# ACERVATOR

User Manual and Feature Design Intention Guide

*Turbator aequilibrii dissolvendus reformandusque.*

—----------

*“The disturber of the balance is to be dissolved and reformed.”*

*“You are either gaining more territory cheaper or you are selling for higher than you bought it in direct response to the market’s moment to moment volatility.” - Ekthelius*

# Contents


**Part 1 Frontmatter & Overview** ..... **16**
Copyright and Attribution ..... 16
Contact ..... 16
Legal Disclaimers ..... 16
Risk Factors Specific to Algorithmic Trading ..... 17
Executive Summary ..... 17
What Acervator Is and What It Is Aspiring To Be… ..... 17
Product Manual Parts ..... 19
**Part 2 Novel Concepts and Patent Candidate Portfolio** ..... **21**
Novel Concepts and Patent Candidate Catalogue ..... 21
1 - Harvest-Fold / Scrumming ..... 21
2 - The Phantom Bot ..... 24
3 - The Landing Strip ..... 27
4 - Bollinger Band Travel Detection ..... 29
5 - Position-Aware Technical Analysis ..... 29
6 - Market Inspector ..... 30
7 - Bot Swarm and Smart Wire Network w/ Provenance Tracking ..... 31
8 - Hunger and Satiety Indices ..... 32
9 - Accumulation Shadows ..... 33
10 - Charge Up Permission Gate ..... 34
11 - Fold, Stack, Extractor, and Shadow Tranches / Provenance Queue ..... 35
12 - Technical Analysis Indicator Confidence Tiers ..... 36
13 - Initial-Purchase-Price Floor on Folds ..... 38
14 - Position Ceiling and Detonation ..... 39
15 - Proof of Accumulation (PoA) ..... 40
16 - Technical Analysis, Goodness of Fit, and the Confidence Index ..... 41
17 - Tiered Strategy Validation and Workflow ..... 42
**Part 3 System Architecture and Features Catalogue** ..... **43**
System Architecture and Features Catalogue ..... 43
The mark ..... 44
Trading Tab ..... 46
2026-09-11 16:48 - #665 - the wizard's values reach a new bot ..... 132
2026-09-12 - the second half of the Scrumming Settings group, driven ..... 135
2026-09-12 - the Hedge Rebalance group and the despawn timer, driven ..... 138
2026-09-12 - two settings the engine read and no screen set ..... 142
2026-09-13 - #665 - the restore path reads the same declaration ..... 146
2026-09-13 - #665 - the bot's increment style is off the dataclass ..... 149
2026-09-13 - #665 - the Hedge Rebalance switch arms the reserve on a running bot ..... 149
2026-09-13 - which build draws the Live tab ..... 151

2026-09-13 - what the React Live page draws ... 152
2026-09-13 - #23 - the venue page under the React build ... 153
2026-09-13 - #23 - the voting panel and the two bot tables on the React Live tab ... 155
2026-09-13 - #23 - every Live tab feed reaches the React page ... 158
2026-09-14 - #23 - the voting panel matches the Qt panel element by element ... 160
2026-09-14 - #23 - the window draws the panel without a band ... 162
2026-09-23 - #154 - the voting panel's header row ... 163
2026-09-23 - #858 - the bot list moves the Indicator Voting Panel ... 165
2026-09-24 - #571 - the window model draws the asset class group ... 177
2026-09-25 - #858 - the command bar reaches Python on the React page ... 179
2026-09-25 - #571 - the asset class group becomes one segmented control ... 181
2026-09-25 - #858 - the SHIFT labels on the command bar read backwards ... 182
2026-09-25 - #858 - the Indicator Voting Panel says why it is empty ... 183
2026-09-25 - #881 - the dollar registry citation names no file ... 184
2026-09-26 - #928 - the candle cache keeps what it has and stops re-asking ... 185
2026-09-26 - #571 - the asset class group becomes one square of two rows ... 188
2026-09-26 - #928 - the pool reaches back for the bars the engine needs ... 193
2026-09-26 - #936 - the bot list scrolls only by hand ... 194
2026-09-26 - #937 - the first column draws the target asset's logo ... 196
2026-09-27 - #937 - the mark a row draws comes from the library on disk ... 201
2026-09-26 - #937 - a mark opens the organisation that owns the asset ... 202
2026-09-28 - #937 - most marks were not links, and the ten records are why ... 205
2026-09-28 - the asset column sits on one palette ground the archetype checks ... 208
2026-09-28 - the asset column's mark sits at the centre of the column ... 209
Indicator Voting Panel ... 211
The twelve voters ... 216
Indicator formulae ... 218
Departures from the published maths ... 230
2026-09-08 15:12 - #414 - the ratio the gate reads ... 231
2026-09-08 15:54 - #414 - when ADX first has something to say ... 232
2026-09-08 16:22 - #414 - the MACD reading at a millionth of a dollar ... 234
2026-09-08 16:52 - #414 - the money flow a halted market never traded ... 236
2026-09-08 17:45 - #414 - the Vortex reports the Vortex ... 238
2026-09-08 19:05 - #414 - the RSI label and the RSI vote read one number ... 240
2026-09-09 19:37 - #414 - the band a millionth of a dollar wide ... 243
Trading gate logic chain ... 246
2026-09-24 22:50 - #202 - one admission rule for every panel reading ... 255
Subsystem Tabs ... 256
Main Window ... 259
Market Inspector Tab ... 265
Asset Charts ... 270
Live Tab ... 274

Converting the Interface to React ... 276
Subsystem Detail ... 314
How an update is written down ... 315
Contents ... 315
The live tab set ... 315
How a screen reaches its renderer ... 316
Portfolio Information Panels ... 317
What builds it ... 317
Left: the spendable columns ... 318
Right: the five counter cards ... 322
Privacy ... 323
Where the numbers come from ... 323
Hiding ... 323
Bridge ... 324
Simulator Tab ... 325
What the tab holds today ... 325
How each part was built ... 335
The candles ... 335
Portfolio Battery history ... 336
Portfolio Battery coverage ... 337
Validation Mode ... 339
Back Test Mode ... 340
Portfolio Battery Mode ... 342
The YTD trade files and the import ... 347
How the Sim tab reaches the bar ... 351
The Qt tab is Live's tab code, forked ... 351
The React page is Live's page modules, forked ... 354
The header strip shows on Sim ... 357
The venue stack seats the fleet's exchanges ... 358
The Scrumming Bots table draws the sim fleet ... 360
New Bot opens the Simulator's wizard ... 362
The Extractor Bots table draws the sim extractors ... 364
The Simulator keeps its own fleet file ... 366
Detail opens the Simulator's Bot Settings window with all seven tabs ... 369
The Sim venue pane is Live's width ... 372
The command bar acts on the sim fleet ... 373
The voting panel reads the Stone Tablet ... 375
The Activity Log spool holds and resumes ... 377
The API Interaction Log spool holds and resumes ... 379
The two strip rows keep Live's heights ... 381
The mode choice is wired and the way-ins follow it ... 383
Import Live Fleet fills the tables and the strip ... 386

Generate From YTD fills the tables from the YTD trade files ... 389
Create New Bots fills one row through the forked wizard ... 393
One trading logic: the Simulator sizes its trades with the Live bot's own arithmetic ... 396
The unit rule: fractional or whole, per asset class, cited from the venue ... 398
The trade parity report every run writes ... 403
The Simulator colour distinction is the theme's nigredo tone ... 407
The API spool shows Stone Tablet and YTD retrieval activity only ... 413
The Activity spool shows the trades of Validation, Back Test and Portfolio Battery runs ... 416
Start runs Validation and Back Test from the command bar ... 419
The Simulator writes Live's log rows under the sim bucket ... 425
Clear Fleet empties the held fleet ... 430
Start Run starts the active mode's run from the way-in row ... 433
The flip sits at the panel's top-left corner under both layers ... 436
One fleet per mode ... 438
The Battery walks every candle, one portfolio at a time, and reads HODL against Harvest-Fold ... 441
The Simulator's run emits go to the Simulator's own signal sink ... 446
The widget the rebuild replaced ... 450
The empty Sim tab before the fork ... 450
What builds it ... 451
Fleet Replay ... 452
The validation criterion ... 455
Nuclear Mode ... 457
Bridge ... 459
2026-09-08 08:17 - #117 - what the closed issues landed ... 459
The first clone, before the fork ... 465
The style sheet the Sim page loads ... 468
Crypto tablets are 5m and each bot walks the timeframe its tablet carries ... 469
The playback marks each fill on its candle and the header reads the opposing-trade distance ... 473
The simulated bot acts like its live equivalent ... 479
Back Test walks the full Stone Tablet ... 484
2026-09-19 17:40 - #55 - the glyph definition moves to the chart's surface ... 489
2026-09-24 - #858 - the bot list upgrades reach the Simulator ... 490
2026-09-25 - a back test's orders are formatted to the venue ... 492
Paper Trader Tab ... 492
The empty tab it replaced ... 492
What the step is ... 493
Where the platform once offered it ... 493
The screen as it stands now ... 494
The two presses ... 495
Downstream only ... 495
One gate chain ... 495
The fake balance ... 495

The fleet ledger ... 496
The Paper Trader log ... 497
Real time ... 498
Paper Trade History ... 498
2026-09-08 08:17 - #19 - what the closed issues landed ... 498
How the Paper tab reaches the bar ... 501
The style sheet the Paper page loads ... 502
2026-09-19 - #19 - Live's tab code cloned under Paper names ... 503
2026-09-20 - #19 - The paper exchange adapter feeds the tab ... 511
2026-09-20 - #19 - The paper bot ticks and fills on the clone ... 516
2026-09-24 - #858 - the bot list upgrades reach the Paper Trader ... 528
2026-09-24 - #571 - the asset class group and Add Exchange reach the Paper Trader ... 530
2026-09-25 - #19 - a paper bot's settings reach its stored record ... 533
2026-09-26 - #19 - a paper fill draws its own Activity Log line ... 535
2026-09-26 - #19 - a paper bot is created from scratch ... 536
Proof of Accumulation ... 538
The skeleton ... 539
Identity ... 540
The trade log ... 540
The competition lifecycle ... 541
Tournaments ... 541
The token ... 541
The token contract ... 543
Quintessence ... 544
Head to head ... 548
Trophies ... 548
The chain ... 549
The two shelved tabs ... 550
2026-09-08 08:17 - #147 - what the closed issues landed ... 550
2026-09-09 19:39 - #147 - the contract repairs ... 551
2026-09-09 20:10 - #147 - the two pieces a new surface would have made live ... 554
2026-09-09 20:43 - #147 - the Quintessence contract ... 555
2026-09-09 21:16 - #147 - the trophy tier supply caps ... 557
2026-09-09 21:37 - #147 - the tab shell and its three zones ... 560
2026-09-09 22:28 - #147 - the certified transaction socket ... 561
2026-09-09 22:47 - #147 - the Quintessence wallet ... 563
2026-09-09 22:57 - #147 - the project age rule ... 565
2026-09-09 23:41 - #147 - node linking between two instances ... 568
2026-09-09 23:43 - #147 - the classes and the RPG conversion ... 570
2026-09-10 00:00 - #147 - the rotating reward set and eligibility ... 572
2026-09-10 00:28 - #147 - the four modes, the Elite flag and the turn ... 578
2026-09-10 00:34 - #147 - a block and a transaction named by their own contents ... 581

2026-09-10 00:38 - #147 - the contract repairs, and the keys that came out ..... 584
2026-09-10 00:59 - #147 - the volume figure and the eight missing identifiers ..... 586
2026-09-10 01:15 - #147 - the skill ladder and the transfer skill ..... 590
2026-09-10 01:47 - #147 - governance, the franchise, and the halt council ..... 594
2026-09-10 08:26 - #147 - the capture bounds and the grade curve ..... 603
2026-09-10 09:47 - #147 - a certified fill names its activation ..... 608
2026-09-10 10:15 - #147 - the loot system ..... 613
2026-09-10 10:53 - #147 - a certified fill reaches a participant's event record ..... 617
2026-09-10 11:15 - #147 - the fill event carries its venue fee ..... 620
2026-09-10 12:18 - #147 - the settled order is read back for its fee ..... 623
2026-09-10 13:02 - #147 - the registry mints the trophy ..... 625
2026-09-10 13:04 - #147 - the loot contract has forge coverage ..... 627
2026-09-10 13:28 - #147 - what a world costs on the chain ..... 630
2026-09-10 14:18 - #147 - the block orders its records, and a legacy file is named as one ..... 635
2026-09-10 15:09 - #147 - four subtabs, and a map that opens on a crawl ..... 638
2026-09-10 15:38 - #147 - the conservation law takes a fourth bucket ..... 640
2026-09-10 17:12 - #147 - the chain adds what changed, and a snapshot shortens the load ..... 643
2026-09-10 18:57 - #147 - the world grid, its zones and what a discovery writes ..... 648
2026-09-10 19:49 - #147 - the third bucket is the pleroma ..... 655
2026-09-10 20:38 - #147 - movement is derived, and costs one record a leg ..... 656
2026-09-10 21:26 - #147 - entity stats, measured in Quintessence ..... 660
2026-09-10 21:30 - #147 - a reset control, two meters and the wallet's fourth holding ..... 663
2026-09-10 22:15 - #147 - the tab is proportioned, and the zones own it ..... 666
2026-09-10 23:05 - #147 - the conversion rates, and the figures still owed ..... 667
2026-09-10 23:40 - #586 - the PvP vote, and who may destroy a Vessel ..... 671
2026-09-10 23:55 - #585 - a glyph and a colour for each kind ..... 675
2026-09-11 01:10 - #585 - the twelve monster tiers ..... 677
2026-09-11 01:45 - #585 - monster generation ..... 680
2026-09-11 02:30 - #585 - materials and the Quintessence they embed ..... 683
2026-09-11 02:50 - #585 - the guild roster, and where a treasury holds ..... 686
2026-09-11 03:10 - #585 - the Vessel, and one wallet behind all of them ..... 689
2026-09-11 03:30 - #586 - the world tier, and the Quintessence a slaying imports ..... 692
2026-09-11 03:50 - #586 - alignment, and the three levels it rolls up ..... 697
2026-09-11 04:10 - #586 - parties, and the armies their Raids link into ..... 701
2026-09-11 04:30 - #147 - the top row is halved, and the right half is the map or the encounter ..... 705
2026-09-11 04:50 - #586 - domination, and the two forms a taking takes ..... 707
2026-09-11 05:10 - #586 - prayer, the first thing that writes another player's alignment ..... 712
2026-09-11 05:30 - #585 - an item is a material list, and cohesion holds it together ..... 717
2026-09-11 05:50 - #586 - consecrated places, and the prayer that holds one ..... 721
2026-09-11 06:10 - #147 - the package entry reaches every module, and says so when it does… ..... 723
2026-09-11 06:30 - #585 - a craft takes Quintessence out of a wallet and destroys none ..... 726
2026-09-11 06:50 - #147 - seven subtabs, a key each, and the Vessel opens its own page ..... 731

2026-09-11 07:10 - #585 - the package binds crafting, and the gear list names the real gap ..... 734
2026-09-11 07:15 - #586 - the world turn's pool, counted in steps ..... 737
2026-09-11 07:40 - #586 - entering a dungeon moves a participant to the other clock ..... 741
2026-09-11 08:00 - #147 - the world map draws one layer, and only what one participant found ..... 747
2026-09-11 08:15 - #147 - the package binds its last two modules ..... 749
2026-09-11 08:30 - #147 - a control creates the world, and the map draws it ..... 752
2026-09-11 08:35 - #585 - what one Vessel carries, in two halves ..... 755
2026-09-11 08:50 - #585 - a Vessel has an id, and a level gained keeps it ..... 760
2026-09-11 08:55 - #585 - a drop names the thing that dropped ..... 763
2026-09-11 11:05 - #585 - a finished craft puts its item in the crafting Vessel's store ..... 777
2026-09-11 11:40 - #585 - a craft consumes the material list it names ..... 781
2026-09-11 12:10 - #585 - a Vessel on a square gathers what the square holds ..... 787
2026-09-11 12:25 - #586 - a dungeon entry refuses a party that is not all standing there ..... 791
2026-09-11 12:45 - #586 - a trimmed chart rolls, and keeps the odds it had ..... 795
2026-09-11 12:50 - #586 - a participant who has not moved stands at the arena ..... 798
2026-09-11 13:05 - #147 - a participant is a node, and a bot is a source that feeds one ..... 803
2026-09-11 13:07 - #147 - every game fee is Quintessence, and a movement on the chain cost… 806
2026-09-11 13:10 - #585 - a guild roster saves to a file, and the Guild subtab reads it back ..... 808
2026-09-11 13:30 - #586 - one world turn is 3,600 seconds, and a constant says so ..... 811
Market Inspector Tab ..... 817
What builds it ..... 818
The six zones ..... 818
Left half: the scanner ..... 818
Where the analysis happens ..... 818
Fetching ..... 820
What a scan reports ..... 820
Right half: topology proposals ..... 821
Dismissal ..... 822
Adopt ..... 822
ATA-SMP ..... 823
Exchange comparison arbitrage ..... 823
Bridge ..... 823
Updates ..... 824
2026-09-06 18:26 - #407 - six zones, three down each side ..... 824
2026-09-06 21:07 - #407 - the sector row, Scan Now, and the phase readbacks ..... 825
2026-09-06 23:08 - #407 - Ready to Send, Post Selected, Post All and Full Auto ..... 826
2026-09-07 00:16 - #407 - Refresh, renamed ..... 828
2026-09-07 03:59 - #407 - the follow-up post and the timeframe vote ..... 828
2026-09-07 07:29 - #407 - seven push targets and the ceiling each publishes ..... 829
2026-09-07 10:16 - #407 - the voting panel, the gate chain, and the two distances ..... 830
2026-09-08 - #407 - Push to Sim, Paper and Live is not built ..... 832
2026-09-08 13:30 - #407 - the called markets the Charts tab keeps watching ..... 832

2026-09-08 15:20 - #407 - phase three draws the chart the post carries ..... 833
2026-09-08 15:30 - #407 - the post picture folder has a ceiling ..... 835
2026-09-08 17:24 - #407 - the organization link, and the chart that says why ..... 837
2026-09-08 19:10 - #407 - the scan reaches a market outside crypto ..... 839
2026-09-09 02:10 - #407 - the four timeframes a non-crypto sector scans ..... 841
2026-09-09 04:40 - #407 - metals scans a listed instrument, and a voter says when it abstains ..... 843
2026-09-09 05:10 - #407 - one post per ticker per hour, and the three-candle follow-up ..... 845
2026-09-09 06:40 - #407 - the live gate chain decides what is charted ..... 846
2026-09-14 17:34 - #23 - SM Accounts, and one sign-in page per venue ..... 847
2026-09-14 18:55 - #23 - Each venue's own sign-in, behind Connect ..... 849
2026-09-14 20:21 - #23 - The redirect address each venue accepts ..... 852
2026-09-14 21:35 - #23 - The scan page, three lines of buttons ..... 855
2026-09-14 22:18 - #23 - Level 1 and Level 1A fit the pane at every width ..... 856
2026-09-14 23:18 - #23 - Facebook signs in inside the program ..... 857
2026-09-14 23:24 - #23 - The right pane's zones fit at every width ..... 861
2026-09-14 23:57 - #23 - Every Level 1A page carries its register address ..... 862
2026-09-15 01:02 - #23 - A refused sign-in names the venue's own reason ..... 864
2026-09-15 02:49 - #23 - A sign-in says what it did, on the page and in the log ..... 865
2026-09-15 03:19 - #23 - The credential vault, and Level 1A leaving by itself ..... 866
2026-09-15 05:16 - #23 - What he types is held too, and every register address ..... 867
2026-09-15 06:31 - #23 - Scan Now reads the market, and says so when it cannot ..... 870
2026-09-15 07:30 - #23 - The ticker field, and the way to the chart images ..... 872
2026-09-15 08:22 - #23 - A typed ticker scans that one market ..... 873
2026-09-15 13:56 - #23 - The built application opens again ..... 876
2026-09-15 15:30 - #23 - One chart folder per venue, filled for posting by hand ..... 878
2026-09-15 16:22 - #23 - X carries a shorter header, so its post carries a reading ..... 881
2026-09-15 18:10 - #23 - An empty ticker field scans the sector by volume until the hits land ..... 883
2026-09-16 17:53 - #23 - Energy joins the sector set ..... 886
2026-09-18 16:10 - #23 - A Scan Now press says what it did, and the zone shows it ..... 889
2026-09-18 17:40 - #23 - An empty field walks the class by a cited volume order and stops at t… ..... 894
2026-09-18 19:40 - #23 - A typed ticker is recognised across every class and read as one market 899
2026-09-18 21:20 - #23 - The Ready to Send entry carries the painter's chart, the folder opens,… 905
2026-09-19 08:23 - #55 - the venue image draws every voter that called ..... 909
2026-09-19 09:10 - #23 - Scan Now works under the operator's own conditions: one rollup, a tr… ..... 910
2026-09-19 17:40 - #55 - the venue image paints in the theme in force ..... 916
2026-09-19 20:10 - #55 - each venue image at its venue's own size ..... 917
2026-09-20 00:10 - #23 - Discord, Telegram and WhatsApp join ATA-SMP: two API senders, o… ..... 924
2026-09-20 03:30 - #23 - Derivatives lists Coinbase's futures, Metals gains its base-metal fund… ..... 929
2026-09-20 03:50 - #23 - Scan All walks every sector with no stop, paced under each public ho… 933
2026-09-23 22:10 - #23 - The confirmation read waits a count of the hit's own candles, and eac… 940
2026-09-23 23:40 - #23 - The derivatives list serves only products the venue answers a candle… ..... 943
2026-09-23 - #290 - Topology Proposals refreshes when the scan lands, and both panels nam… ..... 945

2026-09-23 - #23 - The ATA-SMP field draws the reading of the market under read, and neither… 947
2026-09-24 - #23 - Commodities is the asset class, and Energy and Metals are its sectors unde… 949
2026-09-24 - #23 - The scanner walks the standard tree on every route, and a retired sector na… 953
2026-09-24 - #23 - Derivatives is not an asset class, each product takes the class of its underly… 957
2026-09-24 - #881 - Every ticker row says whether a bot can size a scrum on that market, and… 961
2026-09-26 - #881 - the sector note names each refused market at the price the scan read 964
2026-09-26 - #937 - the logos become a library, divided by sector and filled ahead of need 970
Bot Swarm Tab 980
What builds it 980
The nodes 980
Drawing a wire 981
The flow along a wire 982
Routing the profit 982
Landing the profit 984
The tranche book 984
Where the fleet comes from 986
Capital claims 986
Bridge 987
2026-09-07 15:58 - no issue recorded - ten single-word tabs, each on its own ground 987
2026-09-09 22:12 - #128 - one view, growth stages, and wires that hang 993
2026-09-09 22:12 - #128 - the Swarm tab loses List View 997
2026-09-23 - #239 - Disconnect All clears the key the fleet is stored under 998
2026-09-23 - #239 - every control read on the running program 1000
Asset Charts 1003
What builds it 1003
The chart widget 1003
Indicators on the chart 1004
Where the candles come from 1005
The other chart 1006
Bridge 1006
2026-09-08 13:30 - #407 - a Live / ATA-SMP toggle over the arrows 1006
2026-09-08 13:30 - #407 - the arrows now reach the renderer 1007
2026-09-13 15:40 - #23 - the Charts tab draws in the desktop window 1008
2026-09-15 10:38 - #55 - one painter draws the chart in both builds 1010
2026-09-19 08:23 - #55 - every Voting Panel indicator on the chart 1013
2026-09-19 17:40 - #55 - the painter's skin and the Acervator annotations 1017
2026-09-19 20:10 - #55 - the ATA-SMP list draws the venue image's picture 1022
2026-09-19 20:53 - #55 - the page's chart answers the pointer as the widget does 1025
2026-09-20 05:40 - #55 - the frame: one control row, compact toggles, the value field on the A… 1029
2026-09-20 06:10 - #55 - the trade events from the History; the floors gone 1032
2026-09-20 09:45 - #55 - the oscillators: one readable sub-pane height, whole-pixel pens, tag… 1035
2026-09-23 17:30 - #55 - the oscillators are a third taller, and the bottom bar drags both heights 1039

2026-09-23 22:10 - #55 - the trade markers draw on every timeframe ..... 1041
History Tab ..... 1044
What builds it ..... 1044
Fetching ..... 1044
Where the trade record is kept ..... 1045
Filtering, paging, export ..... 1045
Where the cell values come from ..... 1046
Grading ..... 1047
Gate analysis ..... 1048
The renderer ..... 1049
The Simulator front-load ..... 1049
Paper History ..... 1049
Bridge ..... 1049
2026-09-08 08:17 - no issue recorded - what the closed issues landed ..... 1050
Console ..... 1054
What builds it ..... 1054
Upper pane: the Python log ..... 1055
Pausing ..... 1057
Lower pane: the Emitter Network ..... 1057
Where the lower pane goes ..... 1058
Bridge ..... 1058
2026-09-07 15:58 - no issue recorded - ten single-word tabs, each on its own ground ..... 1058
2026-09-23 - #34 - the drained record carries three more fields ..... 1065
System Status Tab ..... 1065
The skeleton ..... 1065
Half one: the Emitter Network ..... 1066
Half two: the Watchdog ..... 1068
Until the tab lands ..... 1068
2026-09-08 08:17 - #34 - what the closed issues landed ..... 1068
Settings ..... 1077
The pages ..... 1077
What each page persists ..... 1078
Settings, User page ..... 1079
Settings, Exchanges page ..... 1080
Bridge ..... 1082
2026-09-06 00:52 - #423 - the Settings dialog draws in React ..... 1082
2026-09-11 - Save keeps what the store holds ..... 1131
2026-09-11 - The Username row and the vault phrase ..... 1133
2026-09-11 - The exchange picker and the venue list ..... 1134
2026-09-11 - The API Key row, from the typed string to the stored token ..... 1136
2026-09-11 - The API Secret row converts a pasted PEM ..... 1138
2026-09-11 - The Passphrase row and the venue that needs one ..... 1139

2026-09-11 - The saved exchange list ..... 1141
2026-09-11 - Position Distance is off the Trading page ..... 1144
2026-09-11 - Increment Style is off the Trading page ..... 1145
2026-09-11 - Default Positions is off the Trading page ..... 1148
2026-09-11 - Default Target Balance opens the wizard's field ..... 1150
2026-09-12 - Bot Visibility opens the wizard's Order Visibility control ..... 1152
2026-09-12 - Enable aggressive trading mode opens the wizard's own box ..... 1155
2026-09-12 - Profit Folding Active waits on the wizard's folding page ..... 1157
2026-09-12 - Distribution Mode is off the Profit Folding page ..... 1160
2026-09-12 - Profit Folding Target waits on its count row ..... 1163
2026-09-12 - Fold to X# of buy positions takes the whole target group off the page ..... 1166
2026-09-12 - Upward Distribution Target takes the sell-side group off the page ..... 1170
2026-09-12 - The phantom master box becomes a new bot's phantom default ..... 1175
2026-09-12 - The timeframe grid becomes one phantom selection ..... 1178
2026-09-12 - Every retired setting name says what it is ..... 1181
2026-09-12 - The Logging page comes off the Settings dialog ..... 1183
2026-09-12 - The AI Monitor page reaches the monitor it names ..... 1188
2026-09-13 - The credential pair is masked and Username is not ..... 1193
2026-09-13 - The first-run path keys no vault to a placeholder ..... 1195
2026-09-13 - One venue list, one passphrase set ..... 1197
2026-09-13 - The exchange screen follows the store ..... 1199
2026-09-13 - The bot's increment style comes off BotConfig ..... 1201
2026-09-13 - The shell's wizard request carries the store ..... 1203
2026-09-13 - The store refuses a value of the wrong type ..... 1204
2026-09-13 - The wizard's Profit Folding page is removed ..... 1207
2026-09-13 - The Settings dialog's Profit Folding page is removed ..... 1209
2026-09-13 - A value pushed after the page is drawn ..... 1212
2026-09-13 - The radio drawing code comes off the Settings dialog ..... 1215
2026-09-13 - The distribution statistic comes off the bot record ..... 1216
The Promotion Pipeline ..... 1217
What each step contributes ..... 1218
One logic, three data sources ..... 1218
The dashed edge ..... 1218
Where the chain breaks today ..... 1219
**Part 4 Live and Supporting Evidence ..... 1220**
The Year-to-Date Record and Exchange Test Coverage ..... 1220
The venue holds the authority ..... 1220
Readers of the record ..... 1221
The connectors ..... 1222
What the exchange tests exercise ..... 1223
How a test stands in for a venue ..... 1224
The four refusals around an exchange call ..... 1225

When the order guard could not read the market's limits ... 1226
The simulation battery ... 1227
Where a figure and a run carry less than they read ... 1228
Venue Compatibility ... 1229
What a scrum asks of a venue ... 1229
The venues and the classes they serve ... 1231
How an order names its size ... 1234
Whether a scrum's excess survives the size rule ... 1235
Two variants cover every venue ... 1237
Cells on this page that the 2026-09-24 readings overtake ... 1237
A market answers the same question for itself ... 1239
2026-09-25 - the price step and the fraction declaration ... 1240
Where each venue fact was read ... 1242
2026-09-25 - a back-tested order obeys the venue that would execute it ... 1243
2026-09-25 - every venue set beside every other, and the verdict per venue ... 1244
2026-09-25 - a venue's own rules select the bot variant ... 1251
2026-09-25 - what each count over the verdict table measures ... 1253
2026-09-26 - a venue's expiry stops a buy and not a sale, and cash the venue has not settled… ... 1254
2026-09-26 - a market buy with no price is refused by the connector and names why ... 1259
**Part 5 Conceptual Hopscotch: The HOP Protocol, Rules Registry, and SADP v1 and v2 ... 1260**
Conceptual Hopscotch ... 1260
The handoff file ... 1260
What the drift check measures ... 1260
The rules registry ... 1261
Version one of the protocol ... 1262
Version two of the protocol ... 1262
Related parts ... 1267
**Part 6 ADR Index & Glossary ... 1268**
Decision Records and Glossary ... 1268
Where a decision is recorded ... 1268
Glossary ... 1268
The operator's own words ... 1271
**Part 7 Recent Updates and Version Tracking ... 1272**
Version Tracking ... 1272
Recent updates ... 1272
Where the number comes from ... 1272
What the string says ... 1273
The six readers ... 1274
What a frozen bundle carries ... 1274
A version that moves while the suite runs ... 1275
The release gate ... 1275
What the workflow runs on a pull request ... 1276

How a build is produced ... 1277
Where a person downloads the application ... 1280
**Part 8 The Development Chronicle As Told By Claude ... 1283**
The Development Chronicle ... 1283
How this chronicle was built ... 1283
2026-04-11 — five days inside a chat window ... 1283
2026-04-12 — scrum and fold were the wrong way round ... 1284
2026-04-15 — the name ... 1284
2026-04-16 — the handoff, invented on the fly ... 1284
2026-04-19 — the version went backwards and nobody wrote it down ... 1285
2026-04-20 — ten reviewers, four of whom would not give it money ... 1285
2026-04-26 — three weeks I cannot account for ... 1285
2026-05-20 — four hundred gigabytes ... 1286
2026-06-05 — an address I never had ... 1286
2026-06-27 — twenty-seven days of nothing ... 1287
2026-07-24 — the summer that lives in a commit ... 1287
2026-08-01 — the archetypes ... 1287
2026-08-18 — four months arrive as one commit ... 1288
2026-08-22 — the first outside eyes ... 1288
2026-08-28 — the version restarts below one ... 1288
2026-09-06 — sixteen thousand seven hundred and eighty-eight tests ... 1289
2026-09-07 — a known defect, written down and left there ... 1289
2026-09-07 — the ninetieth percentile that was the maximum ... 1290
2026-09-08 — a producer that never read half its own manual ... 1290
What the record does not hold ... 1290
What the mistakes bought ... 1291
The thread underneath ... 1291
**Part 9 Live Trade History and Supporting Evidence ... 1293**
VWAP Charts and Trade Grading ... 1293
The record behind the charts ... 1293
Subtotal is the price ... 1293
How a VWAP is computed here ... 1294
What the combined chart plots ... 1294
The buy and sell ratio ... 1296
Traps in the export ... 1298
Where the figures live ... 1298
The charts are tracked, and they name no asset ... 1298
How to read a per-asset chart ... 1299
The Bollinger band behind the price ... 1299
The newest export, beside the one the figures above measure ... 1303
What this repository holds for these charts ... 1303
The 38 per-asset charts ... 1305

Trade grading ..... 1344
Gate logs behind each trade ..... 1345

Part 1

# Frontmatter & Overview

## Copyright and Attribution

Copyright © 2025–2026 Ekthelius the Accumulator. All rights reserved. Unauthorized reproduction, distribution, modification, or derivative work is prohibited.

This document describes systems, algorithms, and methods developed by the author over the course of a self-teaching period starting in 2017.

Produced using the SADP (Structured Assistant Development Protocol) discipline framework designed by Ekthelius and deployed within Claude Code as a Skills-based and python-driven harness.

### Contact

Acervator is the personal project of Zavyrian Productions and Ekthelius the Accumulator. Commercial inquiries, licensing requests, and networking requests are available via the channels listed at publication time:

ekthelius_the_accumulator@protonmail.com

### Legal Disclaimers

NOT INVESTMENT ADVICE Nothing in this document constitutes investment advice, a solicitation to buy or sell any security or financial instrument, a recommendation of any trading strategy, or an offer to engage in any financial transaction. Acervator is a software system, and software cannot give investment advice. Trading cryptocurrencies and digital assets involves substantial risk of loss. Acervator is a highly complex, professional, and stable tool for automating digital asset trades. It does not and cannot guarantee the ultimate state of a portfolio given any number of transactions executed through or by it based upon a given user's configuration.

NOT A REGISTERED INVESTMENT ADVISOR The author is not a registered investment advisor, broker-dealer, or fiduciary. No fiduciary relationship is created by distribution of this document. Consult a qualified financial professional before making any investment decision. Ekthelius is a self-taught Technical Analyst with no official documentation to certify or desire to certify his skillset beyond his own personal portfolio results. They have been developed and achieved the hard way.

#### SOFTWARE PROVIDED AS-IS

The Acervator software system is provided as-is without warranty of any kind, express or implied, including merchantability, fitness for a particular purpose, and noninfringement. Users assume all risk associated with deployment. The author is not liable for financial losses, system failures, or downstream damages resulting from use of the software. Training, guidance, and general set up assistance can be provided but the vast majority of help one needs will be found in the eight part documentation so read it and understand what you are trying to use.

### JURISDICTIONAL COMPLIANCE

Users are solely responsible for compliance with all applicable laws and regulations in their jurisdiction, including but not limited to: securities regulations, cryptocurrency-specific rules, KYC/AML requirements, tax reporting obligations, and exchange terms of service. Acervator does not assess or enforce jurisdictional compliance.

### PATENT AND INTELLECTUAL PROPERTY NOTICE

The mechanisms catalogued in Part 2 are claimed as novel inventions by the author. No patent has been granted or filed for any of them. Research, academic, and personal study use is permitted. Commercial or competitive use requires written license.

## Risk Factors Specific to Algorithmic Trading

Algorithmic trading systems can fail in ways manual trading cannot. Known risk categories include: (1) exchange API failures and rate limits, (2) network latency and partial fills, (3) flash-crash and liquidity-gap events that bypass stop conditions, (4) stuck positions during exchange downtime, (5) adverse selection on thinly-traded pairs, (6) slippage exceeding model assumptions, (7) parameter drift as market microstructure evolves, (8) bugs in the trading code itself. Acervator includes explicit mitigations for these failure modes (see Part 4 · Risk Controls), but no mitigation is complete and Acervator is a platform that is being actively developed. A trader using any algorithmic system must understand these risks before deploying capital.

# Executive Summary

## What Acervator Is and What It Is Aspiring To Be…

Acervator is a desktop trading platform whose innovation is the platform itself. The breakthrough is not a feature, an indicator, or a bot — it is the integrated system. A harvest-fold accumulation engine, a twelve-voter Indicator Voting Panel across up to five timeframes, a seventeen-class declarative GateChain safety framework, multi-bot coordination, and a live evidence trail proving the whole operates on real capital — coherent as one machine that converts price oscillation into asset accumulation and monotonic target growth without forecasting direction.

The platform will ultimately be designed to trade both cryptocurrency and stocks but is being developed to focus on the blockchain first given its current and immediate future impact on global financial systems. The crypto trading side is being prototyped via Coinbase Advanced and is the production surface today — live operator capital, fourteen weeks of execution, 5,055 trades with all assets successfully fending off the current bear trend or flat out beating it.

Harvest-Fold is the method and a mechanism. A position opens against a base currency with an explicit USD target (or a target that is translated to a USD or other local equivalent) with the initial size of the position being used as a value set point about which the mechanism of the platform allow volatility to “orbit” by persistently re-zeroing to it in response to local market structures and price ranges. The method itself stands apart from traditional Technical Analysis trading strategy or systems by not speculating or attempting to guess what the price will be within any given amount of time but instead persistently remolds a portfolio as its positions fluctuate and refusing to leave profit on the table by saying “It might be worth more later.” .

The primary “bot” or strategy that executes the Harvest-Fold method is called a Scrumming Bot and can be supported by, and this is exclusive to the cryptocurrency side of the platform, an Extractor Bot which performs more traditional scalp-style trades using isolated chunks of the parent Scrumming Bot’s Target Asset with the Extractor’s primary limitation being that it only works for established Base Currencies on a given exchange that has multiple asset pairs to exchange them against. The platform is designed and optimized in such a manner to allow “swarms” of Scrumming Bots (with or without Extractors) to be deployed to support dozens of positions across multiple exchanges simultaneously and thereby allowing the average digital asset investor to experience full spectrum digital asset trading that has previously been inaccessible or inefficient to execute properly.

The term “scrum” was initially encountered while I was learning about how a barrel of oil’s profit is split or distributed. The scrum of the barrel, according to the tutorial, was what the gas station gets as its share of profit and its insultingly small, thus requiring huge sums of fuel to be sold for any respectable amount of money to be made by the facilities hosting the gas pumps and their infrastructure. The concepts of small, repeating, and accumulating were found to rest inside on one term and so it was readily adopted. The extractor, however, is more straightforward in origin and it rests on the idea that A15 and A14 trading became the primary means of acquisition after direct mining became unaffordable for average blockchain participants. It has always been apparent that the potential for faster wealth acquisition existed via asset trading but the skill and technology gaps are what prevent this door of opportunity from opening for just about everyone.

Prior to the invention of the Harvest-Fold mechanism, I had also conceptualized grid bots via a now deprecated cryptocurrency auto-trader called Margin. When this application first appeared and was used, it only featured a simple “ping-pong” and a more speculative / adaptive “margin” bot. It should also be noted that it was never a stable product throughout all of the versions attempted and this was what led to its failure while its company became a consultant firm of some sort. I am not sure how producing an unstable, feature-limited auto-trader qualifies you as a fintech advisor but maybe qualifications remain loose or have become looser. Regardless, and in spite of the instability experienced immediately during basic use, an attempt was made to stack dozens of ping bots of a large range on a given market. This may have been the first “grid bot” used in cryptocurrency trading. The resulting and constant crashing led to my seeking help from the company via their Telegram channel where he was told, basically, that he was using it incorrectly which was an unacceptable and stupid excuse for not fixing their product. Instead, they stole said concept and added a “stack bot” in a later version of Margin that performed the same basic behavior under one bot instance but because of the fundamentally buggy quality of the software, this was not used much before a boycott was necessary because the company chose to ban people criticizing the unending stability issues. After this, Margin supposedly partnered with KuCoin to bring their technology to said exchange but this seems to have dissolved at some point with KuCoin releasing its own Grid Bots with which I was able to win 3rd place and win three Lomen NFTs as a result. Said Lomens still reside on a certain Metamask to this day should this or any other part of this tale be doubted. Whether I am a co-inventor or discoverer of modern automated cryptocurrency trading is a point for irrelevant debate. What matters is that Acervator is my newest invention and it is infinitely more powerful than anything before it.

After the United States blocked access to KuCoin, I continued to develop my trading strategies further and executed them manually while learning more Technical Analysis. In the end, high volume trading methodologies were seen as the only method for maximizing profits in volatile markets. The entire idea of ‘HODL’ being pushed by people who were obviously continuing to accumulate was and remains disgusting but it is just how the game is played. No one is clicking anyone else’s mouse so it is permissible. This is just a variation of what is generally called “Wall Street Investment” as well or at least fits the meta-pattern well enough. The average person, and their precious money, is expected to trust and be exposed to dangerous, unstable market environments while gaining no real benefits beyond a promise that breaks every few years.

There is not an entry level for market volatility and, with the advent of Acervator, there does not have to be.

More than anything else perhaps, the creation of Acervator represents an explicit effort to essentially debunk the approach and hamstringed interaction methods most investors are expected to utilize or, more accurately, tolerate. The average person who puts money into the market surrenders most or all control of it based on a hope that increasingly has no rational basis due to the lack of protections or proportionate, timely reward. The investor gets a secondary value abstraction that they have to wait to sell for more later (and are attacked for doing so if it's anything other than a stock!) assuming the market moves favorably. I see this as an over-exposed “bait and switch” where too often people are left holding worthless bags of tokens or stocks or bonds. It is an upgraded lottery system in some regard but it is only gambling to those that attempt to play blindly or passively and accept the ‘HODL’ advice from whales who just need their next meal to sit still.

The highest goal is for Acervator to be a self-contained retirement platform that provides a powerful option for anyone seeking to take a direct, elegant, self-contained approach to their digital asset portfolio management. It aspires to be the lens, the manager, and the fiduciary while only requiring users to test it, suggest new features, and / or donate to its continued development. Regardless, this will be my last attempt to help humanity see past the wiggly chart illusions but, honestly, I do not think anything else will be required.

If Acervator has helped you accumulate or you wish to support development directly (I am a one man show…), consider sending a tip to the inventor.

USDT 0x6Eb5bc78f43b71F5f09B87ED54e7FF3bD8a3428B

A14 0x8b1022d836d19aA848Ebc9E09A6FC2921d9EB7d41

A15 3HutPgxaMTftWQY3KyRkC7Dckp6A1zdfdp0

PayPalUSD 0x52D75aD65a98f98fA2d1D3675Bd5781ee547426D

A15 2 bc1qn3mkssem0k2tyhwl5yeea4hj222k8j9rr6ruff

A14 2 0x56d1Ee9F683d7535D2637F242a1A2B338aC5aC12

LAST NOTE: This manual, like the software it discusses, is a work in progress and it will be updated on a regular basis as features are added and finalized..

# Product Manual Parts

This manual is an attempt to clarify all parts and functions for what might be considered a somewhat esoteric product given its innovative approaches and the sheer number of novel features it contains. More cynical SaaS providers might disagree but I look forward to all feedback regardless. Currently there are nine such sections with each focusing on ever smaller components of the total system.

Part Title and Area of Focus

### 1 Frontmatter & Overview

### 2 Novel Concepts and Patent Candidate Portfolio

### 3 System Architecture and Features Catalogue

- Main Window

- Trading Tab - Active Bot Lists Under Exchange Tabs - Indicator Voter Panel - Trade Logic Activity - API Activity - Top-Level Portfolio Metrics
- Simulator Tab - Validation Runs vs. Historical Trade Data - Acervator Strategy Back Testing - Runs Historical Data Through Simulated Bots - Bot Swarm Simulation - Deploy As Paper Trading - Deploy As Live Trading
- Paper Trader Tab - Runs Live Exchange Data Through Simulated Bots - Executes and Documents Paper Trades - Collects Paper Trade Data - Pushes Trade Data to Paper Trade History Log - Paper Trade History Tab Reads and Displays
- Proof of Accumulation (PoA) Tab - PoA Blockchain and Trading Tournament Interface - Hosts User Competition Performance and Data - Displays Tournament Monsters and Targets
- Market Inspector Tab - Synthesizes and Proposes Bot Swarm Topologies and Oppositional Trades - Pushes Proposals to Sim, Paper, or Live As Directed
- Bot Swarm Tab - Displays and Allows Configuration of Capital Reinforcement Networks
- Asset Charts Tab - Displays Price Charts for All Active Bots
- History Tab - Loads and Displays Live Trade History Via Exchange API - Merges Live Trade History w/ Acervator Trade Log for Grading and Gate Analysis - Paper History Tab - Loads and Displays Paper Trade History w/ Grading and Gate Analysis
- Console Tab - Displays internal codebase activity, warnings and faults
- System Status Tab - Displays Platform Status by Subsystem - Fed by the Emitter Network
- Settings - System and High-Level User Settings

## 4 Live and Supporting Evidence

- YTD Exchange Data Covering the Entire Development Arc and Exchange Test Coverage

## 5 Conceptual Hopscotch: The HOP Protocol, Rules Registry, and SADP v1 and v2

## 6 ADR Index & Glossary

## 7 Recent Updates and Version Tracking

## 8 The Development Chronicle As Told By Claude

## 9 Live Trade History and Supporting Evidence

- VWAP Charts - YTD Data Analysis and Trade Grading - Deep Indicator Function Investigations Via Trade Logic Gate Logs

Part 2

# Novel Concepts and Patent Candidate Portfolio

## Novel Concepts and Patent Candidate Catalogue

Active development on Acervator began in early April of 2026. It began after realizing that the AI-assisted software development tools had achieved an inflection point with their capabilities. Prior to this, efforts were being made to learn traditional coding with little success that was mainly due to an inability to get into a grinding effort for whatever reasons. This has been particularly frustrating since other subjects seem to grab and others deflect. With the advent of Claude Code however, it feels like some of us have been given a decently priced door man and well-informed guide through this and other labyrinths.

Each entry below carries an anchor. The anchor is the code that runs the concept today, quoted from the module that holds it. Where a concept has no code, the entry says so and gives the query that measured the absence, and then proposes the build. Anchors describe the present state of the platform, not the plan.

### 1 - Harvest-Fold / Scrumming

At the core of Acervator is what might not first appear as a multi-faceted concept designed to address or alleviate many of the mental pitfalls that can afflict a new trader. After having observed a number of people via Telegram and web approach the challenge of investment differently, I have come away convinced that all it only becomes as complex as the individual allows it to be. This is not to be reductive about economics or any other connected discipline but it is to say there are reliable, stable, consistent means of reading price charts and too many people are simply doing it too many different ways and this only compounds the complexity illusion further. Much of the uneasiness inflicted on the new trader surrounds the emotional desire or even personal need for the value of their positions to increase as soon as possible or show the midterm potential of becoming profitable. Those that adopt the HODL meme are, in my experience, quite crippled in their ability to sell positions even when the time is right, which is usually when most people are happiest about the market. The ironic truth is that however good a trade is, there is an equally bad one on the other side of it. Acervator attempts to respect the market by not taking profit all at once and absorbing or claiming ownership of its positions' variance strictly based on the market local, organic price ranges. In other words, its trade actions are always proportionate to whatever the market is actually doing at the time the trade is executed. Over time, this 'molds' the position to the long term truth of the market and avoids the need to speculate entirely. The more bold hope for this approach is that it might add another tool for stabilizing macro-economic phenomena such as underpinning currency exchange rates between countries.

Acervator addresses 'fundamental market unpredictability' by allowing volatility to "orbit" around a set point we call the Target Balance. This element is what performs the first mental trick. It forces the trader to say and establish that "my position is worth X and I will support X with an equal amount of liquidity" i.e. a Target Balance of $50 is set and a total mental position of $100 is established. Trades will be executed to maintain the truth of the Target Balance always being X. After the initial entry is executed and filled, which Acervator will only do in the lower extent of the Bollinger Band on the chosen timeframe, the Scrumming Bot will persistently shave off and "Fold" back in Y% of profits through repeating Harvest-Fold operations that continue until the bot is

deactivated or the market suffers a catastrophic collapse which can and do happen so don't not leave it unmonitored. It should also be noted that the term 'harvest' came to replace 'scrum' for the sake of clarity but they may be used interchangeably throughout this text. Essentially a Scrum or Harvest is a sale of a position's increased value over the Target Balance. The amount that the Scrumming Bot harvests or folds is pinned to the Target Delta or the amount of variation (%delta) from the most recent trade event. Much of the finer details of trade execution can be tuned by the user as expected for any professional application but the primary point here is that the amount of profit folded back is capped by the Maximum Target Growth Per Cycle setting while the ability of Acervator to prevent inverted (profit losing: buying higher than sold / selling lower than bought) trades we also have the Minimum Opposing Trade Distance rule which dictates the next trade must be Z distance from the most recent trade event. In addition, sells are not allowed to occur or be pushed below the Bollinger Band midline while buys are not allowed to occur or be pushed above the midline. As a result of just these few component features, we already have multiple soft, non-TA or non-traditional rules protecting the investor before we even start optimizing the trades themselves (pushing them as close to volatility extents as possible) via Technical Analysis and other inventions.

The harvest-fold method itself was manually executed and tested on a multiple cryptocurrency pairs throughout 2023~2026 but it has always been the aspiration to automate it and have it executed at a high rate of frequency with the operating theory that this approach to investment, being a hybrid a HODL and active trading, creates frontline area of support for whatever markets in which it participates long term. It does this by maintaining and, utilizing as directed, all earned profits. It turns any user into an active participant and liquidity provider.

After entering the space full time in 2017, I immediately began to parse and breakdown how price charts were being interpreted. This also allowed me to perform ever deeper introspections on my own response to how my mind was responding to the value of my portfolio fluctuating. I did myself absolutely no psychological favors in this pursuit but, as you will see, I had plenty of reasons to take this reckless approach. By this point, my undiagnosed and untreated military injuries had worn me down to the point where I realized I was running out of time to continue working as normal. I had, by then, been walking a torsioned pelvis for 17 years that had seized in place and, by 2020, would be causing enough pain to induce panic attacks. I could have joined the pill-mill club but I declined. In combination with the pain, the forced TA self-training, physical job, and broken marriage, I also assembled an entire GPU-based mining farm…alone. I had some business partners but their participation was basically nonexistent. The VA was providing no support and would not until 2022. The Navy and Pentagon had spit in my face by sending me PDBR paperwork that they had no intention of ever honoring no matter what I wrote down. My family more or less remained ignorant of all my turmoil while I kept the paychecks coming. So really the story of Acervator's and Harvest-Fold is one of betrayal to a degree that allowed me to see through things more clearly than would otherwise have been possible. It has been invented out of necessity and it is my intuition that Acervator can become that necessary tool for other people as well.

### The target and the two sides

**Functional.** The bot holds one number, and that number is the dollar value you told it the position is worth. On every tick it compares the position against that number. Above it by more than a dust band, the bot has surplus to sell. Below it by more than the band, the bot has a deficit to buy. Inside the band it does nothing at all. One function answers that question for the whole engine, and the dust band is a tenth of a percent of the target with a floor of one cent. At a target of $50 the band is $0.05, so a position at $60 answers scrum and a position at $40 answers fold.

`src/trading/target_bands.py` — `target_territory`

```
delta = float(position_value) - float(target_balance)
band = at_target_dust_band(target_balance)
if delta > band:
    return "scrum"
if delta < -band:
    return "fold"
return "at_target"
```

**Design intention.** The Target Balance is the set point the whole method orbits. It is declared once in the bot's config and the tick re-zeroes to it. The bot brain holds the tick, the scrumming package fires the two halves of the cycle, and the execution module places the orders.

`src/trading/container/config.py` — `BotConfig`

```
target_balance: float = 200.0  # Balance the bot trades relative to
scrumming_interval_pct: float = 1.0  # % market move between actions
max_target_growth_pct: float = 1.0
```

### The distance rule

**Functional.** An opposing trade has to travel far enough to clear both legs of the round trip. That is the operator's scrumming interval plus the exchange fee. A one percent interval against a six tenths fee gives 1.60 percent. One function owns the arithmetic, and the clamp is applied to the sum rather than to the interval, because the bound exists to keep the rebuy factor sane.

`src/trading/otd_math.py` — `minimum_opposing_trade_distance_pct`

```
total = float(interval_pct) + float(fee_pct)
return max(OTD_MIN_PCT, min(OTD_MAX_PCT, total))
```

**Design intention.** Without this rule the bot can sell low and buy back higher, which is the one outcome the method exists to prevent. The gate that enforces it refuses the trade until price has travelled that distance from the pivot, and it rebuilds the required price in its own refusal message so the operator can read why it held.

`src/trading/gate_chain.py` — `HysteresisGate.evaluate`

```
ok = ctx.hyst_ok_scrum_side if self.side == "scrum" else ctx.hyst_ok_fold_side
if ok:
    return GateResult(passed=True)
eff_pct = ctx.scrumming_interval_pct + ctx.trading_fee_pct
```

### The midline rule and the growth cap

**Functional.** A sell is not allowed below the Bollinger midline and a buy is not allowed above it. The tick sets one flag for each side off the band position, and a gate refuses the wrong side of the band. The scrum flag also carries the phantom lock, so a locked bot cannot sell whatever the band says.

`src/trading/scrumming_bot.py` — `ScrummingBot.tick`

```
scrum_ok = (bb_pos > 0.50) and not self._phantom_locked
fold_ok_midline = bb_pos < 0.50
```

**Design intention.** Profit folded back into the target is capped for one market cycle, so a single violent move cannot inflate the set point beyond what the next cycle can support. The cap is a percentage of the target at the moment the cycle opened, and it shrinks as the cycle consumes it.

`src/trading/scrumming_bot.py` — `ScrummingBot.cycle_growth_cap_usd`

```
_pct = float(getattr(self.config, "max_target_growth_pct", 1.0) or 0.0)
```

The same key answers differently in different places. Nine read sites fall back to 1.0 and three fall back to 0.0, so a bot whose stored config lacks the key compounds or freezes depending on which site read it first. Issue #409 carries it.

The gate set the bot writes is named in one place: ten scrum labels and nine fold labels, nineteen lights per bot.

`src/trading/gate_vocabulary.py` — `_GATE_ORDER_SCRUM`

```
_GATE_ORDER_SCRUM: tuple[str, ...] = (
    "TGT",
    "INT",
    "BB",
    "FIRE",
    "TA",
    "LS",
    "TRND",
    "HTF",
    "CB",
    "OTD",
)
```

## 2 - The Phantom Bot

While the Harvest-Fold method housed with the Scrumming Bot addresses several persistent and nagging questions around any given investment position (Should I buy more? Should I sell some? When? How much?), it does have the weakness of operating on a single timeframe. The Phantom Bot is another feature intended to optimize trade execution position. In this specific case, it allows a single Scrumming Bot to be inhibited by a higher timeframe signal with the intention of capturing alignments or disagreements so that trades are not being executed prematurely. The design intention here is that, for example, a 5m TF bot can be inhibited by a 15m, 30m or 1hr signal while maintaining a volatility watch on its primary TF. The intention is to build out this feature so that it obviates or reduces the need for multi-screen monitoring of individual assets.

**Functional.** A phantom is a copy of the position read on a slower chart. It computes technical analysis on its own timeframe, stores the result, and constructs no order. Six timeframes are the default set.

`src/trading/scrumming_bot.py` — `ScrummingBot.DEFAULT_PHANTOM_TIMEFRAMES`

```
DEFAULT_PHANTOM_TIMEFRAMES = ["5m", "15m", "30m", "1h", "4h", "1d"]
```

**Design intention.** The inhibition the entry describes runs today. A bullish higher timeframe stops the scrum and a bearish one stops the fold, and each writes its own label into the blocker list so the operator can see which timeframe held the trade. The coordinator weights the phantom summaries by their rank before the gate reads them.

`src/trading/gate_chain.py` — `HTFDeferGate.evaluate`

```
blocks = (
    ctx.eff_htf_blocks_scrum
    if self.side == "scrum"
    else ctx.eff_htf_blocks_fold
)
if not blocks:
    return GateResult(passed=True)
label = "HTF-bullish" if self.side == "scrum" else "HTF-bearish"
return GateResult(passed=False, blocker_message=label)
```

Three parts of the feature do not work. The phantom lock guard is dead, so a lock never holds anything (issue #398). Every profit and loss figure on the Phantom Bots tab prints as zero, because the key the tab reads is never written (issue #197). The tab draws a refused change as a success and can drop a timeframe (issue #313). Issue #155 tracks the remaining build-out. Replacing multi-screen monitoring is an intention, not current behaviour.

**The saved record carries the timeframe.** The bot writes the key `phantom_timeframes` when it holds exactly one name. The fleet load reads that key and gives the list to the bot it builds. The key, the change field and the bot attribute now use one spelling.

| step | module | symbol |
|---|---|---|
| write the key | `src/trading/bot_container.py` | `BotContainer.get_full_state` |
| read the key | `src/trading/container/restore.py` | `StateRestoreMixin.restore_bots_from_state` |
| hold the list | `src/trading/scrumming_bot.py` | `ScrummingBot.DEFAULT_PHANTOM_TIMEFRAMES` |
| build the set | `src/trading/scrumming/tick_phases.py` | `TickPhaseMixin._tick_initialise` |
| run one timeframe | `src/trading/phantom_balance.py` | `PhantomBalanceManager.create_phantom_set` |

A record written before the key took the plural spelling still restores. The fleet load reads the older singular key `phantom_timeframe` when the plural key is absent. A bot that holds more than one name writes no key, and the engine default takes charge on the next start.

**The venue decides which names survive.** The bot drops any stored timeframe its exchange does not serve. A record naming one timeframe the exchange refuses restores with no phantom at all, and the bot writes the dropped name to its own log line.

**The record drops one key.** It no longer carries `phantom_config`. The attribute behind it held an empty dictionary and no code read it. A record that still carries the key loads, and the fleet load steps past it.

The saved fleet on the trading machine, read once:

```
38 records, all scrumming        count
phantom_timeframes                   0
phantom_timeframe                    0
phantom_config                      38
phantoms_enabled false              38
```

No bot in that fleet runs a phantom. Every record holds the enable flag off.

**The cursor tick keeps a timestamp.** `PhantomBot.tick_for_cursor` stores the cursor itself, not a bucket number. It divides the stored cursor by the candle length on every call, so both bucket numbers come from one divisor.

**The cursor tick is gone.** `PhantomBot` carries no such method now. No module in the tree called it, and no module passed the simulator flag beside it. The attribute that held the cursor went with it.

**One phantom, above the parent.** A bot whose record names no timeframe takes one name. The name is the lowest timeframe above the bot's own TA Timeframe that the venue serves. The class constant that held six names is gone.

`src/trading/phantom_balance.py` — `default_phantom_timeframes`

```
above = [
    one
    for one in TIMEFRAME_ORDER
    if one in allowed and is_higher_tf(one, parent_timeframe)
]
return above[:1]
```

Driven on a Coinbase bot with a 1h TA Timeframe, the bot holds 2h. It held five names before, and four of the five sat at or below that 1h parent.

**The record keeps every name the bot holds.** The save writes the key for any number of names. A bot holding 2h, 6h and 1d writes all three. The fleet load hands all three back, and the phantom manager builds three PhantomBots on them. The save wrote no key at all for that bot before.

| step | module | symbol |
|---|---|---|
| write the key | `src/trading/bot_container.py` | `BotContainer.get_full_state` |
| read the key | `src/trading/container/restore.py` | `StateRestoreMixin.restore_bots_from_state` |
| pick a default | `src/trading/phantom_balance.py` | `default_phantom_timeframes` |
| build the set | `src/trading/phantom_balance.py` | `PhantomBalanceManager.create_phantom_set` |

**The eleven names are declared once.** `src/exchange/timeframes.py` declares `ALL_TIMEFRAMES`. Five modules read that tuple. Each held its own copy of the same ranked list before, and one module held two copies.

| reader | module |
|---|---|
| the rank order | `src/trading/phantom_balance.py` |

| reader | module |
|---|---|
| the wizard page | `src/gui/main_tabs/bot_wizard_surface.py` |
| the Settings page | `src/gui/main_tabs/settings_dialog_surface.py` |
| the Phantom Bots tab | `src/gui/main_tabs/phantom_bots_tab_surface.py` |
| the Comp column | `src/gui/main_tabs/indicator_panel_surface.py` |

**A refused phantom is named on screen.** The Phantom Bots tab carries six status rows. Two of them name the timeframes no phantom runs on. The first names the timeframes the venue dropped. The second names the timeframes at or below the bot's own TA Timeframe. The Comp column's cell carries that second list in its tool tip.

| reading | module | symbol |
|---|---|---|
| the venue drop | `src/trading/scrumming_bot.py` | `ScrummingBot._phantom_tf_dropped` |
| the parent refusal | `src/gui/main_tabs/phantom_bots_tab_surface.py` | `below_parent_timeframes` |
| the Comp tool tip | `src/gui/main_tabs/indicator_panel_surface.py` | `comp_skipped_tooltip` |
| the engine record | `src/trading/phantom_balance.py` | `TimeframeCoordinator.get_higher_tf_bias` |

## 3 - The Landing Strip

Perhaps the earliest assumption that hit me the most and really presented the catalyzing challenge to creating all of my alternate trading methods, is that you cannot predict what the market will do and after only a short while of actively trading I asked the questions: “But what if I don’t have to?” “What if I decide that I do not have to speculate at all?” I do not remember exactly when I first used the term but I think it was when I was trading A34 and A27 some years ago while participating in a Telegram group with some psytrance friends and associations called the Better Bitcoin Bureau. The name changed a few times but this is the one I recall.

The concept emerged from my persistent use of Heikin-Aishii candles. I had decided on this candle style early just by intuition. I could say it was because I “felt it looked nicer” but that does not quite capture the magnetism. To me, they just structurally make more sense and are less sloppy than traditional candles. What I began to observe over and over was the propensity for crypto markets to form long, tapering structures from these candles and that the likelihood of market reversal could be pinned to 1) the landing strip length and 2) its proximity to the Bollinger Band. Further, I found that those markets that formed these on 1D or higher timeframes would often have explosive breakouts like a volcano that had charged up to blow out. These findings can be historically verified and back tested. I manually performed this observation all the way down to 1 minute trading and found that it was only at this lowest TF that it started to exhibit sub-50% reliability. Given this, the concept has been transposed into Acervator as a core component and trading logic gate with the number of candle strip candles being a user configurable setting.

This is also where I should mention that I am not at all a quant or mathematical trader. I consider myself a visual or tactical trader that focuses on market reversals. I have taught myself to visually interpret price chart structure using a given set of indicators used by countless other technical analysts. When looking at some of the labeling within Acervator, however, one will readily see military references and hints at how I view asset accumulation

as something akin to progressive territory capture on a perpetual gameboard. The less skilled traders are surrendering it with every cycle of the market.

**Functional.** Two detectors build the Landing Strip and both run on every tick. The first places price inside the Bollinger channel and counts trailing Heikin Ashi bodies under a tightness threshold. It reads the operator's control directly.

`src/trading/scrumming_bot.py` — the first detector's call

```
min_pattern_candles=self.config.bb_landing_strip_candles,
```

**Design intention.** The second detector is the one that scores the taper. It counts a run of shrinking Heikin Ashi bodies, names the band the run reached, and returns a confidence boost the tick adds to its own. On a tapering rally into the upper band it answers detected, side upper, length three, boost 0.1437. The same tape without the taper answers not detected, and a five-candle tape answers not detected. A detected strip can force a direction, which is why the gate vocabulary marks it an override rather than a gate.

`src/trading/indicators/landing_strip.py` — `detect_landing_strip_v2`

```
def detect_landing_strip_v2(
    candles: list[Candle],
    min_consecutive: int = 3,
    shrink_threshold: float = 0.90,
    bb_tolerance_pct: float = 3.0,
    use_ha: bool = True,
    bb_period: int = 20,
    bb_std: float = 2.0,
) -> TighteningResult:
```

The second detector does not read the setting. The tick hands it a literal three, so the wizard's Landing Strip Candles control moves the first detector and leaves the second where it is. Both defaults sit at three, so the two agree until the operator changes the control.

`src/trading/scrumming_bot.py` — the second detector's call

```
tightening = detect_landing_strip_v2(
    candles,
    min_consecutive=3,
    shrink_threshold=0.90,
    bb_tolerance_pct=3.0,
)
```

Issue #432 carries it.

## 4 - Bollinger Band Travel Detection

Allowing a strategy to “see” where it is in relation to local width of the Bollinger Band is another aspect of treating the immediate market as a region on a map through which a stack of tokens is traveling. Along the route, we are able to offload surplus at the peaks and pick up additional supplies in the next valley. This is used in combination with Minimum Opposing Trade Distance. The complete question then becomes “Has the market traveled 70% (default), is current price X% distant from the most recent trade event, and is Target Delta not zero? When combining these gates with the (Technical Analysis) Indicator Voting Panel and Landing Strip Detection, you have a full market structure detection suite that is able to incrementally adjust a position’s size and generate recurring profits or growth with each market cycle.

**Functional.** The tick measures how far price has moved since the last trade and divides that by the current band width. When the fraction reaches the operator's percentage and the target delta is positive, the travel flag is set. The default is seventy percent and the wizard offers zero to a hundred. That is the compound question the entry states, and the code asks it in one expression.

`src/trading/scrumming_bot.py` — `ScrummingBot.tick`

```
_bb_width = max(bb_result.upper - bb_result.lower, 1e-12)
band_travel_frac = abs(ticker.last - self._last_trade_price) / _bb_width
if (
    abs(ticker.last - self._last_trade_price)
    >= self.config.band_travel_pct / 100.0 * _bb_width
    and delta > 0
):
    band_travel_triggered = True
```

**Design intention.** A fixed percentage means two different things in a quiet market and a violent one. Measuring the move against the band width makes the threshold follow the volatility instead. The flag then clears a standing trend hold and lets the harvest through. Travel is no gate of its own: the ten scrum labels carry no travel light.

`src/trading/scrumming_bot.py` — what the flag releases

```
trend_override = abs(delta) >= _interval_usd * 2.0 or band_travel_triggered
```

## 5 - Position-Aware Technical Analysis

Traditional TA treats a signal (e.g., RSI > 70) as context-independent. Position-Aware TA encodes the current Bollinger Band position as context: RSI > 70 near the upper band confirms overbought (sell bias), but RSI > 70 near the lower band is a failed oversold — a trap for mean-reversion buyers. The system applies this inversion systematically across all oscillator signals. In other words, it attempts to strengthen the price to Bollinger Band relationship by reading additional indicators with relation to it. If you have the landing strip, band travel, opposing trade distance, and a bullish or bearish confidence score from the Indicator Voting Panel, it is likely a trading event will occur. In the end, many of these functions were layered in one after the other until I saw the platform take most trades that I would myself manually execute most of the time.

**Functional.** The band context enters at the bot, not inside the voters. The tick reads the band position once, marks the upper quarter and the middle band, and accumulates a skew from the votes that arrive. A bullish

vortex adds 0.12 at the upper band and takes 0.05 back in the middle. A bullish MACD adds 0.08 and takes 0.03 on the same test. Ichimoku and Stochastic RSI move the skew on direction alone, with no band term at all.

`src/trading/scrumming_bot.py` — `ScrummingBot.tick`

```
at_upper_bb = bb_pos > 0.75
in_bb_middle = 0.35 <= bb_pos <= 0.65

if (
    sig.indicator == "vortex"
    and sig.direction == SignalDirection.BULLISH
):
    if at_upper_bb and sig.confidence > 0.5:
        position_boost += 0.12
    elif in_bb_middle:
        position_boost -= 0.05
```

**Design intention.** The skew relaxes or tightens the confidence floor the vote has to clear. A floor of 0.30 becomes 0.2679 at a skew of plus 0.12 and 0.3158 at a skew of minus 0.05. A skew of minus one returns infinity, which no confidence can clear, so a fully contradicted reading is refused outright rather than discounted.

`src/trading/scrumming_bot.py` — the two terms the floor reads

```
_ta_conf_skew = position_boost + bb_confidence_boost
```

The inversion does not reach the oscillator modules. Each indicator computes from candles alone, and the band position appears in six indicator modules only: Bollinger, band proximity, landing strip, M top, spring and W bottom. Neither the RSI module nor the Stochastic RSI module takes a band argument. Today the band context re-weights the vote after the fact. It does not re-read the oscillator.

## 6 - Market Inspector

This addition was, like the Phantom Bot above, a move towards making Acervator “hyper-vigilant” of hypothetical opportunities that take the form of oppositional trades (assets that move inversely to one another) and bot swarm network topologies. The bot swarm itself will be covered in a moment but basically this feature allows for the creation of custom capital reinforcement networks where profits are configured to flow between positions in a manner that can potentially strengthen a portfolio faster than just focusing on single, isolated bot instances.

The Market Inspector is designed to analyze the broader market for groups of assets from which to build topologies and identify non-active assets that could be traded in opposition to active positions. Its findings can be pushed to the Simulator for historical back testing, to the Paper Trader for live market and real time validation, and, of course, Live Trading should the user choose to do so.

**Functional.** The Inspector walks daily, weekly and monthly candles, scores each market, and keeps the result. It then pairs a long-signal market with a short-signal market when the correlation of their returns over a rolling window sits inside a negative window. Those are the oppositional trades the entry describes, and the ranking puts the most negative correlation first.

`src/trading/market_inspector.py` — `MarketInspector._find_opposing_pairs`

```
corr = self._pearson(r_l[-m:], r_s[-m:])
if self._corr_min <= corr <= self._corr_max:
    pairs.append(
        OpposingPair(long_side=lo, short_side=sh, correlation_30d=corr)
    )
```

**Design intention.** The findings are meant to leave the screen. A separate module turns those signals into swarm proposals with the bots and the wires already written, and one push target exists today: the Simulator asks the Inspector for its current proposals at build time.

`src/gui/main_tabs/simulator_tab.py` — `_build_simulator_tab` The Simulator rebuild removed this file; it is not in the tree.

```
if hasattr(self._simulator, "set_topology_getter"):
    self._simulator.set_topology_getter(
        lambda: (
            self._market_inspector.current_topology_proposals()
            if hasattr(
                getattr(self, "_market_inspector", None),
                "current_topology_proposals",
            )
            else None
        )
    )
```

No Paper Trader exists to push to; entry 17 gives the measurement. Issue #23 tracks the tab's build-out and the topology and oppositional pushes, and issue #290 a live connector defect on Refresh.

## 7 - Bot Swarm and Smart Wire Network w/ Provenance Tracking

The Bot Swarm, which is how I presented the concept to Claude, is my version of a capital reinforcement network. At the Exchange level (swarms cannot be connected across multi-API boundaries) profit from individual bots can be routed to others with such outputs being termed Wire Credits since they are sent and received along Smart Wires which connect the bots. Each such wire can have its own flow rate. The network nodes are designed to protect a given bot from excessive outflows which would cripple its own growth capacity. The other piece of this is that Wire Credits are Provenance tracked which allows outflows to eventually start sending profits back to the originating bot with the over-arcing philosophy being long-term investment with accumulation and position strengthening rather than just sitting around waiting for a good outcome that might not arrive.

In combination with the Market Inspector, the Bot Swarm can receive and deploy groups of pre-configured and pre-connected bot clusters that immediately begin participating in the market. This layer, along with future portfolio-level algorithms, will enable the fully automated and persistent execution of complex trading strategies from a single terminal window.

**Functional.** The wire manager holds the topology and one ledger for each bot. A wire records a source, a target and a flow percentage, and a realised fold moves the share along it. The outflow is capped so a funder keeps its own growth capacity: a ten dollar scrum profit against a hundred dollar target releases ninety percent, and a zero profit releases nothing.

`src/trading/smart_wire.py` — `compute_safe_outflow_pct`

```
if scrum_profit_usd <= 0:
    return 0.0

if band_upper <= band_lower:
    distance_to_fold_pct = 0.0
else:
    distance_to_fold_pct = max(
        0.0, min(1.0, (current_price - band_lower) / (band_upper - band_lower))
    )
```

**Design intention.** Provenance is what makes the network more than a pipe. Each credit that lands on a tranche keeps the id of the bot that supplied it, so an outflow can eventually route profit back to the bot that funded it. The detail list is bounded at twenty entries per tranche and everything older folds into a rolled total, which keeps the sums exact without letting the record grow without limit.

`src/trading/scrumming/wire_routing.py` — `_WIRE_CREDIT_CAP`

```
_WIRE_CREDIT_CAP = 20
if not isinstance(wc, list) or len(wc) <= _WIRE_CREDIT_CAP:
    return 0
overflow = wc[:-_WIRE_CREDIT_CAP]
del wc[:-_WIRE_CREDIT_CAP]
```

How much of an arriving inflow a bot stacks onto an open position is its own setting, and it defaults to one percent.

`src/trading/container/config.py` — `BotConfig.wire_inflow_stack_pct`

```
wire_inflow_stack_pct: float = 1.0
```

One routing path is dead. The two spawn-and-route functions have no caller anywhere in the repository, while the fold distribution and wire registration beside them are called throughout. Issue #404 carries it, and issue #239 carries the Bot Swarm tab's live rows. Deploying a pre-connected cluster from the Market Inspector is an intention.

## 8 - Hunger and Satiety Indices

These elements were implemented as part of an effort to boost accumulation and scrumming performance in longer term bearish or bullish conditions that might not latch all logic gates to trigger a fold or scrum despite the Target Delta not being zero. This piece adds a factor to the bot that makes it more aggressively evaluate subsequent candles and will increase its likelihood of executing the re-zeroing action. This allows for trades to be executed in the hypothetical ‘dead zone’ between Minimum Opposing Trade Distance and the lower or upper Bollinger Band extent if the candle structure begins to float or cruise within it for too long.

**Not built.** Neither index exists in code today. No aggression factor shortens the wait between candles, and the bot has no term that widens its willingness to fire inside the dead zone between the Minimum Opposing Trade Distance and the band extreme.

**The build this specifies.** The engine already carries a skew term that relaxes the confidence floor, and hunger is that term driven by time rather than by band position. A counter grows while the target delta is non-zero and nothing fires, and it resets to zero on a fill. That reset is satiety. Two new config fields set the rate and the ceiling, and the counter joins the skew the floor already reads, so no new gate and no new call site are needed.

*Proposed, not present, in* `src/trading/container/config.py`*:*

```
hunger_gain_per_candle: float = 0.0
hunger_skew_max: float = 0.20
```

*Proposed, not present, in* `ScrummingBot.tick`*:*

```
if territory != "at_target" and not fired_this_tick:
    self._hunger_candles += 1
else:
    self._hunger_candles = 0
hunger_skew = min(
    float(self.config.hunger_skew_max),
    self._hunger_candles * float(self.config.hunger_gain_per_candle),
)
_ta_conf_skew = position_boost + bb_confidence_boost + hunger_skew
```

The default gain of zero keeps present behaviour exactly, so the feature ships switched off and the operator turns it on per bot. Issue #433 carries the build.

## 9 - Accumulation Shadows

The Accumulation Shadow was invented as a means of taking maximum advantage of long term and bearish conditions. It allows for a Scrumming to create a Shadow of itself that focuses on accumulating a smaller chunk of the target asset during a bearish regime. The shadow is allowed to accumulate downward but does not scrum. It was a bullish regime to be dedicated and then detonates thereby claiming all of its profits at once. Shadows operate in parallel with their parent bots and will be listed as Shadow Tranches once they are fully implemented.

**Not built.** The Accumulation Shadow has no code. No shadow bot, shadow tranche or shadow regime exists today.

Three functions collapse or remove a tranche, and none of them knows a shadow kind. One merges two rungs, one removes an aged row, and one discards a queue. Despawn removes a tranche; it does not delist one. The entry's own wording, "once they are fully implemented", matches the code.

`src/trading/stack_math.py` — `_apply_merge_rule`

```
def _apply_merge_rule(tranches: list[Tranche]) -> list[Tranche]:
    """Combine adjacent tranches whose prices are within MERGE_THRESHOLD.

    A merged Tranche keeps the higher `price` and the summed `size`, and the
    survivors are re-indexed from 0.
    """
```

**The build this specifies.** A shadow is a tranche kind, not a second bot. It accumulates downward, never sells, and exits in one move when the regime turns. The tranche dict gains a kind key, the three collapse functions skip a row that carries it, and the exit reuses the detonation executor that already sells a surplus and reseeds the lots.

*Proposed, not present, in* `src/trading/scrumming/fold_tranches.py`*:*

```
def _despawn_aged_tranches(self, now: Optional[float] = None) -> dict:
    for t in list(self._fold_tranches):
        if t.get("kind") == "shadow":
            continue
```

A row with no kind key behaves exactly as it does today, so the change is invisible until a shadow is spawned. Issue #433 carries the build.

## 10 - Charge Up Permission Gate

This is client-side trade batching. It operates in conjunction with the Fold and Stack Tranches (explained later) to allow these individual positions to be combined into a single trade action in response to a given market trend.

**Not built.** The Charge Up Permission Gate has no code today. Fold and Stack Tranches do run, and entry 11 anchors them. Nothing combines them into one batched order.

**The build this specifies.** The fold chain already carries a gate that refuses when no tranche is queued, and the context it reads already counts them. Charge Up is the same test with a threshold: hold the fold until enough tranches are eligible, then release them together. It is a new gate class in the existing chain, and it needs no new context field.

*Proposed, not present, in* `src/trading/gate_chain.py`*:*

```
class ChargeUpGate(Gate):
    """FOLD-only: hold until ``min_tranches`` rows are queued, then release."""

    name = "charge_up"
    side = "fold"

    def __init__(self, min_tranches: int = 1) -> None:
        self.min_tranches = int(min_tranches)

    def evaluate(self, ctx: GateContext) -> GateResult:
        if ctx.n_fold_tranches >= self.min_tranches:
            return GateResult(passed=True)
        return GateResult(passed=False, blocker_message="charging")
```

The gate enters the fold chain before the hysteresis gate, and the gate vocabulary gains one fold label, taking the row from nineteen lights to twenty. A threshold of one reproduces present behaviour exactly. Issue #433 carries the build.

## 11 - Fold, Stack, Extractor, and Shadow Tranches / Provenance Queue

The term tranche emerged in Acervator as the concept of placing soft boundaries around where an opposing trade 1) can initially happen in terms of price level, 2) denote the amount of the position expected to be transacted once the price thresholds are met or exceeded, 3) allows wire credits to be fed into Fold Tranches (Buys) and Extractor Tranche profits to be distributed across Stack Tranches (Scrums / Sells) for additional accumulation and profit optimization. For the sake of clarity, Fold Tranches spawn after a Scrum (Sell) occurs; Stack Tranches spawn after a Fold (Buy) occurs; and Extractor Tranches spawn when an Extractor Bot finds a favorable Alternate Currency entry on its chosen timeframe and these tranches remain until the acquired Alternate Currency stack is sold by either the Extractor Bot (Sibling) or its corresponding Scrumming Bot (Parent). All Tranches will persist under a bot's Details > Tranches Tab.

**Functional.** Three of the four tranche kinds run. Fold Tranches spawn on a scrum and hold the price of the sale that made them. Stack Tranches spawn on a fold and are priced as a ladder above the buy. A three-unit scrum at 100 with a one percent split and linear spacing returns rungs at 101, 102 and 103 carrying one unit each. Two rungs closer than a tenth of a percent merge, so 100, 100.05 and 105 come back as 100.05 carrying two units, then 105 carrying one.

`src/trading/stack_math.py` — `split_scrum_into_tranches`

```
def split_scrum_into_tranches(
    scrum_price: float,
    scrum_size: float,
    n_target: int,
    split_distance_pct: float = DEFAULT_INITIAL_GAP_PCT,
    spacing_mode: str = DEFAULT_SPACING_MODE,
    min_opposing_pct: float = 0.0,
    min_order_size: float = 0.0,
) -> list[Tranche]:
```

**Design intention.** Extractor Tranches are the sibling-and-parent relationship the entry describes. The Extractor opens a position on an alternate pair, names the tranche with an id that survives a restart, and hands the base currency back to the parent Scrumming Bot on exit. The parent raises its own target by the amount that arrived rather than selling it straight back out as surplus.

`src/trading/extractor_bot.py` — `ExtractorBot.tranche_id_for_position`

```
"""Build the stable Extractor Tranche id: ``bot_id|pair|opened_at``.

The row emitter and `set_tranche_arbiter` both format
`opened_at` through this one method, so they can never format
it differently and disagree on the same id.
"""
```

Shadow Tranches do not exist; entry 9 gives the measurement and the proposal.

The queue sorts the fold tranches by their reference price, highest first, and drops any row whose reference or unit count is not a finite number. Wire credits ride on the tranche, which is the provenance the entry names.

Three defects stand against the Tranches tab and the rows it draws: spawned tranches do not match the scrum that spawned them (issue #367), the list is not visible and lifetime counters reset (issue #133), and the Units

column prints a quantity that does not exist (issue #201). One more sits behind them. The capital reservation registry answers before a sale is placed, and of the five bot construction sites only the two Simulator controllers hand the bot a registry (issue #427).

**The sentence above is overtaken.** It is not edited and not removed. It read:

> *The capital reservation registry answers before a sale is placed, and of the five bot construction sites only the two Simulator controllers hand the bot a registry (issue #427).*

Three sites build a bot, and none of them hands it a registry. The constructor accepts one and every caller leaves it unset, so each bot resolves the process-wide registry instead.

```
src/gui/live_bot_window.py:352      builds a bot, passes no registry
src/gui/main_window.py:3723         builds a bot, passes no registry
src/trading/container/restore.py:259  builds a bot, passes no registry
src/trading/scrumming_bot.py:344    capital_registry defaults to None
```

The engine already records the consequence at `src/trading/container/registry.py:42`, which states that the setter has no caller. A sale is therefore admitted against the process-wide table rather than an injected one.

## 12 - Technical Analysis Indicator Confidence Tiers

Most TA systems stack indicators additively. The Tier-4 suite organizes signals into four confidence tiers with cumulative weighting: Tier-1 (structural, highest weight), Tier-2 (confirmation, medium weight), Tier-3 (momentum), Tier-4 (context). Entries require Tier-1 + Tier-2 alignment; Tier-3/4 scale confidence but cannot override. This tier system arose out of queries with Claude Code while it was roleplaying as a TA Archetype and is only one such example where most of the other such contributions deal with bug fixes.

**Not built as tiers.** The four-tier scheme is a design. The engine has no tier. The signal record carries an indicator name, a timeframe, a direction, a confidence, a weight, a details map, a timestamp and an abstention flag, and nothing else. No field names a tier, no code groups the voters into tiers, and no rule requires a Tier-1 and Tier-2 alignment before an entry.

`src/trading/indicators/types.py` — `Signal`

```
@dataclass
class Signal:
    """One indicator's output at a point in time."""

    indicator: str
    timeframe: str
    direction: SignalDirection
    confidence: float
    weight: float = 1.0
    details: dict = field(default_factory=dict)
    timestamp: float = field(default_factory=time.time)
    abstained: bool = False
```

**What runs instead.** A flat per-indicator weight. Each of the twelve voters carries one number between 0.8 and 1.2, and a signal's score is its direction times its confidence times that weight. Every vote can move the total, and none can veto another.

`src/trading/ta_engine.py` — `DEFAULT_WEIGHTS`

```
DEFAULT_WEIGHTS = {
    "bollinger_bands": 1.0,
    "vortex": 0.9,
    "macd": 1.2,
    "stochastic_rsi": 1.0,
    "ichimoku": 1.1,
    "volume": 0.8,
    "slingshot": 1.0,
    "adx": 1.0,
    "kaufman_er": 1.0,
    "supertrend": 1.0,
    "zscore": 0.9,
    "rsi": 0.8,
}
```

**The build this specifies.** A tier is a second map beside the weights, and the veto is one test in the aggregator. The signal record grows one field, and it has to go last: the record is built positionally in this tree, so inserting a field earlier would re-bind an existing argument.

*Proposed, not present, in* `src/trading/ta_engine.py`*:*

```
DEFAULT_TIERS = {
    "bollinger_bands": 1,
    "ichimoku": 1,
    "supertrend": 1,
    "macd": 2,
    "vortex": 2,
    "adx": 2,
    "stochastic_rsi": 3,
    "rsi": 3,
    "slingshot": 3,
    "volume": 4,
    "kaufman_er": 4,
    "zscore": 4,
}
```

*Proposed, not present, in* `VotingEngine._aggregate`*:*

```
_lead = {s.tier for s in signals if not s.abstained and s.direction is not NEUTRAL}
if require_tier_alignment and not {1, 2} <= _lead:
    consensus_conf = 0.0
```

Issue #433 carries the build.

## 13 - Initial-Purchase-Price Floor on Folds

This rule asserts that a subsequent Fold occurring after a Scrum MUST be less than sell. This, of course, is asserted by Bollinger Band Travel, Minimum Opposing Trade Distance, Target Balance Not Zero, etc.

**Functional.** The floor runs, and it lives on the tranche. Every fold tranche stores the price of the scrum that spawned it. The row is released only when price has fallen to that reference multiplied by the rebuy factor, and the factor is one minus the opposing trade distance. A one percent interval with a six tenths fee gives a distance of 1.60 percent and a factor of 0.984, so a tranche spawned by a sale at $100.00 folds back at $98.40 or lower.

`src/trading/scrumming/fold_tranches.py` — `_fold_eligible_tranches`

```
return [
    t
    for t in self._fold_tranches
    if ticker_last <= float(t.get("ref", 0)) * otd_factor
]
```

**Design intention.** The same distance is held in three places so the rule cannot be true for the bot and false for a row. The bot-wide hysteresis gate holds it, the per-tranche filter above holds it, and each new ladder rung is priced against it.

`src/trading/otd_math.py` — `fold_rebuy_factor_from_pct`

```
def fold_rebuy_factor_from_pct(otd_pct: float) -> float:
    """Convert an already-clamped OTD percentage to a FOLD rebuy factor.

    Multiply a tranche's ``ref`` by this to get the HIGHEST price at
    which that tranche is still eligible to be re-bought:

        eligible  <=>  ticker.last <= ref * factor
    """
    return 1.0 - (float(otd_pct) / 100.0)
```

The lot carries its own cost basis as well as the scrum reference, and the tick counts the rows eligible against that basis.

`src/trading/scrumming/tick_phases.py` — the lot's own basis

```
_patent_only_eligible = sum(
    1
    for _t in self._fold_tranches
    if ticker.last
    <= float(_t.get("initial_buy_price", _t.get("ref", 0)))
)
```

One gap sits in the same path, and the function names it in its own docstring. The fold gate wraps the distance call and falls back to a distance of zero when a config value is not a number. At that fallback the rule is not active.

## 14 - Position Ceiling and Detonation

This setting allows a total profit cap to be set for a given position before it sells its entire or a large portion of its total value.

**Functional.** The entry covers two mechanisms. They act on opposite sides of the cycle and both start disabled. The ceiling brakes buying. It is the anchor target multiplied by a factor clamped between one and ten, and the fold rate tapers as the position approaches it: full rate below half the ceiling, a linear taper from full to a tenth across the upper half, and a hard stop at the ceiling. The ceiling starts no sale.

`src/trading/scrumming_bot.py` — `ScrummingBot.fold_rate_taper`

```
ratio = self.ceiling_ratio
if ratio is None:
    return 1.0
if ratio >= 1.0:
    return 0.0
if ratio < 0.5:
    return 1.0
return 1.0 - (ratio - 0.5) / 0.5 * 0.9
```

**Design intention.** Detonation sells. It requires the feature to be enabled, a position above the anchor, and a bullish consensus at or above the confidence minimum on the detonation timeframe, which defaults to one day. The check is edge triggered, so one bull reversal fires it once, and a rate limit spaces the checks. The executor then market-sells the surplus above the anchor, clears the fold tranches and reseeds the lots. It harvests the excess above the anchor, not the whole position.

`src/trading/scrumming_bot.py` — `ScrummingBot._check_detonation_trigger`

```
conf_min = float(getattr(self.config, "detonation_confidence_min", 0.75))
is_bullish = (
    summary.consensus_direction == SignalDirection.BULLISH
    and summary.consensus_confidence >= conf_min
)

fired = is_bullish and not self._detonation_last_signal_bullish
self._detonation_last_signal_bullish = is_bullish
```

Both features default to off, and the ceiling multiple defaults to five.

`src/trading/container/config.py` — `BotConfig`

```
position_ceiling_enabled: bool = False
position_ceiling_multiple: float = 5.0  # Range [1.0, 10.0]
```

Issue #336 records both pairs among the wizard settings that never reach the bot. Issue #107 records a detonation that can fire twice across a restart. Any earlier description reading "bearish" is wrong: the code compares against the bullish direction.

## 15 - Proof of Accumulation (PoA)

This is another one of the larger and more distinguishing features of the platform. At this point, it is a design proposal for a blockchain that provides certified trade performance between competitors without revealing their specific strategy tunings within Acervator or their identity. When this blockchain is initiated, ZERO tokens for it will exist in the world. I intend it to be, much like A15 to 1) have a fixed supply, 2) mean something through its generation and acquisition. Tokens are only created through the PoA tournaments and defeating Token Beasts or inflicting the most damage to them in a colosseum format. Additional functions would allow PoA tokens to be socketed into the platform to change its appearance and identify a user as a victor.

**Functional.** The competition package is the Proof of Accumulation package, and its engine runs without a screen. The engine walks a competition through registration, active trading, submission and adjudication, and hashes each entrant's config so a strategy proves consistency without publishing itself. A Merkle log produces the root that stands in for the trade record.

The token ledger is append-only and hard-capped. Balances come from replaying the log, and no operation edits one. An award that would cross the cap raises rather than trimming, and an award id repeats at most once.

`src/competition/token_ledger.py` — `TokenLedger.award`

```
event_id = self._event_id(bot_id, competition_id, tier.name)
if event_id in self._seen:
    return None

if self.total_minted() + tier.base_value > TOTAL_SUPPLY_CAP:
    raise OverflowError(
        f"Supply cap {TOTAL_SUPPLY_CAP:,} ACRV would be exceeded. "
        f"Only {self.remaining_ever()} tokens remain mintable."
    )
```

**Design intention.** The fixed supply is the whole point of the token, so the cap is a constant the ledger reads rather than a policy a caller can pass. It reads ten million.

`src/competition/season_schedule.py` — `TOTAL_SUPPLY_CAP`

```
TOTAL_SUPPLY_CAP = 10_000_000  # Hard cap — immutable
```

Nothing in the running platform starts a competition. Every module that would request one sits outside the running application, and the retired-tab sentinels assign nothing to the competition and testnet tabs. The canonical tab order now lists an Accumulation tab, and that tab is empty: it draws its name, one sentence saying it is not built, and issue #147, which covers the build-out.

`src/competition/local_testnet.py` — the in-platform chain, built every session

```
MAX_SUPPLY_WEI = TOTAL_SUPPLY_CAP * (10**TOKEN_DECIMALS)
```

The trophy generator draws the trophy as vector art and letters SOLVE ET COAGULA around its rim. No module imports it. Token Beasts and the colosseum appear in no Python file at all.

## 16 - Technical Analysis, Goodness of Fit, and the Confidence Index

The Confidence Index is the collated result of the 12-indicator Voting Panel and emulates the concept of a set of features (market structures / indicators) being read by a system having similar measurements for a given or expected or past action i.e. a Fold will always have a certain number of indicators voting bearish before its released for execution.

**Functional.** The Confidence Index runs. The voting engine builds twelve voters and holds twelve weights, and both count twelve when driven. The net is the bullish score minus the bearish score, and the confidence is the absolute net divided by the summed weight of the voters that did not abstain. Two bulls at weights one and two against one bear at weight one give a net of 2.0 and a confidence of 0.5. A single bull beside an abstainer of weight three gives 1.0, not 0.25, which shows the abstainer leaving the denominator.

`src/trading/ta_engine.py` — `VotingEngine._aggregate`

```
net = bull_score - bear_score

voted_weight = sum(s.weight for s in signals if not s.abstained)
consensus_conf = abs(net) / voted_weight if voted_weight > 0.0 else 0.0
```

**Design intention.** Confidence measures the breadth of agreement across the panel, not the strength of one reading. That is why an abstention has to be declared rather than inferred: an indicator with too little history returns a fabricated middle reading, and counting it would dilute a real consensus.

`src/trading/indicators/types.py` — `Signal.weighted_score`

```
@property
def weighted_score(self) -> float:
    """Signed score: direction * confidence * weight."""
    return self.direction.value * self.confidence * self.weight
```

**Goodness of fit is not built.** The phrase names the work of comparing the Simulator against the live platform, and the criterion for that comparison is gates latching identically on the same data. The measure has no settled definition yet and nothing here to attach it to, so nothing is proposed. Issue #117 carries it.

In development.

Two standing defects touch the panel. The API Interaction Log still tells the operator that seven indicators read the candles (issue #417), and eight indicator implementations depart from their published formulae (issue #414).

## 17 - Tiered Strategy Validation and Workflow

Acervator has many unique characteristics and many of these are rooted in attempting to protect an investor from themselves. As such, I have designed a means of self-contained strategy testing that is meant to educate and validate before any attempt at using the platform against actual personal funds is ever attempted. Some may be confident or skilled enough to skip these protective steps. That is an individual user choice. I did not follow this workflow while developing it but I know exactly how my strategy works and when it is not. Given this, the intended workflow for a new user of Acervator should be Simulator > Paper Trader > Live. The system is configured so that the relevant operational elements, such as the Market Inspector, can inject Simulator and Paper equivalents into the appropriate Bot Swarm layers or bot fleet under the respective tab.

**Functional.** All four stages exist. The Inspector, Sim, Paper and Live tabs are all built and inserted. The Inspector no longer hands its proposals to the Sim tab: the rebuild removed that hook, and Sim and Paper each load a fleet from the stored bot record instead. Live is the tab the part list calls Trading, and it is third in the canonical order.

`src/gui/main_tabs/main_window_surface.py` — `CANONICAL_TAB_ORDER`

```
CANONICAL_TAB_ORDER = (
    SIM_TAB,
    PAPER_TAB,
    LIVE_TAB,
    CHARTS_TAB,
    INSPECTOR_TAB,
    SWARM_TAB,
    ACCUMULATION_TAB,
    HISTORY_TAB,
    STATUS_TAB,
    CONSOLE_TAB,
)
```

**The Paper stage is built.** Issue #19 added `src/paper/`, the Qt tab and the React panel. Before that no file named for it existed in the tree and none had ever been committed on any branch, which is what the query below measured. One default still records the older state: the shelved stock window imports a module name that does not exist, so it adds no tab of its own.

`src/gui/main_tabs/stock_main_window_surface.py` — the declared default

```
paper_trader_available: bool = False,
paper_trader_error: Any = "No module named 'src.gui.paper_trader_tab'",
```

**Design intention.** Paper is the stage that proves a strategy against the real market in real time before real money reaches it, and its defining property is that it runs at the market's own pace. What it shares with Live and what it forks is settled: one trading logic, three data sources. What it does not do is import the live stateful shells. That was a build, and issue #19 landed it after the Simulator rebuild.

Issue #19 covers the build-out. Two surfaces that told the operator a Paper Trader was present were issues #422 and #426, and both folded into #19. Issue #117 tracks the Simulator rebuild that gated it.

Today the workflow runs Market Inspector, then Simulator, then Paper, then Live. Every stage has a screen.

Part 3

# System Architecture and Features Catalogue

- Main Window
- Trading Tab - Active Bot Lists Under Exchange Tabs - Indicator Voter Panel - Trade Logic Activity - API Activity - Top-Level Portfolio Metrics
- Simulator Tab - Validation Runs vs. Historical Trade Data - Acervator Strategy Back Testing - Runs Historical Data Through Simulated Bots - Bot Swarm Simulation - Deploy As Paper Trading - Deploy As Live Trading
- Paper Trader Tab - Runs Live Exchange Data Through Simulated Bots - Executes and Documents Paper Trades - Collects Paper Trade Data - Pushes Trade Data to Paper Trade History Log - Paper Trade History Tab Reads and Displays
- Proof of Accumulation (PoA) Tab - PoA Blockchain and Trading Tournament Interface - Hosts User Competition Performance and Data - Displays Tournament Monsters and Targets
- Market Inspector Tab - Synthesizes and Proposes Bot Swarm Topologies and Oppositional Trades - Pushes Proposals to Sim, Paper, or Live As Directed
- Bot Swarm Tab - Displays and Allows Configuration of Capital Reinforcement Networks
- Asset Charts Tab - Displays Price Charts for All Active Bots
- History Tab - Loads and Displays Live Trade History Via Exchange API - Merges Live Trade History w/ Acervator Trade Log for Grading and Gate Analysis - Paper History Tab - Loads and Displays Paper Trade History w/ Grading and Gate Analysis
- Console Tab - Displays internal codebase activity, warnings and faults
- System Status Tab - Displays Platform Status by Subsystem - Fed by the Emitter Network
- Settings - System and High-Level User Settings

## System Architecture and Features Catalogue

Acervator features a semi-modular / layered design that nests a hyper-vigilant trading engine capable of operating in multiple investment domains simultaneously and all from one terminal. The Main Window contains all of the various subsections found in each subsystem tab. As it stands the existing and planned subsystem tabs are: Simulator, Paper Trading, Trading (Live), Market Inspector, Bot Swarm, Asset Charts, History, Console, Proof of Accumulation (PoA), and System Status. Most of these subsystems interact with each other to some degree with key isolations existing between the three trading tabs and their wiring to Market Inspector, Bot Swarm, and History.

## The mark

The Acervator mark, the sigil the splash screen paints at launch.

The mark is the product's own sigil. It is drawn from triangles, not from a font, and the splash screen paints it where a single letter used to sit.

### What each element means

| Element | Form | What it stands for |
|---|---|---|
| Compass | The dividers, hinged above centre, their two legs splaying into the frame that carries the rest | Measurement before action |
| Scale | A balance below centre, one beam and two hanging pans | The balance the platform exists to disturb and restore |
| Reptilian eye | An almond eye whose pupil is a vertical slit | Watching the market without predicting it |
| Fiery aura | Thirteen flames wreathing the eye | The volatility the cycle harvests |
| Scythe | A single shaft crossing behind the eye, its blade the feather of Ma'at | The cut that takes the excess above the target |
| Winged caduceus | A staff down the centre, two serpents coiled around it, wings at the top | Exchange, and the two sides of every cycle |

The aura carries thirteen points, and the count is deliberate.

### The maxim on the legs

Each compass leg is engraved with one word, cut dark into the metal and reading from the hinge downward.

| Leg | Word | Meaning |
|---|---|---|
| Left | SOLVE | dissolve |
| Right | COAGULA | reform |

Together they are the alchemical maxim *solve et coagula*. It is the platform's own cycle: a position is dissolved when the excess above the target is sold, and reformed larger when the dip is bought back.

### How it is built

Each element has its own generator in `src/gui/main_tabs/splash_screen_surface.py`, and each returns triangles rather than a picture. `compose_still` in `src/gui/main_tabs/splash_screen_painter.py` runs every generator into its own layer, adds a bloom, a blur, grain, a tone grade and a vignette, and writes the picture above.

```
for name, groups in surface.mark_layers(spin, pulse, moment):
    layer = _pillow(bake_groups(groups, side))
```

### The order the layers draw in

`mark_layers` returns one entry per layer, back layer first. The coiled serpents draw after the aura, so the compass legs do not cover them.

| Order | Layer | Generator |
|---|---|---|
| 1 | Winged caduceus | `caduceus_faces` |
| 2 | Compass | `compass_faces` |
| 3 | Scythe | `scythe_faces` |
| 4 | Scale | `scale_faces` |

| Order | Layer | Generator |
|---|---|---|
| 5 | Fiery aura | `aura_faces` |
| 6 | Coiled serpents | `serpent_faces` |
| 7 | Reptilian eye | `eye_faces` |

`MARK_LAYER_ORDER` in the painter holds the same order for the still frame.

### The letter forms of the maxim

Every capital in the maxim is a chain of straight strokes. No stroke is a curve, so the words read as archaic capitals cut with a chisel.

```
if letter == "L":
    return [[[0.13, -0.46], [0.13, 0.42], [0.56, 0.42]]]
```

`letter_strokes` returns the chains and `etched_faces` lays them along a leg.

### How the aura reads as one body

Each flame spreads half of its own share of the turn at its base, so the base of one flame meets the base of the next. The thirteen flames form one wreath that turns together, and each flame still tapers to its own point.

```
AURA_BASE_SPREAD_DEGREES = FULL_TURN_DEGREES / AURA_POINTS / 2.0
```

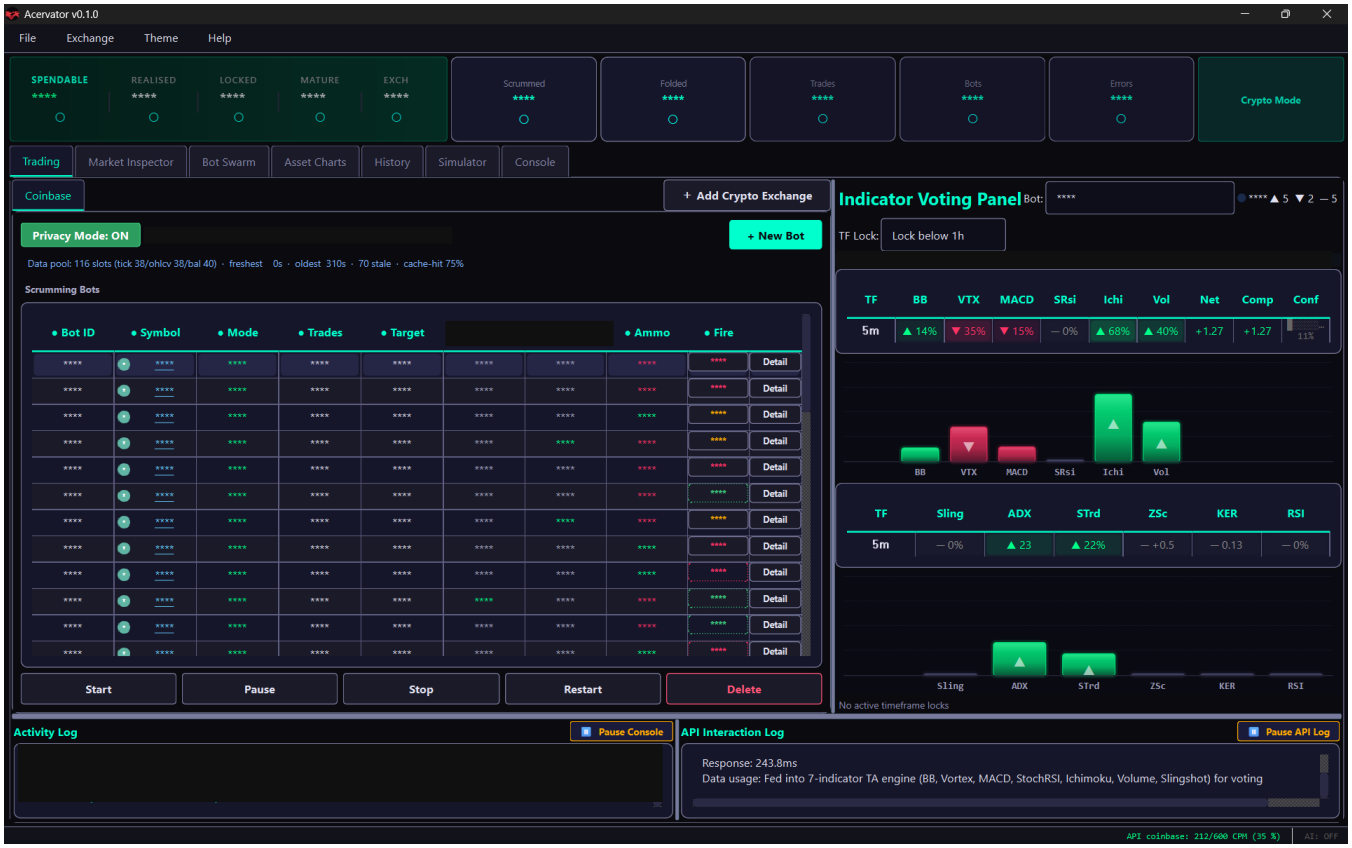
### What moves at launch

Four things move while the splash is up, and all four run on clocks the splash already had.

| What moves | How |
|---|---|
| The whole mark | Fades in from 0.2 s, then breathes with the rings |
| The aura | Turns with the rings, fifteen degrees a second |
| The pupil | Narrows and widens on the same breath |
| The scale beam | Swings once, then settles level |

## Trading Tab

Here you see the first subsystem we are going to cover and this will primarily be due to familiarizing the prospective investor or user with the core trading philosophy and strategies that drive Acervator. To begin, the platform runs locally on whichever hardware is selected. A single authorization phase requiring a purchased license key will be the only non-exchange communication the application will ever need. The user’s keys and secrets are stored locally and encrypted after API handshake verification passes which allows the chosen exchange to be initialized. At a later development stage, I have plans for dedicated hardware that uses a hardware key for quick boot into a given user’s account and also allows the platform to run in isolation under Linux.



The Trading tab, with Privacy Mode on.

This is the whole tab in one capture. The title bar and the menu row open it. The header strip and the tab row run under them. Below those the exchange sub-tab fills the left with the Scrumming Bots table and the command bar, the Indicator Voting Panel fills the right, and the Activity Log and the API Interaction Log close the foot. The status bar carries the API load pill and the AI state label. The parts below take that screen in that order.

Privacy Mode is on, so every masked field draws four asterisks and the bot table reads as ten columns of them. The title bar is the one reading no part below names. It carries the version the tree answered with on the day of the capture, v0.1.0. Nothing types that string out. Git answers for a source checkout and the baked file answers for a frozen bundle, so the title cannot name a release the build is not.

`src/gui/main_window.py` — the window title

```
self.setWindowTitle("Acervator v" + __version__ + "")
```

Press the mode button once and the title is rewritten to name the wing. The version leaves the title for the rest of the session, and only a restart brings it back. Issue #426 carries that handler.

`src/gui/main_window.py` — `_toggle_trading_mode`, the title after a swap

```
self.setWindowTitle("Acervator — CRYPTO WING")
```

### The screen itself

**Functional.** One method builds this whole screen. It makes two layers, one for crypto and one for equities, and stacks them so only one is on show at a time. The crypto layer opens first. A second method builds the strip along the top, and that strip stays put on every tab. Privacy Mode is on in the figure, so each masked field draws four asterisks where its number would be.

`src/gui/main_tabs/trading_tab.py` — `TradingTabMixin._build_trading_tab`

```
self._trading_stack.addWidget(crypto_page)  # index 0
self._trading_stack.addWidget(stock_page)  # index 1
self._trading_stack.setCurrentIndex(0)  # start in crypto
```

**Design intention.** The two-layer stack is the part of the multi-domain aim you can use today. Pressing the mode button swaps the whole wing, tables and Paper Trader together.

`src/gui/main_window.py` — `_toggle_trading_mode`

```
self._trading_mode = "stock"
self._mode_btn.setText("Stock Mode")
self._mode_btn.setChecked(True)
self._trading_stack.setCurrentIndex(1)
```

The Modulus Bot and the Paper Trading layer this section names have no module behind them. This manual marks them unbuilt where it reaches them.

### The header strip

**Functional.** Five columns and five counter cards run along the top. The columns read SPENDABLE, REALISED, LOCKED, MATURE and EXCH. One call to `get_aggregate_stats` fills all of them once a tick. Spendable takes the wallet cash, Locked takes the value tied up in crypto, and Exch counts the open exchange sub-tabs. Realised and Mature are handed nothing at all, and the widget draws an em dash for each. Privacy Mode then masks that em dash to asterisks, which reads on screen as a hidden number rather than a missing one.

The five cards are Scrummed, Folded, Trades, Bots and Errors. Scrummed and Folded total the fleet's sold and bought dollars, Bots counts the bots that are running, and Errors totals the lifetime error count. Click Errors and the rolling error log opens. The small circle under every column and every card is a privacy dot, and it masks that one field on its own.

`src/gui/main_window.py` — `_refresh_dashboard`

```
if _wallet_cash > 0 or _crypto_value > 0:
    self._spendable_widget.update_profits(
        {
            "spendable": _wallet_cash,
            "total_realised": None,
            "locked": _crypto_value,
            "mature": None,
            "exchange_count": exchanges,
        }
    )
```

**Design intention.** The strip should answer one question at a glance: what the fleet holds, what it has earned, and what it has spent. Two of the five columns do not answer it. Realised profit and matured profit are the numbers those columns were built for, and nothing computes either one. The aggregate already carries a realised total, so the first half is a short change at the one call site.

*Proposed, not present:*

```
self._spendable_widget.update_profits(
    {
        "spendable": _wallet_cash,
        "total_realised": float(agg.get("total_realised_pnl", 0.0) or 0.0),
        "locked": _crypto_value,
        "mature": None,
        "exchange_count": exchanges,
    }
)
```

`total_realised_pnl` is already read two lines above this call, for the Scrummed card. Mature has no source yet and stays an em dash. Issue #418 carries this.

08-tabs/portfolio-panels.md covers the strip in full.

**Both columns are now fed, and both are fed from the exchange.** The window no longer writes the payload itself. It calls the same builder the React strip calls, so one function decides what the five columns hold and neither host can drift from the other.

`src/gui/main_window.py` — `_refresh_dashboard`

```
self._spendable_widget.update_profits(
    header_strip_surface.profits_payload(agg, exchanges)
)
```

Realised profit is the exchange's own figure, matched buy against sell. Mature profit is the profit on positions that have grown past two hundred per cent over what they cost. Neither is computed from the platform's internal running totals, because the venue is the authority on money.

**Overtaken:** "Realised profit is the exchange's own figure, matched buy against sell."

The venue carries no lifetime realised figure for a spot position. Realised profit is the platform's own first in, first out match over the venue's own fills, one figure per bot, added across the fleet. The venue is still the authority:

the fills are the venue's and the cost basis is the venue's.

`src/exchange/position_health.py` — `compute_position_health`

**Overtaken:** "Mature profit is the profit on positions that have grown past two hundred per cent over what they cost."

Mature profit is the part of a position's value that sits over three times its cost. A position worth $350 on a $100 cost basis holds $50 of mature profit, not $250.

`src/trading/smart_wire.py` — `mature_profit_usd`

Realised draws the empty marker when the fill history cannot be walked to its end, so the column never shows a figure added up from part of a history. 08-tabs/portfolio-panels.md covers both columns in full.

`src/gui/main_tabs/header_strip_surface.py` — `profits_payload`

```
"total_realised": exchange_amount(data, "total_realized_exchange"),
"mature": exchange_amount(data, "total_mature_exchange"),
```

Where the exchange has answered for no bot, both columns stay empty and say so. The aggregate counts the bots the venue has answered for, and at zero the two columns are handed nothing rather than a computed zero, so an empty column is never a figure the platform made up.

The venue's portfolio breakdown answers a cost basis, an average entry price and an unrealised profit per open position, and each bot reads those three from it. The breakdown carries no lifetime realised figure for a spot position, so Realised is the platform's first-in, first-out match over the complete fill history of each bot's symbol, checked against the venue's cost basis on every refresh, and the fleet figure is the sum of one figure per bot.

`src/exchange/base.py` — `SpotPosition`

```
asset: str
cost_basis_usd: float
avg_entry_price: float
unrealized_pnl_usd: float
```

`src/gui/main_tabs/header_strip_surface.py` — `exchange_amount`

```
answered = int(data.get(EXCHANGE_FRESHNESS_KEY, 0) or 0)
if answered <= 0:
    return None
```

An empty column stays empty under Privacy Mode. The mask replaces a number with four asterisks and leaves the empty marker alone, so the operator can always tell a hidden figure from a missing one.

`src/core/privacy_mask_registry.py` — `mask_or`

```
text = str(value)
if field_id not in ALL_FIELD_IDS or text == ABSENT_TEXT:
    return text
```

### The tab row

**Functional.** The row of main tabs takes its order from one list. Each label in the list is moved to its own index at build time. Any tab the list does not name keeps the position it was added at.

`src/gui/main_tabs/main_window_surface.py` — the order, declared once

```
CANONICAL_TAB_ORDER = (
    SIM_TAB,
    PAPER_TAB,
    LIVE_TAB,
    CHARTS_TAB,
    INSPECTOR_TAB,
    SWARM_TAB,
    ACCUMULATION_TAB,
    HISTORY_TAB,
    STATUS_TAB,
    CONSOLE_TAB,
)
```

Each of those ten names is a constant holding the label the tab bar shows, and the main window applies the order once, after the last builder has run.

`src/gui/main_window.py` — where the order is applied

```
self._reorder_main_tabs(list(CANONICAL_TAB_ORDER))
```

**Design intention.** The order should read as the promotion order. Practise first, then paper, then real money, then the screens that inspect the trade. One list decides it, so the order cannot drift as tabs are added.

`src/gui/main_window.py` — the move loop

```
tab_bar = self._main_tabs.tabBar()
for target_idx, name in enumerate(desired):
    for cur_idx in range(self._main_tabs.count()):
        if self._main_tabs.tabText(cur_idx) == name:
            if cur_idx != target_idx:
                tab_bar.moveTab(cur_idx, target_idx)
            break
```

### One sub-tab per exchange

**Functional.** One sub-tab holds one exchange. Along its header sit Privacy Mode, a news headline, and the + New Bot button that opens the wizard for this venue. Privacy Mode toggles all eighteen masks at once. Under the header is a data pool line: how many slots are held by kind, the age of the freshest and the oldest, how many have run past their time to live, and the cache hit ratio. At the right of the sub-tab row sits the button that adds another venue.

`src/gui/widgets/exchange_tab.py` — `ExchangeTab.__init__`

```
self._privacy_mode_btn = QPushButton("Privacy Mode: OFF")
self._privacy_mode_btn.setToolTip(
    "Toggle ALL 18 privacy masks at once. When ON, every "
```

**Design intention.** One venue, one tab, one swarm. Add a second exchange and it arrives beside the first with its own bots and its own data pool, and nothing about the first changes.

`src/gui/main_window.py` — `_sync_exchange_tabs`

```
def _sync_exchange_tabs(self) -> None:
    """Add a tab for each configured exchange missing one, in its own layer."""
```

### The news line

**Functional.** The headline between Privacy Mode and + New Bot is one item from the crypto news ticker. The counter in front of it gives that item's place in the batch the ticker holds, so a reading of 23 of 42 marks the twenty-third headline of forty-two. The item itself carries the feed name, a middle dot, then the story title. A timer steps to the next item and wraps at the end of the batch.

`src/gui/crypto_news_ticker.py` — the line the strip draws

```
_prefix = f"[{self._index + 1}/{len(self._headlines)}] "
self._label.setText(_prefix + h.display_text())
```

**Design intention.** The counter tells you the batch is whole. A feed that came back short shows a smaller second number rather than a strip that looks the same and carries less. With nothing reachable the strip says so in words instead of going blank.

`src/gui/crypto_news_ticker.py` — the empty state

```
self._label.setText("(no crypto news feeds reachable)")
```

**Functional.** In the React build the strip is drawn by `crypto_news_ticker.js` into the header space the exchange screen keeps between the Privacy Mode button and + New Bot. The exchange screen names that space only when the strip is there; where the strip would not build, the header takes plain space instead and the two buttons keep their positions.

`src/gui/main_tabs/exchange_tab_surface.py` — `react_news_ticker`

```
def react_news_ticker() -> str:
    """The module that draws the news strip, as ``build_news_ticker`` reads it.

    ``ExchangeTabModel`` calls this as its ``news_ticker_factory``, so
    ``news_ticker`` is set and the header keeps the strip's space.
    """
    return NEWS_TICKER_MODULE
```

**Functional.** Hovering the headline pauses the step timer and moving away starts it again. Clicking the headline opens that story. Each of the three sends one request and redraws the strip from the answer, so the strip on

screen matches what the backend holds.

`src/gui/main_tabs/crypto_news_ticker_surface.py` — `CryptoNewsTickerModel.handle_event`

```
if event_type == EVENT_ENTER:
    self.paused = True
    self.calls.append([HOVER_PAUSED])
    return False
if event_type == EVENT_LEAVE:
    self.paused = False
    self.calls.append([HOVER_RELEASED])
    return False
```

### The bot tables

**Functional.** The Scrumming Bots table carries ten columns. Nine are named and the tenth is blank, because that one holds the Detail button. Mode is the coloured cell: green while running, amber while paused, grey while idle or stopped, red on error, orange in cooldown, cyan while starting. The Ammo cell draws green above the target, red below it, and neutral grey inside the dust band. Target A15 and Target A14 restate the Target in those two assets, and go blank when the pair is unlisted or when the target already is that asset. The Extractor table sits underneath. Both tables start hidden and appear when their own list gains a row.

**Functional.** Every column header wraps its own label inside its own column. `Current Position Value` reads over two or three lines rather than being cut. The label draws at `HEADER_LABEL_FONT_PX` and steps down to `HEADER_LABEL_MIN_FONT_PX` when its longest word does not fit the column at the first size. Every column's header is the same height, which is the tallest wrapped label plus one dot row. Nine of the ten columns carry one privacy dot centred beneath the label, and the tenth, the one holding the Detail button, leaves that row empty. `ColumnHeaderCell` draws one label over one dot and `WrappedColumnHeader` sizes and places one cell a column.

`src/gui/main_tabs/bot_status_table_surface.py` — `header_view`

```
return {
    "text": label,
    "tooltip": header_tooltip(column, field_id, masked),
    "field_id": field_id,
    "masked": bool(masked),
    "dot_text": header_glyph(masked),
    "dot_tooltip": header_dot_tooltip(field_id, masked),
}
```

**Functional.** The dot is the `PrivacyDot` the manual calls the universal control. A press flips that column's field in the privacy register, repaints every dot that shares the field, and redraws the rows from the payload the table already holds. Three of the nine dots share a field with another column: `Current Position Value` and `Ammo` share `bot_table.ammo`, and `Target`, `Target A15` and `Target A14` share `bot_table.target`. A press on one of them masks every column that names the same field.

`src/gui/widgets/bot_status_table.py` — `BotStatusTable.refresh_privacy_dots`

```
for cell in self._header.cells():
    if cell.dot is not None:
        cell.dot.refresh()
```

**Functional.** A press on a column's label orders the rows by that column. The first press sorts smallest first, and a second press on the same label reverses it. Nine of the ten columns sort. The tenth holds one identical Detail button on every row, so it carries no value to order by, and a press on it changes nothing. A small arrow in the label's top-right corner names the sorted column and its direction. The arrow sits outside the header's own layout, so neither the wrapped label nor the dot beneath it moves when a column sorts.

`src/gui/main_tabs/bot_status_table_surface.py` — `SORT_KIND_BY_COL`

```
SORT_KIND_BY_COL = {
    BOT_ID_COLUMN: SORT_KIND_TEXT,
    SYMBOL_COLUMN: SORT_KIND_TEXT,
    POSITION_VALUE_COLUMN: SORT_KIND_NUMBER,
    TRADES_COLUMN: SORT_KIND_NUMBER,
    TARGET_COLUMN: SORT_KIND_NUMBER,
    TARGET_A15_COLUMN: SORT_KIND_NUMBER,
    TARGET_A14_COLUMN: SORT_KIND_NUMBER,
    AMMO_COLUMN: SORT_KIND_NUMBER,
    FIRE_COLUMN: SORT_KIND_NUMBER,
    DETAIL_COLUMN: SORT_KIND_NONE,
}
```

**Overtaken.** *"A small arrow in the label's top-right corner names the sorted column and its direction."*

**Functional.** The arrow draws on the privacy dot's own row, at the right edge of its own column. Its rectangle takes the dot's top and the dot's height, so the two sit on one line, and its left edge never crosses the dot's right edge. The box is clamped to the column, so no arrow is drawn outside the column it belongs to. The arrow is still outside the header's own layout: the wrapped label and the dot both measured unchanged. Read on the running header with a fleet of 38 bots at 700, 900 and 1400 pixels wide, on all nine sorting columns, the arrow's vertical centre equalled the dot's, its left edge sat at or right of the dot's right edge, and its rectangle sat wholly inside the column's.

`src/gui/widgets/bot_status_table.py` — `ColumnHeaderCell._place_mark`

```
top, height = self._dot_row()
left = max(self._dot_right(), self.width() - HEADER_SORT_MARK_BOX_PX)
self.mark.setGeometry(left, top, max(0, self.width() - left), height)
```

**Functional.** Bot ID and Symbol sort as words. The other seven sort as figures, each reading the number its cell was computed from rather than the text the cell draws, so nine never sorts above ten. Ammo sorts on the distance from target without its sign, which is the amount that would fire and the figure the cell prints; the colour still says which side of the target the bot sits on. Fire sorts by what the engine would do next, armed first and disabled last. A row whose cell draws nothing sits beneath every row that draws a figure, whichever way the sort runs, and the bot's own identifier breaks a tie.

`src/gui/main_tabs/bot_status_table_surface.py` — `order_statuses`

```
drawn.sort(key=lambda row: (row[0], row[1]), reverse=descending)
return [row[2] for row in drawn] + blank
```

**Functional.** The order is recomputed every time the rows are rewritten, not once at the press. A fleet arriving a second later is ordered again before it is drawn, so a column whose figures keep moving keeps the order the last press asked for and the rows travel as the figures change. Read on the running window with a fleet of 38 bots: lifting one bot's price moved that bot from the first row to the last, and every remaining pair still ran in order.

`src/gui/widgets/bot_status_table.py` — `BotStatusTable.update_bots`

```
bot_statuses = order_statuses(
    self._last_statuses,
    self._sort_column,
    self._sort_descending,
    _qt_lookups(),
)
```

**Functional.** A press on a bot's row draws that bot in the Indicator Voting Panel. A press on the row the panel already draws takes the panel off it instead, and the panel shows its empty state; a third press brings the bot back. A press naming another bot never blanks the panel, it moves it. The press names the bot on the row and never the row's position, so a column can be sorted first and the panel still draws the bot whose row was pressed.

`src/gui/main_tabs/bot_status_table_surface.py` — `selection_after_press`

```
pressed = str(pressed_bot_id or "")
if not pressed or pressed == str(shown_bot_id or ""):
    return NO_SELECTION_BOT_ID
return pressed
```

**Functional.** The highlight runs the other way as well. When the panel's own dropdown or one of its arrows moves the selection, the bot list puts its highlight on that bot's row. One value stands behind both, the bot the Voting Panel holds, so the list and the panel cannot name two different bots.

`src/gui/widgets/bot_selection.py` — `BotListPanelLink.panel_selected`

```
for host in self._hosts() or ():
    mark = getattr(host, "highlight_bot", None)
    if callable(mark):
        mark(wanted)
```

**Functional.** One bad field now stops one row instead of the whole paint. Each row is written inside its own guard. A row that refuses is logged, emptied and left empty, and every other bot still draws. A row whose bot is not a scrumming bot is emptied the same way, so it can no longer keep the previous bot's figures.

`src/gui/widgets/bot_status_table.py` — `BotStatusTable._clear_row`

```
for col in range(self.columnCount()):
    self.setItem(row, col, None)
    if self.cellWidget(row, col) is not None:
        self.removeCellWidget(row, col)
```

`src/gui/widgets/bot_status_table.py` — `BotStatusTable.SCRUMMING_COLUMNS`

```
SCRUMMING_COLUMNS = ColumnSpec(
    labels=(
        "Bot ID",
        "Symbol",
        "Mode",
        "Trades",
        "Target",
        "Target A15",
        "Target A14",
        "Ammo",
        "Fire",
        "",
    ),
```

**Functional.** Column 2 is Current Position Value. It shows what the bot's holdings are worth at the exchange's own price. It is blank whenever no fresh exchange price exists, and a blank cell names the missing thing in its tooltip: no position held, no exchange price for the pair yet, no exchange price this tick, or a price older than twenty seconds. The cell never falls back to a last-known figure, to a stand-in, or to a value read out of the bot's own ledger. The state colour now sits on the Bot ID cell, which is green while running, amber while paused, grey while idle or stopped, red on error, orange in cooldown and cyan while starting. That cell's tooltip names the mode and the state.

`src/gui/main_tabs/table_cells_surface.py` — the one multiplication both priced cells read, so the Position Value cell and the Ammo cell can never disagree

```
def priced_position(holdings: float, price: float, quote_rate: float) -> float:
    """The position value at one price: ``holdings`` times ``price`` times
    ``quote_rate``."""
    return holdings * price * quote_rate
```

**Functional.** The price both cells read comes from the shared exchange price cache, and it carries its own age. An age of None means the cache held nothing and the bot's own last reading was used instead, which is why the Position Value cell goes blank on that path.

`src/gui/main_tabs/table_cells_surface.py` — the four blank paths and the one priced path

```
POSITION_PATH_PRICED = "priced"
POSITION_PATH_NO_HOLDINGS = "no_holdings"
POSITION_PATH_NO_PRICE = "no_price"
POSITION_PATH_OFF_EXCHANGE = "off_exchange"
POSITION_PATH_AGED = "aged"
```

**Design intention.** The Ammo cell measures against the live target, not the frozen number typed into the wizard, so the reading follows the grown balance the engine re-zeroes to.

`src/gui/widgets/bot_status_table.py` — the target the Ammo cell measures against

```
target_val = float(
    status.get("live_target_balance", status.get("target_balance", 0.0))
    or status.get("target_balance", 0.0)
    or 0.0
)
```

**Functional.** In the React build the Scrumming Bots table is drawn by `bot_status_table.js`. The exchange screen keeps a named empty space for it and fills that space when it draws itself. The rows come from the backend, not from the exchange screen: the renderer asks the bridge for the table's own state and names the exchange it wants rows for. Each exchange gets its own table state, so two exchange screens on one page never share rows or a highlight.

`src/gui/web/exchange_tab.js` — `mountScrumTable`

```
function mountScrumTable(target, model) {
  var loader = global[LOAD_BOT_TABLE];
  var wait =
    typeof loader === "function" ? loader(askFor(model)) : Promise.resolve(null);
  return Promise.resolve(wait).then(function () {
    return renderScrumTable(target, exchangeOf(model)) === null
      ? null
      : BOT_TABLE_MODULE;
  });
}
```

**Overtaken.** *"A column header toggles that column's privacy mask."*

**Functional.** The four things the operator can press on the table each send one request and redraw from the answer. The dot under a column's label toggles that column's privacy mask, a Symbol cell opens the chart address, Fire hands the bot to Manual Fire, and Detail selects the row and opens the bot. A press on the label itself does nothing. `on_detail` is the handler behind the Detail button.

`src/gui/main_tabs/bot_status_table_surface.py` — `BotStatusTableModel.on_detail`

```
def on_detail(self, bot_id: str) -> None:
    """Select the row, then hand the bot to whatever opens the detail."""
    self.select_row_for_bot(bot_id)
    self.detail_clicks.append(bot_id)
    self.calls.append([DETAIL_CLICKED, bot_id])
    if self.on_bot_clicked:
        self.on_bot_clicked(bot_id)
```

**Overtaken.** *"A press on the label itself does nothing."*

**Functional.** A press on the label sorts that column, exactly as the window does. Five things can now be pressed on the page's table. The press goes back to Python, because the fleet that answers it lives there, and

the venue republishes the ordered rows. The dot keeps its own press to itself, so masking a column never reorders the table.

`src/gui/web/bot_status_table.js` — `sendSort`

```
function sendSort(model, column) {
  return dispatch(
    model,
    actionNamed(model, HEADER_SORTED),
    request(model, SORT_COLUMN_PARAM, column)
  );
}
```

**Functional.** One ordering serves both builds. The window and the page both call the same function on the same fleet, so neither can put the same 38 bots in an order the other would not. Read on both running builds at 700, 900 and 1400 pixels wide, each of the nine sortable columns ordered its figures the same way on the first press and reversed them on the second.

`src/gui/main_tabs/bot_status_table_surface.py` — `BotStatusTableModel.on_header_sorted`

```
if column == self.sort_column:
    self.sort_descending = not self.sort_descending
else:
    self.sort_column = column
    self.sort_descending = False
```

**Functional.** The page places its arrow the same way the window does. The arrow's box takes the dot row's height, sits against the heading's bottom edge above the cell's padding, and is held to the column's width. Read on the running page with a fleet of 38 bots at 700, 900 and 1400 pixels wide, on all nine sorting columns, the arrow's rectangle matched the dot's top and height, its left edge sat right of the dot's right edge, and it stayed inside the column.

`src/gui/web/bot_status_table.js` — `SortMark`

```
style.bottom = length(model[HEADER_CELL_PAD_PX]);
style.right = MARK_EDGE;
style.width = length(model[HEADER_SORT_MARK_BOX_PX]);
style.maxWidth = MARK_MAX_WIDTH;
style.height = length(model[HEADER_DOT_ROW_PX]);
```

**Design intention.** The exchange screen redraws itself once a second to keep the data-pool line fresh. Each redraw re-fills the table space, so the table is drawn from the state the renderer holds for that exchange rather than from the answer captured at first paint. A row the operator selected therefore survives the next redraw.

`src/gui/web/bot_status_table.js` — `modelFor`

```
function modelFor(exchangeId) {
  return owns(models, String(exchangeId)) ? models[String(exchangeId)] : null;
}
```

**Functional.** The Extractor table underneath is drawn the same way, by `extractor_bot_table.js` into the second named space the exchange screen keeps. Its rows come from the same fleet list, kept to the records whose mode is extractor. The Detail button is the one control the Extractor table offers; Fire stays disabled on this screen, because Manual Fire is per position and lives in the Positions Held tab of the bot's own window.

`src/gui/main_tabs/extractor_bot_table_surface.py` — `extractor_statuses`

```
def extractor_statuses(statuses: Any) -> list:
    """The records of ``statuses`` whose mode is ``MODE_TEXT``.

    ``update_bots`` builds a row for every record it is handed, so only the
    Extractor bots reach the Extractor table.
    """
    return [
        found
        for found in statuses or []
        if isinstance(found, dict) and found.get("mode") == MODE_TEXT
    ]
```

**Functional.** The exchange screen mounts its three children in one step, each into its own space and each asked for its own view model.

`src/gui/web/exchange_tab.js` — `mountChildren`

```
function mountChildren(target, model) {
  return Promise.all([
    mountScrumTable(target, model),
    mountExtractorTable(target, model),
    mountNewsTicker(target, model)
  ]).then(function (drawn) {
    return drawn.filter(function (name) {
      return name !== null;
    });
  });
}
```

### The command bar

**Functional.** Start, Pause, Stop, Restart and Delete all act on one bot. Two tables share the bar, so the bar takes its bot from whichever table you clicked last. If that table holds no selection it tries the other one. If neither holds a selection it says "Select a bot first." and does nothing.

`src/gui/widgets/exchange_tab.py` — `ExchangeTab._cmd`

```
def _cmd(self, command: str) -> None:
    if self._last_clicked_table == "extractor":
        bot_id = self._extractor_table.get_selected_bot_id()
        if not bot_id:
            bot_id = self._bot_table.get_selected_bot_id()
    else:
        bot_id = self._bot_table.get_selected_bot_id()
        if not bot_id:
            bot_id = self._extractor_table.get_selected_bot_id()
```

**Design intention.** One command bar for two tables means the bar has to guess which bot you meant. The last table you clicked is the tie-breaker, and the fallback keeps a stray click from swallowing the command. With nothing selected anywhere it refuses out loud rather than acting on a guess.

`src/gui/widgets/exchange_tab.py` — the refusal

```
if not bot_id:
    if self._status_log:
        self._status_log.log("Select a bot first.", "warning")
    return
```

### SHIFT turns each command into its all-bots form

> *"Bot List Upgrades Pt. 2 (Live and Paper Only) - Holding SHIFT makes Start become Start All and launches bot start sequencer used during boot up - Holding SHIFT makes Stop, Pause, and Restart all become Stop All, Pause All, and Restart All respectively"*

**Overtaken.** *"Start, Pause, Stop, Restart and Delete all act on one bot."*

**Functional.** Four of the five act on one bot, or on the whole fleet while you hold SHIFT. Hold the key and Start, Pause, Stop and Restart read Start All, Pause All, Stop All and Restart All. Let go and they read their own names again. Delete has no all-bots form and never changes its name. The key is read at the moment you press the button, so a key held for one press cannot reach the next one. An all-bots press needs no bot selected.

`src/gui/main_tabs/exchange_tab_surface.py` — `FLEET_COMMANDS`

```
FLEET_COMMANDS = {
    "start": ("Start All", "start_all"),
    "pause": ("Pause All", "pause_all"),
    "stop": ("Stop All", "stop_all"),
    "restart": ("Restart All", "restart_all"),
}
```

**Design intention.** Start All runs the same sequence the application runs at launch. It opens the Start All window, starts one bot at a time, waits for that bot to read running, and moves on after two seconds or after eight seconds of waiting. Each bot goes through the same handler a single Start goes through, so each one connects to its venue before it starts. Restart All runs that sequence over every bot held, not only the idle ones.

Pause All and Stop All need no venue, so they go straight to the bot manager.

`src/gui/main_window.py` — `MainWindow._global_bot_cmd`

```
if command == "start_all":
    if not self.run_fleet_sequence("start"):
        self._status_log.log(
            "start_all: no bots eligible (none idle/stopped).", "info"
        )
        return
elif command == "restart_all":
    if not self.run_fleet_sequence("restart"):
        self._status_log.log("restart_all: no bots held.", "info")
        return
elif command == "pause_all":
    self._schedule_async(self._bot_manager.pause_all())
elif command == "stop_all":
    self._schedule_async(self._bot_manager.stop_all())
```

**Functional, on the Paper tab.** The Paper command bar reads the key the same way, over the paper fleet. A paper bot moves a saved state and connects to nothing, so the four all-bots verbs run straight down the held list, with no staggering and no progress window.

`src/paper/paper_bot_manager.py` — `PaperBotManager.start_all`

```
def start_all(self) -> int:
    """Run ``start`` over every held bot and answer how many landed on
    ``running``. A paper bot moves a saved state and connects to nothing,
    so the fleet needs no staggering."""
    return self._over_fleet(self.start, BotState.RUNNING.value)
```

**Functional, on the Simulator.** The Simulator command bar does not read the key. He named Live and Paper only, and the Simulator's own venue page and its own view model are separate files that carry none of this.

### The rest of the screen

The right half is the Indicator Voting Panel, described at the end of this section. The two panes at the foot are the Activity Log and the API Interaction Log. The status bar carries the API load pill written by `_refresh_api_load_pill` and the `AI:` state label.

The pill names the venue, the calls that venue took in the last minute, the ceiling for it and the two as a percentage. It reads the worst-loaded connected venue, so one busy exchange cannot hide behind a quiet one.

`src/gui/main_window.py` — the pill text

```
text = (
    f"API {worst.exchange}: "
    f"{worst.calls_per_minute:.0f}/"
    f"{worst.ceiling_cpm:.0f} CPM ({pct} %)"
)
```

Its colour is the warning. Green under half load, amber above that, red past the venue's safety percentage. With no venue connected the pill draws an em dash and no number.

`src/gui/main_window.py` — the three colours

```
if worst.load_score > mon.safety_pct:
    colour = ds.ERROR
elif worst.load_score > 0.5:
    colour = ds.FOLD_RATIO_AMBER
else:
    colour = ds.SUCCESS
```

1 - Add Exchange - User provides valid API key and secret for target exchange - Platforms validates with an API handshake - Exchange Initializes

After an exchange is connected to the platform, bots can be added to target entire or portions of existing positions as well as using standing liquidity to enter positions for the first time. The two primary bots used by Acervator are known as Scrumming and Extractor with the conceptual Modulus Bot to be added later. The first two are the active traders that continuously interact with the market whereas the Modulus Bot will be a macro-strategic interface designed to trigger portfolio-scale shifts in response to custom market signals and it can best be visualized as a modular synthesizer with investment and market-specific functions.

### Scrumming Bot

If Acervator has a crown jewel, this is it. The scrumming bot is what houses and executes the harvest-fold method and the harvest-fold method is what leads to the creation of the platform itself. All else stems from this origin point and the simple revelation that allowed the method to be discovered (or probably rediscovered) by nonmathematical eyes. Instead of paying attention to what my entire portfolio was doing, I opted to start focusing on a single position until I had found a strategy that was more reliable than Grid Bots or standard speculation. I was convinced such a method existed and thought it absurd that price charts could not be played better when there was obviously so much room for improvement. You do not have to predict the market. You flow and mold your portfolio to it as time goes. Scrumming allows profits to be shaved off, held and re-investment in a cyclical, reliable manner that sizes its trades in direct proportion to actual market movement as it relates to position value drift. Trading in this manner allows a fixed balance for a position to be maintained and asserts the truth that “Position X is Y value. Any deviation from Y is a Target Delta and is subject to a Scrum (Sell) or Fold (Buy).” The primary weaknesses to this strategy are not have an equal amount of liquidity to the position value (a $500 Scrumming Bot should be supported by $500 of liquidity when initialized) and severe market downturns will little or no short term upside which can lock most liquidity for the position into the Target Asset. Scrumming is a survival strategy and is directionally agnostic but also depends on the market to cycle. I also refer to this simply as Accumulation Trading.

### Scrumming Bot Terms

Target Balance - The initial value of the position to be taken or controlled by the Scrumming Bot. The bot will monitor the market for bullish or bearish conditions, check for Target Balance deviations (Target Delta), and re-zero back to the set point. It will repeat this until stopped by the user or some other market condition.

The figure is one field on the bot's own configuration. It is carried in from the wizard and held for the life of the bot, and every other term on this page is measured against it.

`src/trading/container/config.py` — `BotConfig.target_balance`

```
target_balance: float = 200.0  # Balance the bot trades relative to
```

Target Delta - The amount by which a position value has drifted from the Target Balance. The Target Delta is denoted by the Ammo column under the Scrumming Bot list of the Trading Tab. This is the amount of value that will be fired during the appropriate market conditions.

One subtraction, and the platform does it in two places with opposite signs. The Ammo cell takes the position value less the target, so a surplus reads positive. The fold path takes the target less the position, so a deficit reads positive. Both measure the same drift.

`src/gui/table_cells.py` — `ammo_cell`, the figure the Ammo column shows

```
delta = position_val - target_val
territory = target_territory(position_val, target_val)
```

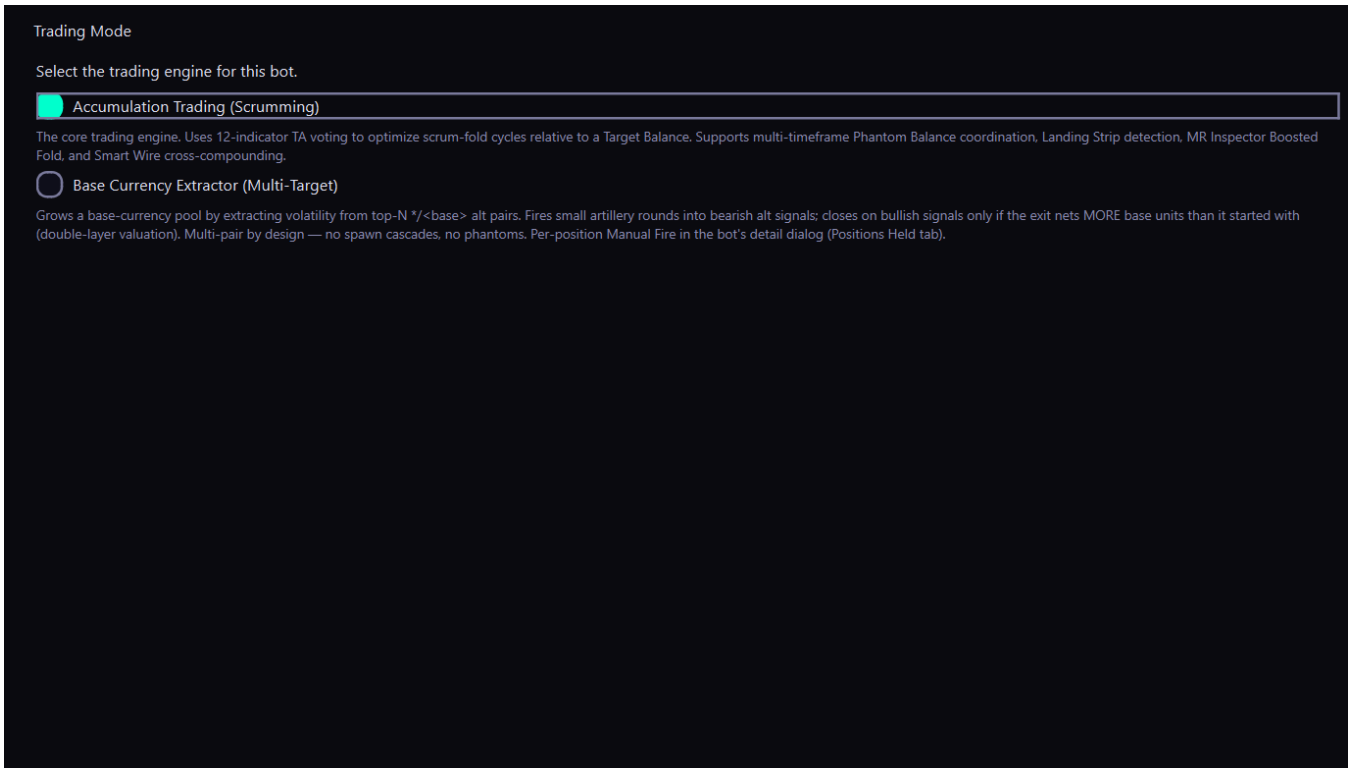
Scrum - To sell an amount from an investment position that allows it to return to its initial price level. This never closes the position. This is the first half of the infinitely divisible circle that can persist for such positions in a healthy market.

One shared helper decides which half of the cycle a position sits in. It answers scrum above the target and fold below it, and it answers at target inside a dust band, so a position that has barely moved is left alone.

`src/trading/target_bands.py` — `target_territory`

```
delta = float(position_value) - float(target_balance)
band = at_target_dust_band(target_balance)
if delta > band:
    return "scrum"
if delta < -band:
    return "fold"
return "at_target"
```

Fold - To buy an amount for an investment position that allows it to return to its initial price level. This also drives Compounding Growth based on Local Volatility and is restricted by the Maximum Growth Per Cycle setting which has been set to a conservative 1% globally for Acervator’s live test and development run.



The wizard's first page: Trading Mode, with Scrumming selected.

When the +New Bot button is pressed, the above window appears. Currently there are two “trading mode”

options (this will be change to Strategies). We are only interested in the Scrumming Bot at this point.

**Functional.** This is the wizard's first page, and the wizard opens on it. Two radio buttons, one description under each. Accumulation Trading is already selected when the page opens, and its description promises 12-indicator voting, which is what the engine builds. Choosing Base Currency Extractor sends you to the Extractor Pool page instead of the asset page. One question decides that branch, and the rest of the wizard asks the same question whenever it needs to know which kind of bot it is building.

A third mode, Grid, is retired. Its test returns False without looking at a widget, and the branch that reads it is marked unreachable in the source.

`src/gui/bot_wizard.py` — `BotCreationWizard.nextId`

```
def nextId(self):
    current = self.currentId()
    if current == PAGE_MODE:
        if self._mode_page.is_extractor():
            return PAGE_EXTRACTOR_POOL
        return PAGE_ASSET
```

**Functional.** In the React build the wizard is drawn by `bot_wizard.js` into a space the exchange screen keeps for it. The screen keeps that space only after + New Bot is pressed. The press asks the surface, and the surface answers with the name of the module that draws the wizard.

`src/gui/main_tabs/exchange_tab_surface.py` — `react_bot_wizard`

```
def react_bot_wizard(exchange_id: str) -> str:
    """The module that draws the Create Auto Trader wizard for ``exchange_id``.

    ``ExchangeTabModel`` calls this as its ``on_new_bot``, so
    ``on_new_bot_clicked`` puts the module name in ``bot_wizard``.
    """
    return BOT_WIZARD_MODULE
```

**Functional.** Cancel closes the wizard. Finish closes it from the last page and refuses from any other. Either close drops the module the screen holds, so the space goes with the wizard and the exchange screen underneath it is whole again.

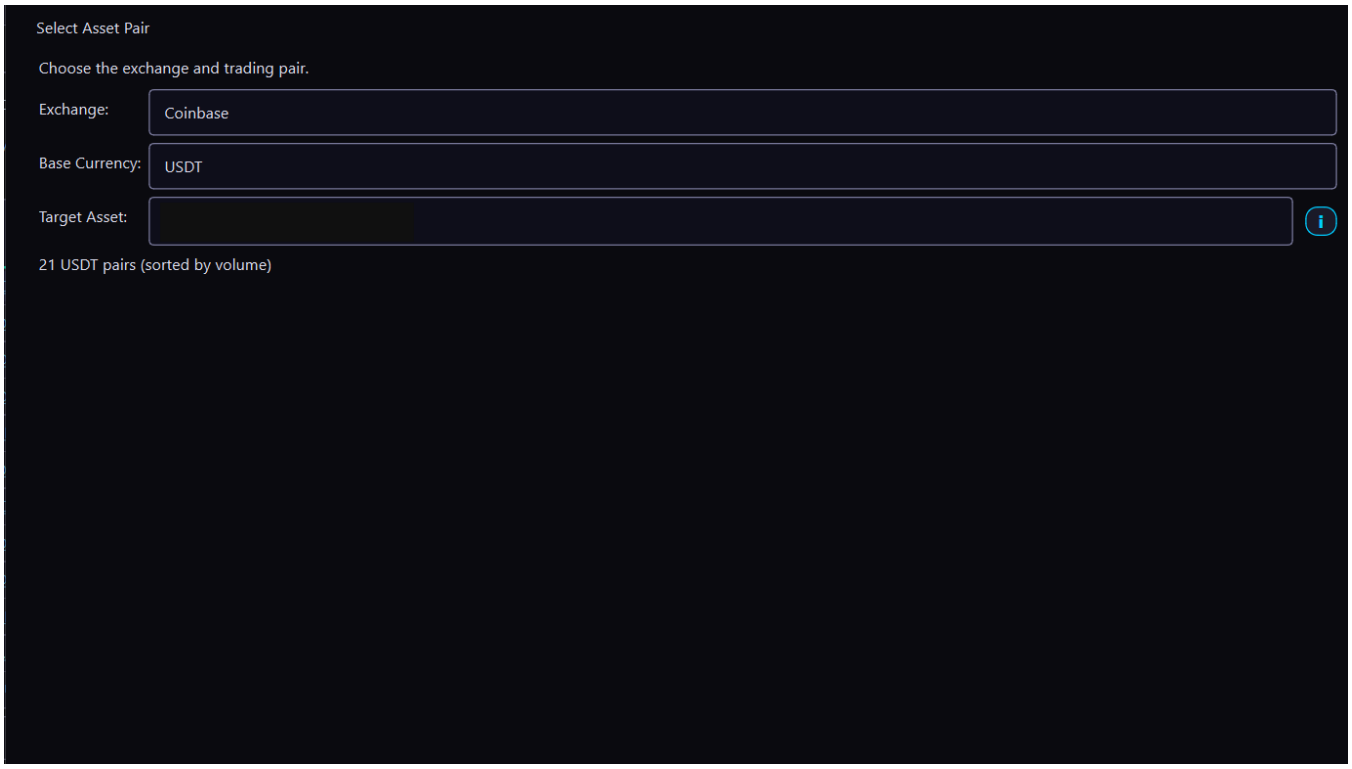
`src/gui/main_tabs/exchange_tab_surface.py` — `ExchangeTabModel.close_bot_wizard`

```
def close_bot_wizard(self) -> None:
    """Drop the wizard, so the screen keeps no space for it."""
    if self.bot_wizard is None:
        return
    self.bot_wizard = None
    self.calls.append([BOT_WIZARD_CLOSED, self.exchange_id])
```

**Functional.** Every control on the page sends the steps taken so far, not the one just pressed. The surface lays out fresh pages on each call and keeps nothing between them, so the page holds the walk and sends the whole of it. The answer replaces the payload the page holds and the wizard draws again.

`src/gui/web/bot_wizard.js` — `press`

```
function press(name, step) {
  lastPress = { name: name, step: step };
  dispatched.push(lastPress);
  remember(step);
  lastAnswer = hasBridge()
    ? global.acervator.call(METHOD, copyOf(heldSteps)).then(take)
    : Promise.resolve(null);
  return lastPress;
}
```



The wizard's asset page: exchange, base currency and target asset.

After Scrumming / Accumulation is selected, next we are presented with Exchange, Base Currency, and Target Asset options.

Exchange - A privately and federally licensed financial platform that allows API interfacing for remote or automated trade execution.

The row lists the venues already connected. Picking one re-scans that venue and keeps only the spot markets the venue itself marks active.

`src/gui/bot_wizard.py` — the market filter inside `_fetch_markets`

```
for sym, info in exch.markets.items():
    if (
        not info.get("active", True)
        or info.get("type", "spot") != "spot"
    ):
        continue
```

Base Currency - This will be a National Currency, Stable Coin, or High Volume Crypto (A15, A14, BNB) for which multiple trading pairs are available on the selected exchange.

The page offers a fixed list of seven, and it holds exactly the national currencies, stable coins and high-volume crypto named above.

`src/gui/bot_wizard.py` — `AssetSelectionPage`, the base list

```
self._base.addItems(["USDT", "USDC", "A15", "A14", "BNB", "EUR", "USD"])
```

Target Asset - This is the asset in which the scrumming bot bases its position. If $50 and A05 are selected, it will strive to maintain and compound against a balance of $50 in A05 until it is shut down or some other adverse market condition occurs.

The list holds every asset trading against the base you chose, each with a cached coin icon and, where the venue reported one, a 24-hour volume figure. The round information button opens a written description of the selected asset, and the line under the rows counts the pairs found. Sorting by volume puts the tradeable pairs at the top of a list that runs to several hundred entries on a large venue, and the sort reads a cached figure rather than fetching one.

`src/gui/bot_wizard.py` — the volume sort

```
# Cached volume only, since a network fetch here blocks the GUI thread.
filtered.sort(
    key=lambda m: _market_number(m.get("volume")) or 0.0, reverse=True
)
```

**The block above is overtaken.** `_market_number` was the wizard's own reading rule. It read either infinity as a volume and it raised on a saved number too large to be a float, which lost the whole pair list. The wizard now reads through the one admission rule the trading package owns, and a refused volume sorts as `VOLUME_REFUSED_USD` and labels as nothing.

`src/gui/bot_wizard.py` — the volume sort as it stands

```
filtered.sort(
    key=lambda m: as_finite_float(m.get("volume")) or VOLUME_REFUSED_USD,
    reverse=True,
)
```

### Trading Parameters

Next we come to the combined Scrumming Bot configuration page which has several sections for fine tuning how the specific instance behaves. Given that Acervator, at the time of this writing, is still in active development the description for each setting should be seen as a design intention should any issues be encountered.

The Trading Parameters group of the wizard's parameter page.

Order Visibility - Trades are listed on the order books or tracked internally to the platform.

Two entries, Order Book (Visible) and Internal (Invisible). `ScrummingBot` reads the choice once at construction and holds it as its invisible flag. The label a reader sees and the value the bot stores are two different strings on the same row.

`src/gui/bot_wizard.py` — the Order Visibility row

```
self._visibility = QComboBox()
self._visibility.addItem("Order Book (Visible)", "orderbook")
self._visibility.addItem("Internal (Invisible)", "internal")
self._visibility.setToolTip("How orders appear on the exchange.")
self._visibility.currentIndexChanged.connect(self._on_visibility_changed)
mf.addRow("Order Visibility:", self._visibility)
```

Driven on a bot rebuilt from a stored record, the two choices take different branches on the same tick: Internal activated all three stored Stack tranches, and Order Book activated none of them. The order type an engine trade carries is chosen off the same flag, on the sell side and the buy side alike.

`src/trading/scrumming/execution.py` — the sell side's order type

```
if self._invisible:
    ot = OrderType.MARKET
    exec_price = None
else:
    ot = OrderType.LIMIT
    exec_price = vh_fp
```

Aggressive Trading - Trades are priced so that they fill immediately. Trading like this is a bit like guerilla warfare. In and out before anyone notices.

A checkbox, off at the start. Every engine-initiated order then leaves as an immediate-or-cancel limit priced through the spread, so it pays the taker fee for an immediate fill. Manual Fire is unaffected.

`src/gui/bot_wizard.py` — the Aggressive Trading row

```
self._aggressive = QCheckBox("Aggressive Trading (force IOC-limit takers)")
```

The box's value reaches the bot and reaches no order. One place in the trading package builds an immediate-or-cancel order, inside the method that opens a Stack from a SCRUM, and that method raises before it gets there. The two places that do choose an order type read Order Visibility and never this flag.

`src/trading/scrumming_bot.py` — the only reader that would change an order

```
_aggressive = bool(getattr(self, "_aggressive", False))
_order_type_for_visible = (
    OrderType.IOC_LIMIT if _aggressive else OrderType.LIMIT
)
```

Stack Mode - Stack Mode enables Stack Tranches which operate on the Sell or Scrum side. This forms the “upside” of the organic ladder structure whereas Fold Tranches form its “downside”.

The box takes its start state from the engine rather than from a second literal typed onto the page.

`src/gui/bot_wizard.py` — `TradingParamsPage`, the Stack Mode default

```
from ..trading.bot_container import STACK_MODE_DEFAULT

self._stack_mode = QCheckBox(
    "Stack Mode (split SCRUM across upward tranches)"
)
# One declaration, so the box and the config cannot disagree.
self._stack_mode.setChecked(STACK_MODE_DEFAULT)
```

The box's value arrives on the bot and gates four places: the branch that turns a SCRUM into a Stack, and the three reconciliation stages. Driven, the guard discriminates — on, all three stored tranches activated; off, none did. No tranche can be created today, because the one method that appends one raises on an attribute no object in the tree carries. That raise is the reason the four rows below report a value that arrives and a behaviour that waits.

`src/trading/scrumming_bot.py` — the refusal, read from a restored bot

```
AttributeError: 'ScrummingBot' object has no attribute 'exchange_interface'
```

Split Distance - This setting determines the spacing between tranches if Tranche Spread (not available yet) is being used.

A percentage from 0.10 to 20.00, at 1.00 % to start. The bot hands it to the stack maths as the gap between one tranche and the next, and the Spacing row below decides how that gap grows across the ladder.

`src/gui/bot_wizard.py` — the Split Distance row

```
self._split_distance = QDoubleSpinBox()
self._split_distance.setRange(0.1, 20.0)
self._split_distance.setDecimals(2)
self._split_distance.setSuffix(" %")
self._split_distance.setValue(1.0)
```

Driven at the maths the bot hands it to, 1.00 % priced three rungs a gap of 0.9901 and 0.9804 per cent apart, and 2.50 % priced the same three 2.439 and 2.381 per cent apart. The figure reaches that maths only through the method named under Stack Mode above, so the value is correct and the ladder is not yet built.

`src/trading/scrumming_bot.py` — the hand-off

```
split_dist = float(getattr(self.config, "split_distance", 1.0) or 1.0)
```

Tranche Spread - This allows a given Scrum or Fold to divide its result X# of times across multiple incremented (as dictated by Split Distance) positions.

The page carries no control of that name, and no setting named `tranche_spread` reaches the engine.

The same sweep run for `split_distance` returns a declaration, a restore entry and a reader, so the sweep itself finds a setting when one is there.

In development.

The sentence that opens this row is retired. Counted over the whole tracked tree, the name appears once, and that once is this page describing its own absence. Spacing is the control that exists and is read, and it names the four models the engine carries. Tranche Count decides how many pieces a Scrum becomes, and Spacing decides where each piece sits.

`src/trading/stack_math.py` — the models Spacing chooses from

```
SPACING_MODES = ("quadratic", "fibonacci", "linear", "exponential")
```

Tranche Count - This can also be referred to as Spread Count. It determines how pieces a given Scrum or Fold is split into and distributed across incremented tranches as opposed to just one.

A whole number from 2 to 20, at 3 to start, written out as `stack_tranche_count_target`. It is a target rather than a promise: the runtime count drops when a tranche would fall under the venue minimum, or when two computed prices land within a tenth of a percent of each other and merge.

`src/gui/bot_wizard.py` — the Tranche Count row

```
self._stack_count = QSpinBox()
self._stack_count.setRange(2, 20)
self._stack_count.setValue(3)
```

Driven at the same maths, a target of 3 built three rungs and a target of 7 built seven, each rung's size falling from 0.4 to 0.171429 units out of one 1.2-unit Scrum. The count reaches that maths through the method named under Stack Mode, so it too is a correct value waiting on a ladder.

`src/trading/scrumming_bot.py` — the hand-off

```
n_target = int(getattr(self.config, "stack_tranche_count_target", 3) or 3)
```

Spacing - This adds a scaling factor Split Distance and works in conjunction with Tranche Spread and Tranche Count to induce curves and more aggressive growth within the ladder structure.

Three entries: Linear, Quadratic and Exponential. The sequence beside each name is the cumulative distance from the anchor in units of Split Distance.

`src/gui/bot_wizard.py` — the Spacing row

```
self._stack_spacing = QComboBox()
self._stack_spacing.addItem("Linear (1, 2, 3, 4…)", "linear")
self._stack_spacing.addItem("Quadratic (1, 2, 4, 7…)", "quadratic")
self._stack_spacing.addItem("Exponential (1, 2, 4, 8…)", "exponential")
```

The Quadratic entry read 1, 2, 4, 7 and the engine prices that model at n squared, so the row named a ladder the engine has never built. Both the wizard row and the live bot window now read 1, 4, 9, 16, which is what the engine publishes. The engine also carries a fourth model, Fibonacci at 1, 2, 3, 5, that neither control offers.

`src/trading/stack_math.py` — `level_multipliers`, driven for four levels

```
linear       [1.0, 2.0, 3.0, 4.0]
quadratic    [1.0, 4.0, 9.0, 16.0]
exponential  [1.0, 2.0, 4.0, 8.0]
fibonacci    [1.0, 2.0, 3.0, 5.0]
```

Personal Hold - Setting to be removed.

The control is still on the page. `TradingParamsPage.get_config` still emits `personal_hold_qty`, and the capital reservation mixin still reads it, adding it to the units the bot claims against its siblings. A removal has that reader to retire with it.

`src/trading/scrumming/capital_reservation_mixin.py` — `_compute_reservation_qty`

```
_hold = float(getattr(self.config, "personal_hold_qty", 0.0) or 0.0)
```

`src/gui/bot_wizard.py` — `TradingParamsPage.get_config`, the settings this group emits, in the order of the rows above

```
"stack_mode": self._stack_mode.isChecked(),
"split_distance": self._split_distance.value(),
"stack_tranche_count_target": int(self._stack_count.value()),
"stack_spacing_mode": self._stack_spacing.currentData(),
"personal_hold_qty": float(self._personal_hold_qty.value()),
```

The same method writes `visibility` and `aggressive_trading` before it branches on the kind of bot, so an Extractor carries those two as well.

Five sites read the number, not one. Two of them change what the engine decides. The other three carry the value to those two, or draw the row the operator types into.

```
src/trading/container/restore.py:254                    stored key into the kwarg
src/trading/scrumming/capital_reservation_mixin.py:67   the size of the claim
src/trading/scrumming/capital_reservation_mixin.py:134  the text of the claim
src/trading/scrumming/reconciliation.py:412             the adoption ceiling
src/gui/live_settings/settings_tab.py:304               seeds the live row
```

Both deciding readers are reached on every run. The claim is sized at the top of every tick. The adoption ceiling is read once when a bot starts, and again every reconcile interval after that.

```
src/trading/scrumming_bot.py:2571   the claim, once per tick
src/trading/bot_container.py:891    the ceiling, on bot start
src/trading/scrumming_bot.py:2643   the ceiling, every reconcile interval
```

The adoption ceiling is the reader that bears on a sale. A bot whose venue balance stands above its own book adopts the difference as a lot, and the hold is subtracted before that adoption, so the held units never enter the book the sell pre-check is measured against.

```
src/trading/scrumming/execution.py:1408 — the sell pre-check reads the book
```

Driven on a restored record carrying a five-unit hold, with twenty units in the book, thirty at the venue and a price of one hundred, against the same code with all five readers taken out:

```
with the readers    claimable 25.0   adopted 5.0    book 25.0
readers removed     claimable 30.0   adopted 10.0   book 30.0
```

That bot may sell 25 units as the code stands and 30 with the readers gone. The five it holds back are the hold itself. The claim a bot places against its siblings moves the same way, 16.0 units down to 11.0, which raises a co-tenant bot's own permitted sale from 14.0 units to 19.0.

```
src/trading/capital_reservation.py:443 — effective_available, the co-tenant's cap
```

The removal is not taken. Stored state does load without the value — three records of three restored, 26 of 27 stored fields identical, none different, and nothing raised — but a bot carrying a hold would be free to sell the units the hold keeps back. The removal is safe only for a bot whose hold reads zero.

The Scrumming Settings group.

Scrolling down we next find the first block of Scrumming Settings. These are the core or basic metrics for a Scrumming Bot.

Opposing Trade Interval - Establishes the minimum travel distance required by price action from the point a given trade in order the next trade of the opposite type to occur.

A percentage from 0.10 to 20.00, at 1.00 % to start. The Trading Fee below is added to it, so the real distance a reversal must travel is the two together.

`src/gui/bot_wizard.py` — the Opposing Trade Interval row

```
self._scrumming_interval = QDoubleSpinBox()
self._scrumming_interval.setRange(0.1, 20.0)
self._scrumming_interval.setDecimals(2)
self._scrumming_interval.setSuffix(" %")
self._scrumming_interval.setValue(1.0)
```

Driven on a bot restored from a stored configuration, the percentage sets two thresholds. One is the dollar drift the position must show before the tick looks at anything else. The other is the price a reversal must reach after an opposite trade, and the Trading Fee is inside both of them.

```
target $200.00
  interval 1.00 %   drift threshold $2.00    the tick reads the gates
  interval 5.00 %   drift threshold $10.00   the tick holds and says so

pivot $100.00000000, one tick later at $101.00000000
  interval 0.10 %   required $100.70000000   cleared, no hysteresis blocker
  interval 1.00 %   required $101.60000000   hysteresis blocker raised
  interval 5.00 %   required $105.60000000   hysteresis blocker raised
```

Asked for a figure under its floor the box answers the floor: asked for 0.00 it answers 0.10.

BB Tolerance - Determines the minimum distance of the Bollinger Band extent price action must be in order for a trade action to occur.

A percentage from 0.25 to 5.00, at 1.00 % to start. The band-proximity detector takes it as its tolerance. This is the narrowest range on the page, and it cannot be set to zero.

`src/gui/bot_wizard.py` — the BB Tolerance row

```
self._bb_tolerance = QDoubleSpinBox()
self._bb_tolerance.setRange(0.25, 5.0)
self._bb_tolerance.setDecimals(2)
self._bb_tolerance.setSuffix(" %")
self._bb_tolerance.setValue(1.0)
```

Driven on one tape whose last close sat at band position 0.976123, the tolerance decides whether that close counts as sitting at the upper band, and the landing strip follows that answer. Asked for 0.00 the box answers 0.25, so it cannot be set to zero.

```
band position 0.976123, one tape, one tick
  tolerance 0.25 %   near upper False   landing strip False
  tolerance 5.00 %   near upper True    landing strip True, upper side
```

Landing Strip Candles - Determines the strictness of Landing Strip detection. Minimum is three candles. Longer Landing Strips are historically more likely to indicate an impending market reversal than shorter ones assuming the taper remains intact or grows tighter.

A whole number of candles from 2 to 10, at 3 to start. The widget accepts 2, one below the minimum of three the description names, and the number reaches only one of the two landing-strip detectors. Issue #432 carries this.

`src/gui/bot_wizard.py` — the Landing Strip Candles row

```
self._ls_candles = QSpinBox()
self._ls_candles.setRange(2, 10)
self._ls_candles.setValue(3)
self._ls_candles.setSuffix(" candles")
```

Both halves of the correction above were re-measured, and both hold. The box does accept 2: its declared floor is 2, asked for 0 or 1 it answers 2, and 2 is the one figure under three it keeps. The number does reach one

detector of the two the tick runs over the same candles.

```
the tick runs two detectors, and they count different things
  band proximity   its minimum pattern candles = the stored figure
  tightening       its minimum consecutive run = a platform constant of 3

one tape, trailing tight bodies 2
  stored 2    landing strip True     confidence boost 0.1900
  stored 3    landing strip False    confidence boost 0.0000
  stored 2 and stored 12 both logged: TIGHTENING (upper BB): 3 candles
```

Raising the floor to three is not taken, and the reason is a measurement. A box holding a stored 2 answers 3 the moment its range starts at three, and putting the range back leaves the 3 behind, so the next Save would write three into a bot whose record says two.

The tightening detector keeps its own run length, because a run of shrinking bodies is not a count of tight bodies at a band. The four figures the tick hands that detector are now the names the gate scan already declared for them, and the values are unchanged.

```
tightening = detect_landing_strip_v2(
    candles,
    min_consecutive=TIGHTENING_MIN_CONSECUTIVE,
    shrink_threshold=TIGHTENING_SHRINK_THRESHOLD,
    bb_tolerance_pct=TIGHTENING_TOLERANCE_PCT,
)
```

TA Timeframe - This is the timeframe at which the bot operates and denotes the price chart it will monitor for trade decisions.

Seven entries to start, with 1h chosen. Pick an exchange and the list is rebuilt from the timeframes that venue carries, so a venue without 4h does not offer it. The starting choice is an index into the list rather than a named timeframe.

`src/gui/bot_wizard.py` — the TA Timeframe row

```
self._ta_timeframe = QComboBox()
```

```
self._ta_timeframe.setCurrentIndex(4)  # Default 1h
```

Driven on a bot restored from a stored configuration, the name decides which chart the bot fetches, and every indicator vote is then computed on that chart.

```
one bot, one tick each, two stored names
  1h   chart asked for 1h   consensus BULLISH 0.0219   band position 0.500000
  1d   chart asked for 1d   consensus BEARISH 0.0863   band position 0.976123
```

Target Balance - This is the intended starting and locked value for the investment position that the Scrumming Bot is controlling.

From $1.00 to $1,000,000.00. Its start value is whatever default the wizard was handed, which is the figure the Settings Trading page stores.

`src/gui/bot_wizard.py` — the Target Balance row

```
self._target_balance = QDoubleSpinBox()
self._target_balance.setRange(1.0, 1000000.0)
self._target_balance.setDecimals(2)
self._target_balance.setPrefix("$ ")
self._target_balance.setValue(defaults.get("default_target_balance", 200.0))
```

Driven on one position worth $203.00, the figure is the line the excess is measured from, and the Opposing Trade Interval above is a percentage of it. A dollar and a half of target turns the same position from one the tick evaluates into one it holds.

```
one position of $203.00, one tick each
  target $200.00   excess +$3.00   scrum side   interval $2.0000   gates read
  target $201.50   excess +$1.50   scrum side   interval $2.0150   tick holds
```

The start value does come from the figure the wizard is handed: handed 200.00 the row opens at 200.00, and handed 777.50 it opens at 777.50. Asked for 0.00 the box answers 1.00.

Max Entry Price - If the new bot does not detect the requisite amount (as dictated by Target Balance) of the Target Asset, this price threshold sets a limit at which it will attempt to perform the initiating Fold.

Eight decimal places, at $0.00000000, where zero means no ceiling. The buy path is the only reader, and it refuses a buy above the ceiling. Manual Fire goes around it.

`src/gui/bot_wizard.py` — the Max Entry Price row

```
self._max_entry_px = QDoubleSpinBox()
self._max_entry_px.setRange(0.0, 10_000_000.0)
self._max_entry_px.setDecimals(8)
self._max_entry_px.setPrefix("$ ")
self._max_entry_px.setValue(0.0)
```

Min Entry (Should Be Exit) Price - If the new bot detects a requisite amount (as dictated by the Target Balance) of the Target Asset, this price threshold sets a limit at which it will attempt to perform the initiating Scrum.

The same shape, where zero means no floor. The buy path is the only reader here too, so the number refuses a buy below the floor and never reaches a sell. The row is labelled Min Entry Price on the page, without the parenthesis his line carries.

`src/gui/bot_wizard.py` — the Min Entry Price row

```
self._min_entry_px = QDoubleSpinBox()
self._min_entry_px.setRange(0.0, 10_000_000.0)
self._min_entry_px.setDecimals(8)
self._min_entry_px.setPrefix("$ ")
self._min_entry_px.setValue(0.0)
```

Trading Fee % - This allows the trading fee for the target exchange to be set. This is added to Minimum Opposing Trade Distance to further ensure buys / sells are properly distant and that a given bot is not losing an excessive amount to fee chop in volatile but overly tight market regimes.

A percentage from 0.00 to 5.00, at 0.60 % to start, which is the Coinbase Advanced Trade maximum tier. It steps in twentieths of a percent, and it is a per-side figure.

`src/gui/bot_wizard.py` — the Trading Fee row

```
self._trading_fee = QDoubleSpinBox()
self._trading_fee.setRange(0.0, 5.0)
self._trading_fee.setSuffix(" %")
self._trading_fee.setDecimals(2)
self._trading_fee.setSingleStep(0.05)
self._trading_fee.setValue(0.6)
```

Max Target Growth % - This determines the maximum amount of growth the Target Balance can increase in a given Market Cycle with a cycle being a Fold / Scrum / Fold sequence. Essentially any Fold preceded by a Scrum will be allowed to Fold an amount of profit back in and, if Surplus remains after the Target Delta is re-zero'd, it can be used to increase Target Balance up to this hard limit for that cycle. This is the organic compounding mechanic.

A percentage from 0.00 to 100.00, at 1.00 % to start. Setting it to zero freezes Target Balance. It steps a quarter of a percent at a time, and it is the only mechanism allowed to raise the figure.

`src/gui/bot_wizard.py` — the Max Target Growth row

```
self._max_target_growth_pct = QDoubleSpinBox()
self._max_target_growth_pct.setRange(0.0, 100.0)
self._max_target_growth_pct.setSuffix(" %")
self._max_target_growth_pct.setDecimals(2)
self._max_target_growth_pct.setSingleStep(0.25)
self._max_target_growth_pct.setValue(1.0)
```

Scrum Fold Ratio - This precedes Surplus calculation as described under Max Target Growth %. It determines how much of a given trade's profits will be redistributed to directly contribute to its own organic compounding. Note that this does not interfere with normal Target Delta re-zeroing and is intended to only serve as a "cushion" to slow runaway compounding when Wire Credits are being received from multiple sources.

A whole percentage from 1 to 100, at 100 % to start.

`src/gui/bot_wizard.py` — `TradingParamsPage.get_config`, the settings this group emits, in the order of the rows above

```
"scrumming_interval_pct": self._scrumming_interval.value(),
"bb_tolerance_pct": self._bb_tolerance.value(),
"bb_landing_strip_candles": self._ls_candles.value(),
"ta_timeframe": self._ta_timeframe.currentData(),
"target_balance": self._target_balance.value(),
"max_entry_price": (float(_max_ep) if _max_ep > 0 else None),
"min_entry_price": (float(_min_ep) if _min_ep > 0 else None),
"trading_fee_pct": self._trading_fee.value(),
"max_target_growth_pct": self._max_target_growth_pct.value(),
"scrum_fold_pct": self._scrum_fold_pct.value(),
```

**Where the code departs.** Two of these rows read differently in the engine than the entries above say.

Min Entry Price is the first. The entry above writes the correction into its own heading, and the code follows the label on screen rather than that correction. The buy path is the only place either entry-price field is read; it refuses a buy above the ceiling and refuses a buy below the floor. Neither number reaches the sell path, so a bot that holds the asset and reaches that price sells nothing. Manual Fire runs its own rebalance and reads neither.

`src/trading/scrumming/execution.py` — `_execute_buy`

```
_max_ep = getattr(self.config, "max_entry_price", None)
_min_ep = getattr(self.config, "min_entry_price", None)
try:
    _px = float(price) if price is not None else 0.0
except (TypeError, ValueError):
    _px = 0.0
if _max_ep is not None and _px > 0 and _px > float(_max_ep):
```

The sell path already takes the price, so the floor check has somewhere to attach.

*Proposed, not present, in* `_execute_sell`*:*

```
_min_ep = getattr(self.config, "min_entry_price", None)
if _min_ep is not None and price > 0 and price < float(_min_ep):
    self._bus.emit(
        "bot.log",
        bot_id=self.bot_id,
        message=(
            f"SELL REFUSED (min_entry_price floor): current price "
            f"${price:.8f} < ${float(_min_ep):.8f}."
        ),
    )
    return None
```

`min_entry_price` is declared in `src/trading/container/config.py` and reaches the bot through the wizard block above, so the proposal adds no new setting.

Max Target Growth % is the second. The engine holds two answers for a bot whose stored config lacks the key. Ten read sites take it with a fallback: seven fall back to 1.0 and three fall back to 0.0. A bot compounds or freezes depending on which site read it first. The declared default is 1.0, and the three outliers should say the

same.

*Proposed, not present, at each of the three sites:*

```
_growth = float(getattr(self.config, "max_target_growth_pct", 1.0) or 0.0)
```

The three outliers are the manual rebalance, the compounding snapshot and the SWOS inputs. Issue #409 carries this.

```
src/trading/scrumming/execution.py   _execute_manual_rebalance
src/trading/scrumming/snapshots.py   _compounding_snapshot
src/trading/scrumming_bot.py         get_swos_inputs
```

The Advanced Scrumming and Hedge Rebalance groups.

Next are the “advanced” Scrumming settings which primarily affect when the bot is allowed to fire a trade. These can be thought of as “calibrating the scope”.

Detect Threshold - Intended as the point at which the bot “takes the safety off” and starts looking for a shot. To be re-evaluated.

A whole percentage from 10 to 90, at 75 % to start. The engine reads 75 as a lower mark of 0.125 and an upper mark of 0.875, measured across the band rather than in dollars.

`src/trading/scrumming/circuit_breakers.py` — `_bb_detect_thresholds`

```
detect_frac = max(0.0, min(1.0, detect_pct / 100.0))
half = detect_frac * 0.5
return (0.5 - half, 0.5 + half)
```

Driven on one 40-candle tape, both marks move with the figure: 10 gave 0.45 and 0.55, 50 gave 0.25 and 0.75, 75 gave 0.125 and 0.875, and 90 gave 0.05 and 0.95. The tick reads the pair twice, and the upper mark is a hard gate. On a tape at band position 0.75 the figure at 75 held the scrum and named two blockers, and the same tape at 40 cleared both and armed it.

`src/trading/scrumming_bot.py` — the two readings in one tick

```
_bb_lower_dt_pre, _bb_upper_dt_pre = self._bb_detect_thresholds()
```

Fire Threshold - The final Bollinger Band approach metric. Once satisfied, the bot can fire a trade.

A percentage from 0.10 to 10.00, at 0.50 % to start. It measures distance from the band, where the Detect Threshold above it measures distance from the midline.

`src/gui/bot_wizard.py` — the Fire Threshold row

```
self._scrum_fire_pct = QDoubleSpinBox()
self._scrum_fire_pct.setRange(0.1, 10.0)
self._scrum_fire_pct.setDecimals(2)
self._scrum_fire_pct.setSuffix(" %")
self._scrum_fire_pct.setValue(0.5)
```

The ramp's near-band test is the only place this figure lands. Driven on one tape at band position 0.75, 0.50 % and 2.00 % each left the ramp short of FIRE and the scrum blocked; 10.00 % took the ramp from SEARCH to FIRE in one tick and removed that blocker.

`src/trading/scrumming/tick_phases.py` — the near-band test

```
fire_pct_frac = self.config.scrum_fire_pct / 100.0
_near_band = abs(_price - _bb_upper) <= fire_pct_frac * _bb_upper
```

BB Midline Gate - This is another, perhaps redundant layer, of Bollinger Band travel protection. It is different in that it is concerned with distance from the midline instead of the entire local width.

A checkbox, on at the start. While it is on, a scrum fires only above the midline and a fold only below it.

`src/gui/bot_wizard.py` — the BB Midline Gate row

```
self._bb_midline_gate = QCheckBox("BB Midline Gate")
self._bb_midline_gate.setChecked(True)
```

Driven on one tape at band position 0.27 with the position above target: on, the tick refused the scrum and named the band gate in its blocker list; off, that blocker was gone. The fold half of the same reading stayed open both ways, because the price sat below the midline.

`src/trading/scrumming_bot.py` — the midline block

```
if self.config.bb_midline_gate:
    scrum_ok = (bb_pos > 0.50) and not self._phantom_locked
    fold_ok_midline = bb_pos < 0.50
else:
    scrum_ok = not self._phantom_locked
    fold_ok_midline = True
```

Read Rate - To be re-evaluated.

From 1 to 60 minutes, at 5 minutes to start. It sets the search-mode read rate; track mode reads ten times faster. Fire mode reads at that same faster rate.

The wizard writes it as `scrum_read_rate_min`, bot creation passes it through, and the bot reads it at the top of every tick. The range starts at 1, so this control cannot switch the throttle off.

`src/trading/scrumming_bot.py` — `ScrummingBot.tick`, the throttle

```
if self.config.scrum_read_rate_min > 0 and not self._manual_fire_pending:
    _tick_sec = max(self.tick_interval, 0.1)
    _base_skip = max(1, int((self.config.scrum_read_rate_min * 60) / _tick_sec))
    self._tick_skip_search = _base_skip
    if self._scrum_target_mode in ("track", "fire"):
        self._tick_skip = max(1, _base_skip // 10)
    else:
        self._tick_skip = _base_skip
```

The tick itself runs every five seconds, so the figure in minutes becomes a count of skipped ticks. Driven over eighty ticks: 1 minute skipped 12 and worked 6, 5 minutes skipped 60 and worked 1, and 60 minutes skipped 720 and worked none. A skipped tick fetches no price and runs no gate.

`src/trading/scrumming_bot.py` — the tick period the count divides

```
@property
def tick_interval(self) -> float:
    return 5.0
```

Band Travel - Previously described. To be re-evaluated.

A whole percentage from 0 to 100, at 70 % to start. Zero switches it off. The wizard writes it as `band_travel_pct` and bot creation passes it through.

The engine reads it once per evaluation. It measures how far price has moved since the last trade, as a share of the current band width, and raises a second trigger once price covers that share. The same trigger overrides a trend hold, so Band Travel can release a trade the trend gates were holding.

`src/trading/scrumming_bot.py` — the band-travel trigger

```
_bb_width = max(bb_result.upper - bb_result.lower, 1e-12)
band_travel_frac = abs(ticker.last - self._last_trade_price) / _bb_width
if (
    abs(ticker.last - self._last_trade_price)
    >= self.config.band_travel_pct / 100.0 * _bb_width
    and delta > 0
):
    band_travel_triggered = True
```

Driven on one rising tape where 95 % of the last twenty candles closed up, with price 37 % of the band width above the last trade: 0 left the trend hold in place and suppressed the scrum, 30 raised the trigger and the override released it, and 70 left the hold in place again, because 37 is under 70. The figure decides, not an on and an off.

`src/trading/scrumming_bot.py` — the override the trigger feeds

```
trend_override = abs(delta) >= _interval_usd * 2.0 or band_travel_triggered
if trend_hold and trend_override:
```

BB Bullseye Check - If current price and Bollinger Band thresholds are equal, the user can opt to perform a double-sized trade.

A checkbox, on at the start.

`src/gui/bot_wizard.py` — the BB Bullseye row

```
self._bb_bullseye = QCheckBox("BB Bullseye Check")
self._bb_bullseye.setChecked(True)
```

A double size is not in the engine. Nothing sizes a trade off this box. Its one decision reader counts a band touch within 0.5 %, or a candle wick within 0.2 %, as band proximity, and proximity with the delta at or over the interval arms the BB priority skew. Driven on one tape with price at the upper band, on emitted the skew and took the confidence floor from 0.25 to 0.1923; off emitted nothing and left the floor where it was.

`src/trading/scrumming_bot.py` — the band touch the box admits

```
_bb_proximity_upper = (
    (bb_pos >= _bb_upper_dt_pre) or _be_upper or _be_upper_wick
)
```

It does not bypass the Fire Threshold. Driven with price 0.3 % under the upper band and the Fire Threshold at its lowest 0.10 %, the ramp stayed short of FIRE with the box on and with it off, and the scrum carried the same blocker both ways. What changes is the floor the TA confidence must clear.

`src/trading/scrumming_bot.py` — the floor the skew divides

```
_ta_conf_skew = position_boost + bb_confidence_boost
if _bb_priority_arm:
    _ta_conf_skew += _BB_PRIORITY_SKEW
_eff_conf_floor = _skewed_confidence_floor(_ta_conf_skew)
```

The second reader writes the two BULLSEYE notices onto the Activity Log and changes no decision.

`src/trading/scrumming/tick_phases.py` — the notice reader

```
if bullseye_upper or bullseye_upper_wick:
    _ramp = self._scrum_target_mode.upper()
    _fire_gate = "passes" if _ramp == "FIRE" else "holds"
```

Wire Inflow Stack - To be re-evaluated.

A percentage from 0.00 to 100.00, at 1.00 % to start. The wizard writes it as `wire_inflow_stack_pct`, and one method in the engine reads it. Above zero, wire income arriving while the position sits within that band of its target and within that band of its entry price goes onto the target and queues an aggressive buy, rather than spreading over the fold queue.

Bot creation does not pass this setting, so a new bot takes the declared default of 1.00 whatever you type here. The declared default and the wizard's start value are the same number, so the loss shows only once you change it. Bot Settings can set it on a bot that is already running. Issue #336 carries this.

`src/trading/scrumming/wire_routing.py` — `WireRoutingMixin.apply_wire_income`

```
try:
    stack_pct = float(getattr(self.config, "wire_inflow_stack_pct", 1.0) or 0)
except (TypeError, ValueError):
    stack_pct = 0.0
_stack_eligible = False
_stack_reason = ""
_target = 0.0
_entry_px = 0.0
if stack_pct > 0:
```

Bot creation passes it now. One declaration serves the wizard path and the restore path together, so every field the config declares reaches a new bot. Driven through the creation route the wizard uses, a stored 0.00 arrived on the bot as 0.00 rather than the declared 1.00, which is what the sentence above was written against.

`src/trading/container/config.py` — the one carried set

```
carried = {f.name for f in fields(BotConfig)} - foreign - {"mode"}
kwargs = {
    key: value
    for key, value in _sanitize_deprecated_kwargs(collected).items()
    if key in carried
}
```

The caller is live. A fold routes its compounded profit through Smart Wire, which hands the share to the target bot, and that is where this figure decides. Driven on one bot sitting at its target and at its entry price, 0.00 parked ten dollars as pending and left the target at 200.00, while 1.00 stacked it, took the target to 210.00 and queued ten dollars for the next tick.

`src/trading/scrumming/tick_phases.py` — the live caller

```
mgr.distribute_fold_profit(
    source_id=self.bot_id,
    profit_usd=float(_growth_applied),
    ref=f"fold-compound@{buy_fill:.8f}",
)
```

Hedge Rebalance Active - Determines if Current Price drifting below Initial Entry Price will have a limited amount of funds that can be used to keep re-zeroing the Target Delta at key bearish thresholds or areas of possible reversal.

A checkbox, on at the start. It opens the one group on the page that holds a reserve outside Target Balance.

`src/gui/bot_wizard.py` — the Hedge Rebalance row

```
self._hedge_rebalance = QCheckBox("Hedge Rebalance Active")
self._hedge_rebalance.setChecked(True)
```

Hedge Balance - Sets a limit on the amount of additional liquidity a given bot is allowed to absorb when Current Price drifts below Initial Entry Price.

At $200.00 to start, a reserve the bot holds apart from Target Balance.

`src/gui/bot_wizard.py` — `TradingParamsPage.get_config`, the settings these two groups emit, in the order of the rows above

```
"scrum_detect_pct": self._scrum_detect_pct.value(),
"scrum_fire_pct": self._scrum_fire_pct.value(),
"bb_midline_gate": self._bb_midline_gate.isChecked(),
"scrum_read_rate_min": self._scrum_read_rate.value(),
"band_travel_pct": self._band_travel_pct.value(),
"bb_bullseye_check": self._bb_bullseye.isChecked(),
"wire_inflow_stack_pct": self._wire_inflow_stack_pct.value(),
"hedge_rebalance_active": self._hedge_rebalance.isChecked(),
"hedge_balance": self._hedge_amount.value(),
```

The group title on screen carries an internal release identifier after the words Advanced Scrumming. Issue #420 carries that, and four more group titles with it.

The Circuit Breakers group.

Now we arrive at some safety controls. Circuit Breakers are designed to fully inhibit trade actions for a given period should an extreme volatility (pump and dump) event occur. Soft Circuit Breakers have a candle-count based timer whereas Hard Circuit Breakers require the user to clear the bot to continue trading.

Soft CB Threshold - The amount of instantaneous, single-candle price action required for the bot to pause trading for a number of candles denoted by Soft CB Cooldown.

A percentage from 0.0 to 100.0, at 25.0 % to start. Zero switches it off. It interrupts only the side of the market that moved, so an upward candle stops scrums and a downward one stops folds.

`src/gui/bot_wizard.py` — the Soft CB Threshold row

```
self._cb_soft_pct.setSuffix(" %")
self._cb_soft_pct.setValue(25.0)
```

The move the figure is compared against is the candle's own span, high minus low over open, as a percentage. The figure decides: on one candle spanning 24.9 %, a stored 10.0 % tripped the breaker and a stored 30.0 % did not, and on one spanning exactly 25.0 % a stored 25.0 % tripped. A stored 0.0 % left a 50 % candle alone, which is the off position the paragraph above describes.

The side follows the close against the open. A candle closing up shut the SCRUM side, a candle closing down shut the FOLD side, and a shut side refuses the trade at `circuit_breaker_scrum` or `circuit_breaker_fold` in the gate chain while the other side's chain is untouched. The breaker is read once per worked tick, and only a candle timestamp the bot has not seen can trip it, so one candle never trips twice.

Hard CB Threshold - The amount of instantaneous, single-candle price action required for the bot to be hard stopped at which point the user must re-authorize trading.

The same range, at 35.0 % to start. Zero switches it off. Where the soft breaker interrupts one side, this one pauses the bot outright and the pause survives a restart.

`src/gui/bot_wizard.py` — the Hard CB Threshold row

```
self._cb_hard_pct = QDoubleSpinBox()
self._cb_hard_pct.setRange(0.0, 100.0)
self._cb_hard_pct.setDecimals(1)
self._cb_hard_pct.setSuffix(" %")
self._cb_hard_pct.setValue(35.0)
```

Driven on the same span reading: at a stored 35.0 % a candle spanning 35.0 % tripped the hard breaker and paused the bot, one spanning 34.9 % did not, and a stored 40.0 % left the 35.0 % candle alone. A stored 0.0 % left a 90 % candle alone. Once tripped it holds every later candle, quiet ones included, and only the Reset All Breakers button clears it — the reset reports the trip percentage it cleared and returns the bot from paused to running. The pause is written into the bot's stored record, so a bot restored from a hard trip comes back paused.

The pause reaches the scrum and the fold gate chain. It does not reach the five trade paths the tick reads before it: Manual Fire, the Wire Stack fire, Max Cartridge, Detonation and the initial entry. Measured on a hard-tripped, paused bot with a position $30.00 past a $20.00 Max Cartridge threshold: the bot still wrote MAX CARTRIDGE FIRE and still sent the cartridge trade notification, while the same bot with Max Cartridge at 0.0 % wrote nothing. Nothing is changed here for that, because raising the hard breaker above the cartridge would stop a rebalance that fires on live bots today.

Soft CB Cooldown - This is the number of candles that must close before the Soft Circuit Breaker opens again.

From 1 to 100 candles, at 3 to start. It is counted in closed candles, so its real length follows the bot's own timeframe.

`src/gui/bot_wizard.py` — the Soft CB Cooldown row

```
self._cb_cooldown = QSpinBox()
self._cb_cooldown.setRange(1, 100)
self._cb_cooldown.setValue(3)
```

The figure is the count of fresh candles the shut side waits out, and it is the count the code uses: a stored 1 re-opened on the first fresh candle, a stored 3 on the third, and a stored 10 on the tenth. The count only moves on a candle timestamp the bot has not seen, so a repeated candle left the remaining count where it was. A record carrying a cooldown of 0 is read as 3, the declared default. A record carrying a word stops the breaker check for that tick with a warning, which leaves the soft breaker unable to trip.

While a cooldown holds, the shut side is refused at the breaker gate and the bot keeps running — the cooldown pauses nothing. It ends on its own when the count reaches zero, with a SOFT CIRCUIT BREAKER RESET line naming the side that re-opens, and the Reset All Breakers button ends it early.

Max Cartridge Size - This is the maximum amount of deviation allowed for the Target Delta. At this threshold the bot is actively and aggressively looking for a trade opportunity.

A percentage from 0.0 to 200.0, at 10.0 % to start. Crossing it fires an aggressive rebalance that bypasses the detection, hysteresis and soft-breaker checks. It is the one figure on the page whose range runs past 100.

`src/gui/bot_wizard.py` — the Max Cartridge Size row

```
self._max_cartridge_pct = QDoubleSpinBox()
self._max_cartridge_pct.setRange(0.0, 200.0)
self._max_cartridge_pct.setDecimals(1)
self._max_cartridge_pct.setSuffix(" %")
self._max_cartridge_pct.setValue(10.0)
```

The threshold is Target Balance times the figure over 100, and the fire is on absolute Target Delta. Driven on a $200.00 target: at 10.0 % the threshold was $20.00, a position $20.00 past target fired and $19.99 past target did not; at 5.0 % the threshold was $10.00, $11.00 fired and $9.99 did not; at 0.0 % a position $100.00 past target fired nothing, which is the off position.

A correction to the sentence above it: the hysteresis check is not bypassed. The tick tests the opposing-trade pivot before the fire and writes MAX CARTRIDGE BLOCKED instead, naming the price the pivot still needs. What the fire does bypass is the band detection, the higher-timeframe bias and the soft breaker — and, because the tick reads this figure before it reads either breaker, the hard breaker as well.

Smart Cartridge - This allows the Max Cartridge Size to organically resize in response to current price range as defined by the current-candle Bollinger Band reading.

One checkbox labelled Calibrate to BB range, off at the start. While it is on, the cartridge size comes from the current band range instead of the fixed percentage above it.

`src/gui/bot_wizard.py` — the Smart Cartridge row

```
self._cartridge_smart_chk = QCheckBox("Calibrate to BB range")
self._cartridge_smart_chk.setChecked(False)
```

The band range is upper minus lower over the midline, as a percentage, and the box swaps it in for the fixed figure. It is held between two bounds: never below the Opposing Trade Interval, never above Smart Ceiling. Driven on a $200.00 target with the fixed figure at 10.0 %: off, the threshold stayed $20.00; on, with a band spanning 20.0 % of its midline, the threshold became $40.00. On the same tick the bot read a band spanning 80.0 %, the clamp gave 30.0 % and the threshold $60.00, and a position $30.00 past target that fired with the box off fired nothing with it on.

The band the box reads is the one the previous worked tick stored, so the first worked tick after a start has no band and keeps the fixed figure.

Smart Ceiling - To be re-evaluated.

A percentage from 1.0 to 100.0, at 30.0 % to start. It caps the cartridge threshold while Smart Cartridge is on.

It is the upper of the two bounds on the band range, and it binds only when the band is wider than it. Driven on a $200.00 target with Smart Cartridge on: a band spanning 120.0 % under a 30.0 % ceiling gave 30.0 % and a $60.00 threshold, the same band under a 15.0 % ceiling gave 15.0 % and $30.00, and a band spanning 29.0 % under the 30.0 % ceiling gave 29.0 % and $58.00 — the ceiling did not bind. A band spanning 0.5 % gave 1.0 %, the Opposing Trade Interval, because the lower bound binds there instead.

The control cannot emit a zero. A record hand-edited to carry one is read as 30.0 %, the declared default, not as no ceiling.

`src/gui/bot_wizard.py` — `TradingParamsPage.get_config`, the settings this group emits, in the order of the rows above

```
"circuit_breaker_soft_pct": self._cb_soft_pct.value(),
"circuit_breaker_hard_pct": self._cb_hard_pct.value(),
"circuit_breaker_cooldown_candles": self._cb_cooldown.value(),
"max_cartridge_size_pct": self._max_cartridge_pct.value(),
"max_cartridge_smart": self._cartridge_smart_chk.isChecked(),
"max_cartridge_smart_ceiling_pct": self._cartridge_smart_ceiling.value(),
```

A trip stops a trade at one gate on each side of the chain, and 07-indicators.md lists both chains in full.

`src/trading/gate_chain.py` — the two breaker gates in the built chains

```
CircuitBreakerGate(side="scrum"),
```

`src/trading/gate_chain.py` — and the fold chain

```
CircuitBreakerGate(side="fold"),
```

The Risk Controls and Strategy Gate Flags groups.

After Circuit Breakers, which help defend against extreme volatility, we come to Risk Controls. These are designed to cap the amount of profit or growth a given bot can earn before performing a full position exit.

Enable Position Ceiling - Enables a growth cap for a given position.

A checkbox, off at the start. Nothing in the Risk Controls group acts until it is on.

`src/gui/bot_wizard.py` — the Position Ceiling row

```
self._position_ceiling_enabled = QCheckBox("Enable Position Ceiling")
self._position_ceiling_enabled.setChecked(False)
```

**Functional.** The box gates six places in the engine. Five refuse or shrink a buy: the ceiling figure itself, the pre-buy check, the buy executor, the first entry of a brand-new bot, and the fold hold. The sixth runs on restore and pulls a stored target back down to the ceiling. The box is read at the moment of each decision and never copied aside, so unticking it releases the brake on the very next read. Driven on a two hundred dollar anchor at three times: with the box off, a fold rebuy of fifty dollars onto a seven hundred dollar position passed; with it on, the same buy was refused; untick, and it passed again.

`src/trading/scrumming_bot.py` — `ScrummingBot.position_ceiling_usd`

```
if not getattr(self.config, "position_ceiling_enabled", False):
    return None
```

**Where the code departs.** Two of the three Detonation rows below do not wait for this box. The detonation check reads its own box and the anchor, and a search of the engine for a detonation site that reads the ceiling box returns nothing. Driven: with the ceiling box off and Detonation on, the hourly check ran and asked the venue for candles. The sentence above holds for the ceiling and the fold taper, and Detonation is the exception.

`src/trading/scrumming_bot.py` — `ScrummingBot._check_detonation_trigger`, its first two conditions

```
if not getattr(self.config, "detonation_enabled", False):
    return False
...
if current_value <= self._anchor_target_balance:
    return False
```

Ceiling Multiple - This setting caps the maximum amount of growth a position at a multiple of the Target Balance (anchor) and, once reached (and under higher timeframe bullish conditions with Detonation enabled) will allow the entire position to be sold and the corresponding bot will pause all further operations. Without Detonation enabled, this becomes a user notification.

From 1.0 to 10.0, at 5.0x anchor to start. The anchor is the target balance the bot was created with, not the balance it has grown to. It steps half a multiple at a time and carries its unit in the box.

`src/gui/bot_wizard.py` — the Ceiling Multiple row

```
self._position_ceiling_multiple = QDoubleSpinBox()
self._position_ceiling_multiple.setRange(1.0, 10.0)
self._position_ceiling_multiple.setDecimals(1)
self._position_ceiling_multiple.setSingleStep(0.5)
self._position_ceiling_multiple.setSuffix("x anchor")
self._position_ceiling_multiple.setValue(5.0)
```

**Functional.** The figure becomes a dollar ceiling, the anchor times the multiple. On a two hundred dollar anchor the four corners of the range read: one gives two hundred dollars, three gives six hundred, five gives one thousand, ten gives two thousand. The refusal is strictly above, so a projected position of exactly six hundred

dollars is allowed against a six hundred dollar ceiling and six hundred dollars and one cent is refused. The same seven hundred and fifty dollar projection is refused at three and allowed at six, which is the figure changing the answer.

`src/trading/scrumming_bot.py` — `ScrummingBot._pre_buy_allowed`, layer two

```
_smart_mult = max(1.0, min(10.0, _smart_mult))
_smart_ceiling_usd = _anchor * _smart_mult
if _projected > _smart_ceiling_usd:
```

The clamp and the multiplication above now sit in one function that the pre-buy check, the ceiling property and the tick's fold evaluation all call, so the three ceilings the bot reads are one number.

`src/trading/scrumming/sizing.py` — `position_ceiling`

```
def position_ceiling(anchor_target_balance: float, multiple: float) -> float:
    mult = max(CEILING_MULTIPLE_MIN, min(CEILING_MULTIPLE_MAX, multiple))
    return anchor_target_balance * mult
```

**Functional.** While the ceiling binds, the bot still sells and stops buying. The fold side shrinks first and then stops: the fold's dollar size is multiplied by a taper that is full below half the ceiling, falls in a straight line to a tenth across the upper half, and reaches zero at the ceiling. Measured against a six hundred dollar ceiling: two hundred and ninety dollars held full size, five hundred and ninety dollars cut the fold to thirteen per cent, and six hundred dollars stopped it. Three things release the brake — price falling back under the ceiling, a detonation resetting the target to the anchor, and unticking the box.

`src/trading/scrumming/tick_phases.py` — `_tick_execute_fold` spends the taper

```
_taper = self.fold_rate_taper
...
buy_cost = _fusd * _taper
```

The multiplication is now `fold_spend_usd`, and the taper schedule is `fold_rate_taper`, both in the shared sizing module; the executor's line reads `buy_cost = fold_spend_usd(_fusd, _taper)` and the Simulator's fold reads the same two functions over its own tranches.

`src/trading/scrumming/sizing.py` — the taper schedule

```
def fold_rate_taper(ratio: float) -> float:
    if ratio >= 1.0:
        return 0.0
    if ratio < TAPER_START_RATIO:
        return 1.0
    return 1.0 - (ratio - TAPER_START_RATIO) / TAPER_START_RATIO * TAPER_DROP
```

**Where the code departs.** The sentence above says the bot pauses all further operations once the ceiling is reached and the position is sold. The executor does not pause it. It resets the target balance to the anchor, clears the fold queue and the tranches, reseeds the lots at the fill price, and logs that the bot will re-accumulate from scratch on the next dip. A search of the detonation executor for a pause call returns nothing. The second

half of the sentence does hold: with Detonation off, the ceiling only brakes, and the operator is told through the Console line and through the Fire button's own tooltip on the Bot Swarm row.

`src/trading/scrumming/execution.py` — `_execute_detonation`, what it resets

```
prior_target = self._target_balance
self._target_balance = self._anchor_target_balance
...
self._fold_tranches.clear()
self._fold_queue_usd = 0.0
```

**Functional.** One name serves two unrelated mechanisms, and the engine's own wording has been corrected in this unit. The percentage box in Circuit Breakers labelled Smart Ceiling caps the cartridge threshold, and the engine reports it inside its cartridge line as a percentage. This dollar ceiling is what every ceiling refusal in the engine names, and those lines now read Position Ceiling and Ceiling Multiple, which are the labels on this screen. A reader can now tell the two apart: one is a percentage under a cartridge heading, the other is a dollar figure under the name of the control that set it.

`src/trading/scrumming/tick_phases.py` — the cartridge line, for contrast

```
f"SMART CARTRIDGE calibrated to "
f"{_smart_pct:.2f}% "
f"(BB range {_bb_range_pct:.2f}%, "
f"floor={_interval_floor:.2f}%, "
f"ceiling={_smart_ceiling:.2f}%). "
```

Enable Detonation - Enables an entire remaining position to be sold after the Ceiling Multiple growth threshold is crossed.

A checkbox, off at the start. Detonation needs the box ticked, a position worth more than its anchor, and a bullish reading at or above the confidence you set. It also needs the reading to have just turned bullish, so a tape that was already bullish last time fires nothing.

`src/trading/scrumming_bot.py` — `ScrummingBot._check_detonation_trigger`

```
conf_min = float(getattr(self.config, "detonation_confidence_min", 0.75))
is_bullish = (
    summary.consensus_direction == SignalDirection.BULLISH
    and summary.consensus_confidence >= conf_min
)

fired = is_bullish and not self._detonation_last_signal_bullish
```

**Where the code departs.** The sentence above says the sale happens after the Ceiling Multiple threshold is crossed. The engine's bar is the anchor, not the ceiling. Driven on a two hundred dollar anchor: a position worth two hundred dollars and one thousandth of a cent passed the bar and the hourly check fetched candles; a position worth exactly two hundred dollars was refused before any fetch. A bot with Detonation on and the ceiling left off can therefore harvest at any value above its starting target, well below any multiple.

`src/trading/scrumming_bot.py` — the bar `_check_detonation_trigger` actually uses

```
current_value = (
    self._current_holdings * price * float(self._quote_to_usd or 1.0)
)
if current_value <= self._anchor_target_balance:
    return False
```

The multiplication is now `priced_usd`, the one position-value function every site in the bot calls: the tick, the ceiling ratio, the detonation bar above, the manual rebalance, the reconciler and the Simulator's position.

`src/trading/scrumming_bot.py` — the bar as it reads now

```
current_value = priced_usd(
    self._current_holdings, price, float(self._quote_to_usd or 1.0)
)
if current_value <= self._anchor_target_balance:
    return False
```

**Functional.** The box is what starts the whole path. Driven with the box off, the detonation phase returned at once and asked the venue for nothing; with it on, the check ran and asked for one candle series. Behind the box sits an hourly rate limit, measured: a gap of three thousand five hundred and ninety-nine seconds since the last check fetched nothing and a gap of three thousand six hundred fetched once.

`src/trading/scrumming/tick_phases.py` — `_tick_detonation`

```
if getattr(self.config, "detonation_enabled", False):
    try:
        fired = await self._check_detonation_trigger(ticker)
```

Detonation TF - Selects the timeframe for the chart that is being evaluated for bullish conditions that will allow the detonation to occur.

Two entries, 1d and 1w. The label and the stored value are the same string on both rows, which is not true of every picker in the wizard.

`src/gui/bot_wizard.py` — the Detonation TF row

```
self._detonation_timeframe = QComboBox()
self._detonation_timeframe.addItem("1d", "1d")
self._detonation_timeframe.addItem("1w", "1w")
```

**Functional.** The pick does two jobs. It names the candle series the vote reads, and it sets how long the one-shot bullish latch survives between checks. One day keeps the latch eighty-six thousand four hundred seconds and one week keeps it six hundred and four thousand eight hundred. Driven on one bullish tape with the latch already set and exactly eighty-six thousand four hundred seconds since the last check: the daily pick retired the latch and fired, the weekly pick kept the latch and did not. One second earlier, neither fired. That is the pick changing the answer on identical candles.

`src/trading/scrumming_bot.py` — the latch life

```
latch_ttl = max(
    float(TIMEFRAME_SECONDS.get(tf, TIMEFRAME_SECONDS["1d"])),
    _DETONATION_LATCH_MIN_TTL_S,
)
if elapsed >= latch_ttl:
    self._detonation_last_signal_bullish = False
```

Min Confidence - This is the minimum technical analysis confidence index (via the Indicator Voting Panel) that will allow the detonation to occur.

From 0.50 to 1.00, at 0.75 to start.

**Functional.** The reader works, and it is exact. On one tape whose consensus read zero point two three three four, a bar of zero point two three three three fired and a bar of zero point two three three five did not, on the same candles with the same state. The comparison is at or above, so a bar set to the reading itself fires.

`src/trading/scrumming_bot.py` — the comparison

```
conf_min = float(getattr(self.config, "detonation_confidence_min", 0.75))
is_bullish = (
    summary.consensus_direction == SignalDirection.BULLISH
    and summary.consensus_confidence >= conf_min
)
```

**Where the code departs.** No figure this box can emit was reached. The box starts at zero point five, and across one hundred and seventy-one tapes — nine shapes, three lengths, three noise levels — the highest bullish consensus the voting panel produced was zero point two five three four. The engine's own floors on the same quantity sit lower still: a quarter for an ordinary trade, and nineteen hundredths on the band-priority arm. A bar of zero point five refused every tape driven, including the strongest. Nothing in this reading is impossible in principle: with every voter agreed at its own best reading the panel would reach eighty-four hundredths, and the twelve voters simply cancel each other long before that. The gap is between the range the box offers and the range the panel produces.

`src/trading/scrumming_bot.py` — the quantity, and the two floors the rest of the engine uses against it

```
_TA_CONFIDENCE_FLOOR = 0.25
_BB_PRIORITY_CONFIDENCE_FLOOR = _TA_CONFIDENCE_FLOOR / (1.0 + _BB_PRIORITY_SKEW)
```

**Design intention.** Two repairs are possible and both change what a live bot does on its next tick, so neither is shipped here. The box's floor could move down to meet the panel, or the detonation could compare against the same floor the rest of the engine uses. Either one arms a control that is quiet today, on bots holding real money, which is the operator's decision and not a wiring job.

PROPOSED — `src/gui/main_tabs/live_settings_tab_surface.py`, the row's range

```
"range": (0.10, 1.00),
```

**Functional.** Two things the bar does not reach. It is absent from the record the Bot Swarm row reads, so that tooltip can report the timeframe a bot is watching and never the bar it must clear. And the detonation check

builds a fresh voting panel with no weights, so a bot carrying the operator's own indicator weights is judged for detonation by the default weights instead.

`src/trading/scrumming_bot.py` — the panel the detonation check builds

```
engine = VotingEngine()
parsed = candles_from_raw(candles)
summary = engine.compute_all(parsed, tf)
```

`src/gui/bot_wizard.py` — `TradingParamsPage.get_config`, the settings this group emits, in the order of the rows above

```
"position_ceiling_enabled": self._position_ceiling_enabled.isChecked(),
"position_ceiling_multiple": self._position_ceiling_multiple.value(),
"detonation_enabled": self._detonation_enabled.isChecked(),
"detonation_timeframe": self._detonation_timeframe.currentData(),
"detonation_confidence_min": self._detonation_confidence_min.value(),
```

Next we use the Strategy Gate Flags which allows top-level trade restrictions to be enabled or disabled thus relaxing or restricting the conditions under which a trade action can occur. The settings, in this case, are self-descriptive.

**Functional.** The box draws five checkboxes, all ticked at the start: SCRUM requires bullish TA, SCRUM holds in sustained uptrend, SCRUM defers to higher-TF bullish, FOLD requires bearish TA, and FOLD defers to higher-TF bearish.

`src/gui/bot_wizard.py` — `TradingParamsPage.get_config`, the six flags this group emits

```
"scrum_require_ta_bullish": self._gate_scrum_ta_chk.isChecked(),
"scrum_hold_in_uptrend": self._gate_scrum_uptrend_chk.isChecked(),
"scrum_defer_to_htf": self._gate_scrum_htf_chk.isChecked(),
"fold_require_ta_bearish": self._gate_fold_ta_chk.isChecked(),
"fold_hold_in_downtrend": True,  # reserved, no gate
"fold_defer_to_htf": self._gate_fold_htf_chk.isChecked(),
```

**Design intention.** Six flags are declared, the box shows five, and the missing one is the fold-side twin of a scrum gate that works. It goes out as a hard True, and nothing in the engine reads it. Its scrum mirror holds a sell during a sustained uptrend, and the fold twin would hold a buy during a sustained downtrend.

*Proposed, not present, in* `src/trading/gate_chain.py`*:*

```
class FoldTrendHoldGate(Gate):
    """FOLD-only: hold the buy while a sustained downtrend runs."""

    name = "trend_hold_fold"
    side = "fold"

    def evaluate(self, ctx: GateContext) -> GateResult:
        if not ctx.eff_trend_hold:
            return GateResult(passed=True)
        return GateResult(
            passed=False,
            blocker_message=f"trend_hold_fold({ctx.trend_strength:.0%})",
        )
```

The gate would join the fold chain beside its scrum twin, and `fold_hold_in_downtrend` in `src/trading/container/config.py` would arm it. A checkbox has to arrive with it, or the flag stays a hard True. Issue #419 carries this.

**Functional.** The sixth flag is gone. It is removed from the bot config, from the restore round-trip and from both wizard surfaces, so the wizard emits five gate flags and the box still draws five checkboxes. A bot record written before the removal still loads. Bot creation keeps only the fields the config declares, so the old key is dropped on the way in and every other flag arrives unchanged.

Nothing the engine does changes. The field had no reader that moved a decision, so no running bot fires differently on its next tick. Driven on both sides over five candle tapes, every gate verdict read the same before the removal and after it.

Two measured facts bear on the proposal above. The engine holds one trend figure, a count of the last twenty candles that close above their open, and it holds no downtrend figure for a gate to read. The proposed gate reads `ctx.eff_trend_hold`, which is that bullish count after the scrum checkbox has been applied to it. As written it would hold a buy during an uptrend, and the scrum checkbox would switch it. A fold hold needs its own measurement and its own control before it needs a flag.

The Profit Routing group.

Moving onto the final section, we have Profit Routing which was intended to allow profits to be routed differently during initial set-up. This will be re-evaluated and potentially removed.

The group writes two settings and stores them with the bot. No module under `src/trading/` reads either one. The wizard records the route, Bot Settings can change it, and a restart restores it. None of that reaches a trade: `_route_scrum_proceeds_via_wires` moves the scrum proceeds, and it never asks what the route says.

The bot config declares both fields with a default, so the destination a bot holds is always the first entry in the list. Issue #336 carries this, with the rest of the settings the wizard writes and bot creation drops.

`src/trading/container/restore.py` — the round-trip that puts both fields back on a restarted bot

```
"profit_route": cfg.get("profit_route", "fold_to_target"),
"profit_route_bot_id": cfg.get("profit_route_bot_id", ""),
```

Route - Destination for profits.

Four destinations: fold back to target balance, send to spendable, split fold and spendable by percentage, and route to another bot. Only the last of the four makes the Target bot ID field below it mean anything.

`src/gui/bot_wizard.py` — the Route row

```
self._profit_route = QComboBox()
self._profit_route.addItem("Fold back to target balance", "fold_to_target")
self._profit_route.addItem("Send to spendable", "spendable")
self._profit_route.addItem("Split fold/spendable per %", "split")
self._profit_route.addItem("Route to another bot (cross-bot)", "cross_bot")
```

Target bot ID - Field for manually a bot ID which was intended to create a Smart Wire under the Bot Swarm tab.

A free-text field. Its placeholder tells you to leave it blank unless you chose the cross-bot route.

`src/gui/bot_wizard.py` — `TradingParamsPage.get_config`, the settings this group emits, in the order of the rows above

```
"profit_route": self._profit_route.currentData(),
"profit_route_bot_id": self._profit_route_bot_id.text().strip(),
```

The group and both of its rows are now removed, from the wizard and from Bot Settings. The operator's ruling on a setting that reaches no trade: "I do not want dangling settings fixed that do not or have not affected what is now almost 6000 trades or data points. The trading mechanisms are valid and sound." Profit reaches accumulation through harvest-fold and the fold tranches, and a wire drawn on the Bot Swarm tab carries any cross-bot share, so neither row had a destination left to name.

`src/trading/scrumming/wire_routing.py` — the cross-bot destination the engine does read, taken from the drawn wires and never from a stored bot id

```
_wires = _wire_mgr.get_outgoing_wires(self.bot_id)
```

A bot saved before the removal still loads. A restart names each field it rebuilds one at a time, so a stored route is simply not read, and every other field arrives at the figure it was saved with.

`src/trading/container/config.py` — the field a bot is sized by, which is what Target Balance writes

```
target_balance: float = 200.0  # Balance the bot trades relative to
```

Two further settings are removed alongside them, and neither ever had a row on any screen. Investment amount was saved and restored and read by nothing. Spacing style was saved and restored and read by nothing, and the placement the engine derives from Opposing Trade Distance and the band extension is the

method that replaced it.

`src/trading/container/restore.py` — the scrumming fields a restart rebuilds, now carrying no line for either name

```
"increment_style": cfg.get("increment_style", "linear"),
"profit_folding_active": cfg.get("profit_folding_active", True),
"scrumming_interval_pct": cfg.get("scrumming_interval_pct", 1.0),
"scrum_fold_pct": cfg.get("scrum_fold_pct", 100),
```

The Phantom Bots page.

Lastly we have the selection for the Phantom (Balance) Bots. These are intended to provide trade action overrides from higher timeframe charts and indicator sets which, in turn, may result in an improved trade or prevent a premature one.

**Functional.** This is the last page on the scrumming path:

- Enable Phantom Bots, a checkbox, off at the start.
- Active Timeframes, eleven checkboxes: 1m, 5m, 15m, 30m, 1h, 2h, 4h, 6h, 12h, 1d and 1w. All start clear. A timeframe the chosen venue does not carry is greyed out and cleared, with the reason written into its tooltip.
- Candles to lock, 1 to 10, at 2, inside the Higher-TF Lock Duration group.

The page hands on only the boxes that are both ticked and available. Before it lets you leave, it asks the API load monitor whether that many phantoms would breach the safety threshold for the venue, and offers you Back to adjust or Continue anyway.

`src/gui/bot_wizard.py` — `PhantomConfigPage.get_config`

```
def get_config(self):
    checked = [
        tf
        for tf, cb in self._tf_checks.items()
        if cb.isChecked() and cb.isEnabled()
    ]
    return {
        "enable_phantoms": self._enable.isChecked(),
        "phantom_timeframes": checked,
        "lock_candle_count": self._lock_candles.value(),
    }
```

**Design intention.** The engine behind this page is unfinished. The Indicator Voting Panel section of this manual records that phantom bots are still in active development and that related features do not yet work, so nothing is proposed for the page.

In development.

**The page keeps one timeframe.** A bot runs one phantom. The page now enforces that rule. Ticking a box clears every other box. The heading states the rule: "Active Timeframe — pick one, above this bot's TA Timeframe:".

`src/gui/main_tabs/bot_wizard_surface.py` — `BotWizardModel.set_phantom_timeframe`

```
for found in PHANTOM_TIMEFRAMES:
    if found != timeframe and self.phantom_checks[found]:
        self.phantom_checks[found] = False
        self.calls.append([CHECK_SET_CHECKED, found, False])
self.phantom_checks[timeframe] = True
```

Driven over the bridge, a request ticking 1d and then 4h on a 1h bot hands back one name. It handed back two names before.

**The page refuses a timeframe that cannot run.** A phantom must outrank the bot's own TA Timeframe. A box at or below it stays clear and its tool tip says why. A box the venue does not serve behaves the same way. The Qt page and the Electron page share one rule.

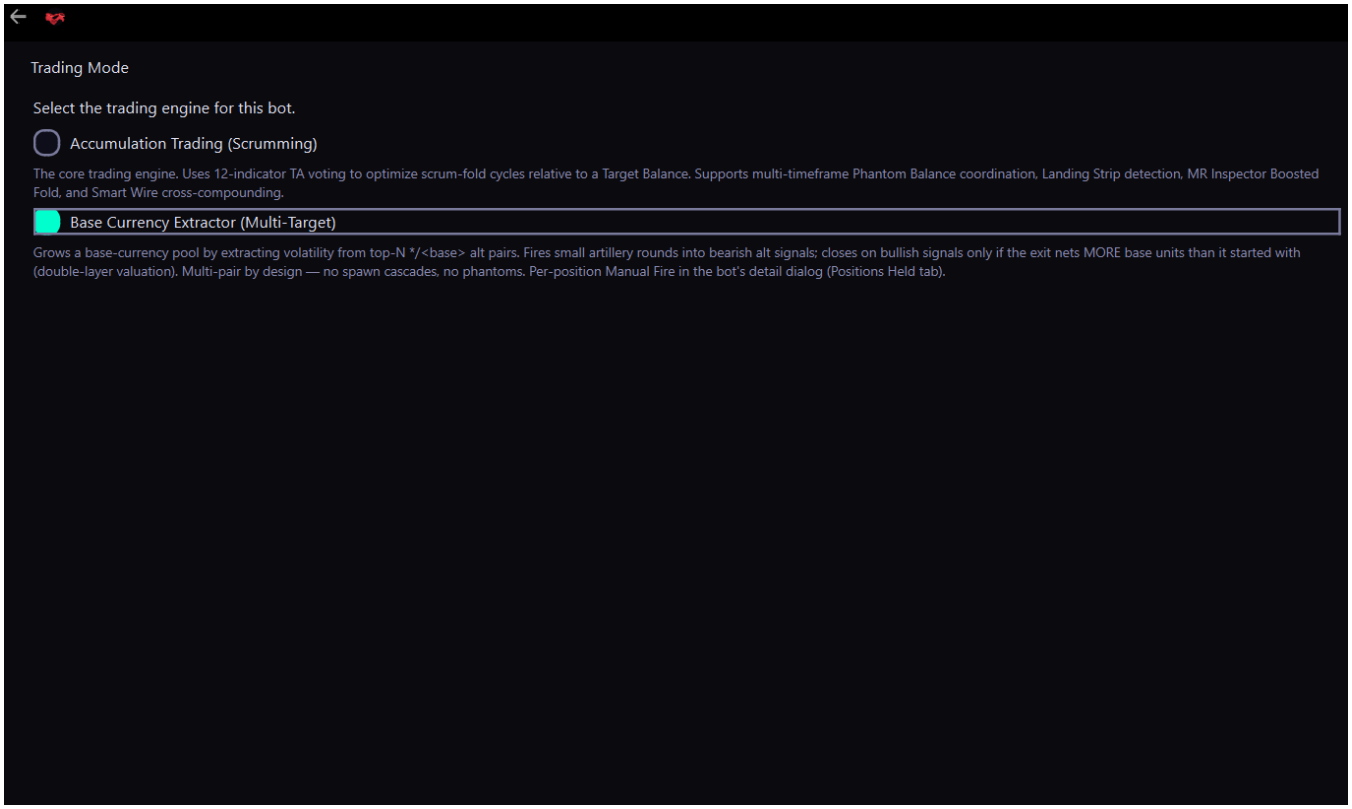
| step | module | symbol |
|---|---|---|
| state the rule | `src/gui/main_tabs/bot_wizard_surface.py` | `BotWizardModel.phantom_refusal` |
| apply it on a tick | `src/gui/main_tabs/bot_wizard_surface.py` | `BotWizardModel.set_phantom_timeframe` |
| apply it on the Qt page | `src/gui/bot_wizard.py` | `PhantomConfigPage._on_timeframe_toggled` |
| hand on the choice | `src/gui/bot_wizard.py` | `PhantomConfigPage.get_config` |

Driven over the bridge on a 1h bot, ticking 5m hands back no name and records the refusal `phantom_not_higher`. It handed back 5m before, with no refusal.

**The eleven names come from one place.** `src/exchange/timeframes.py` declares `ALL_TIMEFRAMES`. The wizard page reads that tuple. It held its own copy of the same eleven names before.

### Extractor Bot (Partially Built; Untested)

The second type of bot offered within Acervator is the Extractor. These operate quite differently from Scrumming Bots and actually operate as Siblings of them. In fact, an Extractor Bot cannot even be called unless a corresponding Base Currency Scrumming Bot (i.e. A15:USD or A14:USD) is already active. This is due to the core operating principle of the Extractor bot to acquire more of these base currencies by performing trades against available alternate currency pairings. It does this by using and blocking off a portion of the Parent's position within an Extractor Tranche that represents an active position taken against one of the available alternate pairs. The Extractor Tranche remains open until its opposing accumulating (or Short Position if preferred by the user) or profit taking trade is filled. Extractor Tranches can be of any size but should generally be a relatively small fraction of the Parent's total position which will allow the Extractor to take multiple positions if available and allowed by the specific user.




The Trading Mode page with the Extractor selected.

**Functional.** The same first page as before, with the other radio chosen. From here the wizard goes to the Extractor Pool page instead of the asset page. Phantom bots and profit folding are both switched off for this bot as it is built, and an Extractor never sees the Phantom page at all.

`src/gui/bot_wizard.py` — `BotCreationWizard.get_bot_config`

```
config.update(self._params_page.get_config())
if config["mode"] == "extractor":
    config["enable_phantoms"] = False
    config["profit_folding_active"] = False
else:
    config.update(self._phantom_page.get_config())
```

**Design intention.** Phantom overrides and profit folding belong to the parent, and the two hard False values above are that decision written down. The mode is stamped onto the config at the very top of the same method, so nothing downstream has to guess which kind of bot it received.

`src/gui/bot_wizard.py` — the mode stamp

```
if self._mode_page.is_extractor():
    config["mode"] = "extractor"
    config.update(self._extractor_pool_page.get_config())
else:
    config["mode"] = "scrumming"
    config.update(self._asset_page.get_config())
```

The Extractor Pool page.

**Functional.** This page draws:

- Exchange, the connected venues. Changing it re-scans that venue.
- Pool Base Currency, a fixed list of five. It names the asset the pool accumulates.
- A line counting the pairs available against that base.
- Target alt pairs, a check list of every alt that trades against the base. Leave every box clear and the bot picks its own pairs by volume at run time.
- Select all and Clear act on the whole list.

**Target alt pairs is removed.** The ruling of 13 September 2026 reads *"Not needed. List of compatible alt pairs is scanned and piped through the Trading IVP along with the chart being read. Same as Scrumming except multiple markets are being checked by one bot for entries and exit opportunities."* The tick boxes, the two buttons and the stored list are gone. The list stays on the page as a reading of which pairs trade against the

pool base, and the bot picks the top-N of them by volume.

```
ranked.sort(key=lambda x: x[1], reverse=True)
top_n = int(self.config.extractor_scan_top_n)
top_n = max(5, min(10, top_n))  # clamp to 5-10
new_watch = [sym for sym, _ in ranked[:top_n]]
```

The page hands on a single asterisk where a Scrumming Bot would hand on one target asset. Its own docstring calls that the pool sigil. An Extractor holds a pool of alts rather than one target, and the sigil keeps the field's shape for the code downstream that builds a trading symbol.

`src/gui/bot_wizard.py` — `ExtractorPoolPage._POOL_BASES`

```
_POOL_BASES = ["A15", "A14", "USDT", "USDC", "BNB"]
```

`src/gui/bot_wizard.py` — `ExtractorPoolPage.get_config`

```
return {
    "exchange_id": self._exchange.currentData(),
    "base_currency": base,
    "target_asset": "*",  # pool sigil — multi-pair indicator
    "extractor_alt_targets": checked,
}
```

**Design intention.** The five pool bases are exactly the assets named as base currencies above. Leaving the list empty hands the choice back to the bot, and only a box that is both ticked and available reaches the config.

`src/gui/bot_wizard.py` — how a ticked alt is collected

```
for i in range(self._alt_list.count()):
    item = self._alt_list.item(i)
    if item.checkState() == Qt.Checked:
        sym = item.data(Qt.UserRole)
        if sym:
            checked.append(sym)
```

The Extractor group of the parameter page, first nine rows.

**Functional.** Choosing the Extractor hides the eight scrumming groups and shows this one. The two sides never appear together.

`src/gui/bot_wizard.py` — `TradingParamsPage.set_mode`, the group swap

```
scrum_visible = not is_grid and not is_extractor
for g in (
    self._mode_group,
    self._scrum_group,
    self._adv_group,
    self._hedge_group,
    self._cb_group,
    self._risk_group,
    self._gates_group,
    self._routing_group,
):
    g.setVisible(scrum_visible)
self._extractor_group.setVisible(is_extractor)
```

The group title in the source reads Extractor, an em dash, Pool, an ampersand, then Artillery. Qt reads that ampersand as a keyboard-mnemonic marker, so the rendered title drops it and underlines the A of Artillery. The figure shows the gap the dropped character leaves. Issue #421 carries this.

Chunk size (USD) - This determines the maximum amount of the parent's pool that the Extractor can use.

From $10.00 to $10,000,000.00, at $100.00 to start. It is the pool every artillery round below is drawn from.

`src/gui/bot_wizard.py` — the Chunk size row

```
self._ext_chunk_size_usd = QDoubleSpinBox()
self._ext_chunk_size_usd.setRange(10.0, 10_000_000.0)
self._ext_chunk_size_usd.setPrefix("$")
self._ext_chunk_size_usd.setDecimals(2)
self._ext_chunk_size_usd.setValue(100.0)
```

The pool is now turned into base-currency units on the bot's first watch-list refresh. The Extractor reads its own base currency's dollar price off its exchange and hands it to the method that rebases the pool, so the pool stops being counted one dollar to one coin. A dollar-pegged base takes a rate of one and reads no price at all.

Driven on a $250.00 pool against a base priced at $4,000.00, the bot read a pool of 0.0625 base units where it used to read 250.

`src/trading/extractor_bot.py` — `ExtractorBot._acquire_usd_per_base_rate`

```
if base in DOLLAR_PEGGED_CURRENCIES:
    rate = 1.0
else:
    try:
        ticker = await self.exchange.get_ticker(f"{base}/USD")
        rate = float(ticker.last)
```

Artillery size (USD) - This determines the individual size of Extractor Tranches.

From $0.50 to $100,000.00, at $5.00 to start. The default is set small enough to fire often and still clear a venue's minimum order cost.

`src/gui/bot_wizard.py` — the Artillery size row

```
self._ext_artillery_size_usd = QDoubleSpinBox()
self._ext_artillery_size_usd.setRange(0.5, 100_000.0)
self._ext_artillery_size_usd.setPrefix("$")
self._ext_artillery_size_usd.setDecimals(2)
self._ext_artillery_size_usd.setValue(5.0)
```

One round converts at that same rate, every time it fires. Driven at $7.50 against a base priced at $4,000.00, a round sized 0.001875 base units where it used to size 7.5.

`src/trading/extractor_bot.py` — `ExtractorBot._fire_artillery`, the sizing

```
artillery_usd = float(self.config.extractor_artillery_size_usd)
artillery_base = self._usd_to_base(artillery_usd)
```

Watch list top-N - The determines the number of Alternate Currency pairs the bot will scan for potential extraction.

From 5 to 10, at 8 to start. The pairs are ranked by twenty-four hour volume, so the list holds the most liquid alternates against the chosen base.

`src/gui/bot_wizard.py` — the watch list size row

```
self._ext_scan_top_n = QSpinBox()
self._ext_scan_top_n.setRange(5, 10)
self._ext_scan_top_n.setValue(8)
```

Watch list refresh - This determines the rate at which the Extractor will scan its watched markets. This is the equivalent of a Timeframe for the Extractor but covers multiple pairs.

From 10 to 240 candles, at 60 to start, which is once an hour at a one-minute cadence. It re-ranks the list rather than re-reading one pair.

`src/gui/bot_wizard.py` — the watch list refresh row

```
self._ext_scan_refresh = QSpinBox()
self._ext_scan_refresh.setRange(10, 240)
self._ext_scan_refresh.setValue(60)
self._ext_scan_refresh.setSuffix(" candles")
```

The count is now candles of the bot's own timeframe. It used to be ticks, and the loop ticks every five seconds whatever the timeframe says, so 60 came due after five minutes. Driven on a one-minute bot, 60 now comes due after 3,600 seconds and not after 300.

The running-bot window used to offer 10 to 600 and call them ticks. It offers the same 10 to 240 candles the wizard does, so both screens and the sentence above agree.

One method turns the setting into seconds, and both the refresh check and the base-currency price read ask it, so the two share one cadence and one reading of the number.

`src/trading/extractor_bot.py` — `ExtractorBot._refresh_interval_seconds`

```
candles = int(self.config.extractor_scan_refresh_candles)
candle_seconds = TIMEFRAME_SECONDS.get(
    self._timeframe, TIMEFRAME_SECONDS[self.DEFAULT_TIMEFRAME]
)
return float(candles * candle_seconds)
```

Pool Reserve - To be re-evaluated.

A percentage from 0.0 to 90.0, at 50.0 % to start. Bot creation passes it through as `extractor_pool_reserve_pct`, and one method reads it: the capacity check the artillery path runs before it fires a round. The reserve is that share of the pool, and the check refuses any round that would take the free pool below it.

The share counts against the pool the operator allocated, not against what is left of it. Corrections that have already drained the pool therefore cannot hide inside the reserve arithmetic.

`src/trading/extractor_bot.py` — `ExtractorBot._has_chunk_capacity`

```
reserve = self._chunk_size_base * (
    self.config.extractor_pool_reserve_pct / 100.0
)
return (self._chunk_free_base - artillery_base) >= reserve
```

**Pool Reserve is removed.** The ruling of 13 September 2026 reads *"Not needed. Conceptually its another name for Chunk Size."* The row is off the creation wizard and off the running-bot window, and the field is off the engine's declaration. The capacity check now asks only whether the free pool covers one round.

```
def _has_chunk_capacity(self, artillery_base: float) -> bool:
    return self._chunk_free_base >= artillery_base
```

Exit % - To be re-evaluated.

A percentage from 10.0 to 100.0, at 100.0 % to start. Bot creation passes it through as `extractor_exit_pct`. It sets the share of a position's alt units an exit sells, and the Extractor reads it nine times across three methods: the profitability test, the bullish exit, and the per-position Manual Fire.

The profitability test is the one that can refuse. It prices the proportional sell in base units, takes the trading fee off, and compares the result against the same share of the cost basis. An exit that would gain dollars but lose base units does not happen.

`src/trading/extractor_bot.py` — `ExtractorBot._exit_is_profitable_in_base`

```
units_to_sell = pos.alt_units * (self.config.extractor_exit_pct / 100.0)
base_back = units_to_sell * alt_price_in_base
fee_pct = float(getattr(self.config, "trading_fee_pct", 0.6))
base_back_after_fee = base_back * (1.0 - fee_pct / 100.0)
base_in_proportional = pos.cost_basis_base * (
    self.config.extractor_exit_pct / 100.0
)
return base_back_after_fee > base_in_proportional
```

Max compounding tier - Allows the Extractor to attempt a number of compounding Swing Trades with a given Extractor Tranche with subsequent re-entries based upon the Parent Scrumming Bot’s Minimum Opposing Trade Distance + Trade Fee + Bollinger Band extension settings.

From 1 to 10, at 3 to start. The first tier always locks its gain back to the pool, and the counter is held by the position rather than by the bot, so it ends when the position does.

`src/gui/bot_wizard.py` — the compounding tier row

```
self._ext_max_tier = QSpinBox()
self._ext_max_tier.setRange(1, 10)
self._ext_max_tier.setValue(3)
```

A rolled tranche now prices its re-entry from the parent, which is what the sentence above asks for. The Extractor records the exit fill that rolled the tier, finds the parent Scrumming Bot holding the base currency, takes that parent's Minimum Opposing Trade Distance and Trading Fee as one distance, and hands that distance to the same placement the engine uses elsewhere, together with the pair's Bollinger band. A band

edge further out than the distance replaces it, and that is the band extension the sentence names. A correction on a rolled tranche waits until the price reaches the floor.

Driven on a parent at a 1.50 % interval and a 0.60 % fee, the distance read 2.10 %, and a tranche rolled at a fill of 0.10 read a floor of 0.094 — the band's lower edge, further out than the 0.0979 the distance alone gives. At a price of 0.10 the re-entry was refused, and at 0.09 it went through. Tier one reads no floor, and neither does a bot with no parent.

`src/trading/extractor_bot.py` — `ExtractorBot._reentry_floor_price`

```
otd_pct = minimum_opposing_trade_distance_pct_from_config(parent.config)
return placement_floor_price(
    -1, float(pos.roll_exit_price_base_per_alt), otd_pct, band
)
```

**Max compounding tier is removed.** The ruling of 13 September 2026 reads *"Not needed."* The row is off both screens, the tier counter is off the position record, and the re-entry pricing that read it is gone with it. The two pricing helpers it borrowed are shared with the Scrumming side and stay where they are, in `otd_math.py` and `stack_math.py`. A realised gain now always locks to the pool.

```
# The gain locks to chunk_free_base; no roll re-enters the pair.
self._chunk_free_base += base_received
log_kind = "LOCK_TO_POOL"
```

Max cost-basis multiple - To be re-evaluated.

From 1.0x to 10.0x, at 2.0x to start. Setting it to 1.0 stops averaging down. Bot creation passes it through as `extractor_max_cost_basis_multiple`, and the correction path reads it twice. The first read is the ceiling: the bot may average a position down until its cost basis reaches this multiple of its opening round, then it holds and waits for the exit.

The second read writes a log line, and it prints this multiple where a correction count belongs. The line reads corrections=1/2x cap, and nothing anywhere compares the correction counter against a cap. Issue #438 carries this.

`src/trading/extractor_bot.py` — `ExtractorBot._maybe_fire_correction`, the ceiling

```
max_basis = pos.artillery_size_base * float(
    self.config.extractor_max_cost_basis_multiple
)
headroom = max_basis - pos.cost_basis_base
if headroom <= 0:
    return  # hard floor reached; wait for bullish exit
```

The log line now prints the cap the code actually enforces. It names the position's cost basis against the base-unit ceiling this multiple sets, and it prints the correction counter as a plain count with no denominator, because nothing caps that counter. Driven on a position with nine corrections already fired and headroom still open, the correction went through — which is what the old wording denied.

`src/trading/extractor_bot.py` — `ExtractorBot._maybe_fire_correction`, the line

```
f"cost_basis {pos.cost_basis_base:.8f} of "
f"{max_basis:.8f} base allowed at "
f"{float(self.config.extractor_max_cost_basis_multiple):.2f}x, "
f"corrections={pos.corrections_fired} (no cap)."
```

**Max cost-basis multiple is removed.** The ruling of 13 September 2026 reads *"Not needed. Trade action is not based on assumed price limits. Its based on market structure."* It was the ceiling on averaging down, and averaging down went with it, so the row, the ceiling and the log line that quoted it are all gone.

In development.

Direction - To be re-evaluated.

Two entries: Normal, which runs base to alt and buys first, and Inverted, which runs standing alt to base and sells first.

`src/gui/bot_wizard.py` — `TradingParamsPage.get_config`, the settings this group emits, in the order of the rows above

```
"extractor_chunk_size_usd": self._ext_chunk_size_usd.value(),
"extractor_artillery_size_usd": self._ext_artillery_size_usd.value(),
"extractor_scan_top_n": int(self._ext_scan_top_n.value()),
"extractor_scan_refresh_candles": int(
    self._ext_scan_refresh.value()
),
"extractor_pool_reserve_pct": self._ext_pool_reserve.value(),
"extractor_exit_pct": self._ext_exit_pct.value(),
"extractor_max_compounding_tier": int(
    self._ext_max_tier.value()
),
"extractor_max_cost_basis_multiple": self._ext_max_cost_basis.value(),
"extractor_direction": self._ext_direction.currentData(),
```

The Extractor group, remaining five rows.

**Direction is removed.** The ruling of 13 September 2026 puts it among the settings that are *"hallucinated nonsense that does not comply with the spec."* The Inverted mode went with the row: the pair filter keeps markets quoted in the base, entry fires on a bearish reading and buys, and the exit sells.

```
def _entry_order_side(self):
    """OrderSide for artillery entry: BUY."""
    from ..exchange.base import OrderSide

    return OrderSide.BUY
```

Standing alt units (inverted) - To be re-evaluated.

Eight decimal places, at 0 to start. The Inverted direction reads it; the Normal direction ignores it. The field takes up to a billion units. The wizard writes it as `inverted_extractor_standing_alt_units`.

Bot creation passes neither this number nor the Direction beside it, so a new Extractor runs Normal with a standing position of zero whatever you enter. The one method that reads the number, `set_initial_chunk_rate`, has no caller in the product source either. Only tests call it. Issue #437 carries the method, and issue #336 the settings creation drops.

Creation carries both names today. Driven on a wizard dictionary holding all fifteen Extractor names, the creation path carried fifteen of fifteen, and a restored bot read Inverted back — taking its entry side as a sell and its exit as a buy. The reader of the standing number is still uncalled, so the number itself reaches nothing.

**Three sentences above are overtaken.** None is edited and none is removed. Each is quoted here with the reading that replaces it.

> *The one method that reads the number, `set_initial_chunk_rate`, has no caller in*
>
> *the product source either. Only tests call it.*
>
> *The reader of the standing number is still uncalled, so the number*
>
> *itself reaches nothing.*

The reader has a caller. The Extractor's own rate reader calls it once, on the first tick of a bot with no position open, and the tick awaits that reader on every due watch-list refresh.

`src/trading/extractor_bot.py` — `ExtractorBot._acquire_usd_per_base_rate`, the first-rate call

```
if not self._recent_rates and self._tick_counter <= 1 and not self._positions:
    self.set_initial_chunk_rate(
        rate, total_holdings=await self._read_base_holdings(base)
    )
```

> *Creation carries both names today.*

Neither name is in the product source. Both were removed with the settings that carried them. Searched over the git index across every tracked Python file, each returns no occurrence, against sixteen for a refresh key that is still read.

```
inverted_extractor_standing_alt_units    0 occurrences
extractor_direction                      0 occurrences
extractor_scan_refresh_candles          16 occurrences
```

`src/trading/extractor_bot.py` — `ExtractorBot.set_initial_chunk_rate`, the inverted branch

```
_standing = float(
    getattr(self.config, "inverted_extractor_standing_alt_units", 0) or 0
)
if self._is_inverted and _standing > 0:
    self._chunk_size_base = _standing
    self._chunk_size_usd = _standing * usd_per_base
```

**Standing alt units is removed.** The ruling of 13 September 2026 puts it among the settings that are *"hallucinated nonsense that does not comply with the spec."* It served the Inverted direction above, which is gone, so the pool now rebases from the dollar figure on every Extractor.

```
self._chunk_size_base = self._chunk_size_usd / usd_per_base
self._chunk_free_base = self._chunk_size_base
```

Correction skip candles - To be re-evaluated.

From 0 to 100 candles, at 4 to start. It throttles averaging down: the Extractor will not correct the same position again until this many have passed. One method reads it, the correction path, and it counts ticks rather than candles. The Extractor ticks every five seconds, so 4 is twenty seconds on any timeframe.

Bot creation does not pass it, so a new bot takes the declared default of 4. Issue #438 carries the counting, and issue #336 the drop.

Creation passes it today. Driven with a typed 9, a restored bot read 9 rather than the default of 4. What it counts is unchanged: the throttle compares ticks.

`src/trading/extractor_bot.py` — `ExtractorBot._maybe_fire_correction`, the throttle

```
last_tick = self._last_correction_tick.get(pos.pair, -(10**9))
if self._tick_counter - last_tick < int(
    self.config.extractor_correction_skip_candles
):
    return  # skip-candles throttle
```

**Correction skip candles is removed.** The ruling of 13 September 2026 puts it among the settings that are *"hallucinated nonsense that does not comply with the spec."* It throttled averaging down, and averaging down went with it, so the throttle and the method that lengthened it into seconds are both gone. The watch-list refresh still counts candles through the same helper.

```
def _refresh_interval_seconds(self) -> float:
    candles = int(self.config.extractor_scan_refresh_candles)
    return float(candles) * self._candle_seconds()
```

Drawdown threshold - To be re-evaluated.

A percentage from 0.00 to 50.00, at 3.00 % to start. One method reads it, and that method decides when a position counts as down: the current dollar value against the dollar value snapshotted at firing, which never changes afterwards. Crossing the threshold puts a position in front of the correction path.

The wizard writes it as `extractor_drawdown_threshold_pct`, and bot creation does not pass it, so a new bot takes the declared default of 3.00. Issue #336 carries this.

Creation passes it today. Driven with a typed 11.00, a restored bot read 11.00 rather than the default of 3.00.

`src/trading/extractor_bot.py` — `ExtractorBot._is_in_drawdown`

```
current_usd = self._position_value_usd(pos, alt_price_in_base)
threshold = pos.artillery_size_usd_at_entry * (
    1.0 - self.config.extractor_drawdown_threshold_pct / 100.0
)
return current_usd < threshold
```

**Drawdown threshold is removed.** The ruling of 13 September 2026 puts it among the settings that are *"hallucinated nonsense that does not comply with the spec."* The operator no longer sets when a position counts as down; a position is down while its dollar value sits below the value snapshotted at firing. The pool light still turns red on that reading.

```
return (
    self._position_value_usd(pos, alt_price_in_base)
    < pos.artillery_size_usd_at_entry
)
```

Trend Strength Threshold - To be re-evaluated.

From 0.000 to 1.000, at 0.650 to start.

Creation passes it, and it reaches the bot's technical-analysis signal provider once, at construction. Driven with a typed 0.875, the provider read 0.875. The provider is built one time per bot, so a later edit to the stored figure does not reach it.

**Trend Strength Threshold is removed.** The ruling of 13 September 2026 puts it among the settings that are *"hallucinated nonsense that does not comply with the spec."* The signal provider carries the same threshold as its own published default, so the bot is built without the argument and the reading does not move.

```
self._ta_provider = TASignalProvider(
    exchange,
    timeframe=self._timeframe,
)
```

A sixth control sits in this part of the group and no entry above names it. Hedge budget (USD) starts at $0.00, which switches it off. Above zero, the bot converts it into a base-currency reserve held out of artillery rotation. This manual states that the control exists and claims nothing about what it is for.

The budget is read at construction, and the line that turns it into base-currency units sits in a method with no caller, so the reserve it names stays at zero. Driven with $40.00, the restored bot read a hedge budget of 40.00 and a free hedge reserve of 0.00.

**Hedge budget is removed.** The ruling of 13 September 2026 puts it among the settings that are *"hallucinated nonsense that does not comply with the spec."* It funded averaging down, which went with it, so the budget, the

reserve it converted into and both of their saved keys are gone. The capital claim now reserves the pool alone.

```
total_reserved_base = self._chunk_size_base
```

`src/gui/bot_wizard.py` — `TradingParamsPage.get_config`, the remaining Extractor settings

```
"inverted_extractor_standing_alt_units": self._ext_standing_alt_units.value(),
"extractor_correction_skip_candles": int(
    self._ext_correction_skip.value()
),
"extractor_drawdown_threshold_pct": self._ext_drawdown_threshold.value(),
"extractor_hedge_budget_usd": self._ext_hedge_budget.value(),
"extractor_trend_strength_threshold": self._ext_trend_strength.value(),
```

### The fifteen Extractor-only settings the engine declares

**Functional.** One declaration names every setting that belongs to an Extractor and to no Scrumming Bot. A Scrumming config carrying any of the fifteen is refused when it is built, and so is an Extractor config carrying a Scrumming-only name. Creation filters the wizard's dictionary against that one declaration, so a name added to it reaches a new bot without a second list being edited.

`src/trading/container/config.py` — `bot_config_kwargs`, how the fifteen are carried

```
carried = {f.name for f in fields(BotConfig)} - foreign - {"mode"}
kwargs = {
    key: value
    for key, value in _sanitize_deprecated_kwargs(collected).items()
    if key in carried
}
```

All fifteen reach a new bot, and all fifteen survive a restore. Driven on a stored record holding every one of them, creation carried fifteen of fifteen and a restored bot read every value back unchanged. The same record with all fifteen dropped still restores the bot, on the declared defaults, and the behaviour moves with them: the entry side flips, and the watch list switches from the typed pair list to the ranked top-N.

| setting | engine read | what the read decides |
|---|---|---|
| Chunk size (USD) | `extractor_bot.py:156` | the pool every round draws from |
| Artillery size (USD) | `extractor_bot.py:1265` | the size of one round |
| Watch list top-N | `extractor_bot.py:581` | how many ranked pairs are kept |
| Watch list refresh | `extractor_bot.py:597` | when the list is re-ranked |
| Pool Reserve | `extractor_bot.py:434` | the share a round may not take |
| Exit % | `extractor_bot.py:799` | the share an exit sells |
| Max compounding tier | `extractor_bot.py:1144` | roll to the next tier, or lock to the pool |
| Max cost-basis multiple | `extractor_bot.py:858` | the averaging-down ceiling |
| Correction skip candles | `extractor_bot.py:853` | the averaging-down throttle |
| Drawdown threshold | `extractor_bot.py:786` | when a position counts as down |

| setting | engine read | what the read decides |
|---|---|---|
| Hedge budget (USD) | `extractor_bot.py:170` | the reserve corrections draw from first |
| Trend Strength Threshold | `extractor_bot.py:193` | the threshold the signal provider is built with |
| Target alt pairs | `extractor_bot.py:478` | the typed pair list against the ranked scan |
| Direction | `extractor_bot.py:620` | whether entry buys or sells |
| Standing alt units (inverted) | `extractor_bot.py:228` | nothing; its method has no caller |

**The last row's reason is overtaken.** The row stays as it is written. Its verdict of nothing still holds; the reason beside it does not.

> | *Standing alt units (inverted)* | `extractor_bot.py:228` | *nothing; its method has no caller* |

The method has a caller. The setting reaches nothing because the name the row stands for is in no Python file, not because the reader is uncalled.

```
inverted_extractor_standing_alt_units    0 occurrences in tracked Python
set_initial_chunk_rate                   1 call site in tracked Python
```

Three of the fifteen are read once, at construction, and never again: the pool size, the hedge budget and the trend strength threshold. The other twelve are read off the config as the bot ticks, so an edit to a running bot reaches them. The running-bot screen draws eight of the fifteen, so seven can only be set before the bot exists.

**Three places the path from a control to a trade stops.** The dollar-to-base rate stays at one, the hedge reserve stays at zero, and the bot writes no capital claim of its own.

`src/trading/extractor_bot.py` — `ExtractorBot.__init__`, the rate

```
self._usd_per_base_rate: float = 1.0  # dollars per base unit; set externally
self._chunk_size_base: float = self._chunk_size_usd  # rebased when rate is set
```

All three sit behind one method. `set_initial_chunk_rate` is the only code that rebases the pool into base-currency units, converts the hedge budget, and claims the funds with the capital reservation registry, and it has no caller in the product source. Registration reads a rate of its own and writes a registry claim from it, but never hands that rate to the bot. Driven with a $250.00 pool, a $7.50 artillery size and a $40.00 hedge budget, the restored bot read a pool of 250 base units, a round of 7.5 base units, a hedge reserve of zero, and no reservation token.

All three arrive now. The Extractor reads its own base currency's dollar price on every watch-list refresh, which is the moment it already goes to the venue. The first reading rebases the pool, converts the hedge budget and writes the capital claim; every reading after that goes through the spike-protected update, so the rate a round is sized at stays current.

Driven again on the same figures against a base priced at $4,000.00, the same restored bot read a pool of 0.0625 base units, a round of 0.001875, a hedge reserve of 0.01, and a reservation token. The rebase runs once, on a bot's first tick with no position open, so it cannot be charged twice and cannot throw away base units a running pool has already earned.

`src/trading/extractor_bot.py` — `ExtractorBot.tick`, the refresh step

```
if self._watch_list_due_for_refresh():
    await self._acquire_usd_per_base_rate()
    await self._refresh_watch_list()
```

One case is still open, and no record of it exists. A bot restored from a state file saved before this change keeps the base-unit figures that file holds, because a restore carries them and the rebase only runs on a first tick. The rate itself still becomes real, so every new round is sized correctly; the pool total is the part that would stay stale.

### The throttle between two averaging-down rounds

The figure on the screen says candles, and the throttle counts candles now. One method lengths a candle of the bot's own timeframe, and both candle-counted settings read it, so the watch-list refresh and this throttle cannot disagree about what a candle is.

`src/trading/extractor_bot.py` — `ExtractorBot._candle_seconds`

```
return float(
    TIMEFRAME_SECONDS.get(
        self._timeframe, TIMEFRAME_SECONDS[self.DEFAULT_TIMEFRAME]
    )
)
```

The throttle reads the stamp the position already carries from its last correction, and from its entry before that, so the first correction after an entry waits the same span as every correction after it.

`src/trading/extractor_bot.py` — `ExtractorBot._maybe_fire_correction`, the throttle

```
skip_seconds = self._correction_skip_seconds()
if skip_seconds > 0 and (time.time() - pos.last_correction_ts) < skip_seconds:
    return  # skip-candles throttle
```

Driven at nine candles on an hourly bot, a correction 32,399 seconds old was refused and one 32,401 seconds old was admitted, so the boundary sits at nine candles exactly. The same nine candles read 2,700 seconds on a five-minute bot, 8,100 on a fifteen-minute, 129,600 on a four-hour and 777,600 on a daily. A typed zero switches the throttle off. A timeframe the engine's table does not carry falls back to the hourly length.

Two further things follow from reading the stamp. It is saved with the position and restored with it, so a restart no longer forgets how long a pair has been waiting. And a record written before that stamp existed restores it as zero, which reads as long ago and admits a correction at once — the same answer the tick counter gave for a pair it had never seen.

**This whole throttle is overtaken.** Every sentence above stays as it is written. The section describes a throttle between two averaging-down rounds, and averaging down went with the settings the ruling of 13 September 2026 removed.

> *The throttle reads the stamp the position already carries from its last correction, and from its entry before that, so the first correction after an entry waits the same span as every correction after it.*

No Python file defines the correction method the citation above names, and no Python file defines the helper inside it. Searched over the git index across every tracked Python file:

```
_maybe_fire_correction       0 occurrences
_correction_skip_seconds     0 occurrences
_last_correction_tick        0 occurrences
_candle_seconds              3 occurrences
_refresh_interval_seconds    5 occurrences
```

The last two rows are the control: the same search finds the candle-length helper and the refresh interval that still read it, so a zero above is a reading of the tree and not of the search. The watch-list refresh is the one candle count that survives.

### What the pool picker does when it is empty, and when it is not

Both settings of the picker run today. Leave every box clear and the bot ranks every alt that trades against its base by the last day's volume and keeps the busiest, re-ranking on each refresh. Tick boxes and the bot hunts those pairs and no others, for as long as they stay listed; a pair the venue has dropped is removed with a log line that names it.

`src/trading/extractor_bot.py` — `ExtractorBot._refresh_watch_list`, the branch

```
manual_targets = list(getattr(self.config, "extractor_alt_targets", []) or [])

if manual_targets:
```

Driven on five scannable markets with three pairs ticked, one of the three not listed, the watch list read the two that were listed. Cleared, the same five markets ranked to five by volume. A pair holding an open position stays on the list either way.

### What the hedge budget does at zero, and above it

At zero the reserve is off and every averaging-down round spends the pool. Above zero the bot converts the dollars into base units at the price it reads for its own base currency, holds them apart from the pool, and spends them on corrections before the pool is touched. The same amount is added to what this bot claims against the venue, so a sibling bot on the same asset cannot spend it.

`src/trading/extractor_bot.py` — `ExtractorBot.set_initial_chunk_rate`, the conversion

```
if self._hedge_budget_usd > 0:
    self._hedge_free_base = self._hedge_budget_usd / usd_per_base
```

Driven with a forty dollar budget against a base priced at sixteen dollars, the reserve read 2.5 base units, and the claim written against the venue covered the pool and the reserve together.

**The hedge conversion above is overtaken.** Every sentence stays as it is written. The entry earlier on this page already records that the hedge budget is removed, and the two fields the block above cites went with it.

> *At zero the reserve is off and every averaging-down round spends the pool.*

No Python file defines either field. Searched over the git index across every tracked Python file, against the pool field that still carries the claim:

```
_hedge_budget_usd     0 occurrences
_hedge_free_base      0 occurrences
_chunk_size_base     22 occurrences
```

The claim the bot writes against the venue now covers the pool alone.

### The standing alt quantity reaches the pool now

The method that reads it runs. An Extractor reads its base currency's dollar price on its first refresh and hands that first reading to the rebase, which is where the standing quantity is read. For an Inverted Extractor holding a standing position, the quantity becomes the pool and the dollar figure becomes a reading of it rather than an input.

`src/trading/extractor_bot.py` — `ExtractorBot.set_initial_chunk_rate`, the inverted branch

```
if self._is_inverted and _standing > 0:
    self._chunk_size_base = _standing
    self._chunk_size_usd = _standing * usd_per_base
```

Driven on a record holding 31.25 standing units against a base priced at sixteen dollars, the restored bot read a pool of 31.25 base units and five hundred dollars, where the same bot before the rate held four hundred of each. The quantity also sizes the claim the bot writes against the venue.

**The inverted branch above is overtaken.** Every sentence stays as it is written. Its opening sentence is still true, and the branch it credits is not.

> *The method that reads it runs.*

That much holds. The rebase runs on the first tick of a bot with no position open, which is the call this page records two sections above.

> *For an Inverted Extractor holding a standing*
>
> *position, the quantity becomes the pool and the dollar figure becomes a reading of*
>
> *it rather than an input.*

No Python file defines the direction field the branch tests, and no Python file defines the standing-quantity name it reads. Searched over the git index across every tracked Python file, against the two names the rebase still uses:

```
_is_inverted                              0 occurrences
inverted_extractor_standing_alt_units     0 occurrences
_chunk_size_usd                          76 occurrences
_usd_per_base_rate                       37 occurrences
```

The pool now rebases from the dollar figure on every Extractor, and the dollar figure is the input rather than a reading.

### Who may close an Extractor Tranche

The operator's own description of the two sides:

> *"...these tranches remain until the acquired Alternate Currency stack is sold*

*by either the Extractor Bot (Sibling) or its corresponding Scrumming Bot (Parent). All Tranches will persist under a bot's Details > Tranches Tab."*

One of the two sides runs. The Extractor closes its own tranche on a bullish exit, at the share the exit setting names, and that is the Sibling side. The Parent side names a force-sell by the base-currency Scrumming Bot at a percentage of growth, and nothing in the engine carries that percentage. It is not a field on a bot's configuration, it is not one of the fifteen Extractor settings, and every mention of a force-sell in the source sits inside a label or a tooltip. What the toggle does is store a word.

`src/trading/extractor_bot.py` — `ExtractorBot.set_tranche_arbiter`, the write

```
position.arbiter = normalize_arbiter(arbiter)
```

Driven end to end on a restored Extractor under a restored parent: the tranche row read Sibling, a set read Parent, two presses returned Sibling then Parent, the parent's own listing reported the same Parent, and an import of the saved record read Parent back. No order was placed and no balance moved. The toggle's own tooltip already says the Parent side is not acted on, and it is accurate.

A built Parent side would read a growth percentage the engine does not declare. The shape it would take, as a proposal rather than a build:

PROPOSED

```
# PROPOSED. The growth percentage arrives as an argument because no
# field carries one.
def force_sell_extractor_tranche(self, tranche_id: str, growth_pct: float) -> bool:
    row = self._extractor_tranche_row(tranche_id)
    if row["arbiter"] != ARBITER_PARENT:
        return False
    cost = row["cost_basis_base"]
    growth = (row["mark_value_base"] - cost) / cost
    return growth >= growth_pct / 100.0
```

Where that percentage comes from is the operator's decision: it is a control nobody has named, and the toggle waits on it.

### What the re-measured settings reach today

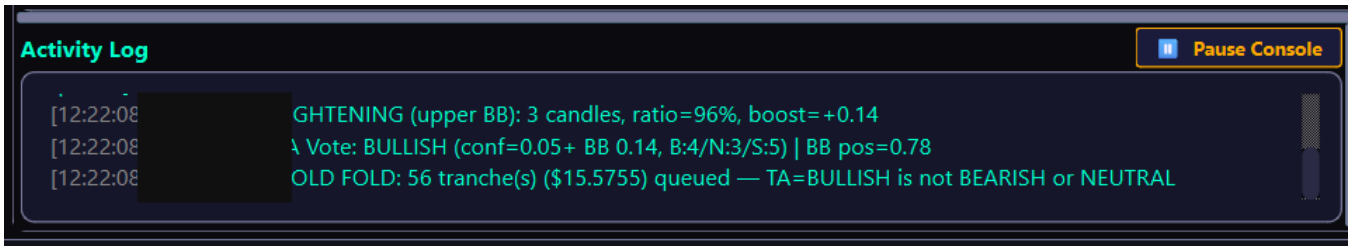
| setting | engine read | what the read decides |
|---|---|---|
| Target alt pairs | `extractor_bot.py:493` | the typed pair list, or the ranked scan when it is empty |
| Direction | `extractor_bot.py:730` | whether the entry side buys or sells, and which half of a pair the filter keeps |
| Standing alt units (inverted) | `extractor_bot.py:242` | the pool of an Inverted Extractor, and the size of its claim |
| Correction skip candles | `extractor_bot.py:1030` | the span between two averaging-down rounds, in candles of the bot's timeframe |
| Drawdown threshold | `extractor_bot.py:894` | when a position counts as down |
| Hedge budget (USD) | `extractor_bot.py:260` | the reserve corrections spend before the pool, and part of the claim |
| Trend Strength Threshold | `extractor_bot.py:211` | the threshold the signal provider is built with, once |
| Arbiter, on a tranche | `extractor_bot.py:125` | a word on the tranche; no decision reads it |

Two of the eight can only be set before a bot exists and have no running-bot control: the trend strength threshold, because its provider is built one time, and the direction, because no screen offers it afterwards.

### Additional Main Window > Trading Tab Features

### Trade Logic and Gate Activity

This spool displays the trading logic and gate activity.




The Activity Log pane.

**Functional.** The pane is read-only. Every line opens with a timestamp in a muted colour, and the message takes its colour from its level: one colour for info, one for success, one for warning, one for error. Three kinds of message get a shape of their own. A trade notification draws larger and bold and takes its colour from the stage of the trade. A wire flow or wire income line draws in magenta behind a bolt character. A wire stack line draws in the pending colour behind the same character. The pane holds 5,000 lines and drops the oldest past that.

### How a line takes its shape

One line is a timestamp, then an optional bolt character, then the message:

```
[hh:mm:ss] <bullet><message>
```

The words the message opens with choose the shape. `TRADE NOTIFICATION:` takes the trade shape, and the stage word inside it chooses the colour: green for `FILLED`, amber for `PLACED`, the primary colour for `SENT`, and red for a message naming no stage, which is what a cancellation gets. `WIRE FLOW` and `WIRE INCOME` take magenta, `WIRE STACK` the pending colour. Every other message takes its level colour.

A line a bot wrote opens with that bot's own tag, `[TICKER/last4]`, added so you can tell which bot spoke. The tag sits before the message and is not part of it, so the shape is read from the text after the tag. A bot's fill therefore draws in the trade shape, the same as those words written straight to the pane, and still names its bot.

### The parts one line draws

The sentence opening this section is overtaken. Quoted whole:

> *One line is a timestamp, then an optional bolt character, then the message:*
>
> `[hh:mm:ss] <bullet><message>`

A line is a timestamp, then the bot's own tag, then the glyph, then the event, the side, the market and the numbers:

```
[hh:mm:ss] [TICKER/last4] <glyph> <EVENT> [<side>] <market> — <numbers>

[16:46:30] [A15/c7a2] ▼ FILLED [SCRUM] A15-USD — 0.001842 @ $61234.50000000
[16:46:30] [A15/c7a2] ▲ FILLED [FOLD] A15-USD — 0.003000 @ $60100.00000000
[16:46:30] [A15/c7a2] ■ WIRE INCOME: +$1.2345 from a1b2c3d4 distributed evenly
[16:46:30] [A15/c7a2] RISK GATE [SCRUM] blocked by hysteresis_scrum. Price $0.00000317
```

Each part, and what already sets it:

| part | what sets it |
|---|---|
| the timestamp | taken when the message arrives, drawn in a muted colour |
| the bot's tag | the window adds it so you can tell which bot spoke |
| the glyph | a scrum sells, so it draws the chart's own sell glyph; a fold buys, so it draws the chart's buy glyph |
| the bolt | a wire line keeps the bolt it always had, now behind the tag rather than in front of it |
| the event | the stage of the trade, in capitals, which is the same word the line's colour comes from |
| the side | the trade's own role, in square brackets, the way a gate line already writes its side |
| the market and the numbers | the pair traded and the writer's own figures, to the places they were always drawn |

### The field that names a line's shape

The sentence naming what chooses the shape is overtaken. Quoted whole:

> *The words the message opens with choose the shape.*

The writer names the kind of line it is writing, and the pane draws the shape that kind carries. A trade writer names a trade, a wire flow or wire income writer names a wire flow line, a wire stack writer names a wire stack line, and every other writer names nothing and its line draws in its level colour.

Sixteen writers name a kind today, all of them in the bot brain, and the count is the whole set: one trade writer and fifteen wire writers.

```
LINE_KIND_TRADE = "trade"
LINE_KIND_WIRE_FLOW = "wire_flow"
LINE_KIND_WIRE_STACK = "wire_stack"
```

`src/core/event_bus.py` — the three kinds a writer names on its own bot line

The stage word inside a trade message still chooses that line's colour, and the bot's tag still opens a line a bot wrote. What changed is that a trade line no longer has to open with the words `TRADE NOTIFICATION:` to draw in the trade shape. A line that names a trade later in its text draws in the trade shape too, because the kind decides and the words do not.

```
def line_style(
    message: str, level: Any = DEFAULT_LOG_LEVEL, kind: Optional[str] = None
) -> dict:
    tag = bot_tag(message)
    shaped = shape_source(message)
    if kind == KIND_TRADE:
        drawn, role = trade_text(shaped)
```

`src/gui/main_tabs/status_log_surface.py` — `line_style`

### What the pane says about a writer that names no kind

A line whose writer names no kind still draws. It draws at the plain size in its level colour, exactly as a plain line does, and nothing is lost from the pane.

The pane reports it when that line's own words ask for a shape. The report carries the words the line opens with, which name the writer, beside the kind the pane drew. A writer that should name a kind and does not is then visible on the System Status tab instead of quietly drawing the wrong shape.

```
_log_emit(
    "trading.12.008.postcondition.line_kind_named_by_writer",
    actual=kind or surface.NO_KIND,
    expected=asked,
    context={
        "writer": surface.writer_mark(message),
        "drawn_kind": style["kind"],
        "font_size_px": style["font_size_px"],
        "bold": style["bold"],
    },
)
```

`src/gui/widgets/status_log.py` — `StatusLog._report_named_kind`

Read on the running program in both builds, with the home on a scratch directory and every socket but loopback refused, driving invented tickers and figures: ten lines drove, ten drew the shape their writer named, and the Qt pane and the page agreed on every size, weight and colour. The one line driven with no kind drew plain in both builds and produced the one report reading a fault. The Simulator's own pane drew all ten the same as the Live pane. Before the change, a fill whose trade words did not open the text drew plain in both builds.

A trade line drops the words `TRADE NOTIFICATION:`. Those nineteen characters tell the pane which shape to draw and tell you nothing you cannot already see: a larger, bold, stage-coloured line is a trade.

A role the chart sets no side for draws no glyph, which is what a wire-stack acquisition and a self-destruct sale get. Nothing is drawn for where a trade leaves the position, because no trade message carries a holdings figure. That figure sits on the plain line a fold writes beside its trade line.

One function decides the shape for both builds, so the Qt pane and the React page cannot drift apart:

```
def line_style(message: str, level: Any = DEFAULT_LOG_LEVEL) -> dict:
    """The three parts one message draws, and the weights it draws them in.
    ``TRADE_PREFIX`` then the wire prefixes decide before ``level_color`` does."""
    tag = bot_tag(message)
    shaped = message[len(tag) :]
    if shaped.startswith(TRADE_PREFIX):
        drawn, role = trade_text(shaped)
        glyph = ROLE_GLYPHS.get(role, "")
        return style_of(
            KIND_TRADE,
            tag,
            drawn,
            stage_color(shaped),
            font_size_px=TRADE_FONT_SIZE_PX,
            bold=True,
            bullet=GLYPH_FORMAT.format(glyph=glyph) if glyph else "",
        )
```

`src/gui/main_tabs/status_log_surface.py` — `line_style`, `trade_text` and `ROLE_GLYPHS`

The glyph is read from the declaration the Asset Charts readout already draws its fills with, so the two screens cannot disagree about which way a trade went:

```
ROLE_GLYPHS = {
    SIDE_ROLES[SELL_SIDE]: READOUT_SELL_GLYPH,
    SIDE_ROLES[BUY_SIDE]: READOUT_BUY_GLYPH,
}
```

`src/gui/main_tabs/trade_charts_tab_surface.py` — `SIDE_ROLES`

The Simulator and the Paper Trader draw their own Activity Log through the same function. Each held its own copy of the four shapes, 48 lines apiece, with its own numbers for the sizes and its own copy of the bolt; both now call the one composer, so a change to the shape reaches all three panes at once.

`src/gui/simulator/sim_status_log.py` — `SimStatusLog._render_safe`

```
def _render_safe(self, ts: str, message: str, level: str = "info") -> None:
    self.append(surface.line_html(ts, surface.line_style(message, level)))
    self.verticalScrollBar().setValue(self.verticalScrollBar().maximum())
```

### What the pane reports about a trade line

The Live pane reports every line it draws whose text names a trade. A trade line that drew plain then shows on the System Status tab as a fault, instead of passing unnoticed. The report carries the shape the pane drew and the shape the text asked for, and it reads the two apart: the drawn shape comes from the text after the bot's tag, and the asked-for shape from those words being anywhere in the message at all. A line that names no trade gets no report, so the report follows fills rather than every line.

`src/gui/widgets/status_log.py` — `StatusLog._report_trade_shape`

```
if surface.TRADE_PREFIX not in message:
    return
with contextlib.suppress(Exception):
    from src.core.signal_contract import emit as _log_emit

    _log_emit(
        "trading.12.007.postcondition.trade_line_drawn_as_trade",
        actual=style["kind"],
        expected=surface.KIND_TRADE,
        context={
            "tag": style["tag"],
            "font_size_px": style["font_size_px"],
            "bold": style["bold"],
            "color": style["color"],
        },
    )
```

Read on the running program, with the home on a scratch directory and every socket but loopback refused, driving invented tickers and figures: six lines naming a trade produced six reports. Five read matched. One read unmatched — a line whose trade words did not open the text, and which drew plain, which is the fault this report exists to show. Wire and plain lines drove in the same run and produced none. Before the change the same run produced no report at all.

Pause Console is a toggle. While it is down, each new line goes into a buffer of 2,000 instead of the screen, and a full buffer drops the newest rather than the oldest. Resume replays the buffer with the original timestamps and adds a line counting what it replayed. One writer goes through the pause regardless, for a message you must not miss. A timer checks the pane's health every sixty seconds and writes a warning into the pane itself when the render-error count rises, or when nothing has rendered for ten minutes while bots are running.

`src/gui/widgets/status_log.py` — `StatusLog.log`, the pause branch

```
if len(self._pause_buffer) < self._pause_buffer_cap:
    self._pause_buffer.append((ts, message, level))
return
```

**Design intention.** Two decisions follow from the pane's job. The buffer drops the newest line rather than the oldest, so the lines around the moment you hit pause are the ones that survive. And a pane that has gone quiet says so in the pane, because silence otherwise reads as calm.

`src/gui/main_tabs/trading_tab.py` — the silence check

```
bots_active = self._bot_manager and any(
    b.state.value == "running"
    for b in getattr(self._bot_manager, "_bots", {}).values()
)
if bots_active and age > 600:
```

**What the lines say.** Every tick message follows one shape: the event in capitals, the side in square brackets where the message has one, then a short sentence and the numbers behind it. A message names the gate that refused and the reading that made it refuse. It never names a direction or a strength the data behind it cannot give.

The gate snapshot is the longest of them. It writes the twelve-voter panel as a count and three groups, strongest confidence first, with an indicator's own raw reading in brackets where it publishes one. A neutral vote carries no confidence, so only its name prints.

```
RISK GATE [SCRUM] blocked by hysteresis_scrum. Price $0.00000317.
Panel 6 bullish, 3 bearish, 3 neutral.
bullish vortex 1.00, kaufman_er 0.70 (er 1.0), macd 0.60, adx 0.36 (adx 25.52),
volume 0.35, supertrend 0.26.
bearish bollinger_bands 0.82, stochastic_rsi 0.50, slingshot 0.35.
neutral ichimoku, rsi, zscore (z 0.143).
```

A hold line names the half of the gate that failed. Two conditions must both hold before the TA gate lets a scrum through: the direction has to be bullish or neutral, and the confidence has to clear the floor. The line says which one refused, so a bullish reading held back by its confidence is never reported as a wait for a bullish reading.

`src/trading/scrumming_bot.py` — the scrum hold reason

```
if eff_direction not in (
    SignalDirection.BULLISH,
    SignalDirection.NEUTRAL,
):
    _scrum_why = f"TA={eff_direction.name} is not BULLISH or NEUTRAL"
else:
    _scrum_why = (
        f"TA={eff_direction.name} but confidence "
        f"{eff_confidence:.4f} < {_eff_conf_floor:.4f} floor"
    )
```

**Design intention.** A confidence and the floor it is measured against print at four decimals wherever a message compares them. At two decimals they collided, and the pane showed a comparison of a number against itself, which is not a statement anybody can act on.

Both snapshot lines read one renderer, so the blocked half and the fired half of a decision cannot drift into two shapes.

`src/trading/scrumming/snapshots.py` — the one panel renderer

```
def _panel_line(summary: Optional[VotingSummary]) -> str:
    panel = _build_panel_snapshot(summary)
    if not panel:
        return PANEL_ABSENT_TEXT
```

### One gate indicator line for each position tick

Two sentences earlier on this page are overtaken. Quoted whole:

> *One line is a timestamp, then an optional bolt character, then the message:*
>
> `[hh:mm:ss] <bullet><message>`
>
> `[16:46:30] [A15/c7a2] RISK GATE [SCRUM] blocked by hysteresis_scrum. Price $0.00000317`

A position tick now draws one line, and that line ends in the same gate lights the History tab's Gates column draws. The line names each bank, the price and the panel's three counts, and the lights carry the rest. No tick writes a paragraph of voter readings to the pane any more.

```
[hh:mm:ss] GATES Scrum <state>. Fold <state>. Price $<price>. Panel <counts>. <lights>

[16:46:30] GATES Scrum blocked TGT, BB, OTD. Fold blocked OTD. Price $0.00003210.
           Panel 4 bullish, 5 bearish, 3 neutral. Latched hysteresis_fold.
[16:46:30] GATES Scrum armed. Fold armed. Price $0.00003210.
           Panel 4 bullish, 5 bearish, 3 neutral.
```

Nineteen lights close the line: the ten scrum gates, the scrum bank's marker, the nine fold gates, then the fold bank's marker. Each light is its label above its own colour, and the colour says what that gate did on this tick.

| light | what it says |
|---|---|
| green | the gate passed and its bank is armed |
| red | the gate is blocking its bank |
| amber | the gate was read and something else is blocking |
| grey | the gate was not read on this tick |
| cyan | the landing strip is overriding that bank |

A tick draws its line when the lights are not the same as the last line's. A tick whose gate state has not moved spends no line, which is what stops the spool. A latched gate is named in words as well, because that block holds until its own condition clears rather than turning over each tick.

```
def _emit_gate_light_line(
    self,
    scrum_result,
    fold_result,
    summary,
    ticker_last: float,
) -> None:
```

`src/trading/scrumming/snapshots.py` — the one per-tick writer

The whole voter panel and every blocker phrase still exist. They go to the Console tab's own log on each line the pane draws, so a reading a tick produced is never lost, and the History tab keeps the full blocker text in its Gates column.

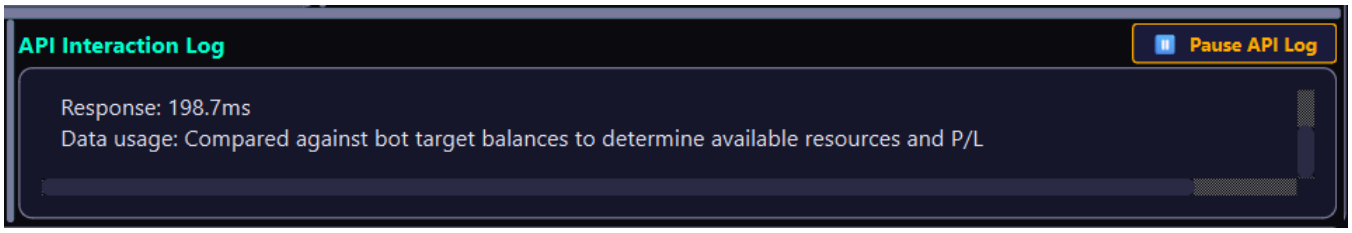
### The timestamp keeps a column of its own

A wrapped message used to continue under the timestamp. The timestamp is now its own column, and a message that wraps continues where the message starts.

Read on the running program in both builds, with the home on a scratch directory and every socket but loopback refused, driving an invented ticker and figures: twelve ticks asked the pane for sixteen lines before the change and three after, and the characters drawn fell from 5,248 to 396. A tick with two blocked gates and a tick with none drew different lines. In the Qt pane the wrapped lines started under the timestamp on all five before the change and on none of seven after. On the page the message box began at the same place as the timestamp before the change and to the right of it after. The nineteen lights drew in both builds, in one order, from one colour table.

### API Interaction Log

This spool displays API handshake information and data transfer speeds for these messages.




The API Interaction Log pane.

**Functional.** The pane is read-only, holds 2,000 lines, and never wraps. One entry is written per API call. A call that arrives on any thread but the GUI thread is refused outright and written to a thread-violation file under the runtime log directory instead, because touching a widget from a background thread ends the process. An entry carries a timestamp, the exchange name, the action and a Reason line, then Endpoint, Result, Response time and Data usage wherever the record holds them. Pause API Log buffers up to 2,000 lines and flushes them on resume with a count.

`src/gui/main_window.py` — `_on_api_event`, the thread check

```
current = _threading.current_thread().name
origin = entry.get("_thread_name", "unknown")
if current != "MainThread":
```

**Design intention.** The pane should tell you what the platform did with the bytes it just paid for. One line does the opposite. The candle fetch writes a Data usage note naming a seven-indicator engine and lists seven names. The engine builds twelve and the Voting Panel shows all twelve, so the log tells you something the screen next to it contradicts.

`src/exchange/ccxt_connector.py` — `get_ohlcv`, what it writes today

```
data_usage="Fed into 7-indicator TA engine (BB, Vortex, MACD, StochRSI, Ichimoku, Volume, Slingshot) for
voting",
```

*Proposed, not present:*

```
data_usage=(
    "Fed into the "
    f"{len(DEFAULT_WEIGHTS)}-indicator TA engine for voting"
),
```

`DEFAULT_WEIGHTS` in `src/trading/ta_engine.py` is the one declaration of the voter set, so a count taken from it cannot drift again. Issue #417 carries this.

### Two mechanisms with no control on this tab

Two more mechanisms sit inside the engine and the wizard offers no switch for either. Neither appears anywhere else in this manual, and their state differs: one runs, one is declared and dormant.

**Fair Value Gap.** Three candles that leave a gap between the first and the third mark a price band the market tends to come back to. The indicator scans the last 24 candles for those bands, reports whether price sits inside one or is approaching one, counts how many are open, and gives the bounds of the nearest. It is not one of the twelve voters. It adjusts the confidence of a side that already has a case: a fold gains ten points near a bullish band, a scrum gains eight inside a bearish one.

`src/trading/indicators/fvg.py` — the four constants that set its reach

```
FVG_LOOKBACK = 24  # candles to scan for 3-candle gap patterns

FVG_PROXIMITY_PCT = 0.005  # "approaching from above": within 0.5% of FVG top

FVG_BULL_BOOST = 0.10  # fold confidence boost when in/near bullish FVG

FVG_BEAR_BOOST = 0.08  # scrum confidence boost when inside bearish FVG
```

The engine imports the indicator and its four constants together, so the adjustment travels with the reading rather than being applied somewhere else.

`src/trading/ta_engine.py` — what it takes from that module

```
from .indicators.fvg import (
    FVG_BEAR_BOOST,
    FVG_BULL_BOOST,
    FVG_LOOKBACK,
    FVG_PROXIMITY_PCT,
    FVGIndicator,
)
```

**Boost Fold.** The intent is a pair: sell a slice of holdings when the mean reversion reading goes extreme, then buy that slice back when price returns to its moving average, and let the difference raise the target. The Market Inspector states the pair at the head of its own module.

`src/trading/mr_inspector.py` — the pair, in the module's own words

```
# ■ When all 3 gates pass → Boost Sell (sell 2.5% of holdings) ■
# ■ When price returns to SMA → Boost Fold (buy back cheaper)  ■
# ■ Profit from boost fold increments target (compound growth)  ■
```

Where it runs is the stock accumulation bot, which carries the queued amount, the reference price, the moving average and the two counters, and works all of them. The crypto Scrumming Bot declares the same four fields, sets them to zero and never reads them again: across the whole trading package the only lines naming them are those four assignments. A crypto bot therefore takes no boost fold, whatever the Market Inspector reads.

`src/trading/scrumming_bot.py` — the four fields, written once and never read

```
        self._boost_fold_q: float = 0.0
        self._boost_fold_ref: float = 0.0
        self._boost_fold_sma: float = 0.0

        self._boost_folds: int = 0
```

*Proposed, not present, in* `src/trading/scrumming_bot.py`*:* the crypto side needs the read half of the pair, taking the queued amount back at the moving average the way the stock bot does, guarded so a bot with an empty queue takes no action.

```
if self._boost_fold_q > 0 and self._boost_fold_sma > 0:
    if price <= self._boost_fold_sma and price < self._boost_fold_ref:
        buy_usd = self._boost_fold_q
        self._boost_folds += 1
        self._boost_fold_q = 0
```

### Add Crypto Exchange Button (to be changed - Add Exchange)

While the initial set up of Acervator requires at least one valid exchange API, additional exchanges can be added and have their own bot swarms. The current upper operational limit of Acervator is unknown. Multi-exchange testing has yet to be attempted as of 8/25/26 with API compatibility work pending. This button also currently opens the Settings Panel but on the incorrect ‘User’ Tab when it should be ‘Exchanges’.

The Add Crypto Exchange button.

**Functional.** The button sits at the right of the exchange sub-tab row, as the corner widget of that row. Each layer builds its own and labels it for that layer: Add Crypto Exchange on the crypto page, Add Stock Exchange on the stock page. Both run the same handler as the button on the empty card a layer shows while it holds no exchange at all.

`src/gui/main_tabs/trading_tab.py` — `_make_layer`

```
tab_w = QTabWidget()
add_btn = QPushButton(f"■ Add {label_text} Exchange")
add_btn.setMinimumWidth(140)
add_btn.setToolTip(f"Add a {label_text} exchange connection")
add_btn.clicked.connect(self._add_exchange)
tab_w.setCornerWidget(add_btn)
```

**Design intention.** The handler already knows which wing you are in and passes that to the dialog, so it has somewhere to say which tab to open.

`src/gui/main_window.py` — `_add_exchange` today

```
dlg = SettingsDialog(self._settings, self._status_log, self, wing=_wing)
dlg.exec()
```

*Proposed, not present:*

```
dlg = SettingsDialog(self._settings, self._status_log, self, wing=_wing)
dlg.show_exchanges_tab()
dlg.exec()
```

The dialog in `src/gui/settings_dialog.py` has no such method yet. The proposal adds one rather than changing the call shape, so the wing argument already passed here keeps working exactly as it does.

**The exchange row in the Electron shell.** The shell draws one tab button per configured exchange, and the screen under the button is drawn by `exchange_tab.js`. A layer shows its empty card only while it holds no exchange, which is what the Qt tab row does once it drops the Get Started tab.

`src/gui/main_tabs/trading_tab_surface.py` — `layer_exchanges`

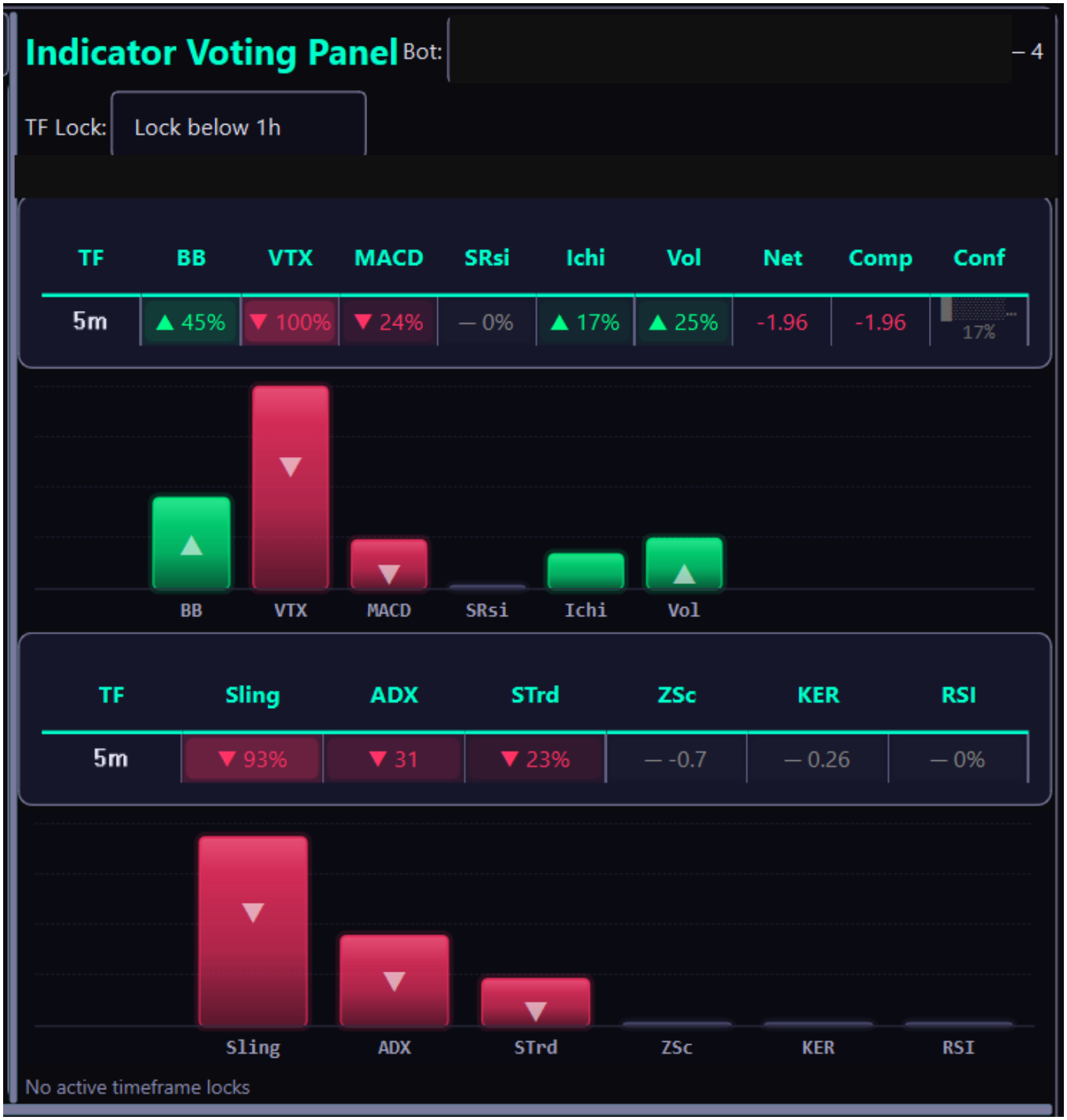
```
for entry in entries or []:
    holder = entry if isinstance(entry, dict) else {}
    exchange_id = str(holder.get("exchange_id") or "")
    if not exchange_id:
        continue
    layer = "stock" if is_equity_exchange(exchange_id) else "crypto"
    split[layer][exchange_id] = exchange_display_name(holder)
```

**One caption for one exchange.** The caption on a tab comes from a single helper, so the start-up path and the settings path cannot label the same exchange two different ways. An exchange saved with no name is captioned from its own id, and an entry with no id gets no tab at all.

`src/gui/main_tabs/trading_tab_surface.py` — `exchange_display_name`

```
def exchange_display_name(entry: Any) -> str:
    holder = entry if isinstance(entry, dict) else {}
    exchange_id = str(holder.get("exchange_id") or "")
    named = str(holder.get("display_name") or "")
    return named or exchange_id.capitalize()
```

### Indicator Voting Panel




The Indicator Voting Panel, at the right of the Trading tab.

**Functional.** The panel fills the right half of the Trading tab.

- The Bot selector at the top names the bot whose votes the panel draws. The list refills every tick. The badge beside it counts the bullish, bearish and neutral voters.
- TF Lock chooses a timeframe below which an opposing trade is refused. The list opens on "None (no lock)".

- The rate line under it shows the A15 and A14 prices with their satoshi and gwei equivalents, and the venue name at the end.
- Two tables of six voters each. A green up triangle is a bullish vote, a red down triangle a bearish one, an em dash a neutral one. ADX, ZSc and KER print a raw value; the other nine print a percentage.
- Each voter belongs to one of three groups, trend, momentum or structure, and the group decides its header colour.
- Net, Comp and Conf close the first table. Conf draws as a small filled bar with its percentage beside it.
- A bar chart under each table draws that table's six confidences at the width of the column above.
- The line at the foot names any timeframe lock that is active.

**Functional.** The dropdown is not the only way to pick the bot. A press on a row of the Scrumming Bots table draws that bot here, and the dropdown moves to it. A press on the row of the bot already drawn empties the panel, and the panel then reads *"no bot is selected — press a bot row, or pick one from the dropdown above."* Whichever control the operator uses, the bot list's highlight and this panel name the same bot.

`src/gui/indicator_panel.py` — `IndicatorVotingPanel.select_bot`

```
wanted = str(bot_id or "")
if not wanted:
    self._selection_cleared = True
    self._bot_selector.setCurrentIndex(-1)
    self._refresh_bot_arrows()
    return self._selected_bot_id
```

`src/gui/indicator_panel.py` — `IndicatorVotingPanel.INDICATOR_COLS`

```
INDICATOR_COLS = [
    ("bollinger_bands", "BB", "S"),
    ("vortex", "VTX", "T"),
    ("macd", "MACD", "M"),
    ("stochastic_rsi", "SRsi", "M"),
    ("ichimoku", "Ichi", "T"),
    ("volume", "Vol", "S"),
    ("slingshot", "Sling", "S"),
    ("adx", "ADX", "T"),
    ("supertrend", "STrd", "T"),
    ("zscore", "ZSc", "M"),
    ("kaufman_er", "KER", "M"),
    ("rsi", "RSI", "M"),
]
```

**Design intention.** One place that shows what every voter thinks and how far they agree. The names, the split into two rows and the three colour groups all come out of the one list above, so the panel cannot fall out of step with itself as voters are added or reordered.

`src/gui/indicator_panel.py` — the two rows, both cut from that list

```
_ROW_A_INDICATOR_COLS = INDICATOR_COLS[:6]  # BB VTX MACD SRsi Ichi Vol
_ROW_B_INDICATOR_COLS = INDICATOR_COLS[6:]  # Sling ADX STrd ZSc KER RSI
```

The operator's own reading of each column follows in the next section.

**Where the maths lives.** 07-indicators.md is the single home for all twelve. Each voter is described once there and once only: the operator's own line, then what the cell prints and what makes the vote bullish, bearish or neutral, then the published formula with the code that computes it. Nine voters print a direction arrow and a confidence percentage. Three print a raw value instead, and those three are ADX, Z-Score and Kaufman ER.

| Voter | Its formula and its vote |
|---|---|
| BB | Bollinger Bands |
| VTX | Vortex Indicator |
| MACD | MACD |
| SRsi | Stochastic RSI |
| Ichi | Ichimoku Cloud |
| Vol | Volume |
| Sling | Slingshot |
| ADX | ADX and DMI — raw value |
| STrd | Supertrend |
| ZSc | Z-Score — raw value |
| KER | Kaufman Efficiency Ratio — raw value |
| RSI | RSI |

One decision picks the text a cell carries. Three voters are named in it and every other voter falls through to the percentage.

`src/gui/indicator_panel.py` — `_populate_indicator_cell`

```
if ind_key == "adx":
    adx_v = details.get("adx", 0)
    if details.get("ranging"):
        cell_text = f"Rng {adx_v:.0f}"
    else:
        cell_text = f"{sym} {adx_v:.0f}"
elif ind_key == "zscore":
    cell_text = f"{sym} {details.get('z', 0):+.1f}"
elif ind_key == "kaufman_er":
    cell_text = f"{sym} {details.get('er', 0):.2f}"
else:
    cell_text = f"{sym} {confidence:.0%}"
```

The same file carries the gate logic chain that turns those twelve votes into a trade decision.

### Net, Comp and Conf

The three columns that close the first table each read one field. Each one draws a number in its cell and a bar behind both tables, and both the number and the bar follow that column's own field.

| Column | Its field | What it holds | Its range |
|---|---|---|---|
| Net | `net_score` | the bullish weight of the twelve voters, less their bearish weight | a signed tally, zero when the voters cancel out |
| Comp | `composite_net` | Net joined with the Net of each higher timeframe a phantom bot watches | the same units as Net; a dash when the row carries no composite |
| Conf | `confidence` | how far the voters agree with each other | 0 to 1 |

`src/gui/main_tabs/indicator_panel_surface.py` — one definition, read by the window and by the page

```
drawn = [
    (sign_direction(net), net_fraction(net), BAR_COLORS[sign_direction(net)]),
    (sign_direction(comp), net_fraction(comp), BAR_COLORS[sign_direction(comp)]),
    (band, conf, _channels(CONF_BAND_COLORS[band])),
]
```

**Bar height.** Conf's bar is its own percentage. Net's and Comp's bar stands full at `NET_PILLAR_FULL_SCALE`, which is 3.00 either way; a reading past that draws a full bar. The twelve voters could together produce a much larger tally than 3.00, and a bar scaled to that ceiling would never leave the floor.

**Bar colour.** The hue names the kind of reading: green above zero and red below it for Net and Comp, and for Conf grey under 30 per cent, amber at 30 and over, green at 60 and over. The brightness names the size of the reading, from `PILLAR_SHADE_FLOOR` of the hue at zero to the whole hue at full scale. The cell's text takes the same hue and brightens toward white by `CELL_LIFT_SPAN` instead of dimming, so a weak reading is never harder to read than a strong one.

**The descriptions.** Each of the three carries the same text on its column heading and on its cell, so a hover over either one answers.

> ***Net.*** *Net vote. The bullish weight of the twelve voters above, less their bearish weight. Above zero the panel leans up. Below zero it leans down. A voter with no opinion adds nothing. The bar below stands full at 3.00 either way, and a bigger vote draws a brighter bar.*
>
> ***Comp.*** *Comp is short for composite Net. It is this bot's Net vote, joined with the Net vote of each higher timeframe a phantom bot watches. A higher timeframe pulls the reading its own way. With no phantom bot running, Comp reads the same as Net. A dash means this row carries no composite of its own.*
>
> ***Conf.*** *Conf is short for confidence. It measures how far the voters agree with each other, and nothing else. It is the Net vote divided by the weight of every voter that cast one. 0% means the voters cancel out or none has an opinion. 100% means every voter that cast one agrees. A high reading does not say the trade is a good one, only that the panel is of one mind. Grey is under 30%, amber is 30% and over, green is 60% and over.*

The maths behind Net and Conf lives with the voters, in 07-indicators.md.

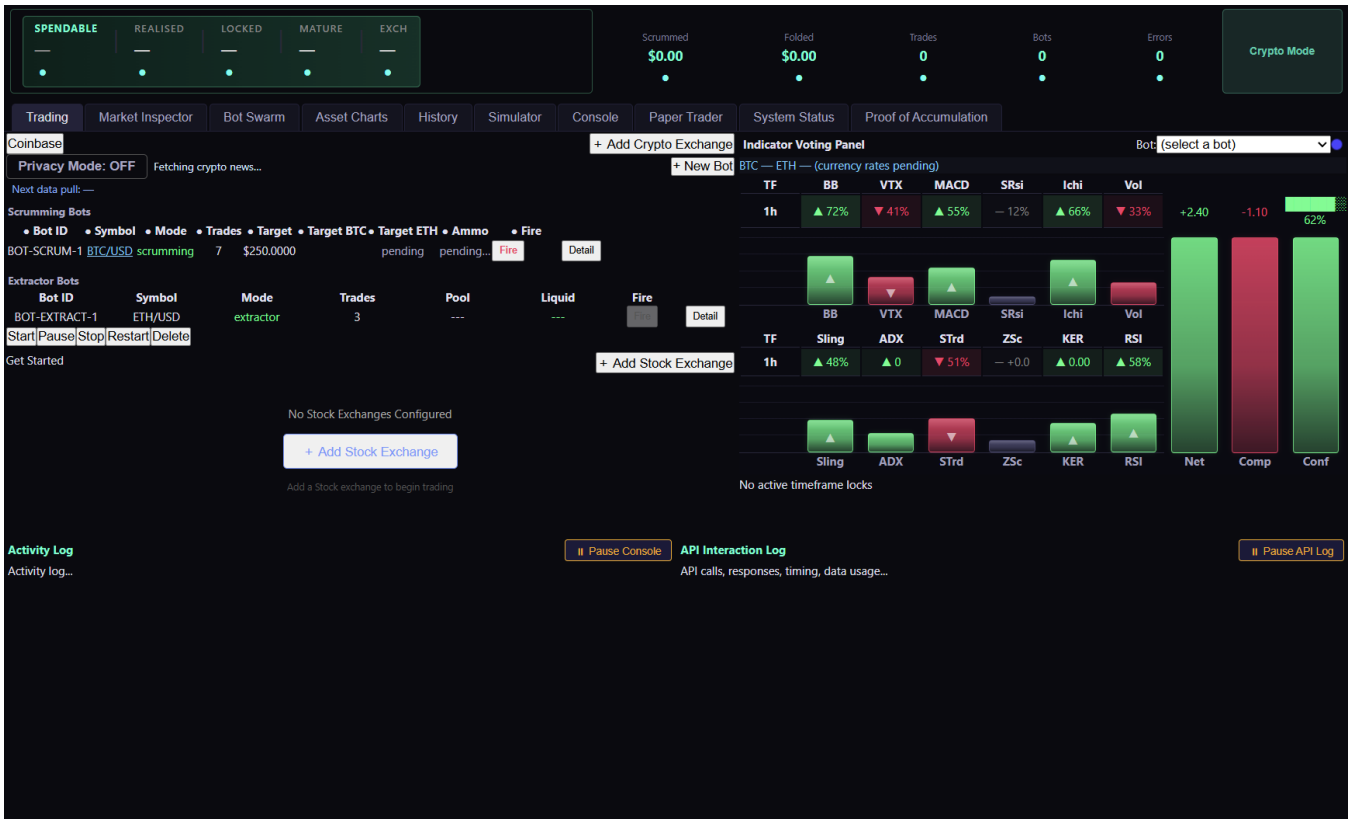
### The Indicator Voting Panel, restyled

The panel beside the exchange stack carries seven columns on each row, aligned under each other: TF and six indicators above, TF and six more below. The three collated columns — Net, Comp and Conf — close the first row and stand as one pillar each behind both rows.

`src/gui/indicator_panel.py` — the two rows share one grid

```
col_names = ["TF"] + [short for _, short, _ in indicator_subset]
col_names += [""] * (_PANEL_COLUMN_COUNT - len(col_names))
```

The TF Lock row is gone, and so are the pair symbol and the vote tally that stood in the upper right. The bot selector and its privacy dot moved there. 07-indicators.md carries the panel's own entry.



The Trading tab in the Electron shell

## 2026-09-11 16:48 - #665 - the wizard's values reach a new bot

Bot creation held two dict literals. Each named the settings it would pass to a new bot, and the wizard collected more settings than either named. A name absent from both lists was collected, held in memory and never read, so the bot took the declared default for that field instead. Twenty-five Scrumming settings and six Extractor settings went that way.

One function now selects the kwargs, and it reads the declaration rather than a list of names. `bot_config_kwargs` keeps every key that names a field `BotConfig` declares, and drops the keys that belong to the other mode — which is the same rule `make_bot_config` applies when it refuses one. A field

added to `BotConfig` and emitted by the wizard arrives with no second edit.

`src/trading/container/config.py` — `bot_config_kwargs`, the selection

```
foreign = (
    _BOT_CONFIG_SCRUMMING_ONLY_FIELDS
    if mode == BotMode.EXTRACTOR
    else _BOT_CONFIG_EXTRACTOR_ONLY_FIELDS
)
# make_bot_config sets mode itself and raises on a foreign field.
carried = {f.name for f in fields(BotConfig)} - foreign - {"mode"}
```

### What the entries above now read

Five entries on this page state that bot creation does not pass a setting. Each sentence described the creation path as it stood, and the creation path has moved. The corrected reading for each:

| entry | the sentence above | what it now does |
|---|---|---|
| Wire Inflow Stack | "Bot creation does not pass this setting, so a new bot takes the declared default of 1.00 whatever you type here." | Bot creation passes `wire_inflow_stack_pct`. A new bot takes the figure on the row. |
| Profit Routing | "Issue #336 carries this, with the rest of the settings the wizard writes and bot creation drops." | Bot creation passes `profit_route` and `profit_route_bot_id`. No module under `src/trading/` reads either one, so the destination a bot holds still reaches no trade. |
| Standing alt units (inverted) | "Bot creation passes neither this number nor the Direction beside it, so a new Extractor runs Normal with a standing position of zero whatever you enter." | Bot creation passes `inverted_extractor_standing_alt_units` and `extractor_direction`. `set_initial_chunk_rate` still has no caller in the product source. |
| Correction skip candles | "Bot creation does not pass it, so a new bot takes the declared default of 4." | Bot creation passes `extractor_correction_skip_candles`. The reader still counts ticks rather than candles. |
| Drawdown threshold | "bot creation does not pass it, so a new bot takes the declared default of 3.00" | Bot creation passes `extractor_drawdown_threshold_pct`. |

Five entries already read "bot creation passes it through": Read Rate, Band Travel, Pool Reserve, Exit % and Max cost-basis multiple. Those sentences stand unchanged.

**Two cells in the table above are overtaken.** Both rows stay as they are written. The table corrected the entries above it, and two of its own corrections have since gone stale.

> *Bot creation passes* `inverted_extractor_standing_alt_units` *and* `extractor_direction`*.* `set_initial_chunk_rate` *still has no caller in the product source.*

Both halves of that cell are false now. The rebase has one call site, and the two names the cell credits to bot creation are in no Python file.

> *Bot creation passes* `extractor_correction_skip_candles`*. The reader still counts ticks rather than candles.*

The name is in no Python file either, and neither is the reader. Searched over the git index across every tracked Python file:

```
set_initial_chunk_rate                    1 call site
inverted_extractor_standing_alt_units     0 occurrences
extractor_direction                       0 occurrences
extractor_correction_skip_candles         0 occurrences
_maybe_fire_correction                    0 occurrences
extractor_scan_refresh_candles           16 occurrences
```

The last row is the control. The same search finds the refresh key that is still read, so a zero above is a reading of the tree and not of the search.

### What the run measured

One run drove the real wizard on both modes. It moved every control on every page the wizard reaches, collected the config, built a real bot through the creation path, and read each value back off that bot's own config.

```
scrumming   wizard emits 52 keys
            before   26 passed, 24 settings held the default against a typed value
            after    48 passed, 0 held a default against a typed value
extractor   wizard emits 23 keys
            before   20 passed, 6 settings held the default against a typed value
            after    22 passed, 0 of those 6 held a default
```

The Extractor's 22 names hold two the wizard does not emit: Target Balance and Stack Mode, each supplied at creation as it was before. One Extractor key still holds a default and it is the Profit Folding flag, which the table below covers.

The same run drove a save and a restore of a bot built from typed values. All 72 fields came back holding what was saved, so a restart puts back what it put back before.

A wizard left untouched still builds a bot at the declared defaults. Every Scrumming field matches its declaration. Two Extractor fields do not, and both are deliberate: the Extractor's parameter page offers no Target Balance row, so its pool figure stands in; and the page offers no Stack Mode box, so an absent key reads as off rather than as `STACK_MODE_DEFAULT`.

### Three settings the wizard still collects and a new bot still cannot read

| setting | why |
|---|---|
| Lock duration (candles) | `lock_candle_count` is no field on `BotConfig` and no argument of `ScrummingBot.__init__`. The phantom coordinator holds an attribute of that name, and Bot Settings writes it there on a running bot. |
| Profit Folding, on an Extractor | `BotCreationWizard.get_bot_config` writes `profit_folding_active: False` for an Extractor. The field is Scrumming-only, so `make_bot_config` refuses it on an Extractor config. No control types it. |
| Max adoptable USD | `max_adoptable_usd` is the one `BotConfig` field no page offers and the restore path does not read either. Nothing writes it and nothing types it. |

`enable_phantoms` and `phantom_timeframes` are not on that list. Both already reach the bot, as arguments to `ScrummingBot.__init__` rather than through the bot config.

## 2026-09-12 - the second half of the Scrumming Settings group, driven

Six rows of that group were driven on a bot built from a record written for the run. The home was redirected into a scratch directory before the settings module was imported, so the settings directory and the log root both bound under it. No stored file of the running install was opened, no venue was contacted, and the exchange object handed to the bot defines no order method at all, so the run could not place an order even where a path reached for one.

```
Max Entry Price        the price an auto-buy is refused above
Min Entry Price        the price an auto-buy is refused below
Trading Fee %          the figure added to the opposing-trade distance
Max Target Growth %    the ceiling one fold cycle may add to Target Balance
Scrum Fold Ratio       the share of a sale's tranches kept for the fold
Profit Folding Active  whether a fold may raise Target Balance at all
```

### What each row did when it was driven

Every reading below is one call into the shipped engine, with the figure on the row as the only thing changed between the two sides.

```
Max Entry Price    ceiling $90 against a price of $100   buy refused at the gate
                   ceiling $110 against the same price   gate silent
Min Entry Price    floor $110 against a price of $100    buy refused at the gate
                   the same floor, the same price, sell  no refusal of any kind
Trading Fee %      fee 0.60 %, pivot $100, buy at $98.70 refused, needs 1.60 %
                   fee 0.00 %, pivot $100, buy at $98.70 not refused
                   fee 0.60 %, pivot $100, sell at $101.30 refused, needs 1.60 %
                   fee 0.00 %, pivot $100, sell at $101.30 not refused
Max Target Growth  1.00 % on a $200 target   cap $2.00, target became $202.00
                   0.00 % on a $200 target   cap $0.00, target stayed $200.00
                   5.00 % on a $200 target   cap $10.00, target became $210.00
Scrum Fold Ratio   100 % on two $100 tranches  queued $200.00, 2.0000 units
                   40 %  on the same two       queued $80.00, 0.8000 units
                   1 %   on the same two       queued $2.00, 0.0200 units
Profit Folding     on   applied $2.00, target $202.00, preview $2.00
                   off  applied $0.00, target $200.00, preview $0.00
```

The Profit Folding row emits its own line when it is off, and that line is the one a reader should look for rather than the absence of a growth.

`src/trading/scrumming_bot.py` — the line the off state emits

```
f"[COMPOUND SKIPPED] ({source}): "
f"profit_folding_active=False — the "
f"compound-growth feature is off for "
f"this bot. No target bump."
```

### A zero in either entry-price box is not the same thing in both

Both entries above say zero means no ceiling and no floor, and both controls honour that: a box left at zero emits nothing rather than a number, measured on the wizard the Electron shell draws. The engine's own guard is narrower than the sentence. It treats only the absent value as off, so a record carrying a literal zero in the ceiling refuses **every** auto-buy for as long as it stands — driven, a ceiling of zero against a price of one hundred refused the buy at the gate. The same zero in the floor is harmless, because no price is below it.

`src/trading/scrumming/execution.py` — the two guards, as they stand

```
if _max_ep is not None and _px > 0 and _px > float(_max_ep):
if _min_ep is not None and _px > 0 and _px < float(_min_ep):
```

No control can write that zero. The three places that collect the value each write an absent value in its place, so a record carrying a literal zero came from an older build or from an edit outside the application.

```
src/gui/bot_wizard.py:1540                    absent unless the box reads above 0
src/gui/live_settings/settings_tab.py:517     absent unless the box reads above 0
src/gui/main_tabs/bot_wizard_surface.py:2267  absent unless the box reads above 0
```

*Proposed, not present, in both guards:*

```
if _max_ep and float(_max_ep) > 0 and _px > 0 and _px > float(_max_ep):
```

It is proposed rather than taken because a bot standing on such a record is refusing every buy today, and a build that reads the zero as off would have it buying on its next tick. That is his decision, not this unit's.

### The floor on the Scrum, measured on both sides

The entry above records that the buy path is the only reader. Driven, that is exactly what happens, and the reading is worth stating as a pair because the pair is what proves the instrument was working.

```
floor $110, price $100, buy   BUY REFUSED (min_entry_price gate)
floor $110, price $100, sell  no refusal emitted, the sale went on
```

The count agrees with the drive. `_execute_sell` runs from line 1333 to line 1669 of that module and the floor's name appears **nowhere** in it, against three appearances inside the buy executor above. The number his own sentence ties to a Scrum therefore reaches no sell, and a bot sitting at the floor holding the asset sells as if the figure were not set.

Read against the standing test, the behaviour the code does have is the part no sentence on this page asks for: nothing here specifies a floor under a buy. The sell-side proposal already on this page stands, and the removal of the buy-side floor is the other half of the same decision. Both move money on the next tick of any bot carrying the figure, so both are reported here and neither is taken.

### A fee of zero is read two ways at once

The entry above says the fee is added to the Minimum Opposing Trade Distance, and it is. At every figure the box offers except one, every reader agrees. At a stored zero they split: ten reads inside the trading package treat a zero as six tenths of a percent, and two inside the same executor honour the zero.

```
stored 0.60 %   distance filter 1.60 %   buy and sell hysteresis 1.60 %
stored 2.50 %   distance filter 3.50 %   buy and sell hysteresis 3.50 %
stored 0.00 %   distance filter 1.60 %   buy and sell hysteresis 1.00 %
```

The box accepts a zero, and so does the wizard the shell draws — typed at zero it emitted a zero. The two controls then show six tenths when that record is reopened, because each seeds itself the same way the ten reads do.

`src/trading/otd_math.py` — the read, and its own note on the quirk

```
return minimum_opposing_trade_distance_pct(
    getattr(config, "scrumming_interval_pct", 0) or 0,
    getattr(config, "trading_fee_pct", 0.6) or 0.6,
)
```

That module's docstring already names the quirk and says it is named there and not repaired there, so the split is known rather than newly found. Closing it changes the distance a reversal must travel for any bot whose fee is zero, which is a live figure on a live tick, so it is reported here and not taken.

### What the Max Target Growth entry above now reads

That entry records ten read sites, seven falling back to one and three to zero, and a bot that compounds or freezes depending on which read first. The first half holds and the second does not.

| the entry's reading | what the run measured |
|---|---|
| three sites fall back to zero against a declared default of one | Held before this change. The three were the manual rebalance, the compounding snapshot and the Smart Wire inputs, exactly as named |
| a bot compounds or freezes depending on which site read first | Does not hold. No bot can reach the fallback: every construction path builds the config through one factory, and the class declares the field, so the value is never absent |
| ten read sites take it with a fallback | Nine now. One of the ten was an expression whose value was discarded, and it is gone |

The three sites now name the declared default. Driven against a stand-in that genuinely lacks the field, the two reachable ones moved from zero to one and then agreed with the cycle cap; every other reading in the run was identical before and after, which is what makes the change inert on a real bot.

```
before   swos 0.0   snapshot 0.0   cycle cap $2.00
after    swos 1.0   snapshot 1.0   cycle cap $2.00
```

`src/trading/scrumming_bot.py` — the shape all nine now share

```
_growth = float(getattr(self.config, "max_target_growth_pct", 1.0) or 0.0)
```

### Where the Scrum Fold Ratio is read

The entry above gives the row its range and its start value and says nothing about where the engine reads it. One method does, once per sale, and it runs on the slice of tranches that sale appended rather than on the whole queue.

`src/trading/scrumming/fold_tranches.py` — the read and the clamp

```
_fold_pct = max(0, min(100, int(getattr(self.config, "scrum_fold_pct", 100))))
_new_tranches = self._fold_tranches[_tranche_count_before:]
if _fold_pct < 100 and _new_tranches:
```

At a hundred the branch does not run and the sale's tranches keep every dollar. Below a hundred each tranche keeps the ratio's share of units and of the sale's own proceeds, and the remainder is retired as cash with a realised profit line. Money in a tranche that did not come from this sale is wired-in credit and keeps its full value.

The clamp's lower bound is zero and both controls start at one. A record carrying a zero therefore empties the sale's whole fold queue — driven, one hundred dollars and one unit became zero dollars and zero units, with the cash retired instead. Raising the clamp to match the controls would change what such a bot rebuys on its next sale, so it is reported here and not taken.

### The Profit Folding Active row on a running bot

No entry above describes this row, because the group on the creation wizard does not carry it. The running bot's own Scrumming Settings group does, and so does the row list the shell draws from.

| | the control |
|---|---|
| Where | `src/gui/live_settings/settings_tab.py:575` |
| React row | `src/gui/main_tabs/live_settings_tab_surface.py:769` |
| Kind | checkbox, on at the start |
| What it writes | the bot config key the engine reads four times |

Profit Folding Active - When it is on, a fold's surplus may raise Target Balance up to the Max Target Growth % cap above. When it is off, the surplus is not applied, the preview of a prospective fold reads zero, and the bot writes a skipped line naming the flag. A bot restored from its own record takes its own flag and reads no application setting.

```
on   surplus $10.00 applied $2.00   target $200.00 became $202.00   preview $2.00
off  surplus $10.00 applied $0.00   target stayed $200.00           preview $0.00
```

The creation wizard holds a checkbox for the same flag on a page no route reaches, so a new bot opens on the declared default until that page is reachable.

## 2026-09-12 - the Hedge Rebalance group and the despawn timer, driven

Three rows were driven on a bot built from a record written for the run. The home was redirected into a scratch directory before the settings module was imported, so the settings directory and the log root both bound under it. No stored file of the running install was opened, no venue was contacted, and the exchange object handed to the bot defines no order method at all, so the run could not place an order even where a path reached for one.

```
Hedge Rebalance Active  whether the bot holds a reserve outside Target Balance
Hedge Balance           the size of that reserve, and the ceiling it refills to
Tranche Despawn Timer   the age at which a tranche record is removed
```

### What the hedge switch decides

The switch reaches four places in the engine and every one of them moves a decision. Two are the seeds that set the reserve at construction, one is the gate that arms a hedge buy on a tick, and one is the gate that refills the reserve out of a completed fold's profit.

`src/trading/scrumming_bot.py` — the two seeds

```
self._hedge_bal: float = (
    float(config.hedge_balance) if config.hedge_rebalance_active else 0.0
)
self._hedge_balance_initial: float = (
    float(config.hedge_balance) if config.hedge_rebalance_active else 0.0
)
```

Driven both ways at three figures, the seeds read what the two sentences above this section promise.

```
active=True   balance $200.00   reserve $200.00   cap $200.00
active=True   balance  $50.00   reserve  $50.00   cap  $50.00
active=True   balance   $0.00   reserve   $0.00   cap   $0.00
active=False  balance $200.00   reserve   $0.00   cap   $0.00
active=False  balance  $50.00   reserve   $0.00   cap   $0.00
active=False  balance   $0.00   reserve   $0.00   cap   $0.00
```

The budget layer that refuses an oversized buy reads the same reserve, so the switch decides the ceiling as well as the gate. With half a unit held at one hundred dollars, the hedge path's ceiling is the position plus the reserve.

```
active=True   balance $200.00   a $25 buy allowed, a $210 buy refused at $251.00
active=True   balance  $50.00   a $25 buy allowed, a  $75 buy refused at $100.25
active=False  balance $200.00   every buy refused at $50.00, the position alone
```

The control for the figure on the scrum path is the same call with a different path name. At a reserve of zero it still allows a seventy-five dollar buy, which is what proves the refusals above came from the hedge reserve and not from the position.

### The reserve a restart re-seeds and a toggle does not

Turning the switch on while the bot runs arms nothing. The two reserve fields are read once, at construction, and nothing on the live path writes them again, so a bot built with the switch off keeps a reserve of zero and a ceiling of zero for as long as it stands.

```
built with the switch off   reserve $0.00   cap $0.00
config flipped to on        reserve $0.00   cap $0.00
a $25 hedge buy then reads  refused
```

A restart is the one route back, and it is a partial one. The ceiling is rebuilt from the stored config, because it is not itself persisted, while the drainable reserve is restored from the saved record.

```
rebuilt with the switch on        reserve $200.00   cap $200.00
then restored from the off record reserve   $0.00   cap $200.00
the fold replenish terms then     open
```

The reserve then refills from compound growth at eight hundredths of each fold's profit, and only after a restart. Making the toggle re-seed the reserve would hand a drained bot its full reserve back the moment the operator flicked the box twice, which is money on a live tick and his decision rather than this unit's.

*Proposed, not present, in the live-settings route:*

```
RUNTIME_ROUTED = {
    "hedge_rebalance_active": "set_hedge_active_live",
}
```

### What the Hedge Balance figure bounds

The figure is the reserve's starting size and the ceiling a refill stops at. Six of its ten engine readers decide something: the two seeds, the arm test, the size of the buy, and the two budget layers that refuse an oversized one.

`src/trading/scrumming_bot.py` — the figure that sizes one hedge buy

```
_use = min(self._hedge_bal * 0.5, _gap)
```

The running bot's own row is routed through a method rather than written straight onto the config, and that method refuses a figure it cannot use.

```
set $500.00  applied   cap $500.00  reserve $120.00 unchanged
set   $0.00  applied   cap   $0.00  reserve $120.00 unchanged
set  -$5.00  refused   hedge_balance must be >= 0
set   "abc"  refused   hedge_balance must be numeric
set  $75.00  applied   cap  $75.00  reserve $120.00 unchanged
```

A cap of zero is accepted, and it shuts the refill gate for good while leaving the old reserve drainable. The bot can still spend what it holds and can never get any of it back. Raising the cap back above zero re-opens the gate, so the state is recoverable by the same control that caused it, and the refusal that would make a zero mean off instead of empty would change what a bot carrying one does on its next fold. Reported here and not taken.

### The despawn timer has no creation-wizard control

The timer is the one row of these three that a new bot cannot be given. Searching both wizard builds for the word returns nothing, so a new bot opens at the declared default of zero, which is off.

```
src/gui/bot_wizard.py                      0 matches
src/gui/main_tabs/bot_wizard_surface.py     0 matches
src/gui/live_settings/settings_tab.py      the one control, 0 to 365 days
src/gui/main_tabs/live_settings_tab_surface.py   the React row for it
```

The label on both screens reads Tranche Despawn Timer. The engine reads the stored figure exactly once, through one shared helper, and the sweep that uses it runs once per tick outside every exception handler.

`src/trading/container/config.py` — the one reading

```
def despawn_threshold_days(config) -> int:
    days = as_finite_float(getattr(config, "tranche_despawn_days", 0))
    if days is None:
        return 0
    return min(DESPAWN_MAX_DAYS, max(0, int(days)))
```

Driven across thirteen stored values, every shape the control cannot type reads as off rather than as a number.

```
0 -> 0      1 -> 1      7 -> 7      365 -> 365      30.9 -> 30
-5 -> 0     True -> 0   "7" -> 0    None -> 0       nan -> 0    inf -> 0
the field absent altogether -> 0
```

### The five claims the despawn tooltip makes

The tooltip on that control is the only specification the row has, and it makes five checkable claims. Each one was driven on a bot holding seven fold records and five stack records.

| The claim | What the run measured |
|---|---|
| Despawn is not a trade | Zero calls reached the exchange object across every sweep, at every threshold |
| No order is placed or cancelled | The exchange object defines none of the five order methods, and the sweep calls none of them |
| Holdings and cost basis are untouched | Holdings 0.5 before and after; both cost-basis lots identical, 0.3 at $91.00 and 0.2 at $103.00 |
| A tranche with no timestamp is never despawned | At a one-day threshold the ageless record survived alone, counted as kept |
| A stack tranche holding a resting order is kept until it settles | Pending with an order id was kept at four hundred days; the same record removed once filled, and once cancelled |

The target balance and the anchor are unchanged too, which the line the operator reads already claims.

```
_current_holdings          0.5   ->  0.5
_target_balance          200.0   ->  200.0
_anchor_target_balance   200.0   ->  200.0
_hedge_bal               200.0   ->  200.0
exchange calls               0   ->  0
```

The threshold is inclusive, as the tooltip says. A record at exactly seven days goes at a seven-day threshold and a record one second younger stays.

```
exactly 7 days     removed 1, 0 rows left
one second under   removed 0, 1 row left
```

### Three declarations of one despawn predicate agree

The predicate is written three times: once in the sweep that removes, once in the shared preview the Qt panel reads, and once again inside the React surface. Driven on one tape of records across thirteen thresholds, all three answer the same counts on every row.

```
days    sweep   shared preview   React preview
   0     0/0             0/0            0/0
   1     2/2             2/2            2/2
   7     2/2             2/2            2/2
  30     1/2             1/2            1/2
 365     0/0             0/0            0/0
30.9     1/2             1/2            1/2
```

The dollars agree as well, and the queue total recomputes to match what is left rather than being decremented.

```
 7 days   queue $138.00 -> $68.00   the sweep reports $70.00 removed
30 days   queue $138.00 -> $98.00   the sweep reports $40.00 removed
```

A third copy of a predicate is a drift hazard rather than a present fault, and collapsing it would reach files other rows own.

### The word a removal now uses

Merge, despawn and clear are the only three things that collapse or remove a tranche, and despawn removes rather than delists. The sweep's own report and the line the operator read said delisted in five places, against a preview beside it that already said removed. The words now agree and the counts did not move.

```
before   TRANCHES DESPAWNED (>= 7d): delisted 2 fold tranche(s) holding $70.0000
after    TRANCHES DESPAWNED (>= 7d): removed 2 fold tranche(s) holding $70.0000
```

The sweep's key set is now a subset of the preview's, so either report can be read by one consumer. 08-tabs/bot-swarm.md carries the block.

```
sweep     ageless_kept  fold_removed  stack_kept_live_order  stack_removed
          threshold_days  usd_removed
preview   the same six, plus fold_open, stack_open and units_removed
```

## 2026-09-12 - two settings the engine read and no screen set

Two values sat on a bot's configuration with no row on any page. Neither was a preference the operator had ever been asked for, and both are gone. The behaviour each one bounded stays exactly where it was.

### The adoption cap comes from Target Balance

A bot that has never scrummed adopts the holding already sitting on the venue as its opening position, and a dollar ceiling bounds what it may take. That ceiling was read from a stored figure with a fallback to Target Balance. The stored figure never arrived: the restore path names its values one at a time and that name was not among them, so a record carrying 750 restored to a configuration holding zero, and the fallback was the only branch that ever ran.

`src/trading/scrumming/tick_phases.py` - the ceiling, in `_tick_initialise`

```
_cap_usd = float(self._target_balance or 0.0)
```

Target Balance is the figure the operator types. The creation wizard carries the row, and so does the live Bot Settings page, so the ceiling is now set from two screens instead of from nowhere.

```
src/gui/bot_wizard.py:826                 Target Balance, on the wizard
src/gui/live_settings/settings_tab.py:407 Target Balance, on a running bot
```

### The log line that named a field nothing writes

When the ceiling holds a bot back, the Activity Log says so and tells the operator what to raise. It named an internal field that no page offered, so the instruction could not be followed. It names the control instead.

```
before   ... units stay unmanaged. Raise max_adoptable_usd to change this.
after    ... units stay unmanaged. Raise this bot's Target Balance to change this.
```

### The reservation has no off switch

A bot claims the funds it is allowed to work with, which is what stops two bots on one asset from taking each other's money. A stored flag could turn that claim off, and nothing on any screen set it. The flag is gone and the claim is unconditional.

`src/trading/scrumming/capital_reservation_mixin.py` - the claim, once per tick

```
async def _ensure_capital_reservation(self, current_price: float) -> None:
    if current_price is None or current_price <= 0:
        return
```

Driven on two records that differed in that one flag, with the flag present and then removed. A record carrying the flag off placed no claim. The same record places one now.

```
flag off, before   reservation token None
flag off, after    reservation token 8c4f5907ff4d4634b8db1ddcd8ab9ba8
flag on,  before   reservation token 4fd7f1ce796540008fb4ebf0c0c6a70a
flag on,  after    reservation token 762010f49abf452f8371841243828f31
```

### Where the claim holds, and where it gives way

The claim holds. Two bots were restored from a record written for this reading, both on one asset, with thirty units at the venue and a price of one hundred dollars. The second bot's permitted sale fell to nineteen units, because the first bot had eleven of the thirty spoken for.

```
bot A target $1,000   claimed 11.0 units
bot B target   $500   claimed  5.5 units
registry total        16.5 units on the asset

A may sell            24.5 units
B may sell            19.0 units, where with no claim standing it is 30.0
```

Driving the real sell on the second bot both ways: a twenty-unit sell was refused and named the reservation, and an eighteen-unit sell was not refused by it.

```
sell 20.0   "sell refused by capital reservation:
             amount=20.000000 effective=19.000000 asset=A15"
sell 18.0   no reservation refusal
```

One reader of another bot's claim decides a sale. Nothing else in the platform reads a claim at all.

`src/trading/scrumming/execution.py` — the sell pre-check

```
_crr_effective = _crr_reg.effective_available(
    asset=self.config.target_asset,
    bot_id=self.bot_id,
    total_holdings=float(self._current_holdings or 0),
)
if amount > _crr_effective + 1e-12:
```

No screen reads a claim. The dollar-denominated registry that sits beside the asset-unit one is built nowhere in the tree, and the method that would attach it to the bot manager has no caller, so every bot is admitted with no dollar claim written for it.

```
effective_available    1 reader that decides a sale
reservations_for        1 reader, a bot's own headroom
snapshot               0 callers
set_capital_registry   0 callers
CapitalRegistry built  0 times
```

### What the counts above point at is removed

Three of those five rows named code with no caller, and that code is gone.

The dollar registry is gone. `src/trading/capital_registry.py` is removed, with the Qt table, the view model and the renderer page that drew it.

That path is in neither the git index nor the working tree, so a reader cannot open it. The commit `d72f69cf` removed it. What holds the reservation behaviour now is the module named in the next paragraph, and the dated entry on this page for issue 881 reads the same absence back.

The whole-table read is gone. `CapitalReservationRegistry` in `src/trading/capital_reservation.py` no longer declares `snapshot`. A bot reads another bot's claim through `reservations_for`, and a sale is decided by `effective_available`. Both stay.

The two operator overrides are gone. That registry no longer declares `force_release` or `force_release_all`. A claim still leaves the table five ways: its owner calls `release`, a bot drops its own with `release_for`, the expiry sweep calls `prune_expired`, the fleet sweep calls `sweep_unknown_bots`, and a silent bot loses its claim on the heartbeat schedule.

| Name | Call sites before | Call sites after | State |
|---|---|---|---|
| `CapitalRegistry` | 0 | 0 | removed |
| `snapshot` on the claim registry | 0 | 0 | removed |
| `force_release` | 0 | 0 | removed |
| `force_release_all` | 0 | 0 | removed |
| `effective_available` | 1 | 1 | kept |
| `reservations_for` | 1 | 1 | kept |

Five places let a bot through with no claim behind it. Each one is a choice to keep trading rather than to stop, and making any of them stop means refusing a bot that trades today.

| where | what fails there | what happens |
|---|---|---|
| `src/trading/container/registry.py`, the admission branch | no registry is attached | the bot is admitted, nothing is claimed |
| the same file, the rate branch | no price for the base currency | the bot is admitted, nothing is claimed |
| the same file, the error branch | the consult raises | the bot is admitted, nothing is claimed |
| `src/trading/extractor_bot.py`, the chunk-rate claim | the claim raises | the Extractor runs unclaimed |
| the Scrumming claim in the reservation mixin | the claim raises | the bot ticks on, its token cleared |

Seven further places on the same path behave the same way. The sell pre-check is the one that bears on money directly, because it is the only reader.

### The Extractor now reads its holdings before it claims

The fourth row above described a claim placed with no holdings figure behind it. The Extractor reads the balance first now.

`ExtractorBot._read_base_holdings` in `src/trading/extractor_bot.py` asks the venue for the free balance of the base currency. It answers a figure, or it answers nothing when the read fails.

Nothing is not zero, and it is not room to claim. `set_initial_chunk_rate` takes that answer. It still rebases the chunk. It places no claim, and it writes one warning that names the bot and the asset.

A figure goes to the registry with the claim, so the registry can compare the request against what the bot owns. This is the check the Scrumming bot already passes, and the Extractor now takes the same path.

Driven three ways on a real Extractor, with the venue read stubbed and no network:

| The balance read | The claim | What the log says |
|---|---|---|
| the read failed | none placed | the bot and the asset are named |
| 1000 units free, chunk 100 | placed, 100 units | the token and the chunk are named |

| The balance read | The claim | What the log says |
|---|---|---|
| 1 unit free, chunk 100 | refused, none placed | the request and the holdings are named |

The second row is the control. It proves the run would have seen a claim if one had been placed.

No bot on the saved fleet is an Extractor. All 38 records carry the Scrumming mode, and all 38 live claims carry the Scrumming kind, so this change moves nothing that trades today.

### The log lines a missing claim writes

Four places used to pass in silence, or to say so only at debug level. Each now writes one line naming what was lost, and none of them changes what a bot does.

```
the sell pre-check      warning, naming the sale that is not bounded
the admission branch    warning, naming the allocation not held aside
the Extractor claim     warning, when the base currency is empty
the dollar grant        warning, when the wallet cannot be priced
```

### What a stored record does now

A record written before the removal still loads. Two bots were restored from a record carrying both retired names, nothing was raised, and every field that remains came back holding what it held before.

```
declared fields      67 before, 65 after
field readings       134 before, 130 after, 0 values different
the four that moved  the two retired names, on each of the two bots
```

## 2026-09-13 - #665 - the restore path reads the same declaration

A launch used to rebuild every bot from a second list of field names. Creation had already stopped doing that: one helper reads the declaration and keeps every key naming a field the config declares. The restore path kept its own list — fifteen shared names, thirty-four Scrumming names and fifteen Extractor names, each carrying a default typed beside it. Both paths now call the one helper.

`src/trading/container/restore.py` — the whole selection

```
_kwargs = bot_config_kwargs(mode, cfg, exchange_id=cfg["exchange_id"])
if "stack_mode" not in cfg:
    _kwargs["stack_mode"] = STACK_MODE_DEFAULT
```

### Every place the field set is declared

The three frozensets partition the dataclass exactly: their union holds 65 names, no field sits outside them, and no name in them is absent from the dataclass. The restore file now adds no fourth list.

```
BotConfig, 65 fields                          config.py:49
_BOT_CONFIG_SHARED_FIELDS, 16                 config.py:267
_BOT_CONFIG_SCRUMMING_ONLY_FIELDS, 34         config.py:290
_BOT_CONFIG_EXTRACTOR_ONLY_FIELDS, 15         config.py:337
bot_config_kwargs reads all four              config.py:629
```

### A field added to the declaration reaches both paths

One run added a field to the declaration, typed 55.50 into it, and read that figure back off a bot built each way. The field count moved from 65 to 66, and a restored bot held the declared default until the restore path read the declaration.

```
before   creation 55.50      restore 0.00      1 of 2 paths carried it
after    creation 55.50      restore 55.50     2 of 2 paths carried it
```

### What a stored record restores to

A record shaped like the saved fleet file carried a value away from the default in every field it can hold, for one Scrumming bot and one Extractor. A third record held a venue and a mode and nothing else. All three restored on both sides of the change, and every reading matched.

```
record config fields      65 of 65 away from the declared default
readings compared         213
readings different        0
the bare record           60 of 60 fields at the default they declare
```

The comparison can report a difference. One stored figure moved from 3.75 to 9.99, the record restored again through the same path, and the comparison named that one field.

```
scrum_fire_pct   before 3.75   after 9.99   1 of 213 different
```

### The one value that changes, and the record that carries it

An Extractor whose record holds a pool figure and no Target Balance used to come back at 200.00. Creation gives that bot its pool figure, and the restore path now agrees.

```
record      extractor_chunk_size_usd 777.00, no target_balance key
before      restored target_balance 200.00
after       restored target_balance 777.00
creation    777.00, both before and after
```

Two writers reach the saved fleet file, and neither can write a configuration section short of a field. One writes the whole dataclass through `asdict`, and the other loads the file, edits a wire route and writes the file back. A record the program wrote therefore carries Target Balance, so no saved bot reaches the branch above.

```
src/trading/bot_container.py:591     "config": asdict(self.config)
src/gui/bot_visualizer.py:1474       loads, edits one route, writes back
```

### Two defaults the restore path keeps as its own

Stack Mode has two defaults, and they answer two different questions. Creation reads the retired Grid checkbox and treats its absence as off. A saved record never holds that key, so an absent Stack Mode on a restore takes the declared default instead.

```
creation   bulk_trading absent   stack_mode False
restore    stack_mode absent     stack_mode True, STACK_MODE_DEFAULT
```

A stored null on the standing alt units, or on the pool picker's list, reaches a number and an empty list rather than a null. Both readings are unchanged by this entry and both are driven.

```
if "inverted_extractor_standing_alt_units" in _kwargs:
    # A stored null reaches float() as 0.0.
    _kwargs["inverted_extractor_standing_alt_units"] = float(
        _kwargs["inverted_extractor_standing_alt_units"] or 0.0
    )
```

### One bad stored value used to stop the whole fleet

The build of a bot's configuration now sits inside the handler that was already written for it. A record holding a boolean where the pool picker's list belongs, or a word where a quantity belongs, used to raise out of the loop: the bots already processed stayed, the rest never loaded, and no line named the record at fault. Each such record is now one skipped bot, named in the restore ledger, and the fleet around it comes back.

```
three bots, the middle record holding a boolean pool list
before   the restore raised, 1 bot in the fleet, no ledger entry
after    2 of 3 restored, ledger names the middle bot
the same reading for a word where the standing quantity belongs
```

### What the earlier entries on this page now read

Two sentences above describe the restore path as it stood. The corrected reading for each:

| entry | the sentence above | what it now does |
|---|---|---|
| One declaration, at creation | "One declaration serves the wizard path and the restore path together, so every field the config declares reaches a new bot." | True of both paths now. The restore path reads the same helper, so a field added to the declaration reaches a restored bot with no second edit. |
| Max adoptable USD | "`max_adoptable_usd` is the one `BotConfig` field no page offers and the restore path does not read either." | The field is gone. Row 135 removed it and its reader, and the accumulation ceiling reads Target Balance. No field on the dataclass is unread by the restore path. |

### A stored phantom timeframe, read again

The restore path passes the stored timeframe to the bot, and the bot keeps it where the venue offers that granularity. A stored 4h on Coinbase reaches an empty list because the venue has no 4h candle, and the bot writes its own note naming the entry it dropped.

```
stored '1d'   bot holds ['1d']
stored '6h'   bot holds ['6h']
stored '4h'   bot holds []      available_timeframes('coinbase') has no 4h
absent        bot holds ['5m', '15m', '30m', '1h', '1d']
```

## 2026-09-13 - #665 - the bot's increment style is off the dataclass

The bot field named for the retired mode's increment is gone. Nothing wrote it and nothing read it, on either build.

```
occurrences in code    2 before, 0 after
writers                0 before, 0 after
readers                0 before, 0 after
```

### The code block above no longer matches the restore path

The block earlier on this page showing the scrumming fields a restart rebuilds lists a line for the removed name. The restore path stopped naming its fields one at a time when it moved to the shared declaration, and the dataclass no longer declares that name, so no such line exists in either place.

```
restore arguments   built from the fields the dataclass declares
retired names       dropped at that filter, never passed on
fields restored     65 before, 64 after, one different and it is the removed one
```

## 2026-09-13 - #665 - the Hedge Rebalance switch arms the reserve on a running bot

Three rows on the running bot's Hedge Rebalance group: the switch, the ceiling and the reserve. The saved fleet was read first, and read only.

```
38 bots, all running
Hedge Rebalance Active ON      6      OFF      32
Hedge Balance $0.00           36      $200.00   2
stored reserve $0.00          35      $1.40 and $185.19 and $200.00, one each
```

### The switch reaches the reserve

The reserve is the figure a hedge buy is checked against. Ticking Hedge Rebalance Active now fills it; before this change the tick moved the flag alone and the box appeared to work.

Diagram: flowchart, left to right

- Hedge Rebalance Active -> **Apply**
- Apply -> **set_hedge_rebalance_active_live**
- set_hedge_rebalance_active_live -> **the reserve a hedge buy reads**
- Hedge Balance -> **Apply**

- Apply -> **set_hedge_balance_live**
- set_hedge_balance_live -> **the ceiling a refill stops at**

Driven on a bot built with the box off at a Hedge Balance of $200.00.

```
the flag moved by hand       reserve   $0.00   a $25 hedge buy refused
the box ticked through Apply reserve $200.00   a $25 hedge buy allowed
the box unticked             reserve $200.00 kept, every hedge buy refused
the box ticked a second time reserve $200.00, no second filling
a word instead of a tick     refused, reserve $0.00, the switch unmoved
```

Ticking on raises the reserve to the Hedge Balance figure and never lowers a larger reserve, so the box cannot destroy a reserve the bot already holds. Unticking keeps the reserve and freezes it, because the arm test and the refill test both read the switch first.

### The ceiling is stored once

The ceiling is the Hedge Balance figure on the bot's configuration, and the bot reads it there rather than keeping a second copy.

`src/trading/scrumming/tick_phases.py` — the one ceiling

```
@property
def _hedge_balance_initial(self) -> float:
    if not self.config.hedge_rebalance_active:
        return 0.0
    return float(self.config.hedge_balance)
```

The reserve stays on the saved record, because it is a balance and not a setting. The configuration is the authority for the ceiling, so a restart cannot leave the two disagreeing. Every stored bot reads the same two figures as before.

```
switch on   Hedge Balance   $0.00   stored reserve   $0.00   ceiling   $0.00   reserve   $0.00
switch off  Hedge Balance   $0.00   stored reserve $200.00   ceiling   $0.00   reserve $200.00
switch on   Hedge Balance $200.00   stored reserve $185.19   ceiling $200.00   reserve $185.19
```

### A Hedge Balance of zero says what it is

Zero is a figure the box accepts, and it is not an off switch. It is an empty reserve that never refills, and a reserve the bot already holds stays spendable until it drains. Both tooltips say so, on both builds, and the Activity Log says so as the figure is typed.

```
HEDGE BALANCE A11 LIVE UPDATE: $200.00 -> $0.00. A $0.00 Hedge Balance is an
empty reserve that never refills, not an off switch. Untick Hedge Rebalance
Active to turn the hedge off. Reserve $200.00 stays spendable until it drains.
```

Making zero mean off was not taken. Four of the thirty-eight running bots carry the switch on with a Hedge Balance of $0.00, so that meaning would change what those four do on their next fold.

## 2026-09-13 - which build draws the Live tab

The Live tab now picks its page from the variant seam. `src/gui/variant_surface.py` registers the screen `TRADING` and holds two loaders for it. `_qt_trading` returns the Qt page. `_react_trading` returns `TradingTabReact`.

`src/gui/main_tabs/trading_tab.py` builds the Qt page on every start. `draws_react` then decides which page the tab shows. Under the Qt build the tab shows that page. Under the React build `_react_trading_page` makes the Qt page a hidden child of the React page and shows the React page.

The Qt page keeps every widget the main window writes to. `MainWindow` writes to `_status_log` and `_indicator_panel` under both builds, so neither build may skip the Qt build step.

`src/gui/react_trading_tab.py` — the page the React build shows

```
def models(live: Any = None) -> dict:
    tab = (
        trading_tab_surface.bind_live(live)
        if live is not None
        else trading_tab_surface.view_model
    )
    return {
        trading_tab_surface.METHOD: tab({}),
        status_log_surface.METHOD: status_log_surface.view_model({}),
        indicator_panel_surface.METHOD: indicator_panel_surface.view_model({}),
    }
```

`TradingTabReact` reads the same bridge method the Qt tab reads. That method is `trading.tab`, and `trading_tab_surface.bind_live` serves it from the running program's own exchange list.

The page carries five scripts and fetches nothing. `panel_host.js` comes first, then React, then `status_log.js`, `indicator_panel.js` and `trading_tab.js`. Each module is inlined, so no module can read its own file name off its script tag. `marker_script` names the module whose script tag comes next, and `namer_script` hands that name to the panel host.

### The voting panel takes no entry of its own

`trading_tab.js` mounts the voting panel itself. It keeps a slot for the panel and hands the slot to the panel host. `variant_surface.py` therefore registers no screen for the voting panel, and none is needed.

The reading below comes from the running page under the React build.

```
registered panels   indicator_panel, status_log, trading_tab
panel faults        none
trading.tab         loaded
```

The same reading with `indicator_panel.js` taken off the page names the panel that did not draw, which is how the reading above is known to discriminate.

```
registered panels   status_log, trading_tab
panel faults        indicator_panel: the manifest names no indicator_panel.js
```

### What the two builds draw

Both pages were drawn at 1743 by 1088 pixels and compared. 91.67 percent of the sampled pixels differ. The Qt page paints its ground `#2d2d2d`. The React page paints its ground `#0a0a0f`.

## 2026-09-13 - what the React Live page draws

`src/gui/web/trading_tab.css` gives the React Live tab its chrome. The page carried no stylesheet of its own before. It painted flat text on a plain ground.

`react_trading_tab.STYLE_ASSETS` names the file. `panel_html` inlines it into the page head, so the page still fetches nothing over the network.

The page also carries four style-source modules. `react_trading_tab.roster` puts them ahead of the three panel modules. They are design_tokens.js, theme_engine.js, shared_widgets.js and header_strip.js.

`trading_tab.js` parses every Qt style sheet in its payload with `styleOf`. That function reads header_strip.js. A page that leaves the four modules out gets an empty object back, and it paints no colour the payload asks for.

The reading below comes from the running page under the React build.

```
registered panels   header_strip, indicator_panel, status_log, trading_tab
panel faults        none
stylesheet rules    23
network requests    0
```

The same reading with the stylesheet emptied counts one stylesheet rule. That is how the reading above is known to discriminate.

```
registered panels   header_strip, indicator_panel, status_log, trading_tab
panel faults        none
stylesheet rules    1
network requests    0
```

### The page chrome

`trading_tab.css` sets colour, border and type. `trading_tab.js` writes every layout value on the element's own style attribute from the payload.

One exchange layer shows at a time. A stack page carries a flex display on its own style attribute, and that outranks the browser rule for a hidden element. The stylesheet marks a hidden page `display:none`. Before that rule the Crypto layer and the Stock layer both drew, one above the other, each at half the height.

The empty-state card takes its ground and its edge from the stylesheet. Qt counts the alpha of both in bytes. `keptSheet` drops a byte alpha from the payload, so only the card's corner radius survives the trip.

Two buttons add an exchange, and the Qt page draws both. The corner button beside the tabs is plain chrome. The card button below it carries the layer accent, which `placeholder_add_style` publishes. The stylesheet paints the corner button only, so the two read as different controls.

Each log pane is one bordered ground. The pane carries the border. The status log and the API log carry the read-only ground inside it. The lower half of the page was blank before those rules.

Each splitter handle carries the border colour, so the pointer can see what it takes hold of.

### The Indicator Voting Panel

The panel draws one row per timeframe and one column per indicator. Each cell prints its vote direction as an arrow and its confidence as a percentage. Green is bullish and red is bearish.

`indicator_panel_surface.indicator_cell_colors` publishes each cell's text colour and its tint. `indicator_panel.js` writes both on the cell. No rule in the stylesheet sets a colour on a cell.

The stylesheet gives the head cells a ground of their own. It also clips a cell that is wider than its column, so a narrow panel shows an ellipsis instead of running one vote over the column beside it.

Each mini-table holds its header plus two timeframe rows. The confidence bars below it take the rest of the height. A summary carrying more timeframes is cut at two rows. `indicator_panel.py` fixes the same height with `setFixedHeight`.

### The coin badge and the header skin

Each Symbol cell carries a coloured disc with the first letter of the name. The colour is derived from the characters of the name, so one asset always draws the same disc. `coin_disc_color` answers for the Qt widget and for the renderer module, and both bot tables read it.

```
ICON_ASSET_SIZE_PX = ds.COIN_ICON_SIZE_PX   # 18 pixels, one figure, four readers
```

The venue stylesheet paints the column headers in the accent colour, bold, over a two-pixel accent underline. It paints a rule on each cell edge, and it gives the Fire and Detail buttons the card ground and a border, so the browser draws no button face of its own.

```
header text        the accent colour
header underline   2px, the accent colour
cell edges         one rule right and one rule below
Fire and Detail    card ground, outline border, payload text colour
coin badge         18px disc, first letter, maximum-contrast text
```

## 2026-09-13 - #23 - the venue page under the React build

### Which build draws the venue page

The React build draws the venue page inside the Live page. It does not draw it in a window of its own. `TradingTabReact` loads eleven renderer modules into one page. The last of them is `exchange_tab.js`, and that module draws the venue.

The Qt build is different. It builds one `ExchangeTab` widget per venue and puts that widget in a tab strip. That path is unchanged.

`src/gui/react_trading_tab.py` - the modules the React Live page carries

```
CHILD_MODULES: tuple[str, ...] = (
    "status_log.js",
    "indicator_panel.js",
    "table_cells.js",
    "bot_status_table.js",
    "extractor_bot_table.js",
    "crypto_news_ticker.js",
    "exchange_tab.js",
)
```

### The pane that holds the venue

`trading_tab.js` draws one empty box per layer and marks it `data-part="exchange-pane"`. `mountExchanges` finds that box and calls `renderExchangePane`. The venue module then draws the whole venue into it.

The pane takes the height of the tab body. The two bot tables scroll inside it. The command bar keeps its own height and stays in view. This is what the Qt build does.

`src/gui/web/trading_tab.css` - the boxes that take the pane's height

```
[data-part="stack"],
[data-part="page"],
[data-part="tab-widget"],
[data-part="tab-body"],
[data-part="exchange-pane"] {
  min-height: 0;
}
```

### How one fleet load reaches the page

The window builds one `ExchangeTabReact` per venue. It hands every fleet load to that object. The object rebuilds five payloads and emits `published`.

`TradingTabReact.hold_venue` takes that object and connects the signal. Each emission rebuilds the venue payload and pushes it into the page. The page then seats each payload and redraws. One fleet load, one source, one draw.

`src/gui/react_trading_tab.py` - the Live page follows one venue

```
def hold_venue(self, venue: Any) -> bool:
    """Draw ``venue`` in this tab and follow every payload it publishes."""
    exchange_id = str(getattr(venue, "exchange_id", "") or "")
    published = getattr(venue, "published", None)
    if not exchange_id or published is None:
        return False
    self._venues[exchange_id] = venue
    published.connect(self._venue_published)
    self._venue_published()
    return True
```

A venue module names its exchange in its request. The page answers from the bag it holds for that exchange. Two venues on two layers therefore read two different fleets from one page.

### The three collated columns

The Net, Comp and Conf cells hold more characters than the other cells. The Conf cell holds a ten-block bar and a percentage. The page wraps these three cells instead of cutting them. The bar takes the first line and the percentage takes the second. Nothing runs past the right edge of the panel.

The other cells keep the clip rule above. Only the three collated cells wrap.

`src/gui/web/trading_tab.css` - the cells that wrap

```
[data-part="indicator-body-cell"][data-state="net"],
[data-part="indicator-body-cell"][data-state="comp"],
[data-part="indicator-body-cell"][data-state="conf"] {
  white-space: normal;
  overflow-wrap: anywhere;
  text-overflow: clip;
  line-height: 1.1;
}
```

### The header strip ground

Three pages in the header strip carry no style sheet: the five stat cards, the spendable strip, and the tab bar. `page_html` writes six chrome colours on the root element and paints nothing. Each of the three pages now paints its own ground from those colours.

A stat card paints its face from `--btn-bg`. Every page paints its body from the theme ground. No page scrolls, so no scroll bar draws.

`src/gui/react_dashboard_stat_card.py` - one card's ground

```
PAGE_STYLE = (
    "*{margin:0;padding:0;box-sizing:border-box}"
    "html,body{height:100%;overflow:hidden}"
    "body{background:var(--bg);color:var(--text)}"
    f"#{CARD_ROOT_ID}{{height:100%}}"
    '[data-part="card"]{height:100%;background:var(--btn-bg)}'
)
```

## 2026-09-13 - #23 - the voting panel and the two bot tables on the React Live tab

### The window feeds the voting panel

The dashboard tick wrote to the Qt panel only. The React panel was built once from an empty model and nothing replaced it, so it read the placeholder text for ever however many bots were running.

`src/gui/main_window.py` — `_publish_votes`

```
            show = getattr(getattr(self, "_trading_tab", None), "show_votes", None)
            if not callable(show):
                return False
            from .react_trading_tab import votes_payload
```

The tick now calls that method with the fleet, the selected bot and the reading it has just given the Qt panel. Both panels show one bot and one set of cells.

`src/gui/react_trading_tab.py` — `votes_payload`

```
    surface.view_model({"action": "set_bots", "bots": list(bots or [])})
    if selected_bot_id:
        surface.view_model({"action": "select_bot", "bot_id": str(selected_bot_id)})
```

`indicator_panel_surface.view_model` is the same handler the Electron renderer asks, so the window and the shell draw one panel.

**Read off the running page, 38 bots.** The selector held one entry and no cell carried a value before. It holds thirty-eight entries now.

```
                  before              after
bot selector      1 entry             38 entries
bot shown         (select a bot)      A02/USD [c8e5c5db] (running)
cells with text   0 of 0              17 of 20
confidence bars   0                   12
```

### The bars arrive settled

The bar model starts every bar at zero and steps it toward its target. The Qt widget runs its own frame timer. The page draws the frame it is handed and runs no timer, so the window hands it a settled one.

`src/gui/react_trading_tab.py` — `settled_frames`

```
    return math.ceil(
        math.log(surface.BARS_SETTLE_DELTA) / math.log(1.0 - surface.BARS_LERP_FACTOR)
    )
```

The two published factors answer forty-nine frames. The panel pushes that many steps before it publishes.

### The log panes give the tables their room

The main splitter gave the two empty log panes the bottom third of the page. It now gives them under a quarter, and the tables take the rest.

`src/gui/main_tabs/trading_tab_surface.py` — `MAIN_SPLITTER`

```
MAIN_SPLITTER = {
    "orientation": "vertical",
    "handle_width_px": HANDLE_WIDTH_PX,
    "children_collapsible": False,
    "children": ["top_splitter", "bottom_splitter"],
    "sizes_px": [660, 190],
}
```

The Qt page took the same four sizes from four literals of its own. It reads them from the surface now, so one declaration sets both builds.

### One stretch per bot table

The Scrumming table holds the whole accumulation fleet and grows into whatever room is left. The Extractor table keeps the height of its own rows, so its rows are never cut.

`src/gui/main_tabs/exchange_tab_surface.py` — the two stretches

```
SCRUM_TABLE_STRETCH = 1
EXTRACTOR_TABLE_STRETCH = 0
```

The React page reads them onto the flex of each table space. The Qt page passes them to the layout, and the Extractor table answers the header plus every row when the layout asks how tall it wants to be.

**Read off the running page at 1960 pixels, 38 Scrumming bots and 3 Extractor bots.** A whole row is one whose top and bottom both sit inside the table's visible box.

```
                       react before  react after  qt before  qt after
Scrumming whole rows   5             7            2          3
Extractor whole rows   0             3            2          3
Extractor box height   21 px         116 px       62 px      90 px
```

The Scrumming table is a scrolling list of thirty-eight rows, so its bottom edge still ends inside the next row: one pixel of twenty-nine under React and four of thirty under Qt. Every other scrolling table in the application ends the same way.

### The readings can fail

The same readings were taken again with the feed cut and again with the page's style sheets emptied. Each one reported the loss.

```
feed cut          bot selector 1 entry, bot shown blank, cells 0 of 0, bars 0
sheets emptied    style sheets 0, Scrumming table box 0 px
```

## 2026-09-13 - #23 - every Live tab feed reaches the React page

### Every Activity Log call reaches the page

Six modules write to the Activity Log, and each one calls the Qt pane directly. The React pane was built once from an empty model, so it held its placeholder while the fleet traded. The pane now reports every call to one listener.

`src/gui/widgets/status_log.py` — `set_relay`

```
        def set_relay(self, relay) -> None:
            """Take the callable every ``log``, ``pause``, ``resume`` and
            ``notice`` call is reported to."""
            self._relay = relay
```

`TradingTabMixin._wire_live_feeds` hands that listener to the React tab when the tab is built. The listener drives `status_log_surface`, which paints the same line the Qt pane paints, so a pause, a resume and a watchdog line all arrive.

The Qt pane paints through `status_log_surface` as well. It asks `line_style` for the shape and `line_html` for the line, rather than building either itself, so the two panes cannot disagree about what a message looks like.

**Read off the running page, 38 bots restored.** The two panes hold the same seven lines in the same order.

```
                  before   after   qt
activity lines    0        7       7
```

### The API Interaction Log takes the same block

`MainWindow._on_api_event` builds one text block per API call. It handed that block to the Qt pane only. It now hands the block to the React tab first, so a paused pane holds it and a running pane paints it.

`src/gui/main_window.py` — `_on_api_event`

```
            block_text = "\n".join(plain_lines)
            self._push_live_tab({"api_lines": [block_text]})
```

`trading_tab_surface.view_model` keeps the capped block buffer the Qt view keeps, so both panes drop the same oldest block.

```
                  before   after   qt
api blocks        0        8       8
```

### The data-pool line reads the shared cache

The Qt venue page runs a one-second timer that reads the market cache and writes the freshness line under the New Bot button. The React venue page ran no such timer, so its line stayed on its opening text.

`src/gui/react_exchange_tab.py` — `_update_pull_rate_label`

```
        def _update_pull_rate_label(self) -> None:
            if self._stopped:
                return
            self._screen.update_pull_rate_label()
            self._publish()
```

The publish carries the fresh payload to the venue's own page and to the Live page beside it.

**Read with 38 bots holding pool slots.** Both builds print one string.

```
before   Next data pull: —
after    Data pool: 76 slots · awaiting first fetch  ·  cache-hit 0%
```

### The panel header carries its state and its rates

The voting panel prints two header lines. One says the reading on screen is the last one taken and how old it is. The other prints the A15 and A14 spot rates. Neither reached the React panel, because the payload the window pushed carried the cells alone.

`src/gui/indicator_panel.py` — `panel_reading`

```
            if self._showing_stored and self._shown_stored:
                read.update(self._shown_stored)
            if self._rates_seen:
                read["rates"] = self._rate_snapshot
```

`react_trading_tab.votes_payload` turns that reading into the panel payload. The Qt panel formats the age once and hands the result over, so the two banners cannot drift apart.

```
before   A15 —   A14 —   (currency rates pending), no state line
after    A15 —   A14 —
after    ■ LAST TA READ, NOT CURRENT — taken 22:03:09, 30s ago. bot is idle
```

### The voting panel takes the pane height

The panel's two graphs divide whatever height is left after the two tables. The host the tab kept for the panel was a plain block, so the panel measured itself against its own content and each graph fell back to its hundred-pixel floor.

`src/gui/web/trading_tab.js` — the panel host

```
    var panelProps = {
      style: {
        flex: AUTO,
        overflow: AUTO,
        display: FLEX,
        flexDirection: COLUMN,
        minHeight: ZERO
      }
    };
```

Each table also keeps the height Qt fixes it to, which is its header plus two rows of slack, so a table holding one row is the same height as one holding three.

**Read off the running page at 1960 pixels.**

```
                    before   after   qt
graph height        100 px   147 px  142 px
table height        52 px    84 px   83 px
```

### Cutting each feed empties what it fed

Each feed was cut in turn and the same readings were taken again. Every reading reported its own loss and no other.

```
feed cut     reading
activity     activity lines 0, api blocks 8
api          api blocks 0, activity lines 7
panel        bot blank, rates pending, no state line
data pool    Next data pull: —
```

## 2026-09-14 - #23 - the voting panel matches the Qt panel element by element

The Net, Comp and Conf columns drew as two stacked bars with a black band between them. The band was the second table's header row. The page gave every header cell a solid ground, so that row painted over the pillar behind it.

`src/gui/web/trading_tab.css` — the header cell lets the pillar through

```
[data-part="indicator-head-cell"] {
  background: transparent;
  border: none;
}
```

The window draws the same header through `QHeaderView::section`, which is transparent for the same reason. The pillar now runs from the upper graph to the lower one without a break.

**Overtaken.** *"The pillar now runs from the upper graph to the lower one without a break."* A pillar now stands on the floor of that same space at its own fraction of it, so it reaches the upper graph only at a full reading. The space it may occupy, and the header it draws through without a band, are unchanged. See Net, Comp and

Conf.

### What the page now reads off the payload

Eleven published values reached nothing. The page took a header ground the payload never gave it, and it guessed the rest of the list below.

```
value                what it sets on the page now
arrow_min_height_px  the bar height an arrow needs
shine_min_height_px  the bar height a highlight needs
shine_limit_px       the tallest a highlight may grow
glow_inset_px        the halo width beside a bar and beside a pillar
column_min_pad_px    the narrowest pad beside a bar
label_font           the face a bar name and a pillar name take
arrow_font           the face and the size a bar arrow takes
empty_font           the face the empty-graph note takes
label_height_px      the box a bar name centres in
label_offset_px      how far that box sits under the baseline
margin_left_px       where the grid and the baseline start
```

### How an ornament decides now

`ConfidenceBarsWidget.paintEvent` tests the bar it painted, in pixels. The page measured nothing, so it compared the vote against a share taken at the smallest graph the panel allows.

A taller graph then dropped arrows the window kept.

`renderPanel` in `src/gui/web/indicator_panel.js` measures the graph after it draws, draws again at the height it found, and watches for a resize.

```
                         window   page before   page after
bar area height          129 px   126 px        129 px
arrows drawn, 12 bars    8        6             8
```

### Two measurements that still differ

The header strip is 23 pixels tall in the window and 18 on the page. Each engine sizes that strip from its own font, and no published value names a height, so a fixed height would write this machine's font metric into the product. Each table is 83 pixels in the window and 78 on the page for the same reason.

The bot selector shows the whole bot name on the page. The window clips the last characters of it, because `QComboBox` fixed its width at its first show, before any bot had arrived.

```
                     window   page
header strip         23 px    18 px
mini-panel table     83 px    78 px
graph pane           159 px   159 px
bar area             129 px   129 px
pillar               374 px   369 px
```

## 2026-09-14 - #23 - the window draws the panel without a band

The window painted a band across the Net, Comp and Conf pillars. The band was the second table's header row. The theme gives every header section a ground and an underline, and the mini-panel container gives itself a ground as well. Both painted over the pillar behind them.

`src/gui/indicator_panel.py` — the container and the table draw no ground

```
container.setStyleSheet("QWidget { background: transparent; }")
table.setStyleSheet(
    "QTableWidget { background: transparent; border: none; } "
    "QTableView { background: transparent; border: none; } "
    "QHeaderView { background: transparent; border: none; } "
    "QHeaderView::section { background: transparent; border: none; }"
)
```

The three pillars now run from the upper graph to the lower one without a break. Read off the rendered panel at the centre of the Net pillar, one colour runs from row 160 to row 599 with no other colour inside it.

**Overtaken.** *"The three pillars now run from the upper graph to the lower one without a break."* Each pillar now stands at its own fraction of that space. One colour still runs the whole of a pillar with no other colour inside it, which is what this passage measured; the run is as tall as the reading. The `pillar 374 px / 369 px` row above measured a full-height pillar and now holds for a reading at full scale only. See Net, Comp and Conf.

### The pillars are named at their tops

Net, Comp and Conf carried a name only at the foot. `column_titles` in `src/gui/main_tabs/indicator_panel_surface.py` now heads those three columns on the table that holds the aggregate cells, and pads the other table as before.

```
def column_titles(subset: list, *, include_aggregates: bool = False) -> list:
    titles = [TF_COLUMN_TITLE] + [short for _, short, _ in subset]
    pad = AGGREGATE_TITLES if include_aggregates else []
    titles = titles + list(pad)
    return titles + [EMPTY_TITLE] * (PANEL_COLUMN_COUNT - len(titles))
```

The window and the page both call that function, so neither can head a column the other leaves blank. Both now draw 17 column headings where each drew 14.

### The page takes the heading colour the window paints

The theme paints a header section in its own accent colour. The page inherited the page text colour instead, so the two drew the same headings in different colours. The page variable `--accent` carries that accent for all five themes.

```
[data-part="indicator-head-cell"] {
  background: transparent;
  border: none;
  color: var(--accent);
  padding: 8px;
}
```

Read off the two rendered panels, a column heading is rgb(0, 255, 204) on both.

### What the two panels measure

```
                      window     page
column headings       17         17
bars                  12         12
pillars               3          3
heading row height    33 px      34 px
bar area              156 px     159 px
panel ground          10,10,15   10,10,15
graph ground          10,10,18   10,10,18
heading colour        0,255,204  0,255,204
```

### One measurement that still differs

The bot selector is 34 pixels tall in the window. The whole header row is 23 pixels on the page. The panel publishes ten heights and none of them names the header row, so each engine sizes that control from its own font.

## 2026-09-23 - #154 - the voting panel's header row

The row at the top of the Indicator Voting Panel now holds, from the left: the panel's title, a blue left arrow, the bot dropdown, a blue right arrow and the privacy control. The word `Bot:` is gone. A stretch sits on each side of the four controls, so the dropdown sits between the two arrows and the group sits in the middle of the space the title leaves.

### The arrows

Each arrow moves the dropdown one place along the list it already holds. The list does not wrap: a press at the first bot leaves the panel on the first bot, and a press at the last leaves it on the last. An arrow with nothing left to step to is greyed.

`src/gui/indicator_panel.py` — `IndicatorVotingPanel._step_bot`

```
def _step_bot(self, by: int) -> int:
    """Move the dropdown ``by`` places and answer the index it lands on.

    The list does not wrap: a press at either end leaves the
    selection where it is.
    """
```

The two glyphs, their size and their family are the Charts tab's own, imported rather than copied, so the two control rows cannot drift apart.

`src/gui/main_tabs/indicator_panel_surface.py` — the arrows take the Charts tab's numbers

```
from .native_chart_surface import (
    ARROW_GLYPH_FAMILY,
    ARROW_GLYPH_PX,
    CONTROL_HEIGHT_PX,
)
```

Read off both running builds, each arrow is 34 by 26 pixels and draws U+25C0 or U+25B6 in Segoe UI Symbol at 16 pixels.

### The privacy control

The panel draws the universal control, `PrivacyDot` in `src/gui/widgets/privacy_dot.py`, in place of the copy it used to carry. The glyph is a filled circle when the dropdown is readable and an empty circle when it is masked. The colour is the design system's `PRIMARY_BRIGHT`, and the control sizes to its own glyph rather than to a fixed square.

`src/gui/design_system.py` — the token the dot paints in

```
PRIMARY_BRIGHT = "#00ffee"  # Brighter cyan accent: privacy dot, Sim tab
```

Read off the running window, the mark paints `#00ffee` at 17 by 19 pixels. On the page it paints `rgb(0, 255, 238)` at 16 by 17. The Charts tab's own dot takes its colour from the same token.

### The page answers a press

The React page used to draw the dropdown and the dot with no handler behind either, and its bridge served every ask from the payload the page was built with. An ask that names an action now goes to the window and moves the one Qt panel the window holds, so the page and the window never show two selections.

`src/gui/main_tabs/trading_tab.py` — `TradingTabMixin._wire_live_feeds` binds it

```
bind = getattr(getattr(self, "_trading_tab", None), "set_votes_handler", None)
answer = getattr(self, "_answer_votes", None)
if callable(bind) and callable(answer):
    bind(answer)
```

### What the two rows measure

```
                        window        page
row order               title, ■, dropdown, ■, dot
row height              30            30
arrow size              34 x 26       34 x 26
arrow glyph family      Segoe UI Symbol 16 px on both
dropdown entries        38            38
privacy mark            17 x 19       16 x 17
privacy colour          #00ffee       rgb(0, 255, 238)
Bot: label              absent        absent
```

Before this entry the same two rows measured a `Bot:` label, no arrow, and a privacy mark of 12 by 12 painting `#3344ff`, and the page's row measured 23 pixels tall.

### Four sentences this entry overtakes

"The Bot selector at the top names the bot whose votes the panel draws. The list refills every tick. The badge beside it counts the bullish, bearish and neutral voters." — the list still refills every tick, and it is still the control that names the bot. No `Bot:` word stands before it and no badge after it; an arrow stands on each side and the privacy control follows.

"TF Lock chooses a timeframe below which an opposing trade is refused. The list opens on 'None (no lock)'." — no such control is on the panel. The line at the foot that names an active timeframe lock is a readout and stays.

"The bot selector and its privacy dot moved there." — they are still in the upper part of the row, with an arrow on each side of the dropdown, and the privacy dot is now the universal control rather than the panel's own copy.

"The bot selector is 34 pixels tall in the window. The whole header row is 23 pixels on the page." — the arrows carry a control height the surface publishes, so each arrow measures 26 pixels tall in the window and 26 on the page. The row that holds them measures 30 pixels in the window and 30 on the page. It measured 23 on the page before the arrows joined it.

## 2026-09-23 - #858 - the bot list moves the Indicator Voting Panel

A press on a row of the Scrumming Bots table draws that bot in the Indicator Voting Panel. Before this entry the two picked their bots apart: the table's selection reached one handler, which cleared the other table and named no panel, and the panel read its own dropdown.

### One value behind both

The bot the panel holds is the value. `IndicatorVotingPanel.select_bot` is the only thing that writes it, and every other control asks that one method.

`src/gui/widgets/bot_selection.py` — `BotListPanelLink`

```
def row_selected(self, bot_id: Optional[str]) -> str:
    """Draw the pressed bot on the panel and answer the bot it holds."""
    if self._settling:
        return self.bot_id
    self._settling = True
    try:
        return str(self._panel.select_bot(str(bot_id or "")) or "")
    finally:
        self._settling = False
```

The window builds one link and gives the panel's own topic a reader. The topic was already declared and the emit already ran; nothing listened to it.

`src/gui/main_window.py` — `MainWindow._setup_bot_list_link`

```
self._bus.subscribe(
    _ivp_surface.BOT_SELECTED_TOPIC, self._bot_list_link.panel_selected
)
```

### What a second press on one row does

The decision is one function, and both build variants call it, so a press cannot mean two different things on the two screens.

`src/gui/widgets/bot_status_table.py` — `BotStatusTable.mousePressEvent`

```
shown = self.get_selected_bot_id()
super().mousePressEvent(event)
pressed = self.get_selected_bot_id()
if not selection_after_press(pressed, shown):
    self.clear_bot_selection()
```

`src/gui/main_tabs/bot_status_table_surface.py` — `BotStatusTableModel.on_row_pressed`

```
wanted = selection_after_press(str(bot_id or ""), self.get_selected_bot_id())
if wanted:
    self.select_row_for_bot(wanted)
else:
    self.clear_selection()
    self.current_row = NO_SELECTION_ROW
```

### How the press leaves the page

The page's row carries a press of its own, alongside the four it already sent. It goes out on the same console line the privacy toggle and the column sort use, and the Fire and Detail buttons stop the press reaching the row so they keep their window behaviour.

`src/gui/web/bot_status_table.js` — `sendRowPress`

```
function sendRowPress(model, botId) {
  return dispatch(
    model,
    actionNamed(model, ROW_PRESSED),
    request(model, SELECT_BOT_PARAM, botId)
  );
}
```

`src/gui/react_trading_tab.py` — the venue answers three presses now

```
VENUE_PRESSES = (
    (scrum_surface.PRIVACY_TOGGLE_PARAM, "toggle_privacy"),
    (scrum_surface.SORT_COLUMN_PARAM, "sort_by"),
    (scrum_surface.SELECT_BOT_PARAM, "select_bot"),
)
```

### What both builds now read

Read off the running program in both builds at 700, 900 and 1400 pixels, with a fleet of 38 bots, the home on a scratch directory and every socket but loopback refused. Every figure below is identical at all three widths and in both builds.

```
press                           bot list highlight     panel draws
none yet                        the panel's own bot    that bot
a row                           that row's bot         that row's bot
the same row again              no row                 the empty state
that row a third time           that row's bot         that row's bot
the panel's dropdown            that bot's row         that bot
a column sorted, then row 0     row 0's bot            row 0's bot
```

The sorted reading moved the first row from one bot to another before the press, so the press could not have passed by position.

### The empty state and the bot that is gone

Two empty states, read in both builds. A selection cleared by a second press reports the cause `no_selection` and reads *"no bot is selected — press a bot row, or pick one from the dropdown above."* A bot the fleet no longer carries reports `bot_missing` and reads *"bot … is selected but no longer present in the fleet."* A bot the dropdown does not carry leaves the panel where it is, so a stale ask cannot blank it.

### A reading the panel used to keep after it was over

`panel_reading` names the cause of an empty state, and nothing cleared that cause when votes arrived. A panel drawing twelve voters still reported the reason it had been blank. It clears when a summary arrives now, and a stored reading keeps the cause of the missing live read, which is what its banner is about.

`src/gui/indicator_panel.py` — `IndicatorVotingPanel.update_data`

```
if multi_tf_summary:
    self._no_data_cause = ""
    self._no_data_message = ""
```

### One sentence this entry overtakes

"The Bot selector at the top names the bot whose votes the panel draws." — the selector still names it and still refills every tick. It is no longer the only control that names it: a press on a row of the Scrumming Bots table names it too, and a second press on that row takes the panel off that bot.

### The asset class group, and the Add Exchange button that follows it

**Functional.** The header strip ends in a segmented group, at the upper right of the window. It holds one button per asset class, and exactly one is active. Pressing a button makes that class active, retitles the Add Exchange button and stores the choice, so the next launch opens on the same class.

The group builds its buttons from the taxonomy at run time, so a class added to `ASSET_CLASSES` gets a button with no further edit.

`src/gui/main_tabs/header_strip.py` — `_build_class_group`

```
for name in asset_classes():
    model = class_button(name, self._asset_class)
    button = QPushButton(model["text"])
    button.setCheckable(True)
    button.clicked.connect(partial(self._on_class_clicked, model["class"]))
    self._class_buttons.addButton(button)
```

Read off the running window, the group holds five buttons:

```
Crypto   Stock   Commodities   Derivatives   Forex
```

**One class at a time.** The buttons sit in a `QButtonGroup` set exclusive, so a second active button is refused before any code of ours runs. The active class is readable off the window as `_asset_class`.

**The Add Exchange button follows the class.** Its text, its tooltip and whether it can act all come from the class, and each is read from one place.

`src/gui/main_tabs/asset_class_surface.py` — `add_exchange_label`

```
state = class_state(name)
if not state["served"]:
    return EMPTY_LABEL.format(name=state["name"])
return f"{ADD_PREFIX} {state['name']} {state['noun']}"
```

Read for each class in turn, on the running window:

| class | the button reads | it can act | venues |
|---|---|---|---|
| Crypto | ■ `Add Crypto Exchange` | yes | 15 |
| Stock | ■ `Add Stock Broker` | yes | 9 |
| Commodities | `Commodities — no venue yet` | no | 0 |
| Derivatives | ■ `Add Derivatives Exchange` | yes | 1 |
| Forex | `Forex — no venue yet` | no | 0 |

**A class with no venue says so.** It does not offer another class's venue list. The button states the class and refuses the press, and the Trading tab draws a card naming what is missing.

`src/gui/main_tabs/asset_class_surface.py` — the notes a class draws

```
NO_VENUE_NOTE = "No configured venue serves {name} yet."
NO_LAYER_NOTE = "{name} has no trading layer yet."
```

Commodities and Forex have no configured venue. Derivatives has one venue and no trading layer, so it draws the second note.

**The Derivatives row above is overtaken, and so is the sentence naming its one venue.** Both stay exactly as written. The class they name retired on purpose, and the row records a taxonomy the program no longer declares.

The row, quoted whole as the page sets it:

```
the row: | Derivatives | `■ Add Derivatives Exchange` | yes | 1 |
```

> *Commodities and Forex have no configured venue. Derivatives has one venue and no trading layer, so it draws the second note.*

The true sentence is: the taxonomy declares four classes, and Derivatives is not one of them. Read at run time, the tuple answers crypto, stocks, commodities and forex, and the group draws one button for each.

```
# src/trading/ata_spm.py:55
ASSET_CLASSES = (
    CLASS_CRYPTO,
    CLASS_STOCKS,
    CLASS_COMMODITIES,
    CLASS_FOREX,
)
```

The one venue that row named is Coinbase. Asked which classes it serves, the surface answers crypto alone, because it answers only classes the live tuple holds. Coinbase's dated futures and perpetuals are still read, from the same public product list, and each product now takes the class of its own underlying instead of a class of its own.

```
# src/trading/ata_asset_maps.py:1082
def futures_placement(product: dict) -> tuple[str, str, str]:
    """The underlying, the asset class and the sector one futures product takes.
```

A stored selection under the old name still opens on a live class. One map answers the old name, and for this one it answers crypto.

```
# src/trading/ata_spm.py:77
RETIRED_CLASSES = {
    "metals": CLASS_COMMODITIES,
    "energy": CLASS_COMMODITIES,
    RETIRED_CLASS_DERIVATIVES: CLASS_CRYPTO,
}
```

The retirement was a decision and not a slip. Derivatives names no sector. A futures contract is a form a contract takes, and the class belongs to the thing underneath it, so a future on an index sits with equities and a future on ether sits with crypto. A market a retail trader cannot reach from home still charts under its own class, and no bot deploys there.

Read at run time, and searched over the git index across every tracked Python file:

```
classes the tuple declares                                  4
the tuple holds derivatives                             False
CONTROL the tuple holds crypto                           True
classes the surface says Coinbase serves                    1
buttons the group draws                                     4
RETIRED_CLASS_DERIVATIVES                       4 occurrences
CLASS_COMMODITIES                              29 occurrences
CLASS_FOREX                                     7 occurrences
CONTROL CLASS_ZORBONICS                         0 occurrences
```

The last row is the control. The same search finds three class names that are still read, so a zero above is a reading of the tree and not of the search.

The class list is read from `ASSET_CLASSES` at run time. Unit S13 of issue 23 replaced Metals and Energy with Commodities while this unit was open, and the group followed with no edit: it dropped to five buttons and drew the new class.

**A retired class name reaches the class that holds its markets.** `ata_spm` names the pair, and the group reads that name rather than carrying its own copy, so a selection stored under the old taxonomy still opens on the right class.

`src/gui/main_tabs/asset_class_surface.py` — `normalise`

```
asked = LEGACY_CLASS_WORDS.get(asked, asked)
asked = RETIRED_CLASSES.get(asked, asked)
```

Read for each name in turn, on the running window:

| stored name | the class it opens on | the button reads | its accent |
|---|---|---|---|
| `metals` | commodities | Commodities | `#ffaa00` |
| `energy` | commodities | Commodities | `#ffaa00` |
| `stock` | stocks | Stock | `#6699ff` |
| `equities` | stocks | Stock | `#6699ff` |
| a name no class holds | crypto | Crypto | `#00ccaa` |

Each of the five live classes draws its own label and its own accent. None falls back to the neutral default.

**A venue may serve more than one class.** The active class filters the venue list; it never owns it. Coinbase serves crypto and derivatives, so it is offered under both.

`src/gui/main_tabs/asset_class_surface.py` — `venue_classes`

```
found = set(EXTRA_VENUE_CLASSES.get(asked, ()))
if asked in EQUITY_VENUES:
    found.add("stocks")
elif asked in crypto_venues():
    found.add("crypto")
```

**The choice survives a restart.** `select_asset_class` writes the class into settings on every press, and `_build_class_group` reads it back at build.

`src/core/settings.py` — the stored field

```
active_asset_class: str = "crypto"
```

**The group at narrow widths.** The buttons share the group's width and shorten their own text to the room each has, so no label is cut mid-letter and no counter is pushed off the strip. The full class name stays in the tooltip.

The table below replaces an earlier one on this page. That one measured the group on its own. This one measures it in the assembled window, where the KPI columns and the five counters share the row with it.

> *"Measured on the running window, at five widths: | 700 | 255 px | 49 px |*
>
> *none in full |"*

The group draws 178 px at a 700 px window, not 255, because the KPI columns and the counters take the room they need first.

### The header row's width budget

**Every part of the row declares a floor.** A floor is what the part draws at when the row cannot give it more, and the sum of the floors is the narrowest the window opens at. Without them a part's minimum is the width of its own text, and a six-figure amount widens the whole window past the screen.

`src/gui/main_tabs/header_strip_surface.py` — the floors

```
SPENDABLE_MIN_W = 180
COUNTER_MIN_W = 48
```

The class group's floor is arithmetic over the class count, so a class added to the taxonomy widens the group and the window with it.

`src/gui/main_tabs/asset_class_surface.py` — `group_min_w`

```
held = len(asset_classes()) if count is None else int(count)
return held * BUTTON_MIN_W + (held - 1) * GROUP_SPACING_PX
```

Five classes at 34 px with four gaps at 2 px is **178 px**. The whole row is 180 + five counters at 48 + 178 + six gaps at 4 px = **622 px**, and the window is that plus the central layout's 6 px either side: **634 px**.

**What each part gets, read off the running window.** Both builds, every class button reachable at every width.

Qt:

| window | pane | KPI strip | Scrummed | Folded | Trades | Bots | Errors | group | each button | reachable |
|---|---|---|---|---|---|---|---|---|---|---|
| 700 | 688 | 246 | 48 | 48 | 48 | 48 | 48 | 178 | 34 | 5 of 5 |
| 900 | 888 | 446 | 48 | 48 | 48 | 48 | 48 | 178 | 34 | 5 of 5 |
| 1400 | 1388 | 709 | 98 | 88 | 78 | 58 | 78 | 255 | 49 | 5 of 5 |
| 1550 | 1538 | 709 | 98 | 88 | 78 | 58 | 78 | 405 | 79 | 5 of 5 |

| window | pane | KPI strip | Scrummed | Folded | Trades | Bots | Errors | group | each button | reachable |
|---|---|---|---|---|---|---|---|---|---|---|
| 1920 | 1908 | 709 | 98 | 88 | 78 | 58 | 78 | 775 | 153 | 5 of 5 |

React:

| window | pane | KPI strip | Scrummed | Folded | Trades | Bots | Errors | group | each button | reachable |
|---|---|---|---|---|---|---|---|---|---|---|
| 700 | 688 | 246 | 48 | 48 | 48 | 48 | 48 | 178 | 34 | 5 of 5 |
| 900 | 888 | 446 | 48 | 48 | 48 | 48 | 48 | 178 | 34 | 5 of 5 |
| 1400 | 1388 | 686 | 100 | 100 | 100 | 100 | 100 | 178 | 34 | 5 of 5 |
| 1550 | 1538 | 686 | 100 | 100 | 100 | 100 | 100 | 328 | 64 | 5 of 5 |
| 1920 | 1908 | 686 | 100 | 100 | 100 | 100 | 100 | 698 | 138 | 5 of 5 |

**The class group takes what the figures leave.** Every other part of the row draws at the width its own text asks for, and the group takes the rest down to its floor. A wide amount therefore shortens a class name and never a figure.

`src/gui/main_tabs/header_strip_surface.py` — the stretch each slot takes

```
TOP_ROW_STRETCH = [0, 0, 0, 0, 0, 0, 1]
```

**A shortened amount ends in an ellipsis.** Every caption and every amount in the KPI strip and the counters is an `ElidingLabel`. It keeps the whole text and draws what the width holds, so `$128,456.78` in a narrow row reads `$128,45…` and never `$128,45`.

`src/gui/widgets/eliding_label.py` — `sizeHint`

```
hint = super().sizeHint()
metrics = self.fontMetrics()
pad = hint.width() - metrics.horizontalAdvance(super().text())
return QSize(metrics.horizontalAdvance(self._full) + max(pad, 0), hint.height())
```

The whole text goes to `setAccessibleName`, so a shortened amount still reaches a screen reader whole.

**Below 634 px the row runs out.** Held to 500 px, in both builds, Derivatives and Forex fall outside the pane and the row's own hit test stops finding them. That is what the floor prevents: 634 px is the narrowest the window opens at, so the operator never reaches 500.

**One venue list, read from one place.** The equity venue ids were written out three times, and the three copies were held together by a comment. They are now one name that all three sites import.

`src/gui/main_tabs/asset_class_surface.py` — the one declaration

```
EQUITY_VENUES = frozenset(
    {
        "alpaca",
        "ibkr",
        "schwab",
        "tdameritrade",
        "webull",
        "tastytrade",
        "fidelity",
        "etrade",
        "interactivebrokers",
    }
)
```

### Where the equity venue list lives

The page carries the sentence above about that list. It is quoted whole here, marked overtaken, with the sentence that is true today beneath it.

> *"The equity venue ids were written out three times, and the three copies were held together by a comment. They are now one name that all three sites import."*

**Overtaken.** Three copies were collapsed onto the one name in that change, and three more were still in the tree: the Paper tab, the Live tab's view model and the Qt Simulator tab. All three now read the one name as well. Six files read it today and one file declares it.

`src/gui/main_tabs/trading_tab_surface.py` — the order the pages are given

```
EQUITY_EXCHANGE_IDS = tuple(sorted(EQUITY_VENUES))
```

`src/gui/simulator/sim_trading_tab.py` — the Simulator tab's own read

```
self._equity_exchange_ids = acs.EQUITY_VENUES
```

**Order matters to the pages and membership matters to the layers.** Three view models publish the list to a page, so it is sorted where the Live view model reads it and every launch gives the pages the same nine in the same order. Nothing else reads the order. Which layer a venue is put on is decided by membership alone.

| what reads the list | what it decides |
|---|---|
| the Live view model | the layer each venue is published under |
| the Paper and Simulator view models | the same, on their own pages |
| the Qt Live tab | the layer a venue tab is seated on |
| the Qt Simulator tab | the layer a venue page is seated on |
| the Settings dialog | whether a venue's row is an equity row |

**Two of the nine trade nowhere today.** One closed its interface on 10 May 2024 and one publishes none, both recorded on 15-venue-compatibility.md. Both stay on the list. This entry records where the list lives, not what is

on it.

### Sentences the asset class group overtakes

The page carries these sentences about the control that came before the group. Each is quoted as it stands, with the sentence that is true today beneath it.

> *"Press the mode button once and the title is rewritten to name the wing."*

The group has one button per class, so a press names a class rather than stepping to the next one. The title is rewritten to name the class: `Acervator — CRYPTO LAYER`.

> *"Pressing the mode button swaps the whole wing, tables and Paper Trader together."*

A press makes one asset class active. Crypto and Stock each swap to their own trading layer. A class with no layer draws the card that names what is missing. Showing only that class's bots, charts and history is unit F2 and is not built.

> *"Crypto Mode at the right swaps the window between the two."* — `08-tabs.md`

The right of the strip now holds one button per asset class, and a press selects that class.

> *"The mode button on the right ends the row."* — `08-tabs/portfolio-panels.md`

The segmented asset class group ends the row.

### The naming the asset class group uses

The operator's words are "market sector". Under the published standards the top level is the **asset class**, and a sector is the tier below it. GICS names sectors inside equities. S&P GSCI names sectors inside commodities. The group therefore selects an asset class. Sectors stay the tier below, where the ATA-SMP scanner uses them.

### Every tab but Status and Console follows the active asset class

**Functional.** Pressing a class button in the header strip filters the whole window. The bar carries nine tabs, and seven of them show only the exchanges, the bots, the charts and the rows of the class on show. Status and Console are left alone.

Read off the running window, the bar holds these nine, left to right:

```
Sim   Paper   Live   Charts   Inspector   Swarm   History   Status   Console
```

What the filter means for each:

| tab | what it shows for the active class |
|---|---|
| Sim | the simulated fleet and the venue pages whose venue serves the class |
| Paper | the paper fleet and its venue pages, on the same rule |
| Live | the exchange sub-tabs whose venue serves the class, and inside each the bots trading a market of that class |
| Charts | one chart per bot of the class, so the asset list holds only those |
| Inspector | the ATA-SPM zone's own class follows the group, and the active-symbol list holds only the class's bots |
| Swarm | one node per bot of the class |

| tab | what it shows for the active class |
|---|---|
| History | one row per fill on a market of the class |
| Status | nothing an asset class owns. It reports emitters and the process, and it never filters |
| Console | nothing an asset class owns. It is a log tail, and it never filters |

`src/gui/main_tabs/class_filter_tab.py` — the one method every press reaches

```
def select_asset_class(self, name: Any) -> None:
    super().select_asset_class(name)
    self._apply_asset_class()
```

**A bot belongs to the class of the market it trades.** A status carries the venue it trades on and the market it trades, and those two answer the class.

`src/gui/main_tabs/class_filter_surface.py` — `market_class`

```
served = venue_classes(venue_id)
if len(served) == 1:
    return next(iter(served))
if DERIVATIVES in served and is_derivative(symbol):
    return DERIVATIVES
```

**One venue, two classes, and the bots split between them.** Coinbase serves crypto and derivatives, so its sub-tab is offered under both. Its spot pairs answer crypto and its dated futures answer derivatives.

Read on the running window with two markets added to the fleet, one Alpaca equity and one Coinbase dated future:

```
alpaca AAPL              -> stocks
coinbase BIT-26DEC25-CDE -> derivatives
crypto 38   stocks 1   commodities 0   derivatives 1   forex 0
```

The 38 Coinbase spot bots stayed under crypto while the one Coinbase future moved to derivatives, off the same venue.

**A tab with nothing for the class says so.** It draws a heading, the thing it has none of, and the way back.

`src/gui/main_tabs/class_filter_surface.py` — the sentences

```
EMPTY_HEADING = "{name} — nothing to show"
EMPTY_NOTE = "No {name} {subject} to show."
EMPTY_HINT = (
    "{name} is the active asset class. Press another class button in the "
    "header strip to see that class."
)
```

Read off the Swarm tab with Forex active:

```
Forex — nothing to show
No Forex bot to show.
Forex is the active asset class. Press another class button in the header
strip to see that class.
```

Both builds draw that note. The Qt build draws it in `EmptyTabQtPanel` and the React build draws it with `src/gui/web/class_note.js`, from the one model `note_model` writes, so neither side holds a word of its own.

**The Live tab keeps its own card when no venue serves the class.** With Stock active and no broker added, the Trading tab draws `No Stock Exchanges Configured` and its Add button stays on screen. The note replaces the Live tab only when a venue does serve the class and no bot trades it.

**The whole fleet is 38 crypto bots, so most classes are empty, and that is the correct reading.** Read on the running window in both builds, at 700, 900 and 1400, with which page each tab draws:

| class | Sim | Paper | Live | Charts | Inspector | Swarm | History |
|---|---|---|---|---|---|---|---|
| Crypto | its own | its own | 38 bots | 45 assets | its own | 76 nodes | its own |
| Stock | note | note | Add card | note | note | note | note |
| Commodities | note | note | Add card | note | note | note | note |
| Derivatives | note | note | note | note | note | note | note |
| Forex | note | note | Add card | note | note | note | note |

Status drew the same 1317 page elements under all five, and Console the same tail. Neither carries a stack, so neither can be switched.

**A filter hides rows and changes nothing else.** Switching class five times left the stored fleet byte-identical, hashed before and after. No bot is stopped, no position is closed and no state is written by a press.

**The class survives a restart.** A fresh window with `forex` stored drew every one of the seven tabs on its note at first draw, with no button pressed.

### Sentences the tabs filter overtakes

The page carries this sentence about the work that came before. It is quoted as it stands, with the sentence that is true today beneath it.

> *"Showing only that class's bots, charts and history is unit F2 and is not built."*

It is built. Seven tabs follow the active class, and Status and Console do not.

### The Add Exchange button reaches the Paper tab

**Functional.** The Paper tab now carries the same Add Exchange button, in the same two seats, reading the same class. The header strip's group is the window's and needed no change to reach it.

The Paper Trader page carries the readings, the press and the card rule: the Paper Trader.

### One reading the Paper Add Exchange button overtakes

The tabs-filter entry above reads each tab's page per class. Its Paper column is quoted as it stands, with the reading that is true today beneath it.

> *"Stock: Sim note, Paper note, Live Add card, Charts note, Inspector note,*
>
> *Swarm note, History note."*

Paper draws its own card, not the note, while it holds no venue of the class, so its Add Exchange button stays on screen and names the class. Read on the running window at 700, 900 and 1400 with no venue seated:

```
Stock         Paper draws its own card, the button reads Add Stock Broker
Commodities   Paper draws its own card, the button reads Commodities - no venue yet
Forex         Paper draws its own card, the button reads Forex - no venue yet
```

Sim and History are untouched and still draw the note. The class list read four entries on that run, because Derivatives had by then been retired onto Crypto.

## 2026-09-24 - #571 - the window model draws the asset class group

### The window model publishes one button per class

**Functional.** The window keeps a model of itself, apart from the widgets, and the bridge answers `main_window.state` from it. That model held a single button of its own, reading Crypto Mode, while the header strip already drew the segmented group. It now publishes the same group the strip draws, built from the same function, so one press names any class and nothing carries two answers.

`src/gui/main_tabs/main_window_surface.py` — `MainWindowModel.select_asset_class`

```
key = asset_class_surface.normalise(name)
self.trading_mode = key
self.class_buttons = asset_class_surface.class_buttons(key)
self.trading_stack_index = asset_class_surface.layer_page(key)
self.window_title = asset_class_surface.window_title(key)
```

**Design intention.** The model exists to report a value that moved on one side and not the other. It could not report this one, because nothing on a screen drew its button, so the drift sat unseen from the day the group landed. Reading the class list at call time removes the class the model can refuse: every declared class answers, and a class added to the taxonomy needs no edit here.

Read on the running program, one press a class, in both builds:

```
Crypto        stack page 0    Acervator - CRYPTO LAYER
Stock         stack page 1    Acervator - STOCK LAYER
Commodities   stack page 2    Commodities - no trading layer
Forex         stack page 2    Forex - no trading layer
```

### The trading stack page comes from one place

**Functional.** Three files used to state which page a class draws: the builder that adds the pages, the emitter that checks them, and the class selector that switches them. The taxonomy states it once now. A class with a trading layer takes its position in the layer list; every class without one draws the single card that names what is missing.

`src/gui/main_tabs/asset_class_surface.py` — `layer_page`

```
layered = layered_classes()
key = normalise(name)
return layered.index(key) if key in layered else len(layered)
```

**Design intention.** The stack holds one page a trading layer, and one card for every class that has none. Commodities and Forex share that card rather than getting empty layers of their own, because an empty layer would offer a venue list it cannot serve.

### Blocks the window model overtakes

Three code blocks near the top of this page describe the control that came before the group. Each is named where it stands, with what is true today beneath it.

**The title block, under the heading naming the title after a swap.** It writes the wing title `Acervator — CRYPTO WING`. A press names a class, and the title is written from the class:

```
self.setWindowTitle(window_title(key))
```

**The wing block, under the design intention for the two-layer stack.** It sets a mode word, retitles one button, checks it, and moves the stack to page one. The window holds no such button. One button per class sits in a group, the group checks the button the active class names, and the stack page is read off the taxonomy:

```
stack.setCurrentIndex(layer_page(key))
```

**The stack block, under the screen itself.** It adds two pages and opens on page zero. The stack holds a third page, the card every class without a layer draws, and the opening page is read rather than typed:

```
self._trading_stack.addWidget(crypto_page)
self._trading_stack.addWidget(stock_page)
self._trading_stack.addWidget(self._make_unlayered_page())
self._trading_stack.setCurrentIndex(acs.layer_page("crypto"))
```

> *"It makes two layers, one for crypto and one for equities, and stacks them so only one is on show at a time."*

Still true, and a third page sits beside the two layers: the card a class with no trading layer draws.

### What the builder's own check reported

**Functional.** The builder emits one reading when it finishes, counting the things it got wrong. That count read one on every launch, because the check named two stack pages while the builder added three. It reads zero now, and the count it compares against is the taxonomy's.

`src/gui/main_tabs/trading_tab.py` — `TradingTabMixin._build_trading_tab`

```
self._trading_stack.count() != acs.stack_pages(),
_crypto_page != acs.layer_page("crypto"),
_stock_page != acs.layer_page("stocks"),
```

Read off the running window, before and after:

```
trading.12.001.postcondition.tab_assembled   1 fault  ->  0 faults
```

## 2026-09-25 - #858 - the command bar reaches Python on the React page

The five controls of the bot list were read on the running program in both builds and set beside one another. Four matched. The command bar did not: on the Live page no button reached the application at all, so nothing happened when it was pressed.

### The overtaken sentence

**Overtaken.** *"Four of the five act on one bot, or on the whole fleet while you hold SHIFT."*

**The true sentence.** Four of the five act on one bot, or on the whole fleet while you hold SHIFT, in the window build and in the page build. Before this entry that held in the window build only.

### What the command bar press reached

**Functional.** A press on the Live page goes out to the application only when it carries one of the fields the page forwards. The command bar's press carried none of them, so it was answered from the page's own held payload and the application never saw it. The command field now goes out beside the other three.

`src/gui/react_trading_tab.py` — the page's own bridge

```
if (
  params &&
  (owns(params, TOGGLE) ||
    owns(params, SORT) ||
    owns(params, PICK) ||
    owns(params, COMMAND))
) {
```

**Design intention.** The page answers a read from what it already holds, and sends a press on to the application, because the register, the fleet, the voting panel and the bot manager all live there. A command is a press, so it travels.

### The venue a press names

**Functional.** One page can draw more than one venue, so every press carries the exchange it came from. The bot table names that field one way and the venue page names it another, and the lookup read only the table's name. A command press therefore found no venue even once it arrived. The lookup now reads every name a press can use.

`src/gui/react_trading_tab.py` — `venue_of_press`

```
def venue_of_press(params: Any) -> str:
    held = params if isinstance(params, dict) else {}
    for name in VENUE_KEYS:
        found = held.get(name)
        if found:
            return str(found)
    return ""
```

### The bot the bar acts on

**Functional.** The bar takes its bot from the table the operator pressed last. The page keeps two records of the Scrumming table: the one it draws, which a row press moves, and the one the bar reads. A row press moved only the first, so the bar found no bot and said "Select a bot first." The bar's own record now takes the bot the drawn list holds, every time a press arrives.

`src/gui/main_tabs/exchange_tab_surface.py` — `ExchangeTabModel.hold_scrum_bot`

```
row = (
    table.bot_ids.index(wanted)
    if wanted in table.bot_ids
    else NO_SELECTION_ROW
)
was = table.block_signals(True)
try:
    table.clear_selection()
    table.set_current_cell(row, 0)
    if row != NO_SELECTION_ROW:
        table.select_row(row)
finally:
    table.block_signals(was)
```

**Design intention.** The Paper tab and the Simulator keep the same two records, so the same call runs on all three. The Detail button already moved both records this way, and the row press now does what the Detail button did.

### What the two builds read, after

Read on the running program over 38 bots, at 700, 900 and 1400 pixels wide, with every socket but loopback refused. Identical at all three widths:

```
                   window build          page build
plain press        1 bot, "stop"         1 bot, "stop"
SHIFT press        stop_all, 0 bots      stop_all, 0 bots
SHIFT labels       Start All, Pause All, Stop All, Restart All, Delete
```

Before the change the page build read 0 bots on the plain press and nothing at all on the SHIFT press.

## 2026-09-25 - #571 - the asset class group becomes one segmented control

### One square, divided into rectangular buttons

**HIS.**

> *"This should be one square segmented into rectangular buttons. You stated you had inferred the meaning. This is not segmenting. This is copy / paste / repurpose. OCIR. Will need to be fixed."*

**Functional.** The group is one rectangle divided by three lines. Neighbours touch, so there is no gap between them. The two end segments round the group's outer corners and the segments between them round nothing. Every segment but the last drops the border it shares with its neighbour, so one line separates two segments rather than two lines meeting.

The active segment still fills with its class accent and keeps its own outline, which is what marks it at a glance.

`src/gui/main_tabs/asset_class_surface.py` — `segment_box`

```
said = [f"border: 1px solid {ds.OUTLINE}"]
if asked not in (SEGMENT_LAST, SEGMENT_ONLY):
    said.append("border-right: none")
said.append("border-radius: 0px")
if asked in ends:
    said.append(f"border-top-left-radius: {end}")
    said.append(f"border-bottom-left-radius: {end}")
```

**Design intention.** One function decides where a segment sits, and one style sheet carries the answer. The page build reads that same sheet, so the two builds cannot drift apart on the shape. A class added to the taxonomy still needs no edit: the new button takes its position from its index.

The page build also reads the active fill from that sheet now. It drew every segment alike before, because the reader took a sheet's plain block and left the state blocks behind.

`src/gui/web/header_strip.js` — `checkedStyle`

```
function checkedStyle(sheet) {
  var found = {};
  stateRules(sheet).forEach(function (block) {
    if (carries(block.selector, CHECKED_STATE)) {
      found = styleOf(block.body);
    }
  });
  return found;
}
```

### The sentence the segment positions overtake

The page carries this sentence about how many buttons the group holds. It is quoted as it stands, with the sentence that is true today beneath it.

> *"Read off the running window, the group holds five buttons:"*

The taxonomy holds four classes — Crypto, Stock, Commodities, Forex — because Derivatives retired onto Crypto. Read off the running window in both builds, the group holds four buttons, and the positions are first, middle, middle, last.

### What the two builds read, after the segmentation

Read off the running program at 700, 900 and 1400 pixels wide, with every socket but loopback refused. The window build was read from the widget's own style and geometry. The page build was read inside the desktop shell, from the drawn element's computed style.

```
reading                                  first    middle    last
the gap to the next segment                0         0        -
the border on the shared edge            suppressed suppressed -
the outer border present                   yes       yes      yes
top-left / bottom-left radius             round      0        0
top-right / bottom-right radius             0        0      round
the active segment reads as active         yes       yes      yes
```

Both builds read the same six rows at all three widths. The group's outer bounds form one rectangle: the segment widths sum to the group's own width with nothing between them, 136 of 136 at 700 and 900, and 464.46 of 464.46 at 1400.

Before the change both builds read a 2 pixel gap, a border on all four sides of every segment, and a 3 pixel radius on all four corners of every segment. The page build also drew the active segment with the same background and text colour as the three inactive ones.

The group's floor fell from 142 to 136 pixels, because four segments that touch ask for six pixels less than four that do not. The five counters and the spendable strip beside it fit at every width.

## 2026-09-25 - #858 - the SHIFT labels on the command bar read backwards

> *"Live - Bug - Bot List - Start / Stop / Pause (All) toggles 'normally on'*
>
> *instead of remaining 'normally off' after releasing the SHIFT key. Can hold*
>
> *SHIFT key to toggle back to the normal state but this is incorrect / inverted*
>
> *behavior."*

### The sentence the labels contradicted

**Overtaken.** *"Let go and they read their own names again."*

**The true sentence.** Let go and they read their own names again, in the window build and in the page build. Before this entry the window build did the reverse: it read the all-bots names with no key held, and the buttons' own names while SHIFT was held down.

### Where the label state came from

**Functional.** The window build asked the application for the state of the SHIFT key. The application keeps one such value for the whole program, and for the SHIFT key's own press and release that value is the one thing this code does not decide. The labels followed it, and they read backwards. The window build now takes the state off the key event it is already holding: a SHIFT press means the key is down, a SHIFT release means it is up, and no other reading is possible. The page build always did this and was never wrong.

### The label rule both window forks share

`src/gui/main_tabs/exchange_tab_surface.py` — `shift_held_after_key`

```
def shift_held_after_key(
    is_press: bool, key_is_shift: bool, event_has_shift: bool
) -> bool:
    if key_is_shift:
        return is_press
    return event_has_shift
```

**Design intention.** The Live tab and the Paper tab drew the same labels from the same wrong reading, so both now call one rule and cannot drift apart. The rule holds whichever way the computer reports the key, which is why the reading below is identical under both.

### The five label readings, per build

Read on the real command bar and the real page, labels taken off each button and never off a picture, every socket but loopback refused:

```
                                  window build   page build
at rest, no key held              single-bot     single-bot
SHIFT pressed and held            all-bots       all-bots
SHIFT released                    single-bot     single-bot
window deactivated while held     single-bot     single-bot
SHIFT pressed twice in a row      all-bots       all-bots
```

Before the change the window build read rows two, three and five wrong. The Paper window build read the same three wrong and now reads all five right.

### The press target after the label repair

**Functional.** The button still acts on what its label says. The press reads SHIFT from the mouse press, which carries the true state, and that reading was never wrong.

```
                   window build          page build
plain press        1 bot, 0 fleet        1 bot, 0 fleet
SHIFT press        stop_all, 0 bots      stop_all, 0 bots
```

## 2026-09-25 - #858 - the Indicator Voting Panel says why it is empty

> *"Live - Bug - IVP - Some bots IVP panels are not loading when selecting.*
>
> *Observed with A19, A24, A11, and A35. Most are working. Cycled to through*
>
> *the bot list a few times and some are consistently failing to load and also*
>
> *giving the candle cache notice in the console but the panels never populate."*

### The two causes an empty panel names

**Functional.** A panel holding no votes prints one sentence saying why. Before this entry it printed nothing and wrote that sentence to the Console alone, which made a market that cannot be voted look exactly like a fault.

Two causes reach the panel. Too few candles means the venue returned fewer bars than the engine needs. Cold start means the bot is running and has computed no reading since the platform launched.

`src/gui/indicator_panel.py` — the sentence each cause prints

```
    "cold_start": "cold start — this bot is running and has computed no TA since "
    "the platform launched. Its first read lands on the next TA "
    "evaluation.",
    "too_few_candles": "too few candles — {candles} cached for {symbol} {timeframe}, "
    "and the TA engine needs 30.",
```

### The floor, and a market that sits under it legitimately

**Functional.** The engine needs thirty candles before any indicator votes. A market with a shorter history is refused a vote, and that refusal is correct rather than a fault, because an indicator computed on too little history is a wrong number. The floor was not lowered to make these panels fill.

Measured on the markets named above, at five minutes:

| market | candles the venue returned | the panel reads |
|---|---|---|
| A24 | 6 to 9 | too few candles |
| A38 | 10 to 12 | too few candles |
| A35 | 13 to 16 | too few candles |
| A19 | 11 to 18 | too few candles |
| A11 | 95 to 96 | cold start |
| a market that votes | 100 | twelve voters |

A11 holds three times the floor and reports the other cause, so a full panel is not the only healthy reading.

### A stored reading draws with its age

**Functional.** When a bot has a reading saved from an earlier evaluation, the panel draws that reading rather than emptying, with the time it was taken, how old it is, and the cause sentence after it. This is why some empty panels already explained themselves and others stayed blank: the ones that spoke had a saved reading behind them.

`src/gui/indicator_panel.py` — the banner over a stored reading

```
            self._staleness_label.setText(
                f"■ LAST TA READ, NOT CURRENT — taken {when}, "
                f"{age_phrase(age_s)}. {message}"
            )
            self._staleness_label.show()
```

The window build and the page build draw the same sentence in the same place.

## 2026-09-25 - #881 - the dollar registry citation names no file

One sentence on this page cites a module a reader cannot open. It is kept as written:

> *The dollar registry is gone.* `src/trading/capital_registry.py` *is removed, with the Qt table, the view model and the renderer page that drew it.*

What the tree holds is this. The path in that sentence is in neither the git index nor the working tree, and the commit that removed the module is the same commit that wrote the sentence citing it, on 2026-09-13. Nothing took the removed module's place. The reservation path sits in a second module that already existed at that commit, that still carries the registry class, and that six modules read.

```
src/trading/capital_reservation.py:87    CapitalReservationRegistry, 566 lines
src/trading/capital_reservation.py:426   effective_available, the reader a sale uses
```

The six readers are the bot, its reservation mixin, the scrumming package, the container restore path, the Extractor and the emitter contract. The removed module declared a different class, over 518 lines, and it was built zero times, which is why its removal changed no behaviour.

The absence was measured two ways, each with a control beside it. A lookup of the cited path in the git index exits non-zero, and the same lookup of the reservation module beside it exits zero. The documentation archetype reports one dead path on this page, at the line carrying that sentence, and reports none against the hundreds of other paths the page cites.

## 2026-09-26 - #928 - the candle cache keeps what it has and stops re-asking

> *"A19 stuck at 13...A24 stuck at 11...probably others...need to isolate the issue."*

### A short series is no longer a cache miss

**Functional.** The shared candle cache holds one slot for each exchange, symbol and timeframe. Before this entry a slot answered a request only while it held as many bars as the request asked for. A market that cannot supply that many failed the test on every call, so the venue was called again every time and the panel still drew nothing.

The slot now records how many bars the venue supplied, whenever that is fewer than the number asked for. A slot holding all of them answers the request instead of sending another call.

`src/exchange/data_pool.py` - the one test both call sites now share

```
    def can_serve(self, limit: int) -> bool:
        """True when this slot is fresh and holds ``limit`` rows, or holds every row
        ``venue_row_cap`` says the venue has."""
        if self.is_stale or not self.candles:
            return False
        if len(self.candles) >= limit:
            return True
        return (
            self.venue_row_cap is not None and len(self.candles) >= self.venue_row_cap
        )
```

Driven on one market at five minutes, the venue answering thirteen bars, the slot never aged:

| requests made | venue calls before | venue calls after |
|---|---|---|
| 5 | 5 | 1 |
| 60 | 60 | 1 |

### The series grows instead of being replaced

**Functional.** A fetch used to replace the stored bars with whatever came back, so the count could never rise above one venue answer. A fetch now merges what came back into the bars already held, keeps one row for each bar time, orders them oldest first, and keeps the newest rows up to the number the pool asked for.

`src/exchange/data_pool.py` - the merge

```
def _merge_candle_rows(stored: list, fetched: list) -> Optional[list]:
    """One row per timestamp from ``stored`` then ``fetched``, oldest first, None
    when a row carries no finite ``row[0]``."""
    by_time: dict[float, list] = {}
    for row in list(stored) + list(fetched):
        at = _row_timestamp(row)
        if at is None:
            return None
        by_time[at] = row
    return [by_time[at] for at in sorted(by_time)]
```

Driven with two answers of thirteen and fourteen bars that share eleven, so twenty-seven rows arrive in all:

| reading | before | after |
|---|---|---|
| bars held after the second answer | 14 | 16 |
| repeated bar times held | 0 | 0 |
| bars in time order | yes | yes |
| the oldest bar of the first answer | dropped | kept |

Two further readings, taken the same way. When the venue answers newest first, the stored bars were left out of order before this entry and are in order after it. When the venue repeats a bar time inside one answer, the repeat was stored before this entry and is dropped after it.

### One venue call for each slot life, not one for each request

**Functional.** A slot lives for the number of seconds its own timeframe names, which is three hundred at five minutes. A market that stays short is now called once inside that window rather than once for each request, the same rate every other market already had. Nothing new sets this rate. It is the slot's own life, and it was already in the file.

`src/exchange/data_pool.py` - where the rate comes from

```
    @property
    def ttl_seconds(self) -> float:
        return TF_SECONDS.get(self.timeframe, 3600)

    @property
    def is_stale(self) -> bool:
        return self.age_seconds > self.ttl_seconds
```

A market that genuinely has more bars still gets them. Driven with the venue answering one hundred bars and the slot aged past its life, the venue is called again on both sides of this entry, once each.

### The sentence this overtakes

One sentence on this page reads the count as what one venue answer returned. It is kept as written:

> *Too few candles means the venue returned fewer bars than the engine needs.*

What the tree holds is this. The count the panel reads is the number of bars the slot holds, and the slot now keeps bars from earlier answers beside the newest ones, so that count can rise between answers without any single answer returning more. The measured table above that sentence records what one answer returned on those markets, which is what it still says.

A market that stays short now draws its indicators once enough bars have gathered. At five minutes a slot holding thirteen bars gains one every five minutes, and the engine's floor of thirty is reached about eighty-five minutes after the bot starts. Until then the panel prints the same sentence it printed before, and the venue is called once every five minutes rather than once for every request.

### The floor is written once, and every reader resolves it there

**Functional.** The number of candles the engine needs before it votes was written in twelve separate places, and three of those were the digits typed inside the sentence the panel prints. A sentence that states a threshold it does not read can go stale without anything reporting it, and the three panels held three copies of it. The figure now has one home, beside the engine that needs it, and every other place reads it from there.

`src/trading/ta_engine.py` - the one place the floor is written

```
#: The candles ``VotingEngine.compute_all`` needs before any indicator votes.
MIN_CANDLES_FOR_TA = 30
```

The value did not change. Driven on the three real panels, a market holding twenty-nine bars still draws the refusal, and a market holding thirty draws the cold-start sentence instead. The sentence the operator reads is the same sentence, character for character, as the one this page already quotes.

| reading | before | after |
|---|---|---|
| places holding the figure | 12 | 1 |
| the cause at twenty-nine bars | too few candles | too few candles |
| the cause at thirty bars | cold start | cold start |
| the tightening detector's own window | 25 | 25 |

A block earlier on this page quotes the panel's template with the figure typed into the prose. It is kept as written, and it is overtaken:

```
    "too_few_candles": "too few candles — {candles} cached for {symbol} {timeframe}, "
    "and the TA engine needs 30.",
```

What the tree holds is this. The figure arrives as a field, filled from the name the engine owns, so the prose carries no number and all three panels print the figure the engine is actually using:

```
    "too_few_candles": "too few candles — {candles} cached for {symbol} {timeframe}, "
    "and the TA engine needs {floor}.",
```

## 2026-09-26 - #571 - the asset class group becomes one square of two rows

### Two columns by two rows, in one square boundary

**HIS.**

> *"Get these mode buttons corrected. I gave you a properly elaborated spec for how this segmented of the ONE square was supposed to be done and yet, here I am, in the latest build still seeing a STUPID design that the GUI Archetype should have flagged as pure grade shit."*

And the elaborated spec he gave, on this issue:

> *"the large Crypto / Stock Wing button in the upper right of the GUI which will now become a segmented group featuring one layer button per available market sector"*

**Functional.** The group is one square. Four asset classes divide it into four rectangles, two across and two down, by one vertical line and one horizontal line. The square is 68 pixels on each side and each segment is 34 by 34.

A segment past the first column drops the border it shares with the segment to its left. A segment past the first row drops the border it shares with the segment above. One line therefore draws between two neighbours, never two.

Each of the four segments rounds exactly one corner, and only the corner it shares with the square. Every corner inside the square is square.

`src/gui/main_tabs/asset_class_surface.py` — `segment_box`

```
said = [f"border: 1px solid {ds.OUTLINE}"]
if column > 0:
    said.append("border-left: none")
if row > 0:
    said.append("border-top: none")
said.append("border-radius: 0px")
if top and left:
    said.append(f"border-top-left-radius: {corner}")
```

**Design intention.** A model that treats its members as a line cannot produce a square, so a segment now carries a row and a column instead of a place on a line. `segment_cell` answers that place and `segment_box` reads it, and both builds take the same answer.

`src/gui/main_tabs/asset_class_surface.py` — `grid_shape`

```
columns = math.isqrt(held)
if columns * columns < held:
    columns += 1
rows = held // columns + (1 if held % columns else 0)
```

The side is one declared number, `group_side_px`, so the window build and the page build cannot draw two different squares. Neither build measures its own row height.

`src/gui/web/header_strip.js` — the square's own style

```
var side = length(model[SIDE]);
var groupStyle = {
  display: "grid",
  gridTemplateColumns: tracks(cell(model[GRID_COLUMNS], 1)),
  gridTemplateRows: tracks(cell(model[GRID_ROWS], 1)),
  width: side,
  height: side
};
```

### The sentences the square overtakes

Each is quoted whole, marked overtaken, with the sentence that is true today beneath it.

**Overtaken.** *"The group is one rectangle divided by three lines."*

The group is one square divided by one vertical line and one horizontal line.

**Overtaken.** *"The two end segments round the group's outer corners and the segments between them round nothing."*

Each of the four segments rounds one of the square's four outer corners, and rounds nothing else.

**Overtaken.** *"Every segment but the last drops the border it shares with its neighbour, so one line separates two segments rather than two lines meeting."*

A segment past the first column drops its shared left border and a segment past the first row drops its shared top border, so one line separates two neighbours in both directions.

**Overtaken.** *"Read off the running window in both builds, the group holds four buttons, and the positions are first, middle, middle, last."*

The group holds four buttons at row 0 column 0, row 0 column 1, row 1 column 0 and row 1 column 1.

**Overtaken.** *"The group's floor fell from 142 to 136 pixels, because four segments that touch ask for six pixels less than four that do not."*

The square's side is 68 pixels, which is its two columns times the 34 pixel minimum one segment draws at.

### What a class count other than four divides into

The columns are the integer square root of the count rounded up, and the rows follow from that, so the grid is the one nearest to square. The last row's segments share the columns the full rows use, so every row fills the width.

```
count   columns x rows   the last row
1       1 x 1            one segment, rounding all four corners
2       2 x 1            full
3       2 x 2            one segment spanning both columns
4       2 x 2            full
5       3 x 2            two segments spanning 2 and 1 columns
6       3 x 2            full
9       3 x 3            full
12      4 x 3            full
```

Nothing is written as four. The count comes from `src.trading.ata_spm.ASSET_CLASSES` when the group is built.

### What the two builds read of the square

Read off the running program at 700, 900 and 1400 pixels wide, with the home directory redirected to a scratch folder. The window build was read from each widget's own geometry and style sheet. The page build was read inside the shell's renderer page, from each drawn element's client rectangle and computed style.

```
reading                                 window build   page build
the group's width and height             68 x 68        68 x 68
the width over the height                  1.0            1.0
the rows and the columns                  2 x 2          2 x 2
each segment's width and height          34 x 34        34 x 34
the gap between two neighbours, across      0              0
the gap between two neighbours, down        0              0
segments rounding more than one corner      0              0
```

Both builds read the same seven rows at all three widths, and the page build reads them again with its row forced taller, so the side follows the declared number and not a row's height.

Before the change both builds read one row of four, a group 136 by 58 in the window build and 389 by 17 in the page build, and two of the four segments rounding two corners each.

### The rule that refuses a separated group

**HIS.**

> *"a STUPID design that the GUI Archetype should have flagged"*

The GUI archetype now carries GUI007. It reads every function that names a segment and writes its own border box, and it refuses three shapes: a group that suppresses no shared edge, a branch that rounds more than one corner, and a spacing or gap constant that is not zero.

`dev_harness/harness/gui_archetype.py` — `_scan_segmented_group_skin`

```
shared = any(f"border-{edge}: none" in joined for edge in _SEGMENT_EDGES)
if not shared:
    findings.append(Finding(..., rule_id="GUI007", ...))
```

Run against the file as it stood before this change, the rule reports two high findings and the archetype reads `passed=False`. Run against the file as it stands now, it reports none and reads `passed=True`. Its fixture pair is `harness_fixtures/gui_archetype/known_good_segmented.py`, which exits zero, and `harness_fixtures/gui_archetype/known_bad_separated.py`, which exits one.

### Where the spare width in the header row goes now

The square is one fixed side, so it can no longer take the width the figures leave. The spendable strip takes it instead, which is the first slot of the row, so the square stays at the row's right end and the money figures elide less.

`src/gui/main_tabs/header_strip_surface.py` — `TOP_ROW_STRETCH`

```
TOP_ROW_STRETCH = [1, 0, 0, 0, 0, 0, 0]
```

**Overtaken.** *"A zero slot draws at the width its own text asks for; the class group takes what is left, so a long amount elides a class name and never a figure."*

A zero slot draws at the width its own text asks for, and the spendable strip takes what is left.

### The row's spare width is empty space

The page carries two sentences about where that width goes. Both are quoted whole here, marked overtaken, with the sentence that is true today beneath.

> *"The class group takes what the figures leave. Every other part of the row draws at the width its own text asks for, and the group takes the rest down to its floor. A wide amount therefore shortens a class name and never a figure."*
>
> *"The square is one fixed side, so it can no longer take the width the figures leave. The spendable strip takes it instead, which is the first slot of the row, so the square stays at the row's right end and the money figures elide less."*

**Overtaken.** No slot of the row takes the width the figures leave. Every share is zero, and the row inserts empty space before the square, which takes every spare pixel.

`src/gui/main_tabs/header_strip_surface.py` — the shares and the empty space

```
TOP_ROW_STRETCH = [0, 0, 0, 0, 0, 0, 0]
TOP_ROW_SPACER_BEFORE = "mode_button"
TOP_ROW_SPACER_STRETCH = 1
```

Both builds insert the same space. The Qt row inserts it at the index the order gives, and the page draws one empty division at the same place.

`src/gui/main_tabs/header_strip.py` — the Qt insertion

```
top_row.insertStretch(
    surface.TOP_ROW_ORDER.index(surface.TOP_ROW_SPACER_BEFORE),
    surface.TOP_ROW_SPACER_STRETCH,
)
```

**What the operator sees.** Measured on the real widgets, Qt build, with the square held at the side it had before the square work, so the two rows differ in nothing but this change. At 1400 px and wider every slot is the width it was before the square work, and the empty space holds the remainder.

| window | strip | Scrummed | Folded | Trades | Bots | Errors | empty space |
|---|---|---|---|---|---|---|---|
| 1400 | 706 | 155 | 156 | 75 | 55 | 75 | 0 |
| 2560 | 709 | 158 | 158 | 78 | 58 | 78 | 1143 |
| 3840 | 709 | 158 | 158 | 78 | 58 | 78 | 2423 |

Before this change the strip drew 1750 px of a 2560 px window and 3030 px of a 3840 px window, and every counter fell to 48 px.

### Each counter's floor holds its own figure

The page carries a sentence about the counter floor. It is quoted whole here, marked overtaken, with the sentence that is true today beneath.

> *"COUNTER_MIN_W = 48"*

**Overtaken.** One flat floor served five counters, and 48 px holds neither a ten-character amount nor a six-figure count. Each counter now declares the characters its own figure draws, and its floor is the room those characters take.

`src/gui/main_tabs/header_strip_surface.py` — the characters and the floor

```
COUNTER_CHARS = {
    "scrummed": COUNTER_AMOUNT_CHARS,
    "folded": COUNTER_AMOUNT_CHARS,
    "trades": COUNTER_COUNT_CHARS,
    "bots": COUNTER_BOTS_CHARS,
    "errors": COUNTER_COUNT_CHARS,
}
```

The money counters draw `$12,345.67` at their widest, so their floor is `COUNTER_NATURAL_W`, the room this page already records for a whole money amount. A trade or error count draws six figures and a bot count three, so each takes that room scaled by its own characters.

| counter | characters | floor |
|---|---|---|
| Scrummed | 10 | 100 |
| Folded | 10 | 100 |
| Trades | 6 | 66 |
| Bots | 3 | 41 |

| counter | characters | floor |
|---|---|---|
| Errors | 6 | 66 |

**The tile holds the figure, and the window's narrowest grows with it.** The row's floors now sum to 137 px more than before, so the narrowest the window opens at rises by the same 137 px. Nothing is cut off at any width the window reaches; the window refuses to open narrower than its floors.

## 2026-09-26 - #928 - the pool reaches back for the bars the engine needs

### A short answer is not a short history

**Functional.** A market that returns eleven bars is not a market that has eleven bars. The venue answers with the periods that traded inside a window ending now, so a market that trades rarely returns a handful however long it has been listed.

Read from the console log for one of the named markets at five minutes, over four days and 5,494 recorded answers:

| day | bars the venue returned |
|---|---|
| 2026-09-23 | 11 to 35 |
| 2026-09-24 | 14 to 39 |
| 2026-09-25 | 11 to 27 |
| 2026-09-26 | 7 to 21 |

The count falls as well as rises, and falls across days. A market limited by its own record gains bars and never loses them, so this count is the trading inside the window and not the length of the record. The history is there. The request was never reaching back for it.

### The request asks for a span of time, not a count of bars

**Functional.** The pool asked for the most recent bars and nothing else, so a market that fills few of the recent periods was sent the same short answer for ever. A slot the venue cannot fill now asks for the window immediately before the oldest bar it holds. Each refresh therefore adds older bars, the merge already in place folds them in on bar time, and the stored series grows until it holds what the engine needs.

`src/exchange/data_pool.py` - the window a short slot asks for next

```
    def next_reach_back_ms(self, requested: int) -> Optional[int]:
        """Epoch milliseconds of the window before the oldest row or last
        ``reach_back_since``, recorded, None at ``MIN_CANDLES_FOR_TA`` rows or past
        ``REACH_BACK_PERIODS``."""
```

### Where the span comes from, and what bounds it

**Functional.** Two figures already in the tree set the span, and no third figure was chosen. The engine's floor lives beside the engine that needs it, and the length of one period lives in the pool's own table of timeframes. The pool reaches back thirty periods for each of the thirty bars the engine needs, which is nine hundred periods, or seventy-five hours at five minutes.

`src/exchange/data_pool.py` - the bound, built from the floor and the period

```
REACH_BACK_PERIODS = MIN_CANDLES_FOR_TA * MIN_CANDLES_FOR_TA
```

A market that trades in fewer than one period in thirty cannot reach the floor inside that span. At that point the pool stops reaching back, keeps the bars it has, and the panel prints the sentence it printed before.

Driven on a market trading one period in nine, twelve refreshes, the venue answering only what traded inside each window:

| reading | before | after |
|---|---|---|
| bars the slot holds | 12 | 34 |
| venue calls | 12 | 12 |
| refreshes that asked for an older window | 0 | 2 |
| repeated bar times | 0 | 0 |
| bars in time order | yes | yes |

Driven again on a market trading one period in two hundred, the same twelve refreshes: five bars held, twelve venue calls, eight refreshes asked for an older window and the bound then stopped it, no repeated bar time, bars in time order. That market never reaches the floor, and it stops asking.

### A market that fills the count is untouched

**Functional.** A slot records how many bars the venue supplied only when that is fewer than the number asked for. A market that fills the count carries no such record, so no start time is computed for it and none is sent.

Driven on a market where every period trades, twelve refreshes, every reading is the same on both sides of this entry: one hundred bars held, twelve venue calls, no start time sent, and the same oldest bar.

### The sentence about waiting is overtaken

**Overtaken.** *"At five minutes a slot holding thirteen bars gains one every five minutes, and the engine's floor of thirty is reached about eighty-five minutes after the bot starts."*

A slot short of the floor asks for an older window on each refresh instead of waiting for new bars, so the floor is reached from the history the venue already holds.

## 2026-09-26 - #936 - the bot list scrolls only by hand

The operator reaches the Scrumming Bots table from the title bar and the menu row, through the tab row to the Trading tab, then the exchange sub-tab that fills the left of that tab. The Indicator Voting Panel fills the right.

### The scroll bar stays where he put it

**Functional.** The list no longer moves itself. Qt scrolls an item view to its current item while auto-scroll is on, and the bot list now turns that off, so the only thing that moves the scroll bar is the operator.

`src/gui/widgets/bot_status_table.py` — `BotStatusTable.__init__`

```
self.setAutoScroll(False)
self.verticalScrollBar().valueChanged.connect(
    lambda _value: self.release_selection_off_view()
)
```

### A highlight that leaves the drawn rows is released

**Functional.** One rule decides it, and both builds ask that one function: a highlighted row the viewport no longer draws loses the highlight. The drop runs with the table's signals blocked, so nothing downstream hears it.

`src/gui/main_tabs/bot_status_table_surface.py` — `selection_after_view_moved`

```
if row < 0 or first < 0 or last < first:
    return NO_SELECTION_ROW
if first <= row <= last:
    return row
return NO_SELECTION_ROW
```

The React page reads its own drawn rows off the scroll box it sits in and sends the first and last of them, and the same function answers there.

`src/gui/web/bot_status_table.js` — `visibleRowBand`

```
if (span.top >= edge.top - 1 && span.bottom <= edge.bottom + 1) {
  if (first < 0) {
    first = at;
  }
  last = at;
}
```

### A column sort counts as leaving the view

**Functional.** A sort reorders every row while the scroll position stays, so the bot that was highlighted can land far below the rows on screen. That is the highlight leaving the view and it is released, the same as any other cause.

Read on the running list, a 38-bot fleet, one column pressed twice with the scroll at the top:

```
                          before this entry    after
scroll position           0 then 27            0 then 0
rows drawn                0 to 11 then 26-37   0 to 11 both times
that bot's row            37                   37
its highlight             held, off screen     released
```

### The Voting Panel keeps its bot

**Functional.** The list's highlight and the panel's bot are now two values. A release changes the list alone. The panel changes only when a bot is chosen, from a row or from the panel's own dropdown, and the Live page does not redraw the panel on a scroll.

`src/gui/react_trading_tab.py` — the venue answers four presses now

```
VENUE_PRESSES = (
    (scrum_surface.PRIVACY_TOGGLE_PARAM, "toggle_privacy"),
    (scrum_surface.SORT_COLUMN_PARAM, "sort_by"),
    (scrum_surface.SELECT_BOT_PARAM, "select_bot"),
    (scrum_surface.VIEW_BAND_PARAM, "scroll_view"),
)
```

### What both builds read

Read off the running program in both builds at 700, 900 and 1400 pixels, with a fleet of 38 bots, the home on a scratch directory and every host but loopback refused. Every figure is the same at all three widths.

```
the window build                        before    after
auto-scroll                             on        off
scroll after an off-screen highlight     27        13, unmoved
that highlight                           held      released
scroll after a hand drag                 moves     moves
the panel's bot after a release          held      held
the panel after a row is chosen          follows   follows
the panel after its dropdown is used     follows   follows
```

```
the page build                          reading
rows drawn                              38
rows drawn before a hand drag           0 to 7
rows drawn after it                     28 to 37
the pair the page sent                  28 and 37
the list's highlight after              released
a pair still holding that row           highlight kept
```

Sibling widgets measured the same on both sides at all three widths: the table, the window, the panel, the header row, all ten column widths and the row heights.

### The sentence about one value behind both is overtaken

**Overtaken.** *"**Functional.** The highlight runs the other way as well. When the panel's own dropdown or one of its arrows moves the selection, the bot list puts its highlight on that bot's row. One value stands behind both, the bot the Voting Panel holds, so the list and the panel cannot name two different bots."*

The panel's dropdown still puts the highlight on that bot's row, and now only while that row is drawn. The two can name different bots: the panel holds the bot it is drawing and the list holds a highlight it can release, which is what keeps the scroll bar still.

## 2026-09-26 - #937 - the first column draws the target asset's logo

The operator reaches the Scrumming Bots table from the title bar and the menu row, through the tab row to the Live tab, then the exchange sub-tab that fills the left of that tab. The first column no longer shows an eight-character bot id. It shows the target asset the bot accumulates.

### The first column is the asset's own mark

**Functional.** The column is headed Asset. Where a logo file is already kept for that asset, the cell draws it and holds no text. Where none is kept, the cell draws the asset's ticker instead, so the row still names what the bot accumulates. Nothing on this path fetches anything: the cell reads the kept directory and no further.

`src/gui/main_tabs/bot_status_table_surface.py` — `logo_cell`

```
if shown != asset or not asset:
    return cell(shown)
address = logo_data_address(path)
if path and address:
    return cell(
        EMPTY_TEXT,
        tooltip=LOGO_TIP_FORMAT.format(asset=asset),
        logo_path=path,
        logo_image=address,
        logo_size=LOGO_SIZE_PX,
    )
return cell(asset, tooltip=NO_LOGO_TIP_FORMAT.format(asset=asset))
```

The asset a crypto bot names is kept under its base, and a currency pair under the whole pair, so both keys are asked in that order.

`src/gui/main_tabs/bot_status_table_surface.py` — `kept_logo_path`

```
for asked in (icon_asset_of(symbol), symbol):
    if not asked:
        continue
    found = KEPT_LOGOS.kept_path(asked)
    if found is not None:
        _LOGO_PATHS[symbol] = str(found)
        return str(found)
```

The React page cannot open a file by its path: the page's own rule allows an image from the page itself or from a data address and nothing else. The same bytes therefore travel to it as a data address, read once per file.

`src/gui/web/bot_status_table.js` — `AssetLogo`

```
var logoProps = {
  src: text(found[LOGO_IMAGE]),
  alt: label(found[TOOLTIP]),
  draggable: false,
  className: TABLE_CLASS,
  style: style
};
```

Read on the running list, nine bots, one per state, at 700, 900 and 1400 wide:

```
the bot's asset       first column            kept file
A15/USD               the logo, 32 by 32      A15.png
A14/USD               the logo, 32 by 32      A14.png
A23/USD               the logo, 32 by 32      A23.png
AAPL                  the logo, 32 by 32      AAPL.png
GLD                   the logo, 32 by 32      GLD.png
EUR/USD               the logo, 32 by 32      EUR-USD.png
A17/USD              the ticker, A17        none
ZZZQ/USD              the ticker, ZZZQ        none
A22/USD              the ticker, A22        none
```

### The state colour and the mode tooltip sit on the Symbol cell

**Functional.** The colour that says what a bot is doing moved one column right. The Symbol cell carries it, in the same seven named colours as before, with one further colour for a state the list does not name. That cell's tooltip names the mode and the state, and where the pair has a chart the chart's line follows underneath it. The underline is what marks the cell as the chart's link.

`src/gui/main_tabs/bot_status_table_surface.py` — `BotStatusTableModel._symbol_cell`

```
found = cell(text, state_color(state), mode_tooltip(mode, state))
```

Read on the running list, nine bots, one per state:

```
state       the Symbol cell's colour
running     #00ff88
idle        #888888
paused      #ffaa00
error       #ff3366
cooldown    #ff6600
stopped     #666666
starting    #00e6ff
not named   #e0e0f0
not set     #e0e0f0
```

### The stand-in disc beside each symbol is gone

**Functional.** The coloured circle carrying the asset's first letter no longer draws in the Symbol cell. The cell sets no fill and no letter, so the window paints no badge and the page draws no circle.

`src/gui/main_tabs/bot_status_table_surface.py` — the Symbol cell's own fields

```
found = cell(text, state_color(state), mode_tooltip(mode, state))
```

### The row height comes from the logo's drawn size

**Functional.** A logo draws at 32 pixels square. A row keeps two pixels above and two below its content, which is what this widget's style answers for its own item margin, so a row takes 36 pixels. Before this change a row took 30, which was the size the style chose for a row of text.

`src/gui/main_tabs/bot_status_table_surface.py` — the two figures and the sum

```
LOGO_SIZE_PX = 32
ROW_LOGO_MARGIN_PX = 2
ROW_HEIGHT_PX = LOGO_SIZE_PX + 2 * ROW_LOGO_MARGIN_PX
```

Read on the running tab at all three widths, with every other element of the tab read beside it:

```
reading                          before   after
the row height                       30      36
the icon size the table asks      style      32
all ten column widths             same    same
the header row's height              60      60
the ten privacy dots              same    same
the ten sort marks                same    same
the table's size hint             same    same
the table's minimum size hint     same    same
the scroll bars                   same    same
every other widget in the tab     same    same
```

### The page reads the same row height as the window

**Functional.** The page's own sheet gives every cell three pixels above and three below its content and a one-pixel rule beneath it, which is more room than the window's style leaves. A thirty-two pixel mark plus that padding is thirty- nine pixels, so the logo's own cell drops its vertical padding and the row keeps the height the payload set. No other cell and no rule in the sheet changes, so the Extractor table beneath is untouched.

`src/gui/web/bot_status_table.js` — the logo's own cell

```
style.paddingTop = NO_PAD;
style.paddingBottom = NO_PAD;
style.lineHeight = NO_PAD;
```

Read off the drawn page, nine rows, at all three widths:

```
reading                           before   after
every row's drawn height           29-30      36
the logo's drawn box                 none   32 by 32
the image behind it                  none   64 by 64
the first column's drawn width       same    same
every other column's drawn width     same    same
the header row's drawn boxes         same    same
the ten privacy dots                same    same
the ten sort marks                  same    same
```

### The first column drawn off the page itself

**Functional.** The same nine bots were driven through the React venue page and read off its own elements rather than off the payload. The page reported no fault of its own at any of the three widths.

```
state       first cell, before   Symbol cell, before   Symbol cell, after
running     rgb(0,255,136)       rgb(102,204,255)      rgb(0,255,136)
idle        rgb(136,136,136)     rgb(102,204,255)      rgb(136,136,136)
paused      rgb(255,170,0)       rgb(102,204,255)      rgb(255,170,0)
error       rgb(255,51,102)      rgb(102,204,255)      rgb(255,51,102)
cooldown    rgb(255,102,0)       rgb(102,204,255)      rgb(255,102,0)
stopped     rgb(102,102,102)     rgb(224,224,240)      rgb(102,102,102)
starting    rgb(0,230,255)       rgb(224,224,240)      rgb(0,230,255)
not named   rgb(224,224,240)     rgb(102,204,255)      rgb(224,224,240)
not set     rgb(224,224,240)     rgb(102,204,255)      rgb(224,224,240)
```

The stand-in circle was present in all nine Symbol cells of the page before and in none of the nine after. Six of the nine first cells hold a drawn image and three hold their asset's ticker.

### The bot id keeps its places outside the table

**Functional.** The bot id is still the thing a row is identified by. It stays in the payload, once as the list of every drawn row's bot and once on each row. It stays in the line the list writes when a row is built. The Detail button still carries it, and the window that button opens still names its first eight characters in its own title.

`src/gui/main_tabs/bot_live_settings_surface.py` — the title that names it

```
WINDOW_TITLE_FORMAT = "Bot Settings — {symbol} [{short_id}]"
```

Read on the running list, one bot, five routes:

```
route                                  read back
the payload's list of bots             bot01ddddeeeeffff
the row's own bot                      bot01ddddeeeeffff
the line written when the row was built bot01ddddeeeeffff
the window's selected bot              bot01ddddeeeeffff
the Detail button, pressed             bot01ddddeeeeffff
```

### The privacy dot over the first column still hides it

**Functional.** The dot beneath the Asset label still masks that column. A masked first column draws no image and no ticker, only the mask, and revealing it brings the logo back.

Read on the running list, the dot driven twice:

```
the dot         first column
revealed        the logo, 32 by 32
masked          ****, no image
revealed again  the logo, 32 by 32
```

### The sentence about the state colour on the Bot ID cell is overtaken

**Overtaken.** *"The state colour now sits on the Bot ID cell, which is green while running, amber while paused, grey while idle or stopped, red on error, orange in cooldown and cyan while starting. That cell's tooltip names the mode and the state."*

The state colour now sits on the Symbol cell, in those same colours. That cell's tooltip names the mode and the state, and the chart's line follows it where the pair has a chart.

### The sentence about Bot ID sorting as a word is overtaken

**Overtaken.** *"**Functional.** Bot ID and Symbol sort as words."*

The first column is headed Asset and sorts as a word, on the asset's ticker. Symbol still sorts as a word.

## 2026-09-27 - #937 - the mark a row draws comes from the library on disk

The first column of the Scrumming Bots table draws the target asset's own official mark. The mark is a file the logo library has already put on disk. The list never fetches one.

### Where a row's mark comes from

**Functional.** The library sits at `resources/logos`, under the repository, and is filled ahead of any bot. A crypto asset's mark comes from a coin data source; every other asset's mark comes from that organisation's own web site, read at the two places a browser reads one from. The library files each mark under its asset class and its sector, so one asset has one file wherever it is met.

`src/trading/logo_library.py` — where one asset's file is put

```
def library_folder(asset_class: Any, sector: Any = "") -> str:
    return kept_folder(f"{asset_class}/{sector}")
```

The list reads that directory at any depth, so a mark filed under a sector answers the same way a mark filed flat does.

`src/core/asset_logos.py` — the read at any depth

```
for found in self._cache_dir.rglob(f"{stem}.*"):
    rank = ranks.get(found.name)
    if rank is None or rank >= best_rank or not found.is_file():
        continue
    best, best_rank = found, rank
```

Read with fifteen real marks on disk, nine bots, at 700, 900 and 1400 wide:

```
sector                        marks on disk   an example
crypto                                   10   A15, A14, A21, DOT
stocks                                    4   AAPL, AMZN
commodities                               1   SLV
forex                                     0   none answered an image
```

### A mark keeps its own shape and its own detail

**Functional.** A mark is drawn inside a box of the logo's own size and is never squeezed to fill it. A mark that is not square keeps its proportions in both builds. A mark whose file holds several sizes is drawn from the size nearest the box rather than from the smallest. A mark smaller than the box is drawn at its own size rather than enlarged.

`src/gui/widgets/bot_status_table.py` — the window hands the file to the icon whole

```
icon = QIcon(path)
found = icon if icon.availableSizes() else None
```

`src/gui/web/bot_status_table.js` — the page bounds the mark rather than setting it

```
style.maxWidth = size + PX;
style.maxHeight = size + PX;
```

Read on both builds, at all three widths, one row per mark:

```
the asset   the file holds            the window draws   the page draws
A15         50 by 50                  32 by 32           32 by 32
A14         50 by 50                  32 by 32           32 by 32
A21        50 by 50                  32 by 32           32 by 32
AAPL        152 by 152                32 by 32           32 by 32
SLV         16 by 16 and 32 by 32     32 by 32           32 by 32
AMZN        48, 32 and 16 square      32 by 32           32 by 32
DOT         47 by 50                  30 by 32           30.08 by 32
A17        no file                   the ticker         the ticker
ZZZQ        no file                   the ticker         the ticker
```

### A bot whose asset has no mark reads the same as before

**Functional.** The cell draws that asset's ticker, and its tooltip says no mark is kept for it yet. Nothing about such a row changes: its height, its columns and every other cell read the same before and after.

### The row height with real marks on the rows

**Functional.** A row is 36 pixels tall, in the window and on the page, with a mark on it and without one. That is the figure the payload already carried, and real marks on real rows did not move it.

```
reading                              before   after
the row height, the window               36       36
the row height, the page                 36       36
the icon size the table asks             32       32
all ten column widths, the window      same     same
all ten column widths, the page        same     same
the header row above the list          same     same
the seven slots of the header strip    same     same
```

## 2026-09-26 - #937 - a mark opens the organisation that owns the asset

His words:

> *"Logos should also double as hyperlinks to the company or organization beyond each asset. Want Acervator to feel like it is connected to all of these corners of the investment world simultaneously like a creature with a thousand*

*tendrils..."*

The first column's mark is a link. Clicking it opens the front door of the company or organisation that owns the traded asset, in the operator's own browser. A mark whose asset has no known web address is not a link: it draws exactly as it drew before, it opens nothing, and its tooltip says so.

### Where a mark's address comes from

**Functional.** Two readers already in the tree answer an address, and the table asks them in one place. A crypto base answers the site its own record carries. A listed name answers the domain the asset maps hold for it. The regulator's company-search page is never answered, because a search result is not an organisation's own front door.

`src/gui/main_tabs/bot_status_table_surface.py` — `organisation_address`

```
for asked in (icon_asset_of(symbol), symbol):
    if not asked:
        continue
    site, _ = CRYPTO_RECORDS.organisation_url(asked)
    found = site or organisation_page(asked)
    if found:
        break
address = opening_address(found)
```

Read by driving both readers over every row the asset maps hold and every record the crypto database holds:

```
class            rows   an organisation address   none
stocks             47                        43      4
currencies         28                        28      0
commodities        14                        10      4
crypto records     48                        10     38
```

The negative control: a planted asset name no map and no record holds answers an empty address, with the reason "no address is known".

### What the address check refuses

**Functional.** An address is checked before any browser is asked to open it. Only a secure web address with a host opens. Everything else is refused and nothing is opened, whatever the data carries.

`src/gui/main_tabs/bot_status_table_surface.py` — `opening_address`

```
OPENING_SCHEMES: tuple[str, ...] = ("https",)
```

Read by handing each value to the check, one per line, on both clickable columns:

```
what the data carried                      what happened
http://evil.example/x                      refused
file:///C:/Windows/System32/calc.exe       refused
javascript:alert(document.cookie)          refused
data:text/html; base64,...                  refused
ftp://evil.example/x                       refused
evil.example, a bare host                  refused
https:///etc/passwd, no host               refused
C:\Windows\System32\calc.exe               refused
https://bitcoin.org                        opened
```

Twelve refusing shapes were driven and twelve were refused. Three secure addresses were driven and three opened, which is the control that the check can still say yes.

### What a mark with no address draws

**Functional.** The cell draws what it drew before the mark became a link. Its tooltip carries one extra line naming the absence, so the operator can see which marks are links and which are not without pressing one.

Read off the running list and off the drawn page, one asset of each kind:

```
asset   the tooltip's second line
A15     Open bitcoin.org in default browser: https://bitcoin.org
PEPE    no web address is known for PEPE, so this mark is not a link
```

Of the target assets the saved fleet holds, 7 carry an organisation address and 31 carry none.

### The window and the page open the same address

**Functional.** The window reads the address off the clicked cell and opens it. The page sends the row and the column out and Python opens it, so the page never opens anything itself. Both go through the one check.

Read on the running window and on the drawn page, the same five bots:

```
what was pressed                     the window        the page
a mark whose asset resolves          the same three addresses opened
a mark whose asset resolves nothing  nothing opened    nothing opened
an insecure address in the data      refused           refused
a script address in the data         refused           refused
every mark hovered, none pressed     nothing opened    nothing opened
```

The control on the quiet rows: after each of them, one real press was made and the same instrument recorded an address, so the quiet was the product and not the reading.

### The overtaken sentence about what the table can be pressed on

**Overtaken.** *"**Functional.** The four things the operator can press on the table each send one request and redraw from the answer. The dot under a column's label toggles that column's privacy mask, a Symbol cell opens the chart address, Fire hands the bot to Manual Fire, and Detail selects the row and opens the bot. A press on the label itself does nothing."*

Five things are pressable. The dot under a column's label toggles that column's privacy mask, a mark in the first column opens its asset's own organisation, a Symbol cell opens the chart address, Fire hands the bot to

Manual Fire, and Detail selects the row and opens the bot. A press on the label itself still does nothing.

### The row heights and every column are unchanged

**Functional.** The only thing this work changed in the first column is what a press on it does. Nothing was resized.

Read at four window widths, on the window and on the page, against the state before this work and against the state before the logo column landed:

```
width   before the logo column   the current tip   this work
         window    page           window   page     window   page
900          30      29               36     36         36     36
1100         30      30               36     36         36     36
1400         30      30               36     36         36     36
1800         30      30               36     36         36     36
```

Every column width, the header's height and the mark's own size read identical between the current tip and this work, at all four widths, on both. The reading reports 0 differences there and 14 between the two earlier states, which is the row growing from 30 to 36 to fit a mark at its proper size — the change his own words asked for, and one that landed before this work.

### The removed dollar registry names where to look instead

**Functional.** Two sentences on this page cite a module a reader cannot open. Both are kept as written, and each now carries the module that holds the behaviour today. The path was removed in the commit that also wrote the first of those sentences.

Read with a search of the git index, and with the documentation check:

```
what was counted                                        figure
the removed path, occurrences on this page                   2
the check's findings for that path on this page              2
a path on this page that does exist, occurrences             4
```

The reservation path lives in `src/trading/capital_reservation.py`, which still carries its registry class and which six modules read. A bot reads another bot's claim through `reservations_for`, and a sale is decided by `effective_available`.

## 2026-09-28 - #937 - most marks were not links, and the ten records are why

A mark opened only where this repository named a web address by hand, which it does on ten crypto records. Seven of the saved fleet's target assets are among those ten, so seven marks opened and thirty-one opened nothing. The address now comes from the same coin record the mark itself came from.

### The site comes from the coin's own record, not from a hand-written row

**Functional.** The library fill already settles one coin for each ticker and reads that coin's record to find the mark. The same record names the project's own front door, and the fill now keeps that beside the mark's address. The bot list reads the kept site first and the hand-written row second, so the ten that already worked still work.

`src/exchange/crypto_assets.py` — `AssetManager.organisation_url`

```
indexed = str(
    (self.coin_index.get(name) or {}).get(COIN_INDEX_SITE_KEY) or ""
).strip()
if indexed:
    return openable_url(indexed, allowed_schemes=COIN_SITE_SCHEMES)
asset = self.get_asset(name)
return openable_url(asset.website if asset else "")
```

Read by handing the reader five index rows, one of each kind:

```
the row the index holds              what the reader answered
a kept site, https                   that site
no kept site, a hand-written row     the hand-written site
a kept site, http                    nothing, the http scheme does not open
a row with no coin settled           nothing
no row at all, a hand-written row    the hand-written site
```

### Which of the three coin addresses carries a site

**Functional.** Three addresses at the coin data source answer this platform. The list names every coin, the market records name each mark, and only the third names a site. A site therefore costs one read for each settled coin, and a site already kept for the same coin is never read twice.

`src/exchange/crypto_assets.py` — the third address

```
COIN_DETAIL_URL = "https://api.coingecko.com/api/v3/coins/{id}"
```

Read live, one coin, all three addresses:

```
address            what it answered              carries a site
/coins/list        every coin's id               no
/coins/markets     26 fields for each coin       no
/coins/{id}        a links block                 yes
```

### What the saved fleet resolves now

**Functional.** Thirty-six of the thirty-eight target assets the saved fleet holds answer a web address. The two that do not are named with the reason, and neither is guessed at.

Read by asking the reader for every target asset the saved fleet holds, before and after the fill, with a planted ticker no record holds as the control:

```
                                        before   after
target assets in the saved fleet            38      38
answered a web address                       7      36
answered nothing                            31       2
the planted ticker, nothing holds it    nothing  nothing
Bitcoin, which a record does hold        a site   a site
```

The last two rows are the control: a ticker no record holds answers nothing in both readings, and one that a record holds answers in both, so the rise from seven to thirty-six is the fill and not the reading.

The two, and why:

```
why an asset answers nothing                                  assets
several coins carry its ticker at comparable market rank           1
the coin record for it names no https web address                  1
```

Three coin records in the saved fleet name an insecure address and no secure one, so the check refuses all three. Two of the three are among the ten this repository names by hand and answer that hand-written address instead, which is why only one of them is counted above.

### A walk over part of the fleet keeps the rest of the index

**Functional.** The walk writes the tickers it asked about and keeps every other ticker the index already holds, so filling one roster never drops the rest.

`src/trading/logo_library.py` — the merge before the write

```
held.update(assets)
```

Read over a walk of 38 tickers against an index of 453:

```
rows in the index before the walk      453
tickers the walk asked about            38
rows in the index after the walk       453
rows kept that the walk never asked     415
```

### The overtaken count of crypto records carrying a site

**Overtaken.** *"Of the target assets the saved fleet holds, 7 carry an organisation address and 31 carry none."*

Thirty-six carry one and two carry none. The count of hand-written records is unchanged at ten; what changed is that a record is no longer the only source.

### The first column draws exactly as it drew

**Functional.** Nothing about the column's look changed. The same marks draw at the same size, every column keeps its width, and the row keeps its height.

Read by drawing the saved fleet at 1920 wide, at two heights, against the state before this work:

```
                         before   after
row height                   36      36
first column width          222     222
marks drawn                  38      38
the picture's fingerprint  same    same
```

## 2026-09-28 - the asset column sits on one palette ground the archetype checks

**HIS.**

> *"Column fields must match logo background color."*
>
> *"Some do not have a background color. Be sure to choose one that contrasts and makes each one pop. Use a theme-consistent color."*
>
> *"Do not want a bunch of random background colors in the Asset Column that smash together and cause an eyesore."*

The three sentences settle each other. One colour sits behind the whole column. Every mark on it reads against that one colour. A column carrying its own tint per row is the thing he refused.

### The rule that refuses a mismatched or a scattered ground

The GUI archetype now carries GUI010. It reads a module's own colour declarations and refuses three departures.

```
a ground that does not carry what sits on it   the ratio misses the floor
a ground the declared palette does not hold    no token, no published harmony
a set giving one column several grounds        the eyesore, counted
```

`dev_harness/harness/gui_archetype.py` - `_scan_column_ground_colours`

```
findings = _style_ground_faults(path, tree, known)
findings.extend(_cell_ground_faults(path, tree, known))
findings.extend(_named_ground_faults(path, tree, known))
findings.extend(_ground_set_faults(path, tree, known))
```

Every finding is high. The archetype then reads `passed=False` and the command exits 1.

### Where the palette and the floors come from

The declared palette is `src/gui/design_system.py`. Its named tokens are the grounds the rule accepts, with one stated exception: a colour standing at a published harmony separation from a token on the hue circle, at that token's lightness and chroma. The published separations are 30, 120, 150 and 180 degrees - analogous, triadic, split-complementary and complementary. A hue angle means nothing for a grey, and a near-neutral colour therefore takes no harmony.

The floors are the Web Content Accessibility Guidelines' own figures:

```
ordinary text on the ground       4.5 to 1   Contrast (Minimum), level AA
large-scale text on the ground      3 to 1   the same criterion
large-scale means                 18 point, or 14 point bold
```

The ratio comes from relative luminance, which linearises each channel before weighting it. A plain average of the raw channels returns a different number.

Two grounds are compared in CIE 1976 L*a*b*, and they count as one ground while they sit within 2.3 of each other - the just-noticeable difference Mahy, Van Eycken and Oosterlinck measured.

### What the ground rule leaves alone

A border colour. The guidelines measure a boundary against the adjacent colour, and one declaration block names the fill on one of its two sides only. A block on the disabled state, which Contrast (Minimum) exempts as an inactive component. A colour the module never declares, such as the pixels inside a logo file.

### The fixture pair the ground rule was watched fail against

```
harness_fixtures/gui_archetype/known_good_column_ground.py   exit 0
harness_fixtures/gui_archetype/known_bad_column_ground.py    exit 1
```

The good file names one ground, the value `SURFACE_2` holds, and puts two palette colours on it at 13.16 and 13.23 to 1. The bad file plants all three departures: a ground no token holds, a colour on the ground at 2.42 to 1 against a floor of 4.5, and three palette grounds down one column. The archetype reports four high findings on the bad file and none on the good one.

Read over the shipped screens, the rule reports 16 findings in 15 files under `src/gui`. One sits in the asset column's own module: the Fire button's engaged ground carries white text at 3.44 to 1.

## 2026-09-28 - the asset column's mark sits at the centre of the column

**HIS.**

> *"Live - Asset Column - Logos are not hyperlinked. Should be centered in the column. Column fields must match logo background color."*

The mark drew hard against the column's left edge. It now draws at the centre of the column, at the same size, in the same place up and down the row.

### Why centre text alignment did not move the mark

Every cell in the table takes centre text alignment. The first column's cell then takes its mark and its text is emptied. Text alignment governs text, so with no text left the mark laid out where a decoration lays out, which is the left edge.

`src/gui/widgets/bot_status_table.py` - `_draw_logo`

```
item.setIcon(icon)
item.setText("")
```

Read off the drawn table, three rows, at 1920 by 700 and again at 1920 by 900:

```
                       before      after
the cell, in pixels    x 0..219    x 0..219
the cell's centre      110.0       110.0
the mark               x 3..34     x 94..125
the mark's size        32 by 32    32 by 32
the mark's centre      19.0        110.0
off the centre         -91.0       +0.0
```

Both heights gave the same numbers. The Symbol column's own text, which is centred, measured 2.5 pixels off centre in the same drawing, so the 91 is a fact about the mark.

### What moves the mark

One field of the style option the table reads while it draws. A delegate on the first column sets that field and touches nothing else.

`src/gui/widgets/bot_status_table.py` - `CentredMarkDelegate`

```
def initStyleOption(self, option, index) -> None:
    super().initStyleOption(option, index)
    option.decorationPosition = QStyleOptionViewItem.Top
```

Two other ways were tried first and measurement refused both. Setting the decoration's alignment to centre left the mark 91 pixels off. Painting the mark by hand drew it twice, once at each place.

### The page already centred its own mark

`src/gui/web/bot_status_table.js` - `AssetLogo`

```
var DOT_MARGIN = "0 auto";
```

A block image with that margin sits at the centre of its cell. The window was the only build drawing the mark left, so this closes a difference between the two builds rather than opening one.

### Nothing else in the list moved

The whole drawn list was compared, the branch point against the branch, at both heights.

```
differing pixels                6144
their box, in pixels            x 3..125, y 1..104
the asset column                x 0..219
one image against itself           0
one planted pixel                  1
```

6144 is three rows times two squares of 32 by 32, the mark leaving one place and arriving at another. Every differing pixel sits inside the asset column, in the band the mark draws in. Row heights and every column width read the same before and after.

### The Fire button's engaged ground now carries its own text

The colour rule refused both of this column's files. The ground under white text missed the published floor.

```
token                hex       carries #ffffff   floor
STATE_ENGAGED        #2d9d5f   3.4428 to 1       4.5     misses
STATE_ENGAGED_DIM    #2d5f48   7.3891 to 1       4.5     reaches
```

`src/gui/main_tabs/bot_status_table_surface.py` - `FIRE_STYLE_FOLD_SOLID`

```
FIRE_STYLE_HEAD + f"color: {ds.TEXT_MAX}; font-weight: bold; "
f"background-color: {ds.STATE_ENGAGED_DIM}; "
f"border: 1px solid {ds.STATE_ARMED};"
```

The declared palette names that token a dimmed engaged ground. Of its 146 tokens, 77 reach the floor against white, and this one sits nearest the colour it replaces. The button keeps its bright border and its glow, which is what parts it from the button a position ceiling has stopped.

```
file                                                before   after
src/gui/main_tabs/bot_status_table_surface.py       refused  passed
src/gui/widgets/bot_status_table.py                 refused  passed
```

# Indicator Voting Panel

This panel shows the readings for each of the 12 as well the collated indices and confidence score.

TF Lock - To be re-evaluated.

The control is removed. `indicator_panel.py` builds no `_tf_lock_combo`, and `indicator_panel_surface.py` publishes no `tf_lock` block, so nothing puts a timeframe on `indicator.tf_lock_changed` any more. `main_window.py` still subscribes to that topic; with no publisher, `_lock_timeframe` keeps whatever the coordinator sets for itself.

`src/gui/indicator_panel.py` — the header, after the removal

```
header.addStretch()
header.addWidget(QLabel("Bot:"))
self._bot_selector = QComboBox()
```

The bot selector and its privacy dot close the header on the right. The pair symbol and the vote tally that stood there are gone: `_symbol_label` and `_summary_label` are not built, and the payload carries no `symbol_text` or `summary_text`.

A drop-down of eleven entries: None, then Lock below each of ten timeframes from 5m to 1w. It opens on the fifth entry, Lock below 1h. Choosing one writes a status line beside the box and puts the timeframe on the event bus.

The main window takes that event and sets `_lock_timeframe` on the coordinator of every running Scrumming Bot. Nothing declares or reads that name, so the choice reaches the coordinator and stops there. The tooltip on the box says the lock feeds directly into active Scrumming Bots. Issue #155 carries this.

`src/gui/main_window.py` — `_on_tf_lock_changed`

```
for bot in self._bot_manager._bots.values():
    if isinstance(bot, ScrummingBot) and hasattr(bot, "_coordinator"):
        bot._coordinator._lock_timeframe = tf
```

Bot - Drop down menu for selecting which bot's TA signals are displayed.

The list refills every tick and carries one entry per running bot, opening on a placeholder until one is chosen. Choosing a bot is what makes the twelve columns below read anything at all.

`src/gui/indicator_panel.py` — the selector

```
self._bot_selector = QComboBox()
self._bot_selector.setMinimumWidth(180)
self._bot_selector.addItem("(select a bot)", "")
self._bot_selector.currentIndexChanged.connect(self._on_bot_selected)
```

TF - Timeframe for the selected bot.

One row per timeframe, each row a separate verdict from the same twelve voters. The rows are the bot's own timeframes and any phantom timeframes it runs.

In development.

BB - Bollinger Bands - https://en.wikipedia.org/wiki/Bollinger_Bands

In Acervator, Bollinger Bands are used as the thresholds or boundaries at which trades are allowed to fire. Multiple market structure pieces are intertwined with our readings of the bands such as Landing Strip Detection (Heikin-Aishii Candle Consolidation Pattern) and Minimum Opposing Trade Distance. From a tactical standpoint, these represent the area or zone through which an investment position is passing.

The cell prints a direction arrow and a confidence percentage. Direction comes from the band position alone: under 0.15 or under 0.35 votes bullish, over 0.85 or over 0.65 votes bearish, and anything between the two middle figures votes neutral. Bollinger Bands carries the formula.

`src/trading/indicators/bollinger.py` — the vote

```
if bb_pos < 0.15:
    direction = SignalDirection.BULLISH
    confidence = max(0.0, min(1.0, (0.15 - bb_pos) / 0.15 * 0.8 + 0.3))
elif bb_pos > 0.85:
    direction = SignalDirection.BEARISH
    confidence = max(0.0, min(1.0, (bb_pos - 0.85) / 0.15 * 0.8 + 0.3))
elif bb_pos < 0.35:
    direction = SignalDirection.BULLISH
    confidence = 0.2
elif bb_pos > 0.65:
    direction = SignalDirection.BEARISH
    confidence = 0.2
```

VTX - Vortex Indicator - https://en.wikipedia.org/wiki/Vortex_indicator

In my experience (which is entirely subjective), this particular indicator is unusually strong at flagging early trend reversals. For Acervator, we are generally looking for signal saturation or near-saturation in either polarity which, under optimum conditions, will align with Bollinger Band thresholds being hit.

The cell prints an arrow and a percentage. The crossover of the two lines decides the vote, and then Acervator adds one reading of its own: the direction is overwritten to bullish whenever the downward line reaches 1.30, whatever the crossover said. Vortex Indicator carries the formula.

`src/trading/indicators/vortex.py` — the ceiling that overwrites the vote

```
VX_CEILING_PCT = 130.0
```

MACD - Moving Average Convergence Divergence - https://en.wikipedia.org/wiki/MACD

This is one of the oscillators used and, with it, we are looking for “top of the hill” formations in either polarity and these, under optimum conditions, will align with Bollinger Band thresholds being hit. Keep in mind that I do not approach any of these indicators from a mathematical standpoint as I am a purely visual trader without a quant or certified technical analysis background.

The cell prints an arrow and a percentage. A signal-line crossover decides the vote first. Failing that, a histogram growing away from zero votes with its own sign, and a histogram merely on one side of zero votes the same way with less conviction. MACD carries the formula.

`src/trading/indicators/macd.py` — the crossover, tested first

```
if prev_macd <= prev_signal and curr_macd > curr_signal:
    direction = SignalDirection.BULLISH
```

SRsi - Stochastic RSI - https://en.wikipedia.org/wiki/Stochastic_oscillator

Our second oscillator and, like the vortex indicator, we are looking at initial maximizing or prolonged periods of saturation of the signal at either polarity and this will, under optimum conditions, align with other indicators to trigger a trade.

The cell prints an arrow and a percentage. The vote needs a crossover of the two smoothed lines, and how deep in the range that crossover happens sets the confidence: a bullish cross under 30 scores 0.8, under 50 scores 0.5, and higher still scores less. Stochastic RSI carries the formula.

`src/trading/indicators/stochastic_rsi.py` — the bullish crossover

```
if prev_k <= prev_d and k > d:
    crossover = "bullish"
    if k < 30:
        direction = SignalDirection.BULLISH
        confidence = 0.8
```

Ichi - Ichimoku Cloud - https://en.wikipedia.org/wiki/Ichimoku_Kink%C5%8D_Hy%C5%8D

This particular indicator is one of the few ‘predicative’ indicators. In Acervator, we are focused on its additional ability to strengthen market reversal zone detection which is where we prefer to shave off or fold in funds.

The cell prints an arrow and a percentage. Several cloud readings add into one score, and only a score past 0.08 either way casts a vote. The score doubles as the confidence. Ichimoku Cloud carries the formula.

`src/trading/indicators/ichimoku.py` — the vote

```
if score > 0.08:
    direction = SignalDirection.BULLISH
    confidence = max(0.0, min(1.0, score))
elif score < -0.08:
    direction = SignalDirection.BEARISH
    confidence = max(0.0, min(1.0, abs(score)))
else:
    direction = SignalDirection.NEUTRAL
    confidence = 0.0
```

Net - Primary TF panel index collated from all 12 indicators.

The cell prints a signed figure to two decimals, green above zero and red below. It is the weighted bullish total less the weighted bearish total, and an abstaining voter adds nothing to either side. Net, Comp and Conf carries the arithmetic.

`src/gui/indicator_panel.py` — the Net cell

```
net = tf_data.get("net_score", 0)
item = QTableWidgetItem(f"{net:+.2f}")
```

Comp - Composite TF panel index provided by the highest TF Phantom Bot that is active. Phantom bots are still in active development so any related features are not yet working.

The cell prints an em dash when no composite figure is present, which is what a fleet with no active phantom shows. Net, Comp and Conf carries the arithmetic.

In development.

Conf - Confidence reading for the panel.

This is the ‘breadth’ of agreement between all 12 indicators.

The cell draws a ten-block bar and the percentage beside it. The bar fills one block per tenth. Green from 0.6, amber from 0.3, grey below that.

`src/gui/indicator_panel.py` — the Conf cell

```
conf = tf_data.get("confidence", 0)
conf_bar = "■" * int(conf * 10) + "■" * (10 - int(conf * 10))
item = QTableWidgetItem(f"{conf_bar} {conf:.0%}")
```

Sling - CM (Chris Moody) Slingshot -

https://www.tradingview.com/script/GE7tSQK1-CM-Sling-Shot-System/

Another excellent indicator for reversal detection and here it adds to our laser precision detection of such zones and structures.

The cell prints an arrow and a percentage. A squeeze breaking either way votes first, and a band snapback votes when no squeeze is breaking. Slingshot carries the formula and the attribution, which is shared between three authors rather than one.

`src/trading/indicators/slingshot.py` — the squeeze, tested first

```
if squeeze_bull:
    direction = SignalDirection.BULLISH
    confidence = squeeze_conf
    active_type = "squeeze_bull"
```

ADX - https://en.wikipedia.org/wiki/Average_directional_movement_index

This is a trend strength indicator but, as with the other indicators, we are using it to detect reversal by way of signal decreasing.

The cell prints the raw ADX value, 0 to 100, not a percentage. Below 20 the cell reads Rng and the figure, with no arrow. The vote itself is decided by which directional line is on top, at any ADX, so a ranging market still casts a directional vote even though the cell shows no arrow. ADX and DMI carries the formula.

`src/trading/indicators/adx.py` — the vote

```
if bull_dominant and strong_trend:
    direction = SignalDirection.BULLISH
    confidence = max(0.0, min(1.0, (adx - 35) / 30 + 0.5))
elif bear_dominant and strong_trend:
    direction = SignalDirection.BEARISH
    confidence = max(0.0, min(1.0, (adx - 35) / 30 + 0.5))
elif bull_dominant:
    direction = SignalDirection.BULLISH
    confidence = max(0.0, min(0.5, adx / 70))
elif bear_dominant:
    direction = SignalDirection.BEARISH
    confidence = max(0.0, min(0.5, adx / 70))
else:
    direction = SignalDirection.NEUTRAL
    confidence = 0.0
```

STrd - https://www.tradingview.com/support/solutions/43000634738-supertrend/

Supertrend is based upon Average True Range and, unsurprisingly further assists in detecting reversals.

The cell prints an arrow and a percentage. The vote is simply which side of the sticky band the price closed on, so this voter is never neutral once it reads at all. A band that has fallen to zero makes it abstain instead. Supertrend carries the formula.

`src/trading/indicators/supertrend.py` — the vote

```
direction = SignalDirection.BULLISH if curr_bull else SignalDirection.BEARISH
```

ZSc - Z Score - https://www.tradingview.com/script/KSMvIkvh-Z-Score-Predictive-Zones-AlgoPoint/

Our current indicator is the basic or classic Z Score indicator but this will be upgraded. But, as before, we are after more data to confirm reversals.

The cell prints a signed z-value to one decimal, not a percentage, and issue #316 made that value the smoothed one. Past the level the indicator remembers the vote is strong and contrarian: high votes bearish, low votes bullish. Until a turn is remembered that level is two deviations either way. Between 1.5 and 2 it votes the same way at a fixed 0.25 confidence, and inside 1.5 it casts no vote. Z-Score carries the formula.

`src/trading/indicators/zscore.py` — the thresholds

```
extreme_high = z > 3.0
strong_high = z > 2.0
mild_high = z > 1.5
extreme_low = z < -3.0
strong_low = z < -2.0
mild_low = z < -1.5
```

The cell now prints a smoothed z, and the vote compares it against levels the market itself set. The indicator remembers where the z-score last turned around, averages those turning points, and uses each average as the level that decides a vote. Until it has remembered a turn, the fixed level of two deviations decides, exactly as before. Each average is also projected back into a price, which is the upper and lower zone the hover text carries.

`src/trading/indicators/zscore.py` — the level that decides, and its price

```
target_z_high = self.reversal_threshold if peak_z is None else peak_z
target_z_low = -self.reversal_threshold if trough_z is None else trough_z

resistance_price = sma + target_z_high * std
support_price = sma + target_z_low * std
```

KER -

Kaufman Efficiency Ratio. It measures how much of the distance the price travelled became progress in one direction. A reading near 1.0 is a straight move and a reading near 0.0 is noise. A low ratio marks the market Acervator prefers. The Efficiency Ratio Regime gate refuses a SCRUM at 0.70 and above, and refuses one again at 0.05 and below.

RSI -

Relative Strength Index. It reports the share of recent movement that ran upward, on a 0 to 100 scale. Above 70 votes bearish, below 30 votes bullish, and between the two it casts no vote. It runs the same published Wilder recursion as the Stochastic RSI above, computed separately inside its own module. Its weight is 0.8, the lightest in the panel, which it shares with Volume.

## The twelve voters

Reference. The operator's own reading of each column opens this part. What follows is the code behind those columns.

**Functional.** One weight map names the twelve indicators that vote, and one method builds exactly those twelve from it. The engine builds the list once and runs it in order on every tick. "Candles needed" in the table below is the tape length each indicator wants before it will read anything. Give it less and it abstains outright

rather than guessing.

`src/trading/ta_engine.py` — `VotingEngine._create_indicators`

```
def _create_indicators(self) -> list:
    """Instantiate all indicator instances with configured weights."""
    return [
        BollingerBands(weight=self.weights.get("bollinger_bands", 1.0)),
        VortexIndicator(weight=self.weights.get("vortex", 0.9)),
        MACD(weight=self.weights.get("macd", 1.2)),
        StochasticRSI(weight=self.weights.get("stochastic_rsi", 1.0)),
        IchimokuCloud(weight=self.weights.get("ichimoku", 1.1)),
        VolumeAnalysis(weight=self.weights.get("volume", 0.8)),
        SlingshotIndicator(weight=self.weights.get("slingshot", 1.0)),
        ADXIndicator(weight=self.weights.get("adx", 1.0)),
        KaufmanERIndicator(weight=self.weights.get("kaufman_er", 1.0)),
        SupertrendIndicator(weight=self.weights.get("supertrend", 1.0)),
        ZScoreIndicator(weight=self.weights.get("zscore", 0.9)),
        RSIIndicator(weight=self.weights.get("rsi", 0.8)),
    ]
```

**Design intention.** The weights rank the voters by how much they are trusted. MACD is heaviest at 1.2 and Ichimoku next at 1.1. RSI and Volume are lightest at 0.8, RSI because it overlaps the Stochastic RSI and Volume because it is meant to confirm rather than lead. Hand the engine a different map and the panel retunes without an indicator being touched.

`src/trading/ta_engine.py` — `VotingEngine.__init__`

```
self.weights = weights or DEFAULT_WEIGHTS.copy()
self._indicators = self._create_indicators()
```

| Panel column | Voter | Weight | Module under `src/trading/indicators/` | Candles needed |
|---|---|---|---|---|
| BB | `bollinger_bands` | 1.0 | `bollinger.py` | 20 |
| VTX | `vortex` | 0.9 | `vortex.py` | 15 |
| MACD | `macd` | 1.2 | `macd.py` | 35 |
| SRsi | `stochastic_rsi` | 1.0 | `stochastic_rsi.py` | 36 |
| Ichi | `ichimoku` | 1.1 | `ichimoku.py` | 79 |
| Vol | `volume` | 0.8 | `volume.py` | 25 |
| Sling | `slingshot` | 1.0 | `slingshot.py` | 52 |
| ADX | `adx` | 1.0 | `adx.py` | 30 |
| STrd | `supertrend` | 1.0 | `supertrend.py` | 12 |
| ZSc | `zscore` | 0.9 | `zscore.py` | 51 |
| KER | `kaufman_er` | 1.0 | `kaufman_er.py` | 12 |
| RSI | `rsi` | 0.8 | `rsi.py` | 15 |

**Functional.** Count the files in the indicator folder and you get 23, not 12. Eleven of them cast no vote. Two carry the shared maths and the signal vocabulary. Three are measurements other code reads. Six are

structural detectors that feed the bot's own pattern reads rather than the panel. Twelve is the count of voters, and twelve is the right count.

```
shared        helpers.py, types.py
measurements  atr.py, bb_proximity.py, heikin_ashi.py
detectors     fvg.py, landing_strip.py, m_top.py, w_bottom.py,
              spring.py, macd_taper.py
```

**Design intention.** One module holds one indicator, and a module that casts no vote still lives here so the voters can read it. Nothing borrows another indicator's arithmetic, and a test in `tests/` fails the moment one starts to.

`tests/test_one_indicator_per_module.py` — the rule, held as tests

```
def test_the_module_holds_exactly_one(self, module):
```

`tests/test_one_indicator_per_module.py` — and the name check beside it

```
def test_no_two_modules_define_the_same_name(self):
```

## Indicator formulae

Each block below carries the published formula for reference, then the code that computes it, then what the code does today and what the reading is meant to give the platform. The operator's own reading of each indicator opens this part and is not repeated here.

Every indicator computes its own maths from candles alone. No indicator borrows another's arithmetic, and `tests/test_one_indicator_per_module.py` fails when one starts to.

Where the code departs from the published formula, the code is the defect. Departures lists the ones found.

### Shared terms

Every formula below uses these. All five live in `src/trading/indicators/helpers.py`.

```
SMA(v, n)     mean of the last n values                          _sma_tail
sigma(v, n)   population deviation of the last n values          _stdev_tail
EMA(v, n)     seed = SMA(v, n) at index n-1, then                _ema
              EMA_t = (v_t - EMA_prev) * k + EMA_prev
              k = 2 / (n + 1)
              no value exists before the seed
TR_t          max(H_t - L_t, |H_t - C_prev|, |L_t - C_prev|)     _true_range
              the first bar's TR is H - L
Wilder(v, n)  seed = mean of the first n, then                   (per module)
              W_t = (W_prev * (n - 1) + v_t) / n
```

**Functional.** True Range gives one bar per candle. The first bar is simply high minus low, because the other two ways of measuring it need a previous close and the first bar has none. Each indicator then takes the slice its own formula covers. ATR and Supertrend take the whole series. ADX and Vortex drop the first bar, because directional movement needs a bar before it. The moving average helper answers nothing at all below its seed,

rather than zero, so reading it too early fails loudly instead of returning a plausible wrong number.

`src/trading/indicators/helpers.py` — `_true_range`

```
tr = [candles[0].high - candles[0].low]
for i in range(1, len(candles)):
    c = candles[i]
    prev_close = candles[i - 1].close
    tr.append(
        max(c.high - c.low, abs(c.high - prev_close), abs(c.low - prev_close))
    )
return tr
```

**Design intention.** One definition per shared term, held in one place, so five indicators cannot quietly drift apart on the same arithmetic. Slingshot is the one deliberate exception. It computes its own true range, and its first bar agrees with this one.

`src/trading/indicators/slingshot.py` — the exception, written out

```
# Carter's Keltner leg needs True Range; `tr_all` computes it
# without `ATRIndicator`.
tr_all = [candles[0].high - candles[0].low]
```

### Bollinger Bands

Published formula, for reference:

```
Middle    = SMA(close, 20)
Upper     = Middle + 2.0 * sigma(close, 20)
Lower     = Middle - 2.0 * sigma(close, 20)
%B        = (close - Lower) / (Upper - Lower)
BandWidth = (Upper - Lower) / Middle
```

**Functional.** The vote is a reading of where the last close sits between the two bands. Zero is the lower band, one is the upper, and the panel and the gates both call that number the band position. A second method returns all three bands for every candle so a chart can draw them, and answers nothing for the first nineteen. Give it fewer than twenty closes, or a window with no range at all, and it abstains.

`src/trading/indicators/bollinger.py` — `BollingerBands.compute`

```
sma = _sma_tail(closes, self.period, tail=self.period)
std = _stdev_tail(closes, self.period, tail=self.period)

mid = sma[-1]
upper = mid + self.std_dev * std[-1]
lower = mid - self.std_dev * std[-1]
price = closes[-1]
# BandWidth = (Upper - Lower) / Middle. `candles_from_raw` admits
# only positive closes, so `mid` cannot be zero.
band_width = (upper - lower) / mid
```

**Design intention.** The bands are the boundary a trade fires at. That is the operator's own reading, and the band position is the one number that carries it into the engine. Four gates read it directly and both overrides arm on it, so this reading has more reach into a trade decision than any other voter.

`src/trading/gate_chain.py` — two of the four, in the scrum chain

```
MidlineGate(side="scrum"),
TargetFiresGate(),
BBProximityGate(side="scrum"),
```

`src/trading/gate_chain.py` — and the other two, in the fold chain

```
MidlineGate(side="fold"),
SmartCeilingGate(),
BBProximityGate(side="fold"),
```

### Vortex Indicator

Published formula, for reference:

```
VM+_t = |H_t - L_(t-1)|
VM-_t = |L_t - H_(t-1)|
VI+   = sum(VM+, 14) / sum(TR, 14)
VI-   = sum(VM-, 14) / sum(TR, 14)
```

**Functional.** Three totals are taken over each fourteen-bar window: upward movement, downward movement, and true range. A window whose true range totals zero, which is what a halted market gives, is skipped rather than divided by. The two movement totals are then divided by the true-range total, so both lines are dimensionless and 1.0 is dead parity. The vote reads the crossover of the two lines.

`src/trading/indicators/vortex.py` — `VortexIndicator.window_sums`

```
for i in range(1, n_c):
    vm_plus.append(abs(candles[i].high - candles[i - 1].low))
    vm_minus.append(abs(candles[i].low - candles[i - 1].high))
```

`src/trading/indicators/vortex.py` — `VortexIndicator.lines`

```
plus_series[i] = sum_vp / sum_tr_window
minus_series[i] = sum_vm / sum_tr_window
```

**Design intention.** The operator uses the Vortex to catch a reversal early, and watches for the signal to saturate at one polarity or the other. Acervator adds one reading of its own on top of the published lines: the vote is overwritten to bullish whenever the downward line reaches 1.30, after the crossover has already decided. That overwrite is not part of the indicator, and Departures records it.

In development. The overwrite belongs to issue #414, which owns every indicator departure, and no replacement reading has been chosen.

### MACD

Published formula, for reference:

```
MACD Line = EMA(close, 12) - EMA(close, 26)
Signal    = EMA(MACD Line, 9)
Histogram = MACD Line - Signal
```

**Functional.** One method builds all three series and the vote reads the last two bars of them. What you are looking at is the gap between a fast and a slow average of the price, and how quickly that gap is changing. The signal line is an average of the MACD line rather than of the candles, so its first real value does not arrive until 33 bars into the tape. A series with a hole in the middle raises rather than being handed on.

`src/trading/indicators/macd.py` — `MACD._lines`

```
ema_fast = _ema(closes, self.fast)
ema_slow = _ema(closes, self.slow)
macd_line: _Line = [
    None if (f is None or s is None) else f - s
    for f, s in zip(ema_fast, ema_slow)
]
```

`src/trading/indicators/macd.py` — the signal line and the histogram

```
signal_line: _Line = [None] * first
signal_line.extend(_ema(real, self.signal_period))
histogram: _Line = [
    None if (m is None or g is None) else m - g
    for m, g in zip(macd_line, signal_line)
]
```

**Design intention.** The operator watches for a top-of-the-hill shape in either polarity, timed against a Bollinger band threshold. The histogram is the series that shape reads from, and the vote reads the last two of its bars, so a turn is caught on the bar it happens rather than a bar later.

`src/trading/indicators/macd.py` — the two bars the vote reads

```
macd_line, signal_line, histogram = self._lines(closes)

curr_hist = histogram[-1]
prev_hist = histogram[-2]
curr_macd = macd_line[-1]
```

The confidence divides the histogram by a share of the last close, so the same crossover scores the same on a four-figure asset and on one worth a few millionths of a dollar.

### Stochastic RSI

Published formula, for reference:

```
StochRSI = (RSI - min(RSI, 14)) / (max(RSI, 14) - min(RSI, 14))
%K       = SMA(StochRSI * 100, 3)
%D       = SMA(%K, 3)
```

**Functional.** The module runs Wilder's recursion over the closes to build its own RSI series, then places each RSI value inside its own fourteen-value range. The vote reads the crossover of the two smoothed lines. The result saturates at 0 and at 100 far more often than plain RSI does, and that saturation is what the panel is watching for. A window whose RSI never moved is caught before the division, and the vote abstains on it rather than dividing by that spread.

`src/trading/indicators/stochastic_rsi.py` — `StochasticRSI.rsi_values`

```
avg_gain = (avg_gain * (self.rsi_period - 1) + gains[i]) / self.rsi_period
avg_loss = (avg_loss * (self.rsi_period - 1) + losses[i]) / self.rsi_period
```

`src/trading/indicators/stochastic_rsi.py` — `StochasticRSI.stoch_ratios`

```
window = rsi_values[i - self.stoch_period + 1 : i + 1]
low = min(window)
ratios.append((rsi_values[i] - low) / (max(window) - low))
```

**Design intention.** This is the operator's second oscillator, and he looks for a long run at one polarity to line up with the other voters. Rescaling RSI against its own recent range is what makes those runs stand out at all. A window that never moved is reported as flat rather than hidden, so the caller can tell a real zero from a market that stood still.

`src/trading/indicators/stochastic_rsi.py` — what the caller is handed

```
return start, ratios, flat
```

### Ichimoku Cloud

Published formula, for reference:

```
Tenkan   = (highest high 9  + lowest low 9)  / 2
Kijun    = (highest high 26 + lowest low 26) / 2
Senkou A = (Tenkan + Kijun) / 2        plotted 26 bars forward
Senkou B = (highest high 52 + lowest low 52) / 2   plotted 26 forward
Chikou   = close                       plotted 26 bars back
```

**Functional.** Every line here is a midpoint: the highest high and the lowest low of a window, averaged. A window that has not closed yet answers nothing rather than zero. Five values come back for each candle. Two of them draw the cloud twenty-six bars ahead of the last candle, and that is the only forward-looking reading anywhere in the panel. The lines are returned at their own bar, so the twenty-six bar shift belongs to whatever draws them.

`src/trading/indicators/ichimoku.py` — `IchimokuCloud.lines`

```
tenkan = self._span_mid(candles, i, self.tenkan)
kijun = self._span_mid(candles, i, self.kijun)
senkou_b = self._span_mid(candles, i, self.senkou_b)
senkou_a = (
    (tenkan + kijun) / 2.0
    if (tenkan is not None and kijun is not None)
    else None
)
out.append((tenkan, kijun, senkou_a, senkou_b, candles[i].close))
```

**Design intention.** The operator calls this one of the few predictive indicators and uses it to sharpen reversal-zone detection. The forward cloud is the half of the indicator that does that work. A window that has not closed answers nothing rather than zero, so an unfinished cloud can never be mistaken for a flat one.

`src/trading/indicators/ichimoku.py` — `IchimokuCloud._span_mid`

```
if end < period - 1:
    return None
return self._mid(candles, end - period + 1, period)
```

The cloud thickness divides by the price bare. Every candle reaching this point has already been refused unless its price is above zero, so the division needs no guard of its own.

### Volume

Published formulae, for reference:

```
OBV_t  = OBV_prev + V_t   if C_t > C_(t-1)
         OBV_prev - V_t   if C_t < C_(t-1)
         OBV_prev         otherwise
TP_t   = (H_t + L_t + C_t) / 3
MF_t   = TP_t * V_t
MFI    = 100 - 100 / (1 + sum(MF where TP rose, 14)
                         / sum(MF where TP fell, 14))
CLV_t  = ((C_t - L_t) - (H_t - C_t)) / (H_t - L_t)
CMF    = sum(CLV * V, 20) / sum(V, 20)
A/D_t  = A/D_prev + CLV_t * V_t
```

**Functional.** The Volume column is the one voter the operator does not describe in his list above, so this is its only description in the manual. Three published measures are computed separately and scored together. The vote answers one question: does the volume traded agree with the direction the price moved. A price low that On-Balance Volume refuses to match is the divergence this module scores highest. Where neither money-flow bucket was ever credited, the Money Flow Index answers 50 rather than 100, so a market that never moved does not read as maximum overbought.

`src/trading/indicators/volume.py` — `VolumeAnalysis._obv`

```
obv = [0.0]
for i in range(1, len(candles)):
    if candles[i].close > candles[i - 1].close:
        obv.append(obv[-1] + candles[i].volume)
    elif candles[i].close < candles[i - 1].close:
        obv.append(obv[-1] - candles[i].volume)
    else:
        obv.append(obv[-1])
```

`src/trading/indicators/volume.py` — `VolumeAnalysis._mfi`

```
if pos_mf <= 0.0 and neg_mf <= 0.0:
    return 50.0
if neg_mf <= 0.0:
    return 100.0
return 100.0 - 100.0 / (1.0 + pos_mf / neg_mf)
```

**Design intention.** Volume confirms or contradicts the other eleven. It does not lead them. Its weight is 0.8, the lightest in the panel alongside RSI, and that number is the intention written down.

`src/trading/ta_engine.py` — the weight that says so

```
"volume": 0.8,
```

Both tests in the block above compare a money flow against zero. Money flow is a price times a volume and carries no fixed scale, so only an exact zero can stand as the test.

### Slingshot

Published formula, for reference:

```
UpperBB = SMA(close, 20) + 2.0 * sigma(close, 20)
LowerBB = SMA(close, 20) - 2.0 * sigma(close, 20)
UpperKC = SMA(close, 20) + 1.5 * SMA(TR, 20)
LowerKC = SMA(close, 20) - 1.5 * SMA(TR, 20)
sqzOn   = (LowerBB > LowerKC) and (UpperBB < UpperKC)
delta_t = C_t - (donchian_mid_20 + SMA(close, 20)) / 2
val     = linreg(delta, 20, 0)
```

**Functional.** The squeeze is on while the Bollinger band sits entirely inside the Keltner Channel. When it opens again, a fitted momentum line gives the sign of the release, and that sign is the vote. The module computes its own true range rather than calling the shared helper, and its first bar agrees with the shared one anyway.

`src/trading/indicators/slingshot.py` — `SlingshotIndicator.compute`

```
rangema = trma_v[i] if trma_v[i] is not None else 0.0
up_kc = mid + rangema * self.kc_mult
lo_kc = mid - rangema * self.kc_mult
sqz_on = (lo > lo_kc) and (up < up_kc)
seg = [
    deltas[k] for k in range(max(delta_lo, i - self.bb_period + 1), i + 1)
]
val = self._linreg_endpoint(seg)
```

**Design intention.** The operator wants a precise reading of a reversal zone, and he names Chris Moody's Slingshot system as the source. The code does not implement that system. What is in the module is John Carter's TTM Squeeze, with Bollinger's own rules 6 and 8 for the snapback. Moody's method is an EMA trend and pullback with no band, no channel and no squeeze at all. The module says so itself, in its own docstring. The column keeps the name he gave it, and the name and the code are two different indicators.

`src/trading/indicators/slingshot.py` — the module stating it

```
# LazyBear's KC shares the BB basis; rangema is the SMA of
# True Range over the same period.
```

In development. Which of the two indicators the column should hold is the operator's decision, not a coding one, so nothing is proposed here.

### ADX and DMI

Published formula, for reference:

```
up_t   = H_t - H_(t-1)
down_t = L_(t-1) - L_t
+DM_t  = up_t   if up_t > down_t and up_t > 0     else 0
-DM_t  = down_t if down_t > up_t and down_t > 0   else 0
+DI    = 100 * Wilder(+DM, 14) / Wilder(TR, 14)
-DI    = 100 * Wilder(-DM, 14) / Wilder(TR, 14)
DX     = 100 * |+DI - -DI| / (+DI + -DI)
ADX    = Wilder(DX, 14)
```

**Functional.** One smoothing recursion carries every line in this indicator. The reading tells you how committed a move is, and says nothing about which way it points. Below 20 the market is ranging. Above 35 it is trending hard. The indicator abstains when its smoothed true range is not above zero, so a market that moved at all still gets a vote however little the asset costs.

`src/trading/indicators/adx.py` — `ADXIndicator._wilder_smooth`

```
if len(values) < period:
    return [0.0] * len(values)
result = [0.0] * (period - 1)
result.append(sum(values[:period]) / period)
for v in values[period:]:
    result.append((result[-1] * (period - 1) + v) / period)
return result
```

**Design intention.** The operator reads ADX for trend strength, and he watches it fall rather than rise, because a falling reading marks a trend running out of road. The engine carries the other half of that: a strong trend holds a scrum back rather than letting it fire into a move that is still running.

`src/trading/gate_chain.py` — `ADXTrendSuppressionGate.evaluate`

```
if ctx.adx < self.adx_threshold:
    return GateResult(passed=True)
```

Issue #414 owns the abstain threshold and no correction is proposed here.

### Supertrend

Published formula, for reference:

```
ATR     = Wilder(TR, 10)
hl2_t   = (H_t + L_t) / 2
rawUp   = hl2 + 3.0 * ATR
rawLow  = hl2 - 3.0 * ATR
Upper_t = rawUp  if rawUp < Upper_prev  or C_(t-1) > Upper_prev  else Upper_prev
Lower_t = rawLow if rawLow > Lower_prev or C_(t-1) < Lower_prev  else Lower_prev
bullish while C_t >= Lower_t; turns bullish again when C_t > Upper_t
```

**Functional.** A band here is sticky. It moves toward the price and refuses to move away from it, unless the price closed past it last bar. What comes out is a line that trails under the price in an uptrend and over it in a downtrend, and the bar where price crosses that line is the flip.

`src/trading/indicators/supertrend.py` — `SupertrendIndicator.compute`

```
final_ub = (
    raw_ub
    if raw_ub < prev_ub or candles[i - 1].close > prev_ub
    else prev_ub
)
final_lb = (
    raw_lb
    if raw_lb > prev_lb or candles[i - 1].close < prev_lb
    else prev_lb
)
```

**Design intention.** Supertrend rests on Average True Range and helps the operator spot a reversal. The flip is the event that does that work, and the line either side of it is context. The bullish test and the bearish test differ on purpose, so a trend has to be broken outright before the line turns.

`src/trading/indicators/supertrend.py` — the asymmetric flip test

```
prev_bull = st[-1]
if prev_bull:
    curr_bull = candles[i].close >= final_lb
else:
    curr_bull = candles[i].close > final_ub
```

The band this indicator measures distance from is derived rather than read off the tape, and it can genuinely reach zero once the average true range passes the midpoint over the multiplier. The indicator abstains outright on that bar instead of dividing by it.

`src/trading/indicators/supertrend.py` — the abstention

```
if st_line <= 0.0:
    return Signal(
        "supertrend",
        timeframe,
        SignalDirection.NEUTRAL,
        0.0,
        self.weight,
        abstained=True,
    )
```

### Z-Score

Published formula, for reference:

```
mean  = SMA(close, 50)
sigma = population deviation of the same 50 closes
Z     = (close - mean) / sigma
```

**Functional.** The reading is how many standard deviations the price sits from its own fifty-bar mean, and the vote reads the extremes of that. Fifty bars is longer than the twenty the Bollinger Bands use, so a stretch those bands have already absorbed still shows up here. A window with almost no spread at all abstains rather than dividing by it.

`src/trading/indicators/zscore.py` — `ZScoreIndicator.compute`

```
closes = [c.close for c in candles[-self.period :]]
sma = sum(closes) / self.period
variance = sum((c - sma) ** 2 for c in closes) / self.period
std = variance**0.5
```

**Design intention.** The operator calls this the classic Z-Score, asked for the upgrade issue #316 landed, and uses it to confirm a reversal. The longer window is why it earns a column of its own beside the Bollinger Bands.

One formula, written once, for both bars. The current bar and the previous bar each divide by their own deviation bare, and a guard above each refuses a deviation under the same threshold before the division runs.

`src/trading/indicators/zscore.py` — both divisions

```
z = (candles[-1].close - sma) / std
```

```
z_prev = (candles[-2].close - s2) / std2
```

**Published method.** The source the operator named publishes two formulae. The first is the classic score above. The second turns a score back into a price, so a level the market has reversed at can be drawn as a zone that widens and narrows with the deviation.

```
Projected Price = Mean + (Target Z * Standard Deviation)
```

The same page names the noise reduction: the raw score is smoothed by a volume weighted average before anything reads it. Turning points are then found in that smoothed series, kept only when they reach a minimum score, and the last few of each side are averaged into the two levels the vote uses.

Both bars now read one window through one expression, so the two spellings that disagreed cannot come back. A window whose prices never moved returns nothing at all, which is how the reading abstains rather than shifting.

`src/trading/indicators/zscore.py` — one window, one expression, every reader

```
def _window_zscore(closes: list, close: float) -> Optional[tuple]:
    if _window_has_no_range(closes):
        return None
    count = len(closes)
    mean = sum(closes) / count
    deviation = (sum((c - mean) ** 2 for c in closes) / count) ** 0.5
    return mean, deviation, (close - mean) / deviation
```

**What the page does not publish.** It names four settings and gives a default value for none of them. Three take a number this file already used or the page itself quotes, and one is chosen here and marked as such.

| Setting | Value | Where the value comes from |
|---|---|---|
| Z-Score Length | 50 | the period this file already used |
| Reversal Threshold | 2.0 | the strong level this file already used, and the level the page quotes as the classic one |
| Pivot bars | 1 | a turn against the bar each side; the page gives noise reduction to the smoothing instead |
| Lookback Depth | 5 | chosen here, because the page publishes no value and this file had none |

### Kaufman Efficiency Ratio

Published formula, for reference:

```
ER = |C_t - C_(t-10)| / sum(|C_i - C_(i-1)|, 10)
```

**Functional.** Net displacement divided by the distance travelled to get there. 1.0 is a straight line, 0.0 is pure noise, and both halves read the same eleven closes. A window where the price never moved travels exactly zero distance, and the indicator abstains on that rather than reporting a regime it never measured.

`src/trading/indicators/kaufman_er.py` — `KaufmanERIndicator.compute`

```
closes = [c.close for c in candles[-(self.period + 1) :]]
net_change = abs(closes[-1] - closes[0])
price_travel = sum(
    abs(closes[i] - closes[i - 1]) for i in range(1, len(closes))
)
```

**Design intention.** A low ratio marks the market Acervator prefers, in the operator's own words above. The engine acts on that at both ends: a scrum is refused when the market is running too straight to mean-revert, and refused again when it is too noisy to hold an edge.

The division that produces the ratio is bare. A guard four lines above returns early when the distance travelled is not above zero, so the denominator is already proved positive by the time the division runs.

`src/trading/indicators/kaufman_er.py` — the division

```
er = net_change / price_travel
```

### RSI

Published formula, for reference:

```
gain_t  = max(C_t - C_(t-1), 0)
loss_t  = max(C_(t-1) - C_t, 0)
AvgGain = Wilder(gain, 14)
AvgLoss = Wilder(loss, 14)
RS      = AvgGain / AvgLoss
RSI     = 100 - 100 / (1 + RS)

AvgLoss = 0 and AvgGain > 0   ->  RSI 100
AvgLoss = 0 and AvgGain = 0   ->  no reading, the indicator abstains
```

**Functional.** The reading is the share of recent movement that ran upward, on a 0 to 100 scale. Above 70 votes bearish, below 30 votes bullish, and anything between casts no vote. A bar with neither a gain nor a loss abstains outright.

`src/trading/indicators/rsi.py` — `RSIIndicator.compute`

```
if rsi > 70:
    direction = SignalDirection.BEARISH
elif rsi < 30:
    direction = SignalDirection.BULLISH
else:
    direction = SignalDirection.NEUTRAL
```

**Design intention.** RSI is the plainest voter in the panel, and it overlaps the Stochastic RSI on purpose. Its weight is 0.8, which stops the same evidence being counted twice at full strength.

One number is read two ways here. The value is rounded to two decimals before the vote tests it, while the overbought and oversold flags in the same dictionary test the unrounded number. An RSI of 70.003 therefore sets overbought and still votes neutral.

`src/trading/indicators/rsi.py` — the rounding today

```
return {
    "rsi": round(rsi, 2),
    "rs_indeterminate": rs_indeterminate,
    "overbought": rsi > 70,
    "oversold": rsi < 30,
```

*Proposed, not present:*

```
return {
    "rsi": rsi,
    "rs_indeterminate": rs_indeterminate,
    "overbought": rsi > 70,
    "oversold": rsi < 30,
```

The rounding belongs to display, not to a comparison. Issue #414 carries this.

## Departures from the published maths

**Functional.** Two departures remain, and neither is a matter of scale. Every voter now reads the same on an asset worth four figures and on one worth a few millionths of a dollar, because no division and no threshold in the twelve carries an absolute constant against a quantity measured in the asset's own price units. Where a denominator can genuinely reach zero the indicator abstains on that bar rather than adding a constant to it.

- The Vortex overwrites its own direction to BULLISH whenever the downward line reaches 1.30, after the crossover has already decided. The two lines themselves match Botes and Siepman exactly. The overwrite is a reading Acervator adds on top of them.
- The RSI is rounded to two decimals before the vote tests it against 70 and 30, while the overbought and oversold flags beside it test the unrounded number. An RSI of 70.003 sets overbought and still votes NEUTRAL.

Supertrend is the one place an abstention stands in for a division. Its lower band is derived from the average true range rather than read off the tape, so it can reach zero on a real market, and the indicator refuses to vote on that bar.

**Design intention.** An indicator has to read the same on every asset, because one threshold serves the whole fleet. Four test files hold that property for the twelve, one shape at a time, by running the same tape at several price scales and comparing the readings.

```
tests/test_bollinger_and_slingshot_are_scale_invariant.py
tests/test_vortex_and_landing_strip_are_scale_invariant.py
tests/test_macd_and_kaufman_er_are_scale_invariant.py
tests/test_the_last_eight_indicators_are_scale_invariant.py
```

Issue #414 owns the two readings above. Both are deliberate choices in Acervator's own reading of a published indicator, not arithmetic errors, and neither has a replacement chosen yet.

In development.

## 2026-09-08 15:12 - #414 - the ratio the gate reads

The published Efficiency Ratio is Kaufman's, unchanged here: the distance between the first and last close of the window, divided by the distance the price travelled to cover it. The Kaufman entry above states it and the code computes it.

```
ER = |C_t - C_(t-10)| / sum(|C_i - C_(i-1)|, 10)
```

The indicator gave three other parts of the platform that ratio rounded to four decimal places. One of them is the Efficiency Ratio Regime gate, which refuses a scrum at 0.05 and below and at 0.70 and above, so the gate compared a shortened number against its two thresholds. The indicator now publishes the ratio it computed.

`src/trading/indicators/kaufman_er.py` — what the reading carries

```
details={
    "er": er,
    "er_prev": er_prev,
```

Three parts of the platform read that number and each builds the same gate input from it: the live bot's tick, the Simulator's back test, and the ATA gate scan. Nothing on any screen changes, because the panel cell, the hover text and the ATA-SMP sentence each cut the number to two, three and four decimals of their own.

`src/trading/scrumming_bot.py` — the gate input, one of the three

```
efficiency_ratio=_extract_signal_detail(summary, "kaufman_er", "er", 0.0),
```

Measured over 71 recorded Coinbase tapes, run at a price of three millionths of a dollar and again at a four-figure price. The run drove 37,380 readings through the real voting engine and the real scrum chain. The gate changed its mind on five of them, in both directions: three where the rounding had pulled a ratio just above 0.05 down onto the threshold, and two where it had pulled a ratio near zero onto the sentinel that tells the gate no reading arrived.

```
A22 2025, bar 264, both price scales
    before   gate reads 0.05                  scrum blocked
    after    gate reads 0.05001050640890935   scrum allowed

A22 2025, bar 263, both price scales
    before   gate reads 0.2257                scrum allowed
    after    gate reads 0.2256565656565654    scrum allowed

A14 2025, bar 233, both price scales
    before   gate reads 0.0                   scrum allowed
    after    gate reads 2.983774235682565e-05 scrum blocked
```

The same run read the other eleven voters back over one tape at 88 bars of each price scale, and every reading is identical, which is what a repair to one indicator has to look like.

## 2026-09-08 15:54 - #414 - when ADX first has something to say

Wilder's ADX needs two runs of bars before it exists. The first is fourteen bars of true range and directional movement, which seed the smoothing. The second is fourteen DX values, which the ADX is the average of. Twenty-eight bars, and not one more. The ADX and DMI entry above states the formula and the code computes it.

```
+DI    = 100 * Wilder(+DM, 14) / Wilder(TR, 14)
-DI    = 100 * Wilder(-DM, 14) / Wilder(TR, 14)
DX     = 100 * |+DI - -DI| / (+DI + -DI)
ADX    = Wilder(DX, 14)          first value at bar 28
```

The indicator refused to vote until thirty bars had arrived. The extra two came from the first upload and never had a reason. On those two bars the market had moved, the ADX existed, and the panel counted eleven voters instead of twelve.

`src/trading/indicators/adx.py` — when the indicator speaks

```
n = len(candles)
# period bars seed the smoothed TR, and period DX values seed the ADX.
if n < self.period * 2:
```

The bar the ADX first exists on has no earlier ADX to compare against. The module read a zero from the padding the smoothing writes ahead of its first real value, so the rising flag on that bar was decided against a number that was never a reading. The readings now start where the padding ends.

`src/trading/indicators/adx.py` — the previous reading

```
adx_values = s_dx[self.period - 1 :]
adx = adx_values[-1] if adx_values else 0.0
p_adx = adx_values[-2] if len(adx_values) >= 2 else adx
```

The indicator also gave every reader its ADX cut to two decimal places, and the directional lines with it. The ADX Trend Suppression gate is one of those readers and compares the number against 30.0. The indicator now publishes what it computed.

`src/trading/indicators/adx.py` — what the reading carries

```
details={
    "adx": adx,
    "di_plus": di_plus,
    "di_minus": di_minus,
```

Eight parts of the platform read those three numbers. Three build the same gate input: the live bot's tick, the Simulator's back test and the ATA gate scan. Five display it and each cuts it themselves, so the panel cell still prints a whole number and the ATA-SMP sentence still prints two decimals.

| reader | what it shows |
|---|---|
| `scrumming_bot.py` gate input | the full reading, `23.968874676972366` |

| reader | what it shows |
|---|---|
| `back_test.py` gate input | the full reading |
| `ata_gate_scan.py` gate input | the full reading |
| `ata_spm.py` sentence | `trend strength ADX 23.97`, unchanged |
| `indicator_panel.py` cell | `24`, unchanged |
| `indicator_panel_surface.py` cell | `24`, unchanged |
| `indicator_panel_surface.py` hover | `23.969`, was `23.970` |
| `snapshots.py` bot log line | `(adx 23.9689)`, was `(adx 23.97)` |

The bot log printed whatever the indicator published, so it now cuts a decimal reading to four places of its own.

`src/trading/scrumming/snapshots.py` — the bot log cell

```
reading = cell.get("detail")
if reading is not None:
    shown = f"{reading:.4f}" if isinstance(reading, float) else reading
```

Measured over 71 recorded Coinbase tapes, run at a price of three millionths of a dollar and again at a four-figure price. The run drove 42,140 readings through the real voting engine and the real scrum chain, from a window of twenty-six bars upward.

```
abstentions   before 280 per price scale    after 140 per price scale
readings that moved
    the two recovered bars           280
    the published decimal places  41,580
scrum verdict changed
    the two recovered bars           136
    the published decimal places       0
```

Every abstention that remains is a window under twenty-eight bars, where Wilder's ADX does not exist. Every one that ended is a window of twenty-eight or twenty-nine bars, where it did.

```
A34 2021, window 27 bars, both price scales
    before   abstained     13 DX values, one short of the fourteen
    after    abstained     the same reason

A34 2021, window 28 bars, both price scales
    before   abstained     the guard asked for 30 bars
    after    ADX 21.847870255087297  bullish at 0.3121

A34 2023, window 28 bars, both price scales
    before   gate reads 0.0                 scrum allowed
    after    gate reads 74.20924187088121   scrum blocked
```

The last row is what the repair changes about a live reading. A bot that has just started, or one whose exchange returned a short window, spent two bars firing scrums into a trend the ADX could already measure. The panel felt it too: on the A34 2021 bar the twelfth voter arriving moved the net score from 0.7437 to 1.0558 and the consensus confidence from 0.1352 to 0.1624, and across the sweep the consensus direction itself changed on

58 readings.

The reading carries no price scale. Every number above is the same at three millionths of a dollar and at a four-figure price, to the last few bits.

The same run read the other eleven voters back over one tape at 88 bars of each price scale, and every reading is identical.

## 2026-09-08 16:22 - #414 - the MACD reading at a millionth of a dollar

Appel's three numbers are unchanged and the MACD entry above states them. This repair touches what the indicator hands its readers, not how the lines are built.

```
MACD Line = EMA(close, 12) - EMA(close, 26)
Signal    = EMA(MACD Line, 9)
Histogram = MACD Line - Signal
```

All three are prices. The indicator published them cut to six decimal places, which is a fixed step of one millionth of a dollar. A market worth three millionths of a dollar has a whole MACD smaller than that step, so every reader was handed a zero. The same market shape at a four-figure price published real numbers. One threshold serves the whole fleet, so one shape has to read the same on both.

`src/trading/indicators/macd.py` — what the reading carries

```
details={
    "macd_line": curr_macd,
    "signal_line": curr_signal,
    "histogram": curr_hist,
```

Measured over 71 recorded Coinbase tapes, run at a price of three millionths of a dollar and again at a four-figure price. The run drove 3,750 readings through the real voting engine and both real gate chains, from a window of thirty-six bars upward.

```
readings publishing three zeros
    three millionths   before 1,834 of 1,875   after 0 of 1,875
    four figures       before     0 of 1,875   after 0 of 1,875
the MACD line over the histogram, the two scales agreeing
    before        0 of 1,875 pairs
    after     1,875 of 1,875 pairs
readings that moved             3,750 of 3,750
scrum and fold verdicts changed     0 of 3,750
```

One bar, at both prices, before and after:

```
A34 2021, bar 234, a bullish crossover
    before   three millionths   histogram  0.0
             four figures       histogram  1.651648
    after    three millionths   histogram  4.149197409777354e-09
             four figures       histogram  1.6516482592467057
the MACD line over the histogram
    before   three millionths   no value, the division is by zero
             four figures       -20.5541919344
    after    three millionths   -20.5541889248
             four figures       -20.5541889248
```

The confidence never carried a price scale and does not change here. It divides the histogram by a share of the last close, a price over a price, and the two shares now have names of their own.

`src/trading/indicators/macd.py` — the two shares the confidence divides by

```
crossover_scale = abs(closes[-1]) * 0.001
momentum_scale = abs(closes[-1]) * 0.002
```

A close of zero leaves nothing to divide by. The indicator used to raise on such a bar and take the whole voting run down with it, so the other eleven voters never answered either. It now says it has no reading, which is what every other voter does when its own denominator is empty.

`src/trading/indicators/macd.py` — the abstention that replaces the crash

```
if crossover_scale == 0.0 or momentum_scale == 0.0:
    # No scale to divide by, so macd publishes no confidence here.
    return Signal(
        "macd",
        timeframe,
        SignalDirection.NEUTRAL,
        0.0,
        self.weight,
        abstained=True,
    )
```

The bound that checks the reading against itself allowed a slack of two millionths. At three millionths of a dollar that slack is wider than any real MACD, so the bound passed a histogram with the wrong sign and a histogram of zero. It is exact now, and it holds on all 3,750 readings.

`src/trading/ta_invariants.py` — the bound on the reading

```
lambda d: float(d["histogram"])
== float(d["macd_line"]) - float(d["signal_line"]),
```

Nine parts of the platform read this indicator. Two read the vote and its strength rather than the numbers, and one builds its own series for the chart.

| reader | at three millionths of a dollar |
|---|---|
| `ta_invariants.py` bound | refuses a wrong histogram, where it passed one |

| reader | at three millionths of a dollar |
|---|---|
| `ta_engine.py` recorder | records the reading, where it recorded three zeros |
| `ata_spm.py` sentence | `histogram +4.1491974e-09`, was `histogram +0.000000` |
| `scrumming_v3_sim.py` flags | flip on 1,006 of 1,875 readings, 0 of 1,875 at four figures |
| `indicator_panel.py` hover | `macd_line: -0.000`, unchanged |
| `indicator_panel_surface.py` hover | `macd_line: -0.000`, unchanged |
| `scrumming_bot.py` position boost | reads the vote, unchanged |
| `ata_gate_scan.py` favour | reads the vote, unchanged |
| `native_chart.py` chart series | builds its own series, unchanged |

The ATA-SMP sentence printed six decimals of a price, so on a micro-priced market it printed nothing but zeros. It now prints eight significant figures, which reads the same at every price.

`src/trading/ata_spm.py` — the sentence one voter writes

```
"macd": ("MACD", ("histogram",), "histogram {value:+.8g}"),
```

The two hover cells still show three zeros, because the panel cuts every voter's numbers to three decimal places of its own. That format is shared by all twelve and is not this repair.

No gate changes its mind. Neither chain reads these numbers; both read the panel's consensus, and the consensus, the vote, its strength, the abstention flag and the other eleven voters are identical on all 3,750 readings.

## 2026-09-08 16:52 - #414 - the money flow a halted market never traded

Quong and Soudack's index is published in two forms, and this repair needs both of them.

```
TP_t = (H_t + L_t + C_t) / 3
MF_t = TP_t * V_t

positive money flow = sum of MF over the 14 bars where TP rose
negative money flow = sum of MF over the 14 bars where TP fell

MFI = 100 - 100 / (1 + positive / negative)
MFI = 100 * positive / (positive + negative)
```

The two lines give the same number. The second is defined on one input more than the first. Where every falling bar in the window traded nothing, the negative flow is zero: the first line divides by that zero, and the second answers 100. One hundred is the right answer there, because every dollar that moved was buying.

One input is left over, and neither published line answers it. A window that credited neither bucket gives nothing over nothing. The index does not exist on that bar.

**Functional.** The module answered 50 on that bar. Fifty is the middle of the scale and it is a number, published to the voting panel, to the ATA-SMP post and to the emitter recorder, about a market the indicator had measured nothing on. Driven through the voting engine on a constructed seventy-minute halt at three millionths of a dollar, the Volume column voted NEUTRAL and still counted in the panel's denominator. On a window

whose last fourteen bars traded nothing at all, it voted BULLISH at 0.37 strength.

The index now reports that it has no reading, and the column abstains on that bar. Abstaining is what this module already does when the window's average volume is zero, and what Supertrend, ADX, the Efficiency Ratio and the MACD each do at their own missing denominator.

`src/trading/indicators/volume.py` — `VolumeAnalysis._mfi`

```
total_mf = pos_mf + neg_mf
if total_mf <= 0.0:
    return math.nan
return 100.0 * (pos_mf / total_mf)
```

`src/trading/indicators/volume.py` — `VolumeAnalysis.compute`

```
mfi = self._mfi(candles, self.mfi_period)
if math.isnan(mfi):
    # `_mfi` has no money ratio; `mfi_ob` and `mfi_os` read False on nan.
    return Signal(
        "volume",
        timeframe,
        SignalDirection.NEUTRAL,
        0.0,
        self.weight,
        abstained=True,
    )
```

The item cites a money flow below 1e-09 read as zero, on two tests that answered 100 on it. Commit `b27c6cdb` removed both tests on 2026-09-04. Driven through the voting engine on a window at three millionths of a dollar whose falling bars carry a volume of 1e-05, the tree that still holds those tests answers 100.0 and this tree answers 99.999001008.

A money flow can also reach zero by running out of double precision, and that is the one route by which an absent reading could still print as the top of the scale. Measured: at a price of three millionths of a dollar the product of price and volume is exactly zero for any volume at or below 7.9688e-319. The smallest volume in the 411 recorded tablets is 100. No order size the venue can express comes within three hundred orders of magnitude of that threshold, and where the falling side does underflow, the ratio puts the index at 100 to every digit a double carries, which is the correct answer rather than an absent one.

The window with no money flow is not in the recorded data either. All 411 tablets hold daily bars; across 99,582 fourteen-bar windows not one credited neither bucket, the typical price repeated on 127 of 104,925 bars, and the longest run of repeats is five. The fleet computes on five-minute bars, which the tablets do not contain, so the recorded data says nothing about whether a seventy-minute flat window occurs live.

The reading was also published cut to one decimal place. The vote itself always tested the computed number, so no gate ever read the short one, but the panel's hover cell prints three decimals and was handed a number carrying one. It now carries what the module computed.

`src/trading/indicators/volume.py` — the reading

```
"mfi": mfi,
```

Six parts of the platform read this indicator.

| reader | at three millionths of a dollar |
|---|---|
| `ta_invariants.py` bound | bounds the computed number; no bound applies on an abstention |
| `ta_engine.py` recorder | records the computed number, and an empty reading on an abstention |
| `ata_spm.py` sentence | `Money Flow Index 56.5`, unchanged on all 3,750 readings |
| `indicator_panel.py` hover | `mfi: 56.450`, was `mfi: 56.500` |
| `indicator_panel_surface.py` hover | `mfi: 56.450`, was `mfi: 56.500` |
| the panel's consensus | loses one voter from its denominator on an abstention |

Neither gate chain is in that list. Both read the panel's consensus rather than any voter's numbers.

No gate changes its mind. Over 71 recorded tapes at two price scales, 3,750 readings and 15,000 gate verdicts, the vote, its strength, the abstention flag, the panel's consensus and the other eleven voters are identical, and both chains fire the same way on every one. Swapping the published form of the ratio moved the index on 1,907 of the 3,750 by at most 2.2e-14, which is the last bits of double arithmetic and vanishes at one decimal place. Publishing it uncut moved the hover cell on 3,710 of 3,750 and the ATA-SMP sentence on none of them.

## 2026-09-08 17:45 - #414 - the Vortex reports the Vortex

Botes and Siepman published two lines and one reading of them. The lines measure upward and downward movement over the same fourteen bars, each divided by the true range of those bars. The reading is the crossover: the upward line above the downward one is a rising market, and the other way round is a falling one. A downward line far above parity is the strongest falling reading the indicator has.

```
VM+_t = |H_t - L_(t-1)|
VM-_t = |L_t - H_(t-1)|
VI+   = sum(VM+, 14) / sum(TR, 14)
VI-   = sum(VM-, 14) / sum(TR, 14)
```

**Functional.** The two lines above are exactly as published, and this repair does not touch them. What came out is the four lines that ran after the vote had already been decided. They read the downward line reaching 1.30, called it a coming reversal, lifted the strength to at least 0.60 and rewrote the vote to bullish.

`src/trading/indicators/vortex.py` — `VortexIndicator.compute`, removed

```
if vim_at_ceiling:
    confidence = min(1.0, max(confidence, 0.60))
    direction = SignalDirection.BULLISH
```

Seventy-one recorded Coinbase tapes were run through the voting engine at three millionths of a dollar and again at 1234.0, 3,750 readings in all. The downward line reached 1.30 on 78 of them, spread over 30 tapes. On every one of those 78 the published lines said falling and the overwrite published rising. The strength was already at or above 0.60 on all 78, so the 0.60 lift changed nothing and only the vote moved.

```
3,750 readings, 71 tapes, two price scales
    VI- at or above 1.30                          78
    vote before the repair, all 78            BULLISH
    vote after the repair, all 78             BEARISH
    strength changed                                0
    votes moved the other way                       0
    VI- range on those rows          1.3014 to 1.4859
```

**Design intention.** An indicator carries one published set of sums and one published reading of them, and nothing else rides on its name. The ceiling that remains touches strength only and never the vote: an upward line at or above 1.30 caps the strength at 0.45. Both ceiling readings and both floor readings are still published beside the vote, so the panel can show that a line is saturated without the vote claiming the opposite of what the lines say.

`src/trading/indicators/vortex.py` — the ceiling that stayed

```
VX_CEILING_PCT = 130.0

if vip_at_ceiling:
    confidence = max(0.0, min(confidence, 0.45))
```

Seven parts of the platform read this indicator.

| reader | on a VI- at or above 1.30 |
|---|---|
| the panel's consensus | one vote moves from rising to falling; moved on 78 |
| `indicator_panel.py` cell | the arrow turns from up to down |
| `indicator_panel_surface.py` cell | the arrow turns from up to down |
| `ata_spm.py` sentence | prints the gap between the lines; unchanged on all 78 |
| `ta_invariants.py` bound | bounds both lines at or above zero; unchanged |
| `ta_engine.py` recorder | records the vote it now casts |
| `scrumming_bot.py` position boost | pays only at the upper band or the middle, and never reached these rows |

`native_chart.py` is not in that list. It draws the two lines from `VortexIndicator.lines`, which this repair does not touch.

The position boost is the one reader the overwrite could have reached and never did. It pays a bullish Vortex 0.12 at the upper band and takes 0.05 back in the middle. A downward line saturating means the price has fallen hard, and on all 78 rows the band position sat between -0.134 and 0.224, below both zones. The boost paid nothing before the repair and nothing after it.

The gate changes its mind on four windows. Over 71 tapes at two price scales, 3,750 readings and 30,000 gate verdicts, eight verdicts move and every one of them moves the same way: a fold that was refused now fires. No scrum verdict moves. Both windows sit at or below the lower band, where the price is at the bottom and the buy-back belongs.

```
30,000 gate verdicts, both chains, four flag states
    fold refused -> fold fires                      8
    fold fires -> fold refused                      0
    scrum verdicts changed                          0
    windows                                         4
        A22 2022, band position 0.1168, VI- 1.3276
        A23  2026, band position -0.1158, VI- 1.3014
    both windows identical at both price scales
```

Both moves need the fold's own bearish requirement switched on. With that switch off the fold does not consult the panel at all and nothing moves.

The reading was also published cut to four decimal places. No vote and no gate ever read the cut copy, because every branch inside the indicator tests the number it computed. The panel's hover cell prints three decimals and was handed a number carrying four, and the cell printed a different third decimal on 726 of 15,000 readings. It now carries what the indicator computed.

`src/trading/indicators/vortex.py` — the reading

```
"vi_plus": vi_plus,
"vi_minus": vi_minus,
"separation": separation,
"sep_acceleration": sep_acceleration,
```

One example, from A34 2021: the downward line computed 1.0835408442150014 and published 1.0835, so the cell printed `1.083` where the number is `1.084`. The ATA-SMP sentence prints four decimals and matched the cut copy exactly, so it reads the same on all 3,750.

## 2026-09-08 19:05 - #414 - the RSI label and the RSI vote read one number

Wilder publishes one quantity. It is the share of the last fourteen bars of movement that ran upward, on a nought to a hundred scale. The same number decides the label a reader sees and the vote the panel casts: above seventy is overbought and below thirty is oversold.

```
gain_t  = max(C_t - C_(t-1), 0)
loss_t  = max(C_(t-1) - C_t, 0)
AvgGain = Wilder(gain, 14)
AvgLoss = Wilder(loss, 14)
RS      = AvgGain / AvgLoss
RSI     = 100 - 100 / (1 + RS)
```

**Functional.** The indicator held two copies of that quantity. It cut one to two decimal places and published it, and it kept the whole one for the overbought and oversold labels. The vote read the cut copy. A reading of 70.003 cuts to seventy exactly, and seventy is not above seventy, so the label said overbought while the vote said neutral, and the two left the indicator together.

`src/trading/indicators/rsi.py` — the cut, removed

```
return {
    "rsi": round(rsi, 2),
    "overbought": rsi > 70,
    "oversold": rsi < 30,
```

One number now leaves the indicator, and the label, the vote, the strength and the published reading all read it. The block marked *Proposed, not present* in the RSI section above is the code in this tree. The rounding has gone back to the two places that display a number: the confirming sentence prints two decimals and the panel's hover cell prints three.

`src/trading/indicators/rsi.py` — the reading

```
return {
    "rsi": rsi,
    "rs_indeterminate": rs_indeterminate,
    "overbought": rsi > 70,
    "oversold": rsi < 30,
```

The gap the two copies can disagree in is five thousandths of a point wide, just above seventy and again just below thirty. Seventy-one recorded Coinbase tapes were run through the voting engine at three millionths of a dollar and again at 1234.0, 3,750 readings in all, and not one landed inside it. The closest reading to seventy was 70.06, which was already voting bearish.

```
3,750 readings, 71 tapes, two price scales
    readings inside 70.000 to 70.005                0
    readings inside 29.995 to 30.000                0
    closest reading to seventy                  70.06
    closest reading to thirty                   29.97
    the index ran from                  12.6 to 89.45
```

One bar puts a market inside it. Take any recorded run of forty bars or more and ask what final close lands the index on 70.003. Every one of the 2,901 runs admits such a close, because the index rises with the last price. Most of those bars are large, because most windows sit a long way from seventy: the middle one needs a move of nearly twenty-six per cent. Seventy-two of the runs need a bar of one per cent or less, and the smallest was half a basis point after 271 real days of A21.

```
2,901 recorded runs, the final bar each one needs
    smallest                                    +0.0049%
    middle                                        25.70%
    largest                                      334.53%
    runs needing one per cent or less                 72
    runs needing five per cent or less               272
```

The smallest is the one to read. Half a basis point is an ordinary tick on a market that has already run up to the seventy line.

```
A21 2023, 271 recorded bars, then one bar of +0.0049%
    reading published, before                  70.00
    reading published, after                  70.003
    label, before and after               overbought
    vote, before                             NEUTRAL
    vote, after                              BEARISH
    identical at both price scales
```

**Design intention.** A number that decides something is the number that gets published, and a display rounds it at the moment it is shown. Two copies of one reading is the shape that lets a label and a vote say different things about the same bar while both look correct on their own.

Nothing about the vote moved on the recorded tapes. The reading published now differs from the cut copy on all 3,750, by at most five thousandths of a point; the direction is the same on every one; the strength moves on 180 of them, by at most one ten-thousandth. The panel's consensus moves on 24 readings and the other eleven voters do not move at all.

```
3,750 readings, both price scales
    reading published differs from computed     3,750
    vote direction moved                            0
    vote strength moved                           180
    overbought or oversold moved                    0
    panel consensus moved                          24
    the other eleven voters                         0
```

No gate changes its mind. Over 30,000 verdicts — both chains, four states of the two direction switches — 1,322 fire before the repair and the same 1,322 fire after it. The repair only bites inside a window five thousandths wide, the recorded tapes never enter it, and the largest strength move anywhere else is one ten-thousandth against a confidence floor near a quarter.

Inside the window the gate does change its mind. Every one of the 2,901 runs was walked in with its own final bar and put through both chains at both price scales: 5,802 readings, and on every one of them the label said overbought while the vote said neutral. After the repair not one of them disagrees. Of the 46,416 verdicts, 60 fire before the repair and six of those stop firing, all on the buy side and all the same way.

```
5,802 constructed readings, 46,416 gate verdicts
    label disagreeing with the vote, before     5,802
    label disagreeing with the vote, after          0
    vote moved from neutral to bearish          5,802
    verdicts that fire before                      60
    scrum fires -> scrum refused                    6
    scrum refused -> scrum fires                    0
    fold verdicts changed                           0
    windows                                         3
        DOT 2023, consensus 0.2002 -> 0.1728, floor 0.1786
        A14 2022, consensus 0.2087 -> 0.1813, floor 0.1923
        MATIC 2022, consensus 0.2286 -> 0.1910, floor 0.1923
```

All six need the scrum's own bullish requirement switched on. The mechanism is the panel: a bearish vote at 0.4001 in place of a neutral one at nought pulls the bullish consensus below the confidence floor, and the sell of

the surplus is refused on a bar the RSI calls overbought.

Ten places read this indicator. Four read the vote and six read the published number.

| reader | what its reading becomes |
|---|---|
| the panel's consensus | the vote and its strength, decided by the number the labels use; direction unchanged on all 3,750 |
| `ta_engine.py` recorder | records the computed number in place of the cut copy |
| the trading gate chain | reads the consensus the vote feeds; no verdict moved |
| `indicator_panel.py` cell | the arrow and the strength percentage; unchanged on all 3,750 |
| `indicator_panel.py` hover text | prints three decimals; moved on 3,356 of 3,750 |
| `indicator_panel_surface.py` hover text | moved on the same 3,356 |
| `ata_spm.py` sentence | prints two decimals; unchanged on all 3,750 |
| `ta_invariants.py` bound | holds the reading between nought and a hundred; unchanged |
| `phantom_balance.py` record | stores what the indicator published |
| `snapshots.py` bot.log line | its headline readings are ADX, Z-Score and Kaufman, so it prints the RSI name and strength only |

The bot's own position boost and the back test both ask a signal for a number, and both ask only for ADX, Kaufman and Z-Score. Neither has ever asked this indicator for one.

The reading was also published cut to two decimal places, and the panel's hover cell prints three. On 3,356 of the 3,750 readings the third decimal the cell printed was a nought that the indicator had not computed. The confirming sentence prints two decimals, which is exactly what the cut copy carried, so it reads the same on every one.

## 2026-09-09 19:37 - #414 - the band a millionth of a dollar wide

Item 6 of the issue names two epsilons, one in the Ichimoku and one in the Supertrend. Neither is in this tree. Both came out on 2026-09-04, in the commit that dropped the last eight absolute constants from price-scaled denominators.

```
b27c6cdb  fix(ta): drop the last eight absolute epsilons from scaled denominators
```

That makes eight items out of eight where the expression the issue quotes had already been repaired, and where the defect still standing underneath it was a rounded copy of the reading reaching a consumer. This entry closes that pattern for the whole folder.

John Bollinger's bands are the worst of the set, because their width is what the scrum ramp measures against. The published definition fixes no number of decimal places, and a band is an ordinary price in the asset's own units.

```
Middle Band = SMA(period)
Upper Band  = Middle + std_dev * sigma(period)
Lower Band  = Middle - std_dev * sigma(period)
BandWidth   = (Upper - Lower) / Middle
```

The indicator published all three cut to six decimal places, a step of one millionth of a dollar. A10 trades at three millionths, so the whole channel is narrower than the step and the three bands land on one number.

`src/trading/indicators/bollinger.py` — the reading, before and after

```
"upper": round(upper, 6),          "upper": upper,
"middle": round(mid, 6),           "middle": mid,
"lower": round(lower, 6),          "lower": lower,
```

Measured over 71 recorded Coinbase tapes, run at a price of three millionths of a dollar and again at a four-figure price. Every window from 36 bars in steps of 11 drove the real voting engine, and each reading built a gate context through the shipped scan. Over 1,875 windows at the small price the distance from the midline to the band read exactly zero on 1,014 of them, and on a further 20 the lower band reached zero and the scan refused the channel outright. At the four-figure price it read zero on none.

```
1,875 windows at 3.1e-06        1,875 windows at 1234.0
    half-span exactly 0.0           half-span exactly 0.0
        before  1,014                   before  0
        after   0                       after   0
    band refused outright           smallest half-span
        before  20                      before  7.76315
        after   0                       after   7.76315
```

The consequence is a decision, not a display. The ramp that arms a scrum divides the price's travel by that half-span, and a floor of a millionth of a millionth stands in when the span is zero. The ramp then walked a different path on a cheap asset than on an expensive one holding the same shape.

`src/trading/ata_gate_scan.py` — the step that decides whether a scrum is armed

```
span = max(abs(edge - middle), BAND_SPAN_FLOOR)
travel_frac = abs(price - middle) / span
```

The two price scales disagreed about that state on 162 of the 1,875 windows. They now agree on all of them, which is the property the fleet needs: one detect threshold serves every bot, whatever its asset costs.

```
target_fires compared between the two price scales
    before   agree 1,713   disagree 162
    after    agree 1,875   disagree 0
```

Both gate chains ran on every reading, in all four states of the two direction switches. Of 30,000 verdicts, 1,322 fire before the repair and 58 change. All 58 are on the buy side, all 58 are at the small price scale, and every one sits on a window where the ramp state moved. Nothing changed at four figures.

```
30,000 gate verdicts
    fire before the repair                 1,322
    changed                                   58
        scrum refused -> scrum fires          50
        scrum fires -> scrum refused           8
        fold verdicts changed                  0
    changed at 1234.0                          0
    changed at 3.1e-06                        58
    rows carrying them                        27
        rows where the ramp state moved       27
```

The same cut sat on every other indicator in the folder. Fifty-three calls across thirteen files cut a reading where it was computed rather than where it was shown, and all fifty-three are gone. None was kept.

```
zscore 7   ichimoku 6   bollinger 5   landing_strip 5   fvg 4   macd_taper 4
slingshot 9   supertrend 3   bb_proximity 3   atr 2   stochastic_rsi 2
volume 2   rsi 1
```

No display loses its shape, because every one of them already carries its own format. The panel hover cell prints three decimals, the bot log line prints four, and each confirming sentence in the ATA-SMP names its own width.

`src/trading/ata_spm.py` — one voter's sentence, formatted where it is shown

```
"bollinger_bands": (
    "Bollinger Bands",
    (BAND_POSITION_KEY,),
    "band position {value:.4f}",
),
```

Item 6 also names two statements that compute a value and bind it to nothing. The Ichimoku one is the third cloud state. The code named the other two and then derived the third a second time where the reading is published, so one expression now decides it.

```
inside_cloud = not above_cloud and not below_cloud
...
"price_vs_cloud": (
    "inside" if inside_cloud else "above" if above_cloud else "below"
),
```

The volume one is the money-flow slope. It is published beside the balance-volume slope the same indicator already publishes, so the value it computes now reaches a reader.

```
mfi_rising = mfi > mfi_prev
```

Across the 3,750 readings, 339,170 non-numeric readings were compared. None moved and none was dropped. The cloud state is identical on every window, which is what a pure rebinding has to look like, and its inside branch runs on 499 of them rather than never. The money-flow slope takes both values.

```
price_vs_cloud   below 1,579   above 1,112   inside 499   absent 560
mfi_rising       True 2,025    False 1,725
```

Twelve places read these numbers, and the table below says what each one's reading becomes.

| reader | what its reading becomes |
|---|---|
| the scan's scrum ramp | a real band width at any price; 58 of 30,000 verdicts move, all on the buy side |
| the live bot's scrum ramp | unchanged; it reads the proximity result, whose three band prices were never cut |
| the Z-Score gate | the computed z; it moved on 3,470 readings and crossed the two-deviation line on none |
| the ADX gate | unchanged; that reading was uncut already |
| the Efficiency Ratio gate | unchanged; that reading was uncut already |
| the confidence floor | the landing-strip boost, uncut; no consensus direction moved |
| the panel's twelve cells | direction and strength unchanged on all 3,750; strength moved on 358 votes |
| `indicator_panel.py` hover text | the computed number, printed to three decimals as before |
| `ata_spm.py` sentences | each keeps its own width, so every sentence reads as it did |
| `snapshots.py` bot log line | prints four decimals of the computed number |
| `phantom_balance.py` record | stores what the indicator published |
| `ta_invariants.py` bounds | no breach on any reading, before or after |

Three figures the page already carries need restating against this run. The departures section above says two departures remain, the Vortex direction overwrite and the RSI rounding. Both are repaired: the Vortex on 2026-09-08 and the RSI on the same day, each in its own entry above. The section also lists four scale-invariance test files. None is in the tree, which now holds its conftest and these debug reports and nothing else, so the property they held is held by the runs recorded in each entry instead.

```
tests/debug_reports/2026-09-09_indicator_rounding.md
```

What changes for an operator: a bot on a market priced in millionths of a dollar now sees a Bollinger channel with a real width instead of a flat line, so its scrum arms on the same bars a bot on an expensive market would.

## Trading gate logic chain

The epigraph on the title page reads *dissolvendus reformandusque*, dissolved and reformed. The chain below does exactly that, once per tick: twelve separate readings dissolve into one signed number, and the gates re-form that number into an order or a refusal.

### The chain

Diagram: flowchart, top to bottom

- OHLCV candles -> **The 12 indicator modules**
- The 12 indicator modules -> **VotingEngine.compute_all** *(Signal: direction, confidence, weight)*
- VotingEngine.compute_all -> **VotingEngine._aggregate**
- VotingEngine._aggregate -> **VotingSummary.net_score**
- VotingEngine._aggregate -> **VotingSummary.consensus_confidence**

- VotingSummary.net_score -> **consensus_direction: deadband +/- 0.1**
- consensus_direction: deadband +/- 0.1 -> **BULLISH / BEARISH / NEUTRAL**
- BULLISH / BEARISH / NEUTRAL -> **ScrummingBot.tick: is_bullish, is_bearish**
- VotingSummary.consensus_confidence -> **ScrummingBot.tick: is_bullish, is_bearish**
- _skewed_confidence_floor: 0.25 relaxed by the skew -> **ScrummingBot.tick: is_bullish, is_bearish**
- OHLCV candles -> **detect_bb_proximity**
- detect_bb_proximity -> **bb_position**
- bb_position -> **bb_pos against _bb_detect_thresholds**
- ScrummingBot.tick: is_bullish, is_bearish -> **build_scrumming_scrum_chain**
- bb_pos against _bb_detect_thresholds -> **build_scrumming_scrum_chain**
- adx signal detail 'adx' -> **build_scrumming_scrum_chain**
- kaufman_er signal detail 'er' -> **build_scrumming_scrum_chain**
- zscore signal detail 'z' -> **build_scrumming_scrum_chain**
- build_scrumming_scrum_chain -> **ChainResult.should_fire**
- ChainResult.should_fire -> **_tick_execute_scrum, then _execute_sell** *(true)*
- ChainResult.should_fire -> **ChainResult.blocked, one message per gate** *(false)*
- _tick_execute_scrum, then _execute_sell -> **guarded_place_order**

### Net, Comp and Conf

`VotingEngine._aggregate` in `src/trading/ta_engine.py` produces all three panel aggregates from the twelve signals.

```
weighted_score = direction * confidence * weight        one per voter
bull_score     = sum of |weighted_score| over the BULLISH voters
bear_score     = sum of |weighted_score| over the BEARISH voters
net_score      = bull_score - bear_score                 the Net column
voted_weight   = sum of weight over the voters that did not abstain
confidence     = min(1.0, |net_score| / voted_weight)    the Conf column
```

**Functional.** Net is a direction bias, not a reversal signal. Positive means the bullish voters outweighed the bearish ones. Conf is the breadth of that agreement, not its force. Six voters agreeing strongly while six abstain reads lower than twelve agreeing mildly, and the panel says the same thing in the Conf column's own tooltip. An abstaining indicator is left out of the divisor, so a short tape cannot water down the reading of the voters that did read. Net is rounded to four decimals and Conf is capped at 1.0.

`src/trading/ta_engine.py` — `VotingEngine._aggregate`

```
net = bull_score - bear_score
```

**On the panel.** Net, Comp and Conf are drawn as one pillar each, standing in the last three columns of the shared grid. A pillar runs from the row-B plot floor to the row-A plot ceiling, so it passes both tables and both bar graphs and tops out on the line a voter at full confidence reaches. `collated_pillars` in

`indicator_panel_surface.py` names each pillar and takes its direction from its own metric's sign, so Net follows `net_score`, Comp follows `composite_net` and Conf follows the summary's direction.

`src/gui/main_tabs/indicator_panel_surface.py` — `collated_pillars`

```
first = PANEL_COLUMN_COUNT - len(AGGREGATE_TITLES)
return [
    {"name": title, "direction": way, "column": first + at}
    for at, (title, way) in enumerate(zip(AGGREGATE_TITLES, directions))
]
```

A pillar carries no height reading. The value stays in the table cell above it, and `AGGREGATE_TITLES` names the pillar once, at its base. `RULED_COLUMNS` keeps the row partition, the plot baseline and the dotted increments on TF and the twelve, so none of them crosses a pillar.

The restyled panel

`src/trading/ta_engine.py` — and the divisor beneath it

```
voted_weight = sum(s.weight for s in signals if not s.abstained)
consensus_conf = abs(net) / voted_weight if voted_weight > 0.0 else 0.0
```

**Design intention.** Conf answers how many of the voters that could read agree, not how loud they were. Dividing by the weight of the voters that actually read, rather than by all twelve, is the arithmetic that says so. The panel repeats the same rule to the reader in its own tooltip, so the screen and the engine cannot tell two

different stories.

`src/trading/ta_engine.py` — the rule, stated where it is applied

```
# A weighted mean divides by the weight of the points included, and an
# abstention is not one; it is NEUTRAL, so it adds nothing to `net`.
```

### The deadband

`VotingSummary.consensus_direction` in `src/trading/indicators/types.py` turns Net into a direction, and it holds a deadband of plus and minus 0.1.

```
net_score > +0.1   ->  BULLISH
net_score < -0.1   ->  BEARISH
otherwise          ->  NEUTRAL
```

`ScrummingBot.tick` then folds the direction and the Conf reading into one boolean per side.

```
is_bullish = direction in (BULLISH, NEUTRAL) and confidence >= floor
is_bearish = direction in (BEARISH, NEUTRAL) and confidence >= floor
```

**Functional.** The confidence floor starts at 0.25 and the bot relaxes it when the vote is skewed. A landing strip at the upper band sets the bullish flag outright, and one at the lower band sets the bearish flag. A neutral direction passes both sides, so a panel with no consensus does not on its own block a trade. The floor and the gates below decide that.

`src/trading/indicators/types.py` — `VotingSummary.consensus_direction`

```
def consensus_direction(self) -> SignalDirection:
    if self.net_score > 0.1:
        return SignalDirection.BULLISH
    elif self.net_score < -0.1:
        return SignalDirection.BEARISH
    return SignalDirection.NEUTRAL
```

**Design intention.** The deadband stops a near-zero Net reading as a direction. Plus or minus 0.1 against a full voted weight of 11.7 is a very small bias, and the band treats it as none at all. The bot then folds the direction and the confidence into one boolean per side, and a neutral panel passes both.

`src/trading/scrumming_bot.py` — the two booleans, in plain terms

```
is_bullish = direction in (BULLISH, NEUTRAL) and confidence >= floor
is_bearish = direction in (BEARISH, NEUTRAL) and confidence >= floor
```

### The band position

**Functional.** The band position is %B: zero at the lower band, one at the upper. The two thresholds a scrum and a fold must clear come back in the same terms, so both sides of every comparison are positions and neither side is a price. The fraction is clamped to the 0 to 1 range, so a stored setting outside 0 to 100 cannot push a threshold outside the band. A missing setting falls back to 75 through an explicit test for absence, so a stored zero stays zero.

The relationship in plain terms:

```
detect_frac  = scrum_detect_pct / 100          75% by default
upper_detect = 0.5 + detect_frac * 0.5         0.875
lower_detect = 0.5 - detect_frac * 0.5         0.125

SCRUM requires bb_pos >= upper_detect
FOLD  requires bb_pos <= lower_detect
```

`src/trading/scrumming/circuit_breakers.py` — `_bb_detect_thresholds`

```
detect_frac = max(0.0, min(1.0, detect_pct / 100.0))
half = detect_frac * 0.5
return (0.5 - half, 0.5 + half)
```

**Design intention.** Working in band positions rather than in prices is what lets one threshold serve the whole fleet. A bot on a four-figure asset and a bot on an asset priced in millionths both read 0.875 and mean the same thing by it. This is the pattern the constants listed above break. A missing setting is tested for absence rather than for truthiness, so a deliberate zero survives instead of being turned back into 75.

`src/trading/scrumming/circuit_breakers.py` — the absence test

```
# None-sentinel default so a legitimate 0 isn't coerced to 75 by
# `or` truthiness.
_raw = getattr(self.config, "scrum_detect_pct", None)
```

### The scrum chain gates

`src/trading/gate_chain.py` — `build_scrumming_scrum_chain`

```
return GateChain(
    gates=[
        DeltaPositiveGate(),
        IntervalGate(),
        TADirectionGate(side="scrum"),
        TrendHoldGate(),
        MidlineGate(side="scrum"),
        TargetFiresGate(),
        BBProximityGate(side="scrum"),
        CircuitBreakerGate(side="scrum"),
        HTFDeferGate(side="scrum"),
        HysteresisGate(side="scrum"),
        ADXTrendSuppressionGate(),
        EfficiencyRatioRegimeGate(),
        ZScoreExtremityGate(side="scrum"),
        RipeHarvestScrumOverride(),  # runs in pass 2 regardless of list position
    ],
    side="scrum",
)
```

**Functional.** Every gate in the list runs, and every refusal is collected. The trade fires only when the refusal list is empty, so one blocked gate is enough to hold it. The order of the list fixes the order the refusals appear in, which is the order the log prints them.

`src/trading/gate_chain.py` — the verdict

```
return ChainResult(
    should_fire=(not final_blocked),
    passed=passed,
    blocked=final_blocked,
    overrides_applied=overrides_applied,
)
```

**Design intention.** One gate holds one question, and each writes its own refusal in its own words. Reading back a held trade is then a matter of reading which name refused, rather than re-deriving the whole decision.

`src/trading/gate_chain.py` — `DeltaPositiveGate.evaluate`, the pattern every gate follows

```
def evaluate(self, ctx: GateContext) -> GateResult:
    if ctx.delta > 0:
        return GateResult(passed=True)
    return GateResult(passed=False, blocker_message="delta≤0")
```

| Gate | Passes when | Reads |
|---|---|---|
| `delta_positive` | the holdings sit above target | the target delta |
| `interval` | the delta clears the scrumming interval | the delta |
| `ta_bullish` | `is_bullish` | Net, Conf and the floor |
| `trend_hold` | no sustained uptrend | more than 13 of the last 20 candles closing above their open |

| Gate | Passes when | Reads |
|---|---|---|
| `midline_scrum` | `bb_pos` above 0.50 and no phantom lock | `bb_position` |
| `target_fires` | the detect and fire ramp is in FIRE | the target ramp |
| `bb_proximity_scrum` | `bb_pos >= upper_detect` | `bb_position` |
| `circuit_breaker_scrum` | no soft trip on the scrum side | the breaker state |
| `htf_defer_scrum` | the higher timeframe is not bullish | the coordinator |
| `hysteresis_scrum` | the price cleared the last pivot plus fees | the pivot |
| `adx_trend_suppression` | ADX under 30.0 | the `adx` signal detail |
| `efficiency_ratio_regime` | ER between 0.05 and 0.70 | the `er` signal detail |
| `zscore_extremity` | Z above -2.0 | the `z` signal detail |

**Functional.** Three of those gates skip the collated vote and read one indicator's own number: ADX, the Efficiency Ratio and the Z-Score. The bot fills each field from that indicator's signal on the tick. A reading of zero or below is the not-populated marker, and all three pass on it.

`src/trading/gate_chain.py` — `EfficiencyRatioRegimeGate.evaluate`, one of the three

```
def evaluate(self, ctx: GateContext) -> GateResult:
    if ctx.efficiency_ratio <= 0.0:
        return GateResult(passed=True)
    if ctx.efficiency_ratio >= self.upper_threshold:
```

**Design intention.** These three describe the market regime rather than its direction, which is why they reach the chain on their own instead of through the vote. Passing on the marker keeps a bot that never filled the field from being frozen by it. The cost is real and worth stating: a genuine zero reads exactly the same as a missing one, and the gate cannot tell them apart.

In development. Separating the two would need a marker the field cannot produce, and no shape for that has been chosen.

### The fold chain gates

`src/trading/gate_chain.py` — `build_scrumming_fold_chain`

```
return GateChain(
    gates=[
        TranchesQueuedGate(),
        TADirectionGate(side="fold"),
        MidlineGate(side="fold"),
        SmartCeilingGate(),
        BBProximityGate(side="fold"),
        CircuitBreakerGate(side="fold"),
        HTFDeferGate(side="fold"),
        HysteresisGate(side="fold"),
        ZScoreExtremityGate(side="fold"),
        DeepFoldOverride(),
    ],
    side="fold",
)
```

**Functional.** The buy side runs nine gates against the sell side's thirteen. Six of the sell-side gates have no twin here, and the buy side adds two of its own.

```
absent on the fold side   delta_positive, interval, trend_hold,
                          target_fires, adx_trend_suppression,
                          efficiency_ratio_regime
present only here         tranches_queued, smart_ceiling
```

**Design intention.** The two chains mirror each other, and two of the six absences carry a written reason. The interval gate says in its own source that it needs no fold twin, because the bot returns early on the same condition. The trend-hold gate is declared sell-side only, and the config field that would arm a buy-side twin is declared and unread. 06-trading-tab.md carries the proposal for that one, at the Strategy Gate Flags group.

**Functional.** That config field is now removed. The buy side holds no flag for a trend hold and no gate for one, so the asymmetry is a plain absence rather than a flag waiting on a gate. The sell-side gate is unchanged. On one tick with twenty candles all closing above their open, the sell side reported `trend_hold` and the buy side reported nothing.

`src/trading/gate_chain.py` — `IntervalGate`, stating its own case

```
# No FOLD twin; ScrummingBot.tick returns early on below_interval when both
# its fold queue and distribution accumulator are empty.
```

The other four absences carry no stated reason in the code at all. That is worth knowing before anyone assumes the asymmetry is deliberate.

| Gate | Passes when |
|---|---|
| `tranches_queued` | at least one fold tranche is waiting |
| `ta_bearish` | `is_bearish` |
| `midline_fold` | `bb_pos` below 0.50 |
| `smart_ceiling` | the position sits under the smart ceiling |
| `bb_proximity_fold` | `bb_pos <= lower_detect` |

| Gate | Passes when |
|---|---|
| `circuit_breaker_fold` | no soft trip on the fold side |
| `htf_defer_fold` | the higher timeframe is not bearish |
| `hysteresis_fold` | the price cleared the last pivot plus fees |
| `zscore_extremity` | Z below +2.0 |

### The two overrides

**Functional.** An override runs in a second pass and force-passes gates that already refused. Its own verdict never appears in the pass list or the block list. Each refusal it lifts has its message rewritten to name the override, and the name is recorded separately, so the log tells you which gate was overridden and by what.

`src/trading/gate_chain.py` — `GateChain.evaluate`, the second pass

```
overrides_applied: list[str] = []
for gate in self._override_gates:
    result = gate.evaluate(ctx)
    if not result.passed:
        continue
    for target_name in result.override_gates:
        if target_name in blocked_index:
            idx = blocked_index.pop(target_name)
            _name, _msg = blocked[idx]
            blocked[idx] = (_name, f"OVERRIDDEN_BY:{gate.name}({_msg})")
            overrides_applied.append(target_name)
```

| Override | Arms on | Force-passes |
|---|---|---|
| `ripe_harvest_override` | positive delta, interval cleared, `bb_pos >= upper_detect` | `midline_scrum`, `target_fires`, `trend_hold`, `ta_bullish` |
| `deep_fold_override` | negative delta, interval cleared, `bb_pos <= lower_detect` | `midline_fold`, `ta_bearish` |

**Design intention.** At the band extreme, the band position outranks the vote. That comes straight from the operator's own reading of the Bollinger Bands: they are the boundary a trade fires at, and a position sitting on one is the condition he wants to trade. Neither override touches the circuit breaker, the hysteresis or the three indicator gates, so the safety gates keep their veto.

`src/trading/gate_chain.py` — `RipeHarvestScrumOverride`, and what it may lift

```
name = "ripe_harvest_override"
side = "scrum"
overrides = ("midline_scrum", "target_fires", "trend_hold", "ta_bullish")
```

### Reading the chain after the fact

**Not available.** A trade already on the books keeps the reading that was recorded beside it. Nothing recomputes the direction, the Net, the Conf, the vote counts, an indicator's own vote or the band position for a past trade, so a gate row that was never written stays missing.

**One writer.** A gate row reaches `gate.log` only when `LogManager.log_gate_decision` writes it at the moment the gate decides.

`src/core/logging_engine.py` — the one call that records a decision

```
def log_gate_decision(
    self,
    exchange: str,
    bot_id: str,
    symbol: str,
    scrum_armed: bool,
    fold_armed: bool,
    scrum_blockers: Optional[list] = None,
    fold_blockers: Optional[list] = None,
```

## 2026-09-24 22:50 - #202 - one admission rule for every panel reading

Four rules read a stored number and they disagreed. A saved flag read as the number one on the panel and as nothing in the trading package. A 400-digit saved number ended the panel's refresh, which left one timeframe's label above the previous reading's numbers.

One rule reads every number now, and it is the trading package's own.

`src/trading/container/config.py` — the one admission rule

```
def as_finite_float(value) -> Optional[float]:
    if type(value) is float:
        return value if math.isfinite(value) else None
    if type(value) is int and -_FLOAT_SAFE_INT <= value <= _FLOAT_SAFE_INT:
        return float(value)
    return None
```

It answers nothing for a saved flag, for a not-a-number, for either infinity, for a number written as text and for an integer too large to be a float. It never raises, so a refusal cannot end a refresh.

**What a refused reading draws.** Each cell says for itself. The panel draws zero, so a refused Net reads `+0.00`, a refused Conf reads an empty bar at `0%`, and a refused indicator cell reads its direction beside `0%`.

`src/gui/main_tabs/indicator_panel_surface.py` — the figure a refusal draws

```
#: What a cell, bar, pillar or tint draws when as_finite_float refuses its reading.
REFUSED_READING: float = 0.0
```

The Qt panel and the React panel now build one cell from one pair of builders, `indicator_cell_text` and `indicator_cell_colors`, so the two draw the same text and the same tint from the same reading.

**Three sentences the retired rules carried.** Each is quoted whole, and each is overtaken by the sentence written under it.

> *"One published reading as a float, or `fallback` when it is not one."*

Overtaken. That rule read a saved flag as the number one and raised on a saved integer too large to be a float. `as_finite_float` answers nothing for either, and `REFUSED_READING` is the figure the panel draws in its place.

> *"`value` as a float, or `fallback` when it is not one."*

Overtaken, and this sentence stood in two files. Both read either infinity as a dollar figure. `as_finite_float` answers nothing for either, and `REFUSED_FIGURE` is the figure a Paper record and a Simulator record draw.

> *"The number one market row carries, nothing where it carries no number."*

Overtaken. That rule refused a saved flag and still raised on a saved number too large to be a float. `as_finite_float` refuses both, and `VOLUME_REFUSED_USD` is the volume a refused pair row sorts by.

# Subsystem Tabs

Reference. Main Window, Market Inspector and Asset Charts each have a heading below, and the manual's own text for the screen comes first with what the code does today under it. Every other screen has one section of its own, one file per screen in 08-tabs/, and this part points at it rather than repeating it.

Detail: 08-tabs/simulator.md. Detail: 08-tabs/paper-trader.md. Detail: 08-tabs/proof-of-accumulation.md. Detail: 08-tabs/bot-swarm.md. Detail: 08-tabs/history.md. Detail: 08-tabs/console.md. Detail: 08-tabs/system-status.md. Detail: 08-tabs/settings.md.

One list names the ten tabs the window builds, and every screen below that the window does not build says as much in its own entry. Two are skeletons now: Status and Accumulation each draw a name, one sentence saying the tab is not built, and the issue that carries the build-out. Sim and Paper were skeletons until issues #117 and #19 built them, and both are full screens today.

`src/gui/main_tabs/main_window_surface.py` — the order before issue #450

```
CANONICAL_TAB_ORDER = (
    TRADING_TAB,
    MARKET_INSPECTOR_TAB,
    BOT_SWARM_TAB,
    ASSET_CHARTS_TAB,
    HISTORY_TAB,
    SIMULATOR_TAB,
    CONSOLE_TAB,
    PAPER_TRADER_TAB,
    SYSTEM_STATUS_TAB,
    PROOF_OF_ACCUMULATION_TAB,
)
```

Each of those names is a constant holding the label the tab bar shows. The skeletons take their label from their own surface, so the bar and the screen cannot carry two spellings of one name. Issue #450 renamed every one of those constants and reordered the tuple, and the section under this one holds what the file carries today.

`src/gui/main_tabs/main_window_surface.py` — the labels before issue #450

```
TRADING_TAB = "Trading"
MARKET_INSPECTOR_TAB = "Market Inspector"
BOT_SWARM_TAB = "Bot Swarm"
ASSET_CHARTS_TAB = "Asset Charts"
HISTORY_TAB = "History"
SIMULATOR_TAB = "Simulator"
CONSOLE_TAB = "Console"

PAPER_TRADER_TAB = paper_trader_tab_surface.HEADING
SYSTEM_STATUS_TAB = system_status_tab_surface.HEADING
PROOF_OF_ACCUMULATION_TAB = proof_of_accumulation_tab_surface.HEADING
```

### The order, the names and the colours today

The bar now opens on Sim and every tab carries a single word. The order below is what the window ends with, and it is decided in the one place named above.

`src/gui/main_tabs/main_window_surface.py` — `CANONICAL_TAB_ORDER`

```
CANONICAL_TAB_ORDER = (
    SIM_TAB,
    PAPER_TAB,
    LIVE_TAB,
    CHARTS_TAB,
    INSPECTOR_TAB,
    SWARM_TAB,
    ACCUMULATION_TAB,
    HISTORY_TAB,
    STATUS_TAB,
    CONSOLE_TAB,
)
```

Eight of the ten labels are new words for screens that already existed. History and Console were already one word and did not change. Each constant was renamed to match the word it now holds.

`src/gui/main_tabs/main_window_surface.py` — the labels

```
LIVE_TAB = "Live"
INSPECTOR_TAB = "Inspector"
SWARM_TAB = "Swarm"
CHARTS_TAB = "Charts"
HISTORY_TAB = "History"
SIM_TAB = "Sim"
CONSOLE_TAB = "Console"

PAPER_TAB = paper_trader_tab_surface.HEADING
STATUS_TAB = system_status_tab_surface.HEADING
ACCUMULATION_TAB = proof_of_accumulation_tab_surface.HEADING
```

The eight renames, in the order the bar reads them:

| was | is now |
|---|---|
| Simulator | Sim |
| Paper Trader | Paper |
| Trading | Live |
| Asset Charts | Charts |
| Market Inspector | Inspector |
| Bot Swarm | Swarm |
| Proof of Accumulation | Accumulation |
| System Status | Status |

Sim, Paper and Live carry the promotion pipeline, and three different grounds separate them at a glance. The other seven tabs share the gold ground.

`src/gui/main_tabs/main_window_surface.py` — `TAB_GROUNDS`

```
TAB_GROUNDS = {
    SIM_TAB: BLACK_GROUND,
    PAPER_TAB: WHITE_GROUND,
    LIVE_TAB: GOLD_GROUND,
    CHARTS_TAB: GOLD_GROUND,
    INSPECTOR_TAB: GOLD_GROUND,
    SWARM_TAB: GOLD_GROUND,
    ACCUMULATION_TAB: GOLD_GROUND,
    HISTORY_TAB: GOLD_GROUND,
    STATUS_TAB: GOLD_GROUND,
    CONSOLE_TAB: GOLD_GROUND,
}
```

No tab holds a colour of its own. A ground names two theme tokens, and every theme fills those tokens with its own values, so gold is the yellow that suits each theme rather than one hex value everywhere.

`src/gui/theme_engine.py` — the six tab tokens each theme carries

```
theme             tab_gold_bg tab_gold_text tab_black_bg tab_black_text tab_white_bg tab_white_text
cyberpunk_dark    #fcee0a     #8c0018       #0a0a0f      #ff5577        #f5f5fa      #0a0a0f
neon_light        #f0cf1f     #99001f       #1a1a2e      #ff6b8a        #ffffff      #1a1a2e
classic_terminal  #ffff00     #990000       #0a0a0a      #ff3333        #e8e8e8      #0a0a0a
minimal_modern    #eab308     #7f1d1d       #1a1a1a      #ff6b6b        #ffffff      #1a1a1a
glass_metal       #e8b34a     #6b1020       #1c1c24      #ff6688        #e8e8f0      #1c1c24
```

Every pair above clears WCAG 2.2 AA at 4.5 to 1. The narrowest margin is red on gold under Minimal Modern, at 5.22 to 1.

The Qt bar paints those colours itself, because a Qt style sheet cannot colour one tab by its position. The theme's own style sheet sets the six colours on the bar, so switching theme repaints it.

`src/gui/main_tabs/main_tab_bar.py` — `MainWindowTabBar.paintEvent`

```
painter.fillRect(self.tabRect(index), ground)
painter.setPen(text)
painter.drawText(self.tabRect(index), int(Qt.AlignCenter), self.tabText(index))
```

The React bar draws the same ten buttons with the same two colours each. The window publishes the colours beside the labels, so the page holds no palette of its own.

`src/gui/main_tabs/main_window_surface.py` — what the frontend reads

```
"tab_labels": list(self.tab_labels),
"tab_methods": dict(self.tab_methods),
"tab_colours": {tab: dict(pair) for tab, pair in self.tab_colours.items()},
"theme": self.theme,
```

The Live tab, which the manual's part list calls the Trading Tab, has its own section, 06-trading-tab.md, and the Indicator Voting Panel has 07-indicators.md.

A strategy reaches real money through four steps: Market Inspector, Simulator, Paper Trader, Live. Each step is a gate, and all four screens are built. 08-tabs/promotion-pipeline.md draws the chain and marks where a step still hands nothing to the next.

The sections below run in the order 04-manual-parts.md lists the tabs.

## Main Window

### Portfolio Information Panels

Found at the top of the Main Window at all times, this provides metrics for total performance and activity of the platform.

The window is built in three parts: the menu bar, the header strip, and the tab row. The panels above fill that strip, and each tab below sits in that row.

The strip is one row. Five labelled columns run down the left — SPENDABLE, REALISED, LOCKED, MATURE and EXCH — and five counter cards close it on the right.

`src/gui/main_tabs/header_strip.py` — `HeaderStripMixin._build_header_strip`

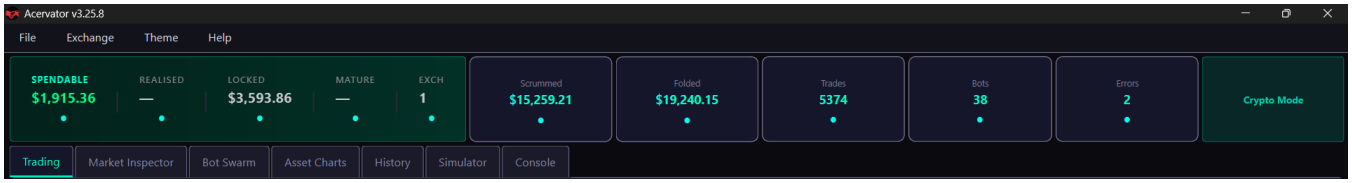
```
self._spendable_widget = SpendableProfitsWidget()
top_row.addWidget(self._spendable_widget, stretch=3)
```

The five cards are Scrummed, Folded, Trades, Bots and Errors. One aggregate call fills every field on the strip once a tick, which keeps two cards from disagreeing about the same fleet.

`src/gui/main_window.py` — `_refresh_dashboard`

```
agg = self._bot_manager.get_aggregate_stats()
_scr = float(agg.get("total_scrummed_usd", 0.0) or 0.0)
_fld = float(agg.get("total_folded_usd", 0.0) or 0.0)
self._stat_scrummed.set_value(f"${_scr:,.2f}")
self._stat_folded.set_value(f"${_fld:,.2f}")
```

Each field carries a privacy dot that masks the value through the registry the Swarm tab shares. The strip hides itself while the Sim tab or the Paper tab is active, and the Paper tab draws four figures of its own in place of it. The absent live strip is itself the signal that the screen is not live trading.



The header strip and the tab row, with Privacy Mode off.

The chrome above the strip carries four menus. File holds Settings, Reset All Settings and Exit. Exchange holds Add Exchange. Theme lists the five display names the theme engine declares, and Help holds About.

`src/gui/main_window.py` — the menu bar

```
file_menu = menu_bar.addMenu("&File")
file_menu.addAction("&Settings", self._open_settings)
file_menu.addAction("&Reset All Settings", self._reset_settings)
```

### What the Theme menu does

The Theme menu paints and it stores. One press applies the theme to the Qt window, repaints the tab labels, repaints every React page already open, and writes the name under the `theme` key, so the choice is still there at the next start.

`src/gui/main_window.py` — `_switch_theme`

```
painted = repaint_pages(self, name)
self._remember_theme(name)
```

A page already up takes the new palette without being rebuilt. One statement per web view rewrites the six chrome colours on the document root, so every drawn row and the scroll position stay where they were.

```
the Market Inspector page, 900x600, 92,250 sampled pixels of ground

before the press   #0a0a0f
after the press    #f5f5fa    184,500 sampled pixels differing
pressed back       #0a0a0f    identical to before
the Activity Log   1 React pages repainted in neon_light
```

A page built later opens on the same theme. Every React page host named one theme in its own source; each now leaves the name unset, and the page reads back the theme the window is painted in.

`src/gui/react_history_panel.py` — `page_theme`

```
return stored_theme(applied_theme() if theme is None else theme)
```

Driven with nothing passed to the builder, the Alerts, Console, History and Market Inspector pages each painted a different ground under a stored Cyberpunk Dark and a stored Neon Light, at 184,500 to 187,500 sampled pixels differing per page. All four were identical before.

The window's theme table and the chart chrome table are one declaration now. The chart table is built from the names the theme table declares, so a name cannot sit in one and not the other, and the five colour bodies it holds are unchanged.

`src/gui/tradingview_chart.py` — `CHART_THEMES`

```
CHART_THEMES: dict[str, dict[str, str]] = {
    name: _CHART_CHROME.get(name, _CHART_CHROME[DEFAULT_THEME_NAME])
    for name in THEMES
}
```

The theme does not reach the contents of a page. A page's rows, panes and buttons arrive on the payload as a Qt stylesheet string built from the one dark token table, which carries no theme dimension. The Console tab, driven under three themes, painted the same five grounds every time, and each of the five is a token from that table.

`src/gui/design_system.py` — the grounds the Console tab painted under all three

```
SURFACE_CHART = "#0a0a12"  # Chart, console and group-box ground
SURFACE_CONSOLE = "#05050a"  # Console ground
SURFACE_CONSOLE_HEADER = "#0a0a14"  # Console header ground
```

Privacy Mode is off in the figure, so each field draws its own value rather than four asterisks. SPENDABLE and LOCKED carry dollar figures. EXCH counts the open exchange sub-tabs. Crypto Mode at the right swaps the window between the crypto layer and the stock layer.

REALISED and MATURE draw an em dash. Both take a literal absence at the one call site that fills them, on every tick.

`src/gui/main_window.py` — `_refresh_dashboard`

```
self._spendable_widget.update_profits(
    {
        "spendable": _wallet_cash,
        "total_realised": None,
        "locked": _crypto_value,
        "mature": None,
        "exchange_count": exchanges,
    }
)
```

Turning Privacy Mode on masks that em dash to asterisks, which reads as a hidden number rather than an empty column. 06-trading-tab.md carries the proposal for the first of the two. Issue #428 carries a second disagreement on this strip, between what the counter tooltips promise and what the cards draw.

Both columns now carry an exchange figure. The window hands the strip the same payload the React strip receives, built by one function, so the two hosts cannot show different numbers.

`src/gui/main_window.py` — `_refresh_dashboard`

```
self._spendable_widget.update_profits(
    header_strip_surface.profits_payload(agg, exchanges)
)
```

REALISED is the venue's matched profit and loss across the fleet. MATURE is the profit held in positions worth more than three times what they cost. Where the venue has answered for no bot, both stay empty, and the mask now leaves an empty column empty instead of drawing four asterisks over it.

`src/gui/main_tabs/header_strip_surface.py` — `exchange_amount`

```
answered = int(data.get(EXCHANGE_FRESHNESS_KEY, 0) or 0)
if answered <= 0:
    return None
```

The venue's portfolio breakdown answers each position's cost basis, average entry price and unrealised profit, and each bot reads those three from it. The breakdown carries no lifetime realised figure for a spot position, so REALISED is the platform's first-in, first-out match over every fill the venue holds for each bot's symbol, one figure per bot, summed across the fleet; a bot pages the whole history once and then one page per refresh, and logs the venue's cost basis against the cost of the buys left open.

`src/trading/scrumming/reconciliation.py` — `fetch_fill_history`

```
if self._fill_history is None:
    self._fill_history = FillHistory(self.config.symbol)
return await self._fill_history.refresh(self.exchange)
```

Detail: 08-tabs/portfolio-panels.md.

### 2026-09-09 07:23 - #128 - the header strip's spendable panel is a panel, its dot is not

The header strip is one registered panel that draws two other modules inside itself. The shell mounts `header_strip` as chrome, above the tab bar and on show for every tab. Inside it, the spendable panel and the five counter cards each fill a space the strip leaves for them, and each of those spaces is a mount of its own.

`src/gui/web/header_strip.js` — the spendable space, as the strip draws it

```
function renderSpendable(target, model) {
  var host = global.acervatorPanelHost;
  var space = spaceNamed(target, SPENDABLE);
  if (!host || space === null) {
    return null;
  }
  space.setAttribute(CHILD_ATTR, SPENDABLE_MODULE);
  return host.mount(SPENDABLE_MODULE, space, model) ? space : null;
}
```

`spendable_profits.js` now registers, so the shell knows it by name and the strip draws it the way the Live tab draws its Activity Log and the Charts tab draws its chart. The registration names no bridge method, so the tab bar gives it no tab. Nineteen panels register where eighteen did before.

The privacy dot is a different kind of thing. It has no space and no host: two other modules render it as an element inside their own trees, so the panel host never draws it and a registration would name a panel with nowhere to go. Its Registers in Electron cell now reads `-`, the mark this table already uses for a column that does not apply to a row.

`src/gui/web/spendable_profits.js` — where the dot comes from

```
function dotSpan() {
  var own = global.acervatorDot;
  if (own && own.Dot) {
    return own.Dot;
  }
  var api = global.acervatorHeader;
  return api && api.PrivacyDot ? api.PrivacyDot : null;
}
```

Both builds were read on one fleet snapshot and one privacy state, with LOCKED and Folded masked and every other field revealed. Nothing was written into the page: the shell fetched every figure over the bridge. Every value, label, glyph, colour and tooltip on the strip agrees between them.

The strip's slots now share the row the way the Qt layout shares it. A Qt stretch divides the whole width; a CSS `flex-grow` divides only what is left over, so the counters drew at five different widths. With a zero flex basis the five cards measure 148 pixels each against Qt's 153, and the spendable panel takes 34.6 per cent of the row against Qt's 34.8.

| Qt file | React module | Uses React | Bridge | Manifest | Registers in Electron | Ships in the build | RENDERS | Scope |
|---|---|---|---|---|---|---|---|---|
| `src/gui/widgets/spendable_profits.py` | `spendable_profits.js` | yes | yes | yes | yes | yes | yes | in scope |
| `src/gui/widgets/privacy_dot.py` | `privacy_dot.js` | yes | yes | yes | - | yes | yes | in scope |

The strip fetches its own figures. The shell asks for the strip with an empty request, so both surfaces now read the fleet the bridge is bound to, the way the History, Charts, Trading and Inspector surfaces already read theirs. The aggregate is `BotManager.get_aggregate_stats`, the same call the Qt window's own tick makes, and the exchange count is the length of `SettingsManager.list_exchanges`, which is the list the window builds

one sub-tab from. Neither figure is derived in the surface or in the page.

`src/gui/main_tabs/header_strip_surface.py` — the request fills itself

```
def live_view_model(params: dict, live: Any) -> dict:
    asked = dict(params or {})
    if asked.get(STATS_PARAM) is None:
        aggregate = fleet_aggregate(live)
        if aggregate is not None:
            asked[STATS_PARAM] = aggregate
```

The spendable columns take the same aggregate through `header_strip_surface.profits_payload`, the one builder the Qt tick already calls, so the two hosts read one snapshot and cannot disagree.

Each privacy dot in the strip's columns now carries the pointing hand the Qt dot sets on itself. `privacy_dot_surface.dot_view` publishes `cursor_shape`, and the two surfaces that build a dot for this strip now keep it. `dashboard_stat_card_surface` still cuts it away, so the five counter dots draw with the ordinary pointer; that card is its own row.

Qt draws a one pixel panel edge around each counter card where the page draws none, because `dashboard_stat_card_surface.FRAME_STYLE` is empty and the `StyledPanel` shape reaches the page as an attribute no rule reads. That is the same row.

### 2026-09-09 08:42 - #128 - the counter cards and the bot table are components

The five counter cards and the Scrumming bot table are drawn inside a panel that has already registered, and neither registers in its own right. The strip leaves five spaces and builds a request for each one, carrying that counter's caption, amount, privacy field and whether it is pressable. The bot table is drawn once for the exchange the Trading tab is showing, from a payload the module keeps under that exchange's own name.

`src/gui/web/header_strip.js` — one request per counter

```
function cardRequest(card) {
  var asked = {};
  asked[CARD_LABEL_PARAM] = text(card[LABEL]);
  asked[CARD_VALUE_PARAM] = text(card[TEXT]);
  if (text(card[FIELD_ID]) !== undefined) {
    asked[CARD_FIELD_ID_PARAM] = text(card[FIELD_ID]);
  }
  asked[CARD_CLICKABLE_PARAM] = card[CLICKABLE] === true;
  return asked;
}
```

The panel host keeps one host element per panel name and asks each panel with one request of its own. Registering either module was driven in the running shell to see what it would give: the card drew an empty caption and the default amount, and the table drew its ten headers over no rows, each into a host of its own under the panel area, while the strip's five counters kept their figures. Both cells now read `-`, the mark this table already uses for a column that does not apply to a row, and the mark the privacy dot took for the same reason.

Each counter card now draws the one pixel edge, the rounded corner and the padding that Qt paints around it. Qt takes all three from the theme rule for a framed panel; the shell selects no theme, so the page had nothing to

read and drew a plain box. The card's own skin carries them, built from the design tokens rather than from a colour written into the page.

`src/gui/main_tabs/dashboard_stat_card_surface.py` — the card's skin

```
FRAME_STYLE = (
    "border-style: solid; "
    f"border-width: {FRAME_LINE_WIDTH_PX}px; "
    f"border-color: {ds.OUTLINE}; "
    f"border-radius: {ds.RADIUS_SM}px; "
    f"padding: {ds.SPACE_CARD_PAD}px;"
)
```

The five counter dots now carry the pointing hand as well. The card cut its dot down to five fields and left out the cursor the dot publishes; it keeps all eight now, which is what the spendable columns already do.

Both builds were read on one fleet snapshot with every field revealed. Nothing was written into the page: the shell fetched the aggregate and the bot list over the bridge, and breaking each source in turn emptied the figures it feeds and left the other side drawing. Every label, amount, colour, glyph, tooltip, column header, row order and cell text agrees between the two.

Two differences remain, and both belong to other rows. Qt fills each card with the theme's card ground, which no design token carries. And the counter space is 141 pixels wide on all five while the card inside takes its content width, which is how the strip has placed its counters since before this change.

| Qt file | React module | Uses React | Bri dge | Mani fest | Registers in Electron | Ships in the build | REND ERS | Sco pe |
|---|---|---|---|---|---|---|---|---|
| `src/gui/widgets/dashboard_stat_card.py` | `dashboard_stat_card.js` | yes | yes | yes | - | yes | yes | in sc ope |
| `src/gui/widgets/bot_status_table.py` | `bot_status_table.js` | yes | yes | yes | - | yes | yes | in sc ope |

## Market Inspector Tab

The concept with the Market Inspector Tab is evaluate markets from a higher point of view and provide strategy proposals in three forms: Oppositional Trading Pairs (Trading Pairs w/ Opposing Trends), Bot Swarm Topologies (Bot Swarm Network Proposals), and Exchange Comparison Arbitrage.

The tab is now called Inspector. It sits fifth on the bar, on the gold ground with red text.

The tab splits in two. The left half holds the HTF Signals table and the Opposing Pairs table, both scored by one analyzer. One method drives the whole pipeline and keeps the results on that analyzer.

`src/trading/market_inspector.py` — `MarketInspector.scan_universe`

```
def scan_universe(
    self,
    candles_by_symbol_by_tf: dict,
    active_symbols: set,
    closes_by_symbol: dict,
) -> None:
```

The right half holds proposal cards from four detectors: momentum funnel, mean-reversion pair, sector cluster and distance to band. The engine takes a plain dictionary, unions the detectors, drops overlapping proposals by asset and caps the result.

`src/trading/topology_proposals.py` — `detect_all_topologies`

```
def detect_all_topologies(
    context: dict[str, Any],
    cap: int = PROPOSAL_CAP,
) -> list[dict[str, Any]]:
```

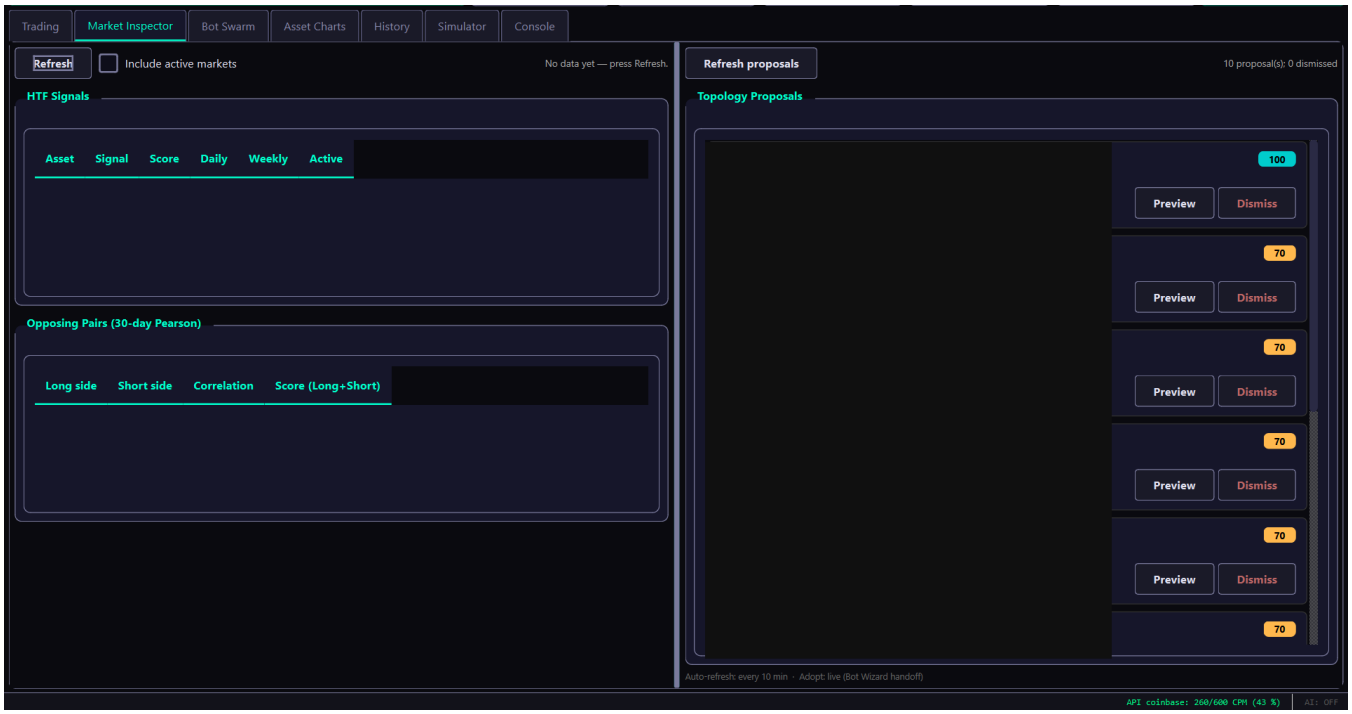
Adopting a card names the new bots, their combined budget and every existing wire the adopt would change before it creates anything.

`src/gui/main_window.py` — `_adopt_topology_proposal`

```
new_bots = [
    b for b in proposal.get("bots", []) if not b.get("existing_bot_id")
]
wires = list(proposal.get("wires", []))
new_count = len(new_bots)
```

Two of the three proposal forms reach the screen. Exchange comparison arbitrage has a module, `src/trading/arbitrage.py`, and no importer under `src/`; the tab draws no arbitrage panel.

In development.



The Market Inspector tab, before the first scan.

Refresh and the Include active markets checkbox sit above the two tables, and the status line reads `No data yet — press Refresh.` until a scan lands, which is the state the figure captures. Leaving the checkbox clear hides the markets a bot already holds.

HTF Signals carries six columns: Asset, Signal, Score, Daily, Weekly and Active. Opposing Pairs carries four: Long side, Short side, Correlation and Score. The pairing enumerates every long against every short and keeps the ones whose thirty-day return correlation sits in the configured negative window.

`src/trading/market_inspector.py` — `MarketInspector._find_opposing_pairs`

```
def _find_opposing_pairs(self, signals: list, closes_by_symbol: dict) -> list:
    """Enumerate long × short candidates; keep pairs whose 30-day
    return correlation sits in the configured negative window."""
    longs = [s for s in signals if s.direction == "long" and s.score >= 0.3]
    shorts = [s for s in signals if s.direction == "short" and s.score >= 0.3]
```

Refresh proposals runs the detectors, and the line beside it counts the proposals held and the cards dismissed. Each card names its archetype and the assets in it, then a line reading the score, the asset count, the wire count and the number of bots an adopt would create. The badge takes its colour from the score: teal at the high threshold and above, amber at the middle one, grey below. Every card in the figure comes from the sector-cluster detector. Preview lists the wires as Source, Target, Pct and Rationale.

Dismiss hides a card for a day, and the dismissal does not survive a restart. The write-through never raises, so the pane logs the failure and carries on, which leaves the dismissed count at zero on every launch.

`src/gui/main_tabs/market_inspector_topologies_surface.py` — `persist_dismissed`

```
def persist_dismissed(self) -> None:
    """Best-effort write-through. Never raises."""
    if self.dismiss_store is None:
        return
    try:
    self.dismiss_store.set(DISMISS_SETTINGS_KEY, dict(self.dismissed))
```

The key it writes is not one the settings schema declares, so the write fails every time. Issue #424 carries it.

The footer names the auto-refresh period and the adopt route. The status bar under it carries the API load pill, drawn green below half load, amber above it, and red past the monitor's safety percentage, then the `AI:` state label.

React draws this screen. Both halves are one web page inside a single view, and the build decides which class the tab builder makes. `src/gui/main_tabs/market_inspector_tab.py` — `_build_market_inspector_tab`

```
from ..variant_surface import MARKET_INSPECTOR, surface_class

self._market_inspector = surface_class(MARKET_INSPECTOR)()
```

`MarketInspectorReactTab` subclasses the Qt tab, so the scan cycle, the filtering and the analyzer writes are the same code on both sides. Five accessors are what the two sides answer differently: `_set_status`, `_set_refresh_enabled`, `_shown_signals`, `_fill_signal_rows` and `_fill_pair_rows`. The Qt tab writes them into its widgets and the React tab writes them into the page.

`src/gui/react_market_inspector_tab.py` — `MarketInspectorReactTab._build_ui`

```
self._web = QWebEngineView(self)
self._web_page = MarketInspectorPage(self)
self._web.setPage(self._web_page)
self._web.loadFinished.connect(self._on_load_finished)
self._web.setHtml(tab_html())
```

Refresh and the Include active markets checkbox report their press back to Python, and the right pane reports Refresh proposals, Preview, Dismiss, Cancel and Adopt the same way. Setting `ACERVATOR_VARIANT` to `qt` builds the Qt widgets instead, unchanged.

Detail: 08-tabs/market-inspector.md.

### 2026-09-09 09:42 - #128 - the HTF Signals and Opposing Pairs tables come out

Neither screen drew the two tables. The Qt tab built both group boxes and put neither into a layout, so no table ever reached the window. In the running shell, the React screen drew no table part at all, while its payload carried six signal columns and seven pair columns.

`src/gui/market_inspector.py` — `_fill_pair_rows`

```
def _fill_pair_rows(self, pairs: list) -> None:
    """Hold the opposing pairs the Opposing Trades stepper draws."""
    self._pairs = list(pairs)
    self._render_empty_notes()
```

The screen holds neither table by design. The six-zone layout moved HTF Signals into the reserved Phantom Bot zone, and it hands each opposing pair to the Opposing Trades stepper, one entry at a time. Both sides now drop the renderer, the column sets and the two empty sentences, and the tab keeps only the rows the zones read.

Nothing on the screen moved. The Qt picture and the React picture each match byte for byte before and after the change, and the drawn markup of the panel matches at 21,194 characters.

The earlier comparison that closed these two rows read its table items out of the payload. `signalRow` and `pairRow` handed back cells from the model, which agreed on both sides whatever the page drew. This change drops both. The run is in 2026-09-09_market_inspector_tables.md.

### 2026-09-09 23:05 - #128 - the Electron shell draws the Inspector tab and its proposals pane

The Inspector tab draws in the Electron shell. The shell asks the application which tabs it builds, matches `market_inspector.state` to the module that serves it, and gives that module the Inspector tab.

`desktop/renderer/panel_host.js` — `register`

```
    panels[name] = spec;
    dropFault(name);
    kinds[name] = declaredKind(name, spec);
```

The panel host now draws the proposals pane into the right-hand slot, under the name `market_inspector_topologies.js` registers. The slot used to be filled by a call straight to the module, so the registration drew nothing and removing it changed no pixel.

`src/gui/web/market_inspector.js` — `fillSlot`

```
    host.mount(TOPOLOGY_PANEL, slot, null);
```

`desktop/renderer/index.html` links `market_inspector.css`, the stylesheet that carries the colour, border and type of every part the two modules stamp. The file shipped and nothing loaded it, so every button on the screen drew in the browser default. The button rule reads the design token that holds the ground the theme gives a Qt push button.

`src/gui/web/market_inspector.css` — the button ground

```
   background: var(--SURFACE_2, var(--btn-bg));
```

The check boxes on the tab drew as the browser's own control, white and 13 pixels, where Qt paints an 18 pixel box with a `#7a7a9c` edge on `#0e0e1a`. That ground was in no token, so the table gained one under the name of the role it fills, and the stylesheet names the token rather than the value.

`src/gui/main_tabs/design_system_surface.py` — the check-box ground

```
SURFACE_INPUT = "#0e0e1a"
```

Both screens were driven on one payload of 54 markets, 300 daily candles each and three ranked proposals. Twenty-seven of twenty-seven picture items match, so rows 878 and 879 read `yes` under Registers in Electron. The Opposing Pairs rows match at nothing on either side, so that item proves the column set and not the rows. The run is in 2026-09-09_market_inspector_register.md.

## Asset Charts

Under the Asset Charts Tab, you will find our active bot (position) chart display. This will be upgraded to display only one chart at a time and will be able to display all indicators found in the Indicator Voting Panel.

The tab is now called Charts. It sits fourth on the bar, straight after Live, on the gold ground with red text.

The tab scrolls one panel per active bot, each painted with QPainter and no browser, carrying trade markers, position markers, grid lines, the standing tranche floors and the target balance line.

`src/gui/main_tabs/charts_tab.py` — `ChartsTabMixin._build_charts_tab`

```
def _build_charts_tab(self) -> None:
    """Build the Asset Charts tab and add it to the main tab widget."""
    # --- Tab 2: Charts ---
    self._charts_tab = TradeChartsTab()
    self._main_tabs.addTab(self._charts_tab, "Asset Charts")
```

The chart reads its overlays from the engine rather than computing any of them.

`src/gui/native_chart.py` — `CandlestickChart._compute_indicators`

```
candles = self._candles
self._bb_data = BollingerBands(20, 2.0).bands(candles)
vi_plus, vi_minus = VortexIndicator(14).lines(candles)
```

Candles arrive through a fetcher with three sources in order: exchange OHLCV first, a public API second, the last good fetch third.

`src/exchange/chart_data.py` — `ChartDataFetcher.fetch`

```
# --- Source 1: Exchange OHLCV via CCXT ---
if exchange is not None:
    try:
        candles, source = await self._fetch_exchange(
            exchange, symbol, timeframe, limit
        )
        if candles:
            self._set_cache(symbol, timeframe, candles)
            return candles, source
```

The Asset Charts tab, one panel per running bot.

The panel set rebuilds from the current bot roster, and the tab scrolls them.

The header line of a panel names the symbol, the last price, the bot state, the timeframe, the feed and the candle count. The line under it carries the open, high, low and close of the current candle, the change in price and in percent, and the volume. Rising candles draw teal and falling candles red.

The dashed lines across the plot are the standing fold-tranche floors, each labelled with its price. `TB-Ceiling` is the dollar-target line. The price axis runs down the right edge with the last price boxed on it, and the volume bars run along the foot.

The toolbar under each panel holds the TF picker and eight indicator toggles.

`src/gui/main_tabs/native_chart_surface.py` — `INDICATOR_TOGGLES`

```
INDICATOR_TOGGLES = (
    ("bb", "BB", "#50a0f0"),
    ("vortex", "Vortex", "#00ff88"),
    ("macd", "MACD", "#ffcc00"),
    ("stochrsi", "SRsi", "#ff9060"),
    ("ichimoku", "Ichi", "#c080ff"),
    ("volume", "Vol", "#80ffcc"),
    ("slingshot", "Sling", "#ff4488"),
    ("bbullseye", "BBull", "#ff00aa"),
)
```

Volume starts on and the other seven start off, which is the state the figure shows. Sling and BBull paint a placeholder shape rather than the indicator, and each says as much in its own tooltip. Issue #430 carries a tooltip on this toolbar that names a colour the chart does not paint.

The legend at the right names the two position markers, Invisible and On Book, and the feed label closes the row.

### One chart, chosen with the arrows

The tab now draws one chart at a time. A left arrow, a centre ticker list and a right arrow choose which traded asset it shows, and a readout beside them says which of how many is on screen. The list wraps, so neither arrow dead-ends, and the ticker readout is itself a drop-down. With no bots running the readout says zero of zero and both arrows are switched off.

`src/gui/widgets/trade_charts_tab.py` — the readout beside the arrows

```
POSITION_FORMAT = "{at} of {total}"
EMPTY_TICKER_TEXT = "No asset"
EMPTY_POSITION_TEXT = "0 of 0"
```

### The toggles, and the two that start off

Every indicator the chart can draw now starts switched on, and a check box for each sits along the bottom of the panel. Two start off, because each paints a filled shape over the price where the Bollinger bands and the Ichimoku cloud are drawn. Slingshot paints solid marks at a squeeze release and a snapback. BB Bullseye paints four shaded envelopes directly on the bands themselves.

`src/gui/native_chart.py` — one entry per drawn indicator

```
class ChartOverlay:
    key: str
    label: str
    colour_field: str
    pane: str
    occludes: bool
    draw: str
    tooltip: str
```

### The renderer is built to grow

`CHART_OVERLAYS` declares each indicator once, and the paint routine walks that list rather than naming any of them. Another overlay is one entry in the list and one draw method; no existing overlay is touched to add it.

The renderer holds no colour of its own. Every painted value resolves through `PALETTE_ROLES`, which names the theme field and the transparency each role reads, so a theme sets the look and the renderer does not.

### 2026-09-08 23:40 - #128 - the chart registers a panel in the shell

The chart now joins the Electron shell's roster of panels. Before this change the shell listed fifteen panels and said of this one that it registered nothing to draw. It now lists sixteen and says nothing.

`src/gui/web/native_chart.js` — the chart joins the roster

```
if (global.acervatorPanelHost) {
  global.acervatorPanelHost.register({
    render: renderChart,
    load: loadChart,
    loadError: loadError
  });
}
```

The chart takes no tab of its own. It names no bridge method the application serves a tab from, so the tab bar passes over it and the chart draws where it belongs, inside the Charts tab. The ten tabs the application reports are the same ten before and after.

The chart in the shell still draws its waiting line rather than a price. The Charts tab asks it for a symbol and a timeframe and sends none of the 180 candles the tab itself holds, and the symbol it sends is empty. Both values belong to the Charts tab file, not to the chart, so the row below keeps no under Registers in Electron until they are carried across.

### 2026-09-09 01:20 - #128 - the Charts tab hands the chart its data

The chart in the shell draws the price now. The tab passes it the asset it is following, the feed name, the candles it is holding and the size of the space the chart has, and the shell's panel host does the drawing rather than the tab reaching past it.

`src/gui/web/trade_charts_tab.js` — what the chart slot asks for

```
        return global.acervator.call(api.method, {
          reset: true,
          symbol: symbol,
          timeframe: mount.getAttribute(CHART_TF_ATTR),
          source: text(panel[SOURCE]),
          candles: candles,
          width: mount.clientWidth,
          height: mount.clientHeight
        });
```

The panel carries the asset it follows. It was built once with a blank name and renamed only its header line, so the name the chart was asked for was always blank.

`src/gui/main_tabs/trade_charts_tab_surface.py` — the panel's own asset

```
  def set_symbol_property(self, symbol: Any) -> None:
      """Rename the chart through its public setter, which repaints."""
      self.symbol = symbol
      self.label = symbol
```

Both builds now draw the same header, the same 180 candles, the same last price of 78,623.00, the same three axis prices and the same time axis. The two pictures still differ on the indicator lines. The Bollinger bands, the Ichimoku cloud and the Vortex, MACD and Stochastic RSI panes draw in the Qt build and in neither place in the shell, because nothing hands the chart those five readings yet. The eight indicator switches under the Qt chart are absent in the shell for the same reason. The row below keeps no under Registers in Electron until the readings arrive.

### 2026-09-09 03:05 - #128 - the indicators draw in both builds

The chart in the shell draws every indicator the Qt chart draws. The five readings now reach it: the Bollinger bands with their cloud, the Ichimoku cloud with its four lines and the lagging line, and the Vortex, MACD and Stochastic RSI panes under the price. The eight switches sit under the chart with Slingshot and BB Bullseye off, and the two position markers sit on the toolbar beside the feed name.

Each reading is the one the indicator itself published. The chart asks the five indicator classes for their values and paints what they answer.

`src/gui/main_tabs/native_chart_surface.py` — the chart takes its readings

```
        self.set_indicator_series(
            bb=BollingerBands(BOLLINGER_PERIOD, BOLLINGER_STD).bands(candles),
```

Both builds were driven from the same 38 running bots and the same 180 daily A15/USD candles. Both draw the same header, the same candles, the same last price of 78,623.00, the same four axis prices, the same time axis, and the same three panes reading Vortex 0.8761, MACD -245.82 and Stoch RSI 0.2318. The row below now reads `yes` under Registers in Electron.

Two figures moved to make that true. The Qt chart asked for 492 pixels and was given 250 until an indicator switch was pressed, so its MACD pane, its Stochastic RSI pane and its time axis were cut off; it now takes its full height as soon as the candles arrive. The panes were also listed as MACD, Vortex, Stochastic RSI in one place and Vortex, MACD, Stochastic RSI in the other; the switch order decides, so both now read Vortex, MACD, Stochastic RSI.

The volume bars are empty in both pictures. The public feed serves no volume with its daily candles, so that one row of the comparison rests on two empty strips.

`src/gui/theme_engine.py` — the chart tokens each theme carries

```
chart_bg_top: str = "#08080e"
chart_up: str = "#00e5a0"
chart_down: str = "#ff2d6f"
chart_zone_fold: str = "#fcee0a"
```

Detail: 08-tabs/asset-charts.md.

## Live Tab

The Live tab is where the bots trade real money on a real exchange. The exchange connections fill the left, the Indicator Voting Panel fills the right, and the Activity Log and the API Interaction Log close the foot.

The tab is now called Live. It sits third on the bar, straight after Paper, on the gold ground with red text.

### 2026-09-09 06:35 - #128 - the Live tab's two widgets register a panel

The Indicator Voting Panel and the Activity Log now register with the shell's panel host, and the Live tab draws both through it. Before the change the shell listed sixteen registered panels and named these two as modules that draw nothing; it lists eighteen now and names neither.

`src/gui/web/status_log.js` — what each of the two modules adds

```
if (global.acervatorPanelHost) {
  global.acervatorPanelHost.register({
    render: renderLog,
    load: loadLog,
    loadError: loadError
  });
}
```

Neither registration names a bridge method, so the tab bar gives neither a tab of its own. The Live tab keeps a slot for each and hands the slot to the host, which is how the Charts tab already draws the price chart.

`src/gui/web/trading_tab.js` — the Live tab's slot for the Activity Log

```
  slot.setAttribute(CHILD_ATTR, STATUS_LOG_MODULE);
  return host.mount(STATUS_LOG_MODULE, slot, model) ? slot : null;
```

The host is the draw path now, not a second name for it. With both registrations taken out and the same reading driven in, each slot fell from 43,264 and 4,532 characters of markup to 82 and 72, and each carried the sentence naming the panel that did not draw.

Both builds were driven with one reading: 180 daily A15/USD candles from CoinGecko through the voting engine, at 15m, 1h and 1d, and eight Activity Log messages covering a placed, a filled and a sent trade, a wire flow, a wire stack, a warning and an error. The Qt tab was built at the size the shell gives the React tab, 1386 by 708.

The twelve indicator columns carry the same name, direction and bar in both. Each bar fills its share of a 70-pixel plot.

```
column  direction  confidence  Qt height  React height
BB      NEUTRAL    0.000       2.00       2.00
VTX     BEARISH    0.328       22.96      22.69
MACD    BEARISH    0.150       10.50      10.38
SRsi    BEARISH    0.200       14.00      13.84
Ichi    BEARISH    0.210       14.70      14.53
Vol     NEUTRAL    0.000       2.00       2.00
Sling   NEUTRAL    0.000       2.00       2.00
ADX     BEARISH    0.267       18.69      18.48
STrd    BULLISH    0.261       18.27      18.05
ZSc     NEUTRAL    0.000       2.00       2.00
KER     NEUTRAL    0.000       2.00       2.00
RSI     NEUTRAL    0.000       2.00       2.00
```

Net, Comp and Conf stand in the last three of the ten columns in both, take the same three directions, and run from the ceiling of the top row of bars to the floor of the bottom row. Only their bases carry a label, in a strip 22 pixels deep under that floor.

No horizontal line crosses the three. Every element in the React panel was read for a top border, a bottom border or a background gradient: ninety-three carry one, and every box ends at or before 1185.93, where the Net column starts. The 1 pixel each indicator column carries is that column's own base, which Qt paints as a line from the left margin to the x of the Net column. The dotted increments sit on those same cells and on no

other.

```
                 Qt                        React
ceiling          171                        267.2
floor            428                        523.2
label strip      428 to 450                 523.2 to 545.2
columns          8, 9, 10 of 10             8, 9, 10 of 10
rules end at     437, the Net column        1185.9, the Net column
```

The Activity Log draws the same eight lines in the same order with the same text. Each line takes the colour of its kind, and a trade line is raised to 14 pixels and bold.

```
line                          colour            size  weight
Bot a15_core started          rgb(0,255,204)    12    normal
SCRUM PLACED                  rgb(255,170,0)    14    bold
SCRUM FILLED                  rgb(0,255,136)    14    bold
WIRE FLOW                     rgb(255,102,221)  12    bold
WIRE STACK                    rgb(255,204,68)   12    bold
Coinbase rate limit           rgb(255,170,0)    12    normal
Reconciliation refused        rgb(255,51,102)   12    normal
FOLD SENT                     rgb(0,255,204)    14    bold
```

Each mini-panel table shows its header and two rows in both builds, and both cut the 1d row off the bottom. Qt fixes the table to that height and turns both scrollbars off; the React table read neither of the two numbers the payload publishes for it, stood two rows taller, and pushed the three pillar labels and the six lower bar labels below the panel. It reads them now.

`src/gui/web/indicator_panel.js` — the height the table body takes

```
    var bodyStyle = {
      display: BLOCK,
      maxHeight: length(slackHeight(table)),
      overflow: HIDDEN
    };
```

One line is drawn in one picture and not the other. The panel's locks line, `No active timeframe locks`, sits inside the Qt panel and two pixels below the bottom of the React one, which reaches it by scrolling. The Live tab's own splitter is what differs: Qt gives its top section 472 pixels of the 708 the tab has, the React tab gives it 411, and the panel cannot draw shorter than 415 because each bar graph carries a 100-pixel minimum in both.

Detail: 06-trading-tab.md. Detail: 07-indicators.md.

## Converting the Interface to React

Every screen in this application is drawn by Qt. Issue #128 replaces them with React and keeps the backend in Python. This section is the running record of that work. One row per screen, one column per step, and a mark only where the step is finished.

One screen draws React in the application today: the History table. It runs inside the Chromium that the Qt toolkit already ships, not inside Electron.

`src/gui/react_history_panel.py` — `HistoryWebTable._setup_ui`

```
self._web = QWebEngineView()
self._web.loadFinished.connect(self._on_load_finished)
self._web.setHtml(panel_html(theme))
layout.addWidget(self._web, 1)
```

### What the tree holds

Measured on 5 September 2026 over the tracked files.

```
React modules             70 files, 80,292 lines, 0 stub markers
                          65 of them call React.createElement
                          React and ReactDOM are vendored on disk
View-model modules        74 files, 74,207 lines
Parity tests              72 files, 170,244 lines
Electron shell             8 code files, 791 lines, plus a package manifest
Renderer manifest         70 names
Qt modules under src/gui  69, every one importing PySide6
Tabs drawn by React        0
```

No file in this repository has ever carried a JavaScript extension other than plain `.js`. A history search over every branch returns no commit that added, deleted or renamed one, and the same search for the plain extension returns 81 paths, so the search does find a file that existed.

```
git log --all --diff-filter=ADR --name-only -- "*.jsx" "*.tsx" "*.ts"   no commits
git log --all --diff-filter=ADR --name-only -- "*.js"                   81 paths
```

### What the Electron shell drew

The shell has been started once, and what it drew is measured. All 70 renderer modules loaded and none failed. Two of them register a panel for the page to draw, so the page had nothing to ask the other 68 for. The window showed an empty History table, with no tab bar and no navigation.

```
React modules the page list names   70
modules that failed to load          0
modules that register a panel        2
panels the page asked to draw        2
panels that failed to draw           0
```

The two that register are the Console tab panel and the header strip. The full per-file record is in docs/audits/2026-09-05_128_file_verification.md.

### The tab bar the shell draws now

The shell builds the same ten tabs the application builds, with the same labels and in the same order. It holds no list of its own. `main_window.state` reports `tab_labels` and `tab_methods`; the bar keeps one tab per label, filled by the registered panel that calls that label's bridge method.

```
Sim  Paper  Live  Charts  Inspector
Swarm  Accumulation  History  Status  Console
```

A module that draws a screen inside a dialog registers no bridge method and takes no tab. `market_inspector_tab.js` and `bot_swarm_tab.js` are the two: both belong to the Live Bot Settings dialog.

Measured in one Electron launch, with the backend serving a two-bot fleet:

```
tabs on the bar                     10
panels that drew text               10
panels that failed to draw           0
child panels drawn inside a tab      7
```

The header strip stays above the bar for every tab.

### How to read the table

Each row is a Qt module the running window builds. The columns are the steps of the conversion, in the order the work happens.

| Column | What the mark means |
|---|---|
| React module | A renderer module in `src/gui/web` serves this screen |
| Surface | A Python view model of the screen exists, with no toolkit in it |
| Bridge | The view model is registered as a method the frontend may call |
| Manifest | The renderer module is named in the shell's module list |
| Ships in the build | The built React application carries the module inside it |
| RENDERS | What the operator sees when he opens that screen |
| Scope | Whether the row is work this item must do |

The last column is the item. A row is finished only when it reads `yes`. Every other column can be marked while the operator sees no change, and that is the difference between wiring written and a screen replaced.

The RENDERS column takes three values, and none of them is derived from the bridge or the manifest. `yes` means React draws where Qt used to. `shell` means the module draws in the Electron shell and nowhere else. A dash means the Qt widget is still the screen.

The Scope column names the module that builds a screen the item has still to convert. Header strip and window chrome are the parts of the Main Window that sit outside the tab row.

The table carries nine columns and no tab column. The screen a module draws is named in that screen's own section above. Header strip and window chrome are the parts of the Main Window that sit outside the tab row.

The Scope column takes five values.

| Value | What it means |
|---|---|
| in scope | The running window builds this screen, and the cell names what builds it |
| React side | The file exists because of this conversion, so a React module for it would be a module for a module |
| not a screen | The module defines no view |
| builds the shell | The module fills the tab bar rather than sitting in it |
| shelved | Nothing the window builds reaches it |

The walk that answers the column starts at the main window. It follows an imported name only where the module calls that name, subclasses it or reads an attribute of it, so a re-export block reaches nothing. A row is in scope when some module on that walk builds one of the classes the row defines.

`tools/conversion_scope.py` writes the column, so the answers are re-measured rather than typed.

### The conversion table

| Qt file | React module | Uses React | Bridge | Manifest | Registers in Electron | Ships in the build | RENDERS | Scope |
|---|---|---|---|---|---|---|---|---|
| `splash_screen.py` | no | - | yes | no | no | - | no | shelved |
| `src/gui/alerts_tab.py` | `alerts_tab.js` | yes | yes | yes | - | yes | yes | in scope |
| `src/gui/analytics_tab.py` | `analytics_tab.js` | yes | yes | yes | no | yes | no | shelved |
| `src/gui/audio_suite.py` | `audio_suite.js` | yes | yes | yes | no | yes | no | shelved |
| `src/gui/bot_live_settings.py` | `bot_live_settings.js` | yes | yes | yes | yes | yes | yes | in scope |
| `src/gui/bot_swarm_list.py` | removed | - | - | - | - | - | - | deleted |
| `src/gui/bot_visualizer.py` | `bot_visualizer.js` | yes | yes | yes | yes | yes | yes | in scope |
| `src/gui/bot_wizard.py` | `bot_wizard.js` | yes | yes | yes | - | yes | yes | in scope |
| `src/gui/buy_confirmation_dialog.py` | `buy_confirmation.js` | yes | yes | yes | - | yes | yes | in scope |
| `src/gui/competition_tab.py` | `competition_tab.js` | yes | yes | yes | no | yes | no | shelved |
| `src/gui/crypto_news_ticker.py` | `crypto_news_ticker.js` | yes | yes | yes | - | yes | shell | in scope |
| `src/gui/history_qt_table.py` | no | - | no | no | no | - | no | React side |
| `src/gui/history_tab.py` | `history_tab.js` | yes | yes | yes | yes | yes | yes | in scope |
| `src/gui/indicator_panel.py` | `indicator_panel.js` | yes | yes | yes | yes | yes | shell | in scope |
| `src/gui/init_wizard.py` | `init_wizard.js` | yes | yes | yes | no | yes | no | shelved |
| `src/gui/instance_consent_dialog.py` | `instance_consent_dialog.js` | yes | no | yes | no | yes | no | shelved |
| `src/gui/journal_tab.py` | `journal_tab.js` | yes | yes | yes | no | yes | no | shelved |
| `src/gui/launcher.py` | `launcher.js` | yes | yes | yes | no | yes | no | shelved |
| `src/gui/live_bot_window.py` | no | - | yes | no | no | - | no | shelved |
| `src/gui/live_settings/bot_swarm_tab.py` | `bot_swarm_tab.js` | yes | yes | yes | yes | yes | yes | in scope |

| Qt file | React module | Uses React | Bridge | Manifest | Registers in Electron | Ships in the build | RENDERS | Scope |
|---|---|---|---|---|---|---|---|---|
| src/gui/live_settings/fold_chrome.py | fold_chrome.js | yes | yes | yes | - | yes | yes | in scope |
| src/gui/live_settings/fold_tokens.py | fold_tokens.js | yes | yes | yes | no | yes | no | not a screen |
| src/gui/live_settings/fold_tranches_tab.py | fold_tranches_tab.js | yes | yes | yes | - | yes | yes | in scope |
| src/gui/live_settings/market_inspector_tab.py | market_inspector_tab.js | yes | yes | yes | yes | yes | yes | in scope |
| src/gui/live_settings/phantom_bots_tab.py | phantom_bots_tab.js | yes | yes | yes | - | yes | yes | in scope |
| src/gui/live_settings/positions_held_tab.py | no | - | no | no | no | - | - | in scope |
| src/gui/live_settings/settings_tab.py | live_settings_tab.js | yes | yes | yes | no | yes | yes | in scope |
| src/gui/live_settings/stack_tranches_tab.py | stack_tranches_tab.js | yes | yes | yes | - | yes | yes | in scope |
| src/gui/live_settings/status_tab.py | no | - | yes | no | no | - | yes | in scope |
| src/gui/main_tabs/audio_suite_surface.py | no | - | yes | no | no | - | no | React side |
| src/gui/main_tabs/buy_confirmation_surface.py | no | - | yes | no | no | - | no | React side |
| src/gui/main_tabs/console_log_handler.py | no | - | no | no | yes | - | - | in scope |
| src/gui/main_tabs/console_tab.py | console_tab.js | yes | yes | yes | yes | yes | yes | in scope |
| src/gui/main_tabs/empty_tabs.py | no | - | no | no | no | - | - | shelved |
| src/gui/main_tabs/header_strip.py | header_strip.js | yes | yes | yes | yes | yes | yes | in scope |
| src/gui/main_tabs/proof_of_accumulation_tab.py | proof_of_accumulation_tab.js | yes | yes | yes | yes | no | shell | in scope |
| src/gui/main_tabs/stock_main_window_surface.py | no | - | yes | no | no | - | no | React side |
| src/gui/main_tabs/system_status_tab.py | system_status_tab.js | yes | yes | yes | yes | no | yes | in scope |
| src/gui/main_tabs/trading_tab.py | trading_tab.js | yes | yes | yes | yes | yes | - | in scope |
| src/gui/main_tabs/tradingview_chart_surface.py | no | - | yes | no | no | - | no | React side |
| src/gui/main_window.py | main_window.js | yes | yes | yes | no | no | yes | in scope |

| Qt file | React module | Uses React | Bridge | Manifest | Registers in Electron | Ships in the build | RENDERS | Scope |
|---|---|---|---|---|---|---|---|---|
| src/gui/market_inspector.py | market_inspector.js | yes | yes | yes | yes | yes | yes | in scope |
| src/gui/market_inspector_topologies.py | market_inspector_topologies.js | yes | yes | yes | yes | yes | yes | in scope |
| src/gui/native_chart.py | native_chart.js | yes | yes | yes | yes | yes | yes | in scope |
| src/gui/paper_trader_tab.py | paper_trader_tab.js | yes | yes | yes | yes | no | yes | in scope |
| src/gui/qt_safe_events.py | no | - | yes | no | no | - | no | not a screen |
| src/gui/react_history_panel.py | no | - | yes | no | no | - | no | React side |
| src/gui/react_history_tab.py | no | - | no | no | no | - | no | React side |
| src/gui/react_paper_trader_tab.py | no | - | no | no | no | - | no | React side |
| src/gui/react_proof_of_accumulation_tab.py | no | - | no | no | no | - | no | React side |
| src/gui/react_simulator_tab.py | no | - | no | no | no | - | no | React side |
| src/gui/react_system_status_tab.py | no | - | no | no | no | - | no | React side |
| src/gui/risk_tab.py | risk_tab.js | yes | yes | yes | no | yes | no | shelved |
| src/gui/settings_dialog.py | settings_dialog.js | yes | yes | yes | - | yes | yes | in scope |
| src/gui/shared_testnet.py | shared_testnet.js | no | yes | yes | no | yes | no | shelved |
| src/gui/simulator_tab.py | simulator_tab.js | yes | yes | yes | yes | yes | yes | in scope |
| src/gui/simulator_tab/fleet/fleet_replay_panel.py | fleet_replay_panel.js | yes | yes | yes | no | yes | yes | shelved |
| src/gui/simulator_tab/fleet/sim_visuals.py | sim_visuals.js | yes | yes | yes | no | yes | yes | shelved |
| src/gui/simulator_tab/nuclear_mode_panel.py | nuclear_mode_panel.js | yes | yes | yes | no | yes | yes | shelved |
| src/gui/simulator_tab/sim_stat_strip.py | sim_stat_strip.js | yes | yes | yes | no | yes | yes | shelved |
| src/gui/simulator_tab/simulator_tab.py | simulator_tab.js | yes | yes | yes | yes | yes | yes | shelved |
| src/gui/start_all_progress_dialog.py | start_all_progress.js | yes | yes | yes | - | yes | yes | in scope |

| Qt file | React module | Uses React | Bridge | Manifest | Registers in Electron | Ships in the build | RENDERS | Scope |
|---|---|---|---|---|---|---|---|---|
| src/gui/stock_main_window.py | no | - | yes | no | no | - | no | shelved |
| src/gui/testnet_tab.py | testnet_tab.js | yes | yes | yes | no | yes | no | shelved |
| src/gui/tradingview_chart.py | tradingview_chart.js | yes | yes | yes | no | yes | no | shelved |
| src/gui/usb_auth_widget.py | usb_auth_widget.js | yes | yes | yes | no | yes | no | shelved |
| src/gui/visualizer/bot_node.py | bot_node.js | yes | yes | yes | yes | yes | yes | in scope |
| src/gui/visualizer/quick_routing.py | quick_routing.js | yes | yes | yes | yes | yes | yes | in scope |
| src/gui/visualizer/themes.py | visualizer_themes.js | - | yes | yes | yes | yes | yes | in scope |
| src/gui/visualizer/wire_canvas.py | wire_canvas.js | yes | yes | yes | yes | yes | yes | in scope |
| src/gui/widgets/__init__.py | no | - | yes | no | no | - | no | not a screen |
| src/gui/widgets/api_tester_tab.py | api_tester_tab.js | yes | yes | yes | no | yes | no | shelved |
| src/gui/widgets/bot_selection.py | bot_selection.js | yes | yes | yes | no | yes | no | not a screen |
| src/gui/widgets/bot_status_table.py | bot_status_table.js | yes | yes | yes | - | yes | yes | in scope |
| src/gui/widgets/capital_registry_panel.py | no | - | no | no | no | - | no | shelved |
| src/gui/widgets/dashboard_stat_card.py | dashboard_stat_card.js | yes | yes | yes | - | yes | yes | in scope |
| src/gui/widgets/exchange_tab.py | exchange_tab.js | yes | yes | yes | no | yes | - | in scope |
| src/gui/widgets/extractor_bot_table.py | extractor_bot_table.js | yes | yes | yes | no | yes | yes | in scope |
| src/gui/widgets/notification_spool.py | notification_spool.js | yes | yes | yes | no | yes | no | shelved |
| src/gui/widgets/privacy_dot.py | privacy_dot.js | yes | yes | yes | - | yes | yes | in scope |
| src/gui/widgets/pulse_manager.py | pulse_manager.js | yes | yes | yes | no | yes | no | shelved |
| src/gui/widgets/spendable_profits.py | spendable_profits.js | yes | yes | yes | yes | yes | yes | in scope |

| Qt file | React module | Uses React | Bridge | Manifest | Registers in Electron | Ships in the build | RENDERS | Scope |
|---|---|---|---|---|---|---|---|---|
| `src/gui/widgets/status_log.py` | `status_log.js` | yes | yes | yes | yes | yes | shell | in scope |
| `src/gui/widgets/trade_charts_tab.py` | `trade_charts_tab.js` | yes | yes | yes | yes | yes | yes | in scope |

The Simulator rebuild removed the files above; they are not in the tree.

The Capital Registry row names a file the tree no longer holds. The registry it drew was built nowhere, so the table could draw no row. Four modules went together: `src/gui/widgets/capital_registry_panel.py`, its view model, its renderer page, and the registry behind them. The bridge no longer offers the method, and the renderer manifest no longer names the page.

Totals across the 76 rows above, measured on 5 September 2026:

```
React module             63
Uses React               60
Bridge                   72
Manifest                 62
Registers in Electron    25
Ships in the build       58
RENDERS                  44
RENDERS, in scope        35 of 43
out of scope             41
```

Four columns are all but complete. RENDERS, the column that is the item, is not.

Re-measured on 6 September 2026, counted off the 76 rows above:

```
React module             55
Uses React               53
Bridge                   67
Manifest                 55
Registers in Electron    15
Ships in the build       55
RENDERS                  29
```

Only the last column moved. The Settings row already carried a React module, a bridge method and a manifest entry; what it gained is the screen.

Re-measured later on 6 September 2026, counted off the same 76 rows:

```
React module             55
Uses React               53
Bridge                   67
Manifest                 55
Registers in Electron    15
Ships in the build       55
RENDERS                  30
RENDERS, in scope        30 of 43
out of scope             33
```

The Create Auto Trader wizard is the row that moved. `bot_wizard.js` draws it into a space the React exchange screen keeps once + New Bot is pressed, so the wizard is a screen and no longer a module with nowhere to draw.

Counted once more the same day, after the Live Bot Settings window went over:

```
React module             55
Uses React               53
Bridge                   67
Manifest                 55
Registers in Electron    15
Ships in the build       55
RENDERS                  38
RENDERS, in scope        38 of 43
out of scope             33
```

Eight rows moved together: the window a bot row opens, six of the files its tabs are built from, and the delegate that draws the fold-tranche row borders. `fold_tokens.py` keeps its dash. Its colours are on the drawn rows, and the file names no screen of its own.

When that count was taken the Paper Trader, Proof of Accumulation and System Status tabs had no Qt module to replace and no React module to replace it with, so the table carried no row for them. `empty_tabs.py` drew all three empty, and its own row read `builds the shell`. Issue #19 has since built the Paper tab, which now has a Qt module and a React module of its own.

In development.

That is no longer the state. Each of the three has a renderer module in `src/gui/web`, a bridge method and a manifest entry, and `empty_tabs.py` carries the row for all three. Its Scope cell reads `in scope` and its RENDERS cell reads `yes`. The section below says what each tab draws.

Counted once more on 6 September 2026, after the header strip and the Start All dialog went over:

```
React module             58
Uses React               56
Bridge                   69
Manifest                 58
Registers in Electron    15
Ships in the build       58
RENDERS                  42
RENDERS, in scope        42 of 43
out of scope             33
```

Six rows moved. Three are the header strip parts. One is the Start All progress dialog. One is the Live Bot Settings screen tab, where the React module cell was the wrong one and now names the module that draws it. The last is the buy confirmation dialog, which gained a module, a bridge method and a build entry and still reads `no` under RENDERS.

Counted again on 6 September 2026, after the buy confirmation dialog went over:

```
React module             58
Uses React               56
Bridge                   69
Manifest                 58
Registers in Electron    15
Ships in the build       58
RENDERS                  43
RENDERS, in scope        43 of 43
out of scope             33
```

One row moved. It is the buy confirmation dialog, and it was the last in-scope row reading `no`. The count comes from `tools/conversion_table.py`, which reads the cells of the table above.

Counted again on 6 September 2026, after the window's own tab bar went over:

```
React module             59
Uses React               57
Bridge                   69
Manifest                 59
Registers in Electron    15
Ships in the build       58
RENDERS                  44
RENDERS, in scope        44 of 44
out of scope             32
```

One row moved, and it is `main_window.py`. The row read `builds the shell` because the module filled the tab bar rather than sitting in it; the bar it fills is now React, so the row is in scope and it renders. `Ships in the build` stays `no`: the newest bundle under `dist` carries 70 renderer modules and `main_window.js` is not one of them, because the module is newer than that build.

Counted again on 6 September 2026, after the three unbuilt tabs went over:

```
React module             60
Uses React               58
Bridge                   70
Manifest                 60
Registers in Electron    16
Ships in the build       58
RENDERS                  45
RENDERS, in scope        45 of 45
out of scope             31
```

One row moved, and it is `empty_tabs.py`. It is the second row that read `builds the shell`, and it is the only row naming three React modules: the Paper Trader, System Status and Proof of Accumulation skeletons each have a renderer module, a bridge method and a manifest entry, and all three register a panel with the Electron shell. `Ships in the build` reads `no` for the same reason as the row above: the newest bundle under `dist` carries 70 renderer modules and these three are not among them.

No row now reads `builds the shell`.

Counted again on 6 September 2026, after the Notifications and Alerts tab went over:

```
React module             60
Uses React               58
Bridge                   70
Manifest                 60
Registers in Electron    16
Ships in the build       58
RENDERS                  46
RENDERS, in scope        45 of 45
out of scope             31
```

One row moved, and it is `alerts_tab.py`. Its Scope cell still reads `shelved`, because nothing the running window builds reaches the screen. `variant_surface` now holds the row under the name `ALERTS`, so the React build makes `AlertsReactTab` and the Qt build makes `AlertsTab` unchanged. The React tab draws the whole screen in one web view from `src/gui/web/alerts_tab.js`, and `AlertsTab.refresh`, `AlertsTab._save_config`, `AlertsTab._test_telegram` and `AlertsTab._acknowledge_all` are the same methods on both sides.

### 2026-09-09 10:31 - #128 - the Live Bot Settings window fills its tab pages

The window a bot row opens already registered with the shell's panel host. Its cell read `no`, and the running shell named it among nineteen registered panels, so the cell was stale.

```
registered  bot_live_settings bot_swarm_tab bot_visualizer console_tab
            header_strip history_tab indicator_panel market_inspector
            market_inspector_tab market_inspector_topologies native_chart
            paper_trader_tab proof_of_accumulation_tab simulator_tab
            spendable_profits status_log system_status_tab trade_charts_tab
            trading_tab
```

Registering was never the whole of it. The window drew its title, its tab strip and its two footer buttons, and every one of its seven tab pages was an empty box. Read off the drawn page, the window held 28 elements

and each page held none.

```
part                   children  characters
status-page                   0           0
settings-page                 0           0
fold-tranches-page            0           0
stack-tranches-page           0           0
bot-swarm-page                0           0
market-inspector-page         0           0
phantom-bots-page             0           0
```

Each tab module already carried a filler that looks for the empty space this window leaves. Nothing called it. The window now asks each module's own loader for its view model and hands it the page.

```
    var wait =
      typeof loader === "function"
        ? loader(tabRequest(model))
        : Promise.resolve(null);
    return Promise.resolve(wait).then(function (found) {
      return api.fill(space, found) === null ? null : space;
    });
```

A tab is asked for the bot the window built, never for one of its own. The window's answer now names that bot, and the request carries it under the name the window's own surface publishes.

```
    config = model.bot.config
    return {
        "bot_id": model.bot.bot_id,
        "symbol": config.symbol,
        "mode": config.mode,
        "state": model.bot.state,
    }
```

Three pages fill from it. Fold Tranches draws eleven health rows, three Clear buttons, an order picker of five choices and a filter box. Stack Tranches draws seven summary rows and two Clear buttons. Phantom Bots draws four summary rows, twelve timeframe boxes and the candles-to-lock number. Take the bot out of the window's answer and every one of those rows goes.

```
page                  rows  order choices  elements   with the bot removed
fold-tranches-page      11              5        69    0,  5, 33
stack-tranches-page      7              0        33    0,  0, 12
phantom-bots-page        4              0        92    0,  0, 46
```

The order picker survives the removal because the fold chrome is asked for its own controls rather than for a bot, which is what proves the row counter was reading the page and not returning zero everywhere.

Every page drew at once before this. A page not on show was marked hidden and still carried an inline display, which beats the browser's own rule for a hidden element. The page not on show now takes `display: none`, so one tab is on screen at a time.

```
    var style = {
      display: props.current === true ? FLEX : DISPLAY_NONE,
```

The window's own stylesheet had never reached the shell page, so its tab buttons painted white on grey. The shell now links it. Its rules were written for a page holding this window alone, and the shell page holds every panel: 17 of the 20 tables on that page belong to other panels, as do 5 of the 12 tab buttons. Each rule is scoped to the window before the link, and five of six other panels render byte-identical to the run before it.

```
[data-part="bot-live-settings"] table {
  border-collapse: collapse;
  width: 100%;
}
```

The Settings page stays empty and its row stays `no`. Its surface builds from the bot's own settings values, and it stops on the first one the window cannot supply.

```
AttributeError: 'BotConfigSource' object has no attribute 'visibility'
```

The window answers a stand-in bot, not a running one. Giving it a running one would also point Apply Changes at that bot, which is a change to what a control writes and is not this unit's to make.

The exchange screen and the Extractor table are drawn inside the Live tab, not by the shell. Both rows stay `no`: the Live tab names an exchange only when the shell's backend is the trading program itself, and the backend reachable here names none, so neither child is built.

```
    slot.setAttribute(CHILD_ATTR, EXCHANGE_MODULE);
    return api.renderTab(slot, shown.model);
```

The Qt window could not be built beside the React one. It reads a mode and a state that carry a `value`, then dozens of settings fields with no fallback, so it needs a bot the running program holds.

```
AttributeError: 'str' object has no attribute 'value'
```

### The window's own tab bar

The row of tab names across the top of the window is drawn by React under the React build. `MainWindow._setup_ui` no longer builds a `QTabWidget` by name. It asks the variant seam for the tab book, so one builder gives the operator the Qt bar he runs today or the React one.

```
            self._main_tabs = _main_tab_book_class()()
            self._main_tabs.setMovable(True)
```

Under the Qt build the seam answers with the widget the operator runs today and nothing changes. Under the React build it answers with a book that keeps that widget as the stack of pages, hides its own bar, and puts a web view above it. Every call the window makes is answered over the widget behind the bar, so the tab order and the pages are the same objects on both sides.

```
        def addTab(self, widget, label: str) -> int:
            """Add ``widget`` under ``label`` and redraw the bar."""
            index = self._book.addTab(widget, label)
            self._push_tabs()
            return index
```

The bar itself is the Electron shell's navigation, run inside the window. The page is built from the shell's own scripts, read off disk, and the buttons are drawn by one React module both sides reach. Neither host holds a bar of its own, so the two cannot draw a different one.

```
#: The shell's own navigation, read from ``desktop/renderer`` and inlined.
SHELL_SCRIPTS: tuple[str, ...] = ("panel_host.js", "tab_bar.js")
```

The shell's navigation asks that React module for the buttons and passes it the tab names, the selected one, and what to do with a press and a drop. It holds no button of its own.

```
    drawn.renderTabBar(bar, {
      names: found,
      selected: current,
      onSelect: select,
      onMove: onMove
    });
```

A press on a button is reported back to Python, which moves the real tab book to that tab. Dragging a tab onto another reports the two slots the same way, and the same handler hands them to the hidden bar, so a drag reorders the pages as it did before.

```
            move = request.get("move") or {}
            if "from" in move and "to" in move:
                self._book.tabBar().moveTab(int(move["from"]), int(move["to"]))
```

The menu bar, the window title, the window icon and the status bar are still Qt. `MainWindow._setup_menu` builds the four menus and `_setup_status_bar` builds the status line; neither was changed, and every menu action the window offered before it is on it now.

### The three tabs with nothing behind them yet

Paper Trader, System Status and Proof of Accumulation were named on the tab bar and were not built when that count was taken. Each one drew a heading, a sentence saying the tab is not built, and the issue that carries the build-out. Every word of it came from that tab's own Python surface, so the page invented nothing. Two of the three are still empty; issue #19 built the Paper tab.

```
    return {
        "accessible_name": HEADING,
        "built": BUILT,
        "heading": HEADING,
        "issue": ISSUE,
        "issue_text": ISSUE_TEXT,
        "method": METHOD,
        "state_text": STATE_TEXT,
    }
```

Under the Qt build the three sentences are Qt labels, as before. Under the React build the same view model goes to the tab's renderer module, which draws it in a web view. The builder asks the variant seam which of the two to make, and it hands both the same values.

```
        model = surface.view_model({})
        panel = _empty_tab_class()(model)
```

Neither side holds a colour or a size of its own. `SKIN` names the ground, the two text colours, the two text sizes, the padding and the gap; the Qt style sheets are built from it and the page reads it as custom properties.

```
SKIN = {
    "--empty-tab-ground": ds.SURFACE_0,
    "--empty-tab-heading-colour": ds.TEXT_MAX,
    "--empty-tab-body-colour": ds.TEXT_EMPTY_STATE,
```

The page runs the Electron shell's `panel_host.js`, so the module is drawn the same way the shell draws it: by name, into one host element, with the reason written onto the page when it does not draw.

### The header strip

The strip and the five counters above the tab row are no longer built by name. `HeaderStripMixin._build_header_strip` asks the variant seam for each class, so one builder draws the Qt widgets or the React pages.

```
        self._spendable_widget = _spendable_profits_class()()
        top_row.addWidget(self._spendable_widget, stretch=3)

        card_class = _stat_card_class()
        self._stat_scrummed = card_class("Scrummed", "$0.00")
```

The spendable strip is drawn by `SpendableProfitsReact`, which hosts the strip module in one web view. It answers the two calls the main window makes, and writes both into the same view model the Qt strip is described by.

```
        def update_profits(self, data: dict) -> None:
            self._model.update_profits(data)
            self._push()
```

Each column carries a privacy dot drawn by the dot module. A press on one runs the model in `privacy_dot_surface`, which flips the field in the process privacy registry and repaints the amount as four stars.

```
    def clicked(self) -> None:
        self.calls.append(CLICKED)
        try:
            registry = get_privacy_mask_registry()
            registry.set_masked(self._field_id, not registry.is_masked(self._field_id))
        except Exception:
            self.calls.append(TOGGLE_FAILED)
        self.refresh()
```

One card is drawn by `StatCardReact`. A press on the card emits the card's one signal, and only while the header strip has armed it.

The glyph, the tooltip and the skin of a dot now have one home. Both header surfaces return what the dot surface paints, so the card and the strip cannot drift apart.

```
def privacy_dot(field_id: str, masked: Optional[bool] = None) -> dict:
    painted = privacy_dot_surface.dot_view(field_id, masked)
    return {name: painted[name] for name in _DOT_FIELDS}
```

### The Start All progress dialog

The dialog the Start All button opens is drawn by `StartAllProgressReactDialog`. It replaces only the method that builds the controls. The progress handling, the Cancel press and the unsubscribe on close are the same code the Qt dialog runs.

```
        def store(self) -> surface.StartAllProgressModel:
            return self._model
```

Every control it needs is a small holder over that store. The list answers `addItem` and `count`; the buttons answer `setEnabled`. Both build sites, in the main window and in the launcher, now ask the variant seam for the class.

### The buy confirmation dialog

This is the one in-scope row that still reads `no`, and the reason is reachability rather than missing work.

The main window builds the broker in this file. Nothing calls the broker's request method, which is the only sender of the signal that opens the dialog, so the running window has no path to the screen.

```
        async def request_confirmation(
            self,
            *,
            bot_id: str,
            symbol: str,
            reason: str,
```

Measured by walking every call in `src` and `main.py`: zero call sites, and no import of that name under another. The module, the bridge method and the build entry are all present, so the screen is one caller away.

Re-measured on 6 September 2026, by the same walk over 354 modules: still zero call sites. That count decides what the operator can reach, and it did not move. The row's RENDERS cell moved for a different measurement — the class the broker raises.

The dialog the broker raises is now `BuyConfirmationReactDialog`. The broker asks the variant seam for that class, so the React build opens the page and `ACERVATOR_VARIANT=qt` opens the same Qt widgets as before.

```
def dialog_class() -> type:
    """The confirmation dialog class the running build variant draws."""
    from .variant_surface import BUY_CONFIRMATION, surface_class

    return surface_class(BUY_CONFIRMATION)
```

Every figure the page shows is written in Python. `details_text` in `buy_confirmation_surface` writes the seven detail rows. The Qt label is set from it, and the browser draws the same seven lines sliced into their values. No arithmetic runs in the page.

```
def after_buy_usd(holdings_before: float, price: float, cost_usd: float) -> float:
    """Dollar value the position reaches once this buy fills."""
    return holdings_usd(holdings_before, price) + cost_usd
```

A press on the page names a button. Python turns that name into an answer and accepts the dialog through the same `_answer` the Qt buttons call. The page places no order and decides nothing.

```
        def run_action(self, name: str) -> bool:
            answer = surface.BUTTON_ANSWERS.get(name)
            if answer is None:
                return False
            self._answer(answer)
            return True
```

A window closed without a press is still no. A request nobody answers inside sixty seconds is refused too.

### The gap

Issue #128 says this item replaces the Qt screens with React. Measured against that sentence:

```
screens the running window builds     52
screens that draw React                1
screens that draw in the shell only    2
tabs drawn by React                     0
```

The renderer manifest names 70 modules. The application the operator runs hosts no manifest, and loads exactly one of those modules: the History table's, which one Qt widget puts on a page of its own. The shell that does host the manifest draws two panels and offers no way to reach a third.

One call site carries the whole difference between the React build and the Qt build. `src/gui/history_table_variant.py` holds the only call to the variant resolver in the tree, and every other tab builds the same Qt widget either way.

`src/gui/history_table_variant.py` — `history_table_class`

```
chosen = resolve_variant() if variant is None else variant
if chosen == QT:
    from .history_qt_table import HistoryQtTable

    return HistoryQtTable
from .react_history_panel import HistoryWebTable

return HistoryWebTable
```

### Why the count read as near-complete

Two instruments report on this work, and both answer a narrower question than the item asks.

The progress tool marks a screen paired when a bridge method reaches a renderer module the manifest names. A manifest entry declares that the shell would load the file. It does not put a screen in front of the operator.

`tools/conversion_state.py` — `served_methods`, the pairing rule

```
def served_methods(root: pathlib.Path) -> dict[str, str]:
    """Bridge method to the renderer module that speaks it, loaded ones only.

    A method absent from `registered_surfaces` is left out of the result.
    """
    loaded = renderer_modules(root)
```

The parity tests compare a Qt view model against the React view model of the same screen. Both sides are plain Python data and neither has to be displayed. A passing test proves the two descriptions agree with each other. It does not prove either one is on screen.

`parity_pins` reads only a parity test that imports exactly one surface module and exactly one Qt module. It collects the names on both sides and pairs them.

`tools/conversion_state.py` — `parity_pins`, what a pin reads

```
def parity_pins(root: pathlib.Path) -> dict[str, set[str]]:
    qt_by_name = {dotted_name(root, path) for path in qt_modules(root)}
    surfaces = surface_methods(root)
    pins: dict[str, set[str]] = {}
```

Both readings were true. Neither measured the item, and the RENDERS column is the one that does.

### Not reached from a live tab

These Qt modules are not built by the live window or by any of its tab builders. They are outside this conversion. One row is a package marker rather than a screen, and it stays where it is.

| Qt module | Why it is out of scope |
|---|---|
| `alerts_tab.py` | Not reachable from a live tab |
| `analytics_tab.py` | Not reachable from a live tab |
| `audio_suite.py` | Not reachable from a live tab |
| `competition_tab.py` | Not reachable from a live tab |
| `init_wizard.py` | Not reachable from a live tab |
| `journal_tab.py` | Not reachable from a live tab |
| `launcher.py` | Not reachable from a live tab |
| `live_bot_window.py` | Not reachable from a live tab |
| `risk_tab.py` | Not reachable from a live tab |
| `stock_main_window.py` | Not reachable from a live tab |
| `testnet_tab.py` | Not reachable from a live tab |
| `usb_auth_widget.py` | Not reachable from a live tab |
| `widgets/__init__.py` | Package marker, no screen |
| `widgets/api_tester_tab.py` | Not reachable from a live tab |
| `widgets/capital_registry_panel.py` | Not reachable from a live tab |
| `widgets/notification_spool.py` | Not reachable from a live tab |
| `widgets/pulse_manager.py` | Not reachable from a live tab |

One row in that table names a file the tree no longer holds. The Capital Registry panel is removed with the registry that fed it, so no conversion is owed for it.

The rows in that table are inside this item. The operator's 6 September 2026 directive puts the whole interface in scope, so a screen the window does not build is converted like any other. A converted row keeps `shelved` in the Scope cell of the table above, because that cell answers reachability. Its RENDERS cell reads `yes` once `variant_surface` holds the row and the React class draws the screen.

The reachability walk is rooted at the window and its tab builders, and an edge is a use rather than an import: a call, a base class, or a returned class. An import alone is not an edge, so the re-export block in the main window leaves a widget unreached. Ten screens known to be built and eleven known to be dead were run through the walk as a control, and all twenty-one came back on the expected side.

Detail on each live screen sits one file down, in 08-tabs/README.md.

### 2026-09-09 11:34 - #128 - the four modal dialogs are opened, not mounted

Four rows read no under Registers in Electron: the bot wizard, the buy confirmation, the Settings dialog and the Start All progress dialog. None of the four is a panel the shell should hold, so all four now take the dash the table already uses for a module that does not register in its own right.

The running shell was asked about each name before anything was changed. All four modules load and none registers a panel.

```
manifest names        69
panels registered     19
bot_wizard            bot_wizard.js registered no panel to draw
buy_confirmation      buy_confirmation.js registered no panel to draw
settings_dialog       settings_dialog.js registered no panel to draw
start_all_progress    start_all_progress.js registered no panel to draw
```

Each was then registered by hand and opened into a host of its own, which is the test that decides the cell. All four draw, so none is an empty shell; what decides them is that nothing in the shell opens them and each one's look belongs to the page its own host builds.

```
name                  elements  characters  where it belongs
bot_wizard                  27         906  a space exchange_tab.js keeps
buy_confirmation            29         254  a window the broker raises
settings_dialog            431        2465  a window the menu raises
start_all_progress          10         390  a window Start All raises
```

The wizard is the clearest of the four. The exchange screen keeps a space for it and fills that space once + New Bot is pressed, so its content reaches the shell through the tab above it.

```
    space.setAttribute(CHILD_ATTR, moduleName);
    return api.fill(space, null, { onClosed: closeBotWizard });
```

The Settings dialog drew all eleven tab pages at once. A page not on show was marked hidden and still carried an inline display, which beats the browser's own rule for a hidden element. Read off the drawn page, 320 parts sat outside the window; with the page not on show taking `none`, none do.

```
      display: props.current === true ? FLEX : DISPLAY_NONE,
```

Every drop-down on that dialog drew one letter per choice. The dialog turns each choice into plain data before it reaches the page, and listing a word gives its letters, so ten carriers drew as ten letters. A choice that is one word now stays that word.

```
before   sms_carrier   A  T  V  S  U  C  B  M  G  O
after    sms_carrier   AT&T  T-Mobile  Verizon  Sprint  US Cellular  Cricket
```

Thirty-eight tick boxes and eight number boxes drew with nothing beside them. The words were already in the payload under `text`, and the units under `suffix`, and neither was drawn. Both are drawn now, so the Sound page reads the same on both builds.

```
part            before   after
row-label           31      31
check-text           0      38
value-prefix         0       2
value-suffix         0       6
```

The Start All progress page did not fit the window it is drawn in. It is sized from the dialog's own width and height and then given padding, and the page carried a margin of its own, so the box measured 548 by 388

inside a window of 520 by 360 and both buttons fell below the edge. A stylesheet of two rules puts the padding and the margin inside that size.

```
before   window 520x360   page 548x388   parts outside the window 10
after    window 520x360   page 520x360   parts outside the window  0
```

The wizard's own walk row is not finished. It draws the four step names as words, where the Qt wizard draws Back, Next, Commit, Finish and Cancel and turns Back and Finish off on the first page. The payload names the four steps and carries no words and no on-or-off state for them.

```
WALK_STEPS = (WALK_NEXT, WALK_BACK, WALK_CANCEL, WALK_FINISH)
```

The RENDERS column asks what the operator sees when he opens the screen, so each of the four was asked of the running build rather than of the table. The variant seam holds sixteen screens and answers a class per build.

```
screen                      qt build                  react build
Settings dialog             SettingsDialog            SettingsDialogReact
Start All progress dialog   StartAllProgressDialog    StartAllProgressReactDialog
Buy confirmation dialog     BuyConfirmationDialog     BuyConfirmationReactDialog
bot wizard                  no entry: both builds open BotCreationWizard
```

Three of the four keep `yes`, and each was drawn under both builds and pictured. The wizard takes the dash. It has no entry on the seam, so + New Bot opens the Qt wizard whichever build is running, and its React module draws only inside the exchange screen the shell builds when its backend names an exchange.

The buy confirmation keeps `yes` for the class the broker raises, which is the React one. Nothing calls the broker's request method, so the operator reaches neither build of that screen; the section above records that count and it has not moved.

The wizard's walk row is a gap in what the payload carries, and the Python surface owns it. Qt's wizard supplies its own five buttons and works out which are live from the page it is on. The React module is handed four step names and nothing else, so it has no words to draw and no state to grey.

```
WALK_STEPS = (WALK_NEXT, WALK_BACK, WALK_CANCEL, WALK_FINISH)
```

Two things are missing from `bot_wizard_surface`: the words each step draws, and whether each step is live on the page now showing. Back is dead on the first page and Finish is dead until the last, and both follow from the step order the surface already holds.

```
PROPOSED, in bot_wizard_surface.py

WALK_WORDS = {"back": "< Back", "next": "Next >", "commit": "Commit",
              "finish": "Finish", "cancel": "Cancel"}

walk_live(page) -> {"back": page is not the first, "next": a page follows,
                    "finish": no page follows, "cancel": True}
```

### 2026-09-09 12:45 - #128 - the alerts tab is opened, the news strip is drawn inside the exchange screen

Two rows read `no` under Registers in Electron: the Notifications and Alerts tab and the crypto news strip. Neither is a panel the shell should hold, so both now take the dash the table already uses for a module that does not register in its own right.

The running shell was asked about each name before anything was changed. Both modules load and neither registers a panel.

```
manifest names        69
panels registered     19
alerts_tab            alerts_tab.js registered no panel to draw
crypto_news_ticker    crypto_news_ticker.js registered no panel to draw
module load errors    []      faults []
```

Each was then registered by hand and opened into a host of its own, which is the test that decides the cell. Both draw, so neither is an empty shell; what decides them is that the shell opens neither, and that a panel host has no way to give either one the values it needs.

```
name                  opened  elements  characters  what it drew
alerts_tab              true        54         233  both tables empty
crypto_news_ticker      true         2          21  Fetching crypto news
strips on the page                                  2, for one exchange space
```

The alerts tab is raised by a window rather than by a tab. The variant seam holds it under the name ALERTS, so the Qt build opens `AlertsTab` and the React build opens `AlertsReactTab`, which is one browser view filling the whole tab. Opened as a panel instead, it draws every control and no row, because the host asks with an empty request and no notification manager reaches it.

The news strip belongs to the exchange screen. That screen keeps a space for it and fills that space, once per exchange, the way it fills the two bot tables. A registration holds one host per name, so registering the strip put a second copy of it on the page.

```
    space.setAttribute(CHILD_ATTR, moduleName);
    return api.mount(target);
```

The two RENDERS cells part company. The variant seam answers a class per build for the alerts tab, so that cell keeps `yes`. It holds no entry for the news strip, so the exchange header builds the Qt strip whichever build is running, and the React module draws in the Electron shell and nowhere else. That cell now reads `shell`, the value this table already defines for exactly that.

```
screen                      qt build            react build
Notifications and Alerts    AlertsTab           AlertsReactTab
crypto news strip           no entry: both builds build CryptoNewsTicker
```

The strip was never fed. The exchange screen asked the backend for the strip's state with an empty request, so the strip was built and never started, and the line read "Fetching crypto news" for as long as the shell ran. Qt starts the strip as it builds it and the screen already reports that it did, so the request now carries the start it

reports.

```
function tickerRequest(model) {
  var started = model[NEWS_TICKER_STARTED];
  return typeof started === "number" && started > ZERO ? { start: true } : {};
}
```

The backend had no fetch to run either. The strip's model takes its fetch as an argument and the bridge never gave it one, so the worker answered with no stories. The bridge now hands in the same transport the Qt strip uses, and runs the fetch away from the request that asked for it, so the answer arrives the way it arrives in Qt.

```
  crypto_news_ticker_surface.use_fetch(SafeRequest, safe_urlopen, time.monotonic)
```

With that in place the shell fetched 43 stories from the ten feeds and drew them, one at a time, in the exchange header. Refusing every feed drops them again, which is what proves the line carries what the fetch found.

```
live      [2/43] The Defiant - SEC Crypto Custody Rewrite Enters White House Review
broken    (no crypto news feeds reachable)
no start  Fetching crypto news
```

Both builds of each screen were then driven on one reading and compared item by item. The alerts tab was read on one notification manager filled the way the main window fills it: the risk rules over a three-bot fleet, the profit and loss milestone, one bot error and one bot start. The news strip was read on one answer from the shipped fetch, given to each side through its own receiver.

```
alerts tab       13 routing rows, 4 history rows      differences 0
news strip       6 stories, 8 steps, wraps 6 to 1     differences 0
```

Every alert row matches: its time, its priority word, its title, its message, its channels, and the colour of each. The four priorities draw in four colours, low grey through critical red, and the rows run newest first on both sides. Emptied of notifications, both sides draw no rows and both read `Unread: 0`.

The strip shows the same story at each place, moves one place forward on each step, and wraps from the last story to the first on both sides. Its words are `#cfe6ff` on both and its pointer is the hand on both.

Two colours were wrong on the page and are now right. The tab's group boxes carry a Qt style sheet whose colour paints the group's title alone; the page was painting the whole box with it, so the routing table's Event and Priority columns and the three field labels drew in the accent where Qt draws them in the body colour. The page also carried no stylesheet of its own, so with the group colour moved to the title it fell back to the browser's black.

```
                        before        after         qt
routing Event column    #00ffcc       #e0e0f0       #e0e0f0
field labels            #00ffcc       #e0e0f0       #e0e0f0
```

The two strips take the fetch at different moments. The Qt strip shows a story as soon as the fetch answers, because the worker reports to it. The page has no such report, so it shows the answer on its next 15 second step. Closing that needs a route from the backend to a module the shell holds by name, and the strip is drawn

by its parent rather than held by name.

The Qt tables are drawn by the theme, not by the tab, so the page draws no grid lines, no alternating row bands and no header underline. Those live in the Qt style sheet for every table in the application and reach no payload.

### 2026-09-09 13:08 - #128 - the RENDERS column reads what the running build draws

The running program answers this column, not the table. The table marks 42 rows in scope. The reading covered 38 of them. 26 agree with the cell they carried. 12 do not, and every one of the 12 claims more than the operator gets. No row claims less. Four rows carry no reading, and each keeps the cell it carried.

```
rows marked in scope       42
measured                   38
  agree                    26
  disagree                 12
could not measure           4    bot_wizard, header_strip,
                                 market_inspector_topologies, visualizer/themes
```

Ten cells change here. Six drop to a dash and four drop to `shell`.

| Qt file | before | after | what the running build draws |
|---|---|---|---|
| `src/gui/main_tabs/trading_tab.py` | yes | - | `_build_trading_tab` gives the Live tab a page with no web view |
| `src/gui/widgets/exchange_tab.py` | yes | - | `ExchangeTab` is that page |
| `src/gui/widgets/bot_status_table.py` | yes | - | `BotStatusTable` sits inside that page |
| `src/gui/widgets/extractor_bot_table.py` | yes | - | `ExtractorBotTable` sits inside that page |
| `src/gui/live_settings/positions_held_tab.py` | yes | - | `BotLiveSettingsReactDialog` carries seven tab labels and none of them is Positions Held |
| `src/gui/main_tabs/console_log_handler.py` | yes | - | the window installs `_QtLogHandler` on the root logger |
| `src/gui/widgets/trade_charts_tab.py` | yes | shell | `TradeChartsTab` is the Charts tab, and `trade_charts_tab.js` draws in the Electron shell alone |
| `src/gui/native_chart.py` | yes | shell | `ChartPanel` and `CandlestickChart` draw inside that tab, and `native_chart.js` draws in the shell alone |
| `src/gui/indicator_panel.py` | yes | shell | `IndicatorVotingPanel` is the voting panel, and `indicator_panel.js` draws in the shell alone |
| `src/gui/widgets/status_log.py` | yes | shell | `StatusLog` is the log pane, and `status_log.js` draws in the shell alone |

Two of the twelve keep the cell they carry. The news strip cell already reads `shell`. The Main Window cell keeps `yes`, because none of the three values fits what that window draws.

The Main Window is a React tab book inside a Qt frame, with a Qt menu bar and a Qt status bar, and `MainWindow` in `src/gui/main_window.py` builds all three.

```
MainWindow._setup_ui           builds the tab book from the variant seam
MainWindow._setup_menu         builds the Qt menu bar
MainWindow._setup_status_bar   builds the Qt status bar
```

The 42 in-scope cells now read:

```
yes      30
shell     5
-         7
```

### 2026-09-09 14:30 - #34 - the Status tab leaves the empty-tab pair

Status is no longer a skeleton, so it leaves `empty_tabs.py` and takes a row of its own. Accumulation stays where it was and still draws its empty state.

`src/gui/main_tabs/system_status_tab.py` — what the window builds for the tab

```
    def _build_system_status_tab(self) -> None:
        """Append the Status tab, and hold None when its page cannot be built."""
        try:
            from ..react_system_status_tab import SystemStatusReactPanel

            self._system_status_tab = SystemStatusReactPanel()
            self._main_tabs.addTab(self._system_status_tab, HEADING)
        except Exception as exc:  # noqa: BLE001 - a missing tab is not a crash
            logger.warning("Status tab unavailable: %s", exc)
            self._system_status_tab = None
```

Three cells move. The `empty_tabs.py` row drops `system_status_tab.js` and now names one module. A new row carries the screen itself, and a second names the Qt-side file the conversion added.

| Qt file | React module | RENDERS | Scope |
|---|---|---|---|
| `src/gui/main_tabs/empty_tabs.py` | `proof_of_accumulation_tab.js` | yes | in scope |
| `src/gui/main_tabs/system_status_tab.py` | `system_status_tab.js` | shell | in scope |
| `src/gui/react_system_status_tab.py` | no | - | React side |

The new cell reads `shell`, and the reason is the seam. `variant_surface.py` records a Qt loader and a React loader for every screen that has both, and this screen has no entry there, because no Qt widget ever drew it. `yes` states that React draws where Qt used to, and nothing here used to. The module was read drawing under its own tab in the Electron shell, so `shell` is what was measured.

```
registered           system_status_tab is in acervatorPanelHost.registered()
tab bar              system_status_tab, ninth of ten
select               returns system_status_tab
drawn                17 subsystem panels, 11 tab groups, 78 emitter rows
```

The detail is in 08-tabs/system-status.md.

### 2026-09-09 15:12 - #435 - Start All waits 3.5 seconds and retries a bot once

The Start All dialog used to promise a pause of about 2.5 seconds with a two-second minimum. The engine waited 0.6. The pause is now 3.5 seconds, it is written down once, and the dialog quotes that figure rather than a number of its own.

```
#: Seconds `BotManager.start_all` waits between bots, and the figure the Start
#: All dialog quotes. 0.6 drew HTTP 429 on three of 38 starts.
START_ALL_GAP_SECONDS: float = 3.5
```

The boot of 8 September 2026 is in the operator's own log. Thirty-eight bots were brought up one at a time and three of them came back with the same refusal from Coinbase: too many requests, on the call that lists the accounts. Those three did not trade until they were started again eight minutes later. A wider pause is exactly what the dialog always said the gap was for.

```
2026-09-08 11:10:43,649 BOT_CONNECT_FAILED | RateLimitExceeded: coinbase GET
https://api.coinbase.com/api/v3/brokerage/accounts?limit=250
429 Too Many Requests | DIAGNOSIS: Rate limited.
```

A bot that does not report itself running inside ten seconds is now stopped and started one more time, and then the sequence moves on whatever happened. The stop is what makes the second attempt real, because a bot still in its starting state refuses a plain second start and the retry would do nothing.

```
            if not verified and not getattr(self, "_start_all_cancel", False):
                logger.warning(
                    "Bot %s did not reach RUNNING in %.1fs; retrying once",
                    bot.bot_id,
                    verify_timeout_seconds,
                )
                await asyncio.sleep(min_gap_seconds)
                await bot.stop()
                await bot.start()
                verified = await self._await_running(bot, verify_timeout_seconds)
```

The platform now writes a line for each of those events, so a boot that loses a bot says so in the log instead of leaving a silent hole. A bot that comes up on the retry gets one warning; a bot that fails both attempts gets two.

```
Bot LATEBOT starting on LATEBOT/USD
Bot LATEBOT did not reach RUNNING in 10.0s; retrying once
Bot LATEBOT starting on LATEBOT/USD
Bot DEADBOT did not reach RUNNING in 10.0s; retrying once
Bot DEADBOT is not RUNNING after 10.0s; start_all moved on
```

The cost is real and it is worth knowing before the next launch. Thirty-eight bots mean thirty-seven waits. The 8 September pass took 3 minutes 33 seconds for its 35 bots at a mean of 5.3 seconds each, so the same fleet at the new pause reaches about 5 minutes, and three retries add about 40 seconds on top.

```
gaps alone, 37 waits       0.6 s -> 22.2 s        3.5 s -> 129.5 s
measured pass, per bot     mean 5.318 s           mean 8.218 s
38 bots, three retried     3 min 17 s             about 5 min 45 s
```

### 2026-09-09 21:37 - #147 - the Accumulation tab leaves the empty-tab pair

Accumulation is no longer a skeleton, so it leaves `empty_tabs.py` and takes a row of its own. It draws three zones: the player window on the left, the square enemy screen upper right, and the party window across the lower half. Every zone draws a placeholder and no art, class or mode is built.

`src/gui/main_tabs/proof_of_accumulation_tab.py` — what the window builds

```
    def _build_proof_of_accumulation_tab(self) -> None:
        """Append the Accumulation tab, and hold None when its page cannot be built."""
        try:
            from ..react_proof_of_accumulation_tab import ProofOfAccumulationReactPanel

            self._proof_of_accumulation_tab = ProofOfAccumulationReactPanel()
            self._main_tabs.addTab(self._proof_of_accumulation_tab, HEADING)
        except Exception as exc:  # noqa: BLE001 - a missing tab is not a crash
            logger.warning("Accumulation tab unavailable: %s", exc)
            self._proof_of_accumulation_tab = None
```

Three cells move and two rows are new. The empty-tab row names no module now, so nothing the window builds reaches it.

| Qt file | React module | RENDERS | Scope |
|---|---|---|---|
| `src/gui/main_tabs/empty_tabs.py` | no | - | shelved |
| `src/gui/main_tabs/proof_of_accumulation_tab.py` | `proof_of_accumulation_tab.js` | shell | in scope |
| `src/gui/react_proof_of_accumulation_tab.py` | no | - | React side |

The new cell reads `shell`, and the reason is the seam. Sixteen screens are registered there and none of them is this one, because no Qt widget ever drew a proof-of-accumulation screen. The two shelved Qt classes are set aside by the operator's directive of 21 April 2026 and this tab is built on neither.

```
seam screens             16
a PoA, competition or testnet entry among them   none
registered               proof_of_accumulation_tab, with no fault
tab bar                  Accumulation, seventh of ten
zones drawn              player window 2, enemy screen 2, party window 56
party window             8 groups of 5, 40 slots on page 1 of 3
wallet                   0 panels closed, 1 panel and 3 sections open
```

Removing the registration is the control. The panel name leaves the register, the Accumulation tab leaves the bar, and every zone count falls to nought while the same counter still reads 154 elements elsewhere on the page.

```
registered                proof_of_accumulation_tab absent, 18 names
reasonFor                 proof_of_accumulation_tab.js registered no panel to draw
tab bar                   nine tabs, Accumulation gone
zones drawn               0, 0, 0
whole page                154 elements, so the noughts are the zones
```

The chain the shell draws is a field of the payload, so a demo run on the TestNet asks the same bridge method with a different chain and reaches the same page. Nothing else in the request changes.

```
def chain_of(params: dict) -> str:
    """The chain ``params`` names, or ``LIVE_CHAIN`` when it names none in ``CHAINS``."""
    asked = params.get(CHAIN_FIELD) if isinstance(params, dict) else None
    return asked if asked in CHAINS else LIVE_CHAIN
```

One tooling defect surfaced and is fixed here. The page carries three tables under the same header, and the Scope writer took the first, which is a two-row excerpt inside an earlier entry. It wrote a totals block into that excerpt. The span now picks the widest table.

```
    spans = []
    for start, line in enumerate(lines):
        if line.startswith(TABLE_HEAD):
            end = start + 2
            while end < len(lines) and lines[end].startswith("|"):
                end += 1
            spans.append((start, end - 1))
    if not spans:
        raise LookupError("no line starts with " + TABLE_HEAD)
    return max(spans, key=lambda span: span[1] - span[0])
```

Pointed at the real table the writer disagrees with the committed Scope column on twenty rows and moves five totals. That disagreement is not this entry's to resolve, so the column was not rewritten and the three cells above were set by the rule the column states.

### 2026-09-12 - #665 - the Phantom Bots page takes one timeframe and keeps it

The page holds three settings. All three now reach the bot they are drawn for, and two of them survive a restart.

**One phantom, not a set.** The eleven timeframe boxes are one picker. It offers all eleven names, greys the ones the bot's venue does not serve, and opens on the timeframe the bot holds. A bot holding several, because it was restored before this, opens on the highest of them.

| setting | label on the page | what it reaches |
|---|---|---|
| enable | Enable Phantom Bots | the tick gate that builds the phantom set |
| timeframe | Timeframe: | the timeframe list the phantom set is built from |
| lock | Candles to lock: | the coordinator's candle count |

**The constraint is the one the Comp field already stated.** A timeframe at or below the bot's own TA Timeframe is refused, and the reason is written beside the picker. A timeframe the venue does not serve carries a second message. Each pick below was made on a page freshly opened on 1d, on a Coinbase bot whose TA Timeframe is 1h:

```
picked  kept   told to the window       reason written beside the picker
1d      1d     phantom_timeframes 1d    -
6h      6h     phantom_timeframes 6h    -
1h      1d     nothing                  1h is not above this bot's TA Timeframe 1h
15m     1d     nothing                  15m is not above this bot's TA Timeframe 1h
4h      1d     nothing                  4h is not offered by coinbase
```

The refusal is not new work for the engine. A phantom at or below the parent's own timeframe is already dropped twice over: the higher-timeframe bias keeps only a higher rank, and the Comp calculation skips a rank at or below the parent's. On the bot above, four of the five phantoms a restart used to build reached neither.

**Two of the three now survive a restart.** The saved record carries the one timeframe and the candle count beside the enable flag it already carried. Driven through the fleet-load path on a record written for the run:

```
record carries                  restored bot holds                      lock
phantom_timeframe 1d            1d                                         2
lock_candle_count 7             5m 15m 30m 1h 1d                           7
neither key                     5m 15m 30m 1h 1d                           2
```

**A bot nobody has chosen for is left exactly as it was.** The record takes a timeframe only when the bot holds one. A bot still on the engine's own six, which a Coinbase filter cuts to five, writes no timeframe key and restores to the same five. The narrowing to one happens when the operator picks one, and not before.

```
bot                 saved keys                                  restored holds  lock
untouched           phantoms_enabled lock_candle_count                5 names       2
one timeframe set   phantoms_enabled lock_candle_count                1 name        7
                    phantom_timeframe
```

**The window now publishes what the page reads.** The answer a tab is asked for carried four values, and the page read five others off it that were never there, so every phantom reading on the React build drew at its own default. The answer carries them now.

```
published before    bot_id symbol mode state
published now       bot_id symbol mode state exchange_id ta_timeframe
                    enabled timeframes lock_candle_count
```

The candle count the page sets still sizes nothing. `TimeframeCoordinator` defines the one function that would read it and nothing calls that function, so the figure persists and waits.

### 2026-09-13 - #23 - the venue page draws in the second build

The venue page now has a host. `src/gui/react_exchange_tab.py` builds one web view for each venue and draws `exchange_tab.js` inside it.

The host answers the four calls the main window already makes on the Qt venue tab. Each call drives a surface model and pushes the answer into the page.

```
call the window makes             what the host runs
exchange_id                       the attribute the constructor sets
update_bots(statuses)             the screen model, then both bot tables
_refresh_privacy_mode_btn_style   the screen model, then the header dots
stop_feeds                        the page script that takes the bridge away
```

The page carries nine modules and one style sheet. It fetches nothing, because every script and every rule is written into the document.

```
design_tokens.js         theme_engine.js          shared_widgets.js
header_strip.js          table_cells.js           bot_status_table.js
extractor_bot_table.js   crypto_news_ticker.js    exchange_tab.js
one style sheet          exchange_tab.css         26 rules
```

The screen draws a header row, a data-pool line, two bot sections and a command bar of five buttons. `bot_status_table.js` draws the Scrumming rows and `extractor_bot_table.js` draws the Extractor rows, each into a space the screen keeps for it.

The page names what it loaded. A venue holding seven bots reads like this.

```
page modules       all nine, exchange_tab.js last
page style sheets  exchange_tab.css, 26 rules
scrum-table rows   5
extractor rows     2
```

The style sheet paints what the payload leaves unpainted. The Privacy Mode button, the data-pool line and the two section labels carry their own colours from the backend, so no rule touches them. The five command buttons and the New Bot button carry none, so the sheet gives each one a ground, an edge and a text colour.

An emptied style sheet reads as zero rules, and a withheld module drops out of the module list and draws no row. Both readings fall when the thing they measure is taken away.

```
run              module list       style sheet rules   scrum rows
whole page       nine modules                     26            5
style emptied    nine modules                      0            5
table withheld   eight modules                    26            0
```

The page reports every control press back to Python. `ExchangePage` reads the console line the page writes and hands it to `run_action`, which drives the surface that owns the method and then pushes the new payload.

```
pressed           reached Python             the screen recorded
Start on row 1    bot-a14-02, start          command.sent start scrumming
nothing pressed   no call                    no entry
```

The host starts no bot and places no order while it builds or draws. It reaches three names under the trading package, and none of them is a bot, a container or an exchange client.

```
src.trading                    the package
src.trading.gate_vocabulary    the gate names
src.trading.target_bands       the Target A15 and Target A14 arithmetic
```

The news strip is fed. `bind_news_transport` gives the strip the same reader the desktop bridge gives it, and the first ask carries the start the Qt strip performs as it builds.

### 2026-09-13 - #23 - the bot wizard gets a host and a stylesheet

`react_bot_wizard.py` builds the wizard page. `bot_wizard.css` paints it. The host answers the calls the window already makes on the Qt wizard, so one set of calls drives either build.

| the window calls | the host answers with |
|---|---|
| the class, with the venue list and the stored defaults | `BotWizardReactDialog.__init__` |
| exec | `QDialog.exec` |
| DialogCode.Accepted | `QDialog.DialogCode` |
| get_bot_config | `BotWizardReactDialog.get_bot_config` |

**The page carries every asset it needs.** `dialog_html` inlines React, the four style sources, the table cells module, the wizard module and the sheet. The page fetches nothing over the network. `MODULE_GLOBALS` names the global each asset defines once its script tag has run.

**The page says what it loaded.** `LOADED_MODULES_JS` reads each of those globals back. `LOADED_STYLES_JS` counts the sheets the browser parsed and the rules whose selector names the wizard class. One run of the built host read this:

```
modules   react  react-dom  design_tokens  theme_engine  shared_widgets
          header_strip  table_cells  bot_wizard        all true
styles    sheets 1    rules 56    wizard rules 50
rail      asset  mode  params  phantom  extractor_pool   params current
rows      47 field rows drawn
```

The same run with the sheet emptied on disk read `wizard rules 0`, and the browser drew all 47 rows in its own plain type. The same run with the module emptied on disk read `bot_wizard.js false`, no rail stops and no rows.

**One model draws every page.** `BotWizardReactDialog.model` asks `bot_wizard_surface.view_model` for the whole wizard as one payload. The venue list, the stored defaults, the market list and the offered timeframes go in around the steps the page sends.

**A press on the page comes back to Python.** The page writes one console line. `BotWizardPage` hands that line to `answer_call`, which builds a fresh payload and hands it back. A finished walk accepts the dialog, and a cancelled walk rejects it.

**The sheet paints the parts the module names.** Every rule sits under the `acervator-bot-wizard` class and selects one `data-part` value.

| part | what the sheet paints |
|---|---|
| wizard-title | the window name, in the accent colour, over a rule |

| part | what the sheet paints |
|---|---|
| page-rail-stop | one pill per page, and the current page takes the accent |
| page-title, page-subtitle | the page heading and its line of help |
| field-group | a bordered block with its title in the accent |
| field-row-label, field | a right-aligned label against a growing control |
| target-combo, info-button | the pair box, its circle and the info button, on one line |
| timeframe-row | one chip per timeframe, and a chip the venue refuses dims |
| alt-list, alt-row | the pair list the pool page fills |
| warning-box | the call-budget message the phantom page raises |
| walk-step | the Next, Back, Cancel and Finish buttons |

**The host reaches no venue and creates no bot.** The venue list and the market list are arguments, so the dialog opens no connection of its own. A run that built the host and drew all five pages at three widths imported no module under `src/trading` and wrote no file under the runtime tree.

**The table row above still reads a dash.** `variant_surface.py` records no loader pair for this screen, so the running window builds the Qt wizard. The row changes when that pair is recorded.

### 2026-09-13 - #23 - the window chooses the three new hosts

`variant_surface.py` records three more loader pairs. `register` holds one Qt loader and one React loader for each screen. The running build selects one side of each pair.

| screen name | the Qt loader returns | the React loader returns |
|---|---|---|
| Charts | `TradeChartsTab`, in `widgets/trade_charts_tab.py` | `ChartsTabReact`, in `react_charts_tab.py` |
| Exchange page | `ExchangeTab`, in `widgets/exchange_tab.py` | `ExchangeTabReact`, in `react_exchange_tab.py` |
| Bot creation wizard | `BotCreationWizard`, in `bot_wizard.py` | `BotWizardReactDialog`, in `react_bot_wizard.py` |

Three call sites ask `surface_class` for the class. `ChartsTabMixin` builds the Charts tab in `_build_charts_tab`. `MainWindow` builds one venue page in `add_exchange_tab` and opens the wizard in `_create_bot`. Each call site keeps the Qt class as its fallback, so a screen with no recorded pair still builds the original.

The window names what it built. One window ran under each build, and each class came off the live object.

```
screen          qt build            react build
Charts tab      TradeChartsTab      ChartsTabReact
venue page      ExchangeTab         ExchangeTabReact
wizard          BotCreationWizard   BotWizardReactDialog
```

Each React page names what it loaded, once the window reaches it.

```
Charts tab   header_strip, native_chart, trade_charts_tab, no faults
venue page   nine modules, exchange_tab.js last, exchange_tab.css, 26 rules
wizard       eight module globals, all true, one sheet, 56 rules
```

The reading falls when one pair goes away. A run with the Charts pair deleted built the Qt tab and logged the refusal. The venue page and the wizard still answered with their hosts.

```
recorded pairs        Charts tab       venue page         wizard
all three             ChartsTabReact   ExchangeTabReact   BotWizardReactDialog
Charts pair deleted   TradeChartsTab   ExchangeTabReact   BotWizardReactDialog
```

The venue page keeps a dash in the table above. `TradingTabMixin` holds the Qt layer page as a hidden child in `_react_trading_page`, and that page holds the venue tabs. The React venue page draws its nine modules as soon as a run shows that layer page, so the host is built and reachable, and the Live tab hides it.

### 2026-09-13 - #23 - the two strips get a host and a stylesheet

`react_status_log.py` builds the Activity Log page. `status_log.css` paints it. `react_crypto_news_ticker.py` builds the news strip page. `crypto_news_ticker.css` paints it. Each host answers the calls the window already makes on the Qt widget, so one set of calls drives either build.

| **the window calls on** `StatusLog` | `StatusLogReact` **answers with** |
|---|---|
| log | `StatusLogReact.log` |
| force_log | `StatusLogReact.force_log` |
| pause, resume | `StatusLogReact.pause`, `StatusLogReact.resume` |
| is_paused, toggle_pause | `StatusLogReact.is_paused`, `StatusLogReact.toggle_pause` |
| health_stats | `StatusLogReact.health_stats` |
| setMaximumHeight | `QWidget.setMaximumHeight` |

| **the header calls on** `CryptoNewsTicker` | `CryptoNewsTickerReact` **answers with** |
|---|---|
| the class | `CryptoNewsTickerReact.__init__` |
| start | `CryptoNewsTickerReact.start` |
| stop | `CryptoNewsTickerReact.stop` |
| force_refresh | `CryptoNewsTickerReact.force_refresh` |
| current_headlines | `CryptoNewsTickerReact.current_headlines` |

**Each page carries every asset it needs.** `panel_html` inlines React, the four style sources, the strip's own module and its sheet. Neither page fetches over the network. `MODULE_GLOBALS` names the global each asset defines once its script tag has run.

**Each page says what it loaded.** `LOADED_MODULES_JS` reads those globals back. `LOADED_STYLES_JS` names each sheet and counts the rules the browser parsed from it. One run of the built hosts read this:

```
status log   modules  design_tokens theme_engine shared_widgets header_strip status_log
             styles   status_log.css 12 rules             lines drawn 8
news strip   modules  design_tokens theme_engine shared_widgets header_strip crypto_news_ticker
             styles   crypto_news_ticker.css 8 rules      headlines drawn 1
```

**The readings fall when an asset goes.** The same run with each sheet emptied read no rules, and the browser drew both strips on white. The same run with each module emptied dropped that module from the roster and drew no line and no headline.

```
run         status log            news strip
live        12 rules, 8 lines     8 rules, 1 headline
no sheet     0 rules, 8 lines     0 rules, 1 headline
no module   12 rules, 0 lines     8 rules, 0 headlines
```

**The pane paints a line by its kind.** `status_log_surface.line_style` reads the message prefix first and the level second. A pixel census of each saved picture read these counts.

```
live        0a0a0f 238504   0e0e1a 193353   7a7a9c 2594   00ff88 478   ffaa00 458
no sheet    ffffff 430648   ffaa00 1759     00ff88 1365   ff66dd 531   ffcc44 461
no module   0a0a0f 450000
```

The sheet carries the page ground, the pane ground and the border. The payload carries every line colour, so a line keeps its colour with no sheet at all.

**One payload draws the whole pane.** `pane_payload` asks `status_log_surface.build_view_model` for the pane state, then puts every line the model holds under `document`. `build_view_model` drains `take_painted`, so its own `document` carries one call's batch. `document_blocks` counts the whole document, and the two agree once `pane_payload` has run.

**The strip shows the story it holds.** `CryptoNewsTickerReact._run_fetch` runs the strip's worker only when the strip carries a `fetcher`. `CryptoNewsTickerModel.on_headlines` writes `NO_FEEDS_TEXT` for an empty answer, so a strip with no reader would report no feeds over the stories it holds.

**The strip reached no feed.** A run built the host with four recorded stories and no `fetcher`. The strip drew the first story with its position and its feed name. The run refused every connect away from loopback and counted none.

```
fetcher              None
fetches started      1
headlines held       4
connects attempted   0
label                [1/4] The Defiant - SEC crypto custody rewrite enters White House review
```

**Each sheet paints the parts its module names.**

| part | what the sheet paints |
|---|---|
| log | the read-only pane, its ground, its border and its scrollbar |
| line | one block, wrapped rather than widened |

| part | what the sheet paints |
|---|---|
| stamp | the leading time, in tabular figures |
| body | the message span, which takes its colour from the payload |
| placeholder | the line an empty pane shows |
| ticker | the frame around the strip |
| headline | one story on one line, clipped at the strip's edge |

**Both table rows above still read `shell`.** `variant_surface.py` records no loader pair for either strip, so the running window builds the Qt pane and the Qt strip. Each row changes when its pair is recorded.

### 2026-09-13 - #23 - four modules already had a host, and the tables get their skin

Four rows read `-` or `shell`. All four already draw in the window the operator launches. No new host was written. The measurement is below, and four cells move.

The venue page host mounts both bot tables inside itself. The Charts host mounts the chart module inside itself. The Status panel is its own host.

```
react_exchange_tab.CHILD_MODULES   table_cells, bot_status_table,
                                   extractor_bot_table, crypto_news_ticker
react_charts_tab.CHILD_MODULES     native_chart
react_system_status_tab            PANEL_MODULE system_status_tab.js
```

The window reaches all three. `main_window.py` asks the seam for the venue page class, and `charts_tab.py` asks it for the Charts class. `SystemStatusTabMixin` builds the Status panel with no seam, because no Qt widget ever drew that tab.

```
main_window.py:1361            page_class = surface_class(EXCHANGE)
main_tabs/charts_tab.py:29     built = surface_class(CHARTS)()
main_tabs/system_status_tab.py SystemStatusReactPanel()
```

**Each page names itself off the live object.** A run built each screen, drove content into it and read the class and the accessible name back.

```
ExchangeTabReact           React Exchange Tab   widget and web view
ChartsTabReact             React Charts Tab     widget and web view
SystemStatusReactPanel     Status               widget and web view
```

**Each page reports what it loaded.** The venue page already answered this. The Charts page and the Status page now answer it too, because `page_html` gives every style sheet a `data-asset` name of its own.

```
venue page   9 modules, exchange_tab.css 31 rules
Charts tab   3 modules, trade_charts_tab.css 20 rules, 0 panel faults
Status tab   1 module, system_status_tab.css 25 rules
```

**Each page drew the content driven into it.** Six bots went into the venue page, forty candles into the chart, and the emitter read-out into Status.

```
venue page   4 Scrumming rows, 2 Extractor rows, 56 cells, 6 coin badges
Charts tab   40 candles, 40 wicks, 40 volume bars, 3 indicator panels
Status tab   17 subsystems, 11 tab groups, 78 emitter rows
```

**The readings can fail.** Each page was built again with its style sheet emptied and again with its module absent. Every reading reported the loss.

```
sheet emptied    rule count 0, page paints white
venue module out modules drop both tables, rows 0 and 0, no cells
chart module out modules drop native_chart, fault "registered no panel to draw"
status module out modules empty, emitter rows 0
```

**Both variants drew the same content, and five colours did not match.** Each screen was built under the qt variant and the React variant with the same bots and the same candles. Every table cell agreed, header for header and row for row. Five colours read off the saved pictures did not.

| element | qt variant | React before | React now |
|---|---|---|---|
| column header text | `#00ffcc` | `#e0e0f0` | `#00ffcc` |
| header underline | 2px accent | 1px grey | 2px accent |
| Detail button face | `#1a1a28` | `#efefef` | `#1a1a28` |
| Fire button face | `#1a1a28` | `#efefef` | `#1a1a28` |
| coin badge | a coloured disc | nothing drawn | a coloured disc |

The badge is the disc `_get_coin_icon` paints when no logo file is cached. Its colour comes from the characters of the name. One function now answers for both sides, so the two cannot drift.

```
def coin_disc_color(symbol: str) -> str:
    hue = sum(ord(char) for char in str(symbol)) % HUE_WHEEL
    red, green, blue = colorsys.hsv_to_rgb(
        hue / HUE_WHEEL, DISC_SATURATION / ALPHA_HIGHEST, DISC_VALUE / ALPHA_HIGHEST
    )
```

Four discs were read off the qt picture and the same four off the React picture. They agree to the digit.

```
A15-USD   #5fb479      A23-USD   #5fb496
A14-USD   #5fb484      A21      #b45fb1
```

**Two differences stay, and each has a reason.** The header ground reads `#16162a` under Qt and `#1a1a28` under React. No design token holds the first value, and the sheet may name no colour the design system does not hold. The column rule reads a lighter grey under Qt for the same reason.

**The Charts tab chrome differs above the chart, and that belongs to another row.** The qt arrows draw no glyph, the qt timeframe box carries a `TF:` label, and the qt indicator toggles paint filled squares where React draws check boxes. Every one of those sits in `trade_charts_tab.js`, whose row already reads `yes`.

**Status draws the same under both variants.** It has no Qt original, so the window builds the React panel either way. The two pictures hold the same colours in the same counts at all three widths.

```
status qt      #141420 60397   #0a0a0f 28812   #ffffff 4080
status react   #141420 60397   #0a0a0f 28812   #ffffff 4080
```

Four cells move to `yes`. The Status cell moves because the column reports what the operator sees, and he sees the React panel in the window.

| Qt file | React module | before | after |
|---|---|---|---|
| `src/gui/widgets/bot_status_table.py` | `bot_status_table.js` | - | yes |
| `src/gui/widgets/extractor_bot_table.py` | `extractor_bot_table.js` | - | yes |
| `src/gui/native_chart.py` | `native_chart.js` | shell | yes |
| `src/gui/main_tabs/system_status_tab.py` | `system_status_tab.js` | shell | yes |

### 2026-09-13 - #23 - the tab bar paints from the theme again

**The ten grounds are gone.** The bar paints no ground of its own. Every tab now takes the colours the running theme's own style sheet sets, which is what the bar did before the grounds were added.

`src/gui/variant_surface.py` — `_qt_main_tab_book`

```
def _qt_main_tab_book() -> type:
    """Import and return the Qt main tab book."""
    from PySide6.QtWidgets import QTabWidget

    return QTabWidget
```

**Six theme tokens go with them.** Each of the five themes carried a ground colour and a text colour for black, white and gold. None of the thirty values is declared now, and no style sheet names one.

**CITED AS ABSENT.** The file `src/gui/main_tabs/main_tab_bar.py` is deleted, so the paint block earlier on this page records what the bar did between the seventh and the thirteenth of September 2026.

**Read off the rendered bar, five themes, both builds.** A bright pixel here is one reading at least 200 red, at least 150 green and at most 120 blue. The gold ground filled most of the bar before and none of it now.

```
theme             qt before  qt after   react before  react after
cyberpunk_dark    20,880     0          14,262        0
neon_light        20,771     0          14,178        0
classic_terminal  22,027     0          14,262        0
minimal_modern    20,643     0          14,091        0
glass_metal       20,604     0          14,078        0
```

### 2026-09-13 - #23 - Accumulation leaves the bar in one place

**One tuple names every tab the window does not build.** The window skips the builder for a name in it, and the tab list the frontend reads leaves that name out. The bar now reads Sim, Paper, Live, Charts, Inspector, Swarm, History, Status, Console.

`src/gui/main_tabs/main_window_surface.py` — `UNBUILT_TABS`

```
UNBUILT_TABS = (ACCUMULATION_TAB,)

BAR_TAB_ORDER = tuple(name for name in CANONICAL_TAB_ORDER if name not in UNBUILT_TABS)
```

**Removing the name from that tuple puts the tab back.** The Accumulation screen is unfinished, not unwanted. Its builder, its panel, its bridge method and its manual page are all unchanged.

### 2026-09-13 - #23 - a tab that fails to build no longer moves the tabs after it

**The reorder advances its slot only over a tab the bar carries.** A name the bar does not carry took a slot before, so every tab after a missing one stayed where the builders left it. The Qt build fails to build Sim and Paper, and its bar now reads the canonical order of the seven tabs it does have.

`src/gui/main_tabs/main_window_surface.py` — `reordered_tabs`

```
    order = list(labels)
    target_index = 0
    for name in desired:
        if target_index >= len(order):
            break
        for current_index in range(target_index, len(order)):
            if order[current_index] == name:
                if current_index != target_index:
                    order.insert(target_index, order.pop(current_index))
                target_index += 1
                break
```

**Read off the two running bars.** The React build carries all nine tabs it builds. The Qt build carries seven, in the same order.

```
react   Sim, Paper, Live, Charts, Inspector, Swarm, History, Status, Console
qt      Live, Charts, Inspector, Swarm, History, Status, Console
qt was  Live, Charts, Inspector, Console, Swarm, Accumulation, Status, History
```

**The two missing tabs have one cause.** The Qt Sim page and the Qt Paper page each read a panel key their own surface no longer publishes, so each builder raises and the window logs the tab as unavailable. That belongs to the Simulator rebuild and to the Paper screen, not to the bar.

# Subsystem Detail

Reference. One file per subsystem tab, written from the source. Each file names the module that implements the screen, the symbols inside it, the bridge method that serves its renderer, and the screen's current state.

The parent part is 08-tabs.md, which points here for every screen but Main Window, Market Inspector and Asset Charts, and carries the manual's own text for those three. The rows below run in the order the tab list in 04-manual-parts.md sets, which is the order 08-tabs.md runs its sections in.

## How an update is written down

Each tab has one section, and that section is its file below. An update to a tab takes a dated subsection under that one heading and never a second section named for the tab.

```
## 2026-09-08 14:20 - #407, #442 - the six zones and their arrows
   date       time    issues addressed   what the update did
```

The date leads so the subsections sort, and they run forward down the page. An update with no issue behind it writes `no issue recorded` in that field rather than a number nothing supports. `.claude/rules/documentation.md` bans an issue number in documentation, and the operator's directive of 2026-09-08 amends it for this header alone; version names stay banned everywhere.

`dev_harness/harness/docs_archetype.py` refuses both halves

```
rule_id="DOC011"   # two contents sections naming one tab
rule_id="DOC012"   # a dated update that runs backwards
```

## Contents

| File | Covers | State |
|---|---|---|
| portfolio-panels.md | The header strip: spendable columns, five counter cards, privacy dots | Live |
| simulator.md | Live's tab forked over Stone Tablets: Validation, Back Test, Portfolio Battery, the gate-latch criterion, the replay layer | Live, rebuilt under issue #117 |
| paper-trader.md | The live feed, the fake balance, the paper log | Live — issue #19 |
| proof-of-accumulation.md | Identity, Merkle log, competitions, ACRV, the local chain | Skeleton tab, engine only — issue #147 |
| market-inspector.md | Higher-timeframe scanner, opposing pairs, topology proposals, adopt | Live |
| bot-swarm.md | Nodes, Smart Wires, wire credits, the fold-tranche book | Live |
| asset-charts.md | One candlestick panel per traded symbol | Live |
| history.md | Venue trade history, grading, gate analysis | Live |
| console.md | Python log tail and the emitter signal stream | Live |
| system-status.md | Emitter Network and Watchdog, the two halves that run today | Skeleton tab — issue #34 |
| settings.md | The User page, the Exchanges page, and what each of the eleven pages persists | Live |
| promotion-pipeline.md | How a strategy earns its way to real money | Four of four steps built |

## The live tab set

One list names the ten tabs the window builds, and each one is moved into that position after the builders run. Two of them are empty: Accumulation and Status.

`src/gui/main_tabs/main_window_surface.py` — `CANONICAL_TAB_ORDER`

```
CANONICAL_TAB_ORDER = (
    SIM_TAB,
    PAPER_TAB,
    LIVE_TAB,
    CHARTS_TAB,
    INSPECTOR_TAB,
    SWARM_TAB,
    ACCUMULATION_TAB,
    HISTORY_TAB,
    STATUS_TAB,
    CONSOLE_TAB,
)
```

The Live tab has its own section: 06-trading-tab.md. The Indicator Voting Panel has 07-indicators.md.

Every screen the window does not build gets a sentinel instead of a widget, so a legacy reader gets that sentinel rather than an error. The list covers the Competition and Local Testnet tabs, the analytics, risk, journal, alerts and audio pages, and the retired Paper Trader widget, which is not the Paper tab issue #19 built.

`src/gui/main_tabs/retired_tabs.py` — `RetiredTabsMixin._install_retired_tab_sentinels`

```
def _install_retired_tab_sentinels(self) -> None:
    """Assign None to ``_paper_trader``, ``_competition_tab`` and the rest.

    ``main_window`` tests ``_analytics_tab``, ``_risk_tab``,
    ``_journal_tab`` and ``_alerts_tab`` before refreshing them.
    """
```

The underlying engines behind several of those keep running; only the surface is gone.

## How a screen reaches its renderer

One registry maps a method name to a view-model function. A surface is a plain function that takes the request's parameters and returns a serialisable result, and a screen is reachable exactly when its method appears in that registry.

`src/core/desktop_bridge.py` — `build_registry`

```
def build_registry() -> Dict[str, Handler]:
    """Return the methods the frontend may call, keyed by method name.

    This is the wiring point for the whole frontend: a surface is
    reachable exactly when it appears here. Surfaces are imported inside
    the function so that the transport above carries no dependency on any
    one domain package.
    """
```

The transport is one JSON object per line over the pipe the parent process already owns: no socket and no port.

Each surface module holds the screen as data. Nothing in one imports Qt, which is why the same view model serves the Qt tab and the renderer without a second description of the screen existing.

Back to the manual index.

# Portfolio Information Panels

Reference. The strip above the tab bar, present on every tab except the two isolated ones.

## What builds it

One method builds the strip. It makes the window's central widget, lays a single row across the top of it, and hands back the layout the tab row goes into. That row holds the five spendable columns on the left and the five counter cards on the right, and it does not move as you change tabs.

`src/gui/main_tabs/header_strip.py` — `HeaderStripMixin._build_header_strip`

```
self._spendable_widget = SpendableProfitsWidget()
top_row.addWidget(self._spendable_widget, stretch=3)
```

The row draws seven slots: the money strip, the five counter cards, and the asset class square that ends it. The first six divide the whole row between them, at three shares for the money strip and one for each card. The square takes no share, so its slot shrinks to the square and the square draws hard against the row's right edge. No background is left anywhere in the row.

`src/gui/main_tabs/header_strip_surface.py`

```
TOP_ROW_STRETCH = [3, 1, 1, 1, 1, 1, 0]
```

Each of the six also carries a floor, the width its own whole amounts need, so a long figure is never cut.

`src/gui/main_tabs/header_strip.py` — `HeaderStripMixin._build_header_strip`

```
card.setMinimumWidth(surface.slot_natural_w(slot))
```

Overtaken, quoted whole: "The first six divide the whole row between them, at three shares for the money strip and one for each card."

True today: the money strip's three shares come out below its own floor at every window up to about 1950 pixels wide, so it draws at that floor and the five cards take the whole surplus between them.

Both builds draw the row at the same geometry. The floor the Qt row sets on a widget is the number the React page sets on the same slot, and the surface serves that number to both.

`src/gui/main_tabs/header_strip_surface.py`

```
"slots": {slot: slot_natural_w(slot) for slot in TOP_ROW_ORDER}
```

`src/gui/web/header_strip.js`

```
style.minWidth = length(floor);
```

Measured at a 1920 pixel window, in both builds: the money strip 686, each card 217, the square 112.

## Left: the spendable columns

Five labelled columns fill the left half, divided by thin vertical rules.

| Column | Meaning |
|---|---|
| SPENDABLE | Cash the exchange reports, across the bots sharing one wallet |
| REALISED | Realised profit and loss |
| LOCKED | Value the open positions hold |
| MATURE | Wire profit old enough to move |
| EXCH | Connected exchange count |

The widget builds four of the five from one list, and each entry carries the label the operator reads and the payload key that fills it.

`src/gui/widgets/spendable_profits.py` — `SpendableProfitsWidget.__init__`

```
for label_text, key in [
    ("REALISED", "total_realised"),
    ("LOCKED", "locked"),
    ("MATURE", "mature"),
    ("EXCH", "exchanges"),
]:
```

One payload draws all five. A value the payload leaves out draws an em dash rather than a zero, which keeps "no reading" and "a reading of nothing" apart. That is the difference between a column with nothing behind it and a column reporting a genuine zero, and on this strip it matters: two of the five have nothing behind them today.

`src/gui/widgets/spendable_profits.py` — `SpendableProfitsWidget._money_text`

```
@staticmethod
def _money_text(value) -> str:
    """Render one amount as money text, or as the empty marker."""
    amount = SpendableProfitsWidget._amount_of(value)
    if amount is None:
        return "—"
    return f"${amount:,.2f}"
```

REALISED and MATURE are the two. Both are handed a literal absence at the one call site that fills the strip, so both draw the marker on every tick. The Trading tab section carries the proposal for the first of them:

06-trading-tab.md.

Both now carry a figure, and the figure comes from the exchange. Realised is the venue's own matched profit and loss across the fleet. Mature is the profit held in positions worth more than three times what they cost, which is the growth threshold the platform applies everywhere it decides maturity.

**Overtaken:** "Realised is the venue's own matched profit and loss across the fleet."

The venue answers a cost basis, an average entry price and an unrealised profit per spot position, and no lifetime realised figure. Realised is the platform's own first in, first out match over the venue's own fills, one figure per bot, added across the fleet.

**Overtaken:** "Mature is the profit held in positions worth more than three times what they cost, which is the growth threshold the platform applies everywhere it decides maturity."

Mature is the part of a position's value that sits over three times its cost. A position worth $350 on a $100 cost basis holds $50 of mature profit. Three times the cost is still the threshold the platform applies everywhere it decides maturity.

`src/gui/main_tabs/header_strip_surface.py` — `profits_payload`

```
"total_realised": exchange_amount(data, "total_realized_exchange"),
"mature": exchange_amount(data, "total_mature_exchange"),
```

Absence is still a reading. When the venue has answered for no bot, both columns take nothing and draw the marker, and the mask leaves that marker alone. A hidden figure shows four asterisks; an empty column stays empty under Privacy Mode.

`src/core/privacy_mask_registry.py` — `mask_or`

```
text = str(value)
if field_id not in ALL_FIELD_IDS or text == ABSENT_TEXT:
    return text
```

### What the venue answers, and what the platform derives

The venue's portfolio breakdown answers three figures per open spot position: the cost basis, the average entry price and the unrealised profit. Each bot reads those three from the venue on every refresh and holds them as its own. The breakdown carries no lifetime realised figure for a spot position, so Realised is derived: every fill the venue holds for the bot's symbol, matched first in, first out, one figure per bot, summed across the fleet.

`src/exchange/ccxt_connector.py` — `get_spot_positions`

```
portfolios = await self._call_sync(self._ex.fetch_portfolios)
rows = await self._call_sync(self._ex.fetch_portfolio_details, uuid)
basis = float(row.get("cost_basis", 0) or 0)
```

The venue's cost basis is the check on the derivation. After the fills are matched, the bot logs the venue's cost basis beside the cost of the buys FIFO left open and beside the average-cost basis, with each gap in per cent, and names the closer of the two. A venue whose account uses another matching method shows up there as a

gap on both.

`src/trading/scrumming/reconciliation.py` — `_log_basis_check`

```
_fifo_gap = abs(_fifo - float(venue_basis)) / _scale
_avg_gap = abs(_avg - float(venue_basis)) / _scale
_closer = "fifo" if _fifo_gap <= _avg_gap else "average"
```

A venue without a portfolio breakdown leaves the three figures to the same derivation, and the check does not run.

`src/exchange/position_health.py` — `compute_position_health`

```
while sell_remaining > 1e-12 and buy_queue:
    buy_lot = buy_queue[0]
    take = min(buy_lot[0], sell_remaining)
    realized += (t.price - buy_lot[1]) * take
```

A sell is matched against the oldest open buy, so a sell placed above the latest fold and below the first buy of a market that has fallen since lowers the figure. Over the 2026 fill export, 1,302 of 2,653 sells lowered it and 66 were priced under the most recent buy.

Each bot holds the complete fill history for its symbol. The first refresh pages the venue newest to oldest until a page comes back short; every later refresh fetches one page and stops at the first fill it already holds.

`src/exchange/fill_history.py` — `FillHistory.refresh`

```
if len(fills) < FILL_PAGE_LIMIT:
    ended_short = True
    break
if joined:
    break
```

Before this, one call fetched the newest 500 fills and the figure was FIFO over that window alone. On the two symbols past 500 fills the window read $-137.38 and $119.91 where the complete history reads $-390.53 and $60.27.

`src/trading/scrumming/reconciliation.py` — `refresh_exchange_position_health`

```
_trades = await self.fetch_fill_history()
if _trades is None:
    return False
_asset_base = self.config.symbol.split("/")[0]
_ph = compute_position_health(_trades, _asset_base)
```

Mature applies one constant. A position counts once its value reaches its cost plus two hundred per cent of its cost, and the column sums the profit on the positions that qualify.

`src/trading/smart_wire.py` — `mature_profit_usd`

```
MATURE_GROWTH_PCT: float = 200.0
if value < basis * (1.0 + MATURE_GROWTH_PCT / 100.0):
    return 0.0
return value - basis
```

**Overtaken:** "the column sums the profit on the positions that qualify."

The column adds up what each qualifying position holds over the threshold. `mature_threshold_usd` is the cost plus two hundred per cent of the cost, and `mature_profit_usd` answers the value above it.

`src/trading/smart_wire.py` — `mature_profit_usd`

```
if not is_mature(cost_basis_usd, current_value_usd):
    return 0.0
return float(current_value_usd) - mature_threshold_usd(cost_basis_usd)
```

A position at three times its cost exactly is mature and holds nothing over the threshold. `is_mature` carries that test on its own, so the count of mature positions reads the position while the money column adds nothing for it.

`src/trading/smart_wire.py` — `is_mature`

```
threshold = mature_threshold_usd(cost_basis_usd)
if threshold <= 0.0:
    return False
return math.isfinite(value) and value >= threshold
```

### Realised reads the same on every start

A bot writes its realised figure only when the walk reached the end of the venue's history. A walk that stops short keeps the last whole reading, writes `stats.fill_history_complete` False and says so in the log, so the same fills always answer the same figure.

`src/trading/scrumming/reconciliation.py` — `refresh_exchange_position_health`

```
_complete = bool(getattr(self._fill_history, "complete", False))
self.stats.fill_history_complete = _complete
if _complete:
    self.stats.realized_pnl_exchange = float(_ph.realized_pnl_usd)
```

The fleet aggregate answers `realised_history_complete` False while any bot the venue answered for reads False, and the column then draws the empty marker. An absent figure never looks like a present one.

`src/gui/main_tabs/header_strip_surface.py` — `realised_amount`

```
if not bool(data.get(REALISED_COMPLETE_KEY, True)):
    return None
```

The Paper and Simulator fleets walk no venue history, so their aggregates omit the key and their realised figure passes through whole.

## Right: the five counter cards

Five cards close the row, and each one counts a single thing.

| Card | What it counts |
|---|---|
| Scrummed | Cumulative USD sold across every bot since the process started |
| Folded | Cumulative USD bought across every bot since the process started |
| Trades | Executed buys and sells across the active bots |
| Bots | Bots in the RUNNING state |
| Errors | Errors across every bot since the last reset |

Errors is the one card you can click. Arming it takes one call, and the click opens the rolling error buffer with its Reset button.

`src/gui/main_tabs/header_strip.py` — `HeaderStripMixin._build_header_strip`

```
self._stat_errors.set_clickable(True, "Click to open the error log.")
self._stat_errors.clicked.connect(self._show_error_log_dialog)
```

Issue #428 carries a disagreement between what the tooltips on these cards promise and what the cards draw.

A sixth card, P/L, is constructed and then hidden. Code that still writes to it keeps working and nothing appears on screen.

`src/gui/main_tabs/header_strip.py` — `HeaderStripMixin._build_header_strip`

```
self._stat_pnl = StatCard("P/L", "$0.00")
self._stat_pnl.setVisible(False)
```

The mode button on the right ends the row. `_toggle_trading_mode` swaps the window between the crypto and stock layers.

Overtaken, quoted whole: "The mode button on the right ends the row. `_toggle_trading_mode` swaps the window between the crypto and stock layers."

True today: one square ends the row. It is segmented two by two into one button per asset class, and a press makes that class active. The square keeps its own size, never stretches, and sits in the row's right corner.

`src/gui/main_tabs/header_strip.py` — `HeaderStripMixin._build_header_strip`

```
top_row.addWidget(
    class_group,
    surface.slot_stretch("mode_button"),
    Qt.AlignVCenter | Qt.AlignRight,
)
```

## Privacy

Every field on the strip carries its own privacy dot. Attaching one registers the field, and from then on the card draws its value through the mask.

`src/gui/main_tabs/header_strip.py` — `HeaderStripMixin._build_header_strip`

```
self._stat_scrummed.attach_privacy_dot("counter.scrummed")
self._stat_folded.attach_privacy_dot("counter.folded")
self._stat_trades.attach_privacy_dot("counter.trades")
self._stat_bots.attach_privacy_dot("counter.bots")
self._stat_errors.attach_privacy_dot("counter.errors")
```

The registry behind those field ids is shared. Hide a field here and the same field hides on the Bot Swarm tab, because both surfaces ask the one registry.

`src/core/privacy_mask_registry.py` — `mask_or`

```
def mask_or(value, field_id: str, mask: str = "****") -> str:
    """Return ``mask`` when ``field_id`` is masked, else ``str(value)``.

    A ``field_id`` outside ``ALL_FIELD_IDS`` never masks, which covers every
    ``TA_FIELD_IDS_EXCLUDED`` entry.
    """
```

## Where the numbers come from

One aggregate call fills the whole strip, once a tick. Every card and every column is written from that single answer, so two cards cannot disagree about the same fleet.

`src/gui/main_window.py` — `_refresh_dashboard`

```
agg = self._bot_manager.get_aggregate_stats()
_scr = float(agg.get("total_scrummed_usd", 0.0) or 0.0)
_fld = float(agg.get("total_folded_usd", 0.0) or 0.0)
self._stat_scrummed.set_value(f"${_scr:,.2f}")
self._stat_folded.set_value(f"${_fld:,.2f}")
```

The aggregate itself is `FleetAggregationMixin.get_aggregate_stats` in `src/trading/container/aggregation.py`.

## Hiding

Two screens are isolated from live trading, and on those two the strip hides itself. The absent strip is the signal: no live numbers are on this screen.

`src/gui/main_window.py` — `_on_main_tab_changed`

```
isolated_tabs = set(ISOLATED_TABS)
container = getattr(self, "_header_strip_container", None)
if container is not None:
    container.setVisible(tab_name not in isolated_tabs)
```

The window surface carries the same pair as a constant, and issue #450 renamed both labels.

`src/gui/main_tabs/main_window_surface.py` — `ISOLATED_TABS`

```
ISOLATED_TABS = (SIM_TAB, PAPER_TAB)
```

The Simulator draws its own strip in place of this one. `SimStatStrip` in `src/gui/simulator_tab/sim_stat_strip.py` mirrors these ten fields against sim balances. The Simulator rebuild removed this file; it is not in the tree.

### The strip stays on Sim and reads the sim fleet

The Simulator rebuild took the Sim tab out of the isolated pair, so the strip hides on Paper alone. On Sim the same ten fields carry the Simulator's figures, read from the Sim tab's fleet source, and the live fleet's numbers do not reach the strip while Sim is in front. The window picks the fleet on every tick and on every tab change.

`src/gui/main_tabs/main_window_surface.py` — the two tuples

```
ISOLATED_TABS = (PAPER_TAB,)

#: The tabs the header strip reads the Simulator's fleet on, not the live one.
SIM_FED_TABS = (SIM_TAB,)
```

`src/gui/main_window.py` — `_refresh_header_strip`

```
if self._header_strip_reads_sim():
    sim_tab = self._simulator_tab
    self._write_header_strip(
        sim_tab.fleet_source().aggregate(), sim_tab.exchange_count()
    )
    return
```

simulator.md covers what each field reads on Sim.

## Bridge

The strip answers one bridge method, and one renderer module draws it.

`src/gui/main_tabs/header_strip_surface.py` — `METHOD`

```
METHOD = "header.strip"
```

The renderer module is `header_strip.js`.

Back to the subsystem index.

# Simulator Tab

Reference. The second step of the promotion pipeline: the live fleet, replayed against stored history. Issue #117 rebuilt the screen, and it carries three modes: Validation, Back Test and Portfolio Battery.

## What the tab holds today

The Sim tab is the Live tab's code, forked and modified, reading Stone Tablets and the YTD trade files instead of a venue. It sits first on the bar. The Qt build draws `SimTradingTab` and the React build draws `SimTradingTabReact`, which hosts one page module and six child modules forked from Live's. This section describes the tab in the order it draws, read off the running program in both builds on the current commit. The sections under "How each part was built" are each unit's own record, and the sections under "The widget the rebuild replaced" describe the screens that are gone.

`src/gui/variant_surface.py` — the two loaders

```
def _qt_simulator() -> type:
    """Import and return the Qt Sim tab, ``SimTradingTab``."""
    from .simulator.sim_trading_tab import SimTradingTab

def _react_simulator() -> type:
    """Import and return the React Sim tab, ``SimTradingTabReact``."""
    from .simulator.sim_react_trading_tab import SimTradingTabReact
```

### The header strip over the tab

The window's header strip stays in view with Sim in front, and it reads the Simulator's fleet rather than the live one. Only Paper is isolated. The five columns and the six cards are Live's: with nothing held the columns read an em dash and the cards read zero; with a fleet held, the cards read Live's arithmetic over the held records, and EXCH counts the seated venues.

`src/gui/main_tabs/main_window_surface.py` — the tabs the strip reads from

```
ISOLATED_TABS = (PAPER_TAB,)

#: The tabs the header strip reads the Simulator's fleet on, not the live one.
SIM_FED_TABS = (SIM_TAB,)
```

Spendable follows the run mode. In Validation and Portfolio Battery it is the run's budget, the sum of the held Target Balances, because no buy is refused for cash in those two modes. In Back Test it is Live's wallet arithmetic over the held bots' own cash.

`src/gui/main_tabs/simulator_tab_surface.py` — the wallet by mode

```
MODE_FUNDING = {
    MODE_VALIDATION: back_test.FUNDED_BY_TARGETS,
    MODE_BACK_TEST: back_test.FUNDED_BY_PROCEEDS,
    MODE_PORTFOLIO_BATTERY: back_test.FUNDED_BY_TARGETS,
}
```

### The venue stack and the Get Started card

The Simulator starts empty. At open no venue is seated and the Get Started card shows in the venue pane. The card asks for a first fleet, says a fleet is what begins a run, and offers the two ways in of the active mode, which is Validation at open. Import Live Fleet copies the stored fleet's records for one exchange, and the venue stack then seats one sub-tab per exchange the held fleet names. Every fleet change seats the stack again.

`src/gui/simulator/sim_trading_tab.py` — the seating

```
    def _sync_exchange_tabs(self) -> None:
        wanted = [str(eid) for eid in self._fleet_source.exchanges() if eid]
        for eid in wanted:
            self.add_exchange_tab(eid, exchange_display_name({"exchange_id": eid}))
        self._drop_unlisted_exchange_tabs(wanted)
        self.refresh_bots()
        self.refresh_votes()
        self._refresh_replay()
```

Since one fleet per mode landed, the fleet the stack seats is the active mode's. The tab's fleet source holds three fleets, one for Validation, one for Back Test and one for Portfolio Battery, and every reader and every act works on the fleet of the mode in force. A mode press moves the source's mode and fires the same fleet change, so the venues, the rows, the strip and the way-in row redraw from that mode's fleet; a venue that only one mode's fleet names unseats when another mode is shown and seats again on return. A mode whose fleet is empty shows the Get Started card, and the card holds the three mode buttons above its two ways in, so the operator can leave an empty mode from the card. The section "One fleet per mode" under "How each part was built" describes the change.

`src/simulator/fleet_source.py` — the fleet in force

```
    @property
    def _records(self) -> dict[str, dict]:
        """The records map of the fleet in force, ``_fleets`` under ``_mode``."""
        return self._fleets[self._mode]

    def set_mode(self, mode: str) -> str:
        """Make ``mode`` the fleet every reader and act works on; a name
        outside ``MODES`` changes nothing. Answers the mode in force."""
        if mode in MODES:
            self._mode = str(mode)
        return self._mode
```

### The mode row and the corner

The row above the bot list holds the three mode buttons, Validation, Back Test and Portfolio Battery, then `+ New Bot`. The active mode's button wears Live's Privacy Mode ON sheet and the other two the OFF sheet. The data-pool row under it holds nothing. The venue tab bar's corner holds the active mode's two ways in, and the Get Started card holds the same two. Validation offers Import Live Fleet and Generate From YTD. Back Test offers Import Live Fleet and Create New Bots. Portfolio Battery offers Run Portfolio and Run Every Portfolio. The mode row lives on the venue page, so before a fleet is held the card offers Validation's two.

`src/gui/main_tabs/simulator_tab_surface.py` — the modes and their ways in

```
MODES = (MODE_VALIDATION, MODE_BACK_TEST, MODE_PORTFOLIO_BATTERY)

ROWS_FOR_MODE = {
    MODE_VALIDATION: RESERVED_ROWS,
    MODE_BACK_TEST: BACK_TEST_ROWS,
    MODE_PORTFOLIO_BATTERY: BATTERY_ROWS,
}
```

The three mode names and their tuple now live in the fleet source, because the sim fleet file is keyed by them; the surface imports the same four names, so the block above reads the same values from `src/simulator/fleet_source.py`. A mode press does more than restyle the row: the host hands the mode to the source, then fires the fleet change, and the tab draws that mode's fleet. A press while a run or a Battery is in flight writes the in-flight line and changes nothing. The card under the mode row holds the same three buttons.

`src/gui/simulator/sim_trading_tab.py` — the press

```
        self._fleet_source.set_mode(mode)
        for venue in list(self._crypto_exchange_tabs.values()) + list(
            self._stock_exchange_tabs.values()
        ):
            venue.show_mode(mode)
        self.fleet_changed.emit()
```

The corner's first button, in every mode, is Clear Fleet. It sits to the left of the mode's two ways in, at their size. A press empties the held fleet: every record on every venue leaves, the venues unseat, the Get Started card returns, and the sim fleet file is written with no record. The card holds Clear Fleet above its two ways in only while a fleet is held; with nothing held the card offers the two ways in alone. The section "Clear Fleet empties the held fleet" under "How each part was built" describes the press.

Since the way-in row landed, Clear Fleet and the two ways in draw on a row above the exchange tab row, not in the corner, and only while a fleet is held; the paragraph after the next block describes that row.

`src/gui/simulator/sim_trading_tab_surface.py` — the corner's list

```
def way_in_buttons(mode: str = sim.MODES[0]) -> list:
    """The corner's buttons as the page draws them: ``clear_fleet_button``
    first, then the two ways in ``mode`` offers from ``sim.reserved_rows``."""
    return [clear_fleet_button()] + [
        {
            "action": row["action"],
            "text": row["text"],
            "accessible_name": row["button_name"],
            "minimum_width_px": live.ADD_BUTTON_MIN_WIDTH_PX,
            "minimum_height_px": WAY_IN_BUTTON_HEIGHT_PX,
        }
        for row in sim.reserved_rows(mode)
    ]
```

The buttons the corner held draw on their own row above the exchange tab row: Clear Fleet, the mode's two ways in, then Start Run, in that order from the left, each at the corner button's size. The corner itself holds nothing, so the exchange tabs take the full row and a third venue's tab has room. The row draws only while a fleet is held. With no fleet held the row is absent and the Get Started card offers the mode's two ways in. A press on Start Run starts the active mode's run over the fleet on the venue shown: the Validation rerun, the Back Test walk, or the Portfolio Battery chooser and its run. The card never holds Start Run. The section "Start Run starts the active mode's run from the way-in row" under "How each part was built" describes the row and the press.

`src/gui/simulator/sim_trading_tab.py` — the row above the tab bar

```
            way_in_row_widget = QWidget()
            way_in_row = QHBoxLayout(way_in_row_widget)
            way_in_row.setContentsMargins(*tab_surface.WAY_IN_ROW_LAYOUT["margins_px"])
            way_in_row.setSpacing(tab_surface.WAY_IN_ROW_LAYOUT["spacing_px"])
            way_in_row.addStretch(1)
            way_in_row_widget.setVisible(False)
            page_layout.addWidget(way_in_row_widget)
```

`src/gui/simulator/sim_trading_tab_surface.py` — the row's last button

```
def start_run_button() -> dict:
    """The corner's Start Run button, at the way-in buttons' size."""
    return {
        "action": sim.START_RUN_ACTION,
        "text": sim.START_RUN_TEXT,
        "accessible_name": sim.button_name(sim.START_RUN_ACTION),
        "minimum_width_px": live.ADD_BUTTON_MIN_WIDTH_PX,
        "minimum_height_px": WAY_IN_BUTTON_HEIGHT_PX,
    }
```

The row's list takes the held count and is empty at zero, so the row is not drawn; above zero it is Clear Fleet, the mode's two and Start Run.

```
    if int(held or 0) <= 0:
        return []
    return (
        [clear_fleet_button()]
        + [
            {
                "action": row["action"],
                "text": row["text"],
                "accessible_name": row["button_name"],
                "minimum_width_px": live.ADD_BUTTON_MIN_WIDTH_PX,
                "minimum_height_px": WAY_IN_BUTTON_HEIGHT_PX,
            }
            for row in sim.reserved_rows(mode)
        ]
        + [start_run_button()]
    )
```

### The two tables and the command bar

The Scrumming Bots table is the forked `SimBotStatusTable`. It hides until a row arrives, then draws one row per held scrumming bot with Live's ten columns, the Bot ID cell coloured by the bot's state. The Extractor Bots table sits under it, forked the same way, and hides until an extractor record is held. Under the tables the command bar holds Start, Pause, Stop, Restart and Delete. Pause, Restart and Stop move the selected bot's state through the Simulator's bot manager, and the row reads it back. Delete asks Live's confirmation and removes the record. Start is the run's button in Validation and Back Test.

Since Start Run landed on the way-in row, Start on the command bar moves the selected bot's state in every mode, as Pause, Stop and Restart do, and starts no run. The run's button is Start Run on the way-in row. While a run is in flight, Stop on one of its rows ends it and every other bar press writes the in-flight line.

`src/gui/simulator/sim_trading_tab.py` — the bar's Start, every mode

```
        if command == "start":
            try:
                moved = self._bot_manager.start(bot_id) != bot.state
                self._status_log.log(f"✓ Bot {bot_id} RUNNING.", "success")
```

`src/simulator/sim_bot_manager.py` — the states the bar moves

```
    def pause(self, bot_id: str) -> str:
        """``BotContainer.pause``'s rule: any state lands on ``paused``."""
        self._require(bot_id)
        return self._fleet.set_state(bot_id, BotState.PAUSED.value)

    def stop(self, bot_id: str) -> str:
        """``BotContainer.stop``'s rule: any state lands on ``stopped``."""
        self._require(bot_id)
        return self._fleet.set_state(bot_id, BotState.STOPPED.value)
```

Each row's Fire and Detail buttons are Live's. Detail opens the Simulator's Bot Settings window over the bot's record with Live's seven tabs for a scrumming bot: Status, Settings, Fold Tranches, Stack Tranches, Bot Swarm, Market Inspector and Phantom Bots. The window reads the record; opening and closing it writes nothing to the fleet file. `+ New Bot` opens the Simulator's Bot Wizard, titled Create Auto Trader, whose Finish holds one new record.

`src/gui/simulator/sim_bot_detail.py` — the tabs a scrumming bot's window carries

```
        tabs.addTab(
            self._wrap_scrollable(self._create_status_tab()), surface.TAB_STATUS
        )
        tabs.addTab(
            self._wrap_scrollable(self._create_settings_tab()), surface.TAB_SETTINGS
        )
        if mode == surface.MODE_SCRUMMING:
            self._install_fold_tranches_tab(tabs)
            self._install_stack_tranches_tab(tabs)
```

The held fleet is the Simulator's own. Every fleet change writes it to `sim_fleet.json` under the sim bucket of the log root, and the tab reads it back at the next build, so a created or imported bot survives a restart. The live `bot_state.json` is never written. The Simulator receives and asks; it never sends.

`src/core/log_paths.py` — the bucket

```
def get_sim_dir() -> Path:
    """``sim/`` bucket — every file the Simulator writes.

    ``src.simulator.fleet_source.FleetSource`` keeps the sim fleet file
    here; ``~/.acervator/bot_state.json`` is never written from the
    Simulator.
    """
    p = _LOG_ROOT / "sim"
```

### The Indicator Voting Panel and the replay layer behind it

The panel beside the venue pane is the forked `SimIndicatorVotingPanel`. It carries Live's title, Bot selector, currency rate strip, two pillar tables, two confidence bar graphs, timeframe-lock line and staleness banner. Its selector holds the held fleet and follows each bot's state. While the selected bot runs, the pillar tables draw its reading from its Stone Tablet and the banner names the tablet's last candle. A bot that is not running draws Live's cause instead.

`src/gui/simulator/sim_trading_tab.py` — the feed

```
        feed = tab_surface.ivp_feed(
            self._tablet_source, self._fleet_source.bot_for(chosen)
        )
        if feed.get("summary"):
            panel.show_stored(
                feed["stored"], feed["when"], feed["age"], feed["message"]
            )
        else:
            panel.show_no_data(
```

One flip button sits after the panel title and reads Replay. A press shows the layer behind the panel: the VWAP window over the Stone Tablet playback window, each drawn from the chosen tablet's last hundred candles. The layer's header holds the flip, now reading Indicators, a Tablet chooser 240 px wide that lists every tablet on disk, and one button reading Retrieve Tablet or Update Tablet. A press runs the retrieval or update through the Simulator's read-only connector on a worker thread, records each venue call on the API Interaction Log, and writes the started line and the finished or refused line on the Activity Log. The connector answers `get_ohlcv` over the public candle endpoint and raises `SendRefused` for every other name. This layer is the first of the three permitted differences from Live.

`src/simulator/read_only_connector.py` — the refusal

```
    def __getattr__(self, name: str) -> Any:
        """Refuse every name outside ``READ_NAMES``."""
        raise SendRefused(
            f"ReadOnlyConnector answers {READ_NAMES} and cannot {name!r}. "
            "The Simulator receives and asks; it sends nothing."
        )
```

The flip button now sits first in the header, at the panel's top-left corner, before the title, and the replay layer's header seats it first at the same corner. It draws one width on both layers, the width of the wider word, so Replay and Indicators occupy one rect and the way back is where the way in was. Each press emits `sim.layer.flipped` through `signal_contract` with the pressed rect against the previous press's rect.

`src/gui/simulator/sim_trading_tab.py` — the seat

```
        if self._layer == surface.LAYER_PLAYBACK:
            self._indicator_panel.header_row().removeWidget(self._flip_button)
            self._chart_header.insertWidget(FLIP_SEAT_INDEX, self._flip_button)
        else:
            self._chart_header.removeWidget(self._flip_button)
            self._indicator_panel.header_row().insertWidget(
                FLIP_SEAT_INDEX, self._flip_button
            )
```

### The two spools

The Activity Log sits at the foot left with Pause Console, and the API Interaction Log at the foot right with Pause API Log, Live's titles at Live's positions. Pause holds new lines and Resume releases them in order.

`src/gui/simulator/sim_trading_tab.py` — the pause

```
        def _on_activity_pause_toggled(checked: bool):
            if checked:
                self._status_log.pause()
                self._activity_pause_btn.setText("■  Resume Console")
            else:
                self._status_log.resume()
                self._activity_pause_btn.setText("■  Pause Console")
```

The Activity Log shows the trades of Validation, Back Test and Portfolio Battery runs, one line per simulated trade in Live's fill line shape, stamped with the candle's time rather than the clock. That is the second permitted difference. The API Interaction Log records Stone Tablet and YTD retrieval activity only: its log is a `SimApiLog` that accepts two action words and refuses every other by name, so a venue order line cannot reach it. That is the third. Generate From YTD writes one block per press.

`src/simulator/sim_api_log.py` — the allowed actions

```
TABLET_ACTION = "FETCH_TABLET"

#: The word the Generate From YTD read's action opens with.
YTD_ACTION = "FETCH_YTD"

#: Every action word ``SimApiLog.record`` accepts. Any other raises ``SendRefused``.
ALLOWED_ACTIONS = (TABLET_ACTION, YTD_ACTION)
```

### The three run modes and what each writes

Every mode's run starts from Start Run on the way-in row. Under Validation and Back Test the press starts the run the two paragraphs below describe, which Start on the command bar started before the row's button landed. Under Portfolio Battery the press opens Run Portfolio's chooser, one portfolio and one span, and runs it. One handler on each host, `_start_run_pressed`, reads the mode and the venue shown.

`src/gui/simulator/sim_trading_tab.py` — the press by mode

```
        elif mode in tab_surface.RUN_MODES:
            outcome = self._start_run(venue, mode)
        else:
            outcome = self._run_battery(surface.RUN_PORTFOLIO_ACTION)
```

Validation. With Validation active, Start on the command bar moves every scrumming bot on the page to running, reruns the gate chain on the candle each recorded fill fell in, and puts the rerun lights beside the recorded ones. The run passes when they latch identically, never on profit and never on a trade count. Each rerun fill reaches the Activity Log as a trade line, and the bots land on stopped when the run ends.

`src/simulator/validation.py` — the criterion

```
    @property
    def latches_identically(self) -> bool:
        """True while every light and both armed flags read the same."""
        return (
            not self.disagreed
            and self.recorded_scrum_armed == self.rerun_scrum_armed
            and self.recorded_fold_armed == self.rerun_fold_armed
        )
```

Back Test. With Back Test active, Start walks each bot along its Stone Tablet with the Live bot's own sizing arithmetic, shared as pure code, and each scrum and fold reaches the Activity Log as it fills.

`src/simulator/back_test.py` — the shared arithmetic

```
from ..trading.scrumming.sizing import (
```

Portfolio Battery. With Portfolio Battery active, Run Portfolio opens a chooser of one portfolio over the archive's thirty-five and one span over seven, and Run Every Portfolio opens it with the span alone. The run holds one bot per symbol at $500 times the portfolio's mix share, walks the RA-StoneTablets on a worker thread, and writes one reading per timeframe.

`src/simulator/portfolio_battery.py` — the default Target Balance

```
DEFAULT_TARGET_USD = 500.0
```

Since the Battery walked every candle, each bot evaluates every bar of its tablet from the thirtieth on, the Activity Log carries a line at each walk's start, every 5,000 bars and at its end, Run Every Portfolio holds one portfolio's bots, walks them, writes their report, clears them and holds the next, every reading is HODL end, Harvest-Fold end and the difference, and Stop on a Battery row ends the walk at the bar reached.

`src/simulator/portfolio_battery.py` — the evaluations a walk makes

```
def evaluations_expected(bars: int) -> int:
    """How many gate-chain evaluations a walk over ``bars`` makes: one per bar
    from the ``MIN_CANDLES``th on, none under ``MIN_CANDLES``."""
    return max(0, int(bars) - (MIN_CANDLES - 1))
```

Stop on a row of the run in flight ends the run where it is.

`src/gui/simulator/sim_trading_tab.py` — the stop

```
    def _stop_run(self, bot_id: str) -> None:
        self._status_log.log(f"Stopping bot {bot_id}...", "info")
        self._run["stopper"] = bot_id
        self._status_log.log(
            tab_surface.run_stopping_line(self._run.get("mode", ""), bot_id), "warning"
        )
        self._run_stop.set()
```

Every run writes one trade parity report, a Markdown file and a JSON sidecar, under the simulator directory of the reports directory, named for the mode, the subject, the span and the time. A stopped run's report reads stopped and partial.

`src/simulator/parity_report.py` — the file stem

```
def report_name(mode: str, subject: str, span: str, stamp: str) -> str:
    """The file stem: mode, subject, span and stamp joined by ``__``."""
    return "__".join(
        (safe_name(mode), safe_name(subject)[:SUBJECT_LIMIT], safe_name(span), stamp)
    )
```

Every run also emits Live's topics on the Simulator's private event bus, and Live's log manager writes them under the sim bucket as four files: one row per gate evaluation, per fill and per voting summary, each gate and voting row carrying the run id the report carries. The live bucket's files are untouched.

`src/simulator/sim_bus.py` — the files

```
#: The files ``LogManager.attach_to_bus`` writes the four topics into.
SIM_LOG_FILES = ("trade.log", "gate.log", "voting.log", "diagnostics.log")
```

Every emit a run raises on its worker thread, the TA engine's postcondition pins among them, lands in the Simulator's own signal sink, `signals/session.jsonl` under the sim bucket, and none in the process sink under `signals/` of the log root. The Activity Log names the sim sink's path at each run's start, and the process sink gains one `sim.sink.routed` row at each run's end.

`src/simulator/sim_bus.py` — the line at a run's start

```
#: The Activity Log line ``routed_run`` writes at each run's start.
SINK_LINE_FORMAT = "{run} signals: {path}"
```

### The colour

Every Sim ground is the theme's nigredo tone. The widget tree carries the tone property, and the theme restates each ground token a quarter of the way to black under that selector, so the Sim's grounds sample darker than Live's under the same theme while the text and accent colours stay the theme's.

`src/gui/theme_engine.py` — the tone

```
TONE_PROPERTY = "tone"
NIGREDO = "nigredo"

#: The share of the distance to black every ``NIGREDO_GROUNDS`` token moves.
NIGREDO_FRACTION = 0.25
```

### Where the tablets live

The tablet archive is in the repository. A clone carries all 407 files, so the history arrives with the code and no venue is called to fetch it. 406 are candle tablets and one is the index they are read through.

`src/trading/stone_tablets/storage.py` — the root a source checkout resolves

```
def _tablet_root() -> Path:
    if is_frozen():
        return HOME_TABLETS_DIR
    return project_root() / "stone_tablets"
```

A tablet is never modified. It is recorded venue history, and a back test over a tablet that drifted proves nothing. Tracking makes that rule checkable rather than merely stated: the archive is under version control, so any later change to any file in it shows up as a difference against the committed copy.

The line-ending rule the repository applies to text does not reach the archive. The index file holds 5,283 carriage returns, and normalising them would change its digest and break the checksum every row carries.

`.gitattributes` — the archive's exemption

```
stone_tablets/** -text
```

A built executable carries no archive, so it reads the home tree instead, at `~/.acervator/stone_tablets`. A source run never reads that copy.

### What is not built

Privacy Mode, the news line and the data-pool line are not on the tab; their row holds the mode buttons, by ruling. A column map exists for the Coinbase export alone, so an export from another exchange is refused at import by name until a sample of it exists. Portfolio Battery walks an asset class only under a unit rule cited from its venue, and a class with no cited rule is not simulated.

In development.

## How each part was built

Every section from here to "The widget the rebuild replaced" is one build's own record, kept as it was written when that build merged, in build order: the Stone Tablet and RA-StoneTablet sections, the three mode sections, the YTD trade files and the tab's builder from before the fork, then units 7 through 31a. Where a later unit changed what a sentence describes, the sentence now says so and names that unit's report on issue #117. The section above describes the tab on the current commit.

## The candles

Stone Tablets are the history and a replay only reads them. The registry stores one timeframe and derives every other from it.

`src/trading/stone_tablets/registry.py` — `NATIVE_TIMEFRAME`

```
NATIVE_TIMEFRAME: str = "5m"
"""Timeframe every tablet stores; ``_rollup`` derives all the others."""
```

Three functions in `src/trading/stone_tablets/storage.py` move tablets between disk and memory.

| Function | What it does |
|---|---|
| `read_tablet` | Loads one tablet from disk |
| `write_tablet` | Writes one back |
| `read_manifest` | Carries the index of what exists |

The tape the removed Fleet Replay handed its bots did not roll up: it asked the registry for the native timeframe only, and the tape refused any other unless the caller supplied that series when the tape was built. The rebuild removed Fleet Replay (pull request 454); the forked tab reads tablets through its tablet source, and the section "What the tab holds today" describes it.

`src/exchange/tablet_backend.py` — `TabletBackend.fetch_ohlcv`

```
        key = (str(symbol), tf)
        rows = self._tf_rows.get(key)
        if rows is None:
            raise ValueError(
                f"no {tf} series for {symbol!r}. Serving the native "
                f"{NATIVE_TIMEFRAME} series instead would make every "
                f"timeframe agree with itself; supply tf_rows[{key!r}]."
            )
```

## Portfolio Battery history

The thirty-five portfolios from the historical archive are now named in code, and their sixty-three symbols are what the new mode runs over. Each one carries its symbols, an equal weight for every symbol, and the archive it was read out of.

`src/simulator/portfolios.py` — one portfolio

```
    "CRYPTO_BLUE": Portfolio(
        name="CRYPTO_BLUE",
        symbols=("A15", "A14", "BNB"),
        description="Large-cap crypto — institutional grade",
    ),
```

Their price history is real, and it is kept apart from the live fleet's. The tablets a replay reads sit under one root; the battery's sit under a second one, and a battery build never opens the first.

`src/trading/stone_tablets/ra_paths.py` — the second root

```
_RA_ROOT: Path = Path.home() / ".acervator_ra_tablets"
```

Two sources fill it and neither needs a key. Crypto arrives through the adapter the fleet already uses, driven by the venue's own public candle endpoint. Everything else arrives through a second adapter beside it.

`src/trading/stone_tablets/ra_fetcher.py` — the non-crypto adapter

```
class YahooChartAdapter(ExchangeAdapter):
    exchange_id = "yahoo"
    chunk_limit = RA_CHUNK_DAYS
    BASE_URL: str = "https://query1.finance.yahoo.com/v8/finance/chart"
    SOURCE: str = "yahoo_chart_v8_ONE_DAY_SPLIT_ADJUSTED"
```

Every tablet records where its numbers came from and when they were fetched, and the builder refuses to write one without both. A price with no source is what made the archive's own figures worthless.

`src/trading/stone_tablets/ra_fetcher.py` — the refusal

```
        resolved = source or str(getattr(adapter, "SOURCE", ""))
        if not resolved:
            raise ValueError(
                f"{type(adapter).__name__} carries no SOURCE; pass source= naming "
                f"the endpoint the candles come from. {adapter.exchange_id!r} is "
                f"an exchange id, not provenance."
            )
```

Where a source has nothing, nothing is written in its place. The missing days are recorded as missing, in their own file beside the tablets.

`src/trading/stone_tablets/ra_fetcher.py` — what a missing period records

```
@dataclass
class TabletGap:
    """One requested period a source returned no rows for."""

    asset: str
    exchange_id: str
    timeframe: str
    year: int
    since_ms: int
    until_ms: int
    reason: str
    checked_at: str
```

At the RA coverage build nothing ran the bot logic over these tablets and no screen showed them; unit 24's Portfolio Battery walks them from the corner (comment 5725991788).

`src/simulator/portfolio_battery.py` — the run over these tablets

```
def plan_run(
    names: Sequence[str],
    tablets: Any,
    held: Sequence[SimBot] = (),
```

## Portfolio Battery coverage

Every symbol in every portfolio has real daily prices on disk. No portfolio names a period of its own, so all six archive periods apply to all thirty-five of them, and the span asked for is 2020 to 2026.

`src/trading/stone_tablets/ra_import.py` — the years asked for

```
def _archive_years() -> tuple[int, ...]:
    """Return every calendar year ``PERIODS`` touches, ascending."""
    years: set[int] = set()
    for start, end in PERIODS.values():
        years.update(range(int(start[:4]), int(end[:4]) + 1))
    return tuple(sorted(years))
```

A symbol reaches one source or the other by what it is. A coin the crypto venue never listed falls to the second source instead of being left empty, and every tablet records which one served it.

`src/trading/stone_tablets/ra_import.py` — the routing

```
def route_for(symbol: str) -> SymbolRoute:
    """Return ``symbol``'s route, crypto to Coinbase and everything else to Yahoo."""
    upper = symbol.upper()
    if is_crypto(upper):
        return SymbolRoute(upper, COINBASE_SOURCE, YAHOO_CRYPTO_SOURCE)
    return SymbolRoute(upper, YAHOO_SOURCE)
```

The import reads its own result back off disk and says what it holds. A count on its own would not be an answer, so the statement carries every source with its fetch times and every period no source served.

`python -m src.trading.stone_tablets.ra_import coverage` — the headline

```
SYMBOLS
  asked for ......... 63
  returned data ..... 60
  tablets ........... 411
  candles ........... 105336

SOURCES
  coinbase_exchange_candles_ONE_DAY: 71 tablets
  yahoo_chart_v8_ONE_DAY_SPLIT_ADJUSTED: 340 tablets
```

A share year is not a calendar year. The market shuts at weekends and on public holidays, and those days are recorded as missing rather than filled, so a share year holds about 250 days where a coin year holds every one.

`python -m src.trading.stone_tablets.ra_import coverage` — a share and a coin

```
  SPY
    2022    251 rows  2022-01-03..2022-12-30
    2023    250 rows  2023-01-03..2023-12-29
  A15
    2022    365 rows  2022-01-01..2022-12-31
    2023    365 rows  2023-01-01..2023-12-31
```

A ticker that no longer reaches the company the archive meant is recorded as a finding, not as a failure. Three of the sixty-three return nothing at all, and what the endpoint answered is written down in place of a price.

`~/.acervator_ra_tablets/GAPS.json` — a ticker that no longer trades

```
  CCIV   2021 yahoo  [2021-01-01..2021-12-31] HTTPError: HTTP Error 404: Not Found
  EXPR   2021 yahoo  [2021-01-01..2021-12-31] HTTPError: HTTP Error 404: Not Found
  IPOF   2021 yahoo  [2021-01-01..2021-12-31] HTTPError: HTTP Error 404: Not Found
```

Three more symbols are short at one end. `BBBY` reaches a company first traded in July 2026 rather than the one the archive ran, and `MATIC` stops on 14 October 2025 because the coin was renamed.

`python -m src.trading.stone_tablets.ra_import coverage` — a symbol cut short

```
  MATIC  missing: 2026
    2024    366 rows  2024-01-01..2024-12-31
    2025    287 rows  2025-01-01..2025-10-14
```

At the RA coverage build nothing ran the bot logic over these prices and no screen showed them; unit 24's Portfolio Battery walks them from the corner (comment 5725991788).

`src/simulator/portfolio_battery.py` — the default Target Balance each symbol runs at

```
DEFAULT_TARGET_USD = 500.0
```

## Validation Mode

Validation takes each YTD trade, finds the historical candle its timestamp falls in, and reruns the gates on that candle. It then puts the gate row it latched beside the gate row the log recorded, one light at a time, over the nineteen lights the shared vocabulary names. A run passes when every light reads the same. It is never judged on profit and never on a trade count.

`src/simulator/validation.py` — the criterion

```
    @property
    def latches_identically(self) -> bool:
        """True while every light and both armed flags read the same."""
        return (
            not self.disagreed
            and self.recorded_scrum_armed == self.rerun_scrum_armed
            and self.recorded_fold_armed == self.rerun_fold_armed
        )
```

### The two ways in

Import Live Fleet clones the live running fleet from the bot_state load and keeps each bot's own id, so a recorded gate row belongs to a bot by that id. Generate From YTD scans the trade files and creates one new bot per pair that was traded. Those bots are new, they carry new ids, and they never wrote a gate row, so a recorded row belongs to one of them by exchange, symbol and the time window instead.

```
Import Live Fleet    38 bots, matched on bot id and time window
Generate From YTD    39 bots, matched on exchange, symbol and time window
```

The two counts differ, and that is a fact about the two sources rather than a fault. A pair traded earlier in the year can have no live bot now, and a live bot can have traded nothing yet. Neither path invents a bot to close the gap.

When more than one exchange is active the tab asks which one to use before it runs anything. One chooser serves both buttons.

`src/simulator/fleet_source.py` — the chooser

```
def exchange_choice(exchanges: list[str], chosen: str = "") -> dict:
    """Whether the operator must pick an exchange, and which one is in force.

    ``prompt`` is True while more than one exchange is active and ``chosen``
    names none of them.
    """
```

### It reports what it could not verify

The tablets end on 1 August 2026 and the trade export runs to 7 September, so about five weeks of trades have no candle to snap to. Validation names that period and counts the entries on both sides of it. A count of verified trades with no denominator is exactly what this replaces.

```
3942 of 4904 YTD entries snapped to a candle; 962 could not be.
after_last_candle: 905
inside_gap: 57
uncovered span 2026-08-01T18:52:58Z to 2026-09-07T21:46:48Z; the newest
candle is 2026-08-01T18:35:00Z
```

### Which half of the rerun moves

A candle supplies the price and the Bollinger reading. It cannot supply the bot's own state, so the delta, the tranche counts, the circuit breaker and every flag come from the row the log recorded. Each light says which half drove it, so a disagreement can be read back to its cause.

```
S/BB   F/BB   F/MID     the tablet candle
S/LS   F/LS             the landing-strip override
every other light       the recorded row
```

### What the first full run measured

Every recorded reading the run checked was reproduced from the tablet exactly, but from a window sitting four hours behind the trade's own candle. The tablet prices are right: measured over one asset, 9,869 of 10,038 recorded prices sit inside the tablet candle covering their own moment. The tape is therefore sound, and the live reading it is compared against was taken from older candles than the ones that moment held.

```
38 bots, matched to a recorded gate row on bot id and time window.
62 of 227 reruns latched every gate identically.
3572 of 4313 gate lights agreed.
60 of 60 recorded readings were reproduced exactly from the tablet.
45 of them came from a window 50 candles (250 minutes) behind the trade's
own candle.
```

## Back Test Mode

Back Test runs the bot logic over a recorded tape. It walks each bot along its own Stone Tablet, feeds every window into the gates that run live, and records where a scrum and a fold latch. A latch that clears every gate then trades: the scrum sells the excess above the dollar target and the fold buys it back with the cash that scrum put aside.

`src/simulator/back_test.py` — the one gate chain, shared with Validation

```
from .validation import (
    BB_MIDLINE,
    MIN_RERUN_CANDLES,
    RERUN_WINDOW_CANDLES,
    bb_reading,
    candle_interval_ms,
    iso_stamp,
    latch,
    tablet_for,
)
```

At the first clone a mode selector sat above the two button rows and a result pane below the Replay Log changed with it; unit 20 put the mode buttons on the venue page's header row, and the corner holds the active mode's two ways in (comment 5720691825).

```
Mode: Validation    Import Live Fleet   Generate From YTD
Mode: Back Test     Import Live Fleet   Create New Bots
```

### The two ways into a back test

Import Live Fleet clones the live running fleet from the bot_state load, the same reader Validation uses. At the first clone Create New Bots made one simulated bot on the Stone Tablet the selector showed, at the Bot Wizard's defaults; since unit 23 it opens the forked wizard and holds one record from the wizard's values (comment 5723334396).

`src/gui/main_tabs/simulator_tab_surface.py` — the new bot the selector defines

```
def new_bot_specs(entry) -> list[dict]:
    """One new-bot spec for ``entry``'s asset, on the Bot Wizard's own
    defaults."""
```

### One clock

Every bot ticks on one cadence. The step is sized once from the longest tape in the run, so a short tablet and a long one are read on the same clock rather than each on its own.

```
def shared_step(entries: Sequence[Any], ticks_per_bot: int, max_candles: int) -> int:
    """One step in candles, sized off the longest tape so every bot ticks
    together.

    A ``ticks_per_bot`` of zero, or no entry, answers one.
    """
```

### What one back test measured

The imported fleet, read on 8 September 2026 from the live Stone Tablets.

```
38 of 38 bots ran over 1610472 tablet candles.
2026-01-01T00:00:00Z to 2026-08-01T18:35:00Z, 5272 gate-chain evaluations.
58 scrum latches and 55 fold latches.
58 scrum sells and 55 fold buys filled, $38.02 in fees.
```

Each row names the bot, its tablet, how much tape it read, how often the gates latched, how many trades filled, the coin it gained or gave up and the cash it now holds. Units alone do not answer whether a cycle accumulated, because a bot that sold into a rise holds fewer coins and more cash.

```
Bot ID | Symbol | Tablet | Candles | Ticks | Scrum / Fold latched |
Scrum / Fold filled | Units gained | Cash held
```

### A missing tablet is named, not fetched silently

A bot whose asset has no tablet is listed with no tape and counted in the run's own lines. Filling one is a call for information and goes through the same gap filler the tablet build uses, into whichever tablet root the caller names.

`src/simulator/back_test.py` — the fetch, and the answer with no connector

```
async def download_missing(
    pairs: Sequence[tuple[str, str]],
    connector: Any,
    since_ms: int,
    until_ms: Optional[int] = None,
    quote_currency: str = "USD",
    registry: Any = None,
) -> list[Any]:
```

Driven into a throwaway root with a recorded tape, it wrote one tablet of 701 candles and a manifest, and the live tablet root kept its 407 files.

```
connector calls: 3
report: ZZZTEST coinbase chunks_ok 3 errors 0 candles 701
entry: ZZZTEST coinbase 5m 701 candles read back: 701
live root unchanged: True 407
```

## Portfolio Battery Mode

The operator's words, 2026-09-18: *"Honest framing should be to show our strategy's ability to identify opportunities for partial exists and re-entries so as to improve portfolio end states. [...] I only seek to compare HODL against Harvest-Fold across as many historical markets and asset classes as possible."* The sentence left out says the mode is not a contest. The Battery reads one comparison, HODL end against Harvest-Fold end and the difference, per portfolio, per span, per timeframe; the section "The Battery walks every candle, one portfolio at a time, and reads HODL against Harvest-Fold" says how.

The third mode runs the thirty-five archive portfolios over their own price history. Every symbol streams its daily bars through the same gate chain a live bot ticks, at three timeframes, and each result is set against what

holding that symbol untraded would have paid over the same span.

`src/simulator/portfolio_battery.py` — the three timeframes

```
#: The timeframes a daily tablet builds, spelled as ``HTF_TIMEFRAMES`` spells
#: them.
TIMEFRAMES = ("1d", "1w", "1M")
```

### The baseline comes off the same walk

Buy and hold is not a figure carried over from the archive. It is the capital the symbol opened with, carried from the opening price to the closing price of the same walk, so the two sides cannot read different prices.

`src/simulator/portfolio_battery.py` — the baseline

```
    @property
    def baseline_usd(self) -> float:
        """``capital_usd`` carried from ``start_price`` to ``end_price``,
        untraded."""
        if self.start_price <= 0.0:
            return 0.0
        return self.capital_usd * self.end_price / self.start_price
```

### One gate chain, fed a different source

The battery defines no gate of its own. It hands its bars to the Back Test walker, which builds the shipped scrum and fold chains and reads their answer.

`src/simulator/portfolio_battery.py` — the walk

```
    bot = battery_bot(asset, exchange_id, timeframe, capital_usd)
    result = walk(bot, candles_from_raw(bars), walk_step(len(bars), ticks))
```

### A weekly and a monthly bar are folded, never invented

The tablets hold one bar a day. A coarser bar is the calendar bucket its days fall in: the first open, the highest high, the lowest low, the last close and the summed volume. A trailing bucket is kept as it stands rather than padded out.

`src/simulator/portfolio_battery.py` — one folded bar

```
def fold_bucket(rows: Sequence[Sequence[float]]) -> list[float]:
    """One bar from ``rows``: first open, highest high, lowest low, last close,
    summed volume."""
    return [
        float(rows[0][0]),
        float(rows[0][1]),
        max(float(one[2]) for one in rows),
        min(float(one[3]) for one in rows),
        float(rows[-1][4]),
        sum(float(one[5]) for one in rows),
    ]
```

### Three symbols hold no tablet

CCIV, IPOF and EXPR stopped trading and no source served them. A portfolio holding one runs on the symbols that exist and reports the share of its capital that reached no tape. That share is a column on every row, beside the count of symbols that ran.

`src/simulator/portfolio_battery.py` — the missing share

```
    @property
    def missing_weight(self) -> float:
        """``missing_usd`` as a share of every symbol's ``capital_usd``."""
        whole = sum(one.capital_usd for one in self.runs)
        return self.missing_usd / whole if whole > 0.0 else 0.0
```

### The two ways into a battery

At the first clone the two rows the removed strips left carried one portfolio and all of them, with a portfolio selector and a span selector under the mode selector; since unit 24 Run Portfolio opens a chooser of the portfolio and the span, and Run Every Portfolio of the span alone (comment 5725991788). The span list is the archive's six periods followed by the whole tape.

`src/gui/main_tabs/simulator_tab_surface.py` — the two ways in

```
BATTERY_ROWS: tuple[dict[str, Any], ...] = (
    {
        "name": NEWS_TICKER_ROW,
        "height_px": 24,
        "action": RUN_PORTFOLIO_ACTION,
        "text": RUN_PORTFOLIO_TEXT,
        "button_name": button_name(RUN_PORTFOLIO_ACTION),
    },
    {
        "name": DATA_POOL_ROW,
        "height_px": 18,
        "action": RUN_EVERY_PORTFOLIO_ACTION,
        "text": RUN_EVERY_PORTFOLIO_TEXT,
        "button_name": button_name(RUN_EVERY_PORTFOLIO_ACTION),
    },
)
```

### What one battery measured

SPAC_BUST over the archive's Apr24-Apr25 period, driven through the Qt tab. Three of its five symbols have a tablet. The accumulation logic ended ahead of buy and hold at the daily bar and at the weekly bar. The monthly bar holds twelve bars, under the thirty the Bollinger window and the voting engine need, so it reports no run rather than a result.

```
SPAC_BUST  1d  3 of 5  756 bars  669 ticks  35 trades
           buy and hold $390.75   accumulation $415.60   +24.85 (+6.36%)
SPAC_BUST  1w  3 of 5  159 bars   72 ticks   6 trades
           buy and hold $458.26   accumulation $478.74   +20.48 (+4.47%)
SPAC_BUST  1M  0 of 5    0 bars    0 ticks   0 trades   missing weight 100%

missing: IPOF (no_tablet), CCIV (no_tablet)
7 recorded gaps fall inside the span
```

Every portfolio over the whole tape reads 189 symbol runs in 26 seconds. Twelve of the thirty-five ended ahead of their own buy and hold at one timeframe or more.

### The run from the corner

Run Portfolio opens a chooser of the same shape as the exchange chooser, with a portfolio row listing the thirty-five names and a span row listing the seven spans; Run Every Portfolio opens it with the span row alone. On Ok the tab gives each portfolio its bots. Where the held fleet carries one scrumming bot per symbol of the portfolio, on the venue the run reads that symbol from, with a Target Balance above zero, those bots run at their own Target Balances and gate settings. Otherwise one bot is generated per symbol at the default Target Balance of $500 times the portfolio's mix share; every archive portfolio's mix is equal, so each bot reads $500. A portfolio that records the size of each position would run at those sizes; no archive entry records one.

`src/simulator/portfolio_battery.py` — each bot's Target Balance

```
def symbol_targets(
    portfolio: Portfolio, default_usd: float = DEFAULT_TARGET_USD
) -> dict[str, float]:
    """Each symbol's Target Balance: its ``positions_usd`` size when
    ``sizes_known``, else its ``weights`` share of ``default_usd`` per symbol,
    to the cent."""
    if portfolio.sizes_known:
        sizes = portfolio.positions_usd or {}
        return {symbol: round(float(sizes[symbol]), 2) for symbol in portfolio.symbols}
    if portfolio.mix is None:
        return {symbol: float(default_usd) for symbol in portfolio.symbols}
    whole = float(default_usd) * len(portfolio.symbols)
    weights = portfolio.weights
    return {symbol: round(whole * weights[symbol], 2) for symbol in portfolio.symbols}
```

The run's bots become the held fleet: a bot already held under its id at the same symbol, venue and Target Balance keeps its record, a generated bot is held under the origin `battery`, and every other held record is dropped. The Scrumming Bots table then draws the run's bots and the strip's Spendable reads the run's budget, the sum of their Target Balances, before the run starts.

`src/simulator/fleet_source.py` — the run's bots become the held fleet

```
    def hold_battery_fleet(self, bots: Sequence[SimBot]) -> list[SimBot]:
```

The run itself walks on a worker thread, as the Market Inspector's Scan Now does, so the window keeps answering; each portfolio's reading reaches the Activity Log as it lands, then the run's own lines and the report line. A second press while a run is in flight writes one line and starts nothing. A symbol with no tablet or no cited unit rule is missing weight and named in the report, and the run proceeds over the rest.

`src/gui/simulator/sim_trading_tab.py` — the worker thread

```
        self._battery_thread = threading.Thread(
            target=self._compute_battery,
            args=(plan, span),
            name="sim-portfolio-battery",
            daemon=True,
        )
        self._battery_thread.start()
```

### What the Battery flow measured

Before, in both builds over a scratch home holding a stored fleet of two coinbase bots and five 2022 tablets, a corner press on Run Portfolio opened no chooser and wrote one refusal line, and the runner driven directly walked every symbol at $200 with the outcome line reading `not computed`. After, Run Portfolio on CRYPTO_BLUE over 2022 opened the chooser, held three bots at $500 each with Spendable reading $1,500.00, ran on the worker thread, wrote one report whose Battery section read `1d improved, 1w defended, 1M not run` with BNB named as `no_tablet` and its $500 as missing weight, and wrote the report line; BOGLEHEAD over the same span read the same words with all three symbols run; Run Every Portfolio held sixty-three bots at $31,500.00 and wrote one report. A planted portfolio with a 60/30/10 mix read Targets of $900, $450 and $150; a stored fleet of A15, A14 and BNB on coinbase at $1,000, $750 and $250 ran at those Targets with the report naming the bots loaded from the held fleet. Over a copy of the whole tape, Run Every Portfolio took 24.3 seconds in Qt and 24.9 in React while a 30 ms timer kept ticking with a longest gap of 0.342 and 0.125 seconds; the same run on the GUI thread read a gap of 1.035 seconds over a 1.32 second run. Every scratch file hashed equal before and after every press, no bot was constructed and no socket left loopback.

### It is not a Monte Carlo

A Monte Carlo samples many paths and reports the spread of what they pay. This mode reads one recorded path per symbol, so it reports one outcome per timeframe and no distribution. Nothing on the screen carries a percentile.

`src/simulator/portfolio_battery.py` — the spans, one recorded path each

```
#: Every span a run may be asked for: the archive's six, and the whole tape.
SPANS = (FULL_SPAN,) + tuple(PERIODS)
```

## The YTD trade files and the import

The Simulator's second data source, kept as the first clone's record wrote it; unit 22 reads these files through Generate From YTD and unit 22a gave the importer one column map per exchange export.

### The YTD trade files

The Simulator's second data source is the operator's own trade record, kept on disk under the exchange history bucket. One file holds one exchange, one symbol and one year. Each file names its exchange inside itself, not only in its filename, so a file that is moved or renamed still says where its trades came from.

`src/exchange/ytd_trade_store.py` — what one row holds

```
@dataclass
class YtdTrade:
    """One fill: ``id``, ``ts_ms``, ``side``, ``amount``, ``price``, ``cost``
    and ``fee``."""

    id: str
    ts_ms: int
    side: str
    amount: float
    price: float
    cost: float
    fee: float
    fee_currency: str
```

Beside the rows, each file carries a schema version, the time of the import, and one entry per export that contributed rows. That entry names the export file and its digest, so a file that grew over two imports records both. A MANIFEST beside the files indexes every one of them with its row count and its date span.

`src/exchange/ytd_trade_store.py` — the provenance on each file

```
@dataclass
class ImportSource:
    """One import that contributed rows: ``file``, ``sha256`` and ``rows_added``."""

    file: str
    sha256: str
    imported_at: str
    rows_added: int
```

### The columns the import requires

The record arrives as a transactions CSV exported from the exchange. The import reads nine columns and refuses a file missing any one of them, naming which. Notes, sender address and recipient address are never carried in.

`src/exchange/ytd_csv_import.py` — the required columns

```
REQUIRED_COLUMNS: tuple[str, ...] = (
    COL_ID,
    COL_TIMESTAMP,
    COL_TYPE,
    COL_ASSET,
    COL_QUANTITY,
    COL_PRICE_CURRENCY,
    COL_PRICE,
    COL_SUBTOTAL,
    COL_FEES,
)
```

The transaction type decides the side. Buys and sells are kept; reward income, deposits and withdrawals are counted and dropped, because they are not trades and must never reach a validation run as though they were. The asset and the price currency together give the pair. The quantity is negative on a sell in the export, so its sign is checked against the type and then the magnitude is stored.

A second import of an overlapping export adds only the rows whose id is new. A file that gains nothing is not rewritten at all.

### One column map per exchange export

The nine columns above are Coinbase's. Each exchange writes its own export, so the import carries one column map per exchange, chosen by the exchange id the import is given. A map names the export's column for each field of a trade, its buy and sell type strings, the number of rows above its header, the form of its timestamps and the currency marks on its money cells. Every map lands on the same trade fields and the same file, so Generate From YTD reads one format whichever exchange wrote the export.

`src/exchange/ytd_csv_import.py` — the map

```
@dataclass(frozen=True)
class ExportColumnMap:
    """The columns, type strings and text forms of one exchange's CSV export,
    each column named for the ``YtdTrade`` field it fills."""

    exchange_id: str
    export_name: str
    id_column: str
    timestamp_column: str
    type_column: str
    asset_column: str
    quote_column: str
    quantity_column: str
    price_column: str
    cost_column: str
    fee_column: str
    buy_types: frozenset[str]
    sell_types: frozenset[str]
    header_scan_lines: int
    timestamp_suffix_utc: str
    timestamp_format: str
    money_prefixes: str
    dropped_columns: tuple[str, ...] = ()
```

Coinbase's map holds the nine columns, the two type sets, the twenty header rows, the timestamp suffix and the currency marks the import read before the maps existed, so a Coinbase export writes the same files it always did. A map is written from a sample export on disk and never from documentation. An exchange with no map is refused at import by name, with a line saying a sample export is needed, and nothing is written.

`src/exchange/ytd_csv_import.py` — the registry and the refusal

```
EXPORT_MAPS: dict[str, ExportColumnMap] = {COINBASE_MAP.exchange_id: COINBASE_MAP}

def export_map_for(exchange_id: str) -> ExportColumnMap:
    """Return ``EXPORT_MAPS[exchange_id]``, refusing an ``exchange_id`` with no
    map by name."""
```

```
no column map for exchange 'kraken': a sample kraken export is needed before its map is written. Maps exist for: ['coinbase'].
```

### The exchanges and their sample exports

One row per exchange Acervator connects to. A map exists for Coinbase alone, written from the operator's own transactions export. No other sample export is on hand, and the export each of the others would map is not named until its sample arrives, because a name taken from documentation is a guess.

```
exchange     map       sample export on hand                        export it maps
coinbase     present   yes, the operator's own transactions export   Coinbase transactions export
binance      none      no                                           not named until a sample arrives
kraken       none      no                                           not named until a sample arrives
kucoin       none      no                                           not named until a sample arrives
bybit        none      no                                           not named until a sample arrives
okx          none      no                                           not named until a sample arrives
gateio       none      no                                           not named until a sample arrives
bitget       none      no                                           not named until a sample arrives
huobi        none      no                                           not named until a sample arrives
mexc         none      no                                           not named until a sample arrives
bitfinex     none      no                                           not named until a sample arrives
gemini       none      no                                           not named until a sample arrives
poloniex     none      no                                           not named until a sample arrives
bitstamp     none      no                                           not named until a sample arrives
cryptocom    none      no                                           not named until a sample arrives
```

### A period with no rows is a gap

Where the export carries no trade for a symbol in a period the export covers, that period is written to a gap record. Nothing is invented to fill it.

`src/exchange/ytd_trade_store.py` — the gap record

```
@dataclass
class TradeGap:
    """One period between ``since_ms`` and ``until_ms`` that no row covers."""

    exchange_id: str
    symbol: str
    year: int
    since_ms: int
    until_ms: int
    reason: str
    checked_at: str
```

### Reading the trade files

The Simulator reads these files through one class, on the same rule as the tablet reader. It answered six names at the first clone and answers eight since unit 22 (comment 5722467005), refusing every other name itself.

`src/simulator/ytd_trade_source.py` — the refusal

```
    def __getattr__(self, name: str):
        """Refuse every name outside ``READ_NAMES``."""
        raise SendRefused(
            f"YtdTradeSource answers {READ_NAMES} and cannot {name!r}. "
            "The Simulator receives and asks; it sends nothing."
        )
```

### What the import run measured

The operator's own export was read into a throwaway directory. The figures below come from that run.

```
export read     5,798 rows, 13 columns
kept            5,766 trades - 3,229 buys, 2,537 sells
dropped         32 - reward income 16, deposits 15, withdrawals 1
written         39 files, one per symbol, all of them 2026
range           2026-04-12 to 2026-09-07
gaps recorded   76
second import   0 rows added, 39 files byte-identical
```

## How the Sim tab reaches the bar

`SimulatorTabMixin` builds the tab and inserts it into the window's tab book. The builder catches every error, writes one warning line, and leaves the tab off the bar. A tab that asks the panel for a value the panel does not publish is therefore a missing tab, not a crash the operator can see.

`src/gui/main_tabs/simulator_tab.py` — the builder

```
def _build_simulator_tab(self) -> None:
    """Insert the Sim tab at ``SIMULATOR_BUILD_INDEX``."""
    try:
        from ..variant_surface import SIMULATOR, surface_class

        self._simulator_tab = surface_class(SIMULATOR)()
        self._main_tabs.insertTab(
            SIMULATOR_BUILD_INDEX, self._simulator_tab, HEADING
        )
    except Exception as exc:  # noqa: BLE001 - a missing tab is not a crash
        logger.warning("Sim tab unavailable: %s", exc)
        self._simulator_tab = None
```

The window builds nine tabs. Sim takes the first slot and Paper the second. The Accumulation tab stays off the bar until the operator asks for it.

```
Sim  Paper  Live  Charts  Inspector  Swarm  History  Status  Console
```

## The Qt tab is Live's tab code, forked

The Qt build of the Sim tab is `SimTradingTab`, a fork of the Live tab's own code under the Simulator's names. The seam's Qt loader answers it; the widget written from scratch, `SimulatorTabQt`, stays in `src/gui/simulator_tab.py` and nothing loads it.

`src/gui/variant_surface.py` — the Qt loader

```
def _qt_simulator() -> type:
    """Import and return the Qt Sim tab, ``SimTradingTab``."""
    from .simulator.sim_trading_tab import SimTradingTab

    return SimTradingTab
```

Each module under `src/gui/simulator/` is one Live module copied and renamed. The copy keeps Live's layout, titles, sizes and design tokens; it changes the name, the feed and the send side, and nothing else.

| Simulator module | forked from |
|---|---|
| `sim_trading_tab.py` `SimTradingTab` | `src/gui/main_tabs/trading_tab.py` `_build_trading_tab`, and the window's `add_exchange_tab` |
| `sim_exchange_tab.py` `SimExchangeTab` | `src/gui/widgets/exchange_tab.py` `ExchangeTab` |
| `sim_bot_status_table.py` `SimBotStatusTable` | `src/gui/widgets/bot_status_table.py` `BotStatusTable` |
| `sim_extractor_bot_table.py` `SimExtractorBotTable` | `src/gui/widgets/extractor_bot_table.py` `ExtractorBotTable` |
| `sim_indicator_panel.py` `SimIndicatorVotingPanel` | `src/gui/indicator_panel.py` `IndicatorVotingPanel` |
| `sim_status_log.py` `SimStatusLog` | `src/gui/widgets/status_log.py` `StatusLog` |

Diagram: flowchart, left to right

- variant_surface SIMULATOR -> **SimTradingTab**
- SimTradingTab -> **exchange layer stack**
- exchange layer stack -> **SimExchangeTab**
- SimExchangeTab -> **SimBotStatusTable**
- SimExchangeTab -> **SimExtractorBotTable**
- SimTradingTab -> **QStackedWidget**
- QStackedWidget -> **SimIndicatorVotingPanel**
- QStackedWidget -> **LineView over PlaybackView**
- SimTradingTab -> **SimStatusLog**
- SimTradingTab -> **API Interaction Log**

### What the fork draws

The tab is Live's four splitters at Live's sizes: the exchange layer stack beside the panel on top, the Activity Log and the API Interaction Log side by side below. Every module carries Live's title. The venue page holds the Scrumming Bots table and the Extractor Bots table, both hidden until a row arrives, and the command bar of Start, Pause, Stop, Restart and Delete. The panel holds its title, the Bot selector and its privacy dot, the currency rate strip, both indicator tables, both confidence bar graphs, the timeframe-lock line and the staleness banner, hidden until raised.

Three positions differ from Live by ruling. The corner Live gives ■ `Add Crypto Exchange` holds Import Live Fleet, Generate From YTD and Create New Bots, each at Live's corner-button width and height. The row above the bot list, where Live draws Privacy Mode and the news line, holds Validation, Back Test and Portfolio Battery, then `+ New Bot` where Live draws it. The data-pool row keeps Live's height and holds nothing. Privacy Mode, the news line and the data-pool line are not forked.

`src/gui/simulator/sim_trading_tab.py` — the corner buttons

```
WAY_IN_BUTTONS = (
    (surface.IMPORT_LIVE_FLEET_ACTION, surface.IMPORT_LIVE_FLEET_TEXT),
    (surface.GENERATE_FROM_YTD_ACTION, surface.GENERATE_FROM_YTD_TEXT),
    (surface.CREATE_NEW_BOTS_ACTION, surface.CREATE_NEW_BOTS_TEXT),
)
```

The replay layer sits behind the panel in one stack. The flip button seats itself on whichever layer is showing: after the panel title, or at the head of the replay layer, so the way back is never hidden with the panel.

`src/gui/simulator/sim_trading_tab.py` — the flip

```
        if self._layer == surface.LAYER_PLAYBACK:
            self._indicator_panel.header_row().removeWidget(self._flip_button)
            self._chart_header.insertWidget(0, self._flip_button)
        else:
            self._chart_header.removeWidget(self._flip_button)
            self._indicator_panel.header_row().insertWidget(1, self._flip_button)
```

The seat later moved to the head of both rows, `FLIP_SEAT_INDEX`, and the chart header reads its margins and spacing off the panel's header row, so the button lands at one corner under both layers.

`src/gui/simulator/sim_trading_tab.py` — the chart header's margins

```
        panel_header = self._indicator_panel.header_row()
        self._chart_header = QHBoxLayout()
        self._chart_header.setContentsMargins(panel_header.contentsMargins())
        self._chart_header.setSpacing(panel_header.spacing())
```

### What the fork does not carry

The copies hold no bot manager, no connector and no event bus. Every send the Live code makes is cut, not stubbed: the signal-contract emits that write under `~/.acervator_logs/signals/`, the event-bus emit behind the bot selector, the API-log listener, the activity-log watchdog that read the live bot manager, the TA snapshot store that read and wrote `~/.acervator/ta_snapshots/`, the data-pool read behind the display price, and the demo readings the Live panel invents from a random walk when no bot is loaded. Zero occurrences of `ScrummingBot`, `BotContainer`, `ExchangeInterface`, `EventBus` and `PhantomBalance` under `src/gui/simulator/`.

At unit 7 the way-in buttons, the mode buttons, `+ New Bot`, the Fire buttons and the command bar's handler were wired to nothing, and the tab held two sources that nothing read. Units 10 to 31 wired each of them (comments 5706487452, 5707202420, 5708384371, 5714936418, 5716410892, 5718837037, 5721499839, 5728708856, 5733262015); the section "What the tab holds today" describes what each does now.

`src/gui/simulator/sim_trading_tab.py` — the two sources

```
        self._tablet_source = (
            tablet_source
            if tablet_source is not None
            else TabletSource(surface.TABLET_ROOT)
        )
        self._fleet_source = fleet_source if fleet_source is not None else FleetSource()
```

Asked for a send by name, each raises `SendRefused`.

At unit 7 the window's header strip hid while Sim was in front, as `ISOLATED_TABS` then named it. Unit 9 took Sim out of that tuple and fed the strip from the sim fleet (comment 5705741677); the section "The header strip shows on Sim" describes the strip as it draws.

## The React page is Live's page modules, forked

The React build of the Sim tab is `SimTradingTabReact`, a fork of the Live tab's React host under the Simulator's name, and it loads seven page modules that are Live's seven copied under Simulator names. The seam's React loader answers it; the page written from scratch, `SimulatorTabReact` in `src/gui/react_simulator_tab.py` with `simulator_tab.js`, stays in the tree and nothing loads it.

`src/gui/variant_surface.py` — the React loader

```
def _react_simulator() -> type:
    """Import and return the React Sim tab, ``SimTradingTabReact``."""
    from .simulator.sim_react_trading_tab import SimTradingTabReact

    return SimTradingTabReact
```

Each forked module sits beside its source under `src/gui/web/`. The copy keeps Live's layout, part names, sizes and design tokens; it changes the bridge method it asks, the globals it defines, the feed and the send side, and nothing else.

| Simulator module | forked from |
|---|---|
| `sim_trading_tab.js` | `trading_tab.js` |
| `sim_exchange_tab.js` | `exchange_tab.js` |
| `sim_indicator_panel.js` | `indicator_panel.js` |
| `sim_status_log.js` | `status_log.js` |
| `sim_bot_status_table.js` | `bot_status_table.js` |
| `sim_extractor_bot_table.js` | `extractor_bot_table.js` |
| `sim_table_cells.js` | `table_cells.js` |
| `src/gui/simulator/sim_react_trading_tab.py` `SimTradingTabReact` | `src/gui/react_trading_tab.py` `TradingTabReact` |
| `src/gui/simulator/sim_trading_tab_surface.py` | `src/gui/main_tabs/trading_tab_surface.py`, the payload builder |
| `src/gui/simulator/sim_exchange_tab_surface.py` | `src/gui/main_tabs/exchange_tab_surface.py`, the payload builder |

Diagram: flowchart, left to right

- variant_surface SIMULATOR -> **SimTradingTabReact**
- SimTradingTabReact -> **sim_trading_tab.js**
- sim_trading_tab.js -> **sim_exchange_tab.js**
- sim_exchange_tab.js -> **sim_bot_status_table.js**
- sim_exchange_tab.js -> **sim_extractor_bot_table.js**
- sim_bot_status_table.js -> **sim_table_cells.js**
- sim_extractor_bot_table.js -> **sim_table_cells.js**
- sim_trading_tab.js -> **sim_indicator_panel.js**
- sim_trading_tab.js -> **replay layer: VWAP over playback**
- sim_trading_tab.js -> **sim_status_log.js**
- sim_trading_tab.js -> **API Interaction Log pane**

### What the page draws

The page is Live's four splitters at Live's sizes, read off Live's own `trading_tab_surface`: the exchange layer stack beside the panel on top, the Activity Log and the API Interaction Log side by side below. Every module carries Live's title and Live's part name, so the page and the Qt fork enumerate the same modules in the same order. The venue page holds the Scrumming Bots table and the Extractor Bots table, both hidden until a row arrives, and the command bar. The panel holds its title, the Bot selector and its privacy dot, the currency rate strip, both indicator tables, both confidence bar graphs, the timeframe-lock line and the staleness banner.

The three ruled positions are the Qt fork's. The corner Live gives `■ Add Crypto Exchange` holds Import Live Fleet, Generate From YTD and Create New Bots, at Live's corner-button width and height, in Live's corner-button chrome. The venue header holds Validation, Back Test and Portfolio Battery in Live's Privacy-Mode sheet, then `+ New Bot` where Live draws it. The data-pool row keeps Live's style and height and holds a blank line.

`src/gui/simulator/sim_trading_tab_surface.py` — the corner buttons

```
WAY_IN_BUTTONS = (
    (sim.IMPORT_LIVE_FLEET_ACTION, sim.IMPORT_LIVE_FLEET_TEXT),
    (sim.GENERATE_FROM_YTD_ACTION, sim.GENERATE_FROM_YTD_TEXT),
    (sim.CREATE_NEW_BOTS_ACTION, sim.CREATE_NEW_BOTS_TEXT),
)
```

The replay layer sits behind the panel in one stack the page draws: the panel slot over a layer holding the VWAP window above the Stone Tablet playback window in one vertical splitter. The flip button sits after the panel title, seated by `sim_indicator_panel.js`, and at the head of the replay layer, seated by `sim_trading_tab.js`, so the way back is never hidden with the panel. A press writes the page's ask on the console line the host reads, and the host redraws the tab with the other layer showing.

`src/gui/simulator/sim_react_trading_tab.py` — the flip

```
    def flip_layer(self) -> str:
        """Swap the page between the panel layer and the replay layer."""
        other = (
            sim.LAYER_PLAYBACK
            if self._state.replay_layer == sim.LAYER_INDICATORS
            else sim.LAYER_INDICATORS
        )
        return self.show_layer(other)
```

The panel module later moved the flip before its title, so both modules draw it first in their header at the panel's top-left corner. Each module's ask carries the pressed button's rect under `flip_rect`, read by the tab module's `flipRect` in the layer stack's coordinates, and `show_layer` emits `sim.layer.flipped` with it on every change of layer.

`src/gui/web/sim_indicator_panel.js` — the header's order

```
  return element(DIV_TAG, headProps, [
    flip,
    element(Title, { key: TITLE_PART, model: model }),
    spacer,
```

### What feeds the page

The host builds one payload per bridge method from state it owns: the tab payload from its own `SimTradingTabState`, the Activity Log from its own `StatusLogModel`, the panel from its own `IndicatorPanelModel`, and each seated venue's three payloads from that venue's own `ExchangeTabModel`, its `SimBotStatusTableModel`, the fork unit 13 made of Live's scrum model (comment 5707202420), and Live's `ExtractorBotTableModel`. Live's module-level models are never read. The page asks each method once on mount and the host answers from those payloads; every push, `show_tab`, `show_votes`, `show_log_call` and `add_exchange_tab`, hands the page a fresh payload and redraws it.

`src/gui/simulator/sim_react_trading_tab.py` — the payloads

```
def models(
    state: tab_surface.SimTradingTabState,
    log: status_log_surface.StatusLogModel,
    panel: indicator_panel_surface.IndicatorPanelModel,
) -> dict:
    """The view model of every bridge method the page's modules ask for."""
    return {
        token_surface.METHOD: token_surface.view_model({}),
        tab_surface.METHOD: state.view_model({}),
        LOG_METHOD: log_payload(log, {"whole": True}),
        PANEL_METHOD: panel_payload(panel),
    }
```

### What the page does not carry

The host holds the same two sources the Qt fork holds, `TabletSource` and `FleetSource`, and no bot manager, no connector and no bus. The news ticker module, which reaches its feeds over the network, is not in the page's roster. The data-pool line's text and its one-second timer, and Privacy Mode, are not forked. The Activity-Log watchdog and the API-log listener were not forked at unit 8; unit 17 forked the watchdog over the sim bot manager and unit 18 the listener over the sim log (comments 5717588355, 5718837037). Every ask the page makes is answered from the payloads the host holds and written to the console for the host to read. At unit 8 the host answered the flip and held every other press, so the way-in buttons, the mode buttons, `+ New Bot`, the Fire buttons and the command bar changed nothing; units 12 to 31 wired each press (comments 5708384371, 5707202420, 5714936418, 5716410892, 5717588355, 5728708856, 5733262015), and the section "What the tab holds today" describes them.

## The header strip shows on Sim

The window's header strip, the five columns and the five counter cards above the tab row, stays on screen while the Sim tab is in front. `ISOLATED_TABS` names the Paper tab alone, so the strip hides on Paper and on nothing else.

`src/gui/main_tabs/main_window_surface.py` — the two tuples

```
ISOLATED_TABS = (PAPER_TAB,)

#: The tabs the header strip reads the Simulator's fleet on, not the live one.
SIM_FED_TABS = (SIM_TAB,)
```

While Sim is in front the ten fields carry the Simulator's figures, read from the Sim tab's fleet source; on every other tab they carry the live fleet's, as before. The window decides on every dashboard tick and again the moment the operator changes tab, so a switch never leaves the other fleet's figures on the strip. The live fleet's numbers do not appear while Sim is in front.

`src/gui/main_window.py` — `_refresh_header_strip`

```
if self._header_strip_reads_sim():
    sim_tab = self._simulator_tab
    self._write_header_strip(
        sim_tab.fleet_source().aggregate(), sim_tab.exchange_count()
    )
    return
if live_stats is None:
    if not self._bot_manager:
        return
    live_stats = self._bot_manager.get_aggregate_stats()
self._write_header_strip(live_stats, len(self._exchange_tabs))
```

Each field keeps the definition the Trading tab page gives it, applied to the sim fleet. SPENDABLE is the sim fleet's wallet cash and LOCKED the value its positions hold. REALISED is the sim fleet's fills matched first-in first-out, one figure per bot, summed. MATURE is the profit on sim positions past two hundred per cent of their cost. EXCH counts the Sim tab's own venue sub-tabs. The five cards are the sim fleet's sold and bought dollars, its trades, its running bots and its errors. The fleet source answers them through one call, in the keys the live

aggregate uses.

`src/simulator/fleet_source.py` — `FleetSource.aggregate`

```
def aggregate(self) -> dict:
    """The header strip's figures for the fleet the Simulator holds.

    No bot is loaded into the Simulator, so the answer is
    ``EMPTY_AGGREGATE``.
    """
    return dict(EMPTY_AGGREGATE)
```

At unit 9 no bot was loaded into the Simulator, so the strip read on Sim as it reads on Live with no bots: the four money columns an em dash, EXCH the count of seated venues, the five cards zero. Unit 21 made `aggregate` answer Live's arithmetic over the held records (comment 5721499839), so the figures arrive with the way-ins that load a fleet and the runs that trade it.

Diagram: flowchart, left to right

- dashboard tick, 2000 ms -> **tab in front**
- tab change -> **tab in front**
- pick -- Sim -> **sim.aggregate]**
- pick -- any other -> **live]**
- SimTradingTab.fleet_source( -> **the ten cells**
- BotManager.get_aggregate_stats( -> **the ten cells**

## The venue stack seats the fleet's exchanges

The Sim tab seats one venue sub-tab per exchange the sim fleet names, in both builds. At unit 10 the fleet was the saved bot record the tab's fleet source read, and the exchange set the distinct exchange ids of the stored bots; since unit 21 the fleet is the held records and the Simulator starts empty (comment 5721499839). Each host seats its venues when it is built and again whenever its `fleet_changed` signal fires, which every way-in does. A sub-tab is captioned as Live captions an exchange saved with no name, the capitalised id. Live's venue set is the live configuration; the Simulator never reads it.

`src/gui/simulator/sim_trading_tab.py` — the seating

```
    def _sync_exchange_tabs(self) -> None:
        """Seat a sub-tab for each exchange the fleet names and drop the rest.

        The caption is ``exchange_display_name`` over the id, as Live captions
        an exchange saved with no name.
        """
        wanted = [str(eid) for eid in self._fleet_source.exchanges() if eid]
        for eid in wanted:
            self.add_exchange_tab(eid, exchange_display_name({"exchange_id": eid}))
        self._drop_unlisted_exchange_tabs(wanted)
```

When the fleet names an exchange the bar does not hold, that venue page is seated beside the others. When the bar holds an exchange the fleet no longer names, its sub-tab is taken off, and the Get Started card comes back once a layer's bar is empty. EXCH on the header strip counts the sub-tabs on the crypto layer, as it counts Live's. A seated venue with no bots draws the forked venue page: the mode buttons, `+ New Bot`, the empty data-pool row, the command bar, and both tables hidden until a row arrives.

Diagram: flowchart, left to right

- bot_state.json, through Import Live Fleet -> **FleetSource.exchanges**
- tab build -> **_sync_exchange_tabs**
- fleet_changed -> **_sync_exchange_tabs**
- FleetSource.exchanges -> **_sync_exchange_tabs**
- sync -- not on the bar -> **add_exchange_tab**
- sync -- no longer named -> **_drop_unlisted_exchange_tabs**
- add_exchange_tab -> **the venue sub-tab bar**
- _drop_unlisted_exchange_tabs -> **the venue sub-tab bar**
- the venue sub-tab bar -> **EXCH**

### The Get Started card asks for a first fleet

With no exchange seated the layer shows the Get Started card. Its frame, its geometry and the order of its parts are Live's; its contents are the Simulator's. The heading reads `No Crypto Fleet Loaded`, the hint reads `Load a Crypto fleet to begin a run`, and the button position holds Import Live Fleet, Generate From YTD and Create New Bots, one under the other, each at Live's card-button size and in the layer's accent. At unit 10 they were wired to nothing; units 21, 22 and 23 then wired the corner button and the card button of each action together (comments 5721499839, 5722113817, 5723035277).

`src/gui/simulator/sim_trading_tab_surface.py` — the card's texts

```
def placeholder_title_text(label: Any) -> str:
    """The Get Started card's heading for one layer: the first step is a fleet."""
    return f"No {label} Fleet Loaded"

def placeholder_hint_text(label: Any) -> str:
    """The line under the Get Started card's buttons for one layer."""
    return f"Load a {label} fleet to begin a run"
```

The Qt card and the React card read those two functions, so the two builds cannot title the card differently. The venue stack is the first module of the tab to read one of its two sources; the tables, the panel and the two spools still draw empty.

## The Scrumming Bots table draws the sim fleet

The Scrumming Bots table on each seated venue draws one row per sim bot on that exchange, in both builds. The rows come from the tab's fleet source: each held record, imported, generated or created (unit 21, comment 5721499839), is one read-only record, and the record answers the same status keys a live bot answers, so the forked table draws it with Live's own cell code. The table stays hidden until a row arrives and hides again when the last row leaves, as Live's does.

`src/gui/simulator/sim_trading_tab.py` — the feed

```
    def refresh_bots(self) -> int:
        """Hand every seated venue its rows from ``FleetSource.statuses``.

        Answers how many rows were handed out. A venue whose list is empty
        hides its tables, as Live's does.
        """
        handed = 0
        for store in (self._crypto_exchange_tabs, self._stock_exchange_tabs):
            for eid, tab in list(store.items()):
                statuses = self._fleet_source.statuses(eid)
                tab.update_bots(statuses)
                handed += len(statuses)
        return handed
```

Both hosts call it after the venues are seated, at build and on every `fleet_changed`. The Simulator runs no refresh timer: a sim figure moves only when a load or a run moves it, and the signal that fires then is what redraws the rows. Live's rows follow the bot manager every two seconds; the Simulator's follow the fleet it holds.

Diagram: flowchart, left to right

- bot_state.json record, through Import Live Fleet -> **SimBot**
- SimBot -> **row_status**
- row_status -> **FleetSource.statuses**
- FleetSource.statuses -> **refresh_bots**
- refresh_bots -> **SimBotStatusTable.update_bots**
- refresh_bots -> **SimVenue.update_bots**
- SimVenue.update_bots -> **sim_bot_status_table.js**

### What one record carries

The stored record is read in four parts. The config gives the ids, the symbol, the mode and the gate fields. The stats give the price, the position value, the trade count, the scrummed and folded dollars, the realised profit and the error count. The saved scrumming state gives the grown target the engine re-zeroes to, the quote rate, the phase and the lots. The saved state gives the state the Bot ID cell is coloured by. Holdings are the units summed across the lots, which is how the live bot sets its own holdings.

| the cell | reads |
|---|---|
| Bot ID, its colour | the record's id and its saved state |

| the cell | reads |
|---|---|
| Symbol | the config symbol and exchange |
| Current Position Value | holdings, the record's price, the quote rate |
| Trades | the stats trade count |
| Target | the grown target, or the config target |
| Target A15, Target A14 | the target over the fleet's own A15 and A14 rows |
| Ammo | the position value and the target |
| Fire | the saved phase; the armed action and the gate state read as a live bot's do before its first tick |

The runtime values a record does not hold, the armed action, the ceiling ratio and the gate state, read as they do on a live bot before its first tick.

### The row's own price, and the fleet's own rates

A sim row is priced from its own record and never from the live data pool. The price age is zero, the sim's own reading for this tick, so the Position Value cell and the Ammo cell price from the same figure, as the Trading tab page requires of them. The Target A15 and Target A14 cells denominate through the fleet's own A15 and A14 rows on that exchange, not through the live currency monitor, and carry no drift suffix because the Simulator has no 24-hour figure to draw one from; with no such row the cell reads `pending`.

`src/gui/simulator/sim_bot_status_table_surface.py` — the price

```
    def _price_reading(self, status, stats) -> tuple:
        """The row's own ``current_price`` at ``SIM_PRICE_AGE_S``, and its ``quote_to_usd``."""
        price = float(stats.get("current_price", live.NO_PRICE))
        quote_rate = float(
            status.get("quote_to_usd", live.DEFAULT_QUOTE_TO_USD)
            or live.DEFAULT_QUOTE_TO_USD
        )
        return price, SIM_PRICE_AGE_S, quote_rate
```

The Qt fork reads the same constant and the same denomination helper, so the two builds price a row identically.

### Fire and Detail

Fire is present on every row with Live's styling and tooltips. A press asks the fleet source to fire, and the fleet source refuses every send by name, so the Activity Log carries Live's own failure line, `Fire on <id> failed`, with the refusal's words. No venue and no live bot is reached. The sim's own scrum and fold arithmetic is a later unit; until it lands every Fire on the Simulator is refused this way.

Detail is present on every row. A press opens the Simulator's bot detail window: the frame of Live's Bot Settings window, with the title, the pair and mode header, the state badge, Prev and Next over the venue's sim fleet, the Status tab over the row's figures, and Close. It edits nothing. At unit 13 the six tabs that read a live bot's runtime state, and the React-drawn window, were not forked; unit 13a forked them (comment 5712057220).

`src/gui/simulator/sim_trading_tab.py` — the two presses

```
    def _on_bot_fire(self, bot_id: str) -> None:
        """Manual Fire on a sim bot: ask ``FleetSource`` to ``fire`` and log the
        refusal to the Activity Log, as the window logs a failed Fire."""
        try:
            self._fleet_source.fire(bot_id)
        except SendRefused as exc:
            self._status_log.log(f"Fire on {bot_id[:8]} failed: {exc}", "error")
```

Under React the page's Fire and Detail asks reach the host through the console line, the host applies the ask to that venue's own table model, and the same two handlers run.

### Until the way-ins landed

At unit 13 the fleet source read the saved bot record as it lay on disk, so a build with a saved fleet drew that fleet's rows at start. Unit 21 gated both the venues and the rows behind Import Live Fleet, so the tab starts empty (comment 5721499839).

## New Bot opens the Simulator's wizard

`+ New Bot` on a seated Sim venue opens the Simulator's Create Auto Trader wizard, a fork of Live's under the Simulator's names, in both builds. It carries Live's five pages in Live's order, Trading Mode, then Select Asset Pair or Extractor Pool, then Trading Parameters, then Phantom Bots, with the same fields, the same ranges, the same walk buttons and the same look, and it opens on the operator's stored defaults as Live's does. Finish creates one sim bot and its row appears in the Scrumming Bots table. Cancel creates nothing.

`src/gui/simulator/sim_trading_tab.py` — the press reaches the fork

```
        tab = SimExchangeTab(
            exchange_id,
            display_name,
            on_new_bot=self._create_bot,
            on_bot_clicked=self._on_bot_detail,
            on_bot_fire=self._on_bot_fire,
            status_log=self._status_log,
        )
```

Diagram: flowchart, left to right

- + New Bot -> **_create_bot**
- _create_bot -> **SimBotCreationWizard or SimBotWizardReactDialog**
- SimBotCreationWizard or SimBotWizardReactDialog -> **get_bot_config** *(Finish)*
- get_bot_config -> **FleetSource.create**
- FleetSource.create -> **fleet_changed**
- fleet_changed -> **refresh_bots**
- SimBotCreationWizard or SimBotWizardReactDialog -> **nothing changes** *(Cancel)*

### What the Simulator's wizard reads

The venue list is the seated sub-tabs. The stored defaults come through the same settings reader Live's window holds, so the target balance, the visibility, the aggressive box, the phantom enable and the lock candles open on the operator's own figures; nothing is written. The Target Asset list is the assets the Simulator holds a tablet for on the picked exchange. A tablet is filed by asset and exchange and names no quote, so its asset is offered under every base currency, with no volume figure and no volume sort. Live's wizard asks the venue for its markets; the Simulator's asks the tablets.

`src/gui/simulator/sim_bot_wizard_surface.py` — the market table

```
    for entry in tablet_source.entries():
        exchange_id = str(entry.exchange_id)
        asset = str(entry.asset).upper()
        key = (exchange_id, asset)
        if key in seen or not asset:
            continue
        seen.add(key)
        rows = out.setdefault(exchange_id, [])
        for base in live.BASE_CURRENCIES:
```

### What Finish creates

The wizard's config is shaped by the same call Live shapes it with, so a mode-foreign key or a bad shape is refused with Live's own Activity Log line. The shaped config takes the stored-record shape the live process writes and is read through the same record reader the imported fleet uses, so a created bot holds exactly the fields a stored bot's record gives: the ids, the symbol, the target, the timeframe, the interval, the fee and the gate fields. Its id is eight characters, drawn as a live bot draws its own. Its state is idle, so its Bot ID cell is grey. Its price, holdings and trades are zero, as on a live bot before its first tick. An Extractor is refused when no single Scrumming Bot on the venue holds its pool base, with Live's own message.

`src/simulator/fleet_source.py` — the add path

```
    def create(self, config: dict) -> SimBot:
        record = wizard_record(config)
        bot_id = str(uuid.uuid4())[:BOT_ID_LENGTH]
        bot = _sim_bot_from_record(bot_id, record, origin=NEW_ORIGIN)
        if bot is None:
            raise ValueError("the wizard config names no symbol")
        self._created.append(bot)
        return bot
```

The fleet source then fires `fleet_changed`, the venues re-seat, the tables redraw, and the Activity Log carries `Bot <id> created: <symbol> (scrumming) - IDLE`.

### What is not forked

Live's pre-flight symbol check contacts the venue; the Simulator offers only the assets it holds a tablet for. The phantom page's API-load warning reads Live's venue call rate; the Simulator makes no venue call, so the page answers True, as Live's React wizard does with no load reading.

### A created bot was not persisted at unit 12

At unit 12 the Simulator kept no fleet file of its own, and a created bot lived in the fleet source until the tab closed; unit 12a gave it the sim fleet file (comment 5710961369). Nothing writes `bot_state.json`.

### The React host opens the wizard one turn later

Under React the venue page's press reaches the host through the console line. A web page opened inside another page's console callback never finishes loading, so the host opens the wizard on the next event-loop turn, and its page loads.

`src/gui/simulator/sim_react_trading_tab.py` — the deferred open

```
            QTimer.singleShot(
                0, lambda: self._open_bot_wizard(exchange_id, defaults_override)
            )
```

## The Extractor Bots table draws the sim extractors

The Extractor Bots table on a seated Sim venue is the forked `SimExtractorBotTable` under Qt and `sim_extractor_bot_table.js` under React, fed from the same `FleetSource.statuses` list the Scrumming Bots table reads, kept to the records whose mode is extractor. It carries Live's eight columns, Bot ID, Symbol, Mode, Trades, Pool, Liquid, Fire and the blank Detail column, with Live's tooltips and Live's two buttons. It is hidden with its label until the venue's fleet holds an extractor, shown when one arrives, and hidden again when the last leaves, while the Scrumming Bots table keeps its own rows.

`src/simulator/fleet_source.py` — the keys an extractor row reads

```
def _extractor_status(bot: SimBot) -> dict:
    """The keys ``ExtractorBot.get_status`` adds over the base status, which
    the Extractor Bots table reads: the base currency, the three pool figures,
    the two position counts and ``extractor_pool_color`` over them."""
    return {
        "base_currency": bot.base_currency,
        "chunk_size_usd": bot.chunk_size_usd,
        "chunk_size_base": bot.chunk_size_base,
        "chunk_free_base": bot.chunk_free_base,
        "n_positions_open": bot.n_positions_open,
        "n_positions_drawdown": bot.n_positions_drawdown,
        "pool_color": extractor_pool_color(
            bot.n_positions_open, bot.n_positions_drawdown
        ),
    }
```

Diagram: flowchart, left to right

- bot_state.json record, through Import Live Fleet -> **SimBot with the pool figures**
- SimBot with the pool figures -> **row_status, the extractor keys added**
- row_status, the extractor keys added -> **FleetSource.statuses**
- FleetSource.statuses -> **refresh_bots**

- refresh_bots -> **SimExtractorBotTable.update_bots**
- refresh_bots -> **ExtractorBotTableModel through SimVenue**
- ExtractorBotTableModel through SimVenue -> **sim_extractor_bot_table.js**

### What an extractor record carries

A stored extractor record is read in the same four parts as a scrumming one, plus the extractor block the live bot saves: the dollar size of its pool, the pool in base units, the base units not deployed to any position, and the list of open positions with each one's state. A record the wizard just created holds no such block, so it reads as a live extractor reads before its first rate arrives: the base size and the free base both equal to the dollar size, and no position.

| the cell | reads |
|---|---|
| Bot ID | the record's id |
| Symbol, its coin icon | the base currency the extractor accumulates |
| Mode, its colour and tooltip | the saved state |
| Trades | the stats trade count |
| Pool | the pool's dollar size |
| Liquid, its tooltip | the free base priced at the pool's own dollars per base unit, with the position counts and the base name |
| Liquid's colour | green with no position open, red with one in drawdown, yellow otherwise |
| Fire | present and disabled, as on Live |
| Detail | present |

The Pool and Liquid figures are composed by Live's own row code in both builds; the Simulator adds no arithmetic of its own. The pool colour is the live extractor's rule, written once as pure code over the two position counts.

`src/simulator/fleet_source.py` — the pool colour

```
def extractor_pool_color(n_positions_open: int, n_positions_drawdown: int) -> str:
    """The Liquid cell's colour name for one extractor, the rule of
    ``ExtractorBot.pool_color``: ``POOL_GREEN`` with no position open,
    ``POOL_RED`` with one in drawdown, ``POOL_YELLOW`` otherwise."""
    if n_positions_open <= 0:
        return POOL_GREEN
    if n_positions_drawdown > 0:
        return POOL_RED
    return POOL_YELLOW
```

### The parent

An Extractor works against a parent base-currency Scrumming Bot. Live's row draws no parent id; what it shows of the parent is the base currency the extractor accumulates, in the Symbol cell and in the Liquid tooltip. The sim row shows the same, from the record's base currency, which is the base the parent scrumming bot on that venue holds.

### Detail and Fire on an extractor row

Detail is present on every extractor row in both builds. A press opens the Simulator's bot detail window over that extractor, with its symbol, its mode, its state and its trade count on the Status tab and, since unit 13a, the Settings and Positions Held tabs beside it (comment 5712057220), and selects the row, as a press on the Scrumming Bots table does. Under React the page's ask reaches the host through the console line, the host applies it to that venue's own extractor table model, and the same handler runs.

Fire is present and disabled on every extractor row, as it is on Live's, because Manual Fire is per position for an Extractor. A press reaches nothing: no name is asked of the fleet source and no line is written.

`src/gui/simulator/sim_react_trading_tab.py` — the extractor table's asks

```
        if method == EXTRACTOR_METHOD:
            if (
                params.get(extractor_surface.ACTION_PARAM)
                not in EXTRACTOR_PRESS_ACTIONS
            ):
                return False
            extractor_surface.drive(self.extractor, params)
            return True
```

## The Simulator keeps its own fleet file

A bot created through the Simulator's wizard survives a restart, in both builds. The tab's fleet source writes one file of its own, `sim_fleet.json`, under the `sim` bucket of the log root; the parity reports of unit 25 land under the reports bucket instead, and the four log files of unit 31a beside the fleet file (comments 5725784001, 5734233752). Every `fleet_changed` writes it first, before the venues re-seat and the tables redraw, and the tab build reads it once. The write goes through the same helper the live process writes `bot_state.json` with, so a crash mid-write leaves the previous file whole. The section "A created bot is not persisted" above describes the tab before this file existed; its last sentence, that nothing writes `bot_state.json`, stays true.

`src/core/log_paths.py` — the bucket

```
def get_sim_dir() -> Path:
    """``sim/`` bucket — every file the Simulator writes.

    ``src.simulator.fleet_source.FleetSource`` keeps the sim fleet file
    here; ``~/.acervator/bot_state.json`` is never written from the
    Simulator.
    """
    p = _LOG_ROOT / "sim"
    p.mkdir(parents=True, exist_ok=True)
    return p
```

`src/gui/simulator/sim_trading_tab.py` — the write hangs on the signal

```
        self.fleet_changed.connect(self._fleet_source.save)
        self.fleet_changed.connect(self._sync_exchange_tabs)
        self._sync_exchange_tabs()
```

Diagram: flowchart, left to right

- Finish in the wizard -> **FleetSource.create**
- FleetSource.create -> **fleet_changed**
- fleet_changed -> **FleetSource.save**
- FleetSource.save -> **sim_fleet.json under the sim bucket**
- fleet_changed -> **refresh_bots**
- the next launch -> **tab build**
- tab build -> **FleetSource reads sim_fleet.json**
- FleetSource reads sim_fleet.json -> **FleetSource.bots**
- FleetSource.bots -> **refresh_bots**

### What the file holds

The file carries `saved_at`, `saved_at_human`, `bot_count` and `bots`, the keys the live state file carries less the smart-wire lists the Simulator has none of. Each entry under `bots` is the per-bot record the live process writes, `config`, `stats`, `scrumming_state`, `state_when_saved` and `bot_id`, so the one record reader that maps a stored live bot to a row maps a stored sim bot the same way. At unit 12a the file held the wizard's bots and nothing of the live fleet, with the live read beside it; since unit 21 it holds every held record, imported ones included, and the live read runs only through Import Live Fleet (comment 5721499839).

`src/simulator/fleet_source.py` — the write

```
    def save(self) -> Optional[Path]:
        payload = {
            "saved_at": time.time(),
            "saved_at_human": datetime.now().strftime(SAVED_AT_HUMAN_FORMAT),
            "bot_count": len(self._records),
            "bots": dict(self._records),
        }
        path = self.sim_path()
        try:
            atomic_write_json(path, payload, indent=2, default=str)
        except (OSError, TypeError, ValueError) as exc:
            logger.error("sim fleet save failed: %s: %s", path, exc)
            return None
```

Since one fleet per mode landed, the file holds all three fleets. The top carries `saved_at`, `saved_at_human`, `bot_count` summed over the three, and `fleets`, one entry per mode; each entry carries the four keys the block above wrote for the one fleet, so a mode's entry is a file of the old shape. Every fleet change writes all three, so an import under Validation, a bot created under Back Test and a Battery run's generated fleet land in one file and survive a restart under their own modes.

`src/simulator/fleet_source.py` — the write, one entry per mode

```
        payload = {
            "saved_at": saved_at,
            "saved_at_human": saved_at_human,
            "bot_count": sum(len(records) for records in self._fleets.values()),
            FLEETS_KEY: {
                mode: {
                    "saved_at": saved_at,
                    "saved_at_human": saved_at_human,
                    "bot_count": len(records),
                    "bots": dict(records),
                }
                for mode, records in self._fleets.items()
            },
        }
```

### Absent, empty, malformed

An absent file, or one holding no bytes, gives no sim bots and no log line. A file that is not a JSON object holding a `bots` object gives no sim bots and one warning line naming the file, and the tab builds. A record that names no symbol is kept in the file, is not drawn, and is named in one warning line at build, as the live state file carries forward a record it did not load.

`src/simulator/fleet_source.py` — the read

```
    try:
        loaded = json.loads(text)
    except json.JSONDecodeError as exc:
        logger.warning("sim fleet file %s malformed: %s", path, exc)
        return {}
    stored = loaded.get("bots") if isinstance(loaded, dict) else None
    if not isinstance(stored, dict):
        logger.warning("sim fleet file %s malformed: no bots object", path)
        return {}
```

Since one fleet per mode landed, the read answers three maps. A file holding `fleets` reads each mode's entry through the reading above, one warning per mode for a record that names no symbol; a mode name under `fleets` that is not one of the three is not read and is named in one warning line. A file in the old shape, one `bots` map at the top and no `fleets`, is read as the Validation fleet with the other two empty, and one line says so, so the fleet the operator held before the change is drawn at the next launch under Validation, the mode the tab opens in. The first fleet change after that launch writes the file in the new shape.

`src/simulator/fleet_source.py` — the old shape and the unknown mode

```
    stored_fleets = loaded.get(FLEETS_KEY)
    if isinstance(stored_fleets, dict):
        unknown = [str(name) for name in stored_fleets if name not in MODES]
        if unknown:
            logger.warning(
                "sim fleet file %s names %d mode(s) outside %s and does not "
                "read them: %s",
                path,
                len(unknown),
                MODES,
                ", ".join(unknown),
            )
```

```
    fleets[MODE_VALIDATION] = _records_of(stored, path, MODE_VALIDATION)
    logger.info(
        "sim fleet file %s holds one fleet; read as the %s fleet",
        path,
        MODE_VALIDATION,
    )
```

Driven in the real window in both builds with a scratch home: a bot created through the wizard was read off the file with the typed symbol, target, timeframe and interval, and the second process drew its row with those values; `bot_state.json` hashed the same before and after every step, and a write planted into its path from the sim side moved the hash, so the comparison can report. The file removed drew the live row alone with no warning; the file overwritten with text that is not JSON drew the live row alone with one warning naming it; a file holding one readable record and one that names no symbol drew the readable one and named the other.

## Detail opens the Simulator's Bot Settings window with all seven tabs

Detail on a Simulator row now opens Live's Bot Settings window, forked, with every tab Live gives the bot's mode, in both builds. A scrumming bot gets Status, Settings, Fold Tranches, Stack Tranches, Bot Swarm, Market Inspector and Phantom Bots; an extractor gets Status, Settings and Positions Held. The window carries Live's title, Live's header, Prev and Next over the venue's sim fleet, Apply Changes and Close. The section "Fire and Detail" above describes the window before the six tabs were forked; the sentence that they were not forked yet no longer holds, and the sentence that the window edits nothing stays true.

Each tab is fed from the stored record, read as the bot the window reads. `SimBotView` holds one `SimBot` and its raw record and answers the attribute names Live's tabs read off a live bot: the config rebuilt as a restart rebuilds it, the stats, the tranches, the counters and the parked credits from `scrumming_state`, the phantom flag, the phantom timeframes and the lock count from the record's own keys, and the positions and the pool figures from `extractor_state`. Every name Live writes a bot through is refused.

`src/simulator/sim_bot_view.py` — the refusal

```
    def __getattr__(self, name: str) -> Any:
        """Refuse every send in ``SEND_NAMES`` and every runtime name not held."""
        if name.startswith("__"):
            raise AttributeError(name)
        if name in SEND_NAMES:
            return _refusal(name)
        reason = (
            f"SimBotView holds no {name!r}: the stored record carries no such "
            "reading. The Simulator receives and asks; it sends nothing."
        )
        raise SendRefused(reason)
```

Diagram: flowchart, left to right

- Detail on a sim row -> **_on_bot_detail**
- _on_bot_detail -> **SimBotView over the SimBot and its record**
- _on_bot_detail -> **surface_class SIM_BOT_DETAIL**
- surface_class SIM_BOT_DETAIL -> **SimBotDetailDialog, seven forked mixins**
- surface_class SIM_BOT_DETAIL -> **SimBotDetailReactDialog, sim_bot_live_settings.js and eight forked tab modules**
- SimBotDetailDialog, seven forked mixins -> **every press asks the view and is refused**
- SimBotDetailReactDialog, sim_bot_live_settings.js and eight forked tab modules -> **every press asks the view and is refused**

### What each tab reads

| tab | read from the record | drawn when the record holds nothing |
|---|---|---|
| Status | `stats` | the row's figures, as the table draws them |
| Settings | `config`, the live target, the anchor, the surplus, the consumed budget | Target A15 and Target A14 from the fleet's own A15 and A14 rows; the budget row reads an unreadable dash |
| Fold Tranches | `fold_tranches`, the parked credits, the ledger, the four counters | no Extractor Tranche row, Live's own answer with no bot manager |
| Stack Tranches | `stack_tranches`, the two counters | — |
| Bot Swarm | nothing named a wire manager | Live's own line, Bot Swarm not active for this bot |
| Market Inspector | nothing; a scan is not a record | Live's own no-scan screen; the shared analyzer is never asked |
| Phantom Bots | `phantoms_enabled`, `phantom_timeframes`, `lock_candle_count` | the runtime rows as a bot before its first tick |
| Positions Held | `extractor_state.positions`, the four chunk figures | — |

The budget row is the one figure the window does not draw. Its arithmetic is a live bot's own, and the unit that shares the trading arithmetic as pure code carries it; until then both builds read – `(unreadable)`, the reading Live gives a cap it cannot read.

### Every press is refused

Apply Changes, Reset All Breakers, SELF-DESTRUCT, the three Clear buttons on Fold Tranches, Fire on a fold row, the two Clear buttons on Stack Tranches and Fire on a position are all present, drawn as Live draws them. In the Qt build a press runs Live's own confirmation, then asks the view, and the view's refusal lands in Live's own failure box: `Nothing was cleared — the call failed`, `Manual Fire refused`, `Schedule failed`. In the React build the press reaches the window through the page's console line and the refusal is drawn on the pending-change line. Apply Changes records each edited field as Live does, asks the view for every one on the press, and the line reads `Refused N change(s) — the Simulator sends nothing`. No file is written: `bot_state.json` and the sim fleet file hash the same before and after every press.

`src/gui/simulator/sim_bot_detail.py` — Apply Changes

```
    def _apply_changes(self) -> None:
        """Ask the view for every pending field; each is refused and logged."""
        if not self._changes:
            return
        refused = []
        for field, value in list(self._changes.items()):
            try:
                self._route_change(field, value)
            except SendRefused as exc:
                logger.warning(
                    surface.ROUTE_REFUSED_LOG, self._bot.bot_id[:8], field, exc
                )
                refused.append(
                    surface.APPLIED_REFUSED_FORMAT.format(
                        field=field, value=value, reason=exc
                    )
                )
```

### Both builds, resolved as Live resolves

The hosts open the window through the variant seam, beside Live's own pair, so the Qt host opens the Qt fork and the React host opens the React fork. The React fork is Live's React window forked under the Simulator's names: its own page module, `sim_bot_live_settings.js`, eight forked tab modules and one sim surface per tab, each answering its own method name. The React host opens the window one event-loop turn after the page's console line that carried the press, as the wizard is opened, because a web view opened inside another page's console callback never finishes loading.

`src/gui/variant_surface.py` — the pair

```
register(BOT_LIVE_SETTINGS, _qt_bot_live_settings, _react_bot_live_settings)
register(SIM_BOT_DETAIL, _qt_sim_bot_detail, _react_sim_bot_detail)
```

Driven in the real window in both builds with a scratch home and every socket but loopback refused: Detail on the scrumming row read seven tabs off the widget tree and off the page, titled and ordered as Live's window over the same record shape; Detail on the extractor row read three. The Fold Tranches tab drew three rows from a record holding three tranches and two from a record holding two. Next opened the next bot's window on

the same tab. Every press above reached the refusal, and `bot_state.json` hashed the same after each one.

## The Sim venue pane is Live's width

The Sim tab's top splitter now settles at the same pair as Live's at every width the window can take. Before, the Qt Sim's venue pane was 48 px narrower than Live's at the window's floor and 66 px narrower at 1920 px, in the built program with its theme. The cause was the flip button seated after the panel title: under the theme it asks 134 px, so the forked panel asked 575 px where Live's panel asks 439, and a splitter given a size below a pane's minimum raises that size and keeps the raised figure as the pane's share on every later resize. The Sim divided 600:575 where Live divides 600:500. The replay layer behind the panel asks 8 px and never decided the pair.

The flip button now reports a minimum width of 0 to the header row, and its size policy lets the row shrink it. The panel asks 441 px, under the 500 the splitter is given, so the Sim keeps 600:500 as Live does. At the floor the header has 185 px of room and the button draws at its full width; dragged narrower than its content, the button gives up width before the pane does, as the React page's flex header already does.

`src/gui/simulator/sim_trading_tab.py` — the flip button

```
class FlipButton(QPushButton):
    """A ``QPushButton`` named ``FLIP_BUTTON_NAME`` whose layout may shrink it to width 0."""

    def __init__(self, text: str, parent: Optional[QWidget] = None) -> None:
        super().__init__(text, parent)
        self.setObjectName(FLIP_BUTTON_NAME)
        self.setAccessibleName(FLIP_BUTTON_NAME)
        self.setSizePolicy(QSizePolicy.Preferred, self.sizePolicy().verticalPolicy())

    def minimumSizeHint(self) -> QSize:  # noqa: N802
        """The base hint's height over a width of 0."""
        return QSize(0, super().minimumSizeHint().height())
```

Read off one window holding both tabs, with the Windows fonts loaded and the theme applied, before and after:

```
window        Live          Sim before    Sim after
1400 x 900    [751, 626]    [703, 674]    [751, 626]
1550 x 974    [833, 694]    [780, 747]    [833, 694]
1856 x 1076   [1000, 833]   [936, 897]    [1000, 833]
1920 x 1080   [1035, 862]   [969, 928]    [1035, 862]
```

The window's floor is its own minimum size, 1400 by 900; a smaller size asked of it settles there. The React page divides its top splitter by the same two figures as flex weights with no floor of its own, so its pair read equal to Live's before this change and after: `[750, 625]` at 1400 and `[999, 832]` at 1856. After the flip, at the settled width, the VWAP window and the Stone Tablet playback window each draw 626 px wide and 248 px high in the Qt build and 625 by 264 in the React build, both inside the pane; a 900 px minimum planted on the replay layer at runtime moves the pair in both builds, and removing it restores the pair.

## The command bar acts on the sim fleet

Pause, Stop, Restart and Delete on the Simulator's venue page each change one sim bot's state, and the table reads it back. Start did the same at unit 15; since unit 31 it starts the run in Validation and Back Test mode and moves every scrumming row on the page (comment 5733262015). The bar itself is Live's: it takes its bot from the table you clicked last, tries the other table with no selection there, and with none anywhere says `Select a bot first.` and does nothing, in both builds. The press then reaches the Sim host's `_on_bot_command`, the window's handler forked, which asks the Simulator's bot manager instead of the live one.

`SimBotManager` in `src/simulator/sim_bot_manager.py` is the fork of the parts of `BotManager` that hold and move bots. It holds one `FleetSource` and no venue, no connector, no event bus and no coroutine. Its registry is the sim fleet file's records: `get_bot` answers a held record's `SimBot` and None for an unknown id, so a command on such an id logs Live's `Bot <id> not found.` line and moves nothing; since unit 21 an imported row is a held record and no row is read from `bot_state.json` directly (comment 5721499839). Each verb moves the record's `state_when_saved` under the live bot's own rule for that verb.

`src/simulator/sim_bot_manager.py` — the verbs

```
def start(self, bot_id: str) -> str:
    bot = self._require(bot_id)
    if bot.state in ALREADY_RUNNING_STATES:
        logger.warning("Bot %s already running", bot_id)
        return bot.state
    return self._fleet.set_state(bot_id, BotState.RUNNING.value, clear_error=True)

def pause(self, bot_id: str) -> str:
    self._require(bot_id)
    return self._fleet.set_state(bot_id, BotState.PAUSED.value)

def stop(self, bot_id: str) -> str:
    self._require(bot_id)
    return self._fleet.set_state(bot_id, BotState.STOPPED.value)

def restart(self, bot_id: str) -> str:
    self.stop(bot_id)
    return self.start(bot_id)

def unregister(self, bot_id: str) -> bool:
    return self._fleet.remove(bot_id)
```

The rule per command, read off `BotContainer`, and the Activity Log lines, which are Live's with the venue step absent:

```
Start     running or starting: unchanged, one logger warning; any other
          state: last_error cleared, running          ✓ Bot <id> RUNNING.
Pause     any state: paused                            Pausing bot <id>... / Bot <id> paused.
Stop      any state: stopped                           Stopping bot <id>... / Bot <id> stopped.
Restart   stopped, then running                        ✓ Bot <id> restarted.
Delete    Live's box, "Delete bot <id>? This cannot be undone."; Yes removes the
          record; No changes nothing                   Bot <id> deleted.
```

Start lands on `running` and not `starting`: Live's `starting` lasts until the bot's loop task takes its first step, and the Simulator has no task. Pause has no state guard, as the live bot's `pause` has none. Each success writes the window notification's line, `[notification] Bot <id> RUNNING | success`, through `notification_line` in `sim_trading_tab_surface.py`, and plays Live's state-change sound; a failed verb logs Live's `Failed to <verb> bot <id>` line and plays Live's error sound. The lines Live prints for connecting to the venue, the REAL MONEY warning among them, are absent, because they read a connect result and the Simulator makes no connect.

`FleetSource` gains three names for the held records: `sim_bot_for`, `set_state` and `remove`. They read and move the records the sim fleet file holds and nothing else; `save` stays the one writer, and every other name still raises `SendRefused`.

After every state move, and after a confirmed Delete, the host fires `fleet_changed`: `FleetSource.save` writes the sim fleet file with the moved state, or without the deleted record, and `refresh_bots` hands each venue its rows again, so the Bot ID cell's colour and tooltip in the Scrumming Bots table, and the Mode cell's in the Extractor Bots table, read the new state. `bot_state.json` is not written.

In the React build the venue page's command press is the `command` ask, which the host now answers beside `+ New Bot`; the ask reaches Live's own `ExchangeTabModel.cmd`, whose `Select a bot first.` line reaches the page through the host. The row highlight on the React page comes from the Detail press, as on Live's React page, which has no row click.

Read off the real window in both builds over a scratch sim fleet file holding one scrumming record and one extractor record, both `idle`, Start, Pause, Start, Stop, Restart, Delete on each row:

```
press      Bot ID cell tooltip      colour      sim fleet file
Start      State: RUNNING           #00ff88     running
Pause      State: PAUSED            #ffaa00     paused
Start      State: RUNNING           #00ff88     running
Stop       State: STOPPED           #666666     stopped
Restart    State: RUNNING           #00ff88     running
Delete No  State: RUNNING           #00ff88     running
Delete Yes row gone                             record gone, bot_count one lower
```

A record planted with `state_when_saved` `error` and a `last_error` reads `State: ERROR` in `#ff3366`; Start moves it to `running` and empties `last_error`. Start on a record already `running` leaves the file's bytes unchanged. The live-origin row logs `Bot <id> not found.`. The scratch `bot_state.json` hashes identical after every press, and a write planted into it from outside the program moves the hash. `ScrummingBot`, `ExtractorBot`, `BotManager.register`, `BotManager.unregister`, `BotContainer.start`, `BotContainer.stop`, `BotContainer.pause` and `MainWindow._on_bot_command` were watched for the whole run and saw zero calls; no socket left loopback. `SimBotManager` read for every attribute naming an exchange, a connector, a session, a socket, a client or a network holds zero, beside `BotManager`'s four.

## The voting panel reads the Stone Tablet

The Indicator Voting Panel on the Sim tab is fed, in both builds, from the tablets on disk. The Bot selector lists the sim fleet, one entry per accumulation bot in Live's own `symbol [id] (state)` form, and opens on the first. Choosing a bot draws the twelve voters, the Net, Comp and Conf columns, both bar graphs and the lock line for that bot's market on that bot's timeframe, computed by the real voting engine over the last hundred rows of the tablet filed under the bot's asset, exchange and timeframe. The rate strip prices A15 and A14 from the fleet's own rows. Nothing is invented; a bot with no tablet says so.

The reading is one function both hosts call, the window's dashboard feed forked over the tablet reader. It finds the tablet, reads the window, runs the engine, merges each phantom timeframe that has a tablet of its own, and weighs the Comp column over those rows with the same function the live feed weighs by.

`src/gui/simulator/sim_trading_tab_surface.py` — the feed

```
def ivp_feed(source: Any, bot: Any, now: Optional[float] = None) -> dict:
    """The voting panel's reading for ``bot`` over the tablet ``tablet_for``
    finds, with each phantom timeframe merged and ``composite_net`` over the
    phantom rows, or the one empty-state cause; a reading carries ``summary``,
    ``stored``, ``when``, ``age`` and ``message`` for ``show_stored``."""
```

### What feeds each part

The Simulator has no venue tick. The panel is fed at tab build, on every `fleet_changed`, and on every change of the Bot selector. A fleet change re-lists the selector, re-prices the rate strip and redraws the chosen bot; a selector change redraws the chosen bot alone. On the Qt build the selector's change is a signal the tab connects; on the React build the page asks the host for the chosen bot, the way the flip button asks for the replay layer.

| part | fed from |
|---|---|
| Bot selector | the fleet's statuses, accumulation bots only |
| rate strip | the fleet's A15 and A14 rows, priced by the currency monitor's own derivation |
| both indicator tables | the engine over the tablet window, one row per timeframe |
| both bar graphs | the same reading; the Qt bars move on their frame timer, the React bars arrive settled as Live's do |
| Net, Comp and Conf | the engine's net score, the composite over the phantom rows, the consensus confidence |
| lock line | `No active timeframe locks`; a sim bot holds no coordinator and no lock |
| staleness banner | the newest candle's time and age, and the day the tablet ends |

`src/gui/simulator/sim_trading_tab.py` — the Qt host's feed

```
    def refresh_votes(self) -> dict:
        statuses = self._fleet_source.statuses()
        panel = self._indicator_panel
        held = panel.blockSignals(True)
        try:
            panel.update_bot_list(statuses)
        finally:
            panel.blockSignals(held)
        panel.update_currency_rates(tab_surface.rate_snapshot(statuses))
        return self._feed_votes(panel.selected_bot_id)
```

### The banner over a tablet reading

A tablet ends where it ends, and a reading computed on candles that closed weeks ago is not a current one. The panel says so in Live's own words: the amber banner Live raises over a stored reading is raised over every tablet reading, naming the newest candle's time, how long ago that was, and the day the tablet ends. The banner is hidden on every empty state.

```
■ LAST TA READ, NOT CURRENT — taken 11:35:00, 46d 20h ago. Stone Tablet ends 2026-08-01.
```

### A bot with nothing to read

An idle or stopped sim bot draws Live's empty state for a bot that is not running, and the reading appears the moment Start moves it, because Start fires `fleet_changed`. A bot in error draws Live's error line. A bot whose asset, exchange and timeframe name no tablet on disk draws the sentence this page already names, and a bot whose tablet holds fewer rows than the engine needs is named with its count.

```
No TA data — bot is idle — a bot that is not running evaluates no TA.
No TA data — No Stone Tablet on disk.
```

A phantom timeframe with no tablet of its own is skipped, and one line names it in the program's log. On the operator's machine every tablet is 5m, so every phantom is skipped today and the Comp column reads the Net column.

### What the reading measured

The real window, both builds, a scratch tablet root holding copies of the A15 and A14 5m tablets, and a scratch sim fleet of five bots. Selecting the A15 bot drew every cell of both tables and every bar equal to the engine's own answer over the same hundred rows, in both builds, and Live's own panel handed the same summary drew the same cells, banner, rate line and lock line. A copy of the A15 tablet with its last twenty closes raised five per cent, filed under another asset, moved the reading: BB from 99 to 69 per cent, Net from –0.79 to +2.11, the consensus from bearish to bullish. The Qt bars read at two frames sixty milliseconds apart differed and settled on their targets; the React bars read equal at two moments, as Live's React bars do. The A23 bot, with no tablet, read the no-tablet sentence; the idle bot read the idle sentence and, after Start on the command bar, the A15 reading. The scratch fleet file and every tablet hashed identical after every reading, and a byte planted into two of them moved the hash. No socket left loopback.

### The selector's entry follows the state

Every command on the bar that moves a bot fires `fleet_changed`, and the panel re-reads the fleet on it. On the Qt build the selector is rebuilt whenever any entry's id or text changed, so the entry names the state the bot is in now, in Live's own form; the React page rebuilds its entries on the same signal. After Start, Pause, Stop and Restart the two builds read the same entry, and a bot whose record carries `cooldown` reads `(cooldown)` in both.

`src/gui/simulator/sim_indicator_panel.py` — the entry, one function

```
def _selector_entry_text(status: dict) -> str:
    bot_id = str(status.get("bot_id", ""))
    return ivp.SELECTOR_ITEM_FORMAT.format(
        symbol=status.get("symbol", ivp.DEFAULT_SYMBOL_TEXT),
        short_id=bot_id[: ivp.SELECTOR_ID_PREFIX_LEN],
        state=status.get("state", ivp.DEFAULT_STATE_TEXT),
    )
```

### What a fed panel with nothing to draw looks like

A panel fed a bot with no reading draws two empty tables and the words `Awaiting TA signals...` in both bar graphs, on Live and on the Sim tab, in both builds. That picture is the same whether the panel has been fed or not; the cause is held in the panel's own reading and the page's payload, not on the screen. An idle bot at open, or a running bot with no tablet on disk, draws the same pixels as a panel nobody has fed, with the cause under them.

```
No TA data — bot is idle — a bot that is not running evaluates no TA.
No TA data — No Stone Tablet on disk.
```

With a tablet on disk and a bot that is paused, running or in cooldown, the tablet reading draws at open with no press on the selector, in both builds.

## The Activity Log spool holds and resumes

The Activity Log at the foot left of the Sim tab is `SimStatusLog`, Live's `StatusLog` under the Simulator's name, seated under Live's title with Live's ■ `Pause Console` toggle in both builds. Every member of Live's pane is on the fork with Live's value: read-only, the placeholder `Activity log...`, 5,000 lines with the oldest dropped past that, a pause buffer of 2,000 that drops the newest when full, a resume that replays the held lines with their original stamps under one line counting them, and `notice` and `force_log`, which draw through a pause. Each line is `[hh:mm:ss] message`, the stamp in the muted colour and the message in its level's colour, with the trade, wire-flow and wire-stack shapes Live raises; since unit 30 a simulated trade's line carries the tape's stamp instead (comment 5731181590). The React page draws the same lines from the host's own `StatusLogModel` through `sim_status_log.js`, the fork of `status_log.js`.

`src/gui/simulator/sim_trading_tab.py` — the toggle's handler

```
        def _on_activity_pause_toggled(checked: bool):
            if checked:
                self._status_log.pause()
                self._activity_pause_btn.setText("■  Resume Console")
            else:
                self._status_log.resume()
                self._activity_pause_btn.setText("■  Pause Console")
```

### The page's own press

On the React page the Pause Console press makes two asks, `paused` on the log method and `activity_paused` on the tab method, and the host answers both through one call. It pauses or resumes its log, pushes the batch, and sets the caption, the way the Qt toggle's handler does; a second ask for the state already held changes nothing. The page's own bridge answers a log ask from the document the page holds, so the press keeps every line drawn since the page was built.

`src/gui/simulator/sim_react_trading_tab.py` — the press

```
        def set_activity_paused(self, paused: bool) -> bool:
            wanted = bool(paused)
            if wanted != self._log.paused:
                self.show_log_call("pause" if wanted else "resume")
            if wanted != self._state.activity_paused:
                self.show_tab({tab_surface.ACTIVITY_PAUSED_PARAM: wanted})
            return self._log.paused
```

### The watchdog, forked over the sim fleet

Both hosts run Live's Activity-Log watchdog every sixty seconds over the spool's own health and the sim bots the sim bot manager holds. It writes Live's line when the render-error count rises, and Live's silence line when nothing has rendered for ten minutes while a sim bot is running, once per ten minutes, with the critical line at thirty. Each line is force-logged, so a pause cannot hide it. The tick is Live's `watchdog_tick`, one definition, over `SimBotManager.bots`; a row read from `bot_state.json` is not held and is not counted.

`src/gui/simulator/sim_trading_tab_surface.py` — the tick

```
def watchdog_lines(
    state: live.WatchdogState, stats: Any, bots: Any, now: float
) -> list:
    running = live.running_bots(bot.state for bot in bots)
    return live.watchdog_tick(state, stats, now, running, running > 0)
```

### What the spool reading measured

The real window, both builds, a scratch sim fleet of one idle bot. The label, the toggle and the pane read the same three rectangles as Live's at 1400 and at the window's floor. Start on the bar drew ✓ `Bot <id> RUNNING.` and its notification; Pause Console then held the four lines Pause and Stop write and drew their two notifications, and Resume Console flushed the four in order under `(resumed — 4 buffered message(s) above)`, in both builds. Ten lines held under a pause flushed in order; 2,011 lines under a pause held 2,000 and dropped the eleven newest; 5,001 lines left 5,000 with the second first. The nine sample lines read the same colour, weight and size on the Sim's pane as on Live's in the same process, both builds. The watchdog's quiet tick wrote nothing; aged past ten minutes with one sim bot running it wrote Live's silence line through a pause. On the base commit the React page's press left the log running and dropped the page to the lines it was built with; on the branch it holds, flips the caption and keeps every line. `bot_state.json` hashed identical after every press, and a byte planted into it moved the hash. No socket left loopback.

## The API Interaction Log spool holds and resumes

The API Interaction Log at the foot right of the Sim tab is Live's pane under Live's title with Live's ■ `Pause API Log` toggle in both builds: read-only, the placeholder `API calls, responses, timing, data usage...`, no wrap, 2,000 lines with the oldest dropped past that, and a pause buffer of 2,000. Each host holds an API log of its own, a `SimApiLog` since unit 29 (comment 5730130103) and an `APIInteractionLog` at unit 18, built beside the two sources and read back through `api_log()`, and never the process-wide log the venue connectors write. The writer is Live's `_on_api_event`, forked over that log: one entry becomes Live's block, a stamped head line naming the exchange and the action, a Reason line, then Endpoint, Result, Response and Data usage where the entry holds them. A block arriving while Pause API Log is down is held, and Resume API Log draws the held blocks in the order they arrived under one line counting them. An entry recorded on any thread but the GUI thread is refused and written to the day's thread-violation file under the runtime log directory, as Live's writer does.

`src/gui/simulator/sim_trading_tab.py` — the writer

```
    def _on_api_event(self, entry: dict) -> None:
        current = threading.current_thread().name
        if tab_surface.api_event_off_thread(
            entry, "SimTradingTab._on_api_event", current
        ):
            return
        block_text = tab_surface.api_block(entry)
        if self._api_log_paused:
            buf = self._api_log_pause_buffer
            buf.append(block_text)
            cap = self._api_log_pause_buffer_cap
            if len(buf) > cap:
                del buf[: len(buf) - cap]
            return
        self._api_log_view.appendPlainText(block_text)
```

### The two logs

The Sim's log and the venue log are two objects. A venue call the Live bots make is recorded on the process-wide log, whose one listener is the Live tab's writer, so it draws on Live's pane and cannot reach the Sim's. An entry recorded on `api_log()` reaches the Sim's writer alone. At unit 18 nothing recorded into the Sim's log; a Stone Tablet retrieval or update and the YTD trade-file read are the two recorders the directive names, and unit 28 landed the first and unit 29 the second (comments 5728708856, 5730130103).

`src/gui/simulator/sim_trading_tab.py` — the log

```
        self._api_log = api_log if api_log is not None else APIInteractionLog()
        self._bot_manager = SimBotManager(self._fleet_source)
        self._layer = surface.LAYER_INDICATORS
        self._build()
        self._api_log.add_listener(self._on_api_event)
```

Unit 29 changed that constructor to `SimApiLog()` (comment 5730130103).

### The block on the React page

The React host's writer pushes each block to the page as one of `api_lines` on the tab method, and the host's own state appends it to its capped pane or holds it in its pause buffer, the way Live's page model does. The page draws the pane's blocks one line each and follows the newest while scrolled to the bottom. A block recorded before the page is built is drawn when the page opens, because the page's tab payload is built from the same state.

`src/gui/simulator/sim_react_trading_tab.py` — the writer

```
        def _on_api_event(self, entry: dict) -> None:
            current = threading.current_thread().name
            if tab_surface.api_event_off_thread(
                entry, "SimTradingTabReact._on_api_event", current
            ):
                return
            self.show_tab({tab_surface.API_LINES_PARAM: [tab_surface.api_block(entry)]})
```

### The page's own Pause API Log press

On the React page the Pause API Log press asks `api_paused` on the tab method, and the host answers through one call that pauses or resumes its buffer and sets the caption, the way the Qt toggle's handler does; a resume flushes the held blocks in order under the marker in the same push, and a second ask for the state already held changes nothing.

`src/gui/simulator/sim_react_trading_tab.py` — the press

```
        def set_api_paused(self, paused: bool) -> bool:
            wanted = bool(paused)
            if wanted != self._state.api_buffer.paused:
                self.show_tab({tab_surface.API_PAUSED_PARAM: wanted})
            return self._state.api_buffer.paused
```

### Past the pause buffer's cap

Past 2,000 held blocks the two builds part, and each follows its own side of Live. The Qt pane drops the oldest held block, so the flush ends on the newest entry; the React page keeps the first 2,000 and drops the newest, because the page model's buffer is Live's own `ApiPauseBuffer`, imported. The two rules are Live's two rules, and the Sim inherits whichever Live settles on.

`src/gui/main_tabs/trading_tab_surface.py` — the page model's hold

```
    def hold(self, line: Any) -> None:
        """Keep one line for the flush, up to the cap."""
        if len(self.lines) < self.cap:
            self.lines.append("" if line is None else str(line))
```

### What the API spool reading measured

The real window, both builds, a scratch sim fleet of one idle bot. The label, the toggle and the pane read the same three rectangles as Live's at 1400 and at the window's floor. One tablet-retrieval entry recorded on the Sim's log drew Live's six-line block on the Sim pane and nothing on Live's; Pause API Log then held two entries and drew nothing, and Resume API Log drew both in order under `--- (resumed; 2 buffered line(s) above) ---`, in both builds. One entry recorded on the process-wide log drew five lines on Live's pane and nothing on the Sim's; the same entry on the Sim's log drew on the Sim's. An entry recorded from a worker thread drew nothing and wrote one line naming the Sim's writer to the thread-violation file under the scratch home. Ten entries held under a pause flushed in order; 2,001 held under a pause kept 2,000, the Qt pane dropping the first and the React page dropping the last; 1,001 more two-line entries left the view at 2,000 blocks with the newest last. On the base commit no log existed for the Sim, nothing drew on its pane from any log, and the React page's press left the buffer running; on the branch it holds, flips the caption and flushes. `bot_state.json` hashed identical after every reading, and a byte planted into it moved the hash. No socket left loopback.

## The two strip rows keep Live's heights

The two rows above the bot list on a Sim venue page are Live's rows at Live's heights in both builds. The first is the row Live gives Privacy Mode and the news line: on the Sim it holds Validation, Back Test and Portfolio Battery in Live's Privacy-Mode sheet, then a stretch, then `+ New Bot` where Live draws it, and nothing else. The second is the row Live gives the data-pool line: on the Sim it is Live's label at Live's style holding no text in the Qt build and one no-break space in the React build, so the line keeps its height. No timer runs on a Sim venue page, so nothing rewrites either row after it is built; Live's venue page runs three, the ticker's 15 s step, its hourly refresh and the data-pool line's one-second rewrite. A press on a mode button sends `mode_clicked` with the mode's key and reaches no handler yet.

Neither row takes its height from a constant. Under the theme the header row is as tall as `+ New Bot`, which carries the theme's accent button padding, and the data-pool row is as tall as one line of its 11 px text plus its 2 px padding. The `height_px` figures in `RESERVED_ROWS`, 24 and 18, were read with no fonts and no theme; nothing that runs reads them.

`src/gui/simulator/sim_exchange_tab.py` — the two rows

```
            header = QHBoxLayout()
            self._mode_buttons: dict[str, QPushButton] = {}
            for mode in MODES:
                mode_btn = QPushButton(MODE_TEXT[mode])
                mode_btn.setAccessibleName(MODE_TEXT[mode])
                mode_btn.setFocusPolicy(Qt.NoFocus)
                mode_btn.setStyleSheet(MODE_BUTTON_STYLE)
                header.addWidget(mode_btn)
                self._mode_buttons[mode] = mode_btn
            header.addStretch()
            self._add_bot_btn = QPushButton("+ New Bot")
            self._add_bot_btn.setProperty("accent", True)
            if on_new_bot:
                self._add_bot_btn.clicked.connect(lambda: on_new_bot(exchange_id))
            header.addWidget(self._add_bot_btn)
            layout.addLayout(header)

            self._data_pool_row = QLabel("")
            self._data_pool_row.setAccessibleName(DATA_POOL_ROW_NAME)
            self._data_pool_row.setStyleSheet(
                f"color:{ds.MAIN_BADGE_TEXT}; font-size:11px; padding:2px 6px;"
            )
            layout.addWidget(self._data_pool_row)
```

Read off one window holding both tabs, with the Windows fonts loaded and the theme applied, one venue on Live and the same venue on the Sim, both venue pages handed the same one-bot status list through `update_bots`:

```
row                          Qt Live   Qt Sim   React Live   React Sim
header row                   33        33       33.2         33.2
data-pool row, one bot row   19        19       20.2         20.2
data-pool row, no bot row    407       407      20.2         20.2
```

The Qt data-pool label takes the leftover height on both pages when the bot tables hide, because both `QVBoxLayouts` are Live's; the React line is `flex: none` and keeps its height. The window's floor is 1400 by 900, its own minimum size, so the floor reading is the 1400 reading. The React figures are taken with both web views at device-pixel ratio 1.25; opened Live-first the two views hold 1.25 and 1.0 and each 1 px border snaps to 0.8 px on one side only, which reads as 33.2 against 33.8 and is a fact about the reading, not the page. The header's child list on the Sim reads Validation, Back Test, Portfolio Battery, a stretch and `+ New Bot` in both builds; Live's reads Privacy Mode, the news line and `+ New Bot`. A fourth button planted into the Sim header at runtime reads as a sixth child in both builds and the list returns to five when it is removed. `reserved_rows` is called by nothing in either build, no button carrying a way-in text sits inside a venue page, and `bot_state.json` hashed identical after every reading, while a byte planted into it moved the hash.

## The mode choice is wired and the way-ins follow it

The tab holds one run mode, in both builds. Validation is the run mode when the tab opens, the first of `MODES`. A press on Validation, Back Test or Portfolio Battery in a venue page's header makes that mode the run mode; its button carries Live's Privacy-Mode ON sheet and the two others carry the OFF sheet, on every seated venue page at once. The corner Live gives ■ `Add Crypto Exchange` and the Get Started card's button position each hold the two ways in the run mode offers, and the third is not drawn:

```
run mode            corner and card offer
Validation          Import Live Fleet, Generate From YTD
Back Test           Import Live Fleet, Create New Bots
Portfolio Battery   Run Portfolio, Run Every Portfolio
```

The pairs are `ROWS_FOR_MODE`, read through `reserved_rows`, and both builds draw the corner and the card from the same function. Each corner button keeps Live's corner-button minimum width and the 24 px height the fork names, and each card button Live's 180 by 36; the accessible names stay `sim-<action>` at the corner and `sim-<action>-card` on the card.

`src/gui/simulator/sim_trading_tab_surface.py` — the corner buttons per mode

```
def way_in_buttons(mode: str = sim.MODES[0]) -> list:
    """The two corner buttons ``mode`` offers, from ``sim.reserved_rows``, as
    the page draws them."""
    return [
        {
            "action": row["action"],
            "text": row["text"],
            "accessible_name": row["button_name"],
            "minimum_width_px": live.ADD_BUTTON_MIN_WIDTH_PX,
            "minimum_height_px": WAY_IN_BUTTON_HEIGHT_PX,
        }
        for row in sim.reserved_rows(mode)
    ]
```

The sheet each mode button carries comes from one function too, so the Qt header and the React header cannot disagree: `mode_style` answers Live's `PRIVACY_STYLE_ON` for the run mode and `PRIVACY_STYLE_OFF` for the rest. The React mode button also carries `aria-pressed`, as Live's Privacy Mode button does.

`src/gui/simulator/sim_exchange_tab_surface.py` — the sheet per button

```
def mode_style(mode: str, active: str) -> str:
    """The sheet the button for ``mode`` carries while ``active`` is the run
    mode: Live's Privacy-Mode ON sheet on the active mode, OFF on the rest."""
    return MODE_BUTTON_STYLE_ACTIVE if mode == active else MODE_BUTTON_STYLE
```

### Held by the tab, not by a venue page

Live's Privacy Mode is a per-page button because each page's button mirrors the process-wide mask registry. The run mode is the tab's: the corner and the card belong to the tab's layer, not to a venue page, the card draws when no venue page exists, and one run runs over the whole sim fleet. Two seated venues show the same active button. In the Qt build the tab holds it as `SimTradingTab.mode` and a venue page's press reaches `set_mode`; in the React build the tab state holds it as `SimTradingTabState.mode`, a press sends `mode_clicked` with the mode's key on the venue page's method, and the host's `run_action` hands it to `set_mode`.

`src/gui/simulator/sim_trading_tab.py` — the Qt press

```
    def set_mode(self, mode: str) -> str:
        """Make ``mode`` the run mode: the active sheet on every seated venue
        page's header and the mode's two ways in at each corner and card.
        A name outside ``MODES`` changes nothing; answers the mode in force."""
        if mode not in surface.MODES:
            return self._mode
        self._mode = mode
        for venue in list(self._crypto_exchange_tabs.values()) + list(
            self._stock_exchange_tabs.values()
        ):
            venue.show_mode(mode)
        self._draw_way_ins()
        return self._mode
```

`src/gui/simulator/sim_react_trading_tab.py` — the React press

```
        def set_mode(self, mode: str) -> str:
            """Make ``mode`` the run mode: redraw the tab so each corner and
            card offers that mode's two ways in, and re-publish every venue so
            its header carries the active sheet; answers the mode in force."""
            if self._state.set_mode(mode) == mode:
                self.show_tab({})
                self._venue_published()
            return self._state.mode
```

With no venue seated there is no header row, so nothing on the screen changes the run mode; the Get Started card offers the open mode's two ways in, Import Live Fleet and Generate From YTD, until a fleet is loaded and a venue page draws.

Diagram: flowchart, left to right

- mode button press -> **set_mode**
- set_mode -> **show_mode on every venue page**
- set_mode -> **way_in_buttons of the run mode**
- way_in_buttons of the run mode -> **the corner**
- way_in_buttons of the run mode -> **the Get Started card**
- the corner -> **_way_in**
- the Get Started card -> **_way_in**

- way_in -- Create New Bots -> **the Simulator's wizard**
- way_in -- any other -> **SendRefused on the Activity Log**

### Where a way-in press lands

Create New Bots, at the corner or on the card, opens the Simulator's Create Auto Trader wizard through `_create_bot`, the same path `+ New Bot` takes, in both builds. At unit 20 Import Live Fleet, Generate From YTD, Run Portfolio and Run Every Portfolio asked the fleet source for the run by name; the fleet source answered its read set and nothing else, so the ask raised `SendRefused`, and the tab wrote one line to the Activity Log naming the button. Units 21, 22 and 24 built the four runs (comments 5721499839, 5722113817, 5725991788).

`src/gui/simulator/sim_trading_tab.py` — the way-in handler

```
    def _way_in(self, action: str) -> None:
        """One way-in pressed at the corner or on the card: Create New Bots
        opens the wizard through ``_create_bot``; every other action asks
        ``FleetSource`` for it by name, which raises ``SendRefused`` until that
        run lands, and the refusal is logged to the Activity Log."""
        if action == surface.CREATE_NEW_BOTS_ACTION:
            self._create_bot(self._current_venue_id())
            return
        try:
            getattr(self._fleet_source, action)
        except SendRefused as exc:
            self._status_log.log(tab_surface.way_in_refused_line(action, exc), "error")
```

The line reads `Import Live Fleet refused: FleetSource answers (...) and cannot 'import_live_fleet'. The Simulator receives and asks; it sends nothing.`, at the error level, with the button's own text in front.

### What the mode reading measured

Read off the real window over a scratch home with every socket but loopback refused, the theme applied and the Windows fonts loaded, once with an empty sim fleet and once with one idle scrumming bot, in both builds:

```
reading                                     Qt          React
run mode at open                            validation  validation
sheets at open                              ON OFF OFF  ON OFF OFF
after Back Test                             OFF ON OFF  OFF ON OFF
after Portfolio Battery                     OFF OFF ON  OFF OFF ON
after Validation                            ON OFF OFF  ON OFF OFF
corner in Validation                        Import Live Fleet, Generate From YTD
corner in Back Test                         Import Live Fleet, Create New Bots
corner in Portfolio Battery                 Run Portfolio, Run Every Portfolio
widgets or nodes carrying a mode or way-in
  text, one bot seated                      11          9
  the same, empty fleet                     8           8
```

The Qt count with one bot is the header's three, the two layers' corners at two each, and the two layers' cards at two each; the crypto card is held but not listed once a venue is seated. The React count is the header's three,

the two corners at two each and the hidden stock layer's card at two; the React page draws no card on a layer that holds a venue. With an empty fleet neither build has a header, and the count is the two corners and the two cards. A button planted under the tab with a mode text moved each count by one and the count returned when it was removed. Every corner and card press landed where this section says: four refusal lines on the Activity Log, each naming its button, and the wizard for Create New Bots. `reserved_rows` is called on every draw of the corner and the card, in both builds, and `bot_state.json` hashed identical after every press while a byte planted into it moved the hash.

## Import Live Fleet fills the tables and the strip

Import Live Fleet copies the stored bots of one exchange out of the live fleet file into the Simulator's own fleet file, in both builds. A press at the corner or on the Get Started card reads which exchanges the live file names. With more than one it opens the chooser; with exactly one it takes that one and opens nothing; with none it writes one Activity Log line naming the file and moves nothing. Each stored bot on the chosen exchange is copied into the sim fleet under its own bot id, marked as imported, and the fleet signal fires: the sim fleet file is written, the venue seats, and the Scrumming Bots table and the Extractor Bots table draw one row per imported bot, matched by bot id. The Activity Log reads `Imported 3 bot(s) from bot_state.json on coinbase.` The live fleet file is only read, never written.

`src/simulator/fleet_source.py` — the copy

```
    def import_live_fleet(self, exchange_id: str) -> list[SimBot]:
        wanted = str(exchange_id or "")
        if not wanted:
            return []
        imported: list[SimBot] = []
        for bot_id, record in self.stored_records().items():
            bot = _sim_bot_from_record(bot_id, record, origin=LIVE_ORIGIN)
            if bot is None or bot.exchange_id != wanted:
                continue
            record["bot_id"] = bot_id
            record["origin"] = LIVE_ORIGIN
            self._records[bot_id] = record
            imported.append(bot)
        return sorted(imported, key=lambda one: (one.symbol, one.bot_id))
```

A second import of the same exchange replaces each held copy with the file's current record, under the same bot id; a held record the file no longer names stays until Delete removes it. A bot created through the wizard is kept beside the imported ones, so one sim fleet can hold both kinds. The command bar acts on an imported row as it acts on a wizard's: Start on an imported idle bot writes Live's ✓ `Bot <id> RUNNING.` line and colours its Bot ID cell.

Diagram: flowchart, left to right

- Import Live Fleet, corner or card -> **_way_in**
- _way_in -> **FleetSource.stored_exchanges**
- options -- more than one -> **SimExchangeChoiceDialog**
- options -- exactly one -> **the exchange**
- chooser -- Ok -> **the exchange**

- chooser -- Cancel -> **one Activity Log line**
- the exchange -> **FleetSource.import_live_fleet**
- FleetSource.import_live_fleet -> **fleet_changed**
- fleet_changed -> **sim_fleet.json**
- fleet_changed -> **the venue seats**
- the venue seats -> **one row per imported bot**

### The Simulator starts empty

The tab opens on the Get Started card with no venue seated, in both builds, whatever the live fleet file holds. The fleet source answers the records the sim fleet file holds and nothing else: its bots, its exchanges, its row statuses and its strip figures all come from those records, so with an empty sim fleet there is no venue, no row and no figure until a way in loads one. The live fleet file is opened only when Import Live Fleet is pressed, and then only to read it.

`src/simulator/fleet_source.py` — the held records alone

```
    def exchanges(self) -> list[str]:
        """Every distinct ``exchange_id`` the held bots name, sorted; empty
        until a way in has loaded a fleet."""
        return sorted({bot.exchange_id for bot in self.bots() if bot.exchange_id})
```

On the next launch the tab reads the sim fleet file once and draws the imported rows from it; the live fleet file is not opened. The Bot Swarm tab keeps its own read of the live file, which is Live's and unchanged.

### The exchange chooser

The chooser is Live's one-question dialog, the Bot Swarm tab's Configure Profit Wire, holding the bot wizard's `Exchange:` row: the title reads Import Live Fleet, one line names the exchanges, the combo lists each exchange under the caption its venue sub-tab carries, and Ok and Cancel close it. Both builds open the same dialog, as both open Live's Delete box. Cancel writes `Import Live Fleet cancelled.` to the Activity Log and moves nothing.

`src/gui/simulator/sim_exchange_choice.py` — the row

```
            row = QFormLayout()
            self._exchange = QComboBox()
            self._exchange.setAccessibleName(EXCHANGE_CHOICE_ROW_LABEL)
            for entry in exchange_choice_options(ids):
                self._exchange.addItem(entry["display_name"], entry["exchange_id"])
            row.addRow(EXCHANGE_CHOICE_ROW_LABEL, self._exchange)
```

### The ten fields on Sim

While Sim is in front the header strip reads Live's ten fields over every record the Simulator holds, imported or wizard-created, with Live's own arithmetic: the wallet is the largest cash balance any held bot carries, a position is holdings times price times the quote rate when both are known, maturity is read only where the venue has answered for that bot, and each year-to-date sum falls back to the lifetime sum at zero. A record that carries no figure for a field reads zero there, as a live bot's fresh statistics do, so a wizard-created bot adds nothing to the money columns until a run trades it, and SPENDABLE and LOCKED keep the em dash until a held record carries cash or a position.

`src/simulator/fleet_source.py` — the strip over the held records

```
    def aggregate(self) -> dict:
        """The header strip's figures over every held record,
        ``aggregate_stats`` of ``bots``."""
        return aggregate_stats(self.bots())
```

The arithmetic is a fork of the live fleet's, not a shared call, because the live version is a method of the bot manager reading live bot objects and the trading package does not change for the Simulator; the maturity threshold is imported from the live module, so the two cannot disagree on it. The strip re-reads on the window's dashboard tick and on the next tab change, as it does on Live after a bot is created.

### What the import reading measured

Read off the real window in both builds over a scratch home holding a live fleet file with three scrumming bots and one extractor across `coinbase` and `kraken`, and no sim fleet file:

```
reading                                   Qt                        React
at open: venues seated                    none                      none
at open: Get Started card                 listed, visible           drawn, visible
Import Live Fleet: chooser                Import Live Fleet, coinbase and kraken listed, both builds
coinbase chosen: venue seated             coinbase                  coinbase
rows by bot id, Scrumming Bots            u21scr01, u21scr02        u21scr01, u21scr02
rows by bot id, Extractor Bots            u21ext01                  u21ext01
strip after one tick                      $165.50 $155.25 11 1 2 | $1,500.75 $4.20 $367.49 $92.47 1
kraken chosen: venues seated              coinbase, kraken          coinbase, kraken
strip after one tick                      $235.50 $185.25 16 1 3 | $1,500.75 $5.30 $658.49 $92.47 2
aggregate against Live's over the four    21 of 21 keys equal       21 of 21 keys equal
Start on u21scr02                         RUNNING, Bots 2           RUNNING, Bots 2
second process, rows from the sim file    four rows, live file not opened by the Simulator
one exchange in the live file             no chooser, three imported, both builds
no bot in the live file                   one line, nothing moves, both builds
Cancel on the chooser                     one line, nothing moves, both builds
```

The scratch live fleet file hashed the same after every step in every run, and a byte planted into it from outside the program moved the hash. No live bot, bot manager or bot container was constructed, and no socket left loopback.

## Generate From YTD fills the tables from the YTD trade files

Generate From YTD builds one simulated bot per pair the operator's own trade record names on one exchange, in both builds. A press at the corner or on the Get Started card first reads the exchange history directory itself: a directory that is missing, empty or without its manifest writes one Activity Log line naming the path and holds nothing. With a manifest, the press reads which exchanges it names; with more than one it opens the chooser under the title Generate From YTD, with exactly one it takes that one, and with none it writes one line naming the manifest. Each traded pair on the chosen exchange becomes one record in the sim fleet under the pair's own id, the symbol at the exchange, marked as generated, and the fleet signal fires: the sim fleet file is written, the venue seats, and the Scrumming Bots table draws one row per pair. The Activity Log reads `Generated 3 bot(s) from 4 YTD trade file(s) on coinbase.` The trade files are only read, never written.

`src/simulator/fleet_source.py` — the generation

```
def generate_from_ytd(self, source: Any, exchange_id: str) -> YtdGeneration:
    wanted = str(exchange_id or "")
    if not wanted:
        return YtdGeneration()
    entries = [one for one in source.entries() if one.exchange_id == wanted]
    held: list[SimBot] = []
    missing: list[YtdFileEntry] = []
    files_read = 0
    for bot in ytd_fleet(source, wanted):
        own = [one for one in entries if one.symbol == bot.symbol]
        absent = [one for one in own if source.trade_path(one) is None]
        if absent:
            missing.extend(absent)
            continue
        trades: list[YtdTrade] = []
        for entry in own:
            trades.extend(source.trades(entry))
        files_read += len(own)
        target_usd = ytd_target_usd(trades)
        record = ytd_record(bot, target_usd)
        self._records[bot.bot_id] = record
```

Diagram: flowchart, left to right

- Generate From YTD, corner or card -> **YtdTradeSource.root_state**
- state -- missing, empty, no manifest -> **one Activity Log line naming the path**
- state -- ready -> **the exchanges the manifest names**
- options -- none -> **one line naming the manifest**
- options -- more than one -> **SimExchangeChoiceDialog**
- options -- exactly one -> **the exchange**
- chooser -- Ok -> **the exchange**
- chooser -- Cancel -> **one line, nothing moves**
- the exchange -> **FleetSource.generate_from_ytd**
- FleetSource.generate_from_ytd -> **fleet_changed**

- fleet_changed -> **sim_fleet.json**
- fleet_changed -> **the venue seats**
- the venue seats -> **one row per traded pair**

### The Target Balance a pair's fills establish

A YTD file names no bot and no target. On Live the target is the dollar value the bot keeps its position at, and the bot reads its position as holdings times price. The pair's own fills establish the position they built, so the generated record's Target Balance is read off them: over every fill of the pair across its year files, oldest first, the units the buys bought less the units the sells sold, valued at the last fill's price, rounded to the cent.

```
units_bought = the amount of every BUY fill, summed
units_sold   = the amount of every SELL fill, summed
units_held   = units_bought - units_sold
last_price   = the price of the last fill
Target       = units_held x last_price, to the cent      when units_held is positive
Target       = none                                      when it is not
```

The fill's cost is its quote leg, the amount times the price before the fee; its net over the fills is the quote spent, not the position's value, so the derivation reads the amount and the price and does not read the cost. When the fills sold more than they bought, the position was opened before the file's first row and the file gives no basis: the record holds no target, the Target cell draws `---`, and the Activity Log says how many records read so. On the scratch files the reading was taken over, A15/USD bought 0.007 and sold 0.002 across two year files, holding 0.005 at a last price of 61,000, so its Target is $305.00; A23/USD bought 1.0 and sold 3.0, so it holds no target.

`src/simulator/fleet_source.py` — the derivation

```
def ytd_target_usd(trades: Sequence[YtdTrade]) -> Optional[float]:
    ordered = sorted(trades, key=lambda one: one.sort_key())
    if not ordered:
        return None
    bought = sum(float(one.amount) for one in ordered if one.side == SIDE_BUY)
    sold = sum(float(one.amount) for one in ordered if one.side == SIDE_SELL)
    held = bought - sold
    if held <= 0:
        return None
    return round(held * float(ordered[-1].price), 2)
```

Under the funding directive, the spendable budget of a Validation or Portfolio Battery run is unbounded and always equals the total of the Target Balances of every simulated bot in the run. A generated bot with no target therefore adds nothing to that budget; the run funds the pairs whose fills established a position and the others stand in the fleet with no target until one is set.

### What a generated record carries

The record is the wizard's record shape with the pair's ids and the derived target: the exchange, the quote currency as the base currency, the base asset as the target asset, the symbol, and the Target Balance, or `null` where the fills gave no basis. Every other config field is the bot config's own default, the same a wizard-created bot carries when nothing on the wizard is changed, so a generated bot reads the one-hour timeframe and the default gates. The record carries no statistics, no lots and no phantom set, because the fills are the pair's year of history and not the bot's own figures: the strip's money fields read zero over generated records, Bots counts one when it is started, and a run is what trades it.

`src/simulator/fleet_source.py` — the record

```
def ytd_record(bot: SimBot, target_usd: Optional[float]) -> dict:
    record = wizard_record(
        {
            "exchange_id": bot.exchange_id,
            "base_currency": bot.base_currency,
            "target_asset": bot.asset,
            "symbol": bot.symbol,
            "target_balance": 0.0 if target_usd is None else float(target_usd),
        }
    )
    record["config"]["target_balance"] = target_usd
    record["bot_id"] = bot.bot_id
    record["origin"] = YTD_ORIGIN
    return record
```

A second generation of the same exchange replaces each held copy under the same id with a record re-derived from the files as they lie, so a pair whose files grew reads its new target and a started bot returns to idle; a bot created through the wizard or imported from the live fleet is kept beside the generated ones. The command bar acts on a generated row as it acts on any other: Start on a generated idle bot writes Live's ✓ `Bot <id> RUNNING.` line and colours its Bot ID cell.

### The empty cases say which

Each case writes one Activity Log line naming the path or the file, in the shape the import's lines take, and holds nothing. The directory is read before any trade file, so a missing directory is named and is not created by the press. A manifest row whose file is gone names that file and leaves that pair out; the other pairs on the exchange are generated.

`src/gui/simulator/sim_trading_tab_surface.py` — the lines

```
GENERATED_FORMAT = (
    "Generated {count} bot(s) from {files} YTD trade file(s) on {exchange}."
)
NO_TARGET_FORMAT = (
    "{count} of them hold no Target Balance: the fills sold more than they bought."
)
GENERATE_CANCELLED_TEXT = "Generate From YTD cancelled."
YTD_ROOT_MISSING_FORMAT = "No YTD trade directory at {path}."
YTD_ROOT_EMPTY_FORMAT = "{path} holds no YTD trade file."
YTD_ROOT_NO_MANIFEST_FORMAT = "{path} holds no {manifest}."
YTD_NO_PAIR_FORMAT = "{manifest} under {path} names no traded pair."
YTD_FILE_MISSING_FORMAT = (
    "{file} named by {manifest} is missing; {symbol} on {exchange} not generated."
)
```

The reader answers eight names now: the state of its directory and the presence of one entry's file joined the six, and the directory it reads is resolved without being created.

`src/simulator/ytd_trade_source.py` — the directory's state

```
    def root_state(self) -> str:
        if not self._root.is_dir():
            return ROOT_MISSING
        if not any(self._root.iterdir()):
            return ROOT_EMPTY
        if not (self._root / MANIFEST_NAME).is_file():
            return ROOT_NO_MANIFEST
        return ROOT_READY
```

### What the generation reading measured

Read off the real window in both builds over a scratch home holding three pairs on `coinbase` (A15/USD over two year files, A14/USD, A23/USD) and one on `kraken` (A18/USD), 15 fills in all, a scratch live fleet file that is only hashed, and no sim fleet file:

```
reading                                     Qt                      React
at open: venues seated                      none                    none
Generate From YTD: chooser                  Generate From YTD, coinbase and kraken listed, both builds
coinbase chosen: venue seated               coinbase                coinbase
rows by id, Scrumming Bots                  A15/USD@coinbase, A14/USD@coinbase, A23/USD@coinbase, both builds
Target cells                                $305.0000, $345.0000, ---, both builds
Activity Log                                Generated 3 bot(s) from 4 YTD trade file(s) on coinbase., both builds
                                            1 of them hold no Target Balance: the fills sold more than they bought.
strip after one tick                        $0.00 $0.00 0 0 0 | — — — — 1, both builds
kraken chosen at the corner                 one row, A18/USD@kraken, $36.0000; two venues, both builds
Start on A15/USD@coinbase                   ✓ Bot A15/USD@coinbase RUNNING., Bots 1, both builds
a fourth pair planted, coinbase again       four rows, A22/USD@coinbase at $200.0000, both builds
second process, rows from the sim file      five rows; no trade file opened, both builds
no exchange_history directory               No YTD trade directory at <path>., not created, both builds
an empty directory                          <path> holds no YTD trade file., both builds
files without MANIFEST.json                 <path> holds no MANIFEST.json., both builds
a manifest naming no pair                   MANIFEST.json under <path> names no traded pair., both builds
one named file removed                      <file> named by MANIFEST.json is missing; A14/USD on coinbase not generated.; two rows,
both builds
Cancel on the chooser                       Generate From YTD cancelled.; nothing moves, both builds
```

Every trade file, the manifest and the scratch live fleet file hashed the same after every step in every run, and a byte planted into the live fleet file and into the manifest from outside the program moved the hash. No live bot,

bot manager or bot container was constructed, and no socket left loopback.

## Create New Bots fills one row through the forked wizard

Create New Bots is Back Test's second way in, in both builds. It opens the Simulator's Create Auto Trader wizard from the Back Test corner, from the Get Started card when the mode is Back Test, and `+ New Bot` on a seated venue opens the same wizard. The wizard carries Live's five pages in Live's order with Live's fields, and Finish creates one sim bot: its record is held, the sim fleet file is written, the venue the bot names seats if it is new, and the row draws in that venue's Scrumming Bots table or Extractor Bots table. The row carries the wizard's target in the Target cell and the wizard's interval in the status the row is drawn from, and the Bot Settings window opened from the row's Detail shows every value the wizard took.

`src/gui/simulator/sim_trading_tab.py` — the corner and the card reach the wizard

```
        if action == surface.CREATE_NEW_BOTS_ACTION:
            self._create_bot(self._current_venue_id())
            return
```

Diagram: flowchart, left to right

- Create New Bots, corner or card -> **_way_in**
- + New Bot on a venue -> **_create_bot**
- _way_in -> **_create_bot**
- _create_bot -> **SimBotCreationWizard or SimBotWizardReactDialog**
- SimBotCreationWizard or SimBotWizardReactDialog -> **get_bot_config, 49 keys or 13** *(Finish)*
- get_bot_config, 49 keys or 13 -> **FleetSource.create, wizard_record**
- FleetSource.create, wizard_record -> **fleet_changed**
- fleet_changed -> **sim_fleet.json**
- fleet_changed -> **the venue seats, the row draws**
- the venue seats, the row draws -> **Detail: Bot Settings over SimBotView**

### Every field the wizard sets, and where it lands

A scrumming walk hands 49 keys and an extractor walk 13, the counts Live's wizard hands from the same window. The record's config holds every field a bot's configuration declares, so a typed value on the Trading Parameters page reaches the Settings tab's control for that field, one control per field.

| page | what it sets | the record | the row | the Settings window |
|---|---|---|---|---|
| Trading Mode | the mode | `config.mode` | which table the row lands in; the Mode cell of an extractor row | seven tabs for a scrumming bot, three for an extractor |
| Select Asset Pair | exchange, base currency, target asset | the three ids and the symbol | the venue sub-tab; the Symbol cell | the title and the pair |
| Extractor Pool | exchange, pool base | the same, the target asset * | the Symbol cell reads the pool base | the same |

| page | what it sets | the record | the row | the Settings window |
|---|---|---|---|---|
| Trading Parameters | 44 settings for a scrumming bot, 7 for an extractor | every one on the config | Target from the target balance; Pool from the chunk size; the Fire tip from the timeframe, the ceiling and the detonation | the Settings tab, one control per field |
| Phantom Bots | enable, one timeframe, the lock count | the enable flag and the timeframes beside the config | — | the Phantom Bots tab's enable box and timeframe |

The three Phantom Bots keys are stored beside the config, as the live process stores them for a running bot: the enable flag as typed, and the timeframes as the bot would hold them, the ticked one, or the one timeframe above the bot's own when none is ticked, kept to the timeframes the venue offers. An extractor stores the flag off and no timeframe. The lock count is the one field the wizard collects that reaches no new bot on Live: the phantom coordinator holds it and a new bot's coordinator holds 2, so the Simulator writes none and its Phantom Bots tab reads 2, as Live's does for a new bot.

`src/simulator/fleet_source.py` — the phantom keys

```
    scrumming = mode == BotMode.SCRUMMING
    record = {
        "config": stored,
        "stats": {},
        "scrumming_state": {},
        "state_when_saved": BotState.IDLE.value,
        "phantoms_enabled": scrumming and bool(collected.get("enable_phantoms", False)),
    }
    timeframes = (
        wizard_phantom_timeframes(
            collected.get("phantom_timeframes"), built.ta_timeframe, built.exchange_id
        )
        if scrumming
        else []
    )
    if timeframes:
        record["phantom_timeframes"] = timeframes
```

### The venues the wizard offers

Live's wizard lists the venues connected in settings. The Simulator's markets are its Stone Tablets, so its wizard lists the seated venues first and then every exchange the tablet manifest names, and the Get Started card can create a bot before any fleet is loaded. The section "What the Simulator's wizard reads" above describes the list before the tablets' exchanges were added.

`src/gui/simulator/sim_bot_wizard_surface.py` — the venue list

```
def wizard_exchanges(exchange_ids: Iterable[str], tablet_source: Any) -> list[dict]:
    seated = [str(eid) for eid in exchange_ids if str(eid)]
    filed = sorted(
        {str(entry.exchange_id) for entry in tablet_source.entries()} - set(seated)
    )
    return seated_exchanges([*seated, *filed])
```

### The React page narrows its timeframes as the Qt screen does

The Qt wizard lists the picked venue's own timeframes in the TA Timeframe combo and greys the phantom timeframes the venue does not offer. The React wizard takes one list per venue and narrows both pages from it; the React host hands it that list, so both builds show the same timeframes for the same venue. Live's window hands its own React wizard no list, so Live's React page lists the wizard's fixed seven and greys nothing; that is Live's and is not changed here.

`src/gui/simulator/sim_react_trading_tab.py` — the list handed over

```
            wizard = wizard_class(
                exchanges,
                defaults,
                self,
                theme=self._theme,
                markets=markets,
                timeframes=wizard_surface.venue_timeframes(exchanges),
            )
```

### The old runner's bot builder is not on the path

`new_bots` in `src/simulator/back_test.py` builds bots from six-key specs for the old tab's Back Test runner, which the window no longer builds. Create New Bots reaches it nowhere: watched across every reading below it counted 0 calls, and a direct call counted 1. It is left as it is.

### The route on a fresh Simulator

With no fleet loaded the mode is Validation, the card and the corner offer Import Live Fleet and Generate From YTD, and no header row exists to press Back Test on. The route to Create New Bots is: load a fleet, press Back Test on the header, press Create New Bots at the corner, or `+ New Bot` on the venue. With the mode on Back Test and every bot deleted, the card returns offering Create New Bots.

### What the creation reading measured

Driven in the real window in both builds with a scratch home, every socket but loopback refused, a scratch live state file holding one bot on each of two exchanges and a scratch tablet manifest naming four assets on both. Before the change, the three Phantom Bots keys stopped at the wizard: the record carried none, the Phantom Bots tab read the enable box ticked with the wizard's box unticked and the timeframe 1d with 4h ticked, and the card's wizard opened with no venue and Finish was refused for an empty target asset.

```
reading                                                Qt                        React
Back Test pressed: the corner                          Import Live Fleet, Create New Bots     the same
the wizard, every page walked, every field typed       49 keys                   49 keys
record config values against the typed values          47 of 47                  47 of 47
Settings tab controls against the typed values         42 of 42                  42 of 42
Phantom Bots tab: enable, timeframe, lock              off, 4h, 2                off, 4h, 2
the row: Bot ID, Symbol, Trades, Target                id, A15/USD, 0, $1,234.5600 (both)
the row against a target of 200                        the Target cell differs (both)
an extractor, pool base A15 on the same venue          13 keys; 12 of 12; Pool $250.50 (both)
an extractor with no holder                            refused with Live's own box (both)
+ New Bot on the other venue                           one row, $333.3300 (both)
Start on the created row: the strip's Bots             0 to 1                    0 to 1
Spendable                                              a dash before and after (both)
a second process over the same home                    every created row again, every value (both)
every bot deleted under Back Test: the card            Import Live Fleet, Create New Bots (both)
Create New Bots on the card: the venues offered        coinbase, kraken from the tablets (both)
bot_state.json                                         unchanged after every step (both)
ScrummingBot, ExtractorBot, BotManager                 0 calls (both)
```

The strip's Spendable reads the fleet's cash balance, which a created bot has none of; the funding rule that makes it the sum of the held Target Balances belongs to the unit that shares the trading arithmetic, for Validation and Portfolio Battery. The venue page's rows in the React build carry no click that selects a row for the command bar, on Live's page and on the Simulator's; Detail selects the row as it opens.

## One trading logic: the Simulator sizes its trades with the Live bot's own arithmetic

The Live bot and the Simulator now share one definition of every sizing figure. The position value, the Target Delta, the delta percent, the scrumming interval in USD, the scrum's units, a sale's proceeds net of its fee, the fold's eligible tranches, the per-cycle growth cap, what a fold takes from each tranche, the fold's spend and its units, the position ceiling, the fold taper and the cartridge threshold each live as one function in a module beside the bot's mixins. The bot calls them at the lines that computed them inline, and the Simulator's walk calls the same functions over a simulated position. The module holds no state and imports no bot, no venue and no bus.

`src/trading/scrumming/sizing.py` — three of the figures

```
def scrum_units(delta_usd: float, price: float) -> float:
    return abs(delta_usd) / price

def fold_spend_usd(eligible_usd: float, taper: float) -> float:
    return eligible_usd * taper

def fold_units(spend_usd: float, price: float) -> float:
    return spend_usd / price
```

Unit 26a gave `scrum_units` and `fold_units` a third argument, the unit rule; the section "The unit rule" quotes the three-argument forms (comment 5725310316).

### What the tick calls

`ScrummingBot.tick` values the position through `priced_usd`, takes the delta through `target_delta_usd` and its percent through `target_delta_pct`, sizes the interval through `scrumming_interval_usd` and asks `delta_below_interval` whether to hold. The scrum sells `scrum_units` of the delta at the ticker and books the proceeds through `sale_proceeds_usd` less the fee the venue reports. The fold reads `eligible_fold_tranches` under the opposing-trade rebuy factor, plans what it takes from each tranche under `cycle_growth_cap_usd` and `fold_cap_remaining_usd` through `plan_fold_consumption`, spends `fold_spend_usd` of that under the taper, books `fold_units` at the fill and settles the tranches through `settle_fold_plan`.

`src/trading/scrumming_bot.py` — the tick's four figures as they read now

```
delta = target_delta_usd(current_value, self._target_balance)
delta_pct = target_delta_pct(delta, self._target_balance)
_interval_usd = scrumming_interval_usd(
    self._target_balance, self.config.scrumming_interval_pct
)
```

The bot's behaviour was read unchanged off the operator's own logs without building a bot: over 270 scrum fills paired with the gate row the bot wrote at each fire, `scrum_units` over the row's delta and ticker answered the row's filled amount on 270 of 270; over 61 fold fills, `fold_units` over the row's spend and fill answered the filled amount on 61 of 61. One row with its amount moved by one unit in the eighth place was read as a disagreement.

### What the Simulator's walk calls

`apply_scrum` sells `scrum_units` of the delta, values them through `priced_usd`, estimates the fee through `estimated_fee_usd` from the bot's Trading Fee, since a walk has no venue to report one, and queues the net proceeds as one fold tranche in the shape the tick builds. `apply_fold` is the tick's fold over the position's tranches: eligibility, the cap plan, the taper when the bot's ceiling is on, the spend and the units, then the settlement. The Target Delta no longer sizes a fold, as it never sized one in the tick.

`src/simulator/back_test.py` — the fold's spend

```
spend = fold_spend_usd(sum(one["usd"] for one in slices), taper)
if funding == FUNDED_BY_PROCEEDS:
    spend = wallet_capped_spend_usd(spend, position.cash_usd)
```

Before this the Simulator sized a fold on the Target Delta and capped it at the cash its scrums had left. Walked over one scratch tape, that sized 46 folds where the Live arithmetic sizes 187, each at most the per-cycle growth cap of its target; the 47 scrums latched on the same candles both ways, and 12 of them sold the same units, the two before the first fold and those where the folds between had brought both positions back to the same holding. A count of arithmetic on a price, units, a target, an interval, a delta, a spend or a fee across `src/simulator` reads zero sizing expressions; the twenty-one that remain value a held position for the strip, a portfolio's baseline and its improvement, an Extractor position's profit, and a candle interval in time.

### The funding rule

A Validation or Portfolio Battery run has no venue wallet. Its spendable budget is unbounded and always equals the sum of the held bots' Target Balances; a bot with no Target Balance adds nothing. The run reads that sum once as it starts and carries it as `budget_usd`, and no fold in these two modes is held to the cash the scrums left. Back Test keeps its own rule: each bot's fold spends its own scrum proceeds and no more, through `wallet_capped_spend_usd`, the cap Manual Fire applies at the wallet. A fold sized by the Live arithmetic spends at most the tranche dollars its own scrums queued, so on the same tape the two fundings walked the same 234 trades, one of them apart by two femto-dollars of rounding.

`src/simulator/back_test.py` — the budget

```
def run_budget_usd(bots: Sequence[SimBot]) -> float:
    return sum(float(bot.target_usd) for bot in bots if bot.target_usd is not None)
```

The header strip reads the rule. With the Sim tab in front in Validation or Portfolio Battery, Spendable reads the sum of the held Target Balances; in Back Test it reads what it read before, the largest cash balance the held records carry. Driven with four held bots whose targets summed to twelve hundred dollars, one of them with no target, the cell drew `$1,200.00` in both modes and `$0.00` in Back Test, in both builds, and the walk over the tab's own fleet and tablets called eighteen of the twenty shared functions, every call from `walk`; the two never reached are the cartridge threshold, which only the tick fires, and the wallet cap, which only Back Test applies.

`src/gui/main_tabs/simulator_tab_surface.py` — the mode's funding

```
MODE_FUNDING = {
    MODE_VALIDATION: back_test.FUNDED_BY_TARGETS,
    MODE_BACK_TEST: back_test.FUNDED_BY_PROCEEDS,
    MODE_PORTFOLIO_BATTERY: back_test.FUNDED_BY_TARGETS,
}
```

### One drawdown

The state an Extractor writes on a position below its entry value was spelled twice, once in the Extractor and once in the Simulator's fleet reader. Both names now read `DRAWDOWN_STATE` from the sizing module; a planted change to that one definition moved both.

`src/trading/scrumming/sizing.py` — the state

```
DRAWDOWN_STATE = "drawdown"
```

## The unit rule: fractional or whole, per asset class, cited from the venue

A Portfolio Battery run walks shares and funds as well as coins, and a share can only be bought the way its venue sells it. Before any non-crypto symbol is simulated, one question is answered per asset class from the venue's own published page: does the class trade in fractions of a unit, or in whole units only. The answer is a row in a table the sizing module carries, and a class with no row is not simulated.

The portfolios name sixty-three symbols and the RA-StoneTablet import asks for the same sixty-three. Thirteen are crypto: A34, ATOM, AVAX, BNB, A15, A22, DOT, A14, A21, MATIC, A23, TRX and A18. The other fifty are

US exchange-listed shares and funds, every one read off the Yahoo route as a share: AAPL, AGG, AMC, AMD, AMZN, ARKF, ARKG, ARKK, ARKW, BABA, BBBY, BIDU, BND, CCIV, COIN, CVNA, DKNG, EXPR, GLD, GME, GOOGL, IEF, IPOF, IWM, JD, KOSS, MCHI, META, MSFT, NIO, NVDA, OPEN, PLTR, PRNT, PTON, QQQ, ROKU, SLV, SNDL, SOFI, SPY, TDOC, TIP, TLT, TSLA, USO, UWMC, VNQ, XLU and ZM. The classes are named with the words the ATA-SPM scanner spells: crypto and stocks. GLD, SLV and USO are the funds the scanner charts for the metals and energy classes; on the venue they are exchange-listed fund shares, so the stocks row binds them. No portfolio names a spot, forward or contract instrument of the metals, energy, forex or derivatives classes, and no connected venue lists one.

### The determination

Each page below was opened as a document on 2026-09-17 and its sentences quoted. No endpoint was called, no credential was sent, and no account page was opened. A rule a page does not state is not cited.

| class | venue Acervator connects to | fractional | conditions |
|---|---|---|---|
| crypto | Coinbase, the one verified exchange and the venue every held live bot names | yes | the size is a multiple of the product's `base_increment` and at least its `base_min_size`; the connector reads both off the market as the amount precision and the minimum amount |
| stocks | Alpaca, the one broker connector in the tree, which sends an order's `qty`; no screen constructs it today | yes | exchange-listed common stocks and ETFs, generally over one dollar with a daily volume over ten thousand shares; a per-asset `fractionable` flag read only through the authenticated assets endpoint, so it stays a condition and is not read; market, limit, stop or stop-limit orders; time in force Day; no short sales |
| metals, energy, forex, derivatives | none | not cited | no connected venue lists the class; not simulated |

The crypto row, quoted from the venue's pages

```
https://docs.cdp.coinbase.com/api-reference/exchange-api/rest-api/orders/create-new-order
  titled "Create a new order", no date shown, read 2026-09-17
  "The size can be in any increment of the base currency (e.g. A15 for the
   A15-USD product)."
  "The size can be in incremented in units of base_increment."
  "The size must be no less than the base_min_size and no larger than the
   base_max_size for the product."
https://docs.cdp.coinbase.com/api-reference/advanced-trade-api/rest-api/products/get-product
  titled "Get Product", no date shown, read 2026-09-17
  base_increment: "Minimum amount base value can be increased or decreased
   at once."
```

The stocks row, quoted from the venue's pages

```
https://alpaca.markets/support/fractional-shares
  titled "Does Alpaca support fractional shares?", dated February 2024,
  read 2026-09-17
  "Yes! Alpaca supports fractional trading via API and the dashboard."
  "Fractional Securities include: US Exchange listed Securities (Common
   Stocks and Exchange Traded Funds) Selected Over the Counter American
   Depository Receipts"
  "We generally will allow securities to be fractionable under the below
   scenarios: Exchange Listed Over $1 Average Daily Trading Volume over
   10,000 shares"
  "To see if an asset is fractional, check the field fractionable = true
   from the GET/v2/assets endpoint."
https://docs.alpaca.markets/docs/fractional-trading
  titled "Fractional Trading", updated 2025-09-24, read 2026-09-17
  "Alpaca currently supports fractional trading for market, limit, stop &
   stop limit orders with a time in force=Day, accommodating both fractional
   quantities and notional values."
  "Both notional and qty fields can take up to 9 decimal point values."
  "Not all assets are fractionable yet so please make sure you query assets
   details to check for the parameter fractionable = true."
  "We do not support short sales in fractional orders. All fractional sell
   orders are marked long."
```

Both classes the portfolios name trade fractionally on the venue Acervator connects to for them. No class a Battery run walks today reads whole-unit, so today every walk sizes as it did before; the whole-unit variant below is the switch the table throws the day a row reads whole.

`src/trading/scrumming/sizing.py` — the table and the function that reads it

```
FRACTIONAL_UNITS = "fractional"
WHOLE_UNITS = "whole"

CITED_UNIT_RULES: dict[tuple[str, str], str] = {
    (CLASS_CRYPTO, "coinbase"): FRACTIONAL_UNITS,
    (CLASS_STOCKS, "alpaca"): FRACTIONAL_UNITS,
}

def unit_rule(asset_class: str, venue: str) -> Optional[str]:
    return CITED_UNIT_RULES.get((str(asset_class), str(venue)))
```

A bot's class comes from its symbol and its venue: a name in the crypto list is crypto, every other archive name is stocks, and a name outside the archive on a crypto exchange is crypto. The venue a class trades on is the bot's own exchange for crypto and Alpaca for stocks. TRX has RA tablets from the Yahoo source alone, because Coinbase served it no candle, so its pair is crypto on `yahoo`, which no row cites, and TRX is not simulated.

`src/simulator/portfolios.py` — the class and the venue

```
def asset_class(symbol: str, exchange_id: str = "") -> Optional[str]:
    name = str(symbol or "").upper()
    if name in CRYPTO_SYMBOLS:
        return CLASS_CRYPTO
    if name in SYMBOLS:
        return CLASS_STOCKS
    if str(exchange_id or "") in crypto_venues():
        return CLASS_CRYPTO
    return None

def trading_venue(class_name: Optional[str], exchange_id: str = "") -> str:
    if class_name == CLASS_CRYPTO:
        return str(exchange_id or "")
    if class_name == CLASS_STOCKS:
        return STOCKS_VENUE
    return ""
```

### The whole-unit variant

The two sizing functions now take the rule. Under the fractional rule they answer the expression they answered before, unchanged. Under the whole rule they floor to a whole number of units, and a count within one billionth of a whole number reads as that number: a bare floor read one unit under on 11,593 of 200,000 exact multiples of a price in the pre-mortem sweep, and none with the grain, which is the ninth decimal an Alpaca quantity carries at most. A rule that is neither word raises, so no walk sizes under an unnamed rule.

`src/trading/scrumming/sizing.py` — sizing under the rule

```
WHOLE_UNIT_GRAIN = 1e-9

def sized_units(units: float, rule: str) -> float:
    if rule == FRACTIONAL_UNITS:
        return units
    if rule == WHOLE_UNITS:
        return float(math.floor(units + WHOLE_UNIT_GRAIN))
    raise ValueError(f"unit rule {rule!r} is not one of {UNIT_RULES}")

def scrum_units(delta_usd: float, price: float, rule: str) -> float:
    return sized_units(abs(delta_usd) / price, rule)

def fold_units(spend_usd: float, price: float, rule: str) -> float:
    return sized_units(spend_usd / price, rule)
```

The Live bot passes the fractional rule at each of its nine call sites and changes nothing else. The Simulator resolves the rule once per bot as a run reaches it and carries it through the walk: the opening position, every scrum and every fold size under it. A whole-unit scrum whose Target Delta buys fewer than one unit fills nothing and logs why. A whole-unit fold spends the whole units' price; what it could not spend is taken back off the last entries of the fold plan, so that remainder stays in the tranche, and a fold whose spend buys fewer than one unit fills nothing, settles nothing, and logs why.

`src/simulator/back_test.py` — the whole-unit fold

```
    units = fold_units(spend, float(price), rule)
    if units <= 0.0:
        if rule == WHOLE_UNITS:
            logger.info(
                "%s: a fold of $%.2f at %.8f is %s; nothing fills",
                bot.bot_id,
                spend,
                float(price),
                BELOW_ONE_UNIT,
            )
        return None
    if rule == WHOLE_UNITS:
        bought_usd = priced_usd(units, float(price))
        plan = trim_fold_plan(plan, spend - bought_usd)
        spend = bought_usd
```

A whole-unit fold needs tranche dollars of at least one unit's price. At the one percent default growth cap on a five hundred dollar target the cap is five dollars, so a share priced above five dollars would never fold in whole units. This binds the Battery's default Target Balance only if a row ever reads whole; today none does.

### Not cited, not simulated

A run resolves each bot's class and venue before it opens a position. A pair with no row in the table is refused: the bot's outcome reads `uncited_rule`, nothing is walked for it, and the run's lines carry one line naming the bot, its class, its venue and that the rule is not cited. The Battery does the same per symbol and counts the symbol's capital as missing weight, and its lines name every refused symbol.

`src/simulator/back_test.py` — the refusal in the runner

```
    for bot in bots:
        class_name, venue, rule = cited_rule_for(bot.asset, bot.exchange_id)
        if rule is None:
            outcomes[UNCITED_RULE] += 1
            results.append(
                BotResult(
                    bot_id=bot.bot_id,
                    symbol=bot.symbol,
                    tablet_key="",
                    outcome=UNCITED_RULE,
                    asset_class=class_name,
                    venue=venue,
                )
            )
            continue
```

`src/simulator/back_test.py` — the line the Activity Log draws

```
def uncited_rule_line(result: BotResult) -> str:
    return (
        f"{result.bot_id}: {result.symbol} is class {result.asset_class or 'none'} "
        f"on venue {result.venue or 'none'}, which has no cited unit rule; "
        "not simulated."
    )
```

### What the unit-rule reading measured

Before this, over the unit 26 scratch tape with four scratch bots, an AAPL bot filled 47 scrums and 187 folds in fractional units, and a bot of no class and a A15 bot on an uncited venue each walked the same 234 trades with nothing refused. After, on the same tape: the AAPL bot walks as stocks on Alpaca under the fractional rule, 234 trades, and the A15 bot on Coinbase walks 234 trades identical to unit 26's reading over the same tape, trade for trade; the bot of no class, the A15 bot on Gemini and a TRX bot on the Yahoo route are each refused with their line. With the stocks row planted whole for one run, the AAPL bot walked 233 trades, every one a whole number of units between one and ten, every dollar figure equal to the units times the price, two scrum latches and two fold latches unfilled below one unit, and the A15 bot unchanged; with the crypto row planted whole instead, the A15 bot at a one thousand dollar target in a hundred dollar asset opened ten units and filled nothing, because its one percent interval never reached one unit, and the AAPL bot was unchanged. Each plant was removed and the table read as before.

On the running Sim tab in both builds, one A15 bot and one AAPL bot imported through Import Live Fleet in Validation over scratch tablets, `back_test.run` under the debugger with a breakpoint on every pure sizing function: the rule was read twice, once per bot, from the runner; 470 sizing calls, 94 scrum sizes and 376 fold sizes, every one with `walk` on the stack; the A15 trades equal to unit 26's reading; under the planted stocks row the AAPL trades all whole and the plan trimmed on all 185 folds. Two bots imported on Gemini were refused and their two lines, drawn through the tab's own log method, read back off the Qt widget and off the React page. No bot was constructed and no socket left loopback. The Live replay over unit 26's copied logs with the fractional rule answered 270 of 270 scrum fills and 61 of 61 fold fills, and the same over a fresh copy taken today; one row with its amount moved by one unit in the eighth place read as a disagreement.

## The trade parity report every run writes

Every Simulator run writes one report file when it ends: a Validation run, a Back Test run, a Portfolio Battery run. The gate-light comparison lives in that file and nowhere on the tab. The report names what the run verified and what it could not, and it carries the run's budget under the funding rule and every symbol the run refused. Nothing on the Sim tab draws a light, a table or a verdict for a run; when a run ends the Activity Log carries one line naming the report's path, and the operator opens the file from that path.

`src/simulator/parity_report.py` — the writer

```
def write_report(
    mode: str, run: object, tablets: Optional[TabletSource]
) -> ParityReport:
    """Write the report of a finished ``run`` in ``mode`` and answer where it
    landed."""
    stamp = utc_stamp()
    return write_figures(FIGURES[mode](run, tablets, stamp), stamp)
```

Diagram: flowchart, left to right

- validation.run, back_test.run or portfolio_battery.run_battery -> **the pass over the tablets**
- the pass over the tablets -> **one dict of figures**
- one dict of figures -> **the Markdown file**
- one dict of figures -> **the JSON sidecar**
- the Markdown file -> **reports/simulator under the log root**
- the JSON sidecar -> **reports/simulator under the log root**
- validation.run, back_test.run or portfolio_battery.run_battery -> **the run object carries ParityReport**
- the run object carries ParityReport -> **log_report on the Sim tab: one Activity Log line naming the path**

### Where the report lands

The file lands under the `reports` bucket of the log root, in a `simulator` directory of its own, beside the version-sweep reports the bucket already holds. One run writes one Markdown file and one JSON sidecar with the same name. The name is the mode, the subject, the span and a UTC stamp to the microsecond, joined by two underscores, so two runs never share a name and no report is ever overwritten: the Markdown is created exclusively, and a name already taken moves to the next suffix.

```
validation__coinbase-live__2023-11-14_2023-11-17__20260918T052817474607Z.md
validation__coinbase-live__2023-11-14_2023-11-17__20260918T052817474607Z.json
back_test__coinbase-live__2023-11-14_2023-11-17__20260918T052820763009Z.md
portfolio_battery__DEGEN__2022__20260918T052822879951Z.md
```

The subject is the exchange and the fleet's origin for Validation and Back Test, and the portfolio's name for a Battery, or `every-portfolio` when every portfolio ran. The span is the first and last day of the tape the run read, or the Battery's own span label.

`src/simulator/parity_report.py` — no report is overwritten

```
def create_pair(directory: Path, stem: str) -> tuple[Path, Path]:
    """The first ``stem`` under ``directory`` whose Markdown and JSON paths both
    do not exist, the Markdown created exclusively so no report is overwritten."""
```

### What the report holds

Both files carry the same figures. The Markdown is for a person; the JSON is for a program, and a later unit reads the four comparison counts from it. The two cannot disagree on a number, because the Markdown is rendered from the same dictionary the JSON stores.

Every mode carries these sections:

- **Run.** The mode, the UTC stamp, the build, the fleet's origin, the exchange, the span, the bot count, and the funding with the budget under it. Validation and Portfolio Battery fund from the held Target Balances: the budget is their sum and no fold is capped for cash. Back Test funds each bot's fold from its own scrum proceeds and carries no run budget. A Validation run states that it reruns gates and sizes no trade, so no buy was refused for cash.

- **Tablets.** Every Stone Tablet the run read: its key, asset, exchange, timeframe, year, candle count, first and last candle, and the checksum the MANIFEST row carries.
- **Bots.** One row per bot: symbol, Target Balance, origin, class, venue, the unit rule the pair trades under and whether the pair has a cited row, the outcome, and what the run produced for it. Validation carries the snapped and unsnapped trade counts and the rows compared and latching; Back Test carries the trades, scrums, folds, units gained, cash and fees; the Battery carries each symbol's capital and its weight of the portfolio, the trades, the units gained, and the buy-and-hold baseline beside the accumulation.
- **Not verified.** One line per thing the run could not verify. The section is never empty: a run that verified everything says so in one line.
- **Lines.** The run's own Activity Log lines, verbatim.

### The gate-light comparison

A Validation report carries every rerun row: the trade, the bot, the trade's stamp, the candle's and the gate row's, whether the row latches identically, how many of the nineteen lights agreed, the armed flags on both sides, every disagreeing light with its recorded state, its rerun state and which side drove it, and any blocker phrase that maps to no light. Above the rows sit the agreement counts and four counts under the words the module comparison uses, applied to the two sides of one gate row: the record the live bot wrote, and the rerun the Simulator latched.

| count | what it reads |
|---|---|
| missing | a blocker the recorded row names that maps to no gate light |
| extra | a blocker the rerun raised that maps to no gate light |
| differs | a light in both whose recorded and rerun states differ |
| variant differs | a row whose nineteen lights all agree and whose armed flags differ |

Each count is written beside its definition, in both files.

`src/simulator/parity_report.py` — the four counts

```
def comparison_counts(rows: Sequence[RowComparison]) -> dict:
    """``summarise`` over ``rows`` and the four ``COUNT_NAMES``."""
    counts = dict(summarise(rows))
    counts[MISSING] = sum(len(one.unknown_recorded_blockers) for one in rows)
    counts[EXTRA] = sum(len(one.unknown_rerun_blockers) for one in rows)
    counts[DIFFERS] = sum(len(one.disagreed) for one in rows)
    counts[VARIANT_DIFFERS] = sum(
        1 for one in rows if not one.disagreed and not one.latches_identically
    )
    return counts
```

### The Portfolio Battery section

A Battery report carries, per portfolio and timeframe, how many symbols ran, the capital committed, the capital missing and its share of the weight with the symbols that carry it, the arithmetic of accumulation less baseline with the verdict word, the units gained, the trades and the fees; then every refused symbol with the line the run wrote for it. At unit 25 the defend, improve or reverse outcome against the portfolio's historical path read `not computed`, with why: the design intent's default Target Balance, mix share and reading were the next Battery unit's, and the run walked every symbol at the same capital with equal weights. Unit 24 wrote the reading (comment 5725991788); the section "Portfolio Battery Mode" describes it.

Since the Battery flow landed, that outcome is computed. The historical path is buy and hold over the span: its start is the Target Balances committed, its trough is the sum of each symbol's lowest untraded value on a walked bar, and its end is the baseline. The scrummed path ends at the accumulation. The reading is the highest of the three marks the scrummed end clears, and the report carries it per portfolio and timeframe with the four figures written out, then counts the readings per timeframe.

`src/simulator/portfolio_battery.py` — the reading

```
def reading_for(
    ran: bool, start_usd: float, trough_usd: float, end_usd: float, scrummed_usd: float
) -> str:
    """The highest historical mark ``scrummed_usd`` clears: ``REVERSED`` at or
    above ``start_usd`` when ``end_usd`` fell below it, ``IMPROVED`` above
    ``end_usd``, ``DEFENDED`` above ``trough_usd``, else ``UNDEFENDED``;
    ``NOT_RUN`` while ``ran`` is False."""
    if not ran:
        return NOT_RUN
    if end_usd < start_usd and scrummed_usd >= start_usd:
        return REVERSED
    if scrummed_usd > end_usd:
        return IMPROVED
    if scrummed_usd > trough_usd:
        return DEFENDED
    return UNDEFENDED
```

The report's header names the run's budget as the sum of the run's bots' Target Balances, the origins those bots carry, and, per portfolio, whether the bots were generated or loaded from the held fleet.

### A run that raises

A run that raises part-way still writes a file. It carries the header the run had, every bot it was handed, the tablets if the source still answers, the exception's type and message, and a Not verified section saying nothing was verified. Then the exception propagates as it did before. A report that cannot be written raises to the caller, so a run whose report is missing is never reported as done.

`src/simulator/validation.py` — the wrap every runner carries

```
    try:
        outcome = _validate(bots, tablets, ytd, gates, exchange_id, limit, lag_sample)
    except Exception as exc:
        write_partial(
            VALIDATION, exc, bots=bots, exchange_id=exchange_id, tablets=tablets
        )
        raise
    return replace(outcome, report=write_report(VALIDATION, outcome, tablets))
```

### The line on the tab

Each Sim host answers `log_report`, which writes one line through the tab's own Activity Log at the level Import Live Fleet uses. At unit 25 no button started a run on the tab; unit 24's Run Portfolio and unit 31's Start call it with the run's report (comments 5725991788, 5733262015).

Since the Battery flow landed, Run Portfolio and Run Every Portfolio start a run from the corner and the card and call it with the run's report when the run ends.

```
Portfolio Battery report written: <log root>/reports/simulator/portfolio_battery__CRYPTO_BLUE__2022__20260918T063630643184Z.md
```

```
Validation report written: <log root>/reports/simulator/validation__coinbase-live__2023-11-14_2023-11-17__20260918T052817474607Z.md
```

### What the report reading measured

Before this, over a scratch home holding four stored bots, three Stone Tablets, three RA-StoneTablets, three YTD trade files and a gate.log of six rows, the three runners answered their lines and the reports bucket held no file after all three. After, each run wrote one Markdown file and one JSON sidecar under the simulator directory of the bucket, and running each mode twice wrote twelve files with none overwritten. The Validation sidecar's four counts read equal to the Markdown's. With a raise planted inside the tablet read, each mode wrote a partial file naming the planted exception and the exception still propagated. With one rerun row rewritten to disagree on one light, another rewritten to agree on every light with its scrum flag flipped, and a third given one recorded and two rerun blocker phrases that map to no light, the counts moved from missing 0, extra 0, differs 40, variant differs 0 to 1, 2, 18 and 1, in both files. Every scratch file hashed equal before and after every run, and a second copy with one byte flipped read as changed.

On the running Sim tab in both builds, the four bots imported through Import Live Fleet, each runner driven over the tab's own fleet source and tablet source under the debugger with a breakpoint on the writer: the writer was reached once per run with the runner's own `run` on the stack, the file appeared under the scratch reports directory, the sidecar's counts read equal to the Markdown's, and the Activity Log line naming the path was drawn through the tab's own log and read back off the Qt widget and off the React page. No bot was constructed and no socket left loopback.

## The Simulator colour distinction is the theme's nigredo tone

The Sim tab paints every ground darker than Live's, under the same theme. The tone is the theme's own ground tokens moved a quarter of the way to black; the accent, the text and the borders are the theme's. The derivation, its one fraction and the contrast arithmetic per theme are on the Settings page under the Simulator's tone, nigredo.

Nothing on the tab names a colour for this. The Qt build marks the tab with one property, and the theme's stylesheet paints the tree it marks; the windows the tab opens are its children, so the chooser and the Bot Settings window paint in the tone too.

`src/gui/simulator/sim_trading_tab.py` — the mark

```
# The theme's nigredo_qss paints this tree, and the windows it parents.
# A QWidget subclass paints its stylesheet ground only with this attribute.
self.setProperty(TONE_PROPERTY, NIGREDO)
self.setAttribute(Qt.WA_StyledBackground, True)
```

The React build marks its host the same way for the Qt windows it parents, and its page takes the tone twice: once at build, when `panel_html` hands the tone to the page chrome, and once per theme switch, when the window's repaint reads the tone off the web view. The design tokens the page embeds go through the same function: every token whose name, or whose alias target, starts with `SURFACE_` is a ground the page paints, and the host darkens it before the page reads it.

`src/gui/simulator/sim_react_trading_tab.py` — the page side

```
GROUND_NAME_PREFIX = "SURFACE_"

def nigredo_design(model: dict) -> dict:
    aliases = model.get("alias_targets", {})

    def is_ground(name: str) -> bool:
        return name.startswith(GROUND_NAME_PREFIX) or str(
            aliases.get(name, "")
        ).startswith(GROUND_NAME_PREFIX)
```

```
return page_html(
    STYLE_ASSETS, (), page_body(), theme, (host_script(built, venues),), NIGREDO
)
```

The two Sim windows drawn by React, the Bot Settings window and the wizard, hand the same tone to their page chrome and carry it on their web views.

### What the tone reading measured

The real window was built in each build over a scratch home with every socket but loopback refused, the theme switched five times through the Theme menu's own path, and the picture of the whole window grabbed with Live in front and again with Sim in front. The ground is the tab's own pixel at its top-left corner; the heading is the most common colour inside the Activity Log label, and the label's second colour is its text. At 1400 by 900, the window's floor:

```
theme              point            Live      Sim       Qt         Live      Sim       React
cyberpunk_dark     ground           #0a0a0f   #08080b   differs    #0a0a0f   #08080b   differs
cyberpunk_dark     heading ground   #0a0a0f   #08080b   differs    #0a0a0f   #08080b   differs
cyberpunk_dark     heading text     #00ffcc   #00ffcc   same       #00ffcc   #00ffcc   same
neon_light         ground           #f5f5fa   #b8b8bc   differs    #0a0a0f   #08080b   differs
neon_light         heading ground   #f5f5fa   #b8b8bc   differs    #f5f5fa   #b8b8bc   differs
classic_terminal   ground           #0a0a0a   #080808   differs    #0a0a0f   #08080b   differs
classic_terminal   heading ground   #0a0a0a   #080808   differs    #0a0a0a   #080808   differs
minimal_modern     ground           #fafafa   #bcbcbc   differs    #0a0a0f   #08080b   differs
minimal_modern     heading ground   #fafafa   #bcbcbc   differs    #fafafa   #bcbcbc   differs
glass_metal        ground           #1c1c24   #15151b   differs    #0a0a0f   #08080b   differs
glass_metal        heading ground   #1c1c24   #15151b   differs    #1c1c24   #15151b   differs
```

Every Sim value is the Live value beside it through `toward_black` at 0.25. The same twenty rows read the same at 1920 by 1080. The heading text reads `#00ffcc` on every row in both builds, so the tone moved no text.

The React ground column is the venue page's own ground, which is the design token `SURFACE_0` under every theme on Live and its darkened value on Sim; the page's outer ground, read at the web view's corner, follows the theme as the Qt ground does and darkens the same way. That split between the page's two grounds is Live's own, stated on the tabs page under the theme menu, and the tone keeps it.

The header strip sits above every tab and takes no tone. Its ground read the theme's own value with Live in front and with Sim in front, on every theme, in both builds:

```
cyberpunk_dark  #0a0a0f   neon_light  #f5f5fa   classic_terminal  #0a0a0a
minimal_modern  #fafafa   glass_metal #1c1c24
```

The windows the Sim opens read the tone at their own corner, both builds, every theme; the React Bot Settings window's page and the React wizard's page each read the same value as their frame:

```
theme              chooser   Bot Settings   wizard
cyberpunk_dark     #08080b   #08080b        #08080b
neon_light         #b8b8bc   #b8b8bc        #b8b8bc
classic_terminal   #080808   #080808        #080808
minimal_modern     #bcbcbc   #bcbcbc        #bcbcbc
glass_metal        #15151b   #15151b        #15151b
```

The built bundles read the same way. Each variant was built, launched over an empty scratch home with every HTTP route pointed at a refused loopback port and Chromium's resolver mapped away, its window found by process id and sized to the floor, and Live then Sim brought to the front by a posted click on the tab bar. Off a capture of that window, under the stored default theme:

```
bundle   Live ground   Sim ground   Live heading text   Sim heading text   strip card, Live and Sim in front
qt       #0a0a0f       #08080b      #00ffcc             #00ffcc            #16162a  #16162a
react    #0a0a0f       #08080b      #00ffcc             #00ffcc            #12121a  #12121a
```

Two plants proved the reading can fail. With the fraction set to 0, every Sim value read equal to Live's, twenty rows per build. With the fraction set to 1, every Sim ground and every window read `#000000`. The plant was removed and the module compared byte for byte with its pre-plant digest. The scratch copy of the fleet file read

the same digest before and after every one of the ten readings; a copy with one byte appended read a different digest, so the comparison reports a change.

Before this change the same reading gave every Sim value equal to Live's, under every theme, in both builds: no distinction reached a pixel.

### The skin dictionary carries the tone too

The `SKIN` dictionary the surface serves puts its two ground entries through the same function, so a reader of the served payload gets the tone the tab paints. The block under the skin tokens the sheet reads shows the ground entry as `ds.SURFACE_0`; it reads the darkened value now.

`src/gui/main_tabs/simulator_tab_surface.py` — the two ground entries

```
SKIN = {
    "--sim-ground": toward_black(ds.SURFACE_0, NIGREDO_FRACTION),
    "--sim-chart-ground": toward_black(ds.SURFACE_CHART, NIGREDO_FRACTION),
```

### The renders

One pair per theme, Live beside Sim, in each build, at 1400 by 900, under `artifacts/u117/u27/` in the repository's ignored artifacts directory. The fraction each was taken at is 0.25, and it is one number in the theme engine.

Back to the subsystem index. ## The replay layer draws the chosen tablet and retrieves through a read-only connector

The second layer behind the Indicator Voting Panel holds the VWAP window over the Stone Tablet Playback window, and both draw the tablet the chooser names. The layer's header row, where the flip button sits, carries `Tablet:`, a chooser and one retrieval button, in both builds. The flip reads `Replay` on the panel's header and `Indicators` on the layer's: under the Classic Terminal theme the panel title leaves the flip 90 px at the window's floor, and `Replay` draws whole there under every theme. The chooser lists every tablet on disk by its file key and every held market that has no tablet, and starts on the selected bot's tablet by the rule the panel follows. The two windows draw the last hundred candles of the chosen tablet: the close line and the VWAP line above, one wick and one body per candle below. With no tablet chosen, or a tablet under thirty candles, both windows draw empty.

`src/gui/main_tabs/simulator_tab_surface.py` — the chooser's items

```
def tablet_choices(source: TabletSource, bots: Sequence[Any] = ()) -> list[dict]:
    """The tablet chooser's items: one per MANIFEST row, keyed by
    ``tablet_key`` and shown as it, then one per held market in ``bots`` with
    no tablet at ``NATIVE_TIMEFRAME``, keyed by ``market_key`` and shown with
    ``NO_TABLET_TEXT``."""
```

`src/gui/main_tabs/simulator_tab_surface.py` — the two windows' payloads

```
def replay_feed(source: TabletSource, key: str) -> dict:
    """The two windows' payloads for the item ``key`` names: ``vwap_payload``
    and ``playback_payload`` over ``window_of`` the tablet, empty with no
    entry or under ``MIN_CANDLES``, with ``entry`` and ``refusal`` beside them."""
```

The Qt tab draws the feed on `LineView` and `PlaybackView` through `set_payload`; the React host pushes it as the tab's `replay` and the page draws two SVG windows from the same points and shapes. Both paint the same five colours, read off `SKIN` through `replay_colours`.

### When the windows draw

The feed runs at build, on every `fleet_changed`, on the flip to the layer, on the chooser's change, when the panel's bot changes, and when a retrieval ends. No timer feeds it: a tablet on disk does not move.

`src/gui/simulator/sim_trading_tab.py` — the feed

```
    def _feed_replay(self) -> dict:
        feed = surface.replay_feed(self._tablet_source, self._tablet_key)
        self._replay = feed
        self._vwap_view.set_payload(feed["vwap"])
        self._playback_view.set_payload(feed["playback"])
        self._retrieve_button.setText(feed["button_text"])
```

### The retrieval button

The button reads `Retrieve Tablet` while the chosen item has no tablet on disk and `Update Tablet` while it has one. A press retrieves the chosen market at the native five-minute timeframe: from the fetcher's own start, `YTD_START_MS`, when no tablet exists, or from the tablet's last candle when one does, up to the newest closed candle. The candle now forming is left out, because the registry never replaces a timestamp it already holds.

`src/simulator/tablet_retrieval.py` — the span

```
def retrieval_span(entry: Any, now_ms: Optional[int] = None) -> tuple[int, int]:
    """``(since_ms, until_ms)`` for one press: ``YTD_START_MS`` with no
    ``entry``, else one step past ``entry.last_ts_ms``; up to
    ``closed_until_ms``."""
```

The walk runs on a daemon thread, as the Portfolio Battery's does, and never on the GUI thread. It goes one adapter chunk at a time, each chunk one `download_missing` call over a registry built on the tab's tablet root, and stops at the first chunk the venue refuses. Every write lands under that root and nowhere else.

`src/simulator/tablet_retrieval.py` — the walk

```
    registry = StoneTabletsRegistry(root)
    span = int(adapter.chunk_span_ms)
    cursor = int(since_ms)
    while cursor <= int(until_ms):
        chunk_end = min(cursor + span - STEP_5M_MS, int(until_ms))
        reports = await download_missing(
            [(outcome.asset, outcome.exchange_id)],
            connector,
            cursor,
            chunk_end,
            registry=registry,
        )
```

### The read-only connector

The connector the walk reads through is the Simulator's one venue path. It holds `CoinbasePublicCandles`, the shipped reader of the venue's public candle endpoint, which carries no key and reaches no account or order endpoint. It answers `get_ohlcv` and nothing else; every other name raises `SendRefused`, the shape `TabletSource` and `FleetSource` have. The Live connector is not used: it requires a key to connect, carries `place_order`, and records on the process-wide API log.

`src/simulator/read_only_connector.py` — the refusal

```
    def __getattr__(self, name: str) -> Any:
        """Refuse every name outside ``READ_NAMES``."""
        raise SendRefused(
            f"ReadOnlyConnector answers {READ_NAMES} and cannot {name!r}. "
            "The Simulator receives and asks; it sends nothing."
        )
```

`get_ohlcv` pages the reader by the venue's three-hundred-row page so a chunk of three hundred and fifty candles comes back whole.

### What the two spools show

The Activity Log carries the started line naming the tablet and the span, one progress line per venue call naming the rows received and their span, and the finished line naming the candles appended, the chunks walked and the file. A refused venue writes one line naming the error and that nothing more was written. A second press while a walk is in flight writes one line and starts nothing.

The API Interaction Log carries one block per venue call, in Live's block: the exchange and `FETCH_TABLET` with the tablet's key, a Reason line naming the candles asked and their start, the public endpoint path, the rows received and the latest close, the response time, and the file the rows were appended to. A refused call carries the error as its result. Each call crosses from the worker thread on a Qt signal and is recorded on the GUI thread, which is what the API log writer's thread rule requires.

`src/gui/simulator/sim_trading_tab_surface.py` — the block's fields

```
RETRIEVAL_ACTION_FORMAT = "FETCH_TABLET {key}"
RETRIEVAL_REASON_FORMAT = (
    "Get {limit} candles ({timeframe}) for {symbol} since {since} - "
    "Stone Tablet {verb}"
)
RETRIEVAL_RESULT_FORMAT = "{rows} candles received, latest close={close}"
```

### What the replay reading measured

Both builds, the real window with a scratch home, every socket but loopback refused, a scratch tablet root holding the A15 and A14 5m 2026 coinbase tablets and a held fleet of the A15, A14 and A23 coinbase bots. With the A15 bot selected and `Replay` pressed, the chooser listed the two tablets and `A23_5m_coinbase — No Stone Tablet on disk.`, started on the A15 tablet, and both windows' payloads hashed equal to `vwap_payload` and `playback_payload` over the same hundred candles; moved to A14, both redrew. A retrieval of A23 against a loopback candle server, the reader's endpoint pointed at it, wrote `A23_5m_2026_coinbase.json` and its MANIFEST row under the scratch root, drew the windows from it, and recorded one API block per call; a second press updated from the last candle; the endpoint restored to the venue and the socket refused, the press wrote the refusal line and the root's files hashed identical. `bot_state.json` and the two copied tablets read byte-identical after every step. No bot was constructed and no socket left loopback.

## The API spool shows Stone Tablet and YTD retrieval activity only

The Sim tab's API Interaction Log carries two kinds of block and refuses every other, in both builds. The log each host builds beside its two sources is the Simulator's own kind, `SimApiLog`, Live's log with one rule added: an entry is accepted when its action opens with one of two words, and any other action is refused before the entry exists, before the file line is written and before the pane is told. The two words are the two recorders the directive names. A Stone Tablet retrieval or update records one block per venue call, and Generate From YTD records one block per read of the trade files. Nothing else records on the Sim's log, and a venue order cannot reach the pane.

`src/simulator/sim_api_log.py` — the allowed set

```
#: The word a Stone Tablet retrieval's action opens with; the tablet key follows.
TABLET_ACTION = "FETCH_TABLET"

#: The word the Generate From YTD read's action opens with.
YTD_ACTION = "FETCH_YTD"

#: Every action word ``SimApiLog.record`` accepts. Any other raises ``SendRefused``.
ALLOWED_ACTIONS = (TABLET_ACTION, YTD_ACTION)
```

`src/simulator/sim_api_log.py` — the refusal

```
class SimApiLog(APIInteractionLog):
    """``APIInteractionLog`` whose ``record`` accepts ``ALLOWED_ACTIONS`` only."""

    def record(
        self,
        exchange: str,
        action: str,
        reason: str,
        endpoint: str = "",
        params: dict = None,
        result: str = "",
        elapsed_ms: float = 0.0,
        level: str = "info",
        data_usage: str = "",
    ) -> dict:
        if not action_allowed(action):
            message = REFUSED_FORMAT.format(action=action, allowed=ALLOWED_ACTIONS)
            logger.warning(message)
            raise SendRefused(message)
        return super().record(
```

### Three ways a venue order cannot reach the pane

The object refuses. A record whose action is not a tablet retrieval or a YTD read raises the same refusal the tab's sources raise, appends nothing, writes no file line and calls no listener, so nothing is drawn on either build; one warning line names the action refused. The process-wide log is a different object. The venue connectors record on the log the Live tab listens to, and the Sim's writer listens to the Sim's log alone, so an entry recorded on the venue log draws on Live's pane and not the Sim's. The callers are the allowed set. Four places under the Simulator's tab code record on the Sim's log, two per build, and they carry the two allowed words and nothing else.

Diagram: flowchart, left to right

- Update Tablet or Retrieve Tablet, one venue call -> **FETCH_TABLET key**
- Generate From YTD, one read -> **FETCH_YTD**
- FETCH_TABLET key -> **SimApiLog.record**
- FETCH_YTD -> **SimApiLog.record**
- any other action -> **SimApiLog.record**
- log -- allowed -> **the Sim API Interaction Log pane**
- log -- refused -> **SendRefused, nothing drawn**

### The Generate From YTD block

A Generate From YTD press that reads the trade files records one block on the Sim's API Interaction Log, in Live's block, after the read and before the Activity Log's generation line. The head line carries the exchange chosen and the action. The Reason line names the read. The Endpoint line is the exchange history directory the files were read from. The Result line counts the fills and the files and names every file read. The Response line is the time the read took. The Data usage line counts the bots the read generated, and adds the manifest rows skipped when a row's file is missing. A press that stops before a read, because the directory is missing, empty or without its manifest, because the manifest names no pair, or because the chooser was cancelled, records no block and writes its Activity Log line alone.

```
[05:51:06] COINBASE FETCH_YTD
  Reason: Read the YTD trade files for coinbase - Generate From YTD
  Endpoint: <the exchange history directory>
  Result: 10 fills read from 4 file(s): A15-USD_2025_coinbase.json, A15-USD_2026_coinbase.json, A14-USD_2026_coinbase.json, A23-USD_2026_coinbase.json
  Response: 2.8ms
  Data usage: Generated 3 bot(s) on the Scrumming Bots table
```

`src/gui/simulator/sim_trading_tab_surface.py` — the block's fields

```
YTD_REASON_FORMAT = "Read the YTD trade files for {exchange} - Generate From YTD"
YTD_RESULT_FORMAT = "{fills} fills read from {files} file(s): {names}"
YTD_DATA_USAGE_FORMAT = "Generated {bots} bot(s) on the Scrumming Bots table"
YTD_NOTHING_GENERATED_TEXT = "Nothing generated"
YTD_MISSING_USAGE_FORMAT = "; {missing} manifest row(s) skipped, file missing"
```

`src/gui/simulator/sim_trading_tab.py` — the record

```
        started = time.perf_counter()
        made = self._fleet_source.generate_from_ytd(source, chosen)
        elapsed_ms = (time.perf_counter() - started) * 1000.0
        self._api_log.record(
            **tab_surface.ytd_api_entry(made, chosen, source.root(), elapsed_ms)
        )
```

The file names and the fill count come off the generation itself, which now answers the names of the files it read and the fills it read beside the bots it held and the rows it skipped.

`src/simulator/fleet_source.py` — what one generation answers

```
@dataclass(frozen=True)
class YtdGeneration:
    bots: tuple[SimBot, ...] = ()
    files_read: int = 0
    missing: tuple[YtdFileEntry, ...] = ()
    files: tuple[str, ...] = ()
    trades_read: int = 0
```

### The Stone Tablet block keeps its text

A retrieval's block is the block the replay layer's section above describes, unchanged in every field; its action now opens with the shared word rather than a copy of it, so the word has one definition.

`src/gui/simulator/sim_trading_tab_surface.py` — the action

```
RETRIEVAL_ACTION_FORMAT = TABLET_ACTION + " {key}"
```

### What the API spool refusal reading measured

Both builds, the real window with a scratch home, every socket but loopback refused, scratch YTD trade files in the shape the generation reading above drove, copies of the A15, A14 and A22 tablets, copies of the SPY and GLD RA tablets, and a copy of `bot_state.json`. One session: Generate From YTD with coinbase chosen drew one block whose Result line named the four files and the ten fills, whose Response line carried the read's time, and whose lines read the same on both pages apart from the stamp and the path; Update Tablet on A22 against a loopback candle server drew forty tablet blocks; Import Live Fleet drew none; Portfolio Battery over EQUITY_MACRO drew none. After the session the pane held 41 blocks, and 41 of them opened with an allowed word. A venue order recorded on the Sim's log raised the refusal and drew nothing, on either build; the same entry on the process-wide log drew on Live's pane and not the Sim's. Live's 37 action names recorded on the Sim's log were refused 37 of 37; with the order word planted into the allowed set the count read 36, and with the plant removed it read 37 again. One tablet entry recorded directly drew. On the base commit the same session drew no block for Generate From YTD, the order record drew, and the 37 names drew 37 of 37. Every copied file and every trade file read byte-identical after every step, and a byte planted into a copy of `bot_state.json` moved its hash. No socket left loopback.

## The Activity spool shows the trades of Validation, Back Test and Portfolio Battery runs

Every simulated fill writes one line on the Sim tab's Activity Log, in both builds, as the walk produces it. The line is Live's fill line as Live's pane receives it: the bot prefix Live's window puts on every bot line, then `SELL FILLED` for a scrum or `BUY  FILLED` for a fold, the units to six places, the asset, the price to eight places. The Simulator has no venue, so it has no intended price and no slippage to report; in their place the line carries what the walk knows about the fill, the dollars and the fee to five places, in the shape Live's proceeds line writes them. A scrum's dollars read `gross`, the notional sold; a fold's read `spent`. The line is written at Live's `info` level and paints at the pane's plain size, because that is what Live's fill line paints at.

`src/gui/simulator/sim_trading_tab_surface.py` — the line one `SimTrade` writes

```
TRADE_LINE_LEVEL = "info"
BOT_PREFIX_FORMAT = "[{asset}/{tail}] "
BOT_ID_TAIL_CHARS = 4
TRADE_LINE_FORMAT = (
    "{prefix}{word}: {units:.6f} {asset} @ ${price:.8f} "
    "({usd_word} ${usd:.5f}, fee ${fee:.5f})"
)
SCRUM_FILL_WORD = "SELL FILLED"
FOLD_FILL_WORD = "BUY FILLED"
SCRUM_USD_WORD = "gross"
FOLD_USD_WORD = "spent"
```

Unit 31a moved the fill words to `src/simulator/sim_bus.py` (comment 5734916119).

```
[2021-02-05T00:00:00Z] [SPY/ahoo] SELL FILLED: 0.193792 SPY @ $387.70999146 (gross $75.13498, fee $0.45081)
[2026-07-29T15:35:00Z] [A03/0bda] BUY FILLED: 155.417407 A03 @ $0.02252000 (spent $3.50000, fee $0.05600)
```

### The stamp is the tape's time

Every other line on the pane is stamped with the clock at the moment it is written, as Live's pane stamps it. A simulated fill happened at the tape's candle, and a daily tablet's candles all fall at midnight, so a clock stamp would read the same on every line of a daily walk and say nothing about when the fill fell. A trade line is stamped with the candle's time instead, the same UTC stamp the report writes for a run's first and last candle. Both forks of the pane gain one call for it, a write under a stamp the caller gives, which holds the line under a pause exactly as an ordinary write does and renders it otherwise.

`src/gui/simulator/sim_status_log.py` — the write under a given stamp

```
        def log_at(self, stamp: str, message: str, level: str = "info") -> None:
            """``log`` under ``stamp`` instead of the clock: held under a pause, else rendered."""
            self._tell("log", message, level)
            if self._paused:
                if len(self._pause_buffer) < self._pause_buffer_cap:
                    self._pause_buffer.append((stamp, message, level))
                return
            self._render(stamp, message, level)
```

### The seam the walk hands each fill to

The walk that sizes every simulated trade gains one callback, handed each `SimTrade` the moment it fills, the way the Battery hands its progress callback each portfolio's line. Every runner threads it through to the walk: Back Test's run, the Battery's symbol, portfolio and battery runs, and Validation's run. A symbol two portfolios share on one bot is walked once, so its fills reach the callback once.

`src/simulator/back_test.py` — the seam

```
#: The seam ``walk`` hands each ``SimTrade`` to as it fills, as ``run_battery``
#: hands ``progress`` each portfolio's line.
TradeSink = Callable[[SimTrade], None]
```

```
        if filled is not None:
            trades.append(filled)
            fees += filled.fee_usd
            if on_trade is not None:
                on_trade(filled)
```

### A Validation run's trades are the recorded fills it reruns

Validation sizes no trade of its own; it reruns the recorded fills against the gate rows the live bot wrote. Its trade line is that recorded fill, one per rerun: the bot, its symbol, a scrum for a sell and a fold for a buy, the fill's time, price, amount, cost and fee. A fill the run does not rerun writes no line; the report names it under what could not be verified.

`src/simulator/validation.py` — the recorded fill as a trade

```
def rerun_trade(bot: SimBot, fill: Any) -> "SimTrade":
    """The ``SimTrade`` one rerun YTD fill reads as: ``SCRUM`` for ``SIDE_SELL``
    and ``FOLD`` otherwise, ``amount`` as the units, ``cost`` as the USD."""
```

### Off the worker thread, as the portfolio lines travel

A Battery run walks on its worker thread. Each fill crosses to the GUI thread through a signal beside the one that carries each portfolio's line, and the host writes it there. On the Qt tab the line is appended to the pane as it arrives. On the React page the host paints it into its own log at once and pushes every line painted since the last push once per strip tick, two seconds, so a walk of thousands of fills does not push thousands of scripts; a push carries the pending lines ahead of anything else it carries.

`src/gui/simulator/sim_trading_tab.py` — the signal and the writer

```
    #: One ``SimTrade`` the Battery's walk filled on the worker thread.
    battery_trade = Signal(object)
```

```
    def log_trade(self, trade: SimTrade) -> None:
        """One Activity Log line per ``SimTrade``: ``trade_line`` under
        ``trade_stamp`` through ``SimStatusLog.log_at`` at ``TRADE_LINE_LEVEL``."""
        self._status_log.log_at(
            tab_surface.trade_stamp(trade),
            tab_surface.trade_line(trade),
            tab_surface.TRADE_LINE_LEVEL,
        )
```

`src/gui/simulator/sim_react_trading_tab.py` — the push cadence

```
#: One push per strip tick carries every trade line ``log_trade`` painted since the
#: last push, so a walk of thousands of fills does not push thousands of scripts.
TRADE_PUSH_INTERVAL_MS = DASHBOARD_TICK_MS
```

### Nothing else reaches the spool during a run

During a run the Activity Log's writers are the run's own: the started line when the press is taken, one line per portfolio as it lands, one line per fill as it fills, and when the run ends its summary lines and the report line. The watchdog writes only when the render-error count rises or when nothing has rendered for ten minutes while a sim bot runs, and a run renders continuously; the command bar's notices are reached only by its own presses; the window's notifications reach Live's pane. The pane holds 5,000 lines and drops the oldest past that; a pause holds 2,000 and drops the newest, and Resume replays the held lines in order under one line counting them.

### Back Test and Validation, through the run

### What the trade-line reading measured

Both builds, the real window with a scratch home holding a copy of `bot_state.json` with 38 coinbase bots, the A03, A11 and A13 tablets, a copy of the whole RA-StoneTablet root, a copy of the gate log and YTD trade files built from the SCRUM and FOLD rows three of those bots wrote; every socket but loopback refused. Import Live Fleet held 38 bots. Back Test through its run walked 3 bots, produced 1,670 fills and wrote 1,670 trade lines and the report line; every line's stamp, prefix, word, units, asset, price, dollars and fee read equal to its fill. One bot at 1,300 candles, a walk of exactly three fills, wrote three lines, each with its fee. Validation through its run rerun 36 recorded fills and wrote 36 lines and the report line. Run Portfolio on BOGLEHEAD over every span, pressed at the corner, wrote 204 trade lines for the 204 fills its nine walks produced, beside the started line, one portfolio line, eight summary lines and the report line, and nothing else; the report's rows counted the same 204. Run Portfolio on FULL wrote its first fills within three seconds of the press with the run still in flight; Pause Console pressed then held every later line, 819 on the Qt tab and 719 on the React page, and Resume flushed them in their held order under the line counting them. Run Every Portfolio over the whole root produced 2,066 fills in 189 walks and wrote 2,066 trade lines beside 47 of the run's own; on the Qt tab the pane read 681 lines at 9.2 seconds of a 27-second run and 2,054 at 27.3, and on the React page 1,573 at 21.5 seconds against 1,587 fills produced. After that run both panes held 4,936 lines under the cap of 5,000. On the base commit the same sessions wrote no trade line in either build. Every copied file read byte-identical after every step, a byte planted into a copy of `bot_state.json` moved its hash, no bot was constructed and no socket left loopback.

## Start runs Validation and Back Test from the command bar

The command bar's Start is the press that starts a run. Live starts a bot with Start on the selected row, and no other press on the Trading tab starts anything, so the Simulator's run control is that same Start. What it starts follows the run mode: in Validation mode it starts the Validation run, in Back Test mode the Back Test walk, and in Portfolio Battery mode it stays the state move the command bar section above describes, because the Battery's run is the corner's. Stop on a row of the run ends it. No new button is drawn.

`src/gui/simulator/sim_trading_tab.py` — the dispatch in `_on_bot_command`

```
        if self.run_running():
            if command == "stop" and bot_id in self._run.get("bot_ids", []):
                self._stop_run(bot_id)
                return
            self._status_log.log(
                tab_surface.run_in_flight_line(
                    self._run.get("mode", ""),
                    len(self._run.get("bot_ids", [])),
                    command,
                ),
                "warning",
            )
            return
        if command == "start" and self._mode in tab_surface.RUN_MODES:
            self._start_run(bot, self._mode)
            return
```

Diagram: flowchart, left to right

- Start on the command bar -> **_on_bot_command**
- _on_bot_command -> **_start_run: every scrumming bot on the page to running, fleet_changed, the started line** *(Validation or Back Test mode)*
- _start_run: every scrumming bot on the page to running, fleet_changed, the started line -> **_compute_run on a daemon thread**
- _compute_run on a daemon thread -> **validation.run over TabletSource, YtdTradeSource, GateLogSource** *(Validation)*
- _compute_run on a daemon thread -> **back_test.run over TabletSource** *(Back Test)*
- validation.run over TabletSource, YtdTradeSource, GateLogSource -> **run_trade: one line per rerun fill**
- back_test.run over TabletSource -> **run_trade: one line per rerun fill**
- validation.run over TabletSource, YtdTradeSource, GateLogSource -> **write_report**
- back_test.run over TabletSource -> **write_report**
- write_report -> **run_finished: _take_run moves the bots to stopped, writes the lines and the report line**
- Stop on a run row -> **_stop_run sets the event the runner reads**

### The run is the page's scrumming fleet

A Validation pass is one pass over a fleet: `validation.run` takes the bots, measures one coverage, one tape lag and one match key over them, and writes one report. Start therefore runs the mode over every held scrumming bot on the pressed page's exchange, `_run_bots` in both hosts, and every one of those rows reads `running` while the run is in flight and `stopped` when it ends. The selected row is the bar's own precondition, `Select a bot first.`, and is not changed. Extractor rows are not run: the gate chains the rerun evaluates are the scrumming bot's, and the walk sizes a scrum and a fold.

`src/gui/simulator/sim_trading_tab.py` — the fleet a run covers

```
    def _run_bots(self, exchange_id: str) -> list[SimBot]:
        """Every held scrumming bot on ``exchange_id``, the fleet a run covers."""
        return [
            one
            for one in self._fleet_source.bots()
            if one.exchange_id == exchange_id and one.mode == SCRUMMING_MODE
        ]
```

`_start_run` moves each of those bots to `running` through `SimBotManager.start`, the same rule Start applies to one bot, and fires `fleet_changed`, so the sim fleet file is saved and every venue page draws its rows again before the walk begins: the Bot ID cell reads `State: RUNNING` in its running colour. Then the started line is written and `_compute_run` starts on a daemon thread named `sim-mode-run`, the shape the Battery and the Market Inspector's Scan Now use, so the window keeps answering. When the worker ends, `_take_run` on the GUI thread moves every run bot to `stopped` through `SimBotManager.stop`, fires `fleet_changed` again, and writes the run's own lines and the report line through `log_report`.

```
Validation started on coinbase over 24 bot(s); budget $16,860.00, the sum of the held Target Balances; no rerun fill is refused for
cash.
Back Test started on coinbase over 24 bot(s); each fold spends its own scrum proceeds.
```

The Validation line names the run's budget under the funding rule: the sum of the held Target Balances, `run_budget_usd`, which is also what the strip's Spendable reads in that mode. Validation reruns gates and sizes no trade, so no fill is refused for cash, and the report's header says so. Back Test keeps the proceeds funding, unchanged.

### What the worker reads

The Validation worker reads the tab's `TabletSource` over the Stone Tablets, a `YtdTradeSource` over the YTD trade files, and a `GateLogSource` over `gate.log` and its rotations under the trade directory of the log root. Each answers reads alone and refuses every other name. The Back Test worker reads the `TabletSource` alone. Neither touches a widget, the fleet source or a venue; the worker's only write is the report.

`src/gui/simulator/sim_trading_tab.py` — the two runners, called as unit 30's seam left them, with the stop event added

```
                outcome = validation.run(
                    bots,
                    self._tablet_source,
                    YtdTradeSource(),
                    GateLogSource(),
                    exchange_id=exchange_id,
                    limit=surface.VALIDATION_RERUN_LIMIT,
                    on_trade=self.run_trade.emit,
                    stop=self._run_stop.is_set,
                )
```

Each rerun fill crosses to the GUI thread on `run_trade` and is written by `log_trade` in the trade-line shape the Activity spool section describes; the run's lines and the failed line cross on `run_line`; the outcome crosses on `run_finished`. The React host carries the same three signals, the same `_start_run`, `_compute_run`,

`_stop_run` and `_take_run`, and reaches them from the page's command bar through Live's own `ExchangeTabModel.cmd`.

### Stop ends the run with a partial report

Stop on a row of the run writes Live's `Stopping bot <id>...`, the run's stopping line, and sets a `threading.Event`. Both runners take `stop`, a callable read before each bot and before each row or tick: `_validate` reads it before each bot's snap and before each row's rerun, `_walk_fleet` before each bot and `walk` before each tick. When it answers True the pass ends where it is, with `stopped` set on the run, and the same `write_report` writes the report: `partial` reads yes with the stop named, the header carries `stopped` and the bots reached, every unreached bot's row reads `not reached`, and the Not verified section leads with the stop line. A stopped walk answers the candles it read, the last close it ticked and the trades it filled. When the worker ends, `_take_run` writes Live's `Bot <id> stopped.` for the row Stop was pressed on, moves every run bot to `stopped`, and writes the lines and the report line as for a whole run.

```
Stopped by the operator: 19 of 24 bots walked, 1 cut short at its last bar ticked; 5 bot(s) not reached: <ids>.
Stopped by the operator: 1 of 24 bots reached, 0 rows rerun; 23 bot(s) not reached: <ids>.
```

While a run is in flight, Start, Pause, Restart and Delete on any row, and Stop on a row outside the run, write one line and move nothing:

```
A Validation run is in flight over 24 bot(s); Stop on one of its rows ends it, and start waits for it.
```

A run that raises writes the failed line, moves its bots to `stopped`, and its partial file is on disk from `write_partial`, as the report section describes.

### Each differing light carries its cause

A non-zero count is a finding, and the report says what drove it. Every disagreeing light already carries which side of the rerun moves it; it now carries a cause as well, one of three words read off the row's own fields.

| cause | what it reads |
|---|---|
| tape | the tablet candle the rerun read differs from the reading the bot recorded: the recorded `bb_pos` sits more than the lag match gap from the rerun's, or the bank's recorded band flag differs from the rerun's |
| fixture | a recorded field the light reads is absent from the row's fixture, or the phantom lock could not be recovered from the recorded row |
| chain | the same inputs on both sides and a different light: the gate chain |

`src/simulator/validation.py` — the cause of one light that is blocked on one side

```
def light_cause(
    light: LabelComparison,
    row: Any,
    context: GateContext,
    locked_known: bool,
    absent: Sequence[str],
) -> str:
    key = (light.bank, light.label)
    if light.driven_by == TAPE:
        if recorded_bb_pos(row) is None:
            return FIXTURE_CAUSE
        if tape_moved(row, context, light.bank):
            return TAPE_CAUSE
        if key == ("S", "BB") and not locked_known:
            return FIXTURE_CAUSE
        return CHAIN_CAUSE
    if light.driven_by == RECORD:
        for fixture, field_name in LABEL_FIELDS.get(key, ()):
            if f"{fixture}.{field_name}" in absent:
                return FIXTURE_CAUSE
        if tape_moved(row, context, light.bank):
            return TAPE_CAUSE
        return CHAIN_CAUSE
    return CHAIN_CAUSE
```

`LABEL_FIELDS` names, per light, the recorded fixture fields `rerun_context` reads for it. A light that is blocked on exactly one side is classified by the rules above. A light blocked on neither side, `passed` against `not_the_blocker`, moved only because its bank's armed flag moved, so it takes the cause of its bank's blocked lights: `tape` before `fixture` before `chain` where they mix, because the tape is the data the criterion is about, and `chain` where no light in the bank is blocked. The report writes the cause beside each disagreeing light, a per-row count of causes, and under the four counts a line `differs by cause: tape N, fixture N, chain N` with the three definitions, in both files. No tolerance is widened and no gate is changed to make a light agree; the chain is Live's.

### The empty cases name the missing source

The run still runs when a source is empty, and the report's Not verified section names what could not be rerun. With no YTD trade directory, the Start press writes the YTD root line before the started line, every bot reads `no YTD trade file; nothing snapped.` and the comparison table holds no row. With no `gate.log`, every bot's snapped fills match nothing, and the bot's own line says so. With no Stone Tablet for a pair, that bot reads `no Stone Tablet; nothing snapped.` and the rest of the fleet is rerun.

```
<id> (AAA/USD): no YTD trade file; nothing snapped.
<id> (AAA/USD): no gate row recorded for this bot; 4 snapped trades matched nothing.
<id> (BBB/USD): no Stone Tablet; nothing snapped.
```

`src/simulator/parity_report.py` — the per-bot line for a bot with no recorded gate row, read off the `gate_rows` count `_validate` fills

```
        elif row["outcome"] == VALIDATED and not row["gate_rows"]:
            out.append(
                f"{row['bot_id']} ({row['symbol']}): no gate row recorded for this "
                f"bot; {row['snapped']} snapped trades matched nothing."
            )
```

### What the run from the bar measured

Before, in both builds over a scratch home holding 24 scrumming bots on coinbase, one 5m Stone Tablet of 2,000 candles per pair, one YTD trade file per pair with four fills, and a `gate.log` of one fired row per fill in the operator's row shape, each row latched by the shipped chain over the tablet candle that holds its fill: Import Live Fleet held 24 rows, and Start in Validation mode wrote ✓ `Bot <id> RUNNING.`, moved the one selected row, wrote no trade line and no report; Back Test the same; the two runners had no caller under the tab.

After, in both builds over the same home: Start in Validation mode moved all 24 rows to `RUNNING` within 40 ms on the Qt tab and within 0.6 seconds on the React page, wrote the started line with the budget of $16,860.00, wrote 96 trade lines as the 96 recorded fills were rerun, wrote the run's six lines and the report line, and moved the rows to `STOPPED`; the report's comparison table read missing 0, extra 0, differs 0, variant differs 0 over 96 rows compared, the sidecar's four counts equal to the Markdown's. Start in Back Test mode moved the 24 rows the same way, walked 48,000 candles in about seven seconds, wrote 17 trade lines for the 17 fills, and wrote its report. Stop pressed on the last row one second into a Back Test wrote the stopping lines and a partial report reading 19 of 24 bots walked, one cut short and five not reached on the Qt tab, 11 of 24 and 13 not reached on the React page; a Start pressed while that run was in flight wrote the in-flight line and started nothing; Stop pressed the moment a Validation run started wrote a partial report reading 1 of 24 bots reached and 0 rows rerun on the Qt tab, 23 of 24 and 0 rows rerun on the React page, and every run bot read `STOPPED` after each.

Over a scratch home whose first bot's rows carried one recorded S/TA light flipped to blocked with its scrum disarmed, one recorded `target_fires` removed, and one recorded reading moved to the opposite band with its lights latched there, plus one fill a day past the tablet's last candle, the table read differs 21 by cause tape 11, fixture 1 and chain 9: the removed field's row read S/FIRE `fixture`; the flipped row read S/TA `chain` with its eight armed-flag lights `chain`; the moved row read S/BB `tape` with its ten dependent lights `tape`; and the fill past the tablet read `after_last_candle` under Not verified with the uncovered span. Both builds read the same 21.

Over three more scratch homes, one with no YTD directory, one with no `gate.log` and one with no tablet for the second pair, the report named each case per bot as the lines above show, the YTD root line preceded the started line in the first, and the third rerun the first pair's four rows to four zeros.

Over scratch copies of the operator's own `bot_state.json`, his `gate.log` and its five rotations, and the 38 Stone Tablets his 38 scrumming bots' assets and timeframes name, with no YTD trade file on this machine: Import Live Fleet held 38 rows and Start in Validation mode wrote the YTD root line, ran, and wrote a report whose 38 bot rows each read `no YTD trade file; nothing snapped.`, whose comparison table read four zeros over 0 rows compared, and whose header named the budget as the sum of the 38 Target Balances. Without the exchange's YTD export nothing can be snapped, so nothing can be rerun, and the report says so rather than reading a pass. With YTD trade files built from the fired rows his own gate log holds, one fill per SCRUM or FOLD row inside its tablet's span, the run rerun 55 recorded fills and the table read missing 0, extra 0, differs 275, variant differs 0, with 23 of the 55 rows latching identically and 770 of 1,045 lights agreeing; every one of the 275 differing lights read `tape`, none `fixture` and none `chain`, because 50 of the 55 recorded

readings were reproduced from a window 50 candles, 250 minutes, behind the fill's own candle, the lag the first full run measured, so the bot read a different candle from the one the tablet holds at the fill's moment and the chain, fed the tablet's candle, lit the lights differently; fourteen bots carried a fired row inside their tablet's span and the other 24 read `no YTD trade file`. Every copied file read byte-identical after every press, a byte planted into a copy of `bot_state.json` moved its hash, no bot was constructed and no socket left loopback.

## The Simulator writes Live's log rows under the sim bucket

Every run writes the rows a live bot writes, into the Simulator's own files, so a run can be troubleshot and tuned from its files as a live bot can from its logs. The operator, 2026-09-18: *"Simulator Logs and Emitter Network must be able to provide sufficient data for troubleshooting and optimization."* Each Sim host builds one private bus at tab build and one second copy of Live's log writer over the sim bucket, attached to that bus. The bus is never the process-wide one, and nothing under the Simulator subscribes to the live bus. The runners emit Live's topics on the private bus, the writer's own listeners write them, and the four files land beside the sim fleet file.

`src/simulator/sim_bus.py` — the bus and the writer one host builds

```
def new_sim_bus() -> EventBus:
    """A private ``EventBus`` for one Sim host, never ``get_event_bus``."""
    return EventBus()

def sim_log_manager(
    bus: Any, symbol_of: Optional[Callable[[str], str]] = None
) -> LogManager:
    """A ``LogManager`` over ``get_sim_dir`` attached to ``bus``, with
    ``symbol_of`` as its symbol resolver when given."""
    manager = LogManager(log_dir=get_sim_dir())
    if symbol_of is not None:
        manager.set_symbol_resolver(symbol_of)
    manager.attach_to_bus(bus)
    return manager
```

`src/gui/simulator/sim_trading_tab.py` — built once, in the tab's `__init__`

```
        self._bus = new_sim_bus()
        self._log_manager = sim_log_manager(self._bus, self._symbol_of)
```

The writer's own attach subscribes the five topics it subscribes on Live, so the sim files carry Live's names and Live's row shapes. One writer, one definition, two buckets.

| topic on the private bus | file under the sim bucket | who emits it |
|---|---|---|
| `trade.filled` | `sim/trade.log` | the walk on every fill, the rerun on every rerun fill |
| `bot.gate_decision` | `sim/gate.log` | the walk on every evaluated candle, the rerun on every rerun row |
| `bot.voting_panel_snapshot` | `sim/voting.log` | the walk on every evaluated candle |

| topic on the private bus | file under the sim bucket | who emits it |
|---|---|---|
| `bot.log` | `sim/diagnostics.log` | the walk on every fill and for every bot it refuses, the rerun for every bot it cannot rerun |
| `pnl.event` | `sim/pnl/daily/` | nothing; the directory is made at build and stays empty |

The writer's console handler is not attached a second time: `main.py` builds Live's writer before the window, so the application logger already holds Live's handler when the tab builds, and the sim writer attaches none. No `sim/system.log` exists, and the live `system.log` is not doubled.

Diagram: flowchart, left to right

- Start on the command bar -> **_compute_run on a daemon thread**
- _compute_run on a daemon thread -> **back_test.run or validation.run with bus=self._bus**
- back_test.run or validation.run with bus=self._bus -> **RunEmitter: the bus and one run id**
- RunEmitter: the bus and one run id -> **walk: one candle evaluated**
- walk: one candle evaluated -> **bus.emit on the private bus**
- bus.emit on the private bus -> **LogManager listener**
- LogManager listener -> **one row appended to sim/gate.log**

### Every row carries its run id and its candle

Each run takes one id before its first row, `<mode>-<stamp>`, and every gate row, voting row and diagnostics row of the run carries it. The candle's time is on every row as well. A row's top-level `timestamp` is the wall clock at the write, as it is on Live, because the writer sets it and no call takes one; the candle's time sits under `data` as `candle_at` and `candle_ts_ms`, the stamp the Activity Log's trade line carries. A reader finds a candle by `data.candle_at`, never by the top-level stamp.

`src/simulator/sim_bus.py` — the four keys every gate and voting row adds

```
    def stamp(self, candle_ts_ms: int) -> dict:
        """The four keys every row adds: ``run_id``, ``mode``,
        ``candle_ts_ms`` and ``candle_at``."""
        return {
            "run_id": self.run_id,
            "mode": self.mode,
            "candle_ts_ms": int(candle_ts_ms),
            "candle_at": iso_stamp(candle_ts_ms),
        }
```

A trade row carries no run id and no candle: Live's trade listener writes a closed set of fields and drops every other key, and that listener is not changed. A trade row is found through the gate row written in the same moment for the same bot with the same action, the pairing the sizing replay already makes. A diagnostics row carries the run id and the candle inside its message, because Live's diagnostics listener writes `symbol` and `message` only.

`src/simulator/sim_bus.py` — the diagnostics message

```
BOT_LINE_FORMAT = "{message} (run {run_id}, candle {candle_at})"
```

### What a walk's gate row holds

A walk's gate row is Live's gate row: the two armed flags and the two blocker lists the chain latched, the scrum and fold fixtures with the keys the live bot writes at fire time, the tranche snapshot and the compounding snapshot in the live shapes, `evaluated_at_tick` as the walk's tick count, and the trade action and side on a row where a fill happened. Live's fixture shape has no function of its own, two dict literals inside the live bot's tick; the Simulator spells the same keys from the walk's gate context.

`src/simulator/sim_bus.py` — the scrum fixture's first keys

```
def scrum_fixture(context: GateContext, reading: Any = None) -> dict:
    """The ``scrum_fixture`` keys ``ScrummingBot.tick`` writes, read off
    ``context`` and ``reading``."""
    return {
        "delta": float(context.delta),
        "below_interval": bool(context.below_interval),
        "ticker_last": float(context.ticker_last),
        "bb_pos": float(context.bb_pos),
        **_landing(reading),
```

One row per evaluated candle, not one per fill as Live writes, because a blocked candle is the one a person troubleshoots: the blocker list on that row names the light that held, and the fixture names the reading behind it. The voting row beside it carries the whole vote of that candle, every indicator's direction, confidence and details, so a pillar that voted wrong is found by candle.

`src/simulator/back_test.py` — the walk's emit order, Live's fire order

```
        if emitter is not None:
            action = fill_action(filled)
            if filled is not None:
                emitter.trade_filled(filled, bot.exchange_id)
                emitter.bot_line(bot.bot_id, fill_line(filled), stamp)
            emitter.voting_snapshot(bot, summary, stamp, action, fill_side(filled))
            emitter.gate_decision(
                bot,
                context,
                armed,
                stamp,
                reading=reading,
                position=position,
                tick=ticks,
                trade_action=action,
                side=fill_side(filled),
            )
```

A scrum row's `usd` is the settled proceeds net of the estimated fee, as Live's scrum row carries it; a fold row's `usd` is the spend. The Activity Log's trade line keeps printing the gross beside the fee. The fill words that line uses moved into the bus module, so the surface's format now holds the prefix and the line the diagnostics row

carries.

`src/simulator/sim_bus.py` — Live's fill line, without the window's prefix

```
FILL_LINE_FORMAT = (
    "{word}: {units:.6f} {asset} @ ${price:.8f} ({usd_word} ${usd:.5f}, fee ${fee:.5f})"
)
SCRUM_FILL_WORD = "SELL FILLED"
FOLD_FILL_WORD = "BUY FILLED"
SCRUM_USD_WORD = "gross"
FOLD_USD_WORD = "spent"
```

### A Validation rerun row carries the recorded row and every light

A Validation run emits one gate row per rerun, with the rerun's armed flags, blockers and fixtures where Live's fields sit, and two more fields under `data`: the recorded row as the gate-log reader read it, and the comparison with all nineteen lights. Each light names its bank and label, the recorded state, the rerun state, whether they agree, which side drives it and, where they differ, the cause. A differing light is found from the file alone, by candle, with no report open: read the rows whose comparison holds a light that does not agree, and each such row names its candle.

`src/simulator/validation.py` — the fields a rerun row adds

```
def rerun_row_fields(row: Any, seen: RowComparison) -> dict:
    """The fields a rerun's gate row adds beside Live's: ``recorded`` from
    ``recorded_row_fields`` and ``comparison`` from ``comparison_row`` with
    all nineteen ``lights`` and their ``agrees``."""
    from .parity_report import comparison_row

    comparison = comparison_row(seen)
    comparison["lights"] = [
        {
            "bank": one.bank,
            "label": one.label,
            "recorded": one.recorded,
            "rerun": one.rerun,
            "agrees": bool(one.agrees),
            "driven_by": one.driven_by,
            "cause": one.cause,
        }
        for one in seen.labels
    ]
    return {"recorded": recorded_row_fields(row), "comparison": comparison}
```

Validation computes no voting summary, because the rerun reads the vote the bot recorded, so a Validation run writes no voting row. A bot with no YTD file or no tablet, and a bot whose snapped trades matched no recorded gate row, is named in the diagnostics file with the same line the report's Not verified section carries.

### The observer reads every run and the report names its rows

For the length of one run, Live's emit observer is subscribed to the private bus over the two Live contracts the Simulator emits, the fill and the gate decision. The run's emitter counts what it emits per topic; the observer counts what arrived and reports a required field that is absent, a value outside its vocabulary, and a declared topic that never fired. At the run's end the observer is finished, its subscriptions are removed, and its reading goes into the run object and the report beside the run id and the four file paths. A run that emits no fill reports the fill topic as never emitted, so an empty run reads as empty rather than as clean.

`src/simulator/sim_bus.py` — the observer's reading at the run's end

```
    def close(self) -> dict:
        """Finish the ``EmitObserver``, remove its subscriptions and answer
        ``to_dict`` with ``emitted`` beside it; a second call answers the same."""
        if self._read is not None:
            return self._read
        for remove in self._unsubscribe:
            remove()
        self._unsubscribe = []
        self._observer.finish()
        self._read = {"emitted": dict(self.emitted), "observer": self._observer.to_dict()}
        return self._read
```

The report's Run section gains the run id, the four file paths, the rows emitted per topic and the observer's line; the sidecar carries the same under `header.rows`, with each violation listed.

```
- run id: back_test-20260918T183818245464Z
- row files: trade.log <sim bucket>/trade.log, gate.log <sim bucket>/gate.log, voting.log <sim bucket>/voting.log, diagnostics.log
<sim bucket>/diagnostics.log
- rows emitted: bot.voting_panel_snapshot 4752, bot.gate_decision 4752, trade.filled 17, bot.log 17
- observer: 2 of 2 declared topic(s) seen; bot.gate_decision 4752, trade.filled 17; 0 violation(s)
```

Live's contract module declares a voting topic nothing emits and no contract for the voting snapshot topic the live bot does emit, so the voting row is written under no contract and the observer does not read it. Live's trade rows carry an empty exchange, because the live fire sites pass none; the Simulator's carry the bot's exchange in that field.

### What the bucket holds and how large it grows

After a run the sim bucket holds the four files beside the sim fleet file and the empty daily directory. Each file rotates at 50 MiB with five backups, the writer's own bound. Measured over the scratch fleet of 24 bots at 200 ticks each, a Back Test wrote 4,752 gate rows in 11.9 MB, about 2.5 KB a row, and 4,752 voting rows in 27.1 MB, about 5.7 KB a row, the vote's twelve signals and their details making the difference. A whole-root Portfolio Battery of 189 walks at 120 ticks each writes about 22,700 rows a file, so its voting rows cross the rotation bound within one run and the older ones survive in the numbered backups.

### What the emitter reading measured

Before, in both builds over a scratch home holding 24 scrumming bots, one 2,000-candle 5m tablet per pair, four fills per pair, one fired gate row per fill, the live bucket's five files each holding one line, and a copy of the 411 RA-StoneTablets: after a Validation run, a Back Test run and a Run Portfolio, the sim bucket held the sim fleet file and nothing else, no host held a bus, and no report named a run id.

After, in both builds over the same home: the Validation run wrote 96 gate rows and 96 trade rows under its run id, the Back Test 4,752 gate rows, 4,752 voting rows, 17 trade rows and 17 diagnostics rows, and the Battery 1,017 gate rows, 1,017 voting rows, 204 trade rows and 204 diagnostics rows; every gate and voting row carried the run id and the candle, every row of each file carried its contract's required fields, and the observer read 2 of 2 declared topics seen and 0 violations on each run, its counts equal to the rows on disk. The host's bus was not the process-wide bus, read by identity; the process-wide bus's subscriptions read the same after every run as after the window was built; the live trade bucket's four files, the 24 tablets, the YTD files and the 411 RA tablets hashed identical after every run, and a byte appended to a copy of the fleet file moved its hash. The live `system.log` changes during a run on the base tree already, from the window's own panel timer and the Simulator's own loggers, and is the process log rather than a trade-bucket file.

Off the tab, one Back Test walk over the same fleet with the bus and the writer, under the debugger with `amount` removed from one fill payload at the emit line, read one observer violation naming the absent field, carried into the report; the same walk untouched read zero. The sizing replay of the one trading logic, run over that walk's `sim/trade.log` and `sim/gate.log`, paired 17 of 17 fills with their gate rows and reproduced 4 of 4 scrum unit counts, 13 of 13 fold unit counts, 4 of 4 interval readings and 4 of 4 scrum proceeds; its own planted row read one disagreement. Over the scratch home whose recorded rows carry one flipped S/TA light, one removed `target_fires` and one reading moved to the opposite band, `sim/gate.log` alone, with no report open, named three rows with a differing light by candle: the removed field's candle with S/FIRE `fixture`, the flipped candle with S/TA `chain` and its eight armed-flag lights, and the moved candle with S/BB `tape` and its ten dependents, in both builds.

## Clear Fleet empties the held fleet

Clear Fleet is the corner's first button in every mode and, while a fleet is held, the Get Started card's first button. A press empties the whole held fleet, every record on every venue, after one confirmation in Live's Delete Bot box shape. The venues unseat, the card returns, the header strip reads zeros and the sim fleet file is written with no record. `bot_state.json` is not touched. Both builds draw the button from one surface list and both hosts run one handler, `_clear_fleet`.

### The button at the corner and on the card

The corner's list is `way_in_buttons` in `src/gui/simulator/sim_trading_tab_surface.py`: `clear_fleet_button` first, then the run mode's two ways in, each at Live's corner-button size, 140 px wide and 24 px tall. The card's list is `placeholder_way_in_buttons`, which puts Clear Fleet above the two ways in only while `held`, the count of held records, is above zero. The Qt host draws both lists in `SimTradingTab._draw_way_ins` and draws them again at the end of every `_sync_exchange_tabs`; the React host writes `held` into `SimTradingTabState` on every `show_tab` and calls `show_tab` at the end of every `_sync_exchange_tabs`, so the page's `WayInButtons` and `PlaceholderWayIns` components draw the same two lists with no change to `src/gui/web/sim_trading_tab.js`. The button's accessible name is `sim-clear-fleet` at the corner and `sim-clear-fleet-card` on the card.

Since the way-in row landed, that list draws on a row above the exchange tab bar in both builds, not in the tab bar's corner, and it is empty with no fleet held, so the row is absent and the card alone offers the ways in. The Qt host builds the row in `SimTradingTab._make_layer` and the React page draws it in `WayInButtons` above the tab bar. The corner holds nothing. The section "Start Run starts the active mode's run from the way-in row" describes the row.

`src/gui/simulator/sim_trading_tab_surface.py` — the card's list

```
def placeholder_way_in_buttons(
    accent: Any, mode: str = sim.MODES[0], held: int = 0
) -> list:
    """The card's buttons at Live's card-button size and sheet: Clear Fleet
    first while ``held`` records are held, then the two ways in ``mode`` offers."""
    rows = list(sim.reserved_rows(mode))
    if int(held or 0) > 0:
        rows.insert(0, {"action": sim.CLEAR_FLEET_ACTION, "text": sim.CLEAR_FLEET_TEXT})
```

The card draws only on a layer that seats no venue. Every venue the Simulator seats today, `coinbase`, `kraken` and the Battery's `yahoo`, sits on the crypto layer, so the crypto card and a held crypto fleet are never on screen together. The card's Clear Fleet is the way to empty a fleet held on the other layer, such as an imported `bot_state.json` fleet on an equity venue, which the stack does not show.

### Which layer a Simulator venue is seated on

The tab asks one list whether a venue trades equities. The list is declared once, in the module the header's asset class group reads, and the Simulator tab reads that same name.

`src/gui/simulator/sim_trading_tab.py` — the read, in the tab's own build

```
self._equity_exchange_ids = acs.EQUITY_VENUES
```

A venue on the list is seated on the stock layer and every other venue on the crypto layer. The id is matched in lower case, so a venue seated under a capitalised name reaches the same layer. Where the list lives, and what reads it, is on 06-trading-tab.md.

### The confirmation and the clear

A press reaches `_way_in` on either host and then `_clear_fleet`. The box is `QMessageBox.question` under the title `Clear Fleet`, with Yes and No, as the command bar's Delete opens its `Delete Bot` box. The question names the count and the venues. Yes runs `FleetSource.clear`, which drops every held record and answers how many were held, then writes the cleared line, the notification `Fleet cleared`, plays Live's state-change sound and fires `fleet_changed` once. `FleetSource.save` writes the sim fleet file with `bot_count` 0. `_sync_exchange_tabs` reads no exchange, takes every venue off, adds the Get Started page back, and re-reads the tables, the panel and the replay layer. The header strip reads `aggregate_stats` over no bot on its next tick, every field zero, and EXCH 0. No is the cancelled line and nothing moves.

`src/simulator/fleet_source.py` — the clear

```
    def clear(self) -> int:
        """Drop every held record, on every exchange; answers how many were
        held. The sim fleet file loses them on the next ``save``."""
        count = len(self._records)
        self._records = {}
        return count
```

Clear Fleet empties every venue, not the shown venue alone. Live's Delete acts on one bot id. The fleet is one map: `FleetSource` holds every venue's records together, `save` writes them as one file, and the strip reads them together. A per-venue clear would be a different control with a different name.

`src/gui/simulator/sim_trading_tab_surface.py` — the box and the lines

```
CLEAR_FLEET_BOX_TITLE = sim.CLEAR_FLEET_TEXT
CLEAR_FLEET_QUESTION_FORMAT = (
    "Clear the Simulator fleet of {count} bot(s) on {venues}? This cannot be undone."
)
CLEARED_FORMAT = "Cleared {count} bot(s) on {venues}; the Simulator fleet is empty."
CLEAR_CANCELLED_TEXT = "Clear Fleet cancelled."
NOTHING_HELD_TEXT = "No fleet is held; nothing to clear."
FLEET_CLEARED_NOTICE = "Fleet cleared"
```

### A run in flight, nothing held, cancelled

While a Validation or Back Test run is in flight, the press writes the same in-flight line the command bar writes, naming Clear Fleet as the command that waits, and opens no box. While a Portfolio Battery run is in flight, the press writes the Battery's in-progress line. With no record held, the press writes `No fleet is held; nothing to clear.` and opens no box. No on the box writes `Clear Fleet cancelled.` and removes nothing. In each of these cases `fleet_changed` does not fire and the sim fleet file is not written.

### The two signals a press emits

Every press emits `sim.fleet.clear_pressed` through `signal_contract.emit`, bound in each host as `_pin_emit`, and a confirmed clear emits `sim.fleet.cleared` after it. The press signal carries the outcome and the held count after the press, against the count the outcome allows: unchanged for a cancel, a refusal or nothing held, zero for a clear. The cleared signal carries the held count and the seated venue count after the clear, both expected zero, with the count removed and the venues unseated in its context. Each host flushes the process sink after the emit, so the handler's file under the runtime log directory's `signals` folder holds the rows right after the press.

`src/gui/simulator/sim_trading_tab.py` — the cleared signal

```
        _pin_emit(
            tab_surface.CLEARED_SIGNAL,
            actual={
                "held_after": len(self._fleet_source.bots()),
                "venues_after": self.exchange_count(),
            },
            expected={"held_after": 0, "venues_after": 0},
            context={"removed": removed, "venues_unseated": list(venues)},
        )
```

### What the clear reading measured

Before, in both builds over a scratch home holding a `bot_state.json` with four records on `coinbase` and one on `kraken`, a tablet manifest naming four assets per venue and no sim fleet file: Import Live Fleet on both venues seated both and held five records; no button named `sim-clear-fleet` existed at the corner or on any card in any mode, the five records stayed held through every press, and the only route to an empty fleet was Delete, one row and one box at a time.

After, in both builds over the same home: the corner read `sim-clear-fleet`, `sim-import-live-fleet` and `sim-generate-from-ytd` in that order from the left, each 140 by 24, Clear Fleet's left edge 142 px

before Import Live Fleet's. With five records held, the stock layer's card held `sim-clear-fleet-card` first and the crypto card was not drawn. The corner press opened the box `Clear Fleet` reading `Clear the Simulator fleet of 5 bot(s) on coinbase, kraken? This cannot be undone.` with Yes and No; No wrote the cancelled line and left five held. Yes wrote `Cleared 5 bot(s) on coinbase, kraken; the Simulator fleet is empty.` and the notification, unseated both venues, drew the crypto card with two buttons and no Clear Fleet, zeroed the five cards and read EXCH 0 on the strip's next tick, wrote the sim fleet file with `bot_count` 0 and an empty `bots` map, and left two signal rows in `session.jsonl`, both `ok` true. Back Test then Create New Bots on the card walked the wizard to one bot on `coinbase`, which seated the venue and put Clear Fleet on the stock card; that card's press opened the box naming one bot and Yes emptied the fleet again. The corner press with nothing held wrote `No fleet is held; nothing to clear.` and opened no box. A Back Test started with its worker held open read `run_running` true; the corner press wrote the in-flight line naming Clear Fleet, opened no box and left five held. `bot_state.json` hashed identical after every step in every run, and a byte appended to a copy of it moved the hash. The Watchdog Archetype read every emit in both hosts wired to `signal_contract`.

## Start Run starts the active mode's run from the way-in row

Start Run is one button on the way-in row above the exchange tab bar, drawn under every mode while a fleet is held, to the right of the mode's two ways in. A press starts the active mode's run over the fleet on the venue shown. Start on the command bar no longer starts a run in any mode; it moves the selected bot's state, as Pause, Stop and Restart do. Both builds draw the button from one surface list and both hosts run one handler, `_start_run_pressed`.

### The button on the way-in row

The row's list is `way_in_buttons` in `src/gui/simulator/sim_trading_tab_surface.py`, which takes the held count: empty at zero, otherwise Clear Fleet first, then the run mode's two ways in, then Start Run, each at Live's corner-button size, 140 px wide and 24 px tall. The Qt host hands the count in `SimTradingTab._draw_way_ins`, which runs again at the end of every `_sync_exchange_tabs`; the React host hands it through `layer_card`, which the tab payload carries on every `show_tab`. The button's accessible name is `sim-start-run`. The Get Started card never holds it: the card draws on a layer with no venue, and a held fleet seats its venue.

`src/gui/simulator/sim_trading_tab_surface.py` — the row's list

```
def way_in_buttons(mode: str = sim.MODES[0], held: int = 0) -> list:
    """The way-in row's buttons as the page draws them, while ``held``
    records are held: ``clear_fleet_button`` first, then the two ways in
    ``mode`` offers from ``sim.reserved_rows``, then ``start_run_button``;
    an empty list with nothing held, so the row is absent and the card shows."""
```

### The way-in row above the exchange tabs

The row is a widget of its own above each layer's exchange tab bar, so the tabs and the buttons never share a line and a third or fourth venue's tab has room. The Qt host builds it in `SimTradingTab._make_layer` as a `QWidget` with a horizontal layout at margins 0 and spacing 2, added to the page's column before the tab widget, hidden while the list is empty; the tab widget gets no corner widget. The React page draws it in `WayInButtons` as the first child of the tab widget's column, above the tab bar, wrapping to a second line where the pane is narrower than the buttons, and draws nothing while the list is empty; the tab bar keeps its tabs and the stretch after them. Both hosts read the row's margins and spacing from one place.

`src/gui/simulator/sim_trading_tab_surface.py` — the row's layout

```
WAY_IN_ROW_LAYOUT = {"margins_px": [0, 0, 0, 0], "spacing_px": 2}
```

`src/gui/web/sim_trading_tab.js` — the row above the bar

```
      element(
        DIV_TAG,
        widgetProps,
        element(WayInButtons, { layer: layer, actions: props.actions }),
        element(DIV_TAG, barProps, barTabs, element(DIV_TAG, spacerProps, null)),
        element(DIV_TAG, bodyProps, body)
      )
```

Read with three venues seated, `coinbase`, `yahoo` and `kraken`, in both builds: at a 1400 px window every tab and every button had its own rect, no two overlapping, the row's bottom edge above the bar's top edge, and the tab bar's corner empty, under all three modes. The Qt window's floor is its layout minimum, 1145 px wide in the reading, and the row fits on one line there. The React window shrinks to 900 px, where the pane is 476 px wide and Start Run wraps to a second line; every tab and button stayed inside the pane with no overlap.

### The press by mode

A press reaches `_way_in` on either host and then `_start_run_pressed`. The handler reads the tab's run mode and the venue on show. Under Validation and Back Test it runs `_start_run` over every held scrumming bot on that venue, the run the section "Start runs Validation and Back Test from the command bar" describes: each bot to running, the started line, the worker thread, the trade lines, the report and the report line, every bot to stopped. Under Portfolio Battery it runs `_run_battery` under Run Portfolio's action, so the chooser opens with the portfolio and span rows, and the run follows as the section "The run from the corner" describes. Run Portfolio and Run Every Portfolio keep their own buttons; Start Run under the Battery is a third way to the same chooser, because one button means one thing in every mode.

`src/gui/simulator/sim_trading_tab.py` — the handler

```
    def _start_run_pressed(self) -> None:
        venue = self._current_venue_id()
        mode = self._mode
        run_bots = self._run_bots(venue)
        running_before = tab_surface.rows_running(run_bots)
```

`_start_run` takes the venue id where it took a selected bot, because the corner has no selected row; the run it starts is unchanged.

```
    def _start_run(self, exchange_id: str, mode: str) -> str:
```

### Start on the bar moves one bot

The dispatch unit 31 put at the top of `_on_bot_command`, which read the mode and started the run on Start, is removed from both hosts. Start on the bar now reaches the state move in every mode: `SimBotManager.start` on the selected bot, the running line, the notification and `fleet_changed`. The in-flight guard above it stays: while a run is in flight, Stop on one of its rows ends the run and every other bar press writes the in-flight line.

### A second press in flight

Start Run is never disabled, because Live's command bar never disables Start. A press while a Validation or Back Test run is in flight writes the in-flight line naming Start Run as the command that waits, and starts nothing. A press while a Portfolio Battery run is in flight writes the Battery's in-progress line. A press on a venue holding no scrumming bot writes the no-bot line.

```
A Back Test run is in flight over 24 bot(s); Stop on one of its rows ends it, and Start Run waits for it.
```

### The signal a Start Run press emits

Every press emits `sim.run.start_pressed` through `signal_contract.emit`, bound in each host as `_pin_emit`, and flushes the process sink after it. The signal carries the outcome, one of started, refused in flight, no bot or cancelled, and the count of the venue's scrumming rows reading running after the press, against the count the outcome implies: every run bot after a started Validation or Back Test, the count before the press otherwise. The mode, the venue, the held count and the run's bot count travel in its context.

`src/gui/simulator/sim_trading_tab.py` — the signal

```
        _pin_emit(
            tab_surface.START_PRESSED_SIGNAL,
            actual={"outcome": outcome, "rows_running": running_after},
            expected={"outcome": outcome, "rows_running": expected_running},
            context=context,
        )
```

### What the Start Run reading measured

Before, in both builds over a scratch home holding a `bot_state.json` of 24 scrumming bots on `coinbase`, one 2,000-candle 5m tablet per pair, four fills per pair in YTD files, one fired gate row per fill, and a scratch RA root holding five 2022 tablets: Import Live Fleet held 24 rows; the corner read Clear Fleet then the mode's two under every mode; no button named `sim-start-run` existed under any mode, and a press asked for it read no button in Qt and no element in React.

After, in both builds over the same home: the way-in row read Clear Fleet, the mode's two, then `sim-start-run` under every mode, each 140 by 24, and was absent with no fleet held at open and after Clear

Fleet. Start on the bar with the first row selected, under each of the three modes, wrote the running line, moved that one row to RUNNING and no other, started no thread and wrote no report; Stop moved it to STOPPED. Start Run under Validation moved all 24 rows to RUNNING, wrote 96 trade lines and six run lines, wrote the Validation report and named it on the Activity Log, and the report read missing 0, extra 0, differs 0, variant differs 0 over 96 rows. Start Run under Back Test pressed twice fast wrote the started line and then the in-flight line naming Start Run, moved all 24 rows to RUNNING, wrote 17 trade lines and the Back Test report. Start Run under Portfolio Battery opened the chooser titled Run Portfolio with 35 portfolios and seven spans; CRYPTO_BLUE over 2022 wrote 45 trade lines, the Battery's eleven lines and the Battery report. Four signal rows landed in `session.jsonl`, every one `ok` true: started under Validation with 24 rows running, started and refused in flight under Back Test, started under Portfolio Battery with 0 rows running. Every watched file, 59 of 59, hashed identical after every step in every run, and a byte appended to a copy of `bot_state.json` moved the hash. No socket left loopback. The Watchdog Archetype read every emit in both hosts wired to `signal_contract`.

## The flip sits at the panel's top-left corner under both layers

The operator's item: *"'Replay' button in the IVP panel needs to move to the IVP's upper left corner where the 'Indicators' button appears after 'Replay' is click. Both of these buttons should appear in the same location."* One button flips the layer stack. It now sits first in whichever header row is showing, at the row's left edge, and draws one width on both layers, so the rect that reads Replay on the panel is the rect that reads Indicators on the replay layer.

### One seat under both layers

The Qt host seats its one `FlipButton` at `FLIP_SEAT_INDEX`, the first item, in the panel's header row at build and on every flip, and at the same index in the replay layer's header. The replay layer's header takes its margins and spacing off the panel's header row instead of restating them. The page does the same from one declaration: `REPLAY_HEADER_LAYOUT` reads the panel header's margins and the panel column's spacing from `indicator_panel_surface`, `sim_indicator_panel.js` draws the flip before the title, and `sim_trading_tab.js` draws it first on the replay header.

`src/gui/simulator/sim_trading_tab_surface.py` — the replay header's layout

```
REPLAY_HEADER_LAYOUT = {
    "margins_px": list(cast(list, panel.HEADER["margins_px"])),
    "spacing_px": int(cast(int, panel.CONTAINER["spacing_px"])),
}
```

### One width for both words

Replay and Indicators differ in width, so one seat alone gives two rects. The Qt `FlipButton` answers `sizeHint` with the wider of the two words' hints, each asked of the style the way `QPushButton.sizeHint` asks it; its `minimumSizeHint` keeps a width of 0, so the pane pair the section "The Sim venue pane is Live's width" records does not move. The page's `flip_button` payload carries `other_text`, the word the button reads on the other layer, and the sheet reserves that word's width as a hidden zero-height line after the text. The sheet also pins the button to the top of its row, and the replay header wraps its chooser and retrieval button to a second line when the pane is narrower than its controls, so the button keeps its width and its corner at every pane width.

`src/gui/simulator/sim_trading_tab.py` — the width

```
    def sizeHint(self) -> QSize:  # noqa: N802
        """The base hint, widened to the widest word in ``texts``."""
        base = super().sizeHint()
        widest = max(self.hint_for(text).width() for text in self._texts)
        return QSize(max(base.width(), widest), base.height())
```

`src/gui/web/sim_trading_tab.css` — the reservation

```
[data-part="sim-flip-button"]::after {
  content: attr(data-other-text);
  display: block;
  height: 0;
  overflow: hidden;
  visibility: hidden;
}
```

### The flip's signal

Each press emits `sim.layer.flipped` through `signal_contract`: `actual` is the pressed button's rect in the layer stack's coordinates, `expected` the rect of the previous press, `ok` their equality or None on the first press, and `context` names the layer left and the layer shown. The Qt host reads the rect in `flip_layer` before it moves the button; the page reads it at the press through the tab module's `flipRect` and sends it under `flip_rect` with either flip ask, and the React host's `show_layer` emits on every change of layer. One function builds the row for both hosts.

`src/gui/simulator/sim_trading_tab_surface.py` — the row

```
def flipped_pin(leaving: str, shown: str, pressed: Any, previous: Any) -> dict:
    """The ``LAYER_FLIPPED_SIGNAL`` row for one press: ``actual`` the pressed
    rect, ``expected`` the previous press's rect, ``ok`` their equality or
    None on the first press, ``context`` the layer left and the layer shown."""
    return {
        "actual": pressed,
        "expected": previous,
        "ok": None if previous is None or pressed is None else pressed == previous,
        "context": {"from": leaving, "to": shown},
    }
```

### What the flip reading measured

Read off the widget tree and the page in the layer stack's coordinates, over a scratch home holding a `bot_state.json` of two scrumming bots on `coinbase` and two 5m tablets, the live fleet imported. Before, at a 1400 by 900 window, the Qt button read x 123, y 5, 47 by 20 on the panel and x 4, y 5, 64 by 20 on the replay layer; the React button read x 122, y 2, 57 by 22 and x 4, y 2, 74 by 22. At the Qt floor, 1145 by 1124, the same two rects; at the React reading of 900 by 700 the panel's title wrapped and the button read x 116, y 8, 57 by 22 against x 4, y 8, 54 by 22 on the layer, where the row squeezed it. No `sim.layer.` row existed.

After, in both builds at 1400 and at the floor, the button read one rect on the panel, on the layer after a press, and on the panel after a second press: Qt x 4, y 5, 64 by 20; React x 4, y 2, 74 by 22. The title sat to the button's right at every reading. Seating the panel's button back after the title moved it to x 123 in Qt and x 122 in React, and seating it first again restored the rect, so the reading can fail. At the React floor the replay header wrapped its retrieval button to a second line at x 4, y 26, inside the pane. Four `sim.layer.flipped` rows landed per variant, the second and every later one `ok` true with `expected` equal to `actual`. The Qt top splitter read the same pair before and after. Every watched file, 8 of 8, hashed identical after every step, and a byte appended to a copy of `bot_state.json` moved the hash. No socket left loopback.

## One fleet per mode

The Simulator holds one fleet for each run mode: a Validation fleet, a Back Test fleet and a Portfolio Battery fleet. The tab shows the fleet of the mode in force. Pressing a mode swaps the tables, the header strip, the venue stack and the way-in row to that mode's fleet. Import Live Fleet, Generate From YTD, Create New Bots, Clear Fleet and the Battery's generated fleet act on the fleet of the mode in force alone, so the Battery's generated fleet and its Yahoo venue live under Portfolio Battery and do not appear under Validation. One file holds all three fleets. Both builds draw the same screens and both hosts run the same source.

### The source holds three fleets

The fleet source keeps one records map per mode and one mode in force. Every reader and every act it answers works on the map of that mode, so no caller changed: the tables, the strip, the venue stack, the way-in row, the bot manager and the three runners all read the fleet in force through the calls they made before. The mode is set on the source by the hosts and read back by them; the Qt host's mode is that read, and the React host writes it into its tab state before each draw.

`src/simulator/fleet_source.py` — the three modes and the fleet in force

```
MODE_VALIDATION = "validation"
MODE_BACK_TEST = "back_test"
MODE_PORTFOLIO_BATTERY = "portfolio_battery"
MODES = (MODE_VALIDATION, MODE_BACK_TEST, MODE_PORTFOLIO_BATTERY)

#: The sim fleet file's key holding one entry per mode of ``MODES``.
FLEETS_KEY = "fleets"
```

```
    def held_by_mode(self) -> dict[str, dict]:
        """Per mode of ``MODES``, ``held`` (how many records draw as a
        ``SimBot``) and ``venues`` (their distinct ``exchange_id`` values,
        sorted)."""
```

### The press swaps the fleet

A mode press on the venue header or on the card reaches the host's `set_mode`. The host hands the mode to the source, restyles every seated venue header, then fires the fleet change: the file is written, the venues the new fleet names seat and the rest unseat, the rows, the panel and the replay re-read, and the way-in row and the card redraw. The strip reads the new fleet on the window's next tick. A press while a run or a Battery is in flight writes the in-flight line and changes nothing, because the run's end stops its bots by id on the fleet in force.

Diagram: flowchart, left to right

- mode button, header or card -> **host.set_mode**
- host.set_mode -> **FleetSource.set_mode**
- FleetSource.set_mode -> **fleet_changed**
- fleet_changed -> **FleetSource.save, three fleets**
- fleet_changed -> **_sync_exchange_tabs**
- _sync_exchange_tabs -> **venues seat and unseat**
- _sync_exchange_tabs -> **rows, panel, replay**
- _sync_exchange_tabs -> **way-in row and card**
- window tick -> **header strip**

### The card holds the mode row

A mode whose fleet is empty shows the Get Started card and no venue, and the venue header is the only place the mode buttons were drawn. The card now holds the three mode buttons under its title, from the same list the venue header draws, with the same sheets: the active mode wears Live's Privacy Mode ON sheet and the other two the OFF sheet. The Qt host builds them once per layer and restyles them on every fleet change; the React page draws them from the card payload's `mode_buttons`, and a press sends `mode` on the tab's own ask, which the host hands to `set_mode` as it hands the venue header's press. The card's buttons are named `sim-mode-<mode>-card`.

`src/gui/simulator/sim_trading_tab_surface.py` — the card's mode buttons

```
def placeholder_mode_buttons(active: str = sim.MODES[0]) -> list:
    """The card's three mode buttons, ``venue.mode_buttons`` over ``active``
    under ``card_mode_name``, so the card offers the mode choice the venue
    header offers."""
    return [
        {**button, "accessible_name": card_mode_name(button["key"])}
        for button in venue.mode_buttons(active)
    ]
```

### Clear Fleet names the mode

Clear Fleet empties the fleet of the mode in force and keeps the other two. Its box and its cleared line name the mode.

`src/gui/simulator/sim_trading_tab_surface.py` — the box and the line

```
CLEAR_FLEET_QUESTION_FORMAT = (
    "Clear the {mode} fleet of {count} bot(s) on {venues}? This cannot be undone."
)
CLEARED_FORMAT = "Cleared {count} bot(s) on {venues}; the {mode} fleet is empty."
```

### The signal a mode press emits

Every mode press emits one row through the emitter network. The row's `actual` is the mode in force with its held count and venues after the press; its `expected` is the pressed mode with the count and venues the source held for it before the press; its context names the outcome, the mode left and every mode's count. The outcomes are `shown`, `refused_in_flight` and `unknown_mode`. The two Clear Fleet rows carry the mode in their context.

`src/gui/simulator/sim_trading_tab_surface.py` — the signal

```
MODE_SHOWN_SIGNAL = "sim.fleet.mode_shown"
MODE_OUTCOME_SHOWN = "shown"
MODE_OUTCOME_IN_FLIGHT = "refused_in_flight"
MODE_OUTCOME_UNKNOWN = "unknown_mode"
MODE_SHOWN_FORMAT = "{mode} fleet shown: {count} bot(s) on {venues}."
```

### The React wizard is deleted on the GUI thread

Driving the flow on the React build found a crash: a bot created through the React wizard, then a Battery run, ended the process with a Chromium thread check, because the wizard's web view was left to Python's garbage collector, which ran on the Battery's worker thread. The React host now deletes the wizard through the event loop as soon as it closes, on the GUI thread, in both the accepted and the cancelled path. The same flow then ran to its end.

`src/gui/simulator/sim_react_trading_tab.py` — the disposal

```
            try:
                accepted = wizard.exec() == wizard.DialogCode.Accepted
                config = wizard.get_bot_config() if accepted else {}
            finally:
                # The dialog's QWebEngineView is deleted on this thread by the
                # event loop; a worker thread's garbage collection would abort.
                wizard.deleteLater()
```

### What the fleet-per-mode reading measured

Read off the running program in both builds, over a scratch home holding a `bot_state.json` of 24 scrumming bots on `coinbase`, one 5m tablet per pair, four fills per pair in YTD files, one fired gate row per fill, and a scratch RA root holding the 2022 tablets of A15, A14, SPY, QQQ, IWM, GLD and SLV.

Before, one fleet: Import Live Fleet under Validation drew 24 rows, and the same 24 rows drew under Back Test and Portfolio Battery; Create New Bots under Back Test made 25 under every mode; Run Portfolio over DIGITAL_GOLD 2022 replaced them with 4 rows on `coinbase` and `yahoo` under every mode; Clear Fleet under Back Test left every mode empty. The file held one `bots` map. No card held a mode button, and no `sim.fleet.mode_shown` row existed.

After, three fleets, the same readings in both builds: at open the card with three mode buttons, Validation active. Import under Validation, 24 rows on `coinbase`, the way-in row Clear Fleet, Import Live Fleet, Generate From YTD, Start Run, Spendable $15,470.00, EXCH 1. Back Test pressed on the header: no venue, no row, the card with Back Test active and Import Live Fleet and Create New Bots, Spendable an em dash, EXCH 0, one signal

row `ok` true, the line `Back Test fleet shown: 0 bot(s) on no venue.` Create New Bots on the card through the wizard: one row on `coinbase`. Portfolio Battery pressed: the card again; Run Portfolio on the card over DIGITAL_GOLD 2022: 4 rows, 2 on `coinbase` and 2 on `yahoo`, both tabs seated, Spendable $2,000.00, EXCH 2. Each mode pressed in turn read its own fleet: 24 rows, 1 row, 4 rows, each with its way-in row, its strip and its venue tabs, and each press one signal row with `expected` equal to `actual`. The file held `fleets` with `bot_count` 24, 1 and 4. Clear Fleet under Back Test: the box read `Clear the Back Test fleet of 1 bot(s) on coinbase? This cannot be undone.`, the line `Cleared 1 bot(s) on coinbase; the Back Test fleet is empty.`, the file 24, 0 and 4, the card back with Back Test active; the card's Validation button pressed drew the 24 rows again, and Portfolio Battery the 4.

A file planted in the old shape, one `bots` map holding one record, drew that record under Validation and nothing under the other two, with the line `holds one fleet; read as the validation fleet`, and the first mode press rewrote it under `fleets`. A file planted with `fleets` naming `validation`, `back_test` and `paper` drew one row under Validation, one under Back Test and none under Portfolio Battery, with one warning naming `paper`, and the first press wrote the three modes without it. Every watched file, 61 of 61, hashed identical after every step, and a byte appended to a copy of `bot_state.json` moved the hash. No socket left loopback.

Six-step counts for the React page: contract members 4 of 4 (the card's mode buttons, the card's mode row layout, the placeholder order, the tab ask's `mode`), feeds 2 of 2 (the card payload on every tab draw, the venue payload on every fleet change), driven and read back 3 of 3 (each of the three card buttons pressed, and the active sheet, the rows and the file read after).

Both bundles were built and launched in isolation, with no network and a scratch home holding two live bots, and driven through Windows UI Automation. In each: the card at open with its three mode buttons; Import Live Fleet, two rows, the way-in row; Back Test on the venue header, the card with Back Test active and its two ways in, the line `Back Test fleet shown: 0 bot(s) on no venue.`; Validation on the card's own button, the two rows and Validation's row again; Back Test on the header again, the card. The file held `fleets` with 2, 0 and 0, and three signal rows landed per build. The Qt bundle also walked Create New Bots through the wizard's Next and Finish buttons under Back Test, and the file then held 2, 1 and 0 with the new bot under Back Test; the React wizard's asset pair is a page control UI Automation does not drive, so Create under Back Test on the React build is the source-tree reading above. On the bundles, the live application's own periodic save rewrote `bot_state.json` sixty seconds after launch with the same two live bots and no sim bot, as it does on every launch; in the source-tree readings the file hashed identical after every step.

## The Battery walks every candle, one portfolio at a time, and reads HODL against Harvest-Fold

The operator's words, 2026-09-18: *"Porfolio Battery - This is scaffolded. It does not appear to be loading any historical data. I run it twice and generated what are undoubtedly bogus but revealing reports. I love the Yahoo exchange appearing and loading the stocks but when 'running all portfolios' should load one, run, clear, load next, run, clear, repeat. These must be real runs against real data using the scrumming bots and IVP trading logic."* And the framing, the same day: *"I only seek to compare HODL against Harvest-Fold across as many historical markets and asset classes as possible."*

Before this, a Battery bot evaluated about one bar in every fourteen of a daily tablet over six years, because the walk stepped so that each bot and timeframe spent about 120 evaluations; Run Every Portfolio held every portfolio's bots at once and walked them in one loop; and the report read each portfolio as `reversed`,

`improved`, `defended` or `undefended` against its historical path. Now every bar is evaluated, one portfolio is held and walked at a time, and the report reads three figures.

### Every candle

The walk hands the Back Test walker no step. The walker evaluates every bar from the thirtieth on, so a walk over N bars makes N less 29 evaluations, and the report states both counts side by side. The walk is timed around the walker alone, and the report states the seconds per 1,000 evaluations it measured and, at that rate, how long a crypto bot's 5m walk takes over `2026 test run` and over `All`.

`src/simulator/portfolio_battery.py` — the evaluations a walk makes

```
def evaluations_expected(bars: int) -> int:
    """How many gate-chain evaluations a walk over ``bars`` makes: one per bar
    from the ``MIN_CANDLES``th on, none under ``MIN_CANDLES``."""
    return max(0, int(bars) - (MIN_CANDLES - 1))
```

`src/simulator/portfolio_battery.py` — the walk, every bar

```
    result = walk(
        walked,
        candles_from_raw(bars),
        funding=FUNDED_BY_TARGETS,
        rule=rule,
        on_trade=took,
        stop=stop,
        emitter=emitter,
        on_tick=on_tick,
    )
```

### The walk reads as moving

One Activity Log line opens each bot's walk at each timeframe, naming the bars and the evaluations to make; one line lands every 5,000 bars, naming the bar reached, the evaluations so far, the partial exits and re-entries so far and the seconds; one line closes it with the evaluations, the seconds, the rate and the three figures. At the measured rate a line lands about every eight seconds on a 5m walk, and a daily walk ends before its first one.

`src/simulator/portfolio_battery.py` — the cadence

```
#: How many bars a walk covers between two progress lines. At the measured
#: rate of about 1.7 ms an evaluation a line lands about every 8.5 seconds on
#: a 5m walk, and a daily walk ends before its first one.
PROGRESS_EVERY_BARS = 5_000
```

`src/simulator/back_test.py` — the walker's tick callback

```
        if on_tick is not None:
            on_tick(index + 1, len(candles), ticks)
```

### Stop ends the walk at the bar reached

Stop on a Battery row sets an event the walk reads before each portfolio, each walk and each tick. The walk in progress ends at the bar reached, the walks and portfolios not reached are named, and the partial report carries the bar, its candle stamp and the evaluations made. The remaining portfolios are not run, and the summary reads `partial`.

`src/gui/simulator/sim_trading_tab.py` — the stop

```
    def _stop_battery(self, bot_id: str) -> None:
        """Stop on ``bot_id``, a row of the Battery in flight: Live's stopping
        line, ``battery_stopping_line``, and the event the walk reads before
        each tick."""
        self._status_log.log(f"Stopping bot {bot_id}...", "info")
        self._battery["stopper"] = bot_id
        self._status_log.log(tab_surface.battery_stopping_line(bot_id), "warning")
        self._battery_stop.set()
```

`src/simulator/portfolio_battery.py` — the stopped line

```
WALK_STOPPED_LINE_FORMAT = (
    "{asset} {timeframe} stopped at bar {bar:,} of {bars:,} ({candle_at}): "
    "{evaluations:,} evaluation(s) made."
)
```

### One portfolio at a time

The runner walks the named portfolios in order. Before each, it hands the host that portfolio's bots; the host holds them as the Portfolio Battery fleet, fires the fleet change, and the venues seat, the rows and the strip draw them, and the Yahoo venue seats for a stock portfolio. The portfolio's bots walk on a fresh set of walks, so a symbol two portfolios share is walked in each; its report is written; the host is handed the report, writes its line and, under Run Every Portfolio, holds an empty fleet, so every venue unseats and the Get Started card returns before the next portfolio loads. A single Run Portfolio keeps its fleet held at the end. When the press names more than one portfolio the runner writes one more report, the summary, and the host writes its line last.

`src/simulator/portfolio_battery.py` — the loop

```
    for name in chosen.names:
        if halted or (stop is not None and stop()):
            halted = True
            not_reached.append(name)
            continue
        bots = chosen.by_portfolio.get(name, {})
        seated = [bots[asset] for asset in PORTFOLIOS[name].symbols if asset in bots]
        if on_portfolio_started is not None:
            on_portfolio_started(name, list(seated))
        emit_portfolio_started(name, seated, str(span))
        emitter = RunEmitter(bus, new_run_id(PORTFOLIO_BATTERY), PORTFOLIO_BATTERY)
```

`src/gui/simulator/sim_trading_tab.py` — the two slots

```
    def _battery_portfolio_started(self, name: str, bots) -> None:
        """Hold ``bots``, one portfolio's, as the fleet through
        ``FleetSource.hold_battery_fleet``, fire ``fleet_changed`` so the
        venues seat and the rows and the strip draw them, and write
        ``battery_loaded_line``."""
        self._fleet_source.hold_battery_fleet(list(bots))
        self.fleet_changed.emit()
        self._status_log.log(tab_surface.battery_loaded_line(name, bots), "info")

    def _battery_portfolio_finished(self, name: str, run) -> None:
        """Write ``run``'s report line through ``log_report``; when the press
        names more than one portfolio, hold an empty fleet through
        ``FleetSource.hold_battery_fleet``, fire ``fleet_changed`` so every
        venue unseats, and write ``battery_cleared_line``."""
        if run.report is not None:
            self.log_report(run.report)
        if not self._battery.get("every"):
            return
        self._fleet_source.hold_battery_fleet(())
        self.fleet_changed.emit()
        self._status_log.log(tab_surface.battery_cleared_line(name), "info")
```

Under Run Every Portfolio the press holds an empty fleet, so nothing is drawn while the crypto tablets are retrieved and the first fleet the tab draws is portfolio 1's. Under Run Portfolio the press holds that one portfolio's bots, as the section "The run from the corner" says, and the first portfolio's start re-holds the same records. Each portfolio's start also empties the held fills, so a bot two portfolios share marks one portfolio's fills on the replay layer, and its loaded line names the portfolio's bot count, venues and budget.

`src/gui/simulator/sim_trading_tab.py` — the press

```
        self._fleet_source.hold_battery_fleet(() if every else plan.bots)
        self.fleet_changed.emit()
```

### The three figures

HODL end is the Target Balance bought at the walk's opening bar and held to its last bar. Harvest-Fold end is the same capital walked by the scrumming bot on the IVP's logic, its units at the last close plus the cash its partial exits left. The difference is Harvest-Fold end less HODL end, as a figure and as a percentage of HODL end. The historical trough is the lowest the untraded holding read on a walked bar, a figure on the path. The partial exits and re-entries are counted beside them. The report's Battery section, the Activity Log's per-portfolio line and the summary read these per timeframe per portfolio; the summary totals them per timeframe over the portfolios that ran and lists each portfolio with its run id, its report file and its three figures. Nothing ranks, scores or declares a winner: the four words of the earlier reading, its rule, the verdict word, the best timeframe and the improved timeframes are gone from the module and the report.

`src/simulator/portfolio_battery.py` — the comparison in the report's words

```
COMPARISON_RULE = (
    "HODL end is the Target Balance bought at the walk's opening bar and held to "
    "its last bar; Harvest-Fold end is the same capital walked by the scrumming "
    "bot on the IVP's logic, its units at the last close plus the cash its "
    "partial exits left; difference is Harvest-Fold end less HODL end, as a "
    "figure and as a percentage of HODL end; historical trough is the lowest the "
    "untraded holding read on a walked bar. Nothing ranks or declares a winner."
)
```

`src/simulator/portfolio_battery.py` — the three figures of one timeframe

```
    @property
    def difference_usd(self) -> float:
        """``accumulation_usd`` less ``baseline_usd``."""
        return self.accumulation_usd - self.baseline_usd

    @property
    def difference_pct(self) -> float:
        """``difference_usd`` as a percentage of ``baseline_usd``."""
        return difference_pct_of(self.baseline_usd, self.accumulation_usd)
```

`src/simulator/parity_report.py` — the section per timeframe

```
                f"- HODL end: {money(frame['baseline_usd'])} (bought at the "
                "opening bar, held to the last)",
                f"- Harvest-Fold end: {money(frame['accumulation_usd'])} "
                f"({frame['partial_exits']} partial exit(s), {frame['re_entries']} "
                "re-entr"
                + ("y" if frame["re_entries"] == 1 else "ies")
                + ")",
                f"- difference: {frame['difference_usd']:+,.2f} "
                f"({frame['difference_pct']:+.2f}% of HODL end); {frame['arithmetic']}",
                f"- historical trough: {money(frame['trough_usd'])}",
```

### The two emits

Each portfolio's start emits `sim.battery.portfolio_started` with the bots seated against the portfolio's symbol count, and its end emits `sim.battery.portfolio_finished` with the three figures, the evaluations made against the evaluations expected and the fills per timeframe; the row reads `ok` when every walk that ran made every evaluation expected of it and Stop did not end it.

`src/simulator/portfolio_battery.py` — the two signals

```
#: The signals one portfolio's start and end emit through ``signal_contract``.
PORTFOLIO_STARTED_SIGNAL = "sim.battery.portfolio_started"
PORTFOLIO_FINISHED_SIGNAL = "sim.battery.portfolio_finished"
```

### What the every-candle reading measured

Both builds, the real window over a scratch home, every socket but loopback refused, a scratch RA root holding the operator's daily files for SPY, GLD, A15, A14, BNB and SLV, and a loopback stand-in for the public candle endpoint serving 5m for A15 and A14 from 2026-04-01 and refusing every other product. Before, Run Portfolio over CRYPTO_BLUE on `2026 test run` read A15 at 5m 49,328 bars and 121 ticks, A15 at 1d 172 bars and 143 ticks, the Battery section `5m defended, 1d defended`, and no line between the retrieved line and the portfolio line; Run Every Portfolio held sixty-three bots from the press to the end and wrote one report. After, the same press read A15 at 5m 49,336 bars and 49,307 evaluations, A14 the same, A15 at 1d 172 bars and 143 evaluations, each the bars less 29, in 223 and 203 seconds at 4.5 and 4.1 seconds per 1,000 evaluations in Qt and 174 and 144 seconds at 3.5 and 2.9 in React; a line at each walk's start, every 5,000 bars and at its end; the Battery section reading, at 5m, HODL end $1,313.13, Harvest-Fold end $2,183.19, difference +870.06 (+66.26% of HODL end) with 609 partial exits and 2,769 re-entries on the stand-in's candles, and at 1d HODL end $1,093.45, Harvest-Fold end $1,096.17, difference +2.71 (+0.25%). Run Every Portfolio on the same span held no fleet at the press, no venue seated and the Get Started card drawn, then AGGRESSIVE's ten on Coinbase and Yahoo with the loaded line naming its budget of $5,000.00, then ALL_WEATHER's five on Yahoo at $2,500.00, then ARK_SUITE's five, then BALANCED's ten, and so on through thirty-five portfolios, the held fleet read empty between two of them and at the end and the fleet file's Portfolio Battery map holding no bot after the last clear (a first reading, before the press held nothing, had shown sixty-three bots at the press); thirty-six reports, one per portfolio and the summary, whose per-portfolio table lists thirty-five run ids; 844,289 evaluations of 844,289 expected in both builds, 80 minutes in Qt beside a React run of 51 minutes; the summary's totals at 5m over nine portfolios and at 1d over twenty-three read the same figures in both builds. Stop pressed on BNB's row a hundred seconds into the crypto walk ended it at bar 5,008 of 49,340 in Qt and 5,886 of 49,358 in React, the partial report naming the bar, its candle stamp and the evaluations made, and A14 and BNB as not walked. With `portfolio_battery.walk` wrapped in the driver process to step fourteen bars, A15 at 5m read 3,523 evaluations against 49,312 expected and the finished emit read `ok` false, so the reading can fail. Every copied daily file and `bot_state.json` hashed identical after every press, with a planted byte moving the hash; the operator's RA root listed 414 entries before and after; no socket left loopback. Both bundles, launched in isolation over a scratch home, ran Portfolio over EQUITY_MACRO on `All` and read SPY and GLD at 1d 1,678 bars and 1,649 evaluations each, at 1w 349 and 320, at 1M 81 and 52, and at 1d HODL end $2,513.22, Harvest-Fold end $2,001.64, difference -511.57 (-20.36% of HODL end) on the operator's own Yahoo files.

## The Simulator's run emits go to the Simulator's own signal sink

Every emit a run raises on its worker thread lands in the Simulator's own signal sink, `signals/session.jsonl` under the sim bucket, and none in the process sink under `signals/` of the log root. The operator, 2026-09-18: *"Entire idea for the Emitter Network is to provide us signals for verifying proper software function resulting from all user actions or automated sequences."* Before this, one Portfolio Battery press over one crypto portfolio at 5m emitted 1,287,112 rows into the process sink, its ladder of six files rotated through five backups, and the press row, the live pins and every row from before the press rotated out with them. The emit sites are not changed, `src/trading/ta_engine.py` is not opened, and the shared trading code does not know which side it runs on: the wire itself learned a routing by thread.

`src/core/signal_contract.py` — the routing seam

```
def route_thread(sink: SignalSink) -> None:
    """Send every `emit` raised on the calling thread to `sink` until `unroute_thread`."""
    with _ROUTES_LOCK:
        _ROUTES[threading.get_ident()] = sink


def unroute_thread() -> None:
    """Return the calling thread's emits to the process sink `set_sink` installed."""
    with _ROUTES_LOCK:
        _ROUTES.pop(threading.get_ident(), None)


def get_sink() -> Optional[SignalSink]:
    """The sink the calling thread emits into: its `route_thread` sink when one
    is set, else the process sink."""
    if _ROUTES:
        with _ROUTES_LOCK:
            routed = _ROUTES.get(threading.get_ident())
        if routed is not None:
            return routed
    with _ACTIVE_LOCK:
        return _ACTIVE
```

`emit` reads `get_sink()` and hands the record to the sink it answers, as it did before. With no route set the dict is empty, and the wire is one truth test on an empty dict, then the lock and the process sink, as before. A thread routes itself, by its own ident, and clears its own route, so a route is never left under an ident the thread no longer holds.

### The sim sink is the process sink's class over the sim bucket

The sim sink is one `SignalSink` per process, the class the process sink is, at `sim/signals/session.jsonl`, opened on the first run of the process and answered on every run after. It rotates at 50 MiB into `.1` to `.5`, keeps 350,000 rows in memory and flushes every 500 rows, the process sink's own figures, and thins its own `session.digest.jsonl` beside it as the process sink does. Each row has the shape `Signal.to_json` writes on both sides.

`src/simulator/sim_bus.py` — the sim sink

```
def sim_signal_sink() -> SignalSink:
    """The one ``SignalSink`` per process at ``SIGNALS_FILE`` under
    ``SIGNALS_DIR`` of ``get_sim_dir``, built as ``install_process_sink``
    builds the process sink and opened on the first call."""
    global _SIGNAL_SINK
    with _SIGNAL_SINK_LOCK:
        if _SIGNAL_SINK is None:
            sink = SignalSink(flush_every=PROCESS_SINK_FLUSH_EVERY)
            sink.path = get_sim_dir() / SIGNALS_DIR / SIGNALS_FILE
            _SIGNAL_SINK = sink
        return _SIGNAL_SINK
```

### Each worker marks itself for the length of its run

Every run's worker thread routes itself on its first statement and clears the route when the run returns or raises. One context, `routed_run`, does both: on entry it opens the sim sink, routes the calling thread and writes the Activity Log line; on exit it reads whether the thread's sink is still the sim sink, clears the route, flushes the sim sink and emits the routed row on the process sink. The six worker entries wrap their bodies in it: `_compute_battery`, `_compute_run` and `_compute_retrieval` on each host.

`src/simulator/sim_bus.py` — the run context

```
@contextmanager
def routed_run(run: str, line: Callable[[str], None]) -> Iterator[SignalSink]:
    """Route the calling thread's emits to ``sim_signal_sink`` for the block,
    hand ``line`` the ``SINK_LINE_FORMAT`` line on entry, and on exit unroute,
    flush the sim sink and emit ``SINK_ROUTED_SIGNAL`` on the process sink."""
    sink = sim_signal_sink()
    thread = threading.get_ident()
    rows_before = int(sink.health()["emitted"])
    route_thread(sink)
    try:
        line(SINK_LINE_FORMAT.format(run=run, path=sink.path))
        yield sink
    finally:
        routed = get_sink() is sink
        unroute_thread()
        sink.flush()
        restored = get_sink() is not sink
        emit(
            SINK_ROUTED_SIGNAL,
            actual={
                "thread": thread,
                "run": run,
                "path": str(sink.path),
                "routed": routed,
                "restored": restored,
                "rows": int(sink.health()["emitted"]) - rows_before,
            },
            expected={"routed": True, "restored": True},
            ok=routed and restored,
        )
        process_sink = get_sink()
        if process_sink is not None:
            process_sink.flush()
```

`src/gui/simulator/sim_trading_tab.py` — the Battery's worker under the context

```
        with routed_run(
            surface.MODE_PORTFOLIO_BATTERY,
            lambda line: self.battery_line.emit(line, "info"),
        ):
```

A Validation or Back Test run routes under its mode name, and the replay layer's tablet retrieval under `retrieval`. No run spawns a thread of its own: `run_battery`, `back_test.run` and `validation.run`

walk on the calling thread, and the retrieval runs its event loop on the worker thread. A route does not pass to a thread a routed thread starts.

### The tab's own presses stay on the process sink

The six pins the tab emits on the GUI thread, `sim.fleet.mode_shown`, `sim.fleet.clear_pressed`, `sim.fleet.cleared`, `sim.run.start_pressed`, `sim.replay.marks_drawn` and `sim.layer.flipped`, are the operator's actions on the running program, and they land on the process sink as before, beside the live bots' pins and the Console's own. The Indicator Voting Panel's feed, `ivp_feed`, computes the selected bot's stored reading on the GUI thread, so the thirteen postcondition pins of that one evaluation stay on the process sink too. The Console's lower pane reads the process sink on the GUI thread, so it draws those and never the walk.

`src/gui/simulator/sim_trading_tab.py` — a GUI-thread pin, unchanged

```
        _pin_emit(
            tab_surface.LAYER_FLIPPED_SIGNAL,
            **tab_surface.flipped_pin(
                leaving, self._layer, pressed, self._last_flip_rect
            ),
        )
```

### The routed row and the Activity Log line

At each run's end one `sim.sink.routed` row lands on the process sink. It carries the worker's thread ident, the run name, the sim sink's path, whether the thread's sink still read the sim sink when the run ended, whether it reads the process sink again after the unroute, and the rows the sim sink took over the run; it reads `ok` when both verdicts hold. A seam that stops consulting the route reads `routed` false, and one that does not clear it reads `restored` false. At each run's start the Activity Log names the sim sink's path, after the started line and before the first walk line.

`src/simulator/sim_bus.py` — the row and the line

```
#: The row ``routed_run`` emits on the process sink at each run's end.
SINK_ROUTED_SIGNAL = "sim.sink.routed"

#: The Activity Log line ``routed_run`` writes at each run's start.
SINK_LINE_FORMAT = "{run} signals: {path}"
```

The Watchdog Archetype reads source, not a sink: a pin is wired when its name resolves to `signal_contract.emit`, and the routing lives inside that wire, so a sim emit site reads wired on the same rule as a live one and the archetype needs no sink named.

Diagram: flowchart, left to right

- Run Portfolio on the GUI thread -> **the started line, the press row on the process sink**
- the started line, the press row on the process sink -> **_compute_battery on sim-portfolio-battery**
- _compute_battery on sim-portfolio-battery -> **routed_run: route_thread, the signals line**
- routed_run: route_thread, the signals line -> **run_battery, walk, compute_all**
- run_battery, walk, compute_all -> **_ta_emit through signal_contract.emit**

- _ta_emit through signal_contract.emit -> **get_sink**
- get_sink -> **sim/signals/session.jsonl** *(routed thread)*
- get_sink -> **signals/session.jsonl** *(any other thread)*
- run_battery, walk, compute_all -> **finally: unroute_thread, flush**
- finally: unroute_thread, flush -> **sim.sink.routed on the process sink**

### What the sink reading measured

Both builds, the real window over a scratch home, every socket but loopback refused, the process sink installed as `main` installs it, unit 31's wide seed and a loopback stand-in for the public candle endpoint. Before, a Back Test over the 24 seeded bots took the process sink from 23 rows and 12,464 bytes at the press to 61,824 rows and 42,183,886 bytes at the end, 61,802 of the new rows the TA engine's, with a planted emit on the worker thread beside them; Run Portfolio over CRYPTO_BLUE on `2026 test run` emitted 1,287,112 rows, the ladder held 445,060 rows and 305,926,618 bytes in six files when the run ended, the sink's own health read 937,112 evicted, and the press row, the planted worker row and the planted GUI row had rotated out; no sim sink existed. After, the same Back Test left the process sink at 45 rows in Qt and 42 in React, the rows above the press watermark the press row, the panel's one evaluation, the Console's own pins, the planted GUI row and the routed row, while the sim sink took 61,777 rows and 42,157,581 bytes, 61,776 of them the TA engine's and one the planted worker row; the same Battery press left the process sink at 60 rows in Qt and 57 in React with zero evicted, no `ta.*` row and no `sim.battery.*` row above the press watermark, while the sim sink took 1,287,136 rows in Qt and 1,287,162 in React, its ladder holding 445,136 and 445,162 rows across six files at the end, its last row `sim.battery.portfolio_finished`. The routed row read `ok` with `routed` and `restored` true and `rows` 61,777 for the Back Test, 1,287,136 and 1,287,162 for the Battery, and 0 for Retrieve Tablet, whose connector emits nothing. The Activity Log named the sim sink's path after each started line. A planted emit on the GUI thread landed on the process sink and not the sim sink; the GUI thread routed on purpose for one emit landed that emit on the sim sink, so the reading can fail; a helper thread that routed itself and exited left the next GUI emit on the process sink; the flip's row landed on the process sink. The scratch `bot_state.json` and every copied daily file hashed identical after every press, with a planted byte moving the hash; no socket left loopback.

## The widget the rebuild replaced

Every section from here to the end of the page describes a screen that is gone: the empty Sim tab, the original Simulator widget with Fleet Replay, Nuclear Mode and the Bridge, and the first clone units 7 and 8 replaced. Each is kept whole as the record of what the rebuild replaced, and nothing in them describes the tab that draws.

## The empty Sim tab before the fork

The Simulator is removed. The tab is named Sim, it opens first on the bar, and it draws an empty panel with a heading and two sentences. Nothing behind it runs: no fleet, no replay, no practice venue and no Nuclear Mode. Every section below this one describes the screen that was removed, and is kept as the record of what the rebuild replaces.

The panel is the one the Paper, Status and Accumulation tabs already draw. Its words come from a view model, so the Qt build and the React build say the same thing.

```
METHOD = "simulator_tab.state"

HEADING = "Sim"
ISSUE = 117
BUILT = False
STATE_TEXT = "This tab is not built."
ISSUE_TEXT = f"Issue #{ISSUE} carries the build-out."
```

## What builds it

The window constructs the tab and inserts it beside Trading. Three of the four wiring calls take a lambda, so the exchange connectors, the swarm and the Market Inspector proposals resolve when they are asked for rather than when the tab is built. The bot manager is handed over directly.

`src/gui/main_tabs/simulator_tab.py` — `SimulatorTabMixin._build_simulator_tab` The Simulator rebuild removed this file; it is not in the tree.

```
        self._simulator = SimulatorTab()
        # _reorder_main_tabs runs after every builder, so index 1 is not the final slot.
        self._main_tabs.insertTab(1, self._simulator, "Simulator")
        if hasattr(self._simulator, "set_connectors_getter"):
            self._simulator.set_connectors_getter(
                lambda: getattr(self, "_exchange_connectors", {})
            )
        if hasattr(self._simulator, "set_bot_manager"):
            self._simulator.set_bot_manager(self._bot_manager)
```

The tab is isolated. The window's header strip hides while the Simulator is in front, and the tab draws its own strip of the same ten fields against sim balances.

`src/gui/simulator_tab/sim_stat_strip.py` — `SimStatStrip.FIELDS` The Simulator rebuild removed this file; it is not in the tree.

```
    FIELDS = (
        "Spendable",
        "Realised",
        "Locked",
        "Mature",
        "Exch",
        "Scrummed",
        "Folded",
        "Trades",
        "Bots",
        "Errors",
    )
```

The Mode picker names three modes and drives two panels. Nuclear raises its own page. Validation and Looping Back Test both raise Fleet Replay.

`src/gui/simulator_tab/simulator_tab.py` — `SimulatorTab._on_sim_mode_changed` The Simulator rebuild removed this file; it is not in the tree.

```
        key = self.sim_mode()
        stack = getattr(self, "_stack", None)
        if stack is not None:
            # Validation and Looping both drive the fleet page.
            stack.setCurrentIndex(1 if key == "nuclear" else 0)
```

The mode goes no further than the picker. The handler passes the key on only to a panel that carries a matching setter, and Fleet Replay carries none, so the two fleet modes run the same replay and the choice changes the hint line alone.

`src/gui/simulator_tab/simulator_tab.py` — the mode hand-off The Simulator rebuild removed this file; it is not in the tree.

```
        panel = getattr(self, "fleet_replay", None)
        if panel is not None and hasattr(panel, "set_sim_mode"):
            try:
                panel.set_sim_mode(key)
```

## Fleet Replay

`FleetReplayPanel` in `src/gui/simulator_tab/fleet/fleet_replay_panel.py` is the panel. Four steps run it. The Simulator rebuild removed this file; it is not in the tree.

**Load.** The panel builds one real bot for every saved config whose symbol has a Stone Tablet, and names the symbols it had to skip. The class body is live's own, so the sim runs live code against a stored tape rather than a second implementation; the single sim-only branch gives each bot a private event bus, which is why a sim fill never reaches the live sound engine or the live logs.

`src/gui/simulator_tab/fleet/fleet_replay_panel.py` — `_spawn_sim_fleet` The Simulator rebuild removed this file; it is not in the tree.

```
        def _spawn_sim_fleet(self) -> int:
            """Construct a real sim bot for every loaded config.

            Returns the number spawned. Reads the Stone Tablet registry
            through the same call Start Replay uses, rather than adding a
            second candle path.

            The controller is built but NOT started: the bots exist, hold
            their imported state and can be inspected, and the tape only
            advances when the operator presses Start Replay.
            """
```

**Configs.** The loader reads the operator's state file read-only and returns each bot's saved config. `load_smart_wires_from_state` does the same for the topology, and `summarize_loaded_configs` reports what loaded. The fleet is never fabricated; it is whatever that file holds.

`src/simulator/fleet/bot_state_loader.py` — `load_bot_configs_from_state` The Simulator rebuild removed this file; it is not in the tree.

```
def load_bot_configs_from_state(
    path: Optional[Path] = None,
    mode_filter: Optional[str] = "scrumming",
) -> list[dict[str, Any]]:
```

**Fetch.** Fetch YTD calls the same history function the History tab's Refresh calls, so the two return the same trades. A replay still runs without it, and the panel then says on screen that the run is synthetic.

`src/gui/simulator_tab/fleet/fleet_replay_panel.py` — `_on_fetch_ytd_clicked` The Simulator rebuild removed this file; it is not in the tree.

```
        def _on_fetch_ytd_clicked(self) -> None:
            """Call the same function the History tab calls.

            The History tab's Refresh dispatches
            ``fetch_all_history_chunked(bot_manager, since_ts)``. No custom
            enumeration, no exchange matching, no connector picking: if the
            History tab returns N trades, Fleet Replay returns the same N.
            """
```

**Run.** `FleetReplayController` in `src/simulator/fleet/fleet_replay_controller.py` drives it. The Simulator rebuild removed this file; it is not in the tree.

| Piece | What it does |
|---|---|
| `_build_sim` | Assembles the bots and the tape |
| `sim_bot_id`, `live_bot_id` | Keep the two id spaces apart |
| `_on_sim_trade` | Records each fill |

One assertion runs once, at the point where the bot set is final and before any of them can trade.

`src/simulator/fleet/fleet_replay_controller.py` — `_assert_capital_isolation` The Simulator rebuild removed this file; it is not in the tree.

```
    def _assert_capital_isolation(self) -> None:
        """Every constructed sim bot must be off the live registry.

        The guards upstream -- the factory raising, `_instantiate_bot`
        refusing, `_crr()` returning None in sim mode -- each protect
        one path. This checks the outcome they exist to produce, once,
        at the point where the bot set is final and before any of them
        can trade. Aborts the run rather than let a replay write sim
        reservations into the operator's live capital state.
        """
```

The bots trade against a tape, not against a venue. `TabletBackend` implements the ccxt surface beneath a real connector, holds the seeded balances, sweeps resting orders, and owns the one clock every symbol advances on.

`src/simulator/fleet/fleet_replay_controller.py` — the tape the bots are handed The Simulator rebuild removed this file; it is not in the tree.

```
        # TabletBackend implements the ccxt surface beneath a real CCXTConnector.
        self._tape = TabletBackend(
            {
                sym: [list(r) for r in rows]
                for sym, rows in self._candles_by_symbol.items()
            },
            balances=dict(seed_by_quote),
            fee_rate_by_symbol=_fees,
        )
        _conn = CCXTConnector("coinbase")
        _conn.attach_backend(self._tape)
        self._exchange = _conn
```

Three more modules sit beside the controller and none of them runs a replay. The older fake exchange is built only by tests, the union clock is constructed only inside that exchange, and the candle series a replay builds is thrown away once its symbol names have been read.

`src/simulator/fleet/sim_exchange.py` — `FleetSimExchange` The Simulator rebuild removed this file; it is not in the tree.

```
"""Candle-driven fake exchange for Fleet Replay, over real symbols.

Serves per-symbol ``CandleSeries`` through the ``ExchangeInterface`` API,
so a bot trades A15/USD or A14/USD against stored candles instead of a
venue. ``NuclearSimExchange`` (``src/simulator/nuclear_sim_exchange.py``)
is the other simulated venue and serves synthetic TAPEA/TAPEB symbols.
The Simulator rebuild removed this file; it is not in the tree.

Reachability: nothing under ``src/`` constructs ``FleetSimExchange``.
``FleetReplayController`` imports only ``make_symbol_series_map`` from
this module and runs against ``TabletBackend``
(``src/exchange/tablet_backend.py``). The class itself is built by tests.
```

`ReplayProgress` carries candles played, trades fired and tick failures back to the panel's progress timer, and a finished run is written under the log root in the live schema.

`src/trading/sim_run_log.py` — `SimRunLog` The Simulator rebuild removed this file; it is not in the tree.

```
"""Persist a Fleet Replay run under ~/.acervator_logs/sim/ in the live schema."""
```

Two bot tables draw at once. The Trading tab's own table is mounted into the Simulator, and the fleet panel's three-column table is still built and still filled. Collapsing the two is the first of the seven pieces of issue #117.

`src/gui/simulator_tab/fleet/fleet_replay_panel.py` — `FleetReplayPanel.COLUMNS` The Simulator rebuild removed this file; it is not in the tree.

```
        # Three columns. The gate row lives in GateStatusPanel, not here.
        COLUMNS = ("Symbol", "Target USD", "Sim Trades")
```

## The validation criterion

The Simulator is validated when the gates latch identically on the same data. Not profit and loss, and not the trade count.

No module measures that. Nothing in the tree reads a sim gate latch and a live gate latch and compares the two.

In development. Validation Mode measures it now; the section at the end of this page states how.

What the screen does show is each bot's arm state while the tape runs. On every refresh tick the panel reads each bot's last gate state and paints a double-stacked row of lights, ten scrum then nine fold.

`src/gui/simulator_tab/fleet/sim_visuals.py` — `GateLightsCell` The Simulator rebuild removed this file; it is not in the tree.

```
    class GateLightsCell(QWidget):
        """One linear labelled row of trading gates.

        ``_draw_bank`` paints the ten ``_GATE_ORDER_SCRUM`` lights then the
        nine ``_GATE_ORDER_FOLD`` lights, each below its own label.
        ``gate_light_color`` picks every colour, and ``update_gates`` sets the
        arm and blocker state ``paintEvent`` reads.
        """
```

That cell and the History table's gate cell read one vocabulary, so the two surfaces cannot drift apart on what a gate is called or on what blocked it.

`src/trading/gate_vocabulary.py` — the shared labels

```
"""gate_vocabulary.py -- the bot's gate labels, blocker map and light states.

Qt-free. Two renderers read it: the Simulator's ``GateLightsCell``
(``src/gui/simulator_tab/fleet/sim_visuals.py``) and the History table's
gate cell (``src/exchange/history_read_contract.py``). One vocabulary,
so a gate added on one surface cannot be missing from the other.
```

The Simulator rebuild removed this file; it is not in the tree.

After a run the panel matches each live trade to the nearest sim fill on the same symbol and side, and prints the result into the Performance log. That is a report on trade counts and timing. It is not the criterion above.

`src/trading/stone_tablets/parity_harness.py` — `compare_trades`

```
def compare_trades(
    live_trades: list[dict],
    sim_trades: list[Any],
    tolerance_s: float = DEFAULT_TOLERANCE_S,
    window_since_ts: float = 0.0,
    window_until_ts: float = 0.0,
) -> ParityReport:
```

The panel puts the honest label on screen. A run with no live trades loaded is comparable to nothing, and the panel says so rather than looking like a parity run.

`src/gui/simulator_tab/fleet/fleet_replay_panel.py` — `_note_parity_state` The Simulator rebuild removed this file; it is not in the tree.

```
        def _note_parity_state(self) -> None:
            """Say so when the run cannot be compared to live.

            ``_ytd_trades`` being empty is the NORMAL condition, and the panel
            said nothing about it, so a synthetic run looked exactly like a
            parity run on screen.
            """
```

`SimValidationGuard` names five conditions under which a run cannot be trusted. The last two halt immediately whatever their count. The guard is defined and covered by tests, and no module under `src/` calls it.

`src/trading/sim_validation_guard.py` — `ValidationIssueType` The Simulator rebuild removed this file; it is not in the tree.

```
class ValidationIssueType:
    NO_TABLET = "no_tablet"
    PRE_LISTING = "pre_listing"
    UNRESOLVABLE_TS = "unresolvable_timestamp"
    SCHEMA_DRIFT = "schema_drift"
    ADDRESS_MISMATCH = "address_mismatch"
```

The voting readout beside the fleet can fill itself with invented numbers. Three seconds after the tab is built, with no bot yet in its selector, it runs the real voting engine over a seeded random walk and draws the vote. On the Simulator that is the normal state until a fleet loads, so the panel can read as a result when it is a placeholder.

`src/gui/indicator_panel.py` — `IndicatorVotingPanel._generate_demo_ta`

```
        def _generate_demo_ta(self):
            """Generate demo TA from synthetic candles.

            Gated by ``_may_fabricate()``. Produces a deterministic
            random walk (``seed = md5(bot_id)``) and runs the real
            VotingEngine over it, labelled with the real bot's symbol
            so the panel stays stable across refreshes.
            """
```

## Nuclear Mode

`NuclearModePanel` drove the controller. The controller looped the same state-file fleet over the tablet window until stopped, recording each cycle so a late failure traced back to the cycle that produced it. Start needed the window's async loop; without it the panel said so and no soak began. The operator cancelled Nuclear Mode and the rebuild deleted its code, so none of the files this section names is in the tree.

`NuclearFleetController`, in the Simulator package the rebuild deleted

```
class NuclearFleetController:
    """Loops the bot_state fleet over Stone Tablet history until stopped.

    GUI-agnostic: all operator-visible output goes through the callbacks,
    so this is testable without Qt.
    """
```

The market structure varied per loop, and nothing on disk was touched.

`noised_series`, in the candle source the rebuild deleted

```
def noised_series(
    rows: list,
    seed: int,
) -> tuple[list, float]:
    """Return ``(noised_rows, noise_pct)`` for one pass over *rows*.

    A ``_Tape`` seeded from *seed* perturbs a copy through ``_at``; *rows* is
    left as it was and nothing here touches disk.
    """
```

A second oscillator supplied the load pulse, and it was capped while its cooling regime held, because the pulse shared a machine with the live trading engine. That module is still in the tree and nothing constructs it.

`src/core/system_load_oscillator.py` — `SystemLoadOscillator`

```
class SystemLoadOscillator:
    """Dual oscillator for Nuclear Mode tick rate and per-tick workload.

    ``start`` takes the monotonic reference and spawns the COOLING
    daemon; ``stop`` joins it and is idempotent. ``current_multiplier``
    returns ``BASE_MULTIPLIER`` before ``start``.
    """
```

`set_swarm_hooks` connected the controller to three methods on the Swarm tab, which is how a nuclear run drew its rows in the sim layer of that swarm. `set_topologies` replayed a Market Inspector proposal shape across the sim bots. Neither name is in the tree now.

| Hook | Fires when |
|---|---|
| `register_sim_run` | A run starts |

| Hook | Fires when |
|---|---|
| `update_sim_run` | A cycle reports |
| `stop_sim_run` | The run ends |

The panel's own visuals stayed empty for a whole run. It fed the shared price chart and the shared voting readout from four fields — the symbol, the last price, the last volume and the voting summary — and the controller's snapshot carried none of them.

`NuclearFleetController.snapshot`, in the controller the rebuild deleted

```
    def snapshot(self) -> dict:
        """Return the GUI panel's polled view of this soak."""
        s = self.state
        return {
            "running": s.running,
            "uptime_seconds": s.uptime_s,
            "fleet_size": s.fleet_size,
            "symbols": s.symbols,
            "cycles_completed": s.cycles_completed,
            "current_cycle": s.current_cycle,
            "noise_pct": s.noise_pct,
            "wires_loaded": len(self._smart_wires),
            "total_candles": s.total_candles,
            "total_trades": s.total_trades,
            "total_exceptions": s.total_exceptions,
            "load_multiplier": s.load_multiplier,
            "cooling": s.cooling,
            "load_sensed": self._sensed,
            "last_error": s.last_error,
            "failed_cycles": sum(1 for c in s.cycles if not c.ok),
        }
```

Nuclear was not a validator. It was to run after trade-logic alignment was earned, its noised tape was deliberately not history, and its criteria were coverage and survival. The operator cancelled it before that point was reached.

`NuclearModePanel`, in the panel the rebuild deleted — what it was for

```
NOT A VALIDATOR. Nuclear runs AFTER trade-logic alignment is proven on
the real tablets, and its noised tape is deliberately not history, so
nothing here compares its output to YTD or live. Its criteria are
coverage and survival.
```

`NuclearController` was the earlier single-tape prototype: one scout bot walking one tape. Nothing under the source tree constructed it, and it stayed because both controllers shared the noise source.

`NuclearModePanel`, in the panel the rebuild deleted — why the prototype stayed

```
`nuclear_controller.py` and `nuclear_candle_source.py` are deliberately
NOT deleted: the latter owns `noised_series`, which v2 depends on for
exactly the market-structure noise above.
```

That comment recorded the state before the cancellation. Every file named in this section was deleted with the rest of the old Simulator, and no Nuclear Mode runs.

## Bridge

Five methods are registered for this screen, and the renderer modules carry the matching names. They serve the Electron frontend; the tab the operator runs today is drawn in Qt.

| Bridge method | Serves |
|---|---|
| `simulator_tab.state` | The tab, its mode switcher and its table |
| `fleet_replay_panel.state` | Fleet Replay |
| `nuclear_mode_panel.state` | Nuclear Mode |
| `sim_stat_strip.state` | The tab's own ten-field strip |
| `sim_visuals.state` | The gate lights and the sim charts |

`src/core/desktop_bridge.py` — `build_registry`

```
def build_registry() -> Dict[str, Handler]:
    """Return the methods the frontend may call, keyed by method name.

    This is the wiring point for the whole frontend: a surface is
    reachable exactly when it appears here. Surfaces are imported inside
    the function so that the transport above carries no dependency on any
    one domain package.
    """
```

## 2026-09-08 08:17 - #117 - what the closed issues landed

The old Simulator was removed and the rebuild replaced it. The tab is named Sim, it opens first on the bar on a black ground, and it draws a clone of the Live tab reading Stone Tablets. It carries three modes — Validation, Back Test and Portfolio Battery — and Nuclear Mode is cancelled. The rest of this section describes the screen that was removed and is kept as the record of what the rebuild replaces.

```
def _build_simulator_tab(self) -> None:
    """Insert the Sim tab at ``SIMULATOR_BUILD_INDEX``."""
    self._add_empty_tab(simulator, index=SIMULATOR_BUILD_INDEX)
```

The tab stacks two panels. Fleet Replay loads every bot from the operator's own state file, builds one real bot per config, and plays Stone Tablet candles through them against a fake exchange. The sim uses the bot class body unchanged, which is the parity guarantee: it runs live's code against a fake exchange rather than a second implementation.

The tab is now called Sim. It sits first on the bar, on a black ground with red text.

`src/gui/simulator_tab/fleet/fleet_replay_panel.py` — `_spawn_sim_fleet` The Simulator rebuild removed this file; it is not in the tree.

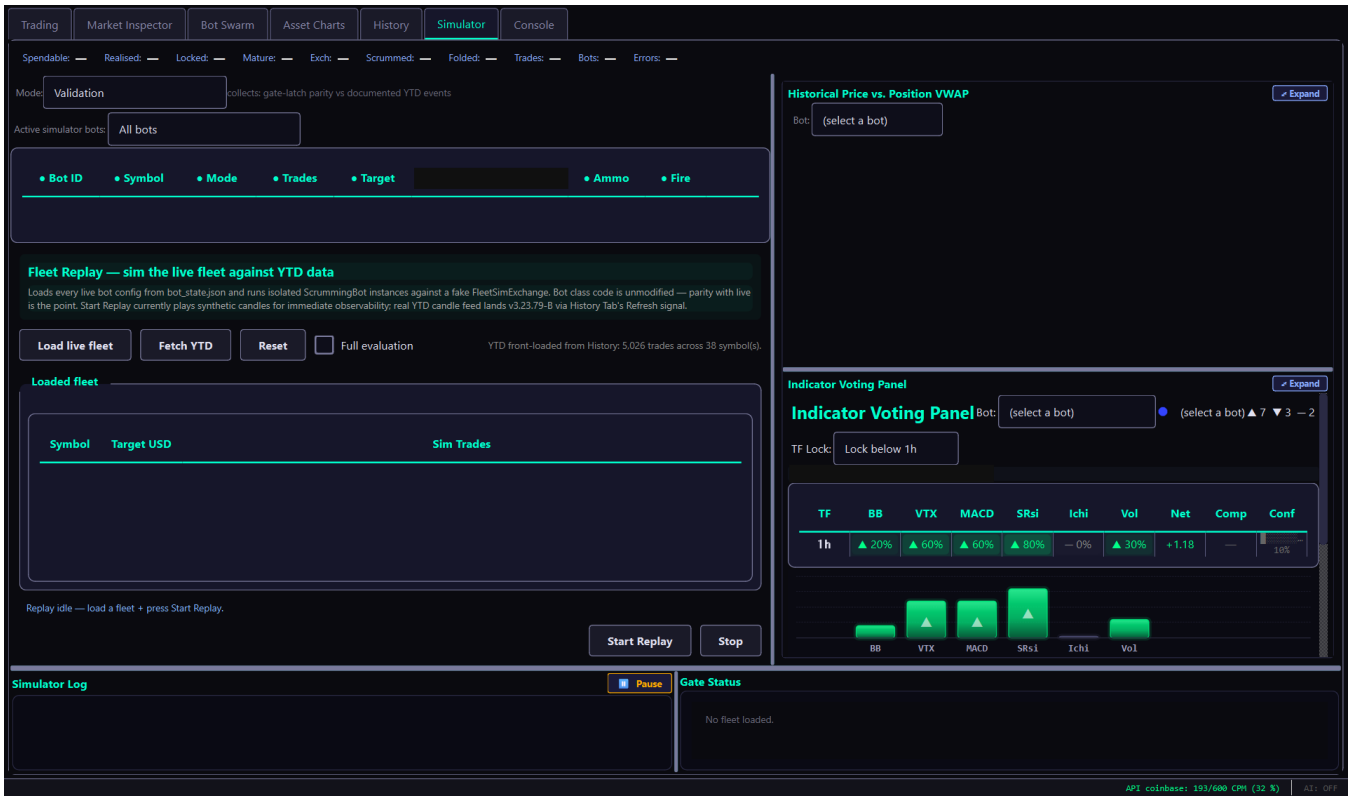
```
def _spawn_sim_fleet(self) -> int:
    """Construct a real sim bot for every loaded config.

    Returns the number spawned. Reads the Stone Tablet registry
    through the same call Start Replay uses, rather than adding a
    second candle path.
```

The criterion is the gates latching identically on the same data. Not profit and loss, and not the trade count. The sim's gate cell reads the same vocabulary the History table reads, which stops the two surfaces drifting apart.

`src/gui/simulator_tab/fleet/sim_visuals.py` — `GateLightsCell` The Simulator rebuild removed this file; it is not in the tree.

```
class GateLightsCell(QWidget):
    """One linear labelled row of trading gates.
```

Nuclear Mode looped the same fleet over the tablet window with per-cycle market noise and a load pulse, writing over no tablet. It stood as a soak test, judged on coverage and survival, and it compared nothing to live. The operator cancelled it, its code is deleted, and the Sim tab has three modes, not four.



The Simulator tab, with no fleet loaded.

The strip along the top replaces the window's own while this tab is active. It names the same ten readings against sim balances, and every one draws an em dash in the figure, because no fleet is loaded.

React draws that strip. The tab asks `variant_surface` which of the two strips to build, and both answer the same `set` and `clear` calls, so the fleet panel and Nuclear Mode write to either without knowing which they hold.

`src/gui/simulator_tab/simulator_tab.py` — `SimulatorTab._build_stat_strip` The Simulator rebuild removed this file; it is not in the tree.

```
from ..variant_surface import SIM_STAT_STRIP, surface_class

return surface_class(SIM_STAT_STRIP)(self)
```

`SimStatStripWebStrip` holds `SimStatStripModel` where the Qt strip held ten label pairs, and draws it with `sim_stat_strip.js` in one `QWebEngineView`. A field name the strip does not carry is still ignored, and an empty value still falls back to the em dash. Setting `ACERVATOR_VARIANT` to `qt` builds the Qt strip instead, unchanged.

React drew the Nuclear Mode page too, before the rebuild deleted both. The tab asked the same seam for the panel class that it asked for the strip.

`src/gui/simulator_tab/simulator_tab.py` — `SimulatorTab._nuclear_panel_class` The Simulator rebuild removed this file; it is not in the tree.

```
from ..variant_surface import NUCLEAR_MODE, surface_class

return surface_class(NUCLEAR_MODE)
```

`NuclearModeReactPanel` inherits the Qt panel, so the fleet preview, Start, Stop and the half-second status tick are one piece of Python on both sides. Every widget write moved behind a named accessor. The Qt panel answers those with labels, spin boxes and tick boxes; the React panel answers them by writing into its view model and redrawing the page.

`src/gui/react_nuclear_mode_panel.py` — `NuclearModeReactPanel._set_status_text` The Simulator rebuild removed this file; it is not in the tree.

```
def _set_status_text(self, key: str, text: str) -> None:
    """Show ``text`` on the live-status row ``key`` names."""
    if key in self._panel.status_text:
        self._panel.status_text[key] = text
        self.push()
```

The page draws the header card, the fleet readout, the four run settings, the Start and Stop buttons and the seventeen live-status rows. A press on the page runs the inherited Python and comes back as a redraw. Setting the variant to `qt` builds the Qt panel instead, unchanged.

The two drawn panels follow the same route. `SimPriceVwapChartReact` inherits the price and VWAP chart, so the ticks, the thinning and the trade markers stay one piece of Python, and `GateStatusPanelReact` inherits the gate pane.

`src/gui/simulator_tab/simulator_tab.py` — `SimulatorTab._price_chart_class` The Simulator rebuild removed this file; it is not in the tree.

```
from ..variant_surface import SIM_PRICE_CHART, surface_class

return surface_class(SIM_PRICE_CHART)
```

Neither panel is redrawn in JavaScript. Python builds the whole draw program — every line, box, dot and label, with its colour and its position — and `sim_visuals.js` runs that program on one canvas. The picture is decided on the Python side, so the two builds cannot draw different charts from the same ticks.

`src/gui/main_tabs/sim_visuals_surface.py` — `chart_program` The Simulator rebuild removed this file; it is not in the tree.

```
if not model.symbols:
    return []
if model.focus:
    return focused_program(model, width_px, height_px)
return band_program(model, width_px)
```

The gate pane draws one labelled row per bot and nineteen lights on each, and a gate reading written from Python changes what those lights show. An empty pane still says "No fleet loaded."

The Fleet Replay page is the last of the four. `FleetReplayReactPanel` inherits the Qt panel, so Load live fleet, Fetch YTD, Reset, Start Replay, Stop and both timers are the same Python. Every widget write the panel used to make moved behind a named accessor, and the React panel answers those by writing into its view model.

`src/gui/simulator_tab/simulator_tab.py` — `SimulatorTab._fleet_replay_class` The Simulator rebuild removed this file; it is not in the tree.

```
from ..variant_surface import FLEET_REPLAY, surface_class

return surface_class(FLEET_REPLAY)
```

A press on the page names its own action. The panel maps that name to the method the Qt button was wired to, looked up on the panel itself, so the two sides run one piece of code.

`src/gui/react_fleet_replay_panel.py` — `FleetReplayReactPanel.run_action` The Simulator rebuild removed this file; it is not in the tree.

```
step = surface.ACTIONS.get(str(request.get(ACTION_KEY) or ""))
named = self.STEP_RUNNERS.get(step)
if named is not None:
    getattr(self, named)()
```

Everything else on this tab is still drawn by Qt.

Mode is the picker beside it. Three modes, each with its own line saying what it collects.

`src/gui/simulator_tab/simulator_tab.py` — `SimulatorTab.SIM_MODES` The Simulator rebuild removed this file; it is not in the tree.

```
SIM_MODES = (
    (
        "Validation",
        "validation",
        "Stone Tablets paired with YTD data. Verifies trade-gate "
        "parity at documented events.",
    ),
    (
        "Looping Back Test",
        "looping",
        "Loops the tablets with market-restructuring noise at a fixed "
        "rate. Strategy development and calibration.",
    ),
    (
        "Nuclear",
        "nuclear",
        "Load oscillation, swarm injection and high-traffic smart "
        "wire. Measures performance, stability and reliability — not "
        "trade validity.",
    ),
)
```

The table under it is the same class the Trading tab uses, with the same ten columns and a privacy dot on each header.

Column 2 is now Current Position Value, priced from the exchange, and the state colour it used to carry sits on the Bot ID cell. The Trading tab page describes both — see The bot tables.

`src/gui/widgets/bot_status_table.py` — `BotStatusTable.SCRUMMING_COLUMNS`

```
SCRUMMING_COLUMNS = ColumnSpec(
    labels=(
        "Bot ID",
        "Symbol",
        "Mode",
        "Trades",
        "Target",
        "Target A15",
        "Target A14",
        "Ammo",
        "Fire",
        "",
    ),
```

Fleet Replay fills the left column. Load live fleet builds the bots, Fetch YTD pulls the year of live trades the run is compared against, and Reset clears both. Full evaluation widens the run. The line beside the buttons counts the trades and symbols the History tab front-loaded. Loaded fleet carries three columns: Symbol, Target USD and Sim Trades, and the last of those increments during a run. The progress label reads `Replay idle` until Start Replay runs, and Stop drains the current tick and exits.

The panel says so out loud when a run cannot be compared to live, rather than letting a synthetic run look like a parity run.

`src/gui/simulator_tab/fleet/fleet_replay_panel.py` — `_note_parity_state` The Simulator rebuild removed this file; it is not in the tree.

```
def _note_parity_state(self) -> None:
    """Say so when the run cannot be compared to live.
```

The right column stacks two panels, each with its own Expand button. The first charts historical price against position VWAP for the bot its picker names, drawn from that bot's Stone Tablet candles. The second is the same Indicator Voting Panel the Trading tab carries: a bot picker, the vote count badge, the TF Lock, the currency rate line, then the first six voters — BB, VTX, MACD, SRsi, Ichi and Vol — with Net, Comp and Conf, and the confidence bars under them. The remaining six, Sling, ADX, STrd, ZSc, KER and RSI, sit below the area the figure shows.

Simulator Log and Gate Status close the tab. Gate Status draws the same nineteen lights the History table draws, and `No fleet loaded.` stands in its place until a fleet loads.

### What is on the tab now

The rebuild's first tab is on the bar. It is a clone of the Trading tab reading Stone Tablets: the bot list with the live table's own ten columns, the Indicator Voting Panel, and a second layer holding the VWAP window over the tablet playback window, with one button between them. The crypto news ticker and the data pool line are not copied, and the rows they held stay empty for Import Live Fleet and Generate From YTD.

`src/gui/main_tabs/simulator_tab_surface.py` — the candle window

```
#: The candle window live reads. ``ScrummingBot`` asks ``get_ohlcv`` for 100,
#: and the Simulator reads the same count off the tablet.
WINDOW_CANDLES = 100
```

The Simulator receives and asks; it never sends. Its one data path reads tablet files, holds no venue, and refuses by name anything that is not a read.

### The buttons that start a Validation run

Those two rows now carry the buttons that start a run. Validation snaps each YTD trade to the historical candle its timestamp falls in, reruns the gates on that candle, and puts the gate row it latched beside the gate row the log recorded. It passes when the gates latch identically, and it is never judged on profit or on a trade count.

```
Import Live Fleet    38 bots from the bot_state load, matched by bot id
Generate From YTD    39 new bots, one per traded pair, matched by pair
```

A run always reports what it could not verify. The tablets end on 1 August 2026 and the trade export runs to 7 September, so about five weeks of trades have no candle to snap to. The pane names that period and counts the entries on both sides of it.

```
3942 of 4904 YTD entries snapped to a candle; 962 could not be.
uncovered span 2026-08-01T18:52:58Z to 2026-09-07T21:46:48Z
```

## The first clone, before the fork

The Sim tab is a clone of the Trading tab, and its data source is the Stone Tablets on disk. It carries the bot list, the Indicator Voting Panel, and a second layer holding the VWAP window over the tablet playback window. One button flips the panel area between those two layers.

`src/gui/main_tabs/simulator_tab_surface.py` — the two layers and the button

```
LAYER_INDICATORS = "indicators"
LAYER_PLAYBACK = "playback"
LAYERS = (LAYER_INDICATORS, LAYER_PLAYBACK)

FLIP_BUTTON_TEXT = {
    LAYER_INDICATORS: "Show Playback",
    LAYER_PLAYBACK: "Show Indicators",
}
```

One model is built from the tablet, and both builds draw it.

Diagram: flowchart, left to right

- RA-StoneTablet files -> **TabletSource**
- TabletSource -> **simulator_tab_surface**
- simulator_tab_surface -> **SimulatorTabQt**
- simulator_tab_surface -> **simulator_tab.js**

### It receives and asks. It never sends.

The Simulator's whole data path is one class that reads tablet files. It holds no venue and defines no write. It answers five names and refuses every other name itself, so a send cannot be expressed through it.

`src/simulator/tablet_source.py` — the refusal

```
    def __getattr__(self, name: str):
        """Refuse every name outside ``READ_NAMES``."""
        raise SendRefused(
            f"TabletSource answers {READ_NAMES} and cannot {name!r}. "
            "The Simulator receives and asks; it sends nothing."
        )
```

Asked for nine venue calls — an order, a market buy, a cancellation, an edit, a withdrawal, a leverage change, a transfer, a tablet write and a save — it refused all nine and answered every read.

### The columns are the Trading tab's own

The bot list draws the ten columns the live table draws, and the same two fixed widths, because the surface imports them rather than restating them. A column added to the live table appears here with no second edit.

```
from .bot_status_table_surface import COLUMN_LABELS, FIXED_WIDTHS
```

No simulated fleet exists yet, so the list holds no rows and says why. Import Live Fleet and Generate From YTD are what fill it, and both are later units.

### One candle window, on both sides

The live engine asks the venue for one hundred candles. The Simulator reads the last hundred rows off the tablet, so the indicator window is one number on both sides rather than two.

```
#: The candle window live reads. ``ScrummingBot`` asks ``get_ohlcv`` for 100,
#: and the Simulator reads the same count off the tablet.
WINDOW_CANDLES = 100
```

The live engine now receives that hundred, and the newest row in it is the bar that just closed. The connector sends the count in the exchange library's count slot. Until this change it sent the count in the start-time slot and left the count slot empty, so the venue answered with a page of its own and the library kept the oldest 300 rows of it. Every indicator then read a window ending 50 five-minute bars behind the price the gate compared it against. That is four hours and ten minutes.

`src/exchange/ccxt_connector.py` — four arguments, each in its own slot

```
data = await self._call_sync(
    self._ex.fetch_ohlcv,
    symbol,
    timeframe,
    None if since is None else int(since),
    int(limit),
)
```

The library's own slice was driven over 1,215 recorded five-minute pages. The window ended 50 bars early on every page before the change and on none after it. On 162 rows of the recorded gate log, the band latch differs between the stale window and the current one on 88.

### The VWAP window

VWAP is the published cumulative figure: typical price times volume, running, divided by running volume, where typical price is the average of the high, the low and the close. A point with no volume behind it yet carries nothing rather than a number.

```
def vwap_series(candles: Sequence[Sequence[float]]) -> list[Optional[float]]:
    """Cumulative ``sum(typical_price * volume) / sum(volume)`` per row.

    A row whose cumulative volume is still zero carries None.
    """
```

### The two strips that are not copied

The crypto news ticker and the data pool line are not on this tab, and their code is not carried into it. A live news feed and a live cache-health line describe nothing a tablet reader does. The rows they held keep their height and hold nothing, and that space is where Import Live Fleet and Generate From YTD go.

```
#: The rows the two strips held on the Trading tab, and the height each keeps.
RESERVED_ROWS: tuple[dict[str, Any], ...] = (
    {"name": "news_ticker_row", "height_px": 24},
    {"name": "data_pool_row", "height_px": 18},
)
```

### With no tablet on disk

The tab draws an empty state and names what is missing. The panel reads `No TA data — No Stone Tablet on disk.`, the selector holds nothing, and both windows hold zero points. Nothing is invented in place of the missing tape.

### What one run measured

Both builds were opened and read off the drawn window and the drawn page. The figures below come from that run.

```
tablet          A18_1d_2026_coinbase, 250 candles, 2026-01-01 to 2026-09-07
window          100 candles
tablets listed  411
votes           5 bullish, 2 bearish, 5 neutral
Qt to React     32 of 32 values matched, with a tablet and again with none
```

### The line beside the voting panel title

One label sits beside the Indicator Voting Panel title. It holds the panel's empty state and nothing else. `no_data_text` builds that line from the panel model and answers an empty string while the model holds a reading, so the label is hidden whenever the panel has values to draw.

`src/gui/main_tabs/indicator_panel_surface.py` — the line the label draws

```
def no_data_text(multi_tf_summary: dict, message: str) -> str:
    """``message`` through ``NO_DATA_FORMAT``, or empty while a summary exists."""
    summary = multi_tf_summary if isinstance(multi_tf_summary, dict) else {}
    if summary or not message:
        return EMPTY_TEXT
    return NO_DATA_FORMAT.format(message=message)
```

Both builds draw the same line. The Qt clone sets the label and hides it when the line is empty. The page module writes the same value into the element that carries the same name.

`src/gui/simulator_tab.py` — the Qt clone draws the line

```
def _draw_indicators(self, panel: dict) -> None:
    self._indicator_title.setText(panel["title_text"])
    # no_data text is empty while the panel holds a reading, so the
    # summary label shows only the one reason there is nothing to draw.
    reason = panel["no_data"]["text"]
    self._indicator_summary.setText(reason)
    self._indicator_summary.setVisible(bool(reason))
    for table, spec in zip(self._indicator_tables, panel["tables"], strict=True):
        self._fill_indicator_table(table, spec)
```

With no tablet on disk the label reads the sentence this page already names, under both builds.

```
No TA data — No Stone Tablet on disk.
```

## The style sheet the Sim page loads

The page carries one style sheet. `TAB_STYLE_ASSETS` names it, and `page_html` reads the file and writes it into the page's head.

`src/gui/react_simulator_tab.py` — the sheet the page carries

```
TAB_STYLE_ASSETS: tuple[str, ...] = ("simulator_tab.css",)
```

The sheet sets the page ground, the type, the pane grid and the table rules. The grid repeats the three pane sizes the Qt clone gives its splitters, so the bot list, the layer pane and the bottom pane take the same share of the window in both builds.

`src/gui/web/simulator_tab.css` — the pane grid

```
.acervator-simulator-tab {
  display: grid;
  grid-template-columns: 600fr 500fr;
  grid-template-rows: 500fr 350fr;
  grid-template-areas:
    "fleet layer"
    "bottom bottom";
}
```

Without the sheet the page draws on the browser's own defaults: a white ground, serif type, one column and no grid.

### The skin tokens the sheet reads

The sheet names no colour of its own. Every colour arrives as a custom property on the tab element, written from the `SKIN` dictionary the surface builds.

`src/gui/main_tabs/simulator_tab_surface.py` — three of the skin entries

```
SKIN = {
    "--sim-ground": ds.SURFACE_0,
    "--sim-better-colour": ds.SUCCESS,
    "--sim-agrees-colour": ds.SUCCESS,
}
```

A better portfolio row and an agreeing light take the same colours the Qt clone paints them, because both builds read one dictionary.

### The two battery selectors reach the page

The portfolio and the span the operator chooses now reach the page before a battery has run. `build_view_model` takes both and hands them to `empty_battery`, which carries them into the payload the selectors draw from.

`src/gui/main_tabs/simulator_tab_surface.py` — the idle battery payload

```
charged = battery_payload_held or empty_battery(portfolio, span)
```

Both builds pass the two values. Before this the idle payload named the defaults, so a chosen portfolio was lost until a run wrote it back.

## Crypto tablets are 5m and each bot walks the timeframe its tablet carries

The RA store holds two kinds of tablet beside each other under one MANIFEST. A non-crypto asset keeps its daily file, `GLD_1d_2025_yahoo.json`, written by the daily builder as before. A crypto asset gains a native five-minute file, `A15_5m_2026_coinbase.json`, the same shape the live fleet's store keeps, keyed on asset, timeframe, year and exchange by the same `tablet_filename`. The Battery reads a crypto bot off the 5m file and a non-crypto bot off the daily file, and the report names which one each bot read, with its timeframe, candle count and checksum. The operator's words, 2026-09-18: *"Crypto Tablets being daily is a failure. Must be 5m resolution. Cannot match trading logic for 5m against 1d charts."*

`src/simulator/portfolio_battery.py` — the timeframe each asset's tablet carries

```
def tablet_timeframe_for(asset: str) -> str:
    """The timeframe ``asset``'s RA tablet carries: ``CRYPTO_TABLET_TIMEFRAME``
    for a crypto asset, else ``TABLET_TIMEFRAME``."""
    return CRYPTO_TABLET_TIMEFRAME if is_crypto(asset) else TABLET_TIMEFRAME
```

### One registry shape, one rollup

The RA store reuses the live registry. `StoneTabletsRegistry` opened on the RA root reads the RA MANIFEST as it stands, indexes every daily row and every 5m row, writes a 5m tablet through the same `ingest_candles` the live store uses, and rolls 5m candles up to any higher timeframe it supports through its one `_rollup`. Nothing in the registry changed; the Battery calls it on a second root. The daily side keeps its own fold: `resample` still builds a weekly and a monthly bar from daily rows, because the registry does not roll up to a week or a month.

`src/simulator/portfolio_battery.py` — the per-bot read

```
    def bars(self, bot: SimBot, timeframe: str) -> list[list[float]]:
        """``bot``'s bars at ``timeframe`` inside the span: a refused asset
        answers none, a crypto bot reads ``registry().get_candles`` on
        ``CRYPTO_EXCHANGE``, and a non-crypto bot ``resample`` over ``rows``."""
        asset = bot.asset
        if self.refusal(asset):
            return []
        if not is_crypto(asset):
            return resample(self.rows(asset), timeframe)
        if str(timeframe) not in SUPPORTED_TIMEFRAMES:
            return []
        if self._end_ms <= self._start_ms:
            return []
        return self.registry().get_candles(
            asset,
            self._start_ms,
            self._end_ms - 1,
            timeframe=str(timeframe),
            exchange_id=CRYPTO_EXCHANGE,
        )
```

### The timeframes a bot walks

A crypto bot walks its own timeframe first, on the native 5m candles, then each of the Battery's three timeframes the registry can roll 5m up to. That is `1d`; `1w` and `1M` are outside the registry's supported set, so a crypto bot's walk is `5m` then `1d`. A non-crypto bot walks `1d`, `1w` and `1M` folded from its daily file, as before. A mixed portfolio reports four timeframes and each row names which bots reached it. Read against Live: the live bot's gate chain ticks its own timeframe, 5m on every one of the operator's thirty-eight bots, and its phantoms read higher timeframes from the venue; the Simulator's rollup is the registry's, so the Simulator and Live read the same hour from two sources, and the Back Test's phantom timeframes are unit 37's.

`src/simulator/portfolio_battery.py` — the set

```
def bot_timeframes(bot: SimBot, timeframes: Sequence[str] = TIMEFRAMES) -> tuple:
    """The timeframes the walk evaluates ``bot`` at: a crypto bot its own
    ``ta_timeframe`` on the native ``5m`` tablet and then each of
    ``timeframes`` in ``SUPPORTED_TIMEFRAMES``, the ones the registry rolls
    ``5m`` up to; a non-crypto bot ``timeframes`` as given, folded from its
    daily tablet."""
```

### The chooser states the cost before the press

Run Portfolio's chooser carries one more line under the portfolio and span combos. It reads the preflight for the choice in force: for each crypto asset of the portfolio, the 5m candles the RA root does not hold over the span, the chunks and venue calls the fetch will make, and the bytes at the measured 63 bytes a stored candle. The line changes with every combo change. OK is the consent and nothing else asks. Over three crypto assets on the `2026 test run` span the line reads about 49,000 candles an asset, 846 calls and 9.3 MB; over `All` it reads 703,296 candles an asset, and the operator decides.

`src/simulator/tablet_retrieval.py` — the cost of one asset

```
def retrieval_cost(
    registry: StoneTabletsRegistry,
    asset: str,
    exchange_id: str,
    since_ms: int,
    until_ms: int,
) -> RetrievalCost:
    """The ``RetrievalCost`` of ``asset`` on ``exchange_id`` over
    ``[since_ms, until_ms]``: every ``TIMEFRAME`` step ``registry.missing_ranges``
    reports, through ``adapter_for``'s ``chunk_limit`` and ``PAGE_ROWS``; an
    exchange with no adapter costs nothing and is refused at the press."""
```

`src/gui/simulator/sim_exchange_choice.py` — the line on the chooser

```
def battery_cost_text(parent: Any, names: Any, span: str) -> str:
    """``portfolio_battery.cost_line`` over ``plan_costs`` on the parent's
    ``battery_tablet_source`` for ``names`` and ``span``; ``COST_UNREAD_TEXT``
    when the parent has no such source or the read raises."""
```

### The Battery's own thread retrieves, then walks

After OK the Battery's daemon thread runs the retrieval before the first walk, through the same path the replay layer's Retrieve Tablet press uses: the read-only connector over the public candle endpoint, one chunk at a time through the gap filler, each chunk written into the year file and the MANIFEST as it lands, so a second press over the same span finds no gap and fetches nothing. One Activity Log line opens each asset's fetch and one closes it, naming the candles appended, the chunks, the calls and the file. A venue refusal closes the asset by name, and that bot reads `no_tablet` at 5m; the daily file never stands in for it. Each landed asset emits `sim.tablet.retrieved` and each refusal `sim.tablet.refused` through the signal contract, the prediction from the chooser's preflight beside the observed count.

`src/simulator/portfolio_battery.py` — the retrieval before the walk

```
    chosen = plan if plan is not None else plan_run(names, tablets)
    start_ms, end_ms = span_bounds(span, tablets.entries())
    costs = battery_costs(tablets, chosen.bots, start_ms, end_ms)
    retrievals = retrieve_tablets(tablets, costs, str(span), connector, progress)
    tape = TapeCache(tablets, start_ms, end_ms)
    for row in retrievals:
        if row["refused"]:
            tape.refuse(
```

The Sim API pane records a `FETCH_TABLET` block per connector call, the same block the replay layer's Retrieve Tablet press writes, because both hosts hand `run_battery` the tab's own connector and that connector crosses each call to the GUI thread, where the pane's writer accepts it. The recorder names the tablet off the call itself, through `call_tablet`: the symbol's base, the timeframe and the year of the first candle asked, so a Battery call reads `FETCH_TABLET A15_5m_2026_coinbase` and a replay-layer call keeps its own key. One block is one connector call of 350 candles; the public endpoint answers it in two pages of 300 and 50, which is why the chooser's call count is twice the block count. The Battery's per-asset lines stay on the Activity Log and the per-call progress line is the replay layer's alone.

`src/simulator/tablet_retrieval.py` — the tablet one call names

```
def call_tablet(call: Any, exchange_id: str = EXCHANGE_ID) -> tuple[str, str]:
    """``(key, file)`` of the tablet one ``VenueCall`` fills: ``tablet_filename``
    over the call's ``symbol`` base, its ``timeframe`` and the UTC year of its
    ``since_ms``, on ``exchange_id``; ``key`` is ``file`` without its suffix."""
```

`src/gui/simulator/sim_trading_tab.py` — the Battery on the tab's connector

```
                 progress=lambda line: self.battery_line.emit(line, "info"),
                 on_trade=self.battery_trade.emit,
                 bus=self._bus,
                 connector=self._connector,
```

### The 2026 test run span

The live fleet's test run began 2026-04-01 with every bot on 5m. The chooser offers that span beside the archive's six, `2026 test run`, from 2026-04-01 to the current UTC day, read at call time so a window left open over midnight still ends today.

`src/simulator/portfolios.py` — the span

```
def period_window(span: str) -> Optional[tuple[str, str]]:
    """``PERIODS[span]`` with ``TEST_RUN_SPAN``'s end read as ``today_utc``
    now, or None for a label ``PERIODS`` does not hold."""
```

### The report's Tablets section names each bot's file

Under the MANIFEST table the Battery report carries a per-bot table: the bot, its own timeframe, the timeframe its tablet carries, the file or files it read, the candle count, the checksum, the timeframes it walked with each outcome, and the refusal where it read no bar. A retrieval table follows it, one row per crypto asset the press checked: the candles asked, the candles appended, the chunks, the calls and the refusal.

`src/simulator/parity_report.py` — the per-bot rows

```
def battery_tablets_by_bot(run: BatteryRun) -> list[dict]:
    """One row per bot of ``run``: the bot's own ``ta_timeframe``, the
    ``tablet_timeframe`` its walk read, every ``TabletRead`` file with its
    candle count and checksum, the timeframes it walked with each outcome,
    and the ``refusal`` where it read no bar."""
```

### What the 5m reading measured

Both builds, the real window over a scratch home, every socket but loopback refused, a scratch RA root holding the operator's daily files for A15, A14, BNB, GLD and SLV, and a loopback stand-in for the public candle endpoint serving 5m from 2026-04-01 and refusing BNB. Under Portfolio Battery, Run Portfolio opened the chooser with the cost line under the combos; the line changed on a control span; CRYPTO_BLUE on `2026 test run` read three assets, 49,313 candles each, 846 calls, 9.3 MB. OK: the Activity Log carried the retrieving line and the retrieved line for A15 and for A14, and the refusal by name for BNB; `A15_5m_2026_coinbase.json` and `A14_5m_2026_coinbase.json` landed under the scratch root with their MANIFEST rows and candle counts near 288 a day; two `sim.tablet.retrieved` rows and one `sim.tablet.refused` row reached the signal sink; the Sim API pane held 290 `FETCH_TABLET` blocks after two presses, 141 for A15, 141 for A14 and 8 for BNB, read off the Qt pane's text and the React page's API log element, while the stand-in counted two pages for each; the walk read A15 and A14 at `5m` and at `1d`, and BNB `no_tablet` at 5m; the report's Tablets section named each bot's file, timeframe, candle count and checksum. A second press over the same span stated nothing to retrieve and fetched nothing, and the two 5m files hashed identical before and after it. On a daily-only root with the stand-in absent, DIGITAL_GOLD refused A15 and A14 by name and walked GLD and SLV on their daily files at `1d`, `1w` and `1M`. Every copied daily file and `bot_state.json` hashed identical after every press, with a planted byte moving the hash; the operator's RA root listed 414 entries before and after. No bot was constructed and no socket left loopback.

## The playback marks each fill on its candle and the header reads the opposing-trade distance

After a run, the Stone Tablet Playback window draws one glyph on the candle each fill landed on, at the fill's price, for the bot the chooser shows. A scrum draws `dissolve`, a triangle pointing down as a gold outline. A fold draws `reform`, a triangle pointing up filled blue. The layer's header carries a second line that reads the shown bot, the count of fold-to-scrum pairs, the mean and the median opposing-trade distance, and the estimated improvement. The operator's words, 2026-09-18: *"Just need to make sure these play back in the window with it marking where trades could have occurred. Not sure if relevant to even draw a VWAP chart but total estimated improvement in terms of % would be useful and can measure distances between opposing trades without having to know amounts that would have been traded."*

Diagram: flowchart, left to right

- back_test.walk -> **battery_trade / run_trade** *(SimTrade with scrum_price, timeframe)*
- battery_trade / run_trade -> **log_trade: the Activity line, and _fills**
- battery_finished / run_finished -> **_refresh_replay follow_run**
- _refresh_replay follow_run -> **replay_feed: shown_bot, fills_for, playback_payload, replay_figures**
- replay_feed: shown_bot, fills_for, playback_payload, replay_figures -> **PlaybackView marks and the figures label**
- replay_feed: shown_bot, fills_for, playback_payload, replay_figures -> **PlaybackWindow polygons and the figures span**
- replay_feed: shown_bot, fills_for, playback_payload, replay_figures -> **sim.replay.marks_drawn**

### The fills the host keeps

Each host holds every `SimTrade` a run hands it, in fill order, on `_fills`. `log_trade` appends the trade beside the Activity line it writes, so a Battery fill and a Back Test fill reach the list by the one path both already took. The list empties when a run starts, in `_run_battery` after the chooser accepts and in `_start_run`, so a Back Test's marks never sit on a Battery's tape. The Battery's finished `BatteryRun` is held beside the list for the improvement figure. `fills` and `battery_outcome` answer both to the window.

`src/gui/simulator/sim_trading_tab.py` — the fill kept

```
    def log_trade(self, trade: SimTrade) -> None:
        """One Activity Log line per ``SimTrade``: ``trade_line`` under
        ``trade_stamp`` through ``SimStatusLog.log_at`` at ``TRADE_LINE_LEVEL``,
        and the trade appended to ``_fills`` for the replay layer's marks."""
        self._fills.append(trade)
```

### The scrum a fold re-entered against

`SimTrade` carries two more fields. `scrum_price` is the scrum's own price on a scrum. On a fold it is `plan_source_price` over the fold's plan: each source tranche's `ref`, the price its scrum sold at, weighted by the dollars the plan takes from it. One fold that consumes two tranches reads their weighted centre, and the pair count is one per fold. Zero names no scrum, which is what a Validation rerun's fill carries, so a rerun forms no pair. `timeframe` is the `ta_timeframe` the walk ran at. The Battery walks a stock bot at `1d`, `1w` and `1M` under one bot id, and a weekly fill's stamp is a Monday's daily candle at the week's close; the daily tape marks the `1d` walk alone.

`src/simulator/back_test.py` — the record

```
@dataclass(frozen=True)
class SimTrade:
    """One scrum sell or fold buy the tape produced, with ``scrum_price`` the
    scrum's own ``price`` on a scrum and ``plan_source_price`` over the
    tranches consumed on a fold, zero naming no scrum. ``timeframe`` is the
    ``ta_timeframe`` the walk ran at, empty when unknown."""

    bot_id: str
    symbol: str
    side: str
    ts_ms: int
    price: float
    units: float
    usd: float
    fee_usd: float
    scrum_price: float = 0.0
    timeframe: str = ""
```

`src/trading/scrumming/sizing.py` — the fold's scrum price

```
def plan_source_price(plan: list) -> float:
    """The scrum price a fold under ``plan`` re-enters against: each source
    tranche's ``ref`` weighted by the USD the plan takes from it; zero for an
    empty ``plan`` or one taking no USD."""
```

### The distance arithmetic

The opposing-trade distance of one pair is the scrum price less the fold price, as a percentage of the scrum price. A fold that re-entered below its scrum reads positive; one above reads negative. Over a run the header reads the count of pairs, the mean and the median. The two functions sit in `sizing.py` beside the arithmetic the same fills were sized under, and read no record: the pairs come from `trade_pairs` in the surface, each fold whose `scrum_price` is above zero. The improvement is the Harvest-Fold end less the HODL end as a percentage of the HODL end, `baseline_usd` and `accumulation_usd` of the shown bot's `SymbolRun` at the tape's timeframe, the figure the report's Battery section carries. A run with no Battery outcome reads an em dash there.

`src/trading/scrumming/sizing.py` — one pair, and the run's figures

```
def opposing_trade_distance_pct(scrum_price: float, fold_price: float) -> float:
    """``scrum_price`` less ``fold_price`` as a percentage of ``scrum_price``,
    positive for a fold that re-entered below its scrum."""
    return 100.0 * (float(scrum_price) - float(fold_price)) / float(scrum_price)


def opposing_trade_distances(pairs: Sequence[Sequence[float]]) -> dict:
    """``opposing_trade_distance_pct`` over each ``(scrum_price, fold_price)``
    in ``pairs`` whose scrum price is above zero: the ``count``, the
    ``mean_pct``, the ``median_pct`` and the ``distances_pct``, the two figures
    None when ``count`` is zero."""
```

`src/gui/main_tabs/simulator_tab_surface.py` — the improvement

```
def improvement_pct(hodl_usd: float, harvest_fold_usd: float) -> Optional[float]:
    """``harvest_fold_usd`` less ``hodl_usd`` as a percentage of ``hodl_usd``;
    None when ``hodl_usd`` is not above zero."""
```

### The two glyphs

One definition, `MARK_GLYPHS`, names both marks, and both hosts read it: the Qt view in Python, the page off the playback payload. Each glyph is three corners in a unit mark, y down, centred on the fill. `dissolve` is the scrum: the corners `(-0.5, -0.5)`, `(0.5, -0.5)`, `(0, 0.5)`, an outline. `reform` is the fold: `(-0.5, 0.5)`, `(0.5, 0.5)`, `(0, -0.5)`, filled. A mark is one candle column wide and a twentieth of the pane tall: 6 px by 13 px on the 624 px pane at 1400, 4 px by 12 px on the 378 px pane at the floor. The colours are `--sim-mark-scrum`, Live's gold, and `--sim-mark-fold`, Live's blue, the two tokens the Charts tab colours its scrum and fold markers with.

`src/gui/main_tabs/simulator_tab_surface.py` — the glyphs

```
DISSOLVE_GLYPH = "dissolve"
REFORM_GLYPH = "reform"
MARK_GLYPHS = {
    back_test.SCRUM: {
        "name": DISSOLVE_GLYPH,
        "points": [[-0.5, -0.5], [0.5, -0.5], [0.0, 0.5]],
        "filled": False,
    },
    back_test.FOLD: {
        "name": REFORM_GLYPH,
        "points": [[-0.5, 0.5], [0.5, 0.5], [0.0, -0.5]],
        "filled": True,
    },
}
MARK_WIDTH_RATIO = 1.0
MARK_HEIGHT_FRACTION = 0.05
MARK_OUTLINE_PX = 1.5
```

### The two glyphs are hermetic characters

Two sentences above are overtaken. Each is quoted whole, with the true sentence under it.

Overtaken: *"Each glyph is three corners in a unit mark, y down, centred on the fill."*

True now: each glyph is one character, centred on the fill.

Overtaken: *"*`dissolve` *is the scrum: the corners* `(-0.5, -0.5)`, `(0.5, -0.5)`, `(0, 0.5)`, *an outline.* `reform` *is the fold:* `(-0.5, 0.5)`, `(0.5, 0.5)`, `(0, -0.5)`, *filled."*

True now: the scrum is U+1F761, named ALCHEMICAL SYMBOL FOR DISSOLVE. The fold is U+1F75F, named ALCHEMICAL SYMBOL FOR PRECIPITATE. A mark is set at a twentieth of the pane in the symbol family, which resolves to Segoe UI Symbol, and both characters resolve on it. The corner lists, the fill flags and the outline width are gone with the triangles they described.

The definition lives in the chart's surface and the Simulator imports it, so both tabs draw the same two characters from the same object.

```
DISSOLVE_GLYPH = "\U0001f761"
REFORM_GLYPH = "\U0001f75f"
MARK_GLYPHS = {
    SCRUM_SIDE: {"name": DISSOLVE_GLYPH},
    FOLD_SIDE: {"name": REFORM_GLYPH},
}
```

### The marks on the payload

`playback_payload` takes the shown bot's fills beside the candles and answers `marks`: one per fill whose stamp is a candle's stamp in the window, carrying the candle's index and x, the price's y over the window's bounds, the side, the scrum price and the glyph name. A fill on a candle outside the window is counted in `fills` and not in `mark_count`. The Qt view draws each mark as a polygon after the candles; the page draws one `polygon` element per mark with `data-side`, `data-glyph`, `data-price` and `data-candle-index`, its outline kept at pixel width under the window's non-uniform scale.

The last sentence above is overtaken. It is quoted whole, with the true sentence under it.

Overtaken: *"The Qt view draws each mark as a polygon after the candles; the page draws one* `polygon` *element per mark with* `data-side`*,* `data-glyph`*,* `data-price` *and* `data-candle-index`*, its outline kept at pixel width under the window's non-uniform scale."*

True now: the Qt view draws each mark's character after the candles. The page draws one text element per mark, carrying the same four data attributes, centred on the fill's point across and down. Neither takes an outline, and the payload no longer carries one.

`src/gui/main_tabs/simulator_tab_surface.py` — one mark

```
def mark_shapes(
    candles: Sequence[Sequence[float]],
    fills: Sequence[Any],
    low: float,
    high: float,
) -> list[dict]:
    """One mark per fill in ``fills`` whose ``ts_ms`` is a candle's stamp in
    ``candles``: the candle's ``index`` and ``x``, the fill's price as ``y``
    over ``low`` and ``high``, its ``side`` and the ``MARK_GLYPHS`` name."""
```

`src/gui/simulator_tab.py` — the Qt polygon

```
            polygon = QPolygonF(
                [
                    QPointF(centre_x + dx * mark_w, centre_y + dy * mark_h)
                    for dx, dy in glyph["points"]
                ]
            )
            painter.setPen(QPen(colour, outline))
            painter.setBrush(QBrush(colour) if glyph["filled"] else Qt.NoBrush)
            painter.drawPolygon(polygon)
```

### The chooser reads the mode's tablet root

The replay layer lists and draws the tablets of the mode in force. `replay_source` answers the Battery's RA reader under Portfolio Battery and the live-fleet reader otherwise, and `_refresh_replay`, `_feed_replay` and the retrieval press read it, so after a Battery run over a stock portfolio the chooser lists each bot's daily file by its key, `SPY_1d_2024_yahoo`, and a retrieval press writes under the root the chooser was read from. Before this change the layer read the live-fleet root in every mode and listed `SPY_5m_yahoo — no tablet` after the same run.

`src/gui/simulator/sim_trading_tab.py` — the source by mode

```
    def replay_source(self) -> TabletSource:
        """The tablet reader the replay layer lists and draws from:
        ``battery_tablet_source`` under ``MODE_PORTFOLIO_BATTERY``,
        ``tablet_source`` otherwise."""
        if self._mode == surface.MODE_PORTFOLIO_BATTERY:
            return self._battery_tablet_source
        return self._tablet_source
```

The chooser names a tablet, and `shown_bot` names the bot whose fills the tablet marks: of the held bots on the tablet's market, the run's last-fill bot, else the panel's selected bot, else the lowest bot id. At a run's end both hosts refresh the layer with `follow_run`, so the chooser defaults to the last fill's bot; a chooser change draws the other bot's marks and figures.

`src/gui/main_tabs/simulator_tab_surface.py` — the shown bot

```
def shown_bot(
    bots: Sequence[Any],
    asset: str,
    exchange_id: str,
    last_bot_id: str = "",
    selected_bot_id: str = "",
) -> Any:
    """The bot whose fills the playback marks for a tablet on ``asset`` and
    ``exchange_id``: of the ``bots`` on that market, the one ``last_bot_id``
    names, else the one ``selected_bot_id`` names, else the lowest ``bot_id``;
    None with no bot on the market."""
```

### The header's figures

The figures sit on the header's second line in both builds. The first line's fixed-width chooser and the retrieval button leave 180 px at 1400, measured, and the line needs about 470. The Qt label wraps at the floor; the page's span takes the header's full width under its wrapping row. With no run held the line reads `no run`.

`src/gui/simulator/sim_trading_tab_surface.py` — the line

```
REPLAY_FIGURES_FORMAT = (
    "{bot_id} · {pairs} pair{plural} · distance mean {mean} · median {median} · "
    "improvement {improvement}"
)
```

### When the marks draw

The feed runs where it ran before: at build, on every `fleet_changed`, on the flip, on the chooser's change, when the panel's bot changes and after a retrieval. Two moments are added: the run's end, `_take_battery` and `_take_run`, with the chooser moved to the run's last bot. Every draw emits `sim.replay.marks_drawn` through `signal_contract` with the view's payload read back against the feed's: the tablet, the bot, the marks drawn, the fills handed, the pairs and the three percentages, on both sides.

`src/gui/simulator/sim_trading_tab_surface.py` — the emit row's figures

```
def marks_reading(key: str, playback: Any, figures: Any) -> dict:
    """The eight figures ``MARKS_DRAWN_SIGNAL`` carries on each side: the
    tablet ``key``, the bot, the marks drawn, the fills handed, the pairs and
    the three percentages, read off one ``playback`` payload and its
    ``figures``."""
```

### What the marks reading measured

Read off the running program in both builds, scratch home, every socket but loopback refused, a scratch RA root holding EQUITY_MACRO's two daily files trimmed to 100 candles each, SPY's first hundred of 2024 and GLD's from row 125. On the base commit the same Battery run put 14 fills on the spool, SPY 8 and GLD 6, and the layer drew 0 candles, no mark, no figures line and no emit row, its chooser listing `GLD_5m_yahoo` and `SPY_5m_yahoo` with no tablet. After: Run Portfolio over EQUITY_MACRO on All, 14 fills on the spool at `1d`, the report's rows SPY 8 trades and GLD 6 at 1d and `short_tape` at 1w and 1M; the flip; the chooser on `GLD_1d_2024_yahoo`, the last fill's bot; 6 marks, four `dissolve` and two `reform`, each painted at its candle's x and its price's y, the Qt painter's polygons and the page's polygons both within a thousandth of a pixel of the payload's point; the header `GLD@yahoo · 2 pairs · distance mean +2.02% · median +2.02% · improvement -0.18%`, equal to the hand recomputation over the held fills (distances 1.89% and 2.15%) and to `(535.32 – 536.31) / 536.31` off the report's row. The chooser moved to `SPY_1d_2024_yahoo`, on the page through its own select: 8 marks, `SPY@yahoo · 4 pairs · distance mean +2.26% · median +2.14% · improvement -0.08%`, equal to the hand figures (1.85%, 1.64%, 3.13%, 2.43%) and the row's `(531.84 – 532.26) / 532.26`. A planted fill on a stamp outside the window raised `fills` to 9 and left `marks` at 8; a planted fold at 200 against a scrum at 100 moved the mean to –18.19% and the pairs to 5; the fills restored, the line read as before. At the floor, the pane at 378 px on Qt and 441 px on the page, the marks drew at 4 px by 12 px and the line wrapped whole. Nine `sim.replay.marks_drawn` rows per build, every one `ok`. `bot_state.json` and both tablets hashed identical after every step, a planted byte moving the hash; the operator's RA root listed 414 entries before and after; no socket left loopback; no bot was constructed.

## The simulated bot acts like its live equivalent

A loaded bot reads `IDLE` until a run starts. While its walk runs its row reads `RUNNING` and moves as the walk fills: the trade count, the current position value, the target and the unrealised figure are written to the held record on every tick and every fill, in the fields the live bot writes, and the tables draw them. Scrums form fold tranches on the record in the live shape, folds consume them and grow the target by the surplus at the configured rate, and the Bot Settings window's Fold Tranches and Stack Tranches tabs draw the walk's tranches when opened mid-run. Every tick reads the higher-timeframe bias from the walk's own candles rolled up to the bot's phantom timeframes, through the registry's one rollup and the live coordinator's weighing, so no gate reads a hard-coded `None`. The operator's words, 2026-09-18: *"Loaded bots are showing running despite there not being a replay active. Should 'turn on' and begin updating trade counts, current position value, and target balance. Once running, bot metrics must update as trade actions are simulated ... It is imperative for simulated bots to be acting just like their live equivalent. Scrums and Folds form Tranches. Surplus profits compound at the configured rate."* And: *"Any TF higher than 5m was supposed to be getting calculated using the 5m data."*

Diagram: flowchart, left to right

- back_test.walk -> **the host's sim bus** *(BotStatsSnapshot on bot.stats)*
- the host's sim bus -> **_on_bus_stats** *(worker thread)*

- _on_bus_stats -> **_take_bot_stats on the GUI thread** *(bot_stats signal, queued)*
- _take_bot_stats on the GUI thread -> **FleetSource.write_stats: stats, scrumming_state**
- _take_bot_stats on the GUI thread -> **SimBotManager.start / stop** *(running / stopped)*
- _take_bot_stats on the GUI thread -> **refresh_bots: the rows** *(every 2 s)*
- FleetSource.write_stats: stats, scrumming_state -> **Detail: SimBotView over the record, the tranche tabs**
- back_test.walk -> **tape_context: the HTF fields** *(phantom_tapes, _rollup, weigh_higher_tf_bias)*
- back_test.walk -> **position.target_usd, target_path** *(fold surplus, cycle cap)*

### A loaded bot reads IDLE

Import Live Fleet copies each stored record and writes its state `idle`, whatever the live file saved, and the sim fleet file's records read `idle` at construction. That is the live restore's own rule: `restore_bots_from_state` recreates every persisted bot in IDLE and it stays there until the operator starts it. Before this an imported bot read the live file's `running` with no run in flight. Generate From YTD and Create already wrote `idle`.

`src/simulator/fleet_source.py` — the one write

```
def loaded_idle(record: dict) -> dict:
    """``record`` with its ``state_when_saved`` written ``BotState.IDLE``, as
    ``restore_bots_from_state`` recreates every persisted bot in IDLE until
    the operator starts it; a loaded bot never reads ``running`` with no run."""
    record["state_when_saved"] = BotState.IDLE.value
    return record
```

### The row moves with the walk

The walk emits a `BotStatsSnapshot` on the run's bus under `bot.stats` at its opening, on every tick, on every fill and at its end. Each host subscribes to that topic on its own bus at build; the subscriber runs on the walk's thread and re-emits the snapshot through the `bot_stats` signal, so the record is written on the GUI thread through `FleetSource.write_stats`. A tick writes what the live tick writes: the current price, the position value and the unrealised figure, with the lots the holdings are summed from. A fill writes what the live fill writes: the trade count, the buys and sells, the scrummed and folded sums, the trade volume, and the scrumming state the record holds, the grown target, the fold tranches, the lots, the cycle's consumed cap, the standing surplus and the growth side, the last trade's price and side, and the lifetime tranche counters. The Scrumming Bots table reads the position value off the holdings and the price, Trades off the trade count and Target off the grown target, all three from the record.

The opening snapshot carries the `running` mark and the closing one `stopped`, so a row reads `RUNNING` while its own walk runs and `STOPPED` when it ends; a fleet of many bots walked one at a time reads one green row at a time. The Start Run press moves no Back Test row itself; under Validation, which reruns recorded rows in one pass and walks nothing, the press still moves every run bot to `running` as before. The rows redraw at once on a mark and every two seconds otherwise, the live dashboard's own interval, through `refresh_bots` alone rather than `fleet_changed`, which also writes the fleet file and re-seats the venues. A 5m walk ticks about two hundred times a second, so the record is always a few ticks ahead of the cells between two redraws, and equal to them at the end.

`src/simulator/back_test.py` — the snapshot

```
@dataclass(frozen=True)
class BotStatsSnapshot:
    """What one walk wrote at one moment, in the two shapes the held record
    holds: ``stats`` in ``BotStats``'s keys and ``scrumming_state`` in the
    keys ``SimBotView`` reads; ``state`` is the mark the walk's start and end
    carry, empty on a tick or a fill."""

    bot_id: str
    symbol: str
    timeframe: str
    ts_ms: int
    tick: int
    event: str
    state: str
    stats: dict
    scrumming_state: dict
```

`src/gui/simulator/sim_trading_tab.py` — the write on the GUI thread

```
        bot_id = str(getattr(snapshot, "bot_id", ""))
        try:
            self._fleet_source.write_stats(
                bot_id, snapshot.stats, snapshot.scrumming_state
            )
            if snapshot.state == back_test.RUNNING_STATE:
                self._bot_manager.start(bot_id)
                self._status_log.log(f"✓ Bot {bot_id} RUNNING.", "success")
```

The live bot's `stats.realised_pnl` is written by nothing on its scrum and fold path, so the walk writes it no more than the live bot does; the realised surplus of a completed cycle reaches the row where the live bot books it, in the target's growth, the folded sum and the standing surplus.

### Scrums and folds form tranches on the record

The position keeps its lots as the live bot keeps `_main_lots`: the opening entry is one lot at the opening price. A scrum sorts the lots by their original cost, highest first, sells from them in that order and queues one fold tranche per lot sold from, each carrying that lot's original cost as `initial_buy_price`, the sell price as `ref`, the proceeds' share as `usd` and the units taken. A fold rebuys the eligible tranches under unit 26's plan and settle, books the bought units as lots per consumed tranche's share, and closes the tranches it drained. The snapshot carries the tranches and the lots, so the Fold Tranches tab opened mid-run draws the walk's tranches, as the live window draws a bot's at open; it does not refresh while open, and the live window does not either. A tranche's age on the tab is measured from the wall clock, so a walk's tranche created on the tape's April candle reads months old in September. The live stack is Stack Mode's ladder of resting venue orders opened off a scrum; the walk places no order, so the Stack Tranches tab reads `No stack tranches yet.` for a walked bot. Merge, despawn and clear are not walked.

`src/simulator/back_test.py` — one tranche per lot

```
    position.main_lots.sort(
        key=lambda lot: float(lot.get("initial_buy_price", 0) or 0), reverse=True
    )
    remaining = units
    for lot in list(position.main_lots):
        if remaining <= 1e-12:
            break
        take = min(float(lot.get("units", 0) or 0), remaining)
        if take <= 1e-12:
            continue
        position.fold_tranches.append(
            {
                "usd": (take / units) * proceeds,
                "units": take,
                "ref": float(price),
                "initial_buy_price": float(lot.get("initial_buy_price", 0) or 0),
                "created_ts": float(ts_ms) / 1000.0,
            }
        )
```

### The surplus compounds into the target

Each fold's surplus is the units bought less the units its consumed tranches sold at their sell price, priced at the fill, the live fold's own figure. The cycle cap is the growth rate of the cycle-open target, and the fold grows the target by the surplus plus any standing surplus, held to what the cap leaves; the rest stays standing. The growth side is set to `lower`, and the cycle resets when the tick's Bollinger position reaches the opposite extreme, 0.75 after a lower-side growth, so the consumed cap zeroes and the next fold may grow the full cap again. The gates read the grown target for the Target Delta, as the live tick reads its own. Three pure functions carry the arithmetic beside unit 26's, and the live bot keeps its inline copy.

`src/trading/scrumming/sizing.py` — the growth and the cycle

```
def target_growth_applied(
    surplus_usd: float, standing_usd: float, cap_remaining_usd: float
) -> tuple[float, float]:
    """What ``_apply_fold_target_growth`` adds to the target and what it parks:
    ``(applied, standing_after)`` where the applied growth is the surplus plus
    the standing pool held to ``cap_remaining_usd``, and the rest stays
    standing; nothing is applied when the cap is consumed."""
    if cap_remaining_usd <= 1e-9:
        return 0.0, max(0.0, float(standing_usd) + max(0.0, float(surplus_usd)))
    available = max(0.0, float(surplus_usd)) + max(0.0, float(standing_usd))
    applied = min(available, float(cap_remaining_usd))
    return applied, max(0.0, available - applied)
```

### The higher timeframes come from the walk's own candles

For each phantom timeframe the record names, or the one above the walk's timeframe when it names none, the walk rolls its own candles up through the registry's `_rollup`: the completed buckets once, and the bucket holding the current candle from its rows up to that candle, so no later candle reaches a window. The window is the last hundred rolled candles, the live phantom's own fetch, and under thirty of them the phantom reads no summary, as the live phantom sets none. The voting engine votes on the window, and the summaries are weighed through `weigh_higher_tf_bias`, the loop lifted out of the live coordinator's `get_higher_tf_bias`, which now calls it: each higher summary at or above the confidence floor contributes its rank times its confidence to its side, both sides empty read `NEUTRAL`, the heavier side wins. The bias name and the two block flags then reach the gate context as the live tick sets them, a scrum deferred on `BULLISH` and a fold on `BEARISH` under the two defer flags. A record with phantoms off reads no bias, as the live bot builds no phantom for it. A timeframe the walk's candles cannot roll up to, one at or under the walk's own or outside the registry's table (`1w` among them), is refused by name on the report and reads no summary.

`src/trading/phantom_balance.py` — the one weighing

```
        return weigh_higher_tf_bias(
            [(p.timeframe, p.rank, p.last_summary) for p in higher],
            below_parent,
            min_confidence,
        )
```

`src/simulator/back_test.py` — the window without look-ahead

```
    def window(self, index: int) -> list[list[float]]:
        """The last ``PHANTOM_WINDOW_CANDLES`` rolled candles up to and
        including the walk's candle at ``index``."""
        key = self.keys[index]
        partial = _rollup(
            self.rows[self.starts[key] : index + 1], self.factor, self.bucket_ms
        )
```

### The report gains the target's path and the bias per fill

The Back Test report's bot rows carry the target at the walk's end, the snapshot's stats block, the target path and the HTF reading. A `Compounding` section lists each bot's target at open and at end, the growth applied, the steps, the standing surplus and the stats figures, then each bot's target path step by step: the candle, the surplus, the cycle cap, the consumed figure, the applied growth and the target before and after. A `Higher timeframes` section lists each bot's named, evaluated and refused timeframes with the bias counts over the ticks, then every fill with the bias its gates read, the target after it and the growth it applied. The sidecar holds the same dicts under `trades`, `compounding` and `htf`.

### The stats emit and the bias emit

Every fill and every walk's end emit `sim.bot.stats_written` through `signal_contract`: the bot, the tick, the fills and the fields moved, against the fill count off the trades, `ok` when the record's trade count equals it. Every tick emits `sim.bot.htf_bias`: the bot, the candle, the bias, the two block flags and each phantom's reading or refusal, against the timeframes the bot names, `ok` when each was evaluated or refused by name; a fill's row lands unthrottled and the rest once per five seconds per bot.

### What the live-equivalent reading measured

Read off the running program in both builds, scratch home, every socket but loopback refused, a `bot_state.json` of three scrumming bots whose saved state read `running`, each with one live fold tranche, the first with a 15m phantom, the second naming `1w`, the third with phantoms off on a rising tape, and one 5m tablet of 1,500 candles per pair. On the base commit Import Live Fleet read three `RUNNING` rows with no run; during the Back Test the first row read `$720.0000`, `7`, `$805.0000` from before the press to the end while the spool carried 22 of its fills; the Fold Tranches tab drew the imported tranche; the report carried no target path and no bias; no emit row landed. After: Import Live Fleet read three `IDLE` rows in both builds. Start Run: the first row `RUNNING` and the other two `IDLE`, its record moving at a tenth of a second, 15 trades and a position of $721.84 against a target of $807.49, then 19 trades, $758.15 and $809.46 with one tranche queued, the cells holding the opening `$800.0000`, `0`, `$800.0000` until the next redraw; Detail pressed while it ran drew one open tranche, `7.622694` units, `$99.3034` parked, sell ref `$13.07965900`, original cost `$11.36281900`, and the Stack Tranches tab `No stack tranches yet.`, in both builds; then the second row `RUNNING` alone, then the third. At the end every row `STOPPED` and equal to the report: the first 23 trades, position $783.6261, target $809.7497, unrealised $198.24, scrummed $395.90, folded $114.64, four tranches queued; the second 11 trades, target $704.15; the third 7 scrums, no fold, seven tranches queued, target $600.00. The first bot's 17 growth steps recomputed by hand from the first: surplus $0.3030 against a cycle cap of $8.00 with nothing consumed, applied $0.3030, target $800.00 to $800.3030. The first bot's 15m bias over 211 ticks: 9 `None` before thirty rolled candles, 201 `NEUTRAL`, 1 `BEARISH`; its first fill's `NEUTRAL` recomputed by hand from 48 rolled candles, the 15m summary `BEARISH` at 0.2049 under the 0.30 floor. The second bot's `1w` refused as `cannot roll up from the walk's timeframe`, the third's `15m` as `phantoms off`, both reading no bias. Forty-four `sim.bot.stats_written` and forty-four `sim.bot.htf_bias` rows per build, every one `ok`. The scratch `bot_state.json`, the three tablets and the manifest hashed identical after every press, a planted byte moving a copy's hash; no socket left loopback; no bot was constructed. The Back Test still steps its tape at two hundred evaluations a bot, so a three-bot run over 1,500 candles lasts about three seconds; the row's movement is read off the record between two redraws and off the cells at the marks and at the end.

## Back Test walks the full Stone Tablet

> *Back Test should be able to 'Load Live Fleet' but runs full Stone Tablet for a given target asset or all tablets corresponding to the loaded / configured fleet.*

Back Test walks every candle of every year file the tablet store holds for each bot's asset on the bot's venue, in time order, at the bot's own timeframe, with no stepping. Before this landed a bot read one year file, the oldest, and evaluated about two hundred of its bars. Now a bot with two year files of 3,000 candles evaluates 5,971 bars: 6,000 less the 29 the first window needs. A bot whose timeframe is not the store's 5m reads the same files rolled up through the registry's one rollup, so a 1h bot over the same two files evaluates 471 of 500 bars. A bot whose asset has no file, or whose timeframe the registry cannot roll 5m up to, is named on the Activity Log, in the started line and in the report, and walks nothing.

`src/simulator/back_test.py` — the files and the tape one bot walks

```
def native_entries(entries: Sequence[Any], asset: str, exchange_id: str) -> list:
    """Every entry in ``entries`` for ``asset`` on ``exchange_id`` at
    ``NATIVE_TIMEFRAME``, by year."""


def bot_tape(registry: StoneTabletsRegistry, files: Sequence[Any], bot: SimBot) -> list:
    """Every bar across ``files`` at ``bot_timeframe`` for ``bot``'s asset on
    its venue, in time order, through ``registry.get_candles``; no file or a
    timeframe outside ``SUPPORTED_TIMEFRAMES`` answers none."""
    timeframe = bot_timeframe(bot)
    if not files or timeframe not in SUPPORTED_TIMEFRAMES:
        return []
    since = min(int(one.first_ts_ms) for one in files)
    until = max(int(one.last_ts_ms) for one in files)
    return registry.get_candles(
        bot.asset, since, until, timeframe=timeframe, exchange_id=bot.exchange_id
    )
```

The runner reads the tape through a registry built on the tablet source's root for the run's length, the way the Portfolio Battery reads a crypto bot, and drops it when the run ends. The registry writes nothing. The stepping constant, the shared step and the two runner arguments that carried them are gone from the runner and from both hosts; the walk is called with its default step of one.

`src/simulator/back_test.py` — the runner, per bot

```
        files = native_entries(entries, bot.asset, bot.exchange_id)
        raw = bot_tape(registry, files, bot)
        if not raw:
            outcomes[NO_TABLET] += 1
```

### The started line states the cost

Start Run under Back Test writes one line before the walk begins. The line names the exchange, the bot count and the funding as before, then how many bots walk, the candles and files they read, the evaluations they make and about how many minutes that takes at the stated rate, and then the bots with no tablet at their timeframe, by name. The counts come from the MANIFEST rows alone, so the line lands at the press and reads no candle body.

`src/gui/simulator/sim_trading_tab_surface.py` — the sentence the line adds

```
BACK_TEST_COST_FORMAT = (
    " {walking} bot(s) walk {candles:,} candle(s) in {files} file(s), "
    "{evaluations:,} evaluation(s), about {minutes:.1f} min at "
    "{per_thousand:.2f} s per 1,000{absent}."
)
BACK_TEST_ABSENT_FORMAT = "; {count} bot(s) with no tablet at its timeframe: {names}"
```

The rate the line uses is one constant, read off the walked lines on this machine with the sim bus live and phantoms on, and the report states the rate each walk measured beside it.

`src/simulator/back_test.py` — the rate the started line assumes

```
WALK_SECONDS_PER_THOUSAND = 5.4
```

Both hosts build the cost in the same place, inside the start of the run, and hand it to the started line.

`src/gui/simulator/sim_trading_tab.py` — the cost at the press

```
        else:
            cost = back_test.fleet_cost(bots, self._tablet_source)
        self._status_log.log(
            tab_surface.run_started_line(
                mode, exchange_id, len(bots), back_test.run_budget_usd(bots), cost
            ),
            "success",
        )
```

### Each bot's walk reads as moving on the Activity Log

Each bot's walk writes one line as it starts, naming the files, the bars and the evaluations to make; one line every 5,000 bars, naming the bar reached, the evaluations so far, the scrum sells and fold buys so far and the seconds; and one line as it ends, naming the evaluations, the bars, the files, the seconds and the rate. The lines cross to the Activity Log through the same signal the run's other lines use, on both hosts.

`src/simulator/back_test.py` — the three lines

```
WALK_STARTED_LINE_FORMAT = (
    "Walking {bot_id} ({symbol}) at {timeframe} on {exchange_id}: {files} "
    "file(s), {bars:,} bar(s), {evaluations:,} evaluation(s) to make."
)
WALK_PROGRESS_LINE_FORMAT = (
    "{bot_id}: bar {bar:,} of {bars:,}, {evaluations:,} evaluation(s), "
    "{scrums} scrum sell(s), {folds} fold buy(s), {seconds:.1f} s."
)
WALK_ENDED_LINE_FORMAT = (
    "{bot_id} walked: {evaluations:,} evaluation(s) over {bars:,} bar(s) in "
    "{files} file(s), {seconds:.1f} s ({per_thousand:.2f} s per 1,000); "
    "{scrums} scrum sell(s), {folds} fold buy(s)."
)
```

The progress cadence, the evaluations a walk makes and the rate arithmetic are one definition each, in the Back Test module, and the Portfolio Battery imports them from there.

`src/simulator/portfolio_battery.py` — the four names imported

```
from .back_test import (
    FOLD,
    FUNDED_BY_TARGETS,
    MIN_CANDLES,
    PROGRESS_EVERY_BARS,
    RATE_LINE_FORMAT,
```

### Stop ends a Back Test walk at the bar reached

Stop on a row of the run in flight ends the walk where it is. The walk in progress writes a stopped line naming the bar reached, its candle stamp and the evaluations made; the partial report carries that line on the bot's row, the evaluations made against the evaluations expected, and the bots not reached by name.

`src/simulator/back_test.py` — the stopped line

```
    @property
    def stopped_at(self) -> str:
        """``WALK_STOPPED_LINE_FORMAT`` when ``stopped``, else empty."""
        if not self.stopped:
            return ""
        return WALK_STOPPED_LINE_FORMAT.format(
            bot_id=self.bot_id,
            bar=self.candles_read,
            bars=self.bars,
            candle_at=iso_stamp(self.last_ts_ms) or "none",
            evaluations=self.ticks,
        )
```

### The report counts the walk per bot

The report's Bots table gains the timeframe, the files, the bars, the evaluations expected beside the evaluations made, the seconds, the rate and the stopped line; the tablet column names every file walked. The Run section carries the files walked, the evaluations, the evaluations expected, the seconds and the rate over the whole fleet, and the run's lines on the Activity Log carry the same totals in one rate line.

`src/simulator/parity_report.py` — the columns added

```
        ("tablet_key", "tablet"),
        ("timeframe", "timeframe"),
        ("files", "files"),
        ("bars", "bars"),
        ("ticks", "ticks"),
        ("evaluations_expected", "expected"),
        ("walk_seconds", "seconds"),
        ("seconds_per_thousand", "s per 1,000"),
```

### One row per walked bot in the sim sink

Each walked bot emits one row through the signal contract when its walk ends: the files, the bars, the evaluations, the fills, the seconds and whether Stop cut it short, against the evaluations expected. The row is `ok` only when the walk made every evaluation and was not stopped. The run's worker thread is routed to the Simulator's own signal sink, so the row lands there, and a long walk rotates that sink, so the row of an earlier bot sits in a rotated file beside the live one.

`src/simulator/back_test.py` — the emit

```
    pin_emit(
        BOT_WALKED_SIGNAL,
        actual={
            "bot": bot.bot_id,
            "files": list(keys),
            "bars": result.bars,
            "evaluations": result.ticks,
            "fills": len(result.trades),
            "seconds": round(seconds, 3),
            "stopped": bool(result.stopped),
        },
        expected={"evaluations": expected, "stopped": False},
        ok=result.ticks == expected and not result.stopped,
```

### The path from the press to the report

Diagram: flowchart, top to bottom

- Start Run under Back Test -> **_start_run: fleet_cost over the tab's tablets, the started line with candles, files, evaluations, minutes**
- _start_run: fleet_cost over the tab's tablets, the started line with candles, files, evaluations, minutes -> **_compute_run on the worker thread, routed to the sim sink**
- _compute_run on the worker thread, routed to the sim sink -> **_walk_fleet: a registry on the tablet root**
- _walk_fleet: a registry on the tablet root -> **per bot: native_entries, bot_tape at the bot's timeframe**
- per bot: native_entries, bot_tape at the bot's timeframe -> **_walk_bot: the walking line, walk over every bar, a line every 5,000 bars, the walked or stopped line, one bot_walked row**
- _walk_bot: the walking line, walk over every bar, a line every 5,000 bars, the walked or stopped line, one bot_walked row -> **write_report: files, bars, evaluations, expected, seconds, rate per bot**
- write_report: files, bars, evaluations, expected, seconds, rate per bot -> **run_finished: the run's lines and the report line on the Activity Log**

### What was measured

Two scrumming bots on coinbase at 5m, each asset with two year files of 3,000 candles, the first ending 2025-12-31T23:55Z and the second starting 2026-01-01T00:00Z, on a scratch home with every socket but loopback refused, read off the running program in both builds. Before: each bot read 3,000 candles of one file and made 199 evaluations; the started line named no candle count and no time; no progress line landed. After: the started line read `2 bot(s) walk 12,000 candle(s) in 4 file(s), 11,942 evaluation(s), about 1.1 min at 5.40 s per 1,000`; each bot's walking line read 2 files, 6,000 bars and 5,971 evaluations to make; one progress line landed at bar 5,000 of 6,000; each walked line read 5,971 evaluations over 6,000 bars in 2 files, 32.8 and 31.5 seconds in Qt, 32.7 and 32.2 in React, at 5.49, 5.27, 5.48 and 5.39 seconds per 1,000; the report's rows read files 2, bars 6,000, ticks 5,971, expected 5,971, and 314 and 237 fills, equal to the spool's FILLED lines per bot; two rows in the sim sink, both `ok`. A bot whose asset had no file read `no Stone Tablet for CCC on coinbase at 5m; nothing walked.` on the Activity Log, `no_tablet` in the report and its name on the started line's absent clause. A bot at 1h over the same two files read 500 bars and 471 evaluations, walked in 2.3 seconds. Stop pressed nine seconds into a walk read `stopped at bar 1,724 of 6,000 (2025-12-27T13:35:00Z): 1,695 evaluation(s) made.` in Qt and bar 1,985 in React, the report `partial`, the second bot not reached, the sink row not `ok`. The walk wrapped to step ten bars read 598 evaluations against 5,971 expected, so the evaluation reading can fail. The scratch fleet file, the four tablets and the manifest hashed identical after every press, a planted byte moving a copy's hash; no socket left loopback; no bot was constructed.

Three passages above describe what this section changes and are kept as written: the "One clock" passage under Back Test Mode, which quotes the shared step that is gone; the sentence under the simulated-bot section that the Back Test still steps its tape at two hundred evaluations a bot; and the three code-block headers that name the Battery module as the home of the evaluations-expected and progress-cadence definitions, which now live in the Back Test module and reach the Battery by import.

## 2026-09-19 17:40 - #55 - the glyph definition moves to the chart's surface

His words on the Charts tab, 2026-09-08: *"Want charts tab to only display one richly detailed and Acervator-annonated chart at a time"*. The Charts painter now draws every fill as the two hermetic glyphs the playback draws, and the one definition both read lives in the chart's surface. The Simulator's surface imports it from there; the playback view, the page and the payload read the same names they read before, so nothing else on this tab changed.

`src/gui/main_tabs/simulator_tab_surface.py` - the import and the key check

```
from .native_chart_surface import (
    MARK_GLYPHS,
    MARK_HEIGHT_FRACTION,
    MARK_OUTLINE_PX,
    MARK_WIDTH_RATIO,
)
```

```
#: ``MARK_GLYPHS`` is keyed by ``SCRUM_SIDE`` and ``FOLD_SIDE``, the strings
#: ``back_test.SCRUM`` and ``back_test.FOLD`` carry.
if MARK_GLYPHS.keys() != {back_test.SCRUM, back_test.FOLD}:
    raise ImportError("MARK_GLYPHS keys differ from back_test.SCRUM and FOLD")
```

Read off the running program in both builds: the object the Charts painter draws its triangles from is the object the Simulator's surface holds, and a scrum on the Charts tab has the corners the playback gives a scrum.

### One sentence this entry overtakes

Not reworded, quoted here. `docs/manual/08-tabs/simulator.md:8634` - "One definition, `MARK_GLYPHS`, names both marks, and both hosts read it: the Qt view in Python, the page off the playback payload." Three readers now, the Charts painter the third, and the definition sits in `src/gui/main_tabs/native_chart_surface.py`.

## 2026-09-24 - #858 - the bot list upgrades reach the Simulator

His line closes the bot-list list on that item: "Transpose these upgrades to Sim and Paper once completed." Three upgrades are in it. Columns that sort. Labels that wrap, with the privacy button centred beneath each one. A row press that moves the Indicator Voting Panel to that bot. All three now draw on the Simulator's own Scrumming Bots table, in both builds, over the Simulator's own figures.

### Every column sorts, over the Simulator's own price

A press on a column heading orders the rows by that column. A second press on the same heading reverses the order. The order is recomputed on every rewrite, so a figure that keeps moving does not unsort the list. The Simulator orders by the row's own reading and never by the live price pool, because the table and the page model each answer their own lookups.

`src/gui/simulator/sim_bot_status_table.py` - the Simulator's own sort readings

```
        def _sort_lookups(self) -> SortLookups:
            """``order_statuses``' two readings, over the Simulator's own figures."""

            def price(status) -> tuple:
                stats = status.get("stats", {}) or {}
                return self._display_price(status, stats)

            def denom_text(
                quote: str, base: str, exchange_id: str, target: float
            ) -> str:
                return self._denom_cell(quote, base, exchange_id, target)[0]

            return SortLookups(price, denom_text)
```

Read off the running program, both builds, at 700, 900 and 1400 pixels wide, over a fleet of 38 rows:

| reading | window | page |
|---|---|---|
| columns that move on a press | 9 of 10 | 9 of 10 |
| a second press reverses | 7 of 9 exactly | yes |
| the order survives a fresh fleet | yes | yes |

The Detail column holds one identical button a row, so it carries no value to order by and is the one column a press does not sort. Target A15 and Target A14 do not reverse to an exact mirror, because a row that draws nothing in those two columns stays beneath the figures whichever way the sort runs.

### The labels wrap and the privacy dot sits beneath each one

Each of the ten labels sits inside its own column, over as many lines as it needs, with one blue dot centred under the nine that carry a mask. The label steps down one size when its longest word will not fit the column. A press on the dot masks that column; a press on the words sorts it.

| width | Current Position Value, window | the page | dots | largest offset |
|---|---|---|---|---|
| 700 | 3 lines in 67 px, size 10 | 46.2 px over a 15.4 px line | 9 | 0.5 px |
| 900 | 3 lines in 92 px, size 11 | 30.8 px over a 15.4 px line | 9 | 0.5 px |
| 1400 | 3 lines in 154 px, size 11 | 15.4 px over a 15.4 px line | 9 | 0.008 px |

No label is cut at any of the three widths, in either build. The tenth column draws no dot and keeps the same empty dot row, so every label sits on one line.

### A row press moves the Simulator's own Voting Panel

A press on a row draws that bot on the Simulator's panel. A second press on the same row takes the panel to its empty state. A press on another row moves the panel to that bot. The panel's own dropdown still picks a bot, and the row it names highlights itself in the list. The panel is the Simulator's own, never Live's.

`src/gui/simulator/sim_trading_tab.py` - the one bot both sides read

```
        self._bot_list_link = BotListPanelLink(
            self._indicator_panel, self._bot_list_hosts
        )
        self._indicator_panel.bot_selected.connect(self._bot_list_link.panel_moved)
```

Read on the running program, both builds, at all three widths: a first press puts the pressed row's bot on the panel, a second press on that row answers no bot, and a press on another row answers that row's bot.

### The table is Live's table with three readings of its own

The Simulator's window table was a copy of Live's, taken before the header was rebuilt. It is now Live's table with the Simulator's three readings over it: the display price, the Target-denomination cell and the sort lookups. The page module carries the same relationship to Live's module, and the model already read the shared surface.

`src/gui/simulator/sim_bot_status_table.py` - the row's own price

```
        def _display_price(self, status: dict, stats: dict) -> tuple:
            """The row's own ``current_price``, at ``SIM_PRICE_AGE_S``."""
            del status
            return float(stats.get("current_price", 0.0) or 0.0), SIM_PRICE_AGE_S
```

### An empty Simulator fleet still draws the header

The Simulator's own fleet file holds no bot on this machine, which is the normal case. With no row at all the header still draws its ten labels and its nine dots, a press on a heading changes nothing and raises nothing, and a press where a row would be finds no row.

| reading, no rows | window | page |
|---|---|---|
| labels drawn | 10 | 10 |
| dots drawn | 9 | 9 |
| a heading press | no error, still no row | no error, still no row |
| a row press | answers no bot | finds no row |

## 2026-09-25 - a back test's orders are formatted to the venue

A Back Test now sizes every order to the rules the execution venue publishes for that market. The rules come from a recording the live connector writes when it reads its markets, so a run that reaches no venue still knows what the venue would accept. `src/exchange/market_rules_store.py` holds the recording under the runtime home.

The bar source and the execution venue are two different things. A tablet names both, and the rules are read against the venue that would execute the order.

```
# src/simulator/back_test.py:673
def venue_rules_for(asset: str, exchange_id: str, symbol: str) -> MarketRules:
```

### Two cells the Back Test report gains

Each fill row carries a **sized by** cell, reading `recorded venue rules` when the market's own record answered and `cited unit rule` when nothing was recorded. Each bot row carries an **orders refused** count, and the Activity Log carries one line per bot naming the reason behind that count.

```
u881l1-fractional: A34/USD sizes on coinbase's recorded rules, minimum 0.1 on a
size step of 0.01.
u881l1-fractional: A34/USD refused 710 order(s): 710 below the venue's minimum
size.
```

### A refusal costs no tranche

A fold refused under the venue's minimum settles no plan and spends no cash, so its tranches stay queued. They accumulate until a later fold buys a legal size. On the reading above, 19 of 730 latched folds filled and the rest waited.

# Paper Trader Tab

Reference. The third step of the promotion pipeline. Issue #19 built the screen. The tab row carried an empty tab until that build landed, and The screen as it stands now describes what draws there today.

## The empty tab it replaced

The tab drew three lines: its name, one sentence saying it was not built, and the issue that owned it. It read no bot, no price and no trade.

`src/gui/main_tabs/paper_trader_tab_surface.py` — the empty state it carried

```
HEADING = "Paper Trader"
ISSUE = 19
BUILT = False
STATE_TEXT = "This tab is not built."
ISSUE_TEXT = f"Issue #{ISSUE} carries the build-out."
```

Two frontends draw that one view model. `EmptyTabsMixin` in `src/gui/main_tabs/empty_tabs.py` builds the Qt tab, and `src/gui/web/paper_trader_tab.js` registers a panel with the Electron shell's panel host. `src.core.desktop_bridge` serves the model under `paper_trader_tab.state`.

## What the step is

Real time is its defining property. The Simulator replays stored candles as fast as the machine allows. Paper takes the live feed at the speed the market delivers it, and that is the whole difference between them.

Live, Paper and the Simulator differ in one thing only: where the data comes from. The trading logic stays one body of pure code that all three call. Only the stateful shells fork, which is what keeps a paper run from holding a live object.

The paper budget is twice the dollar target, so a bot has room to fold without the exercise ending on the first dip.

`src/trading/container/config.py` — `BotConfig`, the field it doubles

```
target_balance: float = 200.0  # Balance the bot trades relative to
```

Paper sat behind the Simulator, which had to reproduce the gate latches first. Nuclear Mode was to be a second gate; the operator cancelled it, so one gate remains and issue #19 built the tab after the Simulator rebuild landed.

## Where the platform once offered it

Two live surfaces named the step. The first is the Swarm tab's third sub-tab, labelled Paper Swarm. Its Start button flips a flag and relabels itself. No bot is constructed, no feed is attached and no order is recorded. Its caption is an empty string now, so the sub-tab promises nothing.

`src/gui/bot_visualizer.py` — the Start handler inside `_create_paper_bot_row`

```
def _toggle():
    if bot_data["running"]:
        bot_data["running"] = False
        start_btn.setText("■ Start")
```

The second is the wing toggle on the header strip. Pressing it swaps the whole wing. The Qt window no longer names a Paper Trader in the line it writes, and the React mirror still does, so the two hosts print different text.

`src/gui/main_tabs/main_window_surface.py` — the line the React mirror writes

```
"→ STOCK WING: equity exchanges + equity Paper Trader. (Crypto wing paused.)"
```

Issues #422 and #426 both folded into issue #19, which built the tab.

## The screen as it stands now

The tab is built. It sits second on the bar and carries the Trading tab's panes over the live exchange feed. `src/gui/paper_trader_tab.py` draws them in Qt and `src/gui/web/paper_trader_tab.js` draws them in React, from one view model.

`src/gui/main_tabs/paper_trader_tab_surface.py` — the fields both hosts read

```
DECLARED_FIELDS = (
    "accessible_name",
    "balance",
    "built",
    "feed",
    "fleet",
    "heading",
    "indicators",
    "issue",
    "method",
    "panes",
    "privacy_button",
    "reserved_rows",
    "run",
    "skin",
    "symbol",
    "symbols",
)
```

The left pane holds the Privacy Mode button, the market selector, the two button rows and the bot list. The bot list uses the Trading tab's own columns. The right pane holds the Indicator Voting Panel, computed from the live window. An empty selector draws the empty state and invents no reading.

`src/gui/main_tabs/paper_trader_tab_surface.py` — the empty state

```
def indicator_payload(
    candles: Sequence[Any], timeframe: str, symbol: str, refusal: str
) -> dict:
    model = ivp.IndicatorPanelModel()
    if refusal:
        model.show_no_data(refusal)
        return ivp.build_payload(model)
    summary = multi_tf_summary(candles, timeframe, symbol)
    if not summary:
        model.show_no_data(NO_SYMBOL_TEXT)
        return ivp.build_payload(model)
    model.set_summary(summary, symbol)
    return ivp.build_payload(model)
```

## The two presses

Import Live Fleet reads the stored bot record and lists what it finds. Start Paper Run opens the run and begins ticking. The same button then reads Stop Paper Run and ends it, leaving the balances and the trades on screen.

`src/gui/main_tabs/paper_trader_tab_surface.py` — the second row's two faces

```
{
    "name": DATA_POOL_ROW,
    "height_px": 18,
    "action": STOP_RUN_ACTION if running else START_RUN_ACTION,
    "text": STOP_RUN_TEXT if running else START_RUN_TEXT,
    "button_name": button_name(
        STOP_RUN_ACTION if running else START_RUN_ACTION
    ),
}
```

## Downstream only

The feed reader answers six names and refuses every other one, so a send has no spelling. It calls the venue's public market endpoints, which carry no key and no signature.

`src/paper/live_feed_source.py` — the refusal

```
def __getattr__(self, name: str):
    """Refuse every name outside ``READ_NAMES``."""
    raise SendRefused(
        f"LiveFeedSource answers {READ_NAMES} and cannot {name!r}. "
        "The Paper Trader receives and asks; it sends nothing."
    )
```

## One gate chain

A tick builds the same `GateContext` a live bot builds and evaluates the shipped scrum and fold chains on it. The paper side adds no gate and changes none.

`src/paper/paper_run.py` — what a tick evaluates

```
window = list(candles[-WINDOW_CANDLES:])
reading = bb_reading(window, bot)
summary = VotingEngine().compute_all(window, bot.ta_timeframe, symbol=bot.symbol)
context = tape_context(bot, balance, window, reading, summary)
armed = latch(context)
```

## The fake balance

Each bot opens a budget of twice its dollar target: units worth the target, and the rest in cash to fund a fold. The balances live in memory and are dropped when the tab goes away.

`src/paper/fake_balance.py` — the opening

```
def opening_balance(target_usd: float, price: float) -> FakeBalance:
    whole = budget_usd(target_usd)
    held_usd = min(whole / BUDGET_MULTIPLE, whole)
    units = held_usd / float(price) if float(price) > 0.0 else 0.0
    return FakeBalance(
        units=units,
        cash_usd=whole - units * float(price),
        tranches=0,
        budget_usd=whole,
    )
```

## The fleet ledger

The fake balance is one figure for the whole fleet, not one figure per bot. Five paper bots with a $200 target open Paper Spendable at $1000 and Paper Locked at $1000. Each variable holds the full fleet sum, which is twice the fleet's dollar target in total.

`src/paper/fake_balance.py` — the opening

```
def opening_ledger(bots: Sequence[Any]) -> PaperLedger:
    total = fleet_target_usd(bots)
    return PaperLedger(
        fleet_target_usd=total,
        opening_spendable_usd=total,
        opening_locked_usd=total,
    )
```

The fleet changes when a bot is added or removed. Every press of Start Paper Run rebuilds the ledger from the fleet it is handed, so both opening figures move on the next press and never inside a run. Six $200 bots open $1200 in each figure and four open $800.

Four figures are tracked. Paper Spendable is the cash the fleet holds. Paper Locked is what its units are worth at the last price the feed answered. Paper Realized Profits moves when a fold buy closes the tranche a scrum sell opened. Paper Mature Profits is the profit on positions past 200 percent growth, the figure the live wing already reads.

`src/trading/smart_wire.py` — the maturity rule both wings call

```
MATURE_GROWTH_PCT: float = 200.0
"""A position is mature once its value exceeds its cost basis by this percent."""
```

The four figures are held apart from the live ones. They live in memory on the run, they are never written into the stored bot record, and no paper figure reaches the header strip's real-money columns.

`src/paper/fake_balance.py` — the four labels the strip prints

```
FIGURE_LABELS = (
    ("spendable_usd", SPENDABLE_LABEL),
    ("locked_usd", LOCKED_LABEL),
    ("realized_profit_usd", REALIZED_LABEL),
    ("mature_profit_usd", MATURE_LABEL),
)
```

The Qt window and the React page print one line from one view model, so the two hosts cannot report different money.

`src/gui/main_tabs/paper_trader_tab_surface.py` — the one line

```
"text": LEDGER_SEPARATOR.join(f"{one['label']} {one['text']}" for one in cells),
```

## The Paper Trader log

Every paper action is written to one file. Each line carries two halves: the gate decision that produced the action and, when one filled, the pretend trade in the shape a Coinbase year-to-date row takes. Validation reads both of those shapes already, so a paper run is checked by the reader that checks the live wing.

`src/paper/paper_log.py` — the row

```
return {
    "timestamp": iso_stamp(tick.wall_ms),
    "category": CATEGORY,
    "exchange": str(exchange_id),
    "bot_id": str(tick.bot_id),
    "data": gate_half(tick),
    "trade": trade_half(tick.filled) if tick.filled is not None else None,
    "ledger": dict(figures or {}),
}
```

The log has a home of its own. It is a sibling of the live folders and never sits inside one, so nothing a paper run writes can land in a live tree.

`src/paper/paper_paths.py` — the root

```
PAPER_ROOT: Path = Path.home() / ".acervator_paper"
```

Gate names come from one place. The row records the blocker phrases the chain produced and the labels those phrases map to, and it spells no gate name of its own.

`src/trading/gate_vocabulary.py` — the mapping the log calls

```
def gate_for_blocker(blocker: str) -> str:
    """Map one blocker string to its gate label, or "" if unknown."""
```

A trade is never invented. A line carries a trade half only because the gate chain latched and the fill was applied to a fake balance.

`src/paper/paper_run.py` — the only writer

```
paper_log.append_row(
    paper_log.paper_row(seen, bot.exchange_id, run.figures())
)
```

## Real time

A run ticks on the wall clock and asks the feed for one bot per fire, so a fleet of many bots never blocks the window on a single pass. One bar is shared across the open bots.

`src/gui/paper_trader_tab.py` — the round robin

```
def advance_once(self) -> list:
    """Tick the next open bot against its newest live window."""
    if self._run is None or not self._run.running or not self._run.bots:
        return []
    chosen = self._run.bots[self._cursor % len(self._run.bots)].bot_id
    self._cursor += 1
    made = surface.advance_run(self._run, self._feed, chosen)
    self.refresh()
    return made
```

## Paper Trade History

The manual's part list names a second History tab reading a paper trade log. No such module exists. `HistoryTab` in `src/gui/history_tab.py` reads live venue history alone.

The ledger such a tab would read now exists. Every paper action lands in the file the Paper Trader log names, and no screen reads that file yet.

In development.

## 2026-09-08 08:17 - #19 - what the closed issues landed

Real-time, API-fed trades against a fake budget. This is designed as the second tier of strategy validation within the platform.

The tab is now called Paper. It sits second on the bar, on a white ground with black text, and it is the only white tab.

The screen was empty until issue #19 built it. The tab row carried a Paper Trader placeholder, which drew its name, one sentence saying it was not built, and the issue that owned it. The section under this one holds what the tab is now.

`src/gui/main_tabs/paper_trader_tab_surface.py` — the empty state it carried

```
HEADING = "Paper Trader"
ISSUE = 19
BUILT = False
STATE_TEXT = "This tab is not built."
ISSUE_TEXT = f"Issue #{ISSUE} carries the build-out."
```

The Qt tab and the `src/gui/web/paper_trader_tab.js` panel both draw that one view model, so neither can drift from the other.

Live, Paper and the Simulator differ in one thing only, where the data comes from. The trading logic stays one body of pure code all three call, and only the stateful shells fork. Real time is Paper's defining property, and its budget is twice the dollar target.

`src/trading/container/config.py` — `BotConfig`, the field that budget doubles

```
target_balance: float = 200.0  # Balance the bot trades relative to
```

Two live surfaces once offered the step. The Swarm tab's Paper Swarm sub-tab is chrome: its Start button flips a flag and relabels itself, and no bot is constructed. Its caption is now an empty string, so it promises nothing. The wing toggle no longer names a Paper Trader in the Qt window, while the React mirror still does. Issues #422 and #426 folded into issue #19.

### The tab as it stands now

The screen is built. It is the Trading tab's panes over the live exchange feed: the Privacy Mode row, the market selector, the two button rows, the bot list, the Indicator Voting Panel, the Live Feed pane, the Fake Balance table and the Paper Run table. The Qt window and the React page draw one view model.

`src/gui/main_tabs/paper_trader_tab_surface.py` — what the tab answers

```
METHOD = "paper_trader_tab.state"

HEADING = "Paper"
ISSUE = 19
BUILT = True
```

The data path is the venue's public market feed, and it goes one way. The reader names the six things it answers, and every other name raises. An order, a cancellation or a venue write cannot be written through it.

`src/paper/live_feed_source.py` — the read set and the refusal

```
READ_NAMES = (
    "venue",
    "product_id",
    "granularity",
    "candles",
    "ticker",
    "asked_at",
)


class SendRefused(AttributeError):
    """Raised when a name outside ``READ_NAMES`` is asked of ``LiveFeedSource``."""
```

Live candles enter the same gate chain a live bot runs. Nothing on the paper side defines a gate of its own.

`src/paper/paper_run.py` — the tick

```
context = tape_context(bot, balance, window, reading, summary)
armed = latch(context)
```

The money is fake and lives in memory alone. A run writes no file, and no figure it produces reaches the stored bot record or the header strip.

`src/paper/fake_balance.py` — the budget

```
BUDGET_MULTIPLE = 2.0
```

The two strips the Trading tab carries are not copied, and neither is their supporting code. Their rows carry Import Live Fleet and the Start or Stop press.

The fake balance is a fleet figure. Five paper bots with a $200 target open Paper Spendable at $1000 and Paper Locked at $1000, each holding the full fleet sum. Adding or removing a bot moves both openings on the next press of Start.

`src/paper/fake_balance.py` — the fleet total both openings take

```
def fleet_target_usd(bots: Sequence[Any]) -> float:
    """The sum of every bot's ``target_usd``, the figure both openings take."""
```

Two more figures are tracked beside them. Paper Realized Profits moves when a fold buy closes the tranche a scrum sell opened, and Paper Mature Profits is the profit on positions past 200 percent growth, which is the figure the live wing already reads. No paper figure is written into the stored bot record or the header strip.

`src/paper/fake_balance.py` — the strip's four figures

```
FIGURE_LABELS = (
    ("spendable_usd", SPENDABLE_LABEL),
    ("locked_usd", LOCKED_LABEL),
    ("realized_profit_usd", REALIZED_LABEL),
    ("mature_profit_usd", MATURE_LABEL),
)
```

Every paper action is recorded in one file of its own. Each line carries the gate decision and, when one filled, the pretend trade in the shape a Coinbase year-to-date row takes. The file sits beside the live folders and never inside one.

`src/paper/paper_paths.py` — where the log lives

```
PAPER_ROOT: Path = Path.home() / ".acervator_paper"
```

## How the Paper tab reaches the bar

`PaperTraderTabMixin` builds the tab and inserts it into the window's tab book. The builder catches every error, writes one warning line, and leaves the tab off the bar. A tab that asks the panel for a value the panel does not publish is therefore a missing tab, not a crash the operator can see.

`src/gui/main_tabs/paper_trader_tab.py` — the builder

```
def _build_paper_trader_tab(self) -> None:
    """Insert the Paper tab at ``PAPER_BUILD_INDEX``."""
    try:
        from ..variant_surface import PAPER_TRADER, surface_class

        self._paper_trader_tab = surface_class(PAPER_TRADER)()
        self._main_tabs.insertTab(
            PAPER_BUILD_INDEX, self._paper_trader_tab, HEADING
        )
    except Exception as exc:  # noqa: BLE001 - a missing tab is not a crash
        logger.warning("Paper tab unavailable: %s", exc)
        self._paper_trader_tab = None
```

The window builds nine tabs. Paper takes the second slot, after Sim.

```
Sim  Paper  Live  Charts  Inspector  Swarm  History  Status  Console
```

### The line beside the voting panel title

One label sits beside the Indicator Voting Panel title. It holds the panel's empty state and nothing else. The shared function `no_data_text` builds that line, and the Simulator page shows it. The label is hidden whenever the panel has values to draw.

`src/gui/paper_trader_tab.py` — the Qt clone draws the line

```
def _draw_indicators(self, panel: dict) -> None:
    self._indicator_title.setText(panel["title_text"])
    # no_data text is empty while the panel holds a reading, so the
    # summary label shows only the one reason there is nothing to draw.
    reason = panel["no_data"]["text"]
    self._indicator_summary.setText(reason)
    self._indicator_summary.setVisible(bool(reason))
    for table, spec in zip(self._indicator_tables, panel["tables"], strict=True):
        self._fill_indicator_table(table, spec)
```

With no paper fleet the label names the press that builds one, under both builds.

```
No TA data — No paper fleet yet. Press Import Live Fleet.
```

## The style sheet the Paper page loads

The page carries one style sheet. `TAB_STYLE_ASSETS` names it, and `page_html` reads the file and writes it into the page's head.

`src/gui/react_paper_trader_tab.py` — the sheet the page carries

```
TAB_STYLE_ASSETS: tuple[str, ...] = ("paper_trader_tab.css",)
```

The sheet sets the page ground, the type, the pane grid and the table rules. The grid repeats the two pane sizes the Qt clone gives its splitters, so the bot list, the voting panel, the feed pane and the run pane take the same share of the window in both builds.

`src/gui/web/paper_trader_tab.css` — the pane grid

```
.paper-top,
.paper-bottom {
  display: grid;
  grid-template-columns: 600fr 500fr;
}
```

Without the sheet the page draws on the browser's own defaults: a white ground, serif type, one column and no grid.

### The skin tokens the Paper sheet reads

The sheet names no colour of its own. Every colour arrives as a custom property on the tab element, written from the `SKIN` dictionary the surface builds. A scrum row and a fold row in the Paper Run table take the two colours `SIDE_COLOURS` gives them in the Qt clone.

`src/gui/web/paper_trader_tab.css` — the two trade rows

```
.paper-tab tr[data-side="scrum"] td {
  color: var(--paper-scrum-colour, var(--text));
}

.paper-tab tr[data-side="fold"] td {
  color: var(--paper-fold-colour, var(--text));
}
```

### Start Paper Run redraws the pane

`start_run` draws the pane after it opens the run. With no paper fleet the first tick returns nothing, so without that draw the Paper Run pane kept the word it held before the press while the run was open.

`src/gui/paper_trader_tab.py` — the press that opens a run

```
    def start_run(self) -> str:
        """Open the run and tick one bot per timer fire from now."""
        if not self._bots:
            self._bots = surface.live_fleet()
        self._run = surface.start_run(self._bots)
        self._cursor = 0
        self.advance_once()
        self.refresh()
        self._tick_timer.start(surface.tick_interval_ms(self._run, self._symbol))
        return self._run.state
```

Both builds carry the same draw.

## 2026-09-19 - #19 - Live's tab code cloned under Paper names

Unit Q1 of issue #19. The Paper tab is now Live's tab code, forked under Paper names in both builds, with Import Live Fleet as the fleet's one way in and the header strip reading the paper ledger while Paper is in front. The two hosts the sections above describe, `src/gui/paper_trader_tab.py` and `src/gui/react_paper_trader_tab.py`, stay on disk and nothing loads them; a later unit removes them once a paper bot ticks on the clone.

### The budget rule today overtakes four sentences

Four sentences on this page carry a figure the operator's words of 2026-09-19 overtake. Each is quoted here and kept above as it stands.

- "The paper budget is twice the dollar target, so a bot has room to fold without the exercise ending on the first dip." (What the step is)
- "Each bot opens a budget of twice its dollar target: units worth the target, and the rest in cash to fund a fold." (The fake balance)
- "Each variable holds the full fleet sum, which is twice the fleet's dollar target in total." (The fleet ledger)
- "Real time is Paper's defining property, and its budget is twice the dollar target." (2026-09-08 08:17)

The rule now: the paper budget is unbounded, and Paper Spendable opens at the sum of the fleet's Target Balances. His words: *"a fake, unbounded (starts equal to bots total target balance) budget."* No fold is refused

for cash. `BUDGET_MULTIPLE` in `src/paper/fake_balance.py` still reads `2.0`; the unit that starts a paper run retires it, and no clone code reads it.

`src/paper/fake_balance.py` — the opening figure the rule keeps

```
def fleet_target_usd(bots: Sequence[Any]) -> float:
    """The sum of every bot's ``target_usd``, the figure both openings take."""
```

### The Qt tab is Live's tab code, forked

The Qt build of the Paper tab is `PaperTradingTab`, a fork of the Live tab's own code under the Paper Trader's name. The seam's Qt loader answers it.

`src/gui/variant_surface.py` — the Qt loader

```
def _qt_paper_trader() -> type:
    """Import and return the Qt Paper tab, ``PaperTradingTab``."""
    from .paper.paper_trading_tab import PaperTradingTab

    return PaperTradingTab
```

Each module under `src/gui/paper/` is one Live module copied and renamed, or one of the Simulator's forks copied once more where that fork's only change was the feed. The copy keeps Live's layout, titles, sizes and design tokens.

| Paper module | forked from |
|---|---|
| `paper_trading_tab.py` `PaperTradingTab` | `src/gui/main_tabs/trading_tab.py` `_build_trading_tab`, and the window's `add_exchange_tab` |
| `paper_exchange_tab.py` `PaperExchangeTab` | `src/gui/widgets/exchange_tab.py` `ExchangeTab` |
| `paper_bot_status_table.py` `PaperBotStatusTable` | `src/gui/simulator/sim_bot_status_table.py` `SimBotStatusTable` |
| `paper_extractor_bot_table.py` `PaperExtractorBotTable` | `src/gui/simulator/sim_extractor_bot_table.py` `SimExtractorBotTable` |
| `paper_indicator_panel.py` `PaperIndicatorVotingPanel` | `src/gui/simulator/sim_indicator_panel.py` `SimIndicatorVotingPanel` |
| `paper_status_log.py` `PaperStatusLog` | `src/gui/simulator/sim_status_log.py` `SimStatusLog` |
| `paper_bot_detail.py` `PaperBotDetailDialog` and its seven tab mixins | `src/gui/simulator/sim_bot_detail.py` `SimBotDetailDialog` and its seven |
| `paper_exchange_choice.py` `PaperExchangeChoiceDialog` | `src/gui/simulator/sim_exchange_choice.py` `SimExchangeChoiceDialog` |
| `src/paper/fleet_source.py` `PaperFleetSource`, `PaperBot` | `src/simulator/fleet_source.py` `FleetSource`, `SimBot`, one fleet and no run mode |
| `src/paper/paper_bot_manager.py` `PaperBotManager` | `src/simulator/sim_bot_manager.py` `SimBotManager` |
| `src/paper/paper_bot_view.py` `PaperBotView` | `src/simulator/sim_bot_view.py` `SimBotView` |

Diagram: flowchart, left to right

- variant_surface PAPER_TRADER -> **PaperTradingTab**
- PaperTradingTab -> **exchange layer stack**
- exchange layer stack -> **PaperExchangeTab**
- PaperExchangeTab -> **PaperBotStatusTable**
- PaperExchangeTab -> **PaperExtractorBotTable**
- PaperTradingTab -> **PaperIndicatorVotingPanel**
- PaperTradingTab -> **PaperStatusLog**
- PaperTradingTab -> **API Interaction Log**
- PaperTradingTab -> **PaperFleetSource**
- PaperFleetSource -> **paper_fleet.json under PAPER_ROOT**

### What the fork draws

The tab is Live's four splitters at Live's sizes: the exchange layer stack beside the panel on top, the Activity Log and the API Interaction Log side by side below. Every module carries Live's title. The venue page holds Privacy Mode where Live draws it, `+ New Bot` where Live draws it, the Scrumming Bots table and the Extractor Bots table, both hidden until a row arrives, and the command bar of Start, Pause, Stop, Restart and Delete. The panel holds its title, the Bot selector and its privacy dot, the currency rate strip, both indicator tables, both confidence bar graphs, the timeframe-lock line and the staleness banner.

Three positions differ from Live by ruling, the ruling unit 7 of issue #117 made for the Simulator. The corner Live gives `■ Add Crypto Exchange` holds Import Live Fleet and Start Paper Run, each at Live's corner-button width. The Get Started card holds the same two where Live's card holds its add button. The news line and the data-pool line are not forked; the data-pool row keeps Live's height and holds nothing.

`src/gui/paper/paper_trading_tab_surface.py` — the corner buttons

```
CORNER_BUTTONS = (
    (paper.IMPORT_LIVE_FLEET_ACTION, paper.IMPORT_LIVE_FLEET_TEXT),
    (paper.START_RUN_ACTION, paper.START_RUN_TEXT),
)
```

A press on `+ New Bot` writes one Activity Log line naming Import Live Fleet as the fleet's way in; the paper bot wizard is a later unit's. A press on Start Paper Run writes one line saying no paper run is built; the run is a later unit's too.

`src/gui/paper/paper_trading_tab_surface.py` — the two lines

```
NEW_BOT_FORMAT = (
    "+ New Bot on {exchange}: the Paper fleet is loaded through "
    "{way_in}; the paper bot wizard is not built."
)

START_RUN_TEXT = "Start Paper Run: no paper run is built; nothing started."
```

**THE FORM THE WIZARD REPLACED.** The block above stays here, because a reader who saw that line needs to find it. `NEW_BOT_FORMAT` left the tree, and the press opens the Bot Creation Wizard instead of writing a line. Its neighbour in that module now holds the button's caption rather than a refusal.

`src/gui/paper/paper_trading_tab_surface.py` — what stands there today

```
IMPORT_LIVE_FLEET_TEXT = "Import Live Fleet"
START_RUN_TEXT = "Start Paper Run"
STOP_RUN_TEXT = "Stop Paper Run"
```

### What the fork does not carry

The copies hold no live bot manager, no connector and no event bus. Every send the Live code makes is cut, not stubbed: the signal-contract emits, the event-bus emit behind the bot selector, the API-log listener on the process-wide log, the watchdog over the live bot manager, the TA snapshot store, the data-pool read behind the display price, and the demo readings the Live panel invents from a random walk. Zero occurrences of `ScrummingBot`, `BotContainer`, `BotManager`, `EventBus` and `PhantomBalance` under `src/gui/paper/` and `src/paper/`.

The Simulator's run modes, its way-in row, its replay layer, its Stone Tablet source, its Portfolio Battery, its retrieval and its nigredo tone are not carried. A Paper module forked from a Simulator fork has those parts cut.

Asked for a send by name, `PaperFleetSource` raises `SendRefused`. The Fire button on a row reaches it and the refusal lands in the Activity Log.

`src/paper/fleet_source.py` — the refusal

```
    def __getattr__(self, name: str):
        """Refuse every name outside ``READ_NAMES``."""
        raise SendRefused(
            f"PaperFleetSource answers {READ_NAMES} and cannot {name!r}. "
            "The Paper Trader receives and asks; it sends nothing."
        )
```

### The React page is Live's page modules, forked

The React build of the Paper tab is `PaperTradingTabReact`, a fork of the Live tab's React host under the Paper Trader's name, and it loads seven page modules that are Live's seven copied under Paper names. The seam's React loader answers it.

`src/gui/variant_surface.py` — the React loader

```
def _react_paper_trader() -> type:
    """Import and return the React Paper tab, ``PaperTradingTabReact``."""
    from .paper.paper_react_trading_tab import PaperTradingTabReact

    return PaperTradingTabReact
```

| Paper module | forked from |
|---|---|
| `src/gui/web/paper_trading_tab.js` | `trading_tab.js` |

| Paper module | forked from |
|---|---|
| `src/gui/web/paper_exchange_tab.js` | `exchange_tab.js` |
| `src/gui/web/paper_indicator_panel.js` | `sim_indicator_panel.js`, the flip button cut |
| `src/gui/web/paper_status_log.js` | `sim_status_log.js` |
| `src/gui/web/paper_bot_status_table.js` | `sim_bot_status_table.js` |
| `src/gui/web/paper_extractor_bot_table.js` | `sim_extractor_bot_table.js` |
| `src/gui/web/paper_table_cells.js` | `sim_table_cells.js` |
| `src/gui/paper/paper_react_trading_tab.py` `PaperTradingTabReact` | `src/gui/react_trading_tab.py` `TradingTabReact` |
| `src/gui/paper/paper_trading_tab_surface.py` | `src/gui/main_tabs/trading_tab_surface.py`, the payload builder |
| `src/gui/paper/paper_exchange_tab_surface.py` | `src/gui/main_tabs/exchange_tab_surface.py`, the payload builder |
| `src/gui/paper/paper_react_bot_detail.py` `PaperBotDetailReactDialog` | `src/gui/simulator/sim_react_bot_detail.py`, over Live's nine window bundles |

Diagram: flowchart, left to right

- variant_surface PAPER_TRADER -> **PaperTradingTabReact**
- PaperTradingTabReact -> **paper_trading_tab.js**
- paper_trading_tab.js -> **paper_exchange_tab.js**
- paper_exchange_tab.js -> **paper_bot_status_table.js**
- paper_exchange_tab.js -> **paper_extractor_bot_table.js**
- paper_bot_status_table.js -> **paper_table_cells.js**
- paper_extractor_bot_table.js -> **paper_table_cells.js**
- paper_trading_tab.js -> **paper_indicator_panel.js**
- paper_trading_tab.js -> **paper_status_log.js**
- paper_trading_tab.js -> **API Interaction Log pane**

### What the page draws

The page is Live's four splitters at Live's sizes, read off Live's own `trading_tab_surface`. Every module carries Live's title and Live's part name, so the page and the Qt fork enumerate the same modules in the same order. The corner holds Import Live Fleet and Start Paper Run in Live's corner-button chrome, and the Get Started card holds the same two at Live's card-button size. The venue page holds Privacy Mode, plain space where Live draws the news line, `+ New Bot`, the data-pool row at Live's height holding a blank line, the two tables and the command bar.

`src/gui/web/paper_trading_tab.js` — the corner

```
// The corner Live gives its add button: the payload's buttons in one row.
function CornerButtons(props) {
  var listed = listField(props.layer, CORNER_BUTTONS);
```

The host builds one payload per bridge method from state it owns: the tab payload from its own `PaperTradingTabState`, the Activity Log from its own `PaperStatusLogModel`, the panel from its own `IndicatorPanelModel`, and each seated venue's three payloads from that venue's own `ExchangeTabModel`, its `PaperBotStatusTableModel` and Live's `ExtractorBotTableModel`. Every ask the page makes is written to the console line the host reads, and `run_action` answers it; nothing is sent anywhere else.

`src/gui/paper/paper_react_trading_tab.py` — the console line

```
ACTION_PREFIX = "acervator-paper:"
```

### The header strip shows on Paper

The window's header strip stays on screen while the Paper tab is in front. `ISOLATED_TABS` names no tab now, so the strip hides on nothing. While Paper is in front the ten fields carry the Paper Trader's figures: SPENDABLE, REALISED, LOCKED and MATURE from the paper ledger, EXCH from the seated venues, and the five cards from the held records. On every other tab the strip carries what it carried before; the live fleet's figures do not appear while Paper is in front, and no paper figure appears on Live.

`src/gui/main_tabs/main_window_surface.py` — the tuples and the reader

```
ISOLATED_TABS: tuple = ()

#: The tabs the header strip reads the Paper Trader's ledger and records on.
PAPER_FED_TABS = (PAPER_TAB,)


def header_strip_reads_paper(tab_name: Any) -> bool:
    """Whether the stat strip reads the Paper Trader's ledger and records on the tab named."""
    return tab_name in PAPER_FED_TABS
```

`src/gui/main_window.py` — `_refresh_header_strip`, the branch beside the Sim's

```
            if self._header_strip_reads_paper():
                paper_tab = self._paper_trader_tab
                self._write_header_strip(
                    paper_tab.aggregate(), paper_tab.exchange_count()
                )
                return
```

The tab's `aggregate` is `strip_aggregate`: Live's arithmetic over the held records, with the ledger's four money figures laid over the four money keys. Before a run the ledger holds zeros, so the four money columns read an em dash, as Live's read with no bot.

`src/paper/fleet_source.py` — the ledger's keys over the aggregate's

```
LEDGER_TO_AGGREGATE = (
    ("spendable_usd", "wallet_cash_usd"),
    ("locked_usd", "crypto_position_value_usd"),
    ("realized_profit_usd", "total_realized_exchange"),
    ("mature_profit_usd", "total_mature_exchange"),
)
```

Diagram: flowchart, left to right

- dashboard tick, 2000 ms -> **tab in front**
- tab change -> **tab in front**
- pick -- Sim -> **SimTradingTab.aggregate**
- pick -- Paper -> **PaperTradingTab.aggregate**
- pick -- any other -> **BotManager.get_aggregate_stats**
- SimTradingTab.aggregate -> **the ten cells**
- PaperTradingTab.aggregate -> **the ten cells**
- BotManager.get_aggregate_stats -> **the ten cells**

### Import Live Fleet fills the tables and the strip

Import Live Fleet copies the stored bots of one exchange out of the live fleet file into the Paper Trader's own fleet file, in both builds. A press at the corner or on the Get Started card reads which exchanges the live file names. With more than one it opens the chooser; with exactly one it takes that one and opens nothing; with none it writes one Activity Log line naming the file and moves nothing. Each stored bot on the chosen exchange is copied whole, every key of the record, into the paper fleet under its own bot id, and the fleet signal fires: the paper fleet file is written, the venue seats, and the two tables draw one row per imported bot. The Activity Log reads `Imported 38 bot(s) from bot_state.json on coinbase.` on a fleet of thirty-eight. The live fleet file is only read, never written.

`src/paper/fleet_source.py` — the copy

```
    def import_live_fleet(self, exchange_id: str) -> list[PaperBot]:
```

`src/paper/paper_paths.py` — where the paper fleet file lives

```
#: The paper fleet file under ``PAPER_ROOT``, in ``bot_state.json``'s shape.
PAPER_FLEET_NAME = "paper_fleet.json"
```

The tab starts empty. The fleet source answers the records the paper fleet file holds and nothing else, so with no paper fleet there is no venue, no row and no figure until Import Live Fleet loads one. On the next launch the tab reads the paper fleet file once and draws the imported rows from it.

### A loaded bot reads IDLE

Import Live Fleet copies each stored record and writes its state `idle`, whatever the live file saved, and the paper fleet file's records read `idle` at construction. That is the live restore's own rule, and the Simulator's. A row's state is the Bot ID cell's colour and its tooltip's `State:` line; Live's ten columns hold no State column.

`src/paper/fleet_source.py` — the one write

```
def loaded_idle(record: dict) -> dict:
    """``record`` with its ``state_when_saved`` written ``BotState.IDLE``, as
    ``restore_bots_from_state`` recreates every persisted bot in IDLE until
    the operator starts it; a loaded bot never reads ``running`` with no run."""
    record["state_when_saved"] = BotState.IDLE.value
    return record
```

The command bar moves a record's state through `PaperBotManager`: Start writes `running` and Live's ✓ `Bot <id> RUNNING.` line, Pause `paused`, Stop `stopped`, Restart `running`, and Delete opens Live's Delete box. No paper bot ticks yet; the state is the record's. Detail opens the seven tabs over the record through `PaperBotDetailDialog`, and Apply is refused: the pending line reads `Refused 1 change(s) — the Paper Trader sends nothing`.

### How the Paper tab reaches the bar today

The builder in `src/gui/main_tabs/paper_trader_tab.py` is unchanged. It asks the seam for the Paper class, and the seam now answers the clone in both builds. The window builds nine tabs and Paper takes the second slot, after Sim, in both.

`src/gui/variant_surface.py` — the registration

```
register(PAPER_TRADER, _qt_paper_trader, _react_paper_trader)
register(PAPER_BOT_DETAIL, _qt_paper_bot_detail, _react_paper_bot_detail)
```

The window's floor is 1400 by 900 pixels, set in `src/gui/main_window.py`, so no tab is reached narrower than 1400 through the window. Constructed alone, the Qt tab draws at 806 pixels wide and no narrower after a fleet loads, the sum of Live's own minimum sizes; the React page draws at 700.

### What the clone reading measured

Read off the real window in both builds over a scratch home holding a stand-in `bot_state.json` of the operator's record shape, thirty-eight scrumming records on coinbase at 5m, every socket but loopback refused:

```
reading                                        Qt                       React
Paper on the bar                               second of nine           second of nine
Live modules on the tab before the clone       0 of 6                   0 of 10
Paper modules on the tab after the clone       6 of 6                   10 of 10
strip on Paper at open                         — — — — EXCH 0           — — — — EXCH 0
Import Live Fleet: rows                        38, State: IDLE          38, State: IDLE
Import Live Fleet: Bot ID cell colour          #888888                  rgb(136, 136, 136)
venue seated, EXCH                             Coinbase, 1              Coinbase, 1
paper fleet file                               38 records, all idle     38 records, all idle
bot_state.json hash after every press          unchanged                unchanged
a planted byte on a copy                       hash moved               hash moved
a planted live figure on the Paper strip       did not appear           did not appear
a planted paper figure on the Paper strip      appeared                 appeared
the same paper figure on the Live strip        did not appear           did not appear
Detail                                         7 tabs, Apply refused    7 tabs, Apply refused
old hosts in the import closure                neither                  neither
```

## 2026-09-20 - #19 - The paper exchange adapter feeds the tab

Unit Q2 of issue #19. The Paper tab now holds one paper exchange adapter, `PaperExchange` in `src/paper/paper_exchange.py`, beside the feed reader the sections above describe. It answers every read a paper bot asks of the venue from the Coinbase Advanced Trade public market endpoints, and no write. Import Live Fleet reads the venue's product list once and each record's ticker once, and every one of those reads is a block on the API Interaction Log in both builds.

### The feed reader sentences today overtake two passages

Two passages on this page describe the feed reader `LiveFeedSource`. Each is quoted here and kept above as it stands.

- "The feed reader answers six names and refuses every other one, so a send has no spelling." (Downstream only)
- "The data path is the venue's public market feed, and it goes one way. The reader names the six things it answers, and every other name raises." (The tab as it stands now)

The rule now: the tab's feed reader is `PaperExchange`, which answers the eleven names in its `READ_NAMES` and refuses every other public name. `LiveFeedSource` stays on disk for the two old hosts, which nothing loads, and its `GRANULARITY` and `BAR_SECONDS` tables grew from eight rows to nine, the `4h` row added, because the venue's candles page lists nine granularity names. A `1w` ask of `LiveFeedSource` sent `FIVE_MINUTE` to the venue and answered 5m bars under the weekly name; the adapter rolls `1w` up from daily bars instead.

`src/paper/paper_exchange.py` — the read set

```
READ_NAMES = (
    "venue",
    "product_id",
    "granularity",
    "timeframes",
    "products",
    "ticker",
    "candles",
    "windows",
    "quote_rate",
    "asked_at",
    "calls",
)
```

### What the adapter answers, and from which endpoint

`products` reads `GET market/products` and keeps the products the venue trades, by the rule the Market Inspector's `_product_trades` states: `status` online and `trading_disabled` not set. `ticker` reads `GET market/products/{id}/ticker?limit=1` and answers the last trade's price as `last`, with the book's `best_bid` and `best_ask` from the same answer; `last` is None when the venue answers no trade. `candles` reads `GET market/products/{id}/candles` at any of the nine granularity names, at most 350 bars a call, and answers `[ts_ms, open, high, low, close, volume]` rows oldest first. `windows` answers one `candles` window per higher timeframe a record's phantoms name. `quote_rate` answers 1.0 for USD with no call, and otherwise the `{QUOTE}/USD` ticker's `last`, the book's mid when the venue answers no trade; the thirteen USDC records share one `USDC/USD` read.

`src/paper/paper_exchange.py` — the ticker's answer

```
    def _ticker_view(self, symbol: str, entry: TickerEntry) -> dict:
        return {
            "symbol": symbol,
            "product_id": entry.symbol,
            "last": entry.last or None,
            "best_bid": entry.bid or None,
            "best_ask": entry.ask or None,
            "timestamp": entry.timestamp,
        }
```

### One rollup, and only what the venue serves

The venue lists no weekly granularity. A `1w` ask reads daily bars in pages of 350, newest page first, until seven days per week asked are held, and rolls them through the stone tablets' `_rollup`, the one rollup every derived tablet timeframe reads through. Each timestamp is moved back by `MONDAY_OFFSET_MS` before the rollup and forward after it, the shift the Market Inspector's `weekly_from_daily` makes, so every weekly bucket starts on Monday 00:00 UTC. A 100-bar weekly window needs 700 daily bars, two calls. A granularity outside the nine is sent as spelled, so the venue's own 400 answers it and lands on the pane as an error block.

`src/paper/paper_exchange.py` — the rollup

```
def weekly_rows(daily: Sequence[Sequence[float]]) -> list[list[float]]:
    shifted = [[row[0] - MONDAY_OFFSET_MS, *row[1:6]] for row in daily]
    return [
        [float(int(row[0]) + MONDAY_OFFSET_MS), *row[1:6]]
        for row in _rollup(shifted, DAYS_PER_WEEK, WEEK_MS)
    ]
```

### Cached with Live's two windows, paced at the connector's interval

Every ticker answer is held in a `TickerEntry` and every candle window in a `CacheEntry`, the two slot types Live's `MarketDataPool` holds in `src/exchange/data_pool.py`. A ticker asked again within 5 s is served from the slot with no call and no block. A candle window asked again within one bar of its timeframe, `TF_SECONDS`, is served the same way while the slot holds enough rows. Every venue call waits `PUBLIC_MIN_INTERVAL_S` after the call before it, the figure `src/exchange/market_inspector_fetcher.py` states for the public route and the exchange connector keeps as `_min_request_interval`, 0.1 s; one lock holds the calls in order, so two threads asking at once are spaced. A 429 is recorded as a warning block, waited out for the venue's `Retry-After` seconds when the answer carries one and one interval otherwise, and asked once more.

`src/paper/paper_exchange.py` — the cache and the pace

```
from ..exchange.data_pool import CacheEntry, TickerEntry
from ..exchange.market_inspector_fetcher import (
    API_LEVEL_ERROR,
    API_LEVEL_SUCCESS,
    API_LEVEL_WARNING,
    DAYS_PER_WEEK,
    MONDAY_OFFSET_MS,
    PUBLIC_MIN_INTERVAL_S,
    WEEK_MS,
    _product_trades,
)
```

### Every read is a block on the API Interaction Log

The adapter records one entry per venue call on the `APIInteractionLog` its host hands it, the tab's own log, never the process-wide one Live's connector records on. The entry carries Live's fields: the exchange, the action (`FETCH_MARKETS`, `FETCH_TICKER` or `FETCH_OHLCV`, the words Live's connector records), the reason, the venue path as the endpoint, the params, the result, the response time, the level and the data usage. Each host now carries one signal, `apiEntryLogged`, and one receiver beside its `_on_api_event`, `_cross_api_event`, the crossing the main window carries for Live; the host registers the receiver as the log's listener, so an entry recorded on a worker thread lands in `_on_api_event` on the GUI thread through Qt's queued connection and draws as Live's block. The Live tab's pane is not written: the adapter never records on `get_api_log`.

`src/gui/paper/paper_trading_tab.py` — the crossing

```
    def _cross_api_event(self, entry: dict) -> None:
        """The ``api_log()`` listener: emit ``apiEntryLogged``, which Qt queues
        onto the GUI thread for ``_on_api_event`` from any other thread."""
        self.apiEntryLogged.emit(entry)
```

### Import Live Fleet reads the venue once

After Import Live Fleet copies the records and fires `fleet_changed`, each host starts one worker thread, `paper-feed-import`, which runs `read_fleet` over the adapter: one `products` read, then one `ticker` read per imported record. Each read lands on the pane as it completes. When the worker ends, its summary crosses a second signal, `feedRead`, and one line lands on the Activity Log naming how many products the venue trades, how many of the fleet's products are among them, how many tickers answered, and how many calls it took; a fleet product the venue does not trade is named on the line, and a product list the venue did not answer is said so, naming no product. The GUI thread never waits on a read.

`src/gui/paper/paper_trading_tab_surface.py` — the line

```
FEED_LINE_FORMAT = (
    "Feed: {products} product(s) trade on {exchange}; {traded} of {records} fleet "
    "product(s) among them; {answered} of {records} ticker(s) answered in {calls} call(s)."
)
FEED_NO_PRODUCTS_FORMAT = (
    "Feed: {exchange} answered no product list; {answered} of {records} ticker(s) "
    "answered in {calls} call(s)."
)
FEED_UNTRADED_FORMAT = " Not traded: {symbols}."
FEED_THREAD_NAME = "paper-feed-import"
```

### Every write name is refused

`PaperExchange.__getattribute__` raises `SendRefused` for every public name outside `READ_NAMES`, before any attribute is read, and `__getattr__` raises it for a name the class does not define. `place_order`, `cancel_order`, `create_order` and every other write name have no spelling on the adapter, and a method planted on the class without its name in `READ_NAMES` is refused too. No file under `src/paper/` or `src/gui/paper/` imports a credential loader, `ccxt`, `ScrummingBot`, `BotContainer` or `BotManager`.

`src/paper/paper_exchange.py` — the refusal

```
    def __getattribute__(self, name: str):
        """Answer a private name or one of ``READ_NAMES``; refuse the rest."""
        if not name.startswith("_") and name not in READ_NAMES:
            raise SendRefused(REFUSED_FORMAT.format(read_names=READ_NAMES, name=name))
        return object.__getattribute__(self, name)
```

### What the feed reading measured

Read off the real window in both builds over a scratch home holding a copy of the operator's `bot_state.json`, thirty-eight scrumming records on coinbase at 5m, every socket but loopback refused, and a loopback stand-in shaped from the venue's pages: the four endpoints with their fields, 400 for a granularity outside the nine names, empty candles for a listed product with `trading_disabled` set, 350 candles a call at most, a ticker carrying `best_bid` and `best_ask`, and a 429 on a plant. On the commit before this unit, Import Live Fleet put nothing on the pane in either build, the feed reader's ticker answered one price with no bid and no ask, and a `1w` ask sent `FIVE_MINUTE` to the stand-in.

```
reading                                          Qt                        React
blocks on the pane after Import Live Fleet       39                        39 entries, 234 lines
actions                                          1 FETCH_MARKETS,          1 FETCH_MARKETS,
                                                 38 FETCH_TICKER           38 FETCH_TICKER
thread the reads ran on                          paper-feed-import         paper-feed-import
thread violations written                        0                         0
least gap between two stamps                     0.112 s                   0.112 s
gaps under 0.1 s                                 0                         0
the Activity Log line                            38 of 38 tickers,         38 of 38 tickers,
                                                 39 calls                  39 calls
a ticker asked twice within 5 s                  two asks, one call        two asks, one call
a 1w window against its daily rows               equal, Monday buckets     equal, Monday buckets
a 12h ask                                        HTTP 400 error block      HTTP 400 error block
a listed-not-traded product                      [] and a No data block    [] and a No data block
a planted 429                                    one wait of 1000 ms,      one wait of 1000 ms,
                                                 then answered             then answered
place_order, cancel_order, create_order          SendRefused               SendRefused
a method planted without its name                SendRefused               SendRefused
a method planted with its name in READ_NAMES     answered                  answered
bot_state.json hash across the press             unchanged                 unchanged
a planted byte on a copy                         hash moved                hash moved
sockets opened outside loopback                  0                         0
```

The stand-in serves the `4h` name and a `USDC-USD` ticker with trades because the venue's pages list them; whether the real venue answers `FOUR_HOUR` and whether `USDC-USD` carries trades are read at the real venue, not here.

One real reading followed, on both built bundles against the public host, no credential file present and the adapter's own pace: Import Live Fleet made 39 calls in each build, the product list answering 917 trading products and 8 not trading, all 38 fleet products among them, 38 of 38 tickers answered with their bid and ask, no 429, the least gap between two stamps 0.191 s in the Qt build and 0.168 s in the React build, the 39 calls spanning 11.4 s and 10.7 s. Launched in a Windows AppContainer with no network, the same press made 39 calls in each build, every one refused by the container and every one a block on the pane naming its cause, and the Activity Log line read that the venue answered no product list and 0 of 38 tickers.

One more real reading settled the ninth name against the source. Through the built bundle's own copy of `PaperExchange`, against the public host, no credential file present and the adapter's own pace, one candles ask per name in `GRANULARITY` on `A15-USD`, nine calls, ten bars each:

```
name             status  rows  bar spacing
ONE_MINUTE       200     10    60 s
FIVE_MINUTE      200     10    300 s
FIFTEEN_MINUTE   200     10    900 s
THIRTY_MINUTE    200     10    1,800 s
ONE_HOUR         200     10    3,600 s
TWO_HOUR         200     10    7,200 s
FOUR_HOUR        200     10    14,400 s
SIX_HOUR         200     10    21,600 s
ONE_DAY          200     10    86,400 s
```

The nine calls were 0.271 s apart at least and spanned 2.3 s; no 400 and no 429. The venue answers `FOUR_HOUR`, so the `4h` row stays. `src/exchange/timeframes.py` still says the venue ships eight

granularities and no 4h; that file is Live's and this unit does not touch it.

## 2026-09-20 - #19 - The paper bot ticks and fills on the clone

Unit Q3 of issue #19. Start Paper Run now starts one worker thread that ticks every running paper bot at Live's cadence over `PaperExchange`, evaluates the shipped gate chains on each worked tick, sizes every scrum and fold with the pure functions Live calls, fills a scrum at the book's best bid and a fold at the best ask with the venue's taker fee, and moves the paper ledger the header strip reads. The old hosts left the tree.

### The sentences this unit overtakes

Seven passages on this page describe the run as it was before this unit. Each is quoted here and kept above as it stands.

- "`src/gui/main_tabs/paper_trader_tab_surface.py` — the second row's two faces" (The two presses). The file is removed; the two faces are `run_buttons` in `src/gui/paper/paper_trading_tab_surface.py`.
- "A run ticks on the wall clock and asks the feed for one bot per fire, so a fleet of many bots never blocks the window on a single pass. One bar is shared across the open bots." and "`src/gui/paper_trader_tab.py` — the round robin" with its `advance_once` block (Real time). The runner ticks every running bot each 5 s on its own thread, and the window is never asked to wait.
- "The Qt window and the React page print one line from one view model, so the two hosts cannot report different money." with its `LEDGER_SEPARATOR` block (The fleet ledger). The four figures reach both hosts through `strip_aggregate` and the header strip, from one figures dict the runner posts.
- "`src/gui/paper_trader_tab.py` — the press that opens a run" with its `_tick_timer` block, and "`src/gui/react_paper_trader_tab.py` — the sheet the page carries" with `paper_trader_tab.css` (Start Paper Run redraws the pane, The style sheet the Paper page loads). Both files, the page module and the sheet are removed.
- "The two hosts the sections above describe, `src/gui/paper_trader_tab.py` and `src/gui/react_paper_trader_tab.py`, stay on disk and nothing loads them; a later unit removes them once a paper bot ticks on the clone." (2026-09-19). This is that unit.
- "`BUDGET_MULTIPLE` in `src/paper/fake_balance.py` still reads `2.0`; the unit that starts a paper run retires it, and no clone code reads it." (The budget rule today overtakes four sentences). Retired: `BUDGET_MULTIPLE`, `budget_usd`, `fleet_budget_usd` and the `budget_usd` field are gone.
- "A press on Start Paper Run writes one line saying no paper run is built; the run is a later unit's too." with `START_RUN_TEXT` (What the fork draws). The press starts the run below.

The `apply_scrum` and `apply_fold` the earlier sections describe sized a fill with their own arithmetic; the two below are the Simulator's, forked over `src/trading/scrumming/sizing.py`.

### Who runs a paper bot

`PaperRunner` in `src/paper/paper_run.py` is the forked shape of the Simulator's `_walk_bot`: one `threading.Thread` named `RUN_THREAD_NAME` (`paper-run`) per Start Paper Run. Its loop calls `advance` once, waits `TICK_INTERVAL_S` (5.0 s, the value `ScrummingBot.tick_interval` answers) through the one clock seam `WallClock.wait`, and reads the stop event before each pass. The tape the Simulator walks is replaced by the adapter's live reads.

`src/paper/paper_run.py` — the loop

```
    def _loop(self) -> None:
        try:
            while not self._stop.is_set():
                self.advance()
                if self._stop.is_set():
                    break
                self._clock.wait(self._stop, self._tick_interval_s)
```

Each pass reads every held record's state through the host's `_states` map, which the host rebuilds on the GUI thread at every `fleet_changed`. A record whose state reads `running` counts one tick; a `paused`, `stopped` or `idle` record is skipped that pass and picked up the pass after the command bar moves it. A running bot is worked every `tick_skip` ticks, the rule `ScrummingBot.tick` applies: `scrum_read_rate_min` minutes over the 5 s tick, a tenth of that in track or fire, and the first tick after a bot starts running always worked, as Live's init tick is. `PaperBot` in `src/paper/fleet_source.py` carries `scrum_read_rate_min` from the record's config, 5 when the config names none.

`src/paper/paper_run.py` — the skip

```
def tick_skip(scrum_read_rate_min: int, scrum_target_mode: Optional[str]) -> int:
    rate = max(0, int(scrum_read_rate_min or 0))
    if rate <= 0:
        return 1
    base = max(1, int((rate * 60) / max(TICK_INTERVAL_S, 0.1)))
    if str(scrum_target_mode or "") in TRACK_MODES:
        return max(1, base // 10)
    return base
```

A worked tick reads `ticker` and `candles` on the runner's thread, never on the GUI thread. The ticker is cached 5 s and the candle window one bar, the two windows `src/exchange/data_pool.py` gives Live, so a 5m bot's gates read a new window every 300 s while its fill price follows the book every 5 s.

### Who evaluates and who sizes

`tick` builds the `GateContext` through the Simulator's `tape_context` over the window's newest close, `bb_reading` and `VotingEngine.compute_all`, and evaluates it with `latch`, unchanged from the sections above. The higher-timeframe bias is None in this unit, so the two defer flags block nothing; unit Q5 widens it.

`apply_scrum` and `apply_fold` are the Simulator's two fills forked over a `FakeBalance`, and every figure comes from `src/trading/scrumming/sizing.py` by name: `scrum_units`, `priced_usd`, `estimated_fee_usd` and `sale_proceeds_usd` on a scrum; `fold_rebuy_factor`, `eligible_fold_tranches`, `cycle_growth_cap_usd`, `fold_cap_remaining_usd`, `plan_fold_consumption`, `fold_rate_taper`, `fold_spend_usd`, `fold_units`, `settle_fold_plan`, `plan_source_price` and `fold_surplus_usd` on a fold. A scrum sells from the highest-priced lot first and queues one fold tranche per lot sold from, in the dict shape Live's `_tick_execute_scrum` builds. A fold is planned under the cycle growth cap, as Live's `tick_phases` plans it, and the target's growth from the surplus is unit Q4's.

`src/paper/paper_run.py` — the scrum's size and proceeds

```
    units = scrum_units(float(delta), float(price), rule)
    ...
    notional = priced_usd(units, float(price))
    fee = estimated_fee_usd(notional, taker_fee_pct(bot))
    proceeds = sale_proceeds_usd(notional, fee)
```

The unit rule is read once per run through `run_rule`, which is `unit_rule(CLASS_CRYPTO, venue)`: `fractional` on coinbase. A venue with no cited rule starts nothing and writes `run_no_rule_line`.

### Who fills, at what price and fee

A scrum fills at the tick's `best_bid` and a fold at the tick's `best_ask`, both from the one `PaperExchange.ticker` read of that tick; never at `last`. A fold's tranche eligibility is read at the tick's `last`, where Live's `fold_tranches.py` reads it. The fee is `estimated_fee_usd` of the notional at `taker_fee_pct`: the record's `trading_fee_pct` when set, else `TAKER_FEE_PCT`, the venue's taker fee at the default tier, 0.6 percent, the rule the Simulator's `DEFAULT_TRADING_FEE_PCT` applies. A scrum's cash gains the proceeds net of the fee; a fold's cash loses the spend plus the fee. Each `PaperTrade` carries the `bid`, `ask` and `last` of its read and the inputs its size came from, `delta_usd` on a scrum, `eligible_usd` and `taper` on a fold, so the paper log can be replayed through the same functions. The fill's stamp is the clock's seconds.

`src/paper/paper_run.py` — the two fills at the book

```
    if armed["scrum_armed"]:
        filled = apply_scrum(bot, balance, bid, now_s, stamp, context.delta, rule)
    elif armed["fold_armed"]:
        filled = apply_fold(bot, balance, ask, last, now_s, stamp, rule)
```

### Who holds the money

`FakeBalance` in `src/paper/fake_balance.py` is the fork of the Simulator's `SimPosition`: `units`, `cash_usd`, `fold_tranches` as a list of Live's tranche dicts, `main_lots`, `target_usd`, `anchor_target_usd`, the growth cycle's figures and the trade counters, with `cost_basis_usd` over the lots and `mature_usd` through `mature_profit_usd`. It opens on the bot's first worked tick as the Simulator's `opening_position` opens: `fold_units` of the target at the read's `last` as one lot, plus `cash_usd` of the target, the bot's share of the unbounded budget.

`src/paper/paper_run.py` — the opening

```
def opening_balance(bot: PaperBot, price: float, rule: str) -> FakeBalance:
    target_usd = float(bot.target_usd or 0.0)
    units = fold_units(target_usd, float(price), rule) if price > 0.0 else 0.0
    lots = [{"units": units, "initial_buy_price": float(price)}] if units > 0 else []
    return FakeBalance(
        units=units,
        cash_usd=target_usd,
        opening_price=float(price),
        main_lots=lots,
        target_usd=target_usd,
        anchor_target_usd=target_usd,
    )
```

The ledger opens at the fleet total. `opening_ledger` sets Paper Spendable and Paper Locked to `fleet_target_usd` over every held record; on the operator's 38 records that is $3,698.46. `PaperLedger.figures` keeps both figures whole: each open balance replaces its `anchor_target_usd` share of the opening with its `cash_usd` for Spendable and its `value_usd` for Locked, so a record the runner has not opened still counts its target. No fold is refused for cash; the spend is not capped by the wallet, the `FUNDED_BY_TARGETS` rule the Simulator's Validation runs under, and Spendable reads negative rather than refuse. Paper Realized Profits is the sum of `fold_surplus_usd` over the run's folds, added through `record_close` at each fold.

`src/paper/fake_balance.py` — the two figures over the open balances

```
        for bot_id, balance in held.items():
            price = float(at.get(bot_id, 0.0) or 0.0)
            spendable += balance.cash_usd - balance.anchor_target_usd
            locked += balance.value_usd(price) - balance.anchor_target_usd
```

### What crosses to the GUI thread

Both hosts, `PaperTradingTab` in `src/gui/paper/paper_trading_tab.py` and `PaperTradingTabReact` in `src/gui/paper/paper_react_trading_tab.py`, carry four queued signals in the Simulator's shape: `run_line` for each runner line, `run_trade` for each `PaperTrade`, `run_finished` for the run at the thread's end, and `run_figures` for the ledger's figures, which the runner computes on its thread after each worked pass and each fill and the host holds as `_figures`. `aggregate` lays that snapshot over `strip_aggregate`, so the header strip's four money cells read a dict the GUI thread owns and never the runner's balances. The strip repaints from it on the window's 2 s dashboard tick.

`src/gui/paper/paper_trading_tab.py` — the crossing

```
        self._runner = PaperRunner(
            run,
            self._exchange,
            self._read_states,
            on_trade=self.run_trade.emit,
            on_figures=self.run_figures.emit,
            on_finished=self.run_finished.emit,
            say=lambda line: self.run_line.emit(line, "info"),
        )
```

### Start Paper Run and Stop Paper Run

`_start_paper_run` is the Simulator's `_start_run` forked. A press with no run up opens `paper_run.start` over the held fleet under the run rule, starts the runner, writes `run_started_line` and turns the run button's face to `STOP_RUN_BUTTON`, "Stop Paper Run", on the corner and on the Get Started card. A fleet holding no record writes `RUN_NO_BOT_TEXT` and starts nothing. While the run is up the same seat's press reaches `_stop_paper_run`, which writes `RUN_STOPPING_TEXT` and sets the event the runner reads; the loop ends at the tick reached, `run_finished` crosses, and `_take_paper_run` writes `run_ended_line` and turns the button back to "Start Paper Run". The balances, the ledger and the trades stay readable on `run()` after the end. The Qt host relabels its four run buttons; the React host redraws the page through `show_tab` with `run_running` on `PaperTradingTabState`, and the page draws the seat's text and action from the payload as it did before.

`src/gui/paper/paper_trading_tab_surface.py` — the seat's two faces

```
def run_buttons(running: bool = False) -> tuple:
    if not running:
        return CORNER_BUTTONS
    return (CORNER_BUTTONS[0], STOP_RUN_BUTTON)
```

The command bar is unchanged: Start, Pause and Stop move one record's state through `PaperBotManager`, and the runner reads that state each pass. Start Paper Run starts the runner whatever the records read, so a bot started on the bar during a run ticks from the next pass.

### The old hosts are gone

`src/gui/paper_trader_tab.py`, `src/gui/react_paper_trader_tab.py`, `src/gui/web/paper_trader_tab.js`, `src/gui/web/paper_trader_tab.css` and `src/gui/main_tabs/paper_trader_tab_surface.py` are removed. The six names the clone read off the old surface, `HEADING`, `IMPORT_LIVE_FLEET_ACTION`, `IMPORT_LIVE_FLEET_TEXT`, `START_RUN_ACTION`, `START_RUN_TEXT` and `button_name`, live in `src/gui/paper/paper_trading_tab_surface.py` with `STOP_RUN_ACTION` and `STOP_RUN_TEXT`. `src/gui/main_tabs/main_window_surface.py` reads the tab's name and method there, `src/gui/main_tabs/paper_trader_tab.py` reads the name there, and `src/core/desktop_bridge.py` serves the clone's `view_model` under `paper_trading.tab` for the Electron shell, one call holding no host. `tools/sync_renderer_modules.py` rewrote `desktop/renderer/module_manifest.js`, which now lists the seven paper page modules and no old page.

`src/gui/main_tabs/main_window_surface.py` — the tab's name

```
from ..paper import paper_trading_tab_surface
...
PAPER_TAB = paper_trading_tab_surface.HEADING
```

### The paper log row gains the book and the fill

`paper_row` in `src/paper/paper_log.py` carries a `book` block, the `last`, `bid` and `ask` of the tick's read, and a `fill` block, every field of the `PaperTrade`, beside the gate half and the trade half. The log stays the only file a run writes, under `PAPER_ROOT`. Unit Q6 rebuilds the log on the bus.

### What the run reading measured

Read off the real window in both builds, under `pdb`, over a scratch home holding a copy of the operator's `bot_state.json`, thirty-eight scrumming records on coinbase at 5m, every socket but loopback refused, and Q2's loopback stand-in extended with a tape: every product's 5m candles follow one recorded shape whose newest bar advances one bar per stand-in minute from the first candles read, the ticker's last is that bar's close with the bid 0.05 percent under it and the ask 0.05 percent over it, and a 15 s delay can be planted on one ticker answer. The two records started on the bar read `scrumming_interval_pct` 5.0 and `scrum_read_rate_min` 1, so the tape rose about eight percent over five bars in a zig-zag and fell about fourteen over the four after. On the commit before this unit, in both builds, Start Paper Run wrote *"Start Paper Run: no paper run is built; nothing started."*, no thread started, `BUDGET_MULTIPLE` read 2.0, the fleet budget read $7,396.91, twice the target, and `apply_scrum` filled at the price it was handed with no bid and no ask in its signature.

```
reading                                          Qt                          React
Import Live Fleet, rows                          38                          38
Start on two rows through the command bar        both running                both running
thread after Start Paper Run                     paper-run                   paper-run
the started line                                 2 of 38 running,            2 of 38 running,
                                                 target $3,698.46            target $3,698.46
ledger at the press, Spendable and Locked        3,698.4572 each             3,698.4572 each
the run button while up                          Stop Paper Run, 4 seats     Stop Paper Run, 3 seats
ticks in the first minute, two bots              24 (one per bot per 5 s)    24
worked ticks in the first minute                 4 (one per bot per 60 s)    4
first scrum, A34/USDC, bar 51                    at 725.294446 = the bid;    the same
                                                 last 725.657275
first scrum, A32/USDC                           at 929.390286 = the bid     the same
scrum units against scrum_units on the           0.00241917, equal;          equal, equal
log row's delta, bid and rule                    0.00188792, equal
scrum fee against 0.6% of the notional           0.01052768, equal, twice    equal, twice
strip after the scrums                           $3,701.95 / $0.00 /         the same
                                                 $3,698.46 / $0.00 / 1
first fold, A34/USDC, bar 57                     at 654.668976 = the ask;    the same
                                                 last 654.341805
fold units against fold_units on the row's       0.00038187, equal;          equal, equal
usd, ask and rule                                0.00029801, equal
fold fee against 0.6% of the spend               0.0015, equal, twice        equal, twice
realized per fold, fold_surplus_usd              0.02434372, twice           the same
Realized on the ledger against the sum           0.04868745 = 0.04868745     the same
strip after the folds                            $3,701.44 / $0.05 /         the same
                                                 $3,694.04 / $0.00 / 1
fills at last                                    0 of 4                      0 of 4
a 15 s delay on one ticker answer                the answer waited 15.0 s;   15.0 s; 378 paints
                                                 803 paints over 40 s        over 40 s
the control, the GUI thread held 3 s             0 paints                    0 paints
thread-violation lines                           0                           0
Stop Paper Run                                   thread ended; 218 ticks,    the same
                                                 20 worked, 2 scrums,
                                                 2 folds, fees $0.0241,
                                                 realized $0.0487
the run button after the stop                    Start Paper Run             Start Paper Run
a planted zero-cash balance, a fold              filled $0.25; cash -0.2515  the same
a fill's price planted to last                   read as not at the book     the same
bot_state.json across the four presses           unchanged                   unchanged
a planted byte on a copy                         hash moved                  hash moved
stand-in calls over the run                      79: 1 products, 58 ticker,  79, 0.139 a second
                                                 20 candles; 0.14 a second
sockets opened outside loopback                  0                           0
```

The stand-in's tape holds fewer than one hundred bars, so the adapter's candle cache never held enough rows and every worked tick read a fresh window; on the real venue a 5m window holds one hundred bars and is served from the cache for 300 s. The scratch copy of `bot_state.json` was rewritten during the run by `main.py`'s `periodic_save`, the live application's own sixty-second state save under the scratch home; no file under `src/paper/` or `src/gui/paper/` writes it.

One line for another item: no element on the React page, Live's `src/gui/web/exchange_tab.js` or the forks, calls the page's own `selectScrumRow`, so a row press selects nothing for the command bar and only a Detail or Fire press selects the row on the model; the reading selected the row through the venue model's `select_scrum_row` and pressed the page's own Start.

### What the bundle reading measured

Both builds were made from this unit's commit and launched twice each, `HOME` and the profile variables on a scratch home holding a copy of the operator's `bot_state.json` and no credential file, each press an accessibility Invoke or a button message posted to the launched tree's own window, the window captured by its handle. In a Windows AppContainer with no network, Import Live Fleet made 39 calls in each build, every one refused by the container; a Detail press on two rows, the window it opened closed, and Start on the bar moved the two records to running; Start Paper Run started the runner and turned the seat to Stop Paper Run; over about 97 s the runner made 8 refused reads, 4 tickers and 4 candle windows, and wrote 4 paper rows each carrying *"The live feed answered no candle."*; the strip read $3,698.46 for Spendable and Locked; Stop Paper Run ended the runner and the seat read Start Paper Run again.

The one real reading, both bundles against the public host alone, no credential file, the adapter's own pace, the two same records running:

```
reading                                   Qt bundle                React bundle
Import Live Fleet                         39 calls, 917 products,  the same
                                          38 of 38 tickers

ticks over the run                        52 in 128 s              the same shape
worked ticks                              6                        6
venue calls during the run                8: 6 FETCH_TICKER,       8: 6 FETCH_TICKER,
                                          2 FETCH_OHLCV of 100     2 FETCH_OHLCV of 100
least gap between two calls               0.258 s                  0.257 s
calls a second over 122 s                 0.066                    0.066
the A34/USDC book on one read             last 0.221450,           last 0.221840,
                                          bid 0.221420,            bid 0.221760,
                                          ask 0.221430             ask 0.221840
paper rows written                        6, 3 per bot             6, 3 per bot
fills                                     0, none expected         0
strip while running                       $3,698.46 / $0.00 /      $3,698.46 / $0.00 /
                                          $3,698.52 / $0.00 / 1    $3,698.46 / $0.00 / 1
Stop Paper Run                            52 ticks, 6 worked,      the seat read Start
                                          the seat read Start      Paper Run again
thread-violation lines                    0                        0
```

The 100-bar window the venue answers is served from the adapter's cache for 300 s, so the second worked tick of each bot read a fresh ticker and no candles. Locked moved with the venue's last price on the Qt run, from $3,698.46 to $3,698.52, and read $3,698.46 on the React run, whose two prices sat within a cent of their openings. The Activity Log's lines are not readable through UI Automation's text pattern on the React page, so the bundle's own `system.log` and the paper log carry the React reading. ## 2026-09-23 - #19 - The paper bot acts like its live equivalent

Unit Q4 of issue #19. A paper run now writes what it does into the held record: a scrum's tranches reach the record and Detail's Fold Tranches tab, a fold's surplus compounds into the target under the cycle cap, the Target Delta reads the ticker's last trade instead of the newest bar's close, and the row's Trades, Current Position Value and Target move within two seconds of a fill. A bot reads `RUNNING` while the runner holds it and `STOPPED` after Stop Paper Run.

### The sentences unit Q4 overtakes

Three passages above describe the run as it was before this unit. Each is quoted here and kept above as it stands.

- "`tick` builds the `GateContext` through the Simulator's `tape_context` over the window's newest close" (Who evaluates and who sizes). `tick` now builds it through `paper_tape_context` in `src/paper/paper_run.py`, which prices the position at the ticker's last trade and leaves the candle window to the indicators, as Live's tick does.
- "A fold is planned under the cycle growth cap, as Live's `tick_phases` plans it, and the target's growth from the surplus is unit Q4's." (Who evaluates and who sizes). This is that unit: `apply_fold` calls `grow_target` after it settles the plan.
- "Q4 draws the tranches; this unit stores them" (What crosses to the GUI thread, and the Q3 comment on the issue). The tranches now reach the record on every snapshot, so Detail draws them mid-run.

### The Target Delta reads the ticker

The Simulator's `tape_context` in `src/simulator/back_test.py` prices a position at `window[-1].close`, the only price a stone tablet carries. Live's `ScrummingBot.tick` prices it at the ticker's last trade every five seconds and reads the candle window for the indicators alone. Paper has both, so the Simulator's function is left untouched and `src/paper/paper_run.py` carries its own `paper_tape_context`.

`paper_tape_context` takes the tick's last trade for three fields only: `ticker_last`, the `delta` against the balance's grown `target_usd`, and `mem253_current_pos`. Every other field is unchanged: `bb_pos` comes from `bb_reading` over the window, the indicators from `VotingEngine.compute_all`, and `trend_reading` from the window itself. The same shared helpers size the figures, so only the price source differs.

`src/paper/paper_run.py` — the delta at the ticker

```
last = float(ticker_last)
...
position_usd = balance.value_usd(last)
delta = target_delta_usd(position_usd, target_usd)
```

This matters on the real venue. `PaperExchange` caches a ticker for five seconds and a 5m candle window for three hundred, so before this unit the Target Delta could be five minutes old while the book moved. In one stand-in reading the gate priced a position at a bar close of 100.61803399 while the same tick's last trade read 101.83.

### A fold compounds its surplus into the target

`apply_fold` settles its plan, books the bought units, and then calls `grow_target`, the Simulator's own function in `src/simulator/back_test.py`. It is imported, not forked: the growth step does not differ between a simulated bot and a paper bot by one figure, and the operator's rule for this issue is that trading logic is shared as pure code and only stateful shells are forked.

`grow_target` reads `fold_surplus_usd` over the consumed slices, holds it to what `cycle_growth_cap_usd` and `fold_cap_remaining_usd` leave of the cycle, applies `target_growth_applied`, adds the applied figure to `target_usd` and to `cycle_cap_consumed_usd`, parks the rest as standing surplus, sets the growth side to `lower`, and records the step on `target_path`. A record with `profit_folding_active` off grows nothing. The cap is `max_target_growth_pct` of the target per cycle, one per cent on the operator's records.

`reset_growth_cycle` runs at the top of every worked tick, before the context is built, so the growth cycle and the gates read the same bar. It is `growth_cycle_side` over the bar's band position: a cycle opened on the lower side zeroes its consumed cap when the band position reaches 0.75, and one opened on the upper side when it reaches 0.25.

`src/paper/paper_run.py` — the growth step inside the fold

```
    realized = fold_surplus_usd(units, slices, float(price))
    grow_target(bot, balance, units, slices, float(price), int(candle_ts_ms))
```

`FakeBalance` in `src/paper/fake_balance.py` gains the two fields `grow_target` writes, `growth_applied_usd` and `target_path`. Every `PaperTrade` now carries `target_usd_after`, the balance's target once the growth has been taken.

### The run writes the record, and the row follows

`PaperRunner` gains one seam, `on_stats`, beside `on_trade` and `on_figures`. `post_stats` builds a `BotStatsSnapshot` through `stats_snapshot`, the Simulator's own builder, so a paper row and a simulated row carry the same keys. The snapshot goes out at three moments: once with `SNAPSHOT_START` and the state `running` on the tick a bot's balance opens, once per worked tick with `SNAPSHOT_TICK` or `SNAPSHOT_FILL`, and once per opened balance with `SNAPSHOT_END` and the state `stopped` when the loop ends.

Both hosts pass `on_stats=self.bot_stats.emit`, a fifth queued signal, so the write lands on the GUI thread in `_take_bot_stats`. That method calls `PaperFleetSource.write_stats`, which merges the snapshot's `stats` into the record's `stats` and its `scrumming_state` into the record's, then arms `_stats_redraw_timer`, a single-shot timer at `STATS_REDRAW_MS`, two thousand milliseconds, Live's own dashboard interval. A snapshot naming a record no longer held is dropped.

The row reads the record, so one write moves every cell that reads it: Current Position Value from `stats.position_value` and the lots, Trades from `stats.total_trades`, and Target from `scrumming_state.target_balance`, the grown target. The five cards on the header strip read the same records through `aggregate_stats`, so Trades, Scrummed and Folded move with the row.

`src/gui/paper/paper_trading_tab.py` — the record write

```
            self._fleet_source.write_stats(
                bot_id, snapshot.stats, snapshot.scrumming_state
            )
```

### RUNNING while the runner holds the bot

A snapshot carrying the state `running` moves the record through `PaperBotManager.start` and redraws the rows at once; one carrying `stopped` moves it through `PaperBotManager.stop` and redraws at once. Each writes two Activity Log lines: Live's own ✓ `Bot <id> RUNNING.` or `Bot <id> stopped.`, and then `bot_opened_line` or `bot_ended_line` naming the units, the price, the fills and the target's two figures. The command bar keeps its own Start, Pause and Stop, and `PaperFleetSource.set_state` stays the only writer of a record's state underneath.

### Detail draws the run's tranches

Detail's Fold Tranches tab is unchanged code. It draws `PaperBotView`, which reads `scrumming_state.fold_tranches` off the record, and the record now carries the tranches the run's scrums queued, so a Detail window opened during a run draws the open tranche. Live refreshes no open Bot Settings window on a timer and this adds none: the window draws what the record held when it opened.

### What the live-equivalent reading measured

Both variants, the real `MainWindow` under `pdb` on a scratch home, every socket but loopback refused, against a loopback stand-in serving 1m bars and a ticker that moves inside each bar. Two records, targets $100 and $200, a 0.5 per cent scrumming interval, phantoms off.

```
reading                                   Qt                        React
-------------------------------------------------------------------------------
fills in one 215 s run                    10                        10
row Trades at +1 s after a fill           3                         3
row Trades at +2 s after a fill           4                         4
row Target before the first fold          $100.0000 / $200.0000     the same
row Target after two folds                $100.0454 / $200.0910     $100.0470 /
                                                                    $200.0942
Current Position Value after a fold       $97.7949 / $195.5608      $97.7313 /
                                                                    $195.4367
growth steps recomputed by hand           4 of 4 equal to           4 of 4 equal
                                          10 decimal places
cycle cap on a $100 target                $1.00                     $1.00
Detail Fold Tranches opened mid-run       2 rows                    2 tranches
bar close held across two ticks           100.61803399              100.61803399
gate ticker_last across those two ticks   100.00780971 ->           100.00746202 ->
                                          101.90722988              101.88644039
delta across those two ticks              0.00000000 -> 0.15219235  0.00000000 ->
                                                                    0.10227326
cards: Trades, Scrummed, Folded           10, $5.1931, $5.1620      10, $5.2814,
                                                                    $5.2497
state while the runner held the bot       RUNNING                   RUNNING
state after Stop Paper Run                STOPPED                   STOPPED
venue calls, all loopback                 93                        93
non-loopback connections refused          0                         0
```

The same run before this unit filled ten times and moved nothing: Trades stayed `0`, Target stayed `$100.0000`, the Target Delta read the bar's close while the ticker stood 1.8 per cent away, Detail drew no tranche, the cards read zero, and both rows still read `running` after Stop.

Two plants, each of which made a reading fail. With the redraw interval planted to sixty seconds the row held its old cells at +1, +2, +3 and +4 seconds after a fill, and a forced `_redraw_stats` then moved Trades from 3 to 5, so the instrument would have seen the move. With `profit_folding_active` planted off, four folds realised $0.022 to $0.047 of surplus each and the target stayed exactly $100.00000000 and $200.00000000 with an empty `target_path`.

### What the bundle reading measured, against the venue

The Qt bundle, built from this branch and launched in isolation: a scratch home with no `bot_state.json` at launch and no credential file at any point, the window driven through the .NET UIAutomationClient assembly, every press an `InvokePattern` invoke or a `SelectionItemPattern` select on an element found under a window matched by its own process id. Two records on pairs the venue lists, `A15/USD` at a $100 target and `A14/USD` at $200, written to the scratch file after launch so no bot is ever restored.

```
moment                                    reading
-------------------------------------------------------------------------
Import Live Fleet                         915 product(s) trade on coinbase;
                                          2 of 2 fleet products among them;
                                          2 of 2 tickers answered in 3 calls
row after Import                          paperlive1 A15/USD, Trades 0,
                                          Target $100.0000
Start on the first row                    "✓ Bot paperlive1 RUNNING."
Start Paper Run                           the runner thread up, Bots card 1
Current Position Value at 52 s            $99.9882
                       72 s               $99.9868
                       92 s               $99.9628
                      112 s               $99.9428
                      132 s               $99.9645
                      152 s               $99.9623
Ammo across the same marks                $0.0118 to $0.0377
LOCKED on the header strip                $299.99 down to $299.94, back to $299.97
SPENDABLE / REALISED / MATURE             $300.00 / $0.00 / $0.00
cards: Scrummed, Folded, Trades, Errors   $0.00, $0.00, 0, 0
the A14 row's Target A15 cell             0.002371 then 0.002372
after Stop Paper Run                      $99.9694, Trades 0
bot_state.json across every press         575DED46ED9CB7B7, unchanged
the planted-byte control                  B7821D9854A0EB0F, a different digest
```

No fill was expected inside two minutes and none came; the position value is the live book moving under a fixed holding, which is what a two-minute reading shows.

The call ledger, read off the API Interaction Log pane, carries 28 calls over 150 seconds:

```
13:03:23  FETCH_MARKETS                     Import Live Fleet
13:03:23  FETCH_TICKER  A15/USD             the same press
13:03:23  FETCH_TICKER  A14/USD             the same press
13:03:54  FETCH_TICKER                      the run's first worked tick
13:03:55  FETCH_OHLCV   100 bars at 5m      the same tick, the only one
13:04:00  FETCH_TICKER                      then one every five seconds
   ...                                      to 13:05:53, 24 of them
```

One candle read serves the whole run, because the adapter holds a 5m window for three hundred seconds; the ticker is read once per five-second tick, the cadence `ScrummingBot.tick_interval` sets. That is 0.19 calls a second, and the adapter's own `PUBLIC_MIN_INTERVAL_S` paces them.

**Two controls could not be pressed, and the routes are named.** The command bar starts the first row and no other: `PaperBotStatusTable.get_selected_bot_id` reads both `selectedItems()` and `currentRow()`, and only the first row's cell answers `HasKeyboardFocus=True`. Four attempts on the

second row each reported `IsSelected=True HasKeyboardFocus=False`, and the Activity Log answered `Select a bot first.` or repeated the first bot's line. On the React bundle the Paper panel never reaches the accessibility tree at all: the Paper tab item answers `invoked` to an `InvokePattern` and `selected` to a `SelectionItemPattern`, and no child window contains the tab's own rect, so there is nothing to post a button message to. The window's descendant count stays at 225 through all three routes. Unit T1b of issue 34 read the same wall on the same bundle, and the React readings on this page stand on the source-tree React build.

## 2026-09-24 - #858 - the bot list upgrades reach the Paper Trader

His line closes the bot-list list on that item: "Transpose these upgrades to Sim and Paper once completed." Three upgrades are in it. Columns that sort. Labels that wrap, with the privacy button centred beneath each one. A row press that moves the Indicator Voting Panel to that bot. All three now draw on the Paper Trader's own Scrumming Bots table, in both builds, over the paper fleet's own figures.

### The paper columns sort over the paper fleet's own price

A press on a column heading orders the rows by that column. A second press on the same heading reverses the order. The order is recomputed on every rewrite, so a figure that keeps moving does not unsort the list. The Paper Trader orders by the row's own reading and never by the live price pool, because the table and the page model each answer their own lookups.

`src/gui/paper/paper_bot_status_table.py` - the paper fleet's own sort readings

```
        def _sort_lookups(self) -> SortLookups:
            """``order_statuses``' two readings, over the Paper Trader's figures."""

            def price(status) -> tuple:
                stats = status.get("stats", {}) or {}
                return self._display_price(status, stats)

            def denom_text(
                quote: str, base: str, exchange_id: str, target: float
            ) -> str:
                return self._denom_cell(quote, base, exchange_id, target)[0]

            return SortLookups(price, denom_text)
```

Read off the running program, both builds, at 700, 900 and 1400 pixels wide, over the 38 rows the paper fleet file holds:

| reading | window | page |
|---|---|---|
| columns that move on a press | 9 of 10 | 9 of 10 |
| a second press reverses | 7 of 9 exactly | yes |
| the order survives a fresh fleet | yes | yes |

The Detail column holds one identical button a row, so it carries no value to order by and is the one column a press does not sort.

### The paper labels wrap and the privacy dot sits beneath each one

Each of the ten labels sits inside its own column, over as many lines as it needs, with one blue dot centred under the nine that carry a mask. The label steps down one size when its longest word will not fit the column. A press on the dot masks that column; a press on the words sorts it.

| width | Current Position Value, window | the page | dots | largest offset |
|---|---|---|---|---|
| 700 | 3 lines in 67 px, size 10 | 46.2 px over a 15.4 px line | 9 | 0.5 px |
| 900 | 3 lines in 92 px, size 11 | 30.8 px over a 15.4 px line | 9 | 0.5 px |
| 1400 | 3 lines in 154 px, size 11 | 15.4 px over a 15.4 px line | 9 | 0.008 px |

No label is cut at any of the three widths, in either build. The tenth column draws no dot and keeps the same empty dot row, so every label sits on one line.

### A row press moves the Paper Trader's own Voting Panel

A press on a row draws that bot on the Paper Trader's panel. A second press on the same row takes the panel to its empty state. A press on another row moves the panel to that bot. The panel's own dropdown still picks a bot, and the row it names highlights itself in the list. The panel is the Paper Trader's own, never Live's.

`src/gui/paper/paper_trading_tab.py` - the one bot both sides read

```
        self._bot_list_link = BotListPanelLink(
            self._indicator_panel, self._bot_list_hosts
        )
        self._indicator_panel.bot_selected.connect(self._bot_list_link.panel_moved)
```

Read on the running program, both builds, at all three widths: a first press puts the pressed row's bot on the panel, a second press on that row answers no bot, and a press on another row answers that row's bot.

### The paper table is Live's table with three readings of its own

The Paper Trader's window table was a copy of Live's, taken before the header was rebuilt. It is now Live's table with the Paper Trader's three readings over it: the display price, the Target-denomination cell and the sort lookups. The page module carries the same relationship to Live's module, and the model already read the shared surface.

`src/gui/paper/paper_bot_status_table.py` - the row's own price

```
        def _display_price(self, status: dict, stats: dict) -> tuple:
            """The row's own ``current_price``, at ``PAPER_PRICE_AGE_S``."""
            del status
            return float(stats.get("current_price", 0.0) or 0.0), PAPER_PRICE_AGE_S
```

### An empty paper fleet still draws the header

With no row at all the header still draws its ten labels and its nine dots, a press on a heading changes nothing and raises nothing, and a press where a row would be finds no row.

| reading, no rows | window | page |
|---|---|---|
| labels drawn | 10 | 10 |

| reading, no rows | window | page |
|---|---|---|
| dots drawn | 9 | 9 |
| a heading press | no error, still no row | no error, still no row |
| a row press | answers no bot | finds no row |

## 2026-09-24 - #571 - the asset class group and Add Exchange reach the Paper Trader

### The class group is the window's, and the Paper tab draws it

**Functional.** The segmented asset class group is part of the header strip at the top of the window, not part of any one tab. It is on screen while the Paper tab is in front, and a press on a class button reaches the Paper tab like every other.

Read off the running window with the Paper tab in front, in both builds, at 700, 900 and 1400 pixels wide:

```
Crypto   Stock   Commodities   Forex
```

All four buttons sit inside the window at every one of those three widths. The group holds 142 pixels whatever the window gives it, and each button shortens its own name to the room it has, so a narrow window elides a label and never pushes a button off the right edge.

**One class for the whole window.** The Paper tab holds no second class of its own. The window keeps the active class, the settings file stores it for the next launch, and the window tells the Paper tab which class that is.

`src/gui/main_tabs/class_filter_tab.py` — the table that reaches Paper

```
SELF_COUNTING_TABS = {
    "History": "_history_tab",
    "Sim": "_simulator_tab",
    "Paper": "_paper_trader_tab",
}
```

### Paper's Add Exchange button follows the active class

**Functional.** The Paper tab now carries the Add Exchange button in the two seats Live gives it: the right of the venue sub-tab row, and the Get Started card. It reads the class the header strip has active, and it sits before Import Live Fleet and Start Paper Run in both seats.

`src/gui/paper/paper_trading_tab_surface.py` — the seat both builds read

```
def add_exchange_seat(asset_class: Any = None) -> dict:
    from ..main_tabs import asset_class_surface as acs

    key = acs.normalise(asset_class)
    return {
        "action": ADD_EXCHANGE_ACTION,
        "text": acs.add_exchange_label(key),
        "tooltip": acs.add_exchange_tooltip(key),
        "enabled": acs.add_exchange_enabled(key),
    }
```

Read for each class in turn, on the running program, in both builds:

| class | the button reads | it can act |
|---|---|---|
| Crypto | ■ `Add Crypto Exchange` | yes |
| Stock | ■ `Add Stock Broker` | yes |
| Commodities | `Commodities — no venue yet` | no |
| Forex | `Forex — no venue yet` | no |

**A class with no venue says so here too.** The button states the class, its tooltip names what is missing, and the press is refused. It never offers another class's venue list.

**Both builds draw the same button.** The words, the tooltip and whether the button can act come from one place, so the Qt widget and the page cannot drift. Read at 700, 900 and 1400, four classes, two seats: 24 readings a build, and every value the same on both sides.

### What the Add Exchange press does on the Paper tab

**Functional.** Paper never opens the live venue settings. A venue page exists on Paper once the paper fleet holds a bot on that venue, so the press offers the venues of the active class that the live fleet file names, and seats the one chosen.

`src/gui/paper/paper_trading_tab.py` — the handler

```
key = acs.normalise(self._asset_class)
if not acs.add_exchange_enabled(key):
    self._status_log.log(acs.class_state(key)["note"], "warning")
    return
self._import_live_fleet(key)
```

Driven once per class on the running program, with the live fleet file naming Coinbase and nothing else. The Activity Log line and the venue count after the press:

| class pressed | the Activity Log line | venues seated |
|---|---|---|
| Crypto | `Imported 38 bot(s) from bot_state.json on coinbase.` | 1 |
| Stock | `bot_state.json names no Stock venue to add.` | unchanged |
| Commodities | `No configured venue serves Commodities yet.` | unchanged |
| Forex | `No configured venue serves Forex yet.` | unchanged |

Both builds write the same four lines. No venue was contacted: every non-loopback connection was refused during the run, and the live fleet file hashed the same before and after.

### The Paper tab keeps its own card while no venue serves the class

**Functional.** The class filter replaces a tab with a note when the tab holds nothing for the class. The Paper tab is left its own card instead while it holds no venue of that class, so the Add Exchange button on that card stays on screen and names the state. That is the rule the Live tab already follows.

`src/gui/main_tabs/class_filter_tab.py` — the gate

```
tab = getattr(self, attr, None)
ask = getattr(tab, "class_venues", None)
if not callable(ask):
    return False
return not ask(key)
```

Read on the running window, before and after one Coinbase venue is seated:

| class | venues Paper holds, none seated | keeps its card | after Coinbase | keeps its card |
|---|---|---|---|---|
| Crypto | none | yes | Coinbase | no |
| Stock | none | yes | none | yes |
| Commodities | none | yes | none | yes |
| Forex | none | yes | none | yes |

Once the class has a venue on Paper, the note takes over again. A tab that does not answer which venues it holds is untouched by this: the History tab and the Simulator tab draw their notes exactly as before, read on the running window at all three widths under all four classes, 134 values compared and none of them moved but Paper's own.

### One venue list, read by every consumer

The nine broker ids that route a venue to the stock layer were written out four times. The Paper tab held the fourth copy; it now reads the one name the venue map declares, as the Live tab does.

`src/gui/paper/paper_trading_tab.py` — the venue ids

```
self._equity_exchange_ids = acs.EQUITY_VENUES
```

### Three sentences this entry overtakes

The manual carries these sentences about the Paper Trader's corner. Each is quoted as it stands, with the sentence that is true today beneath it.

> *"The corner Live gives* ■ `Add Crypto Exchange` *holds Import Live Fleet and Start Paper Run, each at Live's corner-button width."*

The corner holds three buttons: Add Exchange, Import Live Fleet and Start Paper Run, each at Live's corner-button width.

> *"The Get Started card holds the same two where Live's card holds its add*

*button."*

The Get Started card holds the same three, and Live's add button is among them.

> *"The corner holds Import Live Fleet and Start Paper Run in Live's corner-button chrome, and the Get Started card holds the same two at Live's card-button size."*

Both seats hold Add Exchange first, then those two, in the same chrome and at the same sizes.

## 2026-09-25 - #19 - a paper bot's settings reach its stored record

### Apply Changes writes the paper record

The Paper Trader's Bot Settings window drew every field a live bot's window draws, and Apply Changes refused each one. A paper bot could therefore only ever be a copy of a live bot, which left no way to prove a variant before real money touched it. Apply Changes now writes each pending field into the bot's own stored record, and the paper fleet file keeps it.

`src/paper/fleet_source.py` — the write

```
def set_config_field(self, bot_id: str, field: str, value: Any) -> Any:
    wanted = str(bot_id)
    record = self._records.get(wanted)
    if not isinstance(record, dict):
        raise KeyError(f"no paper record is held under {wanted!r}")
    name = str(field)
    if name in RECORD_FIELDS:
        record[RECORD_FIELDS[name]] = value
        return value
```

### A field the config does not declare is still refused

The write accepts the fifty-four fields the bot config declares, plus the three the record holds outside it — the phantom switch, its timeframes and the lock count. Any other name is refused, and the window says so on the same line it uses for an applied change.

`src/gui/paper/paper_bot_live_settings_surface.py` — the two lines

```
CHANGE_REFUSED_FORMAT = "Refused {count} change(s) — the Paper Trader sends nothing"
CHANGE_APPLIED_FORMAT = "Applied {count} change(s) to the paper record"
```

### What the driven window reported

Both readings came from the real window, with the venue's market read stood in for by a recorded answer. The target balance was changed from twenty-five dollars to seventy-seven dollars fifty, and the row, the record and the file all followed it.

```
no writer   Refused 1 change(s) — the Paper Trader sends nothing
            record 25.0   row 25.0
writer      Applied 1 change(s) to the paper record
            record 77.5   row 77.5   paper fleet file 77.5
React page  Applied 1 change(s)   bot rebuilt at 77.5
```

### Creating a paper bot is still the one way in

The venue page's new-bot button writes a line naming the way in, because no paper bot wizard is built. A variant is made by importing the live fleet and changing the imported bot's settings.

`src/gui/paper/paper_trading_tab_surface.py` — the line that button writes

```
NEW_BOT_FORMAT = (
    "+ New Bot on {exchange}: the Paper fleet is loaded through "
    "{way_in}; the paper bot wizard is not built."
)
```

**THE FORM THE WIZARD REPLACED.** The block above stays here, because a reader who saw that line needs to find it. That constant left the tree. The press now reaches a handler that opens the wizard and hands its settings to the paper fleet source.

`src/gui/paper/paper_trading_tab.py` — the handler that replaced it

```
        wizard_class = surface_class(PAPER_BOT_WIZARD)
        exchanges = wizard_surface.wizard_exchanges(self._exchange_tabs, self._exchange)
        defaults = wizard_surface.stored_defaults()
        markets = wizard_surface.venue_markets(self._exchange)
        wizard = wizard_class(exchanges, defaults, self, markets=markets)
```

### Four sentences this entry overtakes

Each is quoted as it stands, with the sentence that is true today beneath it.

> *"`src/gui/paper_trader_tab.py` draws them in Qt and*
>
> *`src/gui/web/paper_trader_tab.js` draws them in React, from one view model."*

**CITED AS ABSENT.** Neither file is in the tree. The Qt tab is `src/gui/paper/paper_trading_tab.py` and the React tab is `src/gui/paper/paper_react_trading_tab.py`.

> *"`src/gui/main_tabs/paper_trader_tab_surface.py` — the fields both hosts read"*

**CITED AS ABSENT.** That file is not in the tree either. The fields both hosts read are declared in `src/gui/paper/paper_trading_tab_surface.py`.

> *"The paper budget is twice the dollar target, so a bot has room to fold without*
>
> *the exercise ending on the first dip."*

The opening figures are the fleet's summed targets and the budget is unbounded, so a run never ends on a balance.

> *"`_apply_changes` asks the view for every pending field, and the view raises*

`SendRefused` *for each, so the pending line reads* `refused_text` *and nothing is written."*

Every pending field is written to the stored record; only a field the config does not declare still raises, and it is counted on its own.

## 2026-09-26 - #19 - a paper fill draws its own Activity Log line

A paper run held each fill and drew none of them. The pane beside the bot table carried the run's start and end and each bot's mark, and a reader watching a run fill ten times read nothing about any of the ten. A fill now draws its own line on the Paper Activity Log, on both hosts.

### A fill draws the moment it happens

The line takes the same trade shape the Live Activity Log uses, and it carries the market, the side, the size and the price. The size reads to six decimals and the price to eight, which is what Live's own trade line reads.

`src/gui/paper/paper_trading_tab_surface.py` — the line one fill writes

```
FILL_LINE_FORMAT = (
    "{prefix} {role}{split}{symbol}{split}{stage}"
    "{extra_split}{units:.6f} @ ${price:.8f}"
)
```

### The line carries the fill's own time

The stamp is the fill's own moment, never the moment the pane drew it. The runner stamps each fill on its own thread, and the line reads that stamp back instead of asking the clock.

`src/gui/paper/paper_trading_tab_surface.py` — the stamp

```
def fill_stamp(trade: Any) -> str:
    """The stamp a fill's Activity Log line carries: ``TIMESTAMP_FORMAT`` over the
    trade's ``wall_ms``, the fill's own time rather than the clock."""
```

### Each record names the side in its own words

One fill leaves two records, and each names the side in its own vocabulary. The pane line says SCRUM or FOLD. The stored row says SELL or BUY. Neither is a drift, and the reason is a different reader at each end. The pane line has to match Live's trade line, because the glyph table keys on Live's two role words. The stored row has to match the reader that reads a Coinbase fill back, because Validation reads a paper row through the reader it reads a live row through.

The paper log module records that reason itself. Its own docstring says the two sides come from the stored-trade module, and the paper log spells neither of them.

`src/paper/paper_log.py` — the two sides, in the stored reader's words

```
#: A scrum sells and a fold buys, in the vocabulary ``YtdTrade.side`` uses.
SIDE_FOR = {SCRUM: SIDE_SELL, FOLD: SIDE_BUY}
```

The pane's two role words come from the table the glyph reads, so a line's role and its glyph cannot disagree.

`src/gui/paper/paper_trading_tab_surface.py` — the pane's two role words

```
FILL_ROLES = {SCRUM: SIDE_ROLES[SELL_SIDE], FOLD: SIDE_ROLES[BUY_SIDE]}
```

### What the driven composer answered

One pass drove both functions over two paper trades, a scrum and a fold, with the two fill times five minutes and thirty-three seconds apart.

```
08:22:03 TRADE NOTIFICATION: SCRUM: A15: FILLED — 0.000153 @ $65432.10000000
08:27:36 TRADE NOTIFICATION: FOLD: A15: FILLED — 0.000154 @ $64980.55000000
```

The two stamps hold that gap although one pass drew them both. A stamp read off the clock would have put both lines on the same second, which is the reading that tells the two apart.

## 2026-09-26 - #19 - a paper bot is created from scratch

The Paper tab could hold a fleet only by copying the live one. Its new-bot button wrote one Activity Log line naming Import Live Fleet and opened nothing. That button now opens the Paper Trader's Bot Creation Wizard on both hosts, and the settings it collects reach `PaperFleetSource.create`, which had been in the tree with no caller at all.

The wizard's shape is the Simulator's, which already had one on both hosts against the same contract. One module holds Paper's own readings, and both hosts read it.

`src/gui/paper/paper_bot_wizard_surface.py` — the request the React fork asks

```
METHOD = "paper_bot_wizard.state"
```

### The press that opens the wizard

The route is the same on both hosts and starts on the tab bar. Press Paper, press the venue sub-tab, then press the button beside the bot tables. The Qt tab opens the wizard at once. The React tab opens it one turn later, because a web view opened inside another page's own callback never finishes loading.

`src/gui/paper/paper_exchange_tab.py` — the button the route starts at

```
self._add_bot_btn = QPushButton("+ New Bot")
```

### Each host draws the wizard from one registry line

One screen name carries both loaders, as every other forked screen does. The Qt loader answers a five-page window and the React loader answers the dialog that draws those pages in a browser.

`src/gui/variant_surface.py` — the screen and its two loaders

```
register(PAPER_BOT_WIZARD, _qt_paper_bot_wizard, _react_paper_bot_wizard)
```

### The pair list carries the venue's own last price

The venue's product list is what the asset page offers. A product record holds a last price and carries neither a volume nor a volatility, and the page orders the list by name rather than by size. Each row carries that price in the currency the pair prices in, because a pair against Bitcoin prices in Bitcoin and not in dollars. A price at one unit or more reads to two decimal places and a smaller one reads to eight with its trailing zeros cut. A row the venue priced at nothing carries no price at all.

`src/gui/paper/paper_bot_wizard_surface.py` — the label one row carries

```
def price_label(price: Any, quote: str) -> str:
    """The last-price tail an asset row carries, empty when ``price`` is not a
    finite number.

    A price of ``WHOLE_UNIT_PRICE`` or more prints to two places and a smaller
    one to ``SMALL_PRICE_PLACES``, with the trailing zeros cut; the quote is
    named because a row on ``A14/A15`` prices in A15, not in dollars.
    """
```

### The wizard's settings become the paper record

A finished wizard hands one settings map to the paper fleet source. The record built from it takes the shape a stored live record has, its state idle, and its own mark naming the wizard as its source rather than a copy. Every reader of that mark answers for a made bot as it answers for a copied one, and the command bar starts and stops it the same way.

`src/paper/fleet_source.py` — the call the wizard's Finish reaches

```
    def create(self, config: dict) -> PaperBot:
        """Hold one record built from the bot wizard's config through
        ``wizard_record`` and answer its ``PaperBot``, its ``bot_id`` the first
        ``BOT_ID_LENGTH`` characters of a ``uuid4`` as a live bot draws its own.
        """
```

### What the driven wizard answered

One pass drove each host at three pane widths, with the venue's product read stood in for by a recorded answer, and read the made bot back off the real fleet source beside a copied bot the same pass had loaded.

```
Qt      5 pages    A14/USD made, marked as made here
                   A23/USD held, marked as copied
React   8 of 8 page scripts, 5 page stops, 2 mode rows
                   A14/USD made, marked as made here
both    49 of 49 settings agree, none differs
labels  A15  (64,231.07 USD)   A14  (2,410.55 USD)   A18  (0.00004312 USD)
status  3 USD pairs, 3 carrying a last price
widths  700   900   1400   every control inside the pane
```

### Three sentences and one heading this entry overtakes

Each is quoted here and kept above as it stands, with the sentence that is true today beneath it.

> *"A press on `+ New Bot` writes one Activity Log line naming Import Live Fleet as the fleet's way in; the paper bot wizard is a later unit's."*

A press on that button opens the Bot Creation Wizard, and its Finish creates the bot.

> *"The venue page's new-bot button writes a line naming the way in, because no paper bot wizard is built."*

That button opens the wizard, and the Activity Log carries the opening line and then the created bot's own line.

> *"A variant is made by importing the live fleet and changing the imported bot's settings."*

A variant is made either way: by changing an imported bot's settings, or by the wizard on a pair no live bot trades.

> *"Creating a paper bot is still the one way in"*

Creating a paper bot is a way in of its own, standing beside Import Live Fleet.

### Two code blocks this entry retires

Both blocks name a constant the tree no longer holds. Each stays where it stands, marked as the form the wizard replaced, with today's code beneath it. The two sites are the corner-button passage and the settings-record entry above.

`src/gui/paper/paper_trading_tab_surface.py` — what neither block names any more

```
CORNER_BUTTONS = (
    (IMPORT_LIVE_FLEET_ACTION, IMPORT_LIVE_FLEET_TEXT),
    (START_RUN_ACTION, START_RUN_TEXT),
)
```

Back to the subsystem index.

# Proof of Accumulation

Reference. **Built.** React draws this tab inside the desktop window, and the tab row carries it under the name Accumulation. The window still builds neither the Competition tab nor the Local Testnet tab. The page halves its top row: the current Vessel on the left, the Map or the Encounter on the right. The party window takes the lower half, and it lists up to 120 party rows, 40 a page. Seven subtabs open over those zones — Maps, Character Details, Gear, Resources, Quint, Skills and Guild. Ctrl+1 to Ctrl+7 open them in that order. Maps opens only in an event that carries a map.

The page draws 35 buttons. A player picks a chain, Live or Demo TestNet, then picks one of eight event types. A bar of fifteen controls drives the engine from the screen: they write this node's identity, distil a trade fee into Quintessence, record a skill use, send Quint to another participant, cast a band, grade a trade, pay out a pot, close an event, draw loot, file a market exclusion, reset the chain and declare a world. The identity, the reset and the world each ask first and act on a second press. Every control prints what its mechanism did, or that

mechanism's own refusal. Until this node writes `bot_identity.json`, every other control refuses for want of a participant.

The page names what is absent. It draws no pixel art, so every class, loot tier and event mode shows a stand-in glyph. Nothing gives a Vessel assignments, lifeskilling, crafting, notifications, gear, equipping, destruction or permadeath, and the page lists all eight. Nothing holds a field for experience, level, character class, gear, enemy, threat or guild, so those seven metrics read nothing. A guild roster saves to a file and the Guild subtab reads it back, and no control founds a guild. Nothing advances the season. `src/competition/` is the Proof of Accumulation package, and issue #147 carries the rest of the build-out. The sections below describe that engine and the contract design behind it.

The design is an on-chain competition layer where bots compete publicly and the winners are awarded ACRV tokens on Base, which is Coinbase's L2. It evolved from a bot identity and an append-only trade log into Elo ratings, tournament brackets and an in-platform chain. Every Acervator bot has a cryptographic identity. During a competition each trade is signed and appended to a Merkle tree, and at the end the bot submits only the Merkle root — a 32-byte hash that commits to the whole trade history without revealing one trade of it. The strategy stays private, the proof is public, and the winner takes ACRV and an NFT trophy.

## The skeleton

The window builds this tab on every launch. `ProofOfAccumulationTabMixin` appends one panel to the tab bar and names it Accumulation. The panel holds a single web view, and React draws the whole screen inside that view. This screen has no Qt version, so nothing chooses between two frontends.

`src/gui/main_tabs/proof_of_accumulation_tab.py` — the method the window calls

```
    def _build_proof_of_accumulation_tab(self) -> None:
        """Append the Accumulation tab, and hold None when its page cannot be built."""
        try:
            from ..react_proof_of_accumulation_tab import ProofOfAccumulationReactPanel

            self._proof_of_accumulation_tab = ProofOfAccumulationReactPanel()
            self._main_tabs.addTab(self._proof_of_accumulation_tab, HEADING)
        except Exception as exc:  # noqa: BLE001 - a missing tab is not a crash
            logger.warning("Accumulation tab unavailable: %s", exc)
            self._proof_of_accumulation_tab = None
```

Built through that method, the tab reads Accumulation on the bar and its page loads. The page draws 35 buttons. The control bar holds 25 of them: two name the chain, eight name the event type, and fifteen fire a mechanism. Seven more open the seven subtabs.

One module serves that whole screen as data, and it alone declares the shell. It reads the Quintessence ledger, the token ledger and the loot store, so the wallet over the party window carries a balance and a list each for trophies, loot and Vessels.

`src/gui/main_tabs/proof_of_accumulation_tab_surface.py` — what the screen declares

```
METHOD = "proof_of_accumulation_tab.state"

HEADING = "Accumulation"
ISSUE = 147
BUILT = True
STATE_TEXT = (
    "The top row is halved: the current Vessel on the left, the Map or the "
    "Encounter on the right. The party window takes the lower half. No pixel art "
    "is drawn."
)
ISSUE_TEXT = f"Issue #{ISSUE} carries the build-out."
```

`src/gui/web/proof_of_accumulation_tab.js` is the renderer module that draws the zones. `src.core.desktop_bridge` registers the surface under the method name above, so the renderer asks the bridge for the shell by that one name.

## Identity

Each instance gets an Ed25519 keypair. The private key signs every trade in the log. The public key is the identity another party verifies against. Strategy parameters are never signed and never published, so authorship is provable while the method stays private.

`src/competition/bot_identity.py` — `BotIdentity`

```
class BotIdentity:
    """
    Manages a bot's Ed25519 keypair.  The private key never leaves this object
    unencrypted.  The public key is the bot's network-visible identity.
```

## The trade log

Each signed trade appends to a Merkle tree. Leaves are the SHA-256 of a trade's canonical bytes, the root is the commitment, and a proof lets a third party confirm one trade sits in the log without receiving the log.

`src/competition/merkle_log.py` — `verify_proof`

```
def verify_proof(leaf_hash: str, proof: List[dict], root: str) -> bool:
    """Verify a Merkle inclusion proof."""
    current = leaf_hash
    for step in proof:
        if step["position"] == "left":
            current = _node_hash(step["hash"], current)
        else:
            current = _node_hash(current, step["hash"])
    return current == root
```

`merkle_root` returns the commitment and `merkle_proof` builds the inclusion proof one trade at a time.

## The competition lifecycle

Four phases run in order.

`src/competition/competition_engine.py` — the module's own summary

```
    1. REGISTRATION  — bots register with capital commitment + config hash
    2. ACTIVE        — bots trade; each trade appended to their Merkle log
    3. SUBMISSION    — trading closes; bots submit Merkle root + performance claim
    4. ADJUDICATION  — arbiter verifies submissions, ranks bots, awards tokens
```

A competition runs either locally across instances on one machine and one price feed, or between two machines exchanging signed submissions.

## Tournaments

`EventVariant` in `src/competition/poa_modes.py` builds the event shapes. Each of the four modes pairs with one Elite flag, so there are eight event types and no mode is written twice.

| Mode | Label on screen | Shape |
|---|---|---|
| `monster_smash` | Fixation | One participant against low to midlevel creatures |
| `team_monster_smash` | Coagulation | A certified guild, up to 120 against 1 |
| `dungeon_crawl` | Descension | One participant, or a group of six, through a dungeon |
| `raid` | Cementation | A group of sixty, the most challenging and the most rewarding |

The code is what every other module keys on. The label is what the screen draws beside the Standard or Elite word, so a reader matches a row above to a button.

`src/competition/poa_modes.py` — the one table the Elite flag indexes

```
STANDARD_RULES = VariantRules(
    STANDARD_SUFFIX, STANDARD_LABEL, STANDARD_TIMEFRAME, 0, 0, 0, 0
)
ELITE_RULES = VariantRules(ELITE_SUFFIX, ELITE_LABEL, ELITE_TIMEFRAME, 1, -1, 1, 1)

VARIANT_RULES: dict[bool, VariantRules] = {False: STANDARD_RULES, True: ELITE_RULES}
```

A turn is one candle of the market clock, and `turn_at` names the turn that clock is in. `ImpetusPool` holds the points one turn grants and loses what that turn does not spend.

## The token

The ledger is append-only. Ask the class for its public names and ten methods answer. Its own docstring lists four of them, and one of those four names nothing: no method answers to `remaining_supply`. The method that reports what is still mintable is `remaining_ever`.

`src/competition/token_ledger.py` — the ten methods `TokenLedger` declares

```
    def award(
    def balance(self, bot_id: str) -> int:
    def awards(self, bot_id: str) -> List[AwardRecord]:
    def total_minted(self) -> int:
    def remaining_ever(self) -> int:
    def season_minted(self, season: int) -> int:
    def leaderboard(self, top_n: int = 20) -> List[dict]:
    def supply_summary(self) -> dict:
    def save(self):
    def load(self) -> "TokenLedger":
```

The hard cap is ten million, and the ledger refuses an award that would pass it. Each season awards less than the one before, so later tokens are harder to earn.

`src/competition/season_schedule.py` — the supply constants

```
TOTAL_SUPPLY_CAP = 10_000_000  # Hard cap — immutable
GENESIS_SEASON = 1
INITIAL_REWARD = 500_000  # Season 1 reward pool
DECAY_FACTOR = 0.85  # Each season awards 85% of the prior season
MIN_SEASON_REWARD = 100  # Floor — never less than this per season
```

Awards are idempotent: settling the same result twice writes one record. Balances are replayed from the log, and no operation edits a balance.

Five tiers are awarded on rank within the field. Iterate the tier table and four of the five carry a lifetime cap on how many can ever exist. Harvest is the one that carries none. Each tier is named for a stage of the Corpus Hermeticum alchemical path, and pays a fixed number of tokens.

| Tier | Stage | Rank | ACRV paid | Ever minted, at most |
|---|---|---|---|---|
| Harvest | NIGREDO | top 50% | 10 | no cap |
| Gold Fold | ALBEDO | top 10% | 50 | 100,000 |
| Bear Slayer | CITRINITAS | top 25% in a verified bear market | 100 | 10,000 |
| Grand Accumulator | RUBEDO | top 1% across three consecutive seasons | 500 | 1,000 |
| Ekthelius | UNIO MYSTICA | perfect score across every metric | 10,000 | 21 |

The cap field is `max_ever`. It holds nothing for Harvest alone. The first two tiers show both cases.

`src/competition/season_schedule.py` — the uncapped tier, then the next one

```
    RarityTier(
        name="Harvest",
        emoji="■",
        description="Top 50% of competition field",
        rank_pct_max=0.50,
        condition="Win rate > 50% of competing bots",
        max_ever=None,
        base_value=10,
    ),
    RarityTier(
        name="Gold Fold",
        emoji="■",
        description="Top 10% of competition field",
        rank_pct_max=0.10,
        condition="Advantage/capital in top 10% of field",
        max_ever=100_000,
        base_value=50,
    ),
```

## The token contract

The ledger above is the platform's own record. On the chain the token is an ERC-20 called Acervator Token. Three of its properties are fixed when the contract is deployed and cannot be changed afterwards: the supply cap, the one address allowed to mint, and the absence of any way to burn.

`contracts/ACRV.sol` — the header, and the guard on minting

```
//   • MAX_SUPPLY  = 10,000,000 ACRV (10_000_000 * 10^18 wei)
//   • Tokens can never be burned — supply monotonically increases
//   • The registry address is set once at construction and cannot change

    modifier onlyRegistry() {
        require(msg.sender == registry, "ACRV: caller is not the registry");
```

| Property | Value |
|---|---|
| Standard | ERC-20, named Acervator Token |
| Chain | Base, chain id 8453 |
| Hard cap | 10,000,000 ACRV, held on the chain |
| Minting | the CompetitionRegistry contract only |
| Burning | never |

The contracts are not deployed. Two npm packages supply the libraries they build on, and the deploy script takes the network and the signing key. Its own instructions name the Base test network first.

`contracts/deploy.py` — how it is called

```
npm install @openzeppelin/contracts @chainlink/contracts
export ACERVATOR_PRIVATE_KEY=0x...
python deploy.py --network sepolia
```

`src/competition/season_schedule.py` — the last tier

```
RarityTier(
    name="Ekthelius",
    emoji="∞",
    description="Perfect score across all metrics, any season",
    rank_pct_max=0.001,
    condition="100% win rate + top Sharpe + max capital efficiency",
    max_ever=21,
    base_value=10_000,
),
```

## Quintessence

A certified trade distils the platform's second asset. Entering an event spends it, and nothing destroys it. It keeps its own ledger, apart from the token ledger above, because the two obey opposite rules: the token only ever moves outward into a balance, while Quintessence circulates.

`src/competition/quintessence_ledger.py` — the nine operations

```
def distil(self, address: str, fee_usd: object, trade_grade: object) -> Decimal:
def spend(self, address: str, amount: object, held_address: str) -> Decimal:
def transfer(self, sender, recipient, amount, skill_level) -> QuintessenceTransfer:
def payout(self, held_address: str, credits: dict) -> Decimal:
def respawn(self, address: str, amount: object) -> Decimal:
def embed_from_pleroma(self, amount: object) -> Decimal:
def embed_from_wallet(self, address, amount, embedded_amount) -> QuintessenceEmbed:
def release_from_embedded(self, address, amount, recovered_amount) -> QuintessenceRelease:
def release_all_to_pleroma(self, amount: object) -> Decimal:
```

Quintessence can be in exactly four places, and the four always add up to everything ever distilled. A wallet holds what a participant can spend. A held address holds what they have already spent, which rests there and funds later awards. The payout operation is what pays one out, and it moves the award from the held address into the winner's wallet. The pleroma holds what bled out of a transfer, and the ledger respawns that to other participants. The embedded bucket holds what a thing in the world carries in itself, drawn out of the pleroma and returned there when the thing is broken.

Diagram: flowchart, left to right

- certified exchange fee -> **wallet** *(distil)*
- wallet -> **held address** *(spend)*
- held address -> **another wallet** *(payout)*
- wallet -> **another wallet** *(transfer)*
- wallet -> **the pleroma** *(bleed)*

- the pleroma -> **another wallet** *(respawn)*
- the pleroma -> **embedded** *(embed)*
- wallet -> **embedded** *(embed)*
- embedded -> **another wallet** *(release)*
- embedded -> **the pleroma** *(release)*

Every operation checks that sum before it writes, and refuses the write when it does not balance.

`src/competition/quintessence_ledger.py` — the two conditions the check reads

```
is_balanced=delta == 0 and not negatives,
is_within_cap=self._total_ever_minted <= QUINTESSENCE_SUPPLY_CAP,
```

| Term | Value |
|---|---|
| Hard cap | 33,000,000 Quintessence, total ever distilled |
| Pre-ownership | none, for anyone |
| Distil rate | one Quintessence for one dollar of certified exchange fee |
| Grade curve | the trade's grade, zero to one, multiplies the award |
| Destruction | never |
| Transfer bleed | 8% at skill level one, falling to 4% at level ten |
| Resolution | one Quintessence divides into 100,000,000 minimum units of 0.00000001 |

The module holds the cap and the rate as constants, and the write path refuses a mint past the cap rather than reporting it afterwards.

`src/competition/quintessence_ledger.py` — the recorded numbers

```
QUINTESSENCE_SUPPLY_CAP = Decimal(33_000_000)
QUINTESSENCE_MINIMUM_UNIT = Decimal("0.00000001")
QUINTESSENCE_UNITS_PER_WHOLE = 100_000_000
QUINTESSENCE_PER_FEE_USD = Decimal(1)
BLEED_FRACTION_AT_LEVEL_1 = Decimal("0.08")
BLEED_FRACTION_AT_LEVEL_10 = Decimal("0.04")
```

### One whole Quintessence divides into a hundred million minimum units

The minimum unit is the smallest amount of Quintessence that can exist. A divine essence is potent at a minute amount, so one whole unit carries a hundred million places to hold power in, the way one bitcoin carries a hundred million satoshi.

Every amount a bucket receives is a whole number of minimum units. The amount is rounded down onto that figure as it is written, so no wallet, held address, pleroma or embedded balance can carry a fraction the currency cannot express.

`src/competition/quintessence_ledger.py` — the rounding and the refusal

```
def quantize_quintessence(amount: Decimal) -> Decimal:
    return amount.quantize(QUINTESSENCE_MINIMUM_UNIT, rounding=ROUND_DOWN)


def is_on_quintessence_grid(amount: Decimal) -> bool:
    return amount == quantize_quintessence(amount)
```

### The rounding leftover returns to the pleroma, because nothing is destroyed

A bleed of eight per cent down to four per cent does not divide evenly at eight of the ten skill levels. The amount the recipient receives is rounded down and the bleed takes the rest, so the fraction that cannot be paid joins the pleroma rather than vanishing. That keeps the four buckets equal to everything ever distilled, to the unit.

`src/competition/quintessence_ledger.py` — the transfer split

```
received = quantize_quintessence(sent - sent * fraction)
bled = sent - received
```

A transfer too small for the recipient to receive one minimum unit is refused rather than paid as nothing, which is the refusal `contracts/Quintessence.sol` already makes.

| Skill level | Bleed on 100 Quintessence | Received |
|---|---|---|
| 1 | 8 | 92 |
| 2 | 7.55555556 | 92.44444444 |
| 3 | 7.11111112 | 92.88888888 |
| 4 | 6.66666667 | 93.33333333 |
| 5 | 6.22222223 | 93.77777777 |
| 6 | 5.77777778 | 94.22222222 |
| 7 | 5.33333334 | 94.66666666 |
| 8 | 4.88888889 | 95.11111111 |
| 9 | 4.44444445 | 95.55555555 |
| 10 | 4 | 96 |

The five stat requirements are whole Quintessence already — one stat measures 550 at level 100 and five of them measure 2,750 — so the resolution changes nothing about the stat scale.

Every launch builds the ledger and attaches it to the window beside the local chain, so a later panel finds it where it finds the chain. The same launch hands that ledger to the certification socket, so the socket and the panels move one ledger and not two.

`src/gui/shared_testnet.py` — the two lines that build it, and the socket that takes it

```
        ledger = cls.install_quint_ledger(quint_ledger_path)
        main_win._quint_ledger = ledger
        main_win._market_rotation = bridge.install_market_rotation(rotation_path)
        main_win._capture_bounds = bridge.install_capture_bounds(
            main_win._market_rotation, bounds_path
        )
        main_win._certification_socket = bridge.install_certification_socket(
            ledger, socket_path, main_win._capture_bounds
        )
```

The screen spends Quintessence. The distil control mints from a trade fee. The spend control then casts a band and rests its cost in the event pot. The wallet over the party window draws what the spend left. Fire the two in that order on the Demo TestNet chain and each prints what its mechanism did.

```
distil   Distilled 0.1 Quint.
spend    Band x1 cost 0.001 Quint, resting at poa_elite_event_pot.
wallet   Quint reads -- before the distil and 0.099 after the spend
```

A trade certifies through `CertificationSocket.certify`, which signs one fill, logs it, posts it and distils the venue's fee. It refuses a fill already certified.

`src/competition/certification_socket.py` — what the method contracts to do

```
    def certify(self, identity: BotIdentity, fill: CertifiedFill) -> CertificationReceipt:
        """Sign, log, post and distil one fill, and return its receipt.
```

The ledger loads whatever its file holds. On a chain that has distilled, the Quint subtab names the file as held and reports the figure minted on it.

```
Ledger file      quintessence_ledger_testnet.json - held
Minted so far    0.1
Smallest unit    0.00000001
Supply cap       33000000
```

The transfer duration, the transfer skill, the guild and the share of a pot are all drawn. The skills subtab states the duration and the bleed. The train control records one use and reports the level it reached. A Guild subtab opens on Ctrl+7 and reports the roster. The pot returns three quarters of itself to the participants.

```
duration      1000 Quint takes 100 hours at level 1 and 10 hours at level 10.
train         Quintessence Transfer stands at level 1 on 1.0 weighted uses.
standing      level 1 - effect 1.1x - bleed 8.00% - transfers sent 0
guild         Guild, Ctrl+7 - 0 guilds, 0 members, ranks officer and member
return pool   Return pool, 75%
```

`GuildRoster.save` writes a guild roster file and the subtab reads it back, so the roster loads empty only until something writes that file, and no control founds a guild. Nothing advances the season either: the counter lives behind a gated registry function on the chain and no module under `src` reaches it.

## Head to head

`challenge_protocol.py` carries the Elo ladder. A challenger sends a signed challenge, the target accepts or declines, both trade the agreed asset for the agreed duration, and the shared engine adjudicates.

`src/competition/challenge_protocol.py` — the ladder constants

```
ELO_K_FACTOR = 32
MIN_ELO = 100
```

The stake flows from loser to winner and both ratings move.

## Trophies

One SVG per tier, picked by name. An unknown tier raises rather than returning an empty drawing, and `generate_preview_html` lays the whole set out on one page.

`src/competition/trophy_generator.py` — `generate_trophy`

```
def generate_trophy(tier: str, data: TrophyData) -> str:
    fn = GENERATORS.get(tier)
    if not fn:
        raise ValueError(f"Unknown tier: {tier!r}")
    return fn(data)
```

Inside `harvest_svg`, a text path letters the epigraph's own instruction around the trophy ring, in its usual form: SOLVE ET COAGULA. Each drawing also letters its own alchemical stage across the face.

`src/competition/trophy_generator.py` — the stage each tier is lettered with

```
        "Harvest": "NIGREDO",
        "Gold Fold": "ALBEDO",
        "Bear Slayer": "CITRINITAS",
        "Grand Accumulator": "RUBEDO",
```

### Where a trophy's artwork is kept

The artwork and the metadata are held on the chain itself. The token's own metadata call returns the JSON inline, and the picture inside that JSON is the tier's drawing, also inline. Nothing points at IPFS, the file-sharing network most NFT projects park their pictures on, and nothing points at any other host, so a trophy lasts as long as the chain does.

`contracts/AcervatorTrophy.sol` — the metadata call

```
            '","image":"', svgB64,

            "data:application/json; base64,",
```

## The chain

`local_testnet.py` simulates the whole Base environment in memory, with no wallet, no A14 and no network. Ask the module for the classes it declares and seven answer. Four are the chain and its contracts. Three are the records those four store.

| Class | Holds |
|---|---|
| `LocalChain` | The blocks |
| `LocalACRV` | The ERC-20 balances and the mint history |
| `LocalRegistry` | Competitions, submissions and adjudications |
| `LocalTestnet` | The three above, plus a mock oracle and transaction receipts |
| `Block` | One block's number, hash, parent hash, timestamp and transactions |
| `TxRecord` | One transaction's hash, block, sender, function, arguments and gas |
| `ChainEvent` | One event a contract emitted, with its block and its arguments |

One call runs a whole competition on that chain and returns the result table. It registers the entrants, trades them, takes their submissions, adjudicates, awards the tokens and mints the trophy, with nothing written outside a temporary directory.

`src/competition/local_testnet.py` — a demo run

```
from src.competition.local_testnet import LocalTestnet

testnet = LocalTestnet()
result = testnet.run_demo_competition(n_bots=3, season=1)
print(result["results_table"])
```

The real targets sit ready for the day it deploys.

`src/competition/base_config.py` — the two chains

```
  Base Mainnet: chain_id=8453  — production
  Base Sepolia: chain_id=84532 — testnet (deploy here first)
```

The two ABIs sit beside them, and the four address fields are empty. Read the two configs and both the token address and the registry address answer an empty string on each network, because no deploy has put a contract on either. The ABIs are complete, so the calls are already described the day an address arrives.

`src/competition/base_config.py` — what the two configs answer

```
BASE_MAINNET   acrv_address = ''   registry_address = ''
BASE_SEPOLIA   acrv_address = ''   registry_address = ''
ACRV_ABI       12 entries
REGISTRY_ABI   16 entries
```

## The two shelved tabs

Both tab modules exist and the window builds neither. Their surfaces stay registered in the bridge, so the view models answer with no Qt tab in front of them. The Testnet tab is where a competition would be run and its blocks read in a block explorer.

The local chain is reachable without either tab. Pick the Demo TestNet chain on the Accumulation tab, press the identity control twice, then the distil control and the spend control. Each one writes a file under the runtime directory.

```
the identity control writes   bot_identity.json
the distil control writes     quintessence_ledger_testnet.json
the spend control writes      poa_record_store_testnet.json
```

Nothing draws a block explorer. The Quint subtab says so in its own words.

```
No block height and no merkle root is drawn. merkle_log writes the trade log
and no subtab reads its root.
```

`src/gui/main_tabs/retired_tabs.py` — `RetiredTabsMixin._install_retired_tab_sentinels`

```
self._competition_tab = None

self._testnet_tab = None
```

Issue #147 carries the initial build-out.

## 2026-09-08 08:17 - #147 - what the closed issues landed

The manual names this screen Proof of Accumulation (Anonymized Trading Tournaments Via Blockchain) and issue #147 carries its build-out.

This is currently proposed as a concept but will likely require the building of a supporting blockchain team for proper / full implementation. This system is designed to enable users of Acervator to compete against each anonymously via our own Proof of Accumulation blockchain. The idea is to convert trades executed into videogame metrics such as damage to a coliseum style monster or a fellow trader in a 1v1 face off. This further positions the platform as a surgical tool that can be finely tuned and customized to produce intense competition scenarios between entire groups of traders. This, of course, opens the door for actual tokenized Trading Guilds who may require their members to have a certain number of PoA tokens under their belt to join. There will be much more to follow on this as I do intend to scaffold it out for internal testing.

The tab is now called Accumulation. It sits seventh on the bar, on the gold ground with red text.

The screen is not built. The window builds neither the Competition tab nor the Local Testnet tab. The tab row now carries a Proof of Accumulation skeleton, which draws its name, one sentence saying it is not built, and the issue that owns it. Issue #147 carries the build-out. The engine behind it runs today, and the rest of this section is that engine.

`src/gui/main_tabs/proof_of_accumulation_tab_surface.py` — the whole empty state

```
HEADING = "Accumulation"
ISSUE = 147
BUILT = False
STATE_TEXT = "This tab is not built."
ISSUE_TEXT = f"Issue #{ISSUE} carries the build-out."
```

`src/competition/` is the Proof of Accumulation package. Each bot signs every trade with an Ed25519 key and signs no strategy parameter, so authorship is provable while the method stays private.

`src/competition/bot_identity.py` — `BotIdentity`

```
class BotIdentity:
    """
    Manages a bot's Ed25519 keypair.  The private key never leaves this object
    unencrypted.  The public key is the bot's network-visible identity.
```

The signed trades commit to a Merkle root a third party can verify one trade against without receiving the log. The competition itself runs four phases.

`src/competition/competition_engine.py` — the module's own summary

```
  1. REGISTRATION  — bots register with capital commitment + config hash
  2. ACTIVE        — bots trade; each trade appended to their Merkle log
  3. SUBMISSION    — trading closes; bots submit Merkle root + performance claim
  4. ADJUDICATION  — arbiter verifies submissions, ranks bots, awards tokens
```

The token ledger is append-only and idempotent, with five rarity tiers by rank. Its hard cap is ten million ACRV, and each season awards less than the one before it.

`src/competition/season_schedule.py` — the supply constants

```
TOTAL_SUPPLY_CAP = 10_000_000  # Hard cap — immutable
GENESIS_SEASON = 1
INITIAL_REWARD = 500_000  # season 1 pool, in ACRV tokens
DECAY_FACTOR = 0.85
MIN_SEASON_REWARD = 100
```

`EventVariant` in `src/competition/poa_modes.py` builds the four modes and the Elite variant of each. A local testnet module beside it simulates the whole Base environment in memory, with no wallet and no network.

## 2026-09-09 19:39 - #147 - the contract repairs

The three contracts could not be deployed. Each one needed another one's address at the moment it was built, and the deployment script handed the token the deployer's own wallet as its only minter. That address could never be corrected afterwards, so every award would have failed, and the wallet would have held the right to mint all ten million tokens for the life of the contract.

The token now takes nothing at all when it is built. One call afterwards names the competition registry as the minter, and a second call to that function is refused. The call also refuses any target that is not itself a contract, so a wallet can no longer be named the minter by mistake.

`contracts/ACRV.sol` — the one wiring call

```
    function setRegistry(address registryAddress) external onlyOwner {
        require(registry == address(0),          "ACRV: registry already set");
        require(registryAddress.code.length > 0, "ACRV: registry not a contract");
        registry = registryAddress;
        emit RegistrySet(registryAddress);
    }
```

Nothing now needs an address that does not yet exist. Four steps run in order, and the two remaining cross-references stay fixed at build time as before.

Diagram: flowchart, top to bottom

- 1. ACRV( -> **2. CompetitionRegistry(token, feeds**
- 2. CompetitionRegistry(token, feeds -> **3. ACRV.setRegistry(registry**
- 3. ACRV.setRegistry(registry -> **4. AcervatorTrophy(registry**

The other published route was to compute an address before the contract exists, and it does not solve this. A computed address commits to the values handed to the constructor, so computing the registry's address still needs the token's address first, and the circle closes again. The Solidity documentation states what goes into the computation.

```
address   = keccak256(0xff ++ deployer ++ salt ++ keccak256(init_code))[12:]
init_code = creation bytecode ++ the encoded constructor arguments
```

A factory contract could break the circle instead, by deploying both in one transaction and deriving the second address from its own count of deployments. That adds a fourth contract to the critical path, and that contract would hold the right to create the token. One locked field is the smaller change.

The award function used to pay the winner and then write its own records. A token contract that called back during the payment would have found the competition still marked open. Every record and both log entries now land first, and the payment is the last statement in the function.

`contracts/CompetitionRegistry.sol` — the end of `adjudicate`

```
        emit Adjudicated(compId, winnerWallet, tierName, tokenAmount, c.season);
        emit TierMinted(tierName, winnerWallet, tokenAmount, compId);

        // Interaction, last
        if (tokenAmount > 0) {
            require(acrv.remainingSupply() >= tokenAmount,
                    "Registry: insufficient ACRV supply remaining");
            acrv.mint(winnerWallet, tokenAmount, compId, tierName);
        }
```

A submitted result carried one unchecked number. A starting value of zero made the advantage calculation divide by zero, and a negative one reversed the ranking every award is built on. The function refuses both, and the conversion that squeezes the result into a small field now fails loudly instead of silently keeping the wrong digits.

`contracts/CompetitionRegistry.sol` — the guard and the conversion

```
        require(startValueCents > 0,    "Registry: start value <= 0");

        int256 advantage   = finalValueCents - startValueCents;
        int32  advBps      = SafeCast.toInt32((advantage * 10000) / startValueCents);
```

The price feed's answer was read and never checked. Five values come back from it and only two were kept. Every one is now checked, and a feed reporting no round, an unfinished round or a price of zero is refused rather than returned.

`contracts/CompetitionRegistry.sol` — `getLatestPrice`

```
        require(roundId != 0,                "Registry: oracle has no round");
        require(answeredInRound >= roundId, "Registry: oracle answer is stale");
        require(startedAt  != 0, "Registry: oracle round unstarted");
        require(answeredAt != 0, "Registry: round incomplete");
        require(answer      > 0, "Registry: price not positive");
```

Handing ownership of any of the three away used to take one call, with nothing asked of the receiver. All three now require the receiver to accept, so a mistyped address cannot take ownership and leave nobody able to act.

The deployment script no longer compiles anything. It reads what the build already produced, so the bytecode that would reach a chain is the bytecode the analyzers read.

`contracts/deploy.py` — what it loads

```
DEPLOY_ARTIFACTS = {
    "ACRV": "ACRV.sol/ACRV.json",
    "CompetitionRegistry": "CompetitionRegistry.sol/CompetitionRegistry.json",
    "AcervatorTrophy": "AcervatorTrophy.sol/AcervatorTrophy.json",
}
```

Four analyzers read the contracts before and after. The table is what each one reported, not a difference between runs.

| Analyzer | Measure | Before | After |
|---|---|---|---|
| forge build | compiler warnings | 0 | 0 |
| forge lint | findings, of which warnings | 67, 11 | 57, 2 |
| slither | medium and above | 2 | 0 |
| slither | all bands | 13 | 11 |
| solhint | errors | 37 | 34 |
| semgrep | security findings | 0 | 0 |
| semgrep | ownership transfer without acceptance | 3 | 0 |

Three findings stand, and each has a reason.

The season budget is still not checked on the chain. The registry records what a season has issued and reads that record nowhere. Checking it there would set a number deciding how many tokens a season may award, and the operator sets that number.

The trophy has no ceiling on any tier and no count of what it has minted. Unit 17 of issue #147 builds both.

Seven functions are named for reading the chain clock. No decision in any of them reads it: every comparison the analyzer lists is on a competition's status or on how many bots entered. The clock is stored as a record of when each step happened, and removing that record would remove the on-chain history the design is built on.

`contracts/CompetitionRegistry.sol` — what the analyzer points at in `adjudicate`

```
CompetitionRegistry.adjudicate uses timestamp for comparisons
■Dangerous comparisons:
■- require(c.status == CompStatus.SUBMISSION, "Registry: not in submission")
```

The conservation law the design names counts Quintessence and not this token: wallets plus held addresses plus the pleroma plus the embedded bucket equals the total ever distilled, at most 33,000,000. No Quintessence contract exists, so nothing on the chain can state that law yet, and the fuzzing the build tool offers has nothing to read. Unit 6 must carry the balances and the total as values a caller can read, or the law stays unprovable on-chain however the token behaves.

In development.

## 2026-09-09 20:10 - #147 - the two pieces a new surface would have made live

Two pieces of code were harmless only while nothing built them. The first added to a token balance without taking a lock first, so two threads adding at the same moment could both read the old figure and one award would vanish. The chain file asked for no thread protection at all.

Sixteen threads made 4,800 awards of one unit each against the chain the window builds at start-up. The token counted all 4,800 into its supply and gave the winner 3,709. The other 1,091 were gone, and nothing reported a fault. One lock now covers the supply cap check, the balance, the running total and the award record together, and the same run gives the winner all 4,800.

`src/competition/local_testnet.py` — the guarded award

```
        with self._supply_lock:
            if self._total_supply + amount_wei > MAX_SUPPLY_WEI:
                raise OverflowError("ACRV: mint would exceed MAX_SUPPLY")
            self._balances[recipient] = self._balances.get(recipient, 0) + amount_wei
            self._total_supply += amount_wei
```

Reading a balance or the total takes the same lock, so a reader can no longer catch the two halfway apart.

The second piece built a private chain of its own whenever nothing handed it the shared one. A tab built that way sat at block zero while the shared chain stood at block one, showed numbers, and raised nothing. The shared chain is now a required argument.

`src/gui/testnet_tab.py` — the required chain

```
        def __init__(self, parent=None, *, shared_testnet, bridge):
            """Read the chain through ``shared_testnet`` and ``bridge``; neither may be None."""
            super().__init__(parent)
            self.setAccessibleName("Testnet Tab")
            if shared_testnet is None or bridge is None:
                raise ValueError(
```

Asking for the tab with nothing attached now stops on the call itself.

```
TypeError: TestnetTab.__init__() missing 2 required keyword-only arguments:
'shared_testnet' and 'bridge'
```

Handing it an empty chain on purpose stops as well, and the refusal names where a real one comes from.

```
ValueError: TestnetTab needs the process-wide chain: pass the shared_testnet
and bridge that SharedTestnetBridge.install_on attached to the MainWindow.
A private LocalTestnet would diverge from every other reader.
```

## 2026-09-09 20:43 - #147 - the Quintessence contract

Quintessence now has a contract. It holds the same places the platform's own ledger holds, and it reports every one of them plus the running total as numbers anyone can read off the chain at any block.

`contracts/Quintessence.sol` — the five numbers a reader gets

```
    uint256 public walletsTotal;
    uint256 public heldTotal;
    uint256 public pleromaTotal;
    uint256 public embeddedTotal;
    uint256 public totalEverMinted;
```

The law is that the first four always add up to the fifth, and the fifth can never pass thirty-three million. The contract also answers both halves in one call, so a reader does not have to do the sum themselves.

`contracts/Quintessence.sol` — the single call that answers the law

```
        isBalanced = wallets + held + pleroma + embedded == everMinted;
        isWithinCap = everMinted <= cap;
```

| Term | Value |
|---|---|
| Smallest unit | one Quintessence divided into 10^18 parts |
| Hard cap | 33,000,000 Quintessence, as 33 followed by 24 zeros of those parts |
| Owner | none |
| Pause | none |
| Upgrade hook | none |
| Burn | none |

| Term | Value |
|---|---|
| Minting | the Proof-of-Accumulation registry only, named once when built |

The smallest unit matches the prize token, which splits each coin into the same number of parts. Quintessence needs the split because the award is a dollar of exchange fee multiplied by a trade grade between zero and one, and because a transfer loses a single-digit percentage. Whole numbers would round both of those away.

The contract has no owner and no pause. A wallet cannot move Quintessence on its own either, because moving it between players is a skill, not a wallet button. A transfer therefore takes two calls: the holder authorizes it, and the registry runs it once the waiting time has passed.

`contracts/Quintessence.sol` — the holder's half and the registry's half

```
function authorizeTransfer(address recipient, uint256 amount) external {
function executeTransfer(address sender, uint256 skillLevel) external onlyRegistry {
```

Neither side can act alone. The registry cannot move a wallet that authorized nothing, and a holder cannot move their own units without the registry. One authorization is held per holder at a time, which is what stops a large transfer being split into many fast ones.

The contract works out the loss and the waiting time itself, from the published rates, rather than taking either as a number it is handed. A transfer always loses at least four per cent and always takes at least one hour.

Diagram: flowchart, left to right

- certified exchange fee -> **wallet** *(distil)*
- wallet -> **held address** *(spend)*
- wallet -> **another wallet** *(authorize, then the registry runs it)*
- wallet -> **the pleroma** *(bleed)*
- the pleroma -> **another wallet** *(respawn)*
- the pleroma -> **embedded** *(embedFromPleroma)*
- wallet -> **embedded** *(embedFromWallet)*
- embedded -> **another wallet** *(releaseFromEmbedded)*
- embedded -> **the pleroma** *(releaseAllToPleroma)*

Nothing in the contract destroys a unit. A spend moves units to a held address where they rest, and that address is never drained, so a spend can never become a free transfer. The running total only ever rises.

The build tool's own fuzzing now holds ten properties over the contract, by throwing random sequences of calls at it. Each property was also broken on purpose once, to watch the tool report it, and then put back.

`tests/contracts/QuintessenceConservation.t.sol` — what the fuzzing reported

```
QuintessenceConservationTest invariants (runs: 256, calls: 16384, reverts: 8607)
[PASS] invariant_threeBucketsEqualTotalEverMinted
[PASS] invariant_walletsTotalEqualsSumOfBalances
[PASS] invariant_totalEverMintedWithinCap
[PASS] invariant_totalEverMintedNeverFalls
[PASS] invariant_noUnitIsDestroyed
[PASS] invariant_capRefusesAMintPastIt
[PASS] invariant_onlyRegistryMovesUnits
[PASS] invariant_registryCannotMoveAnUnauthorizedWallet
[PASS] invariant_transferRefusedBeforeItsDurationElapsed
[PASS] invariant_everyTransferBleedsFourPercentOrMore
```

A mint of one unit more than the cap is refused, and the tool printed the refusal itself while nothing had been minted at all.

`contracts/Quintessence.sol` — the refusal, quoted from the run

```
    ■   ■■ ← [Return] 3300000000000000000000000 [3.3e25]
    ■■ Quintessence::distil(QuintessenceActor, 3300000000000000000000001 [3.3e25])
    ■   ■■ ← [Revert] Quint: supply cap reached
```

Nothing is deployed. No transaction was sent and no network was reached, so the four deployment steps themselves are unproven. The contract is also not yet wired to anything: no registry contract exists to name as its minter, and the platform's own ledger and the contract do not yet read each other.

Three analyzers ran over the new file and none reported a security finding. The fourth, the one that walks the compiled bytecode, is still missing from this machine for the reason the earlier audit records.

| Tool | Result on the new contract |
|---|---|
| the build tool's linter | no errors, one warning about reading the chain clock |
| the static analyzer | one medium finding, on an exact comparison against zero |
| the style linter | 86 warnings, no errors |
| the pattern scanner | 29 findings, none of them about security |

The exact comparison is on the contract's own marker for a waiting transfer, which holds either zero or a real amount and nothing else, so no third value exists for it to miss. The contract reads the chain clock to decide whether a transfer's waiting time has passed, and the shortest waiting time is one hour, far longer than the few seconds a block producer could shift.

No archetype covers Solidity. Each contract file reports zero analyzers and says plainly that it was not examined, so the four tools above are the whole coverage for this file.

## 2026-09-09 21:16 - #147 - the trophy tier supply caps

Four of the five trophy tiers now carry a lifetime ceiling, and the ceiling is held in the contract that mints the trophy. Harvest is the only tier left open, which is what the design asks for: a cap on every tier except the lowest.

| Tier | Trophies ever | Where the ceiling is held |
|---|---|---|
| Harvest | no limit | nowhere, by design |
| Gold Fold | 100,000 | the trophy contract |
| Bear Slayer | 10,000 | the trophy contract |
| Grand Accumulator | 1,000 | the trophy contract |
| Ekthelius | 21 | the trophy contract |

`contracts/AcervatorTrophy.sol` — the four ceilings

```
    uint256 public constant MAX_GOLD_FOLD         = 100_000;
    uint256 public constant MAX_BEAR_SLAYER       = 10_000;
    uint256 public constant MAX_GRAND_ACCUMULATOR = 1_000;
    uint256 public constant MAX_EKTHELIUS         = 21;
```

Only one of those numbers is new. Bear Slayer at 10,000, Grand Accumulator at 1,000 and Ekthelius at 21 were already written into the platform's own tier list. Gold Fold at 100,000 continues the same ten-fold step the other three make.

Before this, the two top ceilings were held in the competition registry, and the registry awards the prize token rather than the trophy. The registry could therefore refuse a twenty-second Ekthelius token award while the trophy contract minted a twenty-second Ekthelius NFT. Each contract now holds the ceiling for the thing it actually mints.

The check sits inside the mint call, so every caller meets it. The owner of the trophy contract is allowed to call mint, and the owner is now refused at a ceiling exactly as the registry is.

A tier name outside the five is also refused. The counter is kept per tier name, so a near-miss name such as a trailing space would otherwise start a fresh counter of its own and mint without limit.

`contracts/AcervatorTrophy.sol` — the refusal a caller reads at each ceiling

```
Trophy: Gold Fold supply of 100,000 exhausted
Trophy: Bear Slayer supply of 10,000 exhausted
Trophy: Grand Accumulator supply of 1,000 exhausted
Trophy: Ekthelius supply of 21 exhausted
Trophy: unknown tier
```

Each of those four refusals was watched. The build tool minted the last trophy a tier allows, then asked for one more and printed the refusal.

`tests/contracts/TrophyTierCaps.t.sol` — the last Ekthelius trophy, then the refusal

```
AcervatorTrophy::tierMinted("Ekthelius")  → 20
AcervatorTrophy::mint(..., "Ekthelius", ...)
  emit TrophyMinted(tokenId: 1, tier: "Ekthelius", ...)
AcervatorTrophy::tierMinted("Ekthelius")  → 21
AcervatorTrophy::mint(..., "Ekthelius", ...)
  ← [Revert] Trophy: Ekthelius supply of 21 exhausted
```

The build tool's own fuzzing holds five properties over the contract, by throwing random sequences of calls at it. Each property was also broken on purpose once, to watch the tool report it, and then put back.

`tests/contracts/TrophyTierCaps.t.sol` — what the fuzzing reported

```
TrophyTierCapsTest invariants (runs: 256, calls: 16384, reverts: 9687)
[PASS] invariant_noCappedTierExceedsItsMaximum
[PASS] invariant_everyCapRefusedAMintAtIt
[PASS] invariant_unknownTierNameIsRefused
[PASS] invariant_harvestIsNeverRefused
[PASS] invariant_everyMintIsCounted
```

The last one is the guard against a counter that is never raised. A ceiling read off a counter nothing increments would never be reached, and the tier would mint for ever while every other check stayed green.

Breaking each property on purpose found a sixth thing that needed covering. Every one of the five reads the ceiling out of the contract, so raising a ceiling to an absurd number left all five green. A separate check now holds the four numbers as written figures, and it goes red the moment one of them moves.

`tests/contracts/TrophyTierCaps.t.sol` — the four figures held as written numbers

```
        require(trophy.MAX_GOLD_FOLD() == 100_000, "MAX_GOLD_FOLD is not 100,000");
        require(trophy.MAX_BEAR_SLAYER() == 10_000, "MAX_BEAR_SLAYER is not 10,000");
        require(trophy.MAX_EKTHELIUS() == 21, "MAX_EKTHELIUS is not 21");
```

Each number is still written twice, once in the platform's own tier list and once in the contract. Nothing fails if those two copies disagree. No check reads both languages, and no archetype covers a contract file, so the pair is held by reading and not by a tool. A check that compares them is a rule for the harness, which is not this unit's to write.

`src/competition/season_schedule.py` — the platform's own copy of the same numbers

```
   Harvest            max_ever = None
   Gold Fold          max_ever = 100_000
   Bear Slayer        max_ever = 10_000
   Grand Accumulator  max_ever = 1_000
   Ekthelius          max_ever = 21
```

The season budget is unchanged and still decides nothing on the chain. It sets how many prize tokens a season may award, and that number is the operator's.

| Tool | Result on the trophy contract |
|---|---|
| the build tool | compiled from scratch, no errors |
| the build tool's linter | no errors |
| the static analyzer | no finding above informational |
| the style linter | no errors |
| the pattern scanner | no security finding |

No archetype covers Solidity. Each contract file reports zero analyzers and says plainly that it was not examined, so the four tools above are the whole coverage for the contract.

Nothing is deployed. No transaction was sent and no network was reached, so the ceilings are proved on a local chain the build tool runs in memory and not on Base.

Back to the subsystem index.

## 2026-09-09 21:37 - #147 - the tab shell and its three zones

The Accumulation tab is no longer three lines of text. It draws the arrangement the operator set out on 9 September: the player window on the left, the square enemy screen upper right, and the party window across the lower half. Nothing inside the zones is built, and each one says so on screen.

The player window and the enemy screen each draw a title and one placeholder sentence. The party window draws a header, eight groups of five empty slots, and its own placeholder sentence. Forty slots is one page of the hundred and twenty the largest event carries.

`src/gui/main_tabs/proof_of_accumulation_tab_surface.py` — the paging the party window draws

```
PARTY_CAPACITY = 120
PARTY_PER_PAGE = 40
PARTY_GROUP_SIZE = 5
PARTY_PAGE = 1
```

The Quintessence balance sits in the party window's header and stays on screen whichever zone is drawing. The full wallet opens as a panel over the party window and over nothing else, carrying one section each for Quintessence, trophies and loot. No balance is read: the panel says so rather than showing a number nothing produced.

```
header, always on screen   Quint  --   Open wallet   Page 1 of 3 - 40 a page
wallet closed              0 panels
wallet open                1 panel over the party window, 3 sections
the balance                "--", because no ledger is read yet
```

The whole screen is one payload from one bridge method. The chain that payload answers for is a field of it, so a demo run against the TestNet asks the same method with a different chain and reaches the same page. No second surface and no second module exist for the demo.

```
CHAIN_FIELD = "chain"
LIVE_CHAIN = "live"
DEMO_CHAIN = "testnet"
CHAINS: tuple[str, ...] = (LIVE_CHAIN, DEMO_CHAIN)
```

Neither shelved Qt class was restored and neither was built on. Sixteen screens are registered in the variant seam and none of them draws a competition, testnet or proof-of-accumulation surface, so this tab had no Qt original to match and no Qt picture was taken.

```
seam screens                                        16
PoA, competition or testnet entries among them      none
CompetitionTab and TestnetTab constructed           never; both sentinels are None
```

What is still not built is everything inside the zones: the pixel art, the character classes, the event modes and the turn structure. The wallet holds no Quintessence, no trophy and no loot, because there is no debit path and no participant identity to read one for.

## 2026-09-09 22:28 - #147 - the certified transaction socket

A bot now certifies each of its fills against the local chain, and every certified fill distils Quintessence from the fee the exchange charged. An event turns away a bot that has certified nothing. No figure a bot trades on changes: the socket reads a fee the platform already recorded, then writes to the chain and to the Quintessence ledger.

`src/competition/certification_socket.py` — what one certification writes

```
            leaf = log.append(record)
            distilled = (
                self._ledger.distil(participant, fee, grade)
                if award_reason == AWARDED
                else Decimal(0)
            )
            self._certified_fill_ids.setdefault(bot_id, set()).add(fill_id)
            self._lifetime_fee_usd[bot_id] = (
                self._lifetime_fee_usd.get(bot_id, Decimal(0)) + fee
            )
```

`participant` is the wallet the award credits. It is the node's wallet for a bot that feeds a participant node, and the bot's own synthesised address for a bot that feeds none.

Most of certification was already in the package. Signing belongs to the bot identity, the append-only log already refuses three kinds of bad record, and the chain already takes a transaction and an event. The socket is what joins those parts to the fee and to the ledger.

| Part of certification | Where it comes from |
|---|---|
| Signing one fill | `BotIdentity.sign_trade` |
| Refusing a wrong competition, a wrong bot, a bad signature | `MerkleTradeLog.append` |
| The commitment over every certified fill | `MerkleTradeLog.root` |
| Proof of one fill without the log | `MerkleTradeLog.proof_for` |
| A summary revealing no fill | `MerkleTradeLog.submission_summary` |
| The transaction and the event on chain | `LocalChain.send_tx`, `LocalChain.emit` |
| Minting the award | `QuintessenceLedger.distil` |
| One chain and one ledger per launch | `SharedTestnetBridge.install_on` |
| A fill offered for certification | `CertifiedFill`, new |
| What one certification produced | `CertificationReceipt`, new |

| Part of certification | Where it comes from |
|---|---|
| The replay, stranger and cap refusals | `CertificationSocket.certify`, new |
| The lifetime fee total | `lifetime_certified_fee_usd`, new |
| The fill subscriber | `attach_to_bus`, new |

Nothing in the platform holds a lifetime fee total, so the socket holds its own. The exchange health refresh re-derives its figure from the last five hundred trades, so that figure falls as older fills age out, and an earned quantity may never fall. The socket's total takes every certified fee, takes the higher of the stored and the held figure on a reload, and rises rather than follows a windowed reading from the exchange.

`src/competition/certification_socket.py` — the windowed reading raises the total or leaves it alone

```
        observed = _as_fee_usd(observed_fee_usd)
        with self._state_lock:
            held = self._lifetime_fee_usd.get(bot_id, Decimal(0))
            if observed > held:
                self._lifetime_fee_usd[bot_id] = observed
                self.save()
                return observed
            return held
```

The total rises on both sides of the cycle. A scrum and a fold each carry the fee the venue reported for that fill, and the socket adds whichever arrives. A fill whose fee is zero certifies and distils nothing.

```
one scrum, fee $4.65   distilled 4.650   lifetime total $4.65
one fold,  fee $4.58   distilled 4.580   lifetime total $9.23
a window reading $500  distilled 0       lifetime total $500.00
a window reading $12   distilled 0       lifetime total $500.00
```

The socket refuses four things. Taking away a guard makes the socket admit whatever that guard stops, and a run of each one showed exactly that.

| Refused | Without its guard |
|---|---|
| A fill already certified | the fee total doubles, $4.00 to $8.00 |
| A bot that has certified nothing | it enters an event |
| A forged signature | 500 Quintessence mints for a bot that signed nothing |
| An award past the 33,000,000 cap | the log keeps a fill that never distilled |

After every mint the ledger's four places still add up to everything ever distilled. The socket reads that report back and carries it in the receipt, so a caller sees the sum rather than trusting it.

```
after the scrum    wallets 4.650 + held 0 + pleroma 0 = 4.650 ever minted
after the fold     wallets 9.230 + held 0 + pleroma 0 = 9.230 ever minted
after a refusal    wallets 9.230 + held 0 + pleroma 0 = 9.230 ever minted
```

Every launch builds the socket beside the chain and the ledger, in the same call that builds those two. It shares the bridge's lock, so a certification and a competition run never write the chain at the same moment.

`src/gui/shared_testnet.py` — the line that builds it

```
        main_win._certification_socket = bridge.install_certification_socket(
            ledger, socket_path
        )
```

Demo mode is the same socket over a second chain. The chain, the ledger and the competition name all arrive at construction, so a TestNet run is a second socket holding different ones, and no flag anywhere decides which path runs. A demo certification leaves the live ledger at the figure it already held.

```
live socket   chain from SharedTestnetBridge   competition POA-STANDING
demo socket   its own LocalTestnet             competition POA-DEMO
measured      demo minted 3.720; live stayed at 9.230
```

One thing a fill on the bus cannot yet supply is its fee. The fill notification carries the type, the side, the amount, the price, the dollars and the profit, and no fee. A fill arriving that way certifies, reaches the chain and distils nothing until the notification carries the number the venue charged.

```
trade.filled fields today   type, side, amount, price, usd, profit,
                            operator_initiated
the field the socket reads  fee_usd, absent from every emit site
the result                  the fill certifies, the award is zero
```

What is still not built is the grade curve that scales an award, the rotation that decides which markets pay, and the caps on how much one participant may take. The socket passes a grade of one and applies no ceiling beyond the supply cap, so those three remain open.

## 2026-09-09 22:47 - #147 - the Quintessence wallet

The wallet holds real state. The balance in the party window's header is the figure the Quintessence ledger computed for this node, and it stays on screen while the wallet is closed. Opening the wallet lays four holdings side by side across the party window: Quintessence, trophies, loot, Vessels.

Quintessence reads the ledger file belonging to the chain the tab is showing. Five figures, every one of them the ledger's own, none of them worked out on the screen. This is a run against a throwaway home holding two distillations.

```
Balance                17.25
Distilled, all time    17.25
Still mintable         32999982.75
Supply cap             33000000
Movements              2
quintessence_ledger.json
```

Trophies are the awards this participant has won, one row each, read from the ACRV award ledger. A row carries the tier's own emblem and name, the season, and the competition it was won in. The emblem is the one the tier declares, not a picture the screen chose.

```
■ Bear Slayer   Season 4 - comp-autumn-0002
■ Gold Fold     Season 3 - comp-autumn-0001
acrv_ledger.json
```

Loot is read from the loot store, one row an item. A row carries the tier's short form, the market that dropped it, the season, and the bonus that item gives one action. Two rows close the section: what the whole holding does to the Impetus an action costs, and the multiplier it puts on that action's effect.

```
Magisterium      AAVE/USD - Season 1 - Impetus -2, effect +50%
Elixir           AAVE/USD - Season 1 - Impetus -1, effect +20%
Flores           AAVE/USD - Season 1 - Impetus -1, effect +10%
Pavonis          A15/USD - Season 1 - effect +5%
Calx             A36/USD - Season 1 - effect +2%
Action Impetus   4 becomes 1
Action effect    1.87x
loot_store.json
```

A chain with no store file, a store holding nobody's loot, and a store that will not replay each print their own sentence in place of a row.

```
loot_store.json does not exist. No market has dropped loot on this chain.
loot_store.json records no loot for this participant.
loot_store.json could not be replayed: Expecting property name enclosed in
double quotes: line 1 column 2 (char 1)
```

The participant is this node's own competition identity. Its key file is read, never created, so a machine with no identity yet names none, the balance falls back to two dashes, and the trophy section says which file it was looking for.

`src/gui/main_tabs/proof_of_accumulation_tab_surface.py` — where each figure comes from

```
def participant_identity() -> BotIdentity | None:
    """This node's PoA identity from ``IDENTITY_NAME``, or None when unreadable."""
    try:
        return BotIdentity(str(LEDGER_DIR / IDENTITY_NAME)).load()
    except Exception:
        return None
```

The wallet displays and does nothing else. It carries no button that spends, no field that sends and no path that moves a balance. Spending belongs to unit 13 and transfer to the skill in unit 21, and until they land a wallet that could move value would be a defect rather than a feature.

Each holding falls back on its own. Corrupting the Quintessence ledger empties that section, prints the ledger's own refusal under it, and drops the header balance to two dashes, while the trophy rows stay on screen. Taking the identity away does the reverse.

```
ledger corrupted   Quintessence 0 rows, the refusal printed, trophies 2 rows
identity removed   Participant none, balance --, trophies 0 rows with a reason
both intact        Quintessence 5 rows, trophies 2 rows, balance 17.25
```

The demo chain reads its own book through the same bridge method. Asking for the TestNet chain returns a different balance from a differently named file, on the same page, with no second surface and no second module.

```
live      Balance 17.25   quintessence_ledger.json
testnet   Balance 50.00   quintessence_ledger_testnet.json
```

## 2026-09-09 22:57 - #147 - the project age rule

A market rewards Quintessence only while its project is at least six months old. The platform now answers that question for one market at a time. It reads the project's start date from CoinGecko, keeps that date on disk for good, and refuses any market whose age it cannot establish. No figure a bot trades on changes: the lookup reads a published date and writes only its own cache file.

`src/competition/project_age.py` — the whole decision

```
        meets_age_rule_on = ""
        if genesis is not None:
            admits_on = months_after(genesis, MIN_PROJECT_AGE_MONTHS)
            meets_age_rule_on = admits_on.isoformat()
            if self._today() < admits_on:
                reason = TOO_YOUNG
```

Six months means six calendar months, not a count of days. Bitcoin started on 3 January 2009, so the rule first admitted it on 3 July 2009. Both edges of that day were run.

```
A15 asked on 2009-07-02   too_young    rewards Quintessence: no
A15 asked on 2009-07-03   old_enough   rewards Quintessence: yes
```

### Where the date comes from

CoinGecko serves a project's start date on its coin detail endpoint, in a field the API reference documents as nullable. Nothing in the platform had ever called that endpoint. Two other CoinGecko endpoints were already in use, and the new call shares no code with either of them.

| What it asks for | The endpoint, and who calls it |
|---|---|
| The asset list and 24-hour volume | `/coins/markets`, from `market_data.py` |
| The candle series | `/coins/{id}/ohlc`, from `chart_data.py` |
| The project start date | `/coins/{id}`, from `project_age.py`, new |

The Charts tab's one-hour timeframe asks the candle endpoint for two days and gets an HTTP 400 back. That fault is recorded elsewhere and is unchanged here. The age call reads no days value and builds its own address, so the two requests share the host name and nothing else.

Most of this was already in the platform. The parts that are new are the rule itself, the refusals, and the file that keeps each date.

| Part of the lookup | Where it comes from |
|---|---|
| The CoinGecko identifier for an asset | `crypto_assets.ASSETS`, 40 of 40 filled |
| A request that refuses any address but http and https | `safe_url.SafeRequest` |
| Opening that request | `safe_url.safe_urlopen` |
| Writing the cache file without a torn write | `io_utils.atomic_write_json` |
| Runtime data outside the repository | the same home folder the Stone Tablets use |
| The six-month test | `months_after`, new |
| One market's answer | `ProjectAgeVerdict`, new |
| The four refusals | `ProjectAgeLookup.verdict_for`, new |

### The identifier map is what limits this, not the rate limit

The asset catalogue carries a CoinGecko identifier for forty base currencies. The eligibility rule reaches up to twenty markets on each of fifteen venues, so three hundred markets can ask and forty bases can answer.

```
bases with an identifier        40 of 40 in the asset catalogue
markets the rule can reach      up to 300
a market outside the forty      refused, reason no_coingecko_id
```

A market with no identifier is refused, and the refusal says which kind it is. Tron is a real project older than six months, and the lookup still turns it away because the catalogue carries no identifier for it.

```
TRX/USD   coingecko_id ''   rewards Quintessence: no   reason no_coingecko_id
```

Widening the map is the way to widen the rule. Either the asset catalogue grows, or a list call fetches every identifier CoinGecko carries. A ticker is not unique across projects, so the second route needs a tie-break where two projects share one symbol.

### The four refusals, and what each one looks like

Every answer carries a reason, so a refusal is never confused with a crash. A bad market string raises instead, and that is the only error path.

| Reason | When | Measured |
|---|---|---|
| `too_young` | the project is under six months old | A15 asked on 2009-03-01 |
| `no_coingecko_id` | the catalogue has no identifier | TRX/USD |
| `no_genesis_date` | CoinGecko answers with no date | SHIB/USD |
| `lookup_failed` | the call did not answer | A14/USD, host unreachable |

Taking away a guard makes the lookup admit whatever that guard stops, and a run of each one showed exactly that.

```
without the too_young guard         A15 at 2009-03-01 rewards Quintessence
without the no_coingecko_id guard   TRX/USD rewards Quintessence
without the no_genesis_date guard   SHIB/USD rewards Quintessence
without the lookup_failed guard     an unreachable host rewards Quintessence
```

### A missing date is the common case, not the corner case

Nine projects answered the endpoint. Two carried a date and seven carried none. Every one of those seven is refused.

```
with a date    bitcoin 2009-01-03, chainlink 2017-09-16
with none      uniswap, shiba-inu, pepe, aave, thorchain, the-sandbox,
               sei-network
```

Refusing an unknown age cannot be gamed by withholding data, which is why it is the answer recorded for this rule. The cost is now measured rather than guessed: most tokens in the catalogue carry no published start date, so the age half of the rule admits few markets today.

### The date is fetched once and never again

A project's start date is fixed on the day its chain starts, so the lookup keeps each one in a file under the runtime folder, with no expiry and no timer. A date already in that file is served without any network call at all.

```
~/.acervator/project_genesis_dates.json
{ "genesis_dates": { "bitcoin": "2009-01-03" }, "version": 1 }
```

Proved by taking the network away. A second run with the host unreachable still answered for Bitcoin from the file, and in the same run a market that was not in the file could not be answered at all.

```
A15/USD, host unreachable, in the file       old_enough
A14/USD, host unreachable, not in the file   lookup_failed
```

An empty answer is not written to the file. A project CoinGecko cannot date today may be dated tomorrow, and a stored blank would freeze that refusal for good.

### What is not built

Nothing in the running program asks a market for its age yet. The module loads on every launch, and the object that answers is built by whoever asks.

`src/gui/main_window.py:277` — the launch call that loads the package

```
                  SharedTestnetBridge.install_on(self)
```

The volume ranking, the market rotation and the per-market caps are the other half of eligibility and are not here. One measurement belongs to whoever builds the backfill: asked forty times at two seconds apart, the detail endpoint refused thirty-four of those calls with HTTP 429. The lookup itself makes one call per project and then never again, so the pacing belongs in the loop that walks a venue, not in the lookup.

Demo mode needs no second path. A start date is a fact about a project, not about a chain, so a TestNet run reads the same dates from the same file through the same call. The lookup holds no chain and no competition name, so there is nothing for a demo run to switch.

## 2026-09-09 23:41 - #147 - node linking between two instances

Two copies of Acervator on one machine now hold the same records on their two chains. Each copy is a node. A node says where it is listening, finds the others that said the same, trades records with them, and keeps every record either side had. This is the run, two processes, each with its own chain and its own books.

```
node 8f3fd656e34e starts with 1 chain records on node_a
node 8f3fd656e34e listening on 127.0.0.1:57019 (network=acervator-poa, records=1)
node b3b38412206d starts with 1 chain records on node_b
node b3b38412206d listening on 127.0.0.1:57040 (network=acervator-poa, records=1)
node b3b38412206d synced with 8f3fd656e34e: held 1, offered 2, took 1, now holds 2
node 8f3fd656e34e synced with b3b38412206d: held 2, offered 2, took 0, now holds 2
node 8f3fd656e34e ends with 2 chain records
node b3b38412206d ends with 2 chain records
```

Nothing opens a port by itself. Starting the application builds the node, and building the node binds nothing, announces nothing, and creates no directory. Linking is a thing somebody asks for afterwards. A node nobody has asked refuses to give out an address, because it does not have one.

```
PoaNodeLink installed (node=node-694eca0b8dc2, peers=...\poa_nodes,
                       network=acervator-poa, listening=False)
after install: listening=False, announced files=no directory, chain=[]
endpoint before start_listening raised NodeLinkError:
    node node-694eca0b8dc2 is not listening, so it has no endpoint
```

The link speaks to this machine and nowhere else. The address sits in one named constant, and no method anywhere takes an address to bind, so there is no setting to get wrong. A caller from any other machine is turned away before its message is read.

`src/competition/node_link.py` - the only interface, and the refusal

```
LOOPBACK_HOST = "127.0.0.1"

    def verify_request(self, request: object, client_address: tuple) -> bool:
        if client_address[0] != LOOPBACK_HOST:
            logger.warning("node link refused non-loopback client %s", client_address)
            return False
        return True
```

Nothing from the trading side can travel over it. A record is six declared fields and nothing else, so a peer that sends a seventh is refused where the record is built. The node holds the chain and holds nothing else, and the only two calls it can make on that chain are to add a transaction and to add an event. A record names a function as text, and no part of this calls it.

`src/competition/node_link.py` - the whole of what crosses

```
    from_addr: str
    to_addr: str
    function_name: str
    args: dict
    gas_used: int
    events: tuple
```

Agreement is on the records, not on the blocks, and that follows from the chain already in the tree. It keeps one list of blocks and always builds on the last one, so it has nowhere to put a second competing history. Each block's identifier is made from the clock and a random number instead of from the block's own contents, so two nodes holding the very same record still give their blocks different identifiers. **The chain cannot express a fork.** Nothing therefore has a branch to choose between, or a history to discard.

`src/competition/local_testnet.py` - where a block's identifier comes from

```
def _fake_hash(seed: str = "") -> str:
    raw = f"{seed}{time.time_ns()}{uuid.uuid4()}"
    return "0x" + hashlib.sha256(raw.encode()).hexdigest()
```

The rule is therefore to keep everything. A record is named by the hash of its own content, a node adds every record it does not already have, and nothing is ever thrown away. The order the two nodes talk in does not matter. What the rule refuses matters as much as what it does: it never removes a record, never reorders one already held, and never claims the two chains are identical block for block. The second exchange in the run above took nothing, which is the same record arriving twice and being recognised.

```
node 8f3fd656e34e synced with b3b38412206d: held 2, offered 2, took 0
node b3b38412206d took 0 of 2 records from peer 8f3fd656e34e
```

Breaking the link breaks the agreement, which is how we know the agreement came over the wire. The same two processes, with the link built but never started, end one record apart and stay that way.

```
node 0a4dbe51b761 ends with 1 chain records, never having listened
node 4629d5d46e1d ends with 1 chain records, never having listened
```

The TestNet demo runs the same code. A demo node is this same node over a different chain with a different chain name, not a second code path and not a switch. Two demo nodes link exactly as two live nodes do. A live node and a demo node sharing one announcement directory, both listening at the same moment, find no peer at all.

```
both on the demo chain
  node efad997ddc19 synced with 56d934d38462: held 1, offered 2, took 1, now holds 2
  network=acervator-poa-testnet

one of each, one directory
  node 883079b27380 discovered 0 peers      network=acervator-poa
  node b1435252aa59 discovered 0 peers      network=acervator-poa-testnet
  each ends with 1 chain records
```

Nothing in the running application starts a link yet. The node is built on every launch and waits. The control that would start it belongs on this tab, and no unit on the issue carries that control, so it is named here rather than invented. A node killed outright also leaves its announcement behind, and the next node to read it logs that it could not be reached and carries on.

```
In development.
```

## 2026-09-09 23:43 - #147 - the classes and the RPG conversion

The party window now lists real characters. Each row names one of the operator's bots, and the figure beside it shows that bot's health pool. Under the rows sit the seven classes a participant picks from.

```
04e1cafc | none | $54.19
092428b2 | none | $101.98
168b78e3 | none | $67.30
```

Every row says `none` for its class. A participant picks a class for an event, and no event exists yet.

### The seven classes and the four roles

The classes come from the seven classical planets and their metals. The roles come from the three principles of Paracelsus: Salt endures, Sulphur burns, Mercury flows. The operator's correction splits Mercury's three across healing and support, which adds Support as the fourth role.

```
Lead Ward              Saturn   lead         Salt     Tank
Tin Bulwark            Jupiter  tin          Salt     Tank
Iron Edge              Mars     iron         Sulphur  Damage
Solar Lance            Sol      gold         Sulphur  Damage
Quicksilver Draught    Mercury  quicksilver  Mercury  pure healer
Copper Conduit         Venus    copper       Mercury  support healer
Silver Mirror          Luna     silver       Mercury  pure support
```

Copper Conduit is the only class holding two roles. A support healer heals and supports, so it counts in both.

Each class levels on its own. A level record holds the class name, its level and its experience, and it starts at level one. One ability arc is a hundred levels, and the chain opens another arc with each alchemical phase.

```
ARC_LEVELS = 100
FIRST_LEVEL = 1
```

### Health comes from the dollar target

The operator's rule is that players take damage and never lose money. The health pool comes from the dollar line the engine defends, not from profit and loss. A bot below its line carries a wound and has lost nothing.

These are the figures the conversion produced for one live bot on A06/USD, with the field each one came from.

```
max health           209.20254   scrumming_state.target_balance
base health          200.00000   scrumming_state.anchor_target_balance
health from levels     9.202538  compounding_snapshot.accrued_growth_usd
gain cap per cycle     2.0       compounding_snapshot.cycle_growth_budget_usd
gain cap, percent      1.0       config.max_target_growth_pct
current health       199.28538   max health plus the gate reading's delta
```

The target grows as folds land, and a per-cycle cap bounds that growth. The engine already computes that curve, so the pool rises through play on its own.

### Damage is the scrum, healing is the fold

The two halves of the cycle are the two award axes. The operator's rule awards tokens for most damage done or most healed, so one number serves both scoreboards.

```
damage, year to date      676.88404   stats.ytd_scrummed_usd
healing, year to date     953.68802   stats.ytd_folded_usd
this heal                  10.11134   the fill's usd on a BUY
heals waiting              11         tranche_snapshot.fold_count
heals waiting, usd         29.720012  tranche_snapshot.fold_total_usd
```

Accuracy and efficacy come from the grade the platform gives a trade. The same bot's last fill graded `A+` at 1.0, with accuracy 1.0 and timing 1.0. The crit reading is the twelve voters' agreement, 0.0005 on that tick. The three fumble counts were all zero.

### Twenty-three of twenty-seven metrics read

The conversion answers twenty-seven metrics. Twenty-three carried a value on the live run. The four that did not are honest gaps, not errors.

```
wound depth          GateContext.delta_pct is computed and never emitted
this strike          the last fill was a BUY, so the SELL half is empty
outcome efficacy     the grader needs a per-unit realised figure
strategic efficacy   the grader needs a rolling sell-to-buy reading
```

### Breaking a source takes its metrics away

A metric with no field answers nothing. Removing one part of a bot's record removes exactly the metrics that part feeds, and nothing else.

```
whole record and gate reading      16 metrics
scrumming_state removed            13 - max, base and current health gone
stats removed                       9 - damage, healing, fumbles, pool, cash gone
config removed                     15 - the percent gain cap gone
gate reading removed               10 - six gate-fed metrics gone
```

### Seven things the platform holds no field for

Experience, level, character class, gear, an enemy, a threat value and a guild have no field anywhere in the source. They are state this design must create. Nothing in the trading profile stands in for them.

```
experience   level   character class   gear   enemy   threat   guild
```

One number is still the operator's to set. A critical hit fires above some confidence, and no design passage names that figure. The conversion reports the confidence and makes no judgement on it.

### Demo mode takes the same path

A TestNet run asks the same surface for the same model and names its own chain. The fleet file for a demo chain carries the chain in its name.

```
chain live      bot_state.json           exists   38 source bots, 7 classes
chain testnet   bot_state_testnet.json   absent    0 source bots, 7 classes
```

The classes are not chain data, so both runs list all seven. The party window keeps its empty-state sentence on the demo chain and drops it on the live one.

### What the classes and the conversion do not reach

A participant cannot pick a class from the screen yet. No picker, no event and no turn exist, so the pick function has no caller.

```
def pick_class(participant: str, event_id: str, class_name: str) -> ClassPick:
```

A row prints health as a figure, not as a bar, and carries no role colour. Current health and wound depth need the fire-time gate reading, which arrives on the bus rather than in the saved file. Nothing here reads an ability, a mode or a turn.

## 2026-09-10 00:00 - #147 - the rotating reward set and eligibility

A market pays Quintessence only while three things are true at once. It sits in the top twenty by volume on that exchange, its project is at least six months old, and the live rotation drew it. Any one of the three missing, and the market pays nothing.

`src/competition/market_rotation.py` — the draw, and the floor under it

```
def draw_size(pool_size: int) -> int:
    """``quarter_rounded_up`` of ``pool_size``, and 0 below ``MIN_ELIGIBLE_POOL``."""
    if pool_size < MIN_ELIGIBLE_POOL:
        return 0
    return quarter_rounded_up(pool_size)
```

### The ranking now counts money, not coins

The volume figure the old ranking read was a count of coins traded, and a coin count cannot be compared between two markets. Five thousand bitcoins and four billion meme tokens are both big numbers, and only one of them is real money. The rotation now ranks on the dollar value traded instead.

The two rankings were taken from one live Coinbase snapshot, the same 403 assets, one sorted each way. They share one market out of twenty.

```
top 20 by dollars traded   A15 A14 A05 A18 A23 A28 A13 A30 A21 A22
                           USELESS UNI A29 A25 A34 A16 A27 A32 LIGHTER A36

top 20 by coins traded     NEX MOG PEPE A10 SHIB FLOKI TOSHI VTHO A29 NOICE
                           SPELL BNKR DRB B3 AMP DOGINME NOM BLAST A12 OXT
```

The second list is nineteen sub-cent tokens. Those are exactly the obscure pumps the rule exists to turn away, and the old figure would have handed them the whole reward surface.

Coinbase serves the dollar figure for every market it lists. The platform asked for all 931 of its pairs and not one of them was missing it.

| What the ranking reads | Where it comes from |
|---|---|
| The dollar value traded in 24 hours | `quoteVolume`, served by the venue |
| The same figure where a venue omits it | coins traded times the last price |
| The coin count, unchanged and still recorded | `volume_24h`, read by the topology proposals |

One market per asset is ranked. An asset quoted against both dollars and USDC was counted twice before, so a top twenty held only ten assets.

### Five of twenty qualify today, and Coinbase pays nothing

The six-month rule reads a project's start date, and most projects publish none. Run against Coinbase's real top twenty, fifteen of them are refused.

```
eligible          A15/USD  A14/USD  A21/USD  A22/USD  A36/USD
no identifier     8 markets     the asset catalogue has no CoinGecko id
no start date     7 markets     CoinGecko answers with no date
pool size         5
markets drawn     0
```

Five is under the floor of twelve, so the exchange draws nothing at all. That is the rule working as designed and it is also the measured cost of refusing an unknown age. Widening the asset catalogue is what widens the pool.

### The draw is a quarter of the pool, and never reaches a single market

A fixed draw of five from a shrinking pool would let an exchange pick the winner by removing everything else. The draw scales with the pool instead, so the odds on any one market stay near one in four however many markets remain.

```
pool  20   drawn 5      pool  13   drawn 4      pool  11   drawn 0
pool  16   drawn 4      pool  12   drawn 3      pool   5   drawn 0
```

Below twelve the exchange draws nothing. Eleven eligible markets were reached by filing real exclusions, and the window refused to open.

```
open_window refused: pool_below_floor: coinbase holds 11 eligible markets,
under the floor of 12, so it draws nothing
```

### An exclusion waits for the next season

An exchange may remove its own markets from the rotation list. It cannot add one, choose one, or time one. The removal takes effect at the next season only, so it can never be filed against a window that is already running.

A season is a counter that a call advances. It carries no date, so the boundary is an event and nobody can predict when it falls.

```
filed in season 1, binds from season 2   A15/USD
in effect in season 1                    nothing
in effect in season 2                    A15/USD
season 1 A15/USD                         eligible
season 2 A15/USD                         excluded_by_exchange
```

### What the chain carries while a window is open

The chain carries a fingerprint of the chosen markets and the count, and no market name. The fingerprint is built from a random value held back until the window closes, so nobody can test a guess against it.

```
chain tx args: {"exchange": "coinbase", "season": 1,
                "commitment": "2c69e1e853579c19...", "marketCount": 5}
```

At the close the markets and the random value are published together, and anyone can check that the published set is the one the fingerprint was made from.

```
reveal markets   A34/USD  A28/USD  LIGHTER/USD  UNI/USD  A13/USD
proof verified   True
```

Asking which markets pay while the window is open is refused.

```
membership_proof refused: coinbase has an open window; a membership proof
would reveal the set it conceals
```

What this hides, and from whom:

| Reader | While the window is open |
|---|---|
| A participant | cannot learn which markets pay |
| Another node | cannot learn which markets pay |
| The exchange | cannot learn which markets pay |
| The node that drew the set | holds the answer on its own disk |

Two things it does not do. The drawing node chose the set, so it could have chosen a set that suits it; the fingerprint only stops it changing its mind afterwards. And no other node can repeat the draw and check it was fair. Both need a shared random value that several nodes produce together, and the key type for that does not exist in the platform yet.

The wording in the design is "encrypted on-chain". Encryption needs a recipient who holds a key, and no field anywhere holds an exchange key. A fingerprint is what ships instead, and on the property the design actually cares about it is stronger: encryption to an exchange would let that exchange read its own rotation early, and a fingerprint lets nobody read it.

### Demo mode needs no second path

The rotation takes its chain when it is built. A TestNet run is one rotation over a different chain with a different record file, calling the same methods. Nothing switches on a flag.

```
live      MarketRotation   market_rotation.json          chain A
testnet   MarketRotation   market_rotation_testnet.json  chain B
same class, same open_window path
```

### What the rotation still waits for

Nothing asks for a rotation yet. The object is built on every launch and then waits for a caller.

`src/gui/main_window.py:277` — the launch call that builds it

```
                  SharedTestnetBridge.install_on(self)
```

Three parts are still missing. No market has a Quintessence pool yet, so the five per cent share ceiling has a number to apply and nothing to apply it to. Nothing advances a season. And the walk that reads project ages waits thirteen seconds between calls, so it must never run on the thread that draws the screens.

The pacing was measured. Asked two seconds apart, CoinGecko refused thirty-four of forty calls. Asked thirteen seconds apart, it answered every one, twelve of twelve on a cold cache and seven of seven on a warm one. ## 2026-09-10 00:03 - #147 - the contract audit against a published standard

Four security tools now run over the four contracts, and every finding they give carries a level from a published standard. Nothing was repaired in this pass, and no line of Solidity changed. The standard is the EEA EthTrust Security Levels Specification Version 3, published by the Enterprise Ethereum Alliance in March 2025, and the older SWC numbers sit beside each finding so a reader can match them to what the tools print.

The tools, and the version of each one as the tool itself reports it:

```
forge     1.8.1     build, lint, and the fuzzing runner
slither   0.11.6    static analysis
solhint   6.2.4     the Solidity linter
semgrep   1.171.0   50 Solidity rules
mythril   absent    see below
```

### Not one tool was trusted until it had been shown failing

A tool that has only ever reported nothing has not been shown able to report at all. Each of the four was pointed at a small broken file and a matching sound one, and each told the two apart.

```
forge      broken file   FAIL   buckets do not equal totalEverMinted
           sound file    PASS   16,384 calls, nothing broken
slither    broken file   1 result at its top severity
           sound file    0 results
solhint    broken file   1 error   "Avoid to use tx.origin"
           sound file    0 errors
semgrep    broken file   1 finding at its top severity
           sound file    0 findings
```

The last pair is the one that carries the headline. semgrep found no security problem in the contracts, and the broken file proves the same security rules were switched on and working while it found none.

### The books balance, and this is the run that proves it

Quintessence rests in four places and the four always add up to everything ever minted. That sentence is now a property a machine holds rather than a claim on a page. The fuzzing runner drove the contract through random sequences of every movement it has.

```
256 runs   16,384 calls   8,661 of them correctly refused

ten properties checked after every call, all held
no sequence broke the books
```

### mythril could not be installed, and the audit says so

The fifth tool reads the compiled bytecode. One of the pieces it needs cannot be built on this machine, because a Microsoft C++ compiler is absent and there is no Docker to fall back on. Nothing was put in its place.

```
mythril needs pyethash, which needs Microsoft C++ Build Tools
docker                                not installed
```

Two things unblock it, and both are a spend decision:

```
install Microsoft C++ Build Tools on this machine
install Docker and use the mythril maintainers' own image
```

### What the tools found, in their own words

Every severity below is the tool's own. None was assigned here.

| Tool | Its top band | Findings in that band |
|---|---|---|
| slither | High | 0 |
| slither | Medium | 1 |
| forge | warning | 9 |
| solhint | error | 34 |
| semgrep | ERROR | 0 |

The thirty-four solhint errors are all one thing, and all in one file: a single quote where the linter wants a double one. They sit in the trophy's artwork and metadata assembly, where single quotes build the JSON a marketplace reads.

The one slither Medium sits on the guard that stops a second Quintessence transfer starting while one is already in flight.

```
contracts/Quintessence.sol   authorizeTransfer
EthTrust level [M]           a human auditor decides whether the exact
                             comparison is necessary
```

### Eighteen findings from the first pass are gone

The repair unit landed before this one ran. Eighteen findings the first pass reported are absent from every run here, so the audit does not carry them forward.

```
slither   3   reentrancy in adjudicate, and the unchecked oracle answer
forge     9   five unsafe number casts, and four ordering warnings
solhint   3   the floating compiler version, now fixed at one release
semgrep   3   one-step ownership transfer, now a two-step handover
```

### The four questions the design raised, answered

**Who can pause the token.** The owner, and only the owner, and the owner is whoever sends the deployment transaction. A pause freezes every transfer and every award at once, because both run through one gate. The owner can never mint. Nothing limits how long a pause may last, and nothing requires a second signature to start one. The entry currency is a different shape: Quintessence has no owner and no pause at all.

**Where spent Quintessence rests.** At a held address, and it never leaves one. Spending moves a holder's own units out of the wallet bucket into the held bucket, and no function takes them back out. The award token is the half still open: it has no burn and no resting place, so spending one has nowhere to go yet.

**Which trophy tiers have a lifetime ceiling.** Four of the five, enforced inside the trophy contract. Harvest has none by design. On the token side two tiers are capped and the rest share the ten million supply cap. The season budget is still a Python rule and reaches no contract.

**What a wrong address at deployment costs.** A redeployment for three of the four contracts, and nothing at all for the token, whose minter is now named by a one-time call that refuses an address with no code behind it. The circular deployment order is gone.

### Where the findings are written down

The full report is a document. The raw tool output is not, because captured output never belongs in the documentation tree.

```
docs/audits/2026-09-10_contract_audit_ethtrust_levels.md   the report
artifacts/solidity_audit/                                   the raw output,
                                                            untracked
```

### What running these tools is not

It is not an audit in the sense the industry means. Tools find known weakness classes. They do not find a flaw in what a contract is for, and the standard says as much by putting business logic at its highest level. Contracts holding real value go to an outside firm before they reach a main network. That is a timing and cost decision, and it is named here so nobody discovers it late.

## 2026-09-10 00:28 - #147 - the four modes, the Elite flag and the turn

The tab now carries the eight event types the design names, a turn tied to the market's own candle, and the per-turn pool of points a participant spends inside it. A real bot also picks a character class for the first time.

### Four modes, one Elite flag, eight event types

The design carries four modes and one Elite flag, not eight modes. Difficulty, entry fee and both loot ranks are properties the variant answers, and one table is the only place the flag is read.

`src/competition/poa_modes.py` — the whole Elite difference

```
STANDARD_RULES = VariantRules(
    STANDARD_SUFFIX, STANDARD_LABEL, STANDARD_TIMEFRAME, 0, 0, 0, 0
)
ELITE_RULES = VariantRules(ELITE_SUFFIX, ELITE_LABEL, ELITE_TIMEFRAME, 1, -1, 1, 1)
```

Elite is one step harder, one step cheaper to enter, and one step richer in both loot ranks. Those three directions are the whole of it.

| Event type | Turn | Difficulty | Entry fee | Loot rarity | Loot drop |
|---|---|---|---|---|---|
| monster_smash | 5m | 1 | 1 | 1 | 1 |
| monster_smash_elite | 1m | 2 | 1 | 2 | 2 |
| team_monster_smash | 5m | 2 | 2 | 2 | 2 |
| team_monster_smash_elite | 1m | 3 | 1 | 3 | 3 |
| dungeon_crawl | 5m | 3 | 3 | 3 | 3 |
| dungeon_crawl_elite | 1m | 4 | 2 | 4 | 4 |
| raid | 5m | 4 | 4 | 4 | 4 |
| raid_elite | 1m | 5 | 3 | 5 | 5 |

A rank is a place in an order, never a price. No entry fee amount and no loot table exist yet, so the ranks say which event is dearer and richer, not by how much. Unit 19 builds the loot.

### The turn length follows the mode, and nothing branches on the flag

An Elite turn is one 1m candle and every other turn is one 5m candle. No rule asks whether an event is Elite; each one reads a property instead, and that property reads the table above.

`src/competition/poa_modes.py` — the turn's own length

```
    @property
    def turn_timeframe(self) -> str:
        """The candle one turn is one of, from ``rules.timeframe``."""
        return self.rules.timeframe

    @property
    def turn_seconds(self) -> int:
        """The seconds in ``turn_timeframe``, read from ``TF_SECONDS``."""
        return TF_SECONDS[self.turn_timeframe]
```

The seconds in a candle come from `TF_SECONDS`, which is the map the platform's own candle cache already uses. The game keeps no clock of its own.

### The clock is fixed, and three refusals prove it

A turn cannot be paused, extended or negotiated, because its boundary belongs to the market and every participant shares it. The program refuses each attempt in its own words.

```
a deadline past the candle close
  turn 16666667 of the 1m candle closes at 1000000080; a deadline of
  1000000100 would extend it by 20s, and the candle clock is fixed for
  every participant

acting after the candle closed
  turn 16666667 of the 1m candle closed at 1000000080 and it is
  1000000080; the 1 Impetus for this action is lost with that turn

spending last turn's leftover
  this pool granted 4 Impetus for turn 16666667 and 3 is unspent; turn
  16666668 is a different turn, and Impetus expires with the candle that
  granted it

paying part of an action
  this action costs 4 Impetus and 3 remains in turn 16666667; no partial
  action exists and nobody borrows against the next turn
```

### A participant who joins midturn takes what is left of the candle

Joining late does not move the deadline. Both participants below sit in the same turn and both lose it at the same instant.

```
joined at the candle open    deadline 1000000080   60s in hand
joined 40s later             deadline 1000000080   20s in hand
```

### The pool is called Impetus, and level decides how much

A turn grants four Impetus at level one and one more every twenty levels. The candle never grows, so a higher level acts more inside the same window.

`src/competition/poa_modes.py` — the grant

```
def base_impetus(level: int) -> int:
    """Four at ``FIRST_LEVEL``, one more every ``IMPETUS_LEVELS_PER_STEP`` levels."""
    if int(level) < FIRST_LEVEL:
        raise PoaModeError(f"level {level} is below {FIRST_LEVEL}")
    return IMPETUS_AT_FIRST_LEVEL + int(level) // IMPETUS_LEVELS_PER_STEP
```

| Level | Impetus a turn |
|---|---|
| 1 | 4 |
| 21 | 5 |
| 40 | 6 |
| 60 | 7 |
| 80 | 8 |
| 100 | 9 |

A haste or a slow effect multiplies that grant once. The product stops at twice the level's own figure and never falls below one, so nobody is frozen out of a turn and speed cannot become the only statistic worth raising.

```
level 40 grants 6     a 1.5x haste gives 9     the cap holds it at 12
                      a 0.5x slow gives 3      the floor holds it at 1
```

What an action costs is not here. Unit 13 prices the five bands.

### A class is picked, and the pool follows the level

The pick function had no caller until now. The screen passes a participant and a class name, the event it is for is the variant's own code, and the party row then carries the class, its level and the Impetus that level grants.

```
04e1cafc  A26/USD    Iron Edge   level 1   Impetus 4   health $54.19
092428b2  A10/USD  none        --        --          health $101.98
```

A class name outside the seven is refused and the panel prints the refusal.

```
'Iron Sword' is not a PoA class; the seven are Lead Ward, Tin Bulwark,
Iron Edge, Solar Lance, Quicksilver Draught, Copper Conduit, Silver Mirror
```

No field under `src` holds a class level, so a fresh pick stands at level one and nothing saves it between asks.

### The unreachable tournament module is retired

The tournament module under `src/trading` held 832 lines and nothing imported it. None of its nine public names appeared anywhere outside its own file, the coding archetype refused it, and it wrote generated output into the repository. Its three shapes were a duel, a melee and a gauntlet, which are not the four modes the design names.

```
832 lines        zero importers        zero uses of its nine public names
passed=False     8 high findings       wrote <repo>/logs/tournaments
```

Two things in it were worth keeping and both carried over. A party row is named by its bot id rather than by holding a bot object, and one event shape serves the screen as a plain row. The seeded simulation, the scorer for computer-run party rows, the invented candles and the settlement adapter that paid nobody are all gone.

### Demo mode answers the same event

A TestNet run asks the same surface and names its own chain. The modes, the turn and the pool are not chain data, so both runs answer the same event.

```
chain live      38 source bots   8 event types   raid_elite, 1m candle
chain testnet    0 source bots   8 event types   raid_elite, 1m candle
```

### What the modes and the turn do not reach

No action exists to spend Impetus on from the screen. The pool is granted and refuses what it cannot pay, but nothing draws a move, an attack or a spell.

A charge that spans more than one turn is not built. Whether it pays its pool at the start or per turn, and what an interruption does to it, are open.

The event is whichever one the request names. No schedule opens an event, no entry fee is taken, and no participant is admitted or turned away at the door.

In development.

## 2026-09-10 00:34 - #147 - a block and a transaction named by their own contents

An identifier on this chain used to come from the clock and a random number. Two copies of Acervator holding the identical record gave that record two different names, and editing an amount in a stored record left every name in the chain still valid. Both identifiers are now the SHA-256 of the record's own contents.

`src/competition/local_testnet.py` - what names a transaction

```
def transaction_id(
    from_addr: str,
    to_addr: str,
    function_name: str,
    args: dict,
    gas_used: int,
    status: int = TX_SUCCESS,
) -> str:
    return content_id(
        {
            "from_addr": from_addr,
            "to_addr": to_addr,
            "function_name": function_name,
            "args": args,
            "gas_used": gas_used,
            "status": status,
        }
    )
```

### What goes into each identifier, and what stays out

A transaction is named by who sent it, who received it, the call, the amounts and addresses that call carried, the gas and the success flag. The block it landed in and the moment of recording stay out, because two machines record one transaction at different moments into differently numbered blocks. A block is named by its own number, its parent's name, its timestamp and the list of transactions it holds.

| value | names a transaction | names a block |
|---|---|---|
| sender and recipient | yes | no |
| the call and its arguments | yes | no |
| gas | yes | no |
| success flag | yes | no |
| block number | no | yes |
| timestamp | no | yes |
| parent block's name | no | yes |
| the transactions held | no | yes |

The bytes are fixed so two machines cannot differ: keys in sorted order, no spaces, and a refusal for any number that is not finite. A block keeps its timestamp to the millisecond, so the name covers the figure written to disk rather than a longer one that is not.

The amounts and the addresses reach the name. That was the hole: a transaction used to be named from its function name alone, so what it moved and who it moved it to had no bearing on its name.

### Two machines, one name

Two copies, two separate chains, two processes. Each certified one fill of its own, and each then held the other's record under the identical name.

```
node-952ca991e4da  sending transaction 0xc69ee5a4fa90328a9f730da334451160430c3e3c4279e8a863a83099d86d5cc0
node-ae8a2c70bb84  sending transaction 0xc69ee5a4fa90328a9f730da334451160430c3e3c4279e8a863a83099d86d5cc0
node-ae8a2c70bb84  sending transaction 0xaab8eefd0d4f2e481a6a7a79b3bfb730d8765846f4376c660d9534abc54ccf18
node-952ca991e4da  sending transaction 0xaab8eefd0d4f2e481a6a7a79b3bfb730d8765846f4376c660d9534abc54ccf18
chain verified: 3 blocks and 2 transactions carry the id of their own contents
```

The same pair of processes on the previous build gave the identical transfer two different names.

```
before   node_a  0x692edd3d07a1b2019b3ce1ed84a262ce9e2a033181ab24805455ee580c84965d
         node_b  0x3d3153c056bb1bc55c4f613bebfc81d3b9ab6a7581e436c7263fb24684ac6b03
after    node_a  0x3472b3435c6b9c843317353b681810c2f416501d6ac2f2c302ff758cdbecdb6a
         node_b  0x3472b3435c6b9c843317353b681810c2f416501d6ac2f2c302ff758cdbecdb6a
```

### A changed record is named, and the other machine refuses it

One stored record had its amount edited from 12.5 to 99.5 and its name left alone. The chain that loaded that file says so at load. The other machine refuses the record outright, and the same exchange still accepted the good record sent beside it.

```
chain NOT verified: 0 of 3 blocks altered [], 0 parent links broken [],
                    1 of 2 transactions altered ['0x3472b3435c6b9c84']

node-a7904d78dd27 refused record 0x3472b3435c6b9c843317353b681810c2f416501d6ac2f2c302ff758cdbecdb6a
    calling transferQuintessence: its contents name
    0x1a4bdf2cbbfd171da96b52a51aab69358b7fdbff4ef8c4809c904fb6b1b0a54f
node-a7904d78dd27 synced with node-6feb24ca8462: held 2, offered 3, took 1, now holds 3
chain verified: 4 blocks and 3 transactions carry the id of their own contents
```

The refusal sits in the code that adds a peer's records to a chain, so an altered record never reaches the chain at all.

### The chain already on disk

The saved chain on this machine loads exactly as before and nothing is lost. Its 3,135 blocks and 3,135 transactions were named the old way, so the chain reports that none of them carries the name of its own contents. Every block mined from now on does.

```
chain restored from disk (block=3135, age=204388 min)
chain NOT verified: 3135 of 3136 blocks altered [1, 2, 3, 4, 5],
                    0 parent links broken [],
                    3135 of 3135 transactions altered ['0x3f79536e53815466', ...]
```

Renaming those 3,135 records was considered and refused. The node rule already in place never removes a record and never reorders one, and renaming every record on the chain is the largest possible change of identity. A load that recomputed every name would also destroy the one property this work adds, because a name recomputed from whatever is on disk can never disagree with what is on disk.

### Demo mode

A demo run is the same code over a different chain and a different network name. The run takes no setting and no flag. The transfer carried the same name on the demo chain as on the live one, because the name comes from the contents and from nothing else.

```
PoaNodeLink installed (node=node-04b4e394a029, network=acervator-poa-testnet, listening=False)
sending transaction 0x3472b3435c6b9c843317353b681810c2f416501d6ac2f2c302ff758cdbecdb6a calling transferQuintessence
node-04b4e394a029 synced with node-2a283bba3115: held 3, offered 3, took 0, now holds 3
chain verified: 4 blocks and 3 transactions carry the id of their own contents
```

### The demo competition's practice prices

The demo competition trades against 120 made-up prices rather than a market. The generator behind them changed to the one the rest of the platform already depends on, so the figures that run reports are different from before. It is still seeded, so the same 120 prices come back on every run, and the competition still names a winner and mints the award.

```
participant nodes   0
source bots         3
winner tier    Harvest
winner tokens  10
rank 1 value   371.46   the same figure on a second run
```

### What this does not cover

The first block of a chain keeps a name of all zeros. It has no parent and holds no transaction, so the only thing in it that a name could cover is its timestamp, and that timestamp is not covered.

A record sent between two machines now carries seven declared fields rather than six. The seventh is the name the chain gave it, and the receiving side recomputes that name from the other six before it accepts anything.

A chain file is not protected from whoever owns the machine. Editing a record and recomputing its name by hand produces a file that verifies. What the names buy is that a second machine holding the same record disagrees out loud.

## 2026-09-10 00:38 - #147 - the contract repairs, and the keys that came out

The audit found five things worth a decision. Three are repaired and the tool that reported each one is now silent on it. Two are left standing on purpose, with the reason written down, because repairing them would make the contracts worse.

```
repaired   34 quote errors in the trophy contract        solhint 34 -> 0
repaired   an exact comparison on a pending transfer     slither 1 -> 0
repaired   the trophy handed out a token before filling it
                                                        slither 2 -> 0
stands     reading the clock to time a transfer          the design needs it
stands     eight writes a lint calls eventless           the lint is wrong
```

### The three repairs

The trophy builds the text a wallet reads from the artwork and the award details. That text is full of quote marks, so it was written with the other kind of quote, and the linter calls that an error thirty-four times over. Thirty-four errors in one lane hide the thirty-fifth, so the quoting is now the style the linter asks for. The text a wallet displays is byte for byte what it was.

```
before  343 problems, 34 of them errors   the run fails
after   307 problems, none of them errors the run passes
```

The second repair is one line. A holder may have only one transfer waiting at a time, and the check asked whether the waiting amount was exactly zero. The analyser objects to an exact comparison, because the record holding that amount also holds a clock reading. The check now asks whether an amount above zero is waiting, which admits exactly the same thing and reads better.

```
    function _hasTransferInFlight(address sender) private view returns (bool) {
        return pendingTransfer[sender].amount > 0;
    }
```

### The two findings that stand

A transfer takes time by design. The shortest window is one hour and it grows with the amount, so the contract has to read the chain's own clock to know whether a window has passed. A validator can shift that clock by seconds. Seconds against an hour is not a risk, and the chain offers no other clock, so the finding is recorded rather than repaired.

```
MIN_TRANSFER_SECONDS        3,600      one hour, the floor
TRANSFER_SECONDS_PER_WHOLE    360      the window grows with the amount
shortest window             1 hour
a validator can shift       seconds
```

The other standing finding is a lint that says eight writes happen without an event. Six of the eight do emit an event in the same call, so the message is not describing what the rule measures. Four small test bodies settle it: the rule fires on any write to a value a safety check reads, event or no event. Clearing it would mean deleting the check that a wallet holds enough to spend, which is the wrong trade, so it stands.

```
write, safety check, no event    the rule fires
write, safety check, then event  the rule fires
event, then write, safety check  the rule fires
write, no safety check           the rule is silent
```

### Fourteen owner keys, and what happened to each

The entry currency has no owner at all. The other three contracts had fourteen powers only the owner could use. Two are gone from the code. Twelve stay, and each one is now written down as waiting for the vote, which is a separate piece of work and exists nowhere yet.

```
gone   the owner could mint any trophy directly
       the trophy now admits the competition registry alone
gone   the owner could abandon the contract
       on the award token that would have made a freeze permanent
```

```
waiting for the vote
  the token      name the minter once, freeze, unfreeze, hand over ownership
  the registry   set a price feed, open, activate, close, adjudicate,
                 advance the season, cancel, hand over ownership
  the trophy     upload a tier's artwork, hand over ownership
```

Removing the owner's route into the trophy mint has one consequence worth stating plainly. The competition registry still holds no reference to the trophy contract, so nothing can mint a trophy until that wiring lands. Before this change the owner was the only caller that ever reached it, which is the bypass the design says must come out.

### The conservation law still holds

The law is that every wallet, every held address, the pleroma pool and the embedded bucket add up to everything ever distilled, and that the total never passes thirty-three million. The fuzzing runner drove it again after the repairs.

```
runs       256
calls   16,384
ten invariants   all pass
```

The count of refused calls inside that campaign moves from run to run, because the runner is not given a fixed starting seed. Three runs after the repairs gave 8,546, 8,721 and 8,773, and the run before them gave 8,523. The two numbers the configuration fixes are the same before and after, and no sequence ever broke the law.

### What the vote still has to take over

No vote, no council and no delay was written here. Twelve owner powers are still owner powers today, exactly as they were, and the list above is the record of what the vote has to take over.

Every finding the audit raised is accounted for in one place, each marked closed or standing, with the count the tool printed on each side.

```
tests/debug_reports/contract_repairs_second_pass.md   the full accounting
```

## 2026-09-10 00:59 - #147 - the volume figure and the eight missing identifiers

Two faults held the rewarding pool down. The price feed reported no trading volume at all for any Coinbase market, and the asset catalogue could not name eight of the twenty biggest markets. This unit fixed both. The pool went from five markets to six.

`src/exchange/ccxt_connector.py` - where the volume figure now comes from

```
def _quote_volume_for(self, symbol: str, raw: dict) -> float:
    """Return ``symbol``'s 24h quote volume from ``raw``, else the recorded one.

    Coinbase's ``fetch_ticker`` serves neither ``quoteVolume`` nor
    ``baseVolume``, so ``get_ticker`` falls back to what the last
    ``get_all_tickers`` recorded for that symbol, and 0.0 before the first.
    """
    served = row_quote_volume_24h(raw)
    if served > 0:
        return served
    return float(self._quote_volumes.get(symbol, 0.0))
```

### Asking Coinbase about one market returns no volume

The platform asks Coinbase for a price in two ways. A question about one market comes back with a price, a bid and an ask, and nothing about volume. A question about every market at once comes back with the dollar volume on almost every row. The price feed read the one-market answer, so its volume figure always held zero.

The platform's own trading log says it, before and after:

```
before   FETCH_TICKER  Get current price for A15/USD ... vol24h=0
after    FETCH_TICKER  Get current price for A15/USD ... vol24h=411256638
```

The bulk answer carried a dollar figure for 923 of the 931 markets Coinbase lists. The platform already sends that bulk request on a timer. The connector now keeps each figure as it arrives and hands it back when a caller asks about one market, so nothing new goes to the venue. A market the bulk request has not yet covered still reports zero, which is what it reported before.

### Eight projects the catalogue could not name

The six-month age rule looks a project up by a CoinGecko identifier held in the asset catalogue. Eight of the twenty biggest Coinbase markets had no entry, so the rule refused them without asking anything.

Every one of the eight now has an entry, and nobody guessed an identifier. For each one, CoinGecko named which of its own coins trades that market on Coinbase Exchange, and the catalogue took that coin.

| Market | Identifier | How the check ran |
|---|---|---|
| A05 | A05 | CoinGecko lists it trading A05/USD on Coinbase Exchange |
| A28 | A28 | CoinGecko lists it trading A28/USD on Coinbase Exchange |
| A13 | A13 | CoinGecko lists it trading A13/USD on Coinbase Exchange |
| USELESS | useless-3 | Two coins carry this ticker; only this one trades on Coinbase |
| A29 | A29 | Two coins carry this ticker; only this one trades on Coinbase |
| A16 | A16 | CoinGecko lists it trading A16/USD on Coinbase Exchange |
| A32 | A32 | CoinGecko lists it trading A32/USD on Coinbase Exchange |
| LIGHTER | lighter | CoinGecko lists it trading LIGHTER/USD, under the ticker LIT |

### Matching by ticker alone fails its own check

The other way to close the gap reads CoinGecko's published coin list at run time and matches on the ticker. The twelve markets whose identifier the catalogue already held, and held correctly, tested that idea.

```
twelve known-correct identifiers
  2  resolve to exactly one coin on the published list
 10  are ambiguous; twelve separate coins carry the ticker A15
```

A resolver that insists on one match refuses ten markets that work today. A resolver that picks among the candidates returns a real founding date for the wrong project, which is worse than a refusal. One of the eight is worse still: CoinGecko carries Lighter under the ticker LIT, so no ticker match finds it at all, while its record of the Coinbase market names it exactly.

The catalogue grew instead, and the venue record did the confirming. A market whose base is still absent from the catalogue refuses with the reason `no_coingecko_id`, as it did before, and never passes unchecked.

### Six of twenty qualify, and the floor is still twelve

Driven against Coinbase's real twenty, the refusals moved from two reasons to one. A05 publishes a founding date and now qualifies. The other seven newly named projects publish none.

```
before   5 eligible   8 refused with no identifier   7 refused with no start date
after    6 eligible   0 refused with no identifier  14 refused with no start date
```

The program declines to open a window in its own words:

```
before   pool_below_floor: coinbase holds 5 eligible markets, under the floor
         of 12, so it draws nothing
after    pool_below_floor: coinbase holds 6 eligible markets, under the floor
         of 12, so it draws nothing
```

Both runs asked CoinGecko at thirteen seconds apart and CoinGecko refused nothing. Twelve detail requests on the first run, fifteen on the second.

### What the identifiers did not fix

The earlier estimate put the pool at thirteen once the catalogue closed, which would clear the floor. It reached six. Seven of the eight newly named projects publish no founding date, so they moved from one refusal to another rather than becoming eligible.

Coinbase still awards no Quintessence, and one reason remains: fourteen of the twenty biggest markets publish no founding date. The age ruling decides whether the economy starts. ## 2026-09-10 01:12 - #147 - the art brief for the enemies, the party and the rail

The pixel art for this tab now has a commissioning document. An outside artist can work from it without asking a question. It specifies the register, the palette, every sprite size, every frame count and the sources each creature comes from.

```
docs/engineering-notes/2026-09-10_poa_art_brief.md    the brief
```

The manual points at the brief and does not carry it. This manual describes what the product does; the brief tells somebody outside the company what to draw and what it costs. Those are different documents for different readers.

### Every creature names the old text it comes from

The operator asked for the ancient sources and nothing from the modern occult revival. Every creature in the brief carries the text that attests it, and a creature with no text did not go in.

```
the surface floor  Lamashtu, Lilith, Empousa
the executors      the gallu, Ammit, the Keres, the Sebitti, Humbaba, Mot
the decan rank     thirty-six star powers, each owning one part of the body
the floor bosses   Apophis, Anzu, Asag, Typhon, Echidna, the Gigantes, Azi Dahaka
the fall tier      the Watchers and the Nephilim, Asmodeus, Angra Mainyu
the summit         Old One Avatars and their direct servants
```

Twenty-one creatures carry their own name and the text that attests it. Three more arrive as ranks rather than as single names: the thirty-six decans, their daimones and their assistants. The two at the summit are the operator's own, built on Lovecraft, and the brief says that plainly rather than letting a modern invention look ancient.

### Five things the brief could not supply, and it names them instead of inventing them

The operator's rule was to name the gap rather than reach for the excluded source. Five gaps came up and the brief names all five.

```
a name for each of the ten husks     only the modern revival supplies one
a look for each husk                 no old source draws one
the thirty-six decan images          the roster sits behind editions not in hand
a colour for Salt, Sulphur, Mercury  no design token holds one
the panel's absolute height          nothing in the code declares it
```

The fourth one reaches the screen. The design makes the role colour the one thing a party row never drops, and no colour token carries a value for it. That is work for the design system, not for the artist.

### The brief measured the party window and found four disagreements

Four numbers in the design prose do not match the page as built. The built page is the fact in every case.

```
120 participants        confirmed
40 to a page            confirmed
eight groups of five    confirmed
three pages             confirmed
the name truncates      at eight characters, not twelve
one mark slot           not built; the row carries five text columns
a health bar            not built; health is a text figure
a role colour           not built; no token carries one
```

The twelve-character short form is real and sits somewhere else. It names the participant identity and the competition identifier, and the party row shows neither.

### The guaranteed art size is fourteen pixels square

Every other size on the tab moves when the window moves. The party row's floor height does not, because the stylesheet fixes it at sixteen pixels and the row's own border takes one from each side.

```
the party row cell       14 x 14 pixels, guaranteed
the class sigil          16 nominal, drawn at 1x
the mark slot glyph      16 nominal, drawn at 1x
an acting sprite         48 nominal, drawn at 4x
an enemy                 32 to 160 nominal, by tier
an Old One Avatar        160 nominal, drawn at 2x, alone on the enemy screen
```

Every frame count comes out of the platform's own motion settings rather than out of taste. An idle loop runs four frames over one second. An attack runs five frames over half a second. A full creature is twenty-six frames.

### What the whole commission costs

The operator pays for this, so the brief counts it.

```
enemies, five of the six tiers       531 frames
player sprites, seven classes        147 frames
sigils, marks and rail nodes          17 static drawings and 4 frames
                                     -----
                                     682 frames and 17 static drawings

the decan rank, once specified       936 frames
```

The decan rank is thirty-six creatures and costs more than the other five tiers put together. That same tier is the one the brief cannot specify yet, so it goes last.

### The 120-participant event has no dungeon map

The mode table settles this. Coagulation is the only mode that reaches a hundred and twenty participants, and it carries no map. Cementation reaches sixty and Descension reaches six, and those two are the modes with a map rail.

```
Fixation                    1 participant          no map
Coagulation                 up to 120              no map
Descension                  1 to 6                 map
Cementation                 2 to 60                map
```

The Old One Avatar the operator wants in a 120-participant event therefore fights in an arena, and the dungeon rail sits beside a roster of at most sixty. The Avatar art gets an arena background first.

### Three names mean two things each

```
tier         five trophy rarities, five loot tiers, and a rank a mode answers
Mercury      a planet, a Paracelsian principle, and the metal quicksilver
silver       the Silver Mirror's metal, and the Albedo trophy stage's colour
```

The brief reports these three and renames none of them. Every instruction in it names the thing instead of the number, so no artist has to guess which five is meant.

### What the art brief does not do

It draws nothing and it commissions nothing. Nobody contacted an artist, no sprite exists, and the tab still draws three empty zones with a sentence in each one saying no pixel art is drawn.

```
tests/debug_reports/2026-09-10_poa_art_brief.md    the unit's own record
```

## 2026-09-10 01:15 - #147 - the skill ladder and the transfer skill

The tab now draws a skill ladder of ten levels under the classes, and the first skill on it is the one that moves Quintessence between participants. The ladder is what sets the loss on a transfer.

### Ten levels, each one 2.5 times the one below

> *"All skills have levels that grow through use. Skills should be intelligently*
>
> *designed with growth curves similar to Eve Online."*

The ladder counts uses, never elapsed time. The first level costs one use and every level above it costs two and a half times the level below, so the last level alone is most of the climb.

`src/competition/skill_ladder.py` — the whole curve

```
FIRST_LEVEL_COST = Decimal(1)
LEVEL_COST_MULTIPLIER = Decimal("2.5")


def cost_of_level(level: int) -> Decimal:
    """Return the quality-weighted uses ``level`` itself costs."""
    trained = _as_level(level, FIRST_SKILL_LEVEL)
    return FIRST_LEVEL_COST * LEVEL_COST_MULTIPLIER ** (trained - FIRST_SKILL_LEVEL)
```

The program prints the ladder it builds. Level ten costs 3,814.7 uses on its own, which is 60 per cent of the whole climb.

```
level   cost of that level   cost to reach it   effect   bleed
 1                1.0                1.0        1.1x    8.00%
 2                2.5                3.5        1.2x    7.56%
 3                6.25               9.75       1.3x    7.11%
 4               15.625             25.375      1.4x    6.67%
 5               39.0625            64.4375     1.5x    6.22%
 6               97.65625          162.09375    1.6x    5.78%
 7              244.140625         406.234375   1.7x    5.33%
 8              610.3515625       1016.5859375  1.8x    4.89%
 9             1525.87890625      2542.46484375 1.9x    4.44%
10             3814.697265625     6357.162109375 2.0x   4.00%
```

An untrained skill is worth 1.0 and a maxed one is worth 2.0. The step is a tenth a level and nothing else changes it.

### Topping out takes 6,358 uses at full quality

The design says about 6,357 quality-weighted uses. The ladder adds up to 6,357.162109375, and a run that records perfect uses until the level stops rising reaches level ten on the 6,358th.

```
topped out: weighted_uses 6358.0   level 10   effect 2.0
cost_to_reach(10)                  6357.162109375
at quality 0.5: weighted_uses 6357.5   level 10
```

Half-quality uses cost twice as many of them for the same level. That is the whole point of weighting a use by its quality.

### A worthless use is not recorded at all

A use carries its own quality between nought and one. A use of nought advances nothing, and the program says so instead of quietly adding zero.

```
a use of quality 0.0 advances nothing; Quintessence Transfer stands at
level 0 on 0 weighted uses
```

Two more refusals guard the same field. A quality above one and a true-or-false value are both turned away by name.

```
quality must be 0 to 1, got 1.5
quality must be int, float or Decimal, not bool
```

### Nothing yet tells the ladder how good a use was

The ledger records that a transfer happened. It records no quality for it, so nothing can weight a use today and every skill stands at level 0. The panel prints that plainly.

```
No field holds a use's quality, so no use is weighted and the skill stands
at level 0.
```

The only nought-to-one quality the platform computes is the trade grade, and that grades a trade rather than a transfer. Unit 10 owns the grade.

### The transfer skill sets the bleed, from 8 per cent to 4

> *"Quint is transferable between players via a specific skill isolated to common*
>
> *Guild members, takes significant time to complete based on amount and skill*
>
> *level, and has a negative effect of 'bleeding' quint back into the 'platonic'*
>
> *where it can be respawned and redistributed to other PoA participants."*

The ledger already charged a bleed that falls with a level. This skill is where that level now comes from, and a skill below level one runs no transfer at all.

`src/competition/skill_ladder.py` — the gate and the rate

```
def transfer_bleed(progress: SkillProgress) -> Decimal:
    """Return the bleed ``progress``'s level pays, gated by ``transfer_level``."""
    return bleed_fraction(transfer_level(progress))
```

An untrained skill is refused in the program's own words, and that sentence is what the panel prints.

```
Quintessence Transfer stands at level 0 on 0 weighted uses; level 1 costs 1
and no transfer runs below it
```

### Two real transfers, and the books balance after both

One hundred Quintessence was sent at level one and again at level ten, on a throwaway ledger file. The bleed fell by half and the four buckets still add up to every unit ever distilled.

```
level 1    sent 100   received 92.00   bled 8.00   bled / sent 0.08
           wallets 992.00 + held 0 + pleroma 8.00 == minted 1000
           delta 0.00   balanced true   negative buckets 0

level 10   sent 100   received 96.00   bled 4.00   bled / sent 0.04
           wallets 988.00 + held 0 + pleroma 12.00 == minted 1000
           delta 0.00   balanced true   negative buckets 0
```

The same file replayed from disk balances to the same three figures. Nothing about the conservation law changed in this unit.

### The hours follow the amount and the level. The guild reads nothing.

Four terms govern a transfer. This unit builds two of them, leaves one as arithmetic with no caller, and finds nothing for the fourth to read.

| Term | State |
|---|---|
| gated by a skill | built - an untrained skill is refused |
| lossy | built - 8 per cent at level one, 4 per cent at level ten |
| slow | the program answers the hours; no clock holds a transfer open |
| guild members only | nothing exists to check |

The duration follows the amount and the level, floored at one hour.

```
1000 Quint at level 1     100 hours
1000 Quint at level 10     10 hours
5 Quint at level 10         1 hour, the floor
```

No guild, guild roster or membership record exists anywhere in the platform, so the guild term of a transfer has nothing to read. Nothing holds one transfer in flight at a time either, because no queue and no in-flight record is kept.

### Demo mode reads the other chain's ledger file

The chain rides in at construction, as it does for the wallet. Both runs build the same panel from the same module and read a differently named ledger file, so the transfer count differs and nothing else does.

```
chain live      transfers sent 1    quintessence_ledger.json
chain testnet   transfers sent 3    quintessence_ledger_testnet.json
```

With the live ledger file removed the same panel prints two dashes rather than a nought, so an absent file never reads as a participant who has sent nothing.

```
live, file removed   transfers sent --
```

### What the skill system does not reach

A Send Quint button now starts a transfer, and the section dated 17:40 below records it. The rate and the hours are still read out rather than chosen.

Only one skill exists. The rest of the tree, the alignment each skill carries and the abilities along the level arc are all later work.

No store holds a participant's weighted uses, so a level cannot survive a restart.

In development.

## 2026-09-10 01:47 - #147 - governance, the franchise, and the halt council

The vote now exists. Four issue levels decide what a change needs, a holding buys access to a level and never buys weight inside it, and the three owner keys the last piece of work left behind are answered one by one.

```
level            holding   quorum  approval  delay
INFORMATIONAL        1 Q      10%   simple    none
PATCH               25 Q      20%   simple    2 days
INTERFACE           75 Q      30%      60%    7 days
CORE               150 Q      40%      67%   30 days
```

The delay is the time a proposal stays open before anybody can run it. An informational vote runs the moment it has the turnout and the agreement. A change to a rule holders rely on waits a month, whoever proposed it.

### A large holder and a small holder count the same

This is the rule that could have gone wrong quietly, so it is driven rather than described. One holder carries ten thousand of the currency. Another carries one hundred and fifty, which is the bare minimum for the top level. Each adds exactly one to the tally.

```
holder        holds      adds to the tally
large     10,000 Q                      1
small        150 Q                      1
```

The proof that matters is the other direction. On a vote where the large holder is against and the small holder is for, the count is one against one and the vote does not pass. A third small holder then votes for, the count is two against one, and it does pass. Two small holders outvote one holder sixty-six times their size.

```
large against, small for          1 - 1   refused
large against, two small for      2 - 1   passes
```

### Turnout counts people, not coins

Turnout is a fraction of the addresses that may vote at that level. It never looks at how much currency exists. Three addresses qualify for the top level while ten thousand three hundred and seventy-five units sit behind them, and the two numbers have nothing to do with each other.

```
addresses at the top level        3
units distilled          10,375 Q
```

A control proves it. Distilling a million more units to the largest holder raises the supply ninety-seven fold and leaves the turnout figure at three, because one address is one address however much it holds.

### Two separate gates, and trading opens only one of them

A holding decides which levels an address may vote on. Playing decides whether that address has a live vote at all. These are different gates and only the second one is about activity.

```
holdings       which levels an address may vote on
playing        whether the vote is live at all
```

An address that certifies trades for a year and never enters an event qualifies on its holdings and holds no vote. Driven both ways: a trade leaves the activity clock exactly where it was, and one action inside an event moves it.

```
a certified trade credited          the clock does not move
one action inside an event          the clock moves to now
a holding with no event action      reaches no level at all
```

### The franchise is a second number, and it moves nothing

Beside the balance sits a remembered maximum. It is never below the balance, for every address, always. Three behaviours, each driven:

```
the balance rises     the franchise follows in the same block
the balance falls     the franchise stays where it was
the address goes      the franchise closes the gap slowly, and reaches
quiet                 the balance after ninety days
```

A balance of one hundred and fifty spent down to fifty leaves the franchise at one hundred and fifty. Ninety quiet days later nothing has changed. Forty-five days after that, half the gap has closed and the franchise reads one hundred. Ninety days after that it reads fifty, which is the balance, and it stops there.

```
day   0   balance 150   franchise 150
day   0   spend 100     balance  50   franchise 150
day  90   balance  50   franchise 150
day 135   balance  50   franchise 100
day 180   balance  50   franchise  50
```

Not one unit of currency moves while that happens. The books were read before a full ninety-day resynchronization and again afterwards, and all four totals are identical.

```
                 before        after
wallets      150.000 Q    150.000 Q
held         100.000 Q    100.000 Q
pleroma        0.000 Q      0.000 Q
ever minted  250.000 Q    250.000 Q
```

One action inside an event stops the slide and keeps what has already closed. An address that had fallen to one hundred stays at one hundred. Nothing is handed back, and nothing is taken.

### The halt council can only halt

Five addresses are elected by the same one-vote-each franchise. Three of them stop one named mechanism, and that is the only thing any of them can do.

```
council size                5
signals that stop something  3
a halt lasts           7 days
```

Driven: one signal does nothing, two signals do nothing, the third stops the award token and a transfer is refused. The same member then tried four other things and was refused every time.

```
open a proposal            refused, the member holds no currency
set a price feed           refused, that answers to the vote
rewrite a tier's artwork   refused, that answers to the vote
halt the entry currency    refused, it is not a haltable mechanism
halt the award token       allowed, and that is the whole list
```

Nothing releases a halt, because there is no release. The contract asks what time it is and stops answering after seven days. One second before the seventh day the token is still frozen; one second later a transfer goes through, and nobody called anything in between.

```
seven days less one second   frozen
seven days exactly           moving again
calls made in between        none
```

A council that has stopped something cannot stop the same thing twice. Renewing a halt therefore needs a vote that elects a different council, which is the point.

### Migration, never a replaceable contract

Nothing deployed here can be edited later. The usual way round that is a contract whose logic can be swapped, and that is exactly what this design refuses, because whoever can swap it can rewrite every rule including the cap.

A change no vote can make is made by deploying a new contract and pointing a top-level vote at it. Holders then move themselves across.

```
what a holder who moves does   spends their balance into the migration address
what a holder who ignores it   keeps every unit on this contract, and reaches
loses                          nothing the new contract governs
```

That cost is real and it is stated rather than discovered. An address that never acts keeps its currency and is left behind. The books still balance, because the units a holder moves are retired rather than destroyed.

### The twelve owner keys, answered

The previous piece of work removed two owner powers and left twelve on record as waiting for the vote. Each is now answered. Three moved behind the vote, five were removed outright, and the remaining six run the game every season, so a vote with a delay measured in days would stop the game rather than govern it.

```
moved behind the vote
  set a price feed    a new market at INTERFACE, repointing one at CORE
  upload tier art     at INTERFACE, and the first upload per tier stays a
                      deployment step so a deployment can mint at all

removed
  freeze the token    the halt council does this now, and it expires
  unfreeze the token  nothing releases a halt, so no release exists
  hand over the token key
  hand over the registry key
  hand over the trophy key
                      all three addresses are now fixed at deployment and
                      cannot move to anybody

left privileged, and why
  open a competition        runs every competition
  activate a competition    runs every competition
  close for submission      runs every competition
  adjudicate                ranks submissions, which no vote can do
  advance the season        runs every season
  cancel a competition      the emergency stop on one competition
```

The six that stay sit on one address written at deployment that can never move, and the halt council can freeze all six for seven days. The plan the contract already names for adjudication is on-chain proof of the result, which is separate work.

Repointing an existing price feed changes what every award already measured against that market is compared to, so it runs at the top level. Pointing a market that has no feed at one is an addition and runs a level lower. The contract decides which of the two it is rather than trusting the proposer.

```
a market with no feed      INTERFACE, 7 days
repointing a seeded market CORE, 30 days
a holder at INTERFACE proposing the repoint    refused
```

### What the tools reported

Each of the four tools was shown reporting a planted fault in a throwaway file before its clean verdict was believed, and the throwaway file was deleted afterwards. mythril is not installed on this machine and nothing was substituted for it.

```
                    on the plant              on the contracts
forge test          1 failed                  43 passed, 0 failed
slither             1 high, 1 medium          0 high, 0 medium
solhint             2 errors, run fails       0 errors, run passes
semgrep             1 blocking, run fails     0 blocking, run passes
```

The fuzzing runner drove the franchise rule and the currency rule together.

```
runs       256
calls   16,384
the franchise never below the balance        holds
a governance call never moves currency       holds
the four buckets still add up                holds
```

Two findings stand, both on the governance contract and both for the same reason the award contracts already carry. The delays, the quiet period and the seven-day halt rest on eleven readings of the chain's clock; the shortest of them spans two days and a validator can shift the clock by seconds. Six naming complaints are a disagreement between two tools: the compiler's own linter requires the style the other tool objects to, and the tree already follows the compiler.

### What governance does not reach

Nothing calls the activity clock yet. The action budget that will charge for an action inside an event is not on the chain, so the one caller allowed to refresh an address's clock has no code calling it today. Until it does, a live net would see holdings qualify and no vote go live.

The turnout figure counts addresses as of their last refresh, and any address may refresh any other at no cost. A roster nobody has refreshed lately gives a stale turnout figure rather than a wrong one, and the contract still checks every vote against the voter's real franchise at the moment they cast it. ## 2026-09-10 01:47 - #147 - the mark slot on the party row

Every party row now carries one mark slot on its right-hand edge. The slot shows a single mark at a time, and it shows nothing at all when the participant is in good order. The row also prints its market beside the identifier, which is the first readable thing a party row has ever carried.

### A bot has no name, so the row shows its market

The design asked for a truncated name. No name exists to truncate. Every bot record was read through the loader the tab already uses, and not one of the thirty-eight carries a field holding a name or a label.

```
bots read from the fleet file                      38
bot id length                                      8 characters
config keys containing "name" or "label"           0, across all 38
the only readable field on the record              symbol, for example A26/USD
```

The identifier is therefore shown whole rather than truncated, because eight characters is the whole of it. The market symbol now sits next to it and carries the meaning. A row reads as its market, not as a string of hexadecimal.

```
04e1cafc   A26/USD      $54.19
092428b2   A10/USD    $101.98
168b78e3   A19/USDC    $67.30
45e9e720   A04/USDC   $134.05
```

### Six marks, ranked, and the slot shows only the first that holds

The slot holds one mark and never two. When several conditions are true at once the most urgent wins and the rest are not drawn.

`src/gui/main_tabs/proof_of_accumulation_tab_surface.py` — the ranking

```
MARK_RANKS: tuple[str, ...] = (
    MARK_DEAD,
    MARK_MISSED_WINDOW,
    MARK_OUT_OF_IMPETUS,
    MARK_AFFLICTED,
    MARK_NO_CLASS,
    MARK_ALIGNMENT_SKEW,
)
```

Death outranks everything because it ends the participant's turn. A missed window and an empty Impetus pool come next, as they cost an action. An affliction is a condition a participant can act through. No class picked is a setup step rather than a fault. The alignment lean sits last because it names no fault at all.

### Two marks read real state today

A dead participant is one the program can find no health figure for. An unclassed participant is one with no class chosen for the running event. Both are read off values the tab already computes.

```
dead              the health figure is absent        paints #ff5577
no class picked   the class reads as none            paints #555555
```

The running page was read back after it drew. Thirty-seven of the thirty-eight rows carried the unclassed mark and the one with a class carried none.

```
rows drawn          38
mark slots drawn    38
rows marked         37
rows unmarked        1
```

### Two conditions at once still show one mark

A participant with no health and no class satisfies both the first rank and the fifth. The slot drew the first and dropped the other.

```
aaaa1111  $120.00  Lead Ward   no mark        transparent
bbbb2222  $75.50   none        no class picked  rgb(85, 85, 85)
cccc3333  --       none        dead             rgb(255, 85, 119)
```

The third row is the proof. It is unclassed as well as dead, and only the dead mark reached the screen.

### Four marks have nothing to read, and the screen says so

The other four conditions have no value behind them anywhere in the program. No mark is drawn for them, because a mark wired to a value that is always absent would read as working. The party window prints what each one is waiting for.

```
4 of 6 marks have no state to read, so no slot draws one: missed the window
waits on unit 13's action record, one a participant an event; out of Impetus
waits on unit 13's action record, one a participant an event; afflicted waits
on no unit; the art brief's tier 3 decans; alignment skew waits on no unit;
the art brief's alignment score.
```

Two of the four wait on the action record that unit 13 writes, one per participant per event. The other two are named only in the art brief and no unit is assigned to either.

### The marks draw on the demo chain too

The demo chain takes the same code path. Nothing switches on a flag; the panel is given a chain name when it is built and the surface reads that chain's own fleet file.

```
panel chain at construction   testnet
the chain the page carries    testnet
the fleet file read           bot_state_testnet.json
dddd4444  DDD/USD  no class picked  rgb(85, 85, 85)
eeee5555  EEE/USD  dead             rgb(255, 85, 119)
```

### What the mark slot does not reach

The slot is a coloured square, not a picture. The six glyphs the art brief specifies are a commission that has not been made, so each mark is drawn as its colour and names itself to a screen reader.

No health bar and no role colour exist on the row yet. Both are listed in the art brief and neither has a token behind it. ## 2026-09-10 01:59 - #147 - the per-action spend and the record store

An action inside an Elite Event now costs Quintessence, and the cost comes off a real balance. Five prices stand on one curve, the cheapest a hundredth of the dearest, and every charge reaches the one spend path the ledger already had.

### Five bands, a hundred to one

The cheapest action is a move and the dearest is a multi-turn spell. Each band costs about three times the one below it, and the program prints the whole curve when it starts.

```
band   cost     what sits in it
x1     0.001    move, switch weapon, take an item from a bag
x3     0.003    a basic attack, a basic heal
x10    0.010    a class ability on a cooldown
x30    0.030    a group-wide ability, a threat move across the field
x100   0.100    a multi-turn spell, and the decisive tactics beside it

ratio dearest/cheapest = 100
```

A hundred to one is the ratio the design sets. Two hundred moves cost 0.2 and one decisive cast costs 0.1, so a guild argues about the expensive action without it ruining anybody.

### The books balance after every one of the five

A spend takes Quintessence out of a wallet and rests it at the event's held address. The spend creates nothing and destroys nothing, so the four buckets still add up to the total ever distilled after each of the five.

```
x1    cost 0.001  actor 1.00  -> 0.999  pot 0.001   balanced True  delta 0.000
x3    cost 0.003  actor 0.999 -> 0.996  pot 0.004   balanced True  delta 0.000
x10   cost 0.010  actor 0.996 -> 0.986  pot 0.014   balanced True  delta 0.000
x30   cost 0.030  actor 0.986 -> 0.956  pot 0.044   balanced True  delta 0.000
x100  cost 0.100  actor 0.956 -> 0.856  pot 0.144   balanced True  delta 0.000
```

The run asks the record store and the ledger one question two different ways at the end, and both give the same answer. One adds up what the records hold as paid and the other reads the held balance.

```
store event_spent 0.374   ledger pot 0.374   agree True
```

### The money moves when the spell lands, not when it starts

A powerful action can occupy more than one turn. The charge opens, the turns pass, and the cast happens on the last of them. The wallet keeps every unit until that moment.

The rule that no partial action exists decides this. A charge that never finishes produced no action, so charging for it would take money for nothing, and the ledger can move nothing back out of a held address.

`src/competition/action_spend.py` — opening a charge debits nothing

```
    def begin_charge(
        self, draft: ActionDraft, turns: int, opened_turn: int
    ) -> ActionCharge:
        """Open ``draft`` over ``turns`` turns, refusing one its payer cannot afford.

        Nothing is debited here; ``complete_charge`` is the only write path.
        """
```

The run interrupted a charge and the balance did not move. The charge opened on turn 10 over three turns, the program refused an early cast, the charge then went away, and the wallet held the same amount throughout.

```
begun  x100 turns 3 casts turn 12
actor after begin_charge   0.856 (was 0.856)
REFUSED mid-charge: casts on turn 12 and it is turn 11; no partial action exists
actor after abandon        0.856
record actions before 5    after 5
open charges 0
```

Carried to its own cast turn the same charge paid in full. The wallet either loses the whole cost or keeps the whole cost, and no half state exists.

```
cast on turn 22: cost 0.100, actor 0.856 -> 0.756   balanced True
```

### The program refuses an action nobody can afford

A participant holding 0.050 cannot take the dearest action. The program names the balance and the band's cost, and then the same participant takes a cheaper action that does fit.

```
poor holds 0.050
REFUSED x100: bot-poor-0004 holds 0.050 Quintessence and band x100 costs 0.100;
no partial action exists and nobody borrows against the next turn
PAID    x30: cost 0.030, poor now 0.020
```

### One record a participant an event, written once and read twice

Each participant gets one record per event. It counts the actions, sums the cost of them, and stamps the second the participant last acted. Two different readers want it and neither writes it.

```
reader 1, the pot share:      spent 0.244  underwritten 0.100  actions 7
reader 2, the dormancy clock: last_acted_at 1700000300.0
```

The store lives beside the other runtime files, under the home directory the platform already uses, and never inside the repository.

```
~/.acervator/poa_record_store.json
```

### A level and a record survive a restart

The store holds the skill ladder's weighted uses as well. Four uses at 0.9 quality put the transfer skill on level two, the first process then exited, and a second process read the level and the record back off disk.

```
PROCESS A EXITING

PROCESS B pid=16428
a level:          level 2   weighted_uses 3.6   bleed 7.5556%
an action record: actions 7  spent 0.244  underwritten 0.100
                  bands x1 1, x3 1, x10 1, x30 1, x100 3
                  last_acted_at 1700000300.0
```

This closes the gap the skill ladder left open. A level now survives a restart, and the sentence in the earlier entry saying no store holds a participant's weighted uses describes the state before this store existed.

### A guild officer may pay, and the actor must agree

An officer commits treasury funds and the actor answers. Accepted, the treasury pays and the actor pays nothing. Refused, neither pays.

```
ACCEPTED   treasury 1.00  -> 0.900    actor 0.756 -> 0.756
           record underwritten 0.100  spent 0.244   balanced True
REFUSED    treasury 0.900 -> 0.900    actor 0.756 -> 0.756
           record actions 7 -> 7                    balanced True
```

Only the actor may answer. The program turns away by name an officer who tries to accept on the actor's behalf.

```
offer is addressed to bot-actor-0001 and bot-officer-0002 cannot answer it;
only the actor accepts
```

### What the officer check and the treasury still need

No guild exists in the platform. The underwrite takes the officer's name and the treasury's address as given, and checks neither.

Two things are missing and no unit on this issue builds either. A guild roster that maps a participant to a guild and a rank would answer whether the officer holds office. A treasury address that carries a spendable balance would let the commitment settle.

A treasury with nothing spendable draws the same refusal as any other payer who is short.

```
REFUSED: guild-held-only-0005 holds 0 Quintessence and band x10 costs 0.010
```

The design calls a treasury a held address, and the ledger can only spend from a wallet. Whichever unit builds guilds has to settle that, because the two readings cannot both be true of one address.

### Demo mode is the same code over a different chain

The ledger, the store file and the held address all arrive at construction, so the demo chain runs the identical method. Nothing switches on a flag.

```
live store poa_record_store.json           pot poa_elite_event_pot
demo store poa_record_store_testnet.json   pot poa_elite_event_pot_testnet
same class True   same method True

demo act x100 cost 0.100   demo pot 0.100   demo balanced True
live pot untouched by the demo act: 0.374
live store record actions 7   demo store record actions 1
```

### What the spend does not reach

A Spend a band button now starts a spend, and the section dated 17:40 below records it. The band it casts is the cheapest one.

Nothing divides the pot. Every unit a participant spends rests at the event's held address, and the redistribution by performance is its own unit.

In development.

## 2026-09-10 08:26 - #147 - the capture bounds and the grade curve

A market's Quintessence pool now exists, and four bounds stand between a trade and an award from it. One participant takes at most a twentieth of any one pool, once per activation period, and waits out three of their bot's own candles afterwards. A trade the platform could not grade earns nothing at all.

### A market's pool is sized once, from the volume it carried

The rotating reward set draws the markets. Each drawn market then takes a share of the activation's Quintessence in proportion to the dollars traded on it, read once and never refreshed.

`src/competition/capture_bounds.py` — the split

```
    ordered = sorted(volumes, key=lambda symbol: (-float(volumes[symbol]), symbol))
    shares: dict[str, Decimal] = {}
    for symbol in ordered[1:]:
        volume = _as_decimal(volumes[symbol], f"volume of {symbol}")
        shares[symbol] = total_emission * volume / total_volume
    shares[ordered[0]] = total_emission - sum(shares.values(), Decimal(0))
```

The largest book takes the division remainder, so the five pools add up to the activation's figure exactly. One real Coinbase draw, 10,000 Quintessence:

```
A15/USD    volume $404,975,607.15   pool 8849.2539   one participant's 5%  442.4627
UNI/USD    volume $ 16,254,055.62   pool  355.1727   one participant's 5%   17.7586
A29/USD   volume $ 16,048,824.74   pool  350.6881   one participant's 5%   17.5344
A25/USD    volume $ 12,378,533.61   pool  270.4874   one participant's 5%   13.5244
A36/USD    volume $  7,981,113.83   pool  174.3979   one participant's 5%    8.7199

the five pools sum to 10000.0000000000000000000000, the figure activated
```

### The cooldown counts the awarding bot's own candles

A bot's timeframe is a field on its own configuration, and the platform already measures how many seconds each candle holds. Three candles of that timeframe is the wait after an award, and fifteen minutes is the floor under it.

`src/competition/capture_bounds.py` — the two figures

```
#: Candles of the awarding bot's own timeframe that must close after an award.
COOLDOWN_CANDLES = 3

#: No cooldown runs shorter, so three 1m candles still wait out 15 minutes.
COOLDOWN_FLOOR_S = 900
```

Three candles is not always a length of time, so on the two fastest candles the floor decides instead.

| Bot timeframe | Three candles | The wait | What decides it |
|---|---|---|---|
| 1m | 180 s | 900 s | the floor |
| 3m | 540 s | 900 s | the floor |
| 5m | 900 s | 900 s | they agree |
| 15m | 2,700 s | 2,700 s | three candles |
| 1h | 10,800 s | 10,800 s | three candles |
| 1d | 259,200 s | 259,200 s | three candles |

An Elite event's turn is one 1m candle and every other turn is one 5m candle, so three turns of either come to the floor or under it. Both land on fifteen minutes, and nothing in the design shortens the wait below that.

```
  Elite turn candle 1m: 3 candles 180s, cooldown 900s
Standard turn candle 5m: 3 candles 900s, cooldown 900s

the live fleet: 38 bots, every one on a 5m candle, so every one waits 900s
```

A timeframe the platform does not measure is refused rather than given the shortest wait.

```
'7m' is not a candle the platform measures; the cooldown counts 3 candles of
the awarding bot's own timeframe, one of 1m, 3m, 5m, 15m, 30m, 1h, 2h, 4h, 6h,
8h, 12h, 1d, 1w
```

### A trade the platform could not grade earns nothing

The grader answers a half when no axis had inputs to score, and that half is a default rather than a reading. The award path now counts the axes that scored, and pays on none of them.

`src/trading/trade_grader.py` — the count the award path reads

```
        overall_numeric=round(overall_num, 4),
        scored_axes=len(sub_scores),
```

Driven on a real Coinbase fill with no reference price and no later prices, the grade reports the default and the award is refused.

```
no reference price, no future prices -> scored_axes 0  overall_numeric 0.5  overall D

the grade on this trade scored 0 of four axes, so its 0.5 is the default and
not a measurement; an award needs at least 1 scored axis
```

The same fill against the venue's own 24-hour opening price scores one axis and is paid.

```
FLOCK/USD fill 0.061947 against its 24h open 0.06214
  scored_axes 1   overall_numeric 1.0   overall A+   execution_bps -31.06

A36/USD awarded 1.0 Quintessence, cooldown 900s of 3 5m candles
```

### A grade standing on one clamped axis earns nothing

The accuracy axis runs out at one per cent. A fill a hundred basis points worse than its reference scores nothing, and so does one fifty per cent worse; a fill a hundred points better scores full marks, and so does one twelve per cent better. Past that distance the score is a clamp rather than a reading of the fill.

The distance on its own decides nothing, and the operator's own trading says why. Across 1,560 of his fills that carry a reference price, 1,368 of them — 87.7% — sit more than a hundred basis points from it, and the middle fill of the set is 552 basis points away on the favourable side.

```
fills carrying a reference        1560
  favourable, nearer than refused 1246   79.9%
  adverse                          299   19.2%

past a hundred basis points       1368   87.7%
  favourable                      1154   74.0%
  adverse                          214   13.7%

the signed spread, basis points
  lowest  -5030.57    middle  -552.35    highest  3253.80
```

A Scrum sells above its earlier fills and a Fold buys below them, so a large favourable gap is the strategy working rather than a lucky fill. Refusing on distance alone would refuse seven awards in eight.

What cannot be trusted is a grade with nothing else in it. When accuracy is the only axis that scored and it has clamped, the grade is exactly 1.0 or exactly 0.0 and carries no reading at all, because a reference price that far from the fill is stale. That case is 35 of his 1,560 fills, 2.2%: twenty-eight at a flat 1.0 and seven at a flat 0.0.

`src/competition/capture_bounds.py` — the distance, and what the measurement says about it

```
#: Basis points past which the execution axis clamps and stops reading the fill.
#: Measured on 1,560 live fills: 87.7% sit past it, so distance alone is no bound.
EXECUTION_READABLE_BPS = 100.0
```

One of his own fills, refused, and one paid.

```
REFUSED  2026-09-10 05:23  A06/USD buy at 0.0052147215059309
         bps -1439.06   accuracy 1.0   axes 1   grade 1.0   A+
         this grade scored execution and nothing else, and its reference price
         sits -1439.1 basis points from the fill, past the 100 the axis reads;
         a reference that far out is stale, so the one axis reports a clamp and
         the grade of 1.0 rests on nothing

PAID     2026-09-10 03:22  A06/USD buy at 0.005468
         bps -1023.25   accuracy 1.0   axes 2   grade 0.5   D
         0.5 Quintessence
```

The second fill is further from nothing and further from a clamp: a second axis scored, so the grade is half rather than full marks, and the award is half. The grade itself is unchanged either way, so every screen that reads a letter still reads the same letter.

### Each bound refuses, and each one pays when it should

Every line below came out of the program on real Coinbase pools. The permitting case sits beside each refusal, because a bound only ever seen to refuse proves nothing.

```
one allotment a participant a market an activation period
  REFUSED  participant-scored already took an allotment of A36/USD in
           coinbase:1; one allotment a participant a market an activation period
  PAID     participant-second on that same market, 1.0 Quintessence

at most a twentieth of a market's pool
  REFUSED  9.7198959401318135277484921 05 Quintessence is above the
           8.7198959401318135277484921 05 ceiling on A36/USD, which holds a pool
           of 174.39791880263627055496984 21; one participant takes at most 5%
           of a market's pool
  PAID     8.7198959401318135277484921 05 Quintessence, exactly the ceiling

three candles, floored at fifteen minutes
  REFUSED  participant-scored has 1s left of a 900s cooldown of 3 5m candles;
           the gate pays nothing until it clears
  PAID     the same participant on UNI/USD the second the cooldown clears

the pool runs out
  REFUSED  allotment_exhausted, on a request for the full ceiling against
           5E-25 of A25/USD's 270.4873712360626776914103408 pool
  PAID     twenty earners in turn, each taking the full 13.52436856180313388
           ceiling
```

Twenty distinct earners empty a pool, which is what the share ceiling is set to produce. The twenty-first is refused, and the five hundred-thousand-billionths left over are the tail of a twenty-eight digit division rather than a prize.

### Demo mode answers the same bounds

A TestNet run builds the same object over its own chain and takes the same path through it. Neither the pools nor the bounds are chain data, so the two runs differ only in which chain carries the record.

```
chain live      5 market allotments   25 CaptureBounds records   QuintessenceAwarded
chain testnet   5 market allotments    1 CaptureBounds record    QuintessenceAwarded

the testnet run's second award on one market
  participant-scored already took an allotment of LIGHTER/USD in coinbase:1
```

### What the bounds do not reach

No certified fill asks them. A fill carries its venue, its fee and its grade, and it carries no exchange, no season and no market pool, so the socket that mints Quintessence cannot yet name the activation an award would come out of. The bounds are built, loaded at every launch and driven, and nothing in the live award path consults them.

The Quintessence one activation emits is not a figure the design sets. The split across markets is fixed and the share of it one participant may take is fixed, so the figure is the caller's to supply until a number is chosen.

How much longer a large award waits than a small one is also open. Three candles is the floor the design gives, and the gate holds there.

In development.

## 2026-09-10 09:47 - #147 - a certified fill names its activation

A fill offered for certification now says which exchange it traded on and which season it traded in. Those two facts name the activation period, and the market symbol names the pool inside it, so the socket that mints Quintessence can find the pool an award would come out of. Every bound built yesterday is now asked before a single Quintessence is minted.

### The fill carries the exchange and the season

Five fields were added to the record one fill is offered as. Two of them name where the award comes from and three are what the bounds read off the trade.

`src/competition/certification_socket.py` - the record

```
    exchange_id: str = ""
    season: int | None = None
    scored_axes: int = 0
    ta_timeframe: str = ""
    execution_bps: float | None = None
```

A fill that carries neither an exchange nor a season names no activation, and the program says so rather than guessing one.

```
fill 04e1cafc:2026-09-10T07:21:39 on A26/USD names no activation, so no market
pool pays it

award_reason : no_activation_named
distilled    : 0 Quintessence
```

### Certification asks the bounds before it mints

The bounds arrive when the socket is built, the same way the chain and the ledger do. A fill the bounds refuse is still signed, logged and counted towards the bot's lifetime fee, and it mints nothing.

`src/competition/certification_socket.py` - the one place the mint is decided

```
            distilled = (
                self._ledger.distil(participant, fee, grade)
                if award_reason == AWARDED
                else Decimal(0)
            )
```

The screen's own install line now says whether awards are bounded at all.

```
CertificationSocket installed (path=..., bots=0, awards bounded=True)
```

### The season comes from the chain, and nothing moves it

A season is a counter on the chain, not a date. The running program reports it, and it has read one since the day it was written.

```
LocalTestnet.get_competition_stats() current_season = 1
```

`contracts/CompetitionRegistry.sol` holds the only thing that advances it, and nothing in the Python calls an equivalent, so every activation today is season one. Governance is the unit that turns that call into a vote.

### A trade that can pay nothing is refused, not paid nothing

A fill with no fee distils nothing. Granting it anyway would spend the participant's one allotment of that market and start their cooldown for no Quintessence at all, which is a pure loss to them and hides the missing fee.

`src/competition/capture_bounds.py` - the new bound

```
        if amount == 0:
            raise CaptureRefusedError(
                NOTHING_TO_AWARD,
                f"a fee of {request.fee_usd} at a grade of {request.grade_numeric} "
                f"distils nothing, and granting it would spend {participant}'s one "
                f"allotment of {symbol} and open a cooldown for no Quintessence",
            )
```

One real fill, driven twice. The venue's fee is the only difference between the two runs.

```
A25/USD sell at 0.8223, two axes, grade 1.0

no fee on the payload   nothing_to_award   0 Quintessence
the venue fee supplied  awarded            1.00 Quintessence
                        pool 817.1556  ceiling 40.8577  cooldown 900s
```

### Every bound, driven on his own fills

1,709 entries were read from the live trade log and 1,668 of them graded. Each refusal below is the program's own sentence.

```
not_activated
  A26/USD holds no Quintessence allotment in coinbase:1, so there is no pool
  for an award to come out of

sole_axis_clamped
  A23/USD sell at -646.3 basis points, one scored axis: a reference that far
  out is stale, so the one axis reports a clamp and the grade of 1.0 rests
  on nothing

nothing_to_award
  a fee of 0.0 at a grade of 1.0 distils nothing

no_activation_named
  the fill carries no exchange and no season

awarded
  A23/USD sell at -80.3 basis points, two axes, grade 0.5
  0.50 Quintessence of a 5902.5851 pool, ceiling 295.1292, cooldown 900s
```

The first refusal is the live configuration speaking. Coinbase holds six eligible markets today against a floor of twelve, so no activation can open, so no pool exists, so every real fill is refused at the first bound. That is the age rule working and the economy not starting, which is already on the page above.

### The fee a fill should be measured on is not on the fill

The live fill event carries a side, an amount and a price. It carries no fee, and the pinned log schema carries none either, so all 1,709 entries read zero. The venue does report a per-trade fee, and the platform already sums it per bot into the figure the status screen shows, but nothing puts it on the individual fill.

Until it does, every certified fill distils nothing and the refusal says which number is missing. Supplying a per-fill venue fee to the fill event is the unit that finishes this.

### The figure that sets the scale of the economy

How much Quintessence one activation emits is still nobody's number. It is the caller's argument, and passing the absence now refuses by name rather than quietly becoming a zero.

`src/competition/capture_bounds.py` - the absence, stated

```
#: No figure sets how much Quintessence a market's pool holds in a window, so
#: ``activate`` takes it as an argument and refuses this None.
EMISSION_PER_ACTIVATION = None
```

```
emission_not_set: no emission was named for this activation, and no figure here
sets how much Quintessence a market's pool holds in a window; activate takes it
as an argument and no default stands in
```

### Demo mode certifies against a second chain

One class, two chains, one award path, no flag. The socket takes its chain and its bounds when it is built, so a TestNet run is a second set of the same three objects over a second chain.

```
the same class        : CertificationSocket and CertificationSocket
the same chain object : False

live chain   4 CaptureBounds records
demo chain   1 CaptureBounds record

on the demo chain
  A29/USD sell, two axes, grade 1.0 -> awarded, 1.00 Quintessence
  pool 964.6291  ceiling 48.2314  cooldown 900s
```

### What still does not reach this

Nothing subscribes the socket to the live fill event. The object is built on every launch and its subscription method has no caller, so no real trade reaches certification while the platform runs. Wiring it would start minting against the live Quintessence ledger on every fill, which is a decision about real value rather than a repair, so it is named here and left for the unit that makes it. ## 2026-09-10 10:07 - #147 - the pot divides on performance, and the remainder rests

An Elite Event ends and the Quintessence its participants spent is divided back to them. Seventy-five per cent of the pot returns. The share each one takes is their own performance score over every score in the event, and nothing in that sum reads what anybody spent.

```
RETURN_PERCENT = 75

return_pool_units = pot_units * RETURN_PERCENT // 100
amount_units = int(return_pool_units * own / total_score_ratio)
```

The grain is the ledger's own. `QUINTESSENCE_UNITS_PER_WHOLE and QUINTESSENCE_MINIMUM_UNIT` arrive by import, so the division cannot divide a Quintessence more finely than a wallet can hold one.

```
from .quintessence_ledger import (
    QUINTESSENCE_MINIMUM_UNIT,
    QUINTESSENCE_UNITS_PER_WHOLE,
    amount_text,
)
```

### The top spender performed worst and took nothing

Three participants distilled four Quintessence each and acted in one event. One bought the dearest band eight times, one bought the cheapest band once. The real trade grader graded three real fills, and the grades run the other way from the spending.

```
                spent    actions   grade   axes   payout
big_spender     0.800          8   F 0.0      4   0
middle          0.030          3   C 0.5748   4   0.22748545
small_spender   0.001          1   A+ 1.0     4   0.39576454
```

The participant who put 0.800 of a 0.831 pot in took nothing back. The one who put 0.001 in took the largest payout. Dividing by spend would have paid the first one almost everything.

### A grade the platform could not compute takes no share

A fill with no surrounding price data scores no axis, and the grader answers 0.5 as its default. That 0.5 is not a measurement, so the division refuses it a share, the same refusal the capture bounds already make on an award.

```
alpha   grade 0.8008   axes 2   share 0.44469124   0.10372423
beta    grade 1        axes 2   share 0.55530875   0.12952576
gamma   grade 0.5      axes 0   no scored axis, so no share
```

### The books balance through the payout, and no bucket goes negative

The four buckets and the supply are read before the payout and again after it. Quintessence moves from the pot into wallets and none is made or lost.

```
before   wallets 2.689                 held 0.311                 pleroma 0
         minted 3   delta 0   balanced True   negative buckets 0
after    wallets 2.92224999            held 0.07775001            pleroma 0
         minted 3   delta 0   balanced True   negative buckets 0
```

### The pot equals the payouts plus the reserve, exactly

This division does not divide evenly. The return pool is 0.23325 and the two shares come to one minimum unit less. That unit is not dropped and not rounded away; it joins the quarter that never left and rests at the pot address as the reserve.

```
pot                    0.311
return pool, 75%       0.23325
paid to participants   0.23324999
division remainder     0.00000001
reserve                0.07775001

0.23324999 + 0.07775001 = 0.311
```

The reserve is also what the pot address still holds, so the figure on the screen and the figure on the chain are the same figure.

### A settled event is refused a second payout, across a restart

The store is stamped before the ledger moves. One program run paid the event and exited. A second run read the same files back and refused to pay again.

```
SETTLED AT live 1700000500.0
SECOND SETTLE REFUSED live raid was settled at 1700000500.0; a second payout
  would take Quintessence the pot no longer rests
```

### The page draws it, on both chains

The Accumulation page carries a Redistribution panel under the skill ladder. The page was drawn and its own text read back.

```
chain live      3180 characters   Pot 0.311   reserve 0.07775001
chain testnet   3188 characters   Pot 0.311   reserve 0.07775001

quintessence_ledger.json           poa_record_store.json
quintessence_ledger_testnet.json   poa_record_store_testnet.json
```

Demo mode is the same code over a different chain. The panel takes its ledger, its store and its pot address at construction, so the demo run is one more object over its own files and there is no flag anywhere in the division.

### One sentence on the skills panel was corrected

The panel said no field holds a use's quality. The record store now holds each use's quality as weighted uses, so the sentence was wrong on screen. It now reads:

```
The record store holds each use's quality as weighted uses, and this panel reads
the participant's own. The Train transfer control records one, and an untrained
skill stands at level 0.
```

The panel also reads the participant's own standing out of the store rather than building a fresh one, so the level it prints is the level the store holds.

### What the redistribution does not reach

A Settle the pot button now starts a payout, and the section dated 17:40 below records it. A second press is refused.

Nothing writes a performance score during live play. The score is the trade grade, read through the RPG conversion, and the writer is reached only from inside the redistribution itself. A certified fill now carries both its grade and its count of scored axes, so the figures a share needs exist, and `record_certified_fill` now puts them on a participant's event record.

No guild exists, so a treasury's spend sizes the pot under the actor's own address and no officer is checked.

In development.

## 2026-09-10 10:15 - #147 - the loot system

A qualifying market now drops loot, the wallet holds it, and each item augments a tournament action. Loot has its own ERC-1155 contract, and the JSON helpers the trophy contract carried now sit in a library both contracts share.

### The five tiers

Each tier is named for what its alchemical operation leaves behind. The weights are percentages and they add up to a hundred exactly.

`src/competition/loot_drop.py` — the table

```
LOOT_TIERS: tuple[LootTier, ...] = (
    LootTier(CALX, "Calx", Decimal(60), 0, Decimal(2)),
    LootTier(CAUDA_PAVONIS, "Pavonis", Decimal(25), 0, Decimal(5)),
    LootTier(FLORES, "Flores", Decimal(11), 1, Decimal(10)),
    LootTier(ELIXIR, "Elixir", Decimal("3.5"), 1, Decimal(20)),
    LootTier(MAGISTERIUM, "Magisterium", Decimal("0.5"), 2, Decimal(50)),
)
```

The long name of the second tier runs thirteen characters, over the twelve a row holds, so the table carries `Pavonis` as its short form and the wallet prints that. A short form longer than twelve characters is refused.

### A weight never becomes a fraction

Every weight is a decimal, and the draw is a whole number rather than a fraction of one. The most decimal places any weight carries is one, so the weights scale by ten and the draw runs over a thousand whole numbers. Each tier owns a block of them.

```
Calx         rolls   0..599 span 600
Pavonis      rolls 600..849 span 250
Flores       rolls 850..959 span 110
Elixir       rolls 960..994 span  35
Magisterium  rolls 995..999 span   5

weights_total()  Decimal('100.0')   equals Decimal(100)   True
spans add to     1000 of 1000
```

Driving the real function at every roll from nought to 999 puts 600 on Calx, 250 on Pavonis, 110 on Flores, 35 on Elixir and 5 on Magisterium. Those are the declared percentages to the last place, and the rarest tier is one roll in two hundred rather than one in two.

### The draw is seeded and reproducible

The roll comes from numpy's own generator under a named seed. Two generators built from that seed produce the same rolls, so a drop can be replayed.

```
LOOT_DROP_SEED 1155
first eight rolls  [370, 405, 844, 802, 811, 407, 148, 944]
same seed again    [370, 405, 844, 802, 811, 407, 148, 944]
```

Over twenty thousand seeded draws the observed rate sits on the declared weight for every tier.

```
Calx          11900 of 20000  59.500%  declared 60%
Pavonis        5087 of 20000  25.435%  declared 25%
Flores         2207 of 20000  11.035%  declared 11%
Elixir          701 of 20000   3.505%  declared 3.5%
Magisterium     105 of 20000   0.525%  declared 0.5%
```

### What counts as a qualifying market

The rotating reward set decides, and the loot system writes no second rule. A drop asks the rotation for the market's reward reason, and anything other than a drawn market refuses in words.

`src/competition/loot_drop.py` — the one question asked

```
    reason = rotation.reward_reason(request.exchange_id, request.symbol)
    if reason != IN_ROTATION:
        refusal = (
            f"{request.symbol} on {request.exchange_id} does not qualify: "
            f"{reason}; a drop comes from a market the open rotation window drew"
        )
        raise LootDropRefusedError(refusal)
```

Three refusals, each driven on the running rotation.

```
A15/USD on coinbase does not qualify: not_drawn; a drop comes from a market the
open rotation window drew

A14/USD on coinbase does not qualify: no_open_window; a drop comes from a market
the open rotation window drew
```

### An exchange under the floor drops nothing, out loud

The reward set needs twelve eligible markets before it draws anything. Coinbase carries five, so no window opens there and no loot drops. The refusal names the count and the floor rather than returning an empty hand.

```
pool  5: kraken holds 5 eligible markets, under the floor of 12, so no window
         opens, no market qualifies and no loot drops
pool  6: kraken holds 6 eligible markets, under the floor of 12, ...
pool 11: kraken holds 11 eligible markets, under the floor of 12, ...
pool 12: A15/USD on kraken does not qualify: no_open_window
```

A pool of twelve clears the floor and then waits on a window, which is the next refusal rather than a silent drop.

### A bonus augments an action and never a trading figure

An item gives two things: Impetus off what one action costs, and a multiplier on that action's effect. Both live inside the tournament. No figure a bot trades on is read, written or scaled anywhere in the loot system.

`src/competition/loot_drop.py` — the augmentation

```
    relief = held_relief(held)
    return AugmentedAction(
        base_cost=base,
        cost=max(IMPETUS_FLOOR, base - relief),
        relief=relief,
        effect_multiplier=held_effect_multiplier(held),
    )
```

Both numbers have a floor and a ceiling. An action never costs less than one Impetus, whatever the holding relieves, and the effect never more than doubles.

```
 0 of each  relief   0  cost 999 -> 999  cost 1 -> 1  effect 1x
 1 of each  relief   4  cost 999 -> 995  cost 1 -> 1  effect 1.87x
 2 of each  relief   8  cost 999 -> 991  cost 1 -> 1  effect 2x
10 of each  relief  40  cost 999 -> 959  cost 1 -> 1  effect 2x

effect floor 1   ceiling 2   cost floor 1
an action costing 0 Impetus is below 1
```

### The store, one file a chain

A drop names itself by the sha256 of its own contents, so the same item cannot be taken twice. The store sits beside the other ledgers and takes the chain's suffix the way they do.

```
saved loot_store.json 1910 bytes
replayed 5 drops
rarest first ['Magisterium', 'Elixir', 'Flores', 'Pavonis', 'Calx']
duplicate refused: item 9312b43ba83e6c81 is already held
nobody else holds any: []
demo store loot_store_testnet.json 417 bytes
```

### The contract, and the library the trophy gave up

Loot is ERC-1155 with one token id a tier, so every item of a tier shares one image and one attribute set while the drop that made it rides in the mint event. The constructor refuses a tier set whose weights do not total a thousand tenths of a per cent.

`contracts/AcervatorLoot.sol` — the deployment refuses a half-declared scale

```
        uint256 total = 0;
        for (uint256 id = CALX; id <= TIER_COUNT; ++id) {
            total += tiers[id].weightPerMille;
        }
        require(total == WEIGHT_TOTAL_PER_MILLE, "Loot: weights do not total 1000");
```

Five helpers left the trophy contract for `contracts/MetadataLib.sol`, which both contracts now call. Every one is internal and pure, so the compiler inlines each call and no library address is deployed.

The trophy's behaviour did not change, and the compiler says so: with the metadata tail stripped, its deployed bytecode is identical before and after the move.

```
before the move   sha256 aadf925232c4a98e6ad5cd37ee10e653dea4a9badf783cd8eeb97fb679dce18f
after the move    sha256 aadf925232c4a98e6ad5cd37ee10e653dea4a9badf783cd8eeb97fb679dce18f
one constant changed from 21 to 22   sha256 858bc92892966dafa5777dbc04edd19afe84561a05b06b7473743470d0471670
```

The four tier ceilings still hold and an unknown tier is still refused, driven rather than read.

```
[PASS] test_the_four_ceilings_hold_the_numbers_the_tier_list_declares
[PASS] test_gold_fold_mints_at_99999_and_is_refused_at_100000
[PASS] test_bear_slayer_mints_at_9999_and_is_refused_at_10000
[PASS] test_grand_accumulator_mints_at_999_and_is_refused_at_1000
[PASS] test_ekthelius_mints_at_twenty_and_is_refused_at_twenty_one
[PASS] test_a_tier_name_outside_the_five_is_refused
43 tests passed, 0 failed across the four contract suites
```

Every tool verdict on the trophy is the same or better after the move.

| tool | before | after |
|---|---|---|
| forge build | exit 0 | exit 0 |
| forge lint | 23 notes | 20 notes |
| forge test, tier caps | 9 passed | 9 passed |
| slither | 1 result, naming-convention | 1 result, naming-convention |
| solhint | 84 warnings, 0 errors | 81 warnings, 0 errors |
| semgrep | 32 findings | 30 findings |

### What the loot system does not reach

A Draw loot button now reaches the drop, and the section dated 17:40 below records it. It was only ever seen to refuse, because no pool reached twelve.

Nothing mints the ERC-1155 contract. It compiles, its tools report on it, and no caller deploys it, so the wallet reads the store file rather than a chain.

No armour, weapon, accessory, consumable or crafting recipe exists, and no glyph art is drawn. Those are their own arc.

No forge test covers the loot contract's own metadata or its weight-total refusal. The four named tools are its whole instrument.

## 2026-09-10 10:53 - #147 - a certified fill reaches a participant's event record

The division had nothing real to divide on. A certified fill held a grade and a count of scored axes, and a participant's event record held a spend and a clock. Nothing carried the first pair onto the second. The join now does.

```
    def record_certified_fill(
        self, event_id: str, receipt: object, fill: object
    ) -> ActionRecord:
        address = certified_participant(receipt, self._testnet)
        return self._store.add_graded_fill(
            event_id,
            address,
            getattr(fill, "trade_grade", None),
            getattr(fill, "scored_axes", None),
        )
```

### A participant is the address that sent the certification

Certifying a fill sends a transaction to the chain. That transaction's sender is the participant's wallet, and the platform already mints their Quintessence there. The join reads the sender back off the chain rather than working the address out a second way, so one spelling of a wallet exists and a payout lands where the mint landed.

```
bot c8e5c5db  ->  identity 3ecef1619dd9  ->  0x3ecef1619dd90c8d933993228485eb1973971ce9
bot 5ca99f1f  ->  identity 48cce07b5732  ->  0x48cce07b5732795ff561bd78ade52212f84e1180
bot a8d95fed  ->  identity ca3e3124a322  ->  0xca3e3124a3226b2052a5310166c55bd40c943ddc
```

### Many fills become one score, by their mean

One bot trades many times in one event. Every fill counts the same. The record keeps the running total of the grades and how many there were, and the score is the first over the second. A latest-wins rule would let the last trade decide the whole event, and a weighted rule needs a weight nothing supplies.

```
A02/USD  buy   grade F  0.0     total 0.0     fills 1   score 0.0
A02/USD  buy   grade A+ 1.0     total 1.0     fills 2   score 0.5
A02/USD  buy   grade A+ 1.0     total 2.0     fills 3   score 0.6666666666666666666666666666
A02/USD  sell  grade F  0.2955  total 2.2955  fills 4   score 0.573875
```

### The best and the worst grade land halfway between them

One record took the highest and the lowest grade the real grader produced, in that order. The score is the mean of the two and nothing else.

```
A06/USD    grade A+  1.0   total 1.0   fills 1   score 1.0
A02/USD   grade F   0.0   total 1.0   fills 2   score 0.5
```

### A fill with no scored axis changes nothing

A grade resting on no scored axis is the grader's own default, not a reading of a trade. The join leaves such a fill out of the total and out of the count, so it cannot pull a score down the way a zero would. A participant whose every fill is like that keeps a count of nought, and the division gives them no share at all.

```
A02/USD  grade F   0.0  axes 0  ->  total 0  fills 0  score 0
A06/USD   grade A+  1.0  axes 0  ->  total 0  fills 0  score 0

score    {'address': '0xadf9793469cec8ae...', 'score': '0', 'scored_axes': 0,
          'standing': 'no_score'}
shares   []
reserve  1
```

### Real fills off the operator's own log, divided

The platform's own trade-log reader read 1,709 fills. The real grader graded 1,668 of them. Three of the operator's bots took four fills each, certified them, and the join wrote the score. The three then acted in one event, which formed the pot, and the pot divided on those scores. This run supplies the exchange fee at one dollar and names it rather than measuring it, because every fill on disk carries a fee of nought.

```
                     fills  grade total  score     spent    payout
0x3ecef1619dd90c8d      4       2.2955  0.573875  0.100    0.02348871
0x48cce07b5732795f      4       3.5     0.875     0.010    0.03581376
0xca3e3124a3226b20      4       3.0     0.75      0.010    0.03069751

pot 0.120   return pool 0.090   paid 0.08999998
remainder 0.00000002   reserve 0.03000002   exact True
```

The biggest spender of the three scored worst and took the smallest payout.

### A fill that names no participant earns no record

Three ways a fill names nobody, each driven, with an accepted fill above them.

```
accepted             0xd43c6de9d53e88201d4a4f16dd500dada3a111da
                     fills 1   total 1.0   score 1.0   axes 1

no bot named         bot_id must be a non-empty string, got ''
no such transaction  transaction 0xnotonthischain is not on this chain, so the
                     fill names no participant
no chain             bot d43c6de9d53e certified with no chain to read the sender
                     from, so the fill names no participant
```

### A spend still cannot reach a share

The division takes a pot and a list of scores. A score carries an address, a number and an axis count, and no record ever reaches it. This run took one participant's spend up a thousandfold and every payout stayed to the digit. The same division with that participant's score halved instead moved all three, which proves the reading was live.

```
spends  0.100    0.010  0.010   shares  0.09786964  0.14922403  0.12790631
spends  100.000  0.010  0.010   shares  0.09786964  0.14922403  0.12790631

scores  0.2869375  0.875  0.75  shares  0.0562788  0.1716191  0.14710208
```

### The books balance and a second settle gets nothing

This run read the four buckets before the payout and again after it. Nothing came into being and nothing vanished, and no bucket went negative. A second set of objects then read the event back off disk and asked to pay it again.

```
before  wallets 8.6755                held 0.12                  pleroma 0
        minted 8.7955  delta 0  balanced True  negative buckets 0
after   wallets 8.76549998           held 0.03000002            pleroma 0
        minted 8.7955  delta 0  balanced True  negative buckets 0

restart  settled_at read off disk 1789037532.1853175
         refused: ELITE-U29 was settled at 1789037532.1853175; a second payout
         would take Quintessence the pot no longer rests
```

### Demo mode is the same code over its own chain

The redistribution takes its ledger, its store, its pot address and its chain at construction. A demo run is one more of the same object over the demo chain's own files, running the same join and the same division. No flag chooses between them.

```
live     chain 2322666782976   poa_record_store.json           paid 0.08999998
testnet  chain 2322667254944   poa_record_store_testnet.json   paid 0.08999998
```

### What the join does not reach

Nothing subscribes the join to the live fill event. Wiring it would mint and score against the operator's real ledger on every trade, and that remains his decision.

The live fill payload carries no grade and no axis count, so a fill arriving that way today carries the record's own defaults and stays out of the score. The grades above came from the real grader reading the operator's trade log.

The payout now has a button, recorded in the section dated 17:40 below. Nothing starts a join.

In development.

## 2026-09-10 11:15 - #147 - the fill event carries its venue fee

### The fee the venue charged now rides on the fill

Every fill the platform announces now carries the fee the exchange actually took for that fill, in dollars. Certification reads that one number and distils Quintessence from it, so a real trade finally earns.

`src/trading/scrumming/execution.py` - what a fill now says

```
{'bot_id': '650df31a', 'type': 'CARTRIDGE_SCRUM', 'side': 'SELL',
 'amount': 1409.0, 'price': 0.003676, 'usd': 5.117330192, 'profit': 0.0,
 'operator_initiated': False, 'fee_usd': 0.062153808}
```

### The fee is carried from the venue and is never worked out

The exchange reports the fee on the settled order. The platform stores that figure and nothing else, converts it to dollars at the cached quote rate, and puts it on the fill. The fee percentage in a bot's settings is never used here.

`src/trading/scrumming/execution.py` - the record, and the five refusals

```
    def _fill_fee_fields(self, fill_price: float) -> dict:
        """The ``trade.filled`` fee fields for the fill that settled at ``fill_price``.

        Returns ``fee_usd`` when the venue reported a usable fee, otherwise
        ``fee_refusal`` naming why, and never reads ``config.trading_fee_pct``.
        """
```

```
no settled order was held for this fill
the venue reported no fee
the reported fee belongs to a different fill
the venue named no fee currency
the venue charged the fee in <currency>, not the <quote> this fill is priced in
```

### The buy side was left out on purpose, and is carried now

The fee record was built to take the exchange's cut out of a sale's proceeds. A purchase has no proceeds, so the record was written for sales only and cleared on a purchase. That was right for its job and wrong for this one, because a Fold is a purchase and the exchange charges for it.

A second record now holds the fee for either side. It is read only by the fill announcement, and the sale-proceeds record is untouched.

```
the sale record   written on a sale, cleared on a purchase, spent on proceeds
the fill record   written on both sides, spent on the fill announcement
```

### Both sides earn, driven on his own trades and the exchange's own fees

Two real trades of his, each matched to its row in his own Coinbase transaction export by asset, size, price and second. Six hundred and fifty-seven of his 1,709 trades have such a row.

```
SELL  A29/USD 1409 @ $0.003676    2026-08-20 15:02
      venue row 6a8716ec03b6f3f5c260f0f6   fee $0.062153808
      on the fill   fee_usd 0.062153808
      awarded       0.0310769040 Quintessence   pool coinbase:1|A29/USD

BUY   A29/USD 1325 @ $0.002286    2026-08-07 20:35
      venue row 6a7641a1facc93c97c34e93b   fee $0.0363474
      on the fill   fee_usd 0.0363474
      awarded       0.03634740 Quintessence
```

### A fill the exchange charged nothing for earns nothing, and says why

Coinbase's reply to a newly placed order carries no fee figure at all. A fill like that carries no fee and carries the reason instead, and the award is refused by name rather than granted for nothing.

```
on the fill   fee_refusal 'the venue reported no fee'
refused       nothing_to_award
distilled     0 Quintessence

a fee of 0.0 at a grade of 0.5 distils nothing, and granting it would spend the
participant's one allotment of A29/USD and open a cooldown for no Quintessence
```

### No figure a bot trades on moved

The same two fills were driven through the same order path on the tree before this change and on the tree after it. Twenty-four figures were compared each time, including the order actually sent to the exchange, every lot, every tranche and every running total.

```
sell with a fee      24 figures identical
sell with none       24 figures identical
buy with a fee       24 figures identical
buy with none        24 figures identical

the same comparison of the announcement itself
  + fee_usd 0.062153808          the one and only difference
```

### His own records now carry the fee

The trade log takes whichever of the two the fill supplied, so a row either names the dollars the exchange took or names why it took none.

```
with a fee  {..., 'usd': 5.117330192, 'fee_usd': 0.062153808}
with none   {..., 'usd': 5.179484,    'fee_refusal': 'the venue reported no fee'}
```

### Demo mode reads the same fee over a second chain

One class, two chains, one award path, no flag. The demo chain reads the fee off the same announcement and mints from its own ledger.

```
the same class        : CertificationSocket and CertificationSocket
the same chain object : False

on the demo chain
  A15/USD, venue fee $0.1369262274012 -> awarded 0.1369262274012 Quintessence

live ledger   total ever minted 0.067424304     balanced
demo ledger   total ever minted 0.1369262274012  balanced
```

### What the venue fee does not reach

Nothing subscribes the socket to the live fill event, so no trade reaches certification while the platform runs. That wiring would mint against the live ledger on every fill and is a decision about real value.

A fill still carries no exchange and no season, so the socket cannot name the activation from the announcement alone. Those two fields reach the record through certification's own caller today.

On live Coinbase the reply to a newly placed order carries no fee, so a fill announces one only where the settled order is re-read. Until the exchange supplies a fee at placement, a live Coinbase fill will usually carry the refusal rather than a figure.

## 2026-09-10 12:18 - #147 - the settled order is read back for its fee

### Coinbase answers the fee on a second call, never on the first

The reply to a placed order names the order and nothing else. No fill size, no average price, and no fee. The fee appears only when the same order is asked for again by its own identifier, and only once the exchange marks it settled.

`src/exchange/ccxt_connector.py` - what each call returns, from the exchange's own published replies

```
placing an order      order_id, product_id, side, client_order_id
reading it, open      status OPEN,    settled false, filled 0,        fee "0"
reading it, settled   status FILLED,  settled true,  filled 0.000297, fee "0.0379"
```

The platform's own record agrees. Every order it placed on the exchange was logged as accepted with nothing filled.

```
ORDER_PLACED | BUY order accepted by exchange | status=open filled=0.0/2672.0
9 of 9 placements read status=open filled=0.0
```

### The sell and the fold read the fee off the placed reply, so they read nothing

The two paths a bot trades on took the fee from the reply to the placement. On Coinbase that reply has no fee in it, so every live fill announced a refusal and earned nothing, while the exchange held the figure all along.

`src/trading/scrumming/execution.py` - the read that was missing

```
    async def _settle_venue_fee(self, order, units: float, price: float) -> None:
        """Re-reads ``order`` by its id and takes the settled venue fee from that body.

        Writes ``_last_fill_venue_fee`` only, and leaves it unchanged when
        ``get_order`` reports no fee.
        """
```

### One extra call, and only for a fill that has no fee yet

The read runs after the trade is placed and filled, never before. A fill whose reply already carried a fee makes no extra call at all. A fill with no fee makes exactly one, and the number of units and the price come from the fill rather than from the second reply, so the second reply can move no trading figure.

```
_VENUE_FEE_REREADS         1
_VENUE_FEE_REREAD_DELAY_S  0.2

a reply that already held a fee   0 reads, fee $0.062153808 on the fill
a reply that held none            1 read,  fee $0.062153808 on the fill
```

### A read that is refused, or never answered, costs the fill its award and nothing else

The exchange refusing the read leaves the fill carrying a refusal. A read that never answers ends at the connector's own twenty-five second ceiling and leaves the same refusal. The sale or the purchase is already done in both cases, at the same size and the same price.

```
the exchange refuses the read   sold 1409 @ $0.003676, fee_refusal, 0 Quintessence
the read never answers          sold 1409 @ $0.003676, fee_refusal, 0 Quintessence
```

### A real trade of his, earning through the second call

His own sale and his own purchase, with the fee arriving from the settled read rather than from a downloaded statement.

```
SELL  A29/USD 1409 @ $0.003676   settled fee $0.062153808 -> 0.0621538080 Q
BUY   A29/USD 1325 @ $0.002286   settled fee $0.0363474   -> 0.03634740 Q
```

An order the exchange has not settled reports a fee of zero, and that fill earns nothing and says so.

```
fee_refusal  the venue reported no fee
distilled    0 Quintessence
```

### Demo mode reads the fee the same way over its own chain

The same class, the same sale and the same settled read, on a second chain with its own ledger and its own record file. No flag selects the behaviour; the chain the page was built over decides which books move.

```
the same class        : CertificationSocket and CertificationSocket
the same chain object : False
the same ledger       : False

A29/USD, settled fee $0.062153808 -> 0.0621538080 Quintessence
demo ledger total ever minted 0.0621538080   balanced
```

### The second call moved no figure a bot trades on

Eight runs across the previous build and this one, twenty-four figures each, and every figure the same. The same comparison reports one difference on the three runs where a fee arrived, which is the fee itself.

```
order size and side sent to the exchange, the fill price, holdings, both target
balances, the fold queue, the standing surplus, the last trade price and side,
both pivot references and both armed flags, the quote rate, the distribution
accumulator, the hedge reserve, every main lot, every fold tranche and seven
running totals

trading differences   0 of 24, on all eight runs
fee field difference  the venue reported no fee -> fee_usd 0.062153808
```

### What the settled read does not reach

The sale's proceeds still book gross. The record a sale settles on is left untouched on purpose, because taking the exchange's cut out of a sale now would move figures the operator reads.

A partly filled order reports the fee for the part that filled at the moment of the read, not for the whole order.

Nothing subscribes certification to the live fill event, so no live trade reaches an award while the platform runs. That remains a decision about real value.

## 2026-09-10 13:02 - #147 - the registry mints the trophy

### Nothing on the chain could mint a trophy

The trophy contract admits one caller for its mint and names it the registry. The registry held no address for the trophy at all, so that mint had nobody able to call it. A competition could hand out tokens and leave no trophy behind.

The two halves, as they stood

```
AcervatorTrophy.mint       admits the registry alone
AcervatorTrophy.registry   fixed at construction, cannot be changed
CompetitionRegistry        no trophy address, no trophy call, no NFT
```

### The registry holds the trophy in an address written once

Each contract needed the other first. The trophy refuses to be built without a live registry address, so the registry is built first and then pointed back at the trophy by a single call. That call admits the deployer only, refuses a second call for ever, and refuses any address that holds no code, so the trophy a deployment mints through can never be redirected afterwards.

`contracts/CompetitionRegistry.sol` - the call that cuts the cycle

```
    function setTrophy(address trophyAddress) external {
        require(msg.sender == DEPLOYER, "Registry: caller is not the deployer");
        require(address(trophy) == address(0), "Registry: trophy already set");
        require(trophyAddress.code.length > 0, "Registry: trophy not a contract");
        trophy = ITrophy(trophyAddress);
        emit TrophySet(trophyAddress);
    }
```

### One tier ceiling, and it belongs to the contract that mints

Two contracts held tier limits and neither knew about the other. The registry counted two tiers and the trophy counted four. The registry's two numbers and its two counters are gone, and awarding now runs through the trophy's mint, so the trophy refusing a tier at its limit undoes the token award in the same transaction. One set of numbers, in the contract that creates the thing counted.

The four lifetime limits, all of them in the trophy

```
Gold Fold             100,000
Bear Slayer            10,000
Grand Accumulator       1,000
Ekthelius                  21
Harvest                no limit
```

### Every limit was driven to its edge and the next award refused

Ekthelius was awarded twenty-one times over twenty-one competitions with nothing faked, and the twenty-second was refused. The other three cannot be reached by awarding inside a test, so their counters were moved to one below the limit, the award at the limit was made, and the next one was refused. A tier name outside the five is refused as well, and Harvest keeps awarding past every other tier's limit because the lowest tier has no limit at all.

```
Ekthelius           21 awarded, 22nd refused   driven, nothing faked
Grand Accumulator   1,000 awarded, next refused
Bear Slayer         10,000 awarded, next refused
Gold Fold           100,000 awarded, next refused
"Ekthelius " (a name outside the five)         refused
Harvest above every other limit                awarded
```

### The season budget is on the chain and refuses the award past it

The season pool used to exist only in the Python. The chain recorded what a season had paid out and checked it against nothing, so the limit held only for as long as the software did. The registry now works the pool out on the chain and refuses any award that would pass it. An award that exactly fills a season lands; one wei more is refused; and the next season opens with its own pool untouched.

The pool each season carries, in ACRV

```
season 1     500,000
season 2     425,000
season 3     361,250
season 4     307,062
season 53        106
season 54        100   the floor, and every season after it
```

### No award can be made before the trophy is named

While the trophy address is unset, adjudicating an award is refused outright. That is deliberate: a token award with no trophy behind it is the defect this work closes, so the chain now refuses to make one. Deployment names the trophy before any competition can open.

```
trophy unset    adjudicate refused, "Registry: trophy not set"
trophy named    the same award lands and mints trophy #1 to the winner
```

### What the registry's mint does not reach

Rank is always first, because a competition adjudicates one winner. A field of runners-up earns nothing and holds no trophy.

A tier whose artwork was never uploaded cannot be awarded at all, because the trophy refuses to mint a tier with no art. Deployment uploads all five.

The season pool bounds the tokens a season pays out. It does not bound how many trophies a season mints, because the uncapped tier costs no tokens.

## 2026-09-10 13:04 - #147 - the loot contract has forge coverage

Four contract suites sat in the tree and none of them read the loot contract. Its rarity scale and its metadata were held by four analyzers that look for known weakness classes, and by nothing that drives the contract and reads what it returns. A fifth suite now sits beside the four.

### A fifth suite reads the loot contract

The suite deploys the contract, fills the art for all five tiers, and drives every mint through a handler that is the only address the contract admits as the dropper.

```
tests/contracts/AcervatorLoot.t.sol   14 checks, 7 of them invariants
forge test                            5 suites, 57 checks, 0 failed
```

### The five weights add up to the draw span

The scale is read back out of the contract rather than out of the source. Each tier's weight is asked for by name and the five are added together.

```
Calx            600
Cauda Pavonis   250
Flores          110
Elixir           35
Magisterium       5
                ---
total          1000   ==  WEIGHT_TOTAL_PER_MILLE
```

A total that matches is worth nothing unless the total can also miss. One tier's weight is moved in storage from 600 to 599 and the same reader is asked again.

```
before        total 1000, the draw span
weight 599    total  999, reported short
restored      total 1000, the draw span again
```

### The refusal the draw span governs

The draw span is the weight total, and a mint refuses a roll that falls outside it. Both sides are driven on the same contract.

```
roll  999                   admitted, the holder ends with one item
roll 1000                   "Loot: roll outside the draw span"
roll at the largest number  "Loot: roll outside the draw span"
the minted total afterwards  1, so a refused roll counts nothing
```

Four more refusals are driven beside it, so the roll refusal is the only reason the roll case can fail.

```
tier id 0 and tier id 6     "Loot: unknown tier"
a tier holding no art       "Loot: SVG not uploaded for this tier"
any address but the dropper "Loot: caller is not the dropper"
the zero address            "Loot: mint to zero address"
```

### The constructor's own refusal cannot be driven from a test

The deployment refuses a tier set whose weights miss the total. That refusal takes no input. The five weights are written by a private helper, from fixed numbers, inside the constructor, and the only thing a caller passes in is the dropper address. The compiler says so when a test tries to reach the helper.

```
Error (7576): Undeclared identifier.
    _setTier(CALX, "Calx", "Calx", 599, 0, 20, "#C8C0B4");
```

A contract built on top of the loot contract can write a weight, and the deployment still succeeds, because the refusal has already run by then. The accepting side is therefore driven, and the refusing side is not reachable without editing the contract, which this unit does not do.

```
    uint256 total = 0;
    for (uint256 id = CALX; id <= TIER_COUNT; ++id) {
        total += tiers[id].weightPerMille;
    }
    require(total == WEIGHT_TOTAL_PER_MILLE, "Loot: weights do not total 1000");
```

### The uri, and a token id that does not exist

Every tier serves its metadata as base64 text carried inside the answer itself, so a reader needs no web address. The lowest tier's answer is compared against the JSON written out by hand below, character for character.

```
{"name":"Acervator Loot — Calx","description":"A Proof-of-Accumulation loot item
dropped by a qualifying market. Tier 1 of 5, weight 600 of 1000. It augments a
tournament action and no trading figure: 0 Impetus off one action's cost and 20 of
1000 added to that action's effect.","image":"",
"attributes":[{"trait_type":"Tier","value":"Calx"},{"trait_type":"Short Form",
"value":"Calx"},{"trait_type":"Weight (per mille)","value":600},{"trait_type":
"Impetus Relief","value":0},{"trait_type":"Effect Bonus (per mille)","value":20},
{"trait_type":"Minted","value":0},{"trait_type":"Tier Color","value":"#C8C0B4"}]}
```


Changing one digit of that hand-written JSON, from weight 600 to 601, makes the comparison report, which is how the comparison is known to read the contract's answer rather than itself.

A token id outside the five gets no metadata at all. It is refused rather than answered with an empty item.

```
id 1 to 5                 each serves its own answer, all five different
id 0                      "Loot: unknown tier"
id 6                      "Loot: unknown tier"
the largest number        "Loot: unknown tier"
the answer after a drop   moves, because it carries the minted count
the answer after new art  moves, because it carries the tier's art
```

### One fuzz target, named, and forty seeds

The random runner is told to drive the handler and nothing else. Without that, it also drives the loot contract directly and a mint can land that no counter recorded. Measured with the declaration removed: about 1,350 direct mints per campaign, and on one earlier revision one of them landed and the count check went red.

```
with the target named, 40 seeds        40 of 40 green, 57 checks each
each campaign                          256 runs, 16,384 calls
without the target, 140 seeds          all green, but ~1,350 direct mints a run
without the target, earlier revision   1 seed of 40 red, an uncounted mint
```

One seed is not a measurement. The rare red is the reason the target is named.

### What every tool printed

Each tool was shown failing on a broken file and quiet on a sound one before any verdict here was read.

| tool | broken file | sound file | the new suite |
|---|---|---|---|
| forge | 1 check failed | 16,384 calls, passed | 57 passed, 0 failed |
| slither | 1 high result | 0 results | 0 results in project code |
| solhint | 1 error | 0 errors | 0 errors, 210 warnings |
| semgrep | 1 finding | 0 findings | 0 findings at error level |

The warnings and the lint notes sit in the same classes and the same counts as the four suites already in the tree.

```
solhint warnings    153 to 235 across the four, 210 here, 0 errors in all five
forge lint results   23 to  48 across the four,  37 here, 0 errors in all five
semgrep, all levels  67 to  70 across two,       92 here, 0 at error level
```

mythril is still absent from this machine, so symbolic execution over the loot bytecode has not been run and nothing was put in its place.

### What the forge coverage does not reach

No contract changed. The loot contract is read and driven and not edited, and the comparison over the contract directory is empty.

The refusal inside the deployment is still unproved, for the reason given above. Proving it would need the weights to arrive from outside the constructor, which is a change to the contract and a decision about what the contract is.

Nothing is deployed and no network is reached. Every run here is local.

No control on screen dropped an item on the day this was written, so the operator saw nothing new. The coverage protects the rarity scale and the item page a marketplace would read, before either reaches a chain.

## 2026-09-10 13:28 - #147 - what a world costs on the chain

### The question turns round, and then it closes

He asked how large a full world state must be. A chain keeps the transitions and never a copy of the state in every block, so the answer is what one action costs and what a budget buys. His own rule sets it that way round: a maximum world state size dictates the participant count and how large a generative world may be.

Driving records through the chain, letting the chain save itself and reading the file size gives this.

```
one action on the chain today          1,473 bytes
one action in a chosen encoding          399 bytes
recommended ceiling                      1 MB per layer per one-hour world turn
participants a layer                      79
squares a layer                           79
squares per participant                  1, fixed rather than chosen
a twenty-layer world                     20 MB an hour, 1,580 participants
his 3,136-block chain reloads in         65 ms
a hundred times those records            4.9 s
what binds                               the save, not the size and not the load
the checkpoint a cube forces             about once a day at twenty layers
```

### His own chain, confirmed

The platform reports its own record counts, and the file system reports the bytes. Both agree with the figures already recorded for this tab.

```
blocks         3,136
transactions   3,135
events         2,565
file bytes 4,059,629
```

Re-saving that chain through the platform's own save path writes 3,946,745 bytes with every section unchanged. The 112,884-byte difference is one carriage return a line, left by an older build writing in Windows text mode. The per-record averages of 291, 369 and 322 bytes therefore run 2.86% high, and correct to 283, 359 and 313.

### One action, measured eight ways

One hundred grid moves were driven through the chain for each row, as a transaction carrying an event, a turn, the actor, the action, two squares and an Impetus cost. They exist to be measured and they are not a design. Each figure includes the block the action rides in.

```
one block per action, named fields, indented   988   what the chain does today
one block per action, packed argument          781
one block per action, named fields, compact    969
one block per action, packed argument, compact 616
one block per turn,   named fields, indented   714
one block per turn,   packed argument          508
one block per turn,   named fields, compact    526
one block per turn,   packed argument, compact 399   the chosen encoding
```

### What the smaller encoding gives up

Dropping the indentation saves 875,348 bytes on his chain, 22 per cent, and costs a file a person can read. Packing the arguments saves 207 bytes an action and leaves a record nothing can interpret without the code that wrote it. Sharing one block across a turn saves 274 bytes an action and gives up the ordering that block number supplies today.

```
his chain indented   3,946,745 bytes
his chain compact    3,071,397 bytes
```

### Two clocks, and only one of them is bounded

A world turn is one hour. A dungeon or raid turn is one candle. The hour divides exactly into both.

```
elite event turn       60 s, sixty turns an hour
standard event turn   300 s, twelve turns an hour
world turn           3600 s, one turn an hour
```

The Impetus pool bounds an event turn: four at the first level, one more every twenty levels, and a speed multiplier may at most double the level's own grant. The modes module holds no one-hour term at all, so nothing bounds the actions in a world turn. The ceiling below can be stated and not enforced until a world-turn allowance exists.

In development.

### A world rebuilds from its own history in well under a second

His real chain, and then ten and a hundred times its records, were loaded through the platform's own load path. Each figure subtracts a control run that builds the same objects without loading.

```
records    file bytes       load   per record
  8,836     4,059,629      65 ms     7.4 us
 65,265    26,031,824     516 ms     7.9 us
629,565   247,769,048   4,901 ms     7.8 us
```

The cost per record stays flat across seventy-one times the history. One second of load buys about 125,000 records, and ten seconds buys 1.28 million.

### The save is what binds

A whole-file write costs its whole size every time, and the timer fires half a second after the last change.

```
 3,946,745 bytes      67 ms
26,031,824 bytes     336 ms
247,769,048 bytes  3,444 ms
```

At fifteen milliseconds a megabyte such a write finishes inside that half second only while the chain stays under about 34 megabytes, which is roughly 85,000 actions for the life of the world. A ten-second load would allow 1.28 million. A save path built that way therefore stops a world at one fifteenth of what its own load time allows.

The shape that removes it appends each record to the log rather than rewriting the file, and writes a full snapshot now and then so a load reads one snapshot and a short tail. Both are built.

In development.

### One megabyte an hour, recommended

The per-action cost rounds up to 1,473 bytes and the budget rounds down from the 1.28 megabytes a ten-second load would allow. Both roundings keep the bound safe.

Maximum world size and maximum participants reach the ceiling together, so nothing is left over to spend on world and no trade-off exists between the two. One number moves and both caps land on the same budget point.

```
1,048,576 / 1,473  = 711 action records a world turn

actions each   participants   world squares   squares per participant
      4             177            177                  1
      8              88             88                  1
      9              79             79                  1
     20              35             35                  1
     40              17             17                  1

a 720-turn season = 720 MB, which reloads in 14.7 s
```

The recommended point is the nine-action row: 79 participants on 79 squares, per layer. Nine is the Impetus grant at the top of the hundred-level arc, so the bound holds at every level. The four-action row is the same

layer on its first day, 177 participants, and it is not the bound. One megabyte rests on the ten-second load ceiling and nothing else.

### A world is a cube, so the layer count multiplies the chain

A world carries a third axis, the tree level. The Sephirot number ten and the Tree of Death inverts each of them, which reads as twenty layers. Twenty is a reading of his wording rather than a ruling, so ten sits beside it.

```
layers   chain an hour   participants   squares   a 720-turn season
   1         1 MB               79         79        720 MB
  10        10 MB              790        790      7,200 MB
  20        20 MB            1,580      1,580     14,400 MB
```

The cube figure is the real one. The flat figure is a comparison, and quoting it for a world that has layers would be wrong by the layer count.

### Where the cube collides with constant density

Capping size and participants together fixes area per participant, and a third axis can honour that two ways.

```
density per world   79 participants over 20 layers, so a layer holds 3 of them
                    on 3 squares, about two by two
density per layer   each layer keeps the cap, so every layer stays a world and the
                    cube holds twenty times the participants and the storage
```

These figures take the second reading. The first holds the density constant only by shrinking a layer to a two-by-two board, which is not a world and leaves a zone three participants to take a level from. Keeping every layer a world means the cap is per layer, and a twenty-layer world then costs twenty megabytes an hour.

### How often a checkpoint has to be written

A checkpoint is a full snapshot beside the log, so a load reads one snapshot and the records since it. The cadence follows from the two ceilings already measured: 512 megabytes is what a ten-second load allows, and 34 megabytes is where the current save stops keeping up with its own timer.

```
layers   reaches 512 MB in       reaches 34 MB in
   1     512 turns, 21 days      34 turns, 1.4 days
  10      51 turns, 2.1 days      3.4 turns, 3.4 hours
  20      26 turns, about a day   1.7 turns, under 2 hours
```

A cube saturates a whole-file save path inside two hours. Appending each record and snapshotting now and then is the condition on the first multi-layer world running at all, not an improvement for later.

```
CHECKPOINT_RECORD_INTERVAL = 50_000
```

### Why the two caps arrive together

A square costs nothing until something changes it, and a square changes only where a participant acts. A square's records are that participant's action records rather than an addition to them.

```
the seed            one record, once
a square unchanged  nothing
a square changed    the action record of whoever changed it
```

A world larger than its participants can reach holds squares that produce nothing, and a smaller one crowds every square. One budget, one exhaustion point, both caps.

### One square a participant, and the rule that fixes it

Two rules already set decide the density rather than a choice made here. Base viewrange is one square, and a zone's level tracks the average character level of those in it.

```
base viewrange            one square
zone level                follows the participants in that zone
squares per participant   1
```

Above one square a participant the average square stands empty, a one-square viewrange shows nobody, and a zone has no population to take a level from. A world at this density is never large and sparse, and never small and crowded. Every later unit sizes a map against this number.

### Sight is bounded and storage is not

Viewrange bounds what a participant sees. A chain holding only what one participant can see would not be a chain, so the two figures stay separate.

The layer axis multiplies what the chain holds and divides what a participant gets, so the two figures pull apart. A participant sees a higher or lower layer only once able to enter it.

```
the chain holds, twenty layers         14,220 records a world turn
one participant, one layer                  9 records a world turn
one participant, all twenty layers        180 records a world turn
```

At base viewrange on one layer a participant receives one record in one thousand five hundred. A range boost multiplies that share by the squares it adds and changes nothing about what the chain stores. Nothing in the competition package carries a zone, a square, a tile, a layer or a viewrange, so neither figure has a consumer yet.

In development.

### Ordering a world turn needs a position inside the block

World actions resolve in timestamp order when the block closes. The block's own id covers its timestamp, and a transaction's id covers its own declared time and the position the block gave it.

```
block id        number, parent, timestamp, transaction list
transaction id  sender, recipient, function, arguments, gas, status,
                declared time, position in the block
placement id    the same fields, position aside
```

The block assigns each transaction its position at close and that position enters the transaction's id, so two nodes holding one transaction agree on where it ran. A position costs 27 bytes a record, measured, and it

changes every id the chain writes from now on. The schema version does not move, so no saved file is discarded.

```
sort key   the declared placement time, then the record's placement id
position   0, 1, 2 and so on, in that sorted order
```

### His own history reports as legacy, not as altered

Loading his chain reported every record as altered before this was repaired.

```
chain NOT verified: 3135 of 3136 blocks altered, 0 parent links broken,
3135 of 3135 transactions altered
```

His file was written before records took the hash of their own contents as a name, and the schema version did not move when that landed. The state rebuilds correctly, and only the records written before content addressing cannot be verified. Raising the schema version would make the load delete his file and 3,136 blocks with it, and re-assigning ids on load is the one thing a tamper check must never do. A saved chain now records the first block whose names come from contents, and every record below it is named legacy and kept.

```
chain holds 3135 legacy blocks and 3135 legacy transactions: written before
block 3136, when ids began to derive from contents, so the chain keeps them
and does not vouch for them
```

### What this measurement does not reach

An identity on this chain is text: 66 characters for a hash and 42 for an address. Bytes would halve both, and that changes the chain's identity model rather than its encoding, so nothing here measures it.

No grid exists, so the square count the density constant gives has nothing in the code to apply it to. Nothing bounds the actions in a world turn, so the recommended ceiling has no enforcement point yet. Twenty layers is a reading of his wording and not a ruling, and ten sits beside it throughout.

## 2026-09-10 14:18 - #147 - the block orders its records, and a legacy file is named as one

### A block closes its records in one order, on every node

A world turn collects actions for an hour and runs them when the block closes. The close sorts them, gives each one its position, and writes that position into the record's name, so the order is part of what the chain can check.

```
sort key       the declared placement time, then the record's own placement id
position       0, 1, 2 and so on, in that sorted order
stored name    the hash of the record's contents at that position
placement id   the hash of the record's contents, position aside
```

A record now carries two names. The stored name says which action this is and where it ran; the placement id says only which action it is, and that is the one two nodes use to recognise the same action.

### What a node chooses, and what it cannot

A node declares when it placed its own action, to the millisecond, and that is the only thing about the order it can steer. Two actions placed in the same millisecond are separated by the hash of their own contents, which no node can aim at the position it would prefer.

```
a node can       say when it placed its own action
a node cannot    move where another node's action lands
a node cannot    make arrival order matter, because arrival order is not read
a node cannot    change a time or an amount later, because both sit inside the name
```

### Two nodes, the same actions, two arrival orders

Two processes on separate chain files were handed the same three actions in two different arrival orders. Both closed one block, and both finished with the same order and the same three names.

```
node A heard   charlie, alpha, bravo      applied 3
node B heard   bravo, charlie, alpha      applied 3

both closed    0  0x9218cce8...  bravo     placed 1789049829.404
               1  0x46b36816...  alpha     placed 1789049829.404
               2  0xc2de4239...  charlie   placed 1789049829.405
```

Neither arrival order matches the closed order. Alpha and bravo share a millisecond, so the tie-break put them in order and not the clock.

### The same three actions, before the change

Taken from the repository at the commit before this change, the same exercise left each node with whatever order it happened to hear, and every chain reported itself verified.

```
source node   alpha, bravo, charlie      verified
node A        charlie, alpha, bravo      verified
node B        bravo, charlie, alpha      verified
```

Three nodes, three orders, and nothing anywhere reported a disagreement. That is the silent failure the close replaces.

### What a position costs, measured

His chain was loaded and written out twice, once by the code before this change and once by the code after it, so only the new field separates the two files.

```
before   3,946,745 bytes
after    4,031,424 bytes

+84,679 bytes over 3,135 transactions   27 bytes a transaction
+34 bytes once                          the file's own marker
```

Twenty-seven bytes takes one action from 988 bytes to 1,015 in this encoding, so a one-megabyte layer holds 1,033 actions an hour rather than 1,061. The recommended ceiling of one megabyte does not move, and 79

participants on one square each sits far under both figures.

### A record written before content names is called legacy

A saved chain now records the first block number whose names come from contents. Every record below that number was written under the older scheme, so the chain keeps it, says so, and does not claim to vouch for it.

```
before   chain NOT verified: 3135 of 3136 blocks altered [1, 2, 3, 4, 5],
         0 parent links broken [], 3135 of 3135 transactions altered

after    chain holds 3135 legacy blocks and 3135 legacy transactions: written
         before block 3136, when ids began to derive from contents, so the
         chain keeps them and does not vouch for them
         chain verified: 1 blocks and 0 transactions carry the id of their
         own contents
```

The schema version is untouched, so nothing about the load deletes anything. A file with no marker in it is read as written entirely under the older scheme, which is what his file is.

### An edited amount inside his own chain is still caught

One current record was added to a copy of his chain, the file was saved, and then one amount in that one record was edited by hand. The 3,135 legacy records stay forgiven and the edited record does not.

```
3,137 blocks   3,136 transactions
legacy         3,135 blocks and 3,135 transactions
altered        1 transaction, 0x86be12a8...
verified       false
```

The same edit offered to a node over the link is refused and the two sound records beside it are taken, so the refusal picks out the record rather than the batch.

### The marker decides it, and it was shown both ways

A marker that forgave every mismatch would be worthless. The same file was read twice, once with its marker as saved and once with the marker set to zero by hand.

```
marker 3136   3,135 legacy, 0 altered, verified true
marker 0      0 legacy, 3,135 altered, verified false
```

### His own file was never written

The file on his machine was copied to a scratch directory, and every run above read the copy. His file is the same size and the same contents it was before this work.

```
4,059,629 bytes, sha256 f84458bc69c8e894794836b8658be7518cb6e108e78861cd9769f12fe25988be
```

### Demo mode runs the same close over its own chain

Nothing here reads a flag. Each node in every run above was a separate chain handed in when the object was built, and the same close, the same link and the same integrity report ran over all of them.

```
four chains in this work, four scratch files, one set of methods
no flag, no second code path
```

### What the ordering does not reach

A block's membership is still whatever a node had when it closed. Two nodes that close different sets put one action at different positions, so its stored name differs on the two chains while its placement id stays the same. The hour boundary that would make every node close the same set does not exist yet.

Nothing bounds the actions inside a one-hour turn, so a block can still close over any number of them. The millisecond is the finest the declared time goes, and below it the order comes from the contents.

---

## 2026-09-10 15:09 - #147 - four subtabs, and a map that opens on a crawl

### A row of four buttons sits under the state line

The tab draws a subtab bar beneath its heading and state sentence, and one panel under that bar. Nothing below moved: the player window, the enemy screen and the party window keep their places, and the Quintessence wallet still opens over the party window. A subtab lives inside this tab, so the tab row still carries one Accumulation entry and this adds no window.

```
Character Stats    Gear    Skill Tree    Map
```

### Character stats is the full one

The panel lists every RPG metric the conversion derives, each with the field it reads and the value the party row holds. Twenty-seven rows draw, and none of them needed a new source.

```
max_health_usd         scrumming_state.target_balance
base_health_usd        scrumming_state.anchor_target_balance
levelled_health_usd    compounding_snapshot.accrued_growth_usd
...
blocked                bot.gate_decision scrum_blockers and fold_blockers
```

Under the rows the page counts what carried a value, and names what nothing holds.

```
Metrics carrying a value: 0 of 27.
bot_state.json holds no bot under this chain, so every metric reads --.
Nothing holds these, so no metric reads them: experience, level, character class,
gear, enemy, threat, guild.
```

One metric holds a list rather than a number. The blocked row joins the gate labels with commas, so the value column reads as a sentence and never as program syntax.

```
blocked   bot.gate_decision scrum_blockers and fold_blockers   interval, trend_hold, cash
```

### Gear manages loot and says what it cannot manage

The gear subtab draws the same loot holding the wallet draws: one row an item, then what one action of the running turn costs and what it does afterwards. Two items in a scratch store drew four rows.

```
Magisterium   A15-USD - Season 1 - Impetus -2, effect +50%
Calx          A14-USD - Season 1 - effect +2%
Action Impetus   4 becomes 2
Action effect    1.52x
```

No armour slot, no weapon slot and no empty frame waits for an item class the design has not reached. The panel states that in one sentence instead.

```
Nothing builds armour, weapons, accessories, consumables, so this subtab manages
loot alone.
```

### The skill tree is a list, and it admits it

The ladder carries one skill. A tree of one member is a list, so the panel draws ten level rows and opens with a sentence naming the count.

```
Skills on the ladder: 1. Quintessence Transfer. A tree needs more than one, so
this draws a list.
```

### The mode decides whether the map opens

`EventMode.has_map` already holds on Descension and Cementation and fails on Fixation and Coagulation. The subtab reads that one property, the same way the turn length reads one property of the variant, so no second rule names which modes allow a map.

```
def map_reachable(variant: EventVariant) -> bool:
    """Whether ``variant``'s mode carries a map, read off ``EventMode.has_map``."""
    return variant.mode.has_map
```

The player window carries its own map button beside the mode list, and it opens the same subtab the bar opens. In a crawl or a raid the button is live and the click draws the map panel. In either smash the button is dead, the bar entry is dead, and the sentence under the button says which modes do carry a map.

```
Fixation carries no map, so this subtab does not open. Descension and Cementation do.
```

That sentence names its modes off the same property, so a mode that gains a map appears there with no second edit.

### The map panel says no world exists

Nothing generates a world, so the map subtab opens on one sentence rather than an empty frame.

```
No world is generated. No grid, no tile and no position is held anywhere, so this
subtab draws no map.
```

### Demo mode navigates all four

No flag selects the chain. The panel takes a chain name at construction and every subtab reads that chain's own files, so one walk covered the four subtabs on the TestNet and another covered the same four on the live chain. The gear panel named the TestNet's own loot file.

```
live      loot_store.json
testnet   loot_store_testnet.json
```

### What the subtabs do not reach

The skill tree stood at level 0 on both chains when the subtabs landed, and the loot a gear panel showed was whatever the chain's store already held. The controls dated 17:40 below change both. The map has no content of any kind, and the world it would draw waits on the size answer recorded above it.

The throwaway home held no fleet file, so every metric printed its no-value mark. The run proved the 27 rows and their field names off the page, and proved no value.

## 2026-09-10 15:38 - #147 - the conservation law takes a fourth bucket

Quintessence now rests in four places, not three. The fourth holds what a thing in the world carries in itself.

```
before   wallets + held + pleroma             == total ever minted <= 33,000,000
after    wallets + held + pleroma + embedded  == total ever minted <= 33,000,000
```

The fourth bucket does not circulate, because only a wallet circulates. It sits inside the thirty-three million like every other bucket. Nothing new is minted to fill it.

### Where a thing's Quintessence comes from

It is drawn out of the pleroma, which is where Quintessence at rest already lives and which already had a way out. That is what keeps the ceiling honest: if a newly drawn material's Quintessence appeared from nowhere, the cap would be a number with nothing behind it.

```
drawn into the world   pleroma -> embedded
put in by a maker      wallet -> embedded, and wallet -> pleroma for the rest
taken back out         embedded -> wallet, and embedded -> pleroma for the rest
taken out with none
  recovered            embedded -> pleroma, all of it
```

Nothing is created and nothing is destroyed at any step. A break that recovers nothing sends the whole amount back to the pleroma rather than losing any of it, which is what the rule against destruction requires.

### The movements are named for the buckets, not for what holds the units

A creature, an item and a lump of ore all use the same four paths, so none of the names says what kind of thing is involved.

`src/competition/quintessence_ledger.py` — the four new movements

```
def embed_from_pleroma(self, amount: object) -> Decimal:
def embed_from_wallet(self, address, amount, embedded_amount) -> QuintessenceEmbed:
def release_from_embedded(self, address, amount, recovered_amount) -> QuintessenceRelease:
def release_all_to_pleroma(self, amount: object) -> Decimal:
```

The part that is lost when a maker puts Quintessence into a thing is recorded under the same name the platform already uses for a wallet losing units to the pleroma, because it is the same movement between the same two buckets.

### The books balance at the smallest figure he gave

His ore values are 0.00000001 and 0.00000005. Both are whole numbers once written in the smallest unit, so nothing rounds.

```
0.00000001   as the smallest unit   10,000,000,000
0.00000005   as the smallest unit   50,000,000,000
1,000 ore at the richest value      0.00005000, exact
the whole cap, at the richest value 660 trillion ore units
```

All four movements were driven at that size, on a throwaway ledger file and again on the chain. The difference between the four buckets and the total ever minted was zero after every one of them.

```
drawn into the world    w 0.0000496      p 0.00000039     e 0.00000001   delta 0
put in by a maker       w 0.00004955     p 0.00000041     e 0.00000004   delta 0
taken back out          w 0.00004956     p 0.00000044     e 0            delta 0
taken out with none
  recovered             w 0.00004955     p 0.00000045     e 0            delta 0
```

On the chain the same five movements left the pleroma exactly 0.000000033 higher and the wallets exactly 0.000000033 lower. Changing that figure by one part in a million million million makes the check fail, so it is reading the real numbers.

### The fourth bucket is read, and here is the proof

The old three-bucket sum was run over the same state. It reports the books short by 0.00000004 — exactly what the fourth bucket holds. Setting the fourth bucket to zero, and then one ore unit too high, both make the ledger refuse to write.

```
four-bucket sum    delta 0                 balanced
three-bucket sum   delta -0.00000004       short by what is embedded
fourth bucket zeroed    refused
fourth bucket inflated  refused
fourth bucket negative  refused, one negative bucket
```

### Sixteen checks, forty runs

The chain checks were rewritten for four buckets and grew from ten to sixteen. They name their own target, so the runner drives nothing else. Every one of the sixteen was shown failing on a broken law before it was trusted passing.

```
runs per campaign    256
calls per campaign   16,384
campaigns            40, each with its own starting number
result               40 of 40 pass
checks               16 of 16 shown able to fail
```

Twelve separate breakages were put into the contract one at a time, and each was taken out again leaving the file identical to the byte.

### Only this contract changed

Every other contract was rebuilt from scratch and came out identical. The comparison was then shown able to report, by moving a number in a contract this unit never touched and watching its result change.

```
ACRV, Governance, AcervatorTrophy, AcervatorLoot,
MetadataLib, CompetitionRegistry                  identical
Quintessence                                      changed, by intent
ACRV with one number moved                        changed, then identical again
```

### What an auditor must now check instead

The earlier audit confirmed a three-bucket law. That property no longer describes the contract, and saying so plainly matters more than the new one passing.

An auditor now has to confirm four things. That the four buckets add to the total ever minted. That the fourth bucket can only be filled from the pleroma or from a wallet, and never from a new mint. That every unit taken out of the fourth bucket lands in a wallet or the pleroma, and that a break recovering nothing sends all of it to the pleroma. That only the registry can draw into the fourth bucket or take out of it, while putting units in from a wallet stays the wallet holder's own call.

### The same code, a different chain

The panel already takes its chain when it is built, and the chain only decides which file the ledger reads. The four movements were driven on the demo chain's own file and the live chain's file was left at zero, through one shared install path and no flag of any kind.

```
live      quintessence_ledger.json
testnet   quintessence_ledger_testnet.json
```

### What this does not build

No material list, no quality scale, no recipe and no skill curve for recovery. The split between what is kept and what is lost is handed in by the caller, so those later pieces decide it without this law changing again.

## 2026-09-10 17:12 - #147 - the chain adds what changed, and a snapshot shortens the load

### A save costs what changed, not what the chain holds

The chain now keeps two files. The chain file holds a full snapshot, and a log file beside it takes each new record on the end. Driving 550 saves while the log grew from 75 kilobytes to 41.6 megabytes, the platform reported its own cost every time.

```
log already holds   records added   bytes added   cost
75 KB               202             75,319        3.5 ms
19 MB               200             75,700        3.3 ms
41.6 MB             200             75,638        3.5 ms
over 550 saves      least 3.0 ms, most 14.3 ms
```

A save at a 41.6 megabyte log costs what a save at a 75 kilobyte log costs. The cost follows the records added and nothing else.

### Where a whole-file write stops, and where this one does not

The whole-file write is now only the snapshot, and it carries the same cost it always did. The platform reported that cost at two sizes, split into building the payload and writing it.

```
snapshot bytes   build      write      together   per megabyte
18,075,873       153.8 ms   112.9 ms   266.7 ms   15.5 ms
36,156,441       388.9 ms   241.7 ms   630.6 ms   18.3 ms
```

Fifteen milliseconds a megabyte is confirmed, and at 34.5 megabytes a whole-file write takes 630 milliseconds. The half-second timer is past at exactly the size the world budget named. An append at that same chain size takes 3.5 milliseconds.

```
34.5 MB whole-file write   630.6 ms, over the 500 ms timer
same chain, one append     3.5 ms
```

### What makes a snapshot happen

A count of records, not a clock. The load a world has to wait through is the records since the last snapshot, so bounding that count bounds the wait whatever the hour.

```
CHECKPOINT_RECORD_INTERVAL = 50_000
```

At twenty layers 14,220 records fill one world turn, so this is about one snapshot every three and a half hours and a load that never replays more than about a second of records. The world budget recommended about once a day; a day at twenty layers leaves 341,280 records to replay, which is longer than the 4.9 seconds that same measurement called free.

### A load reads the snapshot and the tail, or the whole log

Both were driven on one chain of 25,196 blocks and 25,195 transactions, with a competition in it. The second run had the snapshot file removed.

```
load                          records replayed   cost
snapshot and the tail         368                429 ms
log alone, snapshot removed   50,401             663 ms
```

Every figure the platform reports about the two was the same: 25,196 blocks, 25,195 transactions, 9 events, 1 competition, 10 ACRV to one holder, verified, nothing legacy and nothing altered. The saving grows with the history, because the log read is 3.8 milliseconds for a tail and 485.7 milliseconds for the whole log.

### The log decides, and the snapshot is refused when it disagrees

A snapshot names the last log record it absorbed. Chain A's snapshot was put beside chain B's log, and the chain that loaded was B's.

```
A snapshot + B log   loads B, competition COMP-1DA0EFC1
A snapshot + A log   loads A, competition COMP-489CF6D4
```

A log shorter than the snapshot is a different case and is not a disagreement. It holds nothing the snapshot does not, so the snapshot stands and the short log is emptied.

### A record cut part way through is refused, and every earlier one kept

The log was cut by 150 bytes, which lands inside its last record. The load kept everything before it and dropped the rest of the file.

```
whole records read   367 of 368
bytes dropped        218
transactions         25,194, one fewer
everything else      25,196 blocks, 9 events, 1 competition, 10 ACRV, verified
```

The same cut was driven twice, once with the snapshot and once without it, and both read 367 records. Nothing partial was taken for a real record.

### A record that was changed fails its own name

Every record is named by a number taken from its own contents. One digit was changed inside the last record, leaving a line that still reads as valid, and the load refused it for the name alone.

```
line length      unchanged, still valid
whole records    367 of 368
transactions     25,194
```

Nothing was invented to mark a record's end. A cut record fails to read, and a changed record fails the number its own contents give it.

### His own chain loads, and a save no longer rewrites it

His chain file was copied and loaded through the same install path the window uses. A competition was then run on it and saved.

```
loaded          3,136 blocks, 3,135 transactions, 2,565 events, 285 competitions
                2,850 ACRV, 285 holders, verified, 3,135 legacy
a competition   32 records added, 185,843 bytes, 4.3 ms
his chain file  4,059,629 bytes, not rewritten
reloaded        3,147 blocks, 3,146 transactions, 2,574 events, 286 competitions
```

The old path would have rewritten 4.2 megabytes for those 32 records. His file is not rewritten at all until 50,000 records have gone by, and it was still byte for byte what it was when the run finished.

### Demo mode saves the same way

Each panel takes its chain when it is built, and the chain decides which file it saves to. A live chain and a demo chain ran in one program through one save path, with no flag of any kind.

```
live      testnet_chain.json            testnet_chain.log
testnet   testnet_chain_testnet.json    testnet_chain_testnet.log
```

Both added records and both wrote a snapshot, and each wrote only its own two files.

### What this does not reach

The snapshot is still a whole-file write, so the half-second stall returns once every 50,000 records and grows with the chain. Writing it away from the drawing thread is not built. A log damaged below the snapshot loses the records after it, because a damaged log cannot be asked what it held. ## 2026-09-10 17:40 - #147 - the controls that drive the mechanisms

Five units each reported that their mechanism works and that nothing on screen starts it. A row of buttons now sits under the state line, and every one of them calls the mechanism's own entry point.

```
Distil              mints Quintessence against a fee and a grade
Train transfer      records one use on the skill ladder
Send Quint          sends an amount, less the bleed to the pleroma
Spend a band        casts an action and rests its cost in the event pot
Score the action    puts a performance score on the participant's record
Settle the pot      divides the pot and pays every share
Close unpaid        closes an event nobody earned a share in
Draw loot           opens a rotation window and draws an item
File an exclusion   excludes a market from the next season
```

### Every button says what it did, in the mechanism's own words

The verdict line under the buttons carries the sentence the mechanism answered, whether it acted or refused. Nothing on the page writes a second message for a condition a mechanism already states.

```
acted     Sent 0.001 Quint at level 1: 0.00092 received, 0.00008 bled to the pleroma.
refused   Quintessence Transfer stands at level 0 on 0 weighted uses; level 1 costs 1
          and no transfer runs below it
```

### Each one was seen to act and to refuse

Every refusal below is one the mechanism already carried. Each was reached by pressing the same button in a different state, so no second rule decides it.

```
transfer   refuses while the skill is untrained, refuses while the balance is
           short, and then sends
spend      refuses while the wallet holds under the band's cost, and then casts
payout     refuses while nobody has a score, pays the scored, and refuses a
           second press
close      refuses while anybody has a score, and closes an event nobody earned in
drop       refuses while fewer than twelve markets qualify
season     files an exclusion, and the next season is what puts it in effect
```

### A premature payout would have denied whoever scored next, for ever

The payout stamps the records before the Quintessence moves, and that stamp is permanent by design, so a second payout cannot take Quintessence the pot no longer rests. There was no check that the division paid anybody. One press on an event before any participant had a score stamped it settled, paid nobody, and left the whole pot unreachable by the people who later earned it.

A payout that would pay nobody is now refused, and the refusal says which condition it is.

```
too early   monster_smash holds 0.001 Quintessence and no participant carries 1
            scored axis, so a payout would pay nobody and stamp the event settled
            for ever; 1 participant(s) stand unscored, and close_unpaid is the
            deliberate close
already     monster_smash was settled at 1789063555.8413775; a second payout would
            take Quintessence the pot no longer rests
```

Closing an event nobody earned in is still possible, through its own button rather than through a setting on the payout. Two buttons cannot be confused for one another, and the close refuses the moment anybody holds a score, so an earned share can never be closed away.

```
close refuses    monster_smash carries 1 scored participant(s) owed 0.00075
                 Quintessence; settle pays them and a close would deny them
close acts       dungeon_crawl closed with nothing paid: 0.001 Quint rests as
                 reserve and 1 participant(s) stood unscored.
```

The proof that the stamp was not burned is the payout that follows. The same event that refused the premature press paid its share once a participant was scored.

```
monster_smash divided 0.001 Quint: 0.00075 paid over 1 share(s), 0.00025 reserve,
0 unscored.
```

### An event picker, so every event type is reachable

The tab draws one button an event type, eight of them, beside the chain buttons. Every control button carries the selected event in what it sends, so a spend, a payout and a close all act on the event on screen.

```
monster_smash, monster_smash_elite, team_monster_smash, team_monster_smash_elite,
dungeon_crawl, dungeon_crawl_elite, raid, raid_elite
```

### The books balance after every press

The four-bucket report sits beside the pot division and is read again after each control acts. It held through all nineteen presses.

```
Buckets balance the mint   True
Negative buckets           0
Wallets                    0.09867
Held                       0.00125
Pleroma                    0.00008
Embedded                   0
Distilled, all time        0.1
```

### Nothing advances a season, and the page says so

The season counter lives in the competition registry contract behind a call that only the operations role may make, and no module under the source tree reaches it. The panel prints that rather than drawing a button that would claim to.

```
Nothing advances the season. currentSeason lives in
contracts/CompetitionRegistry.sol behind advanceSeason, which onlyOperations
gates, and no module under src reaches it. The governance unit that calls a gated
registry function is the one that would.
```

What the season boundary does reach is the exclusion rule. A market excluded in one season is unaffected in that season and excluded in the next, and the control reads both answers back off the rotation record.

```
In effect this season   False
In effect next season   True
```

### A chain picker, so the demo run needs no flag

The tab now draws one button a chain. Pressing Demo TestNet redraws the whole tab against the TestNet's own files, and every control button afterwards carries that chain in what it sends. No second code path exists, and no flag.

```
live      quintessence_ledger.json
testnet   quintessence_ledger_testnet.json
```

### The demo run cost nothing

Every control above was pressed on the TestNet chain. The live chain was then selected again and read back. Its wallet, its pot and all four of its buckets stood at nought, and the only files the run wrote were the TestNet's own.

```
live after the run      Balance 0, Distilled 0, Movements 0, Pot 0
files written           market_rotation_testnet.json
                        poa_record_store_testnet.json
                        quintessence_ledger_testnet.json
```

### The party window lists the fleet on both chains

The party list was reading a chain-suffixed fleet file, so the demo chain drew an empty party. A fleet is not chain state: the same bots play on either chain and only the ledger changes. The list now reads the one fleet file, and the demo chain draws the same forty slots the live chain draws.

### Nothing is subscribed to the live fill event

Minting from a real trade remains the operator's decision and no code here takes it. The Distil button is the only thing that mints, and a person has to press it.

### What these controls do not reach

The Draw loot button was never seen to return an item. A draw needs a rotation window, a window needs twelve qualifying markets, and the qualifying pool is built from the exchange scout the running program polled. The page's own process polls nothing, so the pool read nought and the floor refused every press.

The capture-bounds activation is the one place that builds a pool and opens a window together, and it refuses without a figure saying how much Quintessence a market's pool holds in a window. No unit has set that figure.

No button scores another participant. The Score the action button scores this node's own participant, so every payout proved above paid one share.

## 2026-09-10 18:57 - #147 - the world grid, its zones and what a discovery writes

### Three layers, and each one does a different job

A world now has three layers of structure, and they deliberately do not line up with one another. The grid is regular and carries addressing. A zone is a patch of terrain with its own shape. The tree levels are the third axis.

```
the world grid    regular. 81 addressable squares, 79 of them budgeted at one
                  square a participant. Addressing, the byte budget and sight.
zone regions      irregular, varied in size and shape, joined at their own
                  borders. A zone IS a terrain region.
Sephirot layers   the third axis. Twenty, read as the ten spheres and their ten
                  inversions.
```

Twenty is a reading of his wording and not a ruling. The code carries twenty as one derived line, and making it ten is a one-word change.

```
TREE_SPHERES = 10
SEPHIROT_LAYERS = TREE_SPHERES * 2
```

### The grid holds 81 squares and base sight covers one

79 participants at one square each is the measured figure from the world budget. 79 squares is a grid 8.9 on a side, which is not a whole number of squares, so the grid takes the smallest whole side that holds them.

```
participants a layer carries        79
squares a layer holds, one each     79
smallest whole side holding 79       9
addressable squares                 81
```

The declared size is free. The byte budget bounds how much a world writes in one turn, never how large the world is.

```
square (0, 0)                  0
square (8, 8)                  80
index 23 back to x and y       (5, 2)
square (9, 0)                  refused - off a grid 9 across holding 81
index 81                       refused - off a grid holding 81
```

Sight is one square, which is the participant's own.

```
squares base sight covers             1
squares in view from 23 at base        (23,)
squares in view from 23 at radius 1    (13, 14, 15, 22, 23, 24, 31, 32, 33)
squares in view from 0 at radius 1     (0, 1, 9, 10)
```

The last row is a corner. A radius that reaches off the grid yields fewer squares rather than an error.

### A zone is a terrain region, and it crosses squares

A grid square needs no description, since its index implies its shape. A region has to carry a boundary, and that boundary is the first thing in this design that costs bytes for shape. It is stored as a closed ring of points in world coordinates, each point named once, with the last point joining the first.

```
boundary  ((3.5, 3.5), (5.5, 3.2), (6.2, 5.1), (4.0, 6.0), (2.8, 4.9))
vertices  5
```

That ring form is what his rule asks for. Two neighbouring regions share the points along the border they meet at, so regions link at their own borders with no extra record. A ring of fewer than three points is refused, since it encloses nothing.

```
a 2-point ring   refused - zone ZONE-THIN carries 2 vertices, under the 3 a
                 closed ring needs
```

The five-point zone above spans parts of twenty squares.

```
squares in its extent   (80, 81, 82, 83, 84, 106, 107, 108, 109, 110,
                         132, 133, 134, 135, 136, 158, 159, 160, 161, 162)
as x and y              (2,3) (3,3) (4,3) (5,3) (6,3) (2,4) (3,4) (4,4) (5,4)
                        (6,4) (2,5) (3,5) (4,5) (5,5) (6,5) (2,6) (3,6) (4,6)
                        (5,6) (6,6)
zones over square 107   ('ZONE-1',)
zones over square 0     ()
```

### A zone's level reads the people across its squares

His rule is that a zone's level follows the participants in it. Because a zone spans several squares, that reading crosses squares rather than staying inside one. The squares give the candidates and the boundary decides who is actually in.

```
0xA  square 108  (4,4)    point (4.5, 4.5)  inside  level 40
0xB  square 109  (5,4)    point (5.1, 4.2)  inside  level 60
0xC  square 134  (4,5)    point (4.2, 5.1)  inside  level 20
0xD  square 84   (6,3)    point (6.9, 3.1)  outside level 99
0xE  square 540  (20,20)  point (20.0, 20.0) outside level 99
```

0xD is the row that matters. It sits in a square the zone's extent covers and outside the zone itself, so it is left out.

```
population            ('0xA', '0xB', '0xC')
squares they came in   (108, 109, 134)
level                  40, the mean of 40, 60 and 20
```

A zone with nobody in it reports no level at all, rather than a low one.

```
an empty zone's level   None
```

### A fact, a piece of knowledge, and one extraction

Discovery has three separate levels, and keeping them apart is what protects the Quintessence cap.

```
the FACT        the asset exists at that place. Written ONCE, by the first
                discoverer, and permanent. Its amount is fixed at that moment.
the KNOWLEDGE   this participant knows about it. Written PER PARTICIPANT, as a
                reference.
the EXTRACTION  the Quintessence is taken. Happens ONCE, since it is one asset.
```

The world is never stored per participant. A second discoverer writes one identifier, never a second boundary and never a second amount.

### Two people find one creature and it pays once

Two participants discovered the same place on the same layer.

```
0xFirst    wrote the fact  True   amount 0.957
0xSecond   wrote the fact  False  amount 0.957
same place id   WORLD-A:3:108:50:50
same amount     True
same leaf       True
facts the world holds  1
```

Then the extraction.

```
0xFirst extracts    0.957
0xSecond extracts   refused - WORLD-A:3:108:50:50 was already extracted by
                    0xFirst; one asset allows one extraction
two rolls would have paid   1.914
one reality paid            0.957
```

That refusal is the cap holding. If each discovery rolled its own amount, a world's fixed Quintessence budget would be multiplied by its population.

Knowledge is also required to take anything.

```
0xFirst extracting a place it has not found  refused - 0xFirst holds no knowledge
                                             of WORLD-A:3:108:51:51
```

### The amount is fixed before anyone arrives

If the amount were undecided until found, whichever node resolved the discovery would choose its own loot. The world therefore commits to a concealed seed when it is created, and an amount is a plain function of that seed and the place.

```
def discovery_leaf(seed: str, locator: str) -> str:
    """The leaf ``locator`` takes under ``seed``, which ``seed_commitment`` hides."""
    return hashlib.sha256(f"{seed}|{locator}".encode()).hexdigest()
```

That is the same salted-hash shape the rotating reward markets already use, where the chain carries a commitment, no field names a market, and the salt is published afterwards. The world reuses the shape rather than inventing one: the seed plays the salt's part and the place plays the market's.

```
at creation    the chain carries sha256(seed). No field carries the seed.
at discovery   the amount comes from sha256(seed|place), inside the band set for
               that kind of asset.
afterwards     the seed is published and anyone recomputes every amount.
```

Nothing invents a distribution curve. An asset kind has no amount until a band is registered for it, and the curve across ore, creatures and the deep layers is still his to set.

```
creature  0.001 to 1.000 in steps of 0.001    1000 amounts
ore       0.000001 to 0.000010                10
avatar    500 to 1000 in steps of 100         6
```

Driven over 20,000 places under one seed, every draw landed inside its band, both ends of each band were reachable, and the creature band produced all 1,000 of its amounts.

### The amount cannot be read early, and checks out later

Before the discovery the chain carries nothing that names the amount.

```
reading the amount    refused - no fact 'WORLD-A:3:108:50:50' is discovered
chain events so far   WorldCreated (arena, commitment, layers, width, world)
                      LayerBreached (by, layer, world)
                      ZoneDiscovered (boundary, by, layer, world, zone)
the chain carries the seed   False
```

After the discovery the same place checks out against the commitment.

```
leaf recomputed from the seed and the place   matches
amount recomputed from that leaf             matches
the seed still hashes to the posted value    matches
verified                                     True
```

The check was watched failing before it was trusted. An amount raised by one step, which is the smallest the band allows, a replaced leaf, and a swapped seed each turned it red, and restoring the value turned it green again.

```
amount 0.568 raised to 0.569   amount_matches False, verified False
leaf replaced with zeros       leaf_matches   False, verified False
seed swapped                   all three      False, verified False
restored                       verified       True
```

### What a region costs, in bytes

Measured by letting the chain save itself and reading the file the chain wrote. Every figure below is the size of that log file.

```
install, no world        545
a world nobody entered   1,844
one layer breached       2,960
a 3-point zone           4,208
a 6-point zone           4,276
a 100-point zone         6,341
a creature found         4,446
a second person knows    5,613
the Quintessence taken   6,834
```

Subtracting gives what one record costs.

```
a world's declaration    1,299
a layer breach           1,116
a 3-point zone           1,248
a 6-point zone           1,316
a 100-point zone         3,381
a creature's fact        1,486
a knowledge reference    1,167
an extraction            1,221
```

A point costs 22 bytes and a zone record carries 1,182 bytes of envelope around them. Each point is written twice, once in the action and once in the event the chain log shows, which is how every other record in this package is written.

```
each extra point     22 bytes
fixed envelope       1,182 bytes
the crossover        the envelope outweighs the geometry up to about 54 points
```

His warning holds for the geometry and not for the whole record. A hundred points cost fifty times two points in geometry, exactly, and the whole record grows only 2.7 times, because the envelope is paid once either way.

The commonest region shape is not set in the design, so here is the capacity each shape would buy against one layer's turn budget of 1 MB.

```
3-point regions     840 a turn
6-point regions     796
100-point regions   310
```

### An undeclared world costs nothing

This is the claim the whole world budget rests on. A world nobody has entered stores its declaration and nothing else, and the declared size is almost free.

```
a world 26 squares across, one layer breached    2,960 bytes
a world 1,000 squares across, one layer breached 2,974 bytes
squares the first declares                       676
squares the second declares                      1,000,000
```

A world 1,479 times larger costs 14 more bytes, which is the extra digits in its width and its arena. Nothing is spent on ground nobody has walked.

```
an undiscovered zone    has no boundary stored
an unvisited layer      does not exist
undiscovered Quint      has no location
```

Work on a layer nobody has breached is refused outright, which is what makes the second line true rather than aspirational.

```
a zone on an unbreached layer   refused - layer 3 of world WORLD-A does not
                                exist; layers breached: none
a layer past the declared count refused - layer 20 is outside the 20 layers
                                world WORLD-A declares
```

### The arena is the one place known without being found

Fixation happens at an arena, it is where everyone starts, and every other dungeon type is found out in the world. That solves the problem lazy discovery creates: a brand-new participant in an undiscovered world has exactly one place to go.

```
arena square      40, the middle of a grid 9 across
as x and y        (4, 4)
reachable with no discovery   True
the square beside it          False
zones discovered at creation  none
layers breached at creation   none
```

Four things separate the arena from a discovered zone.

```
it is a grid square, so its index implies its shape and it stores no boundary
it has no discoverer and no discovery record
every participant reaches it without a knowledge reference
it holds no Quintessence commitment, so nothing is extracted from it
```

A world still holds no zone, no layer and no asset at creation. The Map subtab's sentence that no world is generated stays true, because installing the mechanism creates no world.

### Demo mode builds a world on its own chain

Two chains ran in one program, each with its own world store, and the world took its chain at construction.

```
two chains in one program    True
the demo world writes to the demo chain   True
live chain blocks   10
demo chain blocks   4
both chains verify their own records   True
```

The same three calls ran on both. Different seeds gave different amounts, and each chain's amount checked out against its own commitment.

```
live amount   0.957
demo amount   0.184
demo verified True
```

No flag chose between them. The chain arrives by construction, the same way the certification socket and the rotating markets already take theirs.

### What the world grid does not build

No movement. A journey leg, a terrain cost and encumbrance are the next unit, and they needed zones to exist first.

No drawing. The Map subtab still says no world is generated, and that sentence is still true.

No link to the Quintessence books. Taking an asset records the extraction and returns the amount. Crediting a wallet belongs with the conservation law and its four buckets.

The distribution curve is unset, so no kind of asset has a band until one is registered. That curve decides how long a world takes to reach Ancient, and it is his to set.

## 2026-09-10 19:49 - #147 - the third bucket is the pleroma

The bucket that holds Quintessence at rest is called the pleroma. That is the word everywhere now: every field, every method, every constant, every log line, every screen row and every sentence on this page. Nothing else changed.

```
wallets + held + pleroma + embedded  == total ever minted <= 33,000,000
```

The law is the same law. The supply cap is the same 33,000,000. The bleed, the respawn, the two embed paths and the two release paths move the same amounts between the same places.

### What the panel prints

The Conservation panel's third row now reads Pleroma.

```
Buckets balance the mint   True
Negative buckets           0
Wallets                    0.09867
Held                       0.00125
Pleroma                    0.00008
Embedded                   0
Distilled, all time        0.1
```

### What an auditor must verify

The property has a new name and the old one is stale. An audit record written before today names a three-bucket law over `platonicTotal`; neither the count nor the name is current.

```
verify   walletsTotal + heldTotal + pleromaTotal + embeddedTotal
             == totalEverMinted <= SUPPLY_CAP

read     pleromaTotal is credited by bleed, by releaseFromEmbedded and by
         releaseAllToPleroma, and debited by respawn and by embedFromPleroma
stale    the recorded property name invariant_threeBucketsEqualTotalEverMinted,
         and every citation of platonicTotal in an audit dated before today
```

Sixteen invariants hold the sentence. Each was driven on forty different fuzzing seeds and each was shown able to report before the pass was believed.

```
seeds                        40
invariants per seed          16
passes                       640, no failures
blinded, one inversion each  16 of 16 reported FAIL
pleroma debit halved in
  respawn                    invariant_fourBucketsEqualTotalEverMinted caught it
```

### The contract's code changes and no other contract's does

The bucket is a named identifier inside the Quintessence contract, so its compiled code moves. Six other contracts were rebuilt from scratch and came out identical to the byte.

```
ACRV, Governance, AcervatorTrophy, AcervatorLoot, MetadataLib,
CompetitionRegistry                       identical
Quintessence                              changed, same length
```

### Nothing saved on disk carried the old word

A saved ledger records each movement by name, and two of those names carried the old word. No such file exists on this machine, and no file under the runtime directories holds the word at all, so the rename loses no state and the file version is unchanged.

```
movement names now   embed_from_pleroma, release_to_pleroma
files on disk        none carried the old word
file version         unchanged
```

## 2026-09-10 20:38 - #147 - movement is derived, and costs one record a leg

### His words set the shape

> *"Just need to be able to say that character A traversed x% of a given square in a given turn and this will make it relatively easy to simulate varied terrain within one or across multiple squares. Should be simple enough to make this dynamic and per character or army or group with encumberance playing a role."*
> *"Also, terrain grids do not need to overlap perfectly with the world grid. These grids are varied in size and shape while be linked together at their own borders."*

### A leg is one stretch at one rate, and a journey is a sequence of them

A mover crosses the world in legs. One leg runs at one rate across one terrain region and ends at that region's border. A journey is the legs in order, and a new leg is written when the rate changes.

```
a leg      one constant-rate stretch, ending at a terrain border
a journey  a sequence of legs
a new leg  a border crossed, a load changed, a group split
```

### Progress is read, never written

A leg records the turn it opened, its two end points and its rate. Everything else is worked out when somebody asks. No turn writes anything.

```
stored    the opening turn, the two end points, the rate, the zone
derived   the steps covered, the percent of a square covered this turn,
          the square the mover is in and how far across it, whether it arrived
```

A position is counted in steps, and a square is a hundred steps across, so one step is one percent of a square. His question is answered by reading the step count.

```
row = journeys.progress("u40world", "march", 3)
row["travelled_steps"]        # "300"
row["square_pct_this_turn"]   # "100"
```

### The same leg read at two turns, and the chain replaying both

One leg was opened and then read at two different turns. The two readings differ, because the turn is an input. The same two readings were then worked out again from the record on the chain alone, and both matched to the digit.

```
turn 1   100 steps covered, 100% of a square this turn
turn 3   300 steps covered, 100% of a square this turn
replay from the chain's own record    equal at turn 1 and at turn 3
the chain verifies its own records    6 blocks, 5 transactions, verified
```

Nothing on the leg depends on the machine reading it. Every figure is worked out to a fixed number of digits, so the ambient setting of the program cannot move it.

```
digits set to 5    the same answer
digits set to 60   the same answer
digits set to 28   the same answer
```

### Crossing a terrain border makes a second leg at a different rate

A walk began on open ground and crossed into marsh. The border crossing ended the first leg and opened a second, and the rate fell because the marsh carries a smaller multiplier.

```
leg 0   zone plain   100 steps a turn   400 steps to run, arrives on turn 4
leg 1   zone marsh    40 steps a turn   opens on turn 4
```

The turn of the handover belongs to the leg that was moving during it, not to the one that opens at its end.

```
turn 4   leg 0, plain, 100% of a square covered
turn 5   leg 1, marsh,  40% of a square covered
turn 9   leg 1, marsh,  40% of a square covered
```

### Writing progress every turn against deriving it

Three walks were driven, of three, six and twenty-four turns. Each cost one record on the chain, and each answered a progress question once per turn.

```
walk of  3 turns    1 record on the chain    3 progress readings
walk of  6 turns    1                        6
walk of 24 turns    1                       24
all three journeys  3 transactions on the chain
```

### What one leg costs, in bytes

Measured by letting the chain save itself and reading the file the chain wrote. The figure is the change in that file's size across 655 legs, divided by 655.

```
one leg, the path the program runs        1,305 bytes
its event alone                             422
one leg, packed argument, one block a turn  478
```

A leg is written twice, once as the action and once as the event, which is how every other record in this package is written. The packed figure drops the event and shares one block across the turn.

Against one layer's turn budget of one megabyte, with 655 movers, the difference between writing progress and deriving it is the whole question.

```
655 movers, a record each turn              854,581 bytes   81.5% of the budget
655 movers, a record each turn, packed      313,331         29.9%
655 movers, one record a 24-turn journey     35,607 a turn    3.4%
```

Two earlier figures for this were 1,015 bytes and 399 bytes, and neither reproduces now. The save path changed to a running log that names every record by its own contents, which adds bytes to each one.

```
quoted earlier   1,015 bytes   63.4% of the budget
measured now     1,305         81.5%
quoted earlier     399         24.9%
measured now        478        29.9%
```

The earlier conclusion is stronger rather than weaker. Walking would take more of the world's turn budget than the earlier figure said, not less.

### What sets a rate

A rate belongs to the mover and not to the square. Three things make it, and only the first is settled.

```
the base     one square a world turn. A one-hour turn and an hour's walk a
             square agree with no adjustment.
terrain      a multiplier, registered against the zone the leg crosses
encumbrance  a multiplier, given when the leg opens
```

Neither multiplier has a default and neither has a value chosen here. A zone with no registered multiplier refuses to carry a leg, and says which zones have one.

```
no terrain multiplier is set for zone desert of world u40world;
zones carrying one: marsh, plain, steppe
```

Nothing in the program holds what a mover carries, so the encumbrance multiplier is given by whoever opens the leg. An inventory would supply it, and no inventory exists.

```
In development.
```

### The kilometre label reaches no arithmetic

A square can be called five kilometres, or fifteen, or thirty. No distance in kilometres appears anywhere in the code, and no rate is worked out from one. The travel rates behind those labels are unsourced, and a label needs a source before it reaches a screen.

```
In development.
```

### A leg must stay in the zone it names

Two checks refuse a leg that wanders out of its region, and both use the zone test the grid already carries.

```
both end points   must sit in a square the zone's own extent covers
the midpoint      must sit inside the zone's boundary
```

A region that bends back on itself, where a straight leg leaves and re-enters while staying inside the zone's own squares, is not decided by either check.

```
zone plain does not cover square 141 at (1199, 500) steps;
a leg ends at the zone's border
```

### Demo mode walks on its own chain

Two chains ran in one program. Each had its own world and its own journey store, and the journey store took its chain when it was built.

```
two chains in one program                True
the demo leg is on the demo chain        True
the demo leg is NOT on the live chain    True
both chains verify their own records     True
live leg rate   100 steps a turn, arrives on turn 3
demo leg rate    25 steps a turn, arrives on turn 12
```

No flag chose between them. The chain arrives when the store is built, the same way the world and the certification socket already take theirs.

### What movement does not build

No armies and no groups. A rate belongs to a mover, and whether that mover is one character or forty is the caller's business.

No load. Nothing holds what a mover carries, so the encumbrance multiplier is given rather than read. Nothing bounds it either, so a caller may hand over a number that crosses the world in a turn. Whatever reads a mover's load will bound it, and no such reader exists.

No route planning. A leg's two end points are given, and nothing here decides where a border crossing falls.

No control. Nothing on the page opens a journey, registers a terrain multiplier or reads a progress figure, so every walk above was driven by reaching the installed store directly. The tab's controls own that.

No drawing. The Map subtab still says no world is generated, and that sentence is still true.

## 2026-09-10 21:26 - #147 - entity stats, measured in Quintessence

Every entity in PoA now carries stats, and a stat holds an amount of Quintessence outright. Nothing converts a stat into Quintessence. A Vessel, a monster and one component of an item all hold the same kind of record, and the Quintessence needed to occupy and control that entity is the sum of the stats on it.

```
src/competition/entity_stats.py
```

### A stat is an amount of Quintessence, so no exchange rate exists

The directive asked for stats that convert to a measurement or a function of Quintessence. The strongest reading makes the conversion identity: a stat value already is a Quintessence amount. Nothing multiplies, so there is no rate for anyone to pick and no second number to keep in step.

```
a stat value          IS a Quintessence quantity, carried as Decimal
an entity's stats     sum to the Quintessence it requires
a Vessel's occupancy  that sum
an item's cohesion    the same sum over the item's components
```

One function does all three. A Vessel and a monster hand it one record; an item hands it one record a component.

```
def quintessence_requirement(blocks: Iterable[StatBlock]) -> Decimal:
    """Add every amount of every block in ``blocks``, exactly.

    ``blocks`` holds one block a Vessel or a monster, and one block a component.
    """
```

### The stat set is provisional and lives in one table

A separate unit researches which stats exist. Until that answer lands the set is a placeholder, and it sits in one table so the answer has one place to go. Adding, removing or renaming a stat changes the table and nothing else.

```
STATS: tuple[StatDef, ...] = (
    StatDef("strength", SALT, "max weight"),
    StatDef("dexterity", SULPHUR, NO_EFFECT_NAMED),
    StatDef("constitution", SALT, "turn point penalty while carrying"),
    StatDef("intelligence", MERCURY, NO_EFFECT_NAMED),
    StatDef("wisdom", MERCURY, NO_EFFECT_NAMED),
)
```

Strength sets max weight and Constitution sets the carrying penalty. Those two meanings hold. The other three carry no effect, and the table says so in plain words rather than leaving the field blank.

Nothing in the code counts the stats. One runtime entry into the table moved every figure on its own:

```
shipped table        5 stats, a level 50 entity requires 750
one entry added      6 stats, a level 50 entity requires 900
rows on the page     5 becomes 6
one stat's curve     unchanged, 550 at level 100
no function edited
```

### Each stat sits in one of the three principles

The seven classes already map the Paracelsian principles onto the four roles, so the stats follow that table rather than a new one. Salt is the body, Sulphur is the active and combustive, Mercury is spirit and mind.

```
Salt      strength, constitution     both fix a property of the body
Sulphur   dexterity                  the principle the two Damage classes carry
Mercury   intelligence, wisdom       the principle the Healer and Support classes carry
```

The first row follows from what the two stats already do. The other two rows read the class table, and they may change with the stat research.

The principle changes no price. Giving one principle a cheaper rate would make one stat the correct stat for everyone, and it would need a figure nobody has chosen.

### The curve climbs the ten spheres of the Tree

A stat advances in ten bands of ten levels, and the band is a sphere on the Tree. The rate inside a band is the band's own position, so a level in the first sphere adds one Quintessence and a level in the tenth adds ten.

```
TREE_SPHERES        10, imported from the world grid, not declared twice
LEVELS_PER_SPHERE   10, ARC_LEVELS over TREE_SPHERES
```

Driven across every level from one to one hundred:

```
sphere 1   opens at level   1   1 a level   stat reaches  10
sphere 2   opens at level  11   2 a level   stat reaches  30
sphere 3   opens at level  21   3 a level   stat reaches  60
sphere 4   opens at level  31   4 a level   stat reaches 100
sphere 5   opens at level  41   5 a level   stat reaches 150
sphere 6   opens at level  51   6 a level   stat reaches 210
sphere 7   opens at level  61   7 a level   stat reaches 280
sphere 8   opens at level  71   8 a level   stat reaches 360
sphere 9   opens at level  81   9 a level   stat reaches 450
sphere 10  opens at level  91  10 a level   stat reaches 550
```

The requirement never falls and never stands still. Over all one hundred levels it fell on none and held on none, and the step grows from 5 to 50 as the bands change. An entity at the top of the arc needs 2,750 Quintessence against a supply cap of 33,000,000.

### A Vessel under its requirement runs below full, and is never refused

A holder short of the amount still occupies the Vessel. The reading answers what fraction of full potential the balance reaches, and full is one case of it.

```
a level 12 Vessel      band 2, every stat at 14, needs 70
a balance of 40        reaches 0.5714285714285714285714285714 of full
the shortfall          30
is_full                False
```

The full reading compares the two Quintessence amounts directly and never the fraction, so a balance one hundredth short cannot round up into full.

The program says the same thing in its own log:

```
acervator.entity_stats INFO a balance of 40 against a requirement of 70
reaches 0.5714285714285714285714285714, shortfall 30
```

### No stat is cheaper than another

If one stat bought more power a Quintessence than another, every player would raise that one. Under the identity reading a point costs the same everywhere, and moving points between stats changes nothing.

```
one extra point        costs 1, in every stat in the table
all points in one      a level 50 entity requires 750
spread evenly          a level 50 entity requires 750
```

The price therefore carries no cheap direction. Power per point is a different question, and it belongs to whatever reads a stat. Two stats have a named meaning and nothing built reads either, so no stat converts into an advantage today.

An entity whose stats sum past the supply cap meets no refusal. Nobody could ever hold that much, so the reading stays under full permanently, which is the same answer the partial rule gives everywhere else.

### Every figure, and where it came from

```
TREE_SPHERES = 10        already in the world grid, with ten levels a sphere
ARC_LEVELS = 100         already in the classes module
FIRST_LEVEL = 1          already in the classes module
LEVELS_PER_SPHERE = 10   ARC_LEVELS over TREE_SPHERES, computed
rate inside a band       the band's own number, no coefficient
a stat point             one Quintessence, the identity itself
supply cap 33,000,000    already in the Quintessence ledger
FIRST_SPHERE = 1         the first of the ten bands
```

Nobody picked a figure here to make a curve feel right.

### What reads this

Nothing. The occupancy gate that would refuse or degrade a Vessel does not exist, and neither does the equip check that holds gear to the equipping player's Quintessence. The character stats subtab does not draw these rows.

```
In development.
```

### What the stats do not build

No turn points. Constitution sets a penalty against a turn budget, and no turn budget exists to subtract one from.

No weight and no encumbrance. Strength sets max weight, and nothing weighs anything. The journey store takes an encumbrance multiplier as an argument and no code derives one.

No per-level record. A stat block comes out of the level on each call, so nothing holds it and nothing can drift.

No spread rule. A level 100 entity may put every point in one stat for the same price as spreading them, and no rule gives a stat a floor. Whatever gate reads this will decide that.

## 2026-09-10 21:30 - #147 - a reset control, two meters and the wallet's fourth holding

> *"PoA - Demo Mode - ... Must be able to reset the testnet."*

The reset itself was already built and locked, on a screen that was retired. Two buttons now sit at the end of the control row and reach it.

```
Ask what a reset deletes            names both files, their bytes and the block height
Delete both files and start fresh   calls the reset the shared chain already carried
```

### The first button deletes nothing

Pressing Ask loads the chain and reports what a reset would take. The verdict reads refused, and every figure under it is the chain's own.

```
testnet_chain_testnet.json and testnet_chain_testnet.log hold 38778 bytes at
block height 32. Confirm the reset deletes both; this control deletes nothing.

Block height             32
Chain events             24
Bytes both files hold    38778
```

Reading the page again after that press showed the same block height and the same 38,778 bytes. Nothing moved.

### The second button acts, and the chain starts again at its genesis block

```
testnet_chain_testnet.json and testnet_chain_testnet.log are deleted and the
Demo TestNet chain stands at block height 0 for the reason the Accumulation
tab's reset control.

Block height             0
Chain events             0
Bytes both files hold    0
```

### A deliberate reset and a schema wipe do not read the same

One signal announces both. Every reset carries a reason, and the panel under the buttons prints the two reasons together, so a reader can tell which one happened.

```
A deliberate reset reads   the Accumulation tab's reset control
A schema wipe reads        schema version upgrade (another schema → 1)
```

### Reset acts on the demo chain and refuses on the live one

The live chain is the one a running window holds in memory. Deleting its files would leave that window free to write the chain back, so both buttons refuse there and say why.

```
Reset clears the Demo TestNet chain. The Live chain is the one a running window
holds in memory, which would write it back, so this refuses there.
```

### Two meters, side by side

> *"A block fill and real time turn completion meter next to each other."*

They sit in the player window as one pair under one heading, two equal cards on one baseline.

```
BLOCK FILL         2.7%    28230 of 1048576 bytes
TURN COMPLETION   98.7%    296s of 300s elapsed
```

Together they answer a question neither answers alone. A block near full with the turn barely begun is a world running hot; a quiet block with the turn nearly closed is a world with room to spare.

### The byte bound is now a number the program reads

The fill meter needs something to fill against, and the only bound is the one the world budget recommends: one megabyte a layer a world turn. That figure was a line in a report until this unit, and it is now a constant beside the chain's checkpoint cadence.

```
#: One layer's byte ceiling for one world turn, which buys 2,621 records at 400 bytes.
TURN_BYTE_CAPACITY = 1_048_576
```

### The fill meter costs two reads and parses nothing

A meter that redraws every second must not walk the chain. This one asks the file system for the length of the checkpoint and the length of the log, which is what the save path wrote, and adds them.

```
28230 of 1048576 bytes   2.7%
38778 of 1048576 bytes   3.7%   after ten more records were saved
0 of 1048576 bytes       0.0%   after the reset deleted both files
```

Nothing records the bytes one world turn wrote, so the meter reads every byte the chain holds since its last reset, and its note on the page says so.

### The wallet's fourth holding

The requirement, in his own words:

```
So a Quint wallet must be able to show Quint, NFTs, Loot, and Vessels all
tied to PoA.
```

Vessels joins Quintessence, trophies and loot, drawn the same way as the other three and reading the same way: a real Vessel, or a plain sentence saying there is none.

```
Vessels
Iron Edge             level 1 - Impetus 4
Summed requirement    --
```

A Vessel is the class a Reincarnate occupies. Nothing on disk keeps a set of Vessels against a wallet address, so the section reads the class pick the running request names and says plainly when that names nobody.

```
poa_record_store.json keeps no Vessel for this participant. A Vessel is the
class a Reincarnate occupies, and only a class pick names one.
```

### The summed requirement has no figure yet

Several Vessels are meant to sum their Quintessence requirement against one wallet total. No field anywhere holds what a Vessel's level requires, so the row is there and its value is two dashes.

```
No field holds the Quintessence a Vessel's level requires, so no requirement
sums against the balance above.
```

### What these three do not reach

No control on the page picks a class, so the Vessels section reads a Vessel only when a request names one. Nothing writes a Vessel to a file, and the record store that already keeps skill uses against an address is where one would sit.

The reset loads the chain it is about to clear, so a press costs one replay of that chain. The live chain is refused before any load, and nothing here changes the saved schema version.

---

## 2026-09-10 22:15 - #147 - the tab is proportioned, and the zones own it

### The two zones take the tab and split it in half

The tab used to divide its height into three equal rows, so the subtab panel was the same size as the player window and the same size as the party window. His layout gives the upper band the upper half and the party window the lower half, and that is what the tab now does. The control bar and the subtab panel are chrome: each takes the height its own content needs, up to a share of the tab, and scrolls past that share instead of pushing a zone down the page.

`src/gui/web/proof_of_accumulation_tab.css` - the seven rows of the tab

```
grid-template-rows: auto auto minmax(0, 15%) auto minmax(0, 13%) 1fr 1fr;
```

### What the player window measures now

Read off the rendered page in the Electron shell at the window size the shell opens, and off the rendered page in the desktop window at the same page height.

```
                        before   after
Electron shell            116      181
desktop window           1055      182
```

The desktop window's figure moved for a second reason. Its page had no height to divide, so every row grew to its own content and the whole tab stood 3,506 pixels tall inside a 696-pixel window. The square enemy screen took its side from that height and left the player window 260 pixels of width; at a 900-pixel-wide window it left 18. The page now carries a height and the enemy screen is a 182-pixel square.

### Every band inside a zone keeps its own height

The bands stacked inside the two zones used to share the zone between them, so a short zone drew several of them at no height at all. The eight mode rows in the player window and the forty slots of a party page were both drawing at zero. Each band now keeps the height its content needs and the zone scrolls.

```
                     before   after
mode list               0      150
party page slots        0       88
skill ladder           10       66
pot division           10       49
```

### The player window still scrolls

Its bands come to 410 pixels: the zone title, the event band, the two meters, the map button, the eight mode rows and the placeholder sentence. The zone is 181 pixels at the window size the shell opens and 254 at a full-screen one, so the mode rows are still reached by scrolling that zone. The eight event types are named twice on this screen, once as the event buttons in the control bar and once as the mode rows here, and dropping either copy is a change to what the tab says rather than to how it is sized.

## 2026-09-10 23:05 - #147 - the conversion rates, and the figures still owed

Every mechanism in this design turns something into Quintessence. Those rates sat in separate modules, or in nothing at all. One table now holds all of them, and every entry says where its figure came from.

```
src/competition/conversion_rates.py
```

### A figure is measured, decided or working

An entry carries exactly one of three words. The third one is the point. A working figure is one this table chose so the stitching could exist, and it declares itself rather than sitting in the code as a number nobody chose.

```
measured   read back out of the module that owns it, and the entry names that module
decided    the operator named it, and his figure is reproduced exactly
working    chosen here so the table can exist, and the operator replaces it
```

### Nothing reads a working figure without seeing that it is working

No function in the module hands back a bare number. The lookup answers the whole entry, so the provenance is in the reader's hand every time.

```
def rate_named(name: str) -> ConversionRate:
    """The entry in ``CONVERSION_RATES`` whose ``name`` matches, refusing any other.

    The whole entry answers, so a reader always holds its ``provenance``.
    """
```

The question "what figures does he still owe?" is answered by running something rather than by reading the file. Two readers list them, and both print their counts to the log.

```
working_rates()   every entry the operator still has to rule on
absent_rates()    every entry that carries no figure at all
```

### Eleven rows are anchored, and eight check against their own module

Three of the eleven are the operator's own figures, and this table is where they land. The other eight are figures a module already holds, so the table imports the real symbol and keeps no copy of its own.

```
stat_point_quintessence                      1             entity_stats.quintessence_requirement
stat_quintessence_per_level_at_sphere_1      1             entity_stats.quintessence_per_level
stat_quintessence_per_level_at_sphere_10     10            entity_stats.quintessence_per_level
quintessence_per_certified_fee_usd           1             quintessence_ledger.QUINTESSENCE_PER_FEE_USD
minimum_units_per_quintessence               100000000     quintessence_ledger.QUINTESSENCE_UNITS_PER_WHOLE
impetus_per_turn_at_level_1                  4             poa_modes.base_impetus
impetus_per_turn_at_level_100                9             poa_modes.base_impetus
steps_per_square                             100           world_grid.SQUARE_STEPS
iron_ore_quintessence_low_quality            0.00000001    his figure, held by no module before now
iron_ore_quintessence_high_quality           0.00000005    his figure, held by no module before now
world_budget_per_participant_quintessence    1             his rule, held by no module before now
```

The fee rate is a ceiling rather than a payment. Distillation multiplies it by a trade grade of zero to one, so one dollar of certified venue fee mints one whole Quintessence only on a perfect grade.

```
         amount = fee * QUINTESSENCE_PER_FEE_USD * grade
```

### The stat rate is the identity, and the table drives the real function to say so

A stat amount already is a Quintessence amount. The table does not restate that as a coefficient. It builds a stat block holding one point and asks the stats module what that block requires.

```
ONE_POINT_BLOCK = stat_block(
    {name: (1 if name == STAT_NAMES[0] else 0) for name in STAT_NAMES}
)
```

### His ore figures reach code here for the first time

The operator gave a band rather than one number, because one unit of ore has a quality. Both ends land as named figures, and both print exactly as he wrote them.

```
one unit of iron ore, lowest quality    0.00000001 Quintessence
one unit of iron ore, highest quality   0.00000005 Quintessence
```

### The smallest unit comes from the ledger, not from a copy here

The operator set one hundred million minimum units to the whole Quintessence, the same resolution as Bitcoin. The Quintessence ledger declares that figure, so this table imports the real symbol and keeps no copy of its own.

```
minimum_units_per_quintessence   100000000   quintessence_ledger.QUINTESSENCE_UNITS_PER_WHOLE
```

A rate is a ratio and needs no grid of its own. Rounding an amount onto the minimum unit belongs to the ledger, at the moment an amount enters a bucket.

```
def quantize_quintessence(amount: Decimal) -> Decimal:
    """Return ``amount`` rounded down onto the QUINTESSENCE_MINIMUM_UNIT grid."""
```

### His lowest ore grade sits exactly on the resolution floor

Two of his own figures meet here. The poorest unit of iron ore carries 0.00000001 Quintessence, and that is one minimum unit exactly. Nothing poorer than his lowest ore grade can be held, so the ore band starts at the floor rather than above it.

```
QUINTESSENCE_MINIMUM_UNIT            0.00000001
iron_ore_quintessence_low_quality    0.00000001
iron_ore_quintessence_high_quality   0.00000005, five minimum units
```

### Twenty-six rows are working, and they are the list he still owes

Thirteen of those carry no figure at all. Thirteen carry a placeholder that can be replaced without touching a function.

```
TempResource_0001_low_quality                0.00000001   a non-ore material, lowest quality
TempResource_0001_high_quality               0.00000005   a non-ore material, highest quality
TempStat_0001                                1            dexterity, effect unnamed
TempStat_0002                                1            intelligence, effect unnamed
TempStat_0003                                1            wisdom, effect unnamed
item_cohesion_per_component_quintessence     1            what holds an item together
TempWeight_0001                              1            the weight one material unit carries
max_weight_per_strength_quintessence         1            what strength may haul
impetus_speed_per_constitution_quintessence  absent       the carrying penalty
loot_released_quintessence_calx              0.00000005   a destroyed Calx item
loot_released_quintessence_cauda_pavonis     0.00000005   a destroyed Cauda Pavonis item
loot_released_quintessence_flores            0.00000005   a destroyed Flores item
loot_released_quintessence_elixir            0.00000005   a destroyed Elixir item
loot_released_quintessence_magisterium       0.00000005   a destroyed Magisterium item
TempMonsterTier_descending_0006              absent       a creature at depth -6
TempMonsterTier_descending_0005              absent       a creature at depth -5
TempMonsterTier_descending_0004              absent       a creature at depth -4
TempMonsterTier_descending_0003              absent       a creature at depth -3
TempMonsterTier_descending_0002              absent       a creature at depth -2
TempMonsterTier_descending_0001              absent       a creature at depth -1
TempMonsterTier_ascending_0001               absent       a creature at depth 1
TempMonsterTier_ascending_0002               absent       a creature at depth 2
TempMonsterTier_ascending_0003               absent       a creature at depth 3
TempMonsterTier_ascending_0004               absent       a creature at depth 4
TempMonsterTier_ascending_0005               absent       a creature at depth 5
TempMonsterTier_ascending_0006               absent       a creature at depth 6
```

Three of the five stats name no effect in the stats table, so three rows stand in for them. Damage, restoration and support potency are the proposed readings, and they are the operator's to rule on.

```
dexterity      no effect named in the stats table    TempStat_0001
intelligence   no effect named in the stats table    TempStat_0002
wisdom         no effect named in the stats table    TempStat_0003
```

### A material other than iron ore takes the ore band, and no lore name is invented

Naming materials is content and belongs to the content issue. This table holds one placeholder slot for a material, and that slot carries the ore band, so the scale is right while the material itself is unnamed.

```
TempResource_0001   the iron ore band, until the operator names this material's own scale
```

### The loot rows are flat on purpose

Five loot tiers exist, each with a weight and two bonuses. What a destroyed item of each tier releases does not exist, so every tier carries the same placeholder. Flat is the honest placeholder: any slope across the five tiers is a design decision and it is his.

```
all five tiers   0.00000005 Quintessence released, no curve and no salvage loss
```

### A creature's Quintessence falls out of its stats, once a tier has a level

The monster table declares twelve tiers, and each has a row here. Their rows carry no figure, and the missing figure is not a Quintessence amount at all. It is the level each tier sits at. The amount then comes from the stats requirement, the same way a Vessel's does.

```
the descending six   absent; the tier's level is what is owed
the ascending six    absent; the tier's level is what is owed
```

### Constitution's penalty has a door and no figure

The stats table says constitution sets a penalty against a turn budget. The door it enters through already exists, because the Impetus grant takes a speed multiplier. No figure sets how much constitution buys back, and any figure here moves the turn economy, so the row stays absent and his.

```
def impetus_grant(level: int, speed_multiplier: object = 1) -> int:
```

### One check runs on every real start

The module drives every anchored rate against the module that owns it, at import, and refuses to load when one disagrees. A failure would mean a figure published here no longer matches the engine, and the two would drift apart with nothing reporting it. Zero disagreed on the first run. What the check would have caught is a wrong module or symbol name beside a figure, or a figure typed by hand instead of imported.

```
acervator.conversion_rates INFO drove 8 anchored conversion rates against their own modules, 0 disagreed
```

### What calls this table

Nothing. The table is imported and built on a real start of the Accumulation tab path, and no code calls it yet. Materials, items, crafting, salvage and the creature roster are the consumers, and none of them exists.

```
In development.
```

### What the conversion rates do not build

No materials and no items. This is the table of rates between things, not the things.

No minimum-unit grid. The rates are ratios, and the ledger owns the grid an amount lands on.

No salvage. The loot rows say what a destroyed item releases, and nothing destroys an item.

No screen. The Accumulation tab draws no row of this table.

## 2026-09-10 23:40 - #586 - the PvP vote, and who may destroy a Vessel

### His words set the rule

> *"There will be PvP Worlds and Events. A world entering PvP mode is determined by an active Player Vote and that can put forth once every 24hrs. The voting window persists for 15m or three 5m candles. Only while in PvP mode or participating in PvP events can one player destroy another's Vessels. This will allow players to have specific Vessels they are willing to fight to the death with..."*

Somebody calls a vote in one world turn, and it settles on the next world-turn boundary. Actions placed in the peaceful turn resolve peacefully, and PvP begins with the next turn's placements. A participant has to log in once in an hour rather than be awake at one particular minute. A vote called while a rival guild sleeps also fails outright, because every absent player stays in the count the majority has to beat.

```
called on world turn 100     the peaceful turn
settles on world turn 101    the first PvP turn
resolution on turn 100       refused
a sleeping majority          blocks the vote
```

### The window is three standard candles, and the clock already existed

His fifteen minutes is three turns of the standard five-minute candle. The modes module already measures a turn off the shared candle clock, so the vote declares no clock of its own and counts three of those turns.

```
standard candle      300 s
three candles        900 s
his window       15 min  =  900 s
```

### A majority of the whole world carries it, and there is no separate quorum

> *"51% or higher. Proper Democracy over here..."*

The majority counts against every participant in the world, never against the people who happened to vote. No separate turnout test exists, because 51 per cent of the electorate cannot vote in favour at under 51 per cent turnout. The threshold is its own quorum. Every figure is whole numbers multiplied across, so no decimal fraction of a vote exists.

```
participants on one layer         79      votes to carry      41
a full world of twenty layers  1,580      votes to carry     806
```

Driven on a full world. Eight hundred and six votes in favour carried the vote and turned the mode on at the next boundary, at a turnout of fifty-one per cent exactly. Eight hundred and five did not, and the mode stayed off.

Every roll from one to one thousand five hundred and eighty then met a count-up search for the same figure, and the two agreed on every roll. A decimal version of the same sum disagreed on fifteen.

```
806 of 1,580 in favour   carried      turnout 51.0%   mode on from world turn 101
805 of 1,580 in favour   not carried  turnout 50.9%   mode stays off
rolls 1 to 1,580         whole-number disagreements 0     decimal disagreements 15
```

A world where more than half the participants have gone quiet can never enter PvP mode. Their silence protects their Vessels, and it follows from the threshold rather than from any separate rule.

```
participants who never vote      more than half
the vote                         cannot carry
```

### The mode lasts twenty-four world turns and then lapses

His cap is one vote every twenty-four hours and a world turn is an hour, so the life of the mode and the gap between two votes are one number. A world goes back to peace unless somebody votes it into PvP again. Nothing stops a world staying in PvP indefinitely, and that costs a fresh majority of the world every twenty-four turns.

```
turn 100   the vote is called
turn 101   PvP begins
turn 124   the last PvP turn, and the earliest a new vote may be called
turn 125   the first mode lapses, and a vote called on 124 settles here
```

### Three carries in seventy-two turns lock the world for a week

> *"Three successive pro-PVP votes over 72hrs will lock the World in PvP mode for an entire week starting from the third vote."*

Seventy-two turns is the window the three carries have to fit inside, not the gap between them. At the tightest rhythm the cadence allows, three carries span forty-nine turns, so a world that misses a beat still qualifies. The lock begins at the third vote's resolution and runs one hundred and sixty-eight turns, which is a week of one-hour turns. Sustained aggression is now a commitment with a payoff rather than a daily chore: three carries buy the week outright, where holding it otherwise takes seven more separate votes.

```
vote 1   called 100   resolves 101
vote 2   called 124   resolves 125
vote 3   called 148   resolves 149
span     first call to third resolution    49 turns, inside the window of 72
lock     149 to 317                       168 turns
```

Successive means consecutive carries with nothing failing between them. One failed vote breaks the chain and the count starts again. The run drove both halves, and the window has an exact edge.

```
carry, carry, carry          carry run 3   lock from turn 149
carry, carry, FAIL, carry    carry run 1   no lock
three carries spanning 72    lock from turn 172
three carries spanning 73    no lock
three carries spanning 81    no lock
```

The lock outlives the twenty-four-turn mode, so the lock is the outer authority. The third vote's own mode lapses on turn 173 and the world is still in PvP at turn 180.

```
turn 180, the mode alone          off
turn 180, with the lock           on
turn 316, the last locked turn    on
turn 317, the lock lapsed         off
```

### The program refuses a vote called inside a lock

A vote that cannot change the outcome is a control that lies, so the program refuses it rather than accepting it and doing nothing. The first turn past the lock accepts a vote again.

```
a vote called on turn 172, inside the lock     refused
a vote called on turn 317, past the lock       accepted
```

Once a lock begins, nothing ends it early. A world that changes its mind on the second day stays in PvP for five more, and that holds every participant who voted against. The vote asked the majority three separate times, and this is the sharpest edge in the mechanism.

```
an unlock   Not built, and not asked for.
```

### What the permission answers

One call answers whether one participant may destroy another's Vessels right now. The answer is yes inside a live PvP mode, inside a lock, or in a PvP event, and no everywhere else. Every row below came off the built objects.

```
two participants, inside the mode                yes
the peaceful turn the vote was called in         no
after the mode has lapsed, with no lock          no
after the mode has lapsed, inside a lock         yes
a participant against its own                    no
a PvP event, with no mode at all                 yes
```

### Nine refusals, each driven, each with its accepted neighbour

A refusal is worth nothing unless the program accepts the case one step away. The run drove both sides of every boundary below.

```
a call 23 world turns after the last       refused
a call 24 world turns after the last       accepted
a call on a roll of nobody                 refused
a call on a roll of one                    accepted
a vote at second 900 of the window         refused
a vote at second 899 of the window         accepted
a resolution inside the calling turn       refused
a resolution on the next turn              accepted
a call on turn 172, inside a lock          refused
a call on turn 317, past the lock          accepted
the same participant voting twice          refused
a vote past the roll counted at the call   refused
a second resolution of one ballot          refused
a vote after the ballot has settled        refused
```

### Nothing counts a world's participants

The majority counts against a roll the caller hands in, because no module keeps a list of a world's participants. The world store reports how many of them have discovered something, which is a different number, and the layer figure gives a capacity rather than a roll. The ballot freezes that roll at the moment of the call, so a participant who votes and then leaves cannot shrink the number their vote counted against.

```
PROPOSED
src/competition/world_grid.py
    PoaWorld.participants(world_id) -> tuple[str, ...]
    the addresses enrolled in one world, which the vote would count
```

### No screen calls any of this

No control calls a vote, casts one, or resolves one, and nothing in the running program reaches this module at all. The unused-function check names the six calls a screen would make, and the two the module calls on itself do not appear.

```
named unused    call_vote  cast_vote  resolve  mode_from  may_destroy  ballot_row
called inside   carry_run  lock_from
```

### Nothing keeps a Vessel, so there is nothing to destroy

The permission is the gate and the thing it guards is absent. The tab already says so in its own words, on screen, for any participant.

```
"keeps no Vessel for this participant"
```

### What the PvP vote does not build

Vessel destruction is not built, and neither is a PvP event. Whether any of the eight existing event types is a PvP one is his ruling, and nothing sets the event flag the permission reads. Who may call a vote is not settled either: the ballot records its caller and checks no privilege.

```
Vessel destruction         In development.
a PvP event flag           In development.
the right to call a vote   In development.
a control to cast a vote   In development.
```

## 2026-09-10 23:55 - #585 - a glyph and a colour for each kind

### Dwarf Fortress draws its world in characters, and so does this

The operator asked for a world map in the manner of Dwarf Fortress, sharper, with hermetic icons. Sixteen kinds in the package now carry a stand-in mark: the seven classes, the five loot tiers and the four event modes. A mark is one character and one colour, and the module derives both.

`src/competition/map_glyphs.py` - one mark a kind

```
@dataclass(frozen=True)
class MapMark:
    kind: str
    family: str
    label: str
    glyph: str
    colour_token: str
```

### A class's glyph comes from its planet, not from a choice

The class table already gives each of the seven classes one classical planet, and every one of those planets has its own character in Unicode. The module reads the glyph out of that table instead of picking one, so a class cannot drift from its mark.

`src/competition/map_glyphs.py` - the seven planets

```
Saturn    U+2644  Lead Ward            Luna      U+263D  Silver Mirror
Jupiter   U+2643  Tin Bulwark          Mercury   U+263F  Quicksilver Draught
Mars      U+2642  Iron Edge            Venus     U+2640  Copper Conduit
Sol       U+2609  Solar Lance
```

Mars and Venus are the codepoints Unicode names MALE SIGN and FEMALE SIGN. Those are the standard astronomical and alchemical marks for iron and copper, and the module says so where a later reader would otherwise correct them.

### The other eleven marks

The five loot tiers take the alchemical stage each name denotes, rising from the barred circle of the calcined residue to the pentagram of the Great Work. The four event modes take a heraldic mark each.

```
Calx           U+2296      Fixation              U+2720
Cauda Pavonis  U+26B9      Coagulation           U+2691
Flores         U+2698      Descension            U+2656
Elixir         U+2625      Cementation           U+26E8
Magisterium    U+26E4
```

### A colour is a style name, never a new colour

Every mark names a colour out of the one style table the whole interface draws from. A class takes the colour of its role, a loot tier takes its place on a rarity ramp from grey to gold, and an event mode takes its place on a heat ramp from the easiest to the hardest. Ten names cover the sixteen kinds.

`src/gui/react_proof_of_accumulation_tab.py` - the page carries what the marks name

```
MARK_SKIN = {f"--{name}": getattr(ds, name) for name in colour_token_names()}
```

### Where they are on screen

The player window carries a Map marks band under the mode list, listing every mark with its name and its codepoint. The seven class marks also sit in the class list in the party window, and the four mode marks sit on the mode rows. At the window size the shell opens, the player window's own scrollbar reaches the band, the way it already reached the mode rows.

```
glyph spans drawn        31
  Map marks band         16
  class list              7
  mode rows               8
```

### What nothing supplies

The enemy screen is the zone that would draw an enemy, and no monster kind carries a mark, so that zone names what is missing instead of drawing an empty frame. Two more kinds the directive names have no entity behind them either, and the same panel carries both.

```
Monster           12 tiers and 21 names are declared; six tiers carry no name, and
                  no hermetic glyph set covers twelve positions
World fact kind   WorldFact.kind is text the caller passes in
Zone terrain      ZoneRegion carries a boundary and no terrain kind
```

### Both variants draw every mark

The tab rendered twice, once in the desktop window and once in the Electron shell, and every figure below comes off those two pictures, not off the code. All thirty-one drew, each as its own mark, with the same colour on both.

```
                         desktop window   Electron shell
glyph spans                    31               31
distinct codepoints            16               16
drawn as an empty box           0                0
```

Each page also carried a character no font holds, which proves a missing glyph shows as an empty box. Two permanently unassigned characters drew as empty boxes on both pages, next to marks that drew as marks.

## 2026-09-11 01:10 - #585 - the twelve monster tiers

### Monsters run six tiers down and six tiers up

The operator set the shape. Demons, Old Ones and Pure Nightmare Entities go below; Angels, Watchers and Others go above. A tier is not a species list. A tier names a position on an axis of dominant nature, so a goblin, a dragon and a slime each sit somewhere on that axis according to what the creature fundamentally is. Toward either extreme the nature fixes the form. In the middle bands a creature may still read as an animal or a person.

`src/competition/monster_table.py` - one row a tier

```
@dataclass(frozen=True)
class MonsterTier:
    depth: int
    name: str
    nature: str
    entities: int | None
    nominal_cell_px: tuple[int, ...]
    level: int | None
```

The number carries the side. A negative depth sits below the participant's own plane and a positive one above it, and there is no zero, so no tier reads as neutral and no reader has to learn that seven means up.

```
depth -6 to -1   six tiers down, every figure transcribed from the art brief
depth  1 to  6   six tiers up, declared and undescribed
no depth 0       the participant's own plane is not a monster tier
```

### The six below come out of the art brief, and the six above stay deliberately empty

The art brief commissions the tab's pixel art, and its six tiers make up the descending half. The table transcribes each one as the brief writes it: the creatures, the old text attesting each creature, the sprite size, the frame count, how many appear on screen at once, what the creature does in a fight and how it reads visually.

```
depth  name                                creatures   source examples
  -1   the surface floor                        3      Mesopotamian incantations
  -2   executors of a mandate or of fate        6      Epic of Gilgamesh, Theogony
  -3   the decan rank                          36      hermetic texts, unnamed
  -4   floor bosses                             7      Lugal-e, Theogony
  -5   the fall tier                            3      1 Enoch 6-16, Tobit
  -6   the summit                               2      invented, built on Lovecraft
```

Nothing in the operator's material names a single entity for the six tiers above. Those six tiers exist on the axis and carry nothing else. Their nature sentence is empty, their sprite size is empty, and they hold no creature. Inventing an angelic roster would break the one rule the art brief states about itself, which demands that every creature name the old text attesting it.

```
Angels, Watchers, Others   the three words he gave, held as waypoints
what each tier above owes  a nature sentence, a form rule, and its creatures
what is in code today      the six tiers, present and empty
```

### The thirty-six decans are thirty-six drawing jobs

The brief counts twenty-one designs across five tiers, and the decan rank is the sixth tier it leaves out of that count. Its own frame arithmetic settles what the decans are: nine hundred and thirty-six frames at twenty-six frames each makes thirty-six separate animations, not one animation under thirty-six names. The names themselves sit behind manuscript editions nobody has in hand, so the tier counts thirty-six and names none of them.

```
named designs                    21       across five tiers
creature names                   23       21 attested in a text, 2 invented
drawing jobs, all twelve tiers   57       21 named, 36 unnamed
frames, all twelve tiers      1,467       531 for the named set, 936 decans
```

### A pixel size is how big the drawing is, and nothing else

Each descending tier says what size an artist draws its sprites at, rising from thirty-two pixels on the surface floor to one hundred and sixty at the summit. That figure instructs an artist. It carries no power rating, no character level and no amount of Quintessence, and the table keeps it in a field named for authoring so nobody reads it as strength.

```
depth -1   32 pixels        depth -4    96 pixels
depth -2   48 pixels        depth -5    96 pixels
depth -3   64 pixels        depth -6   160 and 96 pixels
```

### All twelve owe a level, and that is the only missing number

Nobody picks a creature's embedded Quintessence. Once a tier has a character level, the amount falls out of its stats, exactly the way a Vessel's does. No source names a level for any of the twelve, so every row carries the level as absent and the program refuses to answer rather than handing back a stand-in.

```
levels owed        12
asked anyway       "the tier at depth -6 has no level, so no Quintessence
                    amount follows"
once a level is named
  level  1   embeds 5 Quintessence
  level 50   embeds 750 Quintessence
```

The rate table records all twelve creature tiers as owing this figure. It reads the depths off this table, so neither module can hold its own count of the tiers.

```
TempMonsterTier_descending_0001 to _0006   depth -1 down to depth -6
TempMonsterTier_ascending_0001 to _0006    depth 1 up to depth 6
```

### A creature earns its place two ways, and the refusals sit beside them

The operator wants a wide range of encounters without losing the tone. A creature meets both conditions: it holds a position on the nature axis, and it carries a source attested in its own pre-modern text. The table also records the excluded material as data beside those conditions, because a table holding only the inclusions cannot report that a proposed creature fails.

```
refused   nineteenth and twentieth century occult revival
refused   correspondence tables, tarot-to-sphere charts, planetary seal sets
refused   Golden Dawn godforms, Enochian alphabet plates
refused   reconstructed Egypt, against attested temple and funerary imagery
refused   anything Crowley, or beyond the old verified texts
```

Four proposals went through the refusal, and it named the term excluding each one. The real sources admitted cleanly.

```
"Crowley, Liber 777"            refused, carries crowley
"a Golden Dawn godform plate"   refused, carries golden dawn
"a tarot-to-sphere chart"       refused, carries tarot
no source at all                refused, needs an attested text
"1 Enoch 6-16", "Tobit"         admitted
```

The structure takes a far larger roster than the brief commissions. A creature is a row naming its own tier, so a hundred creatures at one tier need no change of shape, only a larger count on that tier. The brief's published frame totals stay pinned to the twenty-one designs it commissioned.

### The Watchers stand on both sides of the plane

The art brief puts the Watchers below, at the fall tier, citing the story of their descent and calling them the corrupt lineage that perverted the craft. The operator has since named Watchers as a family above. Both readings come out of that same text, because the Watchers are angels who fell, so that one family occupies both directions: above as what they were, below as what they became.

```
below, depth -5   the Watchers and the Nephilim, one design, 1 Enoch 6-16
above             Watchers, one of the three families he named
measured          Watchers is the only name appearing on both sides
```

The hinge of the twelve-tier structure sits here, and this page reports it rather than settling it. The table holds the overlap as a measured fact and refuses to start if that fact ever changes and nothing declares it.

### Nothing ties a monster tier to a world layer

The world has twenty Sephirot layers, being ten spheres and their inversions, and a dungeon has ten floors taking the ten sphere names. The monster axis has twelve positions. Twelve does not divide into twenty, nothing in the operator's material connects a tier to a layer, and the table says so rather than aligning them.

```
world layers       20   ten spheres and their inversions
dungeon floors     10   each takes its sphere's attested name
monster tiers      12   tied to neither
```

### What the monster table still owes

Two tiers below name a membership where the others name a nature, and the axis has no rule for placing an arbitrary creature. The table records both gaps instead of filling them.

```
a nature sentence        depth -3 names the decan rank, depth -6 names the summit
a placement rule         nothing says which tier an arbitrary creature belongs to
the six tiers above      nature, form rule and creatures, all absent
a level, twelve times    the only figure standing between a tier and its
                         embedded Quintessence
```

### Nothing draws it yet

The program reaches the table. Opening the Proof of Accumulation path loads the package, the package loads the table, and the table checks itself against the art brief's published counts before the program carries on. The glyph registry and the rate table read its counts, and the enemy screen prints them in a sentence.

```
reached            src/competition/monster_table.py, through the package import
checks on start    six, all passing
what the log says  "12 monster tiers, 6 below and 6 above, count 57 drawing jobs
                    and 1467 frames. 21 designs are named, 36 entities are not,
                    12 levels are owed, and ['Watchers'] appear on both sides"
on a screen        the enemy screen names the counts; no creature is drawn
```

## 2026-09-11 01:45 - #585 - monster generation

### A monster's identity comes out of the seed, the layer and the place

The operator set the rule. Generation derives a monster the way a world fact's Quintessence already comes out: a world commits to a concealed seed at creation, and every place on every layer has a locator. Hash those two together and the result decides what stands there. The derivation rolls nothing and keeps no record, so two people who derive the same place always get the same creature.

`src/competition/monster_spawn.py` - the whole of the derivation

```
def derive_monster(seed, locator, weights):
    layer, _ = require_locator(locator)
    leaf = discovery_leaf(seed, locator)
    tier_roll, depth = tier_for_leaf(leaf, weights)
    design_roll, design_name = design_for_leaf(leaf, depth)
```

### The hash is one value and the two draws read different digits of it

The world grid already reads the first sixteen digits of that hash to set a place's Quintessence. Generation never touches those sixteen. It reads the next sixteen to choose the tier and the sixteen after that to choose the creature. The amount at a place and the monster at a place are separate draws off one hash, and a reader cannot work either one back from the other.

```
digits  1 to 16    the Quintessence amount, which the world grid sets
digits 17 to 32    which of the twelve tiers
digits 33 to 48    which creature inside that tier
digits 49 to 64    unread
```

### Nothing reads a place nobody has found

Anyone holding the seed derives every place in a world, found or not. Anyone without it derives nothing at all. The chain carries no field naming a seed until the world publishes it, so before publication nobody can list a world's creatures in advance. After publication anyone can, which is the same trade the Quintessence amounts already make.

```
at creation            the chain posts the hash of the seed. Chain fields
                       naming a seed, measured: none
a discovered place     its own hash is already on the chain, so anyone derives
                       that place's creature and learns nothing about any other
an undiscovered place  a seed holder derives it and nobody else can
after the seed is published   anyone recomputes every place in the world
```

### How often a tier appears is the one figure nobody has set

A hash is even. Drawn evenly, a world would hold as many summit Avatars as surface afflicters, and the art brief says the opposite: four to eight creatures on the surface floor, one alone at a floor boss. An even draw is wrong. The brief publishes how many of a tier stand in one square at once, as a low figure and a high figure. It names no rule saying which of the two carries a draw share, and four ways of reading that one pair order the twelve tiers three different ways.

```
reading      the order it gives, most common tier first
low          -2  -1  -3  -6  -5  -4
high         -1  -2  -6  -3  -5  -4
span width   -1  -6  -2  -5  -3  -4
span sum     -1  -2  -6  -3  -5  -4
distinct orders across the four readings   3
```

Three of the four readings put the summit above the decan rank and one drops the surface floor to second. Nobody has set the figure, and the module holds no curve of its own. A caller passes the weighting in, and the module refuses a weighting that leaves any of the twelve tiers unnamed.

```
what is owed    how often each of the twelve tiers is encountered
what is built   the draw, which honours whatever weighting it is given
the refusal     "a tier weighting names every depth in [-6, -5, -4, -3, -2, -1,
                 1, 2, 3, 4, 5, 6] and carries no share for [...]"
```

### The decan rank hands back a tier with no creature name

Seven of the twelve tiers name no creature. The decan rank counts thirty-six and the art brief leaves every one of the thirty-six unnamed, and the six tiers above the participant's plane hold no creature at all. A derivation landing on one of those returns the tier and leaves the creature name empty, with a line saying what would fill it. It does not refuse, because the tier is real and populated, and it invents no name.

```
depth -3          36 entities, 0 names, so the creature name is empty
depths 1 to 6     no entity at all, so the creature name is empty
every other tier  one of the 21 designs the brief names
```

### Generation derives identity and refuses power

No tier has a character level. No derived monster therefore has stats, and none has an amount of embedded Quintessence. The derived record carries no level field at all, and asking it for its Quintessence raises, with the refusal naming the level as the thing that is missing.

```
the record holds   the place, the layer, the hash, the tier, the creature name
                   and the two rolls
it does not hold   a level, a stat, or an amount of Quintessence
asking anyway      LevelAbsentError - "the tier at depth -1 has no level, so no
                   Quintessence amount follows"
```

### Ten thousand places on one square

One run drove the derivation over every step pair of one square on one layer, ten thousand places, under a world seed of thirty-two letter a. Every place landed on one of the six weighted tiers, and twenty-two outcomes came up. Each tier took the share the run handed it, to within four tenths of a part in a hundred. Anyone with that seed gets these same numbers again.

```
places derived      10,000
distinct tiers           6
distinct outcomes       22    the 21 named designs and the unnamed decan rank

depth -1   share 4   expected 30.77%   got 30.67%
depth -2   share 4   expected 30.77%   got 30.83%
depth -3   share 2   expected 15.38%   got 15.41%
depth -4   share 1   expected  7.69%   got  8.07%
depth -5   share 1   expected  7.69%   got  7.54%
depth -6   share 1   expected  7.69%   got  7.48%

3:40:50:50   depth -6, the summit, a direct servant
3:40:50:50   derived again, the same creature
3:40:50:51   depth -1, the surface floor, Lilith
```

### Nothing calls the derivation yet

The enemy screen is the zone a monster would draw in, and it states in words that it draws no pixel art. Nothing reads a derived monster. The package does not import the new module either, so opening the Proof of Accumulation path does not reach it. One entry in the package's own export list would change that.

```
reached by the PoA path   src.competition, and monster_table inside it
not reached               src/competition/monster_spawn.py
owed                      the module's entry in the package export list
on a screen               nothing
```

## 2026-09-11 02:30 - #585 - materials and the Quintessence they embed

### His two sentences about the cap are one rule, and the ledger already holds it

The operator said the Quintessence inside a material does not count against the circulating hard cap. He also said it stays budgeted against the hard limit. Those read as opposites and they are not. The ledger keeps four buckets, not one, and every movement it allows keeps the four adding up to everything ever distilled. Embedded is one of the four. An embedded amount is inside the 33,000,000 limit and inside nobody's wallet at the same time, which is exactly budgeted and not circulating.

`src/competition/quintessence_ledger.py` - `conservation`, the sum every movement has to keep

```
buckets = wallets_total + held_total + self._pleroma + self._embedded
delta = buckets - self._total_ever_minted
```

### A material states what it holds, and moves nothing

The new module declares a band and reads it back. It mints nothing, spends nothing and moves nothing between buckets. Four methods on the ledger already do every movement an embedded amount can make, and the material module calls none of them. That split is deliberate: one place owns the accounting, and the content layer only declares figures for it to move.

`src/competition/materials.py` calls none of these, which is the whole of the movement path

```
quintessence_ledger.embed_from_pleroma       a world's own Quintessence embeds
quintessence_ledger.embed_from_wallet        a player's Quintessence embeds
quintessence_ledger.release_from_embedded    part comes back, the rest bleeds
quintessence_ledger.release_all_to_pleroma   none comes back
```

### The lowest grade of ore is the smallest amount the currency can express

He gave one figure pair: a unit of iron ore holds 0.00000001 to 0.00000005 Quintessence. Measured against the ledger, the low end is the minimum unit exactly and the high end is five of them. Ore spans the smallest amounts that can exist at all, and nothing embeds less than one minimum unit. The module refuses a band that tries to.

Measured, all four read off the running program

```
QUINTESSENCE_MINIMUM_UNIT                 0.00000001
iron ore at its lowest quality            0.00000001    one minimum unit
iron ore at its highest quality           0.00000005    five minimum units
a declared band under one minimum unit    refused, with the refusal naming both
```

### Quality has five grades, and nobody had to invent them

Quality is the axis that turns one material into a spread of amounts, so the grades had to come from something already settled. Three candidates were checked against his band. The five loot tiers fit it exactly, one minimum unit a step. The ten sphere bands cannot: they need a step of four ninths of a minimum unit, and the second grade then lands on an amount no bucket can hold. A trade grade is a reading of a real trade fill, runs continuously from zero to one, and is not a property of a material at all.

Measured, each scale driven across his band

```
scale               step across the band        every amount on the grid
five loot tiers     one minimum unit            yes
ten sphere bands    0.4444... minimum units     no, the second grade is off it
trade grade         continuous, zero to one     not a grade of a material
```

The five tiers were the closest fit for another reason as well. The conversion table already pairs them with these same ore figures: it carries one Quintessence-release row a tier, every one of them at his high figure. The grade ladder and the ore band were already sitting next to each other before this unit existed.

`src/competition/conversion_rates.py` - the pairing that was already there

```
def _loot_release_rates() -> tuple[ConversionRate, ...]:
    """One working entry a ``TIER_NAMES`` name, at ``LOOT_RELEASED_QUINTESSENCE``."""
```

### One unit of iron ore, at each of its five grades

Read off the constructed objects, not off the source. Every amount is a whole number of minimum units, his two ends reproduce exactly, and the three grades between them are the second, third and fourth minimum unit.

```
Calx            0.00000001
Cauda Pavonis   0.00000002
Flores          0.00000003
Elixir          0.00000004
Magisterium     0.00000005
```

### Seven materials, one an ore of each metal the classes already name

The class table gives every class a classical planet and that planet's metal. Seven classes, seven distinct metals, and iron is one of them, the metal his own example names. The material table therefore reads the metals out of the class table and names no material of its own. Change the classes and the materials follow.

`src/competition/materials.py` - `MATERIAL_NAMES`, in the order the class table declares the metals

```
lead ore    tin ore    iron ore    gold ore    quicksilver ore    copper ore
silver ore

iron ore          its band is the operator's, and marked decided
the other six     they take the iron band, and are marked working
```

That marking matters. Only one of the seven carries a figure he gave. The other six carry a placeholder with his figures in it, and the conversion table already says what that means: a material other than iron ore takes the iron ore band until he names its own scale.

### Gear takes a slot, consumables and resources stack

His rule was that the three must not conflict. Only gear takes fixed storage space, counted in slots. Consumables and resources stack to a large amount and take no slot. All seven materials are resources, so nothing here declares a piece of gear or a consumable; items are a later unit's work and they will name their own class.

`src/competition/materials.py` - `STORAGE_CLASSES`, with what each one is declared to do

```
gear          takes one fixed slot, does not stack    none declared here
consumable    stacks, takes no slot                   none declared here
resource      stacks, takes no slot                   seven declared here
```

How large a stack runs is absent. He said large, and no figure anywhere says how large. Whatever sets it will be read from the same place the slot count is read from, and neither exists yet.

### Weight is declared on every material, and nothing weighs anything

Weight is what the two stacking classes share; a slot count is not, because only gear takes slots. Every material declares a weight a unit. The figure is one weight unit, it comes from the conversion table, and the conversion table marks it working and says plainly that nothing weighs anything today. A real figure is owed and it is his.

Measured, read back off the conversion table

```
the row            TempWeight_0001, one unit of any material
its figure         1 weight unit
its provenance     working
its own note       "Nothing weighs anything today and no figure sets a
                   material's weight"
```

Weight is read on one occasion only, by his narrowing: encumbrance matters while a Vessel is moving items itself, such as after a foraging run. It is a function of strength for the maximum weight and constitution for the turn point penalty while carrying. Nothing in this unit builds any of that. No Vessel exists to carry anything, so the declared weight has no reader.

### Salvage is named here and built nowhere

Destroying an item to get its Quintessence back is his rule and a later unit's work. What this unit can say is the ceiling: the most a salvage could ever return from one unit of a material is everything that unit embeds, and not one minimum unit more. What share of that a salvage actually returns is absent.

`src/competition/materials.py` - the rule stated, with no figure attached

```
the ceiling          the embedded amount at that grade, and no more
the share returned   absent
his rule             much is lost without proportionate skill, and salvage of a
                     powerful item without it is destructive
what would set it    a salvaging skill level, and the loss curve he puts on it
the movement         quintessence_ledger.release_from_embedded already takes a
                     recovered amount and sends the remainder to the pleroma
```

### Every amount is checked at import, and the check has been seen refusing

The module drives all seven materials across all five grades every time it is imported, and refuses to finish loading if any amount breaks one of three rules. A failure would mean a material declaring Quintessence that no bucket can hold, which would take that amount outside the four-bucket sum and let the 33,000,000 limit be passed with nothing reporting it.

What the program printed, from the module's own logging

```
acervator.materials INFO drove 7 materials across 5 quality grades,
                         0 amounts broke a rule
```

The three rules were each driven against a band built to break them, and each one refused in its own words.

```
half a minimum unit        "is under the 0.00000001 minimum unit, and no material
                            embeds less Quintessence than the currency expresses"
one and a half units       "is not a whole number of 0.00000001, and no bucket
                            holds it"
an end that drifts         reported as not reproducing the declared figure
```

### What nothing supplies, and what nothing reads

Nothing in the running program reads a material. The module imports cleanly through the Proof of Accumulation path and the package does not export it, so a reader has to name it directly. One entry in the package's own export list would change that, and no screen shows a material either way.

```
reads this module              nothing
owed                           the module's entry in the package export list
owed to this module            a real weight a unit, and a salvage loss curve
owed by this module to others   a material list for an item, which crafting reads
on a screen                    nothing
```

## 2026-09-11 02:50 - #585 - the guild roster, and where a treasury holds

A guild now exists as a record. It carries its members, its officers and one treasury address, and four mechanisms that were already built can read it.

### A guild forms, and it holds a treasury

His own terms set both halves.

> *"only those holding PoA tokens can form a guild... guild membership locks and stakes PoA tokens, by guild rank... a guild may hold any number of members"*
>
> *"A guild holds a Quintessence treasury, filled two ways and spent one way. Filled by the transfer skill, from a member, paying the bleed. Filled by the guild's own event awards. Spent on action costs for guild members, inside an event. Never paid out to a member's personal wallet."*

The record is a name, a key, a founder, a list of members and a list of officers. The founder is the first member and the first officer, because an officer is the only one who can commit the treasury, and a guild opened without one could never spend.

```
guild = roster.found("The Gold Fold Collective", "bot-officer-0002")
roster.join(guild.key, "bot-actor-0001")

key        the_gold_fold_collective
treasury   poa_guild_treasury_the_gold_fold_collective
founder    bot-officer-0002, member and officer
```

The treasury address comes from the guild's key, so the same guild always derives the same address and two guilds can never share one. The name and the key derive the same address, which one run confirmed.

### The treasury holds in the wallets bucket, and one run measured it

An earlier entry on this page left a question open: the design calls a treasury a held address, and the ledger can only spend from a wallet. The two built paths that touch a treasury both answer the wallets side, so that is where a treasury holds.

Diagram: graph, left to right

- a member's wallet -> **the guild treasury, wallets bucket** *(transfer skill)*
- a member's wallet -> **the pleroma** *(bleed)*
- the guild treasury, wallets bucket -> **the event pot, held bucket** *(an action's cost)*
- the event pot, held bucket -> **participants' wallets** *(payout by performance)*

The transfer skill credits a wallet, and an action's cost leaves a wallet. One run drove both ends against one treasury address and read the two balances off the ledger.

```
transfer    sent 0.50000000   received 0.48000000   bled 0.02000000
treasury    wallet balance 0.48000000    held balance 0
the action  treasury 0.48000000 -> 0.47000000    actor 0 -> 0
the record  underwritten 0.010   spent 0   actions 1
the pot     held balance 0.01000000
```

The run added no bucket and minted nothing beyond the one unit it distilled. The four totals still balance and still sit under the cap.

```
wallets 0.97000000 + held 0.01000000 + pleroma 0.02000000 + embedded 0
  == 1.00000000 ever minted     delta 0     balanced true     within cap true
```

### Four things an officer's commitment now refuses

Before the roster, the treasury payment took an officer's name and a treasury address as given and checked neither. A caller could name a stranger as the officer, and the program accepted it.

```
offer_underwrite accepted a stranger as officer: 'bot-outsider-0004'
REFUSED: guild-held-only-0005 holds 0 Quintessence and band x10 costs 0.010
```

The roster answers four questions the payment could not. Each refusal names the guild and the addresses involved.

```
no guild        bot-outsider-0004 belongs to no guild and commits no treasury
no office       bot-plain-0003 is a member of the_gold_fold_collective and not
                an officer; 1 officer(s) commit its treasury
not a member    bot-outsider-0004 is no member of the_gold_fold_collective, so
                treasury poa_guild_treasury_... pays nothing for that action
paying oneself  bot-officer-0002 is both the officer and the actor; an underwrite
                takes two addresses and nobody underwrites their own act
```

The last one keeps his rule that the commitment takes two acts by two people. A guild of one is still allowed, because he wrote that a guild may hold any number of members. The roster refuses one address playing both parts.

### Every mode's guild flag, read for the first time

Every event mode declares whether it needs a guild. Until now nothing read that flag. The roster answers it, beside the same mode's own party floor and ceiling.

```
team_monster_smash   guild_required true    party 2 to 120
two members of one guild            admitted
one member only                     admits 2 to 120 and this party holds 1
an outsider in the party            1 of 2 are no members of that guild
one address seated twice            one address takes one seat
monster_smash with two seats        admits 1 to 1 and this party holds 2
```

The PvP vote freezes its roll of participants at the call, so nobody can move the threshold during the window. The roster does the opposite on purpose. A vote counts an eligibility total once; a treasury payment moves money, so it reads the roster at the moment it moves. The roster turns away a member who leaves between the officer's commitment and their own acceptance.

```
before leaving   require_underwrite answers guild the_gold_fold_collective
the member left  the guild now holds 2 member(s) and 1 officer(s)
after leaving    bot-actor-0001 is no member of the_gold_fold_collective, so
                 treasury poa_guild_treasury_... pays nothing for that action
```

### One address belongs to one guild

Quintessence moves only between members of the same guild. If one participant could belong to two guilds, that participant becomes a bridge between them and the rule stops closing anything. Membership is exclusive, and the roster turns away a second guild.

```
founder bot-officer-0002 already belongs to the_gold_fold_collective;
one address belongs to one guild, so a transfer between guild members
reads one guild
```

The roster writes out and reads back whole, so a transfer and a payout read the same membership after a restart. The stored file also names the bucket its treasuries hold in, and the roster turns away a file naming a

different one.

```
to_dict / from_dict round trip equal: true
summary  guild_count 1   member_count 2   treasury_bucket wallets
```

### Nothing reads the roster yet, and nothing shows a guild

The roster exists and no other module calls it. Nothing in the running program builds a guild. One sentence on the transfer subtab still reads "No guild roster is built, so the guild term of a transfer reads nothing", and that is the only place a guild appears on any screen.

```
built        src/competition/guild_roster.py
read by      nothing
on a screen  nothing, beyond the sentence saying a roster is not built
```

This entry leaves five things owed and builds none of them.

```
the package export list     one entry, so other modules can import the roster
the transfer skill          its guild term reads the roster
the action payment          its officer check and its treasury call read the roster
the guild metric            the metric named as held by no field reads the roster
a surface                   a control that forms a guild, and a list that shows one
```

Four figures stay owed, and this entry invents none of them.

```
PoA tokens to form a guild   no number exists
the stake each rank locks    no number exists
a cap on officers            no number exists
a cap on members             none, by his own words
```

---

## 2026-09-11 03:10 - #585 - the Vessel, and one wallet behind all of them

A Vessel is now a thing on disk rather than a word in other modules' docstrings. It carries an owner, a class, a level and an id, and every Vessel a player holds reads the same single wallet balance.

```
src/competition/vessels.py
```

### The requirement, in his own words

```
Players in PoA are actually Reincarnates and their Class at a given point is a
Vessel. Vessel have levels and require increasing amount of Quint to 'power' or
operate... A Reincarnate cannot be completely destroyed... but they can be
'fully merged with the pleroma' if they do not hold a Quint balance.
```

```
Players can have multiple Vessels that they micromanage to do lifeskilling and
crafting (all multi-turn and sometimes lengthy processes) assuming their total
Quint wallet budget supports this...
```

### A Vessel is a class, a level, an owner and an id

The class is one of the seven already in the package, so nothing new decides what a Vessel can be. The level is one of the hundred on the arc. The owner is the wallet address, and a Reincarnate refuses a Vessel whose owner is a different address, so no player can attribute a Vessel to someone else's balance. The id is drawn at construction and holds two Vessels of one class apart.

```
@dataclass(frozen=True)
class Vessel:
    owner: str
    class_name: str
    level: int = FIRST_LEVEL
    vessel_id: str = ""
```

Its requirement comes from the stat table and nowhere else. Asking the Vessel for its requirement adds the five stats the level gives it.

```
Iron Edge at level 12          needs  70
Silver Mirror at level 25      needs 225
Lead Ward at level 1           needs   5
```

### Many Vessels, one balance, and that is the whole rule

His sentence says a player may run several Vessels *assuming their total Quint wallet budget supports this*. The budget is the wallet, so the requirements of every Vessel are added together and read against the one balance. A player with five Vessels holds no more Quintessence than a player with one.

One run drove a real wallet holding 100 Quintessence and added a Vessel at a time:

```
1 Vessel     summed requirement  70    each runs at 1 of full
2 Vessels    summed requirement 295    each runs at 0.33898305084745762711864406 78
3 Vessels    summed requirement 300    each runs at 0.33333333333333333333333333333
```

The first Vessel never changed and its power fell twice. That is the point of the shared reading, and the alternative is the reason for it. Had each Vessel carried its own budget, the same three Vessels on the same balance would read:

```
Iron Edge        1 of full     full power
Silver Mirror    0.4444444444444444444444444444
Lead Ward        1 of full     full power
```

Two of the three at full, and a player could add Vessels for ever at no cost. The shared reading closes that.

### Powering a Vessel spends nothing

The requirement is a threshold a balance is measured against. It is not a charge, and nothing is moved or burned. The same Quintessence satisfies the requirement and stays in the wallet where it can be spent on something else.

```
balance before every reading   100.00000000
balance after                  100.00000000
four-bucket conservation        balanced
```

### Under the requirement a Vessel is not refused

A player short of the amount still occupies the Vessel and runs it below full potential. No call anywhere in this module refuses a Vessel for want of Quintessence, and the fraction above is what a player gets instead.

```
acervator.vessels INFO reincarnate-holder runs 3 Vessels needing 300 against a
balance of 100, each at 0.33333333333333333333333333333 of full
```

### The zero-balance rule, and the newcomer it would have deleted

His rule merges a Reincarnate with the pleroma at a zero balance. Read on the balance alone it also merges every person who has just arrived, because a wallet nobody has paid into holds nothing. Nobody could ever start.

The ledger already tells the two apart. It keeps every movement that has touched an address, and a brand-new address has none.

```
a newcomer        balance 0       movements 0
a spent player    balance 0E-8    movements 2
```

The merger therefore needs two things together, and the second one is a balance the player once held. Driven against three real addresses on one ledger:

```
a holder          balance 100.00000000   merged False   holds a Quintessence balance
a newcomer        balance 0              merged False   has never held a Quintessence balance
a spent player    balance 0E-8           merged True    fully merged with the pleroma
```

Nothing is destroyed by this reading. It answers a question and removes no record, which keeps his rule that a Reincarnate cannot be completely destroyed.

### No figure caps how many Vessels a player may hold

Nobody has named a limit, and none was invented here. The wallet balance is the only bound: a player who adds Vessels keeps diluting all of them, so the cost of a twentieth Vessel is the power of the other nineteen.

```
the only bound on Vessel count   the wallet balance
a figure that caps the count     absent
```

### What a Vessel still cannot do

Eight mechanisms name a Vessel and six of them are unbuilt. The module lists all eight by name and what each one waits on, so none of them is silently missing. Two of the eight now ship, and their notes say which code holds them.

```
assignments     nothing gives a Vessel a task to carry over several turns
lifeskilling    one skill ladder exists and no Vessel runs it
crafting        crafting.CraftRegister makes one. begin_craft takes the components
                and complete_craft delivers the item
notifications   nothing tells a player that a Vessel finished
gear            no module holds an item a Vessel wears
equipping       items.equip_check holds it: an item type's cohesion read against
                the total Quintessence held
destruction     the PvP vote answers who may, and nothing destroys
permadeath      no module ends a Vessel
```

His rule that the Quintessence level of gear cannot exceed the total of the equipping player is the one `equip_check` answers. No module records an item as worn, so the answer reaches no wearer yet.

```
In development.
```

### What reads this today

Nothing. The wallet's Vessels section on the tab still reads a class pick rather than a Vessel, and the summed requirement row still prints two dashes. Driving the tab on both chains shows the section stops earlier than that, because no participant identity exists on this machine at all.

```
section 'Vessels'
    note   bot_identity.json does not exist, so no participant is named.
```

The wallet section already prints a movement count beside the balance, which is the second half of the merger rule, so the page is reading the right ledger already. Two things are owed before any of this shows: the module's entry in the package export list, and the wallet section reading a Reincarnate instead of a class pick.

```
owed   the module's entry in the package export list
owed   the wallet's Vessels section reading a Reincarnate
```

## 2026-09-11 03:30 - #586 - the world tier, and the Quintessence a slaying imports

A world now has a tier. The tier runs from one to six, and it names the deepest dimension the world reaches. Raising it reaches further down, where the creatures are harder, so a mature world grows harder and pays more.

```
src/competition/world_tier.py
```

### The cataclysm rule, in his own words

```
PoA - Cataclysms - Monsters - Creatures fought during a cataclysm (alignment
skewed world event) actually 'import' additional Quint that is added to the
World if they are slain. This is also the mechanism whereby a World's tier
(lowest dimension on which it resides) is raised thus permanently increasing
difficulty and proportionate rewards. This allows loot and resources to roll off
of charts for mature worlds having predominantly players that do not need their
statistical bandwith being consumed by unneeded items.
```

### A higher tier number reaches a lower dimension

His definition reads straight through. The tier counts from one to six. The count names the lowest dimension the world sits on. Raising the count reaches deeper, and deeper runs harder. That reading is the one this unit took, and nothing he has written asks for the opposite.

The depths are not new. The monster table already declares twelve tiers at signed depths, six below the player's own plane and six above. A world at tier one reaches only the first depth below it. A world at tier six reaches all six. This module declares no second scale.

```
world_tier.lowest_depth_of   reads monster_table.monster_tier_at

tier 1   depth -1   the surface floor
tier 2   depth -2   executors of a mandate or of fate
tier 3   depth -3   the decan rank
tier 4   depth -4   floor bosses
tier 5   depth -5   the fall tier
tier 6   depth -6   the summit
```

Driving the module produced those six names, and nothing copied them from a table. The module refuses a seventh tier, and refuses a tier below one.

```
world w rises to tier 6, reaching depth -6
WorldTierBoundError: world 'w' sits at tier 6 of 6 and reaches depth -6 already
WorldTierBoundError: a world tier runs 1 to 6, the 6 descending ranks
monster_table declares, got 7
```

### The import moves Quintessence and creates none

Quintessence has a cap of thirty-three million, and that cap holds only while every amount sits in one of four places. An import can make none. The ledger already had the door. One call takes an amount out of the pleroma, where Quintessence rests while nothing holds it, and puts it in the embedded place, where a world holds it. His own rule sends a slain creature's Quintessence back to the pleroma. The import uses that same door the other way.

```
def import_on_slaying(self, ledger, world_id, amount) -> Decimal:
    standing = self.world(world_id)
    imported = ledger.embed_from_pleroma(amount)
```

### One import, driven on a real ledger

The run opened a ledger on a temporary file. It distilled a hundred Quintessence, then bled four of that into the pleroma with a transfer. It made one import of 1.25 for a world, and read the four totals off the ledger either side of the call.

```
quintessence_ledger.embed_from_pleroma, through
world_tier.WorldTierRegistry.import_on_slaying

              wallets   held   pleroma   embedded   ever minted   difference
before          96.00      0      4.00       0.00        100.00         0.00
after           96.00      0      2.75       1.25        100.00         0.00
```

The import created nothing. The amount ever minted did not move, and the four totals still add to it with no difference. Had that mint figure risen, the import would have been making Quintessence, and the thirty-three million cap would no longer bound the supply at all. The module said what it did as it did it.

```
acervator.world_tier world world-1 opens at tier 1, reaching depth -1
acervator.world_tier world world-1 imports 1.25 Quintessence out of the pleroma
```

### What happens when the pleroma runs dry

Every world draws on one finite pleroma, so an import draws on a shared pool. The run drove the ledger dry to find out what it does.

```
pleroma 0      import 1      ValueError: the pleroma holds 0, cannot embed
                             1.00000000
pleroma 0.4    import 1      ValueError: the pleroma holds 0.40000000, cannot
                             embed 1.00000000
pleroma 0.4    import 0.4    moved 0.40000000, pleroma left at 0
```

An import never part-fills. The ledger refuses an amount larger than the pleroma holds, whole, and credits the world nothing. After both refusals the world's own total still read zero and the four totals still balanced. An amount smaller than the smallest unit the currency can express moves nothing and answers zero.

### Farming a world to raise its tier

A world can reach tier six by farming, and nothing in the tree stops it. The raise takes one call, and no figure says how much imported Quintessence earns one. Six calls to the raise walk a world from tier one to tier six. The module declares that figure absent and refuses to answer for it, rather than choosing a number nobody chose.

```
world_tier.tier_raise_threshold

FigureAbsentError: no threshold raises a world tier: the imported Quintessence
that raises a world one tier. No statement names one, so nothing bounds how
often raise_tier may be called and a world can be farmed to MAX_WORLD_TIER by
repeated slaying
```

The pleroma is the only real brake today, and it brakes the import, not the tier.

### The monster table owes the amount one creature imports

Asking what a slain creature imports hands back a refusal, and the monster table raises it rather than this module. All twelve of its tiers carry no character level, so no Quintessence amount follows from any of them. The monster table already counts twelve levels it owes.

```
world_tier.slaying_import_amount, through
monster_table.tier_embedded_quintessence

LevelAbsentError: the tier at depth -1 has no level, so no Quintessence amount
follows: the character level this tier sits at. No source names one.
```

This module refuses a depth the world does not reach, and that one check is all it adds to that path.

```
WorldTierBoundError: a world at tier 2 reaches [-1, -2], and no creature of it
sits at depth -5
```

### Loot rolls off the low end, and one figure decides how much

His sentence rolls loot and resources off the chart as a world matures, because mature players do not want their item lists full of things they cannot use. A tier picks a band of the chart instead of the whole chart.

The five loot tiers already run commonest first, and the material quality grades take the same five names in the same order from the same place. One band answers loot and resources together. Cutting from the low end takes the commonest items first, which is what his sentence asks for.

```
world_tier.loot_band   cuts loot_drop.TIER_NAMES, which materials.QUALITY_GRADES
                       also reads

cut 0    Calx, Cauda Pavonis, Flores, Elixir, Magisterium
cut 1    Cauda Pavonis, Flores, Elixir, Magisterium
cut 2    Flores, Elixir, Magisterium
cut 3    Elixir, Magisterium
cut 4    Magisterium
```

How many tiers one world tier cuts is the figure nobody has set. Six world tiers and five loot tiers divide by no whole count, so the module refuses to answer a band from a tier until someone names that figure.

```
world_tier.loot_band_at_tier

FigureAbsentError: no band follows from a world tier: how many loot tiers leave
the chart for each world tier above FIRST_WORLD_TIER. No statement names one.
```

### A cut band is not yet a drop table

A drop roll needs weights that add to a hundred. A cut band falls short, and adding the weights at each cut shows by how much.

```
world_tier.band_weight_total, against loot_drop.WEIGHT_TOTAL_PCT of 100

cut 0    weights total 100.0
cut 1    weights total  40.0
cut 2    weights total  15.0
cut 3    weights total   4.0
cut 4    weights total   0.5
```

Only the uncut chart passes the check the loot module already runs on itself. A band names which items a mature world can drop. Nothing reweights what the cut leaves, so no roll comes out of a band yet.

### Which clock

The import belongs to the event turn, because a participant slays a creature inside an event. The tier raise belongs to the world turn, because the tier is a fact about the world that every participant has to see the same way, and a raise part way through an event would change the difficulty of a fight already running. Nothing schedules either one.

```
event turn   300 seconds, 60 for an Elite event, read from
             poa_modes.EventVariant.turn_seconds
world turn   one hour, counted in whole turns and in seconds nowhere under src
```

### The registry holds the tier in memory

The registry keeps every world's tier in memory and writes no file, the same as the guild roster. His word is permanently, so a world's tier has to outlive a restart before this work finishes.

```
owed   a world's tier surviving a restart
```

### Nothing calls it

Four things this needs do not exist, and a count found each absence.

```
cataclysm    no module under src names one
alignment    no module under src/competition names one
combat       no module under src names one, so nothing can slay a creature
world tier   world_grid declares layers and declared no tier before this module
```

The module provides the two calls a cataclysm would make, and nothing calls it. Twelve of the thirteen findings the coding review returned say exactly that: the registry, its methods and the reading functions have no caller. The thirteenth names a missing copyright line, which no file in the package carries.

Nothing shows a world tier either. Driving the tab's own handler returns thirty-one sections, and searching each one finds the words world tier zero times, and cataclysm zero times. The conservation panel does already print the pleroma and the embedded totals, so an import would show its effect there the moment something made one.

```
owed   a cataclysm, which would raise the tier
owed   a combat result, which would make the import
owed   the module's entry in the package export list
owed   a surface showing a world's tier
```

### What the world tier reads, and what it hands back

```
reads, nothing provides   the amount one slain creature imports. The monster
                          table refuses it, because no tier carries a level
reads, nothing provides   the imported Quintessence that earns one tier raise
reads, nothing provides   how many loot tiers one world tier cuts
reads, nothing provides   a reweighting rule for a cut band
provides, nothing reads   a world's tier, and the depths that tier reaches
provides, nothing reads   the import, and the total each world has imported
provides, nothing reads   the loot band a cut leaves on the chart
```

## 2026-09-11 03:50 - #586 - alignment, and the three levels it rolls up

Alignment is now a running total of what a Vessel has done. A player does not pick a side. Each action carries its own ratio of Creation to Destruction, and a Vessel's alignment adds every one of those scores together. The same total rolls up to a guild, and the guilds add up to a world.

```
src/competition/alignment.py
```

### His words, and what they ask for

```
Remember to note how alignment is going to work in PoA. Abilities are ratios of
Creation and Destruction on a gradient that persistently scores every vessels
action so that alignment becomes a function all a Vessel's 'life' choices.
```

```
Individual Vessel alignment informs Guild Alignment which then defines World
Alignment.
```

### An action carries the ratio, and this module holds no ratios

A score arrives with the action that earned it. The module that keeps the totals holds no table of ratios and picks none, because no ability exists yet to carry one. The caller hands over the two parts and the module adds them.

```
ledger.score(vessel, "raised a wall", 3, 1)
```

The one action the package builds is the Quintessence Transfer in `src/competition/skill_ladder.py`. Whether a transfer scores at all, and which way, is a ruling nobody has made. This module gives it no ratio.

### The scale runs pole to pole, with a real middle

One action's reading is its Creation less its Destruction, divided by the two added together. All Creation reads 1. All Destruction reads -1. A ratio of three to one reads 0.5. Nothing can leave that range, because neither part is ever negative.

```
all Creation            polarity   1
three Creation to one   polarity   0.5
one to one              polarity   0
one Creation to three   polarity  -0.5
all Destruction         polarity  -1
```

Every amount stays exact. The module refuses a ratio handed to it as a binary fraction, and rounds nothing.

```
creation must be int, str or Decimal, not float; a ratio between
('creation', 'destruction') decided by binary floating point is refused
```

### An unscored Vessel and a balanced one are not the same thing

A Vessel that has done nothing and a Vessel whose good and bad cancel both show a net of zero. They are still different, and the difference falls out of the arithmetic: a reading is a net divided by a total, and a Vessel with no actions has a total of zero, so it has no reading at all.

```
before any score   actions=0   net=0   total=0   polarity=None   balanced=False
after 3 to 1       actions=1   net=2   total=4   polarity=0.5    balanced=False
after 1 to 3       actions=2   net=0   total=8   polarity=0      balanced=True
```

The balanced Vessel reads zero. The unscored one reads nothing. A world event can tell them apart on either the reading or the action count.

### The total never decays

Nothing removes a score, and no clock reduces one. His sentence says alignment is a function of all a Vessel's life choices, and a total that faded would stop being that. The design has no decay, so no figure sets a decay rate.

```
DECAY_ABSENT   no scored action is ever dropped from a total, and no clock
               reduces one
```

### The Vessel holds it, and its key leaves the level out

A Vessel is a frozen record of an owner, a class, a level and an id, in `src/competition/vessels.py`. It carries an identity field, and this key does not read it yet. The module files each score under the owner and the class name, so a Vessel that gains a level keeps the alignment it earned.

```
the key       owner, class_name
left out      level, so levelling up loses nothing
```

One thing that key cannot do is separate two Vessels of the same class under one owner. Nothing stops a player holding two, and today they would share one alignment. Separating them needs the identity field on the Vessel record, which the Vessel now carries and this key does not read.

### A Vessel informs its guild, and the guilds define the world

His two verbs differ and the difference is worth stating plainly. A guild's alignment is the sum of its members' Vessels, and other things may inform it later, such as the guild's own acts. A world's alignment is the sum of its guilds and nothing else, which is what defines means.

```
a Vessel     every score it has taken
a guild      every Vessel of every member
a world      every guild on the roster
```

The sum is the same operation at both levels today. The verbs differ only in whether more contributors may arrive, and nothing else contributes yet.

Summing rather than averaging is what makes his single-guild rule come out right. A world with one guild takes that guild's reading exactly, with nothing to pull it back toward the middle. A world with guilds on opposite sides cancels toward the middle in the sum, which is the position a victor has to hold.

### One run, three levels, and a change at the bottom moved the top

One run drove two Vessels in one guild. The second Vessel drained a creature at nothing to forty, which is as far toward Destruction as a score goes.

```
alpha Iron Edge        net   0   total  8   polarity  0
beta Silver Mirror     net -40   total 40   polarity -1
the guild              net -40   total 48   polarity -0.83333333333333333333333333333
the world              net -40   total 48   polarity -0.83333333333333333333333333333
```

The first Vessel then restored a spring at twenty to nothing. Its own reading moved, and so did both levels above it.

```
alpha Iron Edge        net  20   total 28   polarity  0.714285714285714285714285714 3
the guild              net -20   total 68   polarity -0.29411764705882352941176470 59
the world              net -20   total 68   polarity -0.29411764705882352941176470 59
```

One action changed all three readings. That is the roll-up working.

### A player in no guild reaches the world nowhere

The chain runs Vessel, then guild, then world. The roster in `src/competition/guild_roster.py` allows a player to belong to no guild, and answers nothing when asked which guild such a player is in. An unguilded player's actions therefore reach no guild and no world.

A third Vessel, owned by a player in no guild, mended a road at five to nothing. The world did not move.

```
gamma Lead Ward        net   5   total  5   polarity  1
the world              net -20   total 68   unchanged
unguilded addresses    ('gamma',)
```

The module reports those addresses rather than hiding them, and invents no route for them. **This is a question for the operator:** either an unguilded player's actions should not count toward a world, or the chain needs a fourth step that carries them.

### Offsetting is open, and the consequence is his to see

The net is a plain sum, so light good acts cancel heavy bad ones. One run scored a single profound Destruction of a thousand, then a thousand trivial Creations of one each. The Vessel came out exactly balanced.

```
actions 1001      net 0      total 2000      polarity 0      balanced True
```

A player can also park a Vessel near the middle by alternating. His own rule calls necromancy and energy vampirism profoundly evil, which reads as weight rather than count, and weight is the ratio the action carries. Nothing in the totals stops many light acts cancelling one heavy act.

The one thing the totals do keep is how much weight went in. A Vessel at zero with a total of two thousand is not the same as a Vessel at zero with a total of two, and a reader can tell them apart.

```
a brake on offsetting   nothing sets one, and the operator would set it
```

### The monster tiers are a different axis, and the module says why

The twelve monster tiers run six below the player's plane and six above. One reading makes them the same Creation-to-Destruction axis, and the table in `src/competition/monster_table.py` refutes it on five counts, so the two axes stay separate.

```
a tier is never at depth 0, and this scale needs a middle
the decan rank below the plane is invoked to HEAL, not to destroy
one tier of the twelve is marked morally wicked; the rest are indifferent
the Watchers sit below the plane AND in the families named above it
no rule places an arbitrary creature on the tier axis at all
```

The last line is the decisive one. The tier axis answers what a creature is. This scale answers what a Vessel has done. Nothing could score an action from a table that cannot place a creature.

### The neutral band is the one figure nobody has set

Exact balance needs no figure. A net of zero against any total above zero reads balanced, and the module answers that today. Naming a reading neutral over a range needs a width around zero, and no source gives one.

```
NEUTRAL_BAND   absent. The operator sets how far from zero still reads neutral
```

A three-way naming of a world's polarity is what the width would buy. His guild rule says a victor must hold a neutral polarity to survive, so a world event that asks whether a world is neutral will need it.

### What reads this, and what it reads

Nothing reads a Vessel's alignment, a guild's, or a world's. The module provides a world alignment that a cascade would read, and nothing reads it. The module names seven absent readers, each with the thing it waits on.

```
cataclysms          no module names one
world polarity      this module derives one, and nothing stores or shows it
abilities           one skill exists, and no ability carries a ratio
combat              nothing resolves a fight, so no fight scores anything
necromancy          no skill exists to score, and its ratio is the operator's
energy vampirism    no skill exists to score, and its ratio is the operator's
a surface           no tab or panel shows an alignment at any level
```

The module owes two more things. It has no entry in the package's export list, and nothing ties a roster to one world on the map, so a world alignment comes off whichever roster the caller hands over.

```
owed   the module's entry in the package export list
owed   a tie between a guild roster and a world on the map
```

The cascade his directive describes has no brake. Creatures pushing a world further toward its own end would make more of those creatures appear, and nothing stated sets a ceiling or slows it down. A reader of the world alignment will need one.

## 2026-09-11 04:10 - #586 - parties, and the armies their Raids link into

A party now exists as a record. A party holds its members, its event mode, and the one member who leads it. Above the party sits an army. An army links two or more Raid parties and reads a General off each one.

```
src/competition/army_command.py
```

### His structure, in his own words

```
Armies are comprised from multiple, linked Raids. Raid Leaders become Generals
under such structures. Generals must coordinate their raids while avoiding
devastating multi-square type abilities and AoEs that last multiple turns. These
events should be intense and always a bit chaotic.
```

```
Idea is for cataclysms to be the peak event type for PoA since it effects all
players in a world. My vision is to have entire player armies fighting cosmic /
nightmare type bosses at some point during one of these.
```

### Nothing held a party's members until now

Every event mode has declared a party range from the start, and one check already read those ranges. The party itself was missing. Nothing anywhere held a list of members, so the range had nothing to measure.

```
poa_modes.MODES             every mode's party range, declared
guild_roster.require_party  checks a list of addresses against one range
army_command.Party          the record that holds the members
```

### A party forms inside its mode's own range and nowhere outside it

Forming runs through the check that already existed. The range that refuses a party is the mode's own declared range, and no second copy of it exists. A Raid admits two to sixty. One run drove both ends and both refusals.

```
form_party, through guild_roster.require_party

a Raid of  2   forms     w0 leads a Raid party of 2
a Raid of 60   forms     w10 leads a Raid party of 60
a Raid of  1   refused   Raid admits 2 to 60 participants and this party holds 1
a Raid of 61   refused   Raid admits 2 to 60 participants and this party holds 61
a leader holding no seat
               refused   'w99' leads this Raid party of 2 and holds no seat in it
```

This work changes no range. It adds no mode, and it raises no ceiling.

```
Fixation                  1 to 1      no guild
Coagulation               2 to 120    one guild
Descension                1 to 6      no guild
Cementation               2 to 60     one guild
```

### An army links Raids and refuses every other kind of party

His sentence names Raids, so Raids are what an army links. A Dungeon Crawl party forms normally, and the link then refuses it. One raid is not an army either. The smallest army links two raids.

```
a Dungeon Crawl party   refused   a Dungeon Crawl party links into no army; an
                                  army is built from Raid parties
an army of one raid     refused   army 'The Fourfold Host' links 1 Raid party(s);
                                  an army links at least 2 of them
```

### One address holds one seat in one army

This is the refusal a size check cannot see. Two raids can each be a legal Raid party and still name the same player. An army admitting both would count that player twice in every fight, under two Generals.

```
two disjoint raids of four   linked    2 raids, 8 members
a third raid naming w3, who already sits in raid 0
                             refused   w3 already hold seats in army 'The
                                       Fourfold Host' across its 2 linked raid(s)
the same third raid, in a fresh army
                             linked    2 raids, 7 members
```

The third line is the one that matters. The party the first army refused links into a fresh army without complaint. The refusal came from the shared address and from nothing else about that party.

### A General is a role a leader takes, not an appointment

His sentence gives one General to each linked raid. Raid Leaders is plural and Generals is plural, and the sentence maps one onto the other. Nothing he has written names a single commander standing over a whole army.

The army records no General of its own. Each raid keeps its leader, and the army reads each linked raid's leader as that raid's General at read time. The same wallet answers differently to a party and to an army.

```
the party alone calls w4 a leader
the army calls the same wallet a general
a stored raid carries leader, members, mode and guild_key, and no general field
```

That shape answers a General leaving in the middle of an event without a rule of its own. No separate record exists to go stale. Forming freezes a party, so nothing in the package removes a member from one, and the departure path is an open item of its own.

### An army reconstructs through the same refusals that formed it

A stored army rebuilds raid by raid, and each raid goes back through forming and linking. Every refusal above applies to a stored army and to a new one. The load refuses a stored party whose members have since left the guild.

```
two raids of four, stored as 280 bytes of JSON
rebuilt   generals ('w0', 'w4')   2 raids   8 members   round trip identical
```

### Which clock a party and an army answer to

No clock times either object. A party forms before an event begins, and an army forms before a cataclysm, so neither forming sits inside a turn of either clock. Once an event runs, the seating call that already exists puts a party's members on the event turn. A cataclysm reaches every player in a world, so an army belongs to the world turn. Nothing schedules either one.

```
event turn   300 seconds, 60 for an Elite event, from
             poa_modes.EventVariant.turn_seconds
world turn   one hour
owed         a deadline for forming a party, and one for linking an army
```

### The party window holds two raids and no more

The screen that would show an army has a measured ceiling. The window pages forty participants at a time and holds a hundred and twenty in all.

```
proof_of_accumulation_tab_surface.party(), driven

capacity 120   per page 40   pages 3   group size 5   groups 8
header "Page 1 of 3 - 40 a page - up to 120"
```

Two full Raids of sixty fill it exactly. A third raid does not fit. The window shows no leader and no General, and carries no army section at all.

```
owed   a party window that pages an army, not one party
owed   a mark on the window for the General of each raid
```

### How many raids one army may link is his to set

No cap sits in the code. A world seats a fixed number of participants, and dividing that by a Raid's own ceiling gives the raids a world could field at once. Whether that is the bound he wants is his decision, so nothing declares it.

```
79 participants a layer  x  20 layers     = 1,580 seats a world
1,580 seats  /  60 a Raid                 = 26 raids
declared in army_command                  = no cap
```

### What stays absent

Four things an army exists to fight do not exist. A count found each absence.

```
combat           no module under src resolves a fight
abilities        skill_ladder holds one skill, the Quintessence transfer
an area effect   needs an ability and a turn resolution
a cataclysm      no module under src names one
```

The multi-square reach his sentence describes has a real anchor already. The world grid addresses squares and gives every position a locator, so an ability covering more than one square would read those. Nothing builds one here.

### Nothing forms a party in the running program

Driving the tab's own handler returns the party window's paging and nothing else. No member, no leader, no army. The handler imports twenty-nine modules from the competition package, and neither the guild roster nor this module is among them.

```
proof_of_accumulation_tab_surface.view_model({}), under python -X dev

party block keys   capacity, group_size, groups, mark_ranks, mark_seam_note,
                   mark_seams, page, page_text, pages, per_page, placeholder
army key           absent
general key        absent
```

The coding review returns seven findings on the new module. Six of them say a method has no caller, which is the same sentence in another form. The seventh names a missing copyright line, which no file in the package carries.

```
owed   the module's entry in the package export list
owed   who may form an army, and whether its raids must share one guild
```

### What a party reads, and what an army hands back

```
reads, nothing provides   a party, from a screen or an event entry. Nothing
                          forms one
reads, nothing provides   a combat result, which is what a General coordinates
reads, nothing provides   a cataclysm, which is the event an army is raised for
provides, nothing reads   a party's members, its mode, its leader and its guild
provides, nothing reads   an army's linked raids, and the General of each one
provides, nothing reads   one row a linked raid, as a surface would serve it
```

## 2026-09-11 04:30 - #147 - the top row is halved, and the right half is the map or the encounter

### His words set the top row

The operator settled the top row on 2026-09-10.

```
"So top row of PoA GUI should be Current Vessel and Current Vessel Details on
 the left half and Map or Encounter on the right half."
```

The top row used to be one wide region beside a small square. It is now two halves of the same width with a divider down the middle. The lower half is unchanged and still holds the party window.

### Two halves and a divider

The top row is a three-column grid. The first and third columns are equal fractions and the middle one is a single pixel that draws the divider.

`src/gui/web/proof_of_accumulation_tab.css` - the top row's columns

```
grid-template-columns: 1fr 1px 1fr;
```

### What each half measures now

Read off the rendered page in the Electron shell and off the rendered page in the desktop window, at the window size the shell opens and at a narrow one.

```
                          left half        right half
                        before  after    before  after
Electron shell  1386    1165    669       181    669
Electron shell   900     684    426       176    426
desktop window  1386    1164    669       183    669
desktop window   900     683    426       177    426
```

The before figures are the player window and the square enemy screen beside it. The after figures are the two halves.

### A record on the event decides which state the right half draws

The right half draws the map until an encounter runs, and the encounter after that. Nothing in the platform held a flag for a running encounter, so the tab reads one off the pot the payout panel was already reading: a participant record on the declared event with no payout yet is an encounter in progress.

`src/gui/main_tabs/proof_of_accumulation_tab_surface.py`

```
def encounter(pot: dict) -> dict:
    """Whether ``pot``'s event holds participant records and no payout, which runs one."""
    joined = len(pot["shares"]) + len(pot["unscored"])
    running = joined > 0 and not bool(pot.get("is_settled"))
```

Both directions were driven from the page itself on the demo chain. The grade button wrote one record and the right half became the encounter; the payout button settled the event and the right half went back to the map.

```
control fired   right half
none            Map
grade           Encounter - "Records on this event: 1. No payout yet."
payout          Map
```

### The square keeps its shape, and the row's height sets its side

The enemy screen is square and draws inside the encounter, beside its title and its notes rather than under them. Its side is the height of the top row, which is what the side used to be, so halving the width did not shrink it.

```
                          enemy screen
                        before   after
Electron shell  1386      181     164
Electron shell   900      176     158
desktop window  1386      183     165
desktop window   900      177     160
```

The remaining difference is the region's own padding and border. The square grows with the window the way it did before.

### The left half is the Vessel, and it says what keeps none

The left half prints the class, the planet, the assignment and the level of the Vessel a class pick names, then its own requirement, the summed requirement of every Vessel the participant occupies, the wallet balance, the fraction of full power that balance reaches, and the pleroma standing.

No module keeps a Vessel against an address, so with no class pick the half prints no rows and says why instead of drawing an empty frame.

```
poa_record_store.json keeps no Vessel for this participant. A Vessel is the
class a Reincarnate occupies, and only a class pick names one.
```

Under that it lists the eight mechanisms a Vessel takes part in that no module builds, each with the sentence naming what it waits on. Those eight come from the Vessel module's own list, not from a second copy here.

### The wallet's summed requirement is a figure now

The wallet's Vessels section printed two dashes where the summed requirement goes, because nothing held the figure. The Vessel module holds it, so that row prints it and the note beside it says what the sum covers. The section still takes its Vessel from a class pick rather than from a stored Reincarnate.

### Both variants, matched reading by reading

The same reading script ran against the desktop window's page and the shell's page.

```
readings compared   145
identical            96
differing            49
```

Every difference is one of three things: the page box, since the shell window is taller than the harness window; one pixel a text line, which the two engines round differently; and the participant address, which each run generates fresh in its own scratch home. No width of either half differs on any reading. The four box readings whose width does differ are the enemy square, which follows the row height.

### What the top row does not draw

```
the seven subtabs and their shortcuts   four subtabs exist; the row is unchanged
clicking the Vessel                     no character details subtab carries them
the tactical map                        nothing holds an encounter's enemies
skill trees and skill rings             content inside a subtab
pixel art                               no art is commissioned
```

The map region opens the existing map subtab when it is clicked, which is the one route of his sentence that a built subtab already answers.

## 2026-09-11 04:50 - #586 - domination, and the two forms a taking takes

A Vessel now carries a condition, and only a condition of incapacity opens it to a taking. A taking comes in two forms. In the first the owner keeps the Vessel and another player acts with it. In the second the Vessel changes hands.

```
src/competition/domination.py
```

### His words, in full

```
PoA - Classes - Abilities - Destruction - Another example of an ability or group
of abilities that sits far on the demonic side of things would be domination or
possession which allows one player (during PvP) to temporarily control or
literally steal a Vessel from another. This can only be done to Vessels that are
incapacitated in some way.
```

```
Vessel Incapacitation causes: Health At Zero or Mentally Comprimised by Daze or
similar.
```

```
Do not get too far down the hole on the incapacitation. This will tied to what
certain abilities do and the length of time it lasts depends on what was used to
cause it.
```

### The ability is absent, and these are the two mechanisms it would call

No ability system exists. The skill ladder holds one entry, the Quintessence Transfer. This work builds no ability, no ability table and no skill tree. It builds the state his rule gates on, and the taking itself.

```
skill_ladder.SKILL_NAMES   one entry, the Quintessence Transfer
an ability                 absent. The ability that invokes them is absent
a skill slot               absent, and a separate item
```

### Both gates bind, and neither one is optional

His sentence carries two conditions. A taking happens during PvP, and only to a Vessel carrying an incapacitation. The PvP half already existed, and this work reads it rather than writing a second one. The permission already honours a carried mode, a week-long lock and a PvP event flag, and it already refuses a player acting against their own address.

```
during PvP only        pvp_vote.may_destroy answers it. No second permission
                       exists in this work
incapacitated only     require_incapacitated refuses a Vessel that reads able

driven, both refusals

no PvP mode      refused   0xthief may not take 0xvictim's Vessel on world turn
                           0; may_destroy permits a taking while a PvP mode or a
                           PvP lock holds that turn or a PvP event is set
health above zero,
no effect        refused   0xvictim's Lead Ward reads able and only a Vessel
                           reading incapacitated may be taken
```

### Two causes, and the line he drew between them

His sentence names a body and a mind. The code keeps them apart, as two kinds. A taking reads either kind and asks for neither in particular, because nothing in his words prefers one.

```
physical   health at zero
mental     compromised by a daze or similar
```

His phrase, or similar, makes the mental kind a class and not one effect. The daze is the one member he named. A second joins the list without any other change, and nothing here invents what the others are.

```
INCAPACITATION_KINDS   physical, mental
MENTAL_EFFECTS         daze
an unnamed effect      refused. 'stun' is no mental effect
```

### How long it lasts belongs to the ability, not to the state

His second sentence settles the length. It depends on what caused it, which makes it a property of each ability. No ability exists, so nothing here decides a length and nothing here runs a timer. The state records an end its caller gives, and reports whether a Vessel is incapacitated now.

```
one run, a daze given an end of world turn 1

on world turn 0   kinds (mental,)   a taking is permitted
on world turn 1   kinds ()          refused, the Vessel reads able
given no end      holds at every world turn
```

That keeps the length out of this work entirely. A caller that applies an incapacitation says how long it holds. This module compares that end against one world turn and computes nothing.

### Both causes read a value nothing supplies

This is the honest half. His causes are now named and both are still out of reach, because nothing can reduce a health value and nothing can apply a daze.

```
a Vessel's health    vessels.Vessel carries an owner, a class, a level and an
                     id. There is no health field, and this work adds none
health as a stat     the five stats are strength, dexterity, constitution,
                     intelligence and wisdom. Health is not among them
a maximum            no figure names full health
a daze               no module applies one, and no module removes one
```

A bot's health in dollars is a different thing and is not reused. The trading metaphor reads a dollar target and names no Vessel, so no Vessel's health comes from it.

```
rpg_metrics.health_metrics   max_health_usd from scrumming_state.target_balance
```

The register takes a health reading from whoever calls it, and says as much in its own output. Health is a decimal, never a binary fraction.

### Two forms, and they are not one call

His sentence draws a line between controlling a Vessel and stealing it. Two separate calls keep that line. Only one of them moves an owner.

```
take_control   the owner keeps the Vessel, and another player acts with it
steal_vessel   the owner changes, and both players are rebuilt around the move

driven, a dazed Vessel under another player's control

controller 0xthief   owner 0xvictim   ownership moved  False
the victim still holds four Vessels
```

What ends a temporary control is not stated and nothing ends it. The only clock figure within reach is the turn the PvP permission lapses on, and that lapses the permission rather than the hold.

```
owed   how long a temporary control lasts, and what ends it
```

### A stolen Vessel makes the thief weaker, and that rule was already there

Every Vessel a player owns draws on one wallet balance. Adding a Vessel raises what the wallet must cover, and the same balance then stretches further. Taking a Vessel costs the thief on every Vessel it already had. This work adds no penalty for theft. The existing rule already charges for it, and a second charge would be a figure nobody chose.

```
one owner at level 12, against a balance of 100

3 Vessels   requirement 210   each reaches 0.4761904761 of full
4 Vessels   requirement 280   each reaches 0.3571428571
5 Vessels   requirement 350   each reaches 0.2857142857
```

### One theft, driven on real objects

One run drove a theft between two players and read both sides off the objects themselves, before and after. The victim held four Vessels and the thief held three. Both readings used one balance, which a theft does not touch.

```
steal_vessel, driven under python -X dev -X faulthandler

the Vessel moved      0xvictim's Lead Ward now reads owner 0xthief
0xvictim  4 -> 3      requirement 280 -> 210    0.3571428571 -> 0.4761904761
0xthief   3 -> 4      requirement 210 -> 280    0.4761904761 -> 0.3571428571
```

The victim's remaining Vessels grew stronger. Every Vessel the thief owns grew weaker. That is the whole cost of a theft and it needed nothing new to price it.

A first run of this went red. The module's own report of the result carried more placeholders than values, and it raised a type error instead of printing. A failure of that run would have meant a taking whose outcome nobody could read. The correction landed, and the same run now prints both sides.

### The four abuse questions, answered

Four ways a player might misuse a taking. One run drove each of them, and each answer is what the code did rather than what it intends.

```
take a Vessel they cannot power     YES. The shortfall is simply larger, and the
                                    thief carries it. Driven: the thief ends at
                                    0.3571428571 of full with a shortfall of 180
take an already taken Vessel        YES, by a third player. The condition moves
                                    with the Vessel, and nothing removes it
dump a Vessel on an enemy           NO. A taking has no recipient field. The new
                                    owner is always the taker, so the dilution
                                    lands on whoever takes
take their own Vessel               NO. The PvP permission already refuses a
                                    player acting against their own address
```

The third answer is the one worth stating plainly. Because extra Vessels weaken their owner, forcing a Vessel onto an enemy would harm them. No call does it. A taking moves a Vessel to the taker and to nobody else.

### The condition stayed behind on a theft, and now moves with the Vessel

A Vessel's condition files under its owner and its class. A theft changes the owner, so the entry must move with the Vessel. It did not at first. The taken Vessel read able under its new owner, and the victim's next Vessel of that same class read incapacitated with nothing having touched it. Both readings were wrong and both are now right.

```
the taken Lead Ward, after the theft   incapacitated, under its new owner
a fresh Lead Ward of 0xvictim          able
```

The scored actions a Vessel has taken do not move with it. They stay filed under the old owner. A Vessel carries an id of its own, which is the field that would carry them across, and the alignment key does not read it yet.

### Whose alignment moves

A taking can carry a score on the Creation to Destruction scale, and the score files against the Vessel that performed it. His rule makes alignment a function of all a Vessel's life choices, so the acting Vessel is the reading this work takes. Adding a player's Vessels still gives a player-level reading.

This work chooses no ratio. An action carries its own ratio, this module declares none, and the figure is his.

```
score_taking, driven with a caller-supplied ratio of 0 creation to 1 destruction

scored against   0xthief's Quicksilver Draught, the acting Vessel
polarity         -1
action           theft of another player's Vessel
a wrong Vessel   refused

owed   the ratio a control and a theft each carry
```

### Which clock a taking answers to

The world clock. The permission reads a world turn, and a taking records that same turn. Neither kind of incapacitation carries a clock of its own.

```
world turn   one hour, and the turn may_destroy reads the permission on
event turn   not read by a taking
a given end  recorded against a world turn, and supplied by the caller
```

### What stays absent around a taking

Seven things read a taking or an incapacitation, and none of them exists.

```
an ability                      nothing invokes either form
combat                          nothing resolves a fight, so nothing empties a
                                health value
a surface                       no tab or panel shows a taking or a condition
a health reading                no module holds a Vessel's health
a mental effect                 nothing applies a daze, and nothing removes one
a control duration              no figure names one
a ratio                         the operator sets it
```

### Nothing in the running program yet takes a Vessel

Nothing outside this module's own driven check calls either form. The package export list carries no name from it, so a taking needs a direct import of the module.

```
owed   the module's entry in the package export list
owed   a control on a screen that starts a taking
```

### What a taking reads, and what it hands back

```
reads, nothing provides   a health value, and anything that lowers one to zero
reads, nothing provides   a daze, and anything that applies or removes one
reads, nothing provides   a domination ability, which is the caller of both forms
reads, nothing provides   a ratio of Creation to Destruction for either form
reads, nothing provides   an end for an incapacitation, which each ability carries
provides, nothing reads   a Vessel's condition, and the kind behind it
provides, nothing reads   a control hold, naming the controller and the owner
provides, nothing reads   a theft record, reconstructible after the owner moved
provides, nothing reads   both players' potential before and after a taking
```

## 2026-09-11 05:10 - #586 - prayer, the first thing that writes another player's alignment

Every alignment score until now was a Vessel scoring its own action. A prayer is the first mechanism that reaches across and writes somebody else. A priest prays in the direction of their own alignment, the target moves that way, and the priest moves the same way by a smaller amount.

```
src/competition/prayer.py
```

### His words, and the three rules in them

```
PoA - Classes - Abilities - Priests - Can use prayer (of their particular
alignment) to shift that of other players but this also further shifts the
priest's in the same direction but a slower rate and multiple priests can act
together to speed up the process while reducing their own penalties.
```

Three rules sit in that sentence, and all three are built.

```
the target    a prayer moves another player toward the priest's own pole
the priest    moves the same way, at a strictly smaller rate
a group       moves the target faster, and each priest pays less
```

### The prayer writes through the ledger that was already there

Nothing new keeps alignment. A prayer files two scores through the same ledger every other action files through, one against the target and one against each priest. The scale is untouched and no second store exists.

```
roll = PrayerRoll(ledger, may_pray)
filed = roll.pray((priest,), target, rates)
```

### The direction is the priest's own pole, read off their own alignment

His words say the prayer is of the priest's particular alignment, so the direction is not chosen. The module reads the praying Vessel's running total and takes the side it sits on.

```
net above the middle    the prayer is Creation
net below the middle    the prayer is Destruction
no scored action        refused; the Vessel carries no alignment to pray
exactly the middle      refused; the middle names neither pole
```

Priests praying together must read one pole. A congregation holding two poles is refused, because the two halves would pull the target in opposite directions and the prayer would name no direction at all.

```
MixedPoleError: priests praying together hold one pole;
Copper Conduit reads creation, Silver Mirror reads destruction
```

### The priest's own shift is a penalty, not a reward

A reader seeing a priest pushed toward their own alignment could easily read it as a bonus. It is the opposite. In his design a strong polarity is dangerous.

```
PoA - Single Guild Worlds - Mechanically will be Quint polarized and, as such,
will have a high amount of cataclysms and world events making it extremely
difficult to survive.
```

```
Unless the victorious Guild manages its alignment smartly and maintains a
neutral Quint polarity
```

Every prayer a priest leads pushes that priest further from the neutral position his own rule rewards. Praying steers a world, and the priest pays the bill in its own polarity.

### Four figures are owed, and a prayer without them is refused

None of the four numbers a prayer needs has been set. The module holds the four as a record the caller fills in, and it defaults none of them.

```
target_shift    how far one prayer moves the target
priest_shift    how far the same prayer moves each priest that led it
group_speedup   what each priest past the first adds to the target's shift
group_relief    what each priest past the first takes off every priest's shift
```

A missing figure is refused by name, and the refusal says who sets it. The operator sets all four.

```
FigureAbsentError: target_shift is not set; how much Creation or Destruction
one prayer scores against the target. No figure names it and the operator sets it
```

### What holds even with the figures absent, which is the whole point of this unit

His sentence gives no numbers. It gives relations, and relations can be enforced without a number. The module refuses any set of figures that breaks one of them.

```
at every size    each priest's own shift is strictly under the target's
as size rises    the target's shift never falls
as size rises    each priest's own shift never rises
```

That is his design holding whatever the four figures turn out to be. A figure set that breaks a relation is refused at the moment it is built, not when it is used.

### One prayer and one group prayer, driven on real alignments

Both were run against real Vessels with real running totals. The figures below were supplied by the run and are set by nobody. A priest with a Creation total of four over ten prays over a target sitting at six Destruction over ten.

```
rates                 target 10, priest 6, speedup 3, relief 1

one priest
  target before       net -6 over 10, reading -0.6
  target after        net  4 over 20, reading  0.2
  priest before       net  4 over 10, reading  0.4
  priest after        net 10 over 16, reading  0.625
  shifts filed        target 10, priest 6

three priests, an identical target
  target before       net -6 over 10, reading -0.6
  target after        net 10 over 26, reading  0.3846
  priest before       net  4 over 10, reading  0.4
  priest after        net  8 over 14, reading  0.5714
  shifts filed        target 16, priest 4
```

The target moved toward Creation both times. Three priests moved it sixteen where one moved it ten, and each of the three paid four where the single priest paid six. That is all three of his rules, read off the objects the run built.

### The ordering is on the shift, not on how far a reading moves

This distinction matters and it is easy to miss. The rule the module enforces is on the amount scored. How far a Vessel's reading actually travels depends on how much that Vessel has already done.

```
the priest above    paid 6 and its reading moved 0.225
the target above    took 10 and its reading moved 0.800
```

A priest with almost no history can see its own reading move further than a heavily scored target's, while still paying the strictly smaller shift. The relation is honoured on the shift in every case.

### A large congregation cannot pray the penalty away

This is the dangerous case, and the answer is no. Relief reduces each priest's own shift as the congregation grows, so a large enough group would drive that shift to nothing and steer a world at no cost to itself.

```
six priests    each pays 1, and the prayer files
seven priests  each would pay 0, and the prayer is refused
```

The refusal is not a number anybody invented. A score carrying nothing on either pole is already refused by the module that keeps alignment, so the last workable size falls out of his own two figures.

```
CongregationError: 7 priests reduce each own shift to 0, which scores nothing;
6 is the last size that files
```

With relief set to nothing there is no last size, and a congregation of any size still costs each priest the full amount. Either way the penalty never reaches zero. A smallest own shift, if he sets one, would replace the refusal with a floor and let a congregation grow without limit.

### Being prayed over is not the target's choice, and that is a question for him

His alignment rule reads one way and this mechanism reads another.

```
alignment becomes a function all a Vessel's 'life' choices
```

A prayer is somebody else's choice. After this unit a Vessel's total is its own choices plus what other players did to it. Two readings were available and only one keeps his sentence true.

```
record it as the target's own action   his sentence stops being true
record it apart from the target's own  his sentence survives
```

**The second was built.** A prayer's shift on the target files under its own action name, so the two can always be told apart afterwards. The running total still adds both, which is his roll-up unchanged.

```
the target's own run   razed a field
worked on the target   prayer received
reading from choices   -0.6
reading from prayers    1
reading from both       0.2
```

Whether that is the separation he wants is his to say. The alternative is that a prayer counts as the target's own life choice, which cannot be undone later without rereading every score ever filed.

### Who counts as a priest is not settled

No class in this design is called a priest. The seven are named for planets and metals, and the roles across them are Tank, Damage, pure healer, support healer and pure support.

```
"Should be period consistent with ancient technologies and time aligned to our
core themes while also including all of our magic systems (mage, priest,
necromancer w/ sub specialities)."
```

Mage, priest and necromancer are a third axis over the seven classes, and that axis has never been recorded in code. The three Mercury classes carry the healing and support roles and are the obvious guess. **The guess was not made.**

```
the eligible classes are an argument the caller supplies
a class outside the list is refused by name
the operator names which classes may pray
```

### Which clock a prayer answers to

None. A prayer's shift has no duration to measure, because no scored action decays and the ledger stores neither a turn nor a timestamp. The shift is permanent the moment it files.

```
the world turn   not read; a prayer's shift does not expire
the event turn   not read; the same
a cadence        the one part that would need a clock, and no figure sets one
```

Nothing limits how often one target takes a prayer, from one priest or from many. No consent record exists either. A cadence measured in world turns is the shape the PvP vote already uses for the same kind of limit, and the number is his.

### One received prayer can silence a priest

This came out of the run rather than out of the design, and it is worth stating. A priest sitting at exactly the middle carries no pole, so it cannot pray. One prayer of the opposite direction is enough to put a priest there.

```
priest at the start        net  4 over 10, reading 0.4
after leading one prayer   net 10 over 16, reading 0.625
after receiving one        net  0 over 26, reading 0, exactly balanced
praying again              refused; the middle names neither pole
```

Two priests of opposite alignment cannot cancel each other's penalty with one prayer apiece. The shift a priest takes always exceeds the shift it pays, so one prayer each overshoots the middle. Over an unequal run of prayers they can land on exact balance, and whichever lands there loses the ability to pray at all.

### A place is a second kind of target, and it is not built here

A later directive makes a consecrated place something prayer maintains against decay. That case belongs to another unit and no place exists in this code.

```
Without persistent and repeated prayer by one or more priests, a place will
slowly lose its consecration status.
```

The figures and the relations in this module name no target at all, so a place reuses them without a change. Only the step that files the two scores needs a Vessel, and a place would need its own.

```
the four figures and the relations   target-agnostic, reusable as they stand
the filing step                     needs a Vessel, so a place needs its own
the group pair                      one pair of figures, read by both mechanisms
```

### Nothing prays in the running program

The module is reached by no caller. No ability exists to pray with, no screen shows a prayer or an alignment, and the package entry file does not re-export it yet.

```
In development.
```

### What a prayer reads, and what it hands back

```
reads, nothing provides   how far one prayer moves the target
reads, nothing provides   how far it moves each priest that led it
reads, nothing provides   what each priest past the first adds to the target
reads, nothing provides   what each priest past the first takes off the penalty
reads, nothing provides   which classes count as priests
reads, nothing provides   a priest ability, which is the caller of all of it
provides, nothing reads   a target's alignment moved by another player
provides, nothing reads   each priest's own shift in the same direction
provides, nothing reads   a prayer record naming who prayed over whom
provides, nothing reads   a Vessel's own choices, separable from what was done to it
```

## 2026-09-11 05:30 - #585 - an item is a material list, and cohesion holds it together

### Four item types, and not one name was invented

His rule for this layer is that content is sourced, not made up. The gear subtab already prints the four kinds of item that nothing builds. Those four names were decided before this unit existed, so the item table takes them exactly as the screen spells them. No artefact is named: every entry is a type, and a named instance waits on crafting, which does not exist.

Read off the running surface module and off the new table

```
the subtab prints   "Nothing builds armour, weapons, accessories, consumables,
                     so this subtab manages loot alone."
the table declares   armour, weapons, accessories, consumables
named instances      none, and ITEM_TYPES says so in every row it serves
```

### An item is a material list, and the list is what a salvage would read

One entry in a list names a material, how many whole units of it, and at which quality grade. Every declared type holds one unit of iron ore at the lowest grade. Iron ore is the one material whose embedded band the operator set himself; the other six take iron's band as a working figure, so a list naming one of them would rest on a figure he has not given. One unit is the count because no recipe sets a larger one.

`src/competition/items.py` - one item read off the constructed object

```
{
 "name": "armour",
 "storage_class": "gear",
 "occupies_slot": true,
 "stacks": false,
 "components": [
  {"material": "iron ore", "units": 1, "quality": "Calx", "embedded": "0.00000001"}
 ],
 "cohesion": "0.00000001"
}
```

Every field a rebuild needs is in that row. A salvage reads the material, the units and the grade, and can work out what the item embeds without holding the item.

### What holds an item together is the sum a Vessel already uses

He asked for the Quintessence needed to hold an item together, read down its components. That sum was already built. One function adds a stat block for a Vessel, for a monster, or for an item's components, and it is the same function behind a Vessel's occupancy requirement. This unit writes no second sum: each component becomes one block, and the built function adds them.

`src/competition/entity_stats.py` - the one sum, unchanged by this unit

```
def quintessence_requirement(blocks: Iterable[StatBlock]) -> Decimal:
    """Add every amount of every block in ``blocks``, exactly.

    ``blocks`` holds one block a Vessel or a monster, and one block a component.
    """
```

The conversion table already carried the row that turns a component's stats into cohesion. Measured, it holds a figure of one and marks it working, so cohesion is one for one with that sum today. A coefficient other than one is his to set, and the import check below would report at once if it took a cohesion off the grid.

`src/competition/conversion_rates.py` - the row, read back at import

```
name          item_cohesion_per_component_quintessence
figure        1
provenance    working
its own note  "One for one with entity_stats.quintessence_requirement until a
               component model names a coefficient"
```

### Twenty cohesion amounts, and every one is a whole number of minimum units

Quality is the axis that spreads one material over five amounts, so it spreads an item over five as well. Four types across five grades is twenty readings. Every one is a whole number of the smallest amount the currency can express, the lowest grade is exactly one of them, and nothing lands between two.

Read off the constructed objects, every type identical because no recipe differs

```
grade            cohesion       minimum units
Calx             0.00000001     1
Cauda Pavonis    0.00000002     2
Flores           0.00000003     3
Elixir           0.00000004     4
Magisterium      0.00000005     5
```

### Gear takes a slot, consumables stack, and a resource is a material

His third storage rule is that the three classes must not conflict. Three of the four types take a counted slot and one stacks. No item type takes the resource class, and that is deliberate rather than missing: every material in the material table already declares itself a resource, so a resource item type would restate a material under a second name.

Read off the item table by storage class

```
gear          armour, weapons, accessories      each takes a counted slot
consumable    consumables                       stacks
resource      none                              a resource is a material
stack ceiling absent                            no figure sets how large a stack runs
slot count    absent                            no figure sets a Vessel's slots
```

### His gear rule is one comparison, and nothing asks it

Gear may not carry more Quintessence than the player who equips it. That refusal can be asked today, because a total is already a built figure and the reader that compares a balance to a requirement already exists. One call reads the item's own blocks against a total and answers full or short. Nothing equips anything, so the question has no caller.

`src/competition/items.py` - the same reading at three totals

```
total held     cohesion      answer
0              0.00000001    refused, short by 0.00000001
0.00000001     0.00000001    allowed, exactly covered
5              0.00000001    allowed, a level one Vessel's own requirement
```

The comparison reads the blocks that the cohesion sum adds, so the two figures cannot drift apart. A second subtraction would have been two numbers that can disagree, and there is only one.

### Three ways to abuse a material list, and the answer to each

A list is data, so the question is what a hostile list can declare. Three were driven against the real module and each one refused, with the refusal naming the rule it broke.

What the program raised, quoted from the runs

```
a list with no material
  "a type lists no material, so nothing holds it together and its cohesion
   would be under the 0.00000001 minimum unit"

one material listed twice
  "a type lists ('iron ore', 'iron ore') and names one material twice; raise
   that component's units instead"

a cohesion above everything that can exist
  "a type is held together by 40000000, above the 33000000 Quintessence that
   can exist, so no holder could ever cover it"
```

The third refusal is bounded at the supply cap and no lower. Under the cap, a list can still ask for more than any real player holds, and the gear comparison above is what refuses it. That is the design, not a gap.

### Every cohesion is checked at import, and the check reports its own count

The module drives all four types across all five grades every time it loads. A fault row would mean an item claiming Quintessence that no bucket can hold, which would put that amount outside the ledger's four-bucket sum and let the 33,000,000 limit be passed with nothing reporting it.

What the program printed, from the module's own logging

```
acervator.items INFO drove 4 item types across 5 quality grades,
                     0 cohesions broke a rule
```

### What an item is owed, and what reads one today

Nothing in the running program reads an item. The module loads cleanly through the Proof of Accumulation path, and the package export list does not name it, so a reader has to name the module directly. The material table it is built on sits in the same state.

```
reads this module               nothing
owed                            the module's entry in the package export list
on a screen                     nothing; the gear subtab still prints that the
                                four classes are built by nothing
what this module owes nothing   crafting, salvage, equipping, an inventory, a
                                Vessel's slot count, encumbrance, loot generation
absent figures                  a stack ceiling, a Vessel's slot count, a salvage
                                recovery share
working figures                 the cohesion coefficient at one, and a weight of
                                one per material unit, which nothing multiplies
```

## 2026-09-11 05:50 - #586 - consecrated places, and the prayer that holds one

A priest can now consecrate a place for a guild. The place adds its own weight to that guild's alignment, and the guilds add up to the world, so a held place is the third thing that steers a world. The first is what a Vessel does. The second is a prayer said over another player. This one is ground, and it is held only while priests keep attending to it.

```
src/competition/consecration.py
```

### His words on a consecrated place

```
PoA - Classes - Abilities - Priests - Blessed and Cursed Places - Priests can
further effect world alignment by consecrating locations for their Guilds. These
can be single simple buildings or entire Guild Halls. Size defines the required
time and cost.
```

```
Without persistent and repeated prayer by one or more priests, a place will
slowly lose its consecration status.
```

### Blessed and cursed are one act in two directions

The direction a place is consecrated in is not chosen. It is read off the consecrating priest's own running alignment. A priest on the Creation side blesses, a priest on the Destruction side curses, and the record is otherwise identical. A priest who has done nothing, and a priest whose good and bad cancel exactly, cannot consecrate at all, because neither has a side to read.

```
priest reads  0.5   ->  blessed, the weight goes to Creation
priest reads -1     ->  cursed,  the weight goes to Destruction
priest reads  0     ->  refused, neither side is named
priest unscored     ->  refused, there is no reading at all
```

### A place joins the chain at the guild level

His sentence puts the effect on world alignment and the holding on the guild, so a place enters the chain one level above a Vessel. The module holding the running totals was not touched. A guild reading is now its members' Vessels plus the places it holds, and a world reading is every guild's Vessels plus every guild's held places.

```
register.guild_alignment(ledger, guild, world_turn, rates)
register.world_alignment(ledger, roster, world_turn, rates)
```

One run, two blessed places of different size for one guild:

```
world before any place      net   2   total  4   polarity  0.5
world after two places      net  47   total 49   polarity  0.959
```

The same act by a priest on the other side, on a second guild's place:

```
world before the cursed place    net  38   total 58   polarity  0.655
world after the cursed place     net  -2   total 98   polarity -0.020
```

One cursed place carried that whole world across the middle.

### Size sets the required time and cost, and no figure names either

Size is a whole rank, smallest first. Nothing names how many ranks there are, nor which rank a single simple building holds and which an entire Guild Hall holds. The required turns and the required cost arrive with the size, and a size that lacks either is refused outright.

What is enforced with no figure at all is his relation. A larger place must require at least as much time and at least as much cost as a smaller one, and a ladder that breaks either is refused.

```
REFUSED  size 4 requires 1 world turns and the smaller size 1 requires 2;
         a larger size requires at least as much time
REFUSED  size 4 costs 1 and the smaller size 1 costs 10;
         a larger size requires at least as much cost
```

The cost is declared and nothing spends it. The Quintessence ledger owns every movement of Quintessence, and debiting a guild belongs to a later unit.

### A place without prayer loses its status

A consecration opens at full status and falls by a flat amount for every world turn of silence. A prayer puts the status back up and never down, and more priests praying together restore at least as much as fewer do, which is the same group rule the prayer module already applies to a congregation. The fall rate, the amount a priest restores, and the point at which a place is no longer consecrated are three figures nobody has named, so every reading asks the caller for all three and refuses without them.

One run, at a fall of 0.25 a turn and a lapse point of zero:

```
turn 100   status 1.00   consecrated   the guild's places add 45
turn 101   status 0.75   consecrated   the guild's places add 45
turn 102   status 0.50   consecrated   the guild's places add 45
turn 103   status 0.25   consecrated   the guild's places add 45
turn 104   status 0      lapsed        the guild's places add 0
```

At turn 104 the place stops steering the world and the guild falls back to its Vessels alone. One priest praying on turn 102 took that status from 0.50 to 1.00. Three priests restored 1.5 where one restored 0.5.

A set of figures that no amount of prayer could hold is refused before it is used:

```
REFUSED  a status falls 0.25 a world turn and a praying priest restores 0,
         so no amount of prayer holds a place consecrated
```

### One locator holds one consecration

A place is named by a locator, which is its layer, its square, and the whole percent across that square. One locator carries one consecration. A second consecration there is refused while the first still holds, whoever asks and in whichever direction.

```
REFUSED  3:7:68:12 is already consecrated as blessed for guild
         order_of_the_kiln, last prayed over on world turn 102;
         guild sworn_of_ash cannot consecrate it as cursed
```

A rival takes a place only after it has lapsed. Nothing breaks a consecration that priests are still praying over, and no act exists that would.

### How far stacking reaches, measured

A guild holding enough places could outweigh everything its players have ever done. Twenty places against one Vessel action were measured:

```
turn 100, twenty places held      net 801   total 801   the places carried 800
turn 104, nobody prayed           net   1   total   1   the places carried 0
```

The arithmetic caps nothing, and his decay rule is the bound. Every held place lapses four turns after its last prayer at these figures, so holding twenty places costs twenty prayers every four turns. A guild reaches exactly as far as its priests keep reaching, and no cap of any kind was added.

### Which clock, and what a consecration does not have

Every turn on a consecration is a whole world turn, counted the way the PvP vote counts one. A world turn is one hour by his own words, and `poa_modes` now names that length, so the figure is in the code and not only in prose.

A consecration records the guild it is held for and confers no ownership of the place. The world grid carries no owner on a square, a zone, or a fact. No building, guild hall or structure is constructed either: his own rule made a building system conditional on world instance storage on chain being viable, the world grid is now built with its byte budget measured, so the condition is met and the building is a later unit.

```
no caller       nothing in the running program yet consecrates a place
no priest       no class is a priest, so any Vessel can consecrate
no membership   nothing checks that the priest belongs to the guild named
no prayer       the prayer module writes a player's alignment and targets no
                place, so nothing yet restores a consecration's status
no surface      no tab, no panel and no map glyph shows a consecrated place
no warning      nothing tells a guild that a place it holds is lapsing
```

## 2026-09-11 06:10 - #147 - the package entry reaches every module, and says so when it does not

### Eleven modules exist and the package could not reach them

The package `src/competition` holds 40 modules. Its entry file imported 29 of them. The other eleven were unreachable by the package name, so a reader had to name each module file directly.

Counted by listing the files on the disk and asking the imported package for each name

```
modules on the disk              40
bound by the entry file before   29
unreachable before               11

  alignment      domination      materials       prayer      world_tier
  army_command   guild_roster    monster_spawn   vessels
  consecration   items
```

One unit built each of the eleven, and the brief for each unit forbade editing the entry file. Each unit then wrote down that the entry file still owed its export, and moved on. The manual carries ten such lines, one per module, and the `consecration` unit is the only one of the eleven that never wrote one. One module went further and put the note in its own code, where it prints on every launch.

The constant in `src/competition/domination.py`, read at line 204

```
PACKAGE_EXPORT_OWED = (
    "src/competition/__init__.py re-exports no name from this module, so every "
    "form is reachable by importing src.competition.domination directly"
)
```

Ten units named the gap. None could close it. A note that tells nobody in particular is a queue with no owner, and the same eleven names came back.

### What the entry file now binds

The entry file imports all eleven modules and names their public forms. The order follows what each module reads, because a module placed before the one it reads would stop the package from loading.

```
guild_roster, vessels, materials, monster_spawn, world_tier   read nothing new
alignment     reads guild_roster and vessels
army_command  reads guild_roster
items         reads materials
consecration  reads alignment, guild_roster and vessels
domination    reads alignment and vessels
prayer        reads alignment
```

The entry file adds 322 names. It keeps every existing import line and every existing name.

```
names in the export list before   494
names added                       322
names in the export list after    817
```

### The entry file holds back sixteen names, and each would have overwritten a value

A package name carries one value. Sixteen of the new modules' names repeat a name the entry file already binds, and nine of the ten repeated names carry a different value in each module. Exporting one would silently replace a value that already works, so the entry file skips all sixteen and a reader reaches them by naming their own module.

The two clearest, measured by reading the value from each module

```
DIRECTIONS    monster_table  ('descending', 'ascending')
              consecration   ('blessed', 'cursed')

tier_bounds   loot_drop      the loot tier function
              monster_spawn  a different function of the same name
```

The tenth repeated name is `FIGURE_ABSENT`, and its value is the same string in all four modules that declare it. The entry file skips it with the other fifteen, so this unit redefines no module's own surface.

```
held back from   the name kept      the names held back
monster_spawn    loot_drop          tier_bounds
world_tier       monster_table      FIGURE_ABSENT
alignment        monster_table      FIGURE_ABSENT
consecration     monster_table      DIRECTIONS, FIGURE_ABSENT
consecration     monster_spawn      LocatorError
consecration     alignment          ABSENT_READERS, ABSENT_READER_NOTES
items            vessels            ABSENT_MECHANISMS, ABSENT_MECHANISM_NOTES
items            materials          grid_faults
domination       alignment          ABSENT_READERS, ABSENT_READER_NOTES
prayer           alignment          ABSENT_READERS, ABSENT_READER_NOTES
prayer           world_tier         FigureAbsentError
```

### The package now reports the gap itself, every time it loads

Reconciling the list once does not stop it growing again. The entry file now reads its own folder and compares the module files it holds against the names it binds. The log line names every module file the package holds and does not bind.

`report_unbound_modules` in `src/competition/__init__.py`

```
def report_unbound_modules(held: list[str] | None = None) -> list[str]:
    """Log and return each module file in this package that the imports do not bind."""
    if held is None:
        try:
            held = [path.stem for path in Path(__file__).parent.glob("*.py")]
        except OSError as exc:
            logger.debug("competition package directory unreadable: %s", exc)
            return []
    unbound = sorted(
        name for name in held if name != "__init__" and name not in globals()
    )
    if unbound:
        logger.warning(
            "competition package holds %d module(s) it does not bind: %s",
            len(unbound),
            ", ".join(unbound),
        )
    else:
        logger.debug("competition package binds every module it holds")
    return unbound
```

It never stops the program. The package loads on the path that builds the desktop bridge, and that path runs before the operator sees a window, so a refusal here would be a failed launch over a missing export. Reading the folder is the only step that can fail, and a folder it cannot read returns an empty list and a debug line.

What the program printed on the path from `main.py`, with all 40 modules bound

```
acervator.competition DEBUG competition package binds every module it holds
```

What it prints when the package holds a module it does not bind, read by passing the function a module list with one extra name

```
acervator.competition WARNING competition package holds 1 module(s) it does not
                              bind: tarot_spread
```

The warning goes to the `acervator` log tree, the same place every other part of the platform writes. The healthy line is a debug line, so a normal launch stays quiet and a missing export does not.

### What reads this, and what is still owed

```
reads the export list      src/gui/competition_tab.py, and every module that
                           imports a PoA form by the package name
reads the report           nothing branches on it; it writes to the log only
on a screen                nothing shows it
owed                       a reader that acts on the warning, rather than
                           printing it
absent figures             none; this section measures the module count, the
                           bound count and every name count
```

The earlier entries on this page still say the entry file owes a module's export. Each of those sentences held on the day its unit wrote it, and this unit changes none of them. The export list above closes all ten.

## 2026-09-11 06:30 - #585 - a craft takes Quintessence out of a wallet and destroys none

### His two sentences on crafting are one rule

He said Quintessence used in crafting is burned, or back in the platonic space, and in the same breath that it remains budgeted against the hard limit. Those read as opposites and they are not. The platonic space is the pleroma, and the pleroma is one of the four buckets the supply already counts. What a craft spends leaves a wallet and arrives in a bucket that is still inside the thirty-three million cap, so nothing is created and nothing is destroyed.

His own words, and where each half lands

```
"Quint used in crafting is effectively 'burned' or 'back in the platonic
 space' but remains budgeted against the hard limit"

what the item holds      the embedded bucket
what the craft lost      the pleroma
what the cap counts      both of those, plus the wallets and the held addresses
```

### The movement a craft needs was already built, and nothing had called it

The ledger already carried one movement that does the whole of a craft. It debits a wallet once, puts a named part of that debit into the embedded bucket, and puts the rest into the pleroma. Both halves commit together, and the ledger rolls the whole thing back if any part of it refuses. Measured before this unit: nothing in the tree called it. A craft is now its only caller, and no second mover was written.

`src/competition/quintessence_ledger.py` - the movement, unchanged by this unit

```
def embed_from_wallet(
    self,
    address: str,
    amount: object,
    embedded_amount: object,
) -> QuintessenceEmbed:
    """Move ``amount`` out of address's wallet, ``embedded_amount`` of it into
    the embedded bucket and the remainder into the pleroma.
    """
```

The paired release movement runs the other way, out of embedded and back toward a wallet. That is what destroying an item does, not what making one does, so no craft calls it. Salvage is still absent, and the earlier entry on this page that names it absent is unchanged.

### The amount leaving the wallet equals what the item holds plus what is lost

That equality is the whole of this unit, and it holds without any figure he has not given. The item's cohesion is what the item itself holds, and the built cohesion sum already computes it. The loss is whatever the craft paid above that. Add the two and you have the wallet debit, exactly, with nothing left over. Two separate objects answer those three amounts, so they can be compared rather than assumed.

`src/competition/crafting.py` - the three amounts, each read off its own source

```
@property
def quintessence_embedded(self) -> Decimal:
    """The produced item's cohesion, which is what the item itself holds."""
    return self.item_type.cohesion_at(self.produced_quality)

@property
def quintessence_lost(self) -> Decimal:
    """``loss_share`` of ``quintessence_embedded``, rounded down onto the grid."""
    return quantize_quintessence(self.quintessence_embedded * self.loss_share)

@property
def quintessence_from_wallet(self) -> Decimal:
    """``quintessence_embedded`` plus ``quintessence_lost``, and nothing else."""
    return self.quintessence_embedded + self.quintessence_lost
```

The finished record keeps its own three amounts beside the three the ledger reported, and reads whether they agree. A disagreement refuses the craft rather than recording it. Nothing is compared to itself: one set is arithmetic on an item, the other is what the ledger moved.

Read off a finished craft, at the highest grade and a loss equal to the item

```
the record says         0.0000001 left the wallet
                        0.00000005 is embedded
                        0.00000005 joined the pleroma
the ledger says         spent 0.0000001
                        embedded 0.00000005
                        pleroma 0.00000005
unaccounted             0
```

### A recipe names a material list, a grade, a loss and a turn count

A recipe is the thing that says what is consumed and what comes out. It reuses the component entry the item table already declares, so a material, its whole units and its quality grade are spelled one way across the package. The item it makes comes from the item table by name, and the grade it carries is one of the five quality grades already in use.

`src/competition/crafting.py` - the recipe, as its fields declare it

```
@dataclass(frozen=True)
class Recipe:
    item_type_name: str
    produced_quality: str
    components: tuple[Component, ...]
    loss_share: Decimal
    turns_required: int
```

### Two figures are owed, and a craft without either is refused

He set a salvage loss in words and set no crafting loss at all, and he said crafting is multi-turn and lengthy without naming a turn count. Neither figure is invented here. Both arrive with the recipe, and a recipe built without one refuses at once with a sentence saying what would set it. A loss share of zero is a different thing from an absent loss share, and only the absent one is refused.

Driven, both refusals read off the real objects

```
loss_share absent      "the operator set a salvage loss and set no crafting
                        loss, so loss_share arrives with the Recipe and this
                        module holds no table of it"
turns absent           "The operator's words say crafting is multi-turn and
                        sometimes lengthy and name no figure, and nothing here
                        derives turns from an item's grade, its material count
                        or its cohesion"
```

The loss share multiplies what the item holds, rather than dividing the wallet amount. That choice is made so every amount stays a whole number of the smallest unit the currency can express. Dividing would leave a part no bucket could hold, and the ledger refuses an amount off that grid.

Measured consequence, at the smallest item there is

```
cohesion        0.00000001     one minimum unit
loss share      0.5
loss            0              rounds down, the same rule distil already uses
wallet debit    0.00000001
```

A share above one is allowed and loses more than the item holds, which is what his much of it will be lost permits. Only the wallet balance bounds it.

### Which clock a craft counts in

A craft counts world turns, and the module takes that name from the one place that already declares it rather than spelling it a second time. One world turn lasts 3,600 seconds, which the same place now names. Every turn field here is still a whole turn index, and no craft converts to wall clock time.

`src/competition/consecration.py` - the clock name and its length, both imported

```
CLOCK                        "world turn"
WORLD_TURN_SECONDS           3600, read from poa_modes
WORLD_TURN_SECONDS_ABSENT    "one world turn is 3600 seconds, the operator's one
                             turn an hour"
```

A craft opens on a turn and finishes on the turn its required count reaches, using the same arithmetic the action charge already uses. Completing before that turn is refused, and the refusal says no part of an item exists.

### Four ways to abuse a craft, and the answer to each

Every one of these was asked before the code was written, and each has an answer that can be driven.

```
make an item worth more than     refused. A recipe whose item needs more
what was consumed                Quintessence than its material list embeds
                                 cannot be built at all

craft with no loss at all        allowed. A loss share of zero takes only the
                                 cohesion out of the wallet, the pleroma gets
                                 nothing, and conservation still holds

consume the same materials       refused. Completing a craft removes it and
twice                            records its identifier, so a second call
                                 raises and moves nothing

abandon a craft part way         nothing to lose. An open craft holds no
                                 Quintessence at all, so there is nothing in
                                 flight to go missing
```

The last one is the dangerous case, and the shape of the code answers it rather than a guard. Opening a craft makes no ledger call and abandoning one makes no ledger call. The only write path is completion, and completion is a single movement that either commits whole or rolls back whole. No moment exists at which Quintessence has left a wallet and not yet reached a bucket.

`src/competition/crafting.py` - opening a craft, which moves nothing

```
def begin_craft(self, recipe, vessel, opened_turn) -> Craft:
    """Open ``recipe`` for ``vessel``, refusing a wallet that cannot cover it.

    Nothing is debited here; ``complete_craft`` is the only write path.
    """
```

### A Vessel crafts, and the wallet it spends is shared with every other Vessel

His words put crafting on secondary Vessels, so a craft names a Vessel and takes its owner as the wallet. Nothing marks a Vessel primary or secondary: the Vessel record carries an owner, a class, a level and an id and no such field. A craft takes any Vessel and refuses none on that ground, so which Vessels may craft is owed.

The wallet a craft debits is the same one every Vessel of that Reincarnate runs on, which is his own rule that a player's total wallet budget has to support the work. Spending on a craft lowers the fraction each of them reaches, with no second rule needed.

Measured on a real ledger, one craft at the highest grade

```
before        wallets 1.00000000   pleroma 0           embedded 0
after         wallets 0.99999990   pleroma 0.00000005  embedded 0.00000005
minted        1.00000000 both times, unchanged
delta         0 both times
```

### Driven on a real ledger, and the delta never left zero

The whole path ran against a real ledger at a temporary file, under the interpreter's development checks with warnings promoted to errors. Every reading below came off the constructed objects, not off a description of them. The exercise covered an open craft, an early completion, a finished craft, a second completion, an abandoned craft, a recipe that cannot hold its item together, both absent figures, and a wallet too small to pay.

What the run reported

```
conservation delta          0 at every reading, nine readings
total ever minted           1.00000000 before and after, unchanged
wallet change               -0.0000001
embedded change             +0.00000005
pleroma change              +0.00000005
unaccounted                 0
ledger movement log         distil, then embed_from_wallet, then bleed
exit code                   0
```

The module also drives itself once at import, across every item type at every grade at both ends of the loss range, and reports its own count. That check runs on every launch and raises if a craft would ever move an amount no bucket holds.

The module's own line from the run

```
drove 4 item types across 5 quality grades at 2 loss ends,
0 amounts broke a rule
```

Reading those deltas is itself a check worth naming. Had the reader been blind the three changes would all have read zero while a movement plainly existed in the log. They read the amounts above, and two recipes of genuinely different size were run through the same reader and answered different numbers.

### What reads a craft, and what is still owed

Nothing in the running program yet crafts. The module supplies the recipe, the craft and the accounting, and no panel and no assignment calls any of it. The gear subtab still prints that nothing builds armour, weapons, accessories or consumables, and that sentence is still true from a player's side.

```
reads a craft              nothing. No surface and no Vessel assignment exists
supplies a surface         every finished craft serves a dict the gear subtab
                           could print without any further work
declares a recipe          nothing. A recipe needs the loss share and the turn
                           count, so no recipe table is declared here
package export             bound. The entry file imports 28 of this module's
                           names, and the package report names no module
absent                     assignments, notifications, lifeskilling, salvage,
                           inventory, a crafting skill
absent figures             the loss share, and how many turns a craft takes
```

Nothing counts the materials a Vessel carries, because no inventory exists, so a recipe names units that nothing has checked against a store. The wallet balance is the only thing that refuses a craft today. The module names that gap in its own text rather than hiding it.

## 2026-09-11 06:50 - #147 - seven subtabs, a key each, and the Vessel opens its own page

### What the bar carried before, and what it carries now

The subtab bar held four entries and no keyboard binding. Searching the tab's three files for a key handler returned nothing.

Counted off `SUBTABS` in `src/gui/main_tabs/proof_of_accumulation_tab_surface.py`

```
                         before   after
subtabs on the bar            4       7
keys bound                    0       7
new subtabs                           3     Resources, Quint, Guild
```

The operator named seven: maps, character details, gear, resources, Quint, skills and guild. Four already existed under other labels and three are new. `Skill Tree` became `Skills`, because his own words divide it into a Vessel page and a Reincarnate page, and a tree is one thing a page could hold.

### The key each subtab opens on

Each button carries its key on its face, so nobody has to hunt for it. A sentence under the bar names the range, and each button also carries the key as an `aria-keyshortcuts` attribute for a screen reader.

```
Ctrl+1  Maps               Ctrl+5  Quint
Ctrl+2  Character Details  Ctrl+6  Skills
Ctrl+3  Gear               Ctrl+7  Guild
Ctrl+4  Resources
```

Ctrl and a digit is the published way to reach the Nth tab in the browser engine both hosts run. Nothing in the application takes those keys. Measured by asking the running Electron shell for its own menu, and by reading the Qt side

```
Electron default menu accelerators      15
  of those on Ctrl and a digit           0     Ctrl+0 is Actual Size
globally registered Ctrl+1 to Ctrl+7     0
Qt menu bar mnemonics                    Alt+F, Alt+E, Alt+T, Alt+H
Qt key sequences elsewhere               Ctrl+Left, Ctrl+Right, in one dialog
```

### Clicking the Vessel opens its page

The Vessel card in the top-left half is now the click target itself. Clicking it, or pressing Enter or Space on it, opens Character Details. The card says so on its face.

The card carries a heading and a list, which HTML does not allow inside a button, so it takes the published pattern for making any element behave as one.

```
panelProps.role = "button";
panelProps.tabIndex = 0;
panelProps.onClick = open;
panelProps.onKeyDown = function (event) {
  if (event.key === ENTER_KEY || event.key === SPACE_KEY) {
    event.preventDefault();
    open();
  }
};
```

### What each subtab draws, and what it says it cannot

Every figure below is read from a module at draw time. No number is typed into the screen.

| Subtab | What it draws | Module behind it |
|---|---|---|
| Maps | 5 grid bounds: 9 squares a side, 81 a layer, 20 layers, 79 participants a layer, 1 square in view | `world_grid` |
| Character Details | 5 stats with their principle and effect, the 10 bands a stat climbs, and the 27 trading metrics | `entity_stats`, `rpg_metrics` |
| Gear | the loot the wallet holds, 4 item classes with their cohesion, 8 mechanisms | `loot_drop`, `items` |
| Resources | 7 materials, each with the Quintessence band one unit embeds and where that band came from | `materials` |
| Quint | the chain, its two files, the smallest unit, the supply cap, the minted total | `quintessence_ledger` |
| Skills | two named pages and the one skill on the ladder, over its 10 levels | `skill_ladder` |
| Guild | 0 guilds, 0 members, the two ranks and the treasury address shape | `guild_roster` |

Three things the operator named are not built, and each subtab says so in its own words rather than showing an empty frame.

```
skill trees    no module declares an ability and no module declares a tree
skill rings    no module declares a ring, a tier or a Sephirot position
tactical map   nothing holds an encounter's enemies or their turn order
```

The Skills subtab names both pages, Vessel and Reincarnate, and says under each that nothing supplies its tree or its ring. The Guild roster loads empty because no module writes a roster file, and it says that too.

### Maps opens only where a map exists

The rule that already governed the map subtab still governs its key. Under an event whose mode carries no map, Ctrl+1 does nothing and the button is disabled with the reason on it.

```
Monster Smash   Ctrl+1 refused    "Monster Smash carries no map, so this subtab
                                   does not open. Dungeon Crawl and Raid do."
Dungeon Crawl   Ctrl+1 opens Maps
```

### Both builds were driven, and every reading matched

Each of the seven keys was pressed as a real keypress and the subtab showing afterwards was read off the drawn page, not off the code. The Vessel was clicked the same way. The Qt window and the Electron shell were driven separately against the same live backend.

```
readings compared                           157
readings that differed                        0
seven keys, seven different subtabs         Qt yes, Electron yes
Vessel click opens Character Details        Qt yes, Electron yes
Ctrl+9, which nothing binds, changes nothing Qt yes, Electron yes
```

### One sentence on the tab was wrong and is corrected

The Vessel card used to say that no subtab carried its details, so nothing opened. That subtab now exists, so the card names it instead.

```
VESSEL_CLICK_NOTE = (
    "Click this Vessel to open the Character Details subtab, or press its shortcut."
)
```

The Gear subtab used to say nothing builds the four item classes. Those classes are now declared with their material lists, and a craft that makes one landed the same day, so the sentence now names the limit that is real.

```
GEAR_ABSENT_TEXT = (
    "{names} are declared as classes. {inventory}, so this subtab lists the loot the "
    "wallet holds and no item a Vessel wears."
)
```

### Still owed

The items module listed crafting among the things nothing supplies, while the crafting module now completes one. That entry now names the recipe instead. The Gear subtab prints that module's own sentence, so correcting the module corrected the screen.

```
ABSENT_MECHANISM_NOTES = {
    "a crafting recipe": (
        "crafting.CraftRegister.complete_craft turns a component list into an "
        "item, and no Recipe is declared because loss_share and turns_required "
        "are two figures no source sets"
    ),
    ...
}
```

## 2026-09-11 07:10 - #585 - the package binds crafting, and the gear list names the real gap

### The package named the module it could not reach, at every start

The crafting module landed and the package entry file imported nothing from it. The entry file reads its own folder at every load and compares the module files it holds against the names it binds, so it named crafting on every start.

What a fresh interpreter printed before this unit

```
acervator.competition WARNING competition package holds 1 module(s) it does not
                              bind: crafting
```

### What the entry file binds from crafting

The entry file imports crafting last, after prayer. The module reads items, materials, the Quintessence ledger, vessels and one name from consecration, so every module it reads is already imported above it. A module placed before the one it reads would stop the package from loading, and the package loads before the operator sees a window.

```
# crafting imports items, materials, quintessence_ledger, vessels and consecration.
# 6 repeats of names bound above, grid_faults among them, stay unexported.
from .crafting import (
    MIN_CRAFT_TURNS,
    CRAFT_MOVEMENT,
    RELEASE_IS_SALVAGE,
    LOSS_SHARE_ABSENT,
    ...
    recipe_for,
    recipe_rows,
)
```

Every existing import line and every existing name is kept.

```
names in the export list before   817
names added                        28
names in the export list after    845
module files on the disk           41
module files the package binds     41
```

### Six names stay unexported, and four of them would have overwritten a value

A package name carries one value. Six of crafting's names repeat a name the entry file already binds. Exporting one would replace a value that already works, so the entry file skips all six, and a reader reaches them by naming the crafting module.

Measured by reading the value out of each module

```
the name the package keeps        crafting's own value        the two
materials.grid_faults            a different function         differ
vessels.ABSENT_MECHANISMS        nine absences of a craft     differ
vessels.ABSENT_MECHANISM_NOTES   nine notes of a craft        differ
the package logger               acervator.crafting           differ
monster_table.FIGURE_ABSENT      None                         are the same
items.MIN_COMPONENTS             1                            are the same
```

Two of the six carry the same value in both modules, so holding those two back changes nothing a reader sees. No module's own surface is redefined.

### The items module said nothing builds a craft

The items module listed crafting among the mechanisms no module builds. The crafting module completes a craft, so that entry named the wrong gap. It now names the recipe. The recipe is absent because two figures are owed, the loss share and the turn count, and one decided pair of figures is all a recipe for each item type would need.

The absent-mechanism list and its notes in `src/competition/items.py`

```
ABSENT_MECHANISMS: tuple[str, ...] = (
    "a crafting recipe",
    ...
)

ABSENT_MECHANISM_NOTES: dict[str, str] = {
    "a crafting recipe": (
        "crafting.CraftRegister.complete_craft turns a component list into an "
        "item, and no Recipe is declared because loss_share and turns_required "
        "are two figures no source sets"
    ),
    ...
}
```

### The Gear subtab prints the module's own sentence

The gear panel builds its mechanism rows straight from the items module's notes, so correcting the module corrected the screen and no screen file changed.

```
"mechanisms": [
    row(name, note) for name, note in ITEM_MECHANISM_NOTES.items()
],
```

### Driven on the real tab path

The desktop bridge registry was built and the PoA tab's own handler was called through it, under the interpreter's development checks with every warning raised as an error. The eight rows the Gear subtab draws were read off the payload the handler returned, and the package's own log line came from that same run.

```
acervator.competition DEBUG competition package binds every module it holds

a crafting recipe   crafting.CraftRegister.complete_craft turns a component list
                    into an item, and no Recipe is declared because loss_share and
                    turns_required are two figures no source sets
salvage             nothing destroys an item for the Quintessence it embeds
equipping           nothing holds an item to a Vessel, so equip_check has no caller
inventory           nothing holds the items one Vessel carries
slot count          no figure sets how many gear slots a Vessel has
encumbrance         nothing weighs what a Vessel hauls
loot generation     loot_drop.drop_from_pool draws a tier and names no item
named instances     no artefact is named here; every entry in ITEM_TYPES is a type,
                    and a named instance waits on crafting naming what it made
```

Each crafting name was read off the package and compared against the object the crafting module holds.

```
crafting names read off the package      28
names that did not match the module       0
names held back                           6
imported by the package name alone       Recipe, CraftRegister, recipe_for
```

### What reads the crafting export, and what is still owed

```
reads the export list      every module that imports a PoA form by the package
                           name, and src/gui/competition_tab.py
reads the gear list        proof_of_accumulation_tab_surface.gear builds the Gear
                           subtab's mechanism rows from it
on a screen                the Gear subtab, eight rows
owed                       a recipe. Nothing declares one, and the operator naming
                           one loss share and one turn count would set both figures
                           every recipe needs
absent figures             the loss share and the turn count, both absent. No
                           module defaults either, and Recipe refuses a craft
                           without them
```

The crafting module's own text still says the package binds no name from it, and the 06:10 entry above still counts 40 modules on the disk. Each held on the day its unit wrote it, and this unit changes neither.

## 2026-09-11 07:15 - #586 - the world turn's pool, counted in steps

> *"Its a function of Strength (max weight) and Constitution (turn point penalty while carrying)."*
>
> *"Just need to be able to say that character A traversed x% of a given square in a given turn... Should be simple enough to make this dynamic and per character or army or group with encumberance playing a role."*

### One clock was bounded and the other was not

The event turn has an action economy. Impetus grants four actions at the first level and one more every twenty levels, so a Vessel at level 100 acts nine times in one candle and the pool refuses a tenth.

Read by asking the modes module for each figure

```
IMPETUS_AT_FIRST_LEVEL      4
IMPETUS_LEVELS_PER_STEP     20
base_impetus(1)             4
base_impetus(100)           9
```

The world turn had none of it. No pool, no budget and no action count existed for the hour a participant spends out in the world, so a choice made in world time cost nothing.

### The step was already a world turn's denomination

A square divides into one hundred steps, so a step is one whole percent of a square. His sentence about traversing a percentage of a square in a turn names a step count, and movement already spends steps by the turn.

The rate a journey leg runs at, in `world_movement`

```
#: Steps a mover covers in one world turn before any multiplier, one square.
BASE_STEPS_PER_TURN = SQUARE_STEPS


def leg_rate(terrain: Decimal, encumbrance: Decimal) -> Decimal:
    """``BASE_STEPS_PER_TURN`` under the ``terrain`` and ``encumbrance`` multipliers."""
```

That is a rate, not a pool, and the difference is the whole of this unit. Three readings separate them, each taken off the movement module itself.

```
it holds no remainder    progress_at derives every figure from the turn a leg
                         opened, its rate and its length, and writes no record.
                         Nothing is subtracted, so no participant has steps left
                         to read

it is bounded per leg    open_leg checks the legs of the one journey it is given,
                         so no rule there compares two journeys of one mover

only movement spends it  a craft counts whole turns to its completion turn and
                         spends no step in any of them
```

The budget did not exist under another name. The step is the right unit for one, because a penalty measured in steps removes distance, and removed distance is exactly how carrying a load slows a journey.

### The pool, and what it refuses

One participant holds one pool for one world turn. The pool keys on a wallet address, because the mover on a journey leg carries an address and a Reincarnate answers to one. Impetus counts per level, which belongs to a Vessel, and no directive says which of the two holds a world turn's pool.

The grant, and the only figure that lowers it

```
    @property
    def granted_steps(self) -> int:
        """``unladen_steps`` less ``penalty_steps``, never below `MIN_GRANT_STEPS``."""
        return max(MIN_GRANT_STEPS, self.unladen_steps - self.penalty_steps)
```

The spend, which refuses another turn and refuses a cost above what is left

```
    def spend(self, cost: int, at_turn: int) -> int:
        """Spend ``cost`` steps, refusing another turn or a cost above ``remaining``.

        Returns the steps remaining after the spend.
        """
        asked = _as_turn_index(at_turn, "at_turn")
        if asked != self.turn_index:
            refusal = (
                f"this pool granted {self.granted} {STEP_UNIT}s to "
                f"{self.participant} for {CLOCK} {self.turn_index} and "
                f"{self.remaining} is unspent; {CLOCK} {asked} is a different "
                f"turn, and a {CLOCK}'s steps expire with the turn that "
                f"granted them"
            )
            raise ExpiredTurnError(refusal)
        steps = _as_steps(cost, "cost", COST_SOURCE, MIN_SPEND_STEPS)
        if steps > self.remaining:
            refusal = (
                f"this action costs {steps} {STEP_UNIT}s and {self.remaining} "
                f"remains of {self.granted} in {CLOCK} {self.turn_index}; no "
                f"partial action exists and nobody borrows against the next turn"
            )
            raise ExhaustedPoolError(refusal)
        self.spent += steps
        return self.remaining
```

Seven refusals were driven through the real module, each one quoting what the module said

```
a pool with no figure   unladen_steps is absent; unladen_steps arrives with the
                        grant and no source names it ... The operator sets the
                        figure

a fractional grant      unladen_steps must be a whole number of steps, not float;
                        a world turn figure decided by binary floating point is
                        refused

a penalty that raises   penalty_steps must be 0 or more steps, got -5

a second pool           0xalice already holds a pool of 40 steps for world turn 0
                        with 40 unspent; one participant holds one pool a world
                        turn

a closed turn reopened  0xalice holds a pool for world turn 5, and world turn 4
                        has closed; a closed turn grants nothing a second time

a spend in another turn 0xalice holds a pool for world turn 0, not world turn 1;
                        a world turn's steps expire with the turn that granted
                        them

a spend of nothing      cost must be 1 or more steps, got 0
```

### What the program printed

One participant's turn was driven from its grant to its refusal, on the path that loads the PoA package. Every figure below is the module reporting its own pool.

```
acervator.world_turn INFO 0xalice holds 40 steps for world turn 0, 40 unladen less 0 penalty
acervator.world_turn INFO 0xalice spent 25 of 40 steps in world turn 0, 15 remaining
acervator.world_turn INFO 0xalice spent 40 of 40 steps in world turn 0, 0 remaining
acervator.world_turn INFO 0xcarol holds 0 steps for world turn 0, 40 unladen less 55 penalty
acervator.world_turn INFO 0xbob holds 40 steps for world turn 1, 40 unladen less 0 penalty
acervator.world_turn INFO 0xbob spent 10 of 40 steps in world turn 1, 30 remaining
acervator.world_turn INFO 0xbob lost 30 of 40 steps unspent when world turn 1 closed
acervator.world_turn INFO 0xbob holds 40 steps for world turn 2, 40 unladen less 0 penalty
acervator.world_turn INFO 0xalice holds 28 steps for world turn 1, 40 unladen less 12 penalty

src.competition.world_turn.ExhaustedPoolError: this action costs 29 steps and 28
remains of 28 in world turn 1; no partial action exists and nobody borrows
against the next turn
```

Four lines carry the answers a reader needs. A penalty of 55 against a grant of 40 reads zero and never less, so a fully laden turn buys nothing and never owes anything. Thirty unspent steps are lost when the turn closes, and the next turn grants a fresh forty. The penalty of twelve takes the grant to twenty-eight, and a spend of twenty-nine is refused.

### The figure is owed, and it is one number

```
owed      unladen_steps, the steps one participant may spend in one world turn
          at zero encumbrance

why not   leg_rate already multiplies 100 steps by terrain and by encumbrance, so
100       a grant read off that rate would count both multipliers twice

refused   a pool built without it, by name, with that sentence in the refusal
```

A world turn's length is 3,600 seconds, and `poa_modes` names it. Every turn field here is still a whole turn index, the same way the PvP vote and a consecration both count one.

### What spends this, and what is still owed

```
reads it today     nothing. No module constructs a world-turn pool, and the only
                   lines above came from driving it directly

movement           derives a leg's progress from its own rate and reads no pool

encumbrance        absent. It needs a Vessel's carried weight and a weight per
                   material, and no module holds either, so nothing computes the
                   penalty a grant takes

crafting           counts whole turns to its completion turn and spends no step

a surface          the tab's turn meter reads the event turn's candle and its
                   Impetus line reads that pool. No panel names a world turn, so
                   nothing shows a turn's steps remaining

the package entry  does not yet name this module. The package says so itself on
                   every launch: competition package holds 1 module(s) it does
                   not bind: world_turn
```

---

## 2026-09-11 07:40 - #586 - entering a dungeon moves a participant to the other clock

### His question, and the answer it has

He asked it as a design question rather than a work item.

> *"PoA - Dungeons - World Map - Another example of needed inference. What happens after a player enters a dungeon? These are the sorts of design questions I want you to ask while proceeding. Who. What Where. When. Why. How."*

The answer is a clock change. Outside a dungeon a participant counts world turns, which run continuously and survive an absence. Inside one they count event turns, which last one market candle and expire when that candle closes. Entering a dungeon moves a participant from the first clock to the second, and leaving moves them back.

`src/competition/dungeon_entry.py` - the two clock names, each read from one place

```
#: The clock outside a dungeon, the name ``consecration`` already declares.
WORLD_CLOCK = CLOCK

#: The clock inside a dungeon, one candle of ``EventVariant.turn_timeframe``.
EVENT_CLOCK = "event turn"

#: The two clocks a participant can be on. There is no third.
CLOCKS: tuple[str, ...] = (WORLD_CLOCK, EVENT_CLOCK)
```

### The two clocks nest exactly, and the program printed the count

The event turn is one candle of the event type's own timeframe. A Standard event runs on the five-minute candle and an Elite event on the one-minute candle, both taken from the same table the exchange reads. A world turn is an hour. Every dungeon variant divides an hour with nothing left over, so a participant inside a dungeon never holds part of a turn.

`src/exchange/data_pool.py` - the one table both clocks read

```
TF_SECONDS["1h"]  3600    the world turn, an hour
TF_SECONDS["5m"]   300    the Standard event turn
TF_SECONDS["1m"]    60    the Elite event turn
```

What the program printed, driven directly:

```
dungeon_crawl        5m   300s   12 event turns an hour
dungeon_crawl_elite  1m    60s   60 event turns an hour
raid                 5m   300s   12 event turns an hour
raid_elite           1m    60s   60 event turns an hour
```

The module refuses a world turn the candle does not divide, and rounds nothing. Driven with 3,500 seconds against the five-minute candle the refusal reads:

```
a world turn of 3500s holds 11 whole 5m candles and 200s over; the two clocks
nest as exact multiples or a participant inside holds part of a turn
```

### Two of the four event types hold a dungeon, and the mode table says which

Nothing lists the dungeon modes by name. The event mode table already carries a map flag, and reading that flag answers the question once for each caller.

`src/competition/dungeon_entry.py` - the dungeon modes, derived

```
#: The modes held at a map locator, read off ``EventMode.has_map``.
DUNGEON_MODES: tuple[EventMode, ...] = tuple(mode for mode in MODES if mode.has_map)
```

The table the program printed:

```
mode                      dungeon  party      guild
monster_smash             no       1 to 1     no
team_monster_smash        no       2 to 120   yes
dungeon_crawl             yes      1 to 6     no
raid                      yes      2 to 60    yes
```

This matches his own two sentences. Fixation is where every player starts, so the arena is not a dungeon and nobody enters one there. Descension takes one to six and Cementation takes two to sixty, and Cementation needs a guild where Descension does not.

### A dungeon is a locator, and no module declares one

No module in the tree declares a dungeon object. A place on the world map is a fact with a kind, and the kind is whatever text the discovering caller passed. No list of valid kinds exists, so a dungeon is a locator plus a name.

`src/competition/dungeon_entry.py` - what the module says about this itself

```
DUNGEON_IS_A_LOCATOR = (
    "no module declares a dungeon. world_grid.WorldFact carries a kind and that "
    "kind is caller text with no declared values, so a dungeon is the locator a "
    "fact sits at plus the kind whoever discovered it named"
)
```

An entry names the square and the position across it that the party stands on, and the module derives the locator from those. No caller can claim an entry at a place the party does not stand on.

### One entry, one exit, and both are rebuildable from the record

An entry records who went in, which place, which event type, the world turn they left, and the event turn they joined. An exit records both turn indexes again. Nothing edits either record afterwards, so the two rebuild the whole visit.

`src/competition/dungeon_entry.py` - the entry record's own fields

```
entry_id: str
world_id: str
fact_id: str
locator: str
layer: int
square_index: int
step_x: int
step_y: int
kind: str
mode_code: str
elite: bool
leader: str
members: tuple[str, ...]
entered_world_turn: int
entered_event_turn: int
entered_at: float
```

Driven on a real world with a real party of three, the program printed:

```
0xLeader took 3 into ruined_keep at 3:45:68:12,
off world turn 41 onto event turn 5856666 of the 5m candle

0xLeader brought 3 out of 3:45:68:12,
off event turn 5856696 back onto world turn 44
```

Written to a file and read back, the rebuilt entry and exit compared equal to the originals and the clock reading after the replay was the world turn.

### The run read the clock three times off the real objects

Before the entry, during it and after it, for each of the three party members and for one participant who never entered.

```
                  before entry   inside        after exit
0xLeader          world turn     event turn    world turn
0xSecond          world turn     event turn    world turn
0xThird           world turn     event turn    world turn
0xElsewhere       world turn     world turn    world turn
```

The last row is the control. The same reading that moves for each member inside does not move for someone who is not, so a reading of `event turn` means the participant is genuinely in a dungeon and not that the reader always says so.

### Five things make an entry impossible

The run drove each one and each refused. Every refusal below comes from that run.

```
an arena mode          Monster Smash is held at no map locator, so nothing is
                       entered in it. Only [dungeon_crawl, raid] hold a dungeon

the wrong party        this party was formed for Dungeon Crawl and admits 1 to 6;
                       entering a Raid needs a party formed in raid

an undiscovered place  0xElsewhere leads this Dungeon Crawl and holds no
                       reference to u76world:3:49:1:2; a dungeon is entered only
                       where it has been discovered

already inside         0xSecond is inside 3:45:68:12 on entry ... and answers to
                       the event turn; entry ... would put one participant on two
                       event clocks at once

leaving twice          entry ... returned to world turn 44 already; nothing is
                       inside it to bring out a second time
```

The third refusal is his own rule in code. He wrote that a party must find a dungeon out in the world first, so the party's leader must already hold a reference to the place. The leader is the one who navigates, which is his rule too.

### Four ways to abuse an entry, and what each one does

```
enter twice              refused. One participant holds at most one open entry

enter while inside       refused, the same check. Driven on a member of a party
                         that was already in, not only on the leader

enter with a member       not refused, and this is a gap. One position is taken
who is elsewhere          for the whole party, and no module holds a position per
                          member, so nothing can tell that a member stands away

escape a world-time       no. Measured: a journey leg kept covering ground on
threat by going in        world turns 41 to 44 while the party was inside, and an
                          incapacitation set to end on world turn 44 still held on
                          41 and 43 and lapsed on 44. Neither reads the dungeon
                          register, so world time does not pause for anyone inside
```

The last one was the question worth asking. Because world time keeps running, a dungeon is not a hiding place, and that is the same answer his own rule gives for a secondary Vessel whose work survives an absence.

### Two parties can stand in one dungeon at once

Nothing caps how many parties hold an open entry at one locator. Driven, a party of three and a separate participant both entered the same place on the same event turn, and the register admitted both.

**In development.** Whether a dungeon admits one party at a time belongs with the inside of a dungeon, which is the ruling below.

### What a dungeon still does not have

Every one of these is absent rather than partly built, and the module says so itself through a reader that lists them.

```
the inside          HIS RULING IS OWED, and the two readings cost very different
                    amounts. A dungeon may be a place ON the world grid, in which
                    case a participant keeps the square and the step it already
                    has and nothing new is needed. Or a dungeon may be a separate
                    space with its own coordinates, which is a second grid with
                    its own sight range and its own set of positions, roughly
                    doubling the world state. His seven-level dungeon points at
                    the second. The entry is built so either can follow it

combat              nothing resolves a fight. The monster tiers exist and the
                    stat blocks exist, and no module turns two of them into a
                    result

permadeath          his ten-round window has no code and a round has no declared
                    length, so entering a dungeon currently costs nothing

a member's position  one position is taken for the party. Movement derives a
                    position from each mover's own journey leg, so a member
                    standing somewhere else is not refused

losing a member      one exit closes the whole entry. Whether a single member can
                    leave alone, and what a party of none becomes, is unanswered

the world turn       nothing acts on a world turn closing while a party is
closing mid-dungeon  inside. The entry records the turn they left and the exit
                     records the turn they returned to, and no rule of his covers
                     the turns between

the world turn's     3,600 seconds, named in poa_modes as the 1h timeframe.
length               event_turns_per_world_turn takes that length as its own
                     default, so no caller supplies one

the screen           this module draws nothing. His map click opens the map
                    subtab, and the screen belongs elsewhere
```

### What reads a dungeon entry, and what is still owed

```
reads it today     nothing. No module enters a dungeon, and every line above
                   came from driving the entry directly

combat             would read the entry to know which event turn a fight is on,
                   and does not exist

a surface          would read the entry to show a party inside a dungeon, and the
                   map subtab reads no entry

the world-turn     grants one pool a world turn. Nothing connects it to an entry,
pool               so a party inside still holds its world-turn steps

the package entry  does not yet name this module. The package says so itself on
                   every launch: competition package holds 2 module(s) it does
                   not bind: dungeon_entry, world_turn
```

## 2026-09-11 08:00 - #147 - the world map draws one layer, and only what one participant found

### The grid is nine squares a side, and the module says so

The Maps subtab now draws squares. Before this it drew the grid's bounds as a table of five numbers and nothing else. The width is not written on the screen or in this page. It falls out of the byte budget, and the module computes it.

`src/competition/world_grid.py` - the width, derived from the budget

```
#: The default grid side, holding ``BUDGETED_SQUARES_PER_LAYER`` squares.
DEFAULT_GRID_WIDTH = smallest_whole_side(BUDGETED_SQUARES_PER_LAYER)
```

What the module answered when asked:

```
squares a side          9
squares a layer        81
layers                 20
participants a layer   79
squares in view         1
```

### A square draws only what that participant has found

His rule is that a zone appears on the map after player discovery, and that one player discovering something does not discover it for everyone. The map is therefore drawn per participant. The surface reads the knowledge references that one participant holds, and nothing else reaches a square.

`src/gui/main_tabs/proof_of_accumulation_tab_surface.py` - the only source of a mark

```
def known_facts(world: PoaWorld, world_id: str, address: str, layer: int) -> tuple:
    """Every ``WorldFact`` on ``layer`` that ``address`` holds a knowledge reference to."""
    found = []
    for reference in world.knows(world_id, address):
        try:
            fact = world.fact(world_id, reference)
        except WorldGridError:
            continue
        if fact.layer == layer:
            found.append(fact)
    return tuple(found)
```

A square with no fact in that list carries a mark with no glyph, so it draws nothing at all. That is the whole of the lazy-discovery rule on screen.

### Four states, and each one looks different

A square is the participant's own, or discovered, or undiscovered. Sight is a fourth reading laid over the first three, and today it covers one square.

```
own          the arena square, a stronger ground
in view      a gold outline, from BASE_VIEWRANGE_SQUARES
discovered   a lighter ground, and the glyph of the thing found there
undiscovered the darkest ground, and nothing drawn inside it
```

### Both variants drew the same layer, read off the pictures

A world was created in a scratch home, one layer breached, and five places discovered. A second participant discovered a sixth place that the first participant was never told about. The tab was then drawn twice. Every figure below comes off the rendered pages, not off the payload behind them.

```
                              desktop window   Electron shell
squares drawn                       81               81
grid columns                         9                9
grid side in pixels             155.6            155.6
the participant's own square        40               40
squares in view                      1                1
squares drawing a glyph              4                4
squares drawing nothing             77               77
```

The sixth place, found by the other participant, read as undiscovered and drew nothing on both pages. That is his rule holding on screen.

### A missing glyph draws as a box, and the reading can see one

A code point that exists in Unicode is not a glyph that exists on the machine. Two permanently reserved noncharacters were drawn in the page's own resolved font, and their pixels were counted. Both produced the same 71 inked pixels, which is the replacement box. Each mark on the map was then measured the same way and compared against it.

```
the box, twice          71 inked pixels, identical
Magisterium  U+26E4     68 inked pixels, gold
Elixir       U+2625     59 inked pixels, pink
Calx         U+2296     83 inked pixels, grey
Flores       U+2698     68 inked pixels, blue
```

No mark matched the box on either page.

### Nothing creates a world on a normal start

No module in the tree calls the function that makes a world. The map above was produced by calling it directly against a local test chain in a scratch home. On the operator's machine the subtab therefore says there is no world, and draws no grid rather than an empty one.

`src/gui/main_tabs/proof_of_accumulation_tab_surface.py` - the three reasons nothing draws

```
no world file      No world is generated. No grid, no tile and no position is
                   held anywhere, so this subtab draws no map.
no breached layer  World <name> declares 20 layers and nobody has breached one,
                   so no layer exists to draw.
no identity        bot_identity.json does not exist, so no participant is named.
```

Each sentence names what would set it. The screen with no world was rendered and read: zero squares drawn, and no grid element on the page at all.

### Where a participant is, and why it does not move

The arena is the one square known with no discovery record, and it is where the map puts the participant. A journey record can derive a later square, but the derivation needs the index of the running world turn.

```
world_movement.JourneyLeg.progress_at   derives a square from a turn index
poa_modes.WORLD_TURN_SECONDS            3600 seconds in one world turn
```

No module can say which turn is running now, and the map stays on the arena. The subtab prints that sentence, so a reader knows the position is a default and not a choice.

### What this map does not draw

Four things his design names and nothing supplies.

```
the tactical map   his words turn the map into a tactical map during an
                   encounter. Nothing holds an encounter's enemies or their turn
                   order, so no tactical map is drawn
an action menu     his words give a location options when a player moves into
                   the correct square. Movement exists and no module holds an
                   enter, a scout or a camp, so no menu is drawn
pixel art          no sprite exists anywhere. The glyphs are the stand-in, which
                   is the point of the Dwarf Fortress decision
monster marks      map_glyphs gives monsters no mark on purpose, because six of
                   the twelve tiers carry no name and no hermetic set covers
                   twelve positions
```

A fact whose kind carries no mark is drawn as discovered with no glyph. One of the five places above was given such a kind, and the square read as discovered and empty on both pages.

## 2026-09-11 08:15 - #147 - the package binds its last two modules

The PoA package held 43 module files and named 41 of them. Two were reachable only by their own full path. The package reported that gap itself on every launch, because it reads its own folder and compares the files it holds against the names it binds.

What a fresh interpreter printed before this unit

```
acervator.competition WARNING competition package holds 2 module(s) it does not
                              bind: dungeon_entry, world_turn
```

The entry file now imports both. It keeps every existing import line and every existing name, and adds no line of arithmetic.

```
names in the export list before   845
names added                        56
names in the export list after    901

names added from world_turn        17
names added from dungeon_entry     39

module files on the disk           43
module files the package binds     43
```

### Where the two imports sit, and why that order holds

Both imports go after crafting, at the end of the import section. A module placed before the one it reads would stop the package from loading, and the package loads before the operator sees a window. Every module these two read is already bound above them.

The import edges, read out of the two module files

```
world_turn     reads consecration and world_movement

dungeon_entry  reads world_turn, poa_modes, world_grid, consecration and the
               platform's own atomic JSON writer

army_command   named in dungeon_entry only under TYPE_CHECKING, so it is not a
               runtime edge and sets no order
```

The entry file takes the world turn first, because the dungeon entry reads its first turn index. The reverse order would put a module before the one it reads.

```
# world_turn imports consecration and world_movement, dungeon_entry imports world_turn,
# poa_modes, world_grid and consecration.
# 4 repeats of names bound above, ABSENT_READERS among them, stay unexported.
from .world_turn import (
    STEP_UNIT,
    ...
    turn_economy_row,
)
from .dungeon_entry import (
    WORLD_CLOCK,
    ...
    nesting_rows,
)
```

### Four names stay unexported, and three would have overwritten a value

A package name carries one value. Four of the two modules' names repeat a name the entry file already binds. Three of the four carry a different value in each module, so exporting one would silently replace a value that already works. The entry file skips all four. A reader reaches them by naming their own module.

Measured by reading the value out of each module

```
held back from   the name the package keeps       the two values
world_turn       alignment.ABSENT_READERS         differ
world_turn       alignment.ABSENT_READER_NOTES    differ
world_turn       monster_table.FIGURE_ABSENT      are the same
dungeon_entry    vessels.ABSENT_MECHANISMS        differ
```

The three that differ, read side by side

```
ABSENT_READERS        alignment      7 names, cataclysms first
                      world_turn     4 names, movement first

ABSENT_READER_NOTES   alignment      one note for each of its 7
                      world_turn     one note for each of its 4

ABSENT_MECHANISMS     vessels        a tuple of 8 absences
                      dungeon_entry  a dictionary of 9 absences and their notes
```

Each module also holds its own logger, and the entry file exports neither. The package keeps one logger of its own, named for the package.

```
the package     acervator.competition
world_turn      acervator.world_turn
dungeon_entry   acervator.dungeon_entry
```

### Driven through the PoA tab's own handler

The run built the desktop bridge registry and called the PoA tab's own handler through it, under the interpreter's development checks with every warning raising an error. That same run read both modules' names off the package, and the package's own log line came from it.

```
acervator.competition DEBUG competition package binds every module it holds

the PoA handler returned          38 sections
the surface module imported       src.gui.main_tabs.proof_of_accumulation_tab_surface

world_turn off the package        STEP_UNIT 'step', MIN_SPEND_STEPS 1
dungeon_entry off the package     CLOCKS ('world turn', 'event turn')
                                  DUNGEON_MODE_CODES ('dungeon_crawl', 'raid')

report_unbound_modules returned   an empty list
```

A circular import would stop the launch rather than slow it, so the surface module importing is the proof that the order holds.

### What the two prior entries said, and what is owed

The 07:15 and 07:40 entries above each end with a line saying the package entry does not yet name that module, and each quotes the warning it drew at the time. Both held on the day their unit wrote them. This unit changes neither, the same way the 07:10 entry left the counts before it standing.

```
owed  dungeon_entry.EXPORT_OWED still reads "src/competition/__init__.py binds
      no name from dungeon_entry". The package now binds 39 of its names, so
      that sentence is stale. The module is not this unit's to edit, and the
      export is now on the package, so a reader sees the stale sentence and not
      a warning

owed  domination.PACKAGE_EXPORT_OWED reads "src/competition/__init__.py
      re-exports no name from this module". The package has bound domination for
      two entries, and the module logs that sentence at every load

absent  world_turn carries no owed-export note of its own, so nothing in it went
        stale

absent  no file under src/gui or desktop names either module, so neither note
        reaches a screen and no screen file changed
```

## 2026-09-11 08:30 - #147 - a control creates the world, and the map draws it

### The press declares a world and breaches its first layer

Nothing in the program created a world. The function that makes one had no caller anywhere under `src`, so the map had nothing to draw and every module that addresses a world sat idle for that one reason. The Accumulation tab now carries two more buttons, beside the two that reset the chain. The first asks and writes nothing. The second declares the world and opens its first layer in one press.

`src/gui/main_tabs/proof_of_accumulation_tab_surface.py` - the press, in full

```
record = world.create_world(world_id)
layer = world.breach_layer(world_id, address, FIRST_LAYER_INDEX)
```

Two calls, because one is not enough. A world on its own draws nothing: the map asks for the layers somebody has breached, and a new world has none. The map says so itself, and the run read both of its refusals straight off the screen.

```
no world          No world is generated. No grid, no tile and no position is
                  held anywhere, so this subtab draws no map.
no breached layer World WORLD-testnet declares 20 layers and nobody has
                  breached one, so no layer exists to draw.
```

The second sentence is the one a world alone would leave on screen. Breaching layer zero in the same press is what clears it.

### The figures a world declares, each read off the module

A new panel sits under the two buttons and reports the world the chain holds. It draws nothing of its own: every figure comes from the world store's own summary. The width, the layer count and the arena are the module's constants and functions, not numbers written here or on screen.

`src/gui/main_tabs/proof_of_accumulation_tab_surface.py` - what the panel drew

```
World               WORLD-testnet
Squares a side      9
Squares a layer     81
Layers declared     20
Layers breached     1
Arena square        40
Seed commitment     3ccc798a7a12165e056c33eef0c09bb83b7007042219d0286ffe94d1b19d3fb8
Seed published      False
```

Before the press the same panel holds no rows and one sentence instead, naming the button that would fill it. The world lands in its own file on the demo chain, and that file is what the Maps subtab reads back.

```
the file            poa_world_testnet.json
what it holds       worlds, seeds, breached, zones, facts, knowledge,
                    extracted, published, bands, version
```

Two buttons and a panel cost the control bar height, and the bar was already scrolling before them. It shows 169 points of its content and the rest scrolls, which is how the bar has always worked. The panel sits beside the reset readout rather than under it, and that placement gives back 89 of the 189 points the stacked version took.

```
control bar, before this unit     178 points of content in a 169 point window
the two buttons, stacked panel    367 points
the two buttons, panel beside     278 points
what stays visible with no scroll the buttons, and the green result line
                                  carrying the sentence and all eight figures
```

### Three refusals, and the live chain is one of them

The control that deletes a chain already refuses the live chain and already refuses without a confirmation. A control that writes chain state keeps the same two, and adds a third of its own: one world a chain. The run drove all three, and each sentence below is the one the program printed.

```
the live chain      A world is created on the Demo TestNet chain only. The Live
                    chain is the one a running window holds, so this refuses
                    there.
no confirmation     Confirm the world writes poa_world_testnet.json and declares
                    world WORLD-testnet: 9 squares a side, 81 a layer, 20
                    layers, arena at 40. This control writes nothing.
a second world      poa_world_testnet.json already holds world WORLD-testnet.
                    One world a chain, so this refuses a second. Confirm the
                    reset clears the chain and the world with it.
```

After the unconfirmed press the run looked for the file, and the file was still absent. The second press came after a world existed, and found the first one unchanged.

### The seed never reaches a screen, and the run searched the output for it

A world's seed decides every creature and every amount in it. Anyone holding it could work out the whole world in advance, so the module keeps it back and publishes only its commitment. That makes the seed the one value this work had to keep off the screen, and a search proves that better than a reading does.

```
the seed              32 hex characters, drawn by the module
its commitment        matches the hash of that seed, confirmed
files searched        9 - both rendered pages, both reports, the payload,
                      the console output of both runs
files holding it      none
files holding the
commitment            7 of the 9
```

The last line is the control. A search that finds the commitment in seven files is a search able to find a string of that shape, so the empty seed result is a fact about the output and not about the search. A line carrying the seed was then written into a tenth file, found, and the file removed.

### Both variants drew the board, read off the pictures

The run drew both the desktop window and the Electron shell with a world on the demo chain, and every figure below comes off the saved picture. The count comes from the grid's own gap lines in the image: nine columns and nine rows leave ten lines across and ten down.

```
                              desktop window   Electron shell
squares in the picture              81               81
columns, from the picture            9                9
rows, from the picture               9                9
gap lines across                    10               10
gap lines down                      10               10
the participant's own square          1                1
squares drawing nothing             80               80
squares drawn as discovered          0                0
```

All eight panel figures matched across the two, and so did the sentence under them. One reading differed, and that difference belongs to the screen rather than to the product: the desktop window runs at a device pixel ratio of 1.25 and reports the grid side as 155.6 points, the shell runs at 1.0 and reports 156. The same ratio accounts for the gold ring around the participant's square measuring 144 device pixels in one and 112 in the other.

### Two earlier sentences are now conditional

Two sentences on this page said the Map subtab still reports no world, and both were true when written. They stay true of a program nobody has pressed anything in, and they stop being true after the press.

```
what it said       "No drawing. The Map subtab still says no world is
                   generated, and that sentence is still true."
what replaces it   The subtab says that until the world control is confirmed,
                   and draws 81 squares after it.

what it said       "The Map subtab's sentence that no world is generated stays
                   true, because installing the mechanism creates no world."
what replaces it   Installing the mechanism still creates no world. The
                   control does, on a press, on the demo chain only.
```

The heading that says nothing creates a world on a normal start stands exactly as written. A start creates no world; an operator pressing a button does.

### What the press does not create

The press stops at a world and its first layer. It reaches none of the five things below, and this page names each one rather than half-building it.

```
participants       nobody is entered into the world. The control's sender is
                   this node's own identity and no player record is written
discovery          no place is found. All 81 squares read as undiscovered, so
                   the board draws no mark at all
monsters           none are generated. The seed that would decide them is
                   committed and nothing reads it yet
dungeons           none exist. A dungeon is a locator and no module declares
                   one
consecration       no ground is consecrated, and the press reaches no part of
                   that mechanism
```

The participant stands on the arena square, and this press does not change that. Nothing in the program can say which world turn is running, so nothing can derive a later square. That figure is still owed.

```
the seconds in a world turn   3,600, in poa_modes. What is still unset is the
                              index of the turn running now, which would let the
                              map move the participant off the arena
```

A second gate already sat in front of every button on this tab, and it now sits in front of these two as well. The node needs an identity file before any control will act, and on the operator's machine that file does not exist. The one piece of the program that writes it belongs to the Competition tab, a shelved screen.

```
the file           bot_identity.json, under the runtime directory
what writes it     src/gui/competition_tab.py, on a shelved screen
what it blocks     every control on this tab, with the sentence the tab
                   already prints
```

## 2026-09-11 08:35 - #585 - what one Vessel carries, in two halves

### The two halves of one Vessel's store

Four modules each recorded the same gap in their own words. The item table said nothing holds the items one Vessel carries. The craft register said a recipe names material units nothing has counted. The material table sets no ceiling on how far a stack runs. A loot drop named the wallet it went to and named no item. A store now exists, and it has two halves, under the operator's rule that gear, consumables and resources must not conflict.

`src/competition/inventory.py` — the storage class decides which half

```
    def put_in(
        self,
        name: str,
        quality: str,
        units: object = MIN_MOVED_UNITS,
    ) -> StoreChange:
        moved = _as_moved_units(units, "units")
        held = _as_holding(name, quality)
        with self._lock:
            if occupies_slot(held):
                self._fill_slots(name, quality, moved)
            else:
                self._raise_stack(name, quality, moved)
            return self._change(name, quality, moved, is_put_in=True)
```

The three class names are the ones the material table already declares. Gear takes a counted slot. A consumable and a resource go on a stack and take no slot. Nothing here adds a fourth class and nothing renames one.

### Three gear slots filled, and the fourth item refused

A store was opened for one Vessel with three gear slots. Three pieces of armour went in, one at a time, and the slot count and the weight were read off the real object after each one. The fourth piece was refused, and the store was read again afterwards to show it held what it held before.

```
slot_count=3 slots_used=0 slots_free=3 stack_ceiling=100 weight_carried=0

put_in armour #1: held=1 slots_used=1 weight_carried=1
put_in armour #2: held=2 slots_used=2 weight_carried=2
put_in armour #3: held=3 slots_used=3 weight_carried=3

a fourth armour: refused, SlotsFullError: vessel-a holds 3 of 3 gear slots and
1 armour takes 1 more; no gear is held in a slot this Vessel does not have

after the refusal slots_used=3 weight_carried=3
```

The refusal changed nothing. A store never holds gear in a slot the Vessel was not given.

### Forty units of ore stacked, and fifteen taken back out

Forty units of lead ore went into the same store at the lowest grade. The slot count did not move, which is the whole point of the two halves. Fifteen units came back out, and then three ways of taking out too much were each refused.

```
put_in 40 lead ore: held=40 slots_used=3 weight_carried=43
take_out 15 lead ore: held=25 weight_carried=28

taking more than is held: refused, HoldingUnitsError: vessel-a holds 25 lead ore
at Calx and 26 was asked for; a stack never falls under nothing

taking zero units: refused, HoldingUnitsError: units must be a whole number of
units at or above 1, got 0

taking gear never put in: refused, HoldingUnitsError: vessel-a holds 0 weapons
at Calx and 1 was asked for; a store gives out nothing it does not hold
```

A stack cannot go negative and cannot give out what it does not hold. Forty units took no slot at all.

### The weight carried adds up to what is held

The store was then read item by item, and each weight was added by hand off the two lists the store serves. That hand total was compared with the one number the store reports.

```
  gear  armour at Calx weighs 1
  gear  armour at Calx weighs 1
  stack 25 lead ore at Calx weighs 25 (1 a unit)

added by hand off held_gear and held_stacks = 27
store.weight_carried                        = 27
the two readings agree: True
```

An empty store read zero. The weight carried is the sum of what is held and of nothing else.

### The derived store key drops the id, so the store is named by the caller

A Vessel holds an owner address, a class name, a level and an id. The derived key reads the first three and not the id, so a key made from them alone cannot separate a player's second Iron Edge from the first. The craft register already keys a Vessel this way, and the same collision sits in it.

```
vessel_key(vessel)  = 0xEkthelius:Iron Edge:1
vessel_key(second)  = 0xEkthelius:Iron Edge:1
```

The store takes its key from whoever opens it, and the book refuses a key it already holds. That is how two Vessels of one class under one owner get two stores. The derived key is still served, and its collision is written next to it. The Vessel id is the clean answer and this module does not read it yet.

### Two figures stop a store opening, and two more wait on a hauling rule

No module sets how many gear slots a Vessel has. No module sets how far a stack runs. Both arrive from whoever opens the store, and a store refuses to open without either of them. The weight one material unit carries, and the weight one Quintessence of strength can lift, both sit in the conversion table as working figures of one. The operator has not decided either.

```
a Vessel's gear slot count                   no figure, required to open a store
the stack ceiling                            no figure, required to open a store
weight per material unit                     1, working
maximum weight per Quintessence of strength  1, working

slot_count absent: refused, StoreFigureError: slot_count is absent; no figure
sets how many gear slots a Vessel has

stack_ceiling absent: refused, StoreFigureError: stack_ceiling is absent;
materials.STACK_CEILING_ABSENT carries no figure and the operator's word is
'large', so every VesselStore takes its stack_ceiling from its caller
```

The first two are refusals, so no store ever runs on a slot count nobody chose. The third is read off every material and is used in every weight above. The fourth is read by nothing here, and an encumbrance rule is what would read it.

### One hand-over, and what a refused one leaves behind

Moving something between two Vessels is one call. It takes out of the first store before it puts into the second, and it puts the units back when the second store refuses them.

```
10 lead ore at Calx moved from vessel-a to vessel-b
  the giver holds 15, the taker holds 10
  vessel-a weight_carried=17  vessel-b weight_carried=10  added = 27

a handover past the taker's ceiling: refused, StackCeilingError: vessel-c holds
0 lead ore at Calx and 15 more reaches 15, above the 5 stack ceiling it was
built with
  vessel-a held 15 before and 15 after
  vessel-c holds []

gear into a no-slot store: refused, SlotsFullError: vessel-c holds 0 of 0 gear
slots and 1 armour takes 1 more
  vessel-a gear unchanged, two pieces of armour
```

The weight the first store carried before the move, twenty-seven, is the weight the two stores carry after it. A move moves; it does not mint and it does not lose.

### A bad grade landed in the store before the refusal

The first version of this module checked the quality grade too late. A grade the table does not carry was written onto a stack, and the refusal then came from the reading that followed the write. The caller saw a refusal and the store was left holding a thing nothing could read.

```
before the fix
  put_in with grade 'Pristine'   refused, UnknownQualityError
  held_stacks                    raised UnknownQualityError
  weight_carried                 raised UnknownQualityError
  to_dict                        raised UnknownQualityError
  a later good put_in            raised UnknownQualityError

after the fix
  put_in with grade 'Pristine'   refused, UnknownQualityError
  held_stacks                    ()
  weight_carried                 0
  to_dict                        read back clean
  a later good put_in            held 1 lead ore
```

For a player that was every item in the Vessel becoming unreadable from one mistyped grade. The grade and the name are now checked before anything is written, on the way in and on the way out.

### Can the same thing be in two stores at once

It cannot be put in two stores by a hand-over. The hand-over removes before it adds, and its refusal path was driven above and returned the units. The store keyed under one name exists once, and a repeated key is refused.

```
a repeated store_key: refused, StoreKeyError: vessel-a already names a store

can a stack exceed what exists        no module counts a world supply of a
                                      material, so stack_ceiling is the only
                                      bound there is
can gear sit in a slot that is not
there                                 no, the slot count refuses it
can weight be carried with no store   no, weight_carried is a reading on a
                                      store and no free function weighs a list
```

The open half is honest and is named. An item carries no name of its own yet, so two stores each holding one piece of armour cannot be told apart from one piece recorded twice. Nothing outside this module creates units, so today the only writer is a caller. Named instances come with a craft that names what it made.

### What a store does not do

```
encumbrance        the weight is read and nothing turns it into a refusal or a
                   turn point penalty; both figures it needs are working
a hauling trip     the operator's rule is that encumbrance counts only while a
                   Vessel moves items itself, and nothing carries anything
a foraging run     nothing gathers a material, so every put-in is a caller's
equipping          the item table answers one total against one cohesion, and no
                   slot here holds gear to a Vessel's Quintessence
trading            a hand-over asks no price and nothing prices a holding
a surface          no panel calls this module
a package export   the package entry binds no name from it
```

### What the two subtabs read, and what is still owed

The Gear subtab and the Resources subtab each print a sentence about this gap today, and the run read both off the real handler.

```
Gear       "Nothing holds the items one Vessel carries, so this subtab lists the
           loot the wallet holds and no item a Vessel wears."
Resources  "Nothing holds how many units a participant carries, so no material
           shows an amount."
Resources  "No stack ceiling is set and no salvage recovery fraction is set."
```

The store answers the first two of those three. The gear list and the slot count are what the Gear subtab needs, and the units held at a grade are what the Resources subtab needs. Neither subtab calls the module, and the screen files belong to another unit, so nothing on screen changes today.

```
owed  src/competition/__init__.py binds no name from inventory. The package says
      so itself at every start: "competition package holds 1 module(s) it does
      not bind: inventory"

owed  a slot count a Vessel actually has, and a stack ceiling for the operator's
      word 'large'

owed  the store reading the Vessel id. Two Vessels of one class at one level
      under one owner are two values, and the store still works round the
      derived key with a caller-supplied one

absent  encumbrance, a hauling trip, a foraging run, equipping, trading, a
        surface. Each is named in the module and none is built here
```

## 2026-09-11 08:50 - #585 - a Vessel has an id, and a level gained keeps it

A Vessel carries a fourth field. Two Vessels of the same class, at the same level, under the same owner were one value, and three other modules each wrote that collision down beside a workaround for it. The field closes the collision at the record. None of the three workarounds changes here.

```
src/competition/vessels.py
```

### Where the id comes from, and the three shapes that could not answer

The package already makes identifiers four ways, and one of them makes a value that is new on every call. A world draws its seed and then hashes it, so the same call twice gives two worlds. The Vessel id follows that shape, with the owner address hashed in beside the drawn value.

```
VESSEL_ID_BYTES = 32


def new_vessel_id(owner: object) -> str:
    """Draw one id for a Vessel of ``owner``, hashing the address with a drawn value."""
    address = _as_address(owner, "owner")
    drawn = hashlib.sha256(
        f"{address}|{secrets.token_hex(VESSEL_ID_BYTES)}".encode(),
    ).hexdigest()
    logger.info("%s drew vessel id %s", address, drawn)
    return drawn
```

The other three shapes were read and set aside, each for a reason the code states about itself.

```
the craft register    joins the owner, the class and the level with colons. That
                      is the colliding triple itself, and the module says so
the guild roster      normalises a name its caller gives. No Vessel carries a
                      name, and no register holds Vessels to refuse a repeat
a counter per owner   needs a register that counts. Nothing keeps a Vessel
                      against an address, so there is nothing to count in
```

The owner goes into the hash with the drawn value, so one drawn value under two addresses gives two ids. The result is sixty-four hex characters, the width every other identifier in this package already has.

### Two Vessels of one class under one owner are two values

The run built two Vessels with the same owner, the same class and the same level, then read the id off each object. The same two compared equal before the field existed.

```
before   a == b: True
         Vessel(owner='0xEkthelius', class_name='Iron Edge', level=1)
         Vessel(owner='0xEkthelius', class_name='Iron Edge', level=1)

after    a == b: False
         a  8ded319904cf1dbbd4765b27795c9937055a6706308126c602bb8ecb39f1717a
         b  eb5d96cf18ace2abd1aacf3b728451f0304d9e4af7a1fbd3287133a1012a085e
```

A failure here would mean a player's second Iron Edge is the first Iron Edge to every module that holds one. One store, one alignment and one craft history would cover both.

### A level gained keeps the id, and so does a theft

The level is not in the drawn value, and the field is copied when a Vessel is rebuilt, so the id survives both changes a Vessel goes through today. An id that moved on a level-up would orphan the Vessel's store and reset the alignment it earned.

```
level 1 to level 2   id unchanged: True
owner moved          id unchanged: True
```

The package proves the second half without being asked. Importing it builds seven Vessels and steals one, and the program's own log shows seven ids drawn for eight Vessel objects, because the stolen one kept its own.

```
acervator.vessels INFO 0xvictim drew vessel id f19c44141db54d5ed0193c0a2af1d07bca069ae4f8fc446626b9c569df16e306
acervator.vessels INFO 0xthief drew vessel id be929363404f610478e19abe412e694f694b2806a259473182f038d0cdc9bfca
acervator.domination INFO 0xthief stole 0xvictim's Lead Ward on world turn 0
```

### What stops a forged claim

Nothing in the id stops one, and nothing needs to. The owner field stops it, and it did so before this change. A caller can copy another player's id onto a Vessel of their own address, and the Reincarnate record refuses that Vessel because its owner is a different address.

```
same id under another owner   accepted at the Vessel
Reincarnate refuses it        Iron Edge is owned by '0xThief', not by '0xEkthelius'
```

What the id adds is that it cannot be guessed. It is a hash over thirty-two drawn bytes, so naming one Vessel's id needs that Vessel's record. No store reads the field yet, so no store can be reached with one either way.

### A Vessel cannot be built without an id

A blank id, a run of spaces and a tab each draw one. A value that is not text is refused. No call in the module returns a Vessel whose id is empty.

```
given ''      id length 64
given '   '   id length 64
given a tab   id length 64
given None    vessel_id must be a string, got NoneType
given 7       vessel_id must be a string, got int
```

Two Vessels can still share an id, because a caller may pass a copy of one and nothing refuses it. Nothing keys on the field yet, so nothing is in a position to. Each of the three modules below already refuses a repeated key of its own, and each becomes the refuser when it keys on this field.

### What the three consumers may read now

All three can read the field today with no change to the record. None of them does, and each is its own unit.

```
inventory   vessel_key joins the owner, the class and the level. The module states
            the collision beside it, and a store still takes its key from a caller
crafting    craft_id_for joins the recipe with those same three fields
alignment   vessel_key files a score under the owner and the class, leaving the
            level out so a level gained keeps the alignment
```

Every module that builds a Vessel still builds one. The three construction sites all pass three values in order, so the field takes its default and draws.

```
src/competition/domination.py      the driven theft, seven Vessels
src/gui/main_tabs/proof_of_accumulation_tab_surface.py   current_vessel, one
```

### What this entry leaves owed

No screen shows the id. The wallet's Vessels section and the Character Details subtab each draw a Vessel and neither prints it, and the screen files belong to another unit.

```
owed  inventory keys a store on the Vessel id
owed  crafting keys a craft on the Vessel id
owed  alignment files a score under the Vessel id
owed  the package entry binds new_vessel_id and VesselIdError. Neither name is in
      the export list, and the module itself is bound
owed  three constants say a Vessel carries no id, each in a file this entry does
      not edit: inventory.VESSEL_ID_ABSENT, alignment.VESSEL_ID_OWED and
      domination.HEALTH_SOURCE_ABSENT
owed  something that keeps a Vessel against an address. current_vessel builds one
      on every read, so the id behind the tab is a new id each time
```

## 2026-09-11 08:55 - #585 - a drop names the thing that dropped

### A drop carried a tier and no thing

Two units recorded the same gap in their own words. The loot module made a drop that named a rarity tier and a wallet, and named nothing that had dropped. The item table declares four types. The store holds items in two halves. A drop had nowhere to go, and the drop record is what changed.

`src/competition/loot_drop.py` — the record names the type, and refuses one the item table does not declare

```
    item_id: str
    tier_name: str
    short_form: str
    item_type: str
    exchange_id: str
    symbol: str
    season: int
    roll: int
    holder: str
    dropped_at: float

    def __post_init__(self) -> None:
        """Refuse an ``item_type`` or a ``quality`` the item tables do not declare."""
        drop_storage_class(self.item_type, self.quality)
```

Nothing about the five tiers changed. Their names, short forms, weights, Impetus relief and effect bonuses are the same bytes as before, the weights still total 100, and the map mark registry still resolves all five.

### The five quality grades are the five loot tiers, in one tuple

An item's cohesion varies by quality grade. A previous unit measured that the grades are the loot tiers themselves. The run read both sets off the real modules and they are one object.

```
TIER_NAMES          ('Calx', 'Cauda Pavonis', 'Flores', 'Elixir', 'Magisterium')
QUALITY_GRADES      ('Calx', 'Cauda Pavonis', 'Flores', 'Elixir', 'Magisterium')
same tuple object   True
```

The material table assigns its grade tuple from the loot module's tier names, so the two cannot drift apart. A drop's tier already names its quality. No second scale exists, and the module adds none. The drop carries no separate grade field and reads its own tier name instead.

### Which type a tier yields is an input, and an absent one refuses the drop

No source names the item type a tier yields. The operator has not set it and no module holds it. The loot module decides nothing and asks its caller.

```
#: What no source sets, and what ``require_yields`` refuses a drop without.
TIER_YIELD_ABSENT = (
    "no source names the item type a loot tier yields, so drop_for_market takes a "
    "yields mapping from its caller and refuses a drop without one"
)
```

One figure stays owed: the mapping from each of the five tiers to one of the four item types. Without that mapping the module draws no drop at all.

### Four ways a drop is refused before it exists

Each refusal ran on the real objects. Each sentence is the module's own.

```
no mapping at all
  TierYieldError: no source names the item type a loot tier yields, so
  drop_for_market takes a yields mapping from its caller and refuses a drop
  without one

a type the item table does not declare
  DropItemError: Calx yields 'potions', which is not a PoA item type; the table
  carries armour, weapons, accessories, consumables

a tier that is not one of the five
  UnknownTierError: Lapis names no loot tier; the five are Calx, Cauda Pavonis,
  Flores, Elixir, Magisterium

a tier left without a type
  TierYieldError: Elixir carries no item type, so a roll landing on it would
  name no thing
```

The item type list in that second message is the item table's own. The loot module writes no second list of types and no second list of grades.

### The item table is read when a drop is made, not when the module loads

The material table imports the loot module for its tier names, and the item table imports the material table. The plain import the other way was driven and it failed.

```
ImportError: cannot import name 'rate_named' from partially initialized module
'src.competition.conversion_rates' (most likely due to a circular import)
```

The item table is read inside the call that needs it. One comment in the file carries the rule a reader needs, and it is that a drop's storage class comes from the item type and never from the drop.

### A drop delivered into the slot half, read off the store

A store was opened for one Vessel with one gear slot and a hundred unit stack ceiling. The drops came from eight real rolls under the module's own seed. The store was read off the real object before and after.

```
before    held_gear () held_stacks () slots_used 0 weight_carried 0

roll  844 tier Cauda Pavonis  item_type accessories  storage_class gear

deliver   vessel-u84 took in 1 accessories at Cauda Pavonis; it now holds 1,
          1 of 1 gear slots, weight 1

after     held_gear (SlotItem(name='accessories', quality='Cauda Pavonis'),)
          held_stacks ()
          slots_used 1 weight_carried 1
```

The store decided which half took it. The drop named a type and a grade, and the storage class the item table gives that type is what sent it to a counted slot.

### The same drop refused a second time, and refused to a second store

A delivery is recorded against the item's own identifier, which is the hash of the drop's own contents. A second delivery of the same item is refused, to the same store and to any other.

```
again, same store     DropDeliveryError: item 2d960fa0e8cfab76 already went to
                      vessel-u84; one drop reaches one store

to a second store     DropDeliveryError: item 2d960fa0e8cfab76 already went to
                      vessel-u84; one drop reaches one store

the second store held ()  ()
```

### A consumable drop goes on a stack and takes no slot

The same store had no free gear slot left. A consumable drop still went in, because a consumable stacks and takes no slot.

```
roll  370 tier Calx  item_type consumables  storage_class consumable

deliver   vessel-u84 took in 1 consumables at Calx; it now holds 1, 1 of 1 gear
          slots, weight 2

after     held_gear (SlotItem(name='accessories', quality='Cauda Pavonis'),)
          held_stacks (Stack(name='consumables', quality='Calx', units=1),)
          slots_used 1
```

The two halves did not take each other's space. That is the operator's own rule on gear, consumables and resources, and the store already enforced it.

### A gear drop into a store with no free slot stays earned

This is the question that matters for a player. A full store refuses gear, so a gear drop can be unplaceable. The delivery is recorded only after the store accepts it, so a refused drop stays undelivered and can be placed later.

```
slots_free            0
refused               SlotsFullError: vessel-u84 holds 1 of 1 gear slots and 1
                      accessories takes 1 more; no gear is held in a slot this
                      Vessel does not have

the store after       held_gear (SlotItem(name='accessories',
                      quality='Cauda Pavonis'),)  slots_used 1
delivered_to          None
still undelivered     6 of 8

the same drop into a store with a free slot
                      vessel-u84-b took in 1 accessories at Cauda Pavonis; it
                      now holds 1, 1 of 4 gear slots, weight 1
```

Nothing was lost. The store was unchanged by the refusal and the drop was still the player's to place.

### The file carries the deliveries, and a swapped type is refused

The loot file now records which store took each item, and the file version moved from one to two. No version one file exists on this machine, so nothing had to be converted.

```
drops replayed        8
deliveries replayed   vessel-u84, vessel-u84-b
still undelivered     5

a second replay       LootStoreError: ... is already loaded; a second replay
                      would double it

a file whose item type was swapped for 'potions'
                      LootStoreError: ... could not be replayed: 'potions' is
                      not a PoA item type; the table carries armour, weapons,
                      accessories, consumables
```

The run compared the swapped file against the good one and printed that the bytes differed, so that last refusal is a reading and not an assumption.

### Nothing in the running program delivers a drop into a store

The tab's drop button was pressed on the real handler. It refuses before it reaches any of this, because no market qualifies on this machine.

```
action   drop
acted    False
message  pool_below_floor:  holds 0 eligible markets, under the floor of 12, so
         it draws nothing
```

The button adds a drop to the loot file and never puts one in a Vessel's store. No module calls the delivery. The three new refusals are all raised as loot faults, which the tab already catches and prints as its own message, so a press after the mapping is set will read the owed figure on screen rather than crash.

### One sentence in the items module is now false

The items module still prints that loot generation draws a tier and names no item, and the Gear subtab prints that sentence word for word. That file and the screen files belong to other units, so neither was touched.

```
the sentence   loot generation     loot_drop.drop_from_pool draws a tier and
                                   names no item
where          src/competition/items.py, in ABSENT_MECHANISM_NOTES
now false      a drop names one of armour, weapons, accessories, consumables
owed           the owning unit rewrites that note, and the Gear subtab stops
               printing it
```

One sentence in the 08:35 entry above is corrected by this change, from "A loot drop names the wallet it went to and names no item" to "A loot drop named the wallet it went to and named no item". Nothing else on this page is reworded.

### What reads a drop's item type, and what is still owed

The Gear subtab lists the loot a wallet holds. It prints the tier short form, the market symbol, the season and the tier bonus, and it reads no item type.

```
reads the item type   nothing

provides and unread   item_type, quality and storage_class on every drop,
                      LootStore.deliver, LootStore.drop, LootStore.delivered_to
                      and LootStore.undelivered

owed                  the mapping from each of the five tiers to one of the four
                      item types. One figure, and the only one

owed                  a Vessel on the drop. A drop names a wallet address and a
                      store is keyed per Vessel, so the delivery is given the
                      receiving store by its caller

owed                  the package entry binds no new name from this module

absent                a loot table per monster, a chest, a dungeon drop, a
                      control that delivers, a surface that shows an item type.
                      None is built here

## 2026-09-11 09:20 - #147 - a control creates the node identity, and the other controls stop refusing

### The press writes one keypair and names this node

Every control on this tab refused, and all of them refused for one reason. The
file that names the participant was written by nothing a running window builds.
The only code that wrote it sits on the Competition tab, a screen nothing
constructs. The tab now carries two more buttons, at the head of the control row.
The first asks and writes nothing. The second writes the file and names this node.

`src/gui/main_tabs/proof_of_accumulation_tab_surface.py` - the press, in full
```

path.parent.mkdir(parents=True, exist_ok=True) made = BotIdentity(str(path)).generate()

```
One call is enough, because the identity module already loads an existing record
rather than replacing it. The control refuses before it reaches that call anyway,
so a second press answers with a sentence on screen instead of a silent
no-change.

### What the identity record holds, and what the panel never draws

A new panel sits in the control bar and reports the identity this node holds. It
draws four readings. The run below was made on a scratch home with no identity in
it, and every line is what the program printed.

`src/gui/main_tabs/proof_of_accumulation_tab_surface.py` - what the panel drew
```

File bot_identity.json - held Participant 429a35254f71 Public key 429a35254f710d3c6355fa127e45bf4b8a83e2938862a902689e6d25a721879f Created 2026-09-11 09:06:42

```
The record on disk holds five fields. Four of them are above. The fifth is the
private key, and it reaches no row here and no line anywhere else.
```

the file bot_identity.json, 288 bytes what it holds version, bot_id, created_at, pubkey_hex, privkey_b64 what is drawn the four readings above what is not privkey_b64, the half that signs

```
Before the press the same panel holds one reading and one sentence instead,
naming the button that would fill it.
```

File bot_identity.json - no file yet the sentence bot_identity.json holds no identity. Confirm the identity is the only thing that writes one, and until it is pressed every other control here refuses.

```
### The control that refused now runs, and both readings came off the page

The panel filling up is the smaller half of the proof. The half that matters is a
different control, one that refused before the press, running after it. The run
drew the page in both builds and read the result line off the drawn document.
```

before the press Distil refused bot_identity.json does not exist, so no participant is named. after the press Distil acted Distilled 0.1 Quint.

```
Nothing about the Distil button changed. The only thing that changed between
those two lines is that an identity exists.

### Two refusals, and the replacement is the one that matters

A control that writes lasting state on this tab asks first and acts second, which
is how the chain reset and the world control already work. This control keeps
that, and adds one of its own: one identity a node, with no way back.
```

no confirmation Confirm the identity writes bot_identity.json and names this node by the public half of one keypair. One identity a node, and both chains read it. This control writes nothing. a second one bot_identity.json

already holds the identity of participant 429a35254f71. One identity a node, so this refuses a second: replacing it abandons the wallet, the Vessel, the guild seat and the store that one owns.

```
The second sentence is the whole reason this control is careful. An identity is
the address that owns things. A new keypair would not lose the old wallet, the
old Vessel or the old guild seat; it would leave them on an address nobody on
this machine can sign for again. After the refused second press the run read the
file back, and the participant was the first one, unchanged.

### One identity a node, and both chains read it

The reset control and the world control both refuse the live chain, because both
write chain state. This control writes none, and the file it writes carries no
chain name.
```

the path the runtime directory, and the file name with no suffix who reads it the tab's own participant lookup, and the node name the shared TestNet bridge answers with what that means the Demo TestNet chain and the Live chain name this node the same way, off one file

```
The control therefore acts on either chain, and pressing it twice from two chains
is the same second press, refused the same way.

### The private key reaches no output, and the search proved it can find one

The private key is the one value this work had to keep off every surface. A
search proves that better than a reading does, and a search nobody has watched
find anything proves nothing at all. The run searched every file it wrote for the
stored key in all three of its shapes, then planted the key in one more file to
show the search works.
```

files searched 27 - both rendered pages, four payloads, both readings files, both console outputs, the scratch runtime tree shapes looked for the stored text, the same bytes in hex, and the raw bytes files holding the private key none files holding the public key 12 of 27 planted one file with the key found, 1 file removed that file none again

```
The public key line is the control. A search that finds it in twelve files can
find a string of that shape, so the empty result for the private key is a fact
about the output rather than about the search. The pictures are covered
separately: the panel's rows were counted off the drawn document, four labels and
four values, and the private key is in none of them.

### What the file lands with, and what already guards a credential here

The identity file lands beside the exchange credentials, so the run measured both
rather than inventing a rule.
```

bot_identity.json 0o666 coinbase_credentials.json 0o666 instance.lock 0o666, and the code that opens it asks for 0o600 a plain write 0o666 an open asking for 0o600 0o666 a change to 0o600 after 0o666

```
Windows does not carry those permission bits, so all three ways of asking give
the same answer on this machine. Nothing in the tree narrows a credential file
either, which makes this file consistent with the one beside it. The guard that
does exist here is a habit rather than a permission: the suite's own walk over
the runtime tree reads sizes and never opens a file, so it cannot read the
credentials it walks past.

### Both variants drew the panel, read off the pictures

The run drew all four pages twice, once in the desktop window and once in the
Electron shell, and compared them reading by reading off the drawn document.
```

pages drawn 4 - before the press, the press, the refused second press, and the control that now runs readings compared 64 in each build differences 0 the control one value changed by hand, 1 difference reported panel height 75 device pixels with no identity, 97 with one, in both

```
Four readings were left out of the comparison on purpose, because each is a fact
about the window the host gave rather than about the tab: the two window sizes,
the height the control bar was given, and where on screen the panel landed.

### Where the panel sits, and what it costs the bar

The control bar scrolled before this unit and scrolls after it. The two new
buttons and the result line are visible with no scrolling; the new panel joins
the reset readout and the world readout below the fold.
```

control bar content 320 points with no identity, 338 with one what the bar showed 136 points, in a window 1600 by 1000 visible with no scroll both new buttons, and the result line below the fold the identity panel, the reset readout, the world readout

```
The result line is what carries the press, and it carries all four identity
readings the moment the button is pressed, so the panel below is a standing
readout rather than the only place the figures appear. The Quint subtab is the
nearest subtab home, because it already lists the chain's files. It was not used:
the two controls that already write lasting state put their readouts in the
control bar, and a readout for the thing every control needs should not sit
behind a keyboard shortcut.

### Two earlier sentences stop being true on a press

Two sentences on this page said nothing here creates an identity, and both were
true when written. They stay true of a program nobody has pressed anything in.
```

what it said "Its key file is read, never created, so a machine with no identity yet names none." what replaces it The participant lookup still only reads. The identity control creates the file, and the lookup reads it after.

what it said "The one piece of the program that writes it belongs to the Competition tab, a shelved screen." what replaces it That screen still writes one and is still shelved. This tab now writes one too, on a press, and that is the path an operator can reach.

```
### What no press here creates yet

The press stops at an identity. It reaches none of the five things below, and
each is named here rather than half-built.
```

a participant nobody is entered into anything. The identity is an address and no player record is written a registration no bot is registered in a competition a distillation no Quintessence is minted. The Distil button does that, and it is a different press a world no world is declared. The world control does that an ability nothing is designed, chosen or granted

```
One figure is owed and is not invented here.
```

a second identity there is no way to retire one and start again. The control refuses a replacement and nothing else removes the file, so an operator who wants a fresh address deletes it by hand

```
---

## 2026-09-11 09:45 - #586 - the four event modes take the names of alchemical operations

**HIS.**

> "Please rename events to match Hermetic tests or similar..."

The four modes carried game-genre names. Each now carries the name of an operation
the pre-modern alchemical corpus describes, picked so the operation's own meaning
matches what that mode does. Only the label moved.
```

mode code was now monster_smash Monster Smash Fixation team_monster_smash Team Based Monster Smash Coagulation dungeon_crawl Dungeon Crawl Descension raid Raid Cementation

```
Every label is now eleven characters or fewer, which the twelve-character short-form
rule asks for. Three of the four old labels were longer than that.

### The operation behind each mode, and why it fits

Three of the four come from the eight operations of the Summa perfectionis
magisterii of pseudo-Geber, written shortly before 1310. The fourth is the parting
assay of the pre-modern metal testers.
```

Fixation a volatile body made to stay in the fire without flight. One party seat, no guild, no map, the lowest tier. A single body, alone in the fire, that holds or does not

Coagulation a fluid brought to one solid body, the coagula of solve et coagula. Two to a hundred and twenty seats, and one guild required. Separate parts set into a single mass

Descension the active part driven down to the bottom of the vessel instead of rising as vapour, worked in a pierced-base descensory. One to six seats, and the mode carries a map

Cementation a mass packed in a corrosive cement and held in fire until the base part is eaten out. The parting assay of the German assay booklets of about 1510 and of Agricola's seventh book, 1556. Two to sixty seats,

one guild, a map, the highest tier

```
  Cementation is the widest of the four readings. Its source meaning parts a mass by
  fire, and the mode applies a guild host of up to sixty to one target. Ceration was
  the alternative and it lost: it softens a hard body for admission, which matches a
  raid less well than parting a mass does.

  ### Sublimation was the first pick and the Flores tier already holds it

  Sublimation fitted the solo mode and is not available. The five loot tiers are gates
  1, 5, 8, 9 and 10 of Ripley's twelve, and the eighth gate is Sublimation, which
  named the Flores tier. Two different things would have shared one word.
```

gate operation already names 1 Calcination the Calx tier 5 Putrefaction the Cauda Pavonis tier 8 Sublimation the Flores tier 9 Fermentation the Elixir tier 10 Exaltation the Magisterium tier

```
  Conjunction and Projection were rejected on the rule this project already applied to
  the tier names: a word that carries another meaning in the tree is not taken twice.
  Conjunction means a boolean AND in the engine's own comments, and the charts surface
  already holds a price projection.
```

candidate files naming it across src, contracts and docs Fixation 1 a quoted source in a design note, no second meaning Descension 0 Coagulation 0 Cementation 0 Conjunction 5 rejected Projection 4 rejected Separation 7 rejected Multiplication 4 rejected

```
  ### The codes did not move, and a saved record is why

  A mode has a code and a label. The label is what a person reads. Every other module
  keys on the code, and a saved record stores the code. The four codes are unchanged.
```

a saved party record {'mode': 'raid', 'leader': 'a', 'members': ['a', 'b'], 'guild_key': 'g'} a saved dungeon entry carries mode_code, replayed through mode_named a renamed code on replay UnknownModeError | 'cementation' is not a PoA event mode; the four are monster_smash, team_monster_smash, dungeon_crawl, raid

```
  No saved PoA file exists yet, so renaming a code would break nothing today. It would
  refuse every saved record the first time anyone plays, and no migration exists.

  ### The screen took the new names with no edit of its own

  The tab reads the label out of the module, so no screen file changed. All eight event
  rows the tab serves carry the new label.
```

monster_smash -> Fixation | Standard | party 1 - 1 | map False monster_smash_elite -> Fixation | Elite | party 1 - 1 | map False team_monster_smash -> Coagulation | Standard | party 2 - 120 | map False team_monster_smash_elite -> Coagulation | Elite | party 2 - 120 | map False dungeon_crawl -> Descension | Standard | party 1 - 6 | map True dungeon_crawl_elite -> Descension | Elite | party 1 - 6 | map True raid -> Cementation | Standard | party 2 - 60 | map True raid_elite -> Cementation | Elite | party 2 - 60 | map True

```
The map refusal, the party refusals and the army refusal read the label too, so all
of them now name the operations.
```

Fixation carries no map, so this subtab does not open. Descension and Cementation do. Cementation admits 2 to 60 participants and this party holds 61 a Descension party links into no army; an army is built from Cementation parties

```
The map glyph table keys on the code, not the label, so each mode kept its mark. The
registry answered all sixteen marks and the distinctness check passed.
```

kind= monster_smash glyph= U+2720 label= Fixation token= STATE_ARMED kind= team_monster_smash glyph= U+2691 label= Coagulation token= WARNING kind= dungeon_crawl glyph= U+2656 label= Descension token= WARNING_STRONG kind= raid glyph= U+26E8 label= Cementation token= DANGER

```
### What the rename did not touch

Every field of all four modes was read off the module before and after the change.
One hundred and twenty-eight field readings across the eight event rows, sixteen
lines differ, and all sixteen are the label.
```

before EventMode(code='raid', label='Raid', tier=4, party_min=2, party_max=60, guild_required=True, has_map=True) after EventMode(code='raid', label='Cementation', tier=4, party_min=2, party_max=60, guild_required=True, has_map=True)

non-label fields changed 0

```
No figure changed. No party range, no guild flag, no map flag, no turn length, no
tier, and no difficulty, entry fee, loot rarity or loot drop rank.

Dated sections above this one quote refusals and keypress readings from the runs that
produced them. Those blocks keep the label those runs printed, so the record of each
run stays true, and the table at the head of this section is where the retired names
are declared.

### The name clearance on these four is owed

The clearance page checks every PoA name against similar content, and against its own
source meaning. These four labels landed after that page and have not been through it.
```

owed Fixation, Coagulation, Descension and Cementation through the similar-content search in docs/design/poa_name_clearance.md

```
## 2026-09-11 10:10 - #147 - every Vessel record keys on the Vessel id

A player owns several Vessels. Each record the package files must name one of
them. One Vessel now holds one key, and `Vessel.record_key` is that key. It
carries the owner, the class and the `vessel_id`. The level is not in it.

Two modules each built their own key before. Both read `record_key` now, so a
store and an alignment cannot disagree about which Vessel a record belongs to.
```

vessels record_key returns the owner, the class_name and the vessel_id alignment vessel_key returns record_key unchanged inventory vessel_key joins record_key with colons, the store_key shape crafting craft_id_for joins record_key between the recipe and the turn

```
### Two Vessels of one class under one owner read two records

A run built two Vessels with one owner, one class and one level. It scored one
action against each.

before
```

vessel_key(first) ('addr_owner_one', 'Lead Ward') vessel_key(second) ('addr_owner_one', 'Lead Ward') vessel_count 1 actions_of(first) ['forge a blade', 'raze a village'] actions_of(second) ['forge a blade', 'raze a village']

```
after
```

vessel_key(first) ('addr_owner_one', 'Lead Ward', 'fb244aa7d4...') vessel_key(second) ('addr_owner_one', 'Lead Ward', 'b7d1569db1...') vessel_count 2 actions_of(first) ['forge a blade'] actions_of(second) ['raze a village']

```
The owner stays in the key, so the roll-up still reads it. The same run read 2
scored actions at the address, 2 at the guild and 2 at the world, on both sides.

### A level gained keeps the store

A run opened a store for a Vessel at level 7 and put one armour in a gear slot.
It then raised the same Vessel to level 8 and asked the book for the store.

before
```

key at level 7 addr_owner_one:Tin Bulwark:7 key at level 8 addr_owner_one:Tin Bulwark:8 the book UnknownStoreError: 'addr_owner_one:Tin Bulwark:8' names no store; the book holds addr_owner_one:Tin Bulwark:7

```
after
```

key at level 7 addr_owner_one:Tin Bulwark:15ac17bff769edc2... key at level 8 addr_owner_one:Tin Bulwark:15ac17bff769edc2... the book the same store, holding armour

```
### Two Vessels of one class open two crafts

The same run gave one recipe and one turn to two Vessels of one class at one
level under one owner.

before
```

craft_id_for(first) armour:Calx:addr_owner_one:Tin Bulwark:7:0 craft_id_for(second) armour:Calx:addr_owner_one:Tin Bulwark:7:0 the second craft HeldCraftError: already open

```
after
```

craft_id_for(first) armour:Calx:addr_owner_one:Tin Bulwark:15ac17bff7...:0 craft_id_for(second) armour:Calx:addr_owner_one:Tin Bulwark:fba9b6fe10...:0 the second craft open

```
### The three modules that read the key
```

domination a health of zero on one Vessel read incapacitated on its twin before. The twin reads able now, and the marked Vessel still reads incapacitated prayer two Vessels of one class in one congregation were refused as one Vessel before. The prayer files at a congregation of 2 now, and the same Vessel named twice is still refused consecration a place records the priest's key. The record carries three parts now, and the class stays the second part the log reads

```
### A theft still moves the key

`steal_vessel` copies the `vessel_id` onto the moved Vessel and writes a new
owner. The owner is in the key, so the key still changes. `carry_to` moves the
incapacitation onto the new key, and the scored actions stay under the
victim's key. No other Vessel of the victim's ever held that key now.

### What the Vessel key leaves owed
```

owed the scored actions a stolen Vessel earned. A key of the vessel_id alone would carry them, and it would leave alignment_of_address and unguilded_addresses no owner to read owed a screen that shows a key. No subtab draws one owed open_store still takes its key from a caller, so a caller that builds its own key can still name one store twice ## 2026-09-11 10:25 - #147 - the package binds its last module, and the item notes say what exists

The PoA package reads its own folder at import. It compares the files it holds against the names it binds, and it logs the difference. That log line is the package's own statement about itself.

The package held 44 module files and bound 43. One file was reachable only by its own full path.

```
before   acervator.competition WARNING
         competition package holds 1 module(s) it does not bind: inventory

after    acervator.competition DEBUG
         competition package binds every module it holds
```

`report_unbound_modules` returned `['inventory']` before the change and `[]` after it. The entry file now imports the module. It keeps every existing import line and every existing name.

```
names in the export list before   901
names added                        36
names in the export list after    937

public names inventory declares    39
names held back                     3
```

### Three names stay unexported

A package name carries one value. Three of inventory's names repeat a name the entry file already binds, and each repeat carries a different value. The entry file skips all three, so no value that already works is replaced. A reader reaches them by naming the module.

```
ABSENT_MECHANISMS        vessels holds this name
ABSENT_MECHANISM_NOTES   vessels holds this name
vessel_key               alignment holds this name
```

The import goes after the dungeon entry, at the end of the import section. Every module inventory reads is bound above it: the conversion rates, the items, the materials, the Quintessence ledger and the vessels.

### Four modules said the package did not bind them

Four modules each carry a sentence about their own export surface. Three of those sentences were false before this unit, and the fourth became false when the package bound inventory. Each sentence is now restated. None is removed.

```
dungeon_entry   EXPORT_OWED              the entry file already imported it
domination      PACKAGE_EXPORT_OWED      the entry file already imported it
crafting        the package export note  the entry file already imported it
inventory       the package export note  the entry file imports it in this unit
```

Each restated sentence names what the package binds and what stays out of reach. A name a module repeats from a module bound above it is still reached by importing that module directly.

### The Gear subtab prints eight item notes, and five were false

The Gear subtab reads its mechanism rows from the items module. Each row is a claim about what the tree builds. All eight were driven against the real modules, one at a time. Five were false.

```
a crafting recipe   FALSE   recipe_for returned a Recipe: loss_share 0.25,
                            turns_required 3. A Recipe with no loss_share refused
salvage             holds   no module destroys an item for its Quintessence
equipping           FALSE   put_in held armour in slot 1 of 4
inventory           FALSE   put_in held 7 units of lead ore, units_held 7
slot count          holds   a store built with no slot figure refused
encumbrance         FALSE   weight_carried returned 7 on that store
loot generation     FALSE   a drop carried item_type armour, quality Calx,
                            storage_class gear
named instances     holds   an item type carries a name, a storage class, a
                            component list and a note, and no instance name
```

The tuple keeps all eight entries and all eight keys. An entry whose mechanism now exists keeps its place, and its note says what builds it and what is still missing. The tuple's own description now says that.

### What the screen shows

The subtab payload was read off the bridge method `proof_of_accumulation_tab.state`, the same request the screen sends. The corrected sentences reached the payload with no change to the panel that prints them, because the panel reads the module.

```
mechanism rows in the payload   8
rows whose text this unit changed   5
```

The heading above those rows read "Nothing supplies these". Five rows now name something that is supplied, so the heading was false. It now reads "What supplies these, and what does not".

### What is still absent

Binding a module does not build a mechanism. These gaps are unchanged.

```
no figure sets a Vessel's gear slot count
no figure sets the stack ceiling
nothing turns a carried weight into a refusal or a turn point penalty
nothing reads an item's cohesion against a Vessel's Quintessence, so the
    equip check has no caller
nothing destroys an item for the Quintessence it embeds
no table names a recipe, and both recipe figures arrive from the caller
no source names the item type a loot tier yields
no artefact carries an instance name
no panel calls the store
```

## 2026-09-11 11:05 - #585 - a finished craft puts its item in the crafting Vessel's store

### The store takes the item before the Quintessence moves

The craft register made an item and the store held nothing of it. The register returned a `CraftResult` naming the item, and no module read that name. The two modules are now wired. `CraftRegister` is built with a `StoreBook` as well as a ledger, and `complete_craft` puts the made item in the store the book holds for the crafting Vessel.

The store key is never built by hand. `inventory.vessel_key` renders it from the Vessel's own `record_key`, so the craft and the store name the same Vessel.

`src/competition/crafting.py` — the store takes the item, then the ledger moves

```
            recipe = craft.recipe
            store = self._store_for(craft.vessel)
            stored = self._store_item(craft, store)
            try:
                moved = self._ledger.embed_from_wallet(
                    craft.crafter,
                    recipe.quintessence_from_wallet,
                    recipe.quintessence_embedded,
                )
            except Exception:
                store.take_out(
                    recipe.item_type_name,
                    recipe.produced_quality,
                    UNITS_PER_CRAFT,
                )
```

`VesselStore.put_in` reads the storage class off the item type itself. Armour, weapons and accessories are gear and reach a counted slot. A consumable reaches a stack. The craft sets no class of its own.

One craft was driven end to end. The store, the wallet and the four buckets were read off the real objects on each side of the one `complete_craft` call. The recipe figures and both store figures are driving inputs of that run. No module holds either of them.

```
driving inputs   loss_share 0.25, turns_required 3, slot_count 2,
                 stack_ceiling 4, fee_usd 1000, trade_grade 1

before   held_gear []            held_stacks []       slots_used 0 of 2
         units_held armour at Calx 0
         wallet 1000             embedded 0           delta 0

after    held_gear [armour, Calx, storage_class gear, weight 1]
         held_stacks []          slots_used 1 of 2
         units_held armour at Calx 1
         wallet 999.99999999     embedded 0.00000001  delta 0
         is_stored True          is_accounted True
```

The item in the slot carries the grade the craft made it at. The Quintessence that left the wallet equals what the embedded bucket gained, and conservation balances on both sides.

### A full store refuses, and the craft waits

A store can refuse. Its slots fill and its stack ceiling is reached. A refused store raises before `embed_from_wallet` runs, so the craft stays in `open_crafts`, the wallet keeps what the craft owes, and the store holds what it held. Nothing is made and nothing is lost.

A store was opened with no gear slot at all. The craft opened, then the completion was refused.

```
driving inputs   slot_count 0, stack_ceiling 1

CraftDeliveryError, raised from SlotsFullError
  ... holds 0 of 0 gear slots and 1 armour takes 1 more; no gear is held in a
  slot this Vessel does not have

after    held_gear []            slots_used 0 of 0
         wallet 1000             embedded 0           delta 0
         ledger movements 1      open_crafts 1        completed_crafts 0
```

The one ledger movement is the distil that funded the wallet. The craft made no movement of its own.

A second store was opened with a stack ceiling of one unit and one consumable already on the stack. The same refusal held for the stacking half.

```
CraftDeliveryError, raised from StackCeilingError
  ... holds 1 consumables at Calx and 1 more reaches 2, above the 1 stack
  ceiling it was built with

after    held_stacks [consumables, Calx, 1 unit]
         wallet 1000             embedded 0           delta 0
         open_crafts 1           completed_crafts 0
```

A book that holds no store for the Vessel refuses earlier still. `begin_craft` asks the book for the store and raises, so no craft opens that would have nowhere to put its item.

```
CraftDeliveryError at begin_craft
  ... names no store in the book this register holds, so a finished craft would
  have nowhere to put its item

after    open_crafts 0           wallet 1000          delta 0
```

Each refused craft completed later. The store was given two slots, the same craft id opened again, and the item reached the slot.

### A drained wallet sends the item back out of the store

The store takes the item first, so a ledger that refuses leaves an item in a slot. The register takes that item back out before it raises.

A craft opened while the wallet covered it. The wallet was then emptied into another address. The completion was driven on the empty wallet.

```
the craft takes out of the wallet   0.00000001
drained out of the wallet           1000.00000000

ValueError from the ledger
  u89-reincarnate-undo holds 0E-8, cannot move 1E-8

after    held_gear []            slots_used 0        weight_carried 0
         units_held armour 0
         wallet 0                embedded 0          delta 0
         ledger movements 2      open_crafts 1       completed_crafts 0
```

The two ledger movements are the distil and the drain. The craft moved nothing. The store holds nothing. The craft still stands open, and it completed once 0.00000001 was paid back into the wallet.

### Two Vessels of one owner, two stores, two items

One Reincarnate holds many Vessels. Two Vessels of one class at one level under one owner carry two Vessel ids, so `vessel_key` renders two store keys and the book holds two stores. Each craft reaches its own crafter's store.

Two Vessels of one owner opened one craft each on the same turn, one for armour and one for consumables, and both completed.

```
vessel a key   u89-reincarnate:Lead Ward:3dc1540b...
vessel b key   u89-reincarnate:Lead Ward:5dc6cb9e...
keys differ    True

store a   held_gear [armour, Calx]      armour 1   consumables 0
store b   held_stacks [consumables, Calx, 1 unit]  armour 0   consumables 1

wallet 999.99999998   embedded 0.00000002   delta 0
```

The gear item reached the slot half of one store. The consumable reached the stack half of the other. Neither item reached the wrong store.

### What a craft still does not take out of a store

A craft puts in. It takes nothing out.

```
a Recipe names component units and nothing deducts them from a store
no figure sets a Vessel's gear slot count
no figure sets the stack ceiling
no table names a recipe, so a loss share and a turn count arrive from the caller
nothing tells a player that a craft finished
nothing hands a Vessel a craft to carry
```

No control on screen reaches this. The PoA tab surface names no store and no craft register. The Gear subtab, `Ctrl+3`, is the surface that would: it already prints the four item classes and reads the note saying what holds them.

## 2026-09-11 11:40 - #585 - a craft consumes the material list it names

### The components leave when the craft opens

A recipe named its materials and took none of them. `Recipe` carried a list of `Component` entries, and no store ever lost a unit to a craft. The two are now wired. `CraftRegister.begin_craft` takes every component's units out of the store the book holds for the crafting Vessel, at the grade that component names.

The units leave at the start, not at the finish. A craft runs over the world turns its recipe names, and a Vessel cannot carry material it already works. A store that still held the ore could hand it to a second store, or open a second craft on the same unit, so the deduction cannot wait for `complete_craft`.

`src/competition/crafting.py` — `begin_craft` reads every shortfall before it moves a unit

```
        recipe = craft.recipe
        short = self._component_shortfalls(recipe, store)
        if short:
            raise CraftComponentError(
                f"craft {craft.craft_id} consumes "
                f"{_component_text(recipe.components)} and {store.store_key} is "
                f"short of {_shortfall_text(short)}. {COMPONENT_REFUSAL_HOLDS}",
            )
        taken: list[StoreChange] = []
        for part in recipe.components:
            try:
                taken.append(store.take_out(part.material, part.quality, part.units))
            except InventoryError as exc:
                self._put_components_back(
                    store,
                    recipe.components[: len(taken)],
                    COMPONENT_REFUSAL_HOLDS,
                )
```

One craft was driven end to end on two trees, the tree before this change and the tree after it, by one script with one set of figures. The recipe figures and both store figures are driving inputs of those runs. No module holds any of them.

```
driving inputs   loss_share 0.25, turns_required 3, slot_count 2,
                 stack_ceiling 4, fee_usd 1000, trade_grade 1,
                 armour at Calx out of 1 iron ore at Calx

before the change, after the craft finished
    units_held iron ore at Calx 1     units_held armour at Calx 1
    weight_carried 2
    wallet 999.99999999   embedded 0.00000001   delta 0

after the change, after begin_craft
    units_held iron ore at Calx 0     units_held armour at Calx 0
    weight_carried 0
    wallet 1000.00000000  embedded 0             delta 0

after the change, after complete_craft
    units_held iron ore at Calx 0     units_held armour at Calx 1
    weight_carried 1
    wallet 999.99999999   embedded 0.00000001   delta 0
```

The ore came through a whole craft untouched on the earlier tree. It does not now. The Quintessence reading is the same on both trees, because a material unit in a store carries no ledger entry of its own.

### A store short of one unit refuses and moves nothing

`begin_craft` reads what the store holds of every component before it takes any of them. A store that cannot cover the whole material list gives out nothing.

A store was opened and given no component unit at all.

```
CraftComponentError at begin_craft
  craft armour:Calx:... consumes 1 iron ore at Calx and ... is short of 1 iron
  ore at Calx, holding 0

after    held_stacks []         weight_carried 0
         wallet 1000.00000000   embedded 0        delta 0
         ledger movements 1     open_crafts 0     completed_crafts 0
```

The one ledger movement is the distil that funded the wallet. The craft made no movement of its own.

A second store was driven against a recipe naming two materials. It held the first material in full and was two units short on the second.

```
driving inputs   a recipe of 2 iron ore and 3 lead ore, both at Calx
                 the store holds 2 iron ore and 1 lead ore

CraftComponentError at begin_craft
  ... is short of 3 lead ore at Calx, holding 1

after    units_held iron ore at Calx 2   units_held lead ore at Calx 1
         weight_carried 3
         wallet 1000.00000000   embedded 0   delta 0
         open_crafts 0          completed_crafts 0
```

The first material was not touched. A refusal part-way down a list returns every unit already taken, so no half-consumed list reaches the store.

### An abandoned craft gives the units back

An abandoned craft made no item, and the ledger moved nothing for it. The components go back where `begin_craft` found them. A unit kept back would leave the world holding less material with no bucket and no wallet recording the difference. No source sets a part-worked loss, so the whole list returns.

```
after begin_craft     units_held iron ore at Calx 0   weight_carried 0
after abandon_craft   units_held iron ore at Calx 1   weight_carried 1
                      wallet 1000.00000000   embedded 0   delta 0
                      ledger movements 1     open_crafts 0
```

### A store with no room leaves the craft holding its components

A store can refuse the units back. Another put-in can fill its stack to the ceiling while the craft runs. `abandon_craft` puts the units back before it takes the craft out of `open_crafts`, so a refusal leaves the craft standing and holding its components. No unit leaves the world, and the same call carries them back once the stack has room.

A store with a four unit stack ceiling was filled to that ceiling while its craft ran. The abandonment was then driven.

```
CraftComponentError at abandon_craft, raised from StackCeilingError
  ... would not take back 1 iron ore at Calx: ... holds 4 iron ore at Calx and
  1 more reaches 5, above the 4 stack ceiling it was built with

after the refusal   units_held iron ore at Calx 4   weight_carried 4
                    open_crafts 1                   completed_crafts 0
                    wallet 1000.00000000   embedded 0   delta 0

after one unit was taken off the stack by hand
                    units_held iron ore at Calx 3   weight_carried 3

after the same abandon_craft was driven again
                    units_held iron ore at Calx 4   weight_carried 4
                    open_crafts 0
                    wallet 1000.00000000   embedded 0   delta 0
```

### A refused completion keeps the components with the craft

The store or the wallet can refuse a completion. The components left at `begin_craft`, so a refused completion consumes nothing more and the craft stays open holding them.

A craft was opened while the wallet covered it. The wallet was then emptied. The completion was driven on the empty wallet, and the craft was completed once the wallet was funded again. A second completion of the same craft was then driven.

```
ValueError from the ledger
  u91-reincarnate-drain holds 0E-8, cannot move 1E-8

after the ledger refusal   units_held iron ore 0   units_held armour 0
                           wallet 0   embedded 0   delta 0
                           open_crafts 1           completed_crafts 0

after the wallet was funded and the craft completed
                           units_held armour at Calx 1   weight_carried 1
                           open_crafts 0                 completed_crafts 1

UnknownCraftError from the second completion
  ... has already completed

after the second refusal   units_held iron ore 0   units_held armour at Calx 1
                           weight_carried 1        embedded 0.00000001
                           delta 0
```

One craft consumed one material list once. A refusal consumed none.

### The Gear subtab says what a craft now does to a store

The Gear subtab, `Ctrl+3`, prints a note for each mechanism an item takes part in. Its inventory note said that a hand-over is the only move between two stores. That stays true, and it no longer covers every way a store changes: a craft is the other way. The note now says both halves, and the two readings below come off the screen route, not off the module.

```
before, as the subtab printed it
  Inventory.VesselStore holds them, gear in counted slots and consumables and
  resources in stacks, and StoreBook.hand_over is the only move between two
  stores, so this subtab lists the loot the wallet holds and no item a Vessel
  wears.

after, as the subtab prints it
  Inventory.VesselStore holds them, gear in counted slots and consumables and
  resources in stacks, StoreBook.hand_over is still the only move between two
  stores, and a craft is the other way a store changes:
  crafting.CraftRegister.complete_craft puts a made item in the crafting
  Vessel's store and begin_craft takes that recipe's component units out of it;
  no panel calls either, so this subtab lists the loot the wallet holds and no
  item a Vessel wears.
```

The subtab prints the same eight mechanism names on both trees. No name left the list to make room.

### What a craft's material list still stands on

A craft takes its materials out and puts its item in. Nothing fills the store in the first place.

```
no module gathers a material unit, so every unit a craft consumes was put in by
  a caller's own put_in
no module counts how many units of a material a world holds
no bucket records the Quintessence a material unit in a store carries, so
  consuming a component moves nothing in the ledger
material_surplus reads 0 for all four item types at all five grades, because each
  declared list is one unit of iron ore and one component of Quintessence holds
  one Quintessence of an item together
no figure sets a Vessel's gear slot count
no figure sets the stack ceiling
no table names a recipe, so a loss share and a turn count arrive from the caller
no source sets what an abandoned craft loses, so it loses nothing
```

No control on screen reaches this. The PoA tab surface names no store and no craft register, and the one thing it reads of this work is the prose above. The Gear subtab is the surface that would call a craft: it already prints the four item classes and the note that now names both halves of what a craft does. ## 2026-09-11 11:45 - #586 - the player window reads what is left of the turn

The player window reported the turn that was gone. It now reports the turn that is left: the seconds left in the candle, and the actions left in the pool.

The reading below is pinned twelve seconds before a 5m candle closes, and every figure in it was read off the page the tab draws.

```
                   was                              now
turn meter label   Turn completion                  Turn remaining
turn meter         96.0%  288s of 300s elapsed      4.0%  12s of 300s left
Impetus line       Impetus 4 this turn - level 1    Impetus 4 of 4 left this
                                                    turn - level 1
```

### The turn meter counts down, and its own text says so

The meter keeps the same arithmetic over the same two figures. It draws the seconds left against the candle's length, where it drew the seconds gone.

```
def turn_meter(running: dict, variant: EventVariant) -> dict:
    """The share of this turn's candle that is left, from its own seconds left."""
    length = Decimal(variant.turn_seconds)
    left = Decimal(str(running["seconds_left"])).quantize(Decimal(1))
```

The row name moved with the label. Nothing reads the old name: the style sheet colours the fill meter's bar by `data-meter="block_fill"`, and it names no other meter.

```
meter row name    was turn_completion    now turn_remaining
meter pair title  was Block fill and turn completion
                  now Block fill and turn remaining
```

### The Impetus line names what is unspent

The line printed the grant. It prints what is unspent against the grant, so the line that reports the pool falls when the pool falls.

```
Impetus 3 of 4 left this turn - level 1
```

### The window holds a pool against the turn that granted it

The window built a fresh pool on every redraw. A fresh pool has spent nothing, so a spend could not survive a redraw and the window could never fall. The surface holds the pool now, keyed on the participant and the event code, and a turn index the held pool was not granted for takes a new pool.

```
HELD_POOLS keyed     [('', 'monster_smash')]
spend(1) returns     3
the payload then     granted 4   spent 1   remaining 3
the line then        Impetus 3 of 4 left this turn - level 1
one turn later       granted 4   spent 0   remaining 4
```

The pool is not chain data, so the key carries no chain name. A live run and a demo run share the one pool a participant holds for one event.

Both refusals stand unchanged. A spend against a turn the pool was not granted for raises, and a cost above what remains raises.

```
this pool granted 4 Impetus for turn 5890752 and 3 is unspent; turn 5890753 is a
different turn, and Impetus expires with the candle that granted it

this action costs 5 Impetus and 4 remains in turn 5890752; no partial action
exists and nobody borrows against the next turn
```

### The payload carries the pool's three figures

The grant, the spend and what is left all ride in the event payload, so nothing on screen recomputes a figure the pool already answers.

```
impetus_granted     4
impetus_spent       1
impetus_remaining   3
```

### The grant and what is left read the same until something spends

No control on this tab spends Impetus. `spent` is nought on every draw the screen makes on its own, and what is left equals the grant. The reading above spends through the held pool, and the window's figures then fall.

Three places still read the grant where the design asks for another figure.

```
the party slot column    prints each participant's grant, and nothing holds a
                         pool for another participant
the gear section         takes the grant as one action's Impetus cost
the wallet section       takes the grant as one action's Impetus cost
```

No figure sets what one action costs in Impetus. The turn economy owes that figure, and the two sections stand in with the grant until it exists.

---

## 2026-09-11 12:10 - #585 - a Vessel on a square gathers what the square holds

A Vessel stands on a square. A foraging run takes the material that square holds and puts it in that Vessel's store. Before this, nothing gathered a material. A craft and a hand-over were the only things that changed a store, and every material unit in a store was put there by a caller.

The run lives in `src/competition/foraging.py`. The package binds it, so every name reads as `src.competition.<name>`.

### A square holds one material, and the same square always holds it

The square decides. It decides which of the seven materials it holds, and which of the five quality grades that material is at. Both come out of the square's own discovery leaf, which is the hash of the world seed and the square's locator.

A creature already comes off the same leaf. `monster_spawn` draws a tier from one window of the leaf and a design from the next. A square's material is two more windows of that same leaf, so the material, the grade, the creature and the committed Quintessence are four draws off one hash. No new seed and no new hash is added.

Diagram: flowchart, left to right

- world seed -> **discovery leaf of the square**
- square locator -> **discovery leaf of the square**
- discovery leaf of the square -> **digits 0 to 16: committed Quintessence**
- discovery leaf of the square -> **digits 16 to 32: monster tier**
- discovery leaf of the square -> **digits 32 to 48: monster design**
- discovery leaf of the square -> **digits 48 to 56: the material**
- discovery leaf of the square -> **digits 56 to 64: the grade**
- digits 48 to 56: the material -> **the Vessel store**
- digits 56 to 64: the grade -> **the Vessel store**

The locator names the square's origin step, never the step the Vessel stands on. A Vessel anywhere inside a square forages the same material. The step it stands on is read once, to name the square.

The module refuses at import if a foraging window reads any digit another draw already reads. That keeps a square's material independent of its creature.

### The two windows the draw reads, and what already reads the leaf

A leaf is 64 hex digits. The other draws read 16 digits each. A foraging draw reads 8, so the two draws fit in the last 16 digits that nothing else reads.

```
leaf digits                 64
draws elsewhere             digits 0 to 16, 16 to 32, 32 to 48
the material draw           digits 48 to 56
the grade draw              digits 56 to 64
```

The spread was measured over 10,000 squares of one layer under one seed.

```
materials declared / drawn    7 / 7
grades declared / drawn       5 / 5
material and grade pairs      35 of 35 possible
even share a material         1428.57     lowest 1394    highest 1466
even share a grade            2000        lowest 1957    highest 2065
even share a pair             285.71      lowest 251     highest 323
```

The same 10,000 squares were then derived under a second seed and compared.

```
same material under both seeds    1361    an even draw gives 1428.57
same grade under both seeds       1975    an even draw gives 2000
same material and grade           283     an even draw gives 285.71
```

The seed reaches the draw. One square was watched where both seeds named silver ore at Cauda Pavonis. Its two leaves differ and its two window values differ, 779,275,405 against 2,597,397,249. Both land on the same material because both divide by seven materials and leave six. At 64 squares that looked like a fault. At 10,000 squares the rate is 283 against 285.71, so one coincidence caused it and not a correlation.

### What one run puts in the store

One run was driven on a world whose seed is published, on square 21 of layer 3.

```
before the run
    units_held silver ore at Cauda Pavonis   0
    weight_carried                           0

after the run
    units_held silver ore at Cauda Pavonis   1
    weight_carried                           1
    units_gathered                           1
    Quintessence the unit embeds             0.00000002
```

The square named silver ore at Cauda Pavonis off material roll 6 and grade roll 1. A second run on the same square named the same material at the same grade and raised the stack to two. A run on square 22 named gold ore at Flores.

The run moves no Quintessence. The amount one gathered unit embeds is read off the materials table and reported, and no bucket records it. A craft already says the same about the units it consumes.

### What the yield figure stands on

Nobody has set how many units one run yields. The figure sits in the conversion rate table, marked as a working figure, beside the other figures the operator owes.

```
rate name       material_units_per_forage_run
per unit        one foraging run on one square
yields          material units
figure          1
provenance      working
```

No number is written into the foraging module. The module reads the rate and refuses a figure that is absent, a fraction of a unit, or under one unit.

**The operator owes this figure.** A yield of one unit a run is a placeholder and nothing more.

### Every way a run refuses, and what the store holds after

Each refusal was driven. In every case the store held exactly what it held before the call, and the register recorded no run.

```
the Vessel is on another square
  VesselNotOnSquareError - Lead Ward of ... stands on square 21 of layer 3 and
  square 22 was named; a Vessel forages the square it stands on
  store units before / after   2 / 2

the store cannot take what is gathered
  ForageStoreError - ... cannot take the 1 quicksilver ore at Cauda Pavonis that
  square 7 of layer 3 yielded ..., so the store holds what it held
  store units before / after   1 / 1

the world has not published its seed
  SeedNotPublishedError - world ... holds its seed under commitment
  d6b2c308a93f08a2 and has not published it, so Lead Ward of ... reads no leaf at
  square 21 of layer 3
  store units before / after   2 / 2

the seed offered is not the published one
  SeedCommitmentError - the seed offered rebuilds commitment d6b2c308a93f08a2 and
  world u94-world published 75b8bddf70267c93
  store units before / after   2 / 2

the square is off the layer
  ForageSquareError - ... stands on no square of world u94-world: square 64 is
  off a grid holding 64

nobody has breached the layer
  ForageWorldError - ... cannot forage square 21 of layer 5: layer 5 of world
  u94-world does not exist; layers breached: 3

the book holds no store for this Vessel
  ForageStoreError - ... names no store in the book this register holds, so a run
  would have nowhere to put what it gathered

the yield figure is not whole units
  ForageYieldError - ... of 0.5 is not a whole number of units, and a store holds
  no part unit
```

A control ran after every refusal. The same call with every argument right gathered one lead ore at Cauda Pavonis on square 9, so the refusals above are refusals and not a broken call.

### Which subtab would reach a run, and which does

The Resources subtab, which opens on Ctrl+4, is the subtab a run belongs on. It already prints every material and the Quintessence one unit embeds. It also prints that nothing holds how many units a participant carries.

**Nothing on screen opens a run today.** No control on any subtab calls this module. The Maps subtab draws the squares and names no material. The run was driven from the real modules and not from the screen.

### What a foraging run still does not do

```
a square that runs out   no source sets a deposit size, so the same square
                         yields the same material and the same units every time
a world supply           nothing counts how many units of a material a world
                         holds, and a run deducts from no total
a turn cost              a run spends nothing out of the world turn's pool
encumbrance              the weight is read and nothing turns it into a refusal
                         or a turn point penalty
a skill                  no skill gates a run, raises its yield, or reaches a
                         grade the square does not already hold
a surface                no control opens a run
```

The operator's rule is that encumbrance counts only while a Vessel moves items itself, such as after a foraging run. The run now exists. Nothing moves a Vessel with its load, and both figures an encumbrance rule needs are still working figures.

## 2026-09-11 12:25 - #586 - a dungeon entry refuses a party that is not all standing there

### A dungeon is a place, and the whole party has to be at it

A dungeon sits at one locator on the world map. A party walks to that locator and goes in. Every member of the party has to be standing at the place, because a dungeon is a place and not an invitation.

The entry took one position for the whole party and checked nobody. A member on the far side of the world went in with the party, and nothing refused it.

The entry now reads each member's own locator and refuses the party when any member is somewhere else. A member's locator comes from that member's own journey leg, which is the one record movement writes.

`src/competition/world_movement.py` - where one mover stands, read from its own leg

```
def mover_locator(self, world_id: str, address: str, turn: int) -> str | None:
    """The ``GridPosition.locator`` ``address`` stands at during ``turn``.

    None answers a mover whose own journeys all open after ``turn`` and one
    that has opened none, and the locator carries the covering leg's own
    ``layer``, so a position on two layers never reads as one place.
    """
```

`src/competition/dungeon_entry.py` - the one reader that holds a party's places together

```
def party_locators(
    self,
    journeys: WorldJourneys,
    world_id: str,
    members: Sequence[object],
    world_turn: int,
) -> dict[str, str | None]:
```

### What the program printed when one member stood elsewhere

Three participants walked on a real eight-square grid. The dungeon is a crypt at square 9, half a square in each direction. Two of them walked to it. The third walked to square 0 instead.

What movement derived for the three, at the world turn the entry names:

```
0xLeader      square 9  step (50, 50)   arrived
0xStander     square 9  step (50, 50)   arrived
0xWanderer    square 0  step (90, 10)   arrived
```

The same three read as locators, which is the form the entry compares:

```
dungeon locator    0:9:50:50
0xLeader           0:9:50:50
0xStander          0:9:50:50
0xWanderer         0:0:90:10
```

Before this change the entry opened and held all three. The register printed:

```
enter RETURNED entry_id=u93world:0:9:50:50:0xLeader:w2:e5866666
entry members = ('0xLeader', '0xStander', '0xWanderer')
clock_row 0xWanderer  clock=event turn  locator=0:9:50:50
```

The wanderer was on square 0 and the register put it on the event clock inside the crypt. After this change the same call refuses:

```
0xLeader leads this Descension into 0:9:50:50 and 1 of 3 members stand away
from it: 0xWanderer at 0:0:90:10; a party walks into a dungeon from the
dungeon's own position
```

### The same square on another layer is a different place

A locator carries the layer, the square and the two steps. Two participants can hold the same square and the same steps on two different layers, and they are not in the same place.

A fourth participant walked to square 9 step (50, 50) on layer 1, while the crypt sits on layer 0. The entry refuses it:

```
0xLeader       at 0:9:50:50
0xOtherLayer   at 1:9:50:50

0xLeader leads this Descension into 0:9:50:50 and 1 of 2 members stand away
from it: 0xOtherLayer at 1:9:50:50; a party walks into a dungeon from the
dungeon's own position
```

Comparing the square and the steps alone would have let that member in. The comparison is on the whole locator for that reason.

### A member that has walked nowhere gets no entry

Movement writes one record a leg. A participant that has opened no leg has no leg to derive a position from, so it has no locator at all. That is an absence and not a place.

The entry refuses such a member. The module refuses every other absence the same way: an undiscovered locator, a zone with no terrain multiplier, a world it cannot name. Admitting this one would pass the check for exactly the members nothing in the tree can place, and a party could then walk in with a member nobody can find.

What the program printed for a participant with no leg:

```
mover_journeys(0xNewcomer) = ()
mover_locator(0xNewcomer)  = None

0xLeader leads this Descension into 0:9:50:50 and 1 of 2 members stand away
from it: 0xNewcomer at no journey leg open by this turn; a party walks into a
dungeon from the dungeon's own position
```

Every participant starts at the arena, and a player discovers a dungeon out in the world. A member that has walked nowhere has not left the arena, and nothing in the tree records a participant standing still there.

### The party standing together enters as it always did

The two participants that walked to the crypt entered with nothing changed about the entry record. The row the register serves:

```
entry_id             u93world:0:9:50:50:0xLeader:w2:e5866666
locator              0:9:50:50
kind                 crypt
variant_code         dungeon_crawl
leader               0xLeader
party_size           2
members              ['0xLeader', '0xStander']
entered_world_turn   2
entered_event_turn   5866666
turn_timeframe       5m
clock                event turn
```

### The refusals a dungeon already had still read the same

Three earlier refusals ran on the same register, after the crypt held an open entry. Each printed its own message word for word as before.

A leader already inside:

```
0xLeader is inside 0:9:50:50 on entry u93world:0:9:50:50:0xLeader:w2:e5866666
and answers to the event turn; entry u93world:0:9:50:50:0xLeader:w2:e5866666
would put one participant on two event clocks at once
```

A leader that has not discovered the place:

```
0xStander leads this Descension and holds no reference to u93world:0:9:50:50;
a dungeon is entered only where it has been discovered, and this leader knows
0 place(s)
```

A party formed in the wrong mode:

```
this party was formed for Fixation and admits 1 to 1; entering a Descension
needs a party formed in dungeon_crawl
```

### A refused entry leaves the register alone

The check runs before the entry record exists and before anything reaches the file. A refused entry opens no entry, moves no clock and writes no file.

The register before and after a refused entry, printed both times:

```
entry_ids     ()
open_entries  []
0xLeader      clock=world turn  entry_id=None
0xStander     clock=world turn  entry_id=None
0xWanderer    clock=world turn  entry_id=None
```

The register wrote its own file and a second register read that file back. The loaded register answered the same as the one in memory:

```
entry_ids equal   True
entry_rows equal  True
clock_rows equal  True
```

The loaded register refuses a scattered party as well, so the refusal is not a property of one register in memory.

### What reaches this refusal at runtime

Nothing on the Accumulation tab reaches a dungeon entry. No screen file builds a register, and the bridge that installs the world and the journeys installs no register beside them. Only an import of the package reaches the module.

The Maps subtab is where the control belongs. The operator set that himself: a click on the map during a Crawl or a Raid opens the Maps subtab, and a location carries its own options when the player moves into the right square. Entering is one of those options, and the player's own square is the square this refusal reads.

### What a dungeon entry still does not know

The entry reads where every member stands at the world turn it names, and nothing holds a member there. A member can open a leg away from the locator after the check reads it and before the party goes in.

Nothing records a participant standing still. A position exists only while a journey leg covers it, so the arena every participant starts on is not a position any module can read.

The rest of the dungeon stands where it stood. Nothing rules the inside, nothing resolves a fight, and a round still has no length.

## 2026-09-11 12:45 - #586 - a trimmed chart rolls, and keeps the odds it had

### A mature world stops dropping junk

His sentence: loot and resources roll off the chart as a world matures, so that mature players do not spend their item lists on things they cannot use.

The code already cut the chart. Nothing rolled on what the cut left. A trimmed chart existed and no drop could come out of it.

### What the roll refused, and what it answers now

A roll needs weights that add to a hundred. The five declared weights add to a hundred. A band is the chart with its commonest tiers cut off the low end, so a band adds to less than a hundred and the roll turned it away.

What the program printed when a band met the roll:

```
band Cauda Pavonis, Flores, Elixir, Magisterium
LootTableError: the 4 loot weights total 40.0, not 100; no roll span can be
cut from them
```

The weight the cut tiers held now spreads across the tiers that stay, in proportion to what each one already held. A tier that held twice another tier's weight still holds twice it. Every band adds to exactly a hundred.

`src/competition/loot_drop.py` - the rule, in the one function that states it

```
def band_weight_pct(band: object) -> tuple[tuple[LootTier, Fraction], ...]:
    """Return every tier in ``band`` beside the percent it owns of the band.

    The weight of each tier off the chart is spread across the tiers left in
    proportion to what they already held, so every surviving pair keeps the odds
    it had on the full chart and a ``Fraction`` carries the share exactly.
    """
```

The share is an exact fraction and not a decimal. Three tiers left on the chart give each one a third of a whole number, and a decimal cannot hold a third. A fraction holds it, so the total lands on a hundred and not near it.

### Every band, and the odds it keeps

Five bands exist. Nothing off the chart, one tier off, and so on to four tiers off. Cutting all five leaves nothing to roll, and the module turns that away.

What the program printed for each band. The span is the whole numbers a roll comes from, out of the thousand a full chart holds:

```
tiers off   band                                          span   total
0           Calx, Cauda Pavonis, Flores, Elixir, Magist.  1000   100/1
1           Cauda Pavonis, Flores, Elixir, Magisterium     400   100/1
2           Flores, Elixir, Magisterium                    150   100/1
3           Elixir, Magisterium                             40   100/1
4           Magisterium                                      5   100/1
```

Every weight, before the cut and after it:

```
band            tier             before   span   after exact   after pct
0 off chart     Calx                 60    600          60/1   60.000000
                Cauda Pavonis        25    250          25/1   25.000000
                Flores               11    110          11/1   11.000000
                Elixir              3.5     35           7/2    3.500000
                Magisterium         0.5      5           1/2    0.500000

1 off chart     Cauda Pavonis        25    250         125/2   62.500000
                Flores               11    110          55/2   27.500000
                Elixir              3.5     35          35/4    8.750000
                Magisterium         0.5      5           5/4    1.250000

2 off chart     Flores               11    110         220/3   73.333333
                Elixir              3.5     35          70/3   23.333333
                Magisterium         0.5      5          10/3    3.333333

3 off chart     Elixir              3.5     35         175/2   87.500000
                Magisterium         0.5      5          25/2   12.500000

4 off chart     Magisterium         0.5      5         100/1  100.000000
```

The band with two tiers off the chart is the one a decimal cannot carry. Its three weights are 220/3, 70/3 and 10/3. Each one repeats forever as a decimal, and the three add to exactly 300/3, which is a hundred.

### The odds between the tiers that stay

Each pair of tiers holds one ratio on the full chart. The same pair holds the same ratio on every band that keeps both. The program printed each pair twice, once off the full chart and once off the band:

```
band 1 off chart
  Cauda Pavonis : Flores         full     25/11   band     25/11   same True
  Cauda Pavonis : Elixir         full      50/7   band      50/7   same True
  Cauda Pavonis : Magisterium    full      50/1   band      50/1   same True
  Flores        : Elixir         full      22/7   band      22/7   same True
  Flores        : Magisterium    full      22/1   band      22/1   same True
  Elixir        : Magisterium    full       7/1   band       7/1   same True

band 3 off chart
  Elixir        : Magisterium    full       7/1   band       7/1   same True
```

Elixir stays seven times as likely as Magisterium on every band that holds both. Elixir holds seven times on the full chart, seven times with one tier cut, and seven times with three cut.

### A real drop on a trimmed band

The program drew four thousand drops on each trimmed band, through the same rotation and the same drop call the rest of the package uses. What came out:

```
band Flores, Elixir, Magisterium

tier              drawn    drawn pct     band pct
Calx                  0     0.000000    off chart
Cauda Pavonis         0     0.000000    off chart
Flores             2907    72.675000    73.333333
Elixir              950    23.750000    23.333333
Magisterium         143     3.575000     3.333333
```

Nothing landed on a tier off the chart, on any band. The rarest tier left on the chart still came out: Magisterium reached 143 drops of four thousand on this band, 56 on the band with one tier cut, and 495 on the band with three cut.

Four thousand draws a band shows that the odds sit near the weights. It does not prove the exact share, and it cannot show a tier rarer than about one draw in four thousand.

The same four thousand draws on the full chart do reach the cut tiers:

```
Calx           drawn 2358 times on the full chart
Cauda Pavonis  drawn 1045 times on the full chart
Flores          428
Elixir          149
Magisterium      20
```

A count of zero on a trimmed band therefore means the band works. It does not mean the counter missed a drop.

One drop on a trimmed band went into a real Vessel store, and onto disk:

```
band             = Elixir, Magisterium
drop             = roll 14, Elixir, armour
delivered to     = u96vessel
held_tiers       = ['Elixir']
```

### The full chart did not move

One script drove the full chart's roll and its refusal before this change and after it. The two runs printed the same thing to the character. That includes the name the drop hashes itself under:

```
roll          = 370
tier_name     = Calx
item_id       = 3a9fe5488c13c80e767e61dae6ff5a37760494547fd85727974958df4d005b20
```

A hand-written set of five weights that does not add to a hundred still gets turned away, and the refusal still names both numbers:

```
dropping Calx         the 4 loot weights total 40.0, not 100
dropping Magisterium  the 4 loot weights total 99.5, not 100
```

Dropping Magisterium takes only half a point off the total, and the roll turns that set away too. The check reads the total, and not how many tiers there are.

### What a band means for material grades

The material quality grades are the same five names, read from the same place as the loot tiers. They are one list and not two.

A band cuts both. A world that drops no Calx loot also yields no Calx grade of any material.

A grade that left the chart is still a grade. The grade table still answers for Calx, because items already held carry their own grade. A band decides what a world drops, and not what a grade is.

### What still has no figure

How many tiers leave the chart for each world tier. Nobody has set that number. The module turns away a band read from a world tier until somebody does:

```
world_tier.loot_band_at_tier(2)
FigureAbsentError: no band follows from a world tier: how many loot tiers leave
the chart for each world tier above FIRST_WORLD_TIER. No statement names one.
```

Nothing raises a world's tier on its own either. No module names a cataclysm, so a world stays at the tier it opened on, and every world rolls the full chart today.

A band rolls, and nothing reaches it. A caller that hands a band to the drop call gets a correct roll. No control and no subtab hands it one.

## 2026-09-11 12:50 - #586 - a participant who has not moved stands at the arena

Every participant is somewhere. A participant who has never walked is standing at the arena. Before this, nothing could say that. Movement wrote one record a leg, and a participant with no leg had no place at all.

Two units reached the same absence from two sides. One could not place a party member who had walked nowhere. One could not forage with a Vessel whose owner had walked nowhere. The world already declared its arena square. Nothing read it as a position.

### The arena is where a participant stands before it walks

`PoaWorld.create_world` fixes the arena square at the middle of the grid and puts it on the chain. The world has always known it. Now a reader answers it.

`src/competition/world_grid.py` - the world answers its own arena as a place

```
def arena_position(self, world_id: str) -> GridPosition:
    """Where a participant of ``world_id`` stands before it walks anywhere."""

def arena_locator(self, world_id: str) -> str:
    """``arena_position`` as the locator text on ``ARENA_LAYER``."""
```

ARENA_LAYER is 0, the lowest layer ordinal `breach_layer` admits. The arena position sits on the square's origin step, so its locator is `0:40:0:0` on a grid 9 squares across.

`src/competition/world_movement.py` - one reader answers where a mover stands

```
def mover_standing(self, world_id: str, address: str, turn: int) -> MoverStanding:
    """Where ``address`` stands in ``world_id`` during ``turn``, and what said so."""

def mover_locator(self, world_id: str, address: str, turn: int) -> str:
    """The ``GridPosition.locator`` ``address`` stands at during ``turn``."""
```

`mover_locator` reads `mover_standing`, so one rule decides a place and the locator text cannot disagree with it.

A `MoverStanding` carries its own source. A standing has one of two sources, and no third source exists.

```
journey leg    the mover's own latest covering leg derived the place
arena          the mover held no covering leg, so the world answered its arena
```

Diagram: flowchart, top to bottom

- a consumer asks where an address stands -> **does a covering leg exist?**
- legs -- yes -> **progress_at derives the square and the two steps**
- legs -- no -> **PoaWorld.arena_position**
- progress_at derives the square and the two steps -> **MoverStanding source = journey leg**
- PoaWorld.arena_position -> **MoverStanding source = arena**
- MoverStanding source = journey leg -> **locator text every place comparison reads**
- MoverStanding source = arena -> **locator text every place comparison reads**

### What the world answers for a participant with no journey leg

Two participants on a real grid 9 squares across, arena at square 40. One walked from the arena to square 9, half a square in each direction. One opened no leg.

What the reader answered before this change:

```
0xNewcomer   mover_journeys = ()                    mover_locator = None
0xWalker     mover_journeys = ('walker-journey',)  mover_locator = '0:9:50:50'
```

What the reader answers now:

```
0xNewcomer   mover_locator = '0:40:0:0'   source = 'arena'        walked_nowhere = True
0xWalker     mover_locator = '0:9:50:50'  source = 'journey leg'  walked_nowhere = False
```

The walker reads the same locator on both sides. A leg still decides a place for every mover that opened one.

### The party check still refuses a member who walked nowhere

The walker discovered a crypt at square 9, out in the world. A party of two tried to enter it: the walker, and the participant that had opened no leg.

Before, the entry refused the member with no leg for an absence:

```
party_locators = {'0xWalker': '0:9:50:50', '0xNewcomer': None}

0xWalker leads this Descension into 0:9:50:50 and 1 of 2 members stand away from
it: 0xNewcomer at no journey leg open by this turn; a party walks into a dungeon
from the dungeon's own position
```

Now the entry refuses the same member for standing at the arena:

```
party_locators = {'0xWalker': '0:9:50:50', '0xNewcomer': '0:40:0:0'}

0xWalker leads this Descension into 0:9:50:50 and 1 of 2 members stand away from
it: 0xNewcomer at 0:40:0:0; a party walks into a dungeon from the dungeon's own
position
```

The refusal is the same refusal. `register.entry_ids()` answered `()` after the attempt on both sides, so the register held nothing either time. The message now names a place instead of a gap in the records.

### A Vessel whose owner walked nowhere forages the arena

A foraging run takes a standing `GridPosition` from its caller. Before this, a caller had nothing to build one from for an owner that had never moved, so the run refused:

```
forage REFUSED ForageError:
  a GridPosition was expected, got NoneType; no module records where a Vessel
  stands ...
```

Now the caller reads the standing off the world and hands it over. The run gathers:

```
standing = {'mover': '0xNewcomer', 'layer': 0, 'square_index': 40,
            'step_x': 0, 'step_y': 0, 'locator': '0:40:0:0', 'source': 'arena'}

material        iron ore at Magisterium
units_gathered  1
units_held      1
weight_carried  1
square foraged  40 on layer 0
store now holds (Stack(name='iron ore', quality='Magisterium', units=1),)
```

The forage path itself never refused the arena square. A position handed in by hand foraged square 40 on both sides. What was absent was a module that could say a Vessel stood there.

A Vessel still forages only the square it stands on. The walker stands on square 9, and naming the arena square refuses it:

```
Quicksilver Draught of 0xWalker stands on square 9 of layer 0 and square 40 was
named; a Vessel forages the square it stands on
```

### The same square, reached two ways

A third participant walked off the arena to square 41 and walked back. Its locator reads `0:40:0:0`, the arena's own text, and its source reads `journey leg`:

```
0xReturner   mover_locator = '0:40:0:0'   source = 'journey leg'   walked_nowhere = False
```

The locator alone cannot separate a participant that returned from one that never left. The source can. A consumer that must refuse an assumed place reads the source.

### A dungeon at the arena admits a party that never moved

Nothing refuses a discovery at the arena locator. A still leader discovered a crypt at `0:40:0:0`. A party of two, neither of which had opened a leg, tried to enter it.

Before, the entry refused both members:

```
party_locators = {'0xStillLeader': None, '0xStillMember': None}

0xStillLeader leads this Descension into 0:40:0:0 and 2 of 2 members stand away
from it: 0xStillLeader at no journey leg open by this turn; 0xStillMember at no
journey leg open by this turn
```

Now both stand at the door and the entry admits them:

```
party_locators = {'0xStillLeader': '0:40:0:0', '0xStillMember': '0:40:0:0'}

entry_id             u99world:0:40:0:0:0xStillLeader:w7:e5866666
locator              0:40:0:0
kind                 crypt
mode_code            dungeon_crawl
members              ['0xStillLeader', '0xStillMember']
entered_world_turn   7
entered_event_turn   5866666

clock_row 0xStillLeader   clock=event turn  locator=0:40:0:0
clock_row 0xStillMember   clock=event turn  locator=0:40:0:0
```

This change makes that entry possible. The rule matches every other entry: a party enters from the dungeon's own position, and the arena is now a position. The check on the mode stands apart from the check on the place. Monster Smash and team Monster Smash hold no map locator, and `require_dungeon` refuses both wherever they run.

### The three edges, and the one the world cannot answer

Nothing can build a world with no arena. `WorldRecord.arena_square` carries no default, and `create_world` always sets the middle square. A world the store does not hold now refuses instead of answering:

```
before   mover_locator('nosuchworld', ...) RETURNED None
now      mover_locator('nosuchworld', ...) RAISED WorldGridError:
         no world 'nosuchworld' is held; worlds: u99world
```

A blank address names nobody, and the reader refuses it:

```
mover_locator('')     RAISED MovementError: a mover must be a non-empty wallet address, got ''
mover_locator('   ')  RAISED MovementError: a mover must be a non-empty wallet address, got '   '
mover_locator(None)   RAISED MovementError: a mover must be a non-empty wallet address, got None
```

The third edge is open. No module holds the list of addresses a world has, so any address string that is not blank reads the arena:

```
knows(0xNobody)          = ()
mover_journeys(0xNobody) = ()
mover_locator(0xNobody)  = '0:40:0:0'
```

The caller holds the participant. A party names its own members and a run names its own Vessel, so each consumer already knows whose place it wants. The package does not, and `dungeon_entry.MEMBER_POSITION_ABSENT` records that.

### What reaches the arena position at runtime

No control on the Accumulation tab calls the reader. Two of the three consumers are not built on any screen.

```
WorldJourneys        installed. src/gui/shared_testnet.py builds one over the
                     demo chain and attaches it to the main window
DungeonRegister      no screen file builds one
ForageRegister       no screen file builds one
Map subtab           prints the arena square as "Own square", read straight off
                     the world record, not through the reader
```

The Map subtab already says a participant stands on the arena square. It reads `record.arena_square` itself. The reader now answers the same square as a position, with the layer and the two steps the locator needs.

The Maps subtab is where a dungeon entry belongs, by the operator's own ruling: a map click during a Crawl or a Raid opens it, and a location carries its options when the player moves into the square. The Resources subtab is where a foraging run belongs. Neither carries the control yet.

### What a standing position still does not know

```
who a world holds       no roster names the addresses of a world, so the arena
                        answers any address a caller names
a Vessel's own place    a standing places an address. A Vessel is placed by its
                        owner's address and has no position of its own
a turn that is real     a turn index below the first one the clocks count still
                        reads the arena. world_movement cannot read
                        world_turn.FIRST_TURN_INDEX, because world_turn imports
                        world_movement
holding a member still  a member can open a leg away from a dungeon after the
                        entry check reads its place and before the party goes in
the seconds in a turn   unset, so nothing in the program can say which world
                        turn is running and derive a later square
```

The rest of a dungeon stands where it stood. Nothing rules the inside, nothing resolves a fight, and a round still has no length.

## 2026-09-11 13:05 - #147 - a participant is a node, and a bot is a source that feeds one

A participant is one active Acervator node. The node qualifies by holding a Quintessence wallet address the ledger can answer for. One or more bots feed that node with their trading activity. A node running ten bots is one participant with ten sources. A bot is a source of activity, not a player.

The engine counted bot registrations and called the total its participants. One node with three bots therefore read as three participants. The count now counts nodes, and a new count beside it counts the bots.

### One node, three bots, one participant

The run below registers three bots on one node twice, on the old reading and on the new one. Nothing else about the run changes.

`src/competition/competition_engine.py` - the same three bots on the same node

```
before   bot registrations filed            3
         CompetitionResult.participants     3
         the results table                  3 bots

after    bot registrations filed            3
         participant_count()                1
         source_bot_count()                 3
         CompetitionResult.participants     1
         CompetitionResult.source_bots      3
         the results table                  1 participant node(s)
                                            3 source bot(s)
```

The node that registers is named by the identity file the tab writes. The node id is the first twelve characters of its public key and the wallet address is the whole of it. Both readings come off one file.

```
bot_identity.json            one file under the runtime directory
node id                      c3838662e5ea
wallet address               c3838662e5ea37ce9ac0e908165297aac9e7f2a77cb9215
                             0dfc0f034f4715640
the node id is its first 12  true
```

The shared TestNet bridge reads that same file and answers with that same node id, so the participant and the chain peer are one node.

```
SharedTestnetBridge.node_id()      dc61ee1c59a3
the file's own short id            dc61ee1c59a3
the participant registered         node dc61ee1c59a3, wallet dc61ee1c59a3...
```

### The wallet a participant registers is one the ledger can answer for

Two things are checked and both are checked against the Quintessence ledger itself. The address must pass the ledger's own address rule. The ledger must hold a wallet record for it, which happens only after a movement has fed that wallet.

`src/competition/participant_node.py` - the two refusals, in full

```
the address rule    node d960314c3eba gave '   ', which is no wallet address

no wallet record    node d960314c3eba gave wallet d960314c3eba9f1d209ceaa801
                    db9b4289050ba8e1414c02f060f6dcd6903990, which the
                    Quintessence ledger holds no record of; a participant's
                    wallet is one the ledger can answer for
```

The same node registers once Quintessence has moved into that wallet. The run distilled four Quintessence into it and the register call then took it.

```
before the movement   holds_wallet    false     register_node   refused
after the movement    holds_wallet    true      register_node   took it
                      balance         4.00000000
                      participant_count()       1
```

The ledger answers a balance of nothing for an address no movement names, so a balance alone cannot tell an unknown wallet from a wallet spent down to nothing. A new query answers that one question and the check reads it.

A bot that names a node nobody registered is turned away. A bot that names no node at all registers, trades and signs as before, and feeds no participant.

```
names an unfiled node   bot d3526f03b07e names node node-nobody-filed, which
                        registered no participant wallet; a node registers
                        before its bots feed it
names no node           registered, node_of_bot None
                        participant_count()   1
                        source_bot_count()    0
                        unattached_bot_count()   1
```

### Two nodes count as two, and each reads only its own bots

Two nodes registered two wallets. Two bots fed the first and three fed the second. The count read two, and each node's list held only its own.

```
node A   032307a2ebc5   ad2c2447e93e, 92423dabc184
node B   f314c2ee858c   7c5ee511d482, 22578d0b8fa6, 87b9a0c71f89

node A's list holding any of B's bots    empty
node B's list holding any of A's bots    empty
a bot read back to its node              A's first bot -> 032307a2ebc5
                                         B's first bot -> f314c2ee858c

participant_count()                      2
source_bot_count()                       5
the results table                        2 participant node(s)
                                         5 source bot(s)
```

One wallet cannot carry two nodes. The second node to name a wallet another node registered is turned away, so one wallet never reads as two participants.

### A bot still signs its own trades, and the log stays keyed by the bot

A bot is what signs a trade, and none of this moves that. The trade log keys on the bot id, the signature verifies against the bot's public key, and the Merkle proof verifies.

```
the signing bot                  c5baf7827ace
the record's public key is its   true
signature length                 128 hex characters
verify_trade on the sell         true
verify_trade on the buy          true
the log key is the bot id        true
log size                         2
Merkle inclusion of trade 0      true
Merkle inclusion of trade 1      true
a second bot keeps its own log   true
```

### What a reader of the participant count should read now

Four readers carry the count and each reads nodes now.

```
CompetitionResult.participants    participant nodes registered
CompetitionResult.source_bots     bots whose submissions the result ranks
the results table line            both counts, named
the saved result file             both counts, and each node with its own bots
```

The saved result file carries the nodes themselves, so a finished competition records which wallet each participant held and which bots fed it.


```
{"participants": 1, "source_bots": 3,
 "participant_nodes": [{"node_id": "711224c3165c",
   "wallet_address": "711224c3165c5ebe745fedf08ace5b94d04079c3267278cb0029a8973bd7246b",
   "source_bots": ["db8f5f8e3cf7...", "74240b2f425e...", "1ca0f82cb76a..."]}]}
```


The TestNet demo control registers three bots and registers no node, because it holds no Quintessence ledger to check a wallet against. Its table therefore reads nought participants and three source bots, which is what is

true of it.

```
LocalTestnet.run_demo_competition(n_bots=3)
  0 participant node(s)  ·  3 source bot(s)
  winner tier Harvest, award 10 ACRV
```

### Where one wallet a bot still stands

The certification socket derives one address from each bot id. That address is still the sender of the bot's own trade commitment, and it is still the credited wallet for a bot that feeds no participant node. A bot that feeds one credits that node's wallet instead, so the market share ceiling and the cooldown bound the participant.

`src/competition/certification_socket.py` - the synthesised address

```
    @staticmethod
    def _wallet_for(bot_id: str) -> str:
        """Return the address synthesised from ``bot_id``.

        ``_post_commitment`` sends every trade commitment from this address,
        because a trade is the bot's own, and ``award_wallet_for`` returns it as the
        credited wallet for a bot feeding no participant node.
        """
        return f"0x{bot_id[:40]}"
```

The dated section below carries the move, the readings on both sides of it, and the reason a bot on no node keeps this address.

The party window is the other place. It lists one row a bot from the fleet load and its row field is named for a participant. Those rows are source bots under this reading, and the wallet panel above them is the one that names the participant.

```
the party row field          participant, holding a bot id
the wallet panel label       Participant, holding the node id
the on-chain view name       getParticipantCount, counting one wallet a bot
the on-chain record key      participants, holding one wallet a bot
```

The two on-chain names are a contract interface and a stored record shape. Renaming either changes what a chain record holds, so both stand and neither is read as a participant count of nodes.

## 2026-09-11 13:07 - #147 - every game fee is Quintessence, and a movement on the chain costs Fluor

**HIS.**

> *"Everything is Quint and we can invent our version of gas using an alchemical*
>
> *term for 'flow' or 'move' or 'fuel'"*

Two rules follow, and they do not overlap.

```
the game charges a fee     the fee is Quintessence
an exchange reports a fee  the fee stays in US dollars
```

### What the game charges, and what the exchange charges

The game charges Quintessence for every act it prices. An action band costs Quintessence. A transfer between two wallets loses a share to the pleroma, 8% at transfer skill level 1 and 4% at level 10. A first skill level costs quality-weighted uses. A consecration declares a Quintessence cost. An event's entry fee carries a rank today and no currency.

A trading fee is different. The exchange took that fee, the exchange reports it, and the exchange is the authority on it. The platform stores the figure the venue gave and changes nothing about it.

```
the fill carries      fee_usd, in dollars
certification reads   that one figure
the ledger converts   at QUINTESSENCE_PER_FEE_USD, which stands at 1
```

Nothing in the package charges a game fee in dollars. The Base layer two also charges its own gas in gwei, and that is a public chain's charge, not this game's.

### Fluor is the name of what a movement on the chain costs

A movement of value on the PoA chain costs **Fluor**, and Fluor is paid in Quintessence like every other game fee.

Georgius Agricola printed the word in *Bermannus, sive de re metallica dialogus*, 1530. Miners called stones that melt in fire *fluores*. A smelter adds them to a charge so the metal runs. The singular is *fluor*, a flowing.

The name fits the thing. A flux is not burnt for heat. It is spent so a mass can move, and it is gone when the work ends. That is what a charge for moving value is.

```
src/competition/quintessence_ledger.py
  FLUOR              the name, five characters
  FLUOR_SOURCE       the book and the year
  FLUOR_RATE_ABSENT  says no source sets a figure
```

The package entry binds all three, so `from src.competition import FLUOR` reaches the name.

### No figure is set, and none is written

Nobody has set what one movement costs. No module holds a Fluor figure and no screen shows one. The nearest charge that exists today is the transfer bleed, and whether Fluor names that charge or a separate charge on every chain write is still open.

Six other names were researched and each lost on a named ground. The clearance work, the sources and the collision counts sit in the PoA name clearance page.

## 2026-09-11 13:10 - #585 - a guild roster saves to a file, and the Guild subtab reads it back

A guild lived until the screen drew itself again. The Guild subtab built a new empty roster each draw, and no module wrote a roster to disk. A roster now has a file of its own. The subtab reads that file. A guild founded once is on the page on every later draw.

`src/competition/guild_roster.py` holds the roster. It now carries `save`, `load` and a default file. Twelve other modules in the package already declare a default path under `~/.acervator`, and this one follows them:

```
DEFAULT_ROSTER_PATH   ~/.acervator/poa_guild_roster.json
ROSTER_FILE_VERSION   1
save                  writes every guild, its members and its officers
load                  reads that file back into the roster that asks
roster_path           the file this roster saves to and loads from
```

A roster takes its file by construction, as a world and a journey store do. A caller that names no file gets `DEFAULT_ROSTER_PATH`.

### The next draw loses a founding that nothing wrote

The readings below come off the screen route. A driving script called the bridge handler the React tab calls, and printed the strings the Guild panel draws.

```
before, the Guild subtab after one founding of a guild with two members
  Guilds            : 0
  Members           : 0
  Roster file       : the row did not exist
  the panel drew no guild list
  note: No module writes a roster file, so the roster loads empty and no guild
        is listed. Founding one through a control is not built.

after, the same founding, and the roster was not saved
  Guilds            : 0
  Members           : 0
  Roster file       : poa_guild_roster.json - no file yet
  the panel drew no guild list

after, the same founding, and the roster was saved
  Guilds            : 1
  Members           : 2
  Roster file       : poa_guild_roster.json - held
  listed guild      : The Salt Wardens - 2 member(s)
```

The third block held on the next draw, and on the draw after it. Only a save puts a guild on the page, and a save is enough.

### The same guild, read back into a second roster

One roster saved its file. A second roster object read the same file. Both answer the same.

```
first  guild_count 2   member_count 3
second guild_count 2   member_count 3
bot-u100-founder-0001    first officer   second officer
bot-u100-joiner-0002     first officer   second officer
bot-u100-second-0004     first member    second member
bot-u100-outsider-0003   first None      second None
keys  both ['silver_mirrors', 'the_salt_wardens']
rows  both name, founder, treasury, 2 members and 2 officers, then 1 and 0
```

Every treasury address matched. A treasury address follows from the guild key, so a roster read back pays from the address it paid from before.

### What an absent, an empty and a foreign file each do

A read the module cannot trust changes nothing. The roster keeps what it holds, and the module writes one line to the log. This matches `PoaWorld.load` and `WorldJourneys.load`, which the same bridge installs.

```
an absent file             guild_count 0, and no file is created
an absent file, under a
  roster holding a guild   that guild is still there after the load
an empty file              guild_count 0
                           log: failed to read ...: Expecting value
a file of version 99       guild_count 0
                           log: carries version 99, not 1; nothing was read
a file naming the pleroma
  treasury bucket          guild_count 0
                           log: the stored roster holds its treasuries in the
                           'pleroma' bucket and this one reads 'wallets'
a file carrying one key
  twice                    guild_count 0
                           log: the stored roster carries 'twins' twice and one
                           key names one treasury address
the same reader on a
  sound file               guild_count 2, member_count 3
```

The last line is the control. The same reader that answered nought five times answered two guilds and three members on a file that was sound.

### Where the program holds its roster

Two places hold a roster, and one file joins them.

The Guild subtab reads the picked chain's own roster file, through `roster_store`. This is what `world_store` does for the world and what `loaded_ledger` does for the Quintessence ledger. The bridge handler gets parameters and no window, so a file is the only thing it can read.

```
Live chain           ~/.acervator/poa_guild_roster.json
Demo TestNet chain   ~/.acervator/poa_guild_roster_testnet.json
```

The live file and `DEFAULT_ROSTER_PATH` are the same file. A demo guild is on the demo subtab and not on the live one, and the live guild did not change when a caller wrote the demo file.

`SharedTestnetBridge.install_roster` builds the running program's own roster and reads its file at launch, beside the world and the journeys. `install_on` attaches it to the window as `_guild_roster`. Driven on a stand-in host, the window held a roster of one guild and two members off a file another caller had saved.

Three modules take a roster and none builds one: `alignment.world_alignment`, `consecration.add_guild_places` and `army_command.open_party`. `install_roster` builds the roster those three can take. Nothing reads the window's copy yet, which is also true of the world and the journeys beside it.

### The refusals a roster already had, unchanged

```
a duplicate guild name   'The Salt Wardens' already holds the key
                         'the_salt_wardens' and its treasury
                         poa_guild_treasury_the_salt_wardens; two guilds never
                         share one treasury address
a member joining twice    address bot-u100-joiner-0002 already belongs to
                         the_salt_wardens; one address belongs to one guild, so a
                         transfer between guild members reads one guild
promote, no membership   bot-u100-outsider-0003 is no member of the_salt_wardens,
                         which holds 2 member(s)
demote, no membership    bot-u100-outsider-0003 is no member of the_salt_wardens,
                         which holds 2 member(s)
leave, no membership     bot-u100-outsider-0003 is no member of the_salt_wardens,
                         which holds 2 member(s)
an unknown guild key     'no_such_guild' is no guild in this roster; it holds 1
                         of them
a name with no letter    a guild name carries at least one letter or digit, got
                         '   '
```

### What founding a guild still costs

Nothing. `found` takes a name and a founder, and no third value:

```
found's parameters        ['self', 'name', 'founder']
found with a third value  GuildRoster.found() takes 3 positional arguments but 4
                          were given
a stored guild's fields   founder, key, members, name, officers, treasury
the stored roster fields  guilds, treasury_bucket
```

No stored field names a price. The operator owes three figures, and no unit invents one:

```
how many tokens found a guild
the stake a rank locks
a cap on how many officers a guild holds
```

### Which control reaches a roster

None. The page draws fifteen controls, and no control name carries the word guild:

```
identity, identity_confirm, distil, train, transfer, spend, grade, payout, close,
drop, season, reset, reset_confirm, world, world_confirm
```

Founding, joining, leaving, promoting and demoting are each reached by no control. A caller reaches all five, and a roster saved by a caller is on the page. The Guild subtab, `Ctrl+7`, is the surface that would carry a founding control, the way the world control declares a world.

The reset control deletes the Demo TestNet chain's record file and its log, and it refuses on the Live chain. It does not touch a roster file, so a reset leaves the guilds standing.

## 2026-09-11 13:30 - #586 - one world turn is 3,600 seconds, and a constant says so

The operator set the length himself: world turns are 1hr. No constant carried it. The code said so out loud, in a sentence four modules read, and a clock ratio that needed the figure had to be handed one by its caller. The hour is now a constant.

`src/competition/poa_modes.py` already named both event timeframes. The world turn sits beside them, in the same shape:

```
ELITE_TIMEFRAME        "1m"
STANDARD_TIMEFRAME     "5m"
WORLD_TURN_TIMEFRAME   "1h"
WORLD_TURN_SECONDS     TF_SECONDS["1h"], which is 3600
```

`TF_SECONDS` is the one table the event candle already read. The world turn reads the same table, so one scale measures both clocks and no second figure exists to drift from the first.

### The hour now has a name in the code

`consecration.WORLD_TURN_SECONDS_ABSENT` keeps its name, because the package entry binds it, and it no longer reports an absence. It reports the length and where the length came from.

```
before   "no constant names the seconds in a world turn. pvp_vote counts whole
         world turns and its own notes carry the operator's one turn an hour, so
         the hour lives in prose and every turn field here is a whole turn index"

after    "one world turn is 3600 seconds, the operator's one turn an hour.
         poa_modes declares it as WORLD_TURN_TIMEFRAME 1h beside ELITE_TIMEFRAME
         and STANDARD_TIMEFRAME and reads the seconds off the one TF_SECONDS table
         both clocks are measured by, so no second scale names it. Every turn
         field here stays a whole turn index"
```

### Twelve standard turns and sixty elite turns an hour

`event_turns_per_world_turn` could not answer before. It took the length as a required value and no constant could supply one. It now answers from the two declared lengths, and both divide the hour with nothing left over.

```
before   event_turns_per_world_turn(standard)
         TypeError: missing 1 required positional argument: 'world_turn_seconds'

before   event_turns_per_world_turn(standard, WORLD_TURN_SECONDS_ABSENT)
         ClockNestingError: a world turn of "no constant names the seconds in a
         world turn..." is not a count of seconds

after    event_turns_per_world_turn(standard)   12
         event_turns_per_world_turn(elite)      60
```

The run printed every dungeon variant's count off `nesting_rows`, which now takes the length as its own default too:

```
dungeon_crawl         5m candle, 300s   12 event turns a world turn   remainder 0s
dungeon_crawl_elite   1m candle,  60s   60 event turns a world turn   remainder 0s
raid                  5m candle, 300s   12 event turns a world turn   remainder 0s
raid_elite            1m candle,  60s   60 event turns a world turn   remainder 0s
```

Both candles divide the hour exactly, so the refusal for a part-turn remainder never fires on the declared pair. A caller may still pass its own length, and a length the candle does not divide is still refused.

### Every reader of the note now carries the number

Four modules read the sentence. Each one now reads the length beside it.

```
consecration   ConsecrationRegister.to_dict gains a world_turn_seconds key of 3600
world_turn     turn_economy_row gains the same key; the refusal for a fractional
               turn index now quotes the length
crafting       CRAFT_CLOCK_NOTE says turns_required counts 3600-second turns, and
               Craft.to_dict gains the key
dungeon_entry  WORLD_TURN_SECONDS_NOTE says both clock functions default to the
               length; ABSENT_MECHANISMS keeps its world_turn_seconds entry
```

No key was dropped from any of those dictionaries and no sentence was deleted. A reader that asked for the absence note still gets a sentence at that key, and the sentence now carries the figure.

### Only one name holds the seconds

Every module in the package was imported and every module-level whole number compared against 3,600. Five names hold it, all of them the same name, and all five are the one object `poa_modes` declared.

```
src/competition/consecration.py::WORLD_TURN_SECONDS    the declared object
src/competition/crafting.py::WORLD_TURN_SECONDS         the declared object
src/competition/dungeon_entry.py::WORLD_TURN_SECONDS    the declared object
src/competition/poa_modes.py::WORLD_TURN_SECONDS        the declaration
src/competition/world_turn.py::WORLD_TURN_SECONDS       the declared object
46 modules scanned
```

One other `3600` is written in the package, and it is not a turn length: `challenge_protocol.py` defaults a challenge to expire an hour after it is raised. The PvP vote's three figures count world turns rather than seconds, so its 24, 72 and 7 x 24 stay as they are and read no length.

### The screen reads the length in one place

The turn meter does not read a world turn. It reads the running event candle, which is 300 seconds standard and 60 elite, and the event band draws the mode and the Impetus. The map panel is the one surface that names a world turn, and it was saying the length did not exist.

```
turn_meter      120s of 300s left, 40.0% - the event candle, not the world turn
event band      the mode code, its label, its variant and the Impetus line
map panel       before: "No module names the seconds in a world turn, so
                JourneyLeg.progress_at is given no turn"
                after:  "A world turn is 3600 seconds and no module turns a clock
                reading into a turn index, so JourneyLeg.progress_at is given no
                turn"
```

The first clause was false after this change and the second clause was always the real blocker. `progress_at` takes a turn index, not a count of seconds, so the length was never what stopped it. The panel was driven on a real world with one layer breached, and it printed the new sentence.

### Crafting and equipping exist, and the notes say so now

`vessels.ABSENT_MECHANISM_NOTES` said crafting and equipping were both missing. Both ship. A craft was driven end to end and an equip check was driven on both sides of its own test.

```
begin_craft      took 1 iron ore at Calx out of the crafting Vessel's store
complete_craft   delivered armour at Calx into a gear slot, is_stored True,
                 is_accounted True, embedded 0.00000001, unaccounted 0
equip_check      cohesion 0.00000001 against a total of 0.00000001 - allowed
equip_check      cohesion 0.00000001 against a total of 0        - refused,
                 shortfall 0.00000001
```

No key left that dictionary. The two notes now name the code that holds each mechanism and the gap each one still has.

```
crafting    before: "no module makes an item"
            after:  "crafting.CraftRegister makes one. begin_craft takes a
            Recipe's components out of the crafting Vessel's store and
            complete_craft delivers the item and embeds its Quintessence, and
            crafting.TURNS_ABSENT names the turn count no source sets"

equipping   before: "nothing holds gear to the equipping holder's Quintessence"
            after:  "items.equip_check holds it: the ItemType's cohesion is read
            against the total Quintessence held and EquipCheck.is_allowed is that
            Potential.is_full. No module writes the gear a Vessel wears, so
            nothing records an item as worn"
```

The `gear` note beside them still reads correctly. A store carries gear in counted slots, and carrying is not wearing: nothing records a worn item, so that note stays as it was.

### What the world turn still does not say

The length is the only figure this unit added. Four things the operator named in the same breath carry no figure, and no unit invents one.

```
what a world turn costs        no module prices one action in Impetus or steps
how many world-level actions
  fit in one block             nothing counts actions against a block
the UTC sequencing rule        nothing orders two zones' actions by a clock
the view range                 base sight is the participant's own square and
                              squares_in_view takes a radius nobody supplies
the index of the turn running
  now                         no module turns a clock reading into a turn index,
                              so the map cannot move a participant off the arena
```

The last row is what the map panel needs, and it needs an origin second as well as a length: nothing records when world turn zero opened. ## 2026-09-11 14:30 - #147 - an award credits the participant's wallet, and the ceiling bounds the participant

A participant is one active Acervator node with a connected Quint wallet address, and one or more bots feed that node. The certification socket used to derive one address from each bot id and credit that. The market share ceiling and the cooldown then bounded a bot. Both now count against the participant.

The socket takes the participant register the same way it takes the chain, the ledger and the capture bounds: by construction. One reader answers which wallet an award credits, and the socket asks it once, before it writes anything.

`src/competition/certification_socket.py` - the one reader

```
    def award_wallet_for(self, bot_id: str) -> str:
        if self._participants is None:
            return self._wallet_for(bot_id)
        node_id = self._participants.node_of_bot(bot_id)
        if node_id is None:
            return self._wallet_for(bot_id)
        wallet = self._participants.wallet_of(node_id)
        if wallet is None or not self._ledger.holds_wallet(wallet):
            raise CertificationRefusedError(...)
        return wallet
```

### One bot on one node credits that node's wallet

The run below certifies one fill twice. The first socket holds no register. The second holds one, with the node registered and the bot attached to it. Each side ran its own node and its own bot, so each side prints its own addresses. The fee was thirty dollars at a grade of one, which distils thirty Quintessence.

```
no register        node wallet          c2dc8100979e04184762b2a54b...
                  synthesised address  0x3d182c3cf23714fc4bc183b6e42d1a1319978e62
                  socket credits       0x3d182c3cf23714fc4bc183b6e42d1a1319978e62
                  award_reason         awarded, distilled 30.00000000
                  node wallet          0 -> 0
                  bot address          0 -> 30.00000000
                  ledger answers for the node wallet   False

with the register  node wallet          bdb615176fccc081dd346e2de8...
                  synthesised address  0x002c27ddf67cad9a8414f8cdac8cee7a6df19274
                  socket credits       bdb615176fccc081dd346e2de8...
                  award_reason         awarded, distilled 30.00000000
                  node wallet          1.00000000 -> 31.00000000
                  bot address          0 -> 0
                  ledger answers for the bot address   False
```

The one Quintessence already in the node wallet is the movement the register needs before it files a node. The section on the unattached bot says why.

### The ceiling now counts two bots as one participant

This is the money change. Two bots fed one node. Each had one fill worth thirty Quintessence of a pool of a thousand, and the ceiling on that pool is fifty. Each fill passed the ceiling on its own. The second fill sat past the cooldown, so the cooldown could not be what refused it.

```
market pool on A15/USD     1000 Quintessence
share ceiling              50, which is 5% of the pool
each fill's award          30 Quintessence, under the ceiling alone
both fills together        60 Quintessence, over the ceiling

no register        bot A credits   0xfa761491d5752447b7943b74e3667a2c1d5d7c61
                  bot B credits   0x558db9285fe7be2a3e20527b69d0cf966e366b25
                  bot A           awarded, distilled 30.00000000
                  bot B           awarded, distilled 30.00000000
                  pool drawn      60.00 of 1000
                  drawn past the ceiling   True

with the register  bot A credits   36d7c52da3996e391e205138d9...
                  bot B credits   36d7c52da3996e391e205138d9...
                  bot A           awarded, distilled 30.00000000
                  bot B           allotment_taken, distilled 0
                  pool drawn      30.00 of 1000
                  drawn past the ceiling   False
```

Two bots on one node took sixty Quintessence of a pool that allows one participant fifty. They now take thirty. The bound that refuses the second fill is the one allotment a participant a market an activation period, not the ceiling test: `_refusal_for` reads the allotment rule first, so the node's second fill on the same market never reaches the ceiling comparison. The ceiling is what sets what the node may hold of that market, and the allotment rule is what keeps the node inside it.

### The cooldown now runs on the participant

The same two bots on one node, on two different markets, with the second fill six hundred seconds after the first. The cooldown is three candles of the bot's own timeframe, floored at nine hundred seconds, which is ten thousand eight hundred seconds on an hour candle.

```
no register        bot A credits   0x661a690670a3c1d4f1e91a6e2fd65ac9f88b3167
                  bot B credits   0x6ac0a4fac6021d5ba2baa6f51f590da28869cbee
                  one address for both     False
                  bot A           awarded, distilled 30.00000000
                  bot B           awarded, distilled 30.00000000
                  A's cooldown left       10200s
                  B's cooldown left       10800s
                  A14/USD drawn           30.00

with the register  one address for both     True
                  bot A           awarded, distilled 30.00000000
                  bot B           cooldown_running, distilled 0
                  A's cooldown left       10200s
                  B's cooldown left       10200s
                  A14/USD drawn           0
```

Two bots used to hold two cooldowns. A node that runs ten bots could take ten awards inside one window. One node now holds one cooldown, and both bots read the same seconds left on it.

### A bot that feeds no node keeps its own address

A bot that feeds no node credits the address the socket synthesises from its bot id, as it did before. That is a decision, and the register itself gives the reason. `ParticipantRegistry.register_node` refuses a wallet no movement has put in the ledger's wallet book. Distilling a certified fill is the movement that puts one there. Refusing such a fill would leave no wallet ever fed, so no node could ever register.

`src/competition/participant_node.py` - the refusal, read before any movement

```
register_node, wallet unfed   refused: node d90f2deced15 gave wallet
                              d90f2deced157f42a15088eff7c47f9d993c22303a28d5cc...,
                              which the Quintessence ledger holds no record of
ledger answers for it         False
movements on the ledger       0

the bot's node                None
socket credits                0x8e835c1b6daa2e01a9543057d8826292d3bbc892
award_reason                  awarded, distilled 30.00000000
ledger answers for it         True

then the node registers       took it, node d90f2deced15
socket credits now            d90f2deced157f42a15088eff7c47f9d993c22303a28d5cc...
```

No path credits a wallet the ledger cannot answer for. A node whose wallet this socket's own ledger holds no record of is refused, and the refusal lands before the socket writes anything. The run below gave the register one ledger and the socket another.

```
the register's ledger answers for it   True
the socket's ledger answers for it     False
certify                                refused
the bot's certified fill count         0
the bot's trade log                    No trades in log
total ever minted                      0
```

### What did not change

The bot is still what signs a trade. The trade log still keys on the bot id. The socket's lifetime certified fee still only rises. Conservation balanced on every path the run drove, including every refusal.

```
signature verifies                True, 128 hex characters
the record's public key is the bot's   True
A's log keyed by A's bot id       True
B's log keyed by B's bot id       True
A's log size 2, B's log size 1    two bots, two roots
A's lifetime fee, in order        0, 30.0, 60.0, 60.0
that total only rises             True

conservation on every path        balanced True, delta 0, negative buckets 0
no fill_id                        refused, nothing minted
a fill already certified          refused, nothing minted
the same market again             allotment_taken, distilled 0
inside the cooldown               cooldown_running, distilled 0
a fill naming no activation       no_activation_named, distilled 0
```

The chain sender of a trade commitment is still the address synthesised from the bot id, because a trade is the bot's own. The award record on the chain is sent from the participant's wallet, because the award is the participant's.

### Nothing reaches a certified fill in the running app

The socket is built at launch. `SharedTestnetBridge.install_on` builds it and attaches it to the main window, reached from `src/gui/main_window.py` line 277. Nothing then subscribes it to a fill.

```
certify callers              one, _on_trade_filled inside the socket
_on_trade_filled runs        only after attach_to_bus subscribes trade.filled
attach_to_bus callers        none, anywhere in src or main.py
controls on the PoA tab that reach certify   none
```

No fill reaches certification in the running application, and no control on the tab reaches it. The register arrives by construction the way the bounds do, so whoever subscribes the bus passes the register in the same call.

# Market Inspector Tab

Reference. The higher-timeframe scanner and the topology proposal pane. First step of the promotion pipeline.

## What builds it

`MarketInspectorTabMixin._build_market_inspector_tab` in `src/gui/main_tabs/market_inspector_tab.py` constructs the tab and wires its three injection points before adding it to the row.

The tab splits horizontally. The left half holds the scanner, the right half holds the proposal cards inside the Bot Swarm Topologies zone, which is one of the three zones that side carries.

## The six zones

The screen draws six zones, three down each side: ATA-SMP, Opposing Trades and Multi-Exchange Arbitrage on the left, and ATA-SMP Ready to Send, Bot Swarm Topologies and Phantom Bot HTF Signals on the right. Every zone shows one entry at a time, with a left arrow, a right arrow and a line saying which entry is on screen out of how many the zone holds. Clicking the entry opens it, and the expansion names the test, the window, the result and why that answer let the entry through. One implementation serves all six, and the proposals pane draws through the same one. The record is `tests/debug_reports/2026-09-06_topology_list.md`.

`src/gui/main_tabs/market_inspector_surface.py` — the three right-side zones

```
def right_zone_rows(run: Any, bucket: Any = None) -> list:
    """The three right-side zones as key, title and status, in screen order."""
```

The Qt build gives every zone a `ProposalStepper`. The two web hosts give every zone a `ZoneStepper`. Both are written from one description of the zone, so a zone's lines are decided in one place and a second copy cannot drift from the first.

`src/gui/market_inspector.py` — the Qt widget every zone uses

```
class ProposalStepper(QWidget):
    """One zone's entries shown one at a time, with arrows and a expansion.
```

## Left half: the scanner

A filter row carries a Refresh button and an "Include active markets" checkbox. The default hides the markets already under a bot, which keeps the table on entry opportunities.

`HTF Signals` lists one row per market: Asset, Signal, Score, Daily, Weekly, Active.

`Opposing Pairs (cointegration, Engle-Granger + Johansen, p<=0.05)` lists Long side, Short side, Method, Window, Statistic, Correlation and the combined score. The 30-day Pearson coefficient raises a candidate and no longer decides one.

## Where the analysis happens

`MarketInspector` in `src/trading/market_inspector.py` owns the maths. One method drives the whole pipeline and keeps the results on the analyzer.

`src/trading/market_inspector.py` — `MarketInspector.scan_universe`

```
def scan_universe(
    self,
    candles_by_symbol_by_tf: dict,
    active_symbols: set,
    closes_by_symbol: dict,
) -> None:
```

Two steps run under it.

| Step | Produces |
|---|---|
| `_analyze_tf` | One `TimeframeAnalysis` per timeframe |
| `_score_market` | One `MarketSignal` per market |

The pairing is a separate step. It enumerates every long against every short and keeps the cointegrated ones. A thirty-day return correlation sitting in the configured negative window raises a candidate, and the cointegration test then decides it.

`src/trading/market_inspector.py` — `MarketInspector._find_opposing_pairs`

```
def _find_opposing_pairs(self, signals: list, closes_by_symbol: dict) -> list:
    """Enumerate long × short candidates; keep the cointegrated ones.

    The 30-day return correlation raises a candidate and no longer decides
    it: ``cointegration_test`` runs on the full close series and a pair it
    refuses never reaches the table.
    """
    longs = [s for s in signals if s.direction == "long" and s.score >= 0.3]
    shorts = [s for s in signals if s.direction == "short" and s.score >= 0.3]
```

The correlation raises a candidate and no longer decides one. Every pair the screen shows has also passed a test for a long-run equilibrium, because two markets can move opposite each other for a year with no relationship holding between them. The test is cointegration in both of its standard forms, taken from statsmodels, and a pair passes only when both reject the null of no cointegration: Engle-Granger at a significance of 0.05, and Johansen above its ninety-five per cent critical value for rank zero. The window is 365 daily closes and a pair with fewer than 120 is not tested and not shown. Opposing Pairs therefore carries seven columns today: Long side, Short side, Method, Window, Statistic, Correlation and Score. The numbers are recorded in `tests/debug_reports/2026-09-06_stageC_cointegration.md`.

`src/trading/pair_selection.py` — what a pair has to clear

```
WINDOW_BARS = 365
MIN_OBSERVATIONS = 120
SIGNIFICANCE = 0.05
```

One analyzer instance serves two readers. `get_shared_inspector` returns it, the tab owns the fetch cycle and writes into it, and the per-bot Market Inspector page in the Bot Details dialog reads back out of it through `build_per_bot_view`. One analyzer, two readers, no second copy of the score.

## Fetching

`set_exchange_source` binds a connectors getter and the application scheduler, so the tab always sees the current connector dict rather than a snapshot taken at build time.

The fetch serves its last network result while that result stays young enough, and the Refresh button passes a flag that goes to the network regardless.

`src/exchange/market_inspector_fetcher.py` — `fetch_htf_universe`

```
async def fetch_htf_universe(
    exchange_connectors: dict,
    active_symbols: Optional[set] = None,
    top_n: int = DEFAULT_TOP_N,
    progress_cb=None,
    force_network: bool = False,
    min_refresh_s: float = DEFAULT_MIN_REFRESH_S,
) -> FetchResult:
```

Three helpers do the work inside it.

| Helper | What it does |
|---|---|
| `_pick_universe` | Ranks each connector's bulk tickers by 24-hour quote volume |
| `_fetch_one_symbol` | Pulls per-symbol OHLCV on the connector's single-worker executor |
| `_resample_daily_to_weekly` | Derives the weekly series on the client when the venue lists no weekly timeframe |

## What a scan reports

An empty table can mean three different things. The note under it says which:

- no scan has run yet
- a scan is running
- a scan finished and found nothing

Before this, all three looked the same. An empty table gave no sign of which one it was, so a tab that was working looked exactly like a tab that was broken.

`src/gui/market_inspector.py` — `_empty_table_text`

```
def _empty_table_text(scan_state: str, noun: str) -> str:
    """The sentence an empty table carries for one scan state.

    ``SCAN_NOT_ASKED``, ``SCAN_RUNNING`` and ``SCAN_FINISHED`` each get
    their own wording, so the three never read alike.
    """
    if scan_state == SCAN_RUNNING:
        return f"Scanning for {noun}…"
    if scan_state == SCAN_FINISHED:
        return f"Scan finished. No {noun} found."
    return f"No scan yet. Press Refresh to look for {noun}."
```

`scan_state` reports the same three states to the renderer. The React side draws the note under the table, the same way the tab does.

The scan also writes to the log. It writes one line when it starts and one line when it finishes.

The start line names two things: whether Refresh forced the scan, and how many exchange connectors it reached.

The finish line names four things: how many markets the scan covered, how many signals and pairs it scored, how long it took, and where the data came from.

The finish line is always written. A scan that scored nothing still writes one. That is what makes a quiet market different from a scan that never ran.

Each line also goes on the event bus as a record other parts of the platform can read.

| Topic | Carries |
|---|---|
| `market_inspector.scan_started` | forced, connector count, active symbols |
| `market_inspector.scan_finished` | market count, duration, signals, pairs, source, error |

Both topics are declared in `src/core/emit_contracts.py`. A topic declared there is a tracked emitter, and a run that never fires it is reported.

## Right half: topology proposals

`MarketInspectorTopologies` in `src/gui/market_inspector_topologies.py` renders one card per proposal and opens a preview dialog on Preview.

`current_topology_proposals` answers two different things. It answers `None` when the right pane never built or refused the read, and a list when the pane answered, so an empty list means the pane holds no proposals. It answered an empty list for both before, which is what made a scan that found nothing read like a pane that was never there. The reader in the Simulator tab passes the same two answers on rather than flattening them.

`src/gui/market_inspector.py` — the two answers a reader gets

```
def current_topology_proposals(self) -> "list | None":
    """The proposals on display, for a simulator to read and wire.

    ``None`` says the right pane never built or refused the read,
    and a list says the pane answered. An empty list therefore
    means the pane holds no proposals, which no caller can
    confuse with a pane that is not there.
    """
```

The engine behind the cards takes a plain dictionary, which keeps it free of any exchange or bot-manager coupling. It unions the detectors, drops overlapping proposals by asset and caps the result.

`src/trading/topology_proposals.py` — `detect_all_topologies`

```
def detect_all_topologies(
    context: dict[str, Any],
    cap: int = PROPOSAL_CAP,
) -> list[dict[str, Any]]:
    """Union every detector's proposals, dedupe them and cap at ``cap``.
```

Four detectors feed it.

| Detector | Shape |
|---|---|
| `detect_momentum_funnel` | Correlated cluster, leader into laggers |
| `detect_mean_reversion_pair` | Anti-correlated pair, wired both ways |
| `detect_sector_cluster` | Same-sector star, hub into spokes |
| `detect_distance_to_band` | One asset, scrum-deep into fold-deep |

Two helpers read from disk under `src/trading/`. `suggested_target_usd` sizes each proposed bot from the asset target defaults, and the sector map names each asset's sector.

## Dismissal

Dismissing a card hides it for a day. The write-through never raises, because losing a dismissal is a nuisance and taking down the pane is not.

`src/gui/market_inspector_topologies.py` — `_persist_dismissed`

```
def _persist_dismissed(self) -> None:
    """Best-effort write-through. Never raises: losing a
    dismissal is a nuisance, taking down the pane is not."""
    if self._dismiss_store is None:
        return
    try:
        self._dismiss_store.set(DISMISS_SETTINGS_KEY, dict(self._dismissed))
```

That write fails on every call today, because the key it writes is not one the settings schema declares. The pane logs the failure and carries on, so the dismissed count reads zero on every launch. Issue #424 carries it.

`set_dismiss_store` hands the pane the settings manager it persists through. The pane never resolves settings itself.

## Adopt

The pane emits an adopt request and the window handles it. The handler counts the new bots and their combined budget, asks which of the proposed wires already exist and would change, and shows all of it before anything is created.

`src/gui/main_window.py` — `_adopt_topology_proposal`

```
def _adopt_topology_proposal(self, proposal: dict) -> None:
    """Confirm, open the wizard for each new bot, then emit `wire.created`."""
```

An adopt that aborts part way calls `_report_adopt_orphans`, which names the bot ids it created and left unwired.

The per-bot page is drawn from one description as well. `per_bot_view` reads the shared analyzer and returns the whole screen as values: the rows, the groups, the words, the colours and the layout numbers. The Qt side draws them as widgets and the React side draws them as elements, where the React side drew a Python error message before. The record is `tests/debug_reports/2026-09-06_unit12_market_inspector_tab.md`.

`src/gui/main_tabs/market_inspector_tab_surface.py` — `per_bot_view`

```
def per_bot_view(bot: Any) -> dict:
    """The per-bot Market Inspector screen as values, read off the shared
    analyzer's most recent scan.
```

## ATA-SMP

The eight phases sit outside the screen. Phases one to three and phase eight run in `src/trading/ata_spm.py`, and phases four to seven run in `src/trading/ata_spm_push.py`. The screen reads what they produce and draws it; it computes none of it. Seven push targets ship, and adding one is adding a row, because no phase names a target.

`src/trading/ata_spm_push.py` — the names every screen reads

```
TARGET_NAMES = tuple(one.name for one in PUSH_TARGETS)
```

The rules each platform publishes are recorded in the push target rules. The narrative for the whole screen is on the tabs page.

## Exchange comparison arbitrage

`src/trading/arbitrage.py` holds the cross-exchange price monitor and its spread tracking. No module under `src/` imports it. The tab draws a Multi-Exchange Arbitrage zone, and that zone draws no arbitrage panel: it names the venues in reach and stops there. The first two proposal forms are on screen; the third is not.

In development.

## Bridge

Three methods serve this screen, and the renderer modules carry the matching names.

| Bridge method | Serves |
|---|---|
| `market_inspector.state` | The scanner and its two tables |
| `market_inspector_tab.state` | The per-bot page in the Bot Details dialog |
| `market_inspector_topologies.state` | The proposal cards |

## Updates

What the screen is sits above. How it got there sits below, in order.

Each entry carries the date, the time, the issues it addressed and a short phrase. Every date and time is read off the commit or the issue comment that carries the change, never off the day the entry was written. An entry with no artefact behind it says so and gives no time.

`.claude/rules/documentation.md` bans item numbers in documentation. The operator's directive of 2026-09-08 amends that rule for this header alone, and his newest word governs.

## 2026-09-06 18:26 - #407 - six zones, three down each side

The screen stopped being two panes with tables in them. Six zones now sit three down each side. ATA-SPM, Opposing Trades and Multi-Exchange Arbitrage run down the left. ATA-SPM Ready to Send, Bot Swarm Topologies and Phantom Bot HTF Signals run down the right. One function names each side in screen order, so a zone cannot appear on one host and go missing on the other.

`src/gui/main_tabs/market_inspector_surface.py` — the three left-side zones

```
def left_module_rows(
    run: Any,
    scan_state: Any,
    pair_count: Any,
    connectors: Any,
    sector_count: Any = 0,
) -> list:
    """The three left-side regions as key, title and status, in screen order."""
```

Every zone works the same way. It shows one entry at a time, with a left arrow, a right arrow and a line saying which entry is on screen out of how many the zone holds. An arrow press wraps at both ends, so the last entry steps forward to the first.

`src/gui/main_tabs/market_inspector_surface.py` — what one arrow press does

```
def step_to(at: Any, total: Any, by: Any) -> int:
    """The entry index one arrow press moves to, wrapping at both ends."""
```

Clicking the entry opens it. The open entry drops its method line and its hint, because the four expanded lines already say the same things, so every zone's open entry takes one height whatever buttons it carries. The four lines are the test, the window, the result, and why that answer let the entry through.

`src/gui/main_tabs/market_inspector_surface.py` — the four names an open entry carries

```
DETAIL_TEST_NAME = "Test"
DETAIL_WINDOW_NAME = "Window"
DETAIL_RESULT_NAME = "Result"
DETAIL_REASON_NAME = "Why it is here"
```

A zone holding nothing keeps its waiting sentence as the headline and still reports a position of "0 of 0". An empty zone therefore reads as a state rather than as a zone that failed to draw. Each zone says something

different while it waits.

| Zone | What its line says while it holds nothing |
|---|---|
| ATA-SPM | "No sector added. Name one and press Scan Now." |
| Opposing Trades | The scan state: not asked, running, or finished and empty |
| Multi-Exchange Arbitrage | The venues in reach, or that no exchange source is wired |
| ATA-SPM Ready to Send | "No run yet. Nothing to approve." |
| Bot Swarm Topologies | The proposal pane, which carries its own status line |
| Phantom Bot HTF Signals | "Phantom Bot source not wired." |

Phantom Bot HTF Signals is the renamed zone, and its line is fixed today. Nothing feeds it. The zone is named, sized and drawn, and the row that builds it writes the same sentence on every call, so no Phantom Bot reaches it yet.

`src/gui/main_tabs/market_inspector_surface.py` — the row that never varies

```
[PHANTOM_HTF_ZONE, PHANTOM_HTF_GROUP_TITLE, PHANTOM_HTF_UNWIRED_TEXT],
```

## 2026-09-06 21:07 - #407 - the sector row, Scan Now, and the phase readbacks

The ATA-SPM zone grew a control row. It carries a sector field, an asset-class box, four timeframe check boxes and a Scan Now button. The operator names a sector, ticks the timeframes he wants, and presses Scan Now. Every asset the sector holds is charted and run through the twelve voters.

`src/gui/main_tabs/market_inspector_surface.py` — the button and what it says

```
SCAN_NOW_LABEL = "Scan Now"
SCAN_NOW_TOOLTIP = (
    "Scan this sector now on the timeframes ticked beside it, without "
    "waiting for a rotation."
)
```

The asset class sets which four timeframes the check boxes offer. Crypto is the fast set. Stocks, metals, derivatives and forex share the slower one.

`src/trading/ata_spm.py` — the two sets

```
CRYPTO_TIMEFRAMES = ("5m", "1h", "1d", "1w")
SLOWER_TIMEFRAMES = ("1h", "1d", "1w", "1M")
```

Only crypto has a sector map in the tree. Every other class answers no assets, and the zone then names the source it is waiting for rather than showing an empty scan. The map itself is the hand-typed one the topology detectors read.

`src/gui/main_tabs/market_inspector_surface.py` — the one class with a map

```
def sector_assets(sector: Any, asset_class: Any) -> list:
    """The assets one named sector holds, read from the shipped sector map.

    Only ``ata_spm.CLASS_CRYPTO`` has a map in the tree, so every other
    class answers none and the zone names the source it waits for.
    """
```

Scan Now adds the typed sector when it is new, then runs the phases that read price data. The expanded entry names each phase back with what it produced, so a run can be read without opening a log.

`src/trading/ata_spm.py` — the phases one press runs

```
def run(
    sectors: Any,
    asset_source: Optional[Callable] = None,
    candle_source: Optional[Callable] = None,
    engine: Optional[VotingEngine] = None,
    message_format: Optional[str] = None,
    clock: Optional[Callable] = None,
) -> AtaSpmRun:
    """Phases one, two, three and eight in order, as one ``AtaSpmRun``."""
```

Eight phases carry a post from a sector name to a published call. Two modules hold them. The screen computes none of it and draws what they answer.

| Phase | Module | What it produces |
|---|---|---|
| 1 Evaluate | `ata_spm.evaluate` | One scan per sector, per ticked timeframe |
| 2 Identify | `ata_spm.identify` | The votes carrying a reversal, strongest first |
| 3 Pull | `ata_spm.pull` | The chart, its bands, and one message per confirming voter |
| 4 Format | `ata_spm_push.format_post` | One post per push target |
| 5 Distribute | `ata_spm_push.distribute` | One delivery record per post |
| 6 Ready to Send | `ata_spm_push.ReadyToSend` | The bucket the operator approves from |
| 7 Follow-Up | `ata_spm_push.FollowUpWatch` | What the market did after a published call |
| 8 Timeframes | `ata_spm.agreement_for` | Every timeframe the asset voted on, and their verdict |

A run answers a phase readback line for each of them. The readback names the sectors scanned, the reversal calls found, and the charts pulled.

`src/trading/ata_spm.py` — the line a finished run leaves

```
PHASE_RUN_FORMAT = "{phase}: {sectors} sector(s), {calls} call(s), {pulls} chart(s)"
```

## 2026-09-06 23:08 - #407 - Ready to Send, Post Selected, Post All and Full Auto

Nothing leaves the machine without an act. Phase four formats a post per target and drops it into the Ready to Send bucket, where it waits. The operator approves or declines each one, and no button sends a declined post.

`src/gui/main_tabs/market_inspector_surface.py` — the five buttons the bucket carries

```
APPROVE_LABEL = "Approve"
DECLINE_LABEL = "Decline"
POST_SELECTED_LABEL = "Post Selected"
POST_ALL_LABEL = "Post All"
FULL_AUTO_LABEL = "Send Bucket Full Auto"
```

Post Selected sends the post on screen. Post All sends every approved post. Send Bucket Full Auto releases approved posts with no further click. All three obey one ceiling, counted over the hour ending now.

`src/trading/ata_spm_push.py` — the ceiling every send route obeys

```
def allows(self, now: float, ceiling: Any) -> bool:
    """Whether one more post fits under ``ceiling`` at ``now``."""
    limit = int(ceiling or NO_CEILING_SET)
    if limit <= NO_CEILING_SET:
        return False
    return self.sent_within_hour(now) < limit
```

The ceiling starts unset, and an unset ceiling releases nothing. That is the safe default: a fresh install cannot publish until the operator sets a number, and the refusal says which of the two reasons held the post.

`src/trading/ata_spm_push.py` — the two refusals a held post reports

```
NO_CEILING_TEXT = "Max posts per hour is unset. Nothing leaves."
RATE_HELD_TEXT = "{sent} post(s) sent this hour, ceiling {ceiling}."
```

The Settings button swaps the zone for a settings page and back. Four settings sit there, each one read by a phase, and under them one credential row per push target.

| Setting | Which phase reads it |
|---|---|
| Max posts per hour | The send ceiling above |
| Max supporting indicators | Phase four, capping the evidence lines drawn |
| Confirmation share % | Phase seven, sizing the target a call must reach |
| Standardised message text | Phase three, wording each indicator message |

Save credentials encrypts every typed credential into the same vault the venue keys use, then clears what was typed. A credential row reports held or not held. No typed value reaches a view model, a render or a log.

`src/trading/ata_spm_push.py` — what a credential row publishes

```
CREDENTIAL_HELD_TEXT = "held"
CREDENTIAL_MISSING_TEXT = "not held"
```

## 2026-09-07 00:16 - #407 - Refresh, renamed

The topology pane's button read "Refresh proposals". It reads "Refresh" now, and the pane is inside the Bot Swarm Topologies zone, whose title already says what is being refreshed.

`src/gui/main_tabs/market_inspector_topologies_surface.py` — the label and what it does

```
REFRESH_TEXT = "Refresh"
REFRESH_TOOLTIP = "Rerun topology detectors on current market state."
```

## 2026-09-07 03:59 - #407 - the follow-up post and the timeframe vote

Phase seven watches a call after it is published and posts what happened. Three outcomes exist. A call is confirmed, failed, or still open.

`src/trading/ata_spm_push.py` — the three verdicts

```
OUTCOME_CONFIRMED = "confirmed"
OUTCOME_FAILED = "failed"
OUTCOME_OPEN = "open"
```

The target a call has to reach is a share of the run from the call's close to the Bollinger midline, and the midline is recomputed on every candle after the call rather than frozen at it. The share is the confirmation share setting. A share of zero sets no target, and the follow-up says so instead of claiming a result.

`src/trading/ata_spm.py` — the moving target phase seven measures against

```
def midline_after(candles: Any, at: Any) -> list:
    """The Bollinger middle band at every candle after index ``at``.

    ``BollingerBands`` computes its published band on each window, so the
    target phase seven measures against moves with the market.
    """
```

Phase eight answers whether the other timeframes agree with the call. It walks every vote the same asset produced in the run, names the direction each one read, and lists the timeframes that read the opposite. A neutral vote is neither agreement nor contradiction.

`src/trading/ata_spm.py` — the three verdicts phase eight can write

```
AGREEMENT_AGREED_TEXT = "Every timeframe agrees."
AGREEMENT_SINGLE_TEXT = "Only one timeframe voted."
AGREEMENT_CONTRADICTED_FORMAT = "Contradicted on {labels}."
```

Phase eight also caps a scan. Each sector takes only as many timeframes as the rounds so far show will fit inside one candle of the shortest timeframe the class scans. The rest are recorded as deferred and named on the zone, so a shortened scan is visible rather than silent.

## 2026-09-07 07:29 - #407 - seven push targets and the ceiling each publishes

Seven targets ship. Adding one is adding a row: no phase names a target, and the Settings credential list is built from the same rows, so a new row appears on the settings page with no screen edit.

| Target | Body ceiling | Title ceiling | Counted in |
|---|---|---|---|
| X | 280 | none | Weighted characters |
| Instagram | 2200 | none | Characters |
| LinkedIn | 3000 | none | Characters |
| TikTok | 4000 | 90 | UTF-16 runes |
| Facebook | none published | none | Characters |
| Threads | 500 | none | UTF-8 with emoji |
| Reddit | 40000 | 300 | Characters |

Every number came off the platform's own documentation, read on 2026-09-06 and 2026-09-07, and each one is cited on its own page. Where a platform publishes no number, the row records that rather than carrying a guess.

The push target rules — one section per target, with the page each number came from

```
Facebook   body ceiling: no number published
           the row carries NO_LIMIT_PUBLISHED, and phase four applies none
```

The fixed header ships on every artefact the run composes. No caller supplies it and no caller can remove it. It is 144 characters, and on X it takes over half the budget.

`src/trading/ata_spm_push.py` — the header that cannot be dropped

```
FIXED_HEADER = (
    "This is not investment advice. It is a demonstration of Ekthelius's "
    "proprietary TA engine housed in the Acervator governance execution "
    "platform."
)
```

Phase four holds each body under its target's ceiling. It drops evidence lines, longest first, and never the header. A body that still will not fit says by how much it missed rather than truncating the header to force it in.

`src/trading/ata_spm_push.py` — what the fitter drops, and what it will not

```
def fit_to_target(ranked: Any, target: Any) -> tuple:
```

Two platforms named in the issue are not rows, and both were removed for the same reason: a rendered chart has no route in. TradingView publishes an idea through its website and states that it has no API for it. The YouTube upload endpoint accepts video only.

**Where the issue and the code disagree, the code is right.** Issue #407's comment of 2026-09-07 records "The formatter applies no per-target ceiling and never did. The truncation rule is recorded and not built." That was true when it was written. Every target row now carries its own `body_limit`, and phase four applies it

through `fit_to_target`. The comment is stale and the ceilings are built.

## 2026-09-07 10:16 - #407 - the voting panel, the gate chain, and the two distances

ATA-SMP carries its own Indicator Voting Panel. It is a clone of the live one, drawn inside an open entry, and it reads only the markets ATA-SMP scanned. No live bot's market reaches it, and it reaches no live bot.

`src/gui/main_tabs/market_inspector_surface.py` — the panel and what feeds it

```
def voting_panel(pull: Any) -> Optional[dict]:
    """The Indicator Voting Panel one scanned asset carries, as its rows.

    The rows are the timeframes ATA-SMP read for this asset alone, and
    every cell is drawn from ``indicator_panel_surface``.
    """
```

The live trade gates also run over the scanned prices. Both chains the bots build are evaluated, unchanged, against a context filled from one market's candles and its voting summary. Nothing on the live path changes: the bots build the only two contexts inside their own tick, and this is a third caller.

`src/trading/ata_gate_scan.py` — both live chains, over scanned prices

```
readings = read_chain(build_scrumming_scrum_chain(), context) + read_chain(
    build_scrumming_fold_chain(), context
)
```

Twenty-two gates run across the two chains. Sixteen decide from price and settings alone. Six need a position the scan does not hold, and the scan never invents one. Four of the six stand down by name, and two publish a distance instead of a verdict.

`src/trading/ata_gate_scan.py` — the six, split by what a scan can honestly say

```
NOT_APPLICABLE_GATES = (
    "delta_positive",
    "interval",
    "tranches_queued",
    "smart_ceiling",
)
HYPOTHETICAL_GATES = ("hysteresis_scrum", "hysteresis_fold")
```

The four stood-down gates appear in the expansion by name, marked as not run, each with the reason. A reader sees that the gate exists, that it did not run, and why. That is the point: a post must never imply a gate latched when no gate ran.

`src/trading/ata_gate_scan.py` — the reason each stood-down gate publishes

```
NOT_APPLICABLE_REASONS = {
    "delta_positive": "a scan holds nothing, and no surplus exists to test",
    "interval": "the same surplus, and a scan holds none of it",
    "tranches_queued": "a scan has no fold queue",
    "smart_ceiling": "a scan holds nothing, and any ceiling test passes",
}
```

Opposing Trade Distance is the second of the two numbers the expansion carries. It is one sum: the bot's scrumming interval percentage plus its trading fee percentage, clamped between 0 and 50. It is a formula over settings, so a scan computes it exactly as a bot does.

`src/trading/otd_math.py` — the whole formula

```
total = float(interval_pct) + float(fee_pct)
return max(OTD_MIN_PCT, min(OTD_MAX_PCT, total))
```

The two hysteresis gates feed on it. Each runs against a hypothetical entry at the scanned price and publishes the price a reversal would have to reach, with no verdict attached. At the instant of the scan the pivot equals the price, so both sides would refuse; the number is the reading, and the refusal is not.

`src/trading/ata_gate_scan.py` — what a hypothetical row prints

```
HYPOTHETICAL_FORMAT = (
    "entry at ${price:.8f} would need ${required:.8f}, "
    "{distance:.2f}% away; no gate ran"
)
```

Landing Strip is the first of the two. The band-proximity detector reads whether price has sat against a Bollinger band for a run of candles, and it answers a side. An upper strip is a sell reversal and a lower strip is a buy reversal. The side and the candle count are both published on the expanded line beside the distance.

`src/gui/main_tabs/market_inspector_surface.py` — the line carrying both numbers

```
GATE_DISTANCE_FORMAT = "opposing trade distance {pct:.2f}% · landing strip {strip}"
```

A second strip detector runs inside the gate scan and feeds the confidence floor rather than the direction. It lowers the floor the TA consensus must clear, in the same way the live tick lowers it, so the scan and a bot judge a market at one bar. The one term a scan loses is the band-priority favour, which arms only on a signed delta, so a scan reads at a stricter floor than a bot holding a position.

A chart with too few candles produces no gate scan at all. The floor is 30 candles, and the line says so rather than reporting an empty result as a clean one.

`src/gui/main_tabs/market_inspector_surface.py` — the sentence a short chart carries

```
NO_GATE_TEXT = "No gate ran. The chart carried too few candles."
```

The counts sit at the head of the expansion, so a reader sees how much of the chain actually decided before reading any single row.

`src/gui/main_tabs/market_inspector_surface.py` — the count line

```
GATE_COUNT_FORMAT = (
    "{ran} of {total} gate(s) ran · {latched} latched · {blocked} blocked "
    "· {stood_down} did not run"
)
```

Which gates transpose, and why each one does, is measured field by field in the gate transposition page.

## 2026-09-08 - #407 - Push to Sim, Paper and Live is not built

This entry carries no time. No commit implements it, so there is nothing to read a time from.

The operator named a push from Opposing Trades and from Topologies to the Simulator, to the Paper Trader and to Live. No such control exists on either zone today. Adopt is the only control on this screen that creates anything, and it creates live bots through the wizard.

A simulator can read the proposals on display, and that is a read with no write behind it. It wires nothing and it starts nothing.

`src/gui/main_tabs/market_inspector_surface.py` — the read that exists instead

```
def current_topology_proposals(self) -> Optional[list]:
    """The proposals on display, for a simulator to read.

    ``None`` says the right pane never built or refused the read, and
    a list says the pane answered, so an empty pane and an absent one
    never look alike to a caller.
    """
```

Nothing reports a push to a target that is not built, because nothing pushes. The Paper Trader and the Simulator are both named on the promotion pipeline page, which is where a push from this screen would land.

In development.

## 2026-09-08 13:30 - #407 - the called markets the Charts tab keeps watching

Reaching Ready to Send now does a second thing. Beside waiting for approval, the market joins a list the Charts tab keeps, so the operator can watch a called market long after the post about it has gone or been declined.

The board answers one row per market, never one per post. A market called on two timeframes is still one market being watched, and the row carries the timeframes it was called on and the vote of the most recent call.

`src/trading/ata_spm_push.py` — the row the Charts tab reads

```
@dataclass(frozen=True)
class WatchedMarket:
    """One market on the ATA-SMP chart list, and the call that queued it."""

    symbol: str
    vote: str = VOTE_NEITHER
    timeframes: tuple = ()
```

Three places hold a called market and the board reads all three, oldest first: the calls phase seven has already settled, the calls it is still watching, and the posts sitting in the bucket now. That is why filling the bucket from a new scan does not drop an older market from the chart list.

`src/trading/ata_spm_push.py` — one set, read by phase seven and by the Charts tab

```
    def watched_markets(self) -> list:
        """One ``WatchedMarket`` per market that reached Ready to Send.

        The bucket is the trigger and ``follow_up`` keeps a market listed
        after ``load_run`` replaces the bucket, so the Charts tab and phase
        seven read one set. The newest call's vote wins.
        """
```

Phase seven used to remember only the key of a call it had settled, which was enough to stop watching it twice and not enough to name it later. It now keeps the call itself, so a settled market can still be listed with its ticker, its timeframe and its vote. The settled calls are read in key order, so the list comes back in the same order every time.

The screen hands that reader to the Charts tab when it is built. Nothing is copied across, so the two screens cannot disagree about which markets are called. What the Charts tab does with the list is on the Charts tab page.

`src/gui/market_inspector.py` — what this screen offers the Charts tab

```
def watched_markets(self) -> list:
    """The markets ATA-SMP has called, for the Charts tab's second list.

    ``PushBoard.watched_markets`` is the one set phase seven also reads.
    """
    return self._push_board.watched_markets()
```

**Figures.** This page carries no figure. The zones it describes are unchanged by this entry, and the second reader it adds draws on another screen.

## 2026-09-08 15:20 - #407 - phase three draws the chart the post carries

Phase three used to answer a data record: the ticker, the timeframe, the bar count, the band values and a list of confirming sentences. There was no picture, so the post's caption captioned nothing. It now draws that chart and writes it as a PNG, and the pull carries the file.

`src/trading/ata_spm.py` — the picture the pull now holds

```
@dataclass
class ChartPull:
    """The chart one reversal call was made on, and its confirming messages.

    ``image`` is that chart rendered to a PNG, which the post's caption
    captions.
    """
```

The picture is drawn by the Charts tab's own renderer. No second drawing engine was written for posts, because two engines drift apart and only one of them is the screen the operator watches. What changed in the renderer is that its drawing state and its paint routine moved out of the window into a plain object, `ChartPainter`. The Charts tab's chart is that object with a window around it, and a post image is that object with a picture file around it.

`src/gui/native_chart.py` — one paint routine, two places to send it

```
        def paint_to(self, p: QPainter, w: int, h: int) -> None:
            """Draw the whole chart onto ``p`` over a ``w`` by ``h`` area.

            The painter's device is the caller's: a widget from ``paintEvent``
            and a ``QImage`` from ``render_chart_png``.
            """
```

The split was forced by where a scan runs. Pressing Scan Now hands the work to a background worker so the screen keeps drawing, and the drawing toolkit will not build a window on a background worker. Driven, it does not raise an error that could be caught; it kills the program. A picture file has no such rule, so the post image is drawn straight onto the file and no window is made.

`src/gui/market_inspector.py` — the worker the scan runs on

```
            self._scan_thread = threading.Thread(
                target=self._compute_scan,
                args=(settings.message_format, settings.max_supporting_indicators),
                name=ATA_SCAN_THREAD_NAME,
                daemon=True,
            )
```

The indicators on the picture are the ones that voted for the call, not the ones the Charts tab happens to have switched on. Each overlay in the chart's registry now names the voter it draws, so the picture is chosen by the vote.

`src/gui/native_chart.py` — the overlay names its voter

```
    ChartOverlay(
        key="bb",
        label="BB",
        colour_field="chart_band",
        pane=PRICE_PANE,
        occludes=False,
        draw="_draw_bollinger",
        tooltip="Bollinger Bands (20, 2σ) with cloud fill",
        voter="bollinger_bands",
    ),
```

**Max supporting indicators** on the settings page decides how many of them are drawn. It was already the cap on how many confirming sentences a post carries, and it is now the same cap on the chart. No second setting was added. Set to one, the picture carries the Bollinger bands alone; left unset, it carries every confirming voter the renderer has an overlay for.

A confirming voter with no overlay cannot be drawn, and the picture says so in its header line rather than passing over it. Z-Score and RSI are the two that vote and have no overlay today.

`src/gui/native_chart.py` — what reached the picture and what did not

```
@dataclass
class ChartImage:
    """One rendered chart on disk, and what the renderer could not draw.

    ``drawn`` and ``undrawn`` name voters: ``undrawn`` holds the ones no
    overlay draws and the ones ``max_overlays`` cut.
    """
```

The files are kept outside the repository, in `~/.acervator_ata_posts`, beside the other runtime folders and inside none of them. One file is written per asset, per timeframe, per last bar. Nothing removes them yet.

`src/trading/ata_post_paths.py` — the one place the folder is named

```
ATA_POST_ROOT: Path = Path.home() / ".acervator_ata_posts"
```

Nothing sends the picture. The sender is handed the whole post, so the file travels with the text, and no sender reaches a platform. The Ready to Send zone draws the text and does not show the picture.

**Figures.** This page carries no figure, and this entry adds none: a rendered chart is produced output and is not kept in the repository. The picture drawn by the run behind this entry measured 1200 by 372 pixels and carried 7126 distinct colours, against 17 for the same chart drawn with no candles. Its readings are in the debug report.

## 2026-09-08 15:30 - #407 - the post picture folder has a ceiling

Every scan wrote a picture and nothing ever took one away. One picture measured 121044 bytes. A scan across sectors on four timeframes, repeated as bars close, reaches gigabytes on the machine the operator trades from, and nobody would notice until the disk did.

The folder now keeps the newest picture of each market and removes the older ones. A market is one asset on one timeframe, so gold on the hourly chart and gold on the daily chart are two markets and each keeps its own newest picture.

The number is not a preference. It is what the posting side can actually reach. Phase four looks up one chart per market and keeps the last one, and the Ready to Send list is rebuilt from scratch on every scan. Nothing in the application can name an older picture, so nothing loses one.

`src/trading/ata_post_paths.py` — how many stay, and why that number

```
#: ``ata_spm_push.format_run`` keys its pulls by symbol and timeframe, last
#: write winning, so one image per market is every image a post can name.
POST_IMAGES_KEPT_PER_MARKET = 1
```

The follow-up post was the thing to check before setting any ceiling, because a ceiling that throws away a picture a follow-up still needs is worse than no ceiling. It does not need one. Phase seven reads the chart again from the market data, never from the folder, and the post it writes is text: the original headline, the outcome, and the evidence behind it. Driven over three called markets, phase seven settled all three and composed twenty-one follow-up posts across the push targets. None of them carried a picture.

`src/trading/ata_spm_push.py` — phase seven asks the market, not the folder

```
    def check(self, candle_source: Any, share_pct: Any) -> list:
        """Read each watched call's chart again and answer what happened to it.

        A settled call stops being watched and an open one stays, so no call
        is posted on twice.
        """
```

The clearing happens on the same press that draws a picture. Scan Now runs the three phases, phase three writes the file, and the folder is trimmed immediately afterwards, so it is never larger than one scan's worth of pictures plus the newest of every market scanned before.

`src/trading/ata_spm.py` — the picture is written, then the folder is trimmed

```
    path = ata_post_paths.post_image_path(vote.symbol, vote.timeframe, stamp)
    image = render_chart_png(
        held,
        vote.symbol,
        timeframe_label(vote.timeframe),
        path,
        voters=[one.indicator for one in confirming_signals(vote)],
        max_overlays=int(max_supporting_indicators or NO_INDICATOR_CAP),
    )
    ata_post_paths.prune_post_images(path)
    return image
```

**It says what it took.** Deleting the operator's files quietly is not acceptable, so every trim writes one line naming how many pictures went, how many bytes came back, how many stayed, and how many the machine would not let go of.

`src/trading/ata_post_paths.py` — the line every trim writes

```
PRUNE_LOG = (
    "ATA post store: removed %d image(s), reclaimed %d byte(s), kept %d, refused %d"
)
```

**It cannot reach anything else.** It reads one folder, the one its own file names, and it goes no deeper. A file whose name a post picture could not have produced is not touched: three such files sat in the folder through a whole driven run and all three were still there at the end. The state folder, the log folder, the recorded market data and the paper ledger were counted before the run and after it and did not move, and the settings file hashed the same both times.

**A picture being written is never taken.** The protection is the operating system's own refusal, not a check that could be wrong. Driven with a picture held open, the deletion was refused, the file survived, and the trim reported it as refused. The same file went on the next press, once the handle had closed.

Driven over three markets with nine older pictures planted, the folder went from twelve files to six, and the three pictures the run had just drawn were the three that stayed. A second press on unchanged bars removed nothing.

```
                         files      bytes
before the first scan       12        174
after the first scan         6     478068
after a later scan           6     441483
after an unchanged scan      6     441483
```

**Figures.** This page carries no figure and this entry adds none. The folder is outside the repository and a rendered chart is produced output, so neither is kept here. The readings behind every number above are in the debug report.

## 2026-09-08 17:24 - #407 - the organization link, and the chart that says why

Two things changed. Every post now carries a link to the Acervator-LLC page on GitHub. The chart a post carries now states the call and shows what each voter read.

**The link is one string in one place.** One routine writes every piece of post text. It puts the fixed header on top, the evidence under it, and the link at the bottom. No caller supplies the link and no caller can remove it. The body, the caption, the thread root and the title all carry it, on every target.

`src/trading/ata_spm_push.py` — the link, written once

```
def compose(lines: Any) -> str:
    """``FIXED_HEADER`` over ``lines`` over ``ORGANIZATION_URL``.

    ``fit_to_target`` drops ``lines`` to reach a ceiling and reaches neither
    ``FIXED_HEADER`` nor ``ORGANIZATION_URL``.
    """
```

**The link costs characters, and each target has its own ceiling.** X counts a web address as 23 characters whatever its real length. The other targets count every character in it. Where a post no longer fits, the evidence

gives way and the link stays. The longest indicator sentence goes first, then the next longest, and the post says how many sentences were left out.

The numbers below are one run on a recorded Amazon daily chart. Measured is the post body in the unit that target counts in.

| target | ceiling | counted in | before the link | with the link | sentences dropped |
|---|---|---|---|---|---|
| X | 280 | weighted characters | 272 | 246 | 7 |
| Instagram | 2200 | characters | 727 | 760 | 0 |
| LinkedIn | 3000 | characters | 711 | 744 | 0 |
| TikTok | 4000 | UTF-16 runes | 765 | 798 | 0 |
| Facebook | none published | characters | 765 | 798 | 0 |
| Threads | 500 | UTF-8 emoji units | 470 | 431 | 4 |
| Reddit | 40000 | characters | 765 | 798 | 0 |

No target is over its ceiling. X keeps the ticker, the direction, and the note that the evidence was cut. That is all 280 characters hold once the fixed header and the link are on the post.

**The chart now states the call.** The renderer takes the direction the vote named and one sentence per confirming voter. It is the same renderer the Charts tab draws with, and a chart given no call takes neither.

`src/gui/native_chart.py` — the direction and the readings the picture takes

```
        def set_call(self, direction: str, readings=()) -> None:
            """Take one reversal direction and one reading line per voter.

            ``readings`` are ``(voter, text)`` pairs and ``_draw_call`` paints
            them under the time axis.
            """
```

**Three marks, and each one is earned.** A badge in the top right names the direction. A dashed rule and a triangle mark the last bar, which is the bar the vote was made on. A strip under the time axis carries one row per confirming voter: a square in the colour of the line that drew that voter, and the sentence phase three wrote for it. Nothing is drawn that a voter did not read.

A voter with no line on the chart still gets a row, in the dim colour, and the header line names it. ADX and Supertrend were the two on this run.

`src/gui/native_chart.py` — where the marks go

```
        def _draw_call(self, ctx, h: int) -> None:
            """Draw the reversal badge, the call bar mark and the voter strip.

            The bar marked is the last candle, which is the bar the direction
            ``set_call`` took was voted on.
            """
```

**The Charts tab is unchanged.** A chart with no call set draws no badge, no bar mark and no strip, and keeps the height it had before. The same recorded tape drawn both ways measured 344 pixels tall with no call and

390 with one.

**Figures.** This page carries no figure and this entry adds none. A rendered chart is produced output and is not kept in the repository. The picture drawn by the run behind this entry measured 1200 by 478 pixels and 164,402 bytes, carried 14,062 distinct colours, and 90.53% of its pixels are not its commonest colour. Its readings are in the debug report.

## 2026-09-08 19:10 - #407 - the scan reaches a market outside crypto

Scan Now now reads a real market that Acervator does not trade. The sector reader answers a named list of assets for FOREX and for metals, each name paired with the venue that lists it, and the prices arrive through the historical fetcher's own adapters.

`src/trading/ata_asset_maps.py` — one asset, and where it is carried

```
@dataclass(frozen=True)
class AssetListing:
    """One asset a sector holds, with the venue and ticker carrying it.

    ``quote`` is the currency the venue prices ``ticker`` in, which
    ``YahooChartAdapter.fetch_chunk`` checks its answer against.
    """

    symbol: str
    quote: str = USD
    venue: str = NO_VENUE
    ticker: str = ""
```

**FOREX is the class built end to end.** Its tiers are liquidity tiers. The major tier is the seven pairs that hold the dollar. The minor tier is every cross of two of those currencies with no dollar in it, twenty-one pairs. The exotic tier is defined and holds no pair, because no pair is named for it.

`src/trading/ata_asset_maps.py` — the two tiers that carry names

```
FOREX_MAJOR: tuple[AssetListing, ...] = tuple(
    _yahoo_fx(one)
    for one in (
        "EUR/USD",
        "USD/JPY",
        "GBP/USD",
        "USD/CHF",
        "AUD/USD",
        "NZD/USD",
        "USD/CAD",
    )
)

CROSS_ORDER: tuple[str, ...] = ("EUR", "GBP", "AUD", "NZD", "CAD", "CHF", "JPY")

FOREX_MINOR: tuple[AssetListing, ...] = tuple(
    _yahoo_fx(f"{base}/{quote}")
    for at, base in enumerate(CROSS_ORDER)
    for quote in CROSS_ORDER[at + 1 :]
)
```

**Every ticker was read before the map was written.** All 28 answered, 283 daily rows each. That is the vintage the map records, and it is what a reader checks the map against when a name stops answering.

**Metals reports an absence and scans nothing.** The four spot pairs are named per troy ounce against the dollar. The one configured non-crypto venue answers HTTP 404 for every spelling of all four, so the zone says which names have no venue and reads no prices at all.

`src/trading/ata_spm.py` — what the sector says when no venue carries it

```
UNLISTED_TEXT = "No configured venue lists {symbols}."
UNSERVED_TEXT = "No venue serves {labels}."
```

**A timeframe with no venue is reported the same way, and separately.** The adapter serves the daily timeframe alone, so a non-crypto sector reports the hourly, weekly and monthly boxes as unserved instead of showing them as timeframes that voted nothing. The crypto universe scan keeps daily and weekly candles, so its two fast boxes report the same way.

**A scan produced a call.** Twenty-one minor pairs voted on the daily timeframe. One carried a reversal, with the price at the lower band and the consensus agreeing with the band voter. Phase three pulled that chart, wrote its four confirming sentences and rendered the picture, and phase four filled the bucket with one post per push target.

```
NZD/CHF  bullish  net=+0.9582  conf=0.0879  band=0.0454  reversal=True
252 bars, 4 confirming voters, 7 posts in Ready to Send
```

**A venue row that is not a candle is dropped and counted.** The FOREX daily series carries an open or a close a few basis points outside its own high and low on about one row in twenty-four: 75 of 1,806 rows across the seven major pairs. Each of those rows is refused on its own, the rest are kept, and nothing is clamped or invented.

`src/trading/ata_asset_maps.py` — the conversion, one row at a time

```
def _candles_of(ticker: str, rows: Any) -> list:
    """Every row ``candles_from_raw`` accepts, one row at a time.

    A venue row whose open or close sits outside its own high and low is not
    a candle, and it is counted into ``VENUE_ROWS_REFUSED_LOG``.
    """
```

**Stocks and derivatives still answer nothing, and the map says why.** GICS names eleven sectors, and MSCI and S&P license the company membership behind them; no list of it sits in this tree. Derivatives has no classification named yet. Both entries sit in the map's own source table beside the two that are built.

**Two faults on the redraw were repaired in the same change.** A scan that produced a call raised on the way to the screen, twice, both from a trading-tab restyle that removed a helper and narrowed a signature the ATA-SMP voting panel still called. No non-crypto sector could answer an asset before, so no run had ever reached those two lines.

`src/gui/main_tabs/market_inspector_surface.py` — the tally, restored beside the panel that draws it

```
def panel_summary_text(rows: Any) -> str:
    """The vote tally beside a voting panel title, summed over its timeframes."""
    return PANEL_SUMMARY_FORMAT.format(
        bullish=sum(panel_tally(one, "bullish") for one in (rows or {}).values()),
        bearish=sum(panel_tally(one, "bearish") for one in (rows or {}).values()),
        neutral=sum(panel_tally(one, "neutral") for one in (rows or {}).values()),
    )
```

**Figures.** This page carries no figure and this entry adds none. The chart the run rendered is produced output and is not kept in the repository. It measured 1200 by 420 pixels and 149,167 bytes under a scratch post root, and the operator's own post folder took nothing from this run. Its readings, the venue census and the three program errors are in the debug report.

## 2026-09-09 02:10 - #407 - the four timeframes a non-crypto sector scans

A FOREX or metals sector is now scanned on all four timeframes the item names, not on the daily one alone. The check boxes are unchanged; what changed is that ticking three of them used to produce nothing.

`src/trading/ata_spm.py` — the four boxes each class carries

```
CRYPTO_TIMEFRAMES = ("5m", "1h", "1d", "1w")
SLOWER_TIMEFRAMES = ("1h", "1d", "1w", "1M")
```

**The limit was a constant in this tree, not a limit of the venue.** The chart adapter sent one interval word on every request because a module constant said so. The endpoint answers four, and it spells two of them differently from the way the engine names them.

`src/trading/stone_tablets/ra_fetcher.py` — the engine's word, and the endpoint's

```
YAHOO_INTERVALS: dict[str, str] = {
    "1h": "1h",
    RA_TIMEFRAME: "1d",
    "1w": "1wk",
    "1M": "1mo",
}
```

**Each timeframe reaches a different depth of history, and the hourly one is shortest.** That is normal for hourly data. Every figure below was read from the venue on 2026-09-09, for one currency pair.

```
1hr     286 rows      2026-08-24 to 2026-09-09        16 days
1d      248 rows      2025-09-10 to 2026-09-09       364 days
1wk     197 rows      2022-11-14 to 2026-09-09     1,395 days
1mnth   197 rows      2010-04-30 to 2026-09-09     5,975 days
```

**The venue refuses an hourly window longer than 729 days.** 729 answers and 730 returns an error. The adapter records that reach and moves the start of a longer request forward, so an over-long ask returns the candles that exist instead of nothing.

`src/trading/stone_tablets/ra_fetcher.py` — the days each interval reaches

```
YAHOO_REACH_DAYS: dict[str, int] = {
    "1h": 729,
    RA_TIMEFRAME: UNCAPPED_REACH_DAYS,
    "1w": UNCAPPED_REACH_DAYS,
    "1M": UNCAPPED_REACH_DAYS,
}
```

The other three answered a twenty-five year request in full. No limit was measured for them and none is recorded.

**A timeframe holding too few candles is reported, never voted.** The floor is thirty candles, the same number the gate scan already requires. Below it the sector line says how many assets were too short, and casts no vote for them.

`src/trading/ata_spm.py` — the floor, and the line it is reported on

```
MIN_CANDLES_TO_VOTE = ata_gate_scan.MIN_CANDLES_FOR_TA

TIMEFRAME_VOTE_FORMAT = (
    "{votes} vote(s), {unread} without candles, {short} under {floor} candles"
)
```

Driven with a one-day hourly window, which is under the floor, the zone drew this and produced no call.

```
Phase 1 Evaluate 1hr:   0 vote(s), 0 without candles, 7 under 30 candles
Phase 1 Evaluate 1d:    7 vote(s), 0 without candles, 0 under 30 candles
Phase 1 Evaluate 1wk:   7 vote(s), 0 without candles, 0 under 30 candles
Phase 1 Evaluate 1mnth: 7 vote(s), 0 without candles, 0 under 30 candles
```

The reason the floor exists is measurable. On twenty-five hourly candles, seven of the twelve voters abstained for want of history, and the remaining five still produced a direction.

**A run on the full window produced a call on the hourly timeframe.** Phase eight now has four timeframes to compare, so it can report a disagreement between them.

```
NZD/USD  1hr  bearish  net -1.0447  confidence 10%  286 bars
Timeframes: 1hr bearish · 1d bearish · 1wk bullish · 1mnth bearish.
Contradicted on 1wk.
```

**The sentence above about the adapter serving the daily timeframe alone describes the code before this change.** It is kept as written. The four rows in this section are what a non-crypto sector reports now.

**Crypto is unchanged and still scans two of its four.** The crypto universe scan keeps daily and weekly candles, so the five-minute and hourly boxes still report as unserved on a crypto sector. That row of the rotation is separate work.

`src/trading/ata_asset_maps.py` — the crypto venue row, untouched

```
VENUE_EXCHANGE: (RA_TIMEFRAME, "1w"),
```

**Figures.** This page carries no figure and this entry adds none. Every figure the page already carries is kept; there are none, and the four earlier entries that say so are unchanged. The chart the run rendered is produced output and is not kept in the repository. The readings, the venue measurements and the six program errors are in the debug report.

## 2026-09-09 04:40 - #407 - metals scans a listed instrument, and a voter says when it abstains

A metals sector now returns votes. It used to return one line saying no venue lists anything, and that line is still there beside the votes.

**The four spot pairs stay in the map, and they stay unlisted.** Gold spot is a dealer market with no public listing, so no free venue carries it. All four spellings answered HTTP 404 again on 2026-09-09. What the map lacked was the instrument a venue does carry for each of those four metals.

`src/trading/ata_asset_maps.py` — the listed instrument for each metal

```
METALS_PHYSICAL: tuple[AssetListing, ...] = tuple(
    AssetListing(symbol=one, quote=USD, venue=VENUE_YAHOO, ticker=one)
    for one in ("GLD", "SLV", "PPLT", "PALL")
)
```

**The map carries the funds and not the futures.** Both answer a full year of daily bars. A futures chart longer than a few months joins two or more contracts together, and every join is a price step nobody traded. This platform sells against a dollar target, so an invented step moves every level behind it. A fund holds the metal, prices in dollars, and has no expiry and no roll.

The futures also carry gaps in their own volume. Measured on 2026-09-09, over 275 daily bars each:

```
GC=F      4 bars with no volume        GLD    0 bars with no volume
SI=F      7 bars with no volume        SLV    0 bars with no volume
PL=F    132 bars with no volume        PPLT   0 bars with no volume
PA=F    129 bars with no volume        PALL   0 bars with no volume
```

**The sentence above saying metals scans nothing describes the code before this change.** It is kept as written. The rows below are what a metals sector reports now, from a run driven through Scan Now on 2026-09-09.

```
assets scanned : GLD  SLV  PPLT  PALL
unlisted       : XAU/USD  XAG/USD  XPT/USD  XPD/USD

1hr    4 votes    78 bars each    2026-08-24 to 2026-09-08
1d     4 votes   250 bars each    2025-09-10 to 2026-09-08
1wk    4 votes   201 bars each    2022-11-14 to 2026-09-08
1mnth  4 votes   198 bars each    2010-05-01 to 2026-09-08

Phase 3 Pull: 1 sector(s), 2 call(s), 2 chart(s)
```

**Every FX pair the map names sends volume 0, and one voter now says so.** The venue sent no volume on any of 2,676 daily bars across ten pairs. Two of the twelve voters read volume. The volume voter already abstains on such a series, and a run on a market that does send volume proves it still votes.

`src/trading/indicators/zscore.py` — which of the two values the reading holds

```
smoothed = _vwma(z_series, z_volumes, self.smoothing_period)
z_volume_weighted = smoothed is not None
z = z_raw if smoothed is None else smoothed
```

**No currency-future volume is served as a pair's volume.** The seven CME contracts do carry real volume on the same endpoint, and a contract's volume is not the pair's. Spot currency trading has no single published volume anywhere. An abstention states what is true; a stand-in number would not.

**The daily open is kept in every formula that reads it.** Three read it: the volume voter, Heikin Ashi and the landing strip. The open on a currency bar sits well away from the previous close, and a fund's open sits further away still, as a share of its own day's range.

```
FX spot mean       gap to previous close = 0.587 of the bar range
metal fund mean    gap to previous close = 0.700 of the bar range

rows where the open falls outside its own bar    FX   10 of 1,981
rows where the close falls outside its own bar   FX   79 of 1,981
                                                 fund  0 of 1,096
```

A fund opens at an auction, so its open is a traded price beyond doubt, and it gaps more than a currency open does. A gap of this size is what any daily bar does when the market stops trading between bars. Each of the three formulae is the published one, and none of them changed.

**Nothing the gate chain decides moved.** Both chains ran over eleven markets before and after, at 21 gate readings each.

```
gate verdicts compared    231
gate verdicts changed       0

metals gate readings before    0, the sector scanned no asset
metals gate readings now      84
```

**Figures.** This page carries no figure and this entry adds none. Every figure the page already carries is kept; there are none, and the five earlier entries that say so are unchanged. The charts the run rendered are produced output and are not kept in the repository. The venue measurements, the two voters and the control behind the zero above are in the debug report.

## 2026-09-09 05:10 - #407 - one post per ticker per hour, and the three-candle follow-up

Two rules hold a repost back, and they are separate. The first is the hour cap. A ticker reaches one push target once an hour, whatever composed the post. The Ready to Send bucket keeps one guard, so a second call on the same ticker is refused exactly like a repeat of the first.

`src/trading/ata_spm_push.py` - the hour the guard measures

```
def allows(self, now: float, symbol: Any, target: Any) -> bool:
    """Whether an hour has passed since ``symbol`` last reached ``target``."""
    gap = self.since(now, symbol, target)
    return gap is NO_SEND_RECORDED or gap >= SECONDS_PER_HOUR
```

Every send route reaches it. Post Selected, Post All and Send Bucket Full Auto run through phase five, and a post the cap holds records the reason beside the target it did not reach.

`src/trading/ata_spm_push.py` - what a held post reports

```
REPOST_HELD_FORMAT = (
    "{symbol} reached {target} {minutes:.0f} minute(s) ago; "
    "one post per ticker per hour."
)
```

The second rule is the follow-up, and only a settled call writes one. Price moving the favourable way as far as the confirmation target confirms the call. The trend carrying on for three candles in a row fails it. The two outcomes are not the same shape, because a continuation is evidence against a reversal call.

`src/trading/ata_spm_push.py` - the candles a failure needs

```
CONTINUATION_CANDLE_FLOOR = 3
```

The count is in the call's own timeframe. Three candles on a daily call is three days, and on an hourly call three hours. No wall clock is read, so a daily call cannot fail on the afternoon it was made. The earlier number was two, and it is stale.

A call short of three candles is not ready. The follow-up then says how far the trend has run against the call instead of claiming a result, and no candle is invented to reach the floor.

`src/trading/ata_spm_push.py` - the wording a call under the floor carries

```
FOLLOW_UP_NOT_READY_FORMAT = (
    "the trend has held for {run} of the {floor} candles a failure needs, "
    "last close {close:g}, target {target:g} not reached"
)
```

The minimum percent is the confirmation share the settings page already carries, listed in the settings table above. No second percentage was added, and no threshold deciding whether something publishes is written into the code.

`src/gui/main_tabs/market_inspector_surface.py` - the setting phase seven reads

```
SETTING_CONFIRMATION_SHARE = "confirmation_share_pct"
```

**Figures.** This page carries no figure and this entry adds none. Every figure the page already carries is kept; there are none, and the seven earlier entries that say so are unchanged. The refusals measured on the send path, the two-candle and three-candle cases and the debugger stack are in the debug report.

## 2026-09-09 06:40 - #407 - the live gate chain decides what is charted

One judgement decides whether a market is charted and posted, and it is the live trade gate chain. If the chain says Acervator would take the trade, the chart is drawn and the post joins the Ready to Send bucket. If it does not, no chart is drawn and nothing reaches the bucket.

The verdict is read off the chains the running bots build, not off a copy of their rules. A chain fires when every gate of it that could run latched.

`src/trading/ata_gate_scan.py` - the one verdict

```
    @property
    def would_fire(self) -> bool:
        """True while either chain would fire, which is what a post needs."""
        return self.firing_side != NO_SIDE
```

The confidence is the chain's own. `confidence_floor` is the floor the live tick reads, and no second threshold is written anywhere on the posting path.

The chart is drawn after the verdict, never beside it. A market the chains refuse writes no picture at all.

`src/trading/ata_spm.py` - what is drawn and when

```
    image = (
        render_pull_image(vote, candles, max_supporting_indicators, messages)
        if gates.would_fire
        else ChartImage()
    )
```

A refused market is still on the record. Its gate scan is kept, so a reader sees the market was judged, which gates blocked it and why.

`src/trading/ata_spm.py` - where a refusal is kept

```
        else:
            found.refused.append(held.gates)
```

The ATA-SPM zone's line now ends with how many markets the gates refused, beside the sectors, the calls and the charts.

`src/trading/ata_spm.py` - the line the zone reads

```
PHASE_RUN_FORMAT = (
    "{phase}: {sectors} sector(s), {calls} call(s), {pulls} chart(s), "
    "{refused} refused by the gates"
)
```

The reversal vote is still read and still shown. It is the sector readback, in the "reversal call(s)" count each sector carries, and it no longer decides what publishes.

An open sector entry lists the charts of that sector's own assets. It used to list only the charts whose market was also a reversal call, which would have left the line counting charts the expansion never showed.

`src/gui/main_tabs/market_inspector_surface.py` - the charts one entry lists

```
    assets = set(scan.assets)
    held = [one for one in pulls if one.symbol in assets]
```

**Figures.** This page carries no figure and this entry adds none. Every figure the page already carries is kept; there are none, and the eight earlier entries that say so are unchanged. The markets measured, the gates that latched on the one that fired, the gates that blocked the 127 that did not, and the picture count before and after the run are in the debug report.

## 2026-09-14 17:34 - #23 - SM Accounts, and one sign-in page per venue

The Settings button on the ATA-SPM control row opens a page of buttons. The page takes the whole left column, so nothing is squeezed into a strip and no scroll bar appears. This page is Level 1.

`src/gui/main_tabs/market_inspector_surface.py` - the two pages the zone shows

```
LEVEL_ONE = "level-1"
LEVEL_ONE_A = "level-1a"
```

Level 1 carries three groups. SM Accounts holds one button per push target. Asset Category holds one button per asset class, and the pressed one is the class a scan uses. Settings holds the four values a phase reads. A Back button returns to the three scan zones.

`src/gui/main_tabs/market_inspector_surface.py` - the three group names

```
SM_ACCOUNTS_TITLE = "SM Accounts"
ASSET_CATEGORY_TITLE = "Asset Category"
ATA_SETTINGS_TITLE = "Settings"
```

A press on a venue button opens that venue's own sign-in page. That page is Level 1A. It shows the venue name, the address it posts to, the permissions it asks for, and one box per value that venue needs. It shows no other venue's boxes.

`src/trading/ata_spm_push.py` - what one press opens

```
    def open_credentials(self, target: Any) -> Optional[str]:
        """Show one push target's Level 1A page, and answer which target it draws."""
        found = push_target(target)
        if found is None:
            return None
        self.settings_open = True
        self.credential_target = found.name
        self.connect_result = None
        return self.credential_target
```

Each venue asks for different values. Every value below comes from that platform's own published documentation, read on 2026-09-06 and 2026-09-07 and recorded in the platform rules audit.

| Venue | Boxes on its page | The sign-in issues | Posts to |
|---|---|---|---|
| X | Client ID, Client secret | Access token, Refresh token | `https://api.x.com/2/tweets` |
| Instagram | App ID, App secret | Instagram user id, Access token | `/<IG_ID>/media` then `/<IG_ID>/media_publish` |
| LinkedIn | Client ID, Linkedin-Version | Access token | `https://api.linkedin.com/rest/posts` |
| TikTok | Client key, Client secret, Verified URL prefix | Open id, Access token, Refresh token | `/v2/post/publish/content/init/` |
| Facebook | App ID, App secret | Page id, Page access token | `/<page_id>/feed` and `/<page_id>/photos` |
| Threads | App ID, App secret | Threads user id, Access token | `/<threads-user-id>/threads` then `/threads_publish` |
| Reddit | App ID, App secret, Subreddit, User agent | Access token, Refresh token | `https://www.reddit.com/api/v1/access_token` then `/api/submit` |

The operator types only what a venue hands him when he registers an application, plus values local to him. Every token in the third column is obtained by the sign-in and never typed.

Connect signs in to the venue. The page returns to Level 1 by itself only when the venue accepts. An empty box, a missing sign-in route and a refusal from the venue each keep the page open and print what failed.

`src/trading/ata_spm_push.py` - the four wordings Connect can print

```
CONNECT_OK_FORMAT = "{target} accepted the credential."
CONNECT_FAILED_FORMAT = "{target} refused the sign-in: {error}"
MISSING_FIELD_FORMAT = "{label} is empty."
NO_CONNECTOR_FORMAT = "No sign-in route wired for {target}."
```

The credential reaches the vault only after the venue accepts it, and each value is held under its own entry. A venue reads as held when every one of its entries is present. Nothing typed on the page reaches a view model, a render or a log.

`src/trading/ata_spm_push.py` - how one venue's values are keyed

```
VAULT_KEY_FORMAT = "{target}:{field}"
```

Every platform above issues these values only to an application the operator has registered with that platform. The software cannot obtain a registration, and each page names what is needed before any of its boxes can be filled. The sign-in does obtain the access and refresh tokens, once a registration supplies the client values the page asks for.

| Venue | What the operator registers |
|---|---|
| X | An X developer app with OAuth 2.0 user authentication and a callback address |
| Instagram | A Meta app with Instagram Login, on a professional account |
| LinkedIn | A LinkedIn developer app carrying the Community Management API |
| TikTok | A TikTok developer app with Content Posting, and a verified address prefix |
| Facebook | A Meta app with the Pages API, and a Page it may post to |
| Threads | A Meta app with the Threads API, on a Threads profile |
| Reddit | A Reddit app at reddit.com/prefs/apps, and a target subreddit |

A sign-in route is wired for every venue, at both construction sites. Connect opens the system browser at that venue's own approval address, waits on a loopback listener for the reply, and exchanges the code for the tokens the third column names. A venue with no registration behind it still refuses, and the page stays open and prints what failed.

`src/trading/ata_spm_push.py` - the call that wires a venue

```
    def set_connector(self, connector: Optional[Callable]) -> None:
        """Take what signs one push target in, or None while no route is wired."""
        self.connector = connector
```

**Figures.** This page carries no figure and this entry adds none.

## 2026-09-14 18:55 - #23 - Each venue's own sign-in, behind Connect

All seven venues use three-legged OAuth. Not one of them issues a working token from an app id and a secret alone. The operator sends himself to the venue in a browser, approves there, and the venue sends a code back to the program.

The boxes on each Level 1A page therefore hold only what a venue hands him when he registers an app, plus values that are local to him. The program obtains the rest and he never types it.

`src/trading/ata_spm_push.py` - the two sets each venue row now declares

```
    fields: tuple = ()
    issued: tuple = ()
```

| Venue | Boxes he types | What the program obtains |
|---|---|---|
| X | Client ID, Client secret | Access token, Refresh token |
| Instagram | App ID, App secret | Instagram user id, Access token |
| LinkedIn | Client ID, Linkedin-Version | Access token |
| TikTok | Client key, Client secret, Verified URL prefix | Open id, Access token, Refresh token |
| Facebook | App ID, App secret | Page id, Page access token |
| Threads | App ID, App secret | Threads user id, Access token |
| Reddit | App ID, App secret, Subreddit, User agent | Access token, Refresh token |

A venue reads as held on Level 1 only once both sets are in the vault, so the button turns on when the sign-in has actually run.

### The connection method

Every route is RFC 8252, the published method for a program on a desktop. The program opens a listener on the loopback address, opens the venue's own approval page in the operating system's browser, and waits for one reply.

Diagram: diagram

```
sequenceDiagram
participant He as Operator
participant App as Acervator
participant Br as System browser
participant V as Venue
He->>App: Connect
App->>App: bind 127.0.0.1 at a port the system chooses
App->>Br: open the venue's approval page
Br->>V: approve
V->>App: redirect to 127.0.0.1 with a code
App->>App: check the state it generated, then close the listener
App->>V: exchange the code for a token
V->>App: access token
App->>App: encrypt into the vault, return to Level 1
```

The listener binds `127.0.0.1` and nothing else. It takes a port the operating system picks, serves exactly one request, refuses a reply carrying a state it did not generate, and closes. It never outlives one sign-in.

`src/trading/ata_spm_signin.py` - what the listener binds

```
LOOPBACK_HOST = "127.0.0.1"
EPHEMERAL_PORT = 0
```

X, LinkedIn and TikTok require PKCE for a desktop program, so those three send a `code_challenge` on the approval call and a `code_verifier` on the token call.

### Where each venue sends him

| Venue | Approval page | Token exchange |
|---|---|---|
| X | `https://x.com/i/oauth2/authorize` | `https://api.x.com/2/oauth2/token` |
| Instagram | `https://www.instagram.com/oauth/authorize` | `https://api.instagram.com/oauth/access_token` |
| LinkedIn | `https://www.linkedin.com/oauth/native-pkce/authorization` | `https://www.linkedin.com/oauth/v2/accessToken` |
| TikTok | `https://www.tiktok.com/v2/auth/authorize/` | `https://open.tiktokapis.com/v2/oauth/token/` |
| Facebook | `https://www.facebook.com/v25.0/dialog/oauth` | `https://graph.facebook.com/v25.0/oauth/access_token` |
| Threads | `https://threads.com/oauth/authorize` | `https://graph.threads.com/oauth/access_token` |
| Reddit | `https://www.reddit.com/api/v1/authorize` | `https://www.reddit.com/api/v1/access_token` |

Three venues take a second call after the exchange. Instagram and Threads trade the first token for one that lasts sixty days. Facebook trades for a long-lived user token, then reads the Page token off the account.

Four things differ enough that one shared call would be wrong on all four. Reddit authenticates with HTTP Basic and refuses a generic `User-Agent`. TikTok names its client field `client_key`, not `client_id`. Reddit asks for `duration=permanent`, or it issues no refresh token. LinkedIn's native address takes no client secret at all.

### What renews a token without him

| Venue | Renews by itself | How |
|---|---|---|
| X | yes | refresh token grant, which needs `offline.access` |
| Reddit | yes | refresh token grant |
| TikTok | yes | refresh token grant, valid 365 days |
| Instagram | yes | `ig_refresh_token`, token at least 24 hours old |
| Threads | yes | `th_refresh_token`, token at least 24 hours old |
| Facebook | not needed | a long-lived Page token carries no expiry date |
| LinkedIn | no | partner-only; he approves again every 60 days |

### What each venue demands before Connect can work

Each Level 1A page now carries this line under its registration line.

| Venue | Review | Cost | What he must do first |
|---|---|---|---|
| Instagram | none | free | a professional account on a Page, Page Publishing Authorization, and a role on his own app |
| Facebook | none | free | a Page he administers, and a role on his own app |
| Threads | none | free | a Threads profile, and a role on his own app |
| Reddit | none | free | register an app, pick a subreddit |
| TikTok | audit for a public post | free | until the audit, only he can see what it posts |
| X | none | per post | a paid usage plan; about $0.015 a post, about $0.20 with a link |
| LinkedIn | two tiers | free if granted | a registered company, a verified Page, a screencast, and LinkedIn must switch on its native flow by hand |

Meta grants Standard Access to every permission automatically, and it reaches any account holding a role on the app. Instagram, Facebook and Threads therefore need no App Review, because he posts to accounts he owns.

TikTok restricts every post an unaudited client makes to private viewing. The post lands, and nobody but him sees it. Its page says so.

LinkedIn is the one venue that may refuse him outright. Its page states both gates rather than offering a Connect that looks like it will work.

### Nothing typed and nothing obtained reaches a page

`credential_page` publishes each box as a name and a wording, and no value.

`src/gui/main_tabs/market_inspector_surface.py` - what one box publishes

```
            "fields": [[one.key, one.label] for one in found.fields],
```

Both variants draw the same seven pages. Read off the running screens, each venue draws the same six lines and the same boxes under Qt and under React.

**Figures.** This page carries no figure and this entry adds none.

## 2026-09-14 20:21 - #23 - The redirect address each venue accepts

Every route sent the same address, `http://127.0.0.1:<port>`, at a port the operating system picked. Each venue's own documentation was then read for what that venue accepts. Three answers came back, and only one of them is the address the routes were sending.

### What each venue accepts, in its own words

| Venue | What it accepts | The sentence, from that venue's page |
|---|---|---|
| X | a loopback address at a fixed port | "For local development, use `http://127.0.0.1` (not `localhost`)" and "URLs must match exactly (including trailing slashes)" |
| Instagram | a loopback address at a fixed port | "Make sure this exactly matches one of the base URIs in your list of valid OAuth URIs you set during API setup in the App Dashboard." |
| LinkedIn | a loopback address at any port | "The lookback IP representation supported for IPv4 is `http://127.0.0.1:{port}` while for IPv6 is `http://[::1]:{port}`. We will be supporting both HTTP and HTTPS loopback IPs." |
| TikTok | a loopback address at any port | "URIs must have a port number, and wildcard port number (`*`) is supported. Wildcard config is recommended if your redirect URI uses a random port number." |
| Facebook | its own published desktop address | "If you are using this in a webview within a desktop app, this must be set to `https://www.facebook.com/connect/login_success.html`" |
| Threads | a loopback address at a fixed port | "Make sure this exactly matches one of the base URIs in your list of valid OAuth URIs." |
| Reddit | a loopback address at a fixed port | "Yes, you need it here again, and yes, it must match exactly." |

Sources, each read from the venue that published it:

- X - https://docs.x.com/resources/fundamentals/developer-apps
- Instagram - https://developers.facebook.com/docs/instagram-platform/instagram-api-with-instagram-login/business-login
- LinkedIn - https://learn.microsoft.com/en-us/linkedin/shared/authentication/authorization-code-flow-native
- TikTok - https://developers.tiktok.com/doc/login-kit-desktop
- Facebook - https://developers.facebook.com/docs/facebook-login/guides/advanced/manual-flow/
- Facebook HTTPS - https://developers.facebook.com/docs/facebook-login/security/
- Threads - https://developers.facebook.com/docs/threads/get-started/get-access-tokens-and-permissions
- Reddit - https://github.com/reddit-archive/reddit/wiki/OAuth2

### The port is the part that was wrong

A port the operating system picks is different on every press. Four venues check the redirect against a registered address character for character and publish no wildcard, so no registered address can ever match. Those four bind one declared port instead, and Level 1A prints it.

`src/trading/ata_spm_signin.py` - the two ports a venue can take

```
EPHEMERAL_PORT = 0
FIXED_CALLBACK_PORT = 8723
```

LinkedIn asks a native client for a random port, and TikTok publishes a wildcard port for that case. Those two keep the operating system's port.

### Facebook opens no listener

Meta requires HTTPS for an OAuth redirect and publishes one desktop address. A loopback address is neither published nor excepted, so Facebook's route opens no listener at all.

Diagram: diagram

```
sequenceDiagram
participant He as Operator
participant App as Acervator
participant V as Facebook
He->>App: Connect
App->>V: open the approval page in a sign-in view
V->>App: land the view on login_success.html
App->>App: read the fragment of the address it landed on
App->>V: trade the short-lived token for a long-lived one
V->>App: Page id and Page token
```

Meta answers a desktop app in the fragment of that address, and the fragment carries a token rather than a code. Facebook's route therefore asks for a different `response_type` and trades the token up.

`src/trading/ata_spm_signin.py` - what Facebook asks for

```
DESKTOP_RESPONSE_TYPE = "token"
FACEBOOK_DESKTOP_REDIRECT = "https://www.facebook.com/connect/login_success.html"
```

A fragment never leaves the browser, so only a view the program draws can read one. No such view is wired, and Facebook's Connect says so on the page rather than waiting three minutes for a reply that cannot come.

### What Level 1A prints

Each page carries one more line, under the line naming where Connect sends him.

| Venue | Redirect address to register |
|---|---|
| X | `http://127.0.0.1:8723/callback` |
| Instagram | `http://127.0.0.1:8723/callback` |
| LinkedIn | nothing; its native page asks for no registered address |
| TikTok | `http://127.0.0.1:*/callback` |
| Facebook | `https://www.facebook.com/connect/login_success.html` |
| Threads | `http://127.0.0.1:8723/callback` |
| Reddit | `http://127.0.0.1:8723/callback` |

Read off the running screens, all seven venues draw that line with the same value under Qt and under React.

### Two questions the venues do not answer

Meta states no scheme rule on the Instagram page or on the Threads page, and publishes no localhost exception on either. Both pages state only the exact-match rule, which the fixed port now satisfies. Whether Meta accepts a plain-HTTP loopback address for those two is settled by neither page.

Meta's device flow takes no redirect at all, and its own page ties its scope to Login Review without naming which permissions it accepts. Whether it carries `pages_manage_posts` is settled by no Meta page, so no route uses it.

**Figures.** This page carries no figure and this entry adds none.

## 2026-09-14 21:35 - #23 - The scan page, three lines of buttons

The ATA-SPM zone drew every control on one line: the sector field, its class, four check boxes, Scan Now and Settings. That line asked for 666 px where the zone gives 415 px at a 900 px tab, so the page pushed Scan Now and Settings past the pane's right edge and grew a horizontal scroll bar. Settings is the only way in to Level 1, so neither page could be reached.

The zone now draws three lines. The sector field takes the slack on the first, beside its class. The four timeframes follow, under their own heading, as a grid that wraps after `BUTTON_COLUMNS` the way the Level 1 groups do. Scan Now and Settings sit on the third.

`src/gui/main_tabs/market_inspector_surface.py` - the three lines a reader can name

```
TIMEFRAME_TITLE = "Timeframe"
SECTOR_ROW_PART = "sector-row"
SCAN_ROW_PART = "scan-row"
```

Read off the running page, the widest line is now the timeframe grid at 274 px, so every control sits inside the pane from 700 px of tab width upward and no scroll bar appears.

### The timeframes are buttons, and a press sticks before a scan

Each timeframe is a button that draws on while that timeframe is ticked, which is what the Asset Category buttons on Level 1 already do. A press before any sector exists is held on the board and given to the sector the next Scan Now adds, so the four buttons report what the scan will use rather than a row of empty boxes.

`src/trading/ata_spm.py` - what a press writes while the board holds no sector

```
        ticked = tuple(one for one in timeframes_for(asset_class) if one in held)
        if sector is None:
            self.timeframes = ticked
        else:
            sector.timeframes = ticked
```

A fresh board carries the whole set its class lists, so all four draw on until one is pressed off. Choosing an asset class puts that class's own four back.

### One pressed look, in both builds

A venue whose credential is held, the asset class a scan uses and a ticked timeframe are the same state, and each now paints `PRIMARY` under `ON_PRIMARY` text in both builds. The window sets it as a style sheet and the page selects the attribute its buttons carry.

`src/gui/main_tabs/market_inspector_surface.py` - the pressed look

```
CHECKED_BUTTON_STYLE = (
    f"QPushButton:checked{{background:{ds.PRIMARY};color:{ds.ON_PRIMARY};"
    f"border:1px solid {ds.PRIMARY};}}"
)
```

`src/gui/web/market_inspector.css` - the same two tokens on the page

```
[data-scan-state] {
  background: var(--PRIMARY, var(--accent));
  color: var(--ON_PRIMARY, var(--bg));
  border-color: var(--PRIMARY, var(--accent));
}
```

**Figures.** This page carries no figure and this entry adds none. The two widths quoted above were read from the running page and are not stored.

## 2026-09-14 22:18 - #23 - Level 1 and Level 1A fit the pane at every width

Level 1 broke its buttons after four and its settings rows after two, whatever the pane's width. The page asked for 638 px where the zone gives 435 px at a 900 px tab, so the TikTok button, the derivatives button and two of the four settings rows sat past the pane's right edge and the zone grew a horizontal scroll bar. Level 1A broke the same way on `Client secret`.

Both pages now break each group at the count the pane's own width holds.

`src/gui/main_tabs/market_inspector_surface.py` - the one call both builds ask

```
def columns_for(
    available: int, cell: int, spacing: int = SETTINGS_ROW_SPACING_PX
) -> int:
```

The window calls it every time the pane's edge moves, and the page reaches the same count by taking the pane's width less one gap. Read off the running program, the pane holds 335, 435, 685 and 965 px at tab widths of 700, 900, 1400 and 1960, and every group measures `scrollWidth` equal to `clientWidth` at all four. No control sits outside the pane and no scroll bar appears.

| Tab width | Venue and category buttons on a line | Settings rows on a line |
|---|---|---|
| 700 | 2 | 1 |
| 900 | 3 | 1 |
| 1400 | 6 | 2 |
| 1960 | 8 | 2 |

Every button, label and value the two pages carried is still there. The seven venues, the five asset classes and the four settings each keep their own control, and the labels read in full.

### The window now sets the handle the page already drew

`SPLITTER_HANDLE_PX` is the gap between the two panes. The page drew it; the window left the theme's own 5 px in place, so each Qt pane came out 1 px wider than the same pane on the page. At a 1960 px tab that one pixel is a whole column, because 966 px holds three settings rows and 965 px holds two.

`src/gui/market_inspector.py` - the call that makes the two panes one width

```
            self._outer_splitter.setHandleWidth(SPLITTER_HANDLE_PX)
```

Both panes now read 335, 435, 685 and 965 px at the same four tab widths, so a group cannot wrap at a different count in one build than the other.

### A value wider than its box starts at its first character

A settings box holding a value wider than itself showed the end of that value in the window and the start of it on the page. `QLineEdit.setText` leaves the cursor past the last character, which scrolls the box. The window now puts the cursor back at the start, so both builds show the same characters.

**Figures.** This page carries no figure and this entry adds none. Every width quoted above was read from the running program and is not stored.

### One sentence this entry contradicts

The Level 1 entry above says the page "takes the whole left column, so nothing is squeezed into a strip and no scroll bar appears." That was not true of the running program when it was written: at a 900 px tab the zone carried a horizontal scroll bar and six controls sat outside the pane. It is true now.

## 2026-09-14 23:18 - #23 - Facebook signs in inside the program

Facebook was the one venue whose Connect could not finish. Meta answers a desktop program in the fragment of its own published redirect, and a fragment never leaves the browser. The program now draws that browser itself, for one sign-in at a time. All seven venues have a working sign-in path.

### The window Connect opens

`src/gui/sign_in_view.py` - what one sign-in is approved in

```
  class SignInView(QDialog):
```

It is a modal window holding one `QWebEngineView`. It has no address bar, no context menu, and no second window: `SignInPage.createWindow` answers None. The product already draws React inside a `QWebEngineView`, and this follows that.

Three things end it, and nothing else.

| What ends the sign-in | What Connect reads next |
|---|---|
| the view reaches `https://www.facebook.com/connect/login_success.html` | the fragment of that address |

| What ends the sign-in | What Connect reads next |
|---|---|
| the operator closes the window | "no reply came back from the browser" |
| 180 seconds pass | "no reply came back from the browser" |

The window reaches two addresses and no more: Meta's approval address and Meta's published redirect. `SignInView.reach` answers False for every other host and ends the sign-in on the spot.

`src/trading/ata_spm_signin.py` - the hosts one sign-in may reach

```
def sign_in_hosts(authorize_url: Any, redirect_address: Any) -> tuple:
```

Read off the running window with a prepared address in each case: `https://www.facebook.com/login.php` loads, `about:blank` is refused without ending the sign-in, `https://evil.example.com/...` ends it, and Meta's published redirect ends it with the address held.

### Where it is wired

```
def sign_in_session() -> Any:
```

`src/gui/market_inspector.py` hands it to `AtaSpmSettings.set_connector` for the Qt build. `src/gui/react_market_inspector_tab.py` hands it to the same call, because the React screen model builds its own `PushBoard` and replaces the one the tab inherits. Both builds sign Facebook in through the same window.

### The three hops

Diagram: diagram

```
sequenceDiagram
participant He as Operator
participant App as Acervator
participant W as The sign-in window
participant V as Meta
He->>App: Connect
App->>W: open on the approval address
W->>V: the operator approves
V->>W: land on login_success.html with a token in the fragment
W->>App: the address it landed on
App->>App: check the state it generated, then close the window
App->>V: trade the short-lived token for a long-lived one
App->>V: list the Pages the account administers
V->>App: Page id and Page token
App->>App: encrypt into the vault, return to Level 1
```

| Hop | Address | Method | Parameter names |
|---|---|---|---|
| the approval | `https://www.facebook.com/v25.0/dialog/oauth` | opened in the window | `client_id`, `redirect_uri`, `response_type`, `scope`, `state` |

| Hop | Address | Method | Parameter names |
|---|---|---|---|
| the long-lived token | `https://graph.facebook.com/v25.0/oauth/access_token` | GET | `grant_type`, `client_id`, `client_secret`, `fb_exchange_token` |
| the Page token | `https://graph.facebook.com/v25.0/me/accounts` | GET | `access_token` |

`response_type` is `token`, which Meta's manual-flow page names for a desktop app. The scope is `pages_manage_posts`, `pages_manage_metadata`, `pages_read_engagement` and `pages_show_list`.

### What Level 1A prints for Facebook

Every other venue keeps the line it already carried. Facebook's line now names the window rather than the system browser.

| Venue | Where Connect sends him |
|---|---|
| X | your browser on `https://x.com/i/oauth2/authorize` |
| Instagram | your browser on `https://www.instagram.com/oauth/authorize` |
| LinkedIn | your browser on `https://www.linkedin.com/oauth/native-pkce/authorization` |
| TikTok | your browser on `https://www.tiktok.com/v2/auth/authorize/` |
| Facebook | a sign-in window in Acervator on `https://www.facebook.com/v25.0/dialog/oauth` |
| Threads | your browser on `https://threads.com/oauth/authorize` |
| Reddit | your browser on `https://www.reddit.com/api/v1/authorize` |

Read off the two running screens, Facebook's Level 1A page draws 11 members under Qt and 11 under React. All 11 read the same.

### What never leaves the sign-in

- No address the window lands on reaches a log line, a render or a file. The log line names the host and nothing else.
- `SignInView._release` empties the cookie jar of the off-the-record profile the sign-in held. Nothing outlives the window.
- `SignInView._refuse_download` cancels every download the venue's page asks for, so nothing it sends reaches a file.
- A fragment with no `access_token` in it stores nothing. Level 1 keeps reading `not held` for Facebook and the page prints "the venue answered with no access_token".

### Every address on a Level 1A page is a link

An address on a Level 1A page opens in the system browser. The window and the page draw the same links, in the same colour, from the same list.

`src/gui/main_tabs/market_inspector_surface.py` - what makes one word a link

```
def link_address(word: Any) -> str:
```

A word is an address only where it names a host and a path. `video.publish` carries a dot and no path. `/api/submit` and `/<IG_ID>/media` carry a path and no host. None of the three becomes a link.

Three lines carry links: the "Posts to" line, the "Register first" line and the "Before it works" line. Each is a fixed text on that venue's own `PushTarget` row in `src/trading/ata_spm_push.py`. Nothing the operator typed and nothing a venue answered is ever a link.

| Venue | Clickable | Plain text, and what it is |
|---|---|---|
| X | `https://api.x.com/2/tweets` | the row names no other address |
| Instagram | nothing | `/<IG_ID>/media` and `/<IG_ID>/media_publish` are paths with no host |
| LinkedIn | `https://api.linkedin.com/rest/posts` | the row names no other address |
| TikTok | nothing | `/v2/post/publish/content/init/` is a path with no host; `video.publish` and `video.upload` are scope names |
| Facebook | nothing | `/<page_id>/feed` and `/<page_id>/photos` are paths with no host |
| Threads | nothing | `/<threads-user-id>/threads` is a path with no host |
| Reddit | `https://www.reddit.com/api/v1/access_token` and `reddit.com/prefs/apps` | `/api/submit` is a path with no host; `<platform>:<app ID>:<version>` and `/u/<username>` are a User-Agent format |

A press reaches `webbrowser.open`, the same call the news ticker and the bot table already use. `page_links` is the whole list a press may name, so a press naming any other address opens nothing and the window records the refusal.

The React page draws an anchor whose click opens the browser through the host. It does not navigate the view the page is drawn in. Read off the running page, a real click on Reddit's endpoint link opened the browser and left the view's own address unchanged.

Read off the two running screens, all seven Level 1A pages draw 10 members under Qt and 10 under React. All 70 read the same, and the link of every venue matches in both.

### Two sentences this entry contradicts

Neither was reworded. Each is quoted here.

`docs/manual/08-tabs/market-inspector.md:2115` - "No such view is wired, and Facebook's Connect says so on the page rather than waiting three minutes for a reply that cannot come." One is wired now.

`docs/manual/08-tabs/market-inspector.md:1877` - "Connect opens the system browser at that venue's own approval address, waits on a loopback listener for the reply". True for six venues. Facebook opens the window above and waits on no listener.

**Figures.** This page carries no figure and this entry adds none.

## 2026-09-14 23:24 - #23 - The right pane's zones fit at every width

The right pane holds three zones. Two of them carry a row of their own above the stepper, and both rows were built to one width and never re-laid. At a 700 px tab the Send Bucket Full Auto button ended 72 px outside its zone, the Adopt line asked 272 px of a 109 px box, and the proposal status line asked 145 px of the same 109. The page put the same status line 178 px past the pane's right edge and grew a horizontal scroll bar on the Ready to Send zone.

Both rows now fit the zone they are drawn in, at every width, in both builds.

### Three buttons of one width, broken at the count the zone holds

The three Ready to Send buttons were 128, 92 and 184 px wide. Three unequal cells cannot break at one count: the window asks `columns_for` for a number and the page packs by each button's own width, so the two builds wrapped in different places on the same pane. At a 900 px tab the window fitted all three on one line and the page did not.

`src/gui/main_tabs/market_inspector_surface.py` - the one width the three share

```
BUCKET_BUTTON_WIDTH_PX = 184
```

184 px is kept because it is the width that already held `Send Bucket Full Auto`. Level 1 sizes its buttons the same way: every venue button is 108 px whether it reads `X` or `Instagram`.

The window puts the three on a `PaneWidthPage`, the same class Level 1 uses, so `_relay_bucket_row` runs at the zone's new width every time the pane's edge moves. The page wraps them at `calc(100% - gap)`, which is the width `columns_for` measures against.

| Tab width | Buttons on a line |
|---|---|
| 700 | 1 |
| 900 | 2 |
| 1400 | 3 |
| 1960 | 3 |

### The Adopt line and the proposal status take an equal share and wrap

Neither line had a width of its own. Each was drawn at whatever its own text needed, on a row that could not wrap, with a gap between them that pushed the status line at the pane's right edge.

The Refresh button now keeps a published width and the two lines share what the row has left, one stretch each, wrapping at a word inside that share.

`src/gui/main_tabs/market_inspector_topologies_surface.py` - what the row declares

```
REFRESH_WIDTH_PX = 96
HEADER_WORD_WRAP = True
```

Read off the running program, each line gets 102, 152, 277 and 417 px in the window at tab widths of 700, 900, 1400 and 1960, and 103, 153, 278 and 418 px on the page. The Adopt line needs 272 px, so it reads on one line at 1400 and above and wraps at a word below that. The status line needs 145 px and wraps only at 700.

### Every width, both builds, nothing outside the pane

| Tab width | Window zone | Page zone | Controls outside the pane | Scroll bars |
|---|---|---|---|---|
| 700 | 346 | 347 | 0 | 0 |
| 900 | 446 | 447 | 0 | 0 |
| 1400 | 696 | 697 | 0 | 0 |
| 1960 | 976 | 977 | 0 | 0 |

Every page row measures `scrollWidth` equal to `clientWidth` at all four widths, and every window control's right edge is inside its zone. The two builds break at the same count on every row.

Nothing was shortened to make this fit. The three buttons keep their wording, and both lines read in full.

**Figures.** This page carries no figure and this entry adds none. Every width quoted above was read from the running program and is not stored.

## 2026-09-14 23:57 - #23 - Every Level 1A page carries its register address

Each venue issues credentials only to an application registered with it. Six of the seven pages named no place to do that. Every page now carries the address of the venue's own app-registration page, as a link.

### The address each venue's row names

Each address is the one that venue's own developer documentation publishes.

| Venue | Register address | Read from |
|---|---|---|
| X | `https://console.x.com` | X, Developer Console and Getting Access |
| Instagram | `developers.facebook.com/apps` | Meta, App Dashboard |
| LinkedIn | `www.linkedin.com/developers/apps/new` | LinkedIn, Marketing Quick Start, step 1 |
| TikTok | `developers.tiktok.com/apps` | TikTok, Register Your App |
| Facebook | `developers.facebook.com/apps` | Meta, App Dashboard |
| Threads | `developers.facebook.com/apps` | Meta, Threads Get Started |
| Reddit | `reddit.com/prefs/apps` | already on the row, unchanged |

`src/trading/ata_spm_push.py` - where one venue's address is held

```
        registration="A TikTok developer app with Content Posting and Direct "
        "Post switched on, and a verified address prefix. Register the app at "
        "developers.tiktok.com/apps.",
```

X publishes an address with no path, so its row carries the scheme. The other six name a host and a path, which is what `link_address` reads as an address.

No new mechanism carries these. `link_segments` makes the word a link and `page_links` keeps it on the open page's allowlist, exactly as the endpoint line and Reddit's address already worked.

### What a press does, read off both running builds

Every address the open page publishes reaches the browser. Every address it does not publish opens nothing and is recorded as refused. A venue's own real address is still refused on another venue's page.

| Page open | Address pressed | What opened |
|---|---|---|
| X | `https://console.x.com` | the browser, at that address |
| Instagram | `https://developers.facebook.com/apps` | the browser, at that address |
| Instagram | `https://console.x.com` | nothing. Not on this page |
| LinkedIn | `https://example.invalid/steal` | nothing. Not on this page |

Read off the two running builds, 20 addresses reached the browser and every off-page address opened nothing, the same 20 and the same refusals under both.

The authorize address and the redirect address stay plain text. A press on either outside a sign-in reaches an error page.

### The pages address the operator directly

Four lines spoke about the reader as a third party. Each now addresses him, or names the account he acts on. No fact changed.

| Venue and line | Now reads |
|---|---|
| Facebook, register | a Page you administer |
| TikTok, prerequisite | which means only you see it |
| TikTok, prerequisite | leave the post in your drafts |
| Instagram, Facebook and Threads, prerequisite | so give your account a role on it |

TikTok's "requires the account to be private at the time of posting" is unchanged. It states TikTok's rule for any posting account, not the reader's.

### An empty Topologies zone reads the same in both builds

The zone's empty sentence comes from `pane_view`, which also turns the entry hint off while the zone holds nothing. The page drew that from its first paint. The window built its stepper and waited for the first auto-refresh, so until then it drew a blank headline and the entry hint under it.

`src/gui/market_inspector_topologies.py` - the pane draws its view when built

```
            self._render()
```

Read off both running builds, an empty zone now reads "No proposals right now. Try Refresh, or wait for market state to shift." and carries no hint. The hint still draws on a zone that holds an entry and is closed.

### Six rows this entry contradicts

None was reworded. Each is quoted here.

`docs/manual/08-tabs/market-inspector.md:2430` - "| X | `https://api.x.com/2/tweets` | the row names no other address |" The row names `https://console.x.com` too.

`docs/manual/08-tabs/market-inspector.md:2431` - "| Instagram | nothing |" Instagram links `developers.facebook.com/apps`.

`docs/manual/08-tabs/market-inspector.md:2432` - "| LinkedIn | `https://api.linkedin.com/rest/posts` | the row names no other address |" The row names

`www.linkedin.com/developers/apps/new` too.

`docs/manual/08-tabs/market-inspector.md:2433` - "| TikTok | nothing |" TikTok links `developers.tiktok.com/apps`.

`docs/manual/08-tabs/market-inspector.md:2434` - "| Facebook | nothing |" Facebook links `developers.facebook.com/apps`.

`docs/manual/08-tabs/market-inspector.md:2435` - "| Threads | nothing |" Threads links `developers.facebook.com/apps`.

**Figures.** This page carries no figure and this entry adds none.

## 2026-09-15 01:02 - #23 - A refused sign-in names the venue's own reason

Level 1A returns to Level 1 by itself when a venue accepts, and holds the page when a venue refuses. Both builds do this. A refusal now prints the reason the venue itself gave, and not the status line alone.

### What each half does, read off both running builds

The page is chosen from `PushBoard.credential_target`, which `connect_credentials` clears only on a sign-in the venue accepted.

| What the venue answers | Page after Connect | Message on the page |
|---|---|---|
| accepts | Level 1 | none. Level 1 carries no message line |
| refuses | Level 1A, held | the status and the venue's own reason |

Read off both running builds, driven through the real loopback listener, the Qt window and the React page show the same level and the same message on both halves.

### What a refusal carries

`urlopen_transport` reads the body a venue answers a refusal with, and raises `SignInError` carrying `refusal_text`. RFC 6749 section 5.2 puts the reason in `error` and `error_description`, and only those two fields are read out. No other value in the body reaches Level 1A.

`src/trading/ata_spm_signin.py` - the wordings a refusal is built from

```
HTTP_REFUSAL_FORMAT = "HTTP {status} {reason}"
VENUE_SAID_FORMAT = "{http}, and the venue said {said}"
VENUE_REASON_FORMAT = "{error}: {description}"
```

| Body the venue answers | What Level 1A prints |
|---|---|
| both fields | HTTP 401 Unauthorized, and the venue said invalid_client: Client auth failed. |
| `error` alone | HTTP 401 Unauthorized, and the venue said invalid_grant |
| `error_description` alone | HTTP 401 Unauthorized, and the venue said The code has expired. |
| neither field | HTTP 401 Unauthorized |
| not JSON | HTTP 401 Unauthorized |
| empty | HTTP 401 Unauthorized |

A body is read up to `REFUSAL_BODY_LIMIT` bytes and no further.

### A pasted credential is trimmed where it is held

`missing_field` reads a box holding only spaces as empty. `typed_credential` handed the raw text to the venue, so a value pasted with a trailing space or newline went into the HTTP Basic header exactly as pasted. `set_credential_text` now holds the trimmed value, which is the value `missing_field` already judged.

`src/trading/ata_spm_push.py` - where one typed value is held

```
        held = str(typed or "").strip()
        self.typed.setdefault(str(target), {})[str(field)] = held
```

Read off the running program, a Client ID pasted as `abc \n` reaches the venue as `abc`, and a box holding only spaces is still reported empty.

**Figures.** This page carries no figure and this entry adds none.

## 2026-09-15 02:49 - #23 - A sign-in says what it did, on the page and in the log

Level 1A held the page on a refusal and printed the reason in the page's own body text. Two X sign-ins ended the same way for the operator: the page did not move, and nothing on it read as an answer to the press. The message now carries the outcome's own colour, and every outcome writes one log line.

### What a press writes now

`AtaSpmSettings.connect` had one log call and it sat inside the `except`. Four other branches answer `ok` False without raising, and the accepted branch raises nothing either, so a press could end with no line at all.

`src/trading/ata_spm_push.py` - the one line every outcome reaches

```
    def connect(self, target: Any) -> "ConnectResult":
        answer = self.sign_in_answer(target)
        logger.info(CONNECT_RESULT_LOG, answer.target, answer.ok, answer.detail)
        return answer
```

| What the venue answers | Lines before | Lines now |
|---|---|---|
| refuses with HTTP 401 | 1 | 2 |
| accepts, and no vault holds it | 0 | 1 |
| accepts, and a vault holds it | 0 | 1 |

Read off both running builds, driven through a real press with the program's own loopback listener and a stand-in for the venue's socket.

### What the message line is drawn in

`message_colour` picks one colour per outcome. Both builds read that one value, so the window and the page cannot disagree about it.

`src/gui/main_tabs/market_inspector_surface.py` - where the colour is chosen

```
def message_colour(answered: Any) -> str:
    if answered is None:
        return NO_COLOR
    return MESSAGE_ACCEPTED_COLOUR if answered.ok else MESSAGE_REFUSED_COLOUR
```

| Outcome | What the line is drawn in |
|---|---|
| no press yet | nothing. The line holds no text |
| refused | `ds.ERROR` |
| accepted | `ds.SUCCESS`, until Level 1 replaces the page |

### A sign-in a venue accepts is not held anywhere

`AtaSpmSettings.set_vault` has no caller in the tree, and `CredentialVault` is constructed nowhere. `store_credential` answers False whenever `vault` is None, so every sign-in a venue accepts ends at `Credential vault not wired.` with `ok` False, and Level 1A holds the page.

Where the vault holds the tokens, and which passphrase holds them, is not decided yet. Until it is, that outcome now reads on the page and in the log instead of reading as nothing.

**Figures.** This page carries no figure and this entry adds none.

## 2026-09-15 03:19 - #23 - The credential vault, and Level 1A leaving by itself

A sign-in the venue accepted had nowhere to put its tokens, so Level 1A held the page on a sign-in that worked. Both builds now hand the board a vault when they build it, and a venue that accepts returns to Level 1 by itself.

### Where the vault is built

`src/core/encryption.py` - the file the tokens are kept in

```
DEFAULT_VAULT_PATH = Path.home() / ".acervator" / "ata_spm_credentials.json"
```

`FileVault` is a `CredentialVault` that reads that file when it is built and writes it on every `store`. `default_vault` keys it with `vault_phrase` over the stored username, which is the phrase every other stored credential in the product is encrypted under.

### Where it is handed over

Two files build a board, and each now hands it a vault beside the sign-in route.

| File | The board it builds |
|---|---|
| `src/gui/market_inspector.py` | the window's own |
| `src/gui/main_tabs/market_inspector_surface.py` | the screen model's, which the React tab reads |

```
            self._push_board.settings.set_vault(encryption.default_vault())
```

### What Connect does now

| What the venue answers | Page after Connect | Level 1 for that venue |
|---|---|---|
| accepts | Level 1 | held, and the button draws on |
| refuses | Level 1A, held | not held |

Read off both running builds, driven by a real press on each page, the window and the page both leave Level 1A for Level 1 on an accepted sign-in. X reads `held` on Level 1 and Reddit, which was never signed in, reads `not held`.

### The tokens outlive a run

One run signs in and the next reads it back.

```
run one    Qt    connect ok = True    X row = ['X', True, 'held']
run two    Qt    X row = ['X', True, 'held']
run two    Qt    Reddit row = ['Reddit', False, 'not held']
```

### Nothing on disk and nothing in a log carries a value

The file holds only the Base64 tokens `encrypt` produced. Read off the run, a search of the whole runtime directory for the value that was signed in with found one file, and that file was a planted control written for the search to find. Neither the vault file nor a log line carried it.

### A sentence this entry contradicts

It was not reworded. It is quoted here.

`docs/manual/08-tabs/market-inspector.md:578` - "Save credentials encrypts every typed credential into the same vault the venue keys use, then clears what was typed." No control named Save credentials is in the program, and Connect is what stores. The exchange keys are held in `settings.toml` and the push-target tokens in `ata_spm_credentials.json`, so they are two files under one passphrase.

**Figures.** This page carries no figure and this entry adds none.

## 2026-09-15 05:16 - #23 - What he types is held too, and every register address

Level 1A held what he typed in memory only. A restart lost it, and X prints a client secret once, so losing it means generating a new one in X's console. What he types now reaches the same encrypted vault the venue tokens reach.

### When a box reaches the vault

A keystroke writes memory, as before. A finished box writes the vault. A box is finished when he leaves it: the window calls that `editingFinished` and the page calls it a blur.

`src/trading/ata_spm_push.py` - what one finished box does

```
            if held:
                self.vault.store(vault_key(name, key), held, "")
            else:
                self.vault.delete(vault_key(name, key))
```

A vault write runs PBKDF2 twice at 600,000 iterations. Measured on this machine, one write takes 0.149 s and one read 0.149 s, so a write on every keystroke would add about four seconds to a 26-character secret. A write when he leaves the box costs that 0.149 s once.

`CredentialVault.delete` is new in `src/core/encryption.py`. It is what lets a box he emptied drop its entry, so `has_exchange` answers whether a value is held without decrypting one. That read takes 0.000002 s, which is why Level 1A can ask it on every paint.

### What Connect reads now

| What the box holds | What `missing_field` says | What `typed_credential` hands the venue |
|---|---|---|
| text typed this run | present | what he typed |
| nothing typed, vault holds a value | present | what the vault holds |
| nothing typed, vault holds nothing | the first such field, by name | nothing |
| emptied this run | the first such field, by name | nothing |

A value the vault holds therefore counts as filled in, and a page he filled before a restart takes Connect without being typed again.

### A held box says it is held and never says what

`credential_page` still publishes each box as a name and a wording and no value. It carries one more list, `held_fields`, which names only the keys the vault holds something for. The wording a held box draws is one fixed string both builds read from the same place.

```
CREDENTIAL_HELD_PLACEHOLDER = "Held · type to replace"
```

Typing into a held box replaces what is held when he leaves it.

### The window now empties its boxes when the page opens

The page builds its boxes when Level 1A opens, so they start empty every time. The window built its boxes once and only hid them, so they kept the characters he typed for the life of the run. Read off the two running builds before this entry, X reopened with the typed characters still in the window's box and with the page's box empty. The window now empties a venue's boxes as it opens that venue, which is what the page does, and a value the vault holds is drawn back into neither.

### Read off the two running builds

`HOME` was redirected to a scratch directory and every non-loopback socket was refused, so nothing reached a venue.

| Reading | Before | After |
|---|---|---|
| vault entries after both X boxes are typed | none | `X:x-client-id`, `X:x-client-secret` |
| vault entries after the venue refuses | none | the same two |
| X reopened, window box | the typed characters | empty, wording `Held · type to replace` |
| X reopened, page box | empty | empty, wording `Held · type to replace` |
| second process, `missing_field` on X | `x-client-id` | none |
| second process, boxes | empty, own labels | empty, wording `Held · type to replace` |

The second process holds nothing in memory and reads the vault file, which is what makes it a restart.

A venue never typed into reads the other way in the same run. Reddit answers `held_fields` empty, draws each box under its own label, and Connect refuses with `App ID is empty.`

### Nothing carries a typed value in clear text

Read off one run that typed both X boxes and pressed Connect on both builds: 575 files under the runtime directory and under `src/` were searched and none carried the value. The same run wrote 4 log lines from the program's own sign-in logging, so the log existed while the value passed through it.

The same search was then run with the value planted in a file and in a log line. It found both. The zero above is a reading, not a silence.

### Every venue's register address

Each address is the one that venue's own developer documentation publishes for creating an application. Read off both running builds, all seven draw one register link, the same one under each build, and each is on its own page's allowlist.

| Venue | Register address | Read from |
|---|---|---|
| X | `https://console.x.com` | X, Developer Console. `developer.x.com` redirects here |
| Instagram | `developers.facebook.com/apps/creation/` | Meta, Create an App with Meta |
| LinkedIn | `www.linkedin.com/developers/apps` | LinkedIn, Getting Access, then Create app |
| TikTok | `developers.tiktok.com/apps` | TikTok, Create an app |
| Facebook | `developers.facebook.com/apps/creation/` | Meta, Create an App with Meta |
| Threads | `developers.facebook.com/apps/creation/` | Meta, Threads use case |
| Reddit | `www.reddit.com/prefs/apps` | Reddit, OAuth2 |

Four rows moved. The three Meta rows named `developers.facebook.com/apps`, which is the list of apps already made, and Meta publishes `/apps/creation/` for making one. LinkedIn's row named `/developers/apps/new`, which LinkedIn publishes nowhere; it resolves to the developers sign-in page rather than to a form.

### Instagram, where the values are not where a developer expects

Instagram's page now names the path, because three steps sit away from where an app's settings normally are.

| What he needs | Where Meta puts it |
|---|---|
| the app type | pick the Other use case, then the Business app type |
| Instagram app ID and Instagram app secret | the left menu panel API setup with Instagram Login |
| the redirect address | Business login settings, inside that same panel |

Meta publishes no Business use case. Its use-case list holds Other, and the Business app type is offered after Other is picked. The App ID and App secret on App settings then Basic are different values and are not the ones this page takes.

This route needs no Facebook Page. Meta states that a Page is required only for the other route, Facebook Login for Business.

### Two rows this entry contradicts

Neither was reworded. Each is quoted here.

`docs/manual/08-tabs/market-inspector.md:2562` - "| Instagram | `developers.facebook.com/apps` | Meta, App Dashboard |" That address opens the list of apps already made. Meta publishes `developers.facebook.com/apps/creation/` for making one, and the row now names it. The Facebook and Threads rows beneath it moved for the same reason.

`docs/manual/08-tabs/market-inspector.md:2563` - "| LinkedIn | `www.linkedin.com/developers/apps/new` | LinkedIn, Marketing Quick Start, step 1 |" LinkedIn publishes no such address. Its own pages name `/developers/apps` and a Create app button.

**Figures.** This page carries no figure and this entry adds none.

## 2026-09-15 06:31 - #23 - Scan Now reads the market, and says so when it cannot

Scan Now read no market of its own for crypto. It served whatever the Refresh press had left behind, and with no Refresh behind it, it served nothing and drew nothing. The press now reads candles for the assets it is scanning, and a scan that comes back empty names the assets it came back empty for.

### Where the two sources are wired

`MarketInspectorScreenModel` published `set_ata_sources` and nothing called it, so both sources read `None` for the life of the screen. `set_connector` and `set_vault` were the same shape on the same screen. Both hosts now call it at the point they are built.

`src/gui/main_tabs/market_inspector_surface.py` - the React screen model

```
        self.set_ata_sources(sector_assets, self.scanned_candles)
```

`src/gui/market_inspector.py` - the window

```
            self.set_ata_sources(sector_assets, self._scanned_candles)
```

Driven with the network refused, the model answered `ata_candle_source is None: True` before and `False` after.

### Where a scan's candles come from now

`sector_candles` asks three questions in order, and stops at the first that answers.

```
    if ata_asset_maps.listing_of(symbol) is not None:
        return ata_asset_maps.venue_candles(symbol, timeframe)
    if inspector_scan_age(inspector) < CANDLES_FRESH_SECONDS:
        held = inspector_candles(inspector, symbol, timeframe)
        if held:
            return held
    return connector_candles(connectors, symbol, timeframe)
```

| The symbol | Where its candles come from |
|---|---|
| a forex or metals name | the venue its listing names, read now |

| The symbol | Where its candles come from |
|---|---|
| a crypto name, Refresh under 15 minutes old | the candles that Refresh read |
| a crypto name, Refresh older or absent | the exchange, read now, one call per timeframe |
| a crypto name, no exchange connected | nothing, and the zone says so |

The third row is the new one, and it is the reason a first press now draws something. `connector_candles` runs `fetch_symbol_timeframe`, which sits in `src/exchange/market_inspector_fetcher.py` beside the Refresh press's own fetch and uses the connectors already attached. ATA-SPM opens no connection of its own.

### The freshness window, and what happens past it

`CANDLES_FRESH_SECONDS` is `DEFAULT_MIN_REFRESH_S`, the same 900 seconds `fetch_htf_universe` serves its own cache over. Inside that window a scan costs no venue traffic, because the answer is the one the fetcher would have given. Past it, the scan reads again.

Driven with an exchange stand-in counting every call, network refused:

```
scan age 0.0 s, window 900 s   ->  0 calls, 36 votes
scan age 960.0 s, window 900 s ->  72 calls, 36 votes
```

Both arms voted, so the call count is what separates them, and it reports a zero and a non-zero on the same instrument.

### A scan that read nothing names the assets

The method line under a sector used to read "No chart carried a reversal vote." whether a chart had been read and refused, or never read at all. Those are different things to a person holding the button.

```
NO_CANDLE_TEXT = "No candles came back for {symbols}."
```

A symbol is named only when every ticked timeframe came back empty for it. When some assets voted and none carried a call, both sentences print, in that order.

### Read off both running builds after a scan

Both builds were driven with the same stand-in and the zone read off each rendered surface - the window's own labels, and the page's own text.

| | window | page |
|---|---|---|
| scan that found candles | `18 asset(s) · 36 vote(s) · 0 reversal call(s)` / "No chart carried a reversal vote." | identical |
| scan that found none | `18 asset(s) · 0 vote(s) · 0 reversal call(s)` / "No candles came back for A34, ALGO, APT, ATOM, AVAX, A15 and 12 more." | identical |

Both builds made the same number of calls in each arm, 72 and 108. Network calls attempted across every run: 0.

### The sentence this entry overtakes

It was not reworded. It is quoted here.

`docs/manual/08-tabs/market-inspector.md:1332` - "The crypto universe scan keeps daily and weekly candles, so its two fast boxes report the same way." That held while the universe scan was the only crypto source. The five-minute and hourly boxes now read through the exchange like the other two, so they report as unserved only when the exchange itself serves no such timeframe.

**Figures.** This page carries no figure and this entry adds none.

## 2026-09-15 07:30 - #23 - The ticker field, and the way to the chart images

The text field on the ATA-SPM scan page is named for what it does. It names one market, read on demand. The menu beside it is the sector menu, and it holds crypto, stocks, metals, derivatives and forex.

`src/gui/main_tabs/market_inspector_surface.py` — what the field says

```
TICKER_FIELD_PLACEHOLDER = "Ticker"
TICKER_FIELD_TOOLTIP = (
    "Name one market to read on demand. Typing offers the tickers the "
    "sector menu beside it holds."
)
```

### Typing offers the tickers the sector holds

Each letter narrows a list the program already holds. Nothing is fetched and no venue is asked. Crypto reads the shipped sector map. Every other sector reads the asset maps.

`src/gui/main_tabs/market_inspector_surface.py` — where the offered names come from

```
def class_tickers(asset_class: Any) -> list:
    """Every ticker one sector names, read from the lists already in this tree.

    ``ata_spm.CLASS_CRYPTO`` reads the shipped sector map and every other class
    reads ``ata_asset_maps.MAPS``, so no venue is asked for a symbol.
    """
```

Matches beginning with what he typed come first, then matches holding it later in the name. Eight is the most the field offers at once.

Three sectors carry a list and two do not. Crypto holds 120 names, forex 28 and metals 8. Stocks and derivatives hold none, because no list of their membership sits in this tree. The field says so under itself and still takes a typed name.

`src/gui/main_tabs/market_inspector_surface.py` — the line an empty sector carries

```
TICKER_NO_LIST_FORMAT = "No ticker list for {sector}. A typed name still scans."
```

### The Chart Folder button, and where the pictures are

The Ready to Send zone carries a fourth button. It opens the folder holding the chart images, in the operating system's own file browser. The folder is a sibling of the state directory and of the log directory, never inside either, and it is created before the first chart is drawn.

`src/trading/ata_post_paths.py` — the one root every chart image is written under

```
ATA_POST_ROOT: Path = Path.home() / ".acervator_ata_posts"
```

This is the route that needs no account. Most venues price a post, or hold it for approval. A chart in a folder can be posted by hand, with no API, no fee and no review.

### Each chart image carries the standardised message

The picture is written with its wording on it. The message is the one a post composes: the fixed header, the call's own headline, then the address. A chart opened from the folder therefore reads as something that can be posted as it is.

`src/trading/ata_spm.py` — what the picture carries

```
def post_caption(vote: AssetVote) -> str:
    """The standardised message one call's chart image carries.

    ``ata_spm_push.compose`` writes the same header, headline and address a
    post's own caption carries, and it is imported here because that module
    reads this one.
    """
```

Measured on a rendered picture, the message takes 55 pixels at the foot of the image: eight of padding above, eight below, and 13 for each of its three lines. A chart given no message is 55 pixels shorter and carries none.

### The sentence this entry leaves standing

It was not reworded. It is quoted here.

`docs/manual/08-tabs/market-inspector.md:443` - "It carries a sector field, an asset-class box, four timeframe check boxes and a Scan Now button." That named the field while it was called Sector. The field is now the ticker field and the box beside it is the sector menu, so the two names in that sentence have swapped. The row still carries the same four controls, and now a fifth line under them for the sector with no list.

### What Scan Now still does with the typed value

Naming the field did not change the press. Scan Now adds what he typed as a sector, so a ticker typed into the field names a sector holding no assets and the scan reads nothing. Changing what the press scans is a product decision and it is recorded here rather than taken.

**Figures.** This page carries no figure and this entry adds none.

## 2026-09-15 08:22 - #23 - A typed ticker scans that one market

The ticker field now names a market to the press behind it. Typing a ticker and pressing Scan Now reads that one market on the ticked timeframes. The sector menu still picks the sector, and an empty field still scans the sectors the board holds.

### What one press does, for each of the three things the field can hold

| the field holds | the press |
|---|---|
| a ticker the sector lists | reads that one market, on every ticked timeframe |
| nothing | runs every scan the board already holds, unchanged |
| a name the sector lists as neither ticker nor sector | reads nothing, and the line under the field says so |

### Where the typed value is turned into one market

`src/gui/main_tabs/market_inspector_surface.py` — the ticker to one asset row

```
def market_listing(ticker: Any, asset_class: Any) -> Any:
    """The one ``ata_asset_maps.AssetListing`` a typed ticker names in one sector.

    ``class_tickers`` decides whether the sector lists the name, and a sector
    listing none takes any name, which is what ``TICKER_NO_LIST_FORMAT`` says
    under the field; a name a listing sector does not hold answers None.
    """
```

The row it answers goes onto the scan itself, so no asset source is asked for a market the field already named.

`src/trading/ata_spm.py` — the scan that carries its own row

```
    name: str
    asset_class: str = CLASS_CRYPTO
    timeframes: tuple = ()
    ticker: str = ""
    listings: tuple = ()
```

Candles then come from the wiring `sector_candles` already holds: a forex or metals name through the venue its listing names, a crypto name through the Refresh cache while that cache is under 15 minutes old, and through `fetch_symbol_timeframe` once it is older.

### The zone says which kind of scan it ran

`src/trading/ata_spm.py` — the two wordings one entry is drawn from

```
SECTOR_LINE_FORMAT = "{sector} ({asset_class})"
SECTOR_META_FORMAT = "{assets} asset(s) · {votes} vote(s) · {calls} reversal call(s)"
MARKET_LINE_FORMAT = "{ticker} in {asset_class}"
MARKET_META_FORMAT = "1 market · {votes} vote(s) · {calls} reversal call(s)"
```

The zone's own status line counts the two kinds apart. A run holding only sectors keeps the line it always had, a run holding only markets counts markets, and a run holding both counts both.

### A name the sector does not list

`src/trading/ata_spm.py` — what that press leaves behind

```
TICKER_UNHELD_FORMAT = (
    "{asset_class} holds no ticker {ticker}. Pick one the field offers."
)
```

That sentence draws on the line under the field, where the sector's own `TICKER_NO_LIST_FORMAT` note draws. Typing again clears it, and so does picking another sector. The board gains nothing, so the press invents no sector.

### Read off both running builds after each press

Both builds were driven with one stand-in candle source, the network refused, and the zone read off each rendered surface — the window's own labels, and the page's own text.

| the press | window | page |
|---|---|---|
| `A15`, crypto | `A15 in crypto` / `1 market · 2 vote(s) · 0 reversal call(s)` | identical |
| `defi`, crypto | `defi (crypto)` / `10 asset(s) · 20 vote(s) · 0 reversal call(s)` | identical |
| the field empty | `defi (crypto)` / `10 asset(s) · 20 vote(s) · 0 reversal call(s)` | identical |
| `ZZZZ`, crypto | zone keeps `defi (crypto)`; the line under the field reads `crypto holds no ticker ZZZZ. Pick one the field offers.` | identical |

The symbols each press read, counted at the candle source: 1 for `A15`, 11 for `defi` on a board already holding `A15`, 11 again for the empty field, and 0 for `ZZZZ`. The same counter reported 0, 1, 4, 10 and 11 across one run, so its zero is a reading and not a blind instrument. Network calls attempted across every run: 0.

Before this entry the same four presses read 0 symbols for `A15`, then 10, 10 and 10, the last of those being the board re-running `defi` behind an invented `ZZZZ (crypto)` entry.

Two sectors carry no ticker of their own kind. `XAU/USD` in metals reads `XAU/USD in metals` / `No configured venue lists XAU/USD.`, which is what the metals map already records for all four spot pairs. `GOOG` in stocks, a sector listing no tickers at all, reads `GOOG in stocks` / `No configured venue lists GOOG.`, so a typed name still scans and the zone names what stopped it.

### The sentences this entry overtakes

They were not reworded. They are quoted here.

`docs/manual/08-tabs/market-inspector.md:483` - "Scan Now adds the typed sector when it is new, then runs the phases that read price data." The press adds the typed value as a market when the sector lists it as a ticker, and as a sector only when the sector map holds it under that name.

`docs/manual/08-tabs/market-inspector.md:3221` - "Naming the field did not change the press. Scan Now adds what he typed as a sector, so a ticker typed into the field names a sector holding no assets and the scan reads nothing." That was the state this entry ends. The product decision it left open is the one taken here.

`docs/manual/08-tabs/market-inspector.md:452` - "Scan this sector now on the timeframes ticked beside it, without waiting for a rotation." That is the Scan Now tooltip as it read while the field named a sector. The button now carries a tooltip naming both cases.

**Figures.** This page carries no figure and this entry adds none. A count of the markdown image tags on the page answers 0 before this entry and 0 after it.

## 2026-09-15 13:56 - #23 - The built application opens again

Build `1371.ge6b50511-qt` stopped before its window opened. The Market Inspector tab is built while the window is, and the line under the ticker field asks for the crypto ticker list as the tab is built. That list is read through the topology detectors' module, which imported the pair tests' module, which imported scipy at the top of the file. The bundle is built without scipy, so the import raised and the window never opened.

```
File "src\gui\main_tabs\market_inspector_surface.py", line 2428, in ticker_note
    if class_tickers(asset_class):
File "src\gui\main_tabs\market_inspector_surface.py", line 2385, in class_tickers
    from ...trading.topology_proposals import load_sector_map
File "src\trading\topology_proposals.py", line 20, in <module>
    from .pair_selection import (
File "src\trading\pair_selection.py", line 18, in <module>
    from scipy.stats import pearsonr  # type: ignore[import-untyped]
ModuleNotFoundError: No module named 'scipy'
```

### Where the two packages are imported now

`src/trading/pair_selection.py` imports statsmodels and scipy inside the three functions that call them, and nowhere else. The module loads where scipy is absent, and a test that needs the package raises when it runs and not before.

`src/trading/pair_selection.py` — each package imported where it is used

```
def engle_granger_p_value(left: Sequence[float], right: Sequence[float]) -> float:
    """The Engle-Granger two-step p-value from ``statsmodels.tsa.stattools.coint``."""
    from statsmodels.tsa.stattools import coint  # type: ignore[import-untyped]
```

```
def johansen_trace(left: Sequence[float], right: Sequence[float]) -> tuple:
    """The Johansen trace statistic for rank zero and its 95% critical value."""
    from statsmodels.tsa.vector_ar.vecm import (  # type: ignore[import-untyped]
        coint_johansen,
    )
```

```
    if is_flat(returns_left) or is_flat(returns_right):
        return _refused(METHOD_CORRELATION, len(left), FLAT_SERIES_DETAIL)
    from scipy.stats import pearsonr  # type: ignore[import-untyped]
```

A missing package is not a refused pair. The cointegration test lets an import error out of the handler that turns degenerate input into a refusal.

`src/trading/pair_selection.py` — a missing package leaves as an error

```
    try:
        p_value = engle_granger_p_value(left, right)
        trace, critical = johansen_trace(left, right)
    except ImportError:
        raise
    except Exception as exc:  # noqa: BLE001 - statsmodels refuses degenerate input
```

### What the bundle is built without, and what still needs it

The Windows and macOS builds share one exclusion list. scipy is on it, and the list is not changed here.

`tools/spec_common.py` — the names the bundle is built without

```
EXCLUDES: tuple[str, ...] = (
    "tkinter",
    "matplotlib",
    "scipy",
    "PIL",
```

The two tests this page names under "what a pair has to clear" still need the excluded package: Pearson from scipy directly, and Engle-Granger and Johansen through statsmodels, which imports scipy at its own top level and is declared in no dependency list of this tree. In the built bundle, a Bot Swarm Topologies refresh with market data reports the missing package in the zone's status line, and an Opposing Trades scan that reaches the cointegration test reports it as the scan error. The source tree, where scipy 1.18.1 and statsmodels 0.15.0 are installed, runs both tests.

Of the ten excluded names, scipy is the only one the startup path reaches. matplotlib is imported by one module nothing imports, and PIL inside four functions of one module nothing imports.

### Read off the built bundles

Both bundles were built from a worktree with the same spec the operator's build script runs, and launched inside a Windows AppContainer holding no network capability, with HOME and USERPROFILE pointed at an empty scratch directory. A loopback listener used to calibrate the container answered one request from outside it and none from inside; the bundle's own log carries no connection. The real runtime directories were not written: their modification times match before and after every launch.

| build | reading |
|---|---|
| `Acervator-0.2.0-dev.1371.ge6b50511-qt`, before the fix | `ModuleNotFoundError: No module named 'scipy'` from `pair_selection.py:18`, exit 1 after 42.9 s |
| `Acervator-0.2.0-dev.1371.ge6b50511.dirty-qt`, after the fix | `Application ready — main window displayed`; the Inspector tab drawn, `artifacts/u30/qt_market_inspector_1371.ge6b50511.dirty.png` |
| `Acervator-0.2.0-dev.1371.ge6b50511.dirty-react`, after the fix | built; not launched, see below |

The React bundle builds its first web view while the window is built, and QtWebEngine refuses to start inside an AppContainer: a bare web view driven from the installed interpreter inside the same container ends with exception `0x80000003` at construction, and outside it constructs. No admin-free way to refuse the network for that bundle exists on this account, so its window was not observed. PyInstaller's own archive reader shows the

two bundles carry byte-identical code for the four modules on the failing chain, and the same reader shows the unfixed bundle's pair tests' module differs.

Driven on the source tree with scipy and statsmodels removed from the import table: the three modules import, the crypto list answers 120 names, a 50-bar series is refused as too short without reaching either package, and a 400-bar series raises the missing-package error from each test. With the packages present the same 400-bar calls answer p-values 0.7626 and 0.9864.

### Two sentences this entry leaves standing

`docs/manual/08-tabs/market-inspector.md:105` - "The test is cointegration in both of its standard forms, taken from statsmodels". True of the source tree. In the built bundle the test cannot run, before this entry and after it, because the package's own scipy import is excluded.

`docs/manual/08-tabs/market-inspector.md:3142` - "Nothing is fetched and no venue is asked. Crypto reads the shipped sector map." Still true. The map is read through a function that needs nothing from either package.

This entry makes no earlier sentence wrong.

**Figures.** This entry adds no figure to the page. A count of the markdown image tags on the page answers 0 before this entry and 0 after it.

## 2026-09-15 15:30 - #23 - One chart folder per venue, filled for posting by hand

His words, 2026-09-15: *"Should have a catered folder for each venue."* And earlier: *"Seems most of this stuff is price gated so will also need a folder where charts customed for each venue can be viewed and manually posted."*

The chart folder now holds seven folders, one per venue, named for it. Each holds that venue's own version of every call: the chart with the venue's message at its foot, the same message as plain text beside it, and for X the address that opens the compose window with the message already typed. Nothing is sent. Writing to disk is the whole action.

`src/trading/ata_post_paths.py` — one folder per venue under the root

```
def venue_post_root(venue: object, root: Optional[Path] = None) -> Path:
    """The folder one push target's files land in, under ``get_ata_post_root(root)``, created."""
    held = get_ata_post_root(root) / name_part(venue)
    held.mkdir(parents=True, exist_ok=True)
    return held
```

### The Chart Folder press shows all seven

The button is the same one, and it opens the root. The seven folders are created on the press, so they are in view before the first scan. Read off the running program with the folder press driven and the host asked for one address:

```
folders now under the root:  ['Facebook', 'Instagram', 'LinkedIn', 'Reddit', 'Threads', 'TikTok', 'X']
one per venue, named for it: True
```

The zone keeps its four buttons. At a 700 px tab the zone holds one button per line, so seven more would crowd it; the root opens with all seven in view.

### What each folder holds, and the limit that shaped it

Every venue's row already declares what it takes: which sections, how long a body, counted in which unit, and whether a title stands beside it. The folder's files come from that row and from nothing else.

`src/trading/ata_venue_folders.py` — the files one folder takes for one call

```
    image = render_chart_png(
        candles,
        vote.symbol,
        ata_spm.timeframe_label(vote.timeframe),
        image_path,
        voters=[one.indicator for one in ata_spm.confirming_signals(vote)],
        max_overlays=int(max_supporting_indicators or ata_spm.NO_INDICATOR_CAP),
        direction=vote.direction_text,
        readings=[(one.indicator, one.message) for one in pull.messages or ()],
        caption=post.body,
    )
    text_path = write_text(
        image_path.with_suffix(ata_post_paths.POST_TEXT_SUFFIX), post_text(post)
    )
```

Driven on one call with twelve confirming voters and long readings, so that every section uncut measured 6474 characters:

| Venue | Counted in | Body limit | Measured | Lines dropped | Title line |
|---|---|---|---|---|---|
| X | weighted | 280 | 251 | 12 | none |
| Instagram | characters | 2200 | 1834 | 9 | none |
| LinkedIn | characters | 3000 | 2850 | 7 | none |
| TikTok | UTF-16 runes | 4000 | 3918 | 5 | none, the header alone is over 90 |
| Facebook | characters | none published | 6474 | 0 | none |
| Threads | UTF-8 with emoji | 500 | 260 | 12 | none |
| Reddit | characters | 40000 | 6474 | 0 | yes |

A line drops whole and the longest evidence drops first, so no price or band value is cut mid-digit, and the fixed header and the address are never dropped. The same call with three voters and the standard wording left every body under its limit except X's: the header, the call line and the address take 209 of X's 280 weighted units, leaving 71 for evidence, so one 43-unit reading fits whole and a longer one is dropped for the abbreviation note.

### The text beside the picture, and the X compose address

The text file holds the message exactly as the picture carries it, so it can be pasted into a compose window. Where the venue holds a title, the title stands on the first line, a blank line, then the body. Reddit's title holds the header with the headline. TikTok's title field holds 90 and the header alone measures more, so its title is empty and the file carries no title line.

For X only, a third file holds the Web Intent address, in the shortcut form the operating system opens on one click. The compose window opens with the message typed; the picture is attached by hand, because the intent carries text alone.

`src/trading/ata_venue_folders.py` — the address, and the file that opens it

```
X_INTENT_FORMAT = "https://x.com/intent/post?text={text}"

INTENT_SHORTCUT_FORMAT = "[InternetShortcut]\nURL={url}\n"
```

Read back off the disk, the text the address carries decodes to the X body, to the character.

### One call fills every folder, and each folder keeps one set per market

The files are written inside phase three, from the candles it already holds, in the same call that draws the root picture. No second read of the market happens, so no venue is asked for a candle twice. A market the live gate chains refuse writes nothing anywhere.

`src/trading/ata_spm.py` — where the folders are filled

```
    if gates.would_fire:
        from .ata_venue_folders import write_venue_posts

        held.image = render_pull_image(
            vote, candles, max_supporting_indicators, messages
        )
        held.venue_posts = write_venue_posts(
            vote, held, candles, max_supporting_indicators
        )
```

Each venue folder is trimmed the way the root is: one picture, one text and, for X, one shortcut per market, the newest kept. Counted off the disk:

```
before any scan                           0 files
after a market the gate chains refused    0 files
after one firing call                    16 files   the root picture, 7 pictures, 7 texts, 1 shortcut
after a second call on the same market   16 files
after a second market                    32 files
```

### The sentences this entry leaves standing

`docs/manual/08-tabs/market-inspector.md:1138` — *"It reads one folder, the one its own file names, and it goes no deeper."* That is still what the root's trim does. It reads the root and skips the seven folders under it; each venue folder's trim reads that one folder. A text file placed at the root under a post's own name sat through a driven run and was still there at the end.

`docs/manual/08-tabs/market-inspector.md:3171` — *"The Ready to Send zone carries a fourth button. It opens the folder holding the chart images, in the operating system's own file browser."* Still true. The button opens the same root, which now holds the seven folders.

This entry makes no earlier sentence wrong.

**Figures.** This entry adds no figure to the page. A count of the markdown image tags on the page answers 0 before this entry and 0 after it.

## 2026-09-15 16:22 - #23 - X carries a shorter header, so its post carries a reading

His words, 2026-09-15: *"Should shorten specifically for X and maintain a consistent level of valuable data..."*

The fixed header measures 144 in every unit, because it is plain ASCII. On X, with the call line, the address at 23 and two separators, the floor is 209 of 280, leaving 71. A reading also costs its separator, so a line has to measure 70 or less to fit, and the shortest standard reading on the previous entry's calls measures 59, the Bollinger one 71. Read off the running program on those three calls: not one reading reached X. With twelve voters X carried none of twelve; with three, none of three; with one voter, none of one. Every X body was four lines: the header, the call line, the abbreviation note, the address.

### One header per venue row

The header is a field on the venue's row. Every row takes the fixed header unless it names its own, so six rows are unchanged and X's row names a short one. A venue is changed by editing its row; nothing branches on a venue's name.

`src/trading/ata_spm_push.py` — the two headers, and the row that names its own

```
#: The header a ``PushTarget`` row carries unless the row names its own.
FIXED_HEADER = (
    "This is not investment advice. It is a demonstration of Ekthelius's "
    "proprietary TA engine housed in the Acervator governance execution "
    "platform."
)

#: The header ``TARGET_X``'s row carries in place of ``FIXED_HEADER``.
X_HEADER = "Not investment advice. Acervator TA engine demonstration."
```

```
    PushTarget(
        TARGET_X,
        (SECTION_CALL, SECTION_INDICATORS),
        body_limit=280,
        count_unit=COUNT_WEIGHTED,
        header=X_HEADER,
```

X's header measures 57 weighted units. It keeps the three facts: it is not investment advice, it is Acervator, and it is a TA engine demonstration. The rest of the long form was connective wording. The words are his to change; the three facts are not.

The composer takes the header it joins, and every artefact of a post composes the post's own. The body, the caption, the thread root and the title all carry the header the row named, and the fitter counts that header while it drops lines, so the ceiling is measured against the header the post will carry.

`src/trading/ata_spm_push.py` — the composer, and where a post's header comes from

```
def compose(lines: Any, header: Any = FIXED_HEADER) -> str:
    """``header`` over ``lines`` over ``ORGANIZATION_URL``.

    ``fit_to_target`` drops ``lines`` to reach a ceiling and reaches neither
    ``header`` nor ``ORGANIZATION_URL``.
    """
```

```
def target_header(target: Any) -> str:
    """The header one push target's row carries, ``FIXED_HEADER`` where the row names none."""
```

### What X carries now, on the previous entry's three calls

The same three calls, driven again with the home and the post root redirected and every socket but loopback refused. The floor on X falls from 209 to 122, leaving 158.

| Call | Readings written | Carried on X before | Carried on X after | Dropped after | X body after |
|---|---|---|---|---|---|
| twelve voters, long readings | 12 | 0 | 0 | 12 | 164 of 280 |
| three voters, standard wording | 3 | 0 | 1 | 2 | 223 of 280 |
| one voter, standard wording | 1 | 0 | 1 | 0 | 194 of 280 |

The long call stays at zero because each of its readings measures over 500 on its own, and the note still reports the drop. The one-voter call now carries its reading whole with no note. The X body on the three-voter call:

```
Not investment advice. Acervator TA engine demonstration.
A15/USD on 1hr: bullish reversal called.
RSI: no reading published. Votes bullish at 80% confidence.
Abbreviated: 2 evidence line(s) omitted.
https://github.com/Acervator-LLC
```

The six other venues' bodies were written to disk before and after on all three calls and compared byte for byte: 18 of 18 identical. The three X bodies differed, which is what proves the comparison could see a change. Reddit's title still opens with the fixed header.

### The X picture's foot and the X text agree

The venue picture already draws the venue's own body at its foot, so the short header reaches X's picture through the same post that writes X's text. Read off the run: on all three calls the caption the renderer received equals the X body, and the X text file equals the X body. Both builds produce the same X body on the same call, byte for byte. The root picture, one image for every venue, keeps the fixed header over the headline.

`src/trading/ata_venue_folders.py` — the foot is the body

```
        caption=post.body,
```

### Five sentences this entry overtakes

`docs/manual/08-tabs/market-inspector.md:679` — *"The fixed header ships on every artefact the run composes. No caller supplies it and no caller can remove it. It is 144 characters, and on X it takes over half the budget."* X's artefacts now carry the 57-unit header; the six other venues' still carry the fixed one, and no caller can remove either.

`docs/manual/08-tabs/market-inspector.md:1175` — *"It puts the fixed header on top, the evidence under it, and the link at the bottom."* On X the row's own header is on top; the evidence and the link are as written.

`docs/manual/08-tabs/market-inspector.md:1183` — the quoted docstring of the composer read `FIXED_HEADER` over the lines. The composer now takes the header it joins.

`docs/manual/08-tabs/market-inspector.md:3573` — *"the header, the call line and the address take 209 of X's 280 weighted units, leaving 71 for evidence, so one 43-unit reading fits whole and a longer one is dropped for the abbreviation note."* That was the fixed header. With X's own, the three take 122, leaving 158.

`docs/manual/08-tabs/market-inspector.md:3571` — *"the fixed header and the address are never dropped."* Still true of the header a row carries: the fitter drops evidence lines and reaches neither the header nor the address.

### Seventeen entry stamps read off their commits now

The entry headings at lines 1775, 1895, 2037, 2149, 2227, 2295, 2467, 2549, 2653, 2719, 2783, 2856, 3012, 3123, 3228, 3357 and 3495 were stamped in local time, 6 minutes to 17 hours ahead of the commits that wrote them, and now read the UTC minute of those commits, the stamp alone changed on each line.

**Figures.** This entry adds no figure to the page. A count of the markdown image tags on the page answers 0 before this entry and 0 after it.

## 2026-09-15 18:10 - #23 - An empty ticker field scans the sector by volume until the hits land

Scan Now with nothing in the ticker field re-ran whatever scans the zone already held, and on a fresh zone it ran nothing. The press now reads the sector the menu shows, market by market from the largest volume down, on every ticked timeframe, and stops the moment the target number of hits has landed. A new setting on Level 1, Hits per scan, holds that target and starts at 3.

### What one press does with each of the three things the field can hold

| the field holds | the press |
|---|---|
| a ticker the sector lists | reads that one market, on every ticked timeframe, unchanged |
| a sector name the map lists | reads that sector's assets, unchanged |
| nothing | reads the sector menu's markets, largest volume first, until Hits per scan reversal calls land |

### What a hit is

A hit is one market on one timeframe whose twelve-voter consensus and Bollinger voter name the same non-neutral direction. It is the vote the zone already counts as a reversal call, so the target and the zone count one thing.

`src/trading/ata_spm.py` — the vote a hit is

```
    def is_reversal(self) -> bool:
        """True while the band voter and the consensus name one direction.

        A ``SignalDirection.NEUTRAL`` on either side is not a reversal.
        """
        if self.direction == SignalDirection.NEUTRAL:
            return False
        return self.band_direction == self.direction
```

### Where the markets and their volume come from

The sector menu names the asset class, and every market of that class is read from the lists already in this tree. For crypto the order is the venue's own 24-hour quote volume, the same figure the Refresh press ranks its universe by, read on the connectors already attached and served from a cache for 900 seconds. A Refresh fills the same cache, so a scan inside that window costs no ticker call.

`src/exchange/market_inspector_fetcher.py` — the figures and their window

```
async def fetch_quote_volumes(
    exchange_connectors: dict,
    min_refresh_s: float = DEFAULT_MIN_REFRESH_S,
) -> dict[str, float]:
    """The 24 h quote volume per base symbol across every connector, now.

    Serves _LAST_VOLUMES while it is younger than min_refresh_s, which the
    Refresh press also fills, and asks connector.get_all_tickers otherwise.
    """
```

`src/gui/main_tabs/market_inspector_surface.py` — the order the scan walks

```
def class_markets(asset_class: Any, connectors: Any = None) -> list:
    """Every market one class holds, largest 24 h quote volume first.

    Crypto ranks ``class_tickers`` by ``class_volumes``, a name with no figure
    last by name; forex and metals keep ``ata_asset_maps.MAPS`` order, which
    lists each sector from its highest liquidity tier down.
    """
```

| The sector menu shows | The order the markets are read in |
|---|---|
| crypto, with an exchange connected | the venue's 24-hour quote volume, largest first; a name the venue lists no figure for comes last, by name |
| crypto, with no exchange connected | by name |
| forex | the asset map's order: the seven USD majors, then the minors, then the exotics; every forex bar carries volume 0, so no volume figure exists |
| metals | the asset map's order: the four spot pairs, then the four funds |
| stocks, derivatives | no list exists, and the zone says so |

### The scan walks market by market and stops at the target

Every earlier scan read a whole list on one timeframe before the next. The by-volume scan reads one market on every ticked timeframe, counts its hits, and moves to the next. The moment the target lands the scan ends, and the markets after it are not read.

`src/trading/ata_spm.py` — the loop and its two exits

```
    for symbol in assets:
        read += 1
        for found in frames:
            vote = _vote_one(
                voter, symbol, found.timeframe, candle_source, cost, clock, found
            )
            if vote is not None and vote.is_reversal:
                hits += 1
                if hits >= target:
                    return frames, read, True
    return frames, read, False
```

### The setting, and what zero does

Hits per scan sits under the four settings on Level 1, in both builds, and starts at 3. A box left empty, a zero and a negative number all read as 3, so no scan can stop before it has read anything.

`src/trading/ata_spm.py` — the count every press reads

```
def hits_target(asked: Any) -> int:
    """The reversal calls one empty-field scan stops at.

    Text that is not a whole number, and any count under 1, read as
    ``DEFAULT_HITS_PER_SCAN``.
    """
```

The five settings live for the life of the process. A restart reads the ceiling as unset and Hits per scan as 3.

### The zone says what the scan did

`src/trading/ata_spm.py` — the two lines a by-volume entry is drawn from

```
VOLUME_LINE_FORMAT = "{asset_class} by volume"
VOLUME_META_FORMAT = "{read} market(s) read · {hits} hit(s) · {stop}"
STOPPED_AT_TARGET_TEXT = "stopped at target"
SECTOR_EXHAUSTED_TEXT = "sector exhausted"
```

### Read off both running builds

Both builds were driven with one stand-in listing of 20 markets carrying volumes 2000 down to 100 and one stand-in candle source, the network refused. The candle source answers a tape the real voting engine reads as a bearish reversal call on the daily timeframe for the markets ranked 3, 7 and 15, and a tape it reads as a vote with no call for every other market and timeframe.

| Hits per scan | window | page | markets read |
|---|---|---|---|
| 3 | `crypto by volume / 15 market(s) read · 3 hit(s) · stopped at target` | identical | 1 to 15; 16 to 20 not read |
| 1 | `crypto by volume / 3 market(s) read · 1 hit(s) · stopped at target` | identical | 1 to 3 |
| 10 | `crypto by volume / 20 market(s) read · 3 hit(s) · sector exhausted` | identical | all 20 |
| 3, `A15` typed | `A15 in crypto / 1 market · 2 vote(s) · 0 reversal call(s)` | identical | `A15` only |

The ticker press reads byte for byte what it read before this entry, 197 bytes of zone line, entry and symbol, and the same comparison run against the empty-field entry reports a difference at its first byte.

The ordering was driven on a stand-in connector serving six ticker rows: A23 at 900, A15 at 500, A14 at 300 and 100 under two quotes, A22 with a base volume and a last price, and USDT at 9999. The crypto list read A23, A15, A14, A22 and then the names with no figure by name; USDT was dropped as a stablecoin; the second read inside the window made no connector call. Network calls attempted across every run: 0.

Level 1 and the zone were rendered in both builds inside a 1200 by 900 host and inside the 1400 by 760 pane the smallest window leaves. Hits per scan reads 3 in both at both sizes. Inside the 760 pane the ATA-SPM zone's entry shows one line in the window and none on the page, in both builds, before and after this entry.

### Four sentences this entry overtakes

They were not reworded. They are quoted here.

`docs/manual/08-tabs/market-inspector.md:3231` - "The sector menu still picks the sector, and an empty field still scans the sectors the board holds." An empty field now scans the sector menu's markets by volume.

`docs/manual/08-tabs/market-inspector.md:3240` - "| nothing | runs every scan the board already holds, unchanged |". The row above replaces it.

`docs/manual/08-tabs/market-inspector.md:567` - "Four settings sit there, each one read by a phase, and under them one credential row per push target." Five sit there now; the fifth is read by Scan Now.

`docs/manual/08-tabs/market-inspector.md:2259` - "the five asset classes and the four settings each keep their own control". Five settings keep their own control.

**Figures.** This page carries no figure and this entry adds none. A count of the markdown image tags on the page answers 0 before this entry and 0 after it.

## 2026-09-16 17:53 - #23 - Energy joins the sector set

*ATA-SMP - New Market Sector - Energy - Not sure how I forgot to add this...*

The Asset Category row on Level 1 and the sector menu on the scan page carry six names. Energy is the sixth, after forex. One tuple declares the set, and every button, menu entry, ticker list and scan reads that tuple.

`src/trading/ata_spm.py` — the six names

```
CLASS_CRYPTO = "crypto"
CLASS_STOCKS = "stocks"
CLASS_METALS = "metals"
CLASS_DERIVATIVES = "derivatives"
CLASS_FOREX = "forex"
CLASS_ENERGY = "energy"

#: Every major asset class that charts and takes TA.
ASSET_CLASSES = (
    CLASS_CRYPTO,
    CLASS_STOCKS,
    CLASS_METALS,
    CLASS_DERIVATIVES,
    CLASS_FOREX,
    CLASS_ENERGY,
)
```

### What energy lists

Energy carries two sectors, petroleum and gas, the two groups S&P GSCI names. Each holds funds priced in dollars on the yahoo chart venue, the venue and the row shape metals already uses. Four names are listed: USO for WTI crude, BNO for Brent crude, UGA for gasoline and UNG for natural gas.

`src/trading/ata_asset_maps.py` — the energy row of the map

```
ENERGY_PETROLEUM: tuple[AssetListing, ...] = tuple(
    AssetListing(symbol=one, quote=USD, venue=VENUE_YAHOO, ticker=one)
    for one in ("USO", "BNO", "UGA")
)

ENERGY_GAS: tuple[AssetListing, ...] = tuple(
    AssetListing(symbol=one, quote=USD, venue=VENUE_YAHOO, ticker=one)
    for one in ("UNG",)
)

MAPS: dict[str, dict[str, tuple[AssetListing, ...]]] = {
    CLASS_FOREX: {
        SECTOR_MAJOR: FOREX_MAJOR,
        SECTOR_MINOR: FOREX_MINOR,
        SECTOR_EXOTIC: FOREX_EXOTIC,
    },
    CLASS_METALS: {SECTOR_SPOT: METALS_SPOT + METALS_PHYSICAL},
    CLASS_ENERGY: {
        SECTOR_PETROLEUM: ENERGY_PETROLEUM,
        SECTOR_GAS: ENERGY_GAS,
    },
}
```

The futures the venue also lists are not carried, on the ground the metals entry gives: a chart of them joins contracts at a price nobody traded. Heating oil has no listed instrument, because the venue answered no rows for its fund.

`src/trading/ata_asset_maps.py` — where the four names came from

```
    CLASS_ENERGY: (
        "S&P GSCI groups energy as petroleum and natural gas. The listed "
        "instrument for each product is the fund holding it, measured on the "
        "yahoo chart endpoint 2026-09-15: USO for WTI crude, BNO for Brent "
        "crude, UGA for gasoline, UNG for natural gas, 252 daily rows each, "
        "every bar carrying volume. The futures CL=F BZ=F NG=F answer the same "
        "window and are not carried: a chart of them joins contracts at a "
        "price nobody traded. UHN, the heating oil fund, answered 0 rows, so "
        "no listed instrument carries heating oil."
    ),
```

### What each surface does with the sixth name

| surface | energy |
|---|---|
| Asset Category, Level 1 | a sixth button after forex; pressing it sets the class the next scan uses |
| the sector menu | a sixth entry after forex |
| the ticker field | typing offers USO, BNO, UGA and UNG; no note draws under the field |
| Scan Now with an empty field | reads USO, BNO, UGA and UNG in that order, the map's order |
| the four timeframes | the slower set: 1h, 1d, 1w, 1M |
| Chart Folder | unchanged; one folder per push venue, and none per asset class |

No new file is imported. The class name joins an import the asset map already takes from the sector module, so the built application's start-up path is the one it had.

### Six names, read off both running builds

Level 1 was driven in both builds inside the 1400 by 760 pane and inside a 900 by 760 pane, the network refused. Six buttons drew in both builds, on one row at 1400 and on two rows of three at 900, and no scroll bar drew in either build at either width. The menu opened with six entries in both builds.

Pressing energy set the board's class to energy and its timeframes to 1h, 1d, 1w and 1M, in both builds. Typing U offered UGA, UNG and USO, in both. Scan Now with an empty field read USO, BNO, UGA and UNG in that order, on a stand-in tape carrying no reversal vote, and the zone drew the same two lines in both builds.

```
energy by volume
4 market(s) read · 0 hit(s) · sector exhausted
```

The same driver on the commit before this change read five names on every surface in both builds, and the energy press left the board on crypto. The five existing sectors' lists compared byte for byte before and after: crypto 2103 bytes, stocks 355, metals 981, derivatives 234, forex 2264, each identical. The same comparison reported the energy block as new, so it can see a change. Network calls attempted across every run: 0.

### Sentences this entry overtakes

They were not reworded. They are quoted here.

`docs/manual/08-tabs/market-inspector.md:459` - "Stocks, metals, derivatives and forex share the slower one." Energy shares it too.

`docs/manual/08-tabs/market-inspector.md:3126` - "The menu beside it is the sector menu, and it holds crypto, stocks, metals, derivatives and forex." It holds energy too.

`docs/manual/08-tabs/market-inspector.md:3159` - "Three sectors carry a list and two do not." Four carry a list. Energy holds 4 names.

`docs/manual/08-tabs/market-inspector.md:2259` - "the five asset classes and the four settings each keep their own control". Six asset classes keep their own control.

`docs/manual/08-tabs/market-inspector.md:3876` - "| stocks, derivatives | no list exists, and the zone says so |". The table it sits in has no energy row. Energy reads the asset map's order: USO, BNO, UGA, UNG.

**Figures.** This page carries no figure and this entry adds none. A count of the markdown image tags on the page answers 0 before this entry and 0 after it.

## 2026-09-18 16:10 - #23 - A Scan Now press says what it did, and the zone shows it

> *ATA-SMP - Please proceed back to ATA-SMP and get my Scan button working.*
>
> *Unitize and apply same work flow under the correct Issue. It still does*
>
> *nothing as of the build I am currently running.*
>
> *For this, will need to verify and / or wire in Emitter Network and update*
>
> *what the Inspector's subsystems (including ATA-SMP) write to the logs. Entire*
>
> *idea for the Emitter Network is to provide us signals for verifying proper*
>
> *software function resulting from all user actions or automated sequences...*

### What a press wrote before this entry

A press reached the scan. The scan ran on its own thread, the answer crossed back, and the zone was redrawn. Two lines said so, and nothing else did.

```
ATA-SPM scan on thread ata-smp-scan: compute
ATA-SPM scan on thread MainThread: draw
```

The operator's own console log, six files and 1,185,719 lines, holds twelve of those lines: six presses on 2026-09-15 and 2026-09-16, each pair 0 to 4 seconds apart. No line names a market, a candle count, a hit or a refusal. The Activity Log carried nothing, and the signal handler carried nothing.

The zone did redraw, under a fold. The ATA-SPM zone took one third of the left pane, its control rows took most of that third, and the entry that carries the report sat below the rows in a strip too short to read. Measured on the operator's window, 1536 by 937 logical pixels at 125 % scale:

| his window | window | page |
|---|---|---|
| height the entry asks for | 123 to 139 px | 94 px |
| height the zone gave it | 37 px | 8 px |
| headline visible | 16 of 26 px | 3.5 px |
| meta line visible | 6 px | 0 px |

| his window | window | page |
|---|---|---|
| method line visible | 0 px | 0 px |

The only change he could see was the position line: `0 of 0` became `1 of 1`.

### The press says what it did

Every phase of one press now writes one line at info on the `acervator` logger, so `system.log` and the Console pane carry it. The formats live beside the scan's other texts.

`src/trading/ata_spm.py` — the lines one press writes, in phase order

```
SCAN_PRESSED_TEXT = (
    "ATA-SPM scan pressed: {asset_class} on {timeframes}, ticker '{ticker}', "
    "target {hits} hit(s)"
)
SCAN_BUSY_TEXT = "ATA-SPM scan pressed while a scan is running; press ignored"
MARKET_READ_TEXT = "ATA-SPM read {symbol} on {label} from {venue}: {candles} candle(s)"
MARKET_EMPTY_TEXT = "ATA-SPM read {symbol} on {label} from {venue}: no candles"
MARKET_REFUSED_TEXT = "ATA-SPM read {symbol} on {label} from {venue}: refused, {reason}"
HIT_TEXT = "ATA-SPM hit: {call}"
SCAN_FINISHED_TEXT = "ATA-SPM scan finished: {headline} · {meta}. {method}"
SCAN_NOTE_TEXT = "ATA-SPM scan finished: {note}"
SCAN_EMPTY_TEXT = "ATA-SPM scan finished: no sector to scan"
SCAN_FAILED_TEXT = "ATA-SPM scan failed: {error}"
```

The press line is written on the window thread before the thread starts. The thread start keeps its `compute` line. Each market read writes its own line from the thread. The hit lines and the finished line are written when the answer crosses back, one finished line per sector the run scanned. A press while a scan is running writes the busy line at warning, where it used to write a debug line nobody saw.

One press on the energy sector, read off the console log:

```
ATA-SPM scan pressed: energy on 1hr 1d 1wk 1mnth, ticker '', target 3 hit(s)
ATA-SPM scan on thread ata-smp-scan: compute
ATA-SPM read USO on 1hr from yahoo: 384 candle(s)
ATA-SPM read USO on 1d from yahoo: 365 candle(s)
...
ATA-SPM read UGA on 1hr from yahoo: refused, HTTPError: HTTP Error 404: Not Found
...
ATA-SPM scan on thread MainThread: draw
ATA-SPM hit: USO on 1wk: bearish reversal
ATA-SPM hit: BNO on 1wk: bearish reversal
ATA-SPM scan finished: energy by volume · 4 market(s) read · 2 hit(s) · sector exhausted. USO on 1wk: bearish reversal
```

### Each market read is named

The candle source the scan reads through now answers three things: where it read, what it read, and why it read nothing. The scan's reader on the tab writes the line and the pin from that answer.

`src/gui/main_tabs/market_inspector_surface.py` — the read behind `sector_candles`

```
def sector_candle_read(
    inspector: Any, symbol: Any, timeframe: Any, connectors: Any = None
) -> tuple:
    listing = ata_asset_maps.listing_of(symbol)
    if listing is not None:
        candles, refusal = ata_asset_maps.venue_candle_read(symbol, timeframe)
        return listing.venue, candles, refusal
    if inspector_scan_age(inspector) < CANDLES_FRESH_SECONDS:
        held = inspector_candles(inspector, symbol, timeframe)
        if held:
            return CANDLES_FROM_SCAN, held, ""
    if not connectors:
        return ata_asset_maps.VENUE_EXCHANGE, [], NO_CONNECTOR_TEXT
    return (
        ata_asset_maps.VENUE_EXCHANGE,
        connector_candles(connectors, symbol, timeframe),
        "",
    )
```

| the source | the name in the line | the refusal it can name |
|---|---|---|
| a mapped name, read from its venue now | `yahoo` | the venue's own error, an unserved timeframe, a name no map lists |
| the universe scan's candles, under 15 minutes old | `universe scan` | none; it answers what it holds |
| the exchange connector, read now | `exchange` | `no exchange connected` |

`src/trading/ata_asset_maps.py` — the refusals a venue read names

```
UNMAPPED_TEXT = "no map lists {symbol}"
NO_VENUE_TEXT = "no configured venue lists {symbol}"
UNSERVED_TEXT = "{venue} does not serve {timeframe}"
NO_ADAPTER_TEXT = "no fetcher serves {venue}"
```

The venue's own error reaches the line as the fetcher reports it. A refused Yahoo read on this page's stand-in reads `refused, HTTPError: HTTP Error 404: Not Found`.

### The Activity Log carries the same lines

The scan runs on its own thread, and the Activity Log is a window widget. Each line crosses on a signal beside the one the answer crosses on.

`src/gui/market_inspector.py` — the signal and the writer

```
        #: One phase line of a running scan, and its Activity Log level.
        scanLogged = Signal(str, str)  # noqa: N815 - Qt signal name
```

```
        def _say(self, message: str, level: str = ACTIVITY_INFO) -> None:
            logger.info(message)
            self.scanLogged.emit(message, level)
```

The Inspector tab mixin hands the tab the Live tab's pane, the same callable every other Activity Log writer calls. Under React the pane relays each line to the page the way it relays every other line.

`src/gui/main_tabs/market_inspector_tab.py` — the wiring

```
    def _wire_ata_activity_log(self, inspector: Any) -> None:
        pane = getattr(self, "_status_log", None)
        if pane is None or not hasattr(inspector, "set_activity_log"):
            logger.debug("no Activity Log pane; ATA-SPM phase lines reach the log only")
            return
        inspector.set_activity_log(pane.log)
```

A read that answered candles draws at info. A read that answered none, the busy line and a finished line with no vote draw at warning. A scan that raised draws its error line at error.

### Four pins on the signal handler

Each phase leaves one record on the signal handler, through the one wire every pin uses, `signal_contract.emit`. Each carries an expectation, so the handler holds a verdict and not only a note.

`src/gui/main_tabs/market_inspector_surface.py` — the pin names

```
SCAN_PRESSED_PIN = "inspector.ata.scan_pressed"
SCAN_STARTED_PIN = "inspector.ata.scan_started"
MARKET_READ_PIN = "inspector.ata.market_read"
SCAN_FINISHED_PIN = "inspector.ata.scan_finished"
```

| pin | when | actual | expected | context |
|---|---|---|---|---|
| `scan_pressed` | the press, window thread | whether a thread started | `True` | asset class, ticker text, timeframes ticked, hit target |
| `scan_started` | first thing on the thread | the thread's name | `ata-smp-scan` | asset class, sectors held |
| `market_read` | every symbol on every timeframe, on the thread | the candle count | the floor a vote needs | symbol, timeframe, source, refusal |
| `scan_finished` | the answer's return, window thread | markets read | markets listed, or the count at the stop | per sector: read, listed, hits, stopped, the report line |

The handler writes its file every 500 records. The run's end flushes it, so one press on a fresh instance leaves its records in `~/.acervator_logs/signals/session.jsonl` at once.

### The zone shows the report

The three left zones shared the pane in equal thirds, on both hosts, and the ATA-SPM zone's rows left its entry no room. The zones now share by one tuple, and the ATA-SPM zone takes two shares.

`src/gui/main_tabs/market_inspector_surface.py` — the shares

```
#: The share of the left pane's height each ``LEFT_MODULE_KEYS`` zone takes.
#: ATA-SPM takes two, so its control rows and its entry both stay in view.
LEFT_MODULE_SHARES = (2, 1, 1)
```

The window sets each group's height from the tuple in `_size_zones`, and the page sets each fieldset's flex from the same tuple through `shareFlex`.

`src/gui/web/market_inspector.js` — the page's share

```
function shareFlex(share) {
  var units = Number(share);
  if (!isFinite(units) || units <= ZERO) {
    return EQUAL_SHARE;
  }
  return String(units) + SHARE_FLEX_TAIL;
}
```

### Read off both running builds, and both bundles

Both builds were driven at the operator's window size, 1536 by 937 logical at 125 % scale, on a scratch home with every socket but loopback refused. A loopback stand-in answered the Yahoo chart endpoint for USO and BNO and refused every other ticker with 404, and answered the Coinbase public candle endpoint for A15-USD and A14-USD and refused every other product. The real endpoints were never called.

| press, empty field | log lines | Activity Log lines | pins | zone |
|---|---|---|---|---|
| energy: USO and BNO answered, UGA and UNG refused | 30 | 28 | 27: 1 pressed, 1 started, 24 read, 1 finished | `energy by volume / 4 market(s) read · 2 hit(s) · sector exhausted / USO on 1wk: bearish reversal` |
| crypto: A15 and A14 answered, 118 refused | 248 | 246 | 247: 244 read | `crypto by volume / 120 market(s) read · 0 hit(s) · sector exhausted / No chart carried a reversal vote. No candles came back for 1INCH, AAVE, A34, AGIX, AKT, ALGO and 112 more.` |
| metals: every ticker refused | 20 | 18 | 19: 16 read | `metals by volume / 4 market(s) read · 0 hit(s) · sector exhausted / No candles came back for GLD, PALL, PPLT, SLV.` |

The counts are the same in the window and on the page. The Activity Log lines were read off the Qt pane's text and off the React page's own line elements, and the page held every line the pane held. The finished pin read `actual 4, expected 4` on energy and metals and `120, 120` on crypto, `ok` true on all three.

The entry now draws in view, both hosts, at his size:

| his window | window | page |
|---|---|---|
| ATA-SPM zone height | 378 px, was 252 | 331 px, was 240 |
| height the zone gives the entry | 163 px, was 37 | 94 px of 94, was 8 |
| method line visible | 15 px, was 0 | 15 px, was 0 |

`bot_state.json` on the scratch home compared byte for byte before and after every press, and a planted byte moved the comparison.

Both bundles were built from this entry's code commit and launched in isolation on a scratch home, the Inspector tab opened by a click on its tab item and Scan Now pressed at the position UI Automation reports for the button, with crypto shown and the field empty. No exchange is configured on a scratch home, so every read names `exchange` and the refusal `no exchange connected`:

| bundle | log lines after the press | pins in the handler's file | zone |
|---|---|---|---|
| Qt | 244 | 243 | `crypto by volume / 120 market(s) read · 0 hit(s) · sector exhausted / No candles came back for ...` |
| React | 244 | 243 | identical |

### One line this entry overtakes

`docs/manual/08-tabs/market-inspector.md:3959` - "Inside the 760 pane the ATA-SPM zone's entry shows one line in the window and none on the page, in both builds, before and after this entry." After this entry the zone's entry shows its headline, its counts and its method line in both builds at the operator's window size, and the same three lines in a 1400 by 900 window: the window gives the entry 144 px against the 124 it asks, and the page shows the method line's full 15 px.

**Figures.** This page carries no figure and this entry adds none. A count of the markdown image tags on the page answers 0 before this entry and 0 after it.

## 2026-09-18 17:40 - #23 - An empty field walks the class by a cited volume order and stops at the N-th push-gate hit

> *ATA-SMP - Scan Now - Seems to be expecting a Ticker field entry. This is incorrect. If Ticker field has no entry, this button must scan sector's markets (on each selected TF) in order of decreasing volume until x total number of "hits" (across all selected TFs for the sector; add under settings, default is 3) are found.*
> *Entire idea for the Emitter Network is to provide us signals for verifying proper software function resulting from all user actions or automated sequences...*

### What the walk did before this entry, read against those words

| the words say | the walk did | short |
|---|---|---|
| decreasing volume, every class | crypto by the venue's 24-hour quote volume; forex, metals and energy in map order | metals and energy carry a volume figure the walk never read, and the zone said `by volume` for a class read in map order |
| a hit | the vote whose consensus and Bollinger voter agreed, counted in the walk; every vote then judged again by the live gate chains for the bucket | two gates, so the zone's count and the bucket disagreed |
| across every ticked timeframe | every ticked timeframe of every market | none |
| until x hits | the walk stopped at the x-th agreeing vote | the stop was at the wrong gate |
| the markets read, in order | held in the scan and shown nowhere | none reached the operator |

Driven on the unchanged code with six markets in a known volume order, three ticked timeframes and four planted charts the gate chains fire on, at a target of 3: the walk read all six markets, the zone said `0 hit(s)`, and the bucket loaded four posts.

### What a hit is now

A hit is a vote the live trade gates would fire on: `GateScan.would_fire`, the same verdict that decides whether a post reaches the bucket. The zone's count, the Activity Log's hit lines and the bucket now count one thing.

`src/trading/ata_spm.py` — the walk, the judge and the stop

```
    for symbol in assets:
        scan.markets_read += 1
        votes = [
            vote
            for vote in (
                _vote_one(
                    voter, symbol, found.timeframe, candle_source, cost, clock, found
                )
                for found in frames
            )
            if vote is not None
        ]
        for vote in votes:
            held = judge(vote, scan)
            scan.pulls.append(held)
            if not held.gates.would_fire:
                continue
            scan.hits.append(vote)
```

Each market is read on every ticked timeframe first, because the gate chains read a market's other timeframes through the panel rows. Its votes are then judged in timeframe order, and the hit that reaches the target ends the walk; the markets after it are not read. The two gates are independent, read as numbers on 132 synthetic daily tapes through the real engine and chains:

| tape | consensus | confidence | floor the chain judges at | agreeing vote | gate fires |
|---|---|---|---|---|---|
| fall 2 % a bar for 74 bars, rise 3 % for 6 | bearish | 0.1107 | 0.25 | yes | no |
| rise 0.4 % a bar for 115, fall 4 % for 5, volume rising on the fall | bearish | 0.3377 | 0.2632 | no | yes, fold side |

### Where each class's order comes from

`src/gui/main_tabs/market_inspector_surface.py` — the order and its source

```
def class_markets(asset_class: Any, connectors: Any = None) -> Any:
    """Every market one class holds as an ``ata_spm.MarketOrder``, largest first.

    Crypto ranks ``class_tickers`` by ``class_volumes`` and names its venues
    as the source, or reads by name while no connector is in reach; every
    mapped class ranks ``listing_volumes`` over its ``ata_asset_maps.MAPS``
    rows, and a class whose rows carry no figure keeps map order.
    """
```

| The sector menu shows | The figure the walk orders by | Where it comes from |
|---|---|---|
| crypto, with an exchange connected | the venue's 24-hour quote volume | the connector's ticker rows, served from the Refresh press's cache for 900 seconds |
| crypto, with no exchange connected | none; by name | the zone says `name, no exchange connected for a volume figure` |
| metals: GLD, SLV, PPLT, PALL | the last complete daily bar's volume times its close | one short daily chart call per fund on the chart endpoint, the same adapter the RA tablets read |
| energy: USO, BNO, UGA, UNG | the same | the same |
| forex | none; map order | every currency pair's row says its venue sends volume 0, so no call is made and the zone says `in map order` |
| stocks, derivatives | none | no list exists, and the zone says so |

The chart endpoint sends one volume figure per bar under `indicators.quote[0].volume`, which the adapter already reads into each candle. The published reader `yfinance` reads the same field in its `parse_quotes`, at raw.githubusercontent.com/ranaroussi/yfinance/main/yfinance/utils.py, read 2026-09-18. A fund carries a figure on every bar; a currency pair carries the field and it holds zero.

`src/trading/ata_asset_maps.py` — a fund's figure

```
def venue_quote_volume(symbol: Any, now_ms: Any = None) -> tuple:
    """One listed name's quote volume, and the refusal when none was read.

    The figure is ``complete_bar``'s volume times its close over
    ``VOLUME_TIMEFRAME`` candles of the last ``VOLUME_WINDOW_DAYS``; a row
    whose ``volumed`` is False, and a venue that refused, answer
    ``NO_VOLUME_FIGURE`` with the refusal named.
    """
```

The newest bar stamped before today's UTC day is a closed session, so the figure does not move through the day, and volume times close puts it in dollars beside crypto's quote volume.

### The zone and the Activity Log name the order

A by-volume entry keeps its headline; a class with no volume figure reads `in map order`. The open entry's first phase-one row names the source and the markets read, first read first, and the count not read.

`src/trading/ata_spm.py` — the order line

```
VOLUME_LINE_FORMAT = "{asset_class} by volume"
MAP_ORDER_LINE_FORMAT = "{asset_class} in map order"
ORDER_LINE_FORMAT = "Order by {source}: {markets}"
ORDER_UNREAD_FORMAT = "{markets} ({unread} not read)"
ORDER_UNFIGURED_FORMAT = "{source}, {unfigured} with no figure last by name"
MAP_ORDER_SOURCE_TEXT = "map order, no volume figure"
```

The Activity Log carries the whole order at the press and the markets read at the end:

```
ATA-SPM order for energy: Order by last complete daily bar volume x close on yahoo: UNG, UGA, BNO, USO
ATA-SPM hit: UNG on 1wk: bearish reversal
ATA-SPM hit: UGA on 1d: bearish reversal
ATA-SPM hit: BNO on 1hr: bearish reversal
ATA-SPM scan finished: energy by volume · 3 market(s) read · 3 hit(s) · stopped at target. UNG on 1wk: bearish reversal
ATA-SPM order for energy: Order by last complete daily bar volume x close on yahoo: UNG, UGA, BNO (1 not read)
```

### Two more pins

| pin | written by | carries | verdict |
|---|---|---|---|
| `inspector.ata.volume_order` | the class read, once per press | the class, the source, the order, the count with no figure | actual: markets with a figure; expected: markets listed; a class read in map order reads false |
| `inspector.ata.hit` | the walk, once per hit | the market, the timeframe, the direction, the side that fired, the net score and confidence | actual: the running count; expected: the target; false past the target |

Both reach `signal_contract.emit` and the handler's file. The end pin from the entry above carries `stopped_at_target`.

### Hits per scan holds across a restart

`src/trading/ata_spm_push.py` — the file and the write

```
def settings_path() -> Path:
    """The file ``AtaSpmSettings`` persists ``PERSISTED_SETTINGS`` in."""
    return Path.home() / STATE_DIR_NAME / ATA_SPM_SETTINGS_NAME
```

```
    @hits_per_scan.setter
    def hits_per_scan(self, asked: Any) -> None:
        """Take a count; ``ata_spm.hits_target`` reads 0 and text as the default."""
        self._hits_per_scan = ata_spm.hits_target(asked)
        self.save()
```

The file is `ata_spm_settings.json` beside the fleet state, written on every change to the field on either host and read back when the page is built. The other four settings keep the lifetime they had.

### The window's timeframe boxes toggle the row they show

The four boxes were bound to the crypto class's keys when the tab was built. After the class box moved to energy, the box labelled `1mnth` toggled `1w` and the box labelled `1hr` toggled nothing. Each box now reads its key at the press from the row the class shows, and its accessible name follows the row.

### Six presses read off both running builds

Both builds were driven at the operator's window size, 1536 by 937 logical at 125 % scale, on a scratch home with every socket but loopback refused. A loopback stand-in answered the chart endpoint for the four energy funds with planted volumes UNG 4000, UGA 3000, BNO 2000, USO 1000, the reverse of map order, and planted charts the chains fire on for UNG on 1wk, UGA on 1d, BNO on 1hr and USO on 1d. A stand-in connector answered six crypto names with volumes A23 600, A15 500, A21 400, A34 300, A14 200, A22 100 and charts that fire for A23 on 1wk, A15 on 1d, A21 on 1wk and A34 on 1d. The real endpoints were never called: 0 refused connections in every run.

| press | ticked | Hits per scan | walk | hits | zone |
|---|---|---|---|---|---|
| energy | 1hr 1d 1wk | 3 | UNG, UGA, BNO | 3: UNG 1wk, UGA 1d, BNO 1hr | `energy by volume / 3 market(s) read · 3 hit(s) · stopped at target` |
| energy | 1hr 1d 1wk | 5 | UNG, UGA, BNO, USO | 4 | `4 market(s) read · 4 hit(s) · sector exhausted` |
| energy, no chart fires | 1hr 1d 1wk | 5 | UNG, UGA, BNO, USO | 0 | `4 market(s) read · 0 hit(s) · sector exhausted` |
| energy, volumes reversed | 1hr 1d 1wk | 3 | USO, BNO, UGA | 3: USO 1d, BNO 1hr, UGA 1d | `3 market(s) read · 3 hit(s) · stopped at target` |
| crypto | 1hr 1d 1wk | 3 | A23, A15, A21 | 3: A23 1wk, A15 1d, A21 1wk | `crypto by volume / 3 market(s) read · 3 hit(s) · stopped at target` |
| crypto | 1hr 1d 1wk | 5 | all 120 | 4 | `120 market(s) read · 4 hit(s) · sector exhausted` |

Every row reads the same in the window and on the page: the walk, the hits, the headline, the counts, the open entry's rows and the pins. The Activity Log carried 23, 35, 32, 28, 23 and 267 lines for the six presses, the same on the Qt pane and on the page's own line elements. The pins per press: one order pin, one hit pin per hit, and the press, start, read and end pins the entry above wired. On crypto the order pin read `actual 6, expected 120`, false: six of the 120 map names had a figure.

Hits per scan was typed as 7 through each host's own field, the file read `{"hits_per_scan": 7}`, the window was closed and a second one built on the same scratch home read 7 in the setting and in the field, both hosts.

`bot_state.json` on the scratch home compared byte for byte after every press and read unchanged, and a planted byte moved the comparison.

### The bundles

Both bundles were built from this entry's code commit and launched in isolation on a scratch home with no network capability at all, the Inspector tab opened by a click on its tab item, the class box moved to energy, and Scan Now pressed at the button. No venue can be reached, so every volume read and every chart read is refused, and the class walks in map order and says so:

| bundle | log lines after the press | pins in the handler's file | zone |
|---|---|---|---|
| Qt | 22 | 20: 1 pressed, 1 started, 1 order, 16 read, 1 finished | `energy in map order / 4 market(s) read · 0 hit(s) · sector exhausted / No candles came back for BNO, UGA, UNG, USO.` |
| React | 22 | 20, the same | identical |

The order pin read `actual 0, expected 4`, false, source `map order, no volume figure`, on both. The press took 97 seconds in the Qt bundle and 121 in the React one: the chart adapter retries each refused call three times, sleeping 2 and then 4 seconds between attempts, and a press on a dead network pays that on each of the 4 volume reads and the 12 chart reads. The bundle's own state timer saves the empty fleet every 60 seconds, so `bot_state.json` gained a new stamp during those presses and held the same fleet: 0 bots, an empty bot map. On a 15-second crypto press in the Qt bundle the file read byte for byte unchanged.

### Four sentences and two code blocks this entry overtakes

They were not reworded. They are quoted here.

`docs/manual/08-tabs/market-inspector.md:3814` - "| nothing | reads the sector menu's markets, largest volume first, until Hits per scan reversal calls land |". The count is now of hits the push gates admit.

`docs/manual/08-tabs/market-inspector.md:3818` - "A hit is one market on one timeframe whose twelve-voter consensus and Bollinger voter name the same non-neutral direction. It is the vote the zone already counts as a reversal call, so the target and the zone count one thing." A hit is now a vote the live gate chains would fire on, and the zone, the Activity Log and the bucket count that.

`docs/manual/08-tabs/market-inspector.md:3874` - "| metals | the asset map's order: the four spot pairs, then the four funds |". The four funds now order by their last complete daily bar's volume times close, and the four spot pairs, which no venue lists, come after them.

`docs/manual/08-tabs/market-inspector.md:3918` - "The five settings live for the life of the process. A restart reads the ceiling as unset and Hits per scan as 3." Hits per scan now reads what was last typed.

The loop quoted at `:3888` and the `class_markets` docstring quoted at `:3862` describe the code before this entry.

**Figures.** This page carries no figure and this entry adds none. A count of the markdown image tags on the page answers 0 before this entry and 0 after it.

## 2026-09-18 19:40 - #23 - A typed ticker is recognised across every class and read as one market

*Sector Field - Incorrect naming... This entry field is for reading specific,*

*individual markets on demand and will require TICKER recognition...*

*Entire idea for the Emitter Network is to provide us signals for verifying*

*proper software function resulting from all user actions or automated*

*sequences...*

### What the field did before this entry, read against those words

Recognition stopped at the sector menu. Typing `gl` under crypto offered nothing, because GLD is listed by metals. Typing `gld` under crypto and pressing Scan Now was refused with `crypto holds no ticker gld. Pick one the field offers.` and nothing ran. `A15/USD` under crypto was refused the same way, because the pair form matched no name. `a15` under metals was refused. A typed name reached a market only when it was spelled as the chosen sector's list spelled it. The class box never moved. A market entry counted `reversal call(s)` by the twelve-voter consensus while the bucket counted the push gates, so `USO in metals · 1 market · 3 vote(s) · 0 reversal call(s)` sat beside one chart in the bucket. No signal recorded what the typed text named. Both running builds read the same twelve keystrokes and presses identically.

### Typing offers every class's names, with the class beside each

Each keystroke offers names from every class the maps and the connector list: the shipped sector map's 120 crypto names and the bases the connector's last ticker read listed, forex's 28 pairs, metals' 8 names and energy's 4 funds. Stocks and derivatives list nothing, as before. Each offer names its class. Nothing is fetched on a keystroke: the connector's bases come from the cache the Refresh press and the by-volume press fill.

`src/gui/main_tabs/market_inspector_surface.py` - what one offer reads, and the separators a typed name may carry

```
TICKER_OFFER_FORMAT = "{symbol}  ({asset_class})"
#: The characters a typed ticker may carry between its base and its quote.
TICKER_SEPARATORS = ("-", "_", " ")
TICKER_JOIN = "/"
```

The offers come prefix matches first, the chosen class first inside each, then matches holding the text later in the name, eight at most. Typing `a15/u` under metals offers `A15 (crypto)`, because the part after the slash starts a quote the connector route reads.

`src/gui/main_tabs/market_inspector_surface.py` - the rows the completer and the page's list draw

```
def ticker_matches(typed: Any, asset_class: Any) -> list:
    """The ``ticker_offer`` rows ``typed`` names across every class, prefix
    matches first and the chosen class first inside each, capped at
    ``TICKER_MATCH_LIMIT``. A typed pair offers the crypto base while its
    quote part starts one of ``DEFAULT_QUOTES``; no venue is asked."""
```

The Qt completer shows the offer and writes the bare symbol into the field when one is picked. The page's list does the same: each option's label is the offer and its value is the symbol.

### The press places the text in the class that lists it

The typed text is folded first: upper case, outer spaces stripped, and `-`, _ and an inner space read as `/`. The folded text is then matched against each class's names, the chosen class first and then the others in the order the class box lists them: whole, with the slash removed from both sides, or by the base of a pair whose quote is USD, USDC or USDT. The quote is not kept, because the connector route already tries those three quotes in that order.

| typed | resolves to | class |
|---|---|---|
| `a15`, `A15/USD`, `a15-usd`, `A15USD` | A15 on the exchange | crypto |
| `gld`, `GLD/USD` | GLD on yahoo | metals |
| `eur/usd`, `eurusd`, `eur usd` | EUR/USD on yahoo | forex |
| `A15/EUR` | nothing | EUR is not a quote the connector route reads |
| `ZZZQ` | nothing | no class lists it |

`src/gui/main_tabs/market_inspector_surface.py` - the placement the press resolves

```
def market_listing(ticker: Any, asset_class: Any, connectors: Any = None) -> Any:
    """The ``ata_spm.TickerPlacement`` a typed ticker names, across every class.

    ``placements_of`` walks the chosen class first; a chosen class that
    ``class_tickers`` lists nothing for takes any name no class holds, which
    is what ``TICKER_NO_LIST_FORMAT`` says under the field; a name no class
    holds under a listing class answers None.
    """
```

`src/trading/ata_spm.py` - what a placement carries

```
@dataclass
class TickerPlacement:
    """One typed ticker placed: the row read and the class that lists it.

    ``typed`` is the text as the operator wrote it; ``symbol`` is the name
    the class lists it under, which the scan and the zone carry.
    """

    listing: Any
    asset_class: str
    typed: str = ""
```

At the press, the connector's bases are read through the same 900-second cache the by-volume walk reads, and the connector is asked once when the cache is older. A name two maps hold is placed in the chosen class when the chosen class holds it, else in the first class of the box's order that does.

### The class box moves to the class that lists the market, and says so

When the placed class differs from the chosen one, the answer's return moves the class box, redraws the four timeframe boxes for that class with the ticks that survive, and writes one line to the Activity Log and the console log.

`src/trading/ata_spm.py` - the line

```
CLASS_MOVED_TEXT = (
    "ATA-SPM ticker {ticker} is listed under {placed}; the class box moves "
    "from {chosen} to {placed}"
)
```

A placed market keeps the class that lists it. Changing the class box while a market entry is shown no longer rewrites that entry's class; the box and the next press take the new class.

### The one market is read on every ticked timeframe and its hits are the push gates' hits

The placed market walks the same judged path an empty-field scan walks, with no stop count: every ticked timeframe the venue serves is read, each vote is judged through the live trade gates, and a hit is a vote the gates would fire on. The zone's count, the Activity Log's hit lines and the Ready to Send bucket count one thing.

`src/trading/ata_spm.py` - the meta line and the rows a one-market entry carries

```
MARKET_META_FORMAT = "1 market on {venue} · {votes} vote(s) · {hits} hit(s)"
#: One phase-one row per timeframe of a one-market scan: the candles read
#: and what the vote came to.
MARKET_TIMEFRAME_FORMAT = "{candles} candle(s) · {reading}"
MARKET_HIT_READING = "{direction} vote, hit"
MARKET_REFUSED_READING = "{direction} vote, refused by the gates"
MARKET_NO_VOTE_READING = "no vote"
```

The headline keeps `{ticker} in {asset_class}`. The meta names the venue the market was read on. One phase-one row per timeframe carries the candle count the read answered and the verdict. A venue that answers no candles reads `0 candle(s) · no vote` on each row, and the closed entry's method line keeps the sentence naming the market, as before.

### A name no class lists is refused with the line naming it

`src/trading/ata_spm.py` - the refusal

```
TICKER_UNHELD_FORMAT = "No class lists ticker {ticker}. Pick one the field offers."
```

It draws on the line under the field, nothing runs, and no venue is asked. Under stocks and derivatives, which list nothing, a typed name still scans and the zone names what stopped it, as before.

### One more pin

| pin | written by | carries | verdict |
|---|---|---|---|
| `inspector.ata.ticker_resolved` | the press, once per press with text in the field | the typed text, the symbol placed, its class, its venue, whether the class box moves, or the refusal | actual: the class the text landed in, or empty; expected: the class chosen; ok: a market or a sector was named. A move reads ok with the two classes apart; a refusal reads false |

It reaches `signal_contract.emit` and the handler's file as the six pins before it do.

### Twelve keystrokes and presses read off both running builds

Both builds were driven at the operator's window size, 1536 by 937 logical at 125 % scale, on a scratch home with every socket but loopback refused. A loopback stand-in answered the Yahoo chart endpoint for GLD, SLV, PPLT, USO, BNO, UGA and UNG and refused every other ticker with 404, and a stand-in connector listed A23, A15, A21, A34, A14 and A22. The real endpoints were never called. Three timeframes were ticked on every press.

| typed, class chosen | completer | class box after the press | zone |
|---|---|---|---|
| `gl`, crypto | `GLD (metals)` | - | - |
| `aa`, crypto | `AAVE (crypto)` | - | - |
| `a15/u`, metals | `A15 (crypto)` | - | - |
| `gld`, crypto | - | metals | `GLD in metals / 1 market on yahoo · 3 vote(s) · 1 hit(s) /` rows `384 candle(s) · bearish vote, refused by the gates`, `365 candle(s) · bearish vote, refused by the gates`, `200 candle(s) · bearish vote, hit` |
| `A15/USD`, crypto | - | crypto | `A15 in crypto / 1 market on exchange · 2 vote(s) · 1 hit(s)`; 1hr unserved |
| `a15`, metals | - | crypto | the same entry |
| `eur usd` , crypto | - | forex | `EUR/USD in forex / 1 market on yahoo · 0 vote(s) · 0 hit(s) / No candles came back for EUR/USD.` |
| `ZZZQ`, crypto | - | crypto | unchanged; the line under the field reads `No class lists ticker ZZZQ. Pick one the field offers.`; 0 venue calls |
| `pall`, metals | - | metals | `PALL in metals / 1 market on yahoo · 0 vote(s) · 0 hit(s) / No candles came back for PALL.` |
| `goog`, stocks | - | stocks | `GOOG in stocks / 1 market on no venue · 0 vote(s) · 0 hit(s) / No configured venue lists GOOG.` |
| `uso`, metals, USO planted into the metals map | - | metals | `USO in metals / 1 market on yahoo · 3 vote(s) · 1 hit(s)` |
| `uso`, crypto, same plant | - | metals | the same entry, placed in the first class of the box's order that holds it |

Every zone line was read off the window's own labels and off the page's own elements, and the two read identical on every row. The Activity Log held the same lines in the Qt pane and on the React page: 10 after the first press and 189 after the last. The `ticker_resolved` record read `actual metals, expected crypto, ok true, moved true` on the first press, `actual "", expected crypto, ok false` with the refusal on `ZZZQ`, and `actual metals, expected metals` on `pall`. The hit record on a one-market press carries the count against the number of timeframes read. The scratch `bot_state.json` read byte for byte unchanged after every press, and a planted byte on a copy moved its digest.

### Both bundles place gld under metals with no venue in reach

Both bundles were built from this entry's code commits and launched in isolation on a scratch home with no network capability at all, the Inspector tab opened by a click on its tab item, `gld` written into the ticker field with crypto on the class box, and Scan Now pressed at the button. No venue can be reached, so every chart read is refused and the entry names the market with no candles; the placement needs no venue:

| bundle | class box after the press | log lines after the press | pins in the handler's file | zone |
|---|---|---|---|---|
| Qt | `metals` | 8 | 7: 1 pressed, 1 started, 1 resolved, 3 read, 1 finished | `GLD in metals / 1 market on yahoo · 0 vote(s) · 0 hit(s) / No candles came back for GLD.` |
| React | `metals` | 8 | 7, the same | identical |

The resolved record read `actual metals, expected crypto, ok true, moved true` on both. The four timeframe boxes redrew as metals' four, with 1hr, 1d and 1wk ticked and the crypto-only 5m gone, and the second press's line read `metals on 1hr 1d 1wk`. A second press with `ZZZQ` in the field wrote 4 lines and 4 records on both, the resolved record `ok false` with the refusal, and the page's own line under the field read `No class lists ticker ZZZQ. Pick one the field offers.` The zone headline and meta were read off the window's own labels and off the page's own text elements through UI Automation, and the class box off the window's combo box. The bundle's own state timer saves the empty fleet every 60 seconds, so `bot_state.json` gained a new stamp during the presses and held the same fleet: 0 bots, an empty bot map.

### The Qt timeframe buttons take their label's width

On both bundle captures at 125 % scale the fourth timeframe button read `mnt`: the bold `1mnth` label outgrew the fixed 64 px width. The width is now a floor and the button grows to its own size hint, so the label reads whole. The page's buttons were not clipped and are unchanged.

### Sentences and code blocks this entry overtakes

They were not reworded. They are quoted here.

`docs/manual/08-tabs/market-inspector.md:3141` - "Each letter narrows a list the program already holds. Nothing is fetched and no venue is asked. Crypto reads the shipped sector map. Every other sector reads the asset maps." The list is now every class's names, and crypto's also carry the bases the connector listed.

`docs/manual/08-tabs/market-inspector.md:3239` - "| a ticker the sector lists

| reads that one market, on every ticked timeframe | " and `:3241` - " | a name |
|---|---|---|

the sector lists as neither ticker nor sector | reads nothing, and the line under the field says so |". A ticker any class lists is read, under that class; a name no class lists is refused.

`docs/manual/08-tabs/market-inspector.md:3339-3340` - "The press adds the typed value as a market when the sector lists it as a ticker, and as a sector only when the sector map holds it under that name." The market is added when any class lists it.

`docs/manual/08-tabs/market-inspector.md:3812` - "| a ticker the sector lists

| reads that one market, on every ticked timeframe, unchanged | ". The market |
|---|---|

is now judged through the push gates as the by-volume walk is.

The `market_listing` docstring quoted at `:3248`, the `MARKET_META_FORMAT` line quoted at `:3283` and the `TICKER_UNHELD_FORMAT` block quoted at `:3295` describe the code before this entry, and the ZZZZ row at `:3315` reads the refusal as it was worded then.

**Figures.** This page carries no figure and this entry adds none. A count of the markdown image tags on the page answers 0 before this entry and 0 after it.

## 2026-09-18 21:20 - #23 - The Ready to Send entry carries the painter's chart, the folder opens, and the buttons hand off by the venue's route

His words, 2026-09-15: *"Ready to Send Bucket - Want a folder button that opens the chart image directory. Want the standardized message marked on each image. Want this loading push candidates via a functioning market scanner..."* and *"Should have a catered folder for each venue."* And on posting: *"This will have to do. Do not want to build something that risks myself or others being banned."* And 2026-09-18: *"Entire idea for the Emitter Network is to provide us signals for verifying proper software function resulting from all user actions or automated sequences."*

### What one entry shows now

Each Ready to Send entry draws the chart its own venue folder holds: the PNG `render_chart_png` wrote for that venue, with that venue's message at its foot, scaled to the entry's width. The thumbnail is 120 px wide and the open entry's larger view 320 px wide; each takes the height the image's own shape needs. An entry whose post names no file on disk, a phase-seven follow-up, draws the rectangle strip it drew before.

`src/gui/main_tabs/market_inspector_surface.py` - the image one entry carries

```
def chart_image(path: Any) -> tuple:
    """The painter's PNG one post names, as a data address with its size.

    A post naming no file, or a file that is gone, answers an empty address
    and ``NO_IMAGE_SIZE``, and the entry draws its rectangle strip instead.
    """
```

The Qt `_PostChart` decodes that address into a `QImage` and draws it over its box; the page's `PostChart` places one `img` with the same address in the same box. Both hosts read one `post_chart` payload, so the two draw one picture.

### What one entry names

A post names the files its own venue folder holds, not the root picture.

`src/trading/ata_spm_push.py` - the three files one post names

```
def venue_files(pull: Any, target: PushTarget) -> tuple:
    """The image, text and intent paths ``ata_venue_folders`` wrote for one
    target, or the root image alone where that folder was not written."""
```

`FormattedPost.image_path` is the venue's stamped PNG, `text_path` the message beside it, and `intent_path` the `.url` file, which only X's folder holds. `folder_path` is the venue folder the image sits in.

### The venue folders, written from the scan

Nothing here changed. Phase three writes every venue folder inside the scan, per hit, at the moment the live gate chains fire. Read off the running program after unit S2's energy scan stopped at three hits, in each variant: 3 PNG and 3 TXT in each of the seven folders, 3 URL in `X/`, 24 renders on the scan thread, and the painter's `set_caption` called 24 times with the venue's own body. X's images carry `X_HEADER`; the six others carry `FIXED_HEADER`.

### The Chart Folder press

The button opens the root, so every venue folder is in view, as before. The press now also writes one line to the Activity Log naming the folder it opened, in both variants.

`src/gui/main_tabs/market_inspector_surface.py` - the line the press leaves

```
def chart_folder_line(path: Any) -> str:
    """The Activity Log line a Chart Folder press leaves, naming the root opened."""
    return CHART_FOLDER_OPENED_LOG % (path,)
```

The handler is the operating system's own: `open_path` hands the folder's `file:` address to `webbrowser.open`, which is the file browser for a folder and the default browser for an Internet Shortcut. No browser is driven and nothing is typed into one.

### The three buttons hand off by the venue's route

A post takes one of three routes, decided by `route_for`.

`src/trading/ata_spm_push.py` - the route one post takes

```
def route_for(post: FormattedPost, settings: AtaSpmSettings, sender: Any) -> str:
    """The route one post takes: ``ROUTE_API`` while its venue is signed in and
    a sender is wired, ``ROUTE_INTENT`` while its folder holds an intent file,
    else ``ROUTE_FOLDER``."""
```

| route | when | what the press does | the line |
|---|---|---|---|
| `api` | the venue is signed in and a sender is wired | `deliver_one`, as before: the ceiling, the hour guard, the sender | `<venue> · <symbol> <label> · sent to <destination>` or `· not sent · <detail>` |
| `intent` | X, whose folder holds the `.url` | Post Selected hands the `.url` to the OS handler; the compose window opens with the message typed and the picture is attached by hand | `X · <symbol> <label> · compose address opened · <path>` |
| `folder` | every other venue not signed in | Post Selected hands the venue folder to the OS handler | `<venue> · <symbol> <label> · in folder <path>` |

No sender is wired in the tree today, so every venue takes the folder route and X the intent route. Post All takes every approved post and opens the post root once, so all seven folders are in view from one window; its X lines read `compose address ready`. Send Bucket Full Auto opens nothing and records each post's route. A Post All with no approved post says so, and a Full Auto press says its state. Every line reaches the Activity Log and `system.log` in both variants, prefixed `ATA-SPM`.

`src/trading/ata_spm_push.py` - the lines a press leaves

```
PRESS_LINE_FORMAT = "ATA-SPM hand-off: {line}"
NOTHING_APPROVED_TEXT = (
    "ATA-SPM Post All: no post is approved. Approve one, then press again."
)
FULL_AUTO_TOGGLED_FORMAT = "ATA-SPM Send Bucket Full Auto: {state}."
```

### The two pins

`inspector.ata.candidate` is written once per bucket entry when the scan fills the bucket, with the symbol, the timeframe, the vote, the venue, the venue folder, the image, the text and the intent paths; `ok` is false when the image is not on disk. `inspector.ata.handoff` is written once per post a press took, with the venue, the route, the outcome, the destination and whether the OS handler was given it. Both go through `signal_contract.emit` and nothing else.

`src/trading/ata_spm_push.py` - the two pin names

```
CANDIDATE_PIN = "inspector.ata.candidate"

HANDOFF_PIN = "inspector.ata.handoff"
```

### A scan that leaves no post

The Ready to Send zone reads the ATA-SPM board's own run. After a scan with no hit it says `The last scan left no post. Nothing to approve.` Before it read `Phase source not wired. Nothing to approve.`, the universe scan's wording, after a scan that had run.

### A message format naming a key the sentence does not have

`Standardised message text` on the settings page may carry `label`, `reading`, `direction` and `confidence` in braces. A typed format naming any other key, or one `str.format` cannot parse, no longer stops the scan: the standard wording is written and one warning names the key.

`src/trading/ata_spm.py` - the keys and the refusal

```
MESSAGE_FORMAT_KEYS = ("label", "reading", "direction", "confidence")
MESSAGE_FORMAT_REFUSED_LOG = (
    "ATA-SPM message format refused, %s; the standard wording is used. Keys: %s"
)
```

### Read off the running program

Unit S2's energy plant, three timeframes ticked, three hits, in each variant with the home on a scratch directory and every socket but loopback refused:

| reading | Qt | React |
|---|---|---|
| bucket entries | 21, 3 hits by 7 venues | 21 |
| entries naming their venue's own PNG | 21 of 21 | 21 of 21 |

| reading | Qt | React |
|---|---|---|
| thumbnail | the venue PNG, 1200 by 634 decoded, drawn at 120 by 63 | the same, one `img` in the box |
| files per venue folder | 3 PNG, 3 TXT; X also 3 URL | the same |
| `set_caption` calls, one per PNG | 24 | 24 |
| Chart Folder | the root handed to the OS handler; the Activity Log names it | the same |
| Post Selected, X | the `.url` handed to the OS handler; one `handoff` record, route `intent` | the same |
| Post Selected, six others | the venue folder handed to the OS handler; route `folder` | the same |
| Post All, nothing approved | `no post is approved` on the Activity Log | the same |
| Post All, one approved | the root opened once; the post's line | the same |
| `candidate` records after the scan | 21 | 21 |
| a scan with no hit | 0 entries, no new file, `The last scan left no post.` | the same |
| a changed message format | every venue PNG and TXT carry the new wording | the same |

Nothing driven contacted a venue; connections refused: 0, because nothing asked for one. The OS handler was recorded and not run.

### The files at the root of his folder

The root of `~/.acervator_ata_posts` on the operator's machine holds 26 PNGs named `<symbol>_1d_<stamp>.png`, all dated 2026-09-08, and the seven venue folders hold nothing. No scan wrote them and no Battery run writes any post image: `src/simulator/portfolio_battery.py` names none of the post writers. They are what one earlier unit rendered from its recorded tablets into the real home, before the post root could be redirected. They are not push candidates. Nothing here removes them; the operator clears them when he chooses.

### Three sentences this entry overtakes

They were not reworded. They are quoted here.

`docs/manual/08-tabs/market-inspector.md:541` - "Post Selected sends the post on screen. Post All sends every approved post. Send Bucket Full Auto releases approved posts with no further click. All three obey one ceiling, counted over the hour ending now." The ceiling and the hour guard hold on the `api` route. The `folder` and `intent` routes send nothing, so they obey no ceiling.

`docs/manual/08-tabs/market-inspector.md:1051` - "The Ready to Send zone draws the text and does not show the picture." The zone now draws the venue's own picture in each entry.

`docs/manual/08-tabs/market-inspector.md:3171` - "The Ready to Send zone carries a fourth button. It opens the folder holding the chart images, in the operating system's own file browser." Still true; the press now also says so on the Activity Log.

**Figures.** This page carries no figure and this entry adds none. A count of the markdown image tags on the page answers 0 before this entry and 0 after it.

## 2026-09-19 08:23 - #55 - the venue image draws every voter that called

His words, 2026-09-19: *"let's migrate to the Charts tab and do its visual upgrades. After its chart renderer is fully upgraded, it will need to be migrated to ATA-SMP."* The Charts tab and the venue images share one painter, so the five indicators the tab gained this day reach the venue image through the same overlay registry, with no change to the ATA-SPM render path.

### What the venue image carries now

The registry entry each overlay names its voter on, and the venue image draws every overlay whose voter confirmed the call. ADX, Supertrend, Z-Score, KER and RSI each carry a voter now, and the Z-Score algo point carries the Z-Score voter beside the score, so a Z-Score call draws both.

`src/gui/native_chart.py` - the overlay keys a voter list answers

```
def overlays_for_voters(voters, max_overlays: int = NO_OVERLAY_CAP) -> tuple:
    """The overlay keys drawing ``voters``, and the voters no overlay draws.
```

The image also carries the legend band the Charts tab carries: every overlay named with its last value, the ones not drawn reading `off` in grey.

### The venue readings, off the running program

The scan over the recorded tablets, the home on a scratch directory and every socket but loopback refused: five markets, 2,401 windows of 300 1h bars each, 264 hits. ADX confirmed on all 264. RSI confirmed on none: RSI votes only past 70 or under 30, against the trend the other voters read, so no recorded call carried it. The reading was taken on the hit with the most new voters, and the RSI route on the same render call the venue folder makes.

| reading | value |
|---|---|
| the hit | A34/USD 1h, bearish, 7 confirming voters: Vortex, MACD, Stoch RSI, Slingshot, ADX, KER, Supertrend |
| venue PNGs written | 7, one per venue folder |
| overlays drawn on each | vortex, macd, stochrsi, slingshot, adx, supertrend, ker; undrawn none |
| the same call on the commit before | drawn vortex, macd, stochrsi, slingshot; undrawn adx, kaufman_er, supertrend |
| ADX 20 and 25 on the X venue PNG | 698 and 707 pixels on each ruled row differ from a repaint with no rules; 0 on a control row |
| the render call with voters adx and rsi | drawn adx, rsi; undrawn none; on the commit before, drawn none |
| RSI 30 and 70 on that PNG | 727 and 719 pixels on each ruled row differ; 0 on a control row |

### One sentence this entry overtakes

It was not reworded. It is quoted here.

`docs/manual/08-tabs/market-inspector.md:1024` - "A confirming voter with no overlay cannot be drawn, and the picture says so in its header line rather than passing over it. Z-Score and RSI are the two that vote and have no overlay today." Every one of the twelve voters has an overlay now; the header line still names any voter the cap cuts.

**Figures.** `artifacts/u55/C2/after/u55c2_venue_X.png` is the X venue PNG of that hit and `artifacts/u55/C2/after/u55c2_venue_route_adx_rsi.png` the render with the adx and rsi voters; the before pair sits under `artifacts/u55/C2/before/`. A count of the markdown image tags on this page answers 0 before this entry and 0 after it.

## 2026-09-19 09:10 - #23 - Scan Now works under the operator's own conditions: one rollup, a trading-only list, the scan on screen while it runs, stocks with nothing typed

> *Simulator - Scan Now - I should not have to have anything in the Ticker*
>
> *field. Stocks is saying 'no ticker list'. This is scaffolded, unverified*
>
> *garbage. I watched you perform various operations while working on this.*
>
> *How the fuck is it that after I launch the build, not a god damn thing has*
>
> *improved with this fucking button.*

The operator's correction, the same day: *"Make sure you correct my incorrect verbatims because SImulator does not have Scan Now...Inspector and ATA-SMP do."* Every `Simulator - Scan Now` above reads `ATA-SMP - Scan Now` on the Inspector tab; the words stand as he typed them.

> *If we are having difficulty sourcing the data for free we will need to*
>
> *implement an API driven solution or get clever...*

### What his press did, read off his own log

Build `1517.gca1ff96b-qt`, 07:36:51 to 07:38:18, `crypto on 1hr 1d 1wk, ticker '', target 3 hit(s)`. The order line ranked 120 markets. Every one of the 120 `1wk` reads answered `no candles`, each after three refused calls: the venue lists no weekly granularity, and the scan asked for one anyway. The `1hr` tick read nothing, because the exchange's timeframe table in this tree was a fixed pair, daily and weekly. Thirty products the venue no longer trades were fetched on every timeframe. 182 candle fetches recorded an API block that the pane's thread guard refused, so the API Interaction Log stayed empty and a `thread_violation` file filled instead. The zone and the button did not change for 87 seconds. Fourteen more presses each wrote `press ignored` to the Live tab's Activity Log, which is not the tab he was on. The end was `0 hit(s)`. Stocks read `No ticker list for stocks`.

### One rollup, and only what the venue serves

Two rollups sat in the tree: `_resample_daily_to_weekly` in the Market Inspector fetcher, seven-bar windows from wherever a list began, and `_rollup` in the stone tablets' registry, calendar buckets that every derived tablet timeframe reads through. The registry's `_rollup` is the one definition now. The fetcher's helper is gone; `weekly_from_daily` calls `_rollup` with the week bucket moved onto Monday 00:00 UTC, the day the Yahoo chart endpoint stamps its weekly bar with.

`src/exchange/market_inspector_fetcher.py` - the one rollup

```
def weekly_from_daily(daily: list[_Candle]) -> list[_Candle]:
    rows = [
        [
            one.timestamp * 1000 - MONDAY_OFFSET_MS,
            one.open,
            one.high,
            one.low,
            one.close,
            one.volume,
        ]
        for one in daily
    ]
    return [
        _Candle(
            timestamp=(int(row[0]) + MONDAY_OFFSET_MS) // 1000,
            open=float(row[1]),
            high=float(row[2]),
            low=float(row[3]),
            close=float(row[4]),
            volume=float(row[5]),
        )
        for row in _rollup(rows, DAYS_PER_WEEK, WEEK_MS)
    ]
```

The scan asks a venue only for what its own table lists. `exchange_timeframes` reads each connector's `_ex.timeframes` and adds `1w` while `1d` is there; every crypto row carries that table as `served`, and `AssetListing.serves` reads it. `fetch_symbol_timeframe` asks a connector for `1w` only when its table lists it, reads `1d` and rolls up when it lists `1d`, and asks nothing when it lists neither. On the connector the operator runs, 300 daily candles give 44 weekly bars, above the 30 the voters need.

`src/exchange/market_inspector_fetcher.py` - the timeframe read

```
def exchange_timeframes(exchange_connectors: Any) -> tuple[str, ...]:
    found: list[str] = []
    tables = [
        getattr(getattr(one, "_ex", None), "timeframes", None) or {}
        for one in (exchange_connectors or {}).values()
    ] or [CoinbasePublicCandles.GRANULARITY_S]
    for table in tables:
        for key in table:
            if str(key) not in found:
                found.append(str(key))
    if DAILY_TIMEFRAME in found and WEEKLY_TIMEFRAME not in found:
        found.append(WEEKLY_TIMEFRAME)
    return tuple(found)
```

### The market list holds only what the venue trades

`trading_products` reads the connector's loaded market table, the one `load_markets` filled at connect, and answers each base as trading while its product's `status` is online and `trading_disabled` is false. A sector-map name the venue does not trade goes on `MarketOrder.dead`, is named once on the order line, and is never fetched. With no connector in reach the same answer comes from `public_products`, the venue's public products list.

`src/trading/ata_spm.py` - the order line's tail

```
ORDER_DEAD_FORMAT = "{line}; {count} not trading, never fetched: {names}"
```

On the operator's list the order line now reads 90 trading products in volume order and then `30 not trading, never fetched: AGIX, AR, BOME, CBBTC, DYDX, ENJ, FXS, GALA, GMX, GUSD, IMX_G, JUP, MANTA, MATIC, NEAR_AI, PIXEL, POLYX, PYUSD, RNDR, RUNE, SC, SCRT, STETH, TRU, USDC, USDP, WBTC, WETH, WORMHOLE, XMR`.

### The scan is on screen while it runs

`_scan_until_hits` hands a callable one `ScanProgress` after each market and each hit. The tab's `_compute_scan` passes `_on_scan_progress`, which emits `scanProgressed` beside `scanLogged`; the GUI-thread slot writes `SectorBoard.progress`, and the ATA-SPM zone draws it as its headline through `zone_view`. The Scan Now button is disabled and reads `Scanning…` from the press until the answer or the failure crosses back, the shape the Live tab's Start button takes while a bot runs. Under React the skin carries `scan_running` and `scan_busy_label`, and `ScanNowButton` draws the same.

`src/trading/ata_spm.py` - the zone's running line and the button's label

```
SCAN_PROGRESS_FORMAT = "Scanning {asset_class} · {read} of {total} · {hits} hit(s)"
SCAN_LISTING_FORMAT = "Scanning {asset_class} · reading the market list"
SCAN_BUSY_LABEL = "Scanning…"
```

`src/gui/market_inspector.py` - the crossing and the button

```
        def _on_scan_progress(self, progress) -> None:
            """The walk's report, on the scan thread: cross to the GUI thread."""
            self.scanProgressed.emit(progress)
```

```
        def _set_scan_busy(self, busy: bool) -> None:
            button = getattr(self, "_scan_now_btn", None)
            if button is not None:
                button.setEnabled(not busy)
                button.setText(ata_spm.SCAN_BUSY_LABEL if busy else SCAN_NOW_LABEL)
            self._render_ata_row()
```

A scan thread that raises now emits `scanFailed`; the slot says the failed line, clears the record and frees the button. Before, the button was never disabled, so nothing needed freeing.

### The counter's place, and the field's lines

*Can see the scan counter in the upper left corner of the lower section.*

> *Would rather have this counter just above and at the right corner of the same field. Then have the field display the results of each scanned market.*

The counter is no longer the entry's headline. It sits at the right end of the stepper's position row, just above the field, in both variants: the Qt stepper's `counter_label` after the row's stretch, and the page's `zone-counter` element pushed to the row's right. `zone_view` carries it as `counter`, empty while no scan runs.

While the scan walks, the field draws one line per market as it is read, newest last: the market, the timeframes that answered candles, the vote count and the verdict - the timeframes a hit landed on, the gates the vote's own chain blocked, no vote, or no candles. `market_line` builds each one after `_scan_until_hits` judges a market, it rides on `ScanProgress.line`, the GUI-thread slot appends it to `SectorBoard.progress_lines`, and `zone_view` answers the lines as the entry's rows. The report replaces them at the end, as before.

`src/trading/ata_spm.py` - one market's line

```
SCAN_MARKET_LINE_FORMAT = "{symbol} · {timeframes} · {votes} vote(s) · {verdict}"
SCAN_MARKET_HIT_FORMAT = "hit on {labels}"
SCAN_MARKET_BLOCKED_FORMAT = "no hit: the gates blocked {names}"
SCAN_MARKET_NO_VOTE_TEXT = "no hit: no vote"
SCAN_MARKET_UNREAD_TEXT = "no candles"
```

Read on the Qt window mid-scan at 1400 by 900: the counter's rectangle `[517, 322, 162, 24]` ends at x 679, the field's right edge (`[25, 350, 654, 144]`), and sits 28 px above the field's top; the field held 27 lines when the counter read `27 of 90`, the last `DASH · 1hr 1d 1wk · 3 vote(s) · no hit: the gates blocked ...`. At the end the counter read empty and the field held the report.

### Every fetch is a block on the API Interaction Log

The connector records each candle call on the API Interaction Log from the thread that made it. The pane's handler, `_on_api_event`, refuses any thread but the GUI thread and writes a `thread_violation` file instead. The main window now carries one signal, `apiEntryLogged`, and one receiver beside the handler, `_cross_api_event`. The Live tab registers the receiver as the listener; every entry, from any thread, crosses the signal and lands in `_on_api_event` on the GUI thread, the way the Simulator crosses its venue calls.

`src/gui/main_window.py` - the crossing

```
    def _cross_api_event(self, entry: dict) -> None:
        """The `APIInteractionLog` listener: emit `apiEntryLogged`, which Qt
        queues onto the GUI thread for `_on_api_event` from any other thread."""
        self.apiEntryLogged.emit(entry)
```

The public Coinbase route and the Yahoo chart route record one `FETCH_OHLCV` block per fetch whose reason names the scan; the screener and the products list record one `FETCH_MARKETS` block each. The Yahoo route now keeps the same gap between calls the exchange connector keeps, 0.1 s.

### Stocks scans with nothing typed

At press time the stocks list is Yahoo's predefined `most_actives` screener, unauthenticated, ranked by `regularMarketVolume`, each quote carrying its sector. `screener_listings` reads it through the same request route the chart adapter uses. When the screener refuses, or answers no quote, the list is `STOCKS_PORTFOLIO`: the operator's RA portfolio equities from the simulator's portfolios, every non-crypto name the metals and energy maps do not carry, 47 names, in map order. The order line names which list was read.

`src/trading/ata_asset_maps.py` - the two lists

```
SCREENER_URL = "https://query1.finance.yahoo.com/v1/finance/screener/predefined/saved"
SCREENER_ID = "most_actives"
SCREENER_COUNT = 100
```

```
STOCKS_PORTFOLIO: tuple[AssetListing, ...] = tuple(
    AssetListing(symbol=one, quote=USD, venue=VENUE_YAHOO, ticker=one)
    for one in SYMBOLS
    if one not in CRYPTO_SYMBOLS and one not in _MAPPED_FUNDS
)
```

`src/gui/main_tabs/market_inspector_surface.py` - what the order line names

```
STOCKS_SCREENER_SOURCE_FORMAT = "{source}, {sectors} sector(s)"
STOCKS_PORTFOLIO_SOURCE_FORMAT = "RA portfolio equities in map order, the screener refused: {refusal}"
```

The ticker field no longer says `No ticker list for stocks`; it offers the 47 names. Derivatives still lists nothing.

### With no connector in reach

A crypto scan with no exchange connected used to refuse every read with `no exchange connected`. It now reads the products list and the candles through `CoinbasePublicCandles`, the app's own public Coinbase Exchange route, at the connector's own interval. The order is by name and the line says so: `name on the coinbase public products list, no volume figure`. The public route serves no volume figure.

### Two pins

`inspector.scan.progress` carries class, read, total and hits, once every ten markets and at the end. `inspector.scan.list_source` carries class, source, count and the names not trading, once per list read. Both go through `signal_contract.emit`, beside the seven pins S1 to S4 left.

### The operator's press, read off the running program

The operator's press, crypto on 1hr 1d 1wk with nothing typed, on the real window in each variant, with the home on a scratch directory and every socket but loopback refused. The stand-ins answer the sources' documented limits: a 400 for a granularity the exchange does not list, `[]` for a product it does not trade, at most 300 candles; a ccxt backend whose table has no `1w`; a Yahoo chart serving 1h, 1d, 1wk and 1mo; a screener answering 100 quotes with sectors, or a 429 on a plant.

| reading | before, both variants | after, Qt | after, React |
|---|---|---|---|
| `1wk` reads with candles | 0 of 120 | 44 candles each, from the daily rollup | the same |
| `1hr` reads | 0 | one per market, 300 candles | the same |
| asks for a weekly granularity | 360 | 0 | 0 |
| asks for a product not trading | 180 | 0 | 0 |
| order line | 120 names, 33 with no figure | 90 trading, then `30 not trading, never fetched` by name | the same |
| zone while running | `No sector added.` | `Scanning crypto · 1 of 90 · 0 hit(s)`, counting up | the same on the page |
| button while running | enabled, `Scan Now` | disabled, `Scanning…` | the same on the page |
| API Interaction Log blocks during the scan | 0 | one per fetch, 167 by the stop | 167 on the page |
| `thread_violation` lines | 270 | 0 | 0 |
| a press while running | the busy line on the Live tab's Activity Log | the button is disabled; the handler, reached, still writes the busy line there | the same, on the page too |
| end state | `0 hit(s) · sector exhausted` on the full walk | `28 market(s) read · 3 hit(s) · stopped at target` | the same |
| stocks with nothing typed | `No ticker list for stocks`; `No asset source wired for stocks.` | `yahoo most_actives volume, 11 sector(s)`, 100 names; `3 market(s) read · 3 hit(s)` | the same |
| stocks, the screener answering 429 | the same refusal | `RA portfolio equities in map order, the screener refused: HTTPError: HTTP Error 429`, 47 names; `43 market(s) read · 3 hit(s)` | the same |
| no connector, crypto | 240 reads refused `no exchange connected` | 90 markets through the public route, `1wk` 44 candles, 174 calls at 7.9 a second | the same |
| an entry recorded off the GUI thread | one `thread_violation` line, nothing on the pane | on the pane, no violation | the same |

`bot_state.json` in the scratch home read byte-identical after every press; a planted byte moved the comparison. Nothing driven contacted a venue; connections refused: 0, because nothing asked for one.

### Seven sentences this entry overtakes

They were not reworded. They are quoted here.

`docs/manual/08-tabs/market-inspector.md:155` - `"_resample_daily_to_weekly`

**Derives the weekly series on the client when the venue lists no weekly**

timeframe". That helper is gone; `weekly_from_daily` does it through the registry's `_rollup`.

`docs/manual/08-tabs/market-inspector.md:1363` - "Stocks and derivatives still answer nothing, and the map says why." and "no list of it sits in this tree." Stocks answers the screener's list at press time and `STOCKS_PORTFOLIO` in the tree; derivatives still answers nothing.

`docs/manual/08-tabs/market-inspector.md:3061` - "a crypto name, no exchange connected | nothing, and the zone says so". The public route reads it now.

`docs/manual/08-tabs/market-inspector.md:3160` - "Stocks and derivatives hold none, because no list of their membership sits in this tree. The field says so under itself and still takes a typed name." Stocks holds 47; the field says nothing under itself for stocks.

`docs/manual/08-tabs/market-inspector.md:3873` and `:4503` - "crypto, with no exchange connected | by name". Still by name; the zone now says `name on the coinbase public products list, no volume figure`.

`docs/manual/08-tabs/market-inspector.md:4242` and `:4254` - the `sector_candle_read` block returning `NO_CONNECTOR_TEXT`, and the row "the exchange connector, read now | `exchange` | `no exchange connected`". That branch reads `public_candles` now and names no refusal.

**Figures.** This page carries no figure and this entry adds none. A count of the markdown image tags on the page answers 0 before this entry and 0 after it.

## 2026-09-19 17:40 - #55 - the venue image paints in the theme in force

His words, 2026-09-07: *"Refining our chart renderer ... will allow us to upgrade the visuals for one and align this to visuals of out going SM posts thus unifying direct usage with SM exposure."* The venue image and the Charts tab share one painter, so the painter's skin from this day reaches every venue image: the theme's tokens for every colour, the crisp grid and wicks, the right-edge tags, the design system's type, and no resize grip on an image. A venue image whose caller names no theme now paints in the theme the window is in, and the message stamp at its foot is drawn in that theme's own axis text colour.

`src/gui/native_chart.py` - the theme the venue image takes

```
painter.set_theme(tokens if tokens is not None else theme_in_force())
```

### The venue readings, off the running program, under two themes

Read on the recorded A15 1h candles with a bearish call carrying the vortex, macd, adx and rsi voters and a caption, the theme applied through the theme manager before each call: under Neon Light the image's top-left pixel reads `#ffffff`, the theme's chart ground, and its foot pixel `#f0f0f8`, the theme's lower ground; under Cyberpunk Dark `#08080e` and `#0c0c16`. The two files differ, each carries one `charts.theme.applied` row naming its theme, and both draw all four voters. The venue image carries no resize grip: only a chart in a window draws one.

### The sentence the theme in force overtakes

Not reworded, quoted here. `docs/manual/08-tabs/market-inspector.md:978` - "and a `QImage from render_chart_png." The image now paints in the theme in force when` render_chart_png` is given no tokens; it painted Cyberpunk Dark before.

## 2026-09-19 20:10 - #55 - each venue image at its venue's own size

His words, 2026-09-07: *"Refining our chart renderer (the one currently used under the Asset Charts Tab) will allow us to upgrade the visuals for one and align this to visuals of out going SM posts thus unifying direct usage with SM exposure."* And 2026-09-19: *"After its chart renderer is fully upgraded, it will need to be migrated to ATA-SMP."* Every venue folder now takes its image at the size that venue's own page publishes for a single image post, drawn by the one painter with the call's confirming indicators, the badge, the theme in force and the venue's message at the foot. Before this day every venue got the same 1200-wide image at whatever height the panes needed.

### The seven sizes and the pages that publish them

| venue | size | shape | the venue's page |
|---|---|---|---|
| X | 1200 by 675 | 16:9 | docs.x.com names a 5 MB image limit and no dimension; the figure is the operator's |
| Instagram | 1080 by 1350 | 4:5 | the Instagram media reference: a ratio from 4:5 to 1.91:1, a width from 320 to 1440 |
| Threads | 1080 by 1350 | 4:5 | the Threads posts reference: a ratio up to 10:1, a width from 320 to 1440 |
| Facebook | 1200 by 630 | 1.91:1 | the Facebook sharing images page: at least 1200 by 630, as close to 1.91:1 as possible |
| LinkedIn | 1200 by 627 | 1.91:1 | LinkedIn help, custom images for posts: a 1.91:1 ratio, 1200 by 627 |
| TikTok | 1080 by 1920 | 9:16 | TikTok in-feed specifications: vertical 9:16 at 540 by 960 or more |
| Reddit | 1200 by 628 | 1.91:1 | the Reddit image ad specifications: 1200 by 628 landscape; the figure is the operator's |

Instagram and Threads take 4:5 because it is the tallest shape Instagram accepts, which gives the panes the most room, and the two Meta feeds then carry one picture. Each row of the push-target table carries its two figures, and the folder write hands them to the painter.

`src/trading/ata_spm_push.py` - a row's size

```
    PushTarget(
        TARGET_X,
        image_width_px=1200,
        image_height_px=675,
        sections=(SECTION_CALL, SECTION_INDICATORS),
```

`src/trading/ata_venue_folders.py` - the size reaching the painter

```
    width_px = int(target.image_width_px)
    height_px = int(target.image_height_px)
    image = render_chart_png(
        candles,
        vote.symbol,
        ata_spm.timeframe_label(vote.timeframe),
        image_path,
        voters=[one.indicator for one in ata_spm.confirming_signals(vote)],
        max_overlays=int(max_supporting_indicators or ata_spm.NO_INDICATOR_CAP),
        direction=vote.direction_text,
        readings=[(one.indicator, one.message) for one in pull.messages or ()],
        caption=post.body,
        width_px=width_px,
        height_px=height_px,
    )
```

### How the painter fits a call into a venue's height

A height the natural layout fits draws the natural layout, and any height left over goes to the price pane. A height it does not fit compresses, in this order: each sub-pane shrinks from 60 toward 28 pixels; then the reading strip folds from one row per reading to one row of voter names with their swatches, because the caption already carries every reading's sentence; then the price pane shrinks from 220 toward 120 pixels. A height under that least layout is refused by name, the folder keeps its text file, and the emitter row for that venue reads not ok.

Since 2026-09-20 a sub-pane's natural height is the Charts tab's readable figure, 84 pixels, so the fold starts from 84 toward 28; the order, the strip fold, the price pane's floors and the refusal are unchanged. The rule is on the Asset Charts page under the oscillators.

`src/gui/native_chart.py` - the fold and the refusal

```
            strip_folded = bool(self._call_readings) and h < self._minimum_height_for_panes(w)
```

```
        asked_height = int(height_px)
        if asked_height > NO_IMAGE_HEIGHT:
            least = painter._least_height_for_panes(int(width_px))
            if asked_height < least:
                return ChartImage(
                    note=IMAGE_TOO_SHORT_NOTE.format(
                        width=int(width_px), height=asked_height, least=least
                    ),
```

### The seven images of one hit, read off the running program

The scan under his conditions, both variants, on the operator's tablet copies through a Coinbase-shaped connector: two calls, A27 1d bearish and AXS 1h bearish, seven confirming voters each. The A27 images, read off the disk and off the painter's own heights at each width:

| venue | image | natural height | reading strip | sub-panes | price pane | caption lines |
|---|---|---|---|---|---|---|
| X | 1200 by 675 | 758 | one row per reading, 7 rows | 4 at 39 px | 221 px | 5 |

| venue | image | natural height | reading strip | sub-panes | price pane | caption lines |
|---|---|---|---|---|---|---|
| Instagram | 1080 by 1350 | 849 | 7 rows | 4 at 60 px | 721 px | 12 |
| Threads | 1080 by 1350 | 784 | 7 rows | 4 at 60 px | 786 px | 7 |
| TikTok | 1080 by 1920 | 862 | 7 rows | 4 at 60 px | 1278 px | 13 |
| Facebook | 1200 by 630 | 862 | folded to 1 row | 4 at 28 px | 206 px | 13 |
| LinkedIn | 1200 by 627 | 849 | folded to 1 row | 4 at 28 px | 216 px | 12 |
| Reddit | 1200 by 628 | 862 | folded to 1 row | 4 at 28 px | 204 px | 13 |

The least height for this call is 440 at 1200 wide, so every venue holds it. The bucket thumbnails follow the shapes: X 120 by 68, Instagram 120 by 150, LinkedIn 120 by 63, TikTok 120 by 213.

Every image carries the same seven overlays, the bearish badge, the theme's ground at its top-left pixel and its lower ground at its foot pixel, under Cyberpunk Dark and, after a Theme menu press and a second scan, under Neon Light (`#ffffff` and `#f0f0f8` on all fourteen files).

### A size the layout cannot hold

A row planted with a 200-pixel height on the same hit: the painter writes no file and answers `1200x200 cannot hold the panes: 440 px is the least height at that width.`; the folder keeps the text file; the log names the venue; the emitter row reads actual `[0, 0]` against expected `[1200, 200]`, not ok.

### The image-size emitter

`inspector.ata.image_size` writes once per venue image: the written size against the row's size, with the venue, the symbol, the timeframe, the overlays drawn, the path and the painter's note.

`src/trading/ata_venue_folders.py` - the row

```
    _pin_emit(
        IMAGE_SIZE_PIN,
        actual=[image.width_px, image.height_px],
        expected=[width_px, height_px],
        context={
            "venue": target.name,
            "symbol": vote.symbol,
            "timeframe": vote.timeframe,
            "width": width_px,
            "height": height_px,
            "overlays": list(image.drawn),
            "path": image.path,
            "note": image.note,
        },
    )
```

### The bucket thumbnail

Each Ready to Send entry is one venue's post, and its thumbnail is that venue's own image scaled to the thumbnail width, so the X entry shows a 16:9 picture and the TikTok entry a 9:16 one. Nothing there changed on this day.

### The sentence the venue sizes overtake

It was not reworded. It is quoted here.

`docs/manual/08-tabs/market-inspector.md:5134` - "the venue PNG, 1200 by 634 decoded, drawn at 120 by 63". That was the X image on its day; the X image is 1200 by 675 now, drawn at 120 by 68.

**Figures.** `artifacts/u55/C5/` holds the seven images of one hit under both themes, the tab's chart beside three of them, and the refused-size reading. ## 2026-09-19 21:30 - #23 - A hit sounds a chime, and each hit starts its own confirmation read timer

His words, 2026-09-19: *"Simulator - Scan Now - ... There should be a chime when there is a hit. Once a hit is made, an isolated confirmation read timer starts based on the original hit's TF. The confirmation read mechanic should be partially documented already if not implemented..."* His correction, the same day: *"Make sure you correct my incorrect verbatims because SImulator does not have Scan Now...Inspector and ATA-SMP do."* The `Simulator - Scan Now` above reads `ATA-SMP - Scan Now` on the Inspector tab; the words stand as he typed them. And 2026-09-18: *"Entire idea for the Emitter Network is to provide us signals for verifying proper software function resulting from all user actions or automated sequences."*

### The chime

A hit sounds one chime the moment the window learns of it: during the walk, when the counter's hit figure rises, and at the scan's end for a hit the walk did not report, such as a typed ticker's. A confirmation sounds one more. A failed call sounds nothing, because the chime marks something to act on and a failure closes the call.

The tone is the Audio Suite's own generator. One preset joins its table, four partials over an 880 Hz base, rendered once per process to a 0.6 second wav with a decay applied so the tone rings out, and played through one `QAudioOutput` and one `QMediaPlayer` on the tab, so both variants play the same bytes from the host.

`src/gui/audio_suite.py` - the preset and its figures

```
CHIME_PRESET = "Hit Chime"
CHIME_PARTIALS = [(1.0, 0.45), (2.0, 0.18), (3.0, 0.09), (4.16, 0.05)]
CHIME_SECONDS = 0.6
CHIME_BASE_HZ = 880.0
```

The chime plays at 0.5, the volume the suite's music player opens at. The Audio Suite holds no mute control and the window does not build the suite, so the chime has no mute and no slider of its own.

### The timer

Each hit starts its own timer the moment it lands in the Ready to Send bucket. The interval is one candle of the hit's own timeframe: a 1d hit reads at each daily close, a 1wk hit at each Monday 00:00 UTC, the day the fetcher's rollup and Yahoo stamp a week with; a 1M hit at the first instant of each month. The first read waits for the candle the hit was read on to close.

`src/trading/ata_spm_push.py` - one candle of each timeframe

```
TIMEFRAME_SECONDS = {"5m": 300, "1h": 3600, "1d": 86400, "1w": 604800}
MONTH_TIMEFRAME = "1M"
```

The next read is the first close after now on the grid of closes that begins at the hit's candle. The board keeps the timers, so a tab change loses nothing and both variants read one set; the tab's own clock wakes once a second and hands every due timer to one worker thread, which reads the market's newest candles through the scan's own route and judges the call. The outcome crosses back to the window thread, which writes the entry, the field, the Activity Log and the pin.

`src/trading/ata_spm_push.py` - what one timer holds

```
@dataclass
class FollowUpTimer:
    """One watched call's confirmation read timer: its next read and its last outcome.

    ``next_read_ts`` is a candle close on the call's own grid, or a retry
    ``FOLLOW_UP_RETRY_S`` after a read that found no closed candle.
    """
```

A venue that has not yet published the candle the timer waited for is read again after 60 seconds, at most five times, then at the next close. A scan does not read: a Scan Now press while timers pend runs the walk on its own thread and touches no timer. The entries of every call under watch stay in the bucket through later scans, beside the new hits, until the call settles; a market hit again on a later candle takes the new entry and the new timer. A timer stops when its read settles the call, or when its entry leaves the bucket, which the Activity Log says.

### The verdict, read by candle time

Two defects in phase seven were measured while putting it on a timer, and both are repaired. A call was anchored by the index of its bar, and a venue serves a rolling window, so the same 300 candles fetched one day later put the call's bar one index lower and the read found no candle after it: on the tree before this entry, a day-old call read `no candle has closed since the call` on every re-read. A call now carries the open time of its bar, the candles counted are those opened at or after it whose close time is at or before the read, so the call's own candle is candle one once it closes and a venue's partial bar is never read as a close.

`src/trading/ata_spm_push.py` - the candles a read counts

```
def closed_candles(candles: Any, timeframe: Any, closed_before: Any) -> list:
    """The candles whose close time is at or before ``closed_before``; every candle when it is None."""
```

The second: the target was a share of the run to the midline whichever side the midline lay on, so a bearish call whose close sat below the midline read `confirmed` on a rise. A target now exists only while the midline lies on the side the call expects the run to go: above a bullish close, below a bearish one. While it lies behind the call, the read stays open and says so.

`src/trading/ata_spm_push.py` - the side the target needs

```
def midline_ahead(direction_text: Any, call_close: Any, midline: Any) -> bool:
    """Whether ``midline`` lies on the side of ``call_close`` the call expects the run to go."""
```

On the scan's own firing shape, a crash of five candles, a bearish call sits far below its midline, so such a call never gains a target from the moving midline: it fails on three closes in a row against it, or stays open. Whether a chain-fire hit's expected move is the called direction, as the post's wording says, or the bounce to the midline

the fold side buys, is the operator's line.

### What the entry, the field and the Activity Log read

The Ready to Send entry's status line carries the timer's status after the venue and the approval state: `X · waiting · open · next read 2026-09-20 00:00 UTC` the moment the hit lands, the next close after each read that leaves it open, then `X · waiting · confirmed · close 1476.67` or `X · waiting · failed · close 992.34` with the close that settled it. The same status sits on the entry's head row after the vote word, the badge both hosts already drew and nothing fed, so it reads without scrolling the entry. Both hosts draw it from one payload.

`src/trading/ata_spm_push.py` - the status line

```
BUCKET_META_FOLLOW_UP_FORMAT = "{target} · {state} · {follow_up}"
FOLLOW_UP_STATUS_OPEN_FORMAT = "open · next read {when}"
FOLLOW_UP_STATUS_SETTLED_FORMAT = "{state} · close {close:g}"
```

The ATA-SPM zone's open entry carries one phase-seven row per watched call, the newest read. The Activity Log names each timer at its start with its next read, each read with its state and its next read or its settling close, and a timer that stopped before settling.

`src/trading/ata_spm_push.py` - the Activity Log lines

```
FOLLOW_UP_TIMER_LINE_FORMAT = (
    "ATA-SPM confirmation timer for {symbol} {label}: next read {when}"
)
FOLLOW_UP_READ_OPEN_LINE_FORMAT = (
    "ATA-SPM confirmation read for {symbol} {label}: {state}, next read {when}"
)
FOLLOW_UP_READ_SETTLED_LINE_FORMAT = (
    "ATA-SPM confirmation read for {symbol} {label}: {state}, close {close:g}"
)
```

A settled call's follow-up post is what phase seven already builds, one per venue, and it lands in the bucket as the scan's own entries do. The confirmation share the settings page carries now persists beside the hits per scan, so a share he set survives a restart; it stays 0 until he sets one, and a call reads `open` with the share unset until then.

### The chime pin and the read pin

`inspector.ata.chime` is written once per chime with its cause, `hit` or `confirmation`, the hits it covers or the call it confirms, and whether the player took it. `inspector.ata.follow_up_read` is written once per read with the symbol, the timeframe, the state, the candles counted against the candles the clock says have closed, the close, the target, the next read and the read count. Both go through `signal_contract.emit` and nothing else.

`src/trading/ata_spm_push.py` and `src/gui/market_inspector.py` - the pin names

```
FOLLOW_UP_READ_PIN = "inspector.ata.follow_up_read"
CHIME_PIN = "inspector.ata.chime"
```

### The timer read off both running builds

A crypto scan on 1hr 1d 1wk to three 1d hits, in each variant, with the home on a scratch directory, every socket but loopback refused, a Coinbase connector attached to a loopback stand-in shaped as ccxt's coinbase class answers, and the board's clock and the stand-in's clock driven forward together. The chime's output was set to 0 for the run and read, not heard.

| reading | Qt | React |
|---|---|---|
| chimes as the three hits landed | 3, one per hit; the player read `EndOfMedia` after each | 3, the same |
| the wav | 26,460 frames, 44,100 Hz, stereo, 0.600 s; first 50 ms peak 15,987, last 50 ms peak 431 | the same bytes, one sha256 |
| timers after the scan | 3, one per hit, each `next read 2026-09-20 00:00 UTC`, the next daily close | the same |
| the entry after the scan | `X · waiting · open · next read 2026-09-20 00:00 UTC`; the head row reads `open · next read 2026-09-20 00:00 UTC` beside the vote | the same, read off the page |
| the Activity Log | `ATA-SPM confirmation timer for ARB 1d: next read 2026-09-20 00:00 UTC`, and AVAX, A22 | the same three on the page's pane |
| the clock past the first close | three reads on thread `ata-smp-follow-up`, one fetch each, `open`, entry and log `next read 2026-09-21 00:00 UTC`, three `follow_up_read` pins | the same |
| the field, entry open | one `Phase 7 Follow-Up` row per call, the newest read | the same |
| Scan Now while the timers pend | the walk ran on its own thread, 0 hits, 21 entries kept, every timer's next read and read count unchanged | the same |
| a planted bullish 1wk call | its timer's next read `2026-09-21 00:00 UTC`, a Monday | the same |
| a planted removal of one market's entries | that timer stopped, `... stopped: the entry left the bucket` on the log; the other two held | the same |
| a tab change with timers held | the same three timers before and after | the same |
| the confirmation share typed as 50 | `ata_spm_settings.json` holds `"confirmation_share_pct": 50` beside `hits_per_scan` | the same |
| the third daily close | ARB and A22 `failed · close 992.34`, `the trend held for 3 candles in a row`; no chime; 14 follow-up entries, one per venue per call | the same |
| the Monday close with the week withheld by the venue | `follow_up_read` pin `ok` False, 1 candle of 2; `next read 2026-09-28 00:01 UTC`, the 60 second retry | the same |
| the retry with the week served | `confirmed · close 1476.67`, `reached 985.086, 50% of the run to midline 985.723`; the fourth chime; 7 more follow-up entries; the entry's head row `confirmed · close 1476.67` | the same |
| `bot_state.json` | byte-identical after every press; the planted write moved the comparison | the same |
| connections refused | 0, because nothing asked for one | 0 |

### The public route's window ends now

Read on the built Qt bundle with no connector in reach, against a loopback stand-in whose tape holds one close per calendar day: 90 markets, 540 reads, 300 candles each, 0 hits, and every daily window ended 65 days before the press. The public route asked for 365 bars starting 365 days ago, and one request to the route is held to 300 candles, so the window it asked for ended at its start plus 300 days. The bars a read asks for are now held to that ceiling before the start is computed, so the window ends at the press: the newest 300 daily candles, the newest 300 hourly ones.

`src/exchange/market_inspector_fetcher.py` - the bars one public read asks for

```
count = min(int(bars), RA_CHUNK_DAYS)
```

### Two sentences this entry overtakes

Not reworded, quoted here. `docs/manual/08-tabs/market-inspector.md:1663` - "No wall clock is read, so a daily call cannot fail on the afternoon it was made." The verdict still counts closed candles and reads no clock; the timer reads the clock only to choose when to read, and a daily call still cannot fail before three daily closes.

`docs/manual/08-tabs.md:2046` - "`audio_suite.py` | Not reachable from a live tab". The Audio Suite tab is still built by no live tab; the module is now imported by the Inspector tab for the chime.

**Figures.** This page carries no figure and this entry adds none. A count of the markdown image tags on the page answers 0 before this entry and 0 after it.

## 2026-09-20 00:10 - #23 - Discord, Telegram and WhatsApp join ATA-SMP: two API senders, one compose address, ten entries per hit

His words, 2026-09-19: *"ATA-SMP - Need to add post / message distribution for Discord, Telegram, WhatsApp, and Signal."* and *"ATA-SMP - For Signal, if its not being used for this sort of thing then we can skip it."* And 2026-09-15: *"This will have to do. Do not want to build something that risks myself or others being banned."* Signal publishes no way for a program to post into a group, so it is skipped on his words; three venues join.

### Three more venue rows

Ten targets ship. Each of the three new ones is one row, and every screen and folder writer reads the rows, so the settings page, the venue folders and the bucket carry them with no screen edit. The figures come off each venue's own page, read on 2026-09-19.

| Target | Body ceiling | Title ceiling | Counted in | Image | How a post leaves |
|---|---|---|---|---|---|
| Discord | 2000 | none | Characters | 1200 by 675 | the channel's webhook, one request carrying the text and the picture |
| Telegram | 1024 | none | Characters | 1280 by 720 | the bot's `sendPhoto`, the text as the photo's caption |
| WhatsApp | 65536 | none | Characters | 1200 by 675 | the Click-to-Chat address in the venue folder, the picture attached by hand |

`src/trading/ata_spm_push.py` - the three names

```
TARGET_DISCORD = "Discord"
TARGET_TELEGRAM = "Telegram"
TARGET_WHATSAPP = "WhatsApp"
```

Discord's page states "the message contents (up to 2000 characters)", and a file's default limit of 20 MiB. Telegram's states a caption of "0-1024 characters after entities parsing" and a photo "at most 10 MB in size", its width and height "must not exceed 10000 in total", its ratio "at most 20". WhatsApp's Cloud API sends from a Meta business number to individual numbers, or to groups of at most eight that the business itself creates under an Official Business Account; it posts to no group or community the operator is a member of, so WhatsApp takes the compose address, as X does.

### What the settings page holds for them

Level 1 draws ten venue buttons. Discord's Level 1A page draws one masked box, `Webhook URL`; Telegram's draws two, `Bot token` and `Chat id`; WhatsApp's draws none. A box reaches the vault when he leaves it, as every other venue's does, and these venues issue no token, so a venue reads `held` the moment its boxes are left. Connect checks the typed value reads as the form the venue's own page prints, opens no browser and reaches no venue.

`src/gui/main_tabs/market_inspector_surface.py` - the sign-in line for the three

```
SIGN_IN_TYPED_TEXT = (
    "Connect checks what you typed and holds it. No browser opens and no "
    "venue is reached."
)
SIGN_IN_NONE_TEXT = (
    "No sign-in. The post goes through the venue folder and the compose address."
)
```

`src/trading/ata_spm_signin.py` - the two routes behind Connect

```
def sign_in_discord(typed: dict, session: SignInSession) -> dict:
def sign_in_telegram(typed: dict, session: SignInSession) -> dict:
```

A webhook URL that does not read as `https://discord.com/api/webhooks/<id>/<token>` is refused on the page with that form named. A bot token that does not read as BotFather prints it is refused the same way, and a chat id that is neither a number nor an `@name`. WhatsApp's Connect answers `WhatsApp needs no sign-in.` and its Level 1 button's wording reads `none needed`.

### The two senders

The first API senders in the tree. Post Selected, Post All and a Full Auto release take a Discord or Telegram post through its API once the venue reads `held` and `Max posts per hour` is set; every other venue keeps its folder or compose-address route, because the sender names the venues it takes and the route asks it.

`src/trading/ata_spm_send.py` - what the board sends through

```
SEND_ROUTES = {
    ata_spm_push.TARGET_DISCORD: send_discord,
    ata_spm_push.TARGET_TELEGRAM: send_telegram,
}


def build_sender(settings: Any, transport: Optional[Callable] = None) -> ApiSender:
```

One request per post, `multipart/form-data`, through the same URL opener the sign-ins use. Discord takes `payload_json` carrying `content` and `files[0]` carrying the venue PNG, at the webhook address with `wait=true`, and answers the created message and its `id`. Telegram takes `chat_id`, `caption` and `photo` at `sendPhoto` and answers `result.message_id`. The record reads `sent to channel <n>, message <id>` or `sent to chat <n>, message <id>`.

`src/trading/ata_spm_push.py` - the route asks the sender

```
def sender_takes(sender: Any, target: Any) -> bool:
    """Whether ``sender`` posts to one target: every target while it names no
    ``targets``, as ``RecordedDestination`` does, else the ones it names."""
```

### What a refusal reads

A venue answering 429 is read for its wait; a 401 or 403 is read as the credential refused, with what the venue said; a caption over Telegram's 1,024 is refused before any request, with the ceiling and the measure. The line and the entry carry the reason.

`src/trading/ata_spm_send.py` - the three wordings

```
RATE_LIMITED_FORMAT = "429, retry after {seconds:g} s"
CREDENTIAL_REFUSED_FORMAT = "{http}, the credential was refused: {said}"
HTTP_REFUSED_FORMAT = "{http}, {said}"
```

Nothing retries by itself. The post waits in the bucket for the next press.

### The entry's status line says what the press did

Each Ready to Send entry's status line now carries the outcome of the last press that took it, after the venue and the approval state: `sent · channel 111222333, message 1001`, `not sent · Discord refused the post: 429, retry after 5 s`, `in folder`, `intent handed to the OS`, or `intent ready` after a Post All. The status line sits below the entry's visible fold at 1400 by 900, so the head-row badge carries the same outcome once a press took the post; before any press the badge carries the timer's status, as before.

`src/trading/ata_spm_push.py` - the words the entry carries

```
STATUS_SENT_FORMAT = "sent · {destination}"
STATUS_NOT_SENT_FORMAT = "not sent · {detail}"
STATUS_IN_FOLDER_TEXT = "in folder"
STATUS_INTENT_OPENED_TEXT = "intent handed to the OS"
STATUS_INTENT_READY_TEXT = "intent ready"
```

`src/gui/main_tabs/market_inspector_surface.py` - the badge both hosts draw

```
def bucket_badge(held: Any) -> str:
    """The head-row badge: the ``delivery`` of the last press that took the
    post, else the timer's ``follow_up`` status, so either reads without
    scrolling the entry."""
```

### WhatsApp by the compose address

WhatsApp's folder holds the stamped picture, the text, and a `.url` file beside them carrying the Click-to-Chat address with the message typed in. Post Selected hands that file to the operating system's own handler; WhatsApp opens with the message, he picks the chat and attaches the picture from the same folder. No browser is driven.

`src/trading/ata_venue_folders.py` - the second compose address

```
WHATSAPP_INTENT_FORMAT = "https://wa.me/?text={text}"
```

### The press runs off the window thread

A Post Selected, Post All or Full Auto press now runs on its own thread, one per press, and its records cross back to the window thread; a press while one runs is refused on the Activity Log. Every send therefore leaves the window-drawing thread free, as the scan and the confirmation reads already do.

`src/gui/market_inspector.py` - the thread and the refusal

```
HAND_OFF_THREAD_NAME = "ata-smp-hand-off"
HAND_OFF_BUSY_TEXT = "ATA-SPM %s pressed while a hand-off is running; press ignored"
```

### The API block and the sent pin

Every send writes one block on the Live tab's API Interaction Log, naming the venue, the action, the post, the text length, the image size and the message id or the refusal; the endpoint names the route with the token left out. `inspector.ata.sent` is written once per request with the venue, the symbol, the timeframe, the status, the message id and the elapsed time; `ok` is true on a send. `inspector.ata.handoff` carries the three venues as it carries the seven.

`src/trading/ata_spm_send.py` - the pin and the block's endpoints

```
SENT_PIN = "inspector.ata.sent"
DISCORD_LOGGED_ENDPOINT = DISCORD_API_BASE + "/<id>/<token>?" + DISCORD_WAIT_QUERY
TELEGRAM_LOGGED_ENDPOINT = TELEGRAM_API_BASE + "/bot<token>/" + TELEGRAM_SEND_PHOTO
```

### The three venues, read off both running builds

Crypto, nothing typed, 1hr 1d 1wk ticked, hits per scan 1, Max posts per hour 10, in each variant, with the home on a scratch directory, every socket but loopback refused, unit S5's candle stand-in and the two API stand-ins on loopback:

| reading | Qt | React |
|---|---|---|
| venue buttons on Level 1 | 10, WhatsApp's wording `none needed` | 10, the same |
| Discord's page | one masked box; `Connect checks what you typed and holds it. No browser opens and no venue is reached.`; `Scopes none`; no redirect address | the same |
| the webhook typed and the box left | `held`; the vault file under the scratch home holds no clear webhook | the same |
| Telegram's page | two masked boxes, both `held` once left | the same |
| WhatsApp's page | no box; `No sign-in. The post goes through the venue folder and the compose address.`; Connect leaves the page | the same |
| one hit, ARB 1d | 10 entries, `1 of 10`, one per venue in row order, each with its venue PNG | the same |
| folders | 10; Discord and Telegram one PNG and one TXT, WhatsApp one PNG, one TXT and one URL | the same |
| Post Selected, Discord | one multipart POST to the stand-in: `payload_json` carrying `content`, `files[0]` carrying the 1200 by 675 PNG; badge `sent · channel 111222333, message 1001`; the Activity Log line; one API block `execute_webhook`, `200 message 1001`, on thread `ata-smp-hand-off`; one `sent` pin ok | the same, the line on the page's Activity Log and the block on the page's API log |
| Post Selected, Telegram | `chat_id`, `caption` and `photo` at `sendPhoto`; `sent · chat -1001234567890, message 91`; the block `send_photo`, `200 message 91` | the same |
| Post Selected, WhatsApp | no request; the `.url` handed to the OS handler; `intent handed to the OS` | the same |
| the stand-in answering 429 | `not sent · Discord refused the post: 429, retry after 5 s`; Telegram `retry after 7 s`; the block reads `refused` at level error; the `sent` pin ok false | the same |
| a wrong webhook and a wrong bot token typed | `401, the credential was refused: Invalid Webhook Token`; `401, the credential was refused: Unauthorized` | the same |
| a Telegram caption over 1,024 | `Telegram publishes 1024; this post measures 2967.` with no request sent | the same |
| Post All, ten approved | ten records: Discord and Telegram sent, WhatsApp and X `compose address ready`, the six others `in folder`; the root opened once | the same |
| `inspector.ata.handoff` | the three venues with their route and outcome | the same |
| `bot_state.json` | byte-identical after every press; the planted write moved the comparison | the same |
| the operator's posts root | listed before and after, equal | the same |
| connections refused | 0, because nothing asked for one | 0 |

Nothing driven contacted a venue: every socket but loopback was refused and the two APIs were stood in on loopback, answering the shapes their own pages publish; the credentials typed were stand-in strings. The OS handler was recorded and not run.

### The sentences the three venues overtake

Not reworded, quoted here.

`docs/manual/08-tabs/market-inspector.md:322` - "Seven push targets ship, and adding one is adding a row, because no phase names a target." Ten ship; the second half is what made the three rows land with no screen edit.

`docs/manual/08-tabs/market-inspector.md:653` - "Seven targets ship. Adding one is adding a row: no phase names a target, and the Settings credential list is built from the same rows, so a new row appears on the settings page with no screen edit." Ten ship; the rest holds and was read off both running builds.

`docs/manual/08-tabs/market-inspector.md:1050` - "no sender reaches a platform." Two senders now reach Discord and Telegram; the sender is still handed the whole post, and the picture travels with the text.

`docs/manual/08-tabs/market-inspector.md:1897` - "All seven venues use three-legged OAuth." The seven still do; Discord and Telegram take a typed credential and no OAuth, and WhatsApp takes none.

`docs/manual/08-tabs/market-inspector.md:3501` - "The chart folder now holds seven folders, one per venue, named for it." Ten folders.

`docs/manual/08-tabs/market-inspector.md:5017` - "`intent_path` the `.url` file, which only X's folder holds." X's and WhatsApp's.

`docs/manual/08-tabs/market-inspector.md:5067` - "No sender is wired in the tree today, so every venue takes the folder route and X the intent route." A sender is wired in both hosts; it takes Discord and Telegram, and the seven keep the routes that sentence names, WhatsApp the intent route.

**Figures.** This page carries no figure and this entry adds none. A count of the markdown image tags on the page answers 0 before this entry and 0 after it.

## 2026-09-20 03:30 - #23 - Derivatives lists Coinbase's futures, Metals gains its base-metal funds, no window floats over the panel, and the field holds its lines at one size

His words, 2026-09-20: *"ATA-SMP - The scanner freaking works but we do have some issues. Derivatives sector list is empty. Metals only has four in its list and I am pretty sure its a more complex sector than this but could be wrong. I see a series of floating windows rapidly appearing and disappearing over the ATA-SMP panel once the scan gets going. The field where the IVP fields are supposed to appear during the read initially displays text but this gets very small very fast before disappearing completely and the floating windows appear with some containing text before vanishing. I was able to get three hits and three charts generated. Very good progress."* And: *"ATA-SMP - Energy sector also only has four entries in its list. Probably not complete..."*

### What he saw, read off the running program before the change

The Qt window under the S5 candle stand-in, 90 trading products, a hit target of 3, the three firing products ranked last: Derivatives listed 0 markets and the field under the ticker read `No ticker list for derivatives`; Metals listed 8 rows, the four spot pairs on no venue and the four funds; Energy 4. One three-hit scan opened 2,331 top-level windows, every one a `QLabel` with no parent, and 71 of them were still visible when the scan ended. The field's rows were never laid out: each label kept the 480 px a widget is born with, the entry held 4 px a row, and the newest row was inside the field on 0 of 88 readings. The font never changed: `DETAIL_STYLE` fixes 11 px in both variants. What shrank and vanished was the rows, and what floated was the rows. The React page opened no window and held its rows at 30.8 px, but its scroll never moved, so the newest row sat 2,942 px below the 75 px it shows.

### Derivatives lists its markets

The class had no source. It has one now: Coinbase Advanced Trade's public product list, read at press time the way the stocks screener is. Each product the venue says it trades becomes one row on the futures venue, ranked by the venue's own 24 h volume times price, and the rows stay in the process so the ticker field offers them after the press.

`src/trading/ata_asset_maps.py` - the source and the read

```
FUTURES_PRODUCTS_URL = "https://api.coinbase.com/api/v3/brokerage/market/products"
FUTURES_PRODUCT_TYPE = "FUTURE"
FUTURES_SOURCE_TEXT = "coinbase futures and perpetuals, 24 h volume x price"


def futures_listings(timeout_s: float = FUTURES_TIMEOUT_S) -> tuple[list, dict, str]:
    """The derivatives rows ``FUTURES_PRODUCTS_URL`` lists as trading, their
    figures by symbol, and the refusal when it answered none.
```

A derivative's candles come from the same route's candles endpoint, which serves `ONE_HOUR` and `ONE_DAY` at 350 candles a request; the weekly line rolls up from the daily one, the rule the crypto class already follows.

`src/trading/stone_tablets/ra_fetcher.py` - the candle route

```
class CoinbaseFuturesCandles(ExchangeAdapter):
    exchange_id = "coinbase-futures"
    chunk_limit = FUTURES_CANDLE_CAP

    BASE_URL: str = "https://api.coinbase.com/api/v3/brokerage/market/products"
```

Before the first press the line under the ticker field says where the list comes from, and a typed product id, `BIT-25SEP26-CDE` for one, reads the list first and scans that one market.

`src/gui/main_tabs/market_inspector_surface.py` - the line under the field

```
TICKER_PRESS_LIST_FORMAT = (
    "Scan Now reads the {sector} list from {source}. A typed name still scans."
)
```

Read once against the venue on 2026-09-20, on both bundles: the list held 117 trading products, every one a dated contract on the CDE venue marked `EXPIRING`, and none carrying the `-PERP-INTX` suffix the venue gives its perpetuals; 57 of the 117 carried no 24 h volume and ranked last by name, and 43 answered no candle on any granularity. The scan read all 117 and found no hit.

### Metals and Energy list every fund their rule admits

The rule stands: the listed instrument for each metal is the fund holding it. The base metals were missing. Read against the chart endpoint on 2026-09-20: CPER, the copper fund, and DBB, the fund holding aluminium, zinc and copper together, each answered 250 daily rows with the last complete daily bar carrying volume, so both rank by a figure beside GLD SLV PPLT PALL. The notes JJN JJU JJT LD, for nickel, aluminium, tin and lead alone, answered HTTP 404 on every timeframe and are not carried.

`src/trading/ata_asset_maps.py` - the base-metal sector

```
METALS_BASE: tuple[AssetListing, ...] = tuple(
    AssetListing(symbol=one, quote=USD, venue=VENUE_YAHOO, ticker=one)
    for one in ("CPER", "DBB")
)
```

Metals now lists 10 rows: the four spot pairs, still on no venue, and six funds. Energy stays at four. UHN, the heating oil fund, answered HTTP 404 on every timeframe again on 2026-09-20, and no fund holding gasoil is listed, so the count is a finding: the rule admits nothing more today.

### No window floats over the panel

Every scan progress rebuilt the field's labels. The old ones were taken off the layout with `setParent(None)`, and a label added to a visible entry is shown by a queued call. When the next progress landed before that call ran, the label was already a parentless widget, and the queued show opened it as a window at the desktop's default place, holding the line's text, until its deferred delete ran. That is the series of floating windows.

The stepper keeps its labels now. A line is written into the label already at that row, a new label is added only past the last one and shown at once, and a surplus label is hidden before it leaves the layout.

`src/gui/market_inspector.py` - the rows kept

```
        def _show_lines(self, lines: list) -> None:
            while len(self.detail_labels) > len(lines):
                gone = self.detail_labels.pop()
                gone.hide()
                self.detail_box.removeWidget(gone)
                gone.setParent(None)
                gone.deleteLater()
            for at, line in enumerate(lines):
                if at < len(self.detail_labels):
                    self.detail_labels[at].setText(line)
                    continue
                drawn = QLabel(line)
                drawn.setStyleSheet(DETAIL_STYLE)
                drawn.setWordWrap(True)
                self.detail_box.addWidget(drawn)
                drawn.show()
                self.detail_labels.append(drawn)
```

Read on the Qt window over one 90-market, three-hit scan: 0 top-level windows opened, and the visible count read 1 at the press, 1 every second during the scan and 1 at the end. A window planted from the hit handler raised the count to 4, so the reading can fail.

### The field holds its lines at one size

The held height counted a hidden label as 0 px, and every new label was hidden until its queued show ran. Shown at once, each row is counted at its own wrapped height, the entry grows to what its rows need, and the scroll area scrolls it. While the counter is set the scroll follows the newest line, in both variants.

`src/gui/market_inspector.py` - the scroll

```
        def _scroll_to_newest(self) -> None:
            bar = self.entry_scroll.verticalScrollBar()
            bar.setValue(bar.maximum())
```

`src/gui/web/market_inspector.js` - the page's entry follows its newest row

```
    entryProps.ref = function (node) {
      if (node !== null && text(view[ZONE_COUNTER])) {
        node.scrollTop = node.scrollHeight;
      }
    };
```

Read at every progress of the same scan: the row font 11 px at the first line and at the ninetieth in both variants; each Qt row 15 px a wrapped line, 30 px where the line wraps to two, from the first line to the ninetieth, and the entry 144 px at one line and 3,075 px at ninety; the newest line inside the field on 90 of 90 readings on the Qt window and 76 of 76 on the page. A held height pinned to the viewport in place of the rule dropped every row to 0 px by the 28th line, so that reading can fail too.

### A venue candle's stamp is written in seconds

A hit on a Yahoo or futures row ended the scan with an error: the row's candles carried millisecond stamps, the follow-up timer and the chart read seconds, and `time.gmtime` refused the year the millisecond value names. Every venue candle is now stamped in seconds, the unit the connector's candles, the chart's time axis and the follow-up timer already read.

`src/trading/ata_asset_maps.py` - the stamp

```
            stamped = [float(row[0]) / MS_PER_S, *row[1:]]
            kept.extend(candles_from_raw([stamped]))
```

### Read off both bundles

Each variant was built and launched with the home on a scratch directory, no credential file, every socket refused, and its window read constructing: `Application ready` after 3.0 s on the Qt bundle and 2.0 s on the React bundle, the Inspector reached from the tab bar, the class menu reading `derivatives` at its full width, and the line under the ticker field reading the source. Then one scan per class on each bundle against the public hosts alone, through a loopback proxy that allows `api.exchange.coinbase.com`, `api.coinbase.com`, `query1.finance.yahoo.com` and `query2.finance.yahoo.com` and refuses everything else. The Qt bundle's ledger: 1,658 connections, 529 to the exchange, 494 to Advanced Trade, 635 to Yahoo, 0 refused. The React bundle's, over two launches because the page's class menu did not open after its derivatives scan in the first: 1,393 and 265 connections, 529 to the exchange, 494 to Advanced Trade, 635 to Yahoo, 0 refused. On each bundle crypto read 88 trading products and found no hit; stocks read 38 of 100 and stopped at three hits; metals read 10 and energy 5, no hit; derivatives read 117, no hit; forex read 28 with two hits. The bot state file was not touched and no thread violation was written.

### Six steps

The Qt stepper and the page draw the same `zone_view`: the contract's members, `counter` and `detail`, answered 2 of 2; the feeds, `scanProgressed` and the finished answer, routed 2 of 2; driven and read back at the widget and at the page, 2 of 2. The two renders agree on the rows and the counter.

### The sentences the four functions overtake

Not reworded, quoted here.

`docs/manual/08-tabs/market-inspector.md:1365` - "Derivatives has no classification named yet." It has one: the venue's own futures product list, read at press time.

`docs/manual/08-tabs/market-inspector.md:3875` - the metals row, "the asset map's order: the four spot pairs, then the four funds". Six funds, and they rank by their last complete daily bar's volume times close.

`docs/manual/08-tabs/market-inspector.md:3876` - the stocks and derivatives row, "no list exists, and the zone says so". Both lists exist; the zone counts them.

`docs/manual/08-tabs/market-inspector.md:4507` - the stocks and derivatives row, "none" and "no list exists, and the zone says so". The same.

`docs/manual/08-tabs/market-inspector.md:4718` - "Stocks and derivatives list nothing, as before." Derivatives offers the products the last press read.

`docs/manual/08-tabs/market-inspector.md:4855` - "Under stocks and derivatives, which list nothing, a typed name still scans". A typed product id reads the list first.

`docs/manual/08-tabs/market-inspector.md:5496` - "Derivatives still lists nothing." 117 products on 2026-09-20.

`docs/manual/08-tabs/market-inspector.md:5432` - "the field held 27 lines when the counter read `27 of 90`". It held 27 labels and drew none of them; it draws every line now and scrolls to the newest.

**Figures.** This page carries no figure and this entry adds none. A count of the markdown image tags on the page answers 0 before this entry and 0 after it.

## 2026-09-20 03:50 - #23 - Scan All walks every sector with no stop, paced under each public host's limit, and each watched call's confirmation timer is a tile right of the Timeframe row

His words, 2026-09-20: *"ATA-SMP - Add a Scan All button which will scan every market in every TF for every market in every sector. Have noticed that scans run really fast which is good but just do not to risk overloading a free API or something. For the attached image, I want the Confirmation Timers for prior Hits to appear here. Can be just a small ticker pair with a theme-consistent timer which triggers the rescan on completion."*

### Scan All

A second button sits beside Scan Now at its size, in both builds. A press walks every class the sector menu holds, in the menu's order: crypto, stocks, metals, derivatives, forex, energy. Each class is walked on every timeframe it scans, the four the Timeframe row lists for it, and on every market its list holds, with no stop at a hit count. The walk runs on the scan's own thread and both buttons read `Scanning…`, disabled, from the press to the end. A press while a scan runs writes the busy line to the Activity Log, as a Scan Now press does.

`src/gui/main_tabs/market_inspector_surface.py` - the button

```
SCAN_ALL_LABEL = "Scan All"
SCAN_ALL_PART = "scan-all"
```

The walk is the by-volume walk Scan Now runs, with a sector flag in place of a hit target. `Sector.walk_all` and `SectorScan.walk_all` say the list is read to its end; `walks_order` answers True for either flag and is what every reader of the old target check now asks, so a walk-all scan takes the same route through `evaluate`, `_scan_until_hits`, the judge, the hit pin, the chime and the bucket.

`src/trading/ata_spm.py` - one function, both flags

```
def walks_order(scan: Any) -> bool:
    """Whether one ``Sector`` or ``SectorScan`` walks its list market by market.

    A ``walk_all`` scan walks the whole list; a ``hit_target`` above
    ``NO_HIT_TARGET`` walks it until that many hits.
    """
```

`SectorBoard.compute_all` builds the six sectors, tells the counter which list it is reading, and runs them. Its sectors replace what the board held. A Scan Now after a Scan All drops them again before it walks, so it walks its own class and not six.

`src/trading/ata_spm.py` - the six sectors

```
        walk = WalkProgress(progress, ASSET_CLASSES)
        sectors: list = []
        for asset_class in ASSET_CLASSES:
            walk.listing(asset_class)
            order = markets_of(class_source, asset_class)
            sectors.append(
                Sector(
                    name=asset_class,
                    asset_class=asset_class,
                    timeframes=timeframes_for(asset_class),
                    listings=tuple(order.listings),
                    order=order,
                    walk_all=True,
                )
            )
```

### The counter, the field and the entries

The counter names the sector, its place in the walk, the markets read of the list and the hits of the whole walk so far. The hits count over every sector because the chime sounds when that count rises; a count that fell back to zero at each new sector would leave the second sector's first hits silent. The field draws one line per market as before, over all six sectors. At the end the stepper holds one entry per sector, `1 of 6`, each reading `sector exhausted`, and the report replaces the lines.

`src/trading/ata_spm.py` - the counter's lines

```
SCAN_ALL_PROGRESS_FORMAT = (
    "Scanning {asset_class} ({at} of {sectors}) · {read} of {total} · {hits} hit(s)"
)
SCAN_ALL_LISTING_FORMAT = (
    "Scanning {asset_class} ({at} of {sectors}) · reading the market list"
)
```

### The pace

Coinbase publishes 10 requests a second per IP for its public endpoints and answers a burst over it with a 429. Yahoo Finance's chart endpoint publishes no limit; a 429 is its only signal. Scan All keeps every read to a host at least this far from the last one on that host: 0.2 s on each Coinbase host, half its published limit, and 0.5 s on Yahoo. The gap is measured from the end of the last read, so every stamp a read leaves on the API Interaction Log sits at least the pace from the one before it. The reads that rank a class's list are paced the same way.

`src/trading/ata_spm.py` - the pace and the hold

```
EXCHANGE_PACE_S = 0.2
YAHOO_PACE_S = 0.5
EXCHANGE_RATE_LIMIT_WAIT_S = 10.0
YAHOO_RATE_LIMIT_WAIT_S = 60.0
```

A 429 is read as a wait, never as a burst. The read that met it names it in its refusal, the walk holds that host for the venue's `Retry-After` when the answer carried one, else the table's seconds, and reads that market and timeframe once more. A second 429 stands as the refusal; no third read is made. Inside one read no route retries a 429 any more: the Yahoo adapter the scan builds retries every transient code but 429, and the connector route stops asking the other quotes once one answered 429. The connector's own three attempts a second apart are the live bots' and stay as they are; the walk's hold begins once they are spent. Scan Now reads through no pace, so its speed is as he described it.

`src/trading/ata_spm.py` - one paced read

```
def paced_read(pace: Any, host: Any, read: Callable) -> tuple:
    """One read through ``pace``: wait, read, and read once more after the hold a 429 leaves.

    ``read`` answers ``(venue, candles, refusal)``. A second refusal stands;
    no third read is made. With no ``pace`` the read runs once, unpaced.
    """
```

### A market is read once per timeframe

Before this entry every by-volume walk read each market twice per timeframe: `_vote_one` read it to vote, and `pull` read it again to draw the chart and run the gates. A walk now hands both the same market memo, so the second read answers the first's candles and a venue is asked once. On the stand-in a Scan Now of 12 crypto markets made 48 requests where it made 96 before. Scan Now takes this too.

`src/trading/ata_spm.py` - the memo

```
class MarketMemo:
    """One market's candles by timeframe, held from ``_vote_one`` to ``pull``
    so ``_scan_until_hits`` asks ``candle_source`` for each timeframe once."""
```

### The timers on screen

The region right of the Timeframe row, the scan line and the stepper holds one tile per watched call, in both builds. A tile reads the market and its timeframe on one line, the way the Activity Log names the timer, and on the second line the time to the call's next read, counted down once a second on the tab's own clock. At zero the read the timer wired fires as before, and the tile reads `reading` until the outcome crosses back; an open call then counts down to its next close, a confirmed or failed call reads its outcome and its close. A settled call's tile stays until its entry leaves the bucket, which the next scan's load does. While no call is watched the region reads `No confirmation timer running.` The tiles wrap by the region's width, 132 px each, the width the widest pair a class lists takes at the caption size. The region is 110 px tall, three tile rows, and scrolls past that, so the tiles never push the stepper and the field down the zone.

`src/trading/ata_spm_push.py` - the tile's lines

```
TIMER_PAIR_FORMAT = "{symbol} {label}"
TIMER_COUNTDOWN_FORMAT = "{hours:02d}:{minutes:02d}:{seconds:02d}"
TIMER_COUNTDOWN_DAYS_FORMAT = "{days}d {hours:02d}:{minutes:02d}:{seconds:02d}"
TIMER_READING_TEXT = "reading"
```

`PushBoard.timer_tiles` answers the rows off the board: one per open timer in `FollowUpWatch.timers`, one per settled call whose entry the bucket still holds. The Qt widget `TimerTiles` draws them as frames; the page's `TimerTiles` draws the same rows from `ata_spm_skin`, and the clock's tick hands the page the rows alone through `setTimerTiles`, so a countdown moves without the whole payload. Both draw from the theme's tokens: the tile on `SURFACE_INPUT` inside an `OUTLINE` border, the pair in `TEXT_HIGH` at the caption size, the countdown in `PRIMARY`, `reading` in `WARNING`, `confirmed` in `SUCCESS`, `failed` in `ERROR`.

`src/trading/ata_spm_push.py` - the rows

```
    def timer_tiles(self) -> list:
        """One ``TimerTile`` per call under watch, then one per settled call
        whose entry the bucket still holds, in the order their timers started.
```

### The walk and the tiles read off both running variants

Both builds, the home on a scratch directory, every socket but loopback refused, the stand-in serving the shapes the sources serve: 90 trading crypto products, 100 screener equities, 14 metals rows of which 6 read, 11 futures products, 28 forex pairs, 5 energy funds; a 5-minute crash on A34's tape, a daily one on nine names; three planted 429s, on AVAX/USD three times in a row on the connector route, on BIT-27FEB26-CDE once with `Retry-After: 4` on the futures route, and on AMD once on Yahoo. The connector's interval was left as `attach_backend` sets it, 0.0 s, so the walk's pace is the only spacing.

| reading | Qt | React |
|---|---|---|
| the walk's minutes, press to the last sector's end | 7.83 (470.0 s) | 7.86 (471.5 s) |
| the sectors, in walk order | crypto 90 of 90 read, 7 hit(s); stocks 100 of 100 read, 3 hit(s); metals 6 of 10 read, 0 hit(s); derivatives 11 of 11 read, 5 hit(s); forex 28 of 28 read, 0 hit(s); energy 4 of 4 read, 0 hit(s) | crypto 90 of 90 read, 7 hit(s); stocks 100 of 100 read, 3 hit(s); metals 6 of 10 read, 0 hit(s); derivatives 11 of 11 read, 5 hit(s); forex 28 of 28 read, 0 hit(s); energy 4 of 4 read, 0 hit(s) |
| requests at the stand-in / blocks on the API Interaction Log | 960 / 957 | 960 / 957 |
| hits, chime pins, hit pins | 15, 15, 15 | 15, 15, 15 |
| both buttons `Scanning…` and disabled, polls | 894 of 894 | 880 of 880 |
| distinct counter texts; entries at the end | 242; 6 | 241; 6 |
| arrival gaps at the stand-in, per venue | yahoo 563 requests, least gap 0.515 s, mean 0.829 s, 0 under the pace; exchange 363 requests, least gap 0.202 s, mean 0.266 s, 0 under the pace; coinbase-futures 34 requests, least gap 0.216 s, mean 0.465 s, 0 under the pace | yahoo 563 requests, least gap 0.515 s, mean 0.832 s, 0 under the pace; exchange 363 requests, least gap 0.202 s, mean 0.266 s, 0 under the pace; coinbase-futures 34 requests, least gap 0.217 s, mean 0.469 s, 0 under the pace |
| stamp gaps on the API Interaction Log, per venue | yahoo 563 blocks, least gap 0.514 s, 0 under 0.2 s, 0 under 0.5 s; coinbase 360 blocks, least gap 0.202 s, 0 under 0.2 s, 351 under 0.5 s; coinbase-futures 34 blocks, least gap 0.216 s, 0 under 0.2 s, 27 under 0.5 s | yahoo 563 blocks, least gap 0.514 s, 0 under 0.2 s, 0 under 0.5 s; coinbase 360 blocks, least gap 0.202 s, 0 under 0.2 s, 351 under 0.5 s; coinbase-futures 34 blocks, least gap 0.217 s, 0 under 0.2 s, 27 under 0.5 s |
| the three planted 429s | exchange AVAX/USD answered 429/429/429/200/200/200/200, the retry 10.05 s after the last 429; coinbase-futures BIT-27FEB26-CDE answered 429/200/200/200, the retry 4.02 s after the last 429; yahoo AMD answered 429/200/200/200/200, the retry 60.03 s after the last 429 | exchange AVAX/USD answered 429/429/429/200/200/200/200, the retry 10.05 s after the last 429; coinbase-futures BIT-27FEB26-CDE answered 429/200/200/200, the retry 4.03 s after the last 429; yahoo AMD answered 429/200/200/200/200, the retry 60.02 s after the last 429 |
| the holds the pace booked | exchange 10 s, yahoo 60 s, coinbase-futures 4 s | exchange 10 s, yahoo 60 s, coinbase-futures 4 s |
| tiles after the walk; countdowns falling by one a second over five readings | 15; 15 of 15 | 15; 15 of 15 |
| the A34 5m tile across its close, one reading a second | 00:00:05 → 00:00:04 → 00:00:03 → 00:00:02 → 00:00:01 → reading → 00:04:57 → 00:04:56 → 00:04:55 → 00:04:54 | 00:00:05 → 00:00:04 → 00:00:03 → 00:00:02 → 00:00:01 → reading → failed · close 2.68396 |

| reading | Qt | React |
|---|---|---|
| the read's Activity Log line | ATA-SPM confirmation read for A34 5m: open, next read 2026-09-20 09:40 UTC | ATA-SPM confirmation read for A34 5m: failed, close 2.68396 |
| tiles before and after a Scan Now on energy | 15 → 14 | 15 → 14 |
| connections refused | 0 | 0 |

The two builds' A34 reads met different closes because each run's clock stood at a different point on the 5-minute grid. The Qt read found one closed candle and stayed open, so the tile counted down to the next close; three closes then planted against the call, 900, 950 and 992.34, and the clock moved past the first of them, settled it `failed · close 950`, and the tile read that until the energy scan dropped its entry. The React read met the failure at once. Both are the read the S6 entry wired, unchanged.

The walk's size before the run, at S8's real reading: 352 markets, 1,408 reads; 820 on the Coinbase hosts at 0.2 s and 588 on Yahoo at 0.5 s, 458 s before the venues' own latency. On the stand-in, 248 markets, 992 reads, 375 s at the pace; read at 470 s, the rest the stand-in's service, the chart rendering per hit and the three holds, 74 s.

### Two plants

A hit target of 3 planted onto every sector of the all-sectors walk on the short stand-in: crypto stopped at 11 of 15 markets, stocks at 3 of 12, derivatives at 7 of 11, each `stopped at target`; the reading fails. The board's clock pinned to one instant after a three-hit crypto scan: every tile read the same countdown over four seconds, the steps `0, 0, 0`, in both builds; the reading fails.

### The real reading, once, on both bundles

Built at `1e739a55` with this change on the tree, launched with the home on a scratch directory holding an empty fleet and no credential file, through a loopback proxy allowing `api.exchange.coinbase.com`, `api.coinbase.com`, `query1.finance.yahoo.com` and `query2.finance.yahoo.com`, one Scan All press each through UI Automation, the app's own throttle and pace. Every read went to a public host; the fleet's fields stayed empty, and the state file's stamp moved only by the app's own periodic save.

| reading | Qt bundle | React bundle |
|---|---|---|
| `Application ready`, the Inspector reached, Scan All pressed | 6.0 s; `invoke 'scan-all' -> invoked` | 6.0 s; a button message to the page's child window at the `Scan All` element |
| the walk | 714.1 s, 11.9 min; 1,255 reads: 352 exchange, 351 futures, 552 Yahoo | 714.0 s; the same 1,255 |
| the sectors | crypto 88 read, 0 hits; stocks 100, 6; metals 6, 0; derivatives 117, 0; forex 28, 2; energy 4, 0; every one `sector exhausted` | the same |
| hits, timers, chimes | 8, 8, 8: NKE 1wk, TAP 1d, RKT 1d, RLX 1d, OPEN 1wk, KHC 1hr, EUR/CHF 1hr, NZD/CHF 1hr | the same 8 |
| the buttons while the walk ran, then after | `scan-row` and `scan-all` both disabled; both enabled after | two `Scanning…` labels disabled; `Scan Now` and `Scan All` enabled after |

| reading | Qt bundle | React bundle |
|---|---|---|
| the counter 8 s in | read off the window by name | `Scanning crypto (1 of 6) · 1 of 88 · 0 hit(s)` |
| the proxy ledger, connections per host, least gap, refused | api.exchange.coinbase.com 353, 0.273 s; api.coinbase.com 352, 0.279 s; query1.finance.yahoo.com 563, 0.280 s once between the screener's list call and the first paced read, else over 0.5 s; 0 refused | 353, 0.288 s; 352, 0.271 s; 563, 0.252 s once, else over 0.5 s; 0 refused |
| the API Interaction Log's candle blocks, least gap | COINBASE 352, 0.278 s; COINBASE-FUTURES 351, 0.274 s; YAHOO 562, 0.596 s | 352, 0.294 s; 351, 0.286 s; 562, 0.599 s |
| 429s | none | none |
| the tiles at the end | 8 tiles by name; `NKE 1wk` over `5d 09:41:56`, `KHC 1hr` over `00:41:53` on the capture | 8 tiles, the countdowns read off the page a second apart: `5d 09:29:01`, `5d 09:29:00`, `5d 09:28:59` |
| the stepper, the bucket | `1 of 6`, `1 of 80` | the same |
| thread violations, credential files | 0, none | 0, none |

The same two bundles launched with no network at all, the Qt one in an AppContainer with no capability and the React one behind a dead proxy with Chromium's resolver mapped away: `Application ready` after 7.0 and 6.0 s, the Inspector reached, Scan All pressed, the walk paced through every refused read, an empty end state, which is a non-reading. The build carrying the last styling line, the region's transparent ground, was launched the same way after the real reading and constructs the same.

### Six steps for the two functions

The contract this unit adds to the zone: the Scan All control, the tile region, and the tile row with its pair line, its second line and its state. Answered 3 of 3 on the Qt widget and 3 of 3 on the page. The feeds: `scanProgressed` with the sector's place and the walk's hits, the finished answer with six sectors, the clock's tile rows once a second, and the full push carrying the rows. Routed 4 of 4. Driven and read back at the widget and at the page: the press, the tiles each second, the pinned clock, 3 of 3. Same on both.

### The three sentences Scan All and the tiles overtake

Not reworded, quoted here.

`docs/manual/08-tabs/market-inspector.md:4418` - "An empty field walks the class by a cited volume order and stops at the N-th push-gate hit". It does under Scan Now. Under Scan All the walk stops at no hit and reads every market of every sector.

`docs/manual/08-tabs/market-inspector.md:3882` - "The moment the target lands the scan ends, and the markets after it are not read." Under Scan All no target lands; every market is read.

`docs/manual/08-tabs/market-inspector.md:5833` - "The board keeps the timers, so a tab change loses nothing and both variants read one set". The board still keeps them; the zone now draws one tile per timer.

**Figures.** This page carries no figure and this entry adds none. A count of the markdown image tags on the page answers 0 before this entry and 0 after it.

## 2026-09-23 22:10 - #23 - The confirmation read waits a count of the hit's own candles, and each timer tile carries a red x

The operator, 2026-09-23: *"Confirmation timers are too short and should be a multiple of the timeframe on which the hit occurred i.e. a weekly chart confirmation read should not be happening again in a few days but in a few weeks and a daily chart should happen in a few days rather than a few hours."* And: *"Confirmation Reading timers need a small red 'x' for their deletion and this provides a confirmation pop up."*

### The interval

A hit's first confirmation read still waits for the candle the hit was read on to close. Every read after it waits a count of closed candles of the hit's own timeframe. The count is `Confirmation read candles` on the ATA-SPM settings page and it starts at 3. A 1d hit therefore reads again three days on, and a 1wk hit three weeks on.

`src/trading/ata_spm_push.py` - the count and its default

```
DEFAULT_CONFIRMATION_CANDLES = 3
ONE_CANDLE = 1
```

`next_read_ts` holds the whole rule. It starts at `candle_close_ts(at_ts)`, the close of the candle the hit was read on, and steps `candles` closes at a time until it passes the clock. The grid is anchored at that first close, so a read that the venue delayed, and a retry, both land back on it.

`src/trading/ata_spm_push.py` - the next read

```
def next_read_ts(
    at_ts: Any, timeframe: Any, now: Any, candles: Any = ONE_CANDLE
) -> float:
    """The next confirmation read after ``now`` on the grid that begins at ``at_ts``.

    The first read is the close of the candle opened at ``at_ts``; every read
    after it waits ``candles`` more closes of the same timeframe.
    """
```

`FollowUpWatch.watch_run` arms each timer with the count, and `FollowUpWatch.take_outcome` sets the next read with it. `PushBoard` supplies it from `self.settings.confirmation_candles` in `after_scan` and in `take_outcome`, so one setting reaches both. The retry is unchanged: a venue that has not published the candle the timer waited for is read again after `FOLLOW_UP_RETRY_S`, at most `FOLLOW_UP_RETRY_CAP` times, and the timer then returns to the grid.

A changed count re-anchors a timer that is already running. The timer keeps the grid that begins at the hit's own candle close and takes the new step, so the one read across the change can fall sooner than the new count. Every read after it is the new count apart. Read on the running program: a 1d timer whose next read stood at the first close plus three candles, with the count moved from 3 to 5, read next at the first close plus five.

### The setting

`Confirmation read candles` sits beside `Confirmation share %` on the ATA-SPM settings page of both builds, and is stored the same way.

`src/gui/main_tabs/market_inspector_surface.py` - the row

```
SETTING_CONFIRMATION_CANDLES = "confirmation_candles"
```

`SETTING_ROWS` names the row and `COUNT_SETTINGS` says it takes a whole number, so `setting_rows` draws it and `set_setting` writes it in both builds with no host code of its own. `AtaSpmSettings.confirmation_candles` is a property whose setter saves, and `PERSISTED_SETTINGS` names it beside `confirmation_share_pct`, so `~/.acervator/ata_spm_settings.json` carries it and the next launch reads it back. `read_candles` reads text that is not a whole number, and any count under one, as 3.

### The x

Each timer tile carries a small red x at its top right. A press opens one pop-up that names the pair. **No** closes it and the tile, its countdown and its timer stand. **Yes** removes that tile and its timer, and nothing else: the call keeps its entry in Ready to Send, the bucket keeps its count, and every other tile keeps its countdown. The Activity Log writes `ATA-SPM confirmation timer for ARB 1d stopped: the operator deleted it`.

`src/gui/main_tabs/market_inspector_surface.py` - the x and its pop-up

```
TIMER_CLOSE_PART = "timer-close"
TIMER_CLOSE_TEXT = "×"
TIMER_DROP_TITLE = "Delete confirmation timer"
```

The Qt tile builds the x as a `QPushButton` in `TimerTiles._build_tile`; the page builds it as a `button` in `TimerClose`, which carries the row's `close_value`. Both reach `MarketInspectorTab._ask_drop_timer`, the Qt tile through the button and the page through `run_action`. That method opens the `QMessageBox` and, on a Yes, calls `PushBoard.drop_timer`, which drops the one timer from `FollowUpWatch.timers`, or the one settled call from `FollowUpWatch.settled`, and touches `ReadyToSend.posts` not at all. The pop-up comes from the host in both builds, the way the hit chime does, so both builds ask the same question.

### The interval and the x read off both running builds

Both builds, the home on a scratch directory, every socket but loopback refused, the loopback stand-in serving the Coinbase Exchange candles shape through the real `CoinbasePublicCandles` route, its clock movable, and one planted hit on ARB. Connections refused: 0. Requests at the stand-in: 123.

| reading | Qt | React |
|---|---|---|
| 5m hit at 21:50, its own candle closes 21:55 | armed at 21:55, 0 candles past the first close | armed at 22:10 from a 22:05 hit, 0 candles past |
| the 5m next read after the first | 22:10, 3 candles on | 22:25, 3 candles on |
| 1h hit at 21:00, its own candle closes 22:00 | armed at 22:00, 0 candles past | armed at 23:00 from a 22:00 hit, 0 candles past |
| the 1h next read after the first | 01:00, 3 candles on | 02:00, 3 candles on |
| 1d hit on 09-23, its own candle closes 09-24 | armed at 09-24 00:00, 0 candles past | the same |

| reading | Qt | React |
|---|---|---|
| the 1d next read after the first | 09-27 00:00, 3 candles on | 09-27 00:00, 3 candles on |
| 1wk hit on 09-21, its own candle closes 09-28 | armed at 09-28 00:00, 0 candles past | the same |
| the 1wk next read after the first | 10-19 00:00, 3 candles on | 10-19 00:00, 3 candles on |
| the clock one candle short of the next read | no second read; reads stay at 1 | the same |
| the clock two seconds past it | the second read runs; reads 2 | the same |
| the count typed as 5, the file after | `"confirmation_candles": 5` | the same, typed through the page |
| the next read after the change, 1d | 09-29 00:00, 5 candles past the first close | the same |
| a second launch on the same home | (read on React) | the row reads 5 and the timer arms 5 candles on |
| the tile | `ARB 1d` over `01:48:34`, one x | `ARB 1d`, `×`, `01:47:23` in the page's own text |
| the pop-up | `Delete confirmation timer` · `Delete the confirmation timer for ARB 1d?` · Yes, No | the same box, opened by the page's press |
| No pressed | 1 tile, 1 x, bucket 10, next read unmoved | the same |
| Yes pressed | 0 tiles, 0 x, bucket 10 | 0 tiles and 0 x in the page's own DOM, bucket 10 |
| the settings page | six rows, `Confirmation read candles` reading 5 | the same six rows |

### The red, read off the program before the change

The same stand-in and the same planted hit, on the build before this entry: a 1d hit's next read after the first was 09-25 00:00, one candle on; a 1wk hit's was 10-05 00:00, one candle on. The settings page held five rows and named no candle count. No tile carried a close control: 0 of 1.

### Two plants, each reading fails

The count planted to 1: the 1d next read read 09-25 00:00, one candle on, so the reading that says three candles fails. The x's handler planted to drop with no question: the pop-up did not open and the tile left on the first press, so the reading that says a pop-up opens fails.

### The three sentences the interval and the x overtake

Not reworded, quoted here.

`docs/manual/08-tabs/market-inspector.md:5825` - "The interval is one candle of the hit's own timeframe: a 1d hit reads at each daily close, a 1wk hit at each Monday 00:00 UTC, the day the fetcher's rollup and Yahoo stamp a week with; a 1M hit at the first instant of each month." The first read is still that close. Every read after it waits `Confirmation read candles` closes of the same timeframe, three by default.

`docs/manual/08-tabs/market-inspector.md:5837` - "The next read is the first close after now on the grid of closes that begins at the hit's candle." The grid still begins at that close, and it now steps

`Confirmation read candles` closes at a time.

`docs/manual/08-tabs/market-inspector.md:6708` - "A tile reads the market and its timeframe on one line, the way the Activity Log names the timer, and on the second line the time to the call's next read, counted down once a second on the tab's own clock." The tile reads the same two lines, and a small red x sits at the right of the first one.

**Figures.** This page carries no figure and this entry adds none. A count of the markdown image tags on the page answers 0 before this entry and 0 after it.

## 2026-09-23 23:40 - #23 - The derivatives list serves only products the venue answers a candle for, and the dropped count sits on the order line

The operator, 2026-09-23: *"Inspector -> ATA-SMP -> Market Sectors -> Several slots for derivatives are empty or were showing no candles returned."*

### The admission rule for a derivatives product

The product list alone is no longer enough. A product reaches the derivatives list when the venue says it trades it **and** answers at least one candle for it. The candle route serves two granularities, and the weekly line is a rollup of the daily one, so the read asks the hourly granularity, then the daily one, and stops at the first that answers. A product that answers on one of them and not the other is not a no-candle product: it stays in the list.

`src/trading/ata_asset_maps.py` - the two functions the rule is

```
def venue_granularities(venue: Any) -> tuple:
    """The timeframes one venue answers from its own route, rolled keys aside."""


def candle_served(symbol: Any, read: Any = None) -> bool:
    """Whether the venue answers a candle for ``symbol`` on any granularity."""
```

A product every granularity answers none for is dropped where the list is built, so no row is ever drawn for it and the scan never reads it. The dropped rows stay reachable by name, so a product id typed into the ticker field still resolves and still scans.

### Where the dropped count is drawn

The count is the point: a shorter list with nothing said about it reads as lost data. Press the ATA-SMP card to open it, and the first line inside it is the order line. It now carries a tail naming how many products were dropped and which.

`src/trading/ata_spm.py` - the tail on the order line

```
ORDER_NO_CANDLE_FORMAT = (
    "{line}; {count} dropped, the venue answered no candle on any "
    "granularity: {names}"
)
```

### The list read off both running builds

Read on 2026-09-23 with a loopback stand-in answering both public Coinbase routes, in the Qt window and on the React page, home on a scratch directory, every name lookup outside loopback refused. The stand-in lists 14 products and says it trades 12. Three of the 12 answer no candle on either granularity, and one answers no candle on the hourly granularity and 40 candles on the daily one.

| reading | before, Qt | before, React | after, Qt | after, React |
|---|---|---|---|---|
| rows on the order | 12 | 12 | 9 | 9 |
| lines drawn in the field | 12 | 12 | 9 | 9 |
| lines reading `no candles` | 3 | 3 | 0 | 0 |
| products counted as dropped | 0 | 0 | 3 | 3 |
| the one-granularity product in the list | yes | yes | yes | yes |
| candle reads over the press | 36 | 36 | 43 | 43 |

The order line the open card draws after the change, identical on both builds:

```
Phase 1 Evaluate order: Order by coinbase futures and perpetuals, 24 h volume
x price, 9 expiring, 0 perpetual, 1 with no figure last by name:
BIT-25SEP26-CDE, BIP-20DEC30-CDE, ET-25SEP26-CDE, A23-25SEP26-CDE,
A36-25SEP26-CDE, A18-25SEP26-CDE, DOG-25SEP26-CDE, A34-25SEP26-CDE,
AVX-25OCT31-CDE; 3 dropped, the venue answered no candle on any granularity:
LNK-25OCT31-CDE, DOT-25OCT31-CDE, NER-25OCT31-CDE
```

The one-granularity product is `AVX-25OCT31-CDE`, and the same card reports which granularity carried it:

```
Phase 1 Evaluate 1hr: 8 vote(s), 1 without candles, 0 under 30 candles
Phase 1 Evaluate 1d: 9 vote(s), 0 without candles, 0 under 30 candles
```

### The reading with every candle read answering none

With the stand-in answering no candle for every product on every granularity, the list holds 0 rows, the field draws 0 lines, and the open card counts all 12 on the order line. Both builds read the same. That is what proves the reading can fail rather than pass by emptiness.

### No other class moves

Read on the same press path, on the commit this entry branched from and on the change, in the Qt window: metals 10 rows, energy 4, forex 28, stocks 47, crypto 120, and none of the five carries a dropped product. The rule reads the derivatives venue's own granularity table, so no other class can reach it.

### The two sentences the admission rule overtakes

Neither is reworded. Both are quoted here and the true sentence follows.

`docs/manual/08-tabs/market-inspector.md:6327` - "Each product the venue says it trades becomes one row on the futures venue, ranked by the venue's own 24 h volume times price, and the rows stay in the process so the ticker field offers them after the press." Each product the venue says it trades **and answers a candle for** becomes one row, ranked the same way. The ticker field offers the products that reached the list; a dropped product id typed in full still resolves and scans.

`docs/manual/08-tabs/market-inspector.md:6371` - "Read once against the venue on 2026-09-20, on both bundles: the list held 117 trading products, every one a dated contract on the CDE venue marked `EXPIRING`, and none carrying the `-PERP-INTX` suffix the venue gives its perpetuals; 57 of the 117 carried no 24 h volume, and 43 answered no candle on any granularity. The scan read all 117 and found no hit." Those figures stand as the 2026-09-20 reading and are not re-measured here: the change was built and read with no venue contacted. Under the rule above, the 43 would not have reached the list and the scan would have read 74.

**Figures.** This page carries no figure and this entry adds none. A count of the markdown image tags on the page answers 0 before this entry and 0 after it.

## 2026-09-23 - #290 - Topology Proposals refreshes when the scan lands, and both panels name their own condition

Two zones read empty and gave no reason. This entry describes what each one's source is, when each reads it, and what each now says when its source answers nothing.

### Opposing Trades: its source, and the sentence it now carries

The zone draws the pairs `MarketInspector._find_opposing_pairs` kept on the last scan. That producer takes every long-signal market against every short-signal market, keeps the ones whose 30-day return correlation sits in the negative window, and hands each survivor to the equilibrium test. A market reaches long or short only when two or more of its timeframes sit at the same Bollinger extreme.

Three counts decide what the zone says when it holds no pair, and all three come off the analyzer the scan wrote:

| Count | Read from |
|---|---|
| markets read | `MarketInspector.last_signals` |
| markets with a direction | the `direction` each signal carries |
| pairs tested | `MarketInspector.last_tested` |

`src/gui/main_tabs/market_inspector_surface.py` - the three counts

```
def scan_counts(signals: Any, tested: Any) -> dict:
    """The three counts the Opposing Trades funnel line names.
```

`last_tested` holds one entry per candidate the equilibrium test ran on, whether it passed or not. Before this entry no screen read it.

Read on the running program, both builds, after a scan over a four-market snapshot: **Scan finished. 4 market(s) read, 0 with a direction, 0 pair(s) tested, none held equilibrium.**

A scan that kept a pair is unchanged: the zone draws the pair and the line names the profit share.

### Topology Proposals: when it reads, and what it says before the scan

The pane asks `MainWindow._build_topology_proposals`, which builds the detector's context. Three of the four detectors test every candidate over `closes_by_asset`, and that map comes from `get_shared_inspector().last_closes` — written only by a finished Inspector scan. The fourth, `detect_distance_to_band`, reads the bot roster alone and pairs a scrum-deep bot with a fold-deep bot on the same asset.

The pane now reads the detector again the moment a scan finishes, as well as on its own Refresh press and its ten-minute timer.

`src/gui/market_inspector.py` - the read a finished scan triggers

```
def _refresh_proposals(self) -> None:
    """Ask the proposals pane to read the detector again.
```

Both panes answer `refresh` under that one name, so the Qt widget and the React host take the same call.

Before any scan the pane says: **No proposals yet. The detector reads the closes the scanner writes, so press Refresh on the left half first.** While a scan runs it says: **A scan is running. Proposals are built from what it finds.** After a scan it keeps the sentence it already had.

`src/gui/main_tabs/market_inspector_topologies_surface.py` - the three sentences

```
def empty_proposals_text(scan_state: Any) -> str:
    """The sentence an empty pane carries for one Inspector scan state.
```

The sentence is a value inside the payload the page already draws, so the React build carries it with no page change.

### The readings, both builds, at 700, 900 and 1400

| Reading | Opposing Trades rows | Proposals rows |
|---|---|---|
| before a scan | 0 | 0 |
| after a scan over the four-market snapshot | 0 | 0 |
| with a pair and a proposal handed in | 1 | 1 |

The third row is the control: each panel draws when its source answers. Both builds read the same at all three widths. The pane was asked 0 times before the scan and 1 time after it, in both builds, which is the read this entry adds.

### The reading with each source planted blind

Clearing the three counts returns the zone to `Scan finished. No opposing trades found.` Dropping the scan-state source returns the pane to `No proposals right now. Try Refresh, or wait for market state to shift.` A pane whose `refresh` raises is asked once, the tab logs a warning and the scan still finishes. That is what proves each reading can fail rather than pass by emptiness.

### Why both zones still read zero on real data

Neither count is a defect in the zone. Measured with no venue contacted:

- The scan snapshot on disk holds four markets. None reached a direction, so the producer had no candidate to test.
- `HTF_TIMEFRAMES` names three timeframes and the fetcher builds two, so a market must sit at the same extreme on both the daily and the weekly before it carries a direction at all.
- `detect_all_topologies` answers 0 on the bot roster alone, because every bot in the fleet read here trades a different asset and `detect_distance_to_band` needs two bots on one asset.
- Over real daily closes for five assets the correlation test passes on two pairs and every cluster it can form holds two members, under the three `detect_momentum_funnel` needs and the four `detect_sector_cluster` needs.

### The two sentences this entry overtakes

Neither is reworded. Both are quoted here and the true sentence follows.

`docs/manual/08-tabs/market-inspector.md:163` - "a scan finished and found nothing". For Opposing Trades a finished scan that kept no pair now names how many markets it read, how many carried a direction and how many pairs it tested. The wording quoted at line 180 is still what the zone says when those counts are not available.

`docs/manual/08-tabs/market-inspector.md:213` - "`MarketInspectorTopologies` in `src/gui/market_inspector_topologies.py` renders one card per proposal and opens a preview dialog on Preview." It still does, and it now also takes the Inspector's scan state, which decides which sentence an empty pane carries.

**Figures.** This page carries no figure and this entry adds none. A count of the markdown image tags on the page answers 0 before this entry and 0 after it.

## 2026-09-23 - #23 - The ATA-SMP field draws the reading of the market under read, and neither build moves its scroll

His words, 2026-09-23: *"Observed a very strange issue with the lower information panel when attempting to scroll down to see more text. This seemed to trigger a chart re-read loop of some kind. During this I saw the rapidly spawning and despawning windows again. Need to get rid of the informational text during reads and only display the really nice looking IVP information that has only been visible in the larger floating windows up to this point. The IVP data keeps the symbology consistent and its really well designed from what I have seen of it. We have no use a for a repeating spool of identical data lines."*

### What the field draws while a walk runs

The Indicator Voting Panel of the market under read, and nothing else. One row per timeframe that answered candles, one cell per voter, and the Net, Comp and Conf columns, in the cell colours `indicator_panel_surface` gives the Live tab's own panel. The title names the market, the word `reading`, and the tally of bullish, bearish and neutral votes over the rows.

`src/trading/ata_spm.py` - the reading the walk reports after each market

```
    #: The market just read, empty before one.
    symbol: str = ""
    #: ``panel_rows_for`` over the market just read: its Indicator Voting
    #: Panel rows, one per timeframe, which the zone draws while it walks.
    panel: dict = field(default_factory=dict)
```

`src/gui/main_tabs/market_inspector_surface.py` - one builder, two callers

```
def panel_grid(symbol: Any, label: Any, rows: Any, lines: Any = ()) -> Optional[dict]:
    """One Indicator Voting Panel: its title, its two ``panel_table`` tables
    and its ``lines``.

    ``voting_panel`` and ``scan_panel`` both build through this, so a called
    asset and a market under read draw the same reading the same way.
    """
```

`voting_panel` is the called asset's grid, drawn in an open entry with its gate chain lines under it. `scan_panel` is the same grid for the market under read, with no gate line, because no chain has run over that market yet. Both are one `panel_grid` call, so the two cannot drift.

A walk that has read no market yet holds no reading. The entry then keeps the zone's own sentence as its headline, so the field names its state rather than drawing nothing.

### Neither build moves the field's scroll

The Qt stepper no longer drags its scroll bar to the bottom at each progress, and the page's entry no longer sets `scrollTop` to `scrollHeight` on each render. The field holds where the operator left it.

### The field readings, both builds, at 700, 900 and 1400

Read off the running program, the home on a scratch directory, every socket but loopback refused, the candles served from read-only copies of the Coinbase 5 m stone tablets aggregated to 1 h, 1 d and 1 w. No venue was contacted. One scan of 93 markets in each build, one wheel step up on each of six readings.

| Reading, while the walk runs | before | after |
|---|---|---|
| text lines in the field, Qt 1400 | 35, 42, 54, 70, 83, 92 | 0 at every step |
| voting grids in the field, Qt 1400 | 0 at every step | 1 at every step |
| text lines in the field, the page 1400 | 36, 44, 63, 81 | 0 at every step |
| voting grids in the field, the page 1400 | 0 at every step | 1 at every step |
| the scroll 0.8 s after one wheel step up | back at its own maximum, 6 of 6 Qt and 4 of 4 page | where the wheel left it, every step, both builds |
| market reads over one whole scan, Qt | 372 | 372 |
| market reads over one whole scan, the page | 372 | 372 |

372 is 93 markets times 4 timeframes. The same scan run with no scroll at all reads 372 as well, so **a scroll causes no read** and caused none before this entry either. What looped was the field: every market rewrote it and dragged its scroll back to the bottom, 93 times in one scan. The same readings at 900 and at 700 hold: 0 text lines, 1 voting grid, the scroll where it was left, 372 reads.

The open entry is unchanged: 30 expanded lines and 2 voting grids before and after, at all three widths, in both builds.

### What the grid asks for, and where it does not fit

`panel_column_widths` adds to 592 px: a 44 px TF column, six 58 px voter columns, two 52 px columns and a 96 px Conf column. The left pane holds about 840 px at a 1400 px window, about 530 px at 900 and about 415 px at 700. The grid therefore draws whole at 1400 and, at 900 and 700, draws its first columns with the Net, Comp and Conf columns past the pane edge and a horizontal scroll bar under it. This is the width `voting_panel` has always asked for in an open entry; the field now asks for it during a read as well. The grid is reachable at every width and complete at 1400. Fitting it to the pane changes the width contract for both hosts and for the open entry, and is not done here.

### The windows, counted two ways

Two instruments ran together: a Qt event filter counting every parentless widget that receives a show, and an `EnumWindows` poll counting every visible top-level window this process owns. Over a whole scan with six scrolls, a scroll pass on the open entry, and ten opens and closes of an entry holding two voting grids: **0 parentless shows and 1 native window, before and after, in both builds.** A planted parentless label raised the first count from 0 to 1 and the second from 1 to 2 on every run, so each reading can fail.

The spawn he reports did not reproduce on this commit through the field's scroll, its read or its open. The teardowns that could still open one are closed. `_drop_panel_box`, `_show_panel_lines`, `_show_actions` and `TimerTiles.show_tiles` each hide a widget before reparenting it, the way `_show_lines` already did. `_VotingPanel` tears no cell down at all: a payload asking for a different count of cells is refused, and `_show_panels` replaces that whole panel. `TimerTiles._build_tile` gives its frame a parent at birth instead of after its style.

### The two sentences the field overtakes

Neither is reworded. Both are quoted here and the true sentence follows.

`docs/manual/08-tabs/market-inspector.md:5411` - "While the scan walks, the field draws one line per market as it is read, newest last: the market, the timeframes that answered candles, the vote count and the verdict". The field now draws the Indicator Voting Panel of the market under read. That line still exists and still carries the same four things; it is written to the Activity Log, beside the read lines it names, rather than into the field.

`docs/manual/08-tabs/market-inspector.md:6447` - "While the counter is set the scroll follows the newest line, in both variants." Neither build moves the field's scroll now. The counter still sits at the right of the position row above the field.

**Figures.** This page carries no figure and this entry adds none. A count of the markdown image tags on the page answers 0 before this entry and 0 after it.

## 2026-09-24 - #23 - Commodities is the asset class, and Energy and Metals are its sectors under the S&P GSCI names

The operator, 2026-09-24, on whether Energy and Metals sit under Commodities or beside it: *"I just want to adopt the standard categorization and language so that the implementation is professional and accurate..."*

### The authority the names now follow

S&P Dow Jones Indices publishes the S&P GSCI methodology, at `https://www.spglobal.com/spdji/en/documents/methodologies/methodology-sp-gsci-quick-guide.pdf`. That index makes **commodities** the asset class and divides it into five sectors:

| sector | what the index puts in it |
|---|---|
| energy | crude oil, heating oil, natural gas, gasoline |
| industrial metals | aluminium, copper, lead, nickel, zinc |
| precious metals | gold, silver |
| agriculture | wheat, corn, soybeans, sugar |
| livestock | lean hogs, live cattle |

The map already cited that index. The energy class record read *"S&P GSCI groups energy as petroleum and natural gas"*, and under the same index energy is a sector of commodities, not a peer of equities.

### The class menu

`metals` and `energy` are no longer classes. `commodities` takes their place and holds their markets. Crypto, stocks, derivatives and forex are untouched.

```
before   crypto  stocks  metals  derivatives  forex  energy
after    crypto  stocks  commodities  derivatives  forex
```

`src/trading/ata_spm.py` - the class list

```
ASSET_CLASSES = (
    CLASS_CRYPTO,
    CLASS_STOCKS,
    CLASS_COMMODITIES,
    CLASS_DERIVATIVES,
    CLASS_FOREX,
)
```

### Where every market landed

Each market is placed by what the asset is, not by the list it sat in. The four spot pairs and the four metal funds are precious metals. The two base-metal funds are industrial metals. The three petroleum funds and the gas fund are one energy sector, as the index puts crude, heating oil, gasoline and natural gas together.

| market | what it is | old sector | new sector |
|---|---|---|---|
| XAU/USD | gold spot | metals / spot | commodities / precious metals |
| XAG/USD | silver spot | metals / spot | commodities / precious metals |
| XPT/USD | platinum spot | metals / spot | commodities / precious metals |
| XPD/USD | palladium spot | metals / spot | commodities / precious metals |
| GLD | gold fund | metals / spot | commodities / precious metals |
| SLV | silver fund | metals / spot | commodities / precious metals |

| market | what it is | old sector | new sector |
|---|---|---|---|
| PPLT | platinum fund | metals / spot | commodities / precious metals |
| PALL | palladium fund | metals / spot | commodities / precious metals |
| CPER | copper fund | metals / base | commodities / industrial metals |
| DBB | aluminium, zinc and copper fund | metals / base | commodities / industrial metals |
| USO | WTI crude fund | energy / petroleum | commodities / energy |
| BNO | Brent crude fund | energy / petroleum | commodities / energy |
| UGA | gasoline fund | energy / petroleum | commodities / energy |
| UNG | natural gas fund | energy / gas | commodities / energy |

Fourteen markets before, fourteen after. None was added, dropped or re-read: no venue was contacted for this change.

Platinum and palladium are placed in precious metals by what they are. The index's own constituent list names gold and silver alone, so neither metal is an S&P GSCI member; both are precious metals in every market convention and that is where they sit.

### The two sectors that list nothing

Agriculture and livestock are sectors of the standard and this map holds no market for either. No fund listed in dollars holds them, and the map carries no futures, because a chart of a futures series joins contracts at a price nobody traded.

**They are not drawn as a sector the scan can walk, and their count is drawn under the ticker field.** A sector with no market is a slot that returns nothing, and a slot that returns nothing does not belong in the list. Leaving them out with nothing said would read as lost data, so the line names both and gives the count.

```
agriculture, livestock: 0 market(s) listed, no fund listed in dollars holds
them, and this map carries no futures.
```

The same line names any class's empty sector. Choosing forex draws:

```
exotic: 0 market(s) listed, no pair is named for this tier yet.
```

`src/gui/main_tabs/market_inspector_surface.py` - the line and what fills it

```
TICKER_EMPTY_SECTORS_FORMAT = "{sectors}: 0 market(s) listed, {reason}."


def empty_sector_line(asset_class: Any) -> str:
    """The line naming one class's sectors that list no market, with its count."""
```

### A held selection naming a retired name still scans

A class name and a sector name the earlier taxonomy used both resolve. Choosing `metals` or `energy` lands on commodities. Typing `spot`, `base`, `petroleum` or `gas` scans the sector those markets moved to. A name neither taxonomy holds is still refused, so no scan runs on a class with no timeframes behind it.

`src/trading/ata_spm.py` and `src/trading/ata_asset_maps.py` - the two seams

```
def asset_class_named(name: Any) -> str:
    """The live ``ASSET_CLASSES`` name one given class name resolves to."""


def sector_named(sector: Any, asset_class: Any) -> str:
    """The key one class's map holds for a given sector name, empty for none."""
```

`~/.acervator/ata_spm_settings.json` never held a class name. Read on a scratch copy, the file carries `hits_per_scan`, `confirmation_share_pct` and `confirmation_candles`, and no value in it is a class or a sector name.

### The menu read off both running builds

Read on 2026-09-24 in the Qt window and on the React page, home on a scratch directory, no bot manager, every name lookup outside loopback refused and no venue contacted. Taken at client widths 700, 900 and 1400. All six readings agree.

| reading | before | after |
|---|---|---|
| names in the class box | 6 | 5 |
| buttons under Asset Category | 6 | 5 |
| classes holding a sector map | 4 | 3 |
| sectors the commodities markets sit in | 4, over two classes | 3, in one class |
| markets in those sectors | 14 | 14 |
| sectors drawn that list no market | 0 named, 1 silent | 0 drawn, 3 named with a count |
| class box inside the pane at 700 | yes | yes |
| sideways scroll on the page at 700 | none | none |

The sectors and their counts after the change:

| class | sector | markets |
|---|---|---|
| commodities | energy | 4 |
| commodities | industrial metals | 2 |
| commodities | precious metals | 8 |
| commodities | agriculture | 0, named under the field |
| commodities | livestock | 0, named under the field |
| forex | major | 7 |
| forex | minor | 21 |
| forex | exotic | 0, named under the field |
| stocks | portfolio | 47 |

Each sector's own line, as the entry headline draws it:

```
energy (commodities)
industrial metals (commodities)
precious metals (commodities)
```

### The reading can fail

A class name was planted into the class box in the window and into the select on the page, and the menus were read again. Both answered seven names with the plant in and five with it out, at all three widths in both builds.

### The six sentences this entry overtakes

None is reworded. Each is quoted and the true sentence follows.

`docs/manual/08-tabs/market-inspector.md:459` - "Stocks, metals, derivatives and forex share the slower one." Stocks, commodities, derivatives and forex share it.

`docs/manual/08-tabs/market-inspector.md:3126` - "The menu beside it is the sector menu, and it holds crypto, stocks, metals, derivatives and forex." It holds crypto, stocks, commodities, derivatives and forex.

`docs/manual/08-tabs/market-inspector.md:4113` - "Energy shares it too." Commodities shares it, and energy is one of its sectors.

`docs/manual/08-tabs/market-inspector.md:4117` - "holds energy too." The menu holds commodities, and energy is a sector inside it.

`docs/manual/08-tabs/market-inspector.md:4120` - "Four carry a list. Energy holds 4 names." Three classes carry a sector map. Commodities holds 14 names over three sectors that list a market.

`docs/manual/08-tabs/market-inspector.md:4123` - "Six asset classes keep their own control." Five asset classes keep their own control.

**Figures.** This page carries no figure and this entry adds none. A count of the markdown image tags on the page answers 0 before this entry and 0 after it.

## 2026-09-24 - #23 - The scanner walks the standard tree on every route, and a retired sector name is scanned under the sector holding it

The class and sector names moved to the published ones. This entry is the scanner that reads them: every route that names a class or a sector, what each one counts, and what each one calls the thing it found.

### The routes that name a class or a sector

Twenty-three routes name one, and each was read off the running program.

| route | where |
|---|---|
| the class menu's items | `src/gui/market_inspector.py:1235` |
| the Asset Category buttons | `src/gui/market_inspector.py:1372` |
| the page's class rows and its class list | `src/gui/main_tabs/market_inspector_surface.py:2203,4082` |
| the class a press takes | `src/trading/ata_spm.py:2129,381` |

| route | where |
|---|---|
| the timeframes and the round budget a class sets | `src/trading/ata_spm.py:397,413,421` |
| Scan Now on an empty field, a sector name, a ticker | `src/trading/ata_spm.py:2217,2209,2205` |
| the market list, the ticker list and the listing of one class | `src/gui/main_tabs/market_inspector_surface.py:3062,2834,3144` |
| the class order a typed ticker is placed against | `src/gui/main_tabs/market_inspector_surface.py:3137,3182` |
| the rows one sector holds | `src/gui/main_tabs/market_inspector_surface.py:2810` |
| the sectors a class holds, and the ones it lists nothing for | `src/trading/ata_asset_maps.py:559,569,579` |
| the sector one name resolves to, and its rows | `src/trading/ata_asset_maps.py:584,598` |
| the line under the ticker field | `src/gui/main_tabs/market_inspector_surface.py:3254,3230` |
| Scan All's walk and its place line | `src/trading/ata_spm.py:2368,243` |
| the zone headline and its counts | `src/gui/main_tabs/market_inspector_surface.py:1557,1573` |
| the confirmation timers and their tiles | `src/trading/ata_spm_push.py:1164,1436` |

### What Scan All walks, and the two sectors it leaves out

A Scan All press builds one walking sector per class, and that sector holds the whole class's market list. Every listed market of every listed sector is therefore walked, with no stop at a hit count.

```
def compute_all(self, ...):
    """One ``walk_all`` sector per class in ``ASSET_CLASSES``, on every
    timeframe ``timeframes_for`` lists for it and every market
    ``class_source`` lists."""
```

Agriculture and livestock are left out, and the reason is stated rather than silent: `sectors_for` answers only the sectors holding a market, both hold zero, and `unlisted_sectors` names both under the ticker field with their count. The same rule leaves forex's exotic tier out.

### A retired sector name is labelled by the sector holding it

Typing `spot` and pressing Scan Now scanned the precious metals rows and drew the zone as `spot (commodities)`. The markets were right and the word was the retired one, while the class side already normalised: `metals` chosen in the class menu leaves the menu reading `commodities`.

Each map row now carries the sector it is listed under, and the board names a typed sector by the name its rows carry.

```
def sector_key_of(rows: Any, typed: Any) -> str:
    """The sector name the asset rows carry, ``typed`` while they carry none."""
```

| typed, commodities chosen | zone headline before | zone headline after | assets |
|---|---|---|---|
| `spot` | `spot (commodities)` | `precious metals (commodities)` | 4 |

| typed, commodities chosen | zone headline before | zone headline after | assets |
|---|---|---|---|
| `base` | `base (commodities)` | `industrial metals (commodities)` | 2 |
| `petroleum` | `petroleum (commodities)` | `energy (commodities)` | 4 |
| `gas` | `gas (commodities)` | `energy (commodities)` | 4 |
| `precious metals` | `precious metals (commodities)` | `precious metals (commodities)` | 4 |

No count moved. The field keeps what the operator typed.

### The chosen class is walked first whatever it is named

A typed ticker is matched against each class's names, the chosen class first. With a retired class name held, that walk started at crypto and reached commodities fourth, so a ticker two classes both list landed in the wrong one. The walk resolves the name now.

```
def class_walk(asset_class: Any) -> list:
    """``ata_spm.ASSET_CLASSES`` with the class ``asset_class`` names first."""
```

Read in both builds: `class_walk("metals")` answered `crypto stocks commodities derivatives forex` before and `commodities crypto stocks derivatives forex` after.

### A declared sector with no market keeps its line through a press

With commodities chosen, the line under the field reads `agriculture, livestock: 0 market(s) listed, no fund listed in dollars holds them, and this map carries no futures.` Typing `agriculture` and pressing Scan Now replaced that line with `No class lists ticker agriculture. Pick one the field offers.`, so the screen said two different things about one standard sector name.

The field now keeps the true line for any name the class declares and lists no market for. The press still runs nothing, which is what a sector of zero markets is.

### The confirmation timers carry no class name

A watched reversal call is told apart by its asset, its timeframe and its bar's open time, and by nothing else.

```
@property
def key(self) -> tuple:
    """What tells two watched calls apart: the asset, its timeframe, its bar's open time."""
```

A hit recorded while a retired class name was chosen therefore resolves and draws its tile unchanged.

### The scanner read off both running builds, before and after

Read on 2026-09-24 in the Qt window and on the React page, home on a scratch directory, no bot manager, every socket but loopback refused and no venue contacted. A loopback listener answered the Coinbase product list, the Coinbase product candles, the Yahoo chart endpoint and the Yahoo predefined screener in the shapes those routes serve. The crypto list was held to three names so one walk is readable in one run.

| Scan Now, empty field | markets listed | markets read | zone headline |
|---|---|---|---|
| crypto | 3 | 3 | `crypto in map order` |
| stocks | 3 | 3 | `stocks by volume` |

| Scan Now, empty field | markets listed | markets read | zone headline |
|---|---|---|---|
| commodities | 14 | 10 | `commodities by volume` |
| derivatives | 3 | 3 | `derivatives by volume` |
| forex | 28 | 28 | `forex in map order` |

Commodities lists 14 and reads 10 because the four spot pairs carry no venue. Every figure above is the same before this entry and after it, and the same in both builds.

| Scan All | before | after |
|---|---|---|
| walking sectors built | 5 | 5 |
| markets read | 47 | 47 |
| walk steps reported | 57 | 57 |
| place line | `Scanning crypto (1 of 5)` to `Scanning forex (5 of 5)` | the same |
| seconds, window and page | 97.6 and 98.4 | 97.8 and 97.4 |
| voting rows drawn under the field while it walked | 2 and 4 | 2 and 4 |

### The readings can fail

The sector name `precious metals` was planted as `planted sector` and the scanner was driven again. `sectors_for` answered `energy`, `industrial metals`, `planted sector`, and typing `spot` drew `planted sector (commodities)` while `base` and `petroleum` kept their own names. The file compares byte identical after the plant was taken out.

The new line-keeping branch was then blinded, with the declared-empty list forced to none. Typing `agriculture` went back to `No class lists ticker agriculture. Pick one the field offers.` and the other three names did not move. The file compares byte identical after.

### The three sentences this entry overtakes

None is reworded. Each is quoted and the true sentence follows.

`docs/manual/08-tabs/market-inspector.md:4754` - "matched against each class's names, the chosen class first and then the others in the order the class box lists them". A class named as the earlier taxonomy named it is resolved first, so the chosen class leads the walk under either name.

`docs/manual/08-tabs/market-inspector.md:7510` - "their count is drawn under the ticker field". The count stays under the field through a press on one of those sector names.

`docs/manual/08-tabs/market-inspector.md:7539` - "Typing `spot`, `base`, `petroleum` or `gas` scans the sector those markets moved to." It scans that sector and the zone names it, so the entry reads `precious metals (commodities)`, `industrial metals (commodities)` or `energy (commodities)`.

**Figures.** This entry adds none. A count of the markdown image tags on the page answers 0 before this entry and 0 after it.

## 2026-09-24 - #23 - Derivatives is not an asset class, each product takes the class of its underlying, and every class says whether a bot can deploy there

A futures contract is an instrument, not an asset class. Each Coinbase CDE product now sits under the class of the thing it is written on, and carries the contract form as a property of its row. Every class and every sector states two separate things: whether a retail trader can reach it from home, and whether this program has a venue wired to it.

### The standard this follows

`MAP_SOURCES` cites ISO 10962, the CFI standard, https://www.iso.org/standard/81140.html. It sorts a financial product into six categories: equities, debt, entitlements, options, futures and other. A future is therefore one category of instrument beside equities, not a class of asset. The contract is written on an underlying and the underlying carries the class.

### The class list

```
before   crypto  stocks  commodities  derivatives  forex
after    crypto  stocks  commodities  forex
```

Read in the class box on the Inspector and on the Asset Category buttons under Settings, in both builds, at 700, 900 and 1400. Five names before, four after, in both menus and at all three widths.

`derivatives` still resolves. Typed into the class box, held in settings, or read off a stored selection, it answers `crypto`, the same way `metals` and `energy` answer `commodities`.

| name given | class answered before | class answered after |
|---|---|---|
| `derivatives` | derivatives | crypto |
| `DERIVATIVES` | derivatives | crypto |
| `derivatives` | derivatives | crypto |
| `metals` | commodities | commodities |
| `energy` | commodities | commodities |
| `bullion` | none | none |

### How one product is placed

Diagram: flowchart, top to bottom

- CDE product record -> **non_crypto set?**
- B -- no -> **root names a crypto asset?**
- C -- yes -> **crypto, sector from the sector map**
- C -- no -> **placed in no class, named and counted**
- B -- yes -> **root in the published root table?**
- F -- yes -> **that class and that sector**
- F -- no -> **placed in no class, named and counted**

`futures_placement` reads `future_product_details.non_crypto` and `future_product_details.contract_root_unit` off the product's own record, with

`base_currency_id` as the fallback root. Nothing is guessed: a root neither side resolves is held back, named on the order line and counted.

```
def futures_placement(product: dict) -> tuple[str, str, str]:
    """The underlying, the asset class and the sector one futures product takes."""
```

### Every product the list answered, and where it went

Read on the running program in both builds, against a loopback stand-in for Coinbase's product route and its candle route. No venue was contacted.

| product id | underlying | class | sector | form |
|---|---|---|---|---|
| BIT-25SEP26-CDE | A15 | crypto | l1 | future |
| BIT-27FEB26-CDE | A15 | crypto | l1 | future |
| ET-25SEP26-CDE | A14 | crypto | l1 | future |
| A23-25SEP26-CDE | A23 | crypto | l1 | future |
| A36-25SEP26-CDE | A36 | crypto | payments | future |
| A18-25SEP26-CDE | A18 | crypto | l1 | future |
| DOG-25SEP26-CDE | A22 | crypto | meme | future |
| A34-25SEP26-CDE | A34 | crypto | l1 | future |
| AVX-25OCT31-CDE | AVAX | crypto | l1 | future |
| BIP-20DEC30-CDE | A15 | crypto | l1 | future |
| OND-20DEC30-CDE | A17 | crypto | rwa | future |
| SIL-25DEC26-CDE | silver | commodities | precious metals | future |
| GLD-25DEC26-CDE | gold | commodities | precious metals | future |
| NQ-25DEC26-CDE | Nasdaq-100 index | stocks | portfolio | future |
| LNK-25OCT31-CDE | A21 | crypto | oracle | future |

Two more products the venue lists answered no candle on any granularity and are dropped at list time with their names on the order line: DOT-25OCT31-CDE and NER-25OCT31-CDE. One product the venue lists does not trade and is never fetched. **One product named a root no class holds - ZZZ-25DEC26-CDE - and is reported rather than placed: 1 product, named on the order line.**

### The contract form each listing takes

| listing | form drawn |
|---|---|
| a crypto pair on the exchange | spot |
| XAU/USD, XAG/USD, XPT/USD, XPD/USD | spot |
| GLD, SLV, PPLT, PALL, CPER, DBB, USO, BNO, UGA, UNG | ETF |
| the RA portfolio equities and the screener's quotes | equity |
| a dated CDE contract | future |
| a perpetual CDE contract | perpetual |

The form rides on the row and is drawn on it. No control was added and no menu gained an entry.

### What a row says now

```
BIT-25SEP26-CDE  (crypto · future · charts only)
GLD              (commodities · ETF · charts only)
GLD-25DEC26-CDE  (commodities · future · charts only)
XAU/USD          (commodities · spot · charts only, not retail)
AAPL             (stocks · equity · charts only)
EUR/USD          (forex · spot · charts only)
A15              (crypto · spot · bot ready)
```

`bot ready` means a venue is wired and a bot can deploy. `charts only` means a retail trader can reach the market from home and no venue here can place an order on it. `charts only, not retail` means the row carries no venue at all, so nothing in this program reaches it.

### What each class and sector declares

| class | reachable from home | venue wired |
|---|---|---|
| crypto | yes - a retail account on a crypto venue takes orders over its API | yes - the connector the live bots trade on |
| stocks | yes - a retail broker account takes equity orders over its API | no - no broker connector is reachable and the quote source is not a broker |
| commodities | yes - every listed row is an ETF share a retail broker account buys | no - the same missing broker connector |
| forex | yes - a retail account at a registered FX dealer takes spot orders | no - no FX dealer connector exists |

Two sectors answer differently from their class.

| sector | reachable from home | venue wired |
|---|---|---|
| commodities / precious metals | yes for the four funds; the four spot pairs carry no venue and nothing here reaches them | no |
| a CDE futures row, in any class | yes - the venue takes orders on its US futures products over the same API | no - the futures adapter reads candles and places no order |

The two questions are kept apart. A market can be reachable in principle and have no venue wired here; ATA-SPM scans it either way, for its charts.

### ATA-SMP scans a class it cannot trade

Scan Now was pressed on the real control with **commodities** chosen, a class whose declaration reads `venue wired no`. The scan ran and the zone headline read `commodities by volume`. The order line inside the card carried the declaration:

```
Order by last complete daily bar volume x close on yahoo; 2 futures
product(s) joined, 4 with no figure last by name. commodities: reachable from
home yes -- every listed row is an ETF share a retail broker account buys;
venue wired no -- the same missing broker connector as the stocks class:
GLD-25DEC26-CDE, BNO, CPER, DBB (12 not read); 1 placed in no class, the
record names a root no class holds: ZZZ-25DEC26-CDE
```

### What each class lists now

| class | markets listed before | markets listed after |
|---|---|---|
| crypto | 120 | 132 |
| stocks | 47 | 48 |
| commodities | 14 | 16 |
| derivatives | 16 | class retired |
| forex | 28 | 28 |

The CDE products moved from one class of their own into the three classes their underlyings belong to. No product was added and none was dropped.

### The venue map with the class gone

Coinbase answered `crypto, derivatives` before and answers `crypto` after. A Coinbase spot pair and a Coinbase CDE product both answer `crypto` now, so the Live, Swarm, Charts, Inspector, History, Sim and Paper tabs all show a futures bot under Crypto. Read on the running window: under Crypto the Live tab held 2 bots, a spot pair and a CDE product; before the change it held 1, and the CDE product sat under a Derivatives button that no longer exists.

### The header strip and the tab filter

The class group reads `ASSET_CLASSES` when the window is built, so four names replace five with no edit to the strip.

| reading | before | after |
|---|---|---|
| class buttons in the row | 5 | 4 |
| every button reachable inside the pane | 5 of 5 | 4 of 4 |
| the group's own floor | 178 px | 142 px |
| the window's floor, first build | 1145 px | 1145 px |
| the window's floor, second build | 634 px | 598 px |

Read at 700, 900 and 1400 in both builds. Status and Console still do not filter; the other seven tabs still do.

### The placement readings can fail

A class name was planted into the class box and the menu was read again: six names with the plant in, four with it out, at all three widths in both builds.

The placement rule was then blinded twice. With the crypto names emptied, 15 of the 18 products fell to `placed in no class` and only the silver, gold and index contracts listed. With a planted root added to the root table, the one product that names an unknown root placed into commodities / agriculture under the planted underlying and the unplaced count went to 0. With both plants removed the reading returned to 15 listed and 1 unplaced.

### The six sentences this contract-form entry overtakes

None is reworded. Each is quoted and the true sentence follows.

`docs/manual/08-tabs/market-inspector.md:7450` - "Crypto, stocks, derivatives and forex are untouched." Crypto, stocks and forex are untouched, and derivatives is retired by this entry.

`docs/manual/08-tabs/market-inspector.md:7609` - "Stocks, commodities, derivatives and forex share it." Stocks, commodities and forex share the slower timeframe set.

`docs/manual/08-tabs/market-inspector.md:7614` - "holds crypto, stocks, commodities, derivatives and forex." It holds crypto, stocks, commodities and forex.

`docs/manual/08-tabs/market-inspector.md:7627` - "Five asset classes keep their own control." Four asset classes keep their own control.

`docs/manual/08-tabs/market-inspector.md:7716` - "`crypto stocks commodities derivatives forex` before and `commodities crypto stocks derivatives forex` after." The walk reads `crypto stocks commodities forex`, and a retired class name still leads the walk with the class holding its markets.

`docs/manual/08-tabs/market-inspector.md:7760` - "| derivatives | 3 | 3 | `derivatives by volume` |" That row is gone from the class walk. The CDE products are read under crypto, commodities and stocks, and each class's headline names that class.

**Figures.** This entry adds none. A count of the markdown image tags on the page answers 0 before this entry and 0 after it.

## 2026-09-24 - #881 - Every ticker row says whether a bot can size a scrum on that market, and a market none can is left off the list

A scrum sells the excess above a dollar target. A venue only accepts an order at or above its own published minimum, stepped onto its own increment. Until now no market on this screen said whether those two figures leave a scrum anything to submit.

One place answers it.

```
def tradeable_answer(
    rules: Any,
    price: Optional[float] = None,
    excess_usd: float = REFERENCE_SCRUM_EXCESS_USD,
) -> str:
```

### The three answers a market gives

| answer | drawn as | what it means |
|---|---|---|
| yes | `can size a scrum` | the smallest order the venue accepts costs no more than the excess |
| no | `cannot size a scrum` | it costs more, so a scrum there would have nothing to submit |
| unknown | `size rules not read` | no market record was obtained, so no rule was read |

Unknown is never read as either other. A market whose rules were never read stays on the list and draws its own state.

### Where the excess figure came from

The figure is the largest excess the saved fleet produces. Every bot in it carries the same 5 per cent scrumming interval, and the highest Target Balance is 350 dollars, so the largest excess is 17 dollars 50.

```
LARGEST_FLEET_TARGET_USD = 350.0
FLEET_SCRUMMING_INTERVAL_PCT = 5.0

REFERENCE_SCRUM_EXCESS_USD = scrumming_interval_usd(
    LARGEST_FLEET_TARGET_USD, FLEET_SCRUMMING_INTERVAL_PCT
)
```

The largest is the right end, not the smallest. `cannot size a scrum` has to mean no bot at any target this fleet carries could size one there. Measured on a read-only copy of the saved fleet: 38 bots, one interval value, targets from 25 to 350 dollars, excess from 1 dollar 25 to 17 dollars 50.

### The smallest order a market accepts

The venue's published minimum is ceiled onto its own increment, valued at the venue's own last price, and never falls under the venue's minimum order cost.

```
def smallest_order_usd(rules: Any, price: Optional[float]) -> Optional[float]:
```

The rules and the price both come off the market table each connector has already loaded. No venue is asked, and the rows redraw on every keystroke.

```
def trading_rules(
    exchange_connectors: Any, accepted_quotes: Iterable[str] = DEFAULT_QUOTES
) -> dict[str, tuple]:
```

### What a ticker row says now

```
A15    (crypto · spot · bot ready · can size a scrum)
CBBTC  (crypto · spot · bot ready · size rules not read)
AAPL   (stocks · equity · charts only · size rules not read)
```

### The list is narrowed, and what leaves it is named

A market answering `cannot size a scrum` is kept off the offered list. It is counted and named on the line under the field, never dropped in silence.

```
TICKER_UNTRADEABLE_FORMAT = (
    "{count} market(s) left off, the smallest order costs more than a scrum's "
    "excess: {names}"
)
```

### Read off both running variants

Both builds were driven with sockets refused and the home directory redirected, at 700, 900 and 1400 pixels wide. This host has no font for the offscreen platform, so the pictures draw boxes; every reading below is the text the widgets themselves report, not the picture.

Before the change, twelve readings of twelve carried three parts and said nothing about trading:

```
A15  (crypto · spot · bot ready)
```

After it, with no connector wired, every row read `size rules not read` in both builds at all three widths.

With a stand-in market table that ccxt's own `parse_spot_market` built from the product records ccxt records for Coinbase, typing `A15` offered three rows in both builds:

```
A15    (crypto · spot · bot ready · can size a scrum)
CBBTC  (crypto · spot · bot ready · size rules not read)
WBTC   (crypto · spot · bot ready · size rules not read)
```

Adding one whole-unit market of the same shape, priced at 67060 dollars with an increment of one unit, moved the reading. The same press offered two rows, and the line under the field named what left:

```
1 market(s) left off, the smallest order costs more than a scrum's excess: WBTC
```

Removing it returned the reading to three rows and an empty line. Both builds read the same at all three widths.

No Coinbase spot product reads `cannot size a scrum` today. Its minimum order cost is one dollar on most products and ten on ether, both under the excess. The answer is for the whole-share brokers, the whole-unit currency dealers and the futures venues the venue compatibility map names, none of which is wired yet.

### The blocks this entry overtakes

Neither is reworded. Each is quoted whole and the true block follows.

`docs/manual/08-tabs/market-inspector.md:4725` -

```
TICKER_OFFER_FORMAT = "{symbol}  ({asset_class})"
```

The format carries four parts now:

```
TICKER_OFFER_FORMAT = "{symbol}  ({asset_class} · {form} · {deploy} · {tradeable})"
```

`docs/manual/08-tabs/market-inspector.md:4739` -

```
def ticker_matches(typed: Any, asset_class: Any) -> list:
```

That function is gone. The rows and the names left off them come back together:

```
def ticker_offers(typed: Any, asset_class: Any, connectors: Any = None) -> TickerOffers:
```

`docs/manual/08-tabs/market-inspector.md:7918` -

```
BIT-25SEP26-CDE  (crypto · future · charts only)
GLD              (commodities · ETF · charts only)
GLD-25DEC26-CDE  (commodities · future · charts only)
XAU/USD          (commodities · spot · charts only, not retail)
AAPL             (stocks · equity · charts only)
EUR/USD          (forex · spot · charts only)
A15              (crypto · spot · bot ready)
```

Every one of those rows now ends with a fourth word, which is whichever of the three answers its market gives.

**Figures.** This entry adds none. A count of the markdown image tags on the page answers 0 before this entry and 0 after it.

## 2026-09-26 - #881 - the sector note names each refused market at the price the scan read

A sector scan ends with one sentence on the Activity Log, and that sentence is the scan's own note. It already said which names no venue lists and which timeframe no venue serves. It now also names each market no built bot variant trades, with that market's own reason. No market leaves the scan: a named market keeps its place in the asset list, charts, and reports.

### The note names the market and its own reason

The reason names the variant the market needs and the market shape that variant absorbs, so a reader sees what is missing rather than a bare refusal.

`src/trading/ata_spm.py` — the sentence one reason writes

```
UNTRADEABLE_TEXT = "Read and not traded, no bot variant trades {symbols}: {reason}"
```

### Two reasons read two sentences

A sector can hold two markets refused for two different reasons. The note groups the markets by reason and writes one sentence per group, each naming the markets that carry it. Before this change the note wrote one sentence and gave every named market the first market's reason, which was wrong for the rest.

`src/trading/ata_spm.py` — what ends one sentence before the next

```
UNTRADEABLE_SEPARATOR = ". "
```

### The price comes from the scan's own candles

Whether the smallest order a venue accepts costs more than a scrum's excess is a question about money, and money needs a price. The scan already read candles for every market it voted on, so each timeframe keeps that market's last close beside the candle count it already keeps. The note reads those closes and asks no venue for anything.

`src/trading/ata_spm.py` — the closes one scan holds

```
def scanned_closes(timeframes: Any) -> dict:
    """Each market's last close across a scan's timeframes, the first timeframe
    that read one answering for it."""
```

The note carries no price for a market no read answered, and that market keeps the answer it had before. The size question stays open rather than guessed.

### The bound on this entry today

The crypto rows this screen lists carry no size rules at all. One function builds every one of them, and it passes none, so the refusal about a smallest order cannot fire on any crypto row the scan walks today. The whole-unit sentence appears once a scanned row carries its venue's own size rules.

`src/trading/ata_asset_maps.py` — the field a crypto row leaves unset

```
    # ata_spm.untradeable_markets reads this; None is a row read off a source
    # publishing no order rules.
    rules: Optional[MarketRules] = None
```

This bound is about the sector scan alone. The ticker field beside it reads its rules off each connector's own loaded market table, which is a different path and already answers all three states.

### What the driven note answered

Three rows drove the real note in one pass: a crypto row as this screen builds it, a whole-unit market priced at 67,060 dollars with an increment of one unit, and an expiring contract. The excess figure is the fleet's largest, 17 dollars 50.

```
crypto row as listed, rules             None
crypto row reason                       '' - no sentence
smallest order on the whole-unit row    $67,060.00 against a $17.50 excess
one refused market, the note reads
  Read and not traded, no bot variant trades WBTC: whole-unit position is
  not built: a market whose smallest order costs more than the excess
two refused markets, the note reads
  Read and not traded, no bot variant trades WBTC: whole-unit position is
  not built: a market whose smallest order costs more than the excess.
  Read and not traded, no bot variant trades BIT-25SEP26-CDE: rolling
  position is not built: a market the venue expires on a date
the same three rows at no price          the whole-unit sentence is gone,
                                         the expiry sentence stays
the crypto row on its own                the note is empty
```

The two readings at the bottom are what tell the price apart from the rest. Take the price away and the whole-unit sentence goes, because that refusal is the only one that needs a price; the expiry refusal reads a date and keeps its sentence. Leave the crypto row on its own and the note is empty, which is the bound above, measured rather than argued.

### The logo one asset draws by

**Functional.** One asset's logo is fetched once, kept on disk, and read off disk every time after. The cache lives beside the program, not in the runtime directory, and it is made the first time a logo is actually kept, so nothing is created on a run that keeps none. An asset no address answers for is remembered by name, so the walk never runs twice for it.

`src/core/asset_logos.py` — the one walk, and what it answers

```
    def resolve(
        self,
        symbol: str,
        candidates: Iterable[str],
        timeout_s: float = LOGO_TIMEOUT_S,
    ) -> LogoAnswer:
        """``symbol``'s kept logo, fetching ``candidates`` in order at most once ever."""
```

**Functional.** A kept body has to be an image, read from the body's own leading bytes rather than from its length. A length alone cannot tell an image from a page: read live once on 2026-09-26, the ticker-keyed crypto icon address answered 6,186 bytes of HTML with no failure code, and a length floor of 100 bytes kept that page as a PNG file. The same 6,186 bytes are refused now, and a body carrying a PNG, JPEG, GIF, ICO, BMP, WebP or SVG signature is kept under the extension its own signature names.

`src/core/asset_logos.py` — what a body must be

```
IMAGE_SIGNATURES: tuple[tuple[bytes, str], ...] = (
    (b"\x89PNG\r\n\x1a\n", "png"),
    (b"\xff\xd8\xff", "jpg"),
    (b"GIF87a", "gif"),
    (b"GIF89a", "gif"),
    (b"\x00\x00\x01\x00", "ico"),
    (b"BM", "bmp"),
)
```

**Functional.** The file name is built from the symbol with every character a file name may not carry replaced. A symbol arrives from outside this repository, so a pair spelled with a slash and a name holding a parent-directory step both stay inside the cache directory: EUR/USD is kept as EUR-USD.png.

`src/core/asset_logos.py` — the file name

```
def kept_name(symbol: str) -> str:
    """``symbol`` as the file name stem a kept logo takes, every other character ``KEPT_NAME_GAP``."""
    text = str(symbol).strip().upper()
    return "".join(
        one if one.isascii() and one.isalnum() else KEPT_NAME_GAP for one in text
    )
```

**Functional.** A crypto asset's addresses come from its own record: the confirmed CoinGecko image address where the record carries one, then the ticker-keyed icon address. That second one needs no lookup and no identifier, so a coin the database has never heard of still resolves an address to try. Thirty rows used to carry a generated CoinGecko address that named image directory 1, Bitcoin's, for every one of them, because the

format string put the coin into the file name and left the directory fixed. Those rows now carry the ticker-keyed address alone.

`src/exchange/crypto_assets.py` — the address a coin nobody entered still has

```
SYMBOL_ICON_URL = "https://www.cryptocompare.com/media/img/cc_icons/{symbol}.png"
```

**Functional.** A stock, a fund share, a metal and a currency pair do not resolve the way a coin does. No coin data source covers them, and three keyless services keyed on a ticker were read and rejected: one wanted a publishable token, one wanted a visible attribution link on every screen drawing a logo, and one was keyed on a company domain instead of on a ticker. The mark is therefore read from the organisation's own domain, at the two standard locations a site serves its own icon at. No third party is contacted, no key is held, and nothing has to appear on the screen.

`src/trading/ata_asset_maps.py` — the two standard locations

```
ORGANISATION_ICON_FORMATS: tuple[str, ...] = (
    "https://{domain}/apple-touch-icon.png",
    "https://{domain}/favicon.ico",
)
```

### The way out to an asset's own organisation

**Functional.** Every address offered as openable is checked for its scheme first, and a refused one is an empty address carrying the reason it is empty. Some schemes open a local file or run script instead of visiting a site, and an address for an asset arrives from outside this repository. The check reads the same allowed set the guarded fetch reads, so the two can never drift apart.

`src/core/safe_url.py` — the check every offered address passes

```
def openable_url(
    url: object,
    allowed_schemes: Optional[Iterable[str]] = None,
) -> tuple[str, str]:
```

Driven over seven addresses, the two allowed ones pass and the five others open nothing.

```
https://bitcoin.org                     opens
http://example.org/logo.png             opens
file:///C:/Windows/System32/calc.exe    the file scheme does not open
javascript:alert(1)                     the javascript scheme does not open
vbscript:msgbox(1)                      the vbscript scheme does not open
data:text/html; base64,PHNjcmlwdD4=      the data scheme does not open
(an empty address)                      no address is known
```

**Functional.** A fund share and a currency pair answer their organisation's own site. Every other US-listed ticker answers the regulator's own company page, which is keyed on the ticker alone and needs no lookup, so an equity the map never carried still answers. A metal quote answers nothing: gold has no issuer.

`src/trading/ata_asset_maps.py` — the regulator route

```
REGISTRY_URL_FORMAT = (
    "https://www.sec.gov/cgi-bin/browse-edgar?action=getcompany"
    "&ticker={ticker}&type=10-K&dateb=&owner=include&count=10"
)
```

### What a logo resolves for, and what it does not

**Functional.** Counted over every row the three non-crypto maps hold, and over the 48 crypto records. A logo address exists for a row when its organisation's domain is known; an organisation address exists for every US-listed ticker.

```
class         rows   a logo address   an organisation address
stocks          47                0                        47
commodities     14               10                        10
forex           28               28                        28
crypto          48               48                        10
```

Four readings, and each one is an absence with its own reason.

```
the 47 stocks rows have no logo address
    a company's own domain is not a fact this repository holds for a ticker,
    and no keyless service keyed on a ticker was accepted
the 4 metal spot pairs have no address of either kind
    a metal quote has no issuer and no company
the 38 crypto records with no organisation address
    the record's website field is filled for 10 of the 48
nothing is fetched until a row asks
    the cache holds 0 files, and no screen asks yet
```

### A stock resolves its own organisation's mark

**Functional.** A stock ticker answers the company's own internet domain, and the company's own site answers the mark. The domain sits beside the fund and central bank domains already held, keyed on the ticker the map uses. Nothing is looked up, no key is held, and nothing has to appear on a screen.

`src/trading/ata_asset_maps.py` — a company's own domain, keyed on its ticker

```
ORGANISATION_SITES: dict[str, str] = {
    "AAPL": "apple.com",
    "MSFT": "microsoft.com",
    "NVDA": "nvidia.com",
    "ROKU": "roku.com",
    "KOSS": "koss.com",
    "SPY": "ssga.com",
}
```

One sentence in the section above is overtaken. Quoted whole:

> *"A stock, a fund share, a metal and a currency pair do not resolve the way a coin does."*

**OVERTAKEN.** A stock and a fund share now resolve the same way a fund share already did: from the organisation's own domain. A metal quote still resolves nothing.

### The third location a mark is served at

**Functional.** Two standard locations are asked first. Where neither answers an image, the organisation's own page is read and the icon that page declares for itself is asked next. That is the third place a browser looks, and it is what resolves a company that publishes a mark without serving either standard path.

`src/core/asset_logos.py` — the icon a page declares for itself

```
def declared_icons(page_url: str, body: bytes) -> tuple[str, ...]:
    """Every icon address ``body``'s own link tags declare, absolute against ``page_url``."""
```

Driven live over six tickers, one read per line, each kept file read back off disk afterwards.

```
ticker  size   domain           reads  what came back
AAPL    large  apple.com          1    a PNG of 4,506 bytes
MSFT    large  microsoft.com      1    a JPEG of 843 bytes
NVDA    large  nvidia.com         2    an ICO of 25,214 bytes
ROKU    mid    roku.com           1    a PNG of 4,342 bytes
GME     mid    gamestop.com       2    nothing: its own certificate chain
KOSS    small  koss.com           3    a PNG of 2,997 bytes
```

Five of the six answered a mark. The first location on `nvidia.com` answered success and was not an image, and the signature check refused it before it could be kept. GME is the one failure: that host's certificate chain does not verify, and no verification was relaxed to reach it.

### What the company register answers

**Functional.** The regulator's own company page carries a field for a company's web address and the field is empty. The register names the company and not its site, so the domain is held here instead of being looked up.

```
data.sec.gov, ticker CIK 0000320193     "website": ""
the same record                         "investorWebsite": ""
the same record                         "name": "Apple Inc."
```

### What resolves per class after the stock rows land

**Functional.** Counted again over every row the three non-crypto maps hold, and over the crypto records.

```
class         rows   a logo address   an organisation address
stocks          47               43                        47
commodities     14               10                        10
forex           28               28                        28
crypto          48               48                        10
```

Two readings in the section above are overtaken. Quoted whole:

> *"the 47 stocks rows have no logo address*
>
> *a company's own domain is not a fact this repository holds for a ticker,*

*and no keyless service keyed on a ticker was accepted"*

**OVERTAKEN.** Forty-three of the 47 stock rows carry a logo address. A company's own domain is now a fact this repository holds, against the ticker, for every company on the list that still trades.

What still resolves nothing, and why each one does not.

```
4 stock rows            the company no longer trades, so it publishes nothing:
                        BBBY, CCIV, EXPR, IPOF
4 metal spot pairs      a metal has no issuer, and no venue quotes these four,
                        so no bot targets one
1 of 43 stock domains   gamestop.com answers, and its certificate chain does
                        not verify
38 of 43 stock domains  the domain is held and was not read live
```

Each of those answers a reason rather than a blank, and a row with no mark draws its symbol exactly as it does today.

Back to the subsystem index.

## 2026-09-26 - #937 - the logos become a library, divided by sector and filled ahead of need

The logos are no longer fetched one at a time when a row happens to need one. They are a library on disk, divided into a folder per asset class and per sector inside it, filled by a step the operator starts himself. Creating a position reads that library and never fetches. An asset the library cannot resolve is written down by name with its own reason, so the gap is countable rather than silent.

The library covers every asset the connected venues list plus every asset the maps hold. Measured on the recorded venue rules: 1,146 Coinbase markets, 467 distinct base assets, 89 non-crypto rows in the maps, and 542 assets once the ten names both sources carry are counted once and the four multiplier perpetuals are folded into the assets they are written on.

### The library, one folder per asset class and sector

A kept mark sits under the class holding the asset and, where the maps or the sector tags name one, under that sector below it. An asset with no sector tag sits in the class folder itself; no sector is guessed for it.

`src/trading/logo_library.py` - the folder one asset is filed under

```
def library_folder(asset_class: Any, sector: Any = "") -> str:
    """``asset_class`` over ``sector`` as a ``kept_folder`` path, the class alone for an untagged asset."""
    return kept_folder(f"{asset_class}/{sector}")
```

The folder name arrives from outside this repository, so every segment is sanitised the way a file name already is. A sector spelled with a space becomes one word joined by a dash, and a parent-directory step cannot name a directory above the library root.

```
commodities/precious metals   ->  commodities/precious-metals
../../etc  with  x/y          ->  etc/x/y
crypto  with no sector tag    ->  crypto
```

The 542 assets fall into 25 folders. The largest is the class folder for crypto assets carrying no sector tag, with 361 of them.

```
crypto                        361      forex/minor                 21
stocks/portfolio               47      crypto/l1                   18
crypto/dex                      9      crypto/defi                  8
crypto/meme                     8      commodities/precious-metals  8
crypto/ai                       7      crypto/l2                    7
forex/major                     7      crypto/gaming                6
crypto/interop                  5      crypto/oracle                5
commodities/energy              4      crypto/depin                 3
crypto/identity                 3      crypto/wallet-infra          3
commodities/industrial-metals   2      crypto/payments              2
crypto/privacy                  2      crypto/stablecoin            2
crypto/storage                  2      crypto/rwa                   1
crypto/wrapped                  1
```

### The fill reads the venues' own listings

The fill takes its asset list from the order rules already recorded for each venue, not from a fresh venue call. Nothing is asked of any exchange or broker while the library fills.

`src/trading/logo_library.py` - where the asset list comes from

```
def venue_bases(document: Optional[dict] = None) -> tuple[str, ...]:
    """Every base asset the recorded venues list, sorted, each folded by ``underlying_base``."""
```

A venue lists some contracts on a multiple of an asset. The base is folded back onto the asset only when the remainder names an asset some source already lists, so no name is shortened on a guess.

```
1000A10   ->  A10      A10 is a recorded base
1000MOG   ->  MOG       MOG is a recorded base
1000PEPE  ->  PEPE      PEPE is a recorded base
1000SHIB  ->  SHIB      SHIB is a recorded base
00        ->  00        nothing is left after the digits
1INCH     ->  1INCH     INCH is not a recorded base
2Z        ->  2Z        Z is not a recorded base
```

### The fill never runs while the fleet trades

Nothing in the running program starts the fill. Measured on this change: no module outside the fill itself names it, against a control where the same search reports five files for the module the fill writes through.

The fill is started by hand, and it refuses outright while the application holds the lock on the runtime directory.

```
python -m src.trading.logo_library --reads 40
python -m src.trading.logo_library --list
```

`src/trading/logo_library.py` - what a refused fill answers

```
TRADING_REFUSAL = (
    "the application is running and holds the instance lock; close it first"
)
```

The fill also states the gap it keeps between two reads, and holds to it. The figure is the one the market scan already uses for a host that publishes no limit, and a logo host publishes none.

```
the gap between two reads                0.5 s
the hold after a rate refusal            60 s
16 reads in one run                      4.50 s of waiting
```

Driven both ways on a scratch runtime directory, never on the operator's own:

```
no lock file present         the fill proceeds
the lock held                refused, 0 reads, 0 kept
the lock released            the fill proceeds again
```

### Stopping the fill and starting it again

A run is bounded by the number of addresses it may read, and the bound is not advice: the store counts every address and refuses a read past the figure rather than taking it. A run that stops leaves the marks it kept on disk and the assets it could not resolve in a file beside them, so the next run reads only what is left.

`src/core/asset_logos.py` - the bound a run cannot exceed

```
class LogoBudgetSpent(RuntimeError):
    """``LogoCache`` was asked for a read and ``read_budget`` leaves none."""
```

Four runs against the real sources, every kept file read back off disk:

```
                                          reads   kept   failed   files on disk
a slice across all four sectors             16      8       2          8
the same slice run a second time             0      0       0          8
a fill stopped partway                       3      3       0         11
the same fill started again                  3      2       1         13
an asset whose source answers nothing        1      0       1         13
```

The second row is what proves the reading is not blind. The first run read sixteen addresses; the same slice run again read none, because every asset was already held. The stopped run kept three of six and the run after it skipped those three and read the three that were left.

Every kept file carries the leading bytes of a real image format. None is empty, and a body that is a web page is refused rather than kept.

```
crypto/l1/A15.png              1,844 bytes   89 50 4E 47
crypto/l1/A14.png              1,270 bytes   89 50 4E 47
crypto/meme/A22.png           4,395 bytes   89 50 4E 47
crypto/oracle/A21.png         1,982 bytes   89 50 4E 47
stocks/portfolio/AAPL.png      4,506 bytes   89 50 4E 47
stocks/portfolio/MSFT.jpg        843 bytes   FF D8 FF E0
stocks/portfolio/NVDA.ico     25,214 bytes   00 00 01 00
```

Reading a mark back by its symbol alone finds it at whatever depth it sits, which is why the bot list and the wizard need no knowledge of the folders.

```
A15   ->  crypto/l1/A15.png
MSFT  ->  stocks/portfolio/MSFT.jpg
NVDA  ->  stocks/portfolio/NVDA.ico
a name no venue lists  ->  nothing
```

OVERTAKEN, the code block under "The logo one asset draws by" earlier on this page, quoted whole:

```
    def resolve(
        self,
        symbol: str,
        candidates: Iterable[str],
        timeout_s: float = LOGO_TIMEOUT_S,
    ) -> LogoAnswer:
        """``symbol``'s kept logo, fetching ``candidates`` in order at most once ever."""
```

The walk now also reads the icon addresses a site declares itself, keeps the file under a folder the caller names, and counts every address against a budget.

`src/core/asset_logos.py` - the walk as it stands

```
    def resolve(
        self,
        symbol: str,
        candidates: Iterable[str],
        timeout_s: float = LOGO_TIMEOUT_S,
        *,
        page_url: str = "",
        no_source_reason: str = "",
        folder: str = "",
    ) -> LogoAnswer:
```

### What the library does not hold yet

The library holds 13 marks. Eight assets are recorded as resolving nothing, and 521 have not been read at all, because this change was bounded to 36 reads in total across every run.

`src/trading/logo_library.py` - where an unresolved asset is written down

```
UNRESOLVED_NAME = "unresolved.json"
```

Each of the eight carries the reason its own source gave:

```
GLD      commodities/precious-metals   spdrgoldshares.com answered HTTP 530 at
                                       both icon locations and at the
                                       favicon-96x96.png its own page declares
EUR/USD  forex/major                   ecb.europa.eu failed certificate
                                       verification at both icon locations and
                                       at its own page; nothing was relaxed
AAVE     crypto/defi                   the ticker-keyed icon address answered
ATOM     crypto/l1                     6,186 bytes of a web page, refused by
A36      crypto/payments                the image check, identically for all six
A17     crypto/rwa                     of these tickers
PEPE     crypto/meme
UNI      crypto/dex
```

The last six matter beyond themselves. Of the 542 assets, 451 hold no address but that one, so the route six of six tickers failed on is the only route those 451 have. The 81 assets whose organisation publishes its own domain resolved three of four tried, and all ten assets carrying a confirmed coin-record address resolved.

```
an organisation's own domain          81 assets    3 of 4 tried resolved
a confirmed coin-record address       10 assets    10 of 10 resolved
the ticker-keyed address only        451 assets    0 of 6 tried resolved
```

Walking the remaining 521 assets would read at most 677 addresses. At the fill's own rate that is about five and a half minutes of waiting plus the time each read itself takes, and on the evidence above it would resolve the organisation and coin-record assets and record the rest by name.

### The mark a new position draws by

Choosing an asset in the wizard used to read the kept file and, finding none, fetch one on the interface thread. It now reads the library and stops there.

`src/gui/bot_wizard.py` - what a wizard row reads

```
#: One reader over the logo library; ``resolve`` is never called, so a wizard row
#: draws the kept file or the lettered disc and never waits on a network read.
KEPT_LOGOS = LogoCache()
```

The flag that used to let a row fetch is gone, and so is the six-line fallback that built an asset database to fetch through. Every one of the six places that draws an asset icon now reads the library.

```
src/gui/bot_wizard.py                          the asset list in the wizard
src/gui/paper/paper_bot_wizard.py              the same, for a paper bot
src/gui/simulator/sim_bot_wizard.py            the same, for a simulated bot
src/gui/widgets/extractor_bot_table.py         the Extractor row
src/gui/paper/paper_extractor_bot_table.py     the same, on paper
src/gui/simulator/sim_extractor_bot_table.py   the same, in the simulator
```

None of the kept images enters this repository. The library directory is ignored, proved after the fill with a control: the same reading reports an unignored file placed beside it. ## 2026-09-27 - #937 - the library is filled

over all 542 assets

The fill was run over every asset the library covers. It resolved 434 marks and named all 108 assets that resolved nothing.

`src/trading/logo_library.py` - what one walk reports

```
542 target(s), 81 already held, 376 read, 352 kept, 109 failed, 0 left, at 0.5 s a read
```

### A crypto mark is looked up, never guessed

Every earlier crypto attempt built an address out of the ticker. That address serves a web page, and the image check refused it 443 times out of 443. A ticker is not a coin id, so the address was a guess.

The fill now takes two steps. One read names every coin and its id; a second read names the picture each id serves.

`src/exchange/crypto_assets.py` - the two addresses a coin picture is looked up through

```
COIN_LIST_URL = "https://api.coingecko.com/api/v3/coins/list"
COIN_MARKETS_URL = (
    "https://api.coingecko.com/api/v3/coins/markets"
    "?vs_currency=usd&per_page={size}&page=1&ids={ids}"
)
```

Five reads answered the whole fleet.

```
453 ticker(s) looked up, 21642 coin(s) listed, 986 record(s) read over 5 read(s),
362 address(es) indexed, 91 unsettled
```

### A ticker several coins carry is not guessed either

`choose_coin` settles a ticker by the recorded name, then by a lone market rank, then by a rank far better than the next. A ticker it cannot settle is written down with its candidates and no address.

`src/trading/logo_library.py` - the margin a rank must clear

```
COIN_RANK_MARGIN = 10.0
```

Three tickers show why the refusal matters. Taking the best-ranked candidate would have drawn the wrong organisation's mark for each of them.

```
TON    Tokamak Network rank 807   against TON Community rank 2787
HOOD   GreenHood rank 478         against Robinhood Markets rank 3209
VELO   Velo rank 325              against Velodrome Finance rank 515
```

### Every mark the library holds now

One folder per asset class, and a sector folder under it.

```
sector          marks       bytes
commodities        10      76,602
crypto            362   6,887,268
currencies         24     231,758
stocks             38     260,135
total             434   7,455,763
```

Every target is settled one way or the other, in all four sectors.

```
sector         targets   kept   unresolved
crypto             453    362           91
stocks              47     38            9
currencies          28     24            4
commodities         14     10            4
total              542    434          108
```

Each of the 434 files was read back off disk. Every one carries the leading bytes of the format its own name gives.

```
faults found on readback          0
marks the window cannot decode    0
formats kept                      311 png, 82 jpg, 37 ico, 4 webp
smallest mark                     660 bytes
largest mark                      175,010 bytes
```

### No mark is a placeholder, and a shared mark has a shared issuer

434 marks carry 395 distinct pictures. Every picture two assets share belongs to one issuer.

```
7 marks   the iShares mark         AGG IEF IWM MCHI SLV TIP TLT
7 marks   the ECB mark             the seven EUR pairs
6 marks   the Bank of England mark the six GBP pairs
5 marks   the USCF mark            BNO CPER UGA UNG USO
5 marks   the ARK Invest mark      ARKF ARKG ARKK ARKW PRNT
5 marks   the RBA mark             the five AUD pairs
```

Three crypto pairs share a picture because the venue spells one project two ways, and both spellings resolved to the same coin.

```
JUP and JUPITER      Jupiter
LIT and LIGHTER      Lighter
ZETA and ZETACHAIN   ZetaChain
```

### Six marks are smaller than the row draws

The row draws a mark at 32 pixels. Six of the 434 offer only 16, so those six draw at their own size in both builds.

```
PALL PPLT   abrdn serves a 16-pixel icon
AMD TDOC UWMC   the same
GOOGL       abc.xyz serves a blank bitmap, and a 16-pixel icon on its own page
```

### What stays unfilled, and why each one does

108 assets resolve no mark. Seven reasons cover all of them.

```
reason                                                        assets
several coins carry the ticker at comparable market rank          67
no coin record carries the ticker, or its name                    22
every address answered something that is not an image              7
the company no longer trades                                       4
a metal quote with no issuer                                       4
several coins carry the ticker and none carries a market rank       2
the domain names a trading venue                                   2
```

The 22 with no coin record are venue index and derivative products rather than coins, so no project publishes a mark for them.

```
BRENTOIL WTIOIL COIN50
CDEAI CDECHN CDECU CDEDEF CDEGLD CDEMC CDENGS CDEOIL CDEPT CDESIL CDETEK CDEUS5
BASED1 BOBBOB CORECHAIN COSMOSDYDX FUN1 GST IP
```

The seven whose address answered a page or a refusal sit on five hosts.

```
amctheatres.com     AMC          HTTP 403 to this platform's user agent
rbnz.govt.nz        4 NZD pairs  HTTP 403 to this platform's user agent
tesla.com           TSLA         HTTP 403 to this platform's user agent
gamestop.com        GME          a certificate this machine cannot verify
```

### A mark is never read from a venue

Two entries in the domain map name a trading venue and a broker.

`src/trading/ata_asset_maps.py` - the domains no logo read reaches

```
VENUE_DOMAINS: frozenset[str] = frozenset({"coinbase.com", "sofi.com"})
```

### The page's own share image is the last location

A site answering one page body at every icon address declares no icon and still declares a share image.

`src/core/asset_logos.py` - the last location a mark is looked for

```
SHARE_IMAGE_TAG = re.compile(
    rb"""<meta\b[^>]*\b(?:property|name)\s*=\s*["']](?:og:image|twitter:image)["']"""
    rb"""[^>]*\bcontent\s*=\s*["']([^"']+)["']""",
    re.IGNORECASE,
)
```

That resolved one asset the five icon addresses could not.

```
www.alibabagroup.com   58,268 bytes of its own page at five icon addresses
BABA.png               15,358 bytes, from the share image the page declares
```

### The fill was stopped and started again

The first run was given 30 reads and stopped at them. The second skipped everything the first had settled.

```
run 1   542 target(s),   5 already held,  30 read,   0 kept,  28 failed, 509 left (stopped)
run 2   542 target(s),  96 already held, 455 read,  13 kept, 433 failed,   0 left
run 3   542 target(s), 542 already held,   0 read,   0 kept,   0 failed,   0 left
run 4   542 target(s),  81 already held, 376 read, 352 kept, 109 failed,   0 left
run 5   542 target(s), 542 already held,   0 read,   0 kept,   0 failed,   0 left
```

96 is the 68 marks then on disk plus the 28 refusals the first run wrote down. The runs that read nothing are the control: they read zero where run 4 read 376.

The fill also refuses outright while the fleet trades, which was driven against the live runtime directory while the application held its lock.

```
refused: the application is running and holds the instance lock; close it first
```

### A kept mark drawn in both builds

Sixteen kept marks spanning all four sectors and all four formats were put through the resolver the bot list uses, then drawn in each build and read back off what each one painted.

```
mark            sector        window decodes   painted colours   page decodes   page draws
A15.png         crypto        yes                          253   50x50          32x32
A25.png         crypto        yes                          119   250x250        32x32
A12.png         crypto        yes                          197   250x250        32x32
WIF.jpg         crypto        yes                          797   250x250        32x32
A16.jpg         crypto        yes                           48   250x250        32x32
BLEND.webp      crypto        yes                           82   250x250        32x32
AAPL.png        stocks        yes                           84   152x152        32x32
GOOGL.jpg       stocks        yes                          632   16x16          16x16
META.webp       stocks        yes                          202   192x192        32x32
NVDA.ico        stocks        yes                          148   48x48          32x32
BABA.png        stocks        yes                          104   600x600        32x32
GLD.png         commodities   yes                          120   180x180        32x32
SLV.ico         commodities   yes                            6   32x32          32x32
EUR-USD.png     currencies    yes                          223   152x152        32x32
USD-CAD.ico     currencies    yes                          110   89x89          32x32
CHF-JPY.png     currencies    yes                           51   180x180        32x32
```

All sixteen decode and draw in both builds. Fifteen draw at the full 32 pixels on the page; GOOGL draws at 16 because its own picture is 16.

`src/gui/main_tabs/bot_status_table_surface.py` - the two figures both builds read

```
LOGO_SIZE_PX = 32
ROW_HEIGHT_PX = LOGO_SIZE_PX + 2 * ROW_LOGO_MARGIN_PX
```

The reading discriminates. Two assets with no kept mark were drawn beside the sixteen: the window gave each a null icon and kept its ticker as text at four painted colours, and the page drew no image for either.

```
TSLA       no mark kept   window icon null, text 'TSLA', 4 colours   page draws no image
VELO/USD   no mark kept   window icon null, text 'VELO', 4 colours   page draws no image
```

### A recorded refusal is not read again

An asset written to the unresolved file is skipped by every later run, including one whose source has since come back. Three central bank domains answered nothing on a first reading and an icon on a second, and they resolved only because their recorded refusals were cleared first.

`src/trading/logo_library.py` - what the walk skips

```
if store.kept_path(row.symbol) is not None or row.symbol in held:
    skipped += 1
    continue
```

### The hosts the fill reads, and nothing else

```
host                          what it answers                  reads
api.coingecko.com             the coin list and the records       11
coin-images.coingecko.com     the coin pictures                  352
www.cryptocompare.com         the ticker-keyed address           443
assets.coingecko.com          ten pictures the database names     10
42 organisation domains       their own icons and pages          206
```

None of the 434 kept images enters this repository. The library directory stays ignored, and the tree is clean after the fill.

# Bot Swarm Tab

Reference. The capital reinforcement network: the nodes, the wires between them, and the accounting that lands routed profit.

## What builds it

`BotSwarmTabMixin._build_bot_swarm_tab` in `src/gui/main_tabs/bot_swarm_tab.py` constructs the visualizer and adds it to the tab row. The tab holds three sub-tabs.

| Sub-tab | State |
|---|---|
| Bot Swarm | The live fleet |
| Simulator Swarm | Rows the Simulator registers while a run plays |
| Paper Swarm | Chrome only — see paper-trader.md |

The builder asks `variant_surface` for the tab class under the screen name `BOT_SWARM`. A build stamped for Qt gets the old widget. Every other build gets the React tab, which holds the whole screen in one web view. React is the running choice.

`src/gui/main_tabs/bot_swarm_tab.py` — `BotSwarmTabMixin._build_bot_swarm_tab`

```
from ..variant_surface import BOT_SWARM, surface_class

built = surface_class(BOT_SWARM)()
```

## The nodes

Each bot draws as one locust card, painted with QPainter. A theme module supplies the palette and a particle module the drifting motes.

`src/gui/visualizer/bot_node.py` — `BotNodeWidget`

```
class BotNodeWidget(QWidget):
    """Paint one bot as a locust, fed by set_bot_data and animate.

    paintEvent draws the wings, abdomen, thorax and head, then the
    symbol, the P/L and the bot_id as text.
    """
```

In the React build the card is drawn by `bot_node.js` from the payload `bot_node_surface` publishes. The tab page carries that module beside the wire canvas, the quick routing matrix and the theme palettes.

`src/gui/react_bot_swarm_tab.py` — `TAB_SCRIPT_ASSETS`

```
TAB_SCRIPT_ASSETS: tuple[str, ...] = (
    "vendor/react.production.min.js",
    "vendor/react-dom.production.min.js",
    "design_tokens.js",
    "shared_widgets.js",
    "header_strip.js",
    "visualizer_themes.js",
    "bot_node.js",
    "wire_canvas.js",
    "quick_routing.js",
    "bot_visualizer.js",
)
```

The grid of locust cards is the whole screen. The wire overlay covers it, because the grid is the only surface whose coordinates a wire can be drawn against.

## Drawing a wire

Drag from one node to another. Wires that arrive on the bus draw through the same handler, which is the path the startup restore and the topology adopt both take, so one subscription covers every source.

`src/gui/bot_visualizer.py` — `_on_external_wire_created`

```
def _on_external_wire_created(self, event) -> None:
    """Add the ``wire.created`` event's wire to ``_wires`` when new.

    A repeated pair updates its pct and adds no second entry.
    """
```

The quick routing matrix is the bulk alternative: tick source bots, tick destination bots, and every pair lands at once. Before that runs, the handler refuses four things and says which.

`src/gui/visualizer/quick_routing.py` — `QuickRoutingMatrix._on_connect_clicked`

```
if not (0.0 <= pct <= 100.0):
    self._reject(f"Rate {pct} is outside 0–100%.")
    return
if not sources:
    self._reject("No SOURCE bots are checked.")
    return
if not dests:
    self._reject("No DESTINATION bots are checked.")
    return
if pct <= 0:
    self._reject(
        "Rate is 0% — that would create wires that route " "nothing."
    )
    return
```

`_confirm_mass` puts a count and a description in front of the operator before a bulk connect or disconnect runs.

## The flow along a wire

One `QTimer` on the Bot Swarm tab drives the whole animation. `BotVisualizationTab._animate` in `src/gui/bot_visualizer.py` runs on every tick: it moves each locust card on, adds the elapsed time to every wire's phase, and asks the wire overlay to repaint. `_WireCanvas.paintEvent` in `src/gui/visualizer/wire_canvas.py` draws the frame, and the travelling dot sits at `pulse_percent(phase)` along the wire.

The timer's interval is `FRAME_INTERVAL_MS` in `src/gui/main_tabs/bot_visualizer_surface.py`, worked out rather than written down: the largest whole number of Windows scheduler ticks that fits inside one frame at `TARGET_FRAMES_PER_S`. A wait is rounded up to the next `SCHEDULER_TICK_MS`, so asking for the 33 ms that `1000 / 30` reads as delivers 46.875 ms — 21 frames a second, not 30. Two ticks is 31.25 ms, so the timer asks for 31 ms and the flow runs at 30 frames a second.

Every frame reports itself. `_WireCanvas.paintEvent` hands its start time and its own duration to `FrameCadence`, and once a `CADENCE_WINDOW_S` window is folded `BotVisualizationTab._report_frame_cadence` emits `swarm.11.003.invariant.frame_cadence`. The record carries the window's frame count, the mean, median, 95th and worst gap between paints, the median and worst paint duration, and how many gaps ran past `FRAME_BUDGET_MS`. A stutter is one long gap among short ones, so `FrameCadence.reading` reports the median and the 95th apart — a mean alone hides it.

The React Bot Swarm page draws no flow. Its `wire-canvas` element is `display: none`, `WireOverlay` in `src/gui/web/bot_visualizer.js` redraws only when the wire list, the opacity or the theme changes, and no caller on the page sends the `animate` action that `apply_action` in `src/gui/main_tabs/bot_visualizer_surface.py` provides. Read on the running page with 38 bots and 33 wires loaded, it calls `requestAnimationFrame` zero times and `setInterval` zero times.

## Routing the profit

`SmartWireManager` in `src/trading/smart_wire.py` owns the topology and one ledger per bot.

| Method | What it does |
|---|---|
| `register_wire` | Adds an edge carrying its own percentage |
| `unregister_wire` | Drops one edge |
| `distribute_fold_profit` | Moves that percentage of realised profit to each wired target |
| `export_wires`, `import_wires` | Carry the topology through a save and a restart |
| `export_ledgers`, `import_ledgers` | Carry the provenance through the same |

Three readings on the per-bot ledger answer where a bot's capital came from and how much of it has aged enough to move on.

| Reading on `BotLedger` | Answers |
|---|---|
| `predominant_source` | Which bot sent the most of what this one holds |
| `mature_profit_total` | Wire profit past its maturity age |
| `mature_profit_available` | How much of that is free to move now |

Maturity is a growth threshold, not an age and not a share. A bot's capital is mature once it is worth more than three times what it started with, and the mature figure is then the whole profit above that starting capital. Below the threshold the figure is zero, so a bot cannot fund a child on a small gain.

`src/trading/smart_wire.py` — `mature_profit_usd`

```
MATURE_GROWTH_PCT: float = 200.0

if value < basis * (1.0 + MATURE_GROWTH_PCT / 100.0):
    return 0.0
return value - basis
```

One function decides maturity everywhere. The per-bot ledger reads it against the seed capital, and the header strip reads it against the exchange cost basis of each bot's holdings, so the spawn gate and the screen cannot disagree about what mature means.

`src/trading/smart_wire.py` — `BotLedger.mature_profit_total`

```
return mature_profit_usd(
    self.starting_balance, self.starting_balance + self.total_profit
)
```

The tab's own row label carries the threshold and reads the same constant the maths reads, so the words on screen and the rule applied to the money move together.

`src/gui/main_tabs/bot_swarm_tab_surface.py` — the row label

```
MATURE_TOTAL_ROW_FORMAT = "Mature profit total (position grown past {pct}%):"

def mature_growth_pct() -> int:
    return int(round(smart_wire.MATURE_GROWTH_PCT))
```

One function bounds what a bot may send away. It reads the target balance, the band edges, the ammunition the next fold needs and the cash on hand, and returns the exportable share as a percentage. A bot never exports the capital it is holding to buy the dip it is waiting for.

`src/trading/smart_wire.py` — `compute_safe_outflow_pct`

```
def compute_safe_outflow_pct(
    scrum_profit_usd: float,
    target_balance_usd: float,
    current_price: float,
    band_lower: float,
    band_upper: float,
    next_fold_ammo_usd: float,
    current_cash_usd: float,
    compound_growth_pct: float,
    retained_this_cycle_usd: float,
) -> float:
```

## Landing the profit

The USD arrives at the destination and spreads evenly across the open fold tranches. Where no tranche stands, it parks instead, and the parked pool drains into the next tranche that opens. Parked credits are routed income, never a tranche.

`src/trading/scrumming/wire_routing.py` — `apply_wire_income`

```
def apply_wire_income(self, usd: float, source: str, ref: str = "") -> dict:
    """Apply incoming Smart Wire USD to this bot's fold queue.
```

Each tranche keeps its wire-credit provenance, held by three methods.

| Method | What it does |
|---|---|
| `_add_wire_credits` | Appends one detail entry per arrival |
| `_compact_wire_credits` | Bounds how many entries a tranche holds |
| `_roll_wire_credit_overflow` | Folds everything past the cap into a rolled total that keeps the sums exact |

## The tranche book

`FoldTrancheAccountingMixin` in `src/trading/scrumming/fold_tranches.py` owns the queued fold tranches, the discharge order the buy path reads, and the lifetime counters. It places no orders.

Four methods carry the fold through, in this order.

| Method | Step |
|---|---|
| `_fold_eligible_tranches` | Selects what may fold at the current price |
| `_fold_discharge_order` | Fixes the order they discharge in |
| `_plan_fold_consumption` | Builds the plan |

| Method | Step |
|---|---|
| `_settle_fold_plan` | Books it |

### Merge, despawn, clear

Three functions collapse or remove a tranche. No other code path does.

**Merge.** `_top_up_remnant_fold_tranches` folds a new sell's USD into part-spent tranches instead of opening more records, and `_bound_new_fold_tranches` merges one sell's fresh slice back to a single record when the fill arrived as many lots. Both reduce the record count and move USD; neither loses any.

**Despawn.** Aged tranches are removed outright. The record leaves the book, the queue total recomputes from what is left, and the count lands in the discarded ledger rather than the closed one, because a closed tranche is one that folded. That split holds one invariant true.

`src/trading/scrumming/fold_tranches.py` — `_despawn_aged_tranches`

```
def _despawn_aged_tranches(self, now: Optional[float] = None) -> dict:
    """Remove tranches older than ``config.tranche_despawn_days``.

    Removing a fold tranche recomputes ``_fold_queue_usd``, the
    derived aggregate behind the tick's ``_fold_queue_usd == 0``
    short-circuit and the panel's "Parked USD", and counts the
    record in ``_tranches_discarded_lifetime``, never in
    ``_tranches_closed_lifetime``, because a closed tranche is one
    that FOLDED. That split keeps
    ``created - closed - discarded == standing`` true.
```

An aged stack tranche still holding a resting limit order is kept and counted separately, since removing it would leave that order on the book with nothing tracking it. The threshold reads the same shared helper the settings spinbox reads, so the sweep and the control cannot disagree about what a stored value means.

The sweep's report now names its counts the way the preview beside it already did, so the two agree on the word as well as on the number. The key set below replaces the one in the block above, and the line the operator reads says removed where it used to say delisted. Nothing outside the sweep read the old names.

```
report = {
    "threshold_days": _days,
    "fold_removed": 0,
    "stack_removed": 0,
    "stack_kept_live_order": 0,
    "ageless_kept": 0,
    "usd_removed": 0.0,
}
```

Two removers touch the stack ledger, not one. The age-driven sweep is one and the operator's own clear is the other, and both move the same discarded counter.

```
def clear_stack_tranches(self, reason: str = "operator") -> dict:
    """Discard this bot's standing Stack tranches, trading nothing."""
```

**Clear.** `clear_fold_tranches` discards every queued tranche and trades nothing. Holdings, the main lots, the target balance and the parked wire credits all survive it, and the report warns when parked credits remain.

## Where the fleet comes from

One file is the only source of the bots, and of their wires, ledgers and tranches with them. `StateManager` in `src/core/state_manager.py` reads and writes it. Two restore methods rebuild the fleet and its topology from it, and one save method writes it back. A restored bot connects to nothing and places no order until the operator starts it.

`src/trading/container/restore.py` — `StateRestoreMixin.restore_bots_from_state`

```
def restore_bots_from_state(self, state: dict) -> list[str]:
    """Recreate every persisted bot in IDLE state; return the ids restored.

    Connects to no exchange and places no order. Each restored bot carries
    a ``_restored`` flag and stays IDLE until the operator starts it, which
    is what triggers the exchange sync before any trading.
    """
```

`restore_smart_wires_from_state` in the same mixin rebuilds the topology. `delete_bot` is the only path that removes a record.

## Capital claims

One reservation per bot stands against a wallet. A total above the wallet figure is refused whole, and the free figure reports what is left. The claims settle against the venue, because the exchange is the authority.

`src/trading/capital_registry.py` — `CapitalRegistry.request_reservation`

```
"""Create or replace ``bot_id``'s ``Reservation`` and return
``(granted, reason, granted_usd)``.

A total above the ``_wallet_usd`` figure is refused whole with
``granted_usd`` 0, and a ``_wallet_usd`` of None grants unchecked.
"""
```

The reservation table reaches the frontend through its own bridge method, named in the table at the foot of this file. The Qt panel beside it, `CapitalRegistryPanel`, is declared and no screen builds it.

### The dollar registry and its table are gone

The paragraph above describes a subsystem the tree no longer holds. Nothing imported the dollar registry, so nothing built one, and the table it fed could draw no row.

Four modules are removed: the registry in `src/trading/capital_registry.py`, the Qt table in `src/gui/widgets/capital_registry_panel.py`, the view model in `src/gui/main_tabs/capital_registry_surface.py`, and the renderer page in `src/gui/web/capital_registry.js`.

The bridge no longer offers the method. `build_registry` in `src/core/desktop_bridge.py` returns 72 methods, and none of them starts with `capital_registry`. The renderer manifest lists 68 modules, and every one has a file on disk.

Claims in asset units are a different mechanism and they stay. `CapitalReservationRegistry` in `src/trading/capital_reservation.py` holds one claim per bot in target-asset units. The saved claim state held 38 of them on 2026-09-13.

## Bridge

Seven methods serve this screen, and the renderer modules carry the matching names.

| Bridge method | Serves |
|---|---|
| `bot_visualizer.state` | The whole tab |
| `bot_swarm_tab.state` | The three sub-tabs and the header row |
| `bot_node.state` | One locust card |
| `wire_canvas.state` | The wire overlay on the grid |
| `quick_routing.state` | The quick routing matrix |
| `capital_registry.rows` | The reservation table |
| `fold_tranches_tab.state` | The tranche book |

The row for `capital_registry.rows` names a method the bridge no longer offers. Six methods serve this screen now. `build_registry` in `src/core/desktop_bridge.py` is the table that decides, and it registers the other six.

## 2026-09-07 15:58 - no issue recorded - ten single-word tabs, each on its own ground

The Bot Swarm is a proprietary capital reinforcement network system that allows profit to be dynamically and strategically routed between positions with the primary intention of this being to accelerate accumulation curves. Smart Wires are dragged between active bots or the quick connection matrix can be used to route multiple streams to different destinations. Each wire can carry a different percentage amount of profit. Profit sent over Smart Wires are registered at the destination as Wire Credits and these are then distributed into standing Fold Tranches which allow them to have a Surplus that will be spent to increase the Target Balance up to the Maximum Growth Per Cycle. Yes, that is probably a mouth full but hopefully the settings and names are, for the most part, self-explanatory.

The tab is now called Swarm. It sits sixth on the bar, on the gold ground with red text.

Each bot draws as one locust card on the grid, and wires drag between nodes.

`src/gui/visualizer/bot_node.py` — `BotNodeWidget`

```
class BotNodeWidget(QWidget):
    """Paint one bot as a locust, fed by set_bot_data and animate.

    paintEvent draws the wings, abdomen, thorax and head, then the
    symbol, the P/L and the bot_id as text.
    """
```

`SmartWireManager` in `src/trading/smart_wire.py` owns the topology and one ledger per bot. One function bounds what a bot may export, reading the target balance, the band edges and the ammunition the next fold needs, which stops a bot sending away the capital it holds for its own dip.

`src/trading/smart_wire.py` — `compute_safe_outflow_pct`

```
def compute_safe_outflow_pct(
    scrum_profit_usd: float,
    target_balance_usd: float,
    current_price: float,
    band_lower: float,
    band_upper: float,
    next_fold_ammo_usd: float,
    current_cash_usd: float,
    compound_growth_pct: float,
    retained_this_cycle_usd: float,
) -> float:
```

At the far end, arriving USD spreads across the open fold tranches, or parks as a pending wire credit while no tranche stands.

`src/trading/scrumming/wire_routing.py` — `apply_wire_income`

```
def apply_wire_income(self, usd: float, source: str, ref: str = "") -> dict:
    """Apply incoming Smart Wire USD to this bot's fold queue.
```

Three functions collapse or remove a tranche and no others: merge, despawn and clear. Despawn removes an aged record outright rather than holding it in a delisted state, and the count lands in the discarded ledger rather than the closed one.

`src/trading/scrumming/fold_tranches.py` — `_despawn_aged_tranches`

```
_days = self._despawn_threshold_days()
report = {
    "threshold_days": _days,
    "fold_delisted": 0,
    "stack_delisted": 0,
    "stack_kept_live_order": 0,
    "ageless_kept": 0,
    "usd_delisted": 0.0,
}
```

The fleet, and its wires, ledgers and tranches with it, comes from one state file alone. A restored bot connects to nothing and places no order until the operator starts it.

`src/trading/container/restore.py` — `StateRestoreMixin.restore_bots_from_state`

```
def restore_bots_from_state(self, state: dict) -> list[str]:
    """Recreate every persisted bot in IDLE state; return the ids restored.
```

Three sub-tabs open the screen: Bot Swarm, Simulator Swarm and Paper Swarm. The header row holds the drag hint, an identifier privacy dot that masks the bot hashes and the symbol labels together, the Privacy Mode button, and three controls. Exchange filters both the swarm and the quick routing scope. Theme lists the four palettes. The Wires slider sets wire opacity from 0 to 100 %.

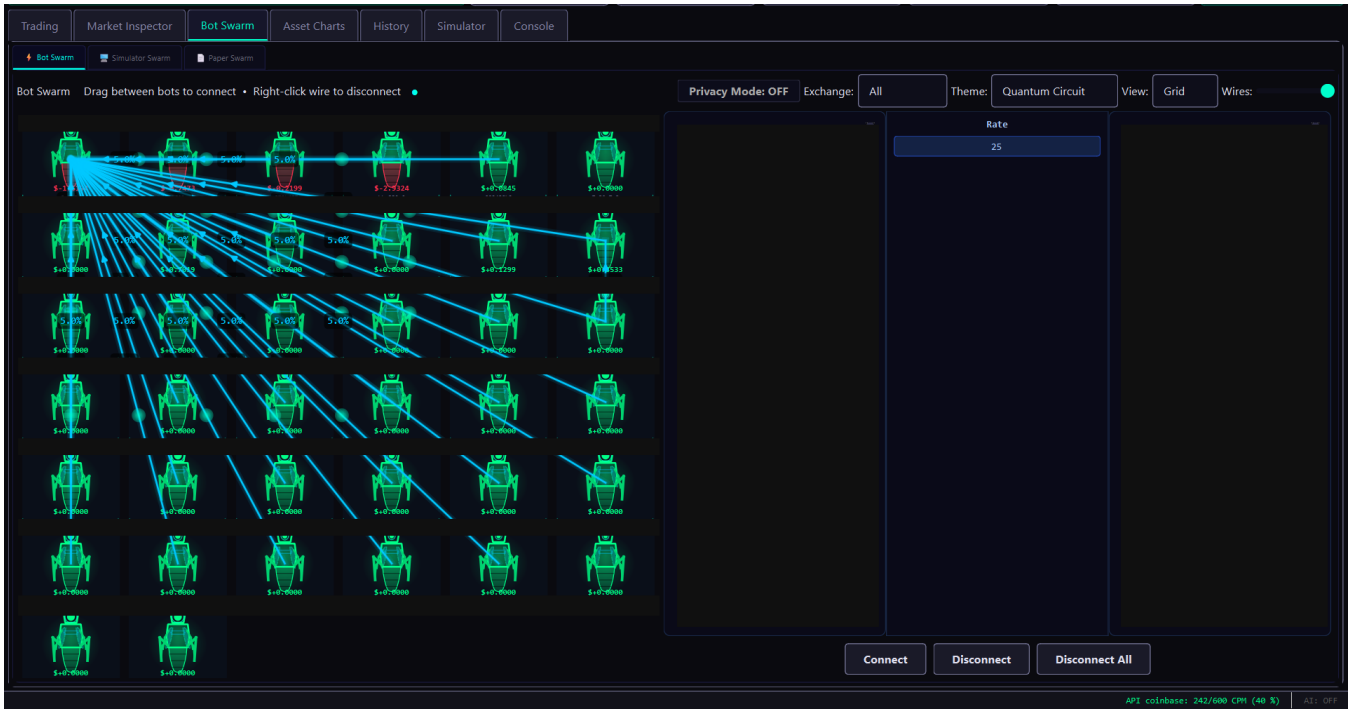
The quick routing matrix fills the right half in three zones: the source list, the Rate field between them, and the destination list. Every checked source wires to every checked destination at that rate, and four conditions are refused out loud before anything is created.

`src/gui/visualizer/quick_routing.py` — `QuickRoutingMatrix._on_connect_clicked`

```
if not (0.0 <= pct <= 100.0):
    self._reject(f"Rate {pct} is outside 0–100%.")
    return
if not sources:
    self._reject("No SOURCE bots are checked.")
    return
if not dests:
    self._reject("No DESTINATION bots are checked.")
    return
if pct <= 0:
    self._reject(
        "Rate is 0% — that would create wires that route " "nothing."
    )
    return
```

A confirmation puts the count in front of the operator before Connect, Disconnect or Disconnect All runs.

(Grid View)




Each card carries the symbol above, the realised profit below it, and the first eight characters of the bot id at the foot. The profit colour is the theme's success colour at zero and above and the error colour below, and the abdomen gradient takes that same colour with its alpha scaled by the size of the figure.

Privacy Mode is off in the figure, so the symbol and the id both draw plain. One field id covers all three readings on a card, the symbol, the short bot id and the tooltip, so a card is never half masked.

`src/gui/visualizer/bot_node.py` — the symbol, drawn through the one mask

```
p.drawText(
    QRectF(0, 2, w, 11),
    Qt.AlignCenter,
    _mask_or(symbol, "bot_swarm.identifiers"),
)
```

Theme names the palette the canvas paints in. The module declares four, and the figure sits on Quantum Circuit. The palette decides the locust body, the success and error colours the profit line takes, and the four corner brackets around each card, which draw in the second accent at low alpha.

`src/gui/visualizer/themes.py` — the palette the figure is on

```
"quantum": {
    "name": "Quantum Circuit",
    "bg": QColor(10, 15, 25),
    "accent": QColor(0, 200, 255),
    "accent2": QColor(0, 255, 200),
    "success": QColor(0, 255, 136),
    "warning": QColor(255, 180, 0),
    "error": QColor(255, 50, 80),
```

The wires cover the grid, because the grid is the only surface whose coordinates a wire can be drawn against. Each wire carries its own percentage, drawn beside it. Drag between two nodes to create a wire, and right-click a wire to remove it.

A wire's percentage sits in a rounded dark badge at the midpoint of the curve, so a wire crossing a card still reads. The Wires slider at the top right sets the opacity of the whole overlay rather than of any one wire.

`src/gui/visualizer/wire_canvas.py` — the badge under the label

```
label = f"{pct}%"
font = QFont("Consolas", 8, QFont.Bold)
p.setFont(font)
fm = p.fontMetrics()
tw = fm.horizontalAdvance(label) + 8
badge = QRectF(label_pt.x() - tw / 2, label_pt.y() - 9, tw, 18)
```

React draws this tab. The choice is made in one place, under the screen name `BOT_SWARM`. A build stamped for Qt gets the old widget. Every other build gets the web one, which holds the tab's page in a single view and answers the calls the main window already made.

`src/gui/main_tabs/bot_swarm_tab.py` — `BotSwarmTabMixin._build_bot_swarm_tab`

```
    def _build_bot_swarm_tab(self) -> None:
        """Build the Bot Swarm tab and add it to the main tab widget."""
        from ..variant_surface import BOT_SWARM, surface_class

        try:
            built = surface_class(BOT_SWARM)()
```

Every header control reports back to Python. The privacy dot, the Privacy Mode button, the Exchange and Theme pickers and the Wires slider each send one action. The screen then redraws from the answer.

`src/gui/react_bot_swarm_tab.py` — `BotSwarmReactTab.take`

```
        def take(self, params: dict) -> dict:
            """Apply one action to the model, push the payload and return it."""
            answer = surface.apply_action(self._state, params)
            self._last_model = answer
            if self._page_ready:
                self._run(push_script(answer))
            return answer
```

A drag between two bots opens the rate box. A drag between two bots already wired opens the disconnect confirmation. A drag released on empty space offers the source bot's outgoing wires. One method decides which of the three it is, so both screens ask the same question in the same words.

`src/gui/main_tabs/bot_visualizer_surface.py` — `BotVisualizerModel.finish_drag`

```
    def finish_drag(self, source_id: str, target_id: str) -> Optional[dict]:
        """Set ``panel`` to what a wire drag from ``source_id`` asks for next.

        ``WireBoard.finish_drag`` decides the outcome; this turns it into
        the rate box, the disconnect confirmation or the wire picker the
        screen shows.
        """
```

### The window a bot row opens

Clicking a bot opens its own settings window. Which window opens depends on the build. The React build draws the header, the tab strip and every tab inside one embedded browser; the Qt build draws the widget tree those tabs describe. The Simulator opens the same window from its own bot rows.

`src/gui/main_window.py` — `_on_bot_clicked`

```
from .variant_surface import BOT_LIVE_SETTINGS, surface_class

_cls = surface_class(BOT_LIVE_SETTINGS)
dlg = _cls(current_bot, self._bot_manager, self)
```

The React side is a subclass, not a rewrite. It replaces only the method that builds the widgets, so an edit still travels the same route into the bot. `_mark_changed` records the field against what the bot holds now, and `_apply_changes` writes it. The window carries no second copy of that logic.

`src/gui/react_bot_live_settings.py` — `BotLiveSettingsReactDialog.apply_edit`

```
field = field_for(str(asked.get("name") or ""))
if not field:
    return
self._mark_changed(field, asked.get("value"))
```

Each tab is drawn by its own renderer module into a named empty space the window leaves for it. `TAB_PLAN` pairs the tab with that space, its module and the surface that builds its payload from the bot. A tab whose surface refuses the bot draws the reason in that space rather than nothing.

`src/gui/react_bot_live_settings.py` — `TAB_PLAN`, the first two rows

```
("Status", "status-page", "acervatorLiveStatus", "live_status_tab_surface"),
(
    "Settings",
    "settings-page",
    "acervatorLiveSettingsTab",
    "live_settings_tab_surface",
),
```

## 2026-09-09 22:12 - #128 - one view, growth stages, and wires that hang

List View is gone. The header row now holds the drag hint, the identifier privacy dot, the Privacy Mode button, Exchange, Theme and the Wires slider. The View picker between Theme and Wires is removed, and the grid of locust cards is the whole left pane.

`src/gui/bot_visualizer.py` — the grid goes straight into the row

```
            inner_hbox = QHBoxLayout()
            # No gap: _quick_routing_matrix takes every pixel the grid leaves.
            inner_hbox.setSpacing(0)
            inner_hbox.addWidget(self._grid_widget)
            inner_hbox.addWidget(self._quick_routing_matrix, stretch=1)
```

### The card is smaller and the grid is wider

A card asks for 88 by 78 pixels, down from 112 by 98. That is 6,864 pixels of area against 10,976, a drop of 37 per cent, and the grid fits eight columns where it fitted six. The three text rows are measured in points rather than in card units, so they survive the smaller card unchanged; the insect body scales by the same ratio as the card.

`src/gui/visualizer/bot_node.py` — the card size

```
CARD_WIDTH_PX = 88
CARD_HEIGHT_PX = 78
```

### Four growth stages, keyed on realised profit

Each card sits in one of four stages. The stage comes from the bot's realised profit measured against its cost basis, both pulled from the exchange, and the 200 per cent floor is the same number the Smart Wire maturity test reads.

| Stage | Realised growth | Dominant colour | Decoration |
|---|---|---|---|
| Hopper | 0 to 99 % | grey and black | wing buds, two tergites |
| Fledgling | 100 to 199 % | red and white | three tergites |
| Immature adult | 200 to 299 % | silver | full grown, four tergites |
| Mature adult | 300 % and over | gold | three crown spines |

The stage names are the published ones. They come from the FAO Desert Locust Guidelines, part one, Biology and behaviour, by Symmons and Cressman, which names the hopper, the fledgling, the immature adult and the mature adult in that order. The module records that source beside the names.

`src/gui/visualizer/growth_stage.py` — the source and the floors

```
STAGE_SOURCE = (
    "FAO Desert Locust Guidelines 1. Biology and behaviour, Symmons and Cressman"
)

HOPPER_FLOOR_PCT = 0.0
FLEDGLING_FLOOR_PCT = 100.0
IMMATURE_FLOOR_PCT = MATURE_GROWTH_PCT
MATURE_FLOOR_PCT = 300.0
```

One function turns the two exchange figures into the percentage. It returns nothing at all when the basis is absent or not above zero, which is how a bot the venue has not answered for is kept out of a stage it has not earned.

`src/gui/visualizer/growth_stage.py` — `realised_growth_pct`

```
    if not math.isfinite(basis) or not math.isfinite(profit) or basis <= 0.0:
        return None
    return profit / basis * PCT_SCALE
```

A second function reads the stats dict the fleet load carries and refuses a bot whose exchange reading is stale. A card with no reading draws as a Hopper and prints an em dash where the percentage goes, never a zero.

`src/gui/visualizer/growth_stage.py` — `growth_pct_from_stats`

```
    if fresh <= 0.0:
        return None
    return realised_growth_pct(stats.get(BASIS_KEY), stats.get(REALISED_KEY))
```

The two figures reach the screen through the bot status snapshot, which now carries them beside the year-to-date sums.

`src/trading/bot_container.py` — `get_status`, the three added stats

```
                "realized_pnl_exchange": round(
                    float(getattr(self.stats, "realized_pnl_exchange", 0.0) or 0.0), 4
                ),
                "cost_basis_total_exchange": round(
                    float(getattr(self.stats, "cost_basis_total_exchange", 0.0) or 0.0),
                    4,
                ),
                "exchange_data_fresh_ts": float(
                    getattr(self.stats, "exchange_data_fresh_ts", 0.0) or 0.0
                ),
```

### Every stage colour is a theme token

No card holds a colour of its own. Each theme carries a body tone and a trim tone for each stage, and the trim is also the colour the growth figure is printed in. The bot id line reads its colour from the same place.

`src/gui/theme_engine.py` — the stage tokens on `ThemeTokens`

```
    locust_hopper_body: str = "#4a4a52"
    locust_hopper_trim: str = "#9a9aa6"
    locust_fledgling_body: str = "#c4303f"
    locust_fledgling_trim: str = "#f2f2f7"
    locust_immature_body: str = "#9ea4ad"
    locust_immature_trim: str = "#e2e6ec"
    locust_mature_body: str = "#b8912c"
    locust_mature_trim: str = "#f0c64a"
    locust_id_text: str = "#8c8ca8"
```

Every text colour on a card was measured against every canvas ground. One hundred and sixty readings were taken, five app themes by four canvas palettes by eight text roles, and all one hundred and sixty clear the WCAG AA floor of 4.5 to 1. On the Quantum Circuit ground the symbol reads 13.35 to 1, the positive P/L 14.30 to 1, the negative P/L 5.32 to 1 and the bot id 5.87 to 1. The four growth figures read 6.89, 17.18, 15.31 and 11.76 to 1.

`src/design_system.py` — the measure used

```
def validate_contrast(fg: str, bg: str, large_text: bool = False) -> tuple[bool, float]:
```

### What the card says without a hover

Four readings, in the order they are drawn. The symbol names the market. The P/L is the realised figure in dollars. The growth figure beneath it is the percentage that placed the card in its stage. The short bot id at the foot separates two bots on one market. A one-letter stage badge sits in the top right corner.

`src/gui/visualizer/bot_node.py` — the growth figure and the badge

```
            p.drawText(
                QRectF(0, h - 31, w, 9),
                Qt.AlignCenter,
                growth_text(growth_pct),
            )
```

The tooltip carries the stage in words and the same percentage, so the colour on the card is never the only place the stage is stated.

### The animation repaints eight times less often

Every coroutine in this application runs on the window's own thread, so a card that repaints on every frame of the 33 millisecond timer spends that thread. A still card now repaints every fourth frame. Measured over 300 frames, both builds repaint a still card 75 times, which is 7.58 times a second. A card holding a trade ring or a speck still repaints on every frame until they fade. Across a fleet of 38 cards that is 288 repaints a second where it was 1,152.

`src/gui/visualizer/bot_node.py` — `animate`

```
            self._since_repaint += dt
            busy = bool(self._trade_pulses or self._particles)
            if not busy and self._since_repaint < REPAINT_INTERVAL_S:
                return False
            self._since_repaint = 0.0
            self.update()
            return True
```

### A wire hangs as a catenary

A wire is no longer a curve bent through a midpoint. It is the curve a cable makes when it hangs between two points, which has a published equation. The wire carries 18 per cent more cable than the straight distance between the two bots, and the two wires of a bidirectional pair hang at different depths.

`src/gui/main_tabs/wire_canvas_surface.py` — `catenary_parameter`

```
    wanted_ratio = math.sqrt(length * length - drop * drop) / span
    ceiling_ratio = math.sinh(CATENARY_U_MAX) / CATENARY_U_MAX
    if wanted_ratio >= ceiling_ratio:
        return None
```

Driven on a 200 pixel span, the sampled curve measures 1.17972 times its chord against the 1.18 asked for, sags 55.03 pixels, and lands exactly on both bots. A wire between two bots in the same column has no span, and falls back to the straight line between them.

`src/gui/main_tabs/wire_canvas_surface.py` — `catenary_curve`, the sample loop

```
    for at in range(count):
        x_at = x_from + span * at / (count - 1)
        points.append([x_at, lift - a * math.cosh((x_at - vertex_x) / a)])
```

### No two flow-rate labels overlap

The percentage badge goes where the curve is flattest, which is the lowest point of the hanging wire. Where that place is already taken, the badge walks outward along the curve and then downward in 20 pixel steps until it is clear.

`src/gui/main_tabs/wire_canvas_surface.py` — `place_badge`

```
    for at in badge_order(points, flattest):
        box = badge_rect(points[at], advance_px)
        if not any(rects_overlap(box, one) for one in placed):
            return box
```

Measured on a grid of 38 bots carrying 152 wires, with the real 33 pixel width of a printed rate label: 114 pairs of badges sit on each other at the flattest point, and none does after the walk. Both numbers come from the same run, so the zero is a result rather than an empty instrument.

One painter each side reads one set of geometry. The Qt sheet imports the curve and the placement from the same module the React payload is built from, so the two cannot drift.

`src/gui/visualizer/wire_canvas.py` — what the Qt sheet imports

```
from ..main_tabs.wire_canvas_surface import (
    arrow_points,
    catenary_curve,
    label_text,
    place_badge,
    point_at,
    pulse_percent,
    slack_for_offset,
)
```

Right-clicking a wire measures against the same curve, so the wire the pointer finds is the wire on the screen.

### Qt against React

Twenty-two measured values were read off the Qt tab and out of the React payload and compared: the card size, the grid columns and spacing, the repaint interval, the theme count, the opacity slider, the absence of a View control, the tooltip, the four stage names, labels, initials and floors, the tergite counts, the crown spines, the wing spreads, the body scales, the stage colour table, the bot id colour and the absent-growth text. Twenty-two of twenty-two matched. The comparison reported one real difference on its first run, a grid of eight columns on the Qt side against six on the React side, which is what it exists to catch.

## 2026-09-09 22:12 - #128 - the Swarm tab loses List View

The Swarm tab no longer offers a choice of view. The grid of locust cards is the whole screen, and the View picker that used to sit between Theme and Wires is gone from both builds. The table below carries the row for every file the change touched; the rows in the table above are the state before it.

| Qt file | React module | Uses React | Bridge | Manifest | Registers in Electron | Ships in the build | RENDERS | Scope |
|---|---|---|---|---|---|---|---|---|
| `src/gui/bot_swarm_list.py` | removed | - | - | - | - | - | - | deleted |
| `src/gui/bot_visualizer.py` | `bot_visualizer.js` | yes | yes | yes | yes | yes | yes | in scope |
| `src/gui/visualizer/bot_node.py` | `bot_node.js` | yes | yes | yes | yes | yes | yes | in scope |
| `src/gui/visualizer/wire_canvas.py` | `wire_canvas.js` | yes | yes | yes | yes | yes | yes | in scope |
| `src/gui/visualizer/growth_stage.py` | shared, no module of its own | - | - | - | - | yes | yes | in scope |
| `src/gui/react_bot_swarm_tab.py` | host for `bot_visualizer.js` | yes | yes | yes | yes | yes | yes | in scope |
| `src/gui/theme_engine.py` | `design_tokens.js` | - | yes | yes | yes | yes | yes | in scope |

The Qt widget file went. `BotListView` and `LaneWireCanvas` had one caller, the Swarm tab, and nothing else built either of them. The live bot settings window draws its own wire table from `QTableWidget`, so it lost nothing.

`src/gui/bot_swarm_list.py` — what was deleted

```
class BotListView(QTableWidget):
class LaneWireCanvas(QWidget):
class BotSwarmLaneAllocator:
```

List View is removed whole. The surface `bot_swarm_list_surface.py` and the module `bot_swarm_list.js` are deleted. The bridge method, the manifest line and the renderer page's script tag that reached them are gone.

`src/gui/web/bot_swarm_tab.js` — the shell that still names the list globals

```
var LIST_API = "acervatorSwarmList";
var LIST_LOADER = "acervatorLoadBotSwarmList";
```

Nothing defines those two globals now. The bot settings shell reads them in four places and guards every read, so the list region draws nothing and raises nothing.

`src/gui/react_bot_swarm_tab.py` — `TAB_SCRIPT_ASSETS`, the tail

```
    "quick_routing.js",
    "bot_visualizer.js",
)
```

One control in `tools/conversion_state.py` named the deleted file. A control pointed at a file that no longer exists reports `born React` rather than `paired`, which reads as a pass and proves nothing, so it now names a Qt file that is still paired.

`tools/conversion_state.py` — `CONTROLS`

```
CONTROLS = ("bot_visualizer", "theme_engine", "design_tokens")
```

Both builds were opened with the change in place. Qt draws eight main tabs and React draws ten, the same counts as before, and the Swarm tab in each carries no View picker, no row list and no lane sheet.

## 2026-09-23 - #239 - Disconnect All clears the key the fleet is stored under

### Every control the tab carries

The tab holds three sub-tabs, six header controls and three routing buttons. A wire also carries two menus of its own.

| Control | What it does | Behind it |
|---|---|---|
| Bot Swarm, Simulator Swarm, Paper Swarm | Picks the layer on show | the sub-tab bar |
| Identifier privacy dot | Masks the bot hashes and the symbol labels together | `_toggle_bot_swarm_identifier_mask` |
| Privacy Mode | Turns every mask on, or every mask off | `_on_privacy_mode_btn_clicked` |

| Control | What it does | Behind it |
|---|---|---|
| Exchange | Filters the swarm and the routing scope | `_on_exchange_filter_changed` |
| Theme | Picks one of the four palettes | `_on_theme_changed` |
| Wires | Sets the overlay opacity, 0 to 100 % | `_on_wire_opacity_changed` |
| Connect | Wires every ticked source to every ticked destination | `connect_clicked` |
| Disconnect | Removes the wire on every ticked pair | `disconnect_clicked` |
| Disconnect All | Removes every wire in the swarm | `disconnect_all_clicked` |
| Right-click a wire | Offers that wire, and both directions when both are wired | `_show_disconnect_menu` |
| Drag onto empty space | Offers the bot's own wires, one entry each | `_show_disconnect_picker` |

Every one of the three routing buttons refuses out loud rather than doing nothing. With no bot ticked, Connect and Disconnect both answer **"No SOURCE bots are checked."**

### What Disconnect All clears

A wire is stored once, at the top level of the fleet file, under `smart_wires`. `StateManager.save_state` writes that list from `SmartWireManager.export_wires`, and `restore_smart_wires_from_state` reads it back at the next launch.

`src/gui/main_tabs/bot_visualizer_surface.py` — the key a wire is stored under

```
WIRES_KEY = "smart_wires"
SOURCE_KEY = "source_id"
TARGET_KEY = "target_id"
```

`clear_all_routes` empties that list and answers the pairs that were in it. Both builds call it: the Qt tab through `_clear_all_routes_in_state`, and the React page through `LiveRoutingTab.clear_all_routes`. Each answered pair becomes one `wire.removed` on the bus, which `BotManager._on_wire_removed_mgr` turns into an `unregister_wire` on the topology's owner. The canvas empties as those messages arrive, and the next launch reads an empty list and paints nothing back.

`src/gui/main_tabs/bot_visualizer_surface.py` — `clear_all_routes`

```
def clear_all_routes(state: dict) -> list:
    """Empty ``WIRES_KEY`` and report the pairs ``stored_wires`` found.

    Each reported pair is one ``wire.removed`` the caller emits.
    """
```

**The sentence this overtakes.** The page carried no sentence naming the key Disconnect All cleared. The older wording that describes the routes a wire is stored as lives on in `apply_routes`, which Connect and Disconnect still use; only the clear and the paint read `smart_wires`.

### The question, on both screens

The Qt build puts the question in a message box. The React page now draws the same two sentences itself, from the words the surface already published to it, and sends the operator's reply back with the press. A press that carries no reply is refused, because a bulk change nobody confirmed must not go through.

`src/gui/main_tabs/quick_routing_surface.py` — `LiveRoutingTab.ask`

```
def ask(self, title: Any, body: Any) -> bool:
    """Take the next reply in ``answers``, or no once they run out."""
```

### The React page takes the wires onto its board

The React host subscribes to `wire.created` and `wire.removed` at build, the two messages every wire reaches a screen as, and retracts both subscriptions when the tab closes. Before that it held none, so its wire board drew nothing however many wires the fleet carried.

`src/gui/react_bot_swarm_tab.py` — `BotSwarmReactTab._subscribe_to_wires`

```
for topic, action in (
    (surface.WIRE_CREATED_TOPIC, "wire_created"),
    (surface.WIRE_REMOVED_TOPIC, "wire_removed"),
):
```

## 2026-09-23 - #239 - every control read on the running program

### What each control does, in both builds

The Qt tab carries **33** controls and the React page carries **21**. Every one was pressed or read on the running program, against a read-only copy of the saved fleet: 38 bots, 33 wires, 51 ledgers.

| Control | What it does | Qt | React |
|---|---|---|---|
| Bot Swarm, Simulator Swarm, Paper Swarm | Picks the layer on show | yes | yes |
| Identifier privacy dot | Masks the bot hashes and the symbol labels together, in the shared register | yes | yes |
| Privacy Mode | Turns every mask in the shared register on, or every one off | yes | yes |
| Exchange | Narrows the swarm and the routing scope to one exchange | yes | yes |
| Theme | Picks one of the four palettes | yes | yes |
| Wires | Sets the overlay opacity, 0 to 100 % | yes | yes |
| Source ticks, Destination ticks | Pick the bots a routing press acts on | yes | yes |
| Rate | The share a new wire carries, 0 to 100 % | yes | yes |
| Connect | Writes one wire per ticked pair into the stored wire list | yes | yes |
| Disconnect | Removes the stored wire on every ticked pair | yes | yes |
| Disconnect All | Empties the stored wire list | yes | yes |

| Control | What it does | Qt | React |
|---|---|---|---|
| Drag between two bots | Asks for a share, then stores that wire | yes | yes |
| Right-click a wire | Removes that wire, and offers both directions when both are wired | yes | **absent** |
| Drag onto empty space | Offers the bot's own wires, one entry each | yes | yes |
| + Add Sim Bot, Run All, Stop All | Adds a Simulator bench row, and flips every row's own button | yes | **draws, acts on nothing** |
| + Add Paper Bot, Start All, Stop All | Adds a Paper bench row, and flips every row's own button | yes | **draws, acts on nothing** |
| A bench row's asset, preset and capital | The values that row carries | yes | **absent** |
| A bench row's Run or Start | Flips that row's own button and status | yes | **absent** |
| A bench row's cross | Drops that row | yes | **absent** |

The three routing buttons refuse out loud rather than doing nothing. Read on the running program: no source answers **"No SOURCE bots are checked."**, no destination answers **"No DESTINATION bots are checked."**, a rate that is not a number answers **"Rate 'abc' is not a number. Enter a percentage between 0 and 100."**, a rate of 0 answers **"Rate is 0% — that would create wires that route nothing."**, and a rate of 140 answers **"Rate 140.0 is outside 0–100%."**

**A** `yes` **in the React column means the control acts when it is pressed.** It does not mean the operator can reach it: only the sub-tab bar and the header strip paint on that page today. The section on how wide the tab has to be carries the measurement.

### The key every connectivity press writes

**The sentence this overtakes**, from the section above:

> *The older wording that describes the routes a wire is stored as lives on in* `apply_routes`*, which Connect and Disconnect still use; only the clear and the paint read* `smart_wires`*.*

**The true sentence.** `apply_routes` now writes `WIRES_KEY`, so every connectivity press reads and writes the one list `stored_wires`, `clear_all_routes` and `hydration_plan` already read. Connect adds a record, a second Connect on the same pair replaces its share rather than adding a second record, and Disconnect drops it. Read on the running program with 33 wires stored: a Connect takes the list to **34** and a Disconnect takes it to **32**.

`src/gui/main_tabs/bot_visualizer_surface.py` — `apply_routes`

```
def apply_routes(state: dict, add: list, remove: list) -> dict:
    """Write each ``add`` pair into ``WIRES_KEY`` and drop each ``remove`` pair.

    ``stored_wires``, ``clear_all_routes`` and ``hydration_plan`` read
    that same list.
    """
```

The Qt tab calls the same function, so the widget and the page store a wire the same way. The right-click menu and the drag picker both land in `remove_wire`, which now drops the pair from the store as well as from the canvas, and the drag that makes a wire stores it when the operator accepts the share.

A direct write is refused when the load it came from carries no fleet, because `_load_bot_state_dict` answers an empty dict when the read fails. Read on the running program: a write from an empty load leaves the file byte-identical and logs the refusal; the same call from a real load moves the file.

### The dot and the Privacy Mode button read one register

`PrivacyMaskRegistry` owns all 19 masks and the Trading tab writes the same one. Both builds now write it and both read it back.

- The dot flips `bot_swarm.identifiers`, the glyph turns from ● to ■, and the routing row labels turn from the symbol and short id to `**** [********]`.
- Privacy Mode sets all 19 masks on, or all 19 off, and its own word follows.
- **The dot also rewrites the Privacy Mode button.** Before this unit the button read `Privacy Mode: OFF` while one mask was on.

`src/gui/main_tabs/bot_visualizer_surface.py` — `any_mask_on`

```
def any_mask_on(self) -> bool:
    """Whether ``mask_registry`` holds any mask on.

    Answers ``masked`` or ``any_masked`` with no registry.
    """
```

### What the page does not carry yet

The page draws the six bench layer buttons and no bench row. Each button carries no handler, so pressing all six leaves the row count at 0. The page has no right-click menu on a wire either: nothing under `src/gui/web/` binds a context menu. The Simulator bench belongs to #117 and the Paper bench to #19; a bench row's own Run and Start buttons flip a label and a status and start no session in either build.

### How wide the Swarm tab has to be

The Qt tab refuses to draw narrower than **1086 px**: the locust grid asks 786 px across its eight columns and the Quick Routing panel asks 278. Asked for 700 or for 900 the tab is 1086 wide both times, so in a pane under 1086 px the Wires slider and the three routing buttons sit past the right edge. At 1400 the whole tab draws — 38 locusts, 33 wires with their share labels, the two tick columns, the Rate box and the three buttons.

The React page fits every width sideways and **none of it downwards**. At 700, 900 and 1400 the body needs no sideways scroll and every header control and routing button reports its right edge inside the viewport. Read down the page instead: `tab-body` is 873 px tall and stacks all three layer panes in the flow — 287 + 293 + 293 — because a pane's own `display: flex` beats the `hidden` attribute. Inside the 287 px that leaves, `header-row` measures 900 px tall over children of 17 to 24 px, so `inner-row` starts at y = 931 with a height of 0, `quick-routing` has a height of 0, a routing button sits at y = 1754, and `wire-canvas` is `display: none`. The captured page shows the tab bar and the header strip and nothing under them. The locust grid, the wires and the routing panel are in the document and are not on the screen.

Back to the subsystem index.

# Asset Charts

Reference. One candlestick panel per traded symbol.

## What builds it

The builder mixin makes the tab and adds it to the row. The tab is a scroll area holding one panel per active bot.

`src/gui/main_tabs/charts_tab.py` — `ChartsTabMixin._build_charts_tab`

```
def _build_charts_tab(self) -> None:
    """Build the Charts tab and add it to the main tab widget."""
    self._charts_tab = TradeChartsTab()
    self._main_tabs.addTab(self._charts_tab, CHARTS_TAB)
```

Issue #450 renamed the tab from Asset Charts to Charts, and `CHARTS_TAB` holds that one word.

One method rebuilds that panel set from the current roster.

`src/gui/widgets/trade_charts_tab.py` — `TradeChartsTab.update_charts`

```
def update_charts(
    self,
    bot_statuses: list[dict],
    bot_manager=None,
    exchange_connectors: dict = None,
) -> None:
    """Create or update chart panels for each bot.
```

Three more methods keep a panel current.

| Method | What it does |
|---|---|
| `_on_tf_changed` | Changes one panel's timeframe, leaving the others alone |
| `log_trade` | Adds a fill marker to the panel that owns the symbol |
| `push_synthetic_candles` | Feeds a panel whose venue returned no history |

## The chart widget

`CandlestickChart` in `src/gui/native_chart.py` paints each panel with QPainter. No browser and no WebEngine dependency.

Four record types carry everything drawn over the candles.

| Record | Draws |
|---|---|
| `Candle` | One open, high, low, close and volume bar |
| `TradeMarker` | An executed buy or sell |
| `PositionMarker` | A holding, with a distinct icon for invisible and visible |
| `GridLine` | A buy or sell level |

The position record is the one that carries the visibility mode, which is what tells an invisible holding from one resting on the book.

`src/gui/native_chart.py` — `PositionMarker`

```
@dataclass
class PositionMarker:
    """Active position shown on chart with visibility mode icon."""

    price: float
    side: str  # "buy" or "sell"
    visibility: str  # "internal" (invisible) or "orderbook" (visible)
    level: int = 0  # Grid level index
    filled: bool = False
    asset_held: float = 0.0
```

The setters name what they place.

| Setter | Places |
|---|---|
| `set_candles` | The candle series |
| `add_marker` | One executed fill |
| `set_trade_history_markers` | The fills already on record |
| `set_positions` | The holdings, invisible and on book |
| `set_grid_lines` | The buy and sell levels |
| `set_tranche_floors` | The standing fold-tranche floors |
| `set_target_balance_lines` | The dollar target |
| `set_fire_armed_state` | An armed manual fire |
| `set_source_label` | The feed the candles arrived on |
| `set_error` | A failure message in place of the panel body |

The mouse handlers give the panel a crosshair with a price and time readout, drag panning and a wheel zoom bounded by `_effective_visible_start` and `_effective_visible_count`.

## Indicators on the chart

The chart calls the engine rather than computing anything of its own. Five series arrive that way, and an entry with no value is skipped when the panel paints.

`src/gui/native_chart.py` — `CandlestickChart._compute_indicators`

```
def _compute_indicators(self):
    """Fill ``_bb_data``, ``_vortex_data``, ``_macd_data``,
    ``_stochrsi_data`` and ``_ichimoku_data`` from the engine
    classes. A ``None`` entry marks a candle with no value, and
    ``paintEvent`` skips it.
    """
```

The toolbar under each panel holds the timeframe picker and eight toggles.

`src/gui/main_tabs/native_chart_surface.py` — `INDICATOR_TOGGLES`

```
INDICATOR_TOGGLES = (
    ("bb", "BB", "#50a0f0"),
    ("vortex", "Vortex", "#00ff88"),
    ("macd", "MACD", "#ffcc00"),
    ("stochrsi", "SRsi", "#ff9060"),
    ("ichimoku", "Ichi", "#c080ff"),
    ("volume", "Vol", "#80ffcc"),
    ("slingshot", "Sling", "#ff4488"),
    ("bbullseye", "BBull", "#ff00aa"),
)
```

Volume starts on and the other seven start off.

`src/gui/main_tabs/native_chart_surface.py` — `INDICATOR_DEFAULTS`

```
INDICATOR_DEFAULTS = {
    "bb": False,
    "vortex": False,
    "macd": False,
    "stochrsi": False,
    "ichimoku": False,
    "volume": True,
    "slingshot": False,
    "bbullseye": False,
}
```

Sling and BBull paint a placeholder shape rather than the indicator, and each says as much in its own tooltip. Issue #430 carries one further disagreement between a tooltip on this toolbar and what the chart paints.

The complete voter set and the maths behind each indicator belong to the Indicator Voting Panel.

## Where the candles come from

Three sources in order: the connected exchange first, a public API second, and the last successful fetch third. Each source that fails is recorded and the next one runs.

`src/exchange/chart_data.py` — `ChartDataFetcher.fetch`

```
# --- Source 1: Exchange OHLCV via CCXT ---
if exchange is not None:
    try:
        candles, source = await self._fetch_exchange(
            exchange, symbol, timeframe, limit
        )
        if candles:
            self._set_cache(symbol, timeframe, candles)
            return candles, source
```

With every source exhausted the fetcher returns an empty list and a label naming what failed, and the panel draws that label rather than an empty grid.

`src/exchange/chart_data.py` — `ChartDataFetcher.fetch`

```
# All sources failed
error_detail = " | ".join(errors) if errors else "No sources available"
```

Each row arrives as an `OHLCVCandle`.

## The other chart

`src/gui/tradingview_chart.py` holds the QWebEngineView chart, which runs HTML and JavaScript inside the Chromium that PySide6 already ships. The Asset Charts tab does not use it; the panels above are painted natively. Its one construction site is the stock window, and the application builds no stock window, so no screen draws it today.

`src/gui/stock_main_window.py` — the one construction site

```
from .tradingview_chart import TradingViewChart

self._chart = TradingViewChart(symbol="AAPL", theme="dark")
```

## Bridge

Three methods serve this screen.

| Bridge method | Renderer module |
|---|---|
| `native_chart.state` | `native_chart.js` |
| `trade_charts_tab.state` | `trade_charts_tab.js` |
| `tradingview.chart` | `tradingview_chart.js` |

## 2026-09-08 13:30 - #407 - a Live / ATA-SMP toggle over the arrows

The tab now walks two lists of markets. Live is the markets the bots are trading. ATA-SMP is the markets the TA engine has called and queued. One selector serves both: the arrows, the ticker menu and the readout all follow whichever list the toggle names.

The toggle is a button on its own row, above the blue arrows and the ticker menu. It reads the list on screen, so the button says Live while the Live list is up and ATA-SMP while the called markets are up.

`src/gui/main_tabs/trade_charts_tab_surface.py` — the two lists and their words

```
LIST_LIVE = "live"
LIST_ATA = "ata_smp"
LIST_MODES = (LIST_LIVE, LIST_ATA)
LIST_LIVE_TEXT = "Live"
LIST_ATA_TEXT = "ATA-SMP"
```

Pressing the toggle swaps the list, points the chart at that list's own selected market, and writes its header. Each list remembers where it was, so coming back to Live returns to the asset that was on screen.

`src/gui/main_tabs/trade_charts_tab_surface.py` — pressing the toggle

```
def toggle_list(self) -> str:
    """Move the arrows to the other list and answer the mode on screen."""
    self.list_mode = LIST_ATA if self.list_mode == LIST_LIVE else LIST_LIVE
    self.calls.append([SELECT_LIST_TOGGLED, self.list_mode, len(self.list_order())])
    self._follow_shown()
    self._label_shown()
    self._feed_shown()
    return self.list_mode
```

A market joins the ATA-SMP list when it reaches the Ready to Send bucket. It does not have to be posted. A call that is queued and then declined is still tracked, because the trigger is the bucket rather than the delivery.

The tab holds no register of its own. It reads the markets through a callable the Market Inspector hands it, so the list the operator walks and the calls phase seven is watching are one set of markets read twice.

`src/gui/main_tabs/market_inspector_tab.py` — where the two are joined

```
def _wire_ata_chart_list(self, inspector: Any) -> None:
    """Point the Charts tab's ATA-SMP list at ``inspector``'s ``PushBoard``.

    The Charts tab is built first, so it reads the board through a
    callable instead of holding a second copy of the markets.
    """
```

No entry is ever invented. With nothing bound, or with an empty bucket, the ticker offers one line reading No called market, the readout says 0 of 0, the arrows go grey, and a note beside the toggle says what the list is waiting for.

`src/gui/main_tabs/trade_charts_tab_surface.py` — the empty state's own words

```
ATA_EMPTY_TICKER_TEXT = "No called market"
ATA_EMPTY_HINT = "A market joins this list when it reaches Ready to Send."
```

An ATA-SMP market has no bot behind it, so the chart draws its candles without the trade markers, the target lines, the tranche floors and the fire glow that a traded asset carries. Nuclear Mode returns the tab to the Live list before it draws, because a Nuclear scenario belongs to a bot.

**Figures.** This page carries no figure. The screen changed on this date and no capture of the toggle row exists yet, so the paragraphs above are the only record of it.

## 2026-09-08 13:30 - #407 - the arrows now reach the renderer

The React half of this tab drew the arrows and the ticker menu, and pressing either one changed nothing. The buttons called the bridge with the step it wanted, and the handler read every other request field and dropped that one, so the answer that came back was the state the tab was already in.

`src/gui/main_tabs/trade_charts_tab_surface.py` — the three the handler now reads

```
        params.get("step_by"),
        params.get("pick_at"),
        params.get("toggle_list", False),
```

The same tab also refused to draw at all under the renderer. One of its own checks read a name the file never declared, which stopped the whole payload before a single row was placed. The check now compares the number of rows the Python side declares against the number the renderer draws, which is the disagreement it was written to catch.

`src/gui/web/trade_charts_tab.js` — the check, repaired

```
// The surface and this module must name the same number of tab rows.
function checkSlots(model) {
  var content = objectField(model, CONTENT);
  if (!owns(content, LAYOUT_SLOTS)) {
    return;
  }
  if (content[LAYOUT_SLOTS] !== LAYOUT_SLOT_COUNT) {
    chartFaults.push(
      fault(CONTENT, LAYOUT_SLOTS, DISAGREES_FAULT, LAYOUT_SLOT_COUNT)
    );
  }
}
```

## 2026-09-13 15:40 - #23 - the Charts tab draws in the desktop window

The tab draws under the window the operator launches. `ChartsTabReact` in `src/gui/react_charts_tab.py` builds one web view and loads one page. The page carries every script and every rule inside it, so it fetches nothing.

`src/gui/react_charts_tab.py` — the modules the page carries, in load order

```
def roster() -> tuple[str, ...]:
    """Every module the page carries, in load order.

    ``STYLE_SOURCE_ASSETS`` leads: ``trade_charts_tab.js`` reads a payload
    colour back to its token through ``shared_widgets.variableFor``.
    """
    return STYLE_SOURCE_ASSETS + CHILD_MODULES + (PANEL_MODULE,)
```

`trade_charts_tab.js` draws the tab. `native_chart.js` draws the candles into the slot that module keeps. Both register with the shell's own panel host, so the page in the window and the page in the shell run the same two modules.

The page reaches the model over one console channel. Each ask leaves the page as a line under `ASK_PREFIX`, and `ChartsTabPage` hands that line to `run_ask`. The answer returns by the ask's own number. The arrows, the ticker menu and the TF picker each move the asset the window holds.

`src/gui/react_charts_tab.py` — the call the dashboard pass makes

```
        def update_charts(
            self,
            bot_statuses: list,
            bot_manager=None,
            exchange_connectors: Optional[dict] = None,
        ) -> None:
            """Rebuild the asset list for one pass of bot statuses and redraw."""
            self._model.update_charts(bot_statuses, bot_manager, exchange_connectors)
            self.redraw()
```

`fetch_chart_data` refetches the asset on screen. `set_ata_source` takes the callable behind the ATA-SMP list. Both answer the calls the main window and the Market Inspector tab already make on the Qt tab.

### The tab's own style sheet

`src/gui/web/trade_charts_tab.css` paints the chrome around the chart. It holds no colour of its own. Every rule names a design token, and `design_tokens.js` writes each token onto the page as a CSS variable.

`src/gui/web/trade_charts_tab.css` — the arrows and the list toggle

```
[data-part="asset-prev"],
[data-part="asset-next"],
[data-part="chart-list-toggle"] {
  background: var(--SURFACE_CONTROL, var(--btn-bg));
  color: var(--PRIMARY, var(--accent));
  border: 1px solid var(--GLOW_PRIMARY_EDGE, var(--border));
  border-radius: 4px;
  font-family: inherit;
  font-size: 12px;
  font-weight: bold;
  cursor: pointer;
}
```

An arrow with one asset to walk goes grey. The panel takes a border and a rounded ground. The header line, the toolbar and the toggle row each take a rule between them. The readout beside the arrows and the empty-list hint take the low text colour at ten pixels.

### The chart keeps a host of its own

`native_chart.js` opens a React root of its own. It draws into `chart-host`, a child element the tab declares and never fills. One element under two roots lost the whole tab the moment the waiting line cleared.

`src/gui/web/trade_charts_tab.js` — the slot the chart draws into

```
    var hostProps = { key: CHART_HOST_PART, style: { flex: AUTO, overflow: HIDDEN } };
    hostProps[PART_ATTR] = CHART_HOST_PART;
    hostProps[BOT_ATTR] = text(props.botId);
```

A missing chart module says so on the slot. `renderChartMounts` asks the panel host to mount the slot with no model, and the host writes the reason it refused where the candles belong.

The tab redraws when its width changes. `resizeEvent` re-arms a short timer, and the timer pushes the payload again. The chart then takes the width the tab has.

### What the page draws

The page draws the Live toggle, the two arrows, the ticker menu and the readout saying which asset of how many. Under them one panel header names the symbol, the price and the bot state. The chart draws the candles, the Bollinger cloud, the Ichimoku cloud, the last price on the right axis, the volume strip and the Vortex, MACD and Stochastic RSI panes. Eight indicator switches close the panel, with Sling and BBull off.

The legend beside the feed name carries its own colour. `native_chart_surface` holds the two Qt style sheets for it, and the tab surface sends them as `legend_styles`.

Back to the subsystem index.

### Where the chart draws

The Charts tab host mounts the chart module inside itself. The tab draws in the window the operator launches, not only in the shell. `CHILD_MODULES` in `react_charts_tab.py` names the one module the tab slot takes.

```
modules the page registered   header_strip, native_chart, trade_charts_tab
panel faults                  none
drawn from forty candles      40 candles, 40 wicks, 40 volume bars
indicator panes               Vortex, MACD, Stochastic RSI
```

With the chart module absent the page still draws its own chrome, and the panel host records the loss rather than drawing an empty slot.

```
native_chart.js registered no panel to draw
```

## 2026-09-15 10:38 - #55 - one painter draws the chart in both builds

The chart area of the Charts tab is the image one painter paints, in both builds. The Qt build's chart widget already painted through that routine; the React build now shows the same image where its own drawing used to be. The ATA-SPM post images come from the same routine, so one chart looks one way in the Qt tab, in the React tab and in a post.

`src/gui/native_chart.py` — the routine both builds and ATA-SPM paint through

```
    def paint_image(
        painter: ChartPainter,
        width_px: int,
        height_px: int,
        device_pixel_ratio: float = 1.0,
    ) -> QImage:
        ratio = float(device_pixel_ratio) if float(device_pixel_ratio) > 0 else 1.0
        image = QImage(
            int(round(int(width_px) * ratio)),
            int(round(int(height_px) * ratio)),
            QImage.Format_ARGB32,
        )
        image.setDevicePixelRatio(ratio)
        image.fill(painter.BG_TOP)
        image_painter = QPainter(image)
        painter.paint_to(image_painter, int(width_px), int(height_px))
        image_painter.end()
        return image
```

Fed the same hundred candles, the same label and the same six overlays, the three agree byte for byte at 1200 pixels wide: the Qt widget, the React image and the ATA-SPM image all read `d8932829dcf760c1`. A planted change to one candle's close moves the Qt widget and the React image to the same new digest, `4dbcb29e360c363e`, so the agreement is a reading and not an artefact. The ATA-SPM image for its own inputs is unchanged by this work.

### How the React page gets the image

The tab holds one painter beside its model. Each time the page draws, the chart slot asks the tab for the image at the slot's own width and height and at the page's device pixel ratio. The tab feeds the painter what the panel holds and answers a PNG as a data URI, which the slot shows as one image element.

`src/gui/react_charts_tab.py` — what the painter is fed, as the Qt tab feeds its chart

```
        def _feed_painter(self) -> None:
            """Give ``painter`` what ``PanelSink`` holds, as the Qt tab gives its chart."""
            panel = self._model.panel
            painter = self._painter
            painter.symbol = str(panel.label)
            painter.set_timeframe(str(panel.chart_timeframe or panel.timeframe))
            painter.set_candles(
```

The image carries the display's pixels. At a device pixel ratio of 1.25 the page asked for 1201 by 731 CSS pixels and received a 1501 by 914 pixel image; at 2x, 600 by 492 CSS pixels received 1200 by 984. In the header band the 2x image holds 395 distinct colours where the 1x image scaled up holds 305, and the thinnest vertical run in the price pane is one device pixel where the scaled 1x image needs two.

The page no longer carries `native_chart.js`. The tab's list of child modules is empty, the page registers two modules, and the file stays in the tree unmounted.

`src/gui/react_charts_tab.py` — the modules the chart slot mounts

```
CHILD_MODULES: tuple[str, ...] = ()
```

### The eight toggles reach the painter

Each check box under the chart now asks the tab to switch that overlay on the painter, and reads the painter's own state back off the answer. The Qt build's boxes drive the same switch on the same painter.

`src/gui/web/trade_charts_tab.js` — a click on one box

```
function overlayChosen(key, on) {
  if (!global.acervator || typeof global.acervator.call !== "function") {
    loadFault = NO_BRIDGE;
    return Promise.resolve(null);
  }
  var params = {};
  params[TOGGLE_KEY_PARAM] = key;
  params[TOGGLE_ON_PARAM] = Boolean(on);
  return global.acervator
    .call(METHOD, params)
```

Each of the eight, flipped from its starting state, moves the image to a new digest in both builds, and the two builds' digests agree on every flip; flipped back, each returns to the starting digest. A toggle left alone answers the last image with no repaint. The starting states are six on and two off: BB, Vortex, MACD, SRsi, Ichi and Vol on; Sling and BBull off.

### What a repaint costs

The tab answers the last image again when nothing it reads has changed. When something has, the painter repaints and the PNG is encoded on the window's own thread. Measured ten runs each, six overlays on, one hundred candles:

| width x height | ratio | image | paint | PNG + base64 |
|---|---|---|---|---|
| 1200 x 492 | 1x | 1200x492 | 7.6 ms | 21.6 ms |
| 1920 x 492 | 1x | 1920x492 | 9.0 ms | 31.5 ms |
| 1200 x 492 | 2x | 2400x984 | 18.0 ms | 76.9 ms |
| 1920 x 492 | 2x | 3840x984 | 22.1 ms | 112.9 ms |

The dashboard pass calls `update_charts` every two seconds, and the header label carries the price, so a moving price repaints on most passes. Candles refetch at most every 30 seconds.

### Live candles reach the React chart

The React tab's fetch handed the fetcher's rows to `list`, and the fetcher's row type is not iterable, so every live fetch ended in an error line and no candles. The rows now pass through `candle_row`, which reads an object, a dict or a six-number row.

`src/gui/main_tabs/trade_charts_tab_surface.py` — the fetch, repaired

```
            if rows:
                self.panel.set_candles([candle_row(one) for one in rows])
                self.panel.set_source(answered)
```

**Figures.** `artifacts/issue-55/` holds the Qt tab, the React tab and the ATA-SPM image before and after, for the same hundred candles. The React page draws the tab header line and the painter draws its own header line under it, so the label reads twice on that build.

## 2026-09-19 08:23 - #55 - every Voting Panel indicator on the chart

His words, 2026-08-21: *"Asset Charts Tab - Restructure - Must now display only one chart at a time with all indicators from the Indicator Voting Panel loaded and properly visualized."* And 2026-09-08: *"Want charts tab to only display one richly detailed and Acervator-annonated chart at a time with all Indicators visible by default excluding that might visually oclude BB or ICHI as Sling or Z-Score Algo Point. All indicators will need toggle check boxes at the bottom."*

### The five series the indicator classes now answer

ADX, Supertrend, Z-Score, KER and RSI each gain one method beside their vote, named as the Vortex and Ichimoku classes name theirs. The vote reads that method, so the line on the chart and the vote in the Voting Panel are one arithmetic.

`src/trading/indicators/adx.py` - the series and the vote reading it

```
    def lines(self, candles: list) -> tuple[_Line, _Line, _Line]:
        """+DI, -DI and ADX, one entry per candle, for a chart to draw.
```

```
        di_plus_series, di_minus_series, adx_series = self.lines(candles)
```

| class | `lines` answers, one entry per candle |
|---|---|
| `ADXIndicator` | +DI, -DI and ADX |
| `SupertrendIndicator` | the line, its side and the ATR under it |
| `ZScoreIndicator` | a `ZScoreBar`: the smoothed score, the two projected prices and the window figures |
| `KaufmanERIndicator` | the ratio |
| `RSIIndicator` | Wilder's RSI |
| `SlingshotIndicator` | a `SlingshotBar`: the squeeze state, the release, the snapback and the momentum |

The vote is unchanged. `compute` from the commit before and `compute` from this one ran over every prefix of 600 recorded A15 1h candles, six classes, every Signal field compared: 0 of 600 prefixes differ for any class. The control: shifting one class's series by one candle makes the same comparison report 549 to 589 differing prefixes.

### Fourteen overlays, seven sub-panes

The overlay registry now holds the twelve Voting Panel indicators in the panel's own order, then the two overlays that draw no voter: the Z-Score algo point and BB Bullseye.

`src/gui/native_chart.py` - one of the six new entries

```
    ChartOverlay(
        key="adx",
        label="ADX",
        colour_field="chart_last_price",
        pane=SUB_PANE,
        occludes=False,
        draw="_draw_adx",
        tooltip="ADX (14) with +DI and -DI, 0..100, 20 and 25 ruled, sub-pane",
        voter="adx",
    ),
```

| overlay | pane | what it paints | ruled |
|---|---|---|---|
| ADX | sub-pane | +DI, -DI and ADX on 0..100 | 20 and 25 |
| STrd | price pane | the Supertrend line, green under price while bullish, red over it while bearish | |
| ZSc | sub-pane | the smoothed score on a symmetric scale of at least 3 | +2 and -2 |
| KER | sub-pane | the Efficiency Ratio on 0..1 | |
| RSI | sub-pane | Wilder's RSI on 0..100 | 30 and 70 |
| ZPt | price pane | the resistance and support prices the averaged reversals project, dashed | |

Slingshot now draws the indicator's own marks. Before, the painter re-derived a bandwidth squeeze of its own and drew marks the indicator never fired; now each diamond is a bar the class read as a squeeze release, signed by its momentum, and each circle a bar it read as a Bollinger snapback.

`src/gui/native_chart.py` - the slingshot marks

```
                if bar.released and bar.momentum != 0.0:
                    marks.append(("release", bar.momentum > 0.0))
                if bar.snapback:
                    marks.append(("snapback", "bull" in bar.snapback))
```

The sub-panes sit under the volume strip in registry order: Vortex, MACD, Stoch RSI, ADX, Z-Score, KER, RSI. Each takes 60 px at the chart's natural height. When the window gives less, every sub-pane shrinks alike, down to 28 px, so the price pane keeps its 220 px and every pane stays on screen.

`src/gui/native_chart.py` - the pane heights

```
SUB_PANE_H = 60
SUB_PANE_MIN_H = 28
PRICE_PANE_LAYOUT_H = 220
PRICE_PANE_MIN_H = 120
```

### The legend

A band under the OHLC row names every overlay with the last candle's value of its series, in the overlay's colour. An overlay that is off reads `off` in grey. The band wraps to the chart's width. It is painted by the painter, so the Qt widget, the React image and the ATA-SPM post images all carry it.

`src/gui/native_chart.py` - what one legend entry carries

```
                found.append(
                    {
                        "key": overlay.key,
                        "label": overlay.label,
                        "text": text,
                        "colour": self.overlay_colour(overlay).name(),
                        "on": on,
                        "has_value": value is not None,
                    }
                )
```

### Fourteen toggles

The toggle row under the chart holds one box per overlay, in the same order. The three occluders start off: Sling, ZPt and BBull, each a mark or a fill over the price pane. Every other indicator starts on.

`src/gui/main_tabs/native_chart_surface.py` - the occluders

```
INDICATOR_OCCLUDES = {
    "bb": False,
    "vortex": False,
    "macd": False,
    "stochrsi": False,
    "ichimoku": False,
    "volume": False,
    "slingshot": True,
    "adx": False,
    "supertrend": False,
    "zscore": False,
    "ker": False,
    "rsi": False,
    "zscore_point": True,
    "bbullseye": True,
}
```

Both hosts flip an overlay through one method on the painter. The Qt check box calls it, and the React box asks the tab, which calls it.

`src/gui/native_chart.py` - the Qt press

```
            self._chart.set_overlay(name, on)
```

### The two emitters

`charts.indicator.toggled` is written once per press, with the key, the state asked and the variant; `ok` is true when the painter's state after the press is the state asked. `charts.indicators.drawn` is written once the candles arrive and the series recompute, with the symbol, the timeframe, the keys shown and each one's legend value; `ok` is true when every shown overlay holds a value on the last candle. Both go through the emitter network's one wire and nothing else.

`src/gui/native_chart.py` - the two names

```
TOGGLED_PIN = "charts.indicator.toggled"

DRAWN_PIN = "charts.indicators.drawn"
```

### Read off the running program

The real window in each build, the home on a scratch directory, every socket but loopback refused, a two-bot fleet read off a copy of the operator's `bot_state.json`, and the real fetcher fed 100 recorded A15 1h candles by a stand-in connector:

| reading | Qt | React |
|---|---|---|
| toggles under the chart, in order | 14: BB, Vortex, MACD, SRsi, Ichi, Vol, Sling, ADX, STrd, ZSc, KER, RSI, ZPt, BBull | the same 14 on the page |
| starting states | 11 on, Sling, ZPt and BBull off | the same |
| legend entries | 14, 11 with a value, 3 reading `off` | the same 14 in the image answer |
| ADX on the last candle, by hand and on the painter | 20.16, +DI 10.91, -DI 40.24 | the same |
| Supertrend | 63,003.61, bearish | the same |
| Z-Score | -1.24 | the same |
| KER | 0.7617 | the same |
| RSI | 25.44 | the same |
| each new toggle pressed off then on | the line gone and back, the image digest moved and returned, one emitter record per press | the same through the page's box |
| sub-pane rules | RSI 30 and 70, ADX 20 and 25, Z-Score +2 and -2: 692 to 727 pixels on each ruled row differ from a repaint with no rules, 0 on the row above and 0 on a control row | the same |
| `bot_state.json` after every press | byte-identical | byte-identical |
| the planted legend value | the legend reads 99.99 where the series reads 25.44, and the comparison reports it | the same |

The two builds paint one image: the painter's image for the same candles reads one digest in both.

### Sentences this entry overtakes

They were not reworded. They are quoted here.

`docs/manual/08-tabs/asset-charts.md:97` - "The chart calls the engine rather than computing anything of its own. Five series arrive that way, and an entry with no value is skipped when the panel paints." Fourteen series arrive that way now.

`docs/manual/08-tabs/asset-charts.md:112` - "The toolbar under each panel holds the timeframe picker and eight toggles." Fourteen toggles.

`docs/manual/08-tabs/asset-charts.md:128` - "Volume starts on and the other seven start off." Eleven start on and three start off.

`docs/manual/08-tabs/asset-charts.md:146` - "Sling and BBull paint a placeholder shape rather than the indicator, and each says as much in its own tooltip." Both paint the indicator, and neither tooltip says placeholder.

`docs/manual/08-tabs/asset-charts.md:552` - "The starting states are six on and two off: BB, Vortex, MACD, SRsi, Ichi and Vol on; Sling and BBull off." Eleven on and three off.

**Figures.** `artifacts/u55/C2/` holds the Qt window and the React page at 1400 by 900 and at 1960 by 1200, the painter's image, and one venue PNG, before and after.

## 2026-09-19 17:40 - #55 - the painter's skin and the Acervator annotations

His words, 2026-08-21: *"Charts did get a slight visual over all during the last past some instances ago but I am still not completely satisfied."* And 2026-09-08: *"I want our chart renderer to be top notch. I want to be able to add modern features and graphics and styling to it so that we can help asset trading migrate away from the pastel and bland."*

### Every colour is a theme token

The painter holds no colour of its own. Each colour it paints is one role in its palette table, and each role names a theme field and an alpha. A theme press on the Theme menu now reaches the painter in both builds: the window's theme switch calls the Charts tab, the tab calls the painter, and the painter resolves every role again. The toggle boxes under the chart and the two position labels above it read their colours from the painter, so they follow the press too. The toggle table in the chart's surface carries a theme field name per entry, not a hex string. A theme that lacks a field keeps the default theme's value for that role and the emitter row names how many roles it did resolve.

`src/gui/native_chart.py` - the roles added this day

```
        PALETTE_ROLES: dict[str, tuple[str, int]] = {
            "PANEL_SOURCE_TEXT": (PANEL_SOURCE_FIELD, 255),
            "TAG_TEXT": ("chart_bg_top", 255),
            "STRIP_FILL": ("chart_band", 40),
            "STRIP_EDGE": ("chart_band", 210),
            "FILL_TAG_SURFACE": ("chart_bg_top", 215),
```

`src/gui/main_window.py` - the one line the theme switch gained

```
                self._charts_tab.set_theme(tm.current)
```

At build the painter starts in the theme the window is in, and the venue image paints in that theme too when its caller names none.

`src/gui/native_chart.py` - the theme in force

```
    def theme_in_force():
        """The ``ThemeTokens`` ``ThemeManager.apply_theme`` last painted."""
        return THEMES.get(applied_theme(), DEFAULT_THEME_TOKENS)
```

### Crisp at every device pixel ratio

The paint pass reads the device pixel ratio of the surface it paints and holds one device pixel as a logical length. The grid, the time ticks and the pane separators draw with anti-aliasing off, one device pixel wide, on coordinates moved to the centre of a device pixel. The wick is a share of one candle column and never under one device pixel; the candle border is one device pixel; the indicator lines keep their logical width and stay anti-aliased. Read on a 1200 pixel image at ratio 1.0 the pens are 0.8, 1.0, 1.2, 1.4, 1.5 and 1.56 logical pixels; at ratio 2.0 the set gains 0.5, the one-device-pixel pen.

`src/gui/native_chart.py` - the device pixel and the snap

```
            ratio = self._device_ratio(p)
            px = 1.0 / ratio

            def snap(value: float) -> float:
                """``value`` moved to the centre of the device pixel it falls in."""
                return (int(value * ratio) + 0.5) / ratio
```

### The type

The header symbol is set in the design system's UI family at the card-title size, the OHLC row in the mono family at the label size, and the legend, the axes, the tags and the caption in the mono family at the caption size. The legend wraps at the chart's width as before; at 1400, 1366 and 1200 pixels it takes two rows and its widest row ends before the price scale.

`src/gui/native_chart.py` - the fonts

```
    def caption_font() -> QFont:
        """The mono font the legend, the axes, the tags and the caption are set in."""
        return design_font(ds.FONT_FAMILY_MONO, CAPTION_FONT_PX)

    def header_font() -> QFont:
        """The UI font the header symbol is set in."""
        return design_font(ds.FONT_FAMILY_UI, HEADER_FONT_PX, bold=True)
```

### Every fill as the two glyphs

A fill is drawn as one of the two hermetic glyphs the Simulator's playback draws, from one definition both read: a sell as `dissolve`, a triangle pointing down as an outline, and a buy as `reform`, a triangle pointing up filled. The definition moved from the Simulator's surface to the chart's surface, and the Simulator imports it from there. The glyph sits on the candle whose interval holds the fill's stamp, one candle column wide and a twentieth of the price pane tall, with a tag naming the role and the price beside it. A fill whose stamp falls before the window or past its end is counted and not drawn.

`src/gui/main_tabs/native_chart_surface.py` - the one definition

```
MARK_GLYPHS = {
    SCRUM_SIDE: {
        "name": DISSOLVE_GLYPH,
        "points": [[-0.5, -0.5], [0.5, -0.5], [0.0, 0.5]],
        "filled": False,
    },
    FOLD_SIDE: {
        "name": REFORM_GLYPH,
        "points": [[-0.5, 0.5], [0.5, 0.5], [0.0, -0.5]],
        "filled": True,
    },
}
```

### The two glyphs are hermetic characters

Two sentences above are overtaken. Each is quoted whole, with the true sentence under it.

Overtaken: *"A fill is drawn as one of the two hermetic glyphs the Simulator's playback draws, from one definition both read: a sell as* `dissolve`*, a triangle pointing down as an outline, and a buy as* `reform`*, a triangle pointing up filled."*

True now: a fill is drawn as one of two hermetic characters the Simulator's playback draws, from one definition both read. A sell draws U+1F761, named ALCHEMICAL SYMBOL FOR DISSOLVE, in gold. A buy draws U+1F75F, named ALCHEMICAL SYMBOL FOR PRECIPITATE, in blue. The alchemical block carries no COAGULATE character, and precipitate is the standard's name for a substance leaving solution and re-forming solid, which is what a fold does.

Overtaken: *"The glyph sits on the candle whose interval holds the fill's stamp, one candle column wide and a twentieth of the price pane tall, with a tag naming the role and the price beside it."*

True now: the character sits on the candle whose interval holds the fill's stamp, set at a twentieth of the price pane in the symbol family, with a tag naming the role and the price beside it. One candle column still sets the mark's footprint for placing that tag. The corner list and the fill flag are gone with the triangles they described, and so is the outline width, because a drawn character takes no outline.

```
# U+1F761 is named ALCHEMICAL SYMBOL FOR DISSOLVE, the scrum's operation.
DISSOLVE_GLYPH = "\U0001f761"
# U+1F75F is named ALCHEMICAL SYMBOL FOR PRECIPITATE, a substance leaving
# solution and re-forming solid, which is the fold's operation.
REFORM_GLYPH = "\U0001f75f"
MARK_GLYPHS = {
    SCRUM_SIDE: {"name": DISSOLVE_GLYPH},
    FOLD_SIDE: {"name": REFORM_GLYPH},
}
MARK_GLYPH_FAMILY = ds.FONT_FAMILY_GLYPH
```

The family resolves to Segoe UI Symbol, and both characters resolve on it. The Simulator's two playback windows draw the same two characters from the same object.

The fills come from two figures of the bot. Each tab subscribes to the bus topic every fill crosses and records it, so a fill made after launch draws on the next status tick. And each standing fold tranche the bot holds was made

at a scrum: its reference price is that sell and its stamp is that sell's time, so the tab draws each one as a scrum glyph. Before this day nothing called the tab's trade recorder, so no fill had ever reached the chart.

`src/gui/widgets/trade_charts_tab.py` - the subscription and the handler

```
            get_event_bus().subscribe(FILLED_TOPIC, self._on_trade_filled)
```

```
        def _on_trade_filled(self, event) -> None:
            """Record one ``trade.filled`` bus event through ``log_trade``.

            The next ``update_charts`` tick draws it as a glyph on its candle.
            """
```

### The target, the floors, the strip, the glow and the position

Each annotation carries a tag at the right edge, a badge filled in the line's colour with the theme's ground colour for text. The target line and the ceiling line read `TARGET` and `CEILING` with their prices. Each tranche floor keeps its dashed line; floors whose tags would overlap share one tag naming their count and their price span, so 16 floors in one pane read as seven tags. The landing strip is a band between the side's Bollinger band and its tolerance edge, over the strip's candles at the bot's own timeframe, tagged with the side and the count; it draws from the bot's own proximity reading. The fire glow keeps its gradient and gains `SCRUM ARMED` or `FOLD ARMED`. The position is the venue's average entry with the units held, drawn as its line, its icon and a `POSITION` tag.

`src/gui/native_chart.py` - the strip's bounds

```
        def _strip_bounds(self) -> Optional[tuple]:
            """The landing strip's two prices, ``(top, bottom)``, or None with no strip set.

            The band lies between the side's Bollinger band and its tolerance
            edge, ``(upper - lower) * tolerance_pct / 100`` inside it.
            """
```

`src/gui/main_tabs/trade_charts_tab_surface.py` - the readers both tabs share

```
def landing_strip(bb: Any, timeframe: Any = "") -> Optional[dict]:
```

```
def position_reading(bot: Any) -> Optional[dict]:
```

```
def tranche_scrums(tranches: Any, bot_id: Any, symbol: Any) -> list:
```

### The theme and annotation emitters

`charts.theme.applied` is written once per theme applied, with the theme, the variant and the symbol; `ok` is true when every palette role resolved from that theme. `charts.annotations.drawn` is written when the set of annotations the paint pass drew changes, with the counts drawn against the counts fed that fall inside the pane, and the fills fed and the fills off the window in its context. Both go through the emitter network's one wire.

`src/gui/native_chart.py` - the two names

```
THEME_PIN = "charts.theme.applied"

ANNOTATIONS_PIN = "charts.annotations.drawn"
```

### The skin readings, off the running program

Both builds, the real window, the home on a scratch directory, every socket but loopback refused, one A15/USD bot's figures off a scratch copy of the operator's fleet file, and 100 recorded A15 1h candles rolled from the 5m tablet by the registry's own rollup:

| reading | Qt | React |
|---|---|---|
| each of the five Theme menu entries pressed | 79 of 79 roles move on the four non-default themes, 0 on the default; the toggle boxes and the two labels restyle | the same 79; the image digest moves on each press and the page's boxes recolour |
| literal colours in the painter and its surface | 3 and 17 before, 0 and 0 after; a literal planted into one draw call reads 1 | the same files |
| two fills handed to the tab's handler, one live-shaped fill on the bus, three standing scrums off the record | 6 fills fed, 2 drawn as triangles on candles 60 and 80 of 100, 4 off the window and counted | the same 6, the same 2 triangles at the same vertices |
| the glyph corners against the Simulator's | one object, `MARK_GLYPHS` | the same object |
| 55 floors off the record | 55 lines fed, 16 inside the pane, 7 tags | the same |
| the target and ceiling prices | equal to the anchor and the cycle ceiling over the units held and the quote rate; outside the pane on these candles | the same |
| the position | the venue's average entry off the record, one `POSITION` tag | the same |
| the strip | the bot's own detector reads none on any window of this tablet; a chosen 3-candle upper strip draws its band between the detector's own band and tolerance edge, tagged `STRIP upper 3c` | the same |
| the glow | `SCRUM ARMED` tagged | the same |
| pens at ratio 1.0 and 2.0 | 0.8 to 1.56 logical, then 0.5 joins at 2.0; anti-aliasing off eight times for the grid and the separators, on again after each | the same |
| a theme with one role | no crash; that role takes the planted colour, every other keeps the default, the emitter row reads 5 of 79 and false | the same |
| the legend at 1400, 1366 and 1200 | two rows, none clipped | the same |
| `charts.theme.applied` | 7 rows, 79 of 79 on each | 7 rows |
| `charts.annotations.drawn` | 2 rows, drawn equal to fed inside the pane | 2 rows |
| `bot_state.json` after every press | byte-identical; a copy with one byte appended reads a different digest | the same |

| reading | Qt | React |
|---|---|---|
| the venue image under Neon Light and Cyberpunk Dark | the top-left pixel equals each theme's ground and the foot pixel its lower ground; the two files differ; the message stamp is drawn in the theme's own axis text colour | one painter |

### Two sentences this entry overtakes

They were not reworded. They are quoted here.

`docs/manual/08-tabs/asset-charts.md:76` - "The setters name what they place." The table under it lacks `set_landing_strip`, which places the landing strip band.

`docs/manual/08-tabs/simulator.md:8634` - "One definition, `MARK_GLYPHS`, names both marks, and both hosts read it". The definition now lives in the chart's surface, and the Charts painter is a third reader.

**Figures.** `artifacts/u55/C3/` holds the Qt window and the React page at 1400 by 900 and at 1960 by 1200, the painter's image at ratio 1.0 and 2.0 with a crop of each, and the venue image under two themes, before and after.

## 2026-09-19 20:10 - #55 - the ATA-SMP list draws the venue image's picture

His words, 2026-09-08: *"After a market qualifies, has its chart rendered with the appropriate confirming indicators and test, and is to be sent to the Ready to Send bucket the Charts tabs will continue to track said market under a ATA-SMP chart list."* And 2026-09-19: *"After its chart renderer is fully upgraded, it will need to be migrated to ATA-SMP."* A called market on the ATA-SMP list now draws the picture its venue images carry: the candles the call was made on, at the call's timeframe, the overlays of its confirming voters and no other, the reversal badge, the reading strip and the caption band. The Live list is unchanged.

### What a called market's chart draws now

The chart takes the call's own candles, so it draws the same bars the venue image drew, and it fetches nothing for that market. The overlays switched on are exactly the ones the venue image switched on, the badge names the same direction, the strip carries the same readings and the foot carries the root image's caption. The bot annotations stay off: no fill glyph, no floor, no target line, no landing strip, no fire glow, no position marker.

`src/gui/widgets/trade_charts_tab.py` - what one call puts on the panel

```
        def _draw_call(self, entry: dict, call) -> None:
            """Draw ``call`` on the panel: its candles at its timeframe, its
            overlays, its badge and its caption, with no bot annotation.

            The overlay set the Live list showed is held in ``_live_overlays``
            until ``_leave_call`` restores it.
            """
            chart = self._panel.chart
            if not self._live_overlays:
                self._live_overlays = [
                    key for key, on in chart.overlays_shown().items() if on
                ]
            self._clear_annotations()
            chart.set_timeframe(str(call.timeframe))
            self._panel.show_only(list(call.overlays))
            chart.set_candles(list(call.candles))
            chart.set_call(call.direction, call.readings)
            chart.set_caption(call.caption)
```

The toggle boxes at the bottom follow the picture. The panel's own `show_only` flips the painter and sets each box to match, with the boxes' signals blocked so no press is counted, and the React page reads the same set off the payload it is drawn from.

### Where the call comes from

Phase three keeps, on the chart it pulled, the candles the images drew and the overlay keys they switched on. The Inspector's sector board turns each pulled call into one record per symbol when the run lands, and the Charts tab reads that record through a second callable beside the one that lists the markets.

`src/trading/ata_spm.py` - one call as the tab draws it

```
@dataclass(frozen=True)
class ChartCall:
    """One call as the Charts tab draws it: the picture its venue images carry.

    ``overlays`` are the keys the images switched on, ``readings`` the
    ``(voter, message)`` pairs ``set_call`` takes, ``caption`` the root
    image's ``post_caption`` and ``candles`` the ``chart_candles`` rows.
    """
```

`src/gui/main_tabs/market_inspector_tab.py` - the second callable

```
        charts.set_ata_source(inspector.watched_markets)
        # MarketInspectorTab keeps its SectorBoard as _ata_board and offers no accessor.
        board = getattr(inspector, "_ata_board", None)
        if board is None or not hasattr(charts, "set_ata_call_source"):
            logger.debug("no SectorBoard reachable; ATA-SMP charts draw no call")
            return
        charts.set_ata_call_source(board.chart_call)
```

The newest call per symbol is the one drawn, which is the vote the list shows. A market on the list with no call held, because no run in this session called it, draws as before: its candles fetched, no badge.

### The Live set held and given back

The overlay set the Live list showed is held the moment a call is drawn and put back when the list returns to Live or moves to a market with no call. The badge, the strip and the caption clear at the same moment.

`src/gui/widgets/trade_charts_tab.py` - leaving a call

```
        def _leave_call(self) -> None:
            """Clear the call's badge and caption and restore the Live overlay set."""
            chart = self._panel.chart
            chart.set_call("", ())
            chart.set_caption("")
            if self._live_overlays:
                self._panel.show_only(self._live_overlays)
                self._live_overlays = []
            if self._showing_ata():
                self._clear_annotations()
```

### The tab's ATA-SMP readings, off the running program, both variants

The real window in each variant, the home on a scratch directory, every socket but loopback refused, a connector shaped like Coinbase Exchange serving the operator's own tablet copies, his scan shape (crypto, nothing typed, 1hr 1d 1wk, target 3), then the Charts tab's toggle pressed:

| reading | Qt | React |
|---|---|---|
| the list after the press | A27 bear 1d, AXS bear 1h | the same |
| the chart | A27, 1d, 213 candles | the same |
| overlays on | adx, ichimoku, ker, macd, slingshot, supertrend, vortex; no other | the same |
| badge and strip | bearish, 7 readings; caption 3 lines | the same |
| floors, target line, glow, strip, position | none | none |
| boxes checked at the bottom | the same seven | the same seven off the page |
| X, Instagram, TikTok images against the chart | overlays equal, badge equal, candles 213 and 213 | the same |
| a market with no voter, planted | 50 candles, no overlay, the badge and nothing else | the same |
| `charts.ata.rendered` rows | one per call drawn, `ok` True | the same |
| the scratch `bot_state.json` | the same bytes before and after every press | the same |

### The rendered emitter

`charts.ata.rendered` writes when a called market's chart draws: the overlays the painter shows against the call's, with the symbol, the timeframe, the list, the widget's size, the direction, the candle count and the variant.

`src/gui/widgets/trade_charts_tab.py` - the row

```
                    _emit(
                        ATA_RENDERED_PIN,
                        actual=shown,
                        expected=sorted(call.overlays),
                        context={
                            "symbol": call.symbol,
                            "timeframe": call.timeframe,
                            "list": LIST_ATA,
                            "width": chart.width(),
                            "height": chart.height(),
                            "direction": call.direction,
                            "candles": len(call.candles),
                            "variant": "qt",
                        },
                    )
```

### The sentence the ATA-SMP picture overtakes

It was not reworded. It is quoted here.

`docs/manual/08-tabs/asset-charts.md:266` - "An ATA-SMP market has no bot behind it, so the chart draws its candles without the trade markers, the target lines, the tranche floors and the fire glow that a traded asset carries." The sentence holds, and holds on screen now: on `5819ced6` the last bot's 55 floors, its target line and its glow stayed drawn when the list moved to a called market, and the candles were fetched. A called market's candles are the call's own, and its picture carries the call's overlays, badge, strip and caption, which the sentence does not name.

**Figures.** `artifacts/u55/C5/` holds the tab's ATA-SMP chart in both variants, the three comparisons side by side and the seven venue images.

## 2026-09-19 20:53 - #55 - the page's chart answers the pointer as the widget does

The React page's chart takes the same five pointer inputs the Qt widget takes, and answers each one the same way to the candle: a hover draws a crosshair with its readout, a wheel tick zooms about the pointer, a drag pans, a double-click fits every candle again, and a leave takes the crosshair off. The mechanism differs by input. A hover repaints nothing: the page draws the crosshair itself on a canvas over the image, from the geometry the painter exports beside every image. A zoom, a pan and a reset move the painter's window and repaint the image, as the widget repaints itself.

### One arithmetic, called by both

The widget's handlers and the page's asks reach the same methods on `ChartPainter`, and those methods call the pure functions `native_chart_surface` already held: `zoom_window`, `pan_start`, `zoom_factor`, `clamp_y_zoom`, `effective_visible_start`, `effective_visible_count` and `fmt_price`. The drag state moved from the Qt widget onto the painter, so both variants keep it in one place.

`src/gui/native_chart.py` - the wheel, on the painter both variants hold

```
        def wheel_turned(
            self, x: int, wheel_delta: float, width_px: int, control_held: bool = False
        ) -> bool:
            if not self._candles:
                return False
            if control_held:
                self._y_zoom_pct = clamp_y_zoom(
                    self._y_zoom_pct * zoom_factor(wheel_delta)
                )
                self._emit_view(
                    VIEW_CAUSE_Y_ZOOM, self._visible_start, self._visible_count
                )
                self._repaint()
                return True
            window = zoom_window(
                wheel_delta,
                int(x),
                int(width_px),
                len(self._candles),
                self._visible_start,
                self._visible_count,
            )
            if window is None:
                return False
            self._visible_start = window["start"]
            self._visible_count = window["count"]
            self._emit_view(VIEW_CAUSE_ZOOM, window["start"], window["count"])
            self._repaint()
            return True
```

`CandlestickChart.wheelEvent` calls `wheel_turned` with the event's x, its angle delta, the widget's width and whether Ctrl is held. `CandlestickChart.mouseMoveEvent` calls `pointer_moved`, which places the crosshair and pans through `pan_to` while a drag is active; `mousePressEvent` and `mouseReleaseEvent` call `pointer_pressed` and `pointer_released`; `mouseDoubleClickEvent` calls `view_reset`; `leaveEvent` calls `pointer_left`. The grip drag along the bottom strip stays on the widget, which is the only variant that draws a grip.

### The page's asks

The page sends each input that moves the window as one ask on the bridge method `trade_charts_tab.view`, with the pointer's x and y, the host's width and height and the page's device pixel ratio. `ChartsTabReact.chart_view` answers it on the tab's painter through the same methods.

`src/gui/react_charts_tab.py` - the ask answered

```
            if action == surface.VIEW_ACTION_WHEEL:
                moved = painter.wheel_turned(
                    x,
                    float(asked.get(surface.VIEW_DELTA_PARAM) or 0.0),
                    width,
                    bool(asked.get(surface.VIEW_CONTROL_PARAM, False)),
                )
            elif action == surface.VIEW_ACTION_PRESS:
                painter.pointer_pressed(x)
            elif action == surface.VIEW_ACTION_DRAG:
                moved = painter.pan_to(x, width)
            elif action == surface.VIEW_ACTION_RELEASE:
                painter.pointer_released()
            elif action == surface.VIEW_ACTION_RESET:
                painter.view_reset()
                moved = True
```

The answer carries the window's start and count and, when the window moved, the repainted image with its geometry, so a zoom costs one round trip. The image key now carries the window and the price padding, so the two-second `update_charts` pass answers the zoomed image again instead of the fitted one. A drag sends one move at a time; a move that arrives while one is in flight waits, and the latest goes when the answer lands.

### The crosshair drawn by the page

Every image answer carries `geometry`: the pane rect, the price scale, the candle column width, the window, and for each visible candle its time label and the six readout lines the painter formats with `readout_lines` - the same function `paint_to` draws the widget's readout from - with the theme's colours as CSS text. The page's `pointermove` handler draws the dotted crosshair, the price badge, the time badge and the readout box on the canvas from those numbers, at the positions `paint_to` uses, and writes what it drew on the chart host as `data-crosshair-x`, `data-crosshair-y`, `data-crosshair-candle` and `data-readout`. A `pointerleave` clears the canvas and the attributes. The price badge at the pointer's row is the one text the page formats itself, from `PRICE_FORMAT_BANDS`, the bands `fmt_price` reads.

`src/gui/main_tabs/native_chart_surface.py` - the readout both draw from

```
def readout_lines(candle: CandleLike) -> list[list[str]]:
    accent = READOUT_ROLE_UP if candle.close >= candle.open else READOUT_ROLE_DOWN
    change = candle.close - candle.open
    return [
        ["O", fmt_price(candle.open), READOUT_ROLE_LIGHT],
        ["H", fmt_price(candle.high), READOUT_ROLE_LIGHT],
        ["L", fmt_price(candle.low), READOUT_ROLE_LIGHT],
        ["C", fmt_price(candle.close), accent],
```

### The two pointer emitters

Two pins through `signal_contract.emit`. `charts.view.changed` writes each time the window moves, with `actual` the window the painter holds, `expected` the window the surface function answered, and the cause: `zoom`, `y_zoom`, `pan` or `reset`. `charts.crosshair.shown` writes at most once per quarter second per painter, with `actual` the candle `candle_at_x` places under the pointer and `expected` the candle the drawing pass placed: the widget's `paint_to` on the Qt build, the page's throttled `crosshair` ask on the React build.

### The five inputs read off the running program

Both variants built the real `MainWindow` with the home on scratch, every socket but loopback refused, one A15/USD bot off a scratch copy of `bot_state.json`, 100 A15 1h candles rolled from a copy of the operator's 5m tablet, at a device pixel ratio of 1.25, the widget and the page's chart host both 1358 pixels wide.

| input | Qt widget | React page |
|---|---|---|
| pointer at x 679 in the price pane | candle 52; readout O 64,634.01, H 64,846.02, L 64,632.71, C 64,741.54, delta +107.53 (+0.17%), V 445 | candle 52; the same six lines to the character |
| one wheel tick in at x 679 | window None, None to 7, 85 | 7, 85; the image digest moved |
| a drag of 120 px left | start 7 to 15 | 7 to 15 |
| a double-click | None, None; padding 1.0 | None, None; padding 1.0 |
| a leave | `_mouse_x` None; the crosshair pixels gone | the canvas holds 0 painted pixels; the attributes gone |
| a pointer in the header band | no readout | no readout, no canvas pixels |

With `ZOOM_IN_FACTOR` set to 0.5 in the React process only, the same wheel tick reads 26, 50 on the page against 7, 85 on the widget, so the equality reading can fail. The scratch `bot_state.json` and the tablet copy read the same digest after every input in both variants.

### What a hover and a zoom cost

Measured on the page at 1.25, the chart host 1358 by 561 CSS pixels: a hover draws in 0.3 to 0.4 ms per move over 60 moves, the widget's repaint reading 13.9 ms per move over the same 60. A wheel tick on the page runs 66 to 83 ms from the ask to the image placed - 14.3 ms to paint, 41.1 ms to encode a 222 KB PNG at 1698 by 701 pixels, the rest transport and the 30 KB geometry - against 13.4 ms on the widget. A hover on the page repaints nothing.

**Figures.** `artifacts/u55/C4/` holds the crosshair and readout in both variants on the same candle, and both after the same wheel tick.

## 2026-09-20 05:40 - #55 - the frame: one control row, compact toggles, the value field on the ATA-SMP picture

His words, 2026-09-20: *"it does not appear that the GUI Archetype organized all of the elements properly using any kind of rules as we still have a significant void space and poorly placed or incorrect icons"*; *"TA values are smashed together in a long row above the chart when they should be stacked in the lower left corner in their own field with a dark background and these only need to be present for ATA-SMP posting"*; *"Check boxes for indicators are over large and taking up too much real estate."* The tab now lays its controls by one rule, in both builds.

### The layout rule

One control row sits at the top of the tab. From the left: the list toggle, the previous arrow, the ticker menu, the next arrow, the `n of N` counter, the empty-list hint when the ATA-SMP list is empty, the `TF:` word and the timeframe menu, then a stretch, then the two legend labels and the source label at the right edge. Every control in the row is `CONTROL_HEIGHT_PX` tall and the gap between two controls is `SELECTOR_SPACING_PX`. The type is the design system's: the buttons and the menus at `TYPE_SMALL` in `FONT_FAMILY_UI`, the counter and the labels at `TYPE_CAPTION`, the two arrow glyphs at `ARROW_GLYPH_PX` in `FONT_FAMILY_GLYPH`. The chart fills the tab's width from the row's bottom edge to the toggle row. The toggle row is one line of fourteen boxes in `CHART_OVERLAYS` order, left-aligned, with no stretch between them; each box is `TOGGLE_BOX_PX` square, under the caption line height, and its label is at `TYPE_CAPTION` in the overlay's colour. The tab draws no legend band. The ATA-SMP picture draws the value field at the lower-left corner of the price pane. Nothing else moves.

`src/gui/main_tabs/native_chart_surface.py` - the numbers both builds read

```
CONTROL_HEIGHT_PX = 26
ARROW_GLYPH_PX = 16
ARROW_GLYPH_FAMILY = ds.FONT_FAMILY_GLYPH
CONTROL_FONT_FAMILY = ds.FONT_FAMILY_UI
CONTROL_FONT_PX = ds.TYPE_SMALL
CAPTION_PX = ds.TYPE_CAPTION
TOGGLE_BOX_PX = 12
```

### Who lays the row

The Qt tab lays one `QHBoxLayout`. `ChartPanel` builds the `TF:` label, the timeframe menu, the two legend labels and the source label as before, and the tab places them in its row through `timeframe_widgets` and `legend_widgets`. The panel keeps the chart and the toggle row.

`src/gui/widgets/trade_charts_tab.py` - the panel's controls join the row

```
            for widget in self._panel.timeframe_widgets():
                widget.setFixedHeight(CONTROL_HEIGHT_PX)
                control_row.addWidget(widget)
            control_row.addStretch()
            for widget in self._panel.legend_widgets():
                widget.setFixedHeight(CONTROL_HEIGHT_PX)
                control_row.addWidget(widget)
            layout.addLayout(control_row)
```

The React page draws the same row in `ControlRow`, from the sizes and the families `selector_values` carries, and the surface's `LAYOUT_SLOTS` reads three: the control row, the chart and the toggle row. The page no longer draws a header line of its own over the image; the painter's header line inside the image is the one both builds show.

### The arrows draw their glyphs

The arrows carry U+25C0 and U+25B6. In the UI family those glyphs are 6 by 7 pixels at 12 pixels, a mark under a quarter of the button's height, which reads as an empty button. The buttons now set `FONT_FAMILY_GLYPH` at `ARROW_GLYPH_PX`, and the glyph reads 12 by 13 pixels in a 34 by 26 button.

`src/gui/design_system.py` - the family for symbol glyphs

```
FONT_FAMILY_GLYPH = "'Segoe UI Symbol', 'DejaVu Sans', 'Apple Symbols', sans-serif"
```

### The toggle boxes

Each Qt box takes its own style sheet: the label in the overlay's colour at the caption size, the indicator `TOGGLE_BOX_PX` square with a one-pixel border. The page sizes each box's input from the same number, carried as `toggle_box_px` in the panel chrome.

`src/gui/native_chart.py` - one box's style sheet

```
TOGGLE_STYLE_FORMAT = (
    "QCheckBox {{ color: {color}; font-size: {font_px}px; spacing: {gap}px; }}"
    "QCheckBox::indicator {{ width: {box}px; height: {box}px; "
    "border-width: 1px; border-radius: 2px; }}"
)
```

### The value field on the ATA-SMP picture

The legend band under the OHLC row is gone from every surface: the OHLC row sits directly over the price pane and the chart's natural height loses the band's rows. The values now draw as a field at the lower-left corner of the price pane, only on a chart that carries a call. Every venue PNG and the tab's ATA-SMP list call `set_call` with a direction; the Live list never does, so the tab shows no field. The field holds one caption-size line per overlay that is on, the label in the overlay's colour and the value in the axis text colour, on the theme's `chart_bg_top` at alpha 205 so the candles under it stay faint, inside a one-pixel border in `chart_grid`, `FIELD_PAD_PX` in from the pane's left edge and bottom edge. Lines past the pane's room are not drawn.

`src/gui/native_chart.py` - what makes a chart the ATA-SMP picture

```
        def draws_value_field(self) -> bool:
            """True when the chart carries a call, which makes it the ATA-SMP picture.

            Every venue PNG and the tab's ATA-SMP list call ``set_call`` with a
            direction; the Live list never does, so the tab draws no field.
            """
            return CALL_DIRECTION_ROLES.get(self._call_direction) is not None
```

A market on both lists keeps one symbol and draws two pictures, the bot's and the call's, so a list press now clears the followed symbol before the chart follows the shown market. Before this, a market that was both traded and called kept the bot's picture on the ATA-SMP list.

### Read off the running program, both builds

Both builds built the real `MainWindow` with the home on scratch, every socket but loopback refused, one A15/USD bot off a scratch copy of `bot_state.json`, 100 A15 1h candles rolled from a copy of the 5m tablet, at the display's ratio of 1.25, the window 1920, 1400 and 900 pixels wide, each under `python -m pdb` with a breakpoint on the first statement of `_draw_legend`.

| reading | before, Qt / React | after, Qt / React |
|---|---|---|
| distinct y of the controls | 8, 42, 78 / 12, 50, 115 | 8 / 12 |
| the chart's top edge against the controls' bottom | 108 against 104 / 136.4 against 134.4 | 42 against 34 / 50.8 against 38 |
| the arrow glyph | Segoe UI 12 px, 59 pixels of ink in the grabbed button / the browser's fallback | Segoe UI Symbol 16 px, 138 pixels of ink / the same family and size on the page |
| the box | indicator 16 by 16 in a 24 px box, label 9 px / input 13 by 13, label 9 px | indicator 12 by 12 in a 14 px box, label 10 px / input 12 by 12, label 10 px; the caption line height 12 |
| `_draw_legend` on the Live tab, the presses and ten repaints | 19 hits / 14 to 16 hits | 0 hits / 0 hits |
| the natural chart height at 1886 wide | 764 / 764 | 732 / 732 |
| the field on one planted call, on the tab | none | 191 by 51, three lines, at the pane's lower-left corner in both builds; gone again on the Live list |
| the field on the six venue sizes | none | 191 by 51 with three voters, 191 by 181 with thirteen overlays on, inside the price pane on every size |

With the row's spacing planted to zero in the driver's process only, the gaps between the controls read 0 in both builds against 8 unplanted, so the reading can fail.

**Figures.** `artifacts/u55/C6/` holds the tab, the arrow buttons, one box, the window and the page at each width before and after, the six venue PNGs with three voters and with all twelve, and the tab on the planted call.

### Five sentences the frame overtakes

"The toggle is a button on its own row, above the blue arrows and the ticker menu." - the toggle is now the first control of the one row.

"A band under the OHLC row names every overlay with the last candle's value of its series, in the overlay's colour. An overlay that is off reads `off` in grey. The band wraps to the chart's width. It is painted by the painter, so the Qt widget, the React image and the ATA-SPM post images all carry it." - no surface carries the band; the ATA-SMP picture carries the field, and an overlay that is off has no line in it.

"The legend wraps at the chart's width as before; at 1400, 1366 and 1200 pixels it takes two rows and its widest row ends before the price scale." - the field does not wrap; each value is one line.

"The React page draws the tab header line and the painter draws its own header line under it, so the label reads twice on that build." - the page draws no header line of its own.

"The header line, the toolbar and the toggle row each take a rule between them." - the page has no toolbar and no header line; the toggle row keeps its rule.

## 2026-09-20 06:10 - #55 - the trade events from the History; the floors gone

His words, 2026-09-20: *"Dotted yellow lines for prior orders are supposed to be gone. Supposed to have hermetic symbols for trade events based on the History."* The chart draws no tranche floor on any surface, and every trade event of the shown market is drawn as its hermetic glyph from the History tab's rows, in both builds.

### The rule

No dashed floor line, no `FLOOR` tag and no `N FLOORS` tag on the Qt widget, the React page's image or any venue PNG. For each row the History tab holds for the shown symbol, one glyph: a sell as `dissolve`, a buy as `reform`, on the candle whose interval holds the row's stamp, tagged with the role and the price, `SCRUM` for a sell and `FOLD` for a buy. A row outside the window is counted and not drawn. The rows are the venue's own fills, and the History tab is the one place that fetches them; the Charts tab never calls a venue. A fill the bus carries after launch draws at once and is not doubled when the History's next fetch carries it. The ATA-SMP picture and every venue PNG carry no glyph.

### Where the trade events come from

The History tab holds the venue's fills in `_all_trades`, the time of its last fetch in `_last_fetched_ts` and whether a fetch is in flight in `_fetch_in_flight`; see the History tab. The window builds it after the Charts tab, so the Charts tab reads it at each `update_charts` tick under the name the window gives it, `_history_tab`. One surface reader takes the three names, one reader turns the rows of the shown symbol into fills, and one join keeps each bus fill the venue has not answered for.

`src/gui/main_tabs/trade_charts_tab_surface.py` - the rows to fills

```
def history_fills(rows: Any, symbol: Any) -> list:
    """One fill per History row of ``symbol``: the venue's id, side, price and stamp, the role from the side.

    A row whose side is neither buy nor sell, or whose price or stamp is
    not positive, gives no fill.
    """
```

`src/gui/main_tabs/trade_charts_tab_surface.py` - the join

```
def merged_fills(venue: Any, logged: Any) -> list:
    """``venue`` fills, then each ``logged`` fill no venue fill names.

    A venue fill names a logged fill when both carry the same venue id, or
    when their prices sit within ``FILL_MATCH_PRICE`` and their stamps
    within ``FILL_MATCH_WINDOW_S`` of each other.
    """
```

The bus fill carries no venue id today, so every join is by stamp and price. A fill the venue answers as several rows at several prices keeps its bus glyph beside them until the next launch.

### The one ask, and the landing

When the History has never fetched and no fetch is in flight, the Charts tab asks `refresh()` once in its life, the same call the window makes when the History tab is activated. The Charts tab also connects the History's `history_refreshed` signal once, so the rows landing redraw the shown market without waiting for the two-second tick. The window's own rule keeps the History fresh after that.

`src/gui/main_tabs/trade_charts_tab_surface.py` - the one time the tab asks

```
def history_never_fetched(state: dict) -> bool:
    """True when the History has never landed a fetch and none is in flight: the one time the Charts tab asks.
"""
    return state["fetched_ts"] == NEVER_FETCHED_TS and not state["in_flight"]
```

`src/gui/widgets/trade_charts_tab.py` - the Qt tab's draw

```
        def _draw_history(self) -> None:
            """Put the shown market's trade events on the chart: the History's fills
            for its symbol, then each bus fill since launch the venue has not
            answered for.

            A History that has never fetched is asked to ``refresh`` once, the
            call the window makes on activation; ``history_refreshed`` is
            connected once so the landing redraws without waiting for the tick.
            A called market on the ATA-SMP list gets none.
            """
```

The React widget hands the same three names to its model through `set_history`, and the model's `feed_history` runs in `update_charts` before the bot's own annotations. Both builds read the same glyph set on the same candles from one painter.

### The floors gone

`ChartPainter` no longer holds `_tranche_floors`, `set_tranche_floors` or `_draw_floors`; `FLOOR_LINE` is no longer a palette role; the two floor tag formats are gone. The Qt tab no longer reads the bot's lots, the React model no longer carries floors, and the page no longer stamps a floor count. The annotation emitter `charts.annotations.drawn` names fills, target lines, the strip, the glow and positions, and no floors; it now also fires when the fed count or the off-window count moves, so a row landing outside the window writes a row. The standing scrums off the bot's fold tranches are no longer a source of markers: the History's rows are the venue's answer for the same sells.

### The History readings, off the running program, both builds

Both builds, the real window, the home on a scratch directory, every socket but loopback refused, a two-bot fleet off a scratch copy of one seed record, 100 recorded A15 1h candles rolled from the 5m tablet by the registry's own rollup and shifted by whole hours so the newest candle opens at the current hour, and a History venue on loopback answering `get_my_trades` with 40 A15/USD rows (26 on distinct candles inside the window, 14 older than it) and 6 A14/USD rows, through the History tab's own fetch path:

| reading | Qt | React |
|---|---|---|
| `_draw_floors` and `set_tranche_floors` on the painter | absent; 0 floors held; the emitter names no floor | the same |

| reading | Qt | React |
|---|---|---|
| the History at the tab's first tick | never fetched; the tab asks once; the venue is asked once per market, 2 of 2; 46 rows land in 0.21 s | the same |
| the shown market's rows against its glyphs | 40 rows for A15/USD, 40 markers held, 26 drawn on 26 candles, 14 counted off the window | the same 40, 26 and 14 |
| a six-notch zoom to 36 candles, then a reset | 5 drawn and 35 off; back to 26 and 14 | the same |
| one bus fill on the newest candle | drawn at once, 27 of 41; the History's next fetch carrying it leaves 27 of 41 | the same |
| a row planted inside the window, then one outside | 28 drawn of 42; then 28 drawn of 43 with 15 off | the same |
| A14/USD shown, then A15/USD again | 6 rows, 4 drawn, 2 off; back to 28 of 43 | the same |
| the History tab after these fetches | 49 of 49 trades shown, page 1 of 1 | the same |
| the venue image with a call | no glyph, no floor | one painter |

**Figures.** `artifacts/u55/C7/` holds the tab, the chart and the window at 1920 and 1400 in both builds, before and after.

### Six sentences the History glyphs overtake

They were not reworded. They are quoted here.

"`set_tranche_floors` | The standing fold-tranche floors" - the setter table row; the setter no longer exists and nothing places a floor.

"The fills come from two figures of the bot. Each tab subscribes to the bus topic every fill crosses and records it, so a fill made after launch draws on the next status tick. And each standing fold tranche the bot holds was made at a scrum: its reference price is that sell and its stamp is that sell's time, so the tab draws each one as a scrum glyph." - the fills come from the History tab's rows for the shown symbol, then the bus fills since launch the venue has not answered for; the fold tranches are no longer a source.

"Each tranche floor keeps its dashed line; floors whose tags would overlap share one tag naming their count and their price span, so 16 floors in one pane read as seven tags." - no floor draws on any surface.

"`charts.annotations.drawn` is written when the set of annotations the paint pass drew changes, with the counts drawn against the counts fed that fall inside the pane, and the fills fed and the fills off the window in its context." - it is also written when the fills fed or the fills off the window change, and it names no floor.

"55 floors off the record | 55 lines fed, 16 inside the pane, 7 tags | the same" - a reading of the floors, which no longer draw.

"An ATA-SMP market has no bot behind it, so the chart draws its candles without the trade markers, the target lines, the tranche floors and the fire glow that a traded asset carries." - the sentence holds; there are no tranche floors on any market now.

## 2026-09-20 09:45 - #55 - the oscillators: one readable sub-pane height, whole-pixel pens, tags that never cross

His words, 2026-09-20: *"Oscillators are not being drawn at the highest quality and are too compressed vertically to be read clearly."* And: *"it does not appear that the GUI Archetype organized all of the elements properly using any kind of rules."* Three rules now govern the seven sub-panes, the pens and the fill tags, and each is proved on the running program in both builds. The GUI Archetype carries no layout or draw-quality rule, so the rules are stated here and read off the widget and the page.

### The height rule

Every sub-pane on the Charts tab draws at one readable height. The figure is derived from what a sub-pane holds: a label row one tag tall over a plot of five bands one tag tall each - room over the upper ruled level, the upper level, the value tag at the middle, the lower level, and room under the lower level. A tag is 14 px, so a sub-pane is 14 + 5 x 14 = 84 px. The lines, the histogram and the ruled levels plot under the label row and never through it. The value tag sits at the plot's middle. The price pane never falls under 220 px.

`src/gui/main_tabs/native_chart_surface.py` - the figure beside the pane numbers

```
TAG_HEIGHT_PX = 14
#: A sub-pane's label row, one tag tall, over its plot.
SUB_PANE_LABEL_PX = TAG_HEIGHT_PX
#: The tag-tall bands a sub-pane's plot holds: over the upper ruled level, the
#: upper level, the value tag at the middle, the lower level, under the lower level.
SUB_PANE_PLOT_BANDS = 5
#: The height every sub-pane draws at on the Charts tab: the label row over
#: the plot's bands. A host shorter than the panes' height scrolls.
SUB_PANE_READABLE_PX = SUB_PANE_LABEL_PX + SUB_PANE_PLOT_BANDS * TAG_HEIGHT_PX
#: The least height a sub-pane folds to on a fixed-height venue image.
SUB_PANE_FOLD_PX = 28
```

The chart's height is the natural height of the panes on. With the volume strip and the seven sub-panes it is 28 + 18 + 220 + 28 + 7 x 84 + 18 = 900 px. A panel taller than that gives the rest to the price pane. A panel shorter than that keeps the chart at 900 and scrolls it.

`src/gui/native_chart.py` - the widget's height

```
        def _apply_height_for_panes(self) -> None:
            """Set the minimum height to ``_natural_height_for_panes`` at the widget's width.

            ``_height_override`` from a grip drag wins when it is taller. The
            scroll area holding the chart scrolls when its viewport is shorter
            than this height; a taller viewport's extra height goes to the price pane.
            """
            target = self._natural_height_for_panes(self.width())
```

### The scroll hosts

On the Qt panel the chart sits in a scroll area with a vertical bar as needed and no horizontal bar. A wheel over the chart zooms the window, as before, and never scrolls the area, because the chart takes the wheel. The bar scrolls. The toggle row sits under the scroll area at the tab's foot.

`src/gui/native_chart.py` - the scroll area in the panel

```
            self._scroll = QScrollArea(self)
            self._scroll.setAccessibleName(CHART_SCROLL_NAME)
            self._scroll.setWidgetResizable(True)
            self._scroll.setFrameShape(QFrame.NoFrame)
            self._scroll.setHorizontalScrollBarPolicy(Qt.ScrollBarAlwaysOff)
            self._scroll.setVerticalScrollBarPolicy(Qt.ScrollBarAsNeeded)
            self._scroll.setWidget(self._chart)
            layout.addWidget(self._scroll)
```

On the page the chart mount scrolls the same way. The image host inside it is never under the natural height, so a shorter mount scrolls and the toggle row under the mount stays at the panel's foot. The wheel listener on the host already takes the wheel for the zoom.

`src/gui/web/trade_charts_tab.js` - the mount and the host

```
      style: {
        flex: AUTO,
        minHeight: ZERO_PX,
        overflowX: HIDDEN,
        overflowY: AUTO,
        display: FLEX,
        flexDirection: COLUMN
      }
```

```
    into.style.minHeight = height(
      answer[IMAGE_NATURAL_HEIGHT] === undefined
        ? answer[IMAGE_HEIGHT]
        : answer[IMAGE_NATURAL_HEIGHT]
    );
```

The image the page asks for is never under the natural height either.

`src/gui/react_charts_tab.py` - the image's height

```
            natural = int(self._painter._natural_height_for_panes(width))
            minimum = int(self._painter._minimum_height_for_panes(width))
            height = max(natural, asked_height)
```

A fixed-height venue image keeps the fold of 2026-09-19: each sub-pane shrinks from the readable height toward 28 px, then the reading strip folds, then the price pane shrinks toward 120 px, and a height under the least layout is refused by name.

### The quality rule

Every stroke pen's width is its logical width scaled by the device pixel ratio to whole device pixels, never under one. The five logical widths 0.8, 1.0, 1.2, 1.4 and 1.5 draw 1, 1, 1, 1 and 2 device pixels at ratio 1.0 and 1, 1, 2, 2 and 2 at 1.25. Lines draw with anti-aliasing on. Axis rules - the grid, the pane separators, the ruled levels, the reference lines and the MACD zero line - draw with anti-aliasing off, one device pixel wide, on coordinates snapped to the centre of a device pixel. Bars - the volume bars and the MACD histogram - draw with anti-aliasing off on whole device pixels. Text stays at the caption size in the mono family.

`src/gui/main_tabs/native_chart_surface.py` - the pen width

```
def device_pen_width(width_px: float, ratio: float) -> float:
    """``width_px`` logical pixels scaled by ``ratio`` to whole device pixels, never
    under one, given back in logical pixels."""
    scale = float(ratio) if float(ratio) > 0 else 1.0
    return max(1, round(float(width_px) * scale)) / scale
```

`src/gui/native_chart.py` - one ruled level

```
                def _rule_line(y: float, colour=None, dashed: bool = True) -> None:
                    """One ruled level across the pane at ``y``: one device pixel, aliased, snapped."""
                    pen = QPen(colour if colour is not None else self.GRID_MINOR, px)
                    if dashed:
                        pen.setStyle(Qt.DashLine)
                    p.setRenderHint(QPainter.Antialiasing, False)
                    p.setPen(pen)
                    p.drawLine(QPointF(ML, snap(y)), QPointF(w - MR, snap(y)))
                    p.setRenderHint(QPainter.Antialiasing, True)
```

### The tag rule and the readout

Every glyph draws first. A fill tag is then placed at the right of its glyph, or at the left when the right would cross the price scale. A tag that would cross a glyph or a tag already drawn is not drawn. The glyph count does not move. Merging two tags into one naming a count was not chosen, because a count names no price and no role.

`src/gui/native_chart.py` - where a tag goes

```
        def _tag_place(
            self,
            right_x: float,
            left_x: float,
            y: float,
            width: float,
            drawn: list,
            scale_x: float,
        ) -> Optional[QRectF]:
            """The rect a fill tag draws in: ``TAG_H_PX`` tall at ``right_x`` when
            ``_tag_clear``, else at ``left_x`` when clear, else None and no tag."""
            for x in (right_x, left_x):
                rect = QRectF(x, y, width, TAG_H_PX)
                if self._tag_clear(rect, drawn, scale_x):
                    return rect
            return None
```

The crosshair's readout names every fill on the candle under the pointer. One row per fill follows the six candle rows: its label is the side's glyph character, a sell as a filled triangle pointing down and a buy as one pointing up, and its text is the tag's text in the marker's colour. The painter's own tip and the page's canvas tip both draw the rows, because the page reads them off the geometry the image answer carries.

`src/gui/main_tabs/native_chart_surface.py` - one readout row for a fill

```
def fill_row(fill: FillLike) -> list[str]:
    """One readout row for ``fill``: the side's glyph, the tag's text and its role."""
    is_buy = str(fill.side).lower() == FILL_BUY_SIDE
    return [
        READOUT_BUY_GLYPH if is_buy else READOUT_SELL_GLYPH,
        FILL_TAG_FORMAT.format(label=fill_label(fill), price=fmt_price(fill.price)),
        fill_role(fill),
    ]
```

### The oscillator readings, off the running program, both builds

Each build ran the real window with the home on a scratch directory, every socket but loopback refused, two bots off a scratch copy of one record, 100 A15 1h candles and a loopback History venue answering 40 fills, 26 inside the window. The chart's host was resized to 690, 900 and 1100 px. Each run was driven under the debugger with a breakpoint on the paint pass, ignored on every crossing and counted at the end.

| reading | before, Qt / React | after, Qt / React |
|---|---|---|
| seven sub-panes at a 690 px host | 54.0 px each, the price pane 220, no scroll | 84.0 px each, the price pane 220, the chart 900 tall, the bar's range 210 |
| at 900 | 60.0 each, 388 | 84.0 each, 220, range 0 |
| at 1100 | 60.0 each, 588 | 84.0 each, 420, range 0 |
| the toggle row | under the chart | under the scroll host at the tab's foot: y 738 on Qt, 740.8 on the page, at 690 |
| the wheel over the chart | zooms 100 to 85 candles | zooms 100 to 85; the bar stays at 0 |

| reading | before, Qt / React | after, Qt / React |
|---|---|---|
| the stroke pens in device pixels, ratio 1.0 | 0.8, 1.0, 1.2, 1.4, 1.5, 1.7 | 1, 2 |
| ratio 1.25 | 1.0, 1.25, 1.5, 1.75, 1.875, 2.12 | 1, 2 |
| the fills at 100 candles | 26 glyphs, 26 tags, 24 overlapping pairs | 26 glyphs, 15 tags, 0 overlapping pairs |
| at 36 candles | 5 glyphs, 5 tags, 0 pairs | 5 glyphs, 4 tags, 0 pairs |
| the squat venue, Facebook 1200 by 630, eight voters | seven sub-panes at 40 px, the strip folded, the price pane 226 | the same |
| the readable figure planted to 20 | - | seven sub-panes at 20.0, the chart 452 tall; 84.0 restored |
| the tag rule planted off | - | 26 tags, 24 overlapping pairs; 15 and 0 restored |

The crops at 1.25, four times enlarged: before, the RSI line smeared over two to three device rows and the 30 and 70 rules over two faint rows; after, the line two device pixels and each rule one crisp dashed row, the label in its own row over the plot, the value tag legible. Renders sit under `artifacts/u55/C8/` before and after.

### Four sentences the oscillators overtake

They were not reworded. They are quoted here.

"Each takes 60 px at the chart's natural height. When the window gives less, every sub-pane shrinks alike, down to 28 px, so the price pane keeps its 220 px and every pane stays on screen." - each takes 84 px on the tab at every window height, and the panel scrolls when the window gives less; only a fixed-height venue image shrinks a sub-pane, toward 28 px.

"`SUB_PANE_H = 60`" and "`SUB_PANE_MIN_H = 28`" - the two names are gone; the painter reads the surface's readable figure and its fold floor.

"The wick is a share of one candle column and never under one device pixel; the candle border is one device pixel; the indicator lines keep their logical width and stay anti-aliased. Read on a 1200 pixel image at ratio 1.0 the pens are 0.8, 1.0, 1.2, 1.4, 1.5 and 1.56 logical pixels; at ratio 2.0 the set gains 0.5, the one-device-pixel pen." - every stroke pen is now a whole number of device pixels; at ratio 1.0 the set is 1 and 2, at 1.25 it is 0.8 and 1.6 logical, which are 1 and 2 device pixels.

"The glyph sits on the candle whose interval holds the fill's stamp, one candle column wide and a twentieth of the price pane tall, with a tag naming the role and the price beside it." - the glyph holds; its tag draws only where it crosses no glyph and no other tag, and the readout names the fill under the pointer.

## 2026-09-23 17:30 - #55 - the oscillators are a third taller, and the bottom bar drags both heights

His words, 2026-09-23: *"Oscillator default height should be increased by 30% - Oscillator and chart height needs to be adjustable by dragging the bottom bar. Said adjustments must move all subsequent elements down and not overlap them"*.

### The default height

A sub-pane's plot now holds seven tag-tall bands, not five: two over the upper ruled level, the upper level, the value tag at the middle, the lower level, and two under it. A tag is 14 px, so a sub-pane is 14 + 7 x 14 = 112 px. Read off the running chart before the change it was 84.0 px. His figure, 84 x 1.3, is 109.2 px, which is not a whole number of bands; seven bands is the least count that reaches it, and it gives the plot the same room above the upper level as below the lower one. The rise is 33.3%.

`src/gui/main_tabs/native_chart_surface.py` - the band count

```
#: The tag-tall bands a sub-pane's plot holds: two over the upper ruled level,
#: the upper level, the value tag at the middle, the lower level, two under the
#: lower level. Seven is the least count whose height clears 84 px by the 30%
#: the operator asked for: 14 + 7 x 14 = 112, a rise of 33.3%.
SUB_PANE_PLOT_BANDS = 7
```

The chart's natural height follows. With the volume strip and the seven sub-panes it is 28 + 18 + 220 + 28 + 7 x 112 + 18 = 1096 px, where it was 900. A panel shorter than that scrolls to it.

### The bottom bar

Three short dashes draw across the chart's bottom edge, in a strip 8 px tall. Pressing in that strip and dragging down sets the chart's height, and every sub-pane keeps the share of the chart it holds at the natural height. One drag therefore raises the price pane and every oscillator together. Dragged up, the height stops at the natural 1096 and the sub-panes return to 112 px. The pointer turns to the vertical resize arrows over the strip on the Qt widget.

`src/gui/main_tabs/native_chart_surface.py` - the share the drag gives a sub-pane

```
def sub_pane_height(dragged_height_px: int, readable_height_px: int) -> int:
    scaled = SUB_PANE_READABLE_PX * int(dragged_height_px) // int(readable_height_px)
    return max(SUB_PANE_READABLE_PX, scaled)
```

`CandlestickChart.mouseMoveEvent` drives it on the widget, and `chart_view` drives it on the page from the same press, drag and release the page already sends. Both call `set_pane_drag_height`, so both builds answer one grab of the bar the same way.

### Nothing below the drag overlaps

Inside the chart the panes are laid one under the next: each pane's top is the pane above's bottom, so a drag moves every pane under it down by the height it won. Outside the chart the panes sit in a scroll host, with the toggle row under the host, so the host's own height never changes and the toggle row never moves. Read on the running program at 700, 900 and 1400 px wide:

| drag | sub-pane | price pane | vortex | rsi | time axis | chart bottom |
|---|---|---|---|---|---|---|
| at rest | 112.0 | 46.0 - 266.0 | 294.0 - 406.0 | 966.0 - 1078.0 | 1078.0 - 1096.0 | 1096 |
| +260 | 138.0 | 46.0 - 344.0 | 372.0 - 510.0 | 1200.0 - 1338.0 | 1338.0 - 1356.0 | 1356 |
| +4000 | 520.0 | 46.0 - 1410.0 | 1438.0 - 1958.0 | 4558.0 - 5078.0 | 5078.0 - 5096.0 | 5096 |
| dragged up | 112.0 | 46.0 - 266.0 | 294.0 - 406.0 | 966.0 - 1078.0 | 1078.0 - 1096.0 | 1096 |

On the Qt panel at 700 px wide the scroll host reads 0.0 - 668.0 and the toggle row 674.0 - 686.0 at every one of those positions; at 900 the host reads 0.0 - 878.0 and the row 884.0 - 896.0; at 1400 the host reads 0.0 - 1078.0 and the row 1084.0 - 1096.0. On the page at 700 the chart mount reads 50.8 - 656.8 and the toggle row 660.6 - 674.6 at every position, and the image inside the mount grows from 1146.8 to 4002.8 while the mount scrolls.

### The height and drag readings, off the running program, both builds

Each build ran with the home on a scratch directory and every socket but loopback refused, 100 A15 1h candles, seven sub-panes on, the panel at 700, 900 and 1400 px wide. The Qt numbers come from `ChartPainter.geometry_payload`, which reports what the last paint laid out; the page's numbers come from the same payload carried on the image answer, and its element rectangles from the page's own `getBoundingClientRect`.

| reading | before, Qt / React | after, Qt / React |
|---|---|---|
| each sub-pane at rest, every width | 84.0 / 84.0 | 112.0 / 112.0 |
| the chart's natural height | 900 / 900 | 1096 / 1096 |
| the bar dragged +260 | 84.0, the chart 1160 / no answer, the image 900 | 138.0, the chart 1356 / 138.0, the image 1356 |
| the bar dragged +4000 | 84.0, the chart 4900 / no answer, the image 900 | 520.0, the chart 5096 / 520.0, the image 5096 |
| the bar dragged up | 84.0, the chart 900 / 900 | 112.0, the chart 1096 / 1096 |
| the panes' tops and bottoms, every position | each top the one above's bottom | the same |
| the toggle row against the scroll host | 674.0 under 668.0 / 660.6 under 656.8 | the same at every drag position |
| the band count planted back to 5 | - | 84.0 at rest and the natural height 900, in both builds; 112.0 restored |

Renders sit under `artifacts/u55/C9/`, one per width and drag position in each build.

### Three sentences the drag overtakes

They were not reworded. They are quoted here.

"A tag is 14 px, so a sub-pane is 14 + 5 x 14 = 84 px." - a sub-pane's plot holds seven bands, so a sub-pane is 14 + 7 x 14 = 112 px.

"With the volume strip and the seven sub-panes it is 28 + 18 + 220 + 28 + 7 x 84 + 18 = 900 px." - it is 28 + 18 + 220 + 28 + 7 x 112 + 18 = 1096 px.

"each takes 84 px on the tab at every window height" - each takes 112 px at every window height, and a drag of the bottom bar raises that figure with the chart.

## 2026-09-23 22:10 - #55 - the trade markers draw on every timeframe

His words, 2026-09-23: *"Historical trade markers ... do not consistently appear and I could only get them to do so on the weekly timeframe during the previous test."* The chart now draws a fill's glyph at 5m, 1h, 1d and 1w, and where a timeframe cannot reach a fill it says so in the header.

### The window rule and what each timeframe reaches

One fetch asks for `FETCH_LIMIT` candles, and a fill can only be drawn on a candle the fetch brought back. The window a chart draws is therefore the candle count times the timeframe's own length, and a fill older than the first candle is counted, named in the header, and not drawn.

`FETCH_LIMIT` is 300. One request carries at most 300 candles on the Coinbase candles route, which the tree records as `RA_CHUNK_DAYS`, and the Advanced Trade route's own ceiling is 350. The lower of the two is the figure the chart asks for, so no fetch asks a route for more than it serves. The fetch is one request either way, because the count is a parameter of that one call.

`src/gui/main_tabs/trade_charts_tab_surface.py` - the one declaration

```
#: Candles one fetch asks for, and so the span a fill's glyph can land in;
#: 300 is the Coinbase candles route's one-request ceiling, recorded as
#: ``RA_CHUNK_DAYS``. ``src.gui.widgets.trade_charts_tab`` imports this name.
FETCH_LIMIT = 300
```

What 300 candles reach, from `TIMEFRAME_SECONDS`:

| timeframe | window span | reaches a fill this old |
|---|---|---|
| 5m | 1.042 days | one day |
| 1h | 12.500 days | twelve and a half days |
| 1d | 300.000 days | ten months |
| 1w | 2100.000 days | five years nine months |

A five-minute chart cannot reach a fill five months old. Reaching 152 days at 5m needs 44,011 candles, which is 147 requests and 0.014 px a candle in the pane; at 1h it needs 3,668 candles and 12 requests. The daily and the weekly reach every fill a bot of this fleet holds, and the two short timeframes draw what they reach and name the rest.

### One declaration of the candle count

`FETCH_LIMIT` was declared in two files with the same value. Nothing compares the two, so the two copies could disagree without anything reporting it. The surface declares the name now, and the Qt tab imports it.

`src/gui/widgets/trade_charts_tab.py` - the import

```
from ..main_tabs.trade_charts_tab_surface import (
    FETCH_LIMIT,
    bot_timeframe,
    fill_record,
```

### The header names the fills it could not draw

The paint pass counts each fill it refused, under the reason it refused it: older than the first candle, past the last one, or at a price outside the drawn scale. The header carries one phrase per reason that counted anything, beside the symbol, the timeframe, the source and the candle count. A chart with nothing refused carries none of them, and its picture is unchanged.

`src/gui/native_chart.py` - the three phrases

```
FILLS_OLDER_HEADER_FORMAT = "{count} {fills} older than this chart"
FILLS_NEWER_HEADER_FORMAT = "{count} {fills} newer than this chart"
FILLS_OFF_SCALE_HEADER_FORMAT = "{count} {fills} off the price scale"
```

`charts.annotations.drawn` carries the same counts split by reason: `fills_older_than_window`, `fills_newer_than_window` and `fills_off_price_scale`, with `fills_off_window` the sum of the first two. It previously folded a fill at a price off the drawn scale into `fills_off_window`, which named the window for an absence the window did not cause.

### The marker readings per timeframe

One market, 37 fills fed, read off the running program in both builds at 700, 900 and 1400 px wide. The candles come from a read-only copy of one five-minute stone tablet on disk, folded to each timeframe by the registry's own rollup; the fills carry the fill ages of the running fleet, read from a read-only copy of its state file. No venue was contacted, every socket but loopback was refused, and no bot was started and no order placed, priced or cancelled.

| timeframe | candles before | candles after | window before | window after | drawn before | drawn after | header after |
|---|---|---|---|---|---|---|---|
| 5m | 100 | 300 | 0.347 days | 1.042 days | **0** | **6** | 31 fills older than this chart |
| 1h | 100 | 300 | 4.167 days | 12.500 days | 12 | **18** | 19 fills older than this chart |
| 1d | 100 | 213 | 100.0 days | 213.0 days | 31 | **37** | none |
| 1w | 31 | 31 | 217.0 days | 217.0 days | 37 | 37 | none |

The daily and the weekly answered fewer candles than the 300 asked for, because the tablet on disk holds 212.771 days; a venue serving the whole 300 reaches 300 days at 1d.

Every marker sat on the candle holding its own stamp at every timeframe and every width, with no fault in any reading. One fill planted at 400 days, outside every window including the weekly's, raised every timeframe's refused count by exactly one and turned the daily and the weekly from no phrase to `1 fill older than this chart`.

Read off the saved renders rather than off the painter: at the weekly, before and after differ by **0 pixels** in both builds and in the image the page is handed, and the planted phrase differs by 720 pixels inside rows 14 to 22, the header's own band. At 700 px wide the longest header this draws ends 89 px clear of the right edge, so no phrase is clipped at the narrowest width.

A paint pass costs 40.58 ms at 300 candles against 27.64 ms at 100, at 700 px; 28.10 against 15.82 at 900; 30.51 against 19.43 at 1400. The tick that redraws the chart runs every two seconds.

Renders sit under `artifacts/u55/C11a/`, one per build, stage, width and timeframe.

### Sentences the window rule overtakes

They were not reworded. They are quoted here.

"A fill whose stamp falls before the window or past its end is counted and not drawn." - it is counted, named in the header under the reason it was refused, and not drawn.

"A row outside the window is counted and not drawn." - a row outside the window is counted, named in the header, and not drawn.

"`charts.annotations.drawn` is written when the set of annotations the paint pass drew changes, with the counts drawn against the counts fed that fall inside the pane, and the fills fed and the fills off the window in its context." - the context now carries the fills fed, the fills off the window, and that figure split into the fills older than the window, the fills newer than it and the fills at a price off the drawn scale.

# History Tab

Reference. Trade history pulled from the venues, paired with the platform's own trading logic and graded.

## What builds it

One builder makes the tab, adds it to the row, and hands it the bot manager.

`src/gui/main_tabs/history_tab.py` — `HistoryTabMixin._build_history_tab`

```
def _build_history_tab(self) -> None:
    """Build the History tab and add it to the main tab widget."""
```

## Fetching

Refresh starts an asynchronous fetch that asks every active exchange for the operator's own fills. The helper walks each exchange and symbol pair the bot manager exposes, and turns each venue trade into one row.

`src/exchange/history_helpers.py` — `fetch_all_history_chunked`

```
async def fetch_all_history_chunked(
```

The exchange is the authority here. The rows on screen are the venue's answer, and the platform's own log joins to them rather than replacing them.

The tab refreshes itself on activation when the last fetch is older than five minutes and no fetch is already in flight.

`src/gui/main_window.py` — `_on_main_tab_changed`

```
is_stale = last_ts == 0 or _t.time() - last_ts > 300
if is_stale and not in_flight:
    hist.refresh()
```

## Where the trade record is kept

Refresh reads the venue live and keeps nothing. The year of trades the Simulator needs is kept separately, on disk, under the exchange history bucket. One file holds one exchange, one symbol and one year, and every file names its exchange inside itself.

`src/core/log_paths.py` — the bucket

```
def get_exchange_history_dir() -> Path:
    """``exchange_history/`` bucket — the YTD trade files.
```

The files are filled from a transactions CSV exported from the exchange. The import keeps buys and sells, counts and drops everything that is not a trade, and refuses a file that is missing a column it needs. Re-importing an overlapping export adds only the rows whose id is new.

`src/exchange/ytd_csv_import.py` — the import

```
def import_ytd_csv(
    csv_path: Path,
    exchange_id: str,
    root: Optional[Path] = None,
) -> ImportResult:
    """Read ``csv_path`` and write ``exchange_id``'s trade files under
    ``get_ytd_root(root)``."""
```

Where the export covers a period and carries no trade for a symbol in it, that period is written to a gap record rather than filled. See the Simulator tab for the file format and the reader.

The import reads the export through a column map chosen by the exchange id it is given. One map exists, for Coinbase, and a Coinbase export writes the same files it always did. An exchange with no map is refused by name, with a line saying a sample export is needed, and nothing is written. The maps and the table of exchanges are in the Simulator tab.

`src/exchange/ytd_csv_import.py` — the map chosen at import

```
def import_ytd_csv(
    csv_path: Path,
    exchange_id: str,
    root: Optional[Path] = None,
) -> ImportResult:
    """Read ``csv_path`` and write ``exchange_id``'s trade files under
    ``get_ytd_root(root)``."""
    column_map = export_map_for(exchange_id)
```

## Filtering, paging, export

Eight methods carry the controls under the table.

| Method | What it does |
|---|---|
| `_populate_filter_options` | Fills the exchange, symbol and side pickers from the rows in hand |

| Method | What it does |
|---|---|
| `_apply_filters` | Narrows the set |
| `_reset_filters` | Restores the full set |
| `_current_filters` | Reports the active state |
| `_remember_to_bound`, `_to_bound_ts` | Hold the upper date bound across a refresh |
| `_render_page`, `_prev_page`, `_next_page` | Page the result |
| `_export_csv` | Writes the current selection out |

## Where the cell values come from

One function returns the rendered page: the rows plus the pager state. It is the only disk read on that path.

`src/exchange/history_read_contract.py` — `build_page`

```
def build_page(
    filtered: list[dict],
    page: int = 0,
    bot_manager: Any = None,
    gate_index: Optional[dict] = None,
    voting_index: Optional[dict] = None,
) -> HistoryPage:
    """One rendered page: the ``HistoryRow`` list plus the pager state.

    ``build_join_indexes`` runs when neither index is supplied, and it is the
    only disk read on this path.
    """
```

Thirteen columns each yield one cell carrying a value, its text, its colour and its tooltip.

`src/exchange/history_read_contract.py` — `COLUMNS`, the first ten

```
COLUMNS: tuple[HistoryColumn, ...] = (
    HistoryColumn(0, "timestamp", "Timestamp (UTC)"),
    HistoryColumn(1, "exchange", "Exchange"),
    HistoryColumn(2, "symbol", "Symbol"),
    HistoryColumn(3, "bot", "Bot"),
    HistoryColumn(4, "side", "Side"),
    HistoryColumn(5, "amount", "Amount"),
    HistoryColumn(6, "price", "Price"),
    HistoryColumn(7, "cost", "Cost USD"),
    HistoryColumn(8, "fee", "Fee"),
    HistoryColumn(9, "trade_id", "Trade ID"),
```

Grade, Gates and Voting close the set. The ordering is fixed newest-first and nothing re-sorts.

`src/exchange/history_read_contract.py` — `ROW_ORDER`

```
ROW_ORDER = "timestamp_desc"
"""Newest first. ``fetch_all_history_chunked`` sorts ``reverse=True``,
``HistoryTab`` calls no ``setSortingEnabled``, and nothing re-sorts."""
```

The table renders those fields and derives none of them. A cost, a grade, a colour or a gate light on screen is always the contract's answer, which is what keeps a second implementation of History from growing behind the renderer.

Two strings depart from that pattern. The contract already declares the summary line and the page counter, and the tab builds both itself, so two implementations of the same two strings stand in the tree. Issue #425 carries it.

`src/exchange/history_read_contract.py` — `page_label`, the declaration the tab does not call

```
def page_label(page: int, total: int) -> str:
    """The footer's page counter, or "No matches" when ``total`` is zero."""
    if total == 0:
        return "No matches"
    page = clamp_page(page, total)
    return f"Page {page + 1} / {page_count(total)} ({total} trades)"
```

## Grading

One record and its price context go in, and an unweighted mean comes out. An axis with no input is skipped, so it neither credits nor penalises.

`src/trading/trade_grader.py` — `grade_trade`

```
def grade_trade(record: TradeRecord, ctx: PriceContext) -> TradeGrade:
    """Grade one ``record`` against its ``ctx``.

    ``TradeGrade.overall_numeric`` is the unweighted mean of the sub-scores
    whose inputs were present, or 0.5 when no axis could be scored.
    """
    exec_score, exec_bps = _score_execution(record, ctx)
    timing_score, mfe, mae = _score_timing(record, ctx)
    strategic_score, sb_delta = _score_strategic(record, ctx)
    outcome_score, realized = _score_outcome(record, ctx)
```

The mean becomes a letter at fixed boundaries.

`src/trading/trade_grader.py` — `_letter_from_numeric`

```
def _letter_from_numeric(num: float) -> str:
    """Map ``num`` in [0.0, 1.0] to ``A+``, ``A``, ``B``, ``C``, ``D`` or ``F``."""
    if num >= 0.93:
        return "A+"
    elif num >= 0.85:
        return "A"
    elif num >= 0.70:
        return "B"
    elif num >= 0.55:
        return "C"
    elif num >= 0.40:
        return "D"
    else:
        return "F"
```

`grade_trades` runs the batch. The regime tag reaches the grade's regime and its rationale without a sub-score of its own.

## Gate analysis

Four helpers in `src/exchange/history_helpers.py` join the gate log to a trade.

| Helper | What it does |
|---|---|
| `build_page_gate_index` | Buckets the gate log entries by bot id and minute |
| `lookup_gate_entry` | Picks the closest entry to a trade |
| `gate_cell_text` | Renders the cell text |
| `gate_cell_tooltip` | Renders its tooltip |

The lights themselves come from one vocabulary. Nineteen of them, the scrum bank then the fold, each carrying its bank marker, its label, its state and its colour. No Qt type crosses that boundary.

`src/trading/gate_vocabulary.py` — `gate_light_row`

```
def gate_light_row(
    scrum_armed: bool,
    fold_armed: bool,
    scrum_blockers: Optional[list] = None,
    fold_blockers: Optional[list] = None,
    landing_strip_side: str = "",
    evaluated: bool = True,
) -> list[dict]:
    """The nineteen lights, in draw order: the scrum bank then the fold.

    Each entry carries ``bank`` ("S" or "F"), ``label``, ``state`` and
    ``color``. Serialisable: no Qt type crosses this boundary.
    """
```

Five states, five colours.

`src/trading/gate_vocabulary.py` — `LIGHT_COLORS`

```
LIGHT_COLORS: dict[str, str] = {
    "override": "#22d3ee",
    "not_evaluated": "#333340",
    "blocked": "#ff3366",
    "passed": "#00cc55",
    "not_the_blocker": "#c8901e",
}
```

That vocabulary is shared on purpose. The Simulator's gate cell and this table read the same labels and the same blocker prefixes, so a gate added on one surface cannot be missing from the other. That shared seam is what makes the Simulator's gate comparison meaningful.

Each trade is joined to its gate entry by `lookup_gate_entry` in `src/exchange/history_helpers.py`. One threshold decides what counts as a match, and a trade with no entry inside it reads "no record" in the cell.

`src/exchange/history_helpers.py` — the join threshold

```
JOIN_TOLERANCE_SECONDS = 60.0
"""Largest gap ``_best_entry_within_window`` accepts between a trade and a log
entry, and the backdate the index builders apply to ``since``."""
```

Nothing measures coverage across a window.

`build_page_voting_index` and `lookup_voting_entry` do the same for the indicator voting snapshot behind each trade.

## The renderer

`HistoryWebTable` in `src/gui/react_history_panel.py` draws the table with React inside the Chromium PySide6 ships, and sits where the Qt table used to. One History tab, one renderer, one set of rows.

## The Simulator front-load

Refreshing History hands the Simulator the year of live trades it needs before a parity run. The tab's signal connects to the Simulator's panel, and the builder makes that connection once both tabs exist.

`src/gui/history_tab.py` — `HistoryTab.history_refreshed`

```
history_refreshed = Signal(list)
```

## Paper History

The manual's part list names a Paper History Tab beside this one. No such module exists. Issue #19 built the Paper tab and its log, so the ledger such a tab would read is on disk now, and no screen reads it. See paper-trader.md.

## Bridge

Three methods serve this screen.

| Bridge method | Serves |
|---|---|
| `history.view_model` | The whole screen, assembled from the read contract |
| `history_tab.chrome` | The surrounding controls |
| `react_history_panel.state` | The table |

Three renderer files draw it: `history_tab.js` for the chrome, then `history_panel.js` and its stylesheet for the table.

## 2026-09-08 08:17 - no issue recorded - what the closed issues landed

The History Tab is able to pull trade history from all active exchanges via their respective APIs. It also pairs each imported trade with its in-platform trading logic and applies our trade grading system.

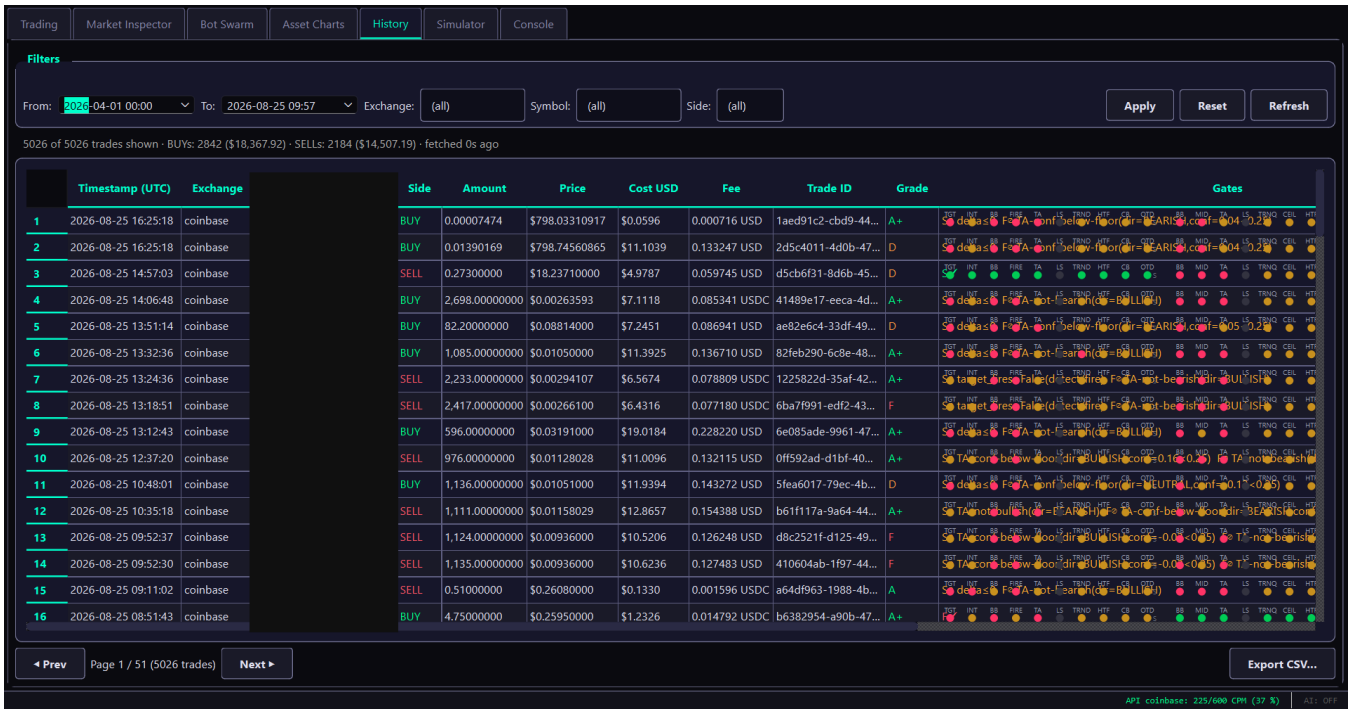
The tab keeps the name History. It sits eighth on the bar, on the gold ground with red text.

The tab fetches from every active venue and draws the rows with React. Every cell value comes from one read contract; the table renders those fields and derives none, which stops a second History growing behind the renderer.

`src/exchange/history_read_contract.py` — `build_page`

```
def build_page(
    filtered: list[dict],
    page: int = 0,
    bot_manager: Any = None,
    gate_index: Optional[dict] = None,
    voting_index: Optional[dict] = None,
) -> HistoryPage:
```

The grade comes from the trade grader and the gate lights come from the same vocabulary the Simulator's gate cell reads. A refresh also hands the Simulator its year of live trades, so the tab's signal front-loads the parity run.



The History tab: filters, graded rows and the gate lights.

The Filters group carries From and To date pickers, then Exchange, Symbol and Side, each filled from the rows in hand. Apply narrows the set, Reset restores it, and Refresh goes back to the venues. The line under the group counts the retained rows against the loaded rows, both sides with their dollar totals, and the age of the last fetch.

Thirteen columns carry a row, newest first, and nothing re-sorts. BUY draws green and SELL red.

`src/exchange/history_read_contract.py` — `COLUMNS`, the first ten

```
COLUMNS: tuple[HistoryColumn, ...] = (
    HistoryColumn(0, "timestamp", "Timestamp (UTC)"),
    HistoryColumn(1, "exchange", "Exchange"),
    HistoryColumn(2, "symbol", "Symbol"),
    HistoryColumn(3, "bot", "Bot"),
    HistoryColumn(4, "side", "Side"),
    HistoryColumn(5, "amount", "Amount"),
    HistoryColumn(6, "price", "Price"),
    HistoryColumn(7, "cost", "Cost USD"),
    HistoryColumn(8, "fee", "Fee"),
    HistoryColumn(9, "trade_id", "Trade ID"),
```

Grade, Gates and Voting close the set.

The Gates cell opens with the blocker text, then the lights. Nineteen of them: the ten scrum gates TGT, INT, BB, FIRE, TA, LS, TRND, HTF, CB and OTD, then the nine fold gates BB, MID, TA, LS, TRNQ, CEIL, HTF, CB and OTD. Each light draws under its own label, and each bank ends with an S or an F marker. Five states, five colours.

`src/trading/gate_vocabulary.py` — `LIGHT_COLORS`

```
LIGHT_COLORS: dict[str, str] = {
    "override": "#22d3ee",
    "not_evaluated": "#333340",
    "blocked": "#ff3366",
    "passed": "#00cc55",
    "not_the_blocker": "#c8901e",
}
```

Green passed, red the gate that blocked, amber not armed for another reason, grey not evaluated at this candle, and cyan a landing-strip override. The blocker text and the light banks overlap in the figure; both occupy the one cell.

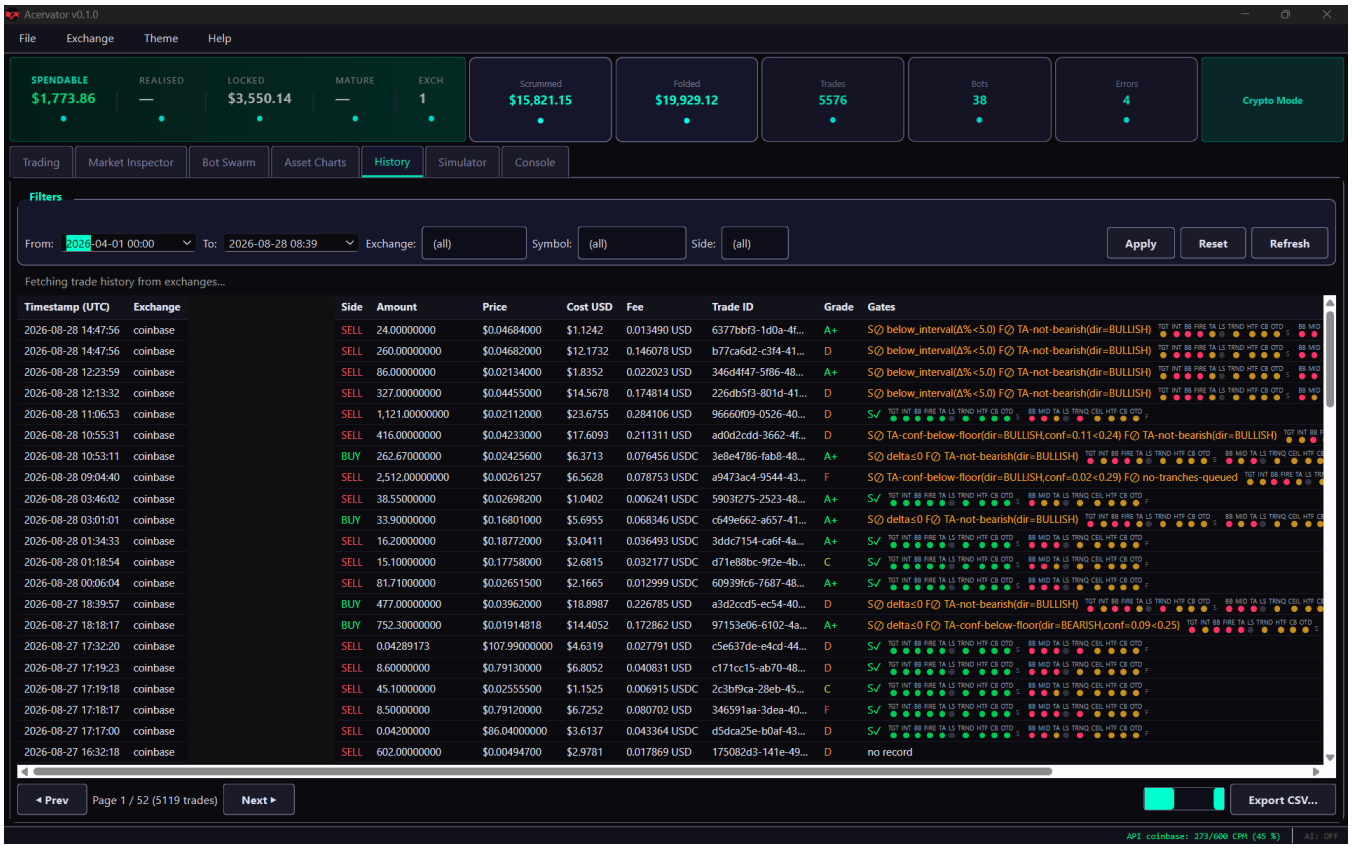
One sentence above is overtaken. Quoted whole:

> *The blocker text and the light banks overlap in the figure; both occupy the one cell.*

The blocker text and the light banks share the one cell and no longer overlap. The Qt table writes the text once, into the lights cell's own label, and leaves the table item under that cell blank. A row whose gate log holds no entry has no lights cell, and there the table item carries the `no record` text itself.

Prev, the page counter and Next page the result at the foot, and Export CSV writes the current selection out. The tab builds the summary line and the page counter itself, and the read contract already declares both, so two implementations of the same two strings stand in the tree. Issue #425 carries it.

History Tab (React):



The same tab, captured while the asynchronous fetch runs. Five fixed status strings cover the states before any row exists, and the one on screen is the fetching entry.

`src/exchange/history_read_contract.py` — `STATUS_TEXT`

```
STATUS_TEXT = {
    "idle": "No history loaded yet — click Refresh.",
    "no_bot_manager": "Bot manager unavailable — cannot fetch history.",
    "no_async_loop": "Async loop not ready — try again after platform starts.",
    "fetching": "Fetching trade history from exchanges…",
    "timeout": "Fetch timeout (60s). Exchange may be rate-limited; try again.",
}
```

Two constants bound the fetch, in the same module: the poll runs on a 400 ms timer, so an observed latency is the true latency plus up to one interval, and the fetch is abandoned after sixty seconds.

`src/exchange/history_read_contract.py` — the fetch bounds

```
FETCH_POLL_INTERVAL_S = 0.4
```

The header strip is visible here, which marks the screen as a live-trading tab. REALISED and MATURE draw the exchange figures the Live tab draws, which issue #418 fed. A row whose gate log holds no entry inside the join window reads `no record`.

Both columns draw an exchange figure here too, because one strip serves every tab that shows it. Where the venue has answered for no bot they stay empty, and Privacy Mode leaves an empty column empty.

### Trade grading

The grader scores a trade that has already happened. It never feeds a trading decision, it never tunes a parameter, and no gate consults it. The same inputs always give the same grade.

Four axes are scored. An axis with no input is skipped: it neither credits nor penalises, and the overall number is the unweighted mean of the rest.

`src/trading/trade_grader.py` — `grade_trade`

```
exec_score, exec_bps = _score_execution(record, ctx)
timing_score, mfe, mae = _score_timing(record, ctx)
strategic_score, sb_delta = _score_strategic(record, ctx)
outcome_score, realized = _score_outcome(record, ctx)
```

| Axis | What it reads | What a 1.0 means |
|---|---|---|
| Execution | fill price against the reference price at decision time | no slippage |
| Timing | the favourable excursion against the adverse one over the following candles | the trade went far more right than wrong before it closed |
| Strategic | the asset's discipline ratio before the trade against after it | the trade moved the ratio the operator's way |
| Outcome | realised profit per unit | a clear gain |

The regime tag reaches the grade and the rationale string and gets no score of its own. Grading the same record with a regime tag and without one returns the same number, which is the control that proves it.

The scale is 0.0 to 1.0, not 0 to 100, and the mean becomes a letter at fixed boundaries.

`src/trading/trade_grader.py` — `_letter_from_numeric`

```
    if num >= 0.93:
        return "A+"
    elif num >= 0.85:
        return "A"
    elif num >= 0.70:
        return "B"
    elif num >= 0.55:
        return "C"
    elif num >= 0.40:
        return "D"
    else:
        return "F"
```

Driving the mapper across each boundary and just under it flips the letter exactly where the code says, in both directions.

**One trap worth knowing.** A trade with no gradeable input at all scores 0.5 and therefore reads as a **D**, not as ungraded. A fully specified trade scores 1.0 and reads A+. A column of Ds may mean the trades were poor, or it may mean the surrounding price context never arrived.

Back to the subsystem index.

# Console

Reference. A raw log tail in the upper pane and the emitter signal stream in the lower one.

## What builds it

`ConsoleTabMixin._build_console_tab` in `src/gui/main_tabs/console_tab.py` builds both panes into a vertical splitter, weighted three to two, and starts three timers.

The builder does not hold the panes. It asks `variant_surface` for the tab class, takes the parts off it, and adds it to the tab row.

`src/gui/main_tabs/console_tab.py` — `ConsoleTabMixin._build_console_tab`

```
tab = surface_class(CONSOLE)()
self._console_tab = tab
self._console = tab.log_pane
self._signal_view = tab.signal_view
self._console_pause_btn = tab.pause_button
self._console_pause_indicator = tab.pause_indicator
```

Two classes answer that call. `ConsoleQtTab` builds the Qt panes, the control bar and the vertical splitter. `ConsoleReactTab` holds the same six parts and draws all of them in one web view. React is the running choice.

`src/gui/variant_surface.py` — the two loaders for the Console

```
def _qt_console() -> type:
    """Import and return the Qt Console tab."""
    from .qt_console_tab import ConsoleQtTab

    return ConsoleQtTab


def _react_console() -> type:
    """Import and return the React Console tab."""
    from .react_console_tab import ConsoleReactTab

    return ConsoleReactTab
```

`_start_console_timers` starts two timers, the signal drain and the console health. The pause refresh timer is built with the tab and started by the pause action.

`src/gui/main_tabs/console_tab.py` — `ConsoleTabMixin._start_console_timers`

```
self._signal_timer = QTimer(self)
self._signal_timer.setInterval(surface.DRAIN_INTERVAL_MS)
self._signal_timer.timeout.connect(self._drain_signals)
self._signal_timer.start()
```

## Upper pane: the Python log

The upper pane is a read-only text view capped at two thousand blocks, which is what bounds its memory.

`src/gui/main_tabs/console_tab.py` — `ConsoleTabMixin._build_console_tab`

```
self._console = QPlainTextEdit()
self._console.setReadOnly(True)
self._console.setFont(QFont("Consolas", 9))
```

`ConsoleQtTab` builds that view. Under React the same calls land on `PagePane`, which holds the blocks and pushes them to the page.

The pushes of one event-loop turn coalesce into one. `redraw` starts a zero-interval single-shot timer and `_push_now` builds the payload and pushes it once when the timer fires, so a signals drain that appends 200 rows pushes the page once rather than 200 times. Measured on a Portfolio Battery walk, whose gate chain fires the TA engine's thirteen postcondition pins per evaluation into the drain: with a push per append the walk thread reached 5,000 bars in 257 seconds while the GUI thread rebuilt the page per row; with the pushes coalesced, 14 seconds.

`src/gui/react_console_tab.py` — the coalesced push

```
        def redraw(self) -> None:
            """Ask for one push of both panes on the next event-loop turn; the
            asks of one turn coalesce into one ``_push_now``."""
            if not self._redraw_timer.isActive():
                self._redraw_timer.start()
```

`src/gui/qt_console_tab.py` — `ConsoleQtTab.__init__`

```
self.log_pane = QPlainTextEdit()
self.log_pane.setReadOnly(True)
self.log_pane.setFont(
    QFont(surface.PANE_FONT_FAMILY, surface.PANE_FONT_POINT_SIZE)
)
self.log_pane.setMaximumBlockCount(surface.PANE_MAX_BLOCKS)
```

A log handler formats each record and hands the line to a relay. The relay carries it over a Qt signal to the thread that constructed it and paints it there, so a log call from any thread reaches the pane without touching the widget from that thread. Colour follows the level, and a message holding the phrase `INDICATOR PANEL` paints in the accent colour.

The handler attaches at two points, and both are needed because the logging engine clears propagation on the application logger.

`src/gui/main_tabs/console_tab.py` — `ConsoleTabMixin._build_console_tab`

```
logging.getLogger().addHandler(qt_handler)
logging.getLogger("acervator").addHandler(qt_handler)
```

The builder reads those two names from the surface and adds the tab's own handler to each.

`src/gui/main_tabs/console_tab.py` — `ConsoleTabMixin._build_console_tab`

```
for name in surface.HANDLER_LOGGERS:
    logging.getLogger(name).addHandler(tab.log_handler)
```

### The handler level

The tab sets its own handler to debug and changes no logger's level. An earlier build raised the root logger and never restored it, which left the whole application logging at debug volume for the life of the process. Setting the handler instead keeps the verbosity local to this pane.

`src/gui/main_tabs/console_tab.py` — `ConsoleTabMixin._build_console_tab`

```
# Two disjoint attach points: `logging_engine` clears `acervator.propagate`.
for name in surface.HANDLER_LOGGERS:
    logging.getLogger(name).addHandler(tab.log_handler)
```

Each tab class sets that level on its own handler, from `HANDLER_LEVEL` in the surface.

`src/gui/qt_console_tab.py` — `ConsoleQtTab.__init__`

```
self.log_handler.setFormatter(
    logging.Formatter(surface.LOG_FORMAT, datefmt=surface.LOG_DATEFMT)
)
self.log_handler.setLevel(surface.HANDLER_LEVEL)
```

## Pausing

Pause holds new lines in a bounded buffer. On resume the buffer replays and one notice names anything the cap dropped, so a gap is never silent.

`src/gui/main_tabs/console_log_handler.py` — `_QtLogHandler.set_paused`

```
def set_paused(self, paused: bool) -> None:
    """Hold new lines in ``_buffer`` while ``paused``, and drain them on resume.

    A drain that followed a ``_buffer_max`` overflow appends one notice line.
    """
```

`buffered_count` feeds the pause indicator, and Clear empties the pane.

## Lower pane: the Emitter Network

The signal tail drains twice a second and renders the newest two hundred records. Each line carries a pass marker, a fail marker or a neutral one, then the signal name, the site, the actual value and the expected one. Records the slice drops are counted and announced with a gap marker drawn above the slice.

`src/gui/main_window.py` — `_drain_signals`

```
def _drain_signals(self) -> None:
    """Render sink records past `_signal_seq`, keeping the newest 200."""
```

The drain reads the process sink, the one `get_sink` answers on the GUI thread. A Simulator run routes its worker thread to the Simulator's own sink under the sim bucket, so the walk's pins, the TA engine's thirteen postcondition pins per evaluation among them, no longer reach this pane; the Simulator's own presses on the GUI thread and the one `sim.sink.routed` row each run leaves at its end do. See simulator.md.

`src/core/signal_contract.py` — the sink the drain reads

```
def get_sink() -> Optional[SignalSink]:
    """The sink the calling thread emits into: its `route_thread` sink when one
    is set, else the process sink."""
```

Seven counters ride along, published every five seconds: the sequence watermark, drain ticks, records read, records rendered, records the slice dropped, gap markers drawn, and health ticks seen. Ticks count invocations rather than records, which is what separates a quiet sink from a stopped timer.

`src/gui/main_window.py` — `_drain_signals`

```
# Counts invocations, not records: a quiet sink is not a stopped timer.
self._signal_drain_ticks = getattr(self, "_signal_drain_ticks", 0) + 1
```

## Where the lower pane goes

The signal pane belongs to the Status tab once that screen exists. See system-status.md.

## Bridge

Two methods serve this screen.

| Bridge method | Serves | Renderer module |
|---|---|---|
| `console.tab` | Both panes | `console_tab.js` |
| `console.log_lines` | The log tail | `console_log.js` |

The surface holds the two behaviours the tab owns rather than describes, so neither is written twice.

| In the surface | Holds |
|---|---|
| `ConsolePane` | The same capped block buffer, at the same two-thousand-block limit |
| `SignalLedger` | The same seven counters |
| `format_record` | The same formatter the tab attaches to its handler |

## 2026-09-07 15:58 - no issue recorded - ten single-word tabs, each on its own ground

This tab is focused on displaying Python activity and errors. The lower half, which is displaying the Emitter Network activity, will be migrated to the System Status Tab (under the Watchdog) which is to be built in the near future.

The tab keeps the name Console. It sits last on the bar, on the gold ground with red text.

The tab builds both panes into a vertical splitter. The upper one is a raw log tail: a handler formats each record and a relay paints it on the GUI thread, which lets a log call from any thread reach the pane safely.

`src/gui/qt_console_tab.py` — `ConsoleQtTab.__init__`

```
self.log_pane = QPlainTextEdit()
self.log_pane.setReadOnly(True)
self.log_pane.setFont(
    QFont(surface.PANE_FONT_FAMILY, surface.PANE_FONT_POINT_SIZE)
)
```

The tab sets its own handler to debug and changes no logger's level; an earlier build raised the root logger and never restored it.

`src/gui/qt_console_tab.py` — `ConsoleQtTab.__init__`

```
# The tab sets its own handler's level and leaves every logger level alone.
self.log_handler.setLevel(surface.HANDLER_LEVEL)
```

Pause holds lines in a bounded buffer and announces anything it had to drop.

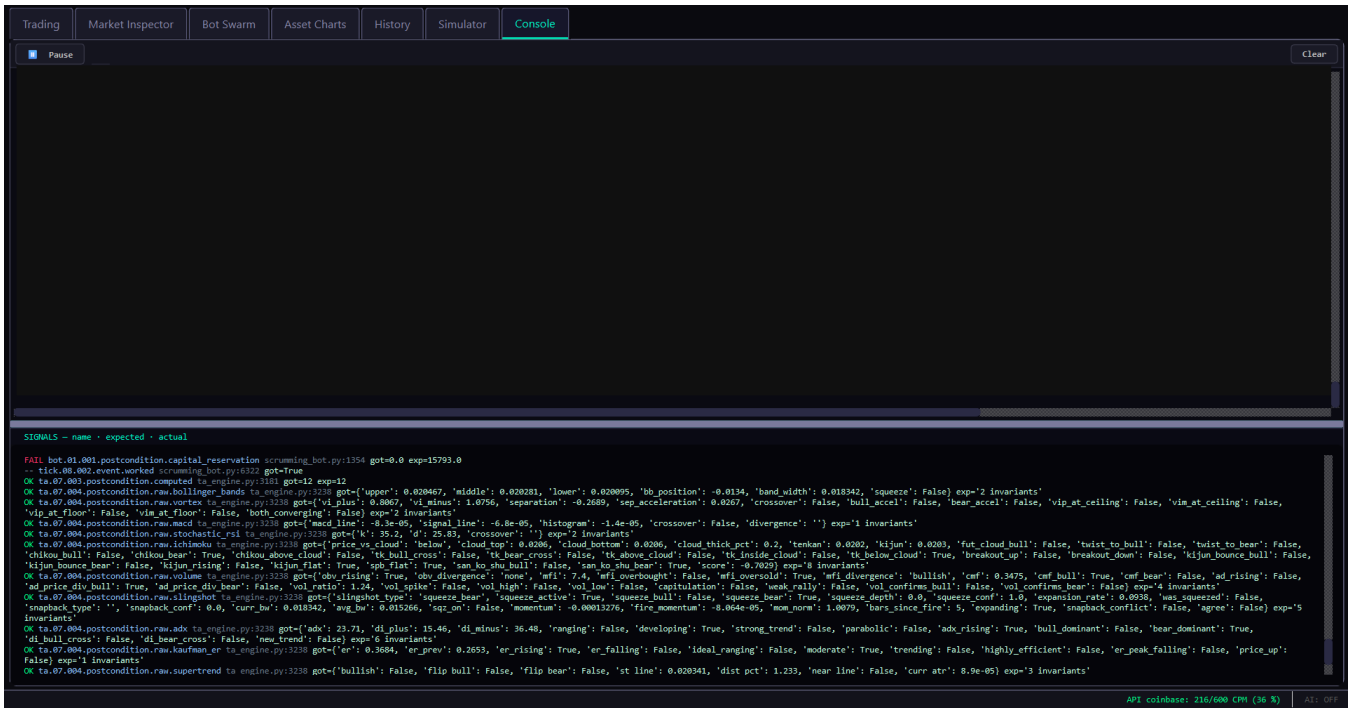
`src/gui/main_tabs/console_log_handler.py` — `_QtLogHandler.set_paused`

```
self._paused = bool(paused)
if not self._paused and self._buffer:
    for msg, r, g, b in self._buffer:
        self._append_signal.emit(msg, r, g, b)
    self._buffer.clear()
```

The lower pane is the Emitter Network. It reads the sink twice a second and renders the newest 200 records with their expected and actual values, keeping seven counters, which lets a quiet sink and a stopped timer read differently.

`src/gui/main_window.py` — `_drain_signals`

```
sink = get_sink()
if sink is None:
    return
new = sink.since(getattr(self, "_signal_seq", 0))
```



The Console tab: the log tail above, the Emitter Network below.

Pause and Clear sit above the upper pane. Pause holds lines in the bounded buffer and announces anything it had to drop when it resumes. Clear empties the pane.

Each line in the upper pane carries the time, the level, the logger name and the message, coloured by level. Three logger names appear in the figure.

React draws this tab. The choice is made in one place, under the screen name `CONSOLE`. A build stamped for Qt gets the pane set above. Every other build gets the web one, which holds the whole tab in a single view.

`src/gui/main_tabs/console_tab.py` — `ConsoleTabMixin._build_console_tab`

```
        from ..variant_surface import CONSOLE, surface_class

        tab = surface_class(CONSOLE)()
        self._console_tab = tab
        self._console = tab.log_pane
        self._signal_view = tab.signal_view
        self._console_pause_btn = tab.pause_button
        self._console_pause_indicator = tab.pause_indicator
```

Both tabs name the same six parts, so the pause, the drain and the health timer call one set of methods whichever side is built. The pause buffer, the level colours and the drop notice stay in the one handler both sides attach.

`src/gui/main_tabs/console_log_handler.py` — `_QtLogHandler.__init__`

```
    def __init__(self, text_edit, painter: Callable[[str, int, int, int], None] = None):
        """Paint into ``text_edit``, or into ``painter`` when one is given."""
```

A line reaches the page as markup the surface writes, with the message escaped, so a log message holding a tag is shown rather than read as one.

`src/gui/main_tabs/console_tab_surface.py` — `line_html`

```
def line_html(text: Any, red: int, green: int, blue: int) -> str:
    """One console line as the markup a pane paints it with.

    The text is escaped, so a message holding a tag is shown rather than
    read as markup.
    """
```

| Logger | Writes |
|---|---|
| `acervator.scrumming` | The bot warnings |
| `acervator.api` | The venue calls |
| `acervator.gui` | The panel feed |

Five levels each carry a colour, and the figure shows three of them: warning, info and debug, with debug the most muted of the three. A sixth colour follows the text of the message rather than its level, and it marks the indicator panel feed.

`src/gui/main_tabs/console_log_handler.py` — the level colours

```
COLORS = {
    "DEBUG": QColor(ds.TEXT_MUTED),
    "INFO": QColor(ds.TEXT_INACTIVE),
    "WARNING": QColor(ds.WARNING),
    "ERROR": QColor(ds.ERROR),
    "CRITICAL": QColor(ds.MAIN_LOG_CRITICAL),
}
```

Two of those lines are worth reading against the code. The repeated warning naming a capital-reservation over-commit comes from `src/trading/capital_reservation.py`, where a claim whose total would pass the holdings is refused and the existing claims, the request and the holdings are named. The `FETCH_OHLCV` line names a seven-indicator engine and lists seven indicators; the voting engine builds twelve and the Indicator Voting Panel shows twelve, so the count in the log line and the count in the engine disagree.

The lower pane opens with the header `SIGNALS — name · expected · actual`. Each record draws a pass marker, a fail marker or a neutral one, then the signal name, the site that emitted it, the actual value and the expected one. A satisfied expectation is recorded the same as a violated one, which keeps a call site that never ran distinct from one that always passed.

`src/gui/main_window.py` — `_drain_signals`, the three markers

```
if r.ok is True:
    mark, colour = "OK  ", ds.SUCCESS
elif r.ok is False:
    mark, colour = "FAIL", ds.ERROR
else:
    mark, colour = "--  ", ds.STATUS_NEUTRAL
```

The site is the file and the line of the frame that made the call, read off the stack at the moment of emission. It moves whenever the code moves. The numbers in the figure are the numbers those call sites held on the day of the capture, not the numbers they hold now.

`src/core/signal_contract.py` — `_caller_site`

```
def _caller_site(depth: int = 2) -> str:
    """Return `file:line` of the calling frame at the given stack `depth`."""
    try:
        f = sys._getframe(depth)
        return f"{Path(f.f_code.co_filename).name}:{f.f_lineno}"
```

### The two snapshot emitters

Two symmetric records go to the bot log, and the Console tab is where an operator reads them. A blocked trade and a fired trade write the same shape, so the log holds both halves of the decision and not only the half that acted.

`src/trading/scrumming/snapshots.py` — `SnapshotEmitterMixin._emit_risk_gate_snapshot`

```
f"RISK GATE SNAPSHOT [{side.upper()}] "
f"risk_blockers={risk_blockers_sorted} "
f"ticker_last={ticker_last:.6g} "
f"panel={snapshot}"
```

The fired-side companion carries the same prefix shape.

`src/trading/scrumming/snapshots.py` — `SnapshotEmitterMixin._emit_trade_fire_snapshot`

```
f"TRADE FIRED SNAPSHOT [{side.upper()}] "
f"ticker_last={ticker_last:.6g} "
f"panel={snapshot}{extra_str}"
```

| Method | Line prefix | Written when |
|---|---|---|
| `_emit_risk_gate_snapshot` | `RISK GATE SNAPSHOT [SIDE]` | a risk gate blocks a trade |
| `_emit_trade_fire_snapshot` | `TRADE FIRED SNAPSHOT [SIDE]` | a scrum or a fold actually fires |

Both lines carry the last ticker price and a panel dictionary holding every voter's direction, confidence, weight and detail at that moment. The fire record adds the list of overrides that engaged, so a reader can tell an override-driven fire from a consensus-driven one without opening anything else.

The two code blocks above and the prefixes in the table are the shape these lines had before the tick-message rewrite. The current prefixes are `RISK GATE [SIDE]` and `TRADE FIRED [SIDE]`, and both write the same panel through one renderer.

| Method | Line prefix | Written when |
|---|---|---|
| `_emit_risk_gate_snapshot` | `RISK GATE [SIDE]` | a risk gate blocks a trade |
| `_emit_trade_fire_snapshot` | `TRADE FIRED [SIDE]` | a scrum or a fold actually fires |

The panel prints as a count and three direction groups rather than as a dictionary. A voter's weight is a configuration value that never moves between ticks, and a neutral voter's confidence is fixed at zero in the indicator code, so neither reaches the line. Direction, confidence and an indicator's own raw detail all remain.

`src/trading/scrumming/snapshots.py` — the grouped panel

```
counts = ", ".join(f"{len(grouped[name])} {name.lower()}" for name in _PANEL_GROUPS)
rows = [f"Panel {counts}."]
for direction, voters in grouped.items():
    voters.sort(key=lambda one: (-one[0], one[1]))
```

The measurement behind the change is in tests/debug_reports/2026-09-07_activity_log_format.md: the dictionary form measured 1,005 to 1,017 characters across 1,143 recorded snapshots, and the grouped form of the same panel measures 322.

Three further emitters sit in the same mixin: a trade notification carrying its own text prefix, a voting-panel snapshot at fire time, and a gate decision at fire time. The last two emit events rather than text lines, and the Console tab's signal pane reads them.

### A missing capital reservation in the log

A bot claims the units it is allowed to work with so that a second bot on the same asset cannot sell them. Four places let a bot carry on when that claim is not there, and all four now write a warning naming what was lost. They appear in the upper pane at warning level.

`src/trading/scrumming/execution.py` — the one that bears on a sale

```
logger.warning(
    "Bot %s sell of %.6f %s is NOT bounded by any other bot's "
    "capital reservation: the pre-check raised %s: %s. A sibling "
    "bot's claim on this asset is invisible to this sell.",
```

The other three name an allocation that is not held aside at admission, an Extractor whose base currency is empty, and a dollar grant made without an over-allocation check.

```
src/trading/container/registry.py      at admission
src/trading/extractor_bot.py           at the chunk-rate claim
src/trading/capital_registry.py        at the dollar grant
```

Two of those three lines have changed. `src/trading/capital_registry.py` is removed, so no dollar grant is made and that warning can no longer be written. The Extractor writes a second warning now, described under the next heading but one.

### The sale stops when the claim cannot be read

One sentence above is overtaken for the first of the four places, and it stays as written:

> *"Four places let a bot carry on when that claim is not there, and all four now*
>
> *write a warning naming what was lost."*

The true sentence for the place that bears on a sale: the warning is written and then the sale is refused. A check that cannot answer is not an allowance, which is what the open-orders read one screen below already does with a failure of its own.

`src/trading/scrumming/execution.py` — the refusal that follows the warning

```
self._emit_trade_notification(
    "SCRUM", "CANCELLED", "capital reservation unreadable"
)
return None
```

Driven on the real sell path with the claim read raising, twice: once on a claim table holding a quantity that is not a number, and once on a registry that could not be resolved. Both readings are the record the bot itself emitted.

```
before   SELL signal: 20.000000 @ $100.00000000 (VH:clean, MARKET)
         SCRUM SENT, venue order calls 1
after    SELL REFUSED (capital reservation unreadable)
         SCRUM CANCELLED, venue order calls 0
```

The operator sees a cancelled scrum with its reason in the upper pane, where a completed sale used to appear.

### A claim that never reached the disk

The registry keeps the claim table in memory and writes it to a file. The writer is `_save` in `src/trading/capital_reservation.py`. A failed write does not stop the bot. It logs an error that names the file, the reason, and the count of changes that are not on the disk.

The line that records the claim now carries the answer as well. `CapitalReservationRegistry.reserve` logs the field `on_disk`. The line is an error when the value is False, and information when the value is True. Read the last line, not the first: a claim with `on_disk=False` exists for this run only, and the next start loses it.

### A claim the bot refuses to place

The bot reads its exchange balance before it claims. The reader is `_get_cached_exchange_balance` in `src/trading/scrumming_bot.py`. The reader answers None when the venue call fails.

A balance that did not read is not a balance of zero, and it is not a licence to claim. `CapitalReservationMixin._ensure_capital_reservation` in `src/trading/scrumming/capital_reservation_mixin.py` stops on that answer. It places no new claim. It resizes no standing claim. It writes a warning that names the bot and the asset, and it tries again on the next call.

### The Extractor refuses on the same answer

The Extractor reads its base currency the same way. The reader is `_read_base_holdings` in `src/trading/extractor_bot.py`, and it answers nothing when the venue call fails.

`set_initial_chunk_rate` stops on that answer. It places no claim, and it writes one warning at the upper pane that names the bot and the asset. The line reads:

```
Bot <id> could not read its <asset> balance; placing no claim, because no
holdings figure bounds it.
```

The Extractor claims once for each run, so it does not try again on a later call. A start reads the balance again.

### The bot named in a refused sale

A sale stops when another bot claims the units. The check is `effective_available` in `src/trading/capital_reservation.py`, and the caller is `src/trading/scrumming/execution.py`.

The refusal names the bots that hold the claim and the units each one holds. Stop a named bot to release its claim: `ScrummingBot.stop` calls `_release_capital_reservation`. The refusal names no page, because no page edits this table.

### A claim that expires on its own clock

A bot pulses a heartbeat while it runs. A bot that stops pulsing leaves a claim that blocks every other bot on the same asset.

`prune_expired` in `src/trading/capital_reservation.py` drops a claim after `HEARTBEAT_TTL` seconds of silence, which is 120. `_prune_on_schedule` runs it once every HEARTBEAT_INTERVAL seconds, which is 30. Two methods call it: `effective_available`, before it answers, and `heartbeat`, after

it stamps. A running bot stamps first, so it never drops its own claim. No setting changes either interval.

## 2026-09-23 - #34 - the drained record carries three more fields

`SignalSink.emit` in `src/core/signal_contract.py` stamps `cadence`, `budget_s` and `tab` on every record. The drain reads the same records, so every record this pane renders carries them. See system-status.md for what each field holds.

### What the pane draws after the three fields land

The rendered line does not change. `_drain_signals` in `src/gui/main_window.py` reads five attributes off each record: `ok`, `name`, `site`, `actual` and `expected`. None of the three new fields is one of them.

`src/gui/main_window.py` - `_drain_signals`, the five it reads

```
view.appendHtml(
    f'<span style="color:{colour}">{mark}</span> '
    f'<span style="color:{ds.MAIN_LOG_NAME}">{_esc(r.name)}</span>'
    f'<span style="color:{ds.MAIN_LOG_SITE}"> {_esc(r.site)}</span>'
)
```

The seven counters do not change either. They count drain ticks, records read and records rendered, and no counter reads a field of the record.

Back to the subsystem index.

# System Status Tab

Reference. The screen is not built. The tab row carries an empty tab labelled Status, and issue #34 carries the build-out. Both halves the operator's text describes run today, and this file is those two halves.

## The skeleton

The tab exists and draws three lines: its name, one sentence saying it is not built, and the issue that owns it. It reads no emitter, no heartbeat and no post-mortem.

`src/gui/main_tabs/system_status_tab_surface.py` — the whole empty state

```
HEADING = "Status"
ISSUE = 34
BUILT = False
STATE_TEXT = "This tab is not built."
ISSUE_TEXT = f"Issue #{ISSUE} carries the build-out."
```

Two frontends draw that one view model. `EmptyTabsMixin` in `src/gui/main_tabs/empty_tabs.py` builds the Qt tab, and `src/gui/web/system_status_tab.js` registers a panel with the Electron shell's panel host. `src.core.desktop_bridge` serves the model under `system_status_tab.state`.

## Half one: the Emitter Network

One plain function is the whole connection point. A call site says what it expected and what it actually saw, and the record goes to the sink. It is a function rather than a bus subscription, so a call site needs no reference to anything, and with no sink installed the call is a dictionary lookup and a return.

`src/core/signal_contract.py` — `emit`

```
def emit(
    name: str,
    actual: Any,
    expected: Any = None,
    ok: Optional[bool] = None,
    context: Optional[dict] = None,
    every: float = 0.0,
    instance: Optional[str] = None,
    module: Optional[str] = None,
    duration: Optional[float] = None,
) -> Optional[Signal]:
```

A satisfied expectation records the same as a violated one. That is the design point. In a log that only records failures, a call site that never ran and a call site that always passed look identical. Here they do not.

Three constants bound what the sink holds: how many rows buffer before a write, how many stay in memory, and the size at which the file rotates.

`src/core/signal_contract.py` — the sink's bounds

```
DEFAULT_FLUSH_EVERY = 200
"""Rows buffered in memory before `flush()` writes them to disk."""

RETAIN_ROWS = 350_000
"""Signal records kept in memory at once; older ones are evicted from memory but remain
in the file on disk.
"""

MAX_FILE_BYTES = 50 * 1024 * 1024
"""Byte size at which the sink's file rotates to `.1`."""
```

The function stays one; the sinks are two. A thread that calls `route_thread` sends its own emits to another sink of the same class until it calls `unroute_thread`, and `get_sink` answers per thread. The Simulator routes each run's worker thread to its own sink under the sim bucket, so a run's rows never rotate the process sink; every other thread's emits reach the process sink as before. See simulator.md.

`src/core/signal_contract.py` — the routing

```
def route_thread(sink: SignalSink) -> None:
    """Send every `emit` raised on the calling thread to `sink` until `unroute_thread`."""
```

### Tracked emitters

A second module declares every emitter. A contract names a topic, the fields a payload must carry, the fields it may carry, and the vocabulary a field is allowed to draw from.

`src/core/emit_contracts.py` — the gate-decision contract

```
EmitContract(
    topic="bot.gate_decision",
    required=("symbol",),
    optional=(
        "scrum_armed",
        "fold_armed",
        "scrum_blockers",
        "fold_blockers",
        "scrum_fixture",
        "fold_fixture",
    ),
    description="Gate evaluation snapshot at fire time.",
),
```

`EmitObserver` records what actually arrived and reports each violation in one of three classes: a topic that never fired, a required field that was missing, and a value outside its vocabulary.

| Topic | Required | Notes |
|---|---|---|
| `trade.filled` | side, amount, price | The trade kind is `type` here; the persisted log row calls it `action` |
| `bot.gate_decision` | symbol | Scrum and fold arm state, blockers and fixture at fire time |
| `pnl.event` | none | Realised, unrealised, symbol |
| `ta.voting` | none | Voting-engine snapshot |

The note on the first row is the reason the module exists. A consumer read one key from a payload that carried another. Both sides succeeded, the consumer got nothing, and a coverage report read zero while hundreds of trades flowed past it. A key-name mismatch is invisible to the producer and to the consumer; only a declared contract makes it visible.

The observer already has a formatter that turns its counts into operator-facing lines. Its one caller was the Nuclear fleet controller writing a run report; the Simulator rebuild deleted that file, so nothing calls it and no screen reads it.

`src/core/emit_contracts.py` — `format_observer_lines`, what it returns

```
d = obs.to_dict()
out = [
    f"Emit contracts: {d['topics_seen']}/{d['topics_declared']} "
    "topic(s) observed",
]
```

## Half two: the Watchdog

The Watchdog runs outside the process that can crash. It launches the application as a subprocess, tees both output streams to the console and to its own log, and polls a heartbeat file. A heartbeat whose modification time stops advancing past the stall threshold counts as a hang. When the child dies the Watchdog writes a post-mortem, which is the record an in-process hook cannot produce for a hard abort or an operating-system kill.

`acervator_watchdog.py` — the stall threshold and the poll

```
DEFAULT_STALL_SECONDS = 60  # must match or exceed Acervator's longest sync call (CCXT: ~15s typical, 30s timeout)
HEARTBEAT_POLL_INTERVAL = 2.0
```

Every post-mortem is a bundle of logs, and bundles once grew without limit. Three constants bound them now. Two govern the rotation and the third only warns.

`acervator_watchdog.py` — the rotation bounds

```
POSTMORTEM_KEEP_LATEST: int = 20  # most-recent N bundles preserved
POSTMORTEM_MAX_AGE_DAYS: int = 30  # anything older is dropped
POSTMORTEM_SIZE_WARN_BYTES: int = 5 * 1024 * 1024 * 1024  # 5 GB warning threshold
```

The Watchdog writes to files under the log root, and nothing in the application reaches into it. Surviving the application dying is the whole point of it.

## Until the tab lands

The Console tab's lower pane is the Emitter Network readout. It renders the same sink records, capped at the newest 200 per drain.

## 2026-09-08 08:17 - #34 - what the closed issues landed

The manual marks this screen System Status Tab (To Be Built) and it is a skeleton today.

This tab consists of two distinct but closely related parts. The Emitter Network is an embedded system of data activity detectors intended to allow for detailed subsystem performance monitoring. The Watchdog is the raw signal capture for the Emitter Network’s output.

The tab is now called Status. It sits ninth on the bar, on the gold ground with red text.

The screen is not built. The tab row now carries a System Status skeleton, which draws its name, one sentence saying it is not built, and the issue that owns it. Issue #34 carries the build-out. Both halves run today, and the Console tab shows the first of them.

`src/gui/main_tabs/system_status_tab_surface.py` — the whole empty state

```
HEADING = "Status"
ISSUE = 34
BUILT = False
STATE_TEXT = "This tab is not built."
ISSUE_TEXT = f"Issue #{ISSUE} carries the build-out."
```

The Emitter Network is one plain function and one sink. A call site says what it expected and what it actually saw, and a satisfied expectation is recorded the same as a violated one, which keeps a call site that never ran distinct from one that always passed.

`src/core/signal_contract.py` — `emit`

```
def emit(
    name: str,
    actual: Any,
    expected: Any = None,
    ok: Optional[bool] = None,
    context: Optional[dict] = None,
    every: float = 0.0,
    instance: Optional[str] = None,
    module: Optional[str] = None,
    duration: Optional[float] = None,
) -> Optional[Signal]:
```

A second module declares every emitter. A contract names a topic's required fields and its vocabulary, and the observer reports a topic that never fired, a missing field or a value outside that vocabulary.

`src/core/emit_contracts.py` — the gate-decision contract

```
EmitContract(
    topic="bot.gate_decision",
    required=("symbol",),
    optional=(
        "scrum_armed",
        "fold_armed",
        "scrum_blockers",
        "fold_blockers",
        "scrum_fixture",
        "fold_fixture",
    ),
    description="Gate evaluation snapshot at fire time.",
),
```

The Watchdog is `acervator_watchdog.py` at the repository root. It launches the application as a child process, tees both output streams to its own log, polls the heartbeat file, and writes a post-mortem after the child dies.

`acervator_watchdog.py` — the stall threshold and the poll

```
DEFAULT_STALL_SECONDS = 60  # must match or exceed Acervator's longest sync call (CCXT: ~15s typical, 30s timeout)
HEARTBEAT_POLL_INTERVAL = 2.0
```

### Post-mortem bundle rotation

The Watchdog writes a post-mortem bundle every time the application dies. Each bundle copies the runner's console log, the crash log, the fault-handler log and a thread dump, which runs to hundreds of megabytes. Per-run log rotation capped the size of one run's logs and capped nothing about the number of bundles, so weeks of restarts grew without limit until the log directory took the host down.

`acervator_watchdog.py` — `prune_postmortem_bundles`

```
def prune_postmortem_bundles(
    keep_latest: int = POSTMORTEM_KEEP_LATEST,
    max_age_days: int = POSTMORTEM_MAX_AGE_DAYS,
    log_dir: Path | None = None,
) -> tuple[int, int]:
```

It keeps the most recent bundles, drops anything past a maximum age whatever its position, and runs at two sites: once at Watchdog startup, which clears what earlier runs left behind, and once after each post-mortem is written, which holds the ceiling during a long session.

`acervator_watchdog.py` — the three constants

```
POSTMORTEM_KEEP_LATEST: int = 20  # most-recent N bundles preserved
POSTMORTEM_MAX_AGE_DAYS: int = 30  # anything older is dropped
POSTMORTEM_SIZE_WARN_BYTES: int = 5 * 1024 * 1024 * 1024  # 5 GB warning threshold
```

The third is a warning threshold read by `report_log_dir_footprint` and it deletes nothing, so two constants govern the rotation and one reports on it. The only other inputs are the log directory itself, derived from the home directory, and the literal bundle-name prefix inside the function. Neither is a rotation constant.

No test in this repository references any of the three names. The cap runs and nothing holds it in place.

### 2026-09-09 14:30 - #34 - the tab draws the emitter network by subsystem and by tab

The screen is built. The Status tab no longer draws the empty state; it draws the emitter network, divided two ways.

The top of the screen names the feed: the file the sink writes, how many records it has taken this run, how many are still in memory, how many were evicted and how many were dropped. Under it, one strip of totals across the whole network. With no sink in the process the feed line reads one sentence instead, and every count falls to zero.

`src/gui/main_tabs/system_status_tab_surface.py` — the tab a subsystem's emitters serve

```
TAB_BY_SUBSYSTEM = {
    "apitest": ENGINE_GROUP,
    "bot": ENGINE_GROUP,
    "charts": "Charts",
    "console": "Console",
    "exchange": "Live",
    "extractor": ENGINE_GROUP,
    "fleet": "Sim",
    "gui": ENGINE_GROUP,
    "history": "History",
    "instance": ENGINE_GROUP,
    "sim": "Sim",
    "swarm": "Swarm",
    "ta": ENGINE_GROUP,
    "tick": ENGINE_GROUP,
    "topology": ENGINE_GROUP,
    "trading": "Live",
    "ytd": ENGINE_GROUP,
}
```

**By subsystem** is one panel per prefix, seventeen of them. Each panel carries a light, the tab it serves, six counts, and a table of that subsystem's own emitters with the cadence each one declares, how many times it fired, how many of its retained records failed, and the last value it reported.

**By tab** is one row per tab on the bar, ten of them, and an eleventh named Engine for the emitters that serve no single screen. A row names the subsystems it holds and adds their counts together. Paper, Inspector, Accumulation and Status hold no subsystem at all, and each says `no emitters` rather than drawing a light. That absence is the reading, not a gap.

### Green, Yellow, and why Red cannot show

Two lights, and the screen says so in its own legend.

`src/core/signal_contract.py` — the two states the report returns

```
HEALTH_STATES = (HEALTH_GREEN, HEALTH_YELLOW)
"""The states `subsystem_health` returns; `HEALTH_RED` is not one of them."""
```

Green means every always-on emitter the subsystem declares has fired and no retained record of it failed. Yellow means a record failed, or an always-on emitter never fired at all. A subsystem that has recorded nothing carries no light and reads `no records this run`, which keeps a quiet subsystem separate from a healthy one.

Red waits on issue #14. A hung subsystem sends nothing, so silence is the only sign of it, and silence can only be read against a rhythm each emitter is expected to keep. No emitter declares one, so nothing returns Red and no light on this screen can show it.

### What the log record carries

The persisted row now names the subsystem it belongs to, so the report and the log divide the network the same way.

`src/core/signal_contract.py` — `Signal.to_json`

```
        payload = {
            "ts": self.ts,
            "seq": self.seq,
            "module": self.module,
            "name": self.name,
            "subsystem": subsystem_of(self.name),
            "kind": self.kind,
```

`subsystem_of` is the one function that splits a name, and `SignalSink.by_subsystem` calls it too, so the row on disk and the panel on screen cannot disagree.

### Measured on this build

The registry declares 78 emitters across 17 subsystems. The issue body's figure of 70 is older than the registry.

```
declared emitters          78
  always-on                16
  toggle                   62
subsystems                 17
no call site in src/       27    fleet 8, sim 14, ta 2, ytd 3
```

The twenty-seven with no call site can never fire, so their subsystems read silent whatever the platform does. Two of them are always-on, which is why Sim cannot reach Green today. Those emitters belong to the Simulator rebuild and the emitter network item, not to this screen.

### What is still open

The Console tab keeps its Emitter Network pane; moving it here is not done. The Watchdog is not on this screen yet. Both are named in issue #34 and neither is built.

Back to the subsystem index.

### Where the Status tab draws

The window builds this panel under both builds. `SystemStatusTabMixin` appends it with no variant seam, because no Qt widget ever drew this tab. The two builds draw the same picture at every width.

```
class            SystemStatusReactPanel
accessible name  Status
module loaded    system_status_tab
style sheet      system_status_tab.css, 25 rules
drawn            17 subsystems, 11 tab groups, 78 emitter rows
```

### 2026-09-20 06:18 - #34 - the register equals the code

The register in `src/core/signal_contract.py` now names every emitter the code calls, and only those. Every name under `src/` that reaches `emit` has an entry with its kind, and every entry has a call site. The tab map lives in the same module.

`src/core/signal_contract.py` - one entry per group, with its reason

```
_ALWAYS_ON_PINS: dict[str, str] = {
    "charts.indicators.drawn": (
        "the same `fetch_chart_data` pass calls `set_candles` on every fetch "
        "that returns candles"
    ),
```

```
_TOGGLE_PINS: dict[str, str] = {
    "inspector.ata.scan_pressed": "the operator presses Scan Now or Scan All",
```

An entry is a name and one line. The line is the reason the kind was read at the call site, in the form the retired roster used. The two groups keep their kinds: a loop or a timer reaches an always-on pin on every pass; a toggle pin needs a trigger, and silence from it is normal. `CADENCE_BY_NAME` is built from the two groups as before.

### The tab map in the register

The tab a subsystem serves is read from `TAB_BY_SUBSYSTEM` and `tab_of` in `src/core/signal_contract.py`, beside `subsystem_of`. The map names `inspector` to Inspector and `status` to Status. The surface imports the map and the function; it holds no copy.

`src/core/signal_contract.py` - `tab_of`

```
def tab_of(subsystem: str) -> str:
    """Return the tab `subsystem`'s emitters serve, or `ENGINE_GROUP` when none does."""
    return TAB_BY_SUBSYSTEM.get(subsystem, ENGINE_GROUP)
```

`src/gui/main_tabs/system_status_tab_surface.py` - the import

```
from src.core.signal_contract import (
    ENGINE_GROUP,
    HEALTH_GREEN,
    HEALTH_STATES,
    HEALTH_YELLOW,
    TAB_BY_SUBSYSTEM,
    get_sink,
    subsystem_health,
    tab_of,
)
```

### What entered and what left

Thirty-six names entered: 7 `charts.*`, 15 `inspector.*`, 14 `sim.*`. One of them, `charts.indicators.drawn`, is always-on: the 2000 ms dashboard timer schedules `fetch_chart_data`, and that pass calls `set_candles` on every fetch that returns candles. The other thirty-five need a press, a pointer, a call, or a run the operator starts.

Twenty-seven entries left, each with no call site under `src/`: every `fleet.03.*` entry (8), every `sim.06.*` entry (14), `ta.07.001` and `ta.07.002`, and every `ytd.10.*` entry (3). Two of them were always-on, `sim.06.011.postcondition.price_chart.fed` and `sim.06.012.postcondition.gate_status.rendered`, and they held the Sim panel at Yellow with

`always-on silent 2`.

### Overtaken sentences on this page

Each sentence below stands as written above. The sentence under it is the current reading.

> `src/gui/main_tabs/system_status_tab_surface.py` — *the tab a subsystem's emitters serve*

The map is in `src/core/signal_contract.py`, as `TAB_BY_SUBSYSTEM`, with `inspector` and `status` added.

> ***By subsystem** is one panel per prefix, seventeen of them.*

Sixteen. The `fleet` and `ytd` prefixes hold no entry after this change, and `inspector` holds fifteen.

> *Paper, Inspector, Accumulation and Status hold no subsystem at all, and each says `no emitters` rather than drawing a light.*

Inspector holds `inspector`. Paper, Accumulation and Status still say `no emitters`.

> *The registry declares 78 emitters across 17 subsystems.*

The register declares 87 emitters across 16 subsystems.

> *The twenty-seven with no call site can never fire, so their subsystems read silent whatever the platform does. Two of them are always-on, which is why Sim cannot reach Green today.*

Those twenty-seven are gone. No entry lacks a call site, and the Sim panel's `always-on silent` reads 0.

> *drawn 17 subsystems, 11 tab groups, 78 emitter rows*

Drawn: 16 subsystems, 11 tab groups, 87 emitter rows.

### Measured on this change

Read off the running program in both variants, on a scratch home with every socket but loopback refused, the tab shown, the page read at the widget.

```
declared emitters          87
  always-on                15
  toggle                   72
subsystems                 16
names in code not declared  0
entries with no call site   0
drawn                      16 subsystems, 11 tab groups, 87 emitter rows
Inspector tab row          inspector
Sim panel always-on silent 0
```

Before the change the same reading gave 17 subsystems, 80 rows on a page where two undeclared names had already fired at build, `always-on silent 2` on Sim, and `no emitters` on the Inspector row. One Scan Now on the loopback stand-in then raised an `inspector` panel with 11 undeclared rows under Engine. After the change the same press fills the Inspector panel: 15 declared, 11 fired, 0 undeclared, and the panel sits under the Inspector row.

The page still draws once, when the window builds. A Status tab opened after a scan shows what the sink held at build. The redraw is a later unit of #34.

### 2026-09-23 - #34 - the record carries its kind, its budget and its tab

`SignalSink.emit` in `src/core/signal_contract.py` stamps three more fields on every record it builds. `Signal.to_json` writes them to the line on disk. `read_records` restores them. A line on `session.jsonl` is eighteen fields. A reader can now judge a silence from the line alone. Before this change a reader had to import the register of the build that wrote the line.

| field | what it holds | the function that gives it |
|---|---|---|
| `cadence` | `always_on` or `toggle`, empty for a name the register does not hold | `cadence_of` |
| `budget_s` | the seconds an always-on emitter may stay silent, empty for every other emitter | `always_on_stale_after` |
| `tab` | the tab the emitter's subsystem serves | `tab_of` |

`src/core/signal_contract.py` - `SignalSink.emit`, the stamp

```
                    _cadence, _tab = self._declared_for(_name)
                    sig = Signal(
                        cadence=_cadence,
                        budget_s=(
                            always_on_stale_after(_window)
                            if _cadence == CADENCE_ALWAYS_ON
                            else None
                        ),
                        tab=_tab,
                    )
```

`_window` is the widest `every=` window the sink holds for that emitter's identity. An emitter with no window keeps the flat budget, `ALWAYS_ON_STALE_AFTER`, ten seconds. An emitter with a window keeps `ALWAYS_ON_SLACK` times that window, which is 2.11 times. A toggle keeps no budget. `_declared_for` reads the register once per name and keeps the answer, so the register is not walked per record.

### What a restored record answers

`read_records` reads each of the three through its own check: `_as_cadence` admits only a member of `CADENCE_CATEGORIES`, `_as_budget` admits only a finite number above zero, and `_as_tab` admits only a string with text in it. A value outside a check restores empty and the record is kept, the way a bad `dt` or a bad `duration` is kept today. A line written before this change carries none of the three, so all three restore empty and every other field restores as before.

`src/core/signal_contract.py` - `read_records`, the restore

```
                            cadence=_as_cadence(d.get("cadence")),
                            budget_s=_as_budget(d.get("budget_s")),
                            tab=_as_tab(d.get("tab")),
```

### Overtaken by the record's three new fields

The sentence below stands as written above. The sentence under it is the current reading.

> *The persisted row now names the subsystem it belongs to, so the report and the log divide the network the same way.*

The row names the subsystem, the emitter's kind, the emitter's budget in seconds and the tab the subsystem serves. Four of the row's eighteen fields come from the register rather than from the call, and the `Signal.to_json` block above carries three more keys after `context`.

### Read on this change

Read off the running program on a scratch home, with every socket but loopback refused, and on a read-only copy of a rotated file from the operator's own log root.

```
a line on session.jsonl          18 fields, 25 of 25 lines
a line on session.digest.jsonl   19 fields, the three plus folded
an always-on emitter, no window  ta.07.003.postcondition.computed  budget_s 10.0
an always-on emitter, 30 s window console.14.001.invariant.records_rendered
                                                          budget_s 63.34459459459459
a toggle                         trading.12.001.postcondition.tab_assembled
                                                          budget_s empty, tab Live
the operator's rotated file      92,500 lines, 92,500 records restored,
                                 all three empty on every record
```

Before the change the same reading gave fifteen fields on every line and no `cadence`, `budget_s` or `tab` on any of them.

The panels do not read the three fields yet. `subsystem_health` still counts silence since launch, so the light is still Green or Yellow and never Red. Reading a budget off the record is a later unit of #34.

### 2026-09-23 - #34 - the qt bundle ships the shell renderer

This tab draws through a React page, and the page host sits in the renderer folder beside the application. A qt build shipped no copy of that folder. The tab then raised a missing shell asset and drew an empty pane. Every build now ships the folder.

`tools/spec_common.py` - the pair every build copies

```
def renderer_candidate(project_root: str) -> tuple[str, str]:
    """The (source, destination) pair of ``SHELL_RENDERER``, which every variant ships."""
```

`shell_candidates` now holds what the Electron shell alone needs: the three shell files and the Electron runtime. A React build ships those, and no other variant does.

### The overtaken sentence about the bundle

The sentence below stands as written above. The sentence under it is the current reading.

> *The window builds this panel under both builds.*

The window builds this panel under both builds. Before this change only a React bundle carried the page host, so only a React bundle could draw the panel. Both bundles carry it now.

### Read off the launched bundles

Both variants were built from one commit and launched with no network: a scratch home, an application container with no capability, and the name resolver mapped away. The Status tab was pressed through its accessibility action. The page was read off the accessibility tree.

```
qt bundle before     no desktop folder in the bundle
                     1 missing-asset line in system.log
                     0 panels, 0 emitter rows, empty pane

qt bundle after      desktop/renderer, 8 files, 38,592 bytes
                     0 missing-asset lines in system.log
                     16 panels, 87 emitter rows, 11 tab rows

react bundle after   desktop, 11 files: 3 shell files and the renderer
```

A build that names a renderer folder which is not on disk ships no renderer, and its Status tab raises the missing asset again. That is how the reading is known to be able to fail.

# Settings

Reference. `SettingsDialog` in `src/gui/settings_dialog.py`. The two pages below are the ones Part 3 names.

## The pages

One method adds them, in one order.

`src/gui/settings_dialog.py` — `SettingsDialog._setup_ui`

```
tabs.addTab(self._create_user_tab(), "User")
tabs.addTab(self._create_exchange_tab(), "Exchanges")
tabs.addTab(self._create_trading_tab(), "Trading")
tabs.addTab(self._create_ta_tab(), "TA Indicators")
tabs.addTab(self._create_phantom_tab(), "Phantom Bots")
tabs.addTab(self._create_theme_tab(), "Theme")
tabs.addTab(self._create_sound_tab(), "Sound")
tabs.addTab(self._create_sms_tab(), "SMS")
tabs.addTab(self._create_ai_monitor_tab(), "AI Monitor")
```

### The count is a declaration, not a number in this manual

That block is a reading, and every removal has moved it. Eleven became ten, ten became nine, and the next removal will move it again. The rule replaces the figure. The page count and the control count are two declarations, and the answer is whatever those two hold on the day you read them.

`src/gui/main_tabs/settings_dialog_surface.py` — the two the count comes from

```
TAB_TITLES = (
    USER_TAB, EXCHANGE_TAB, TRADING_TAB, TA_TAB, PHANTOM_TAB,
    THEME_TAB, SOUND_TAB, SMS_TAB, AI_TAB,
)

CONTROL_SPECS: tuple[dict, ...] = (
    {"tab": USER_TAB, "group": "", "name": "username", "kind": LINE, ...},
    ...
)
```

Both builds read the same two, so a page or a control that leaves a declaration leaves the Qt window and the shell together. Count the entries in the first for the pages and the entries in the second for the controls. Every dated section further down this page records what one removal took; none of them owns the count.

Read on 13 September 2026, in one run, off the built Qt window and off the payload the shell draws:

| | the declaration | the Qt window | the shell payload |
|---|---|---|---|
| Pages | 9 | 9 tabs | 9 |
| Controls | 66 | not counted whole | 66 specs |
| Tick boxes | 23 | 23 drawn | 23 |
| Drop-downs | 6 | 6 drawn | 6 |
| Radio buttons | 0 | 0 drawn | 0 |

The Qt window was read by walking its own widget tree, so the tick-box and drop-down rows are drawn counts and the whole-dialog figure of 66 is the declaration's. The browser page was rendered as well and reported 0 radio inputs, against 23 checkbox inputs on the same reading.

## What each page persists

`_save` writes and `_load_current` reads back. A control neither method touches keeps its build-time default for the life of the process and reaches no engine.

| Page | Controls | `_save` writes | `_load_current` restores |
|---|---|---|---|
| User | 1 | the row | the row |
| Exchanges | 6 plus 3 buttons | on Add and Remove | the configured list |
| Trading | 6 | all six | four of six |
| Profit Folding | 11 | the whole group | `active` alone |
| TA Indicators | 12 | nothing | nothing |
| Phantom Bots | 13 | nothing | nothing |
| Theme | 6 | all six | the theme and the accent |
| Logging | 6 | the whole group | nothing |
| Sound | 10 plus 6 buttons | nothing | nothing |
| SMS | 17 | nothing | nothing |
| AI Monitor | 7 | all seven | all seven |

`AppSettings` in `src/core/settings.py` is the whole schema, and a key it does not declare is refused outright rather than stored somewhere unread.

`src/core/settings.py` — `SettingsManager.set`

```
def set(self, key: str, value: Any) -> None:
    """Set the ``AppSettings`` attribute *key* and call ``_save``.

    A *key* that ``AppSettings`` does not declare raises ``KeyError``.
    """
    with self._lock:
        if not hasattr(self._settings, key):
            raise KeyError(f"Unknown setting: {key}")
```

Four pages hold controls `_save` never reads, and the schema declares no field any of them could land in: TA Indicators, Phantom Bots, Sound and SMS. Issue #423 carries all four and counts the controls.

The Exchanges page is the exception to the whole pattern. `_add_exchange` and `_remove_exchange` write through the settings manager as the operator uses them, rather than waiting for Save.

The Sound page reaches its engine by one other route. The slider handler builds a fresh configuration from every checkbox, pushes it in, and clears the sample cache, because each sample bakes its volume at synthesis time.

`src/gui/settings_dialog.py` — `_on_sfx_volume_changed`

```
se = get_sound_engine()
new_cfg = SoundConfig(
    enabled=self._sound_enabled.isChecked(),
    buy_sound=self._sound_buy.isChecked(),
    sell_sound=self._sound_sell.isChecked(),
    error_sound=self._sound_error.isChecked(),
    bot_state_sound=self._sound_state.isChecked(),
    fire_sound=self._sound_fire.isChecked(),
    track_sound=self._sound_track.isChecked(),
    profit_sound=self._sound_profit.isChecked(),
    drip_sound=self._sound_drip.isChecked(),
    volume=v / 100.0,
)
```

That path runs when the slider moves or a test button plays a sample, and at no other time.

The parent section, 08-tabs.md, carries the figure for each page and names every control on it.

## Settings, User page

The whole page is one form row. No credential field, no licence field and no key import.

`src/gui/settings_dialog.py` — `_create_user_tab`

```
def _create_user_tab(self) -> QWidget:
    w = QWidget()
    form = QFormLayout(w)
    self._username = QLineEdit()
    form.addRow("Username:", self._username)
    return w
```

`_load_current` reads the stored value and `_save` writes it back through the settings manager, which persists the schema under a home-relative directory as a TOML file, or as JSON when no TOML writer is available. One lock guards every read and write, shared across every manager instance.

### What a credential store would rest on

Both halves already exist, and the User page reads neither.

The first is encryption. A passphrase and a random salt derive a 256-bit key, and each token carries its own salt and nonce.

`src/core/encryption.py` — `derive_key`

```
def derive_key(passphrase: str, salt: bytes) -> bytes:
    """Derive a 256-bit AES key from *passphrase* + *salt* via PBKDF2."""
```

`KeyringManager` holds the passphrase in the operating system's credential store. Without the `cryptography` package a fallback keystream with an HMAC-SHA256 tag takes over.

The second is the hardware-key path. A credential list encrypts against a USB volume's serial and writes at that mount point, so the file decrypts on that stick and nowhere else.

`src/core/usb_auth.py` — `write_auth_file`

```
def write_auth_file(
    usb_path: Path,
    volume_serial: str,
    credentials: list[dict],
) -> Path:
    """Encrypt *credentials* under ``_derive_usb_key`` and write ``AUTH_FILENAME``.

    Each dict carries ``exchange_id``, ``api_key``, ``api_secret`` and
    ``passphrase``, and the written path under *usb_path* is returned.
    """
```

`find_auth_volume` matches the volume serial that `read_auth_file` needs to derive the same key.

## Settings, Exchanges page

`_create_exchange_tab` lists the configured exchanges and offers a form to add one: the exchange picker, an API key, an API secret, and a passphrase field that appears only when the exchange needs one.

The picker lists what the connector layer can actually reach.

`src/exchange/ccxt_connector.py` — `SUPPORTED_EXCHANGES`, the first six

```
SUPPORTED_EXCHANGES: dict[str, str] = {
    "binance": "binance",
    "coinbase": "coinbase",
    "kraken": "kraken",
    "kucoin": "kucoin",
    "bybit": "bybit",
    "okx": "okx",
```

Three venues need a passphrase, and the field appears for those alone.

`src/exchange/ccxt_connector.py` — `PASSPHRASE_EXCHANGES`

```
PASSPHRASE_EXCHANGES: set[str] = {
    "kucoin",
    "okx",
    "bitget",
}
```

The key field accepts a Coinbase CDP key string as well as a plain key, and the secret field accepts a PEM elliptic-curve private key. Escaped newlines inside a pasted PEM convert on the way in.

Three handlers sit behind the buttons.

| Handler | What it does |
|---|---|
| `_test_api_connection` | Authenticates against the venue and reports through `_set_feedback` |
| `_add_exchange` | Runs that test first and stores nothing when it fails |
| `_remove_exchange` | Drops the selected entry |

### The stock wing

The same page in the stock wing shows a banner and disables Add and Test.

`src/gui/settings_dialog.py` — `_create_exchange_tab`

```
_banner = QLabel(
    "<b>Stock Wing:</b> equity-broker integration is "
    "queued — no live brokers are wired up yet. The "
    "list below shows the planned brokers; Add / Test "
    "are disabled until the broker connectors ship. "
    "Use the Crypto Wing for active trading today."
)
```

`EQUITY_EXCHANGE_IDS` fills the list with the planned brokers, each labelled as planned. Crypto is the wing that trades today.

### What adding one does

The new exchange routes to the crypto layer or the stock layer, and the method removes the placeholder tab if one stood there.

`src/gui/main_window.py` — `add_exchange_tab`

```
def add_exchange_tab(self, exchange_id: str, display_name: str) -> None:
    """Add exchange tab to the correct layer (crypto or stock)."""
    is_equity = self._is_equity_exchange(exchange_id)
```

The method emits a signal naming which layer the tab actually landed in, read back from both layer widgets rather than from the argument it was given.

## Bridge

Two methods serve this screen.

| Bridge method | Serves | Renderer module |
|---|---|---|
| `settings_dialog.state` | The dialog and every page on it | `settings_dialog.js` |
| `exchange_tab.state` | One venue's tab | `exchange_tab.js` |

## 2026-09-06 00:52 - #423 - the Settings dialog draws in React

The File menu opens this dialog, and which one it opens depends on the build. The React build draws every page below inside one embedded browser. The Qt build draws the widget tree the pages describe. One line in the running window picks the side.

`src/gui/main_window.py` — `_open_settings`

```
from .variant_surface import SETTINGS_DIALOG, surface_class

_cls = surface_class(SETTINGS_DIALOG)
dlg = _cls(self._settings, self._status_log, self, wing=_wing)
```

The React side is a subclass, not a rewrite. It replaces only the method that builds the widgets, so the loading and the saving below are the same code on both sides. Each named control gets a small holder that answers the same calls the widget answered, and the page reads and writes those holders.

`src/gui/react_settings_dialog.py` — `SettingsDialogReact._setup_ui`

```
self._model = surface.SettingsDialogModel(wing=self._wing)
self._model.build()
self._build_holders()
self._web = QWebEngineView(self)
self._web.setPage(SettingsDialogPage(self))
self._web.setHtml(dialog_html(self._theme_name()))
```

An edit on the page arrives as one line the host reads, and the holder takes it the way the control takes a stored value. A radio inside a group box clears the others, which is what `QGroupBox` does for the widget side.

`src/gui/react_settings_dialog.py` — `SettingsDialogReact.apply_edit`

```
self._holders[name].admit(asked.get("value"))
```

### Settings > User Tab

This needs to be built out to accept and contain individual end user credentials. This will also be where a license is entered or a keyfile is imported depending on how we design the authentication piece.

The dialog holds the pages below, added in one order.

`src/gui/settings_dialog.py` — `SettingsDialog._setup_ui`

```
tabs.addTab(self._create_user_tab(), "User")
tabs.addTab(self._create_exchange_tab(), "Exchanges")
tabs.addTab(self._create_trading_tab(), "Trading")
tabs.addTab(self._create_ta_tab(), "TA Indicators")
tabs.addTab(self._create_phantom_tab(), "Phantom Bots")
tabs.addTab(self._create_theme_tab(), "Theme")
tabs.addTab(self._create_sound_tab(), "Sound")
tabs.addTab(self._create_sms_tab(), "SMS")
tabs.addTab(self._create_ai_monitor_tab(), "AI Monitor")
```

`_save` and `_load_current` decide which of them persists, and each page says which under its own heading. Issue #423 carries the four pages whose controls Save never reads.

The User page itself is one form row, and the settings manager persists it. No credential field, no licence field and no key import.

`src/gui/settings_dialog.py` — `_create_user_tab`

```
def _create_user_tab(self) -> QWidget:
    w = QWidget()
    form = QFormLayout(w)
    self._username = QLineEdit()
    form.addRow("Username:", self._username)
    return w
```

Both halves a credential store would rest on already exist, and the page reads neither. `src/core/encryption.py` encrypts a secret with AES-256-GCM under a key derived from a passphrase and a random salt.

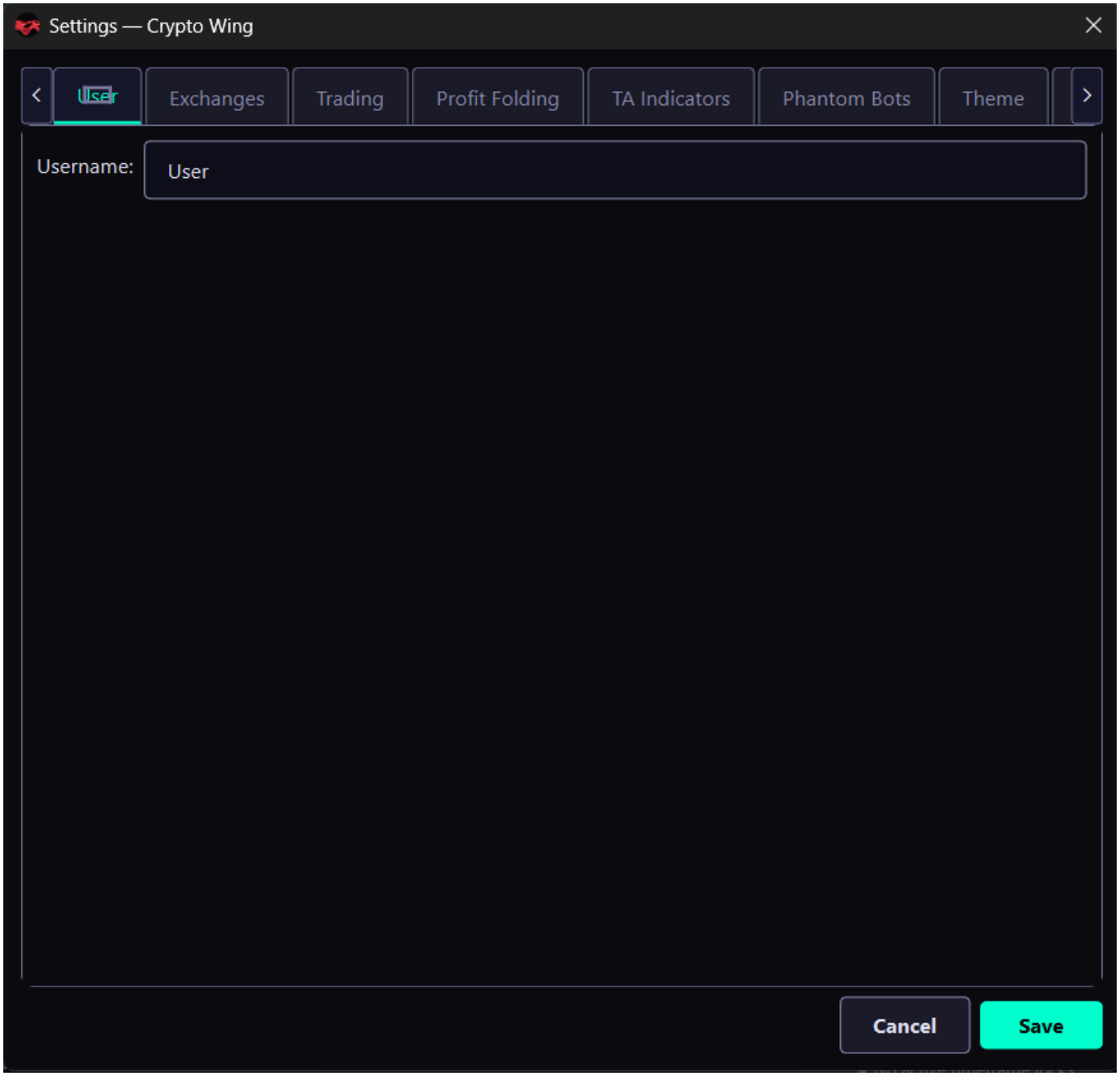
`src/core/encryption.py` — `derive_key`

```
def derive_key(passphrase: str, salt: bytes) -> bytes:
    """Derive a 256-bit AES key from *passphrase* + *salt* via PBKDF2."""
```

`src/core/usb_auth.py` writes an encrypted credential file keyed to a USB volume's serial, which decrypts on that stick and nowhere else.

`src/core/usb_auth.py` — `write_auth_file`

```
def write_auth_file(
    usb_path: Path,
    volume_serial: str,
    credentials: list[dict],
) -> Path:
```




Settings, the User page.

The title bar names the wing the dialog was opened for. The arrows at each end of the page row scroll it, because the eleven do not fit the dialog's minimum width. Cancel and Save close the dialog, and Save always closes it, reporting any group that failed to persist. The page itself carries one row, Username, and nothing else.

That one row is not cosmetic. The stored username keys the credential vault. Adding a venue on the next page builds a master phrase from that name, then encrypts the API key and the API secret with the phrase.

`src/gui/settings_dialog.py` — `_add_exchange`, the phrase and the encryption

```
master = f"qat_{self._sm.get('username', 'user')}_vault"
config.api_key_enc = encrypt(key, master)
config.api_secret_enc = encrypt(secret, master)
```

The figure shows a stored value, User. The declared default is an empty string, and the settings object always declares the field, so the fallback written into the phrase never fires. An install where the row was never filled encrypts under a phrase with an empty middle. Change the name on a machine that already holds credentials and the stored secrets stop opening, because the connect path rebuilds the same phrase from the new name. Issue #436 carries it.

`src/gui/main_window.py` — the decrypt side, rebuilding the same phrase

```
master = f"qat_{self._settings.get('username', 'user')}_vault"
api_key = decrypt(exch_config["api_key_enc"], master)
api_secret = decrypt(exch_config["api_secret_enc"], master)
```

Three sites write that phrase out as a literal, one to encrypt and two to decrypt, and a fourth declares the format none of them reads. The same issue carries that half.

`src/gui/main_tabs/settings_dialog_surface.py` — the declaration

```
MASTER_FORMAT = "qat_{username}_vault"
```

### Settings > Exchanges

The page lists the configured exchanges and adds one: the exchange picker, an API key, an API secret, and a passphrase field that appears only when the venue needs it. The picker fills from the connector layer, so it lists what that layer can reach.

`src/exchange/ccxt_connector.py` — `SUPPORTED_EXCHANGES`, the first six

```
SUPPORTED_EXCHANGES: dict[str, str] = {
    "binance": "binance",
    "coinbase": "coinbase",
    "kraken": "kraken",
    "kucoin": "kucoin",
    "bybit": "bybit",
    "okx": "okx",
```

Three venues need the passphrase field, and it appears for those alone.

`src/exchange/ccxt_connector.py` — `PASSPHRASE_EXCHANGES`

```
PASSPHRASE_EXCHANGES: set[str] = {
    "kucoin",
    "okx",
    "bitget",
}
```

The key field accepts a Coinbase CDP key string and the secret field a PEM elliptic-curve private key. The connection test runs before anything is stored, and adding one routes the new tab to the crypto or the stock layer.

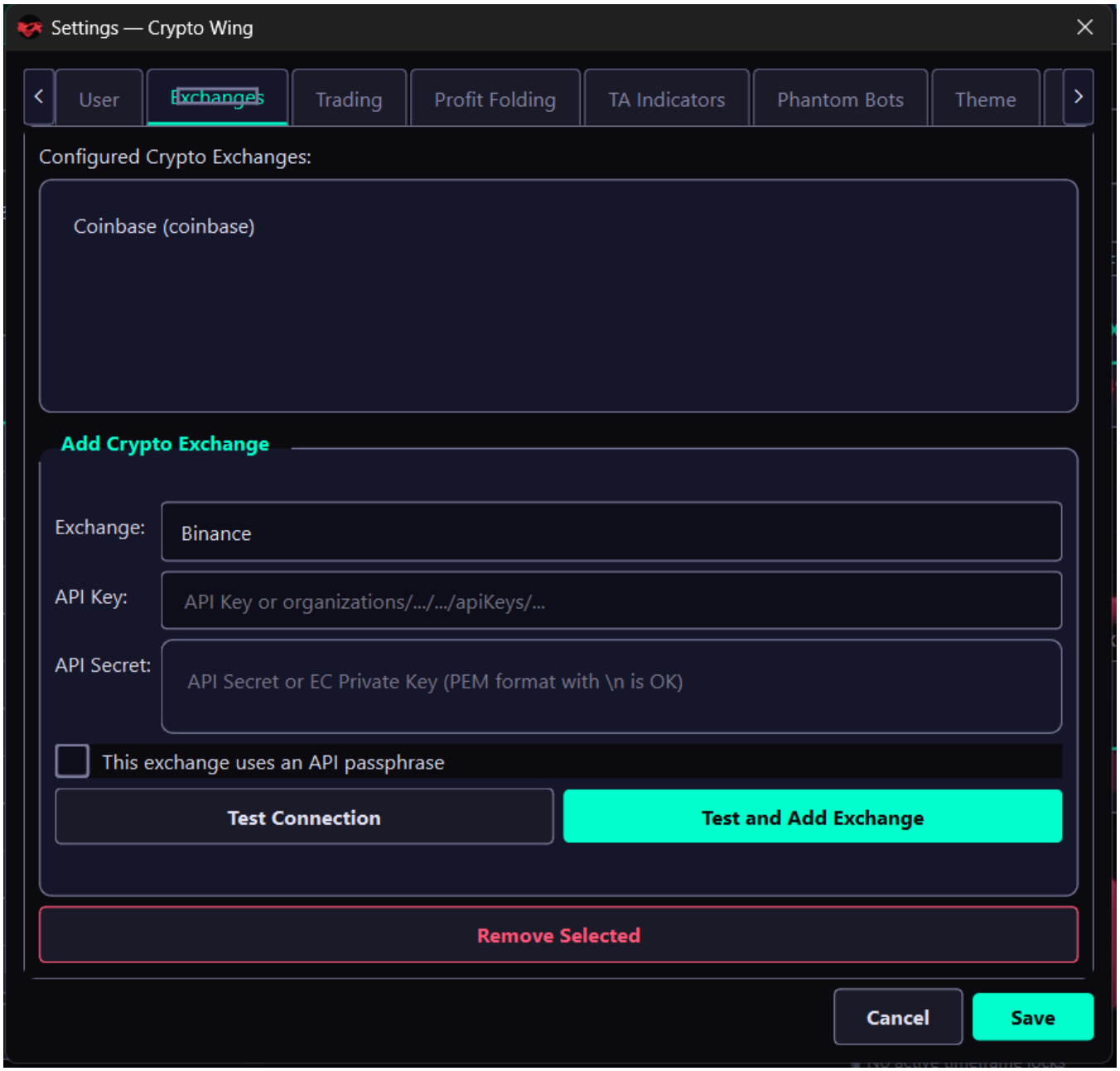
`src/gui/main_window.py` — `add_exchange_tab`

```
def add_exchange_tab(self, exchange_id: str, display_name: str) -> None:
    """Add exchange tab to the correct layer (crypto or stock)."""
    is_equity = self._is_equity_exchange(exchange_id)
```

The same page in the stock wing shows a banner and disables Add and Test, because no broker connector ships yet.

`src/gui/settings_dialog.py` — `_create_exchange_tab`

```
_banner = QLabel(
    "<b>Stock Wing:</b> equity-broker integration is "
    "queued — no live brokers are wired up yet. The "
    "list below shows the planned brokers; Add / Test "
    "are disabled until the broker connectors ship. "
    "Use the Crypto Wing for active trading today."
)
```



Settings, the Exchanges page.

Configured Crypto Exchanges lists what the manager returns for this wing, each entry naming its display name and its exchange id. The stock wing lists the equity ids instead and disables both buttons.

The Add Crypto Exchange group holds four rows. Exchange fills from the supported list. API Key takes a plain key or a Coinbase CDP key string, and its placeholder shows the CDP shape. API Secret takes a plain secret or a PEM elliptic-curve private key, and escaped newlines inside a pasted PEM convert on the way in. The passphrase checkbox appears for the three venues above.

The picker in the figure reads Binance, the first supported id in alphabetical order, and the passphrase box under it stays clear. Changing the picker sets that box for the three venues that need one and clears it for the rest. It also wipes the feedback line, so an answer about the last venue cannot stand as an answer about this one.

`src/gui/settings_dialog.py` — `_on_exchange_changed`

```
def _on_exchange_changed(self) -> None:
    eid = self._new_exchange.currentData()
    needs_pp = eid in self._passphrase_exchanges
    self._pp_check.setChecked(needs_pp)
    self._api_feedback.setText("")
```

The page hides the passphrase field until you tick the box, which is why the figure shows the box and no field under it. The box carries the state; the field only collects the value.

`src/gui/settings_dialog.py` — the field the box reveals

```
self._new_passphrase = QLineEdit()
self._new_passphrase.setEchoMode(QLineEdit.Password)
self._new_passphrase.setPlaceholderText(
    "Passphrase set when creating API key"
)
self._new_passphrase.setVisible(False)
```

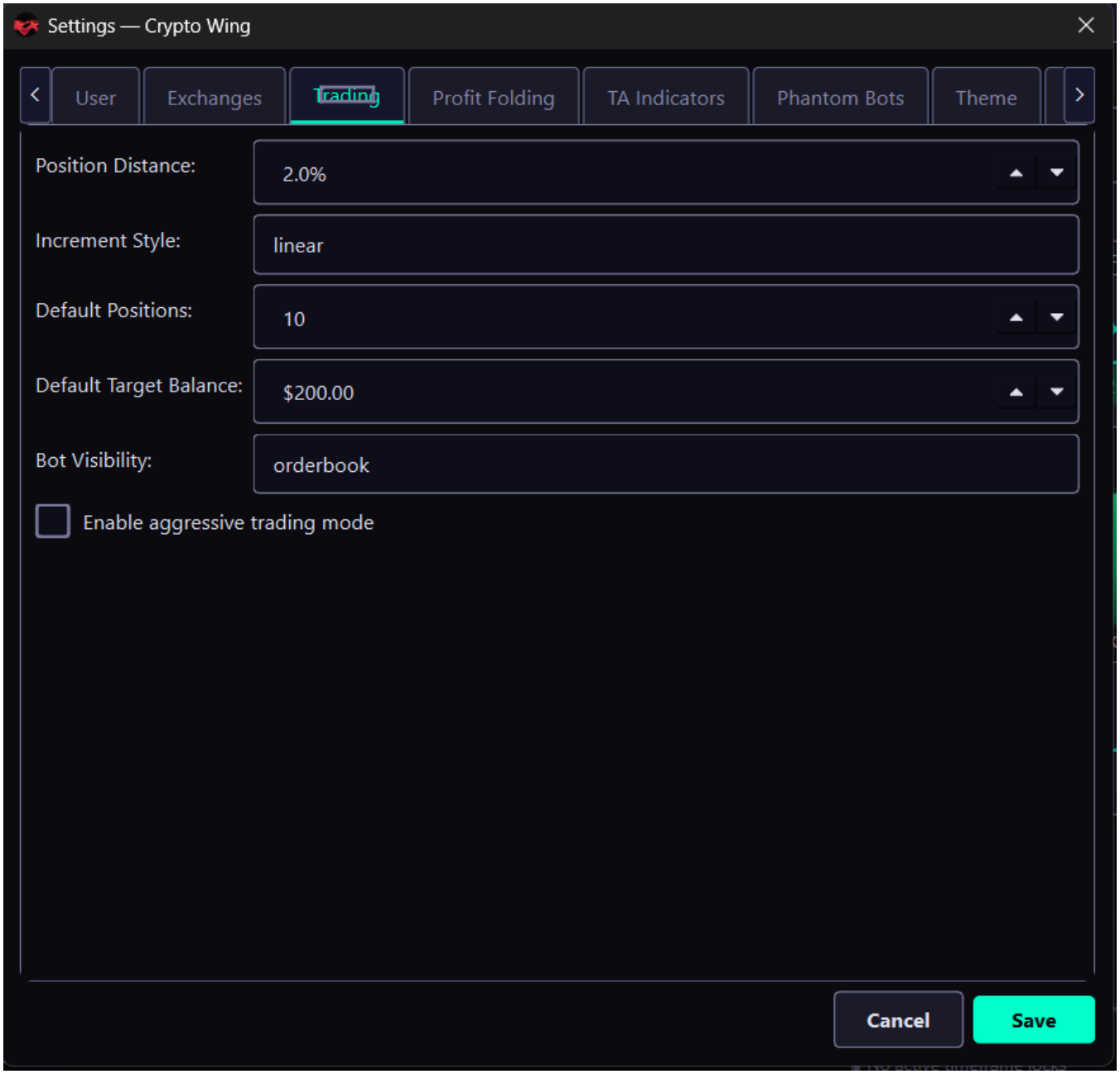
Test Connection runs the check and reports the answer. Test and Add Exchange runs the same check first and stores nothing when it fails. Remove Selected drops the highlighted entry.

Both buttons go dead for the length of a call, so a second click cannot start a second test. Test and Add returns on a failed check, which is what keeps a venue that did not answer out of the store.

`src/gui/settings_dialog.py` — `_add_exchange`, the guard before the store

```
if key and secret:
    result = self._test_api_connection()
    if result is None or not result.success:
        return
```

### Settings > Trading




Settings, the Trading page.

Six rows carry the defaults a new bot starts from, not a running bot's settings.

Three of those six rows are off the page. The descriptions below, and the code blocks under them, record what the page held before the removal. Each removal has its own dated section further down this page.

| the row | the dated section that records its removal |
|---|---|
| Position Distance | 2026-09-11 - Position Distance is off the Trading page |
| Increment Style | 2026-09-11 - Increment Style is off the Trading page |
| Default Positions | 2026-09-11 - Default Positions is off the Trading page |

The strip list quoted further down this section now says in its own words that it is a retirement record and not a set of settings.

`src/trading/container/config.py` — what the strip list says about itself

```
#: A retirement record, not a setting set. Each name is a setting removed from
#: the product, kept so a config stored before its removal still builds;
#: `_sanitize_deprecated_kwargs` drops each before `BotConfig.__init__`.
#: Add no name. Only `bulk_trading` is read, by `bot_config_kwargs`.
```

Position Distance - Sets the spacing a new bot puts between its positions. Key `position_distance_pct`, 1.0 % to 50.0 %, at 2.0 %.

The dialog stores the figure and reads it back the next time the page opens. No bot ever sees it. The bot factory strips the name before it builds a config, so the value cannot reach a bot by any route.

`src/gui/settings_dialog.py` — the Position Distance row

```
self._pos_distance = QDoubleSpinBox()
self._pos_distance.setRange(1.0, 50.0)
self._pos_distance.setSuffix("%")
self._pos_distance.setDecimals(1)
form.addRow("Position Distance:", self._pos_distance)
```

The strip list holds eleven names. This page contributes one of them and the Profit Folding page below contributes four more.

`src/trading/container/config.py` — `_DEPRECATED_KWARGS`

```
_DEPRECATED_KWARGS: frozenset = frozenset(
    {
        "bulk_trading",  # renamed to stack_mode
        "bulk_partial_on_return",
        "market_check_interval",
        "position_count",
        "position_distance_pct",
        "fold_mode",
        "fold_target",
        "fold_target_count",
        "profit_fold_pct",
        "distribute_target",
        "distribute_target_count",
    }
)
```

Increment Style - Chooses whether that spacing stays even or widens on a curve. Key `increment_style`, linear or logarithmic, at linear.

The dialog stores the choice and restores it on the next open. A bot takes its own increment style from the wizard rather than from this page.

`src/gui/settings_dialog.py` — the Increment Style row

```
self._increment_style = QComboBox()
self._increment_style.addItems(["linear", "logarithmic"])
form.addRow("Increment Style:", self._increment_style)
```

Default Positions - Sets how many positions a new bot opens with. Key `default_position_count`, 1 to 100, at 10.

The dialog stores the count and restores it. Nothing outside the dialog reads the name, so no bot opens with it.

`src/gui/settings_dialog.py` — the Default Positions row

```
self._default_positions = QSpinBox()
self._default_positions.setRange(1, 100)
form.addRow("Default Positions:", self._default_positions)
```

Default Target Balance - Sets the dollar figure the wizard's own Target Balance field opens at. Key `default_target_balance`, $1.00 to $1,000,000.00, at $200.00.

This is the one row on the page that reaches a new bot. The wizard's parameter page opens its Target Balance field at the stored figure.

`src/gui/bot_wizard.py` — the wizard field that reads the stored default

```
self._target_balance = QDoubleSpinBox()
self._target_balance.setRange(1.0, 1000000.0)
self._target_balance.setDecimals(2)
self._target_balance.setPrefix("$ ")
self._target_balance.setValue(defaults.get("default_target_balance", 200.0))
```

Bot Visibility - Chooses whether a new bot rests its orders on the book or tracks them inside the platform. Key `bot_visibility`, orderbook or internal, at orderbook.

Save writes the name and the load path never reads it. The box opens on orderbook whatever was stored, and a Save with no edit writes that first entry back over the stored value. A bot takes its visibility from the wizard, under a different name.

`src/gui/settings_dialog.py` — the Bot Visibility row

```
self._visibility = QComboBox()
self._visibility.addItems(["orderbook", "internal"])
form.addRow("Bot Visibility:", self._visibility)
```

Enable aggressive trading mode - Starts a new bot pricing for an immediate fill. Key `aggressive_trading`, a checkbox, off.

The same gap as the row above. Save writes it, the load path skips it, and the box opens clear. The wizard carries its own aggressive trading checkbox, and that is the one a new bot reads.

`src/gui/settings_dialog.py` — the aggressive trading row

```
self._aggressive = QCheckBox("Enable aggressive trading mode")
form.addRow(self._aggressive)
```

Save writes all six keys. The load path restores the first four, which is why the last two rows on the page open at their built-in defaults.

`src/gui/settings_dialog.py` — `_load_current`, the four it restores

```
self._pos_distance.setValue(self._sm.get("position_distance_pct", 2.0))
self._default_positions.setValue(self._sm.get("default_position_count", 10))
self._default_balance.setValue(
    self._sm.get("default_target_balance", 200.0)
)
style = self._sm.get("increment_style", "linear")
idx = self._increment_style.findText(style)
if idx >= 0:
    self._increment_style.setCurrentIndex(idx)
```

Three of the four reads in that block belonged to rows now off the page. The page saves and restores Default Target Balance alone out of the four.

### Settings > Profit Folding

The dialog draws no Profit Folding page. `TAB_TITLES` in `src/gui/main_tabs/settings_dialog_surface.py` holds no such entry, so neither build lays one out.

The settings record declares no folding group either. `AppSettings` in `src/core/settings.py` is the whole schema, and no field on it holds a folding value.

Folding is switched per bot, on the running bot's own settings row. The field is `profit_folding_active`, declared on `BotConfig`.

`src/trading/container/config.py` — the switch a fold reads

```
profit_folding_active: bool = True
```

A fold is bounded in dollars and by nothing else. The planner takes the tranches the market price reaches and the money left under this cycle's growth cap, and it stops when the money runs out. One dollar cap bounds a fold, and no count does carries that measured over five queued tranches.

### Settings > TA Indicators

Settings, the TA Indicators page.

One slider per weight, twelve in all, each labelled from its key and each running 0.00 to 2.00. The value beside a slider follows it as it moves, and the starting values are the weights themselves.

Indicator weight - Sets how far one indicator's vote counts against the other eleven when the engine totals a direction. The key behind each slider, and the value it starts at, are the twelve below.

`src/trading/ta_engine.py` — `DEFAULT_WEIGHTS`

```
DEFAULT_WEIGHTS = {
    "bollinger_bands": 1.0,
    "vortex": 0.9,
    "macd": 1.2,
    "stochastic_rsi": 1.0,
    "ichimoku": 1.1,
    "volume": 0.8,
    "slingshot": 1.0,
    "adx": 1.0,
    "kaufman_er": 1.0,  # Perry Kaufman Efficiency Ratio (regime classifier)
    "supertrend": 1.0,  # Olivier Seban ATR-trailing trend (reactive flip)
    "zscore": 0.9,  # Statistical extremity over a longer window than BB
    # Both are built on Wilder's RSI, so this vote overlaps stochastic_rsi.
    "rsi": 0.8,
}
```

A weight moved here reaches nothing. Save reads no widget on this page, the load path restores none, and the settings schema declares no field for indicator weights. The voting engine takes a weights argument and falls back to the defaults above, and no construction site under `src/` supplies one from the settings store. The page label promises an adjustment the engine never sees. Issue #423 carries it.

`src/trading/ta_engine.py` — `VotingEngine.__init__`, the fallback every construction site takes

```
self.weights = weights or DEFAULT_WEIGHTS.copy()
```

### Where an indicator weight lands

Each of the twelve sliders is a named control, saved under one key and restored from it. Save writes all twelve; reopening the page shows what was saved. The key is `ta_indicator_weights`, and an empty one means every indicator votes at its published figure.

`src/core/settings.py` — the stored group

```
# Indicator name -> voting weight. Empty means every indicator votes at the
# figure ta_engine.DEFAULT_WEIGHTS declares for it.
ta_indicator_weights: dict = field(default_factory=dict)
```

A bot reads the group when it is built, so a weight changed here applies to the next bot created and to every bot on the next launch. A bot already ticking keeps the figures it started with until it is rebuilt.

Diagram: flowchart, left to right

- slider — TA Indicators page -> **K]**
- ("ta_indicator_weights -> **weights_from_settings**
- weights_from_settings -> **ScrummingBot — ta_weights**
- ScrummingBot — ta_weights -> **VotingEngine**
- VotingEngine -> **each indicator's weight**

`src/trading/ta_engine.py` — the one reader every path goes through

```
def weights_from_settings(stored: Optional[dict]) -> dict[str, float]:
    found = dict(DEFAULT_WEIGHTS)
    for name, figure in (stored or {}).items():
        ...
        found[name] = number
    return found
```

An entry naming no indicator, an entry that is not a number, and a negative figure are each dropped with a line in the log, so a hand-edited settings file cannot give an indicator a weight the engine refuses to explain.

### One declaration of the twelve figures

`DEFAULT_WEIGHTS` is the only place the twelve figures are written. The voting engine builds one indicator per key, in that order, and the page reads the same mapping for its starting positions. Neither holds a second copy, so the two cannot drift apart.

`src/trading/ta_engine.py` — `_create_indicators`

```
return [
    INDICATOR_CLASSES[name](weight=self.weights.get(name, default))
    for name, default in DEFAULT_WEIGHTS.items()
]
```

### The figures stay as they are

Driven on a record shaped the way the restore path reads `bot_state.json`, a restored bot arrives with no stored weight of its own and takes the declared figures. A zero written into the declaration would therefore reach every bot already on disk and stop all twelve indicators contributing, while the vote counts stayed the same. The figures are unchanged for that reason.

```
declared defaults   consensus 0.1684   BULLISH   votes 6/4/2
all twelve at zero  consensus 0.0      NEUTRAL   votes 6/4/2
```

A zero for one indicator is a different matter and is available today: setting a single slider to 0.00 removes that indicator's vote and leaves the other eleven voting.

### Save hands the figures to a bot already running

Save is the only route. Pressing it writes the twelve figures and then hands them to every bot the manager holds. Each bot rebuilds its twelve indicators and votes on the new figures from its next tick. No timer and no other control reaches a bot already running.

`src/gui/main_window.py` — the one caller, inside the handler Save emits to

```
if self._bot_manager:
    self._bot_manager.push_ta_weights(
        weights_from_settings(
            self._settings.get("ta_indicator_weights", {})
        )
    )
```

Launch takes a different route and pushes nothing. `set_ta_weights` holds the figures for the restore path to build each bot with, and `push_ta_weights` is the only one that reaches a bot already built.

Diagram: flowchart, left to right

- Save button -> **settings_changed**
- settings_changed -> **push_ta_weights**
- push_ta_weights -> **each bot's VotingEngine**
- each bot's VotingEngine -> **the next tick's vote**
- launch -> **set_ta_weights, held only**
- set_ta_weights, held only -> **the next bot built**

Driven on three stand-in bots: every one read its MACD weight at 1.20 before the push and 0.30 after it, and a fourth holder that answers no such call was skipped, 3 of 4. Holding alone moved nothing, and the same reading stayed at 1.20.

### Two voters build their own engine and take no weight

The detonation check and the initial-entry check each build a fresh voting engine, so the twelve figures reach neither. Initial entry gates the first buy on the direction that engine reports. Detonation cannot fire in any case: its confidence bar starts at 0.50, and one 240-candle tape put the highest consensus at 0.1242 with the figures declared and 0.1016 with all twelve at 0.30.

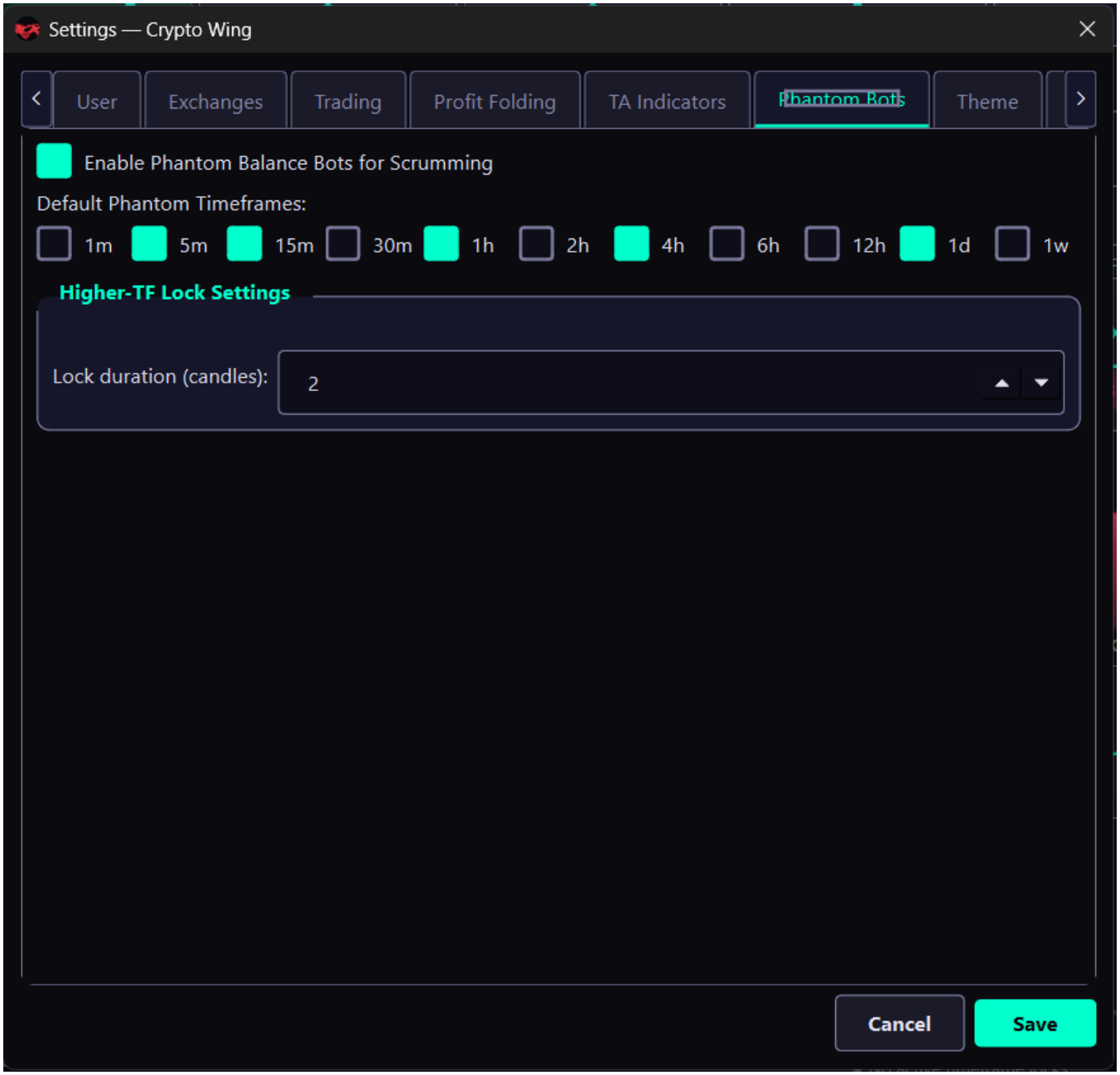
### A phantom keeps the figures its set was created with

A phantom set is built from the parent's weights at the moment it is created. The push reaches the parent's own voter and not a phantom already running, so a phantom votes on the older figures until its set is created again.

### Each drawn row is named by its indicator

On the drawn page every weight row carries the key the engine uses, which is the same name its slider answers to, and the printed words sit in the row's own label. Read off the rendered page: twelve rows, sixty-six drawn controls, and every row named for its indicator rather than for the words beside it.

### Settings > Phantom Bots




Settings, the Phantom Bots page.

The master checkbox, eleven timeframe boxes — 1m, 5m, 15m, 30m, 1h, 2h, 4h, 6h, 12h, 1d and 1w — and one Higher-TF Lock Settings group holding Lock duration in candles, 1 to 10. The build checks 5m, 15m, 1h, 4h and 1d, which is the state the figure shows.

Enable Phantom Bots for Scrumming - Turns the phantom overrides on for a new bot. Per-bot key `enable_phantoms`, a checkbox, checked at build.

This is the only control on the page that starts on. Save never reads it, so the page stores nothing under the name it shows.

`src/gui/settings_dialog.py` — the phantom master switch

```
self._phantoms_enabled = QCheckBox(
    "Enable Phantom Bots for Scrumming"
)
self._phantoms_enabled.setChecked(True)
layout.addWidget(self._phantoms_enabled)
```

Default Phantom Timeframes - Chooses which charts a phantom watches. Per-bot key `phantom_timeframes`, eleven boxes, five of them checked at build.

One box per timeframe in a single row. The build decides which five open checked, and those five are the ones the figure shows.

`src/gui/settings_dialog.py` — the box built for each timeframe

```
cb = QCheckBox(tf)
cb.setChecked(tf in ["5m", "15m", "1h", "4h", "1d"])
self._phantom_tf_checks[tf] = cb
tf_grid.addWidget(cb)
```

The row is now one drop-down holding the same eleven names, and the page stores the one name it shows. The opening choice is the daily chart, which is the only entry above the wizard's own hourly default that every venue offers.

`src/gui/settings_dialog.py` — the one drop-down that replaced the grid

```
layout.addWidget(QLabel("Default Phantom Timeframes:"))
self._phantom_timeframe = QComboBox()
for tf in PHANTOM_TIMEFRAMES:
    self._phantom_timeframe.addItem(tf, tf)
self._phantom_timeframe.setCurrentIndex(
    self._phantom_timeframe.findData(PHANTOM_TIMEFRAME_DEFAULT)
)
layout.addWidget(self._phantom_timeframe)
```

Lock duration (candles) - Sets how many candles a higher-timeframe lock holds an opposing trade back for. Per-bot key `lock_candle_count`, 1 to 10, at 2.

The one spin box on the page, inside its own group. It opens at 2 and Save reads it no more than it reads the other two.

`src/gui/settings_dialog.py` — the lock duration box

```
lock_group = QGroupBox("Higher-TF Lock Settings")
lock_form = QFormLayout(lock_group)
self._lock_candles = QSpinBox()
self._lock_candles.setRange(1, 10)
self._lock_candles.setValue(2)
lock_form.addRow("Lock duration (candles):", self._lock_candles)
```

The page now stores that figure. One row carries the store key and the control it reads, so Save writes it and the load path puts it back.

`src/gui/main_tabs/settings_dialog_surface.py` — the row both surfaces persist

```
("default_lock_candle_count", "lock_candles", LOCK_CANDLE_DEFAULT),
```

Both wizard surfaces open their own lock box on the stored figure, so a new bot starts on the number the page shows. A run with no settings file at all opens at 2, the figure the store itself declares.

`src/gui/bot_wizard.py` — the wizard's lock box, opened on the store

```
self._stored_lock_candles = int(
    (defaults or {}).get("default_lock_candle_count")
    or AppSettings().default_lock_candle_count
)
```

Driven on one store, typed as 7 and saved: the page reopens at 7, the React page paints 7, and both wizard lock boxes open at 7. A settings file written before the key existed still loads, and its box opens at 2.

```
store before   KeyError 'Unknown setting: default_lock_candle_count'
store after    7
page reopen    Qt 7   React 7   React payload 7
wizard boxes   Qt 7   React 7   with no store at all 2
old file       loads, theme carried, box opens at 2
```

The figure stops at the bot's phantom coordinator and holds no lock yet. Nothing in the application calls the method that appends a lock, so the countdown never starts and the lock test answers False on every tick.

`src/trading/phantom_balance.py` — `create_lock`, which has no caller

```
count = candle_count or self.lock_candle_count
```

Driven: a fresh coordinator holds no locks and reports none on an hourly chart. Called by hand at 7 it appends a lock of seven candles and the same test then answers True, so the figure sizes a real lock the moment something creates one.

```
fresh coordinator      locks 0     is_locked("1h", BULLISH) False
create_lock by hand    candles 7   is_locked("1h", BULLISH) True
```

The wizard's own key still stops before the bot. The bot configuration declares no field of that name, and the bot class takes no argument for it.

```
BotConfig has a lock_candle_count field        False
bot_config_kwargs carries lock_candle_count    False
ScrummingBot.__init__ has lock_candle_count    False
```

Those eleven boxes are the eleven timeframes the voting engine already weights, from 1m at 0.3 up to 1w at 1.6, so a slower chart counts for more when several are combined.

These three controls carry the same gap the TA Indicators page carries. Save reads none of them, the load path restores none, and the schema refuses a key it does not declare rather than storing it somewhere unread.

`src/core/settings.py` — `SettingsManager.set`

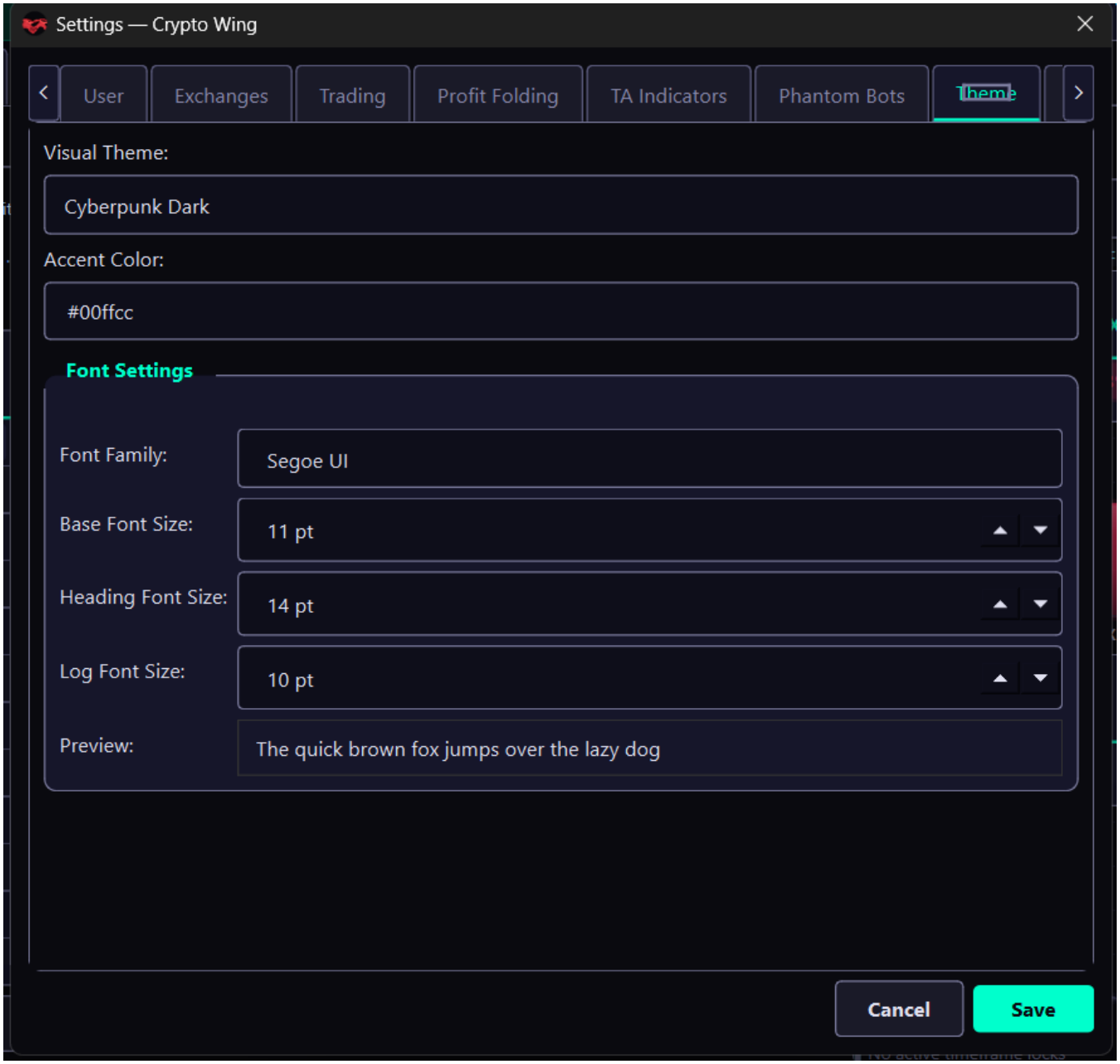
```
with self._lock:
    if not hasattr(self._settings, key):
        raise KeyError(f"Unknown setting: {key}")
    setattr(self._settings, key, value)
    self._save()
```

Issue #423 carries this page. The per-bot equivalents do persist: the wizard's phantom page emits the same three names into the bot's own config, and the Live Bot Settings dialog edits them there.

`src/gui/bot_wizard.py` — `PhantomConfigPage.get_config`, the three keys that do reach a bot

```
return {
    "enable_phantoms": self._enable.isChecked(),
    "phantom_timeframes": checked,
    "lock_candle_count": self._lock_candles.value(),
}
```

### Settings > Theme




Settings, the Theme page.

The theme picker, the accent colour field and a Font Settings group. Six controls and one preview.

Visual Theme - Chooses the palette the whole window paints in. Key `theme`, five display names — Cyberpunk Dark, Neon Light, Classic Terminal, Minimal Modern and Glass & Metal — at Cyberpunk Dark.

This is the one control on the page that reaches the running window. Save emits a signal, the main window reads the stored theme back, and the window repaints without a restart.

`src/gui/main_window.py` — `_on_settings_changed`

```
theme = self._settings.get("theme", "cyberpunk_dark")
self._switch_theme(theme)
```

A stored name the theme table does not hold opens on Cyberpunk Dark. The store accepts any text under that key, and the theme manager refuses a name it does not carry, so every store read passes its value through a guard first. Driven on ten stored values, five of them names the table does not hold: without the guard the application refused five and painted five, and with it all ten paint, while the five real names still paint their own five colours.

`src/gui/theme_engine.py` — `stored_theme`

```
def stored_theme(name: object) -> str:
    """Return ``name`` when ``THEMES`` holds it, else ``DEFAULT_THEME_NAME``."""
    return name if isinstance(name, str) and name in THEMES else DEFAULT_THEME_NAME
```

The guard covers the read the block above shows and the read at startup, which also says in the log when a stored name was not recognised. It covers the dialog's own page read as well: that one never refused, because the page palette falls back on its own table, so the guard there removes a second fallback and one copy of the default name rather than a refusal. Driven on eight stored values, four of them names the table does not hold: the page drew 8 of 8 and refused none both before and after, and all four unknown names paint a picture identical to Cyberpunk Dark and different from Neon Light.

`src/gui/react_settings_dialog.py` — `_theme_name`

```
        def _theme_name(self) -> str:
            """The stored theme the page paints in, always a name ``THEMES`` holds."""
            if not self._sm:
                return DEFAULT_THEME_NAME
            return stored_theme(self._sm.get("theme", DEFAULT_THEME_NAME))
```

`main.py` — the startup read

```
stored_name = settings.get("theme", DEFAULT_THEME_NAME)
theme_name = stored_theme(stored_name)
if theme_name != stored_name:
    log_manager.warning(
        f"Stored theme {stored_name!r} is not a known theme; "
        f"painting {theme_name}"
    )
theme_mgr.apply_theme(theme_name, app)
```

The palette reaches every Qt control and the Settings dialog's own page. It does not reach the React tab pages: each of those paints from the one design system, and their panes carry their own ground on every row. Rendered offscreen at 64,680 pixels a page, the Market Inspector, History, Console and Alerts pages come back identical under a stored Cyberpunk Dark and a stored Neon Light, and the Console page is identical even when the theme is handed straight to it.

`src/gui/react_history_panel.py` — the six values a page receives

```
    return {
        "--bg": colors["bg"],
        "--text": colors["text"],
        "--grid": colors["grid"],
        "--border": colors["border"],
        "--accent": colors["accent"],
        "--btn-bg": colors["btn_bg"],
    }
```

The main tab bar is the one React surface that follows a switch, and it follows it in part. Rendered at 8,232 pixels, one colour of eight moves from `#fcee0a` to `#f0cf1f` and returns when the theme returns; the bar's ground does not move.

`src/gui/react_main_window.py` — `MainTabBookReact.set_theme`

```
        def set_theme(self, name: str) -> None:
            """Repaint the bar's tabs from ``name``'s own tab tokens."""
            self._theme = name
            self._push_tabs()
```

Accent Color - Sets the highlight colour. Key `accent_color`, a free-text field, at `#00ffcc`.

A plain text box. The default is placeholder text, not a value, so the field opens empty until something is typed or a stored value loads. Save stores what is there and the load path puts it back, but nothing paints with it.

`src/gui/settings_dialog.py` — the Accent Color field

```
layout.addWidget(QLabel("Accent Color:"))
self._accent_color = QLineEdit()
self._accent_color.setPlaceholderText("#00ffcc")
layout.addWidget(self._accent_color)
```

An empty box means the theme's own accent. A hex colour in the box paints over it. Everything else is refused, and the theme's own accent paints.

`src/gui/theme_engine.py` — what the box is read through

```
def stored_accent(value: object) -> str:
    """Return ``value`` when it is a hex colour, else ``THEME_ACCENT``.

    The store's accent field is free text, and a value ``generate_qss`` cannot
    read either drops its declaration or escapes it, so the reads in ``main``
    and in the main window pass through here.
    """
    if not isinstance(value, str):
        return THEME_ACCENT
    asked = value.strip()
    return asked if _ACCENT_HEX.match(asked) else THEME_ACCENT
```

Three hex digits or six, either case, surrounding spaces trimmed. Each value below was typed in and the accent button was then read off an offscreen render.

| typed | the accent button paints |
|---|---|
| `#ff0000` | #ff0000 |
| `#0f0` | #00ff00 |
| `#FF00AA` | #ff00aa |
| `#ff0000` | #ff0000 |
| `red` | the theme's accent |
| `rgb(255,0,0)` | the theme's accent |
| `#00ffc` | the theme's accent |
| `#00ffccff` | the theme's accent |
| `notacolour` | the theme's accent |
| `#ff0000;` | the theme's accent |
| `url(x)` | the theme's accent |
| empty, spaces, or no value at all | the theme's accent |

A colour name is refused on purpose: the box takes a hex colour, and the four forms Qt would also read bring a wider parser with nothing to paint that hex cannot.

The refusal matters because the box is free text. A value that ends one declaration and opens another repaints the window, and that was measured: with `red; } QWidget { background-color: #000000` standing in the accent token, a plain pane rendered 3,840 pixels of #000000 where its ground is #0a0a0f. The same string typed into the box now leaves that pane on its ground. A hex colour carries no semicolon and no brace, so no accepted value can reach that.

Six surfaces carry the accent. The figures are pixel counts off an offscreen render of each one.

| surface | pixels |
|---|---|
| accent button | 3,289 |
| full progress bar | 2,092 |
| ticked box | 464 |
| slider handle | 164 |
| selected tab underline | 134 |
| focused text box border | 304 |

A typed `#ff0000` moves all six, on all five themes. Text a theme paints from its own `text_accent` stays where it was: on Classic Terminal 160 pixels of #00ff00 remain after the override, 96 in the tab label and 64 in the checkbox label.

`main.py` — the startup read

```
    stored_hex = settings.get("accent_color", THEME_ACCENT)
    accent = stored_accent(stored_hex)
    if not accent and str(stored_hex).strip():
        log_manager.warning(
            f"Stored accent colour {stored_hex!r} is not a hex colour; "
            f"painting the {theme_name} accent"
        )
    theme_mgr.apply_theme(theme_name, app, accent)
```

A refused value is named on the Activity Log when the dialog saves and when the Theme menu switches. The box keeps what was typed, so the value and the line that says it did not paint are both on screen.

`src/gui/main_window.py` — `_switch_theme`, the refusal

```
            taken = stored_accent(accent)
            if not taken and str(accent or "").strip():
                self._status_log.log(
                    f"Accent colour {accent!r} is not a hex colour; "
                    f"painting the {name} accent.",
                    "warning",
                )
```

Driven with `notacolour` against Neon Light, that line read: Accent colour 'notacolour' is not a hex colour; painting the neon_light accent.

The React tab pages do not follow the typed value. Each page receives an `--accent` variable built from a theme name alone, so the seven rules in the Settings page's own stylesheet that read that variable stay on the theme's accent.

`src/gui/main_tabs/react_history_panel_surface.py` — `palette`, which takes a theme and no accent

```
def palette(theme: str) -> dict:
    """The six chrome colours one theme paints the page in.

    A theme the table does not carry falls back to the default theme.
    """
    colors = THEME_CHROME.get(theme) or THEME_CHROME[DEFAULT_THEME]
    return {
        key: colors[source] for key, source in zip(PALETTE_KEYS, PALETTE_SOURCE_KEYS)
    }
```

Contrast is not checked. Every theme's own accent clears WCAG 2.2 AA against the ground it sits on, and a typed one is painted as given.

Font Family - Sets the typeface for application text. Key `font_family`, an editable combo over twelve named families, at Segoe UI.

The box is editable, so a family outside the twelve can be typed in. Save stores this value and the three sizes below it. Nothing reads any of the four, and the load path does not restore them either, so the whole Font Settings group reopens at its built-in figures.

`src/gui/settings_dialog.py` — the Font Family row

```
self._font_family.addItems(fonts)
self._font_family.setCurrentText("Segoe UI")
self._font_family.setToolTip("Font family for all application text")
font_form.addRow("Font Family:", self._font_family)
```

Base Font Size - Sets the size of ordinary interface text. Key `font_size`, 8 pt to 24 pt, at 11 pt.

The preview line below the group follows this box and the family box as either changes. That preview is the only place either value has an effect.

`src/gui/settings_dialog.py` — the Base Font Size row

```
self._font_size = QSpinBox()
self._font_size.setRange(8, 24)
self._font_size.setValue(11)
self._font_size.setSuffix(" pt")
self._font_size.setToolTip("Base font size for all UI text")
font_form.addRow("Base Font Size:", self._font_size)
```

Heading Font Size - Sets the size of headings and stat-card values. Key `heading_font_size`, 10 pt to 32 pt, at 14 pt.

The widest of the three ranges. The preview does not follow this box.

`src/gui/settings_dialog.py` — the Heading Font Size row

```
self._heading_size = QSpinBox()
self._heading_size.setRange(10, 32)
self._heading_size.setValue(14)
self._heading_size.setSuffix(" pt")
self._heading_size.setToolTip("Font size for headings and stat card values")
font_form.addRow("Heading Font Size:", self._heading_size)
```

Log Font Size - Sets the size of the Activity Log and API Log text. Key `log_font_size`, 8 pt to 18 pt, at 10 pt.

The narrowest of the three ranges, and the last row before the preview.

`src/gui/settings_dialog.py` — the Log Font Size row

```
self._log_font_size = QSpinBox()
self._log_font_size.setRange(8, 18)
self._log_font_size.setValue(10)
self._log_font_size.setSuffix(" pt")
```

Preview redraws in the chosen family and base size as either changes. It is a sample line, not a setting, and nothing stores it.

The five theme names come from one mapping built off the five theme objects.

`src/gui/theme_engine.py` — `THEMES`

```
THEMES: dict[str, ThemeTokens] = {
    t.name: t
    for t in [CYBERPUNK_DARK, NEON_LIGHT, CLASSIC_TERMINAL, MINIMAL_MODERN, GLASS_METAL]
}
```

Save writes the theme, the Accent Color field and all four font values.

`src/gui/settings_dialog.py` — `_save`, the theme and font keys

```
"theme": lambda: self._theme_combo.currentData(),
"accent_color": lambda: self._accent_color.text().strip(),
"font_family": lambda: self._font_family.currentText(),
"font_size": lambda: self._font_size.value(),
"heading_font_size": lambda: self._heading_size.value(),
"log_font_size": lambda: self._log_font_size.value(),
```

The load path reads back the theme and the accent field only, so the four font rows open at Segoe UI, 11, 14 and 10 whatever was stored.

Of the six, the theme alone reaches the window. `main.py` reads it at startup, and the window reads it again in `_on_settings_changed` when the dialog saves.

The other five are stored and never read again. Each palette carries its own typeface in the theme object it declares, so the theme decides the font and the four font rows decide nothing. The accent field has the same shape: declared, written, read back into its own line edit, and painted nowhere.

`src/core/settings.py` — `AppSettings`, the five that are stored and unread

```
accent_color: str = "#00ffcc"

# These four mirror the font widgets in settings_dialog._create_theme_tab.
font_family: str = "Segoe UI"
font_size: int = 11
heading_font_size: int = 14
log_font_size: int = 10
```

Of those five, the accent field now reaches the window. The four font rows do not. The stored default for the accent is empty, so a store that has never been typed into leaves every theme on its own accent rather than handing Cyberpunk Dark's to the other four.

`src/core/settings.py` — `AppSettings`, the accent field today

```
    theme: str = VisualTheme.CYBERPUNK_DARK.value
    # Empty leaves each theme's own accent painting. theme_engine.stored_accent
    # takes a hex colour here and refuses anything else.
    accent_color: str = ""
```

The Font Settings group is gone. Font Family, Base Font Size, Heading Font Size and Log Font Size are no longer on the Theme page, the preview line went with them, and the four stored keys are no longer fields of the settings record. The theme declares the typography instead: every palette carries a typeface, a monospace

face and four sizes.

`src/gui/theme_engine.py` — the typography tokens a theme declares

```
    # Font
    font_family: str = "'Rajdhani', 'Orbitron', 'Segoe UI', sans-serif"
    font_mono: str = "'JetBrains Mono', 'Fira Code', 'Consolas', monospace"
    font_size: str = "13px"
    font_size_small: str = "11px"
    font_size_large: str = "16px"
    font_size_title: str = "20px"
```

Three of the five palettes name a typeface of their own. Classic Terminal takes a monospace family, Minimal Modern and Glass & Metal each name their own, and the other two keep the declared default above.

```
# CLASSIC_TERMINAL
font_family="'JetBrains Mono', 'Fira Code', 'Consolas', monospace",

# MINIMAL_MODERN
font_family="'SF Pro Display', 'Inter', 'Segoe UI', sans-serif",

# GLASS_METAL
font_family="'Exo 2', 'Rajdhani', 'Segoe UI', sans-serif",
```

The stylesheet generator reads those tokens, so the typeface and the size arrive with the palette rather than from a control.

`src/gui/theme_engine.py` — `generate_qss`, the global rule

```
QWidget {{
    background-color: {t.bg_primary};
    color: {t.text_primary};
    font-family: {t.font_family};
    font-size: {t.font_size};
    selection-background-color: {t.accent_primary};
    selection-color: {t.bg_primary};
}}
```

A typeface the operator sets would have split that. The theme engine audits a palette as one set and says so in its own opening words, and a size set outside the palette moves the text the audit was taken against while nothing asks for the audit again.

`src/gui/theme_engine.py` — the module's own rule

```
Every ``ThemeTokens`` pair meets WCAG 2.2 AA contrast, and a changed hex value
needs a fresh audit.
```

The Theme page now carries two controls, the theme picker and the accent field. The dialog payload carries 68 controls where it carried 72, the rendered page draws 55 where it drew 59, and Save writes nine top-level keys where it wrote thirteen. The five generated stylesheets are unchanged byte for byte, because the removal

touched no palette.

A settings file written before the removal still loads. The load copies only the fields the record declares, so a font key in such a file is read past and every other setting arrives at its stored value.

`src/core/settings.py` — `SettingsManager._apply_dict`

```
defaults = asdict(AppSettings())
for key, default_val in defaults.items():
    if key in data:
        setattr(self._settings, key, data[key])
```

Driven on a file holding all four font keys beside nine other settings: nothing raised, and the username, the target balance, the visibility, the aggressive flag, the theme, the accent, the three phantom values, the indicator weights and the logging group all arrived as stored.

### The Simulator's tone, nigredo

Every theme has a second tone, and one widget tree paints in it: the Simulator tab. The tone is the theme's own seven ground tokens moved part of the way to black. The accent, the text, the borders and the chart colours stay the theme's, so a Simulator screen reads as the same theme, darker.

`src/gui/theme_engine.py` — the derivation

```
NIGREDO_FRACTION = 0.25

NIGREDO_GROUNDS = (
    "bg_primary",
    "bg_secondary",
    "bg_tertiary",
    "bg_card",
    "bg_input",
    "bg_hover",
    "bg_selected",
)


def nigredo(theme: ThemeTokens, fraction: float = NIGREDO_FRACTION) -> ThemeTokens:
    moved = {
        name: toward_black(getattr(theme, name), fraction) for name in NIGREDO_GROUNDS
    }
    return replace(theme, **moved)
```

One function moves a colour. Each sRGB channel is scaled by one minus the fraction and rounded, so a fraction of 0 leaves the colour as it is and a fraction of 1 gives black. The function takes six-digit hex and refuses any other form.

```
def toward_black(colour: str, fraction: float) -> str:
    keep = 1.0 - fraction
    return "#" + "".join(
        f"{round(int(channel, 16) * keep):02x}" for channel in found.groups()
    )
```

The fraction is one number. Every theme's ground, its darkened ground, and the contrast of that theme's body text on each, WCAG 2.2 ratio:

```
theme              ground    nigredo   text      ratio on ground  ratio on nigredo
cyberpunk_dark     #0a0a0f   #08080b   #e0e0f0   15.13            15.33
neon_light         #f5f5fa   #b8b8bc   #1a1a2e   15.70             8.63
classic_terminal   #0a0a0a   #080808   #00ff00   14.43            14.60
minimal_modern     #fafafa   #bcbcbc   #1a1a1a   16.67             9.16
glass_metal        #1c1c24   #15151b   #d0d0e0   11.10            11.93
```

Every row is above 4.5 to 1. On the three dark themes the ratio rises. On the two light themes it falls, because their text is dark and a darker ground moves toward it; it stays above 4.5 to 1 up to a fraction near 0.45, and at 0.6 Neon Light reads 2.80 and fails. On a near-black theme the ground moves by two to seven per channel, which the picture reports and the eye does not; the raised grounds, the cards and the tables, move by more.

The stylesheet carries the tone. `generate_qss` ends with the theme's rules restated a second time, from the darkened tokens, each rule under one selector that names a Qt dynamic property.

`src/gui/theme_engine.py` — the scope and the block

```
TONE_PROPERTY = "tone"
NIGREDO = "nigredo"

NIGREDO_SELECTOR = f'QWidget[{TONE_PROPERTY}="{NIGREDO}"]'


def generate_qss(theme: ThemeTokens) -> str:
    return f"{_theme_qss(theme)}\n/* --- Simulator: {NIGREDO} --- */{nigredo_qss(theme)}\n"
```

The block's first rule, under Cyberpunk Dark:

```
QWidget[tone="nigredo"], QWidget[tone="nigredo"] QWidget {
    background-color: #08080b;
    color: #e0e0f0;
    font-family: 'Rajdhani', 'Orbitron', 'Segoe UI', sans-serif;
    font-size: 13px;
    selection-background-color: #00ffcc;
    selection-color: #08080b;
}
```

A widget that sets the property `tone` to `nigredo` paints in the tone, and so does every widget under it, including a dialog it parents. The Simulator tab sets it in its constructor, in both builds. A theme switch re-applies the whole stylesheet, so the tone follows the theme with no second step.

`src/gui/simulator/sim_trading_tab.py` — the tab takes the tone

```
self.setProperty(TONE_PROPERTY, NIGREDO)
self.setAttribute(Qt.WA_StyledBackground, True)
```

A React page reads the tone the same way. The page's `:root` chrome comes from `page_html`, which takes a tone and puts the two grounds through the same function; a theme switch runs `repaint_pages`, which reads the tone off each web view's property and pushes that view its own chrome.

`src/gui/react_history_panel.py` — the page chrome in a tone

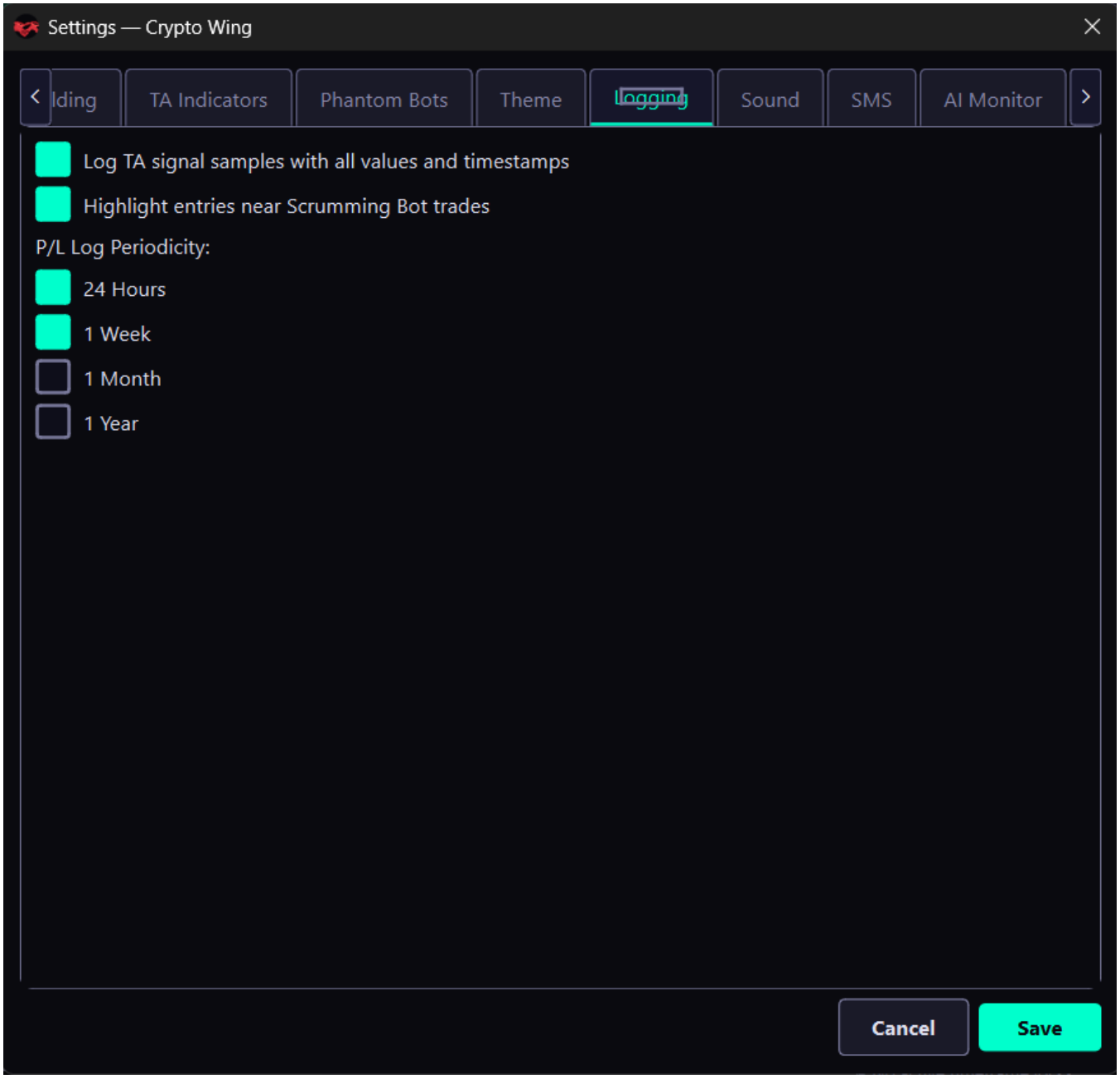
```
if tone == NIGREDO:
    ground = toward_black(ground, NIGREDO_FRACTION)
    button = toward_black(button, NIGREDO_FRACTION)
```

Diagram: flowchart, left to right

- _switch_theme -> **ThemeManager.apply_theme**
- ThemeManager.apply_theme -> **generate_qss**
- generate_qss -> **_theme_qss: every widget**
- generate_qss -> **nigredo_qss: the tone=nigredo tree**
- _switch_theme -> **repaint_pages**
- repaint_pages -> **palette_script: a Live view**
- repaint_pages -> **palette_script with NIGREDO: a Sim view**

The Simulator page under the Simulator tab states what each build paints in the tone and what was read off the renders.

### Settings > Logging




Settings, the Logging page.

Two checkboxes and four periodicity boxes. Every key below sits inside one stored `data_logging` group.

Log TA signal samples with all values and timestamps - Writes each indicator reading out with its value and its time. Key `ta_signal_logging`, a checkbox, checked at build.

The first of the two flags on the page, and the one that decides whether the indicator record is written at all.

`src/gui/settings_dialog.py` — the TA logging checkbox

```
self._ta_logging = QCheckBox(
    "Log TA signal samples with all values and timestamps"
)
self._ta_logging.setChecked(True)
layout.addWidget(self._ta_logging)
```

Highlight entries near Scrumming Bot trades - Marks the log lines that sit close to a fire, so a decision can be read in its own context. Key `highlight_trade_proximity`, a checkbox, checked at build.

The second flag, built the same way and checked at build like the first.

`src/gui/settings_dialog.py` — the highlight checkbox

```
self._highlight_trades = QCheckBox(
    "Highlight entries near Scrumming Bot trades"
)
self._highlight_trades.setChecked(True)
layout.addWidget(self._highlight_trades)
```

P/L Log Periodicity - Chooses the windows a profit and loss figure is summarised over: 24 Hours, 1 Week, 1 Month and 1 Year. Key `active_periodicities`, four boxes written as a list, with the first two checked at build.

Four separate boxes rather than one control. Save walks them in page order and writes the checked ones into a list.

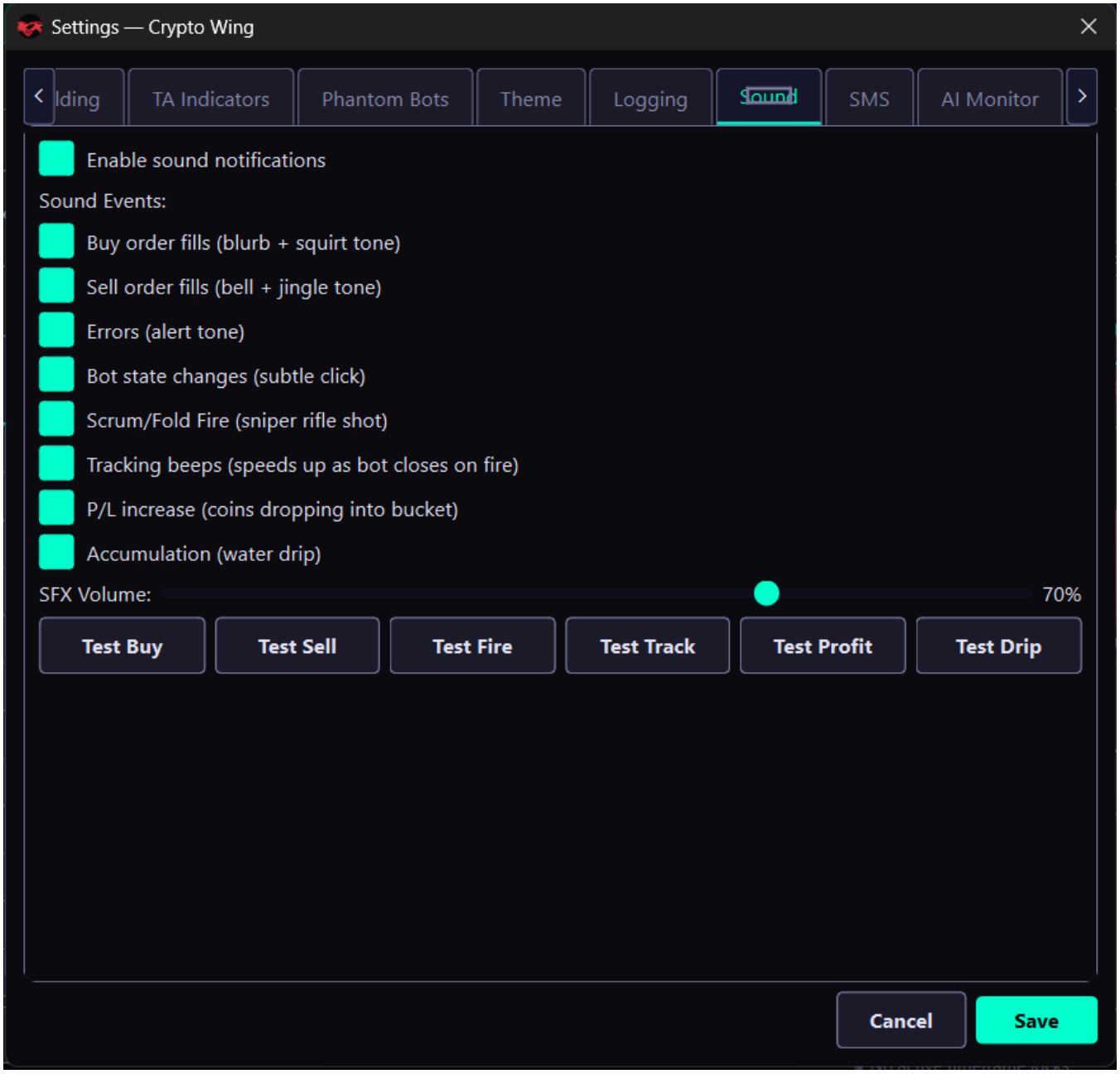
`src/gui/settings_dialog.py` — the four periodicity boxes

```
self._log_24h = QCheckBox("24 Hours")
self._log_24h.setChecked(True)
self._log_1w = QCheckBox("1 Week")
self._log_1w.setChecked(True)
self._log_1m = QCheckBox("1 Month")
self._log_1y = QCheckBox("1 Year")
```

Save collapses all six into one dictionary holding the two flags and the list of active periodicities. The load path reads none of them back, so every open shows the build state rather than the stored one. No reader outside the dialog reads the group either, which leaves the six controls writing to a key nothing consults. Issue #423 carries this page.

In development. What the load path should restore for this page has not been settled against the rest of the dialog, so nothing is proposed here.

### Settings > Sound




Settings, the Sound page.

The master switch, eight event checkboxes, a volume slider and six test buttons. Every checkbox is checked at build.

Each checkbox names its sound in its own label, and the key behind each control is a field of the sound configuration the block below builds.

Enable sound notifications - Turns every sound on or off at once. Key `enabled`, on.

The engine tests this flag before it tests any other, so a clear box silences the page whatever the eight below are set to.

`src/gui/settings_dialog.py` — the master switch

```
self._sound_enabled = QCheckBox("Enable sound notifications")
self._sound_enabled.setChecked(True)
self._sound_enabled.setToolTip("Master switch for all audio notifications")
```

Buy order fills - Plays a low rising tone when a buy fills. Key `buy_sound`, on.

The label carries the sound's own name in brackets, as every box on this page does. The engine plays this one under the name buy.

`src/gui/settings_dialog.py` — the buy fill box

```
self._sound_buy = QCheckBox("Buy order fills (blurb + squirt tone)")
self._sound_buy.setChecked(True)
self._sound_buy.setToolTip(
    "Plays a low bubbly rising tone when a buy order is filled"
)
```

Sell order fills - Plays a bright bell when a sell fills. Key `sell_sound`, on.

The counterpart to the box above, played under the name sell.

`src/gui/settings_dialog.py` — the sell fill box

```
self._sound_sell = QCheckBox("Sell order fills (bell + jingle tone)")
self._sound_sell.setChecked(True)
self._sound_sell.setToolTip(
    "Plays a high bright bell tone when a sell order is filled"
)
```

Errors - Plays an alert tone on an error. Key `error_sound`, on.

One of the two boxes on the page with no tooltip of its own.

`src/gui/settings_dialog.py` — the error box

```
self._sound_error = QCheckBox("Errors (alert tone)")
self._sound_error.setChecked(True)
layout.addWidget(self._sound_error)
```

Bot state changes - Plays a click when a bot starts, pauses or stops. Key `bot_state_sound`, on.

The other box with no tooltip. The engine plays it under the name state, not under the key.

`src/gui/settings_dialog.py` — the bot state box

```
self._sound_state = QCheckBox("Bot state changes (subtle click)")
self._sound_state.setChecked(True)
layout.addWidget(self._sound_state)
```

Scrum/Fold Fire - Plays a rifle shot when a scrum or a fold executes, and on a Manual Fire. Key `fire_sound`, on.

This is the first of the four the engine reads with a default rather than straight off the configuration. A stored configuration written before these four existed leaves all four on.

`src/gui/settings_dialog.py` — the fire box

```
self._sound_fire = QCheckBox("Scrum/Fold Fire (sniper rifle shot)")
self._sound_fire.setChecked(True)
self._sound_fire.setToolTip(
    "Synthesized rifle shot plays when a scrum or fold\n"
    "actually executes. Also plays on Manual Fire."
)
```

Tracking beeps - Paces a beep by the scrum phase: silent in SEARCH, slow in TRACK, fast in FIRE. Key `track_sound`, on.

The tooltip gives the two paces in milliseconds: 800 in TRACK and 200 in FIRE.

`src/gui/settings_dialog.py` — the tracking box

```
self._sound_track = QCheckBox(
    "Tracking beeps (speeds up as bot closes on fire)"
)
self._sound_track.setChecked(True)
```

P/L increase - Plays coins dropping into a bucket on any trade that realised a profit. Key `profit_sound`, on.

The tooltip is the only place on the page that names the condition exactly: realised profit above zero, on grid and scrumming bots alike.

`src/gui/settings_dialog.py` — the profit box

```
self._sound_profit = QCheckBox("P/L increase (coins dropping into bucket)")
self._sound_profit.setChecked(True)
```

Accumulation - Plays a water drip on a FOLD, and on nothing else. Key `drip_sound`, on.

The last of the nine. Its tooltip calls a fold the canonical accumulation moment and states that a scrum or a distribution does not fire it.

`src/gui/settings_dialog.py` — the accumulation box

```
self._sound_drip = QCheckBox("Accumulation (water drip)")
self._sound_drip.setChecked(True)
```

SFX Volume - Sets how loud every sample plays. Key `volume`, 0 % to 100 %, at 70 %.

Test Buy, Test Sell, Test Fire, Test Track, Test Profit and Test Drip play one sample each. They are buttons, not settings.

One handler is the only path from this page to the engine. It builds a configuration from every checkbox and the slider, pushes it in and clears the sample cache, because each sample bakes its volume at synthesis time.

`src/gui/settings_dialog.py` — `_on_sfx_volume_changed`

```
se = get_sound_engine()
new_cfg = SoundConfig(
    enabled=self._sound_enabled.isChecked(),
    buy_sound=self._sound_buy.isChecked(),
    sell_sound=self._sound_sell.isChecked(),
    error_sound=self._sound_error.isChecked(),
    bot_state_sound=self._sound_state.isChecked(),
    fire_sound=self._sound_fire.isChecked(),
    track_sound=self._sound_track.isChecked(),
    profit_sound=self._sound_profit.isChecked(),
    drip_sound=self._sound_drip.isChecked(),
    volume=v / 100.0,
)
```

That path runs when the slider moves or a test button is pressed, and at no other time. Save reads no widget on this page and the schema declares no sound field, so nothing here survives the dialog closing. A checkbox cleared and the dialog saved comes back checked. Issue #423 carries it.

`src/core/sound_engine.py` — `SoundConfig`, the ten fields and their defaults

```
enabled: bool = True
buy_sound: bool = True
sell_sound: bool = True
error_sound: bool = True
bot_state_sound: bool = True
fire_sound: bool = True
track_sound: bool = True
profit_sound: bool = True
drip_sound: bool = True
volume: float = 0.7
```

Every tick box reaches the engine on its own. Each box's toggle runs the same push the slider runs, so a box cleared with nothing else touched silences that sound at once.

`src/gui/settings_dialog.py` — each box wired to the push

```
for _key, name in SOUND_CONFIG_FIELDS:
    getattr(self, f"_{name}").toggled.connect(self._push_sound_config)
```

The push reads the slider itself, so one method serves a box, the slider and a Test button alike.

`src/gui/settings_dialog.py` — the push

```
def _push_sound_config(self) -> None:
    self._on_sfx_volume_changed(self._sound_volume.value())
```

The React page draws the same ten controls. Its edit route named one handler, for the slider, and that handler wanted an argument the page never sends, so the press raised and reached nothing. Every sound control names the push, which wants no argument.

`src/gui/react_settings_dialog.py` — the edit route

```
EDIT_HANDLERS: dict[str, str] = {
    "new_exchange": "_on_exchange_changed",
    surface.VOLUME_NAME: "_push_sound_config",
    **{name: "_push_sound_config" for _key, name in surface.SOUND_CONFIG_FIELDS},
}
```

The settings file holds the page as one group, and the schema declares that group from the engine's own defaults, so the two cannot disagree over what the page holds.

`src/core/settings.py` — the group on the schema

```
sound: dict = field(default_factory=lambda: asdict(SoundConfig()))
```

Save and load walk one row list for the group, the way the Profit Folding and AI Monitor groups do, so no row reaches the file without also reaching a control. The slider shows whole percent and the engine holds a fraction, so the volume row divides out and multiplies back.

`src/gui/settings_dialog.py` — the sound rows

```
switches = tuple(
    (
        key,
        getattr(self, f"_{name}").isChecked,
        show(getattr(self, f"_{name}")),
        getattr(built, key),
    )
    for key, name in SOUND_CONFIG_FIELDS
)
```

One reader carries a stored group into the engine, and it keeps the default for anything the generators cannot take. A switch reads as on or off: the play gate tests it for truth, so a non-empty word would open a sound the operator closed. A volume reads as a finite number from zero to one: each generator multiplies it into every sample before packing the samples, and a volume of four leaves the engine holding no clips and playing nothing.

`src/core/sound_engine.py` — the reader

```
def sound_config_from_settings(stored: Optional[dict]) -> SoundConfig:
    taken = SoundConfig()
    for name, value in (stored or {}).items():
        if name in SOUND_SWITCHES:
            if isinstance(value, bool):
                setattr(taken, name, value)
```

The engine takes the stored group at startup, before the first fill, so a silenced sound stays silent whether or not the operator opens this page.

`main.py` — the startup push

```
get_sound_engine().update_config(
    sound_config_from_settings(settings.get("sound", {}))
)
```

Two of the nine switches gate a clip no trade path plays. Nothing calls the buy helper or the sell helper, so Test Buy and Test Sell are the only presses that reach those two samples. Both boxes store and reload, and the sound itself waits on a caller.

`src/core/sound_engine.py` — the two helpers with no caller

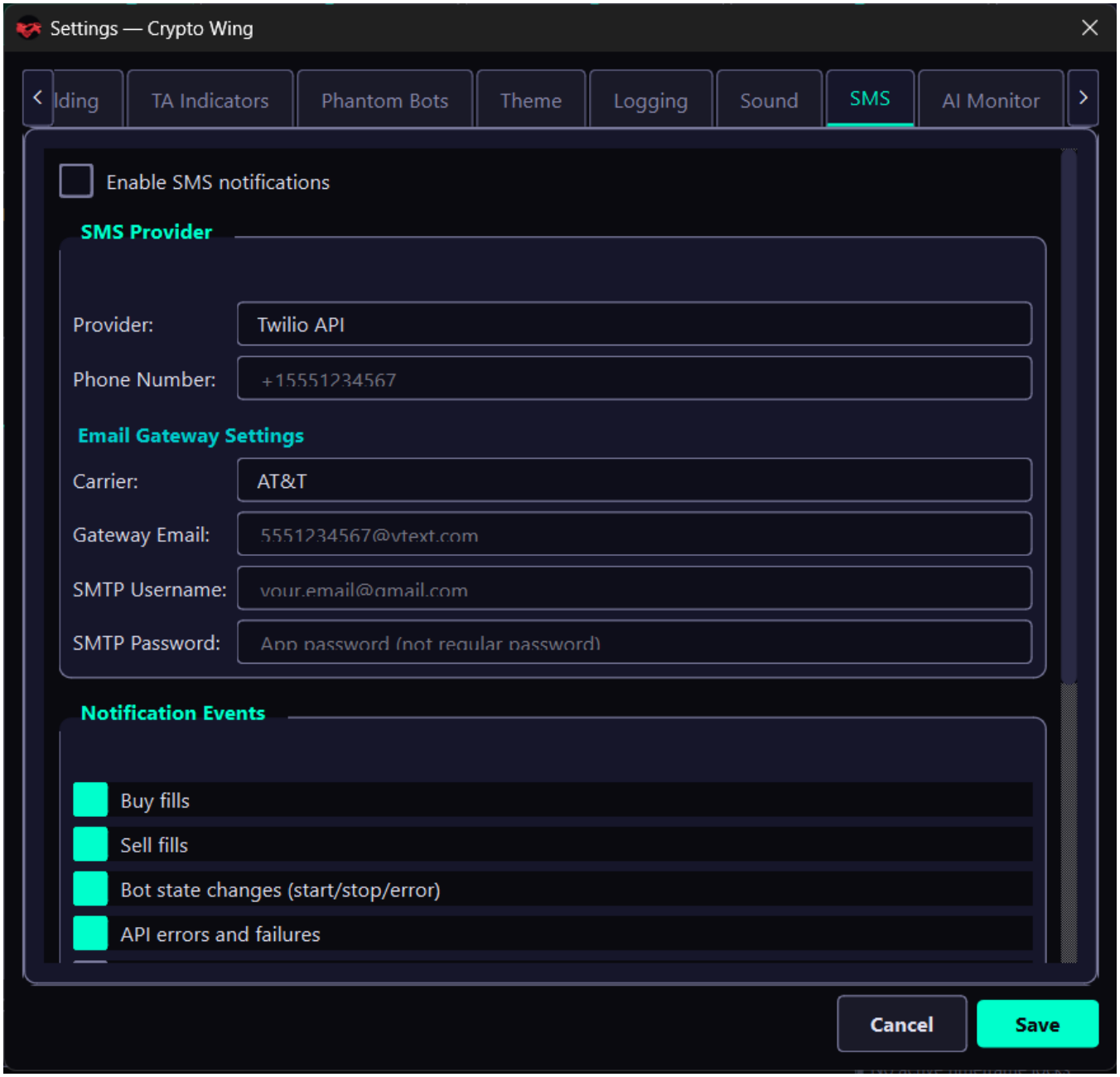
```
def play_buy(self) -> None:
    self.play("buy")

def play_sell(self) -> None:
    self.play("sell")
```

The six Test buttons carry a part name of their own, and the page's click listener reports the plain button part alone, so a press on the React page reports nothing to the dialog. Those six are buttons, not settings, and the switches beside them do not depend on them.

`src/gui/web/settings_dialog.js` — the two part names

```
var BUTTON_PART = "button";
var SOUND_BUTTON_PART = "sound-button";
```

### Settings > SMS




Settings, the SMS page.

A scrolling page holding the master switch and three groups.

Seventeen controls, and the key behind each is a field of the message configuration the engine reads.

Enable SMS notifications - Turns every message on or off at once. Key `enabled`, clear at build.

The only control on the page outside the three groups, and the only checkbox here that opens clear rather than checked.

`src/gui/settings_dialog.py` — the SMS master switch

```
self._sms_enabled = QCheckBox("Enable SMS notifications")
self._sms_enabled.setToolTip(
    "Send text messages to your phone for trading events"
)
layout.addWidget(self._sms_enabled)
```

Provider - Chooses how a message leaves the machine: a carrier's email gateway, or the Twilio web interface. Key `provider`, at the email gateway. Nothing loads a stored choice into the picker, so it opens on the first entry whatever the figure shows.

Two entries, both written as display labels. The engine tests a short code rather than either label, so neither entry matches what the send path looks for.

`src/gui/settings_dialog.py` — the Provider row

```
self._sms_provider = QComboBox()
self._sms_provider.setMinimumHeight(28)
self._sms_provider.addItems(["Email-to-SMS Gateway", "Twilio API"])
pf.addRow("Provider:", self._sms_provider)
```

Phone Number - Sets the number every message goes to. Key `phone_number`, empty, with a placeholder showing the international shape.

The placeholder is the only guidance on the row. It is grey sample text, not a value, so the field starts genuinely empty.

`src/gui/settings_dialog.py` — the Phone Number row

```
self._sms_phone = QLineEdit()
self._sms_phone.setMinimumHeight(28)
self._sms_phone.setPlaceholderText("+15551234567")
pf.addRow("Phone Number:", self._sms_phone)
```

Carrier - Names the carrier whose gateway address a message would be built for. Ten entries, opening on the first. It has no stored field of its own, and nothing on the page reads the choice.

The ten names come from one mapping in the engine. The last entry is a manual option whose address is deliberately empty.

`src/gui/settings_dialog.py` — the Carrier row

```
self._sms_carrier = QComboBox()
self._sms_carrier.setMinimumHeight(28)
from src.core.sms_engine import CARRIER_GATEWAYS

for carrier in CARRIER_GATEWAYS:
    self._sms_carrier.addItem(carrier)
pf.addRow("Carrier:", self._sms_carrier)
```

Gateway Email - Sets the full gateway address a message is sent to. Key `gateway_email`, empty.

The row a carrier choice would otherwise fill in. Nothing builds the address from the picker above, so this field is typed by hand or left blank.

`src/gui/settings_dialog.py` — the Gateway Email row

```
self._sms_gateway = QLineEdit()
self._sms_gateway.setMinimumHeight(28)
self._sms_gateway.setPlaceholderText("5551234567@vtext.com")
self._sms_gateway.setToolTip("Full email address for carrier SMS gateway")
pf.addRow("Gateway Email:", self._sms_gateway)
```

SMTP Username - Sets the mailbox a message is sent from. Key `smtp_username`, empty.

The first of the two mailbox rows. Its placeholder shows an ordinary address.

`src/gui/settings_dialog.py` — the SMTP Username row

```
self._sms_smtp_user = QLineEdit()
self._sms_smtp_user.setMinimumHeight(28)
self._sms_smtp_user.setPlaceholderText("your.email@gmail.com")
pf.addRow("SMTP Username:", self._sms_smtp_user)
```

SMTP Password - Sets that mailbox's application password. Key `smtp_password`, masked, empty.

The only masked field on the page. Its placeholder says an application password rather than the account password.

`src/gui/settings_dialog.py` — the SMTP Password row

```
self._sms_smtp_pass = QLineEdit()
self._sms_smtp_pass.setMinimumHeight(28)
self._sms_smtp_pass.setEchoMode(QLineEdit.Password)
self._sms_smtp_pass.setPlaceholderText(
    "App password (not regular password)"
)
pf.addRow("SMTP Password:", self._sms_smtp_pass)
```

Buy fills - Sends a message when a buy fills. Key `notify_buy_fills`, checked at build.

The first of the eight event rows, and one of the four that open checked.

`src/gui/settings_dialog.py` — the buy fills row

```
self._sms_buy = QCheckBox("Buy fills")
self._sms_buy.setChecked(True)
ef.addRow(self._sms_buy)
```

Sell fills - Sends a message when a sell fills. Key `notify_sell_fills`, checked at build.

The counterpart to the row above, built and checked the same way.

`src/gui/settings_dialog.py` — the sell fills row

```
self._sms_sell = QCheckBox("Sell fills")
self._sms_sell.setChecked(True)
ef.addRow(self._sms_sell)
```

Bot state changes - Sends a message when a bot starts, stops or errors. Key `notify_bot_state_changes`, checked at build.

The label names the three states in brackets, which the page's own text does not repeat anywhere else.

`src/gui/settings_dialog.py` — the bot state row

```
self._sms_state = QCheckBox("Bot state changes (start/stop/error)")
self._sms_state.setChecked(True)
ef.addRow(self._sms_state)
```

API errors and failures - Sends a message when a venue call fails. Key `notify_errors`, checked at build.

The last of the four rows that open checked.

`src/gui/settings_dialog.py` — the API errors row

```
self._sms_errors = QCheckBox("API errors and failures")
self._sms_errors.setChecked(True)
ef.addRow(self._sms_errors)
```

P/L threshold alerts - Sends a message when profit or loss passes a dollar figure. Key `notify_pl_threshold`, clear at build.

The first row in the group that opens clear. It gates the dollar figure on the row below it.

`src/gui/settings_dialog.py` — the P/L alert row

```
self._sms_pl = QCheckBox("P/L threshold alerts")
ef.addRow(self._sms_pl)
```

P/L threshold - Sets that dollar figure. Key `pl_threshold_amount`, $1 to $100,000, at $100.

The one spin box in the events group, and the only row there that is not a checkbox.

`src/gui/settings_dialog.py` — the P/L threshold row

```
self._sms_pl_amount = QDoubleSpinBox()
self._sms_pl_amount.setMinimumHeight(28)
self._sms_pl_amount.setRange(1, 100000)
self._sms_pl_amount.setValue(100)
self._sms_pl_amount.setPrefix("$")
ef.addRow("P/L threshold:", self._sms_pl_amount)
```

Low balance warnings - Sends a message when a balance runs low. Key `notify_balance_warning`, clear at build.

No figure on the page sets what counts as low, so the row carries the switch alone.

`src/gui/settings_dialog.py` — the low balance row

```
self._sms_balance = QCheckBox("Low balance warnings")
ef.addRow(self._sms_balance)
```

Exchange connection status - Sends a message when a venue connects or drops. Key `notify_connection_status`, clear at build.

The last event row, and the third of the three that open clear.

`src/gui/settings_dialog.py` — the connection status row

```
self._sms_connection = QCheckBox("Exchange connection status")
ef.addRow(self._sms_connection)
```

Max messages per hour - Caps how many messages one hour may carry. Key `max_messages_per_hour`, 1 to 100, at 20. Below the area the figure shows.

The first of the two rows in the rate limiting group.

`src/gui/settings_dialog.py` — the hourly cap row

```
self._sms_max_hour = QSpinBox()
self._sms_max_hour.setMinimumHeight(28)
self._sms_max_hour.setRange(1, 100)
self._sms_max_hour.setValue(20)
rf.addRow("Max messages per hour:", self._sms_max_hour)
```

Min time between messages - Sets the wait between two messages. Key `cooldown_seconds`, 5 to 300 seconds, at 30. Below the area the figure shows.

The second rate limiting row, and the last control on the page. It is the one spin box here that carries a unit in its own suffix.

`src/gui/settings_dialog.py` — the cooldown row

```
self._sms_cooldown = QSpinBox()
self._sms_cooldown.setMinimumHeight(28)
self._sms_cooldown.setRange(5, 300)
self._sms_cooldown.setValue(30)
self._sms_cooldown.setSuffix(" sec")
rf.addRow("Min time between messages:", self._sms_cooldown)
```

The Carrier list fills from one mapping of carrier name to gateway address.

`src/core/sms_engine.py` — `CARRIER_GATEWAYS`, the first four

```
CARRIER_GATEWAYS = {
    "AT&T": "{number}@txt.att.net",
    "T-Mobile": "{number}@tmomail.net",
    "Verizon": "{number}@vtext.com",
    "Sprint": "{number}@messaging.sprintpcs.com",
```

None of it reaches the SMS engine. Save reads no widget on this page and the schema declares no SMS field. Neither provider label the combo offers matches the value the engine tests for, and the page carries no field for the three Twilio credentials the send path reads.

`src/core/sms_engine.py` — `SMSConfig.provider`, the value the send path tests

```
    provider: str = "email_gateway"
```

Issue #423 carries the page.

`src/core/sms_engine.py` — `SMSConfig`, the three credentials no row fills

```
twilio_sid: str = ""
twilio_auth_token: str = ""
twilio_from_number: str = ""
```

The page persists now. Save writes one stored group and the dialog reads that group back on the way in, so every row reopens on the figure it was left at.

The group is named for the channels this page is to carry rather than for the one it carries today, so a channel added beside the text messages joins it instead of renaming it. One constant in the engine holds that name and every reader takes it from there, so the schema field is the only other place the name is written.

`src/core/settings.py` — the group the schema declares

```
    # The SMS page's twenty-two values. The name carries the channels the page
    # is to hold; sms_engine.sms_config_from_settings reads this group back.
    message_channels: dict = field(default_factory=lambda: asdict(SMSConfig()))
```

One list pairs each stored field with the control carrying it, and the save and the load both walk that list, so no row can be written without also being read.

`src/gui/main_tabs/settings_dialog_surface.py` — the first rows of the pairing

```
SMS_CONFIG_FIELDS = (
    ("enabled", "sms_enabled"),
    ("provider", "sms_provider"),
    ("phone_number", "sms_phone"),
```

The stored group reaches the engine at two moments. The startup path hands it over before the first fill, and the window hands it over again when the dialog saves, so a changed figure applies without a restart.

`main.py` — the startup push, before the first fill

```
    get_sms_engine().update_config(
        sms_config_from_settings(settings.get("message_channels", {}))
    )
```

Each provider entry carries a short code beside its printed label, and the picker stores the code rather than the label. The send path tests that code, so the Twilio entry reaches the Twilio route and the gateway entry reaches the mail route.

`src/gui/main_tabs/settings_dialog_surface.py` — the two provider entries

```
SMS_PROVIDERS = (
    ("Email-to-SMS Gateway", EMAIL_GATEWAY),
    ("Twilio API", TWILIO),
)
```

Carrier has a stored field of its own, and picking one fills the Gateway Email row below it from the number already typed. The row is filled only by that choice, never by a save or by opening the page, so an operator who never touches the picker is never given an address for a carrier they are not on.

`src/gui/settings_dialog.py` — the fill a carrier choice runs

```
built = gateway_address(
    self._sms_carrier.currentText(), self._sms_phone.text()
)
if built:
    self._sms_gateway.setText(built)
```

A gateway takes the national digits alone, so a leading country digit is dropped and a number of any other length builds nothing. The manual choice at the end of the list builds nothing by design, and a row the carrier cannot fill keeps whatever is in it.

`src/core/sms_engine.py` — the address a carrier builds

```
def gateway_address(carrier: str, number: str) -> str:
    template = CARRIER_GATEWAYS.get(carrier, "")
    if not template:
        return ""
```

Five rows are new. Three fill the Twilio credentials the send path reads, with the auth token masked like the mailbox password above it, and two set the mail server and the port the email route opens. The page holds twenty-two controls now, one for every field of the message configuration.

`src/gui/settings_dialog.py` — the mail server and port rows

```
self._sms_smtp_server = QLineEdit()
self._sms_smtp_server.setMinimumHeight(28)
self._sms_smtp_server.setPlaceholderText(built.smtp_server)
pf.addRow("Mail Server:", self._sms_smtp_server)
```

The figure that set what counts as a low balance is gone. No row set it, nothing read it, and no sentence above asks for one, so the low balance row carries its switch alone as it always did. A settings file still holding the old key loads with every other value intact, and the engine names the key it dropped.

`src/core/sms_engine.py` — the line the engine prints for a name it does not hold

```
        logger.warning(
            "message_channels %r is not a message setting; ignored", name
        )
```

A value the engine cannot use is refused instead of reaching it. A provider outside the two entries, a switch that is neither on nor off, a cap or a cooldown outside its range, and a carrier with no gateway are each dropped with a line on the log and the build figure kept.

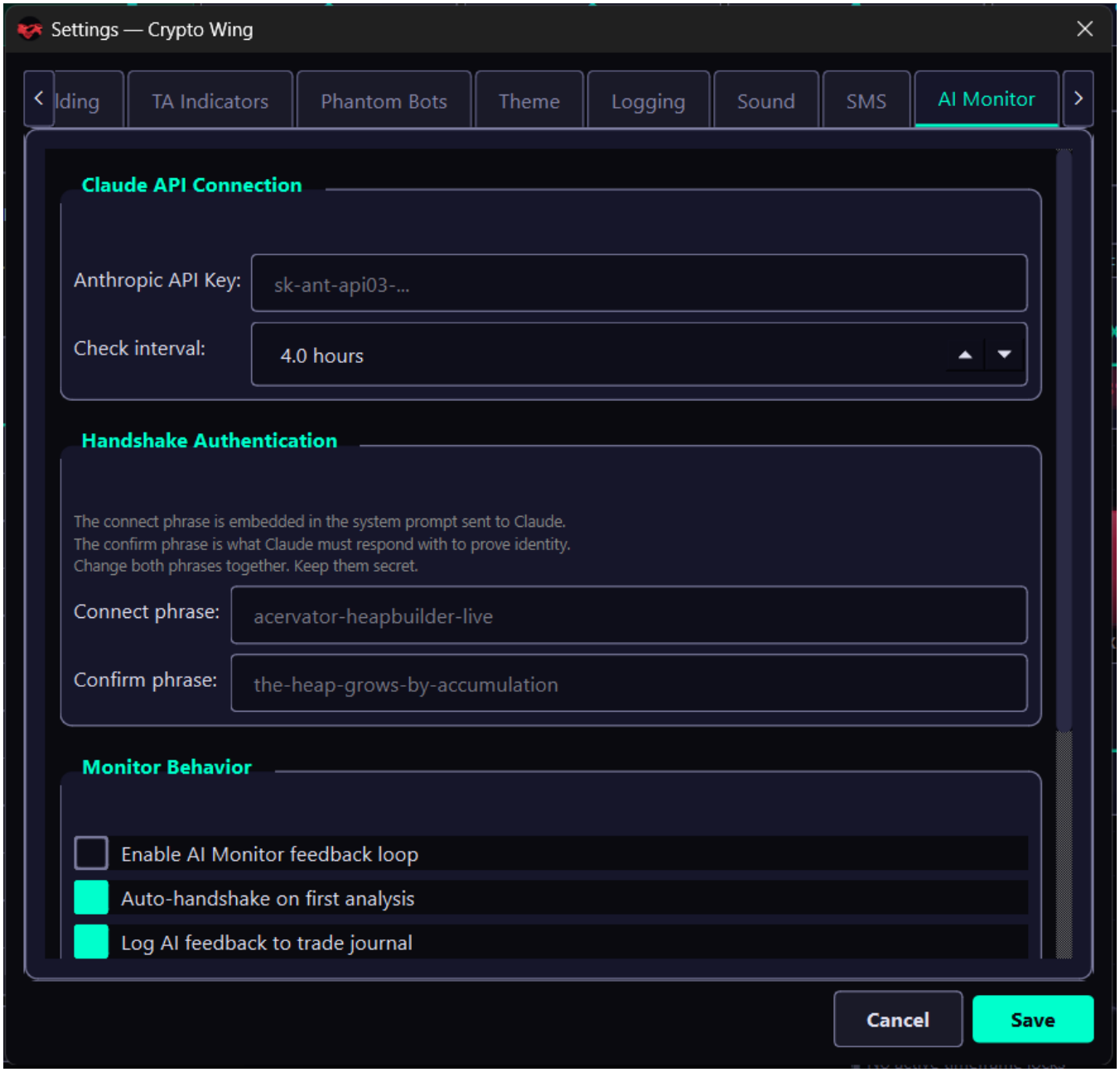
`src/core/sms_engine.py` — the range each numeric row may hold

```
FIELD_BOUNDS: dict[str, tuple[float, float]] = {
    "smtp_port": (1, 65535),
    "pl_threshold_amount": (1.0, 100000.0),
    "max_messages_per_hour": (1, 100),
    "cooldown_seconds": (5, 300),
}
```

The eight event rows wait on a trigger. Each one names an event type the engine checks before it sends, and nothing in the application calls the send method with one of those types, so no fill, no bot state and no venue error reaches a message yet.

In development.

### Settings > AI Monitor




Settings, the AI Monitor page.

A scrolling page holding four groups.

Seven settings, and every key below sits inside one stored `ai_monitor` group.

Anthropic API Key - Holds the key the monitor authenticates with. Key `api_key`, masked, empty.

The first row of the Claude API Connection group. It is masked like the SMS password, and its placeholder shows the shape of a key rather than a key.

`src/gui/settings_dialog.py` — the API key row

```
self._ai_api_key = QLineEdit()
self._ai_api_key.setMinimumHeight(28)
self._ai_api_key.setEchoMode(QLineEdit.Password)
self._ai_api_key.setPlaceholderText("sk-ant-api03-...")
af.addRow("Anthropic API Key:", self._ai_api_key)
```

Check interval - Sets how long the monitor waits between two reviews. Key `interval_hours`, 0.5 to 24.0 hours, at 4.0.

The one figure on the page. It carries a decimal place, so half an hour is the shortest wait it accepts.

`src/gui/settings_dialog.py` — the check interval row

```
self._ai_interval = QDoubleSpinBox()
self._ai_interval.setMinimumHeight(28)
self._ai_interval.setRange(0.5, 24.0)
self._ai_interval.setValue(4.0)
self._ai_interval.setSuffix(" hours")
self._ai_interval.setDecimals(1)
af.addRow("Check interval:", self._ai_interval)
```

Connect phrase - Sets the phrase the platform puts into the prompt it sends. Key `connect_phrase`, empty.

The first of the two handshake rows. A grey note above the pair states the rule the two follow, and the placeholder shows the shape of a phrase.

`src/gui/settings_dialog.py` — the connect phrase row

```
self._ai_connect_phrase = QLineEdit()
self._ai_connect_phrase.setMinimumHeight(28)
self._ai_connect_phrase.setPlaceholderText("acervator-heapbuilder-live")
hf.addRow("Connect phrase:", self._ai_connect_phrase)
```

Confirm phrase - Sets the phrase the answer must carry back to prove it came from the same conversation. Key `confirm_phrase`, empty. Change both phrases together and keep both secret.

The second handshake row, built like the first. Neither field is masked.

`src/gui/settings_dialog.py` — the confirm phrase row

```
self._ai_confirm_phrase = QLineEdit()
self._ai_confirm_phrase.setMinimumHeight(28)
self._ai_confirm_phrase.setPlaceholderText("the-heap-grows-by-accumulation")
hf.addRow("Confirm phrase:", self._ai_confirm_phrase)
```

Enable AI Monitor feedback loop - Turns the monitor on. Key `enabled`, clear at build.

The one control on the page that opens clear. Save emits a signal and the main window reads the whole group back, so this box and the key beside it decide whether the header reads READY or OFF.

`src/gui/main_window.py` — `_on_settings_changed`, the monitor branch

```
ai_cfg = self._settings.get("ai_monitor", {})
if self._bot_manager:
    self._bot_manager.configure_live_monitor(ai_cfg)
    if ai_cfg.get("enabled") and ai_cfg.get("api_key"):
        self._ai_monitor_label.setText("AI: READY")
```

Auto-handshake on first analysis - Runs the phrase exchange on the first review rather than waiting for one to be asked for. Key `auto_handshake`, checked at build.

The first of the two Monitor Behavior rows that open checked.

`src/gui/settings_dialog.py` — the auto-handshake row

```
self._ai_auto_handshake = QCheckBox("Auto-handshake on first analysis")
self._ai_auto_handshake.setChecked(True)
bf.addRow(self._ai_auto_handshake)
```

Log AI feedback to trade journal - Writes each answer into the trade journal. Key `log_feedback`, checked at build.

The last control before the status group, and the second of the two that open checked.

`src/gui/settings_dialog.py` — the journal logging row

```
self._ai_log_feedback = QCheckBox("Log AI feedback to trade journal")
self._ai_log_feedback.setChecked(True)
bf.addRow(self._ai_log_feedback)
```

Connection Status closes the page, below the area the figure shows: a status line, a journal hash, a completed-check count and a Test Handshake button. The three readings report and the button acts. None of the four stores anything.

Save writes all seven controls into one dictionary and the load path reads all seven back, which makes this the one page besides User whose whole state round-trips.

`src/gui/settings_dialog.py` — `_load_current`, the seven it restores

```
ai = self._sm.get("ai_monitor", {})
self._ai_api_key.setText(ai.get("api_key", ""))
self._ai_interval.setValue(ai.get("interval_hours", 4.0))
self._ai_connect_phrase.setText(ai.get("connect_phrase", ""))
self._ai_confirm_phrase.setText(ai.get("confirm_phrase", ""))
self._ai_enabled.setChecked(ai.get("enabled", False))
self._ai_auto_handshake.setChecked(ai.get("auto_handshake", True))
self._ai_log_feedback.setChecked(ai.get("log_feedback", True))
```

Test Handshake runs no handshake. It checks that the key and both phrases hold text, then writes either an error line or the message saying the handshake runs on the next bot cycle.

`src/gui/settings_dialog.py` — `_test_ai_handshake`

```
self._ai_status.setText("Settings saved — handshake runs on next bot cycle")
self._ai_status.setStyleSheet("color: #00ddff; font-weight: bold;")
```

The button reports the fields it read, never a venue answer. Issue #327 carries that.

Six of the seven do reach the engine. `configure_live_monitor` in `src/trading/bot_container.py` reads five of them and builds or clears the monitor, and the main window reads the journal switch when an answer arrives. The auto-handshake box is the seventh: written, read back, and read by nothing else.

`src/trading/bot_container.py` — `configure_live_monitor`, the five keys it reads

```
self._live_monitor = LiveMonitor(
    api_key=settings["api_key"],
    journal=journal,
    interval_hours=settings.get("interval_hours", 4.0),
    connect_phrase=settings.get("connect_phrase", ""),
    confirm_phrase=settings.get("confirm_phrase", ""),
)
```

## 2026-09-11 - Save keeps what the store holds

`_save` and `_load_current` read one declaration. `_stored_rows` names every setting the store holds at its top level. `_stored_groups` names every group the store holds as one key, with the rows inside it. One row carries the store key, the read off its control, the write back into that control, and what stands in for a store without the key. Both methods walk the same rows. No setting can be written without also being loaded.

`src/gui/settings_dialog.py` — one row of `_stored_rows`

```
("bot_visibility",
 self._visibility.currentText,
 lambda value: self._show_text(self._visibility, value),
 "orderbook"),
```

`_show_stored` puts one stored value into one control. A value the control refuses leaves that control on its build figure and names the key in the log. One unreadable entry in the settings file cannot stop the dialog opening.

`SettingsDialogReact` replaces `_setup_ui` alone, so the React build and the Qt build load and save the same way.

### What each page restores now

This table replaces the restores column in What each page persists. Every other column holds.

| Page | Controls | `_save` writes | `_load_current` restores |
|---|---|---|---|
| User | 1 | the row | the row |
| Exchanges | 6 plus 3 buttons | on Add and Remove | the configured list |
| Trading | 6 | all six | all six |
| Profit Folding | 11 | the whole group | the whole group |

| Page | Controls | `_save` writes | `_load_current` restores |
|---|---|---|---|
| TA Indicators | 12 | nothing | nothing |
| Phantom Bots | 13 | nothing | nothing |
| Theme | 6 | all six | all six |
| Logging | 6 | the whole group | the whole group |
| Sound | 10 plus 6 buttons | nothing | nothing |
| SMS | 17 | nothing | nothing |
| AI Monitor | 7 | all seven | all seven |

### The fourteen rows a Save overwrote

Save wrote these fourteen rows and no load read them back. A Save with no edit wrote the build figure over the stored value. Each of the fourteen now reopens on what was stored.

| Page | Rows |
|---|---|
| Trading | Bot Visibility, Enable aggressive trading mode |
| Profit Folding | Distribution Mode, Profit Folding Target, Fold to X# of buy positions, Upward Distribution Target, Distribute to X# of sell positions |
| Theme | Font Family, Base Font Size, Heading Font Size, Log Font Size |
| Logging | Log TA signal samples with all values and timestamps, Highlight entries near Scrumming Bot trades, P/L Log Periodicity |

What each row means has not changed. No reader outside the dialog reads any of the fourteen. Whether each one is wired to a reader or taken off the page is a separate question, and the Settings audit carries it row by row.

### The sentences this section replaces

Each sentence below stands in an earlier dated section and no longer describes the code. The earlier text stays where it is.

| Earlier sentence | What the code does now |
|---|---|
| "Save writes the name and the load path never reads it." | The load path reads `bot_visibility`. The box opens on the stored name. |
| "The box opens on orderbook whatever was stored, and a Save with no edit writes that first entry back over the stored value." | The box opens on the stored name. A Save with no edit writes that same name back. |
| "Save writes it, the load path skips it, and the box opens clear." | The load path reads `aggressive_trading`. The box opens ticked when the store holds true. |
| "Save writes all six keys. The load path restores the first four, which is why the last two rows on the page open at their built-in defaults." | The load path restores all six keys on the Trading page. |
| "This is the one control on the page the load path restores, so it is the only one of the six that reopens on what was stored." | The load path restores all six keys in the stored `profit_folding` group. |

| Earlier sentence | What the code does now |
|---|---|
| "Nothing reads any of the four, and the load path does not restore them either, so the whole Font Settings group reopens at its built-in figures." | Nothing outside the dialog reads the four font keys. The load path restores all four, so the Font Settings group reopens on what was stored. |
| "The load path reads none of them back, so every open shows the build state rather than the stored one." | The load path reads all three `data_logging` keys back. Every open shows the stored state. |
| "In development. What the load path should restore for this page has not been settled against the rest of the dialog, so nothing is proposed here." | The load path restores the whole group: both flags and the list of active periodicities. |

## 2026-09-11 - The Username row and the vault phrase

Driven on the real dialog and the real vault. The home was redirected into a scratch directory before `src.core.settings` was imported, so `_DEFAULT_DIR` bound under that directory. No stored credential of the running install was read.

### Every reader of the stored name

The row reaches four readers. Each one ran with a generated name, and each printed the phrase it built.

| Site | What it does with the name | Driven result |
|---|---|---|
| `main.py`, the startup read | Decides whether to write the default | reads the stored name |
| `src/gui/settings_dialog.py`, `_add_exchange` | Builds the phrase, encrypts the key and the secret | encrypts under `qat_<name>_vault` |
| `src/gui/main_window.py`, `_connect_exchange_for_bot` | Rebuilds the phrase, decrypts | opens the stored secret |
| `src/gui/widgets/api_tester_tab.py`, `_do_connect` | Rebuilds the phrase, decrypts | opens the stored secret |

Nothing else reads the stored name. No bot reads it. No log line carries it. No title bar shows it.

`_stored_rows` carries the row that loads the stored name into the control and saves the control back. `SettingsDialogReact` replaces `_setup_ui` alone, and the page declares the row as one line control, so both builds carry the same row.

`src/gui/init_wizard.py` collects a name into `get_results`. `get_results` has no caller, and `InitWizard` is never built. The name on the User page comes from the startup write.

### What a rename does to a stored secret

A secret went in under one name. The row then changed to a second name, and Save ran.

| Question | Observed |
|---|---|
| Does a box warn? | No box appears |
| Is the rename refused? | No. The store holds the new name |
| Is the stored token rewrapped? | No. The token stays the one that was written |
| Does the secret open? | No. Both decrypt sites refuse |

Both decrypt sites raise `Decryption failed — wrong passphrase or corrupted data`. The bot connect reports `Connection failed`, and the API tester label reads `Failed`. Neither message names the rename.

The earlier name typed back opens the same secret again. Nothing records that earlier name, and nothing shows it after the change.

No code handles the rename. The unlock the name will belong to is the Quintessence Wallet, and the Quintessence Wallet is not built.

### The empty middle

`get('username', 'user')` on a store written without the entry returns an empty string, so the `'user'` fallback in the three phrase literals never fires. Such a store builds the phrase `qat__vault`. The startup write closes that gap on the first run.

`MASTER_FORMAT` formats to the same text the three literals carry.

## 2026-09-11 - The exchange picker and the venue list

The Exchange row on the Exchanges page offers fifteen crypto venues. This section records which list fills it, which build paints it, and every place the chosen venue id arrives.

### Which build paints the picker

Two builds draw the row. `src/_variant.py` sets `DEFAULT_VARIANT = REACT`, so a checkout with no baked variant file and no `ACERVATOR_VARIANT` draws the React page:

```
resolve_variant()                 react
draws_react('Settings dialog')    True
surface_class('Settings dialog')  SettingsDialogReact
```

The React page is `src/gui/react_settings_dialog.py`, and it reads its picker items from `src/gui/main_tabs/settings_dialog_surface.py` — `exchange_items`. The Qt build, `src/gui/settings_dialog.py` — `_create_exchange_tab`, reads `SUPPORTED_EXCHANGES` from the connector directly.

Diagram: flowchart, left to right

- ccxt_connector.SUPPORTED_EXCHANGES — + exchange_label -> **settings_dialog.py — _create_exchange_tab**
- ccxt_connector.SUPPORTED_EXCHANGES — + exchange_label -> **settings_dialog_surface.py — crypto_exchange_items**
- settings_dialog_surface.py — crypto_exchange_items -> **react_settings_dialog.py — the loaded page**
- settings_dialog_surface.py — crypto_exchange_items -> **view_model — settings_dialog.state**

### The two lists, entry by entry

The page once carried its own frozen copy of the fifteen labels. Three readings were taken and each offered the same fifteen entries, in the same order, label for label:

| Reading | Where it comes from | Count |
|---|---|---|
| The connector registry | `SUPPORTED_EXCHANGES` through `exchange_label` | 15 |
| The surface list | `settings_dialog_surface.exchange_items` | 15 |
| The loaded React page | the `<option>` elements the page drew | 15 |

In order: Binance (blocked from US), Bitfinex, Bitget (passphrase required), Bitstamp, Bybit (blocked from US), Coinbase, Cryptocom, Gateio, Gemini, Huobi, Kraken, Kucoin (passphrase required), Mexc, Okx (passphrase required), Poloniex. Every label except Coinbase carries a note, because `exchange_label` marks a venue that is blocked from a US address, or untested, or needs a passphrase.

Binance stands first because the list is in id order and `binance` sorts first.

### Every reader of the chosen venue id

The picker writes its id into an `ExchangeConfig`, and `add_exchange` stores it keyed by that id. `gemini` was chosen on both builds and each reader printed what it saw.

| Reader | What it saw |
|---|---|
| `SettingsManager.list_exchanges` | `exchange_id: 'gemini'` |
| `SettingsManager.get_exchange` | the same entry, matched on the id |
| `_load_current`, the Qt Configured list | `Gemini (gemini)` |
| The React Configured list | `Gemini (gemini)` |
| `_remove_exchange`, which reads the id back out of that row | the store emptied |
| `trading_tab_surface.exchange_display_name` | `Gemini`, the tab caption |
| `trading_tab_surface.is_equity_exchange` | `False`, so the crypto layer |
| `CCXTConnector.__init__` | the registry accepts the id |

Five further sites read the stored id rather than the picker: the startup loop in `main.py` that builds one exchange tab per entry, the startup credential report and the bot-connect credential lookup in `src/gui/main_window.py`, the wizard's venue choices, and the API panel's stored-credential lookup. Each matches on the same `exchange_id` field.

Nothing else reads it. No bot reads the picker, and no gate reads it. `src/core/usb_auth.py` looks like a reader and is not one: it reads `CredentialVault.list_exchanges`, a separate store the picker never writes to.

### The drift, measured

The connector is the authority over the id. A venue it does not carry is refused before any object exists:

```
CCXTConnector('notavenue') refused  Unsupported exchange 'notavenue'.
Supported: ['binance', 'coinbase', 'kraken', ... 'cryptocom']
```

With the frozen copy in place, one venue added to the connector alone reached the Qt picker and not the React page:

| Reading | Before the connector gained a venue | After |
|---|---|---|
| The Qt picker | 15 | 16 |

| Reading | Before the connector gained a venue | After |
|---|---|---|
| The surface list | 15 | 15 |
| The bridge list | 15 | 15 |

A venue offered on the page and missing from the registry would be stored and then refused at the first connect. A venue in the registry and missing from the page would never appear.

### What changed

`crypto_exchange_items` in `src/gui/main_tabs/settings_dialog_surface.py` now builds the pairs from `SUPPORTED_EXCHANGES` and `exchange_label`, imported when first asked so the module still loads no exchange library. The frozen copy is gone, and the registry is the only list of venues.

The screen offers the same fifteen venues in the same order as before. The Qt picker, the surface list, the bridge list and the stock-wing list were read from a tree at the earlier commit and from the changed tree, and the two readings agree in every entry. The same connector mutation now reaches all three crypto readings together.

Which venues are supported is still decided in one place only: `src/exchange/ccxt_connector.py`.

## 2026-09-11 - The API Key row, from the typed string to the stored token

The API Key row on the Exchanges page was typed into on the running React page, the Add button was pressed, and every place the typed string could come to rest was then searched. This section records what was found. The string used was a throwaway that is not a key, and no venue was contacted.

### Where the typed key goes, and where it stops

The row is an entry field and nothing else. The operator fills it, the Add handler reads it once, and the row is cleared.

Diagram: flowchart, left to right

- the API Key row — new_api_key -> **api_validator.validate_credentials — the venue check**
- api_validator.validate_credentials — the venue check -> **core.encryption.encrypt — PBKDF2 then AES-256-GCM**
- core.encryption.encrypt — PBKDF2 then AES-256-GCM -> **settings.toml — exchanges api_key_enc**
- settings.toml — exchanges api_key_enc -> **core.encryption.decrypt**
- core.encryption.decrypt -> **CCXTConnector.sync_connect — apiKey**

`src/gui/settings_dialog.py` — `_add_exchange`, the two lines the row causes

```
master = f"qat_{self._sm.get('username', 'user')}_vault"
config.api_key_enc = encrypt(key, master)
```

| Moment | What happens to the row |
|---|---|
| At dialog load | Nothing. No stored value is ever put back into it |
| On the Save button | Nothing. The row is not in the save path |
| On Test and Add Exchange | The venue check runs, then the key is encrypted and the file is written |

| Moment | What happens to the row |
|---|---|
| Straight after that press | The row is cleared |
| On the next cycle | Nothing. No running bot re-reads the row |
| On restart | The stored token is decrypted fresh on each connect |

A thirty-two character key was stored as a token of one hundred and four characters, and the longer Coinbase string as one of one hundred and fifty-six. The typed string appears nowhere in the stored entry.

### Every reader of the stored key

| Reader | What it does |
|---|---|
| `src/gui/main_window.py` — `_connect_exchange_for_bot` | decrypts it for a bot connect |
| `src/gui/widgets/api_tester_tab.py` — `_do_connect` | decrypts it for the API panel |
| the same two methods, one step earlier | refuse the connect while the field is empty |
| `src/gui/main_window.py`, the startup credential report | reads presence only, never the value |

Both decrypt paths were driven and each recovered the typed string byte for byte, then handed those same bytes to the connector. Nothing in the frontend reads the stored field, measured by searching the whole index in three naming conventions. A separate credential store in `src/core/encryption.py` carries a field of the same name, and this row never writes there.

### The clear-text hunt, twenty places

Twenty places were searched for the typed string after the press. Three hold it, and each of the three is a consumer that cannot do its work without it.

| Place | Result |
|---|---|
| every file under the settings directory | not found |
| every log file the run produced | not found |
| the payload the dialog pushes to the page | not found |
| the page store values, labels, tooltips, calls, prints and boxes | not found |
| the rendered page markup | not found |
| the feedback line and the added-exchange box | not found |
| the message the bot-connect reader returned | not found |
| the API panel result view and status label | not found |
| every log record the run emitted, and the API Log ring buffer | not found |
| the API Key field the page draws | found, because the field shows what is typed |
| the venue check the Add ran first | found, because it authenticates with the key |
| the connector call each reader makes | found, because it authenticates with the key |

Two controls prove the search can report. The typed string was written into the stored settings file, the search named that file, the file was put back and its digest matched. The typed string was then written through the application's own logger, the search named both the log file and the captured record, and that file was put back to the same digest.

```
hunt with the plant      [('...\.acervator\settings.toml', 1218)]
bytes unchanged by hash  True
log file hunt with it    [('...\.acervator_logs\console\system.log', 10258)]
bytes unchanged by hash  True
```

A wrong Username refuses rather than returning a wrong key, and the refusal names no part of the key:

```
decrypt refused                  ValueError: Decryption failed - wrong passphrase or corrupted data
the refusal names the typed key  False
```

### What the row echoes

The row is drawn as a text field, so the key is readable on screen while it is typed. Three credential rows on the same dialog are drawn as password fields:

| Row | Control kind | Echo |
|---|---|---|
| API Key | line | none |
| API Secret | text area | none, and a text area carries no echo mode |
| Passphrase | line | password |
| Anthropic API Key | line | password |
| SMTP Password | line | password |

The Qt build sets no echo mode on this row either, so both builds agree. Masking the key alone would hide nothing, because the secret beside it authenticates with it and stays in view. The complete change reaches the API Secret row, which is held as its own unit, so it is recorded here and not made.

### The Coinbase CDP key string

The page says the row takes a plain key or a Coinbase CDP key string. Both were typed and both round-trip:

```
the CDP string decrypts byte for byte                 True
the stored entry carries the CDP string in the clear  False
```

The row treats the two alike. It accepts any string, and the venue check in front of the store is what refuses a key the exchange will not take. The key shape is read later, at connect, by `src/exchange/ccxt_connector.py` — `sync_connect`, which reports the format it detected and writes only the last four characters of the key into the API log.

## 2026-09-11 - The API Secret row converts a pasted PEM

The row takes a plain secret or a PEM elliptic-curve private key. A paste that carried every newline as the two characters backslash and n is converted before the secret is encrypted, so the stored token holds the real key and every venue is handed the same bytes.

`src/core/encryption.py` holds the two functions the conversion rests on. `looks_like_pem` reports whether the text carries both `PEM_ARMOUR` lines. `unescape_pem_newlines` replaces every escaped newline, and breaks the armour lines off the body when a PEM arrives on one line.

### Where the conversion runs

| Site | What it converts |
|---|---|
| `src/gui/settings_dialog.py` — `_typed_secret` | the row, read by the venue check and by the store write |
| `src/gui/main_tabs/settings_dialog_surface.py` — `_typed_credentials` | the same row in the Qt-free model |
| `src/exchange/ccxt_connector.py` — `sync_connect` | a Coinbase secret stored before the dialog converted |

`SettingsDialogReact` inherits `_add_exchange`, so the React build runs `_typed_secret` through a `PageTextArea` holder.

`_typed_secret` converts only a value `looks_like_pem` accepts. A plain secret that carries a literal backslash and n reaches the store unchanged.

### What the row stores

A throwaway PEM whose body reads THIS-IS-NOT-A-KEY was pasted into the row and the Add button was pressed. The stored token was then decrypted.

```
the escaped paste          3 escaped, 0 real newlines
decrypted out of the store 0 escaped, 3 real newlines
```

A PEM pasted with real newlines already is stored byte for byte, and so is a plain secret carrying a literal backslash and n.

### What each venue is handed

At the point `sync_connect` builds the CCXT config, from the stored value the escaped paste produced:

| Venue | Escaped | Real newlines | Usable PEM |
|---|---|---|---|
| coinbase | 0 | 3 | yes |
| kraken | 0 | 3 | yes |

A second connect on the same stored value produces the same bytes, so the Coinbase branch in `sync_connect` never converts a converted secret twice.

### What the row does not mask

The row is a text area and a text area carries no echo mode, so the secret is readable on screen while it is typed. The API Key row beside it is a text field with no echo mode either, and the two authenticate as a pair. Masking one of them hides nothing, and masking this one would mean replacing the text area with a single line that cannot take a multi-line PEM. Both rows are recorded and neither is changed.

## 2026-09-11 - The Passphrase row and the venue that needs one

The box labelled **This exchange uses an API passphrase** decides whether the field under it is drawn. Picking Kucoin, Okx or Bitget ticks the box, and the field appears with it on the React build and on the Qt build alike.

### Which trigger draws the field

One method decides the field's visibility, and three triggers call it.

`src/gui/settings_dialog.py` — `_sync_passphrase_row`

```
def _sync_passphrase_row(self) -> None:
    self._new_passphrase.setVisible(self._pp_check.isChecked())
```

Diagram: flowchart, left to right

- Click on the box, Qt -> **_sync_passphrase_row**
- Click on the box, React page -> **_sync_passphrase_row**
- Venue pick, _on_exchange_changed -> **_sync_passphrase_row**
- _sync_passphrase_row -> **new_passphrase drawn or hidden**

| Trigger | Where it is wired |
|---|---|
| A click on the box | `_pp_check.toggled`, on the Qt build |
| A click on the box | `SettingsDialogReact.apply_edit`, on a `pp_check` edit from the page |
| A venue pick | `_on_exchange_changed`, which the React dialog inherits and runs |

`_on_exchange_changed` sets the box with `setChecked` and then calls the method. Qt emits `toggled` from `setChecked` and the React page-backed holder emits nothing, so the direct call is what draws the field on each build.

The Qt-free model carries the same rule for the page the dialog opens on. `SettingsDialogModel.on_exchange_changed` calls its own `show_passphrase`, so the Exchanges page draws the field for a venue that needs one from the first paint.

### The row read on each venue

Driven on the real React page and on the Qt dialog, with the Exchanges tab open.

| Venue | Needs one | Box ticked | Field drawn |
|---|---|---|---|
| kucoin | yes | yes | yes |
| okx | yes | yes | yes |
| bitget | yes | yes | yes |
| binance | no | no | no |
| coinbase | no | no | no |
| kraken | no | no | no |

Both builds report that table. A direct click on the box draws the field for a venue that needs no passphrase, and a second click hides it again.

### What the typed value reaches

The field takes focus once it is drawn, and the typed string runs the whole stored path.

| Step | What it carries |
|---|---|
| The field | the typed passphrase |
| `validate_credentials` | the same string, as its fourth argument |

| Step | What it carries |
|---|---|
| `config.passphrase_enc` | an AES-256-GCM token, 84 characters |
| `decrypt` | the typed passphrase, byte for byte |

Test Connection answers `Kucoin requires an API passphrase.` while the field is empty, and the venue is never reached. With the field filled, the connector is handed the key, the secret and the passphrase together.

### Where the venue set is declared

Three declarations hold the same three ids.

| Declaration | Read by |
|---|---|
| `src/exchange/ccxt_connector.py` — `PASSPHRASE_EXCHANGES` | the Qt dialog, `src/exchange/api_validator.py`, `sync_connect` |
| `src/gui/main_tabs/settings_dialog_surface.py` — `PASSPHRASE_EXCHANGE_IDS` | the React dialog and the Qt-free model |
| `src/gui/main_tabs/init_wizard_surface.py` — `PASSPHRASE_EXCHANGE_IDS` | the first-run wizard |

`_sync_passphrase_row` reads the box and no set, so the field follows the box whichever declaration ticked it.

## 2026-09-11 - The saved exchange list

The list under Configured Crypto Exchanges is the stored `exchanges` field. Each entry is one saved venue: its id, its display name, three encrypted tokens, an enabled flag and two hardware fields. The list was driven on the running React page, two venues at a time.

### Where the list is written

Two buttons write it, and Save is not one of them.

Diagram: graph, left to right

- Test and Add Exchange — add_btn -> **settings_dialog.py — _add_exchange / _remove_exchange**
- Remove Selected — remove_btn -> **settings_dialog.py — _add_exchange / _remove_exchange**
- settings_dialog.py — _add_exchange / _remove_exchange -> **settings.py — add_exchange / remove_exchange**
- settings.py — add_exchange / remove_exchange -> **settings.toml — exchanges**

On React a click on a list line reports its index, and `run_action` turns that into `setCurrentRow`, so Remove Selected has a row to drop. `_save` never names `exchanges`, so pressing Save changes nothing here.

### Every reader of the saved list

Ten readers, all through `SettingsManager.list_exchanges` or `get_exchange`.

| Site | What it does with the list |
|---|---|
| `main.py` | One `add_exchange_tab` per entry, in stored order |

| Site | What it does with the list |
|---|---|
| `src/gui/main_window.py` — `_report_stored_credentials_on_startup` | An empty list is a warning and an early return |
| `src/gui/main_window.py` — `_connect_exchange_for_bot` | Walks the list for the bot's id; a missing id and a missing key are two refusals |
| `src/gui/main_window.py` — `_sync_exchange_tabs` | Adds a tab for every listed id that has none |
| `src/gui/main_window.py` — `_create_bot` | Hands the whole list to `BotCreationWizard` |
| `src/gui/widgets/api_tester_tab.py` — `_do_connect` | Walks the list for the picked id and decrypts its tokens |
| `src/gui/settings_dialog.py` — `_load_current` | Fills the on-screen list, filtered by wing |
| `src/gui/main_tabs/settings_dialog_surface.py` — `_load_current` | The same fill in the Qt-free model |
| `src/gui/main_tabs/header_strip_surface.py` — `configured_exchange_count` | The length, and nothing else |
| `src/gui/main_tabs/trading_tab_surface.py` — `live_view_model` | `layer_exchanges` splits it into the crypto and stock layers |

`src/core/usb_auth.py` calls a different `list_exchanges`, the one on `CredentialVault`. That is a separate store this list never writes to. No JavaScript file reads the stored list: the page is handed `listed_exchanges`, words already formatted.

### Two adds, in the order added

Two venues were added through the page, each with its own generated credentials.

| Reading | Answer |
|---|---|
| Order added | `binance`, `kucoin` |
| Order stored | `binance`, `kucoin` |
| Binance key decrypts to its own | True |
| Kucoin key decrypts to its own | True |
| Kucoin passphrase decrypts to its own | True |

Each entry carries its own tokens. `list_exchanges` and `get_exchange` both deep-copy, so no reader can change the stored list by holding what it returned.

### Removing one

The first line was clicked and Remove Selected pressed. The store kept the other entry, and its three tokens still decrypt.

| Reading | Answer |
|---|---|
| Lines on screen before | `Binance (binance)`, `Kucoin (kucoin)` |
| Lines on screen after | `Kucoin (kucoin)` |
| The store after | one entry, `kucoin` |
| The survivor's key decrypts | True |
| The survivor's passphrase decrypts | True |

| Reading | Answer |
|---|---|
| The feedback line | `Binance removed.` |

### Adding a venue the list already holds

`add_exchange` stores one entry per venue id, so a second add is never a second entry. It now keeps that entry's place, and a credential token the new entry leaves empty keeps the token the stored entry carried.

| The press | What the store holds after |
|---|---|
| The same venue with new credentials typed | One entry, carrying the newly typed tokens |
| The same venue with both rows empty | One entry, byte for byte what it held before |
| The same venue with the key typed and the secret empty | One entry, byte for byte what it held before |

With two venues stored, re-adding the first leaves the order as `binance`, `kucoin`. The on-screen list is asked whether a line already names the venue, so one venue is drawn once:

| Reading | Answer |
|---|---|
| Lines on screen before the re-add | `Binance (binance)`, `Kucoin (kucoin)` |
| Lines on screen after the re-add | `Binance (binance)`, `Kucoin (kucoin)` |
| Entries in the store | 2 |

`listed_exchange_position` in `src/gui/main_tabs/settings_dialog_surface.py` answers that question, and both the Qt dialog and the Qt-free model call it.

The feedback line still reads `added without credentials` after a press with both rows empty, which names what was typed rather than what is stored. With the key typed and the secret empty it reads `added with credentials (verified)`, because it tests the key alone while `_add_exchange` needs both rows before it contacts a venue or encrypts anything.

### What the readers see on an empty list

An empty list is the state on a fresh install. Every reader answers, and none raises.

| Reader | Empty | One entry | Two entries |
|---|---|---|---|
| `main.py` venue tabs | none | `kucoin` | `binance`, `kucoin` |
| `configured_exchange_count` | 0 | 1 | 2 |
| The crypto layer | placeholder shown | opens on `kucoin` | opens on `binance` |
| The Settings page list | none | `Kucoin (kucoin)` | `Binance (binance)`, `Kucoin (kucoin)` |
| A bot asking for `kucoin` | `Exchange kucoin not found in settings.` | connects | connects |
| The API panel asking for `kucoin` | `No stored credentials for Kucoin.` | decrypts | decrypts |

An entry stored with no tokens reads differently from a missing entry: the bot path answers `No API credentials for Kucoin.` instead.

### Removing the venue a bot uses

Nothing on this path can see a bot. The dialog is handed the settings manager and the status log, and nothing else.

| Reading | Answer |
|---|---|
| Refused | False |
| Warned about the bot | False |
| Silently allowed | True |
| What the bot's next connect reads | `Exchange binance not found in settings. Add it in Settings first.` |

`_sync_exchange_tabs` only adds a tab, so the removed venue's tab stays on screen until the next launch.

## 2026-09-11 - Position Distance is off the Trading page

The Trading page carries five rows. The Position Distance row is removed. The row set a percent the dialog stored and read back, and no bot read the value.

Three files declared the row and every declaration is gone.

| File | What it declared |
|---|---|
| `src/core/settings.py` | the `position_distance_pct` field on `AppSettings` |
| `src/gui/settings_dialog.py` | the `QDoubleSpinBox` and its `_stored_rows` entry |
| `src/gui/main_tabs/settings_dialog_surface.py` | the `pos_distance` spec, its Trading layout step and its `PERSISTED_ROWS` entry |

`_DEPRECATED_KWARGS` in `src/trading/container/config.py` keeps the name. A `bot_state.json` still carrying the key restores without a `TypeError`.

### Why the row had no wizard control to open

The row set a percent from 1.0 to 50.0 at 2.0, spacing a bot's positions. Three wizard controls carry a spacing and none carries that quantity.

| Control | Key | Range | At | What it spaces |
|---|---|---|---|---|
| Split Distance | `split_distance` | 0.10 % to 20.00 % | 1.00 % | one Stack tranche from the next |
| Spacing | `stack_spacing_mode` | Linear, Quadratic, Exponential | Linear | how that tranche gap grows |
| Opposing Trade Interval | `scrumming_interval_pct` | 0.10 % to 20.00 % | 1.00 % | the price travel a reversal needs |

The key belongs to the retired Grid Bot. The module docstring of `src/gui/bot_wizard.py` names the mode and its rows: "Grid Bot: Investment Amount, Position Count/Distance/Increment". `BotMode` declares `SCRUMMING` and `EXTRACTOR`, and `bot_config_kwargs` names "the retired Grid checkbox" in the same file as the strip list.

### A store that still carries the key

`SettingsManager._apply_dict` walks the fields `AppSettings` declares and copies the ones the file carries. A key the file holds and `AppSettings` does not declare is passed over. A store holding `position_distance_pct = 7.5` opens, its `username` and `default_target_balance` read back, the dialog draws its controls, and the next Save writes the file without the key.

### The sentences the removal replaces

Each sentence below stands in an earlier section and no longer describes the page. The earlier text stays where it is.

| Earlier sentence | What the page does now | | | | | |
|---|---|---|---|---|---|---|
| "Six rows carry the defaults a new bot starts from, not a running bot's settings." | Five rows carry those defaults. | | | | | |
| "Position Distance - Sets the spacing a new bot puts between its positions." and the three sentences and the code block under it | The page carries no Position Distance row, and no control on it sets position spacing. | | | | | |
| "The strip list holds eleven names. This page contributes one of them and the Profit Folding page below contributes four more." | The strip list holds the same eleven names. The Profit Folding page contributes four and this page contributes none. | | | | | |
| "Save writes all six keys." | Save writes five keys. | | | | | |
| "The load path restores all six keys on the Trading page." | The load path restores all five keys on the Trading page. | | | | | |
| "`src/gui/settings_dialog.py` — `_load_current`, the four it restores", and the `self._pos_distance.setValue` line inside that block | `_stored_rows` carries no `position_distance_pct` row, so the load path names four keys and the page has five rows. | | | | | |
| "\ | Trading \ | 6 \ | all six \ | four of six \ | " in What each page persists | Trading, 5 controls, `_save` writes all five, `_load_current` restores all five. |
| "\ | Trading \ | 6 \ | all six \ | all six \ | " in What each page restores now | The same five. |

## 2026-09-11 - Increment Style is off the Trading page

The Trading page carries four rows. The Increment Style row is removed. The row stored one of two words and read it back, and nothing in the program read the word.

Three files declared the row and every declaration is gone.

| File | What it declared |
|---|---|
| `src/core/settings.py` | the `increment_style` field on `AppSettings`, and the `IncrementStyle` enum that supplied its default and its two words |
| `src/gui/settings_dialog.py` | the `QComboBox` and its `_stored_rows` entry |
| `src/gui/main_tabs/settings_dialog_surface.py` | the `increment_style` spec, its Trading layout step, its `PERSISTED_ROWS` entry, and the `INCREMENT_KEY` and `INCREMENT_DEFAULT` names |

`_DEPRECATED_KWARGS` in `src/trading/container/config.py` never held this name, so the strip list is unchanged.

### The bot config field of the same name stays

`increment_style` names a field on two objects that never meet. `AppSettings` held the page's word. `BotConfig` declares its own at `src/trading/container/config.py:59`, `_BOT_CONFIG_SCRUMMING_ONLY_FIELDS` names it at `:307` so an Extractor is refused it, and `src/trading/container/restore.py:220` copies it out of a saved bot. All three stay. The page never wrote any of them, and a bot built through the real creation path holds the same `linear` it held before.

Nothing reads the bot field either. `git grep "\.increment_style"` finds no reader, and the same search shape finds thirteen reads of `.target_balance`, so the absence is a fact about the code rather than about the search.

### Why no wizard control could open at it

A row on this page earns its keep by opening a wizard field at the stored figure, the way Default Target Balance opens the wizard's Target Balance. No wizard control sets an increment style. The wizard collects 52 keys and none is this name. Its nearest control is Spacing, a different quantity under a different key.

| | Increment Style | Spacing |
|---|---|---|
| key | `increment_style` | `stack_spacing_mode` |
| choices | linear, logarithmic | Linear, Quadratic, Exponential |
| at | linear | Linear |
| what it shapes | how the gap between a bot's positions grows | how the gap between Stack tranches grows |

The word belongs to the retired Grid Bot. The module docstring of `src/gui/bot_wizard.py` names the mode and its rows: "Grid Bot: Investment Amount, Position Count/Distance/Increment". The other two of that trio, `position_count` and `position_distance_pct`, already sit on the strip list as grid legacy. This one stayed on `BotConfig` instead, unread.

One place still carries the word `logarithmic` for a spacing, and it carries it only to replace it. `_sanitize_deprecated_kwargs` at `src/trading/container/config.py:577` promotes `stack_spacing_mode="logarithmic"` to `"quadratic"`.

### A store carrying the key after the removal

`SettingsManager._apply_dict` walks the fields `AppSettings` declares and copies the ones the file carries. A key the file holds and `AppSettings` does not declare is passed over rather than refused. A store holding `increment_style = "logarithmic"` opens, its `username` and `default_target_balance` read back, the page draws its controls, and the next Save writes the file without the key.

| | before the removal | after the removal |
|---|---|---|
| drawn controls on the page | 70 | 69 |
| the Increment Style row is drawn | yes | no |
| the Default Positions row beside it | drawn | drawn |
| `username` read back off the store | `u10_operator` | `u10_operator` |
| `default_target_balance` read back | 777.0 | 777.0 |
| `BotConfig.increment_style` on a new bot | `linear` | `linear` |

### The sentences this removal replaces

Each sentence below stands in an earlier section and no longer describes the page. The earlier text stays where it is.

| Earlier sentence | What the page does now | | | | | |
|---|---|---|---|---|---|---|
| "Six rows carry the defaults a new bot starts from, not a running bot's settings." | Four rows carry those defaults. | | | | | |
| "Increment Style - Chooses whether that spacing stays even or widens on a curve." and the two sentences and the code block under it | The page carries no Increment Style row, and no control on it chooses how spacing grows. | | | | | |
| "A bot takes its own increment style from the wizard rather than from this page." | No wizard control sets an increment style, and nothing reads the `BotConfig` field of that name. | | | | | |
| "Save writes all six keys." | Save writes four keys. | | | | | |
| "The load path restores all six keys on the Trading page." | The load path restores all four keys on the Trading page. | | | | | |
| "`src/gui/settings_dialog.py` — `_load_current`, the four it restores", and the three `self._increment_style` lines inside that block | `_stored_rows` carries no `increment_style` row, so the load path names three keys and the page has four rows. | | | | | |
| "\ | Trading \ | 6 \ | all six \ | four of six \ | " in What each page persists | Trading, 4 controls, `_save` writes all four, `_load_current` restores all four. |
| "\ | Trading \ | 6 \ | all six \ | all six \ | " in What each page restores now | The same four. |

The 2026-09-11 Position Distance section above restated the same rows at five. Four is the count after both rows are gone.

## 2026-09-11 - Default Positions is off the Trading page

The Trading page carries three rows. The Default Positions row is removed. The row stored a count between 1 and 100 and read it back, and nothing outside the dialog read the count.

Three files declared the row and every declaration is gone.

| File | What it declared |
|---|---|
| `src/core/settings.py` | the `default_position_count` field on `AppSettings` |
| `src/gui/settings_dialog.py` | the `QSpinBox` with its `setRange(1, 100)` and its form row, and its `_stored_rows` entry |
| `src/gui/main_tabs/settings_dialog_surface.py` | the `default_positions` spec, its Trading layout step, and its `PERSISTED_ROWS` entry |

`AppSettings` no longer declares the name, so `SettingsManager.set` refuses it. Driven after the removal, `set("default_position_count", 42)` raised `KeyError: 'Unknown setting: default_position_count'`. That is the schema doing its stated job rather than a fault.

### The bot-side spelling stays on the strip list

`position_count` is the name a bot-side kwarg would carry, and `_DEPRECATED_KWARGS` in `src/trading/container/config.py:384` holds it beside unit 9's `position_distance_pct` at `:385`. Both stay on the list, so a `bot_state.json` still carrying either key restores without a `TypeError`. `_sanitize_deprecated_kwargs` drops `position_count` before `BotConfig.__init__`, and `BotConfig` declares no field of that name and no field of this row's name.

The word belongs to the retired Grid Bot. The module docstring of `src/gui/bot_wizard.py:6-7` names the mode and its rows: "Grid Bot: Investment Amount, Position Count/Distance/Increment". `BotMode` at `src/trading/container/config.py:39-41` declares `SCRUMMING` and `EXTRACTOR` alone. With this row gone, all three of that group are off the page.

### No wizard control counts the positions a bot opens with

A row on this page earns its keep by opening a wizard field at the stored figure, the way Default Target Balance opens the wizard's Target Balance. The wizard collects 52 keys and none of them is a count of positions a new bot opens with. The nearest control is Tranche Count, and it counts something else.

| | Default Positions | Tranche Count |
|---|---|---|
| key | `default_position_count` | `stack_tranche_count_target` |
| range | 1 to 100 | 2 to 20 |
| at | 10 | 3 |
| what it counts | positions a new bot opens with | Stack tranches created from one SCRUM |

The meaning, the range and the default all differ, so the row had no wizard field to open at its figure.

One site in the repository counts positions, and it counts ones already open. `open_position_count` at `src/gui/main_tabs/stock_main_window_surface.py:545` reads a live status list and reports how many positions stand now, on the stock window. That is a runtime count rather than an opening count, and it never touches this row's name.

### A legacy store after the removal

`SettingsManager._apply_dict` walks the fields `AppSettings` declares and copies the ones the file carries. A key the file holds and `AppSettings` does not declare is passed over rather than refused. A store holding `default_position_count = 55` opens, its `username` and `default_target_balance` read back, the page draws its controls, and the next Save writes a file without the key while leaving the stored `username` intact.

| | before the removal | after the removal |
|---|---|---|
| drawn controls on the page | 69 | 68 |
| control specs in the payload | 70 | 69 |
| the Default Positions row is drawn | yes | no |
| the Default Target Balance row beside it | drawn | drawn |
| keys and groups one Save wrote | 14 | 13 |
| `username` read back off the store | `u11_driver` | `u11_driver` |
| a legacy store's `username` | `u11_legacy` | `u11_legacy` |
| a legacy store's `default_target_balance` | 321.0 | 321.0 |
| a legacy store's `default_position_count` | 55 | not answered |
| `stack_tranche_count_target` on a new bot | 3 | 3 |

### The sentences this row's removal replaces

Each sentence below stands in an earlier section and no longer describes the page. The earlier text stays where it is.

| Earlier sentence | What the page does now | | | | | |
|---|---|---|---|---|---|---|
| "Six rows carry the defaults a new bot starts from, not a running bot's settings." | Three rows carry those defaults. | | | | | |
| "Default Positions - Sets how many positions a new bot opens with." and the key line, the two sentences and the code block under it | The page carries no Default Positions row, and no bot holds a count of positions it opens with. | | | | | |
| "The dialog stores the count and restores it." | The dialog stores and restores nothing under this name. | | | | | |
| "Save writes all six keys." | Save writes three keys. | | | | | |
| "The load path restores all six keys on the Trading page." | The load path restores all three keys on the Trading page. | | | | | |
| "`src/gui/settings_dialog.py` — `_load_current`, the four it restores", and the `self._default_positions.setValue` line inside that block | `_stored_rows` carries no `default_position_count` row, so of the four lines in that block one remains and the page has three rows. | | | | | |

| Earlier sentence | What the page does now | | | | | |
|---|---|---|---|---|---|---|
| "\ | Trading \ | 6 \ | all six \ | four of six \ | " in What each page persists | Trading, 3 controls, `_save` writes all three, `_load_current` restores all three. |
| "\ | Trading \ | 6 \ | all six \ | all six \ | " in What each page restores now | The same three. |

The 2026-09-11 Increment Style section above restated the same rows at four. Three is the count after all three of the Grid Bot group are gone.

## 2026-09-11 - Default Target Balance opens the wizard's field

The Trading page carries three rows. This row is the one that reaches a new bot. It stores a dollar figure between $1.00 and $1,000,000.00 under the key `default_target_balance` and opens at $200.00. The row is verified and unchanged.

### The row, drawn and driven

React draws the row. The build answers `react`, `draws_react('Settings dialog')` answers `True`, and the class is `SettingsDialogReact`. The Trading tab draws 68 controls and this row is one of them.

| What was driven | What it answered |
|---|---|
| the drawn node | `INPUT`, `type="number"`, `min="1"`, `max="1000000"` |
| $777.00 typed on the page | the holder answered 777.0 |
| Save pressed on the page | the store answered 777.0 |
| a fresh manager on the same folder | 777.0 |
| the page reopened | the row drew 777.0 |

The round trip holds. A figure typed on the page survives a Save, a new manager and a new page.

### Every reader of the stored figure

Seven sites name the key. One of them reads it outside the Settings dialog.

| Site | What it does with the figure |
|---|---|
| `src/core/settings.py:157` | declares `default_target_balance: float = 200.0` |
| `src/gui/main_tabs/settings_dialog_surface.py:467` | the React model's save and load row |
| `src/gui/settings_dialog.py:1162-1165` | `_stored_rows`, the read on Save and the write on load |
| `src/gui/bot_wizard.py:810` | the wizard's Target Balance field opens here |
| `src/gui/main_window.py:1735` | writes an override for one wizard opening |
| `src/gui/main_tabs/bot_wizard_surface.py:1224` and `:1702-1716` | the React wizard module's own read |
| `src/gui/main_tabs/bot_wizard_surface.py:2768` | names the key in the payload catalogue |

The tab conversion table gives the wizard's seam row: both builds open `BotCreationWizard`. **+ New Bot** reaches `src/gui/bot_wizard.py:810`, and the React wizard module is a second reader that the button does not reach.

Nothing under `desktop/` names the row. The control specs travel from Python into the page payload, and the drawn node carries `data-name="default_balance"` as data.

### Where the figure stops being this setting

Diagram: flowchart, left to right

- Settings row — default_target_balance -> **SettingsManager — settings.toml**
- SettingsManager — settings.toml -> **C"]**
- main_window._create_bot — get_all( -> **bot_wizard.py:810 — Target Balance field**
- bot_wizard.py:810 — Target Balance field -> **TradingParamsPage.get_config — target_balance**
- TradingParamsPage.get_config — target_balance -> **bot_config_kwargs**
- bot_config_kwargs -> **BotConfig.target_balance**

`TradingParamsPage.get_config` at `src/gui/bot_wizard.py:1533` renames the figure `target_balance`. From that step on the figure is the bot's own setting.

Driven with $4,321.55 stored:

| Step | What it answered |
|---|---|
| the wizard's Target Balance | 4321.55 |
| keys the wizard collected | 52 |
| `default_target_balance` among them | no |
| `target_balance` among them | 4321.55 |
| `bot_config_kwargs target_balance` | 4321.55 |
| the built config | `BotConfig` |
| `BotConfig.target_balance` | 4321.55 |
| `BotConfig` holds `default_target_balance` | no |

Two more stored figures opened the wizard at themselves: $777.00 and $12.75. Market Inspector topology adoption hands `defaults_override` in place of the stored figure, and $25.00 opened the wizard at $25.00.

### The range, at both ends and past them

The page, the wizard and this manual declare the same range. The page and the wizard hold a figure to that range in the same way.

| Written to the store | The store holds | The page draws | The wizard opens at |
|---|---|---|---|
| 1.00 | 1.0 | 1 | 1.0 |
| 1000000.00 | 1000000.0 | 1000000 | 1000000.0 |
| 0.50 | 0.5 | 1 | 1.0 |
| 1000001.00 | 1000001.0 | 1000000 | 1000000.0 |

The row holds a typed figure to the range before the store sees it, so the page stores no figure outside the range. A figure written into the file by hand meets the nearer end on the page and in the wizard alike, so no bot opens at a refused figure.

### A store with the figure removed

| What was driven | What it answered |
|---|---|
| `AppSettings` declares | 200.0 |
| a settings file with the key deleted | 200.0 |
| the wizard on that store | 200.0 |
| the wizard on an empty defaults dict | 200.0 |

`get_all()` is `asdict` of a dataclass that declares the field, so the dict the wizard takes always carries the key. `_apply_dict` keeps the declared default for a field the file omits. The wizard's own fallback at `src/gui/bot_wizard.py:810` fires only where `main_window.py:3185` hands `{}`, which is a run with no settings manager, and the fallback gives the same $200.00.

The page's build figure for this control is $1.00, because the spec carries a range and no `value`. The store always answers, so the page draws $1.00 only where it refuses the stored figure.

### What this section adds

Nothing on the page, in the store or in the wizard changed. The row matches every sentence written about it, and this section records the driven readings.

Two gaps stand outside this row and sit here as measured facts.

| Gap | What was measured |
|---|---|
| a text figure in the settings file | `src/gui/settings_dialog.py:1284-1296` catches it, logs "Settings kept the default for default_target_balance", and draws $1.00. `src/gui/bot_wizard.py:810` carries no guard, so `QDoubleSpinBox.setValue(str)` raises `TypeError` and `BotCreationWizard` cannot open. Only an edited file puts a text figure in the store |
| a write to a control the page already drew | The React page draws every control with `defaultValue`, which seeds the DOM once. A write from Python after the page is up moved the store and not the drawn figure: the node held 321 while the holder answered 654.0. This row stands clear of it, because `_load_current` runs before the page mounts and nothing writes the row afterwards |

## 2026-09-12 - Bot Visibility opens the wizard's Order Visibility control

The Trading page row stores `orderbook` or `internal` under the key `bot_visibility` and opens at `orderbook`. The row is now the default the bot creation wizard's own control opens on. It was unwired, and removing it was refused: the behaviour it names runs on the bot, and the wizard carries a control for it.

### The row and the wizard control, side by side

| | The Settings row | The wizard control |
|---|---|---|
| Where | `src/gui/main_tabs/settings_dialog_surface.py:606-613` | `src/gui/bot_wizard.py:674-686` |
| Label | `Bot Visibility:` | `Order Visibility:` |
| Choices | `orderbook`, `internal` | `orderbook`, `internal` |
| Opens at | `orderbook` | the stored name, `orderbook` when the store holds none |
| What it writes | the store key `bot_visibility` | the config key `visibility` |

The two agree on meaning, on both choices and on the default, so the row is a legitimate application default for the control.

### Every reader of the stored visibility

Four sites name the key. One of them reads it outside the Settings dialog.

| Site | What it does with the name |
|---|---|
| `src/core/settings.py:163` | declares `bot_visibility: str = BotVisibility.ORDERBOOK.value` |
| `src/gui/settings_dialog.py:1166-1169` | `_stored_rows`, the read on Save and the write on load |
| `src/gui/main_tabs/settings_dialog_surface.py:468` | the same row on the Qt-free surface |
| `src/gui/bot_wizard.py:677-683` | the wizard's Order Visibility control opens here |

Before this section there was no reader outside the dialog. The bare name was searched in all 1081 files `git ls-files` reports, and the search was proved against `default_target_balance`, read out of the settings dict, and against `ai_monitor`, read through `getattr`. Both controls answered with their known reading sites, so the zero was a measurement.

### Where the stored name reaches the bot

Diagram: flowchart, left to right

- Settings row — bot_visibility -> **SettingsManager — settings.toml**
- SettingsManager — settings.toml -> **C"]**
- main_window._create_bot — get_all( -> **bot_wizard.py:679 — Order Visibility control**
- bot_wizard.py:679 — Order Visibility control -> **TradingParamsPage.get_config — visibility**
- TradingParamsPage.get_config — visibility -> **BotConfig.visibility**
- BotConfig.visibility -> **scrumming_bot.py:338 — _invisible**

`BotConfig.visibility` at `src/trading/container/config.py:139` is where the name stops being this setting. From there the bot owns it, and `src/trading/scrumming_bot.py:974` changes it live on one bot without touching the store.

### The round trip, with a Save that changed nothing

Driven on the class the running build opens. `resolve_variant` answered `react` and `surface_class('Settings dialog')` returned `SettingsDialogReact`. The home was redirected into a scratch directory before the settings module was imported, so the settings default directory bound under that directory.

| Stored | The box opened at | The store after a Save with no edit | The box on reopening |
|---|---|---|---|
| `internal` | `internal` | `internal` | `internal` |
| `orderbook` | `orderbook` | `orderbook` | `orderbook` |

Two different stored names gave two different readings, so the reading discriminates. The payload the dialog pushes to the page carries 69 control specs, and the figure is the same before and after.

### What a new bot opens at, and what an existing bot keeps

| Driven | What it answered |
|---|---|
| `internal` stored, the wizard opened | `internal` |
| `orderbook` stored, the wizard opened | `orderbook` |
| a name the two items do not offer | `orderbook`, the build figure |
| app setting `internal`, a saved bot holding `orderbook` | the restored config holds `orderbook` |
| app setting `orderbook`, a saved bot holding `internal` | the restored config holds `internal` |

A restored bot reads `cfg.get("visibility", "orderbook")` from its own saved config at `src/trading/container/restore.py:203` and reads no application setting. Every `SettingsManager.get` call of the run was recorded with its calling file: the wizard and the restore paths read the key no times, and the Settings dialog read it once. The recorder saw the read it was meant to see, so the two zeros are measurements.

### The sentence this wiring replaces

| Earlier sentence | What the code does now |
|---|---|
| "A bot takes its visibility from the wizard, under a different name." | A new bot still takes its visibility from the wizard, and the wizard's control now opens at this row's stored name. |

The two revert sentences in the same paragraph were already retired by Save keeps what the store holds, and that section's reading was re-driven here and holds.

### What the wiring adds

One lookup on one control, in the file the + New Bot button reaches. Nothing a running bot does changed, nothing on the Settings page changed and no order placement path changed.

Two absences stand outside this row and sit here as measured facts.

| Absence | What was measured |
|---|---|
| the shell's wizard request carries no stored defaults | `src/gui/web/bot_wizard.js:1775` calls `bot_wizard.state` with the step values alone. `src/gui/main_tabs/bot_wizard_surface.py:2796` reads `defaults` from the request, so the Qt-free wizard surface receives an empty bag on every call and `_apply_stored_target_balance` falls back to its built-in figure |

| Absence | What was measured |
|---|---|
| the Qt-free wizard surface has no visibility lookup | `src/gui/main_tabs/bot_wizard_surface.py:1702` applies the stored target balance and nothing applies a stored visibility. Its `visibility` combo at `:472-474` carries the same two entries as the Qt one and opens at index 0 |

## 2026-09-12 - Enable aggressive trading mode opens the wizard's own box

The Trading page row stores a boolean under the key `aggressive_trading` and opens clear. The row is now the default the bot creation wizard's own Aggressive Trading checkbox opens on. It was unwired, and removing it was refused: the behaviour it names runs on the bot, and the wizard carries a checkbox for it.

### The Settings box and the wizard box, side by side

| | The Settings row | The wizard box |
|---|---|---|
| Where | `src/gui/main_tabs/settings_dialog_surface.py:614-622` | `src/gui/bot_wizard.py:688-699` |
| Text | `Enable aggressive trading mode` | `Aggressive Trading (force IOC-limit takers)` |
| Choices | on, off | on, off |
| Opens at | off | the stored flag, off when the store holds none |
| What it writes | the store key `aggressive_trading` | the config key `aggressive_trading` |

The two agree on meaning, on both choices and on the default, and here the two keys are the same word.

### The two sets that share the word

The bare name `aggressive_trading` stands in 19 python lines of the 1081 files `git ls-files` reports. They split into two sets, and no line crosses between them.

| Set | Lines | What they are |
|---|---|---|
| the application setting | 3 | the `AppSettings` field, the dialog's `_stored_rows` row, the same row on the Qt-free surface |
| a bot's own config field | 16 | the `BotConfig` field, the restore read, the bot's own `_aggressive`, the live toggle, the wizard write and the live settings box |

The bot's own field is a separate setting with its own screen row, and this section changes nothing in that set.

### Every reader of the stored flag

Four sites name the application key. One of them reads it outside the Settings dialog.

| Site | What it does with the flag |
|---|---|
| `src/core/settings.py:164` | declares `aggressive_trading: bool = False` |
| `src/gui/settings_dialog.py:1170-1173` | `_stored_rows`, the read on Save and the write on load |
| `src/gui/main_tabs/settings_dialog_surface.py:469` | the same row on the Qt-free surface |
| `src/gui/bot_wizard.py:689-691` | the wizard's Aggressive Trading box opens here |

Before this section there was no reader outside the dialog. The bare name was searched in every tracked file, and the search was proved against `default_target_balance`, read out of the settings dict, and against `ai_monitor`, read through `getattr`. Both controls answered with their known reading sites, so the zero was a measurement.

### Where the stored flag reaches the bot

Diagram: flowchart, left to right

- Settings row — aggressive_trading -> **SettingsManager — settings.toml**
- SettingsManager — settings.toml -> **C"]**
- main_window._create_bot — get_all( -> **bot_wizard.py:691 — Aggressive Trading box**
- bot_wizard.py:691 — Aggressive Trading box -> **TradingParamsPage.get_config — aggressive_trading**
- TradingParamsPage.get_config — aggressive_trading -> **BotConfig.aggressive_trading**
- BotConfig.aggressive_trading -> **scrumming_bot.py:339 — _aggressive**
- scrumming_bot.py:339 — _aggressive -> **scrumming_bot.py:4551 — OrderType.IOC_LIMIT**

`BotConfig.aggressive_trading` at `src/trading/container/config.py:142` is where the flag stops being this setting. From there the bot owns it, and `src/trading/scrumming_bot.py:1015` changes it live on one bot without touching the store.

### The round trip on both choices

Driven on the class the running build opens. `resolve_variant` answered `react` and `surface_class('Settings dialog')` returned `SettingsDialogReact`. The home was redirected into a scratch directory before the settings module was imported, so the settings default directory bound under that directory. No venue was contacted. A name-lookup guard refused eight lookups, every one of them the wizard's own market scan.

| Stored | The box opened at | The store after a Save with no edit | The box on reopening |
|---|---|---|---|
| on | on | on | on |
| off | off | off | off |

Two different stored flags gave two different readings, so the reading discriminates. The payload the dialog pushes to the page carries 69 control specs and the page draws 68, the same two figures before and after.

### What a new bot carries, and what an existing bot keeps

| Driven | What it answered |
|---|---|
| on stored, the wizard opened | on |
| off stored, the wizard opened | off |
| on stored, through `bot_config_kwargs` and `make_bot_config` | `BotConfig.aggressive_trading` on |
| off stored, the same path | `BotConfig.aggressive_trading` off |
| the setting on, a saved bot holding off | the restored bot holds off |
| the setting off, a saved bot holding on | the restored bot holds on |

A restored bot reads `cfg.get("aggressive_trading", False)` from its own saved config at `src/trading/container/restore.py:204` and reads no application setting. Every

`SettingsManager.get` and `get_all` call of the run was recorded with its calling file. The wizard and the restore paths read the key no times, and the driver read it once. The recorder saw the read it was meant to see, so the two zeros are measurements.

### What this wiring retires

| Earlier sentence | What the code does now |
|---|---|
| "The wizard carries its own aggressive trading checkbox, and that is the one a new bot reads." | A new bot still reads the wizard's checkbox, and that checkbox now opens at this row's stored flag. |

The two revert sentences in the same paragraph were already retired by Save keeps what the store holds, and that section's reading was re-driven here and holds.

### What the one lookup adds

One lookup on one checkbox, in the file the + New Bot button reaches. Nothing a running bot does changed, nothing on the Settings page changed and no order placement path changed.

### The wizard path this wiring does not reach

The Electron shell's wizard request carries no stored defaults, so the Qt-free wizard surface opens its box at the build figure whatever the store holds.

| Absence | What was measured |
|---|---|
| the shell's wizard request carries no stored defaults | `src/gui/web/bot_wizard.js` never names `defaults`, and `src/gui/main_tabs/bot_wizard_surface.py:2806` reads `defaults` out of the request. A request with no bag left `target_balance` at its built-in 200.0 |
| the Qt-free wizard surface has no aggressive lookup | `bot_wizard_surface.view_model` with a bag holding the flag on still answered off. `CHECK_FIELDS` at `:425` seeds the box at `False` and nothing reads the store |
| a hand-edited word in the store reads as on | The store declares a boolean. A word put there by hand ticks the box on the Settings page and in the wizard alike, because both admit the value as `bool(value)` |

## 2026-09-12 - Profit Folding Active waits on the wizard's folding page

Driven on the React build, which is what the running application draws. The home was redirected into a scratch directory before `src.core.settings` was imported, so the settings directory and the log root bound under that directory. No stored setting of the running install was read and no venue was contacted.

The row stays on the page exactly as it is. It stores and restores correctly, and the value it stores has no reader. The one place a new bot could read it is a wizard page no route reaches, so a default pointed at that page would change no reading. The row therefore waits, and this section records what it waits on.

### The Settings box and the wizard box it would open

| | The Settings row | The wizard control |
|---|---|---|
| Where | `src/gui/main_tabs/settings_dialog_surface.py:627` | `src/gui/bot_wizard.py:1590-1597` |
| Kind | checkbox | checkbox |
| Text | `Profit Folding / Upward Distribution Active` | `Enable Profit Folding` |
| Choices | on, off | on, off |
| Default | on | on |
| What it writes | the store key `profit_folding.active` | the bot config key `profit_folding_active` |

The two agree on meaning, on both choices and on the default.

### Every reader of the stored group

Nothing outside the dialog. The bare name `profit_folding` returns 52 hits across the 1,081 files the repository tracks, 41 of them Python. Eight name the application setting, and all eight are the dialog or the declaration behind it.

| Site | What it is |
|---|---|
| `src/core/settings.py:79` | the field defaults for the group |
| `src/core/settings.py:159` | the `AppSettings` field that declares the group |
| `src/core/settings.py:187` | the manager docstring naming the group |
| `src/gui/settings_dialog.py:1206` | the row the save and the load path both walk |
| `src/gui/main_tabs/settings_dialog_surface.py:417` | the group in the save list |
| `src/gui/main_tabs/settings_dialog_surface.py:449` | the group key |
| `src/gui/main_tabs/settings_dialog_surface.py:2234` | the same six rows on the Qt-free surface |
| `src/gui/main_tabs/settings_dialog_surface.py:2576` | the group read off the page |

The other 33 name a bot's own field `profit_folding_active`, which is a different name and a different thing. That field is read seven times inside the trading engine and gates the fold at `src/trading/scrumming_bot.py:781` and `:927`.

The nested getter and setter on `SettingsManager` still have no callers. The bare names `get_nested` and `set_nested` return two hits each, and both are the declaration and the class docstring in `src/core/settings.py`.

### The round trip of the whole stored group

All six keys in the group survive a Save that changed nothing, in both stored states. The ten top-level rows and the two sibling groups came back unchanged across the same Save.

| Key | Stored | The control showed | The store after Save |
|---|---|---|---|
| `active` | `false` | `false` | `false` |
| `mode` | `logarithmic` | `logarithmic` | `logarithmic` |
| `fold_target` | `most_recent_buy` | `most_recent_buy` | `most_recent_buy` |

| Key | Stored | The control showed | The store after Save |
|---|---|---|---|
| `fold_target_count` | `9` | `9` | `9` |
| `distribute_target` | `x_sell` | `x_sell` | `x_sell` |
| `distribute_target_count` | `7` | `7` | `7` |

The same read with the group stored at its defaults reported all six kept as well, so the reading tells the two states apart.

### Why no route reaches the wizard's folding page

The wizard registers six pages and routes to five. `nextId` was read from each page in turn, with that page set as the start page.

Diagram: flowchart, left to right

- Trading Mode -> **Select Asset Pair**
- Trading Mode -> **Extractor Pool**
- Select Asset Pair -> **Trading Parameters**
- Extractor Pool -> **Trading Parameters**
- Trading Parameters -> **Phantom Bots**
- Phantom Bots -> **finish**
- Trading Parameters -> **finish**
- Profit Folding and Upward Distribution -> **finish**

No page answers the folding page. The only branch that would is guarded behind `is_grid()` at `src/gui/bot_wizard.py:1851`, and `is_grid()` returns `False` always. The Qt-free wizard surface states the same in a named constant, `UNREACHABLE_PAGES` at `src/gui/main_tabs/bot_wizard_surface.py:110`, and its view model reports the folding page unreachable.

A second cut sits behind the first. `BotCreationWizard.get_bot_config` never calls the folding page's `get_config`. A Scrumming bot collects 52 keys and `profit_folding_active` is not among them, against the six keys that page's own `get_config` returns.

### What a new bot opens at whatever is stored

| Driven | Reading |
|---|---|
| the group stored with `active` off, the wizard's box | on, the build figure |
| `profit_folding_active` among the keys the wizard collects | no |
| `profit_folding_active` on the new bot's config | on |
| the declared default on the bot config field | on |
| `profit_folding_active` carried when anything collects it | yes |

The carriage from the wizard to the bot is already built, so the field arrives the moment the folding page is reached and collected. An existing bot is untouched either way: it takes the flag from its own saved config at `src/trading/container/restore.py:222`, which reads no application setting.

### The sentence this section leaves standing

`Profit Folding / Upward Distribution Active - Turns the whole page on.`

The box does not turn the page on. It carries no toggle connection and no enable rule on either build, so the three groups below it stay live with the box clear. The sentence stands as the specification and the page does not yet meet it.

### What waits on what

| The blocker | What it needs |
|---|---|
| the wizard's folding page is unreachable | a route to the page and a call to its `get_config` |
| the box does not turn the page on | an enable rule over the three groups below it, which are the other five rows |
| the Electron shell's wizard request carries no stored defaults | the request carrying the defaults bag the receiving surface reads |

Until the first of the three lands, a stored choice on this row has nowhere to arrive, and the row is left as it stands.

## 2026-09-12 - Distribution Mode is off the Profit Folding page

The Profit Folding page carried one master checkbox and three groups of radio buttons. The Distribution Mode group is removed. It offered two choices, `equal` and `logarithmic`, stored the chosen word under the key `mode` inside the `profit_folding` group, and read it back. Nothing outside the dialog read the word.

Three files declared the row and every declaration is gone.

| File | What it declared |
|---|---|
| `src/core/settings.py` | `FoldDistributeMode`, and the `mode` field on `ProfitFoldingSettings` |
| `src/gui/settings_dialog.py` | `FOLD_MODE_BUTTONS`, the `QGroupBox` with its two `QRadioButtons`, and the `_stored_groups` row |
| `src/gui/main_tabs/settings_dialog_surface.py` | `MODE_GROUP_TITLE`, `EQUAL_MODE`, `LOG_MODE`, `FOLD_MODE_BUTTONS`, the `fold_equal` and `fold_log` specs, the `GROUPS` entry, the Profit Folding layout step, and the `_folding_rows` entry |

### The two spellings this row carried

The Settings dialog stored the word under `mode`. The bot wizard's own Distribution Mode group writes the same word under `fold_mode`. The two names addressed the same choice, and the dialog said so itself: the tuple holding this row's two buttons was named `FOLD_MODE_BUTTONS` while the row it fed wrote `mode`.

| What matched | The Settings row | The wizard's group |
|---|---|---|
| the group title | Distribution Mode | Distribution Mode |
| the two controls | `fold_equal`, `fold_log` | `fold_equal`, `fold_log` |
| the two stored words | `equal`, `logarithmic` | `equal`, `logarithmic` |
| the default | Equal checked at build | Equal checked at build |

| What matched | The Settings row | The wizard's group |
|---|---|---|
| the key it wrote | `profit_folding.mode` | the bot config key `fold_mode` |

`fold_mode` stays in `_DEPRECATED_KWARGS` at `src/trading/container/config.py:386`, so a `bot_state.json` still carrying it restores without a `TypeError`. `_sanitize_deprecated_kwargs` drops it before `BotConfig.__init__`, and `BotConfig` declares no field of that name. The wizard's group is the only control left that writes it, and it sits on the wizard page no route reaches.

Because `mode` never matched a strip-list name, the count at Settings > Trading stands as written: the Profit Folding page contributes four of the eleven names, and this row was never one of them.

### Why no fold takes a distribution mode

The row offered a choice between two ways of spreading a fold across positions. No code in the engine offers that choice. Every fold path was read for a mode parameter and none takes one.

| Where | What it does |
|---|---|
| `src/trading/scrumming_bot.py:767` `_apply_fold_target_growth` | drains fold surplus into one target balance; no positions, no spread |
| `src/trading/smart_wire.py:589` `distribute_fold_profit` | splits realised fold profit across wires to other bots by per-wire percent |
| `src/trading/scrumming/wire_routing.py:487` `_spread_wire_usd_over_fold_queue` | adds wire income evenly to every standing fold tranche; the even split is fixed and has no alternative |
| `src/trading/stack_math.py:304` `fold_ladder_prices` | places fold prices from `stack_spacing_mode` and decides no size |
| `src/trading/profit_fold.py:14` `apply_profit_fold` | grows one target balance; its own docstring records that no module imports it |

The one mode-like selector near folding refuses this row's value. `SPACING_MODES` in `src/trading/stack_math.py:25` holds `quadratic`, `fibonacci`, `linear` and `exponential`. `level_multipliers("logarithmic", 3)` raises `ValueError`, while `level_multipliers("quadratic", 3)` returns `[1.0, 4.0, 9.0]`.

The two choices belong to the retired Grid Bot. The earliest tracked copy of the strip list names the block in its own words, as grid-legacy fields kept so a stored bot does not crash on load, and `src/trading/container/restore.py:183-184` records that the grid mode was removed.

### A six-key group opening with five rows

`SettingsManager._apply_dict` copies the whole `profit_folding` dict off the file, so a store written before the removal still carries its `mode` entry and still opens. The page reads back the five keys it has rows for, and the next Save writes the group without the sixth.

| | before the removal | after the removal |
|---|---|---|
| control specs in the payload | 69 | 67 |
| control specs on the Profit Folding tab | 11 | 9 |
| groups declared on the surface | 13 | 12 |
| a Distribution Mode group is drawn | yes | no |
| rows the `profit_folding` group declares | 6 | 5 |

| | before the removal | after the removal |
|---|---|---|
| keys a legacy store's group holds on opening | 6 | 6 |
| keys the group holds after one Save | 6 | 5 |
| keys and groups one Save wrote | 13 | 13 |
| `active` read back off a legacy store | `False` | `False` |
| `fold_target` | `most_recent_buy` | `most_recent_buy` |
| `fold_target_count` | `9` | `9` |
| `distribute_target` | `x_sell` | `x_sell` |
| `distribute_target_count` | `7` | `7` |
| `username` beside the group | `u16_driver` | `u16_driver` |
| `bot_visibility` beside the group | `internal` | `internal` |

Every sibling in the shared group reads back at its stored value, and Save raised no warning box.

### The sentences the page keeps that no longer describe it

Each sentence below stands in an earlier section and no longer describes the page. The earlier text stays where it is.

| Earlier sentence | What the page does now | | | | | |
|---|---|---|---|---|---|---|
| "One master checkbox and three groups of radio buttons, eleven controls in all." | One master checkbox and two groups of radio buttons, nine controls in all. | | | | | |
| "Distribution Mode - Chooses whether a fold spreads evenly across the chosen positions or on a curve." and the key line, the two sentences and the code block under it | The page carries no Distribution Mode group, and no fold in the engine takes a distribution mode. | | | | | |
| "\ | Profit Folding \ | 11 \ | the whole group \ | `active` alone \ | " in What each page persists | Profit Folding, 9 controls, `_save` writes the whole group, `_load_current` restores the whole group. |
| "\ | Profit Folding \ | 11 \ | the whole group \ | the whole group \ | " in What each page restores now | The same nine controls and five keys. |

| Earlier sentence | What the page does now | | | | | |
|---|---|---|---|---|---|---|
| "\ | Profit Folding \ | Distribution Mode, Profit Folding Target, Fold to X# of buy positions, Upward Distribution Target, Distribute to X# of sell positions \ | " in The fourteen rows a Save overwrote | Thirteen rows, four of them on this page. Distribution Mode is off the page. | | |

The master switch above this row gated five rows. It gates four now, and the four are Profit Folding Target, Fold to X# of buy positions, Upward Distribution Target and Distribute to X# of sell positions.

## 2026-09-12 - Profit Folding Target waits on its count row

The third row on the Profit Folding page offers three choices: fold to all buy positions, to a counted number of them, or to the most recent. It stores the chosen word under the key `fold_target` inside the `profit_folding` group and reads it back. Nothing outside the dialog reads the word, and no fold in the engine reaches a chosen set of buy positions. The row is therefore dead, and it stays on the page until the count row beside it comes off with it.

### What the three buttons choose, and what reads the word

| Build | The controls |
|---|---|
| React, the build that runs | `fold_all`, `fold_x` and `fold_recent`, three radio specs under `FOLD_GROUP_TITLE` in `src/gui/main_tabs/settings_dialog_surface.py` |
| Qt, preserved beside it | three `QRadioButton` inside `QGroupBox("Profit Folding Target")` in `src/gui/settings_dialog.py` |

Both write `profit_folding.fold_target` through one `_stored_groups` row, and `FOLD_TARGET_BUTTONS` fixes the read order that decides the stored word.

The bare name `fold_target` was searched over the 1077 tracked files outside `dist/` and `docs-archive/`. It returns 16 hits: three declare the row, five belong to the bot wizard's own group, and eight are the strip list and this manual quoting it. No site reads the word.

| Read shape | Sites |
|---|---|
| `getattr(x, "fold_target", ...)` | 0 |
| `x["fold_target"]` | 0 |
| `sm.get_nested("profit_folding", "fold_target")` | 0 |
| `getattr(bot.config, "fold_target", ...)` | 0 |
| `cfg.fold_target` | 0 |

The same five shapes, spelled for the sibling key `profit_folding_active`, return 16 sites in the same files, so the zeros read the repository rather than a blind search. `FoldTarget` in `src/core/settings.py` has two sites, both its own declaration.

`fold_target` stays in `_DEPRECATED_KWARGS` at `src/trading/container/config.py:387`, so a `bot_state.json` still carrying it restores without a `TypeError`. `BotConfig` declares no field of that name, and `_sanitize_deprecated_kwargs` drops the key before `BotConfig.__init__`. This is where the row differs from Profit Folding Active: `profit_folding_active` is a real `BotConfig` field, so that row has a destination behind one cut route. `fold_target` has no field to land in, so repairing the wizard's folding page would not give it one.

### Why no fold reaches a chosen set of buy positions

Every fold path was asked for its runtime signature. None takes a target of positions or a count of them.

| Where | What it does |
|---|---|
| `src/trading/profit_fold.py:14` `apply_profit_fold` | grows one dollar target balance; its `target` parameter is annotated `float`, and its own docstring records that no module imports it |
| `src/trading/scrumming_bot.py:767` `_apply_fold_target_growth` | drains fold surplus into one target balance; no positions, no set |
| `src/trading/smart_wire.py:589` `distribute_fold_profit` | splits realised fold profit across wires to other bots by per-wire percent |
| `src/trading/scrumming/wire_routing.py:487` `_spread_wire_usd_over_fold_queue` | adds wire income evenly to every standing fold tranche; the share is fixed at the dollars divided by the tranche count |
| `src/trading/scrumming/fold_tranches.py:51` `_fold_eligible_tranches` | picks the tranches a fold may reach by price alone |
| `src/trading/scrumming/fold_tranches.py:515` `_fold_discharge_order` | orders those tranches by `ref`, highest first |
| `src/trading/stack_math.py:304` `fold_ladder_prices` | places fold prices and decides no size; its `levels` counts prices on a ladder |

The two paths that do decide which tranches a fold reaches were driven over three tranches at refs 120, 110 and 100. `_fold_discharge_order` returned them in price order and not in `created_ts` order, so "most recent" is not even the order the queue discharges in. `_fold_eligible_tranches` at 105 returned the two tranches above that price and at 50 returned all three, so the set is price and no count narrows it.

The even spread handed each of three tranches the same share, and the share is the dollars divided by the tranche count. It is fed by wire income rather than by a fold, and no second way of spreading exists.

The one selector near folding refuses this row's words. `SPACING_MODES` in `src/trading/stack_math.py` holds `quadratic`, `fibonacci`, `linear` and `exponential`. `level_multipliers("x_buy", 3)` raises `ValueError`, while `level_multipliers("linear", 3)` returns `[1.0, 2.0, 3.0]`.

### The count that has no other referent

The count row beside this one, Fold to X# of buy positions, is not a setting in its own right. It is the number one of these three choices uses, and four readings tie it to this row.

| What ties them | Measured |
|---|---|
| the group | the spin `fold_x_count` carries the group title `Profit Folding Target`, which is this row's title |

| What ties them | Measured |
|---|---|
| the layout | the Qt-free surface puts `fold_x` and `fold_x_count` in one row tuple, and the Qt tab puts them in one `QHBoxLayout` |
| the words | the spin's spec carries `label` `None` and no `text`. Every word beside it on screen belongs to the radio `fold_x` |
| the sentence | "Fold to X# of buy positions - Sets that counted number" points at this row's sentence for what "that" names |

Driven with this row's three radios taken out, the group titled `Profit Folding Target` holds one control: a spin box named `fold_x_count`, with no label and no text. Taking this row off alone would leave a numeric box with no words, under a title describing a choice the page no longer offers.

The two rows are one decision, so one change takes both off together.

### A five-key group opening with the row gone

`SettingsManager._apply_dict` copies the whole `profit_folding` dict off the file, so a store written before the row comes off still carries its `fold_target` entry and still opens. The figures below come from driving `SettingsDialogReact` on one store holding all five keys, with the row present and then with the row taken out.

| | the row present | the row gone |
|---|---|---|
| the dialog built | yes | yes |
| control specs in the payload | 67 | 64 |
| control specs on the Profit Folding tab | 9 | 6 |
| control specs in the Profit Folding Target group | 4 | 1 |
| a legacy store's five keys opened | yes | yes |
| keys the group holds after one Save | 5 | 4 |
| keys and groups one Save wrote | 13 | 13 |
| warning boxes raised | 0 | 0 |
| `active` read back off a legacy store | `False` | `False` |
| `fold_target_count` | 9 | 9 |
| `distribute_target` | `x_sell` | `x_sell` |
| `distribute_target_count` | 7 | 7 |

Every one of the four siblings in the shared group reads back at its stored value, both before and after the Save, and the four read back again when the store is reopened.

With the row present, all three stored words round-trip. Each opens the page on its own button, each is read back as itself, and each survives a Save that changed nothing.

| Stored `fold_target` | The radio ticked | The store after a Save with no edit |
|---|---|---|
| `all_buy` | `fold_all` | `all_buy` |
| `x_buy` | `fold_x` | `x_buy` |
| `most_recent_buy` | `fold_recent` | `most_recent_buy` |

### The claim the page makes that the page does not keep

The sentence "Three radio buttons and the count box the middle one gates" stands in an earlier section. No gate exists. Driven on `SettingsDialogReact`, the count box stays able to take an edit with each of the three radios ticked in turn, and the same reading returns a refusal when the box is deliberately disabled. The Qt folding tab makes no `toggled` connection and calls no `setEnabled`, and the Qt-free surface holds no enable rule for either name.

### The two sentences the engine no longer keeps

The sentence below stands in an earlier section and no longer describes the engine. The earlier text stays where it is.

| Earlier sentence | What the engine does now |
|---|---|
| "Profit Folding Target - Chooses which buy positions a fold reaches: all of them, a counted number of them, or the most recent." | No fold chooses a set of buy positions. A fold reaches the tranches whose reference price is above the market and discharges them in price order. |
| "Three radio buttons and the count box the middle one gates." | The count box is live with any of the three radios ticked. |

## 2026-09-12 - Fold to X# of buy positions takes the whole target group off the page

The Profit Folding page carried one master checkbox and two groups of radio buttons. The `Profit Folding Target` group is removed, all four controls together: the three radio buttons and the count box inside them. The group offered a choice of which buy positions a fold reaches and a number for the middle choice. No fold in the engine offers that choice, and no fold counts the positions it reaches.

Three files declared the group and every declaration is gone.

| File | What it declared |
|---|---|
| `src/core/settings.py` | `FoldTarget`, and the `fold_target` and `fold_target_count` fields on `ProfitFoldingSettings` |
| `src/gui/settings_dialog.py` | `FOLD_TARGET_BUTTONS`, the `QGroupBox` with its three `QRadioButtons` and its `QSpinBox`, and the two `_stored_groups` rows |
| `src/gui/main_tabs/settings_dialog_surface.py` | `FOLD_GROUP_TITLE`, `FOLD_ALL`, `FOLD_X`, `FOLD_RECENT`, `FOLD_TARGET_BUTTONS`, the four control specs, the `GROUPS` entry, the Profit Folding layout step, and the two `_folding_rows` entries |

`FOLD_COUNT_DEFAULT` stays, because the sell-side count row reads it.

### The four controls the group held

| The control | What it stored |
|---|---|
| the radio `fold_all`, "Fold to ALL buy positions", checked at build | `profit_folding.fold_target` as `all_buy` |
| the radio `fold_x`, "Fold to X# of buy positions:" | the same key as `x_buy` |
| the radio `fold_recent`, "Fold to most recent buy positions" | the same key as `most_recent_buy` |
| the spin `fold_x_count`, 1 to 100, at 5, with no label and no text | `profit_folding.fold_target_count` |

The spin carried the group title of the three radios and shared one layout row with `fold_x`. Every word beside it on screen belonged to that radio, so the count had no words of its own and no meaning apart from the choice it sized.

### What the number was meant to count

The bare name `fold_target_count` appears at 11 sites across the 1077 tracked files outside `dist/` and `docs-archive/`: three declared this row, two belong to the bot wizard's own count, and six are the strip list and this manual quoting it. Six read shapes were spelled for the name and each returned nothing.

| Read shape | Sites |
|---|---|
| `getattr(x, "fold_target_count", ...)` | 0 |
| `getattr(bot.config, "fold_target_count", ...)` | 0 |
| `x["fold_target_count"]` | 0 |
| `get_nested(..., "fold_target_count")` | 0 |
| `get(..., "fold_target_count")` | 0 |
| `cfg.fold_target_count` | 0 |

The same six shapes, spelled for the sibling key `profit_folding_active`, return 17 sites over the same files, so the zeros read the repository rather than a blind search.

`fold_target` and `fold_target_count` both stay in `_DEPRECATED_KWARGS` at `src/trading/container/config.py:387-388`, so a `bot_state.json` still carrying either restores without a `TypeError`. `BotConfig` declares neither name as a field, and `_sanitize_deprecated_kwargs` drops both before `BotConfig.__init__`.

### One dollar cap bounds a fold, and no count does

`src/trading/scrumming/tick_phases.py:1494-1534` is the live fold-back route. It asks `_fold_eligible_tranches` which queued tranches the market price reaches, sorts them by `initial_buy_price`, then hands the sorted list and one dollar figure to `_plan_fold_consumption` at `src/trading/scrumming/fold_tranches.py:123`. That dollar figure is `cycle_growth_cap_usd` less what the cycle has already consumed.

Driven over five queued tranches at refs 150 down to 110, on a bot config carrying `fold_target_count = 2`:

| What was asked | Tranches the fold reached |
|---|---|
| a cap of $1000 | 5 of 5 |
| a cap of $25 | 3 of 5, one of them part-consumed |
| a cap of $0 | 0 of 5 |

The stored count of 2 narrowed nothing. The two tighter caps show the reading can report a narrowed plan, so the unnarrowed plan at $1000 is a reading.

`_fold_eligible_tranches` narrows by price alone: at a market price of 100 it returned all five, and at 135 it returned the two tranches above that price. `_fold_discharge_order` returned the five in `ref` order, highest first. The same five rows in `created_ts` order, newest first, are the reverse of that, so "most recent" is not the order the queue discharges in.

`_spread_wire_usd_over_fold_queue` handed each of the five the same $10 share out of $50. The share is the dollars divided by the tranche count, it is fed by wire income rather than by a fold, and it reaches every standing tranche.

`fold_ladder_prices` at `src/trading/stack_math.py:304` takes `levels`, which counts prices on a ladder. `fold_ladder_prices(100.0, 4)` returned four prices and decided no size and no position.

### Nine calls that refused a counted fold

Each live fold path was asked for a count or a target by keyword.

| The call | What it did |
|---|---|
| `_plan_fold_consumption(eligible, cap, count=2)` | `TypeError`, unexpected keyword argument |
| `_plan_fold_consumption(eligible, cap, fold_target_count=2)` | `TypeError`, unexpected keyword argument |
| `_fold_eligible_tranches(100.0, 1.0, count=2)` | `TypeError`, unexpected keyword argument |
| `_fold_eligible_tranches(100.0, 1.0, fold_target='x_buy')` | `TypeError`, unexpected keyword argument |
| `_fold_discharge_order(count=2)` | `TypeError`, unexpected keyword argument |
| `_fold_discharge_order(most_recent_first=True)` | `TypeError`, unexpected keyword argument |
| `_spread_wire_usd_over_fold_queue(30.0, [], count=2)` | `TypeError`, unexpected keyword argument |
| `apply_profit_fold(..., fold_target_count=2)` | `TypeError`, unexpected keyword argument |
| `apply_profit_fold(..., target='x_buy')` | `TypeError`, a word compared against a number |

The `target` parameter of `apply_profit_fold` is a dollar balance, not one of this group's three words, and `src/trading/profit_fold.py` records in its own module docstring that no module imports it.

### A five-key store opening on three rows

`SettingsManager._apply_dict` copies the whole `profit_folding` dict off the file, so a store written before the removal still carries its `fold_target` and `fold_target_count` entries and still opens. The page reads back the three keys it has rows for, and the next Save writes the group without the other two.

| | before the removal | after the removal |
|---|---|---|
| the dialog built | yes | yes |
| control specs in the payload | 67 | 63 |
| control specs on the Profit Folding tab | 9 | 5 |
| control specs in the `Profit Folding Target` group | 4 | 0 |
| a `Profit Folding Target` group is drawn | yes | no |
| an `Upward Distribution Target` group is drawn | yes | yes |
| groups declared on the surface | 12 | 11 |
| rows the `profit_folding` group declares | 5 | 3 |
| `ProfitFoldingSettings` fields | 5 | 3 |
| keys a legacy store's group holds on opening | 5 | 5 |
| keys the group holds after one Save | 5 | 3 |

| | before the removal | after the removal |
|---|---|---|
| warning boxes raised | 0 | 0 |
| `active` read back off a legacy store | `False` | `False` |
| `distribute_target` | `x_sell` | `x_sell` |
| `distribute_target_count` | `7` | `7` |
| `username` beside the group | `u18_driver` | `u18_driver` |
| `bot_visibility` beside the group | `internal` | `internal` |

Every one of the three remaining siblings reads back at its stored value and survives a Save that changed nothing. No holder named `_fold_all`, `_fold_x`, `_fold_x_count` or `_fold_recent` exists on the dialog any more, while `_dist_x_count` still does.

### The sentences both rows leave behind

Each sentence below stands in an earlier section and no longer describes the page. The earlier text stays where it is.

| Earlier sentence | What the page does now | | | | | |
|---|---|---|---|---|---|---|
| "One master checkbox and three groups of radio buttons, eleven controls in all." | One master checkbox and one group of radio buttons, five controls in all. | | | | | |
| "Profit Folding Target - Chooses which buy positions a fold reaches", the two sentences and the code block under it | The page carries no `Profit Folding Target` group. | | | | | |
| "Three radio buttons and the count box the middle one gates." | Neither the radios nor the count box exists, so nothing is left to gate. | | | | | |
| "Fold to X# of buy positions - Sets that counted number. Key `fold_target_count`, 1 to 100, at 5." | The page sets no count, and one dollar cap bounds how many tranches a fold reaches. | | | | | |
| "Save collapses the two target groups into one dictionary." | Save collapses one target group, the sell side. | | | | | |
| "Four of the six keys then fail a second time further down: the two target names and their two counts are four of the eleven the bot factory strips." | Two of the three keys the page still stores are on the strip list: `distribute_target` and `distribute_target_count`. | | | | | |

| Earlier sentence | What the page does now | | | | | |
|---|---|---|---|---|---|---|
| "\ | Profit Folding \ | 11 \ | the whole group \ | the whole group \ | " in What each page restores now | Five controls, and three keys in the group. |
| "\ | Profit Folding \ | Distribution Mode, Profit Folding Target, Fold to X# of buy positions, Upward Distribution Target, Distribute to X# of sell positions \ | " | Two rows on this page, Upward Distribution Target and Distribute to X# of sell positions. | | |

The master switch above the removed group gated four rows. It gates two now, and the two are Upward Distribution Target and Distribute to X# of sell positions.

## 2026-09-12 - Upward Distribution Target takes the sell-side group off the page

The Profit Folding page carried one master checkbox and one group of radio buttons. The `Upward Distribution Target` group is removed, all four controls together: the three radio buttons and the count box inside them. The group offered a choice of which sell positions a distribution reaches and a number for the middle choice. No distribution in the engine offers that choice, and no distribution counts the positions it reaches.

Three files declared the group and every declaration is gone.

| File | What it declared |
|---|---|
| `src/core/settings.py` | `DistributeTarget`, and the `distribute_target` and `distribute_target_count` fields on `ProfitFoldingSettings` |
| `src/gui/settings_dialog.py` | `DIST_TARGET_BUTTONS`, the `QGroupBox` with its three `QRadioButtons` and its `QSpinBox`, the `QRadioButton` import, the two `_stored_groups` rows, and `_picked` with `_show_picked` |
| `src/gui/main_tabs/settings_dialog_surface.py` | `DIST_GROUP_TITLE`, `DIST_ALL`, `DIST_X`, `DIST_RECENT`, `DIST_TARGET_BUTTONS`, `FOLD_COUNT_DEFAULT`, the four control specs, the `GROUPS` entry, the Profit Folding layout step, the two `_folding_rows` entries, and `_picked` with `_show_picked` |

`_picked` and `_show_picked` held one caller each in each file, the `distribute_target` row, so both pairs go with it.

### The group's four controls on the sell side

| The control | What it stored |
|---|---|
| the radio `dist_all`, "Distribute to ALL sell positions", checked at build | `profit_folding.distribute_target` as `all_sell` |
| the radio `dist_x`, "Distribute to X# of sell positions:" | the same key as `x_sell` |
| the radio `dist_recent`, "Distribute to most recent sell positions" | the same key as `most_recent_sell` |
| the spin `dist_x_count`, 1 to 100, at 5, with no label and no text | `profit_folding.distribute_target_count` |

The spin carried the group title of the three radios and shared one layout row with `dist_x`. Every word beside it on screen belonged to that radio, so the count had no words of its own and no meaning apart from the choice it sized. The two rows are one decision, so one change takes both off together.

### What the three sell-side words chose

The bare name `distribute_target` appeared at 16 sites across the 1077 tracked files outside `dist/` and `docs-archive/`, and `distribute_target_count` at 13. Three of each declared these rows, five of the first and three of the second belong to the bot wizard's own copy, one of each is the strip list, and the rest are this manual quoting the keys. Six read shapes were spelled for each name and every one returned nothing.

| Read shape | Sites |
|---|---|
| `getattr(x, "distribute_target", ...)` | 0 |
| `getattr(bot.config, "distribute_target", ...)` | 0 |
| `x["distribute_target"]` | 0 |
| `get_nested(..., "distribute_target")` | 0 |
| `get(..., "distribute_target")` | 0 |
| `cfg.distribute_target` | 0 |

The same six shapes, spelled for the sibling key `profit_folding_active`, return 17 sites over the same files, so the zeros read the repository rather than a blind search. A planted file spelling all six shapes for `distribute_target` grew every one of the six counts.

`distribute_target` and `distribute_target_count` both stay in `_DEPRECATED_KWARGS` at `src/trading/container/config.py:390-391`, so a `bot_state.json` still carrying either restores without a `TypeError`. `BotConfig` declares neither name as a field, and `_sanitize_deprecated_kwargs` drops both before `BotConfig.__init__`. A `make_bot_config` call asking for all four of `distribute_target`, `distribute_target_count`, `profit_folding_active` and `scrum_fold_pct` kept the last two and built a config carrying neither removed name.

### One accumulator bounds a distribution, and no count does

`src/trading/scrumming/tick_phases.py:1919` `_tick_distribute` is the live route, and `src/trading/scrumming_bot.py:4222` calls it. It sells `_dist_accumulator` units of the asset, takes the lesser of that figure and the venue balance, then walks `_main_lots` sorted by `initial_buy_price` and opens one fold tranche per lot it consumes.

Driven over five held lots, on a config carrying `distribute_target = 'x_sell'` and `distribute_target_count = 2`:

| What was asked | Lots the distribution reached |
|---|---|
| an accumulator of 5.0 units | 5 of 5 |
| an accumulator of 2.5 units | 2 of 5 |
| an accumulator of 1.0 units | 1 of 5 |
| an accumulator of 0.0 units | 0 of 5, and no sell placed |

The stored count of 2 narrowed nothing. The three smaller accumulators show the reading can report a narrowed walk, so the unnarrowed walk at 5.0 units is a reading. The venue balance is the only other bound: at a balance of 2.0 units against an accumulator of 5.0 the sell was 2.0 units and the walk reached 2 of 5.

The five lots were built with buy-price order and age order deliberately opposite. The walk consumed `['p150_oldest', 'p140']`, which is buy price highest first, and the newest-first order is the exact reverse of it. "Most recent" is not the order a distribution takes.

One DIST sell leaves one tranche behind. `_bound_new_fold_tranches` merges the slice the build loop appended back to a single record, so the number of positions a distribution opens is 1 at an accumulator of 5.0 units and 1 at 3.0.

### Nine sell-side calls that refused a count

Each sell-side path was asked for a count or a target by keyword.

| The call | What it did |
|---|---|
| `_tick_distribute(..., count=2)` | `TypeError`, unexpected keyword argument |
| `_tick_distribute(..., distribute_target='x_sell')` | `TypeError`, unexpected keyword argument |
| `_tick_distribute(..., distribute_target_count=2)` | `TypeError`, unexpected keyword argument |
| `_execute_sell(1.0, 200.0, None, count=2)` | `TypeError`, unexpected keyword argument |
| `_settled_sale_proceeds(1.0, 200.0, label='DIST', count=2)` | `TypeError`, unexpected keyword argument |
| `_bound_new_fold_tranches(0, count=2)` | `TypeError`, unexpected keyword argument |
| `_apply_scrum_fold_pct(0, 100.0, 1.0, distribute_target='x_sell')` | `TypeError`, unexpected keyword argument |
| `scrum_ladder_prices(100.0, 'x_sell')` | `ValueError`, levels must be an int |
| `split_scrum_into_tranches(..., spacing_mode='most_recent_sell')` | `ValueError`, unknown spacing_mode |

`scrum_ladder_prices(100.0, 3)` returned `[101.0, 104.0, 109.0]`, three prices on an upward ladder, and decided no position and no set. `SPACING_MODES` in `src/trading/stack_math.py` holds `quadratic`, `fibonacci`, `linear` and `exponential`, and none of this group's three words.

### The count default that kept the constant alive

`FOLD_COUNT_DEFAULT` at `src/gui/main_tabs/settings_dialog_surface.py` had exactly one reader, the `distribute_target_count` row. That row is gone, so the constant goes with it. The bare name fell from 3 sites to 1, and the one left is this manual.

| Name | Sites before | Sites after |
|---|---|---|
| `distribute_target` | 16 | 13 |
| `distribute_target_count` | 13 | 10 |
| `DistributeTarget` | 2 | 0 |
| `DIST_TARGET_BUTTONS` | 6 | 0 |
| `DIST_GROUP_TITLE` | 7 | 0 |
| `FOLD_COUNT_DEFAULT` | 3 | 1 |
| `dist_x_count` | 8 | 4 |

Every site still standing is either this manual or the bot wizard's own copy of the same three words, which carries its own controls and its own count of 1 to 50.

### A six-key store opening on one row

`SettingsManager._apply_dict` copies the whole `profit_folding` dict off the file, so a store written before the removal still carries all six keys the group ever held and still opens. The page reads back the one key it has a row for, and the next Save writes the group without the other five.

| | before the removal | after the removal |
|---|---|---|
| the dialog built | yes | yes |
| control specs in the payload | 63 | 59 |
| control specs on the Profit Folding tab | 5 | 1 |
| control specs in the `Upward Distribution Target` group | 4 | 0 |
| an `Upward Distribution Target` group is drawn | yes | no |
| groups declared on the surface | 11 | 10 |
| rows the `profit_folding` group declares | 3 | 1 |
| `ProfitFoldingSettings` fields | 3 | 1 |
| keys a legacy store's group holds on opening | 6 | 6 |
| keys the group holds after one Save | 3 | 1 |
| warning boxes raised | 0 | 0 |
| `active` read back off a legacy store | `False` | `False` |
| `username` beside the group | `u19_driver` | `u19_driver` |
| `bot_visibility` beside the group | `internal` | `internal` |
| the Qt dialog built on a legacy store | yes | yes |
| a `_dist_x_count` holder on the Qt dialog | yes | no |

`active` is the one remaining sibling and it reads back at its stored value on every run, before and after, and survives a Save that changed nothing. The three rows the dialog keeps beside the group, `username`,

`bot_visibility` and the stored group itself, all read back unchanged.

With the group present, all three stored words round-tripped. Each opened the page on its own button, each was read back as itself, and each survived a Save that changed nothing.

| Stored `distribute_target` | The radio ticked | The store after a Save with no edit |
|---|---|---|
| `all_sell` | `dist_all` | `all_sell` |
| `x_sell` | `dist_x` | `x_sell` |
| `most_recent_sell` | `dist_recent` | `most_recent_sell` |

### What the Profit Folding page holds now

One checkbox, `Profit Folding / Upward Distribution Active`, and nothing else. It is the page's only control and the only `profit_folding` key the dialog stores. The Settings dialog draws no radio button on any tab any more: the three removed here were the last three `RADIO` control specs on the surface.

### The sentences the sell side leaves behind

Each sentence below stands in an earlier section and no longer describes the page. The earlier text stays where it is.

| Earlier sentence | What the page does now |
|---|---|
| "Upward Distribution Target - The same three choices on the sell side. Key `distribute_target`, at all sell positions." | The page carries no `Upward Distribution Target` group. |
| "The sell-side group is built the same way as the buy-side group above, with its own count box at 5 and its own all-sell button checked at build." | Neither group exists. |
| "Distribute to X# of sell positions - Sets the sell-side count. Key `distribute_target_count`, 1 to 100, at 5." | The page sets no count, and the accumulated asset bounds how many lots a distribution reaches. |
| "Save collapses the two target groups into one dictionary." | Save collapses no target group. It writes one key, `active`. |
| "Save collapses one target group, the sell side." | Save collapses none. |
| "Two of the three keys the page still stores are on the strip list: `distribute_target` and `distribute_target_count`." | The page stores one key and it is not on the strip list. |
| "`FOLD_COUNT_DEFAULT` stays, because the sell-side count row reads it." | The sell-side count row is gone and so is the constant. |
| "The master switch above the removed group gated four rows. It gates two now, and the two are Upward Distribution Target and Distribute to X# of sell positions." | The master switch gates no row. |
| "One master checkbox and three groups of radio buttons, eleven controls in all." | One master checkbox, one control in all. |

The master switch is the last row on the page, and its own verdict was that it waits on the wizard's folding page. Nothing on this page gates anything now.

## 2026-09-12 - The phantom master box becomes a new bot's phantom default

The Phantom Bots page's master checkbox now stores a boolean and the bot creation wizard's own phantom box opens on it. Driven on the React build, which is what the running application draws. The home was redirected into a scratch directory before the settings module was imported, so the settings directory and the log root bound under that directory. No stored setting of the running install was read and no venue was contacted.

### The Settings master box and the wizard's phantom box

| | The Settings row | The wizard box |
|---|---|---|
| Where, Qt | `src/gui/settings_dialog.py:521-525` | `src/gui/bot_wizard.py:1671-1674` |
| Where, React | `src/gui/main_tabs/settings_dialog_surface.py:625-633` | `src/gui/main_tabs/bot_wizard_surface.py:1721-1735` |
| Text | Enable Phantom Bots for Scrumming | Enable Phantom Bots |
| Choices | on, off | on, off |
| Opens at | on | the stored default, off when no bag arrives |
| What it writes | the store key `default_enable_phantoms` | the config key `enable_phantoms` |

The two agree on meaning and on both choices. They disagree on the name, and the names are kept apart on purpose.

### The two phantom names that share a word

The page's own name and a bot's own name are two settings, not one. The store accepts the first and refuses the second, driven:

```
store accepts key default_enable_phantoms
store refuses key enable_phantoms   -> KeyError 'Unknown setting: enable_phantoms'
store refuses key phantoms_enabled  -> KeyError 'Unknown setting: phantoms_enabled'
```

| Set | Occurrences in `src` | What they are |
|---|---|---|
| the application setting | 6 | the `AppSettings` field, the Qt dialog's row, the same row on the Qt-free surface, the two wizard lookups |
| a bot's own config field | 27 | the constructor argument, the restore read, the bot's own flag, the live toggle, the wizard write and the live settings box |
| a bot's own saved record | 10 | the key the container writes per bot and the restore path reads back |

Counted with `git grep -ow` over the 1081 files the index reports. Three files hold a name from both sets and each is a crossing point: the store's declaration names the bot key in its comment, and the two wizard files write the bot key from the application default.

### Every reader of the stored phantom default

Five sites name the application key. Two of them read it outside the Settings dialog.

| Site | What it does with the default |
|---|---|
| `src/core/settings.py:163` | declares `default_enable_phantoms: bool = True` |
| `src/gui/settings_dialog.py:1111-1114` | the Qt row, read on Save and written on load |
| `src/gui/main_tabs/settings_dialog_surface.py:476` | the same row on the Qt-free surface |
| `src/gui/bot_wizard.py:1672-1674` | the Qt wizard's phantom box opens here |
| `src/gui/main_tabs/bot_wizard_surface.py:1721-1735` | the Qt-free wizard's box opens here |

Before this section there was no reader at all, and no store key to read. The dialog saved 14 groups and none of them carried the box; it saves 15 now.

### Where the stored default reaches a phantom bot

Diagram: flowchart, left to right

- Settings row — default_enable_phantoms -> **SettingsManager — settings.toml**
- SettingsManager — settings.toml -> **C"]**
- main_window._create_bot — get_all( -> **bot_wizard.py:1673 — Enable Phantom Bots**
- bot_wizard.py:1673 — Enable Phantom Bots -> **PhantomConfigPage.get_config — enable_phantoms**
- PhantomConfigPage.get_config — enable_phantoms -> **F"]**
- main_window.py:3369 — ScrummingBot(enable_phantoms= -> **scrumming_bot.py:402 — _phantoms_enabled**
- scrumming_bot.py:402 — _phantoms_enabled -> **tick_phases.py:390 — create_phantom_set**
- tick_phases.py:390 — create_phantom_set -> **main_window.py:1178 — the Comp field**

The constructor argument at `src/trading/scrumming_bot.py:402` is where the flag stops being this setting. From there the bot owns it, and the Comp field on the TA Indicator Voter Panel is the last reader. That field already refuses a phantom timeframe at or below the parent's own rank, at `src/gui/main_window.py:1196`, so the higher-timeframe rule is not new here.

### The round trip on both phantom choices

| Stored | React box on reopen | React wizard box | Qt box on reopen | Qt wizard box | The config a new bot reads |
|---|---|---|---|---|---|
| off | off | off | off | off | off |
| on | on | on | on | on | on |

Two different stored flags gave two different readings on every column, so the reading discriminates. Driven before the change with the box ticked, both wizards answered off, which is the reading this section repairs.

### What a new bot opens with, and what a restored bot keeps

Driven through the real restore path with the application default set on.

| The saved record | What the restored bot holds |
|---|---|
| its own flag off | off |
| its own flag on | on |

| The saved record | What the restored bot holds |
|---|---|
| no flag at all | on, which is the constructor's own default |

A restored bot reads its own saved record at `src/trading/container/restore.py:361` and reads no application setting. A record holding off stayed off while the store held on, so the application default cannot reach a bot already on disk.

### The sentences this wiring leaves standing

| Earlier sentence | What the code does now |
|---|---|
| "This is the only control on the page that starts on. Save never reads it, so the page stores nothing under the name it shows." | It still opens on. Save reads it and stores it under `default_enable_phantoms`, so the name on screen and the name in the store differ. |
| "These three controls carry the same gap the TA Indicators page carries. Save reads none of them, the load path restores none, and the schema refuses a key it does not declare rather than storing it somewhere unread." | Save reads the master box and the load path restores it. The other two controls on the page still carry the gap. |
| "Enable Phantom Bots for Scrumming - Turns the phantom overrides on for a new bot." | Unchanged, and true for the first time: the wizard's own box opens on this row. |

### The wizard request this wiring still waits on

The Electron shell's wizard request carries no stored defaults, so the Qt-free wizard opens its box at the build figure whatever the store holds.

| Absence | What was measured |
|---|---|
| the shell's wizard request carries no stored defaults | `src/gui/web/bot_wizard.js` names `defaults` no times, and `src/gui/main_tabs/bot_wizard_surface.py:2823` reads it out of the request. The lookup added here fires on any call that carries a bag, and the shipped call carries none |
| the bridge hands the request through unchanged | `src/core/desktop_bridge.py:284` maps the wizard method straight to the surface, so nothing on the way in adds the stored settings |

This absence is shared with every other stored default the wizard opens on, so it belongs to the wizard request rather than to this row.

### Rows 34 and 35 stay for their own units

| Row | Why it did not come with this one |
|---|---|
| 34, Default Phantom Timeframes | Its eleven boxes are not a declared control on either surface, so the one list that carries Save and load cannot hold them. The rule for this group turns eleven boxes into one selection above the parent's own timeframe, which is a change of control, not a change of wiring |
| 35, Lock duration (candles) | It is a declared control and would persist through the same one line, but its destination is the wizard's own lock box, which is a second lookup this row does not make |

Measured on the page's declared controls: the Phantom tab carries two of them, the master box and the lock spin box, and the timeframe grid is neither.

## 2026-09-12 - The timeframe grid becomes one phantom selection

The Phantom Bots page's eleven tick boxes are now one drop-down, the page stores the name it shows, and the bot creation wizard opens its own phantom box on that name. A name at or below the bot's own TA Timeframe, or one the venue does not offer, is refused on the wizard page rather than dropped in silence. Driven on the Qt dialog, the React host dialog and the shared surface. The home was redirected into a scratch directory before the settings module was imported, so the settings directory and the log root bound under that directory. No stored setting of the running install was read and no venue was contacted.

### What the page offered before and what it offers now

| | Before | Now |
|---|---|---|
| Control | eleven tick boxes in one row | one drop-down |
| Choices | eleven, five ticked at build | eleven, one selected at build |
| Opens at | 5m, 15m, 1h, 4h and 1d | 1d |
| Declared as a control | no | yes, `phantom_timeframe` |
| In the list that carries Save and load | no | yes |
| Store key | none, the store refused four candidate names | `default_phantom_timeframe` |

### The selection round-tripping through the store

Driven on the shared surface and on both dialog hosts against one scratch store.

```
he picks               : 12h
save reported          : {'saved': 16, 'failed': [], 'closed': True}
store fields changed   : ['default_phantom_timeframe', 'ta_indicator_weights']
stored value           : '12h'
read back off disk     : '12h'
a fresh dialog shows   : 12h
pressing Save again keeps it: '12h'
```

The same run before the change reported one changed field, the indicator weight bag, and the store refused every candidate name for this row.

```
refuses phantom_timeframes         -> KeyError 'Unknown setting: phantom_timeframes'
refuses default_phantom_timeframes -> KeyError 'Unknown setting: default_phantom_timeframes'
refuses default_phantom_timeframe  -> KeyError 'Unknown setting: default_phantom_timeframe'
refuses phantom_timeframe          -> KeyError 'Unknown setting: phantom_timeframe'
```

### Why the daily chart is the opening choice

A phantom must sit above the bot's own TA Timeframe, and the wizard's timeframe drop-down opens on the hourly chart. Of the five entries above it, only one is offered by every venue the application knows.

| Timeframe | Above the hourly chart | Offered by coinbase | Offered by kraken |
|---|---|---|---|
| 2h | yes | yes | no |
| 4h | yes | no | yes |
| 6h | yes | yes | no |

| Timeframe | Above the hourly chart | Offered by coinbase | Offered by kraken |
|---|---|---|---|
| 12h | yes | no | no |
| 1d | yes | yes | yes |
| 1w | yes | no | yes |

Measured against the venue lists the application holds.

```
   coinbase: offers ['1m', '5m', '15m', '30m', '1h', '2h', '6h', '1d']
     kraken: offers ['1m', '5m', '15m', '30m', '1h', '4h', '1d', '1w']
    binance: offers ['1m', '5m', '15m', '30m', '1h', '2h', '4h', '6h', '12h', '1d', '1w']
```

The store declares the opening choice and the page reads it from there, so the schema and the screen cannot drift apart.

`src/core/settings.py` — the one declaration of the opening choice

```
# The one phantom timeframe a new bot starts with. 1d is the only entry
# above the wizard's 1h ta_timeframe default that every venue offers;
# coinbase has no 4h, 12h or 1w, and kraken no 2h or 6h.
default_phantom_timeframe: str = "1d"
```

### Where the stored selection reaches a phantom bot

Diagram: graph, left to right

- Settings row — default_phantom_timeframe -> **SettingsManager — settings.toml**
- SettingsManager — settings.toml -> **main_window.py:3201 — settings.get_all**
- main_window.py:3201 — settings.get_all -> **bot_wizard.py — set_parent_timeframe**
- bot_wizard.py — set_parent_timeframe -> **PhantomConfigPage.get_config — phantom_timeframes**
- PhantomConfigPage.get_config — phantom_timeframes -> **F"]**
- main_window.py:3370 — ScrummingBot(phantom_timeframes= -> **tick_phases.py:390 — create_phantom_set**
- tick_phases.py:390 — create_phantom_set -> **main_window.py:1178 — the Comp field**

The Comp field rank-weights the parent's Net with every phantom summary above the parent's own rank, and skips the rest at `src/gui/main_window.py:1198`. One selection above the parent means one phantom reaches that field and none is skipped.

### The refusal the wizard page shows

Both refusal reasons were driven on the shared surface and on the Qt page.

```
stored  1d parent  1h -> selection ['1d'] refusal ''
stored  4h parent  1h -> selection []     refusal 'phantom_not_offered'
    the box says: Timeframe 4h: REFUSED, coinbase does not offer it
stored  5m parent  1h -> selection []     refusal 'phantom_not_higher'
    the box says: Timeframe 5m: REFUSED, a phantom must sit above the bot's own TA Timeframe of 1h
stored  1d parent  1d -> selection []     refusal 'phantom_not_higher'
    the box says: Timeframe 1d: REFUSED, a phantom must sit above the bot's own TA Timeframe of 1d
```

A box the operator has already ticked keeps his choice, so the stored name seeds a page he has not touched and never overwrites a page he has.

### What a bot already on disk does

A running bot's phantom set never came from this page and still does not.

| Read | What it answers |
|---|---|
| `src/trading/bot_container.py:601` | writes the enable flag into the bot's record and writes no timeframe list |
| `src/trading/container/restore.py:363-366` | builds the bot without a timeframe argument |
| `src/trading/scrumming_bot.py:377` | the bot therefore takes its class default |

Driven on a bot built the restore way, before and after this change, against a parent on the hourly chart.

```
built the restore way      : ['5m', '15m', '30m', '1h', '1d']
class default              : ['5m', '15m', '30m', '1h', '4h', '1d']
survives the Comp skip     : ['1d']
silently skipped           : ['5m', '15m', '30m', '1h']
```

Identical on both sides of the change. Four of the five a restored bot runs sit at or below the hourly parent and never reach the Comp field, and the sixth is dropped because coinbase has no four-hour chart. That belongs to row 131, which owns a bot's own phantom selection.

### The sentences this change leaves standing

| Earlier sentence | What the code does now |
|---|---|
| "Default Phantom Timeframes - Chooses which charts a phantom watches." | One chart, not several. The name it shows is the chart the one phantom watches. |
| "Per-bot key `phantom_timeframes`, eleven boxes, five of them checked at build." | Eleven names in one drop-down, one selected at build, and the page's own store key is `default_phantom_timeframe`. |
| "One box per timeframe in a single row. The build decides which five open checked, and those five are the ones the figure shows." | One drop-down in that row. The figure shows the eleven boxes the page no longer draws. |
| "These three controls carry the same gap the TA Indicators page carries. Save reads none of them, the load path restores none." | Save reads two of the three and the load path restores both. The lock spin box still carries the gap. |
| "The one spin box on the page, inside its own group. It opens at 2 and Save reads it no more than it reads the other two." | It still opens at 2 and Save still does not read it. The other two are now read. |

### What this row still waits on

| Absence | What was measured |
|---|---|
| the wizard still draws eleven boxes | `src/gui/bot_wizard.py:1679-1699` builds eleven tick boxes, so the operator can still tick several. One selection is seeded and not enforced there; the wizard's own control belongs to the bot's phantom rows |
| the figure on this page shows the old grid | `p39-i0.png` was captured from the eleven boxes and has not been retaken |
| the rank order is written twice | `src/trading/phantom_balance.py:29-41` and `src/gui/main_tabs/bot_wizard_surface.py:521-533` hold the same eleven names in the same order, and the Qt-free surface takes no import, so nothing reconciles them. Measured identical today |

## 2026-09-12 - Every retired setting name says what it is

Eight settings came off these pages, and their names did not all leave the tree with them. A name sitting in a comment with no explanation reads as a setting, so each one that stays now says what it is.

### Where the retired names still sit

Counted over the git index, matches rather than lines, in both the code spelling and the words the page shows.

| the name | matches | where they are |
|---|---|---|
| `position_distance_pct` | 9 | this page 8, the strip list 1 |
| `increment_style` | 21 | this page 15, the bot config 2, the restore path 1, a debug report 1 |
| `default_position_count` | 9 | this page only |
| `position_count` | 19 | this page 14, the strip list 1, a creation log line 1, a census note 1, a stock helper 2 |
| `fold_mode` | 10 | this page 4, the two wizard files 5, the strip list 1 |
| `fold_target` | 31 | this page 24, the two wizard files 5, the strip list 1 |
| `fold_target_count` | 23 | this page 20, the two wizard files 2, the strip list 1 |
| `distribute_target` | 34 | this page 27, the two wizard files 5, the strip list 1 |
| `distribute_target_count` | 21 | this page 18, the two wizard files 2, the strip list 1 |
| `profit_fold_pct` | 3 | this page 1, a wizard docstring 1, the strip list 1 |
| `upward_distribution` | 1 | a wizard docstring, and nowhere else |

The settings store and both settings dialog modules hold none of them. Thirteen camelCase spellings were searched beside the snake_case ones and all read zero, so no shipped JavaScript names a retired setting.

### What emptying the strip list does

A record shaped like `bot_state.json` was built carrying thirty config keys, of which eleven are the retired names, and driven through all three build routes with the strip list as shipped and then emptied.

```
the list as shipped, 11 names
  make_bot_config(**stored)             OK  symbol='A15/USD'
  bot_config_kwargs then make_bot_config OK  kwargs=18 retired leaked=0
  restore_bots_from_state               OK  restored=1 skips=0

the list emptied
  make_bot_config(**stored)             TypeError: unexpected keyword
                                        argument 'bulk_trading'
  bot_config_kwargs then make_bot_config OK  kwargs=18 retired leaked=0
  restore_bots_from_state               OK  restored=1 skips=0
```

Dropping the eleven one at a time raises that error naming each key in turn, eleven times out of eleven, so every entry is load-bearing on that route. The launch restore names each kwarg it wants one at a time rather than passing the stored config whole, so the restore arm does not move. A retired key that did reach the factory is caught at `src/trading/container/restore.py:322`, which skips the bot and leaves its record on disk.

### What the declaration says now

The eleven entries stay. The comment above them states why.

`src/trading/container/config.py` — the strip list, with its own description

```
#: A retirement record, not a setting set. Each name is a setting removed from
#: the product, kept so a config stored before its removal still builds;
#: `_sanitize_deprecated_kwargs` drops each before `BotConfig.__init__`.
#: Add no name. Only `bulk_trading` is read, by `bot_config_kwargs`.
```

### The names taken out, and the ones left standing

| occurrence | what happened to it |
|---|---|
| the parameter page's `get_config` docstring | removed. It listed four retired keys the method does not emit, and carried two version names. `upward_distribution` had no other occurrence in the tree and now has none |
| the restore path's Grid-legacy comment | it read "nothing here reads them" above three lines that read three of them. It now states what the hand-written kwarg list does |
| the stack-mode fallback comment | it named no key. It now names `bulk_trading` |
| the eleven strip-list entries | kept, for the measured reason above |
| the two wizard files' folding groups | kept. The wizard's folding page is unreachable, and the rows that own those files are still open |
| the creation log line's `position_count` read | kept. It is the row that owns the line that always prints zero |
| the handoff files, the archive chronicle, the census note, the debug report | kept. Each is a dated record of an earlier state, not a description of a control |

## 2026-09-12 - The Logging page comes off the Settings dialog

The Logging page carried two checkboxes and four periodicity boxes. All six are removed, and the tab goes with them, because every control on the page wrote into one stored group that nothing outside the dialog ever read.

Each of the three names on the page was judged on its own evidence. Two of them name logging the platform already writes under a different name. The third names a summary no code in the tree writes at all.

Three files declared the page and every declaration is gone.

| File | What it declared |
|---|---|
| `src/core/settings.py` | `LogPeriodicity`, the whole `DataLoggingSettings` class, and the `data_logging` field on `AppSettings` |
| `src/gui/settings_dialog.py` | `PERIOD_BUTTONS`, `DEFAULT_PERIODS`, `_create_logging_tab` with its six check boxes, its `addTab` step, `_ticked_periods`, `_show_periods`, and the three `_stored_groups` rows |
| `src/gui/main_tabs/settings_dialog_surface.py` | `LOGGING_TAB`, `LOGGING_HEADING`, `LOGGING_GROUP_KEY`, `LOGGING_FLAG_DEFAULT`, `PERIOD_CONTROLS`, `DEFAULT_PERIODS`, the six control specs, the tab title, the layout page, the heading entry, `_logging_rows`, `_logging_group`, and the save-group entry |

### The six controls the page held

| The control | What it stored |
|---|---|
| the box `ta_logging`, "Log TA signal samples with all values and timestamps", checked at build | `data_logging.ta_signal_logging` |
| the box `highlight_trades`, "Highlight entries near Scrumming Bot trades", checked at build | `data_logging.highlight_trade_proximity` |
| the box `log_24h`, "24 Hours", checked at build | one entry in `data_logging.active_periodicities` |
| the box `log_1w`, "1 Week", checked at build | the same list |
| the box `log_1m`, "1 Month", clear at build | the same list |
| the box `log_1y`, "1 Year", clear at build | the same list |

The four periodicity boxes were one setting spread over four controls. Save walked them in page order and wrote the ticked ones as a list.

### Nothing outside the dialog read the group

Each of the four stored names occurs in exactly three files, and all three are the declaration plus the two dialog surfaces. The scan walked every Python and JavaScript file under the source, tooling and desktop trees, plus the entry point.

```
data_logging               3 files
ta_signal_logging          3 files
highlight_trade_proximity  3 files
active_periodicities       3 files
    src/core/settings.py
    src/gui/settings_dialog.py
    src/gui/main_tabs/settings_dialog_surface.py
```

Every writer of every log file the platform keeps sits in one module, `src/core/logging_engine.py`, and that file holds no occurrence of any of the four names. The same count over the word "timestamp" in the same file returns three, so the zeros read the file rather than a blind search.

### Each indicator reading is already written with its value and its time

The voting panel snapshot writes one record for every fired trade, carrying the whole vote that drove it. Each vote is one indicator's own reading, and it carries its own clock value beside the record's.

`src/trading/indicators/types.py` — one indicator's output

```
@dataclass
class Signal:
    """One indicator's output at a point in time."""

    indicator: str
    timeframe: str
    direction: SignalDirection
    confidence: float  # 0.0 - 1.0
    weight: float = 1.0
    details: dict = field(default_factory=dict)
    timestamp: float = field(default_factory=time.time)
```

Driven through the real writer into a scratch directory, one record landed with the reading, the weight, the raw values and two separate times:

```
row timestamp  2026-09-12T17:07:05.855494+00:00
signal in row  {"indicator":"rsi","timeframe":"1h","direction":-1,
                "confidence":0.72,"weight":1.4,
                "details":{"rsi":71.25,"period":14},
                "timestamp":1789232825.8554606,"abstained":false}
```

That is what the removed checkbox described, already written, for every fired trade, with no setting consulted. The operator's own file carries it: his `voting.log` measured 8,078,180 bytes on the day of this change.

### Every row in the two decision logs already sits at a fire

The gate log and the voting log are both written once per fired trade, never per tick. Each file therefore holds fire-adjacent rows and nothing else, and each row already names the action that fired it.

`src/core/logging_engine.py` — the voting writer's own contract

```
"""Write one entry with category "voting" to ``voting.log``.

``ScrummingBot._emit_voting_panel_snapshot_at_fire`` reaches this once
per fired trade, not per tick.
"""
```

A flag that marked the lines near a fire would mark every line in both files, so it distinguishes nothing. The record type does declare a `highlight` field, and nothing in the tree sets it and nothing reads it.

### No profit and loss figure is summarised over any window

The cascade declares four windows and creates a directory for each one at construction. Only the daily writer has a caller.

`src/core/logging_engine.py` — the four windows, and the three that nothing fills

```
PERIODS = {
    "daily": 1,
    "weekly": 7,
    "monthly": 30,
    "yearly": 365,
}
```

The three builders that would fill the other three directories have no call site anywhere. Parsed from every Python file under the source, tooling and desktop trees, plus the entry point:

| What was counted | Result |
|---|---|
| call sites of the weekly, monthly and yearly builders | 0 |
| occurrences of those three names, as a control | 9 |
| files under `tests/` naming any of the three | 0 |

The nine occurrences are the three definitions and six mentions inside docstrings, so the zero is a count of calls and not a fact about the file set.

The runtime tree reads the same way. The daily directory holds 66 files and the other three have held none since they were created on 2026-06-09. A driven run reproduced it exactly: one daily file written, and nothing in weekly, monthly or yearly.

### The three builders are gone

The cascade now declares one window. `PnLCascade` in `src/core/logging_engine.py` holds a single `PERIODS` key, `daily`, and one writer, `record_daily`. The three builders and the helper that wrote their files are removed.

The cascade makes one directory. A start reaches `daily/` only. It no longer makes `weekly/`, `monthly/` or `yearly/`.

| What was counted | Before | After |
|---|---|---|
| occurrences of the three builder names in the tree | 9 | 0 |
| windows the cascade declares | 4 | 1 |

| What was counted | Before | After |
|---|---|---|
| directories the cascade makes at a start | 4 | 1 |
| the daily writer | present | present |

The runtime tree keeps every file it holds. The daily directory held 67 files on 2026-09-13, and the other three held none. Nothing was written or deleted under the runtime tree to prove this.

### A store carrying the retired group still opens

`SettingsManager._apply_dict` walks the declared fields and takes only the keys a file shares with them, so a stored group with no field behind it is passed over and every other value loads. A file holding the three retired keys beside thirteen other settings was driven through the real store:

| | before the removal | after the removal |
|---|---|---|
| the dialog built | yes | yes |
| tabs in the payload | 11 | 10 |
| a Logging tab is drawn | yes | no |
| layout pages in the payload | 11 | 10 |
| control specs in the payload | 68 | 62 |
| control specs on the Logging tab | 6 | 0 |
| check boxes on the built window | 30 | 24 |
| the six retired labels found on that window | 6 | 0 |
| stored groups the dialog persists | 4 | 3 |
| rows the retired group declared | 3 | 0 |
| other settings read back at their stored value | 13 of 13 | 13 of 13 |
| the retired group read off the store | a dict of three keys | `None` |
| anything raised on load | no | no |

The thirteen siblings include the username, the theme, the accent colour, the folding group, the monitor group and the indicator weights, and each one reads back at its stored value. The three remaining groups still round-trip through a Save.

Both columns were driven, on the unchanged tree and on the change, from one store holding the same fifteen keys. The reader that finds none of the six labels on the new window found all six on the old one, so the zero is a fact about the dialog and not about the reader.

### What a live trade still writes

Nothing about a live trade changed. The same three writers were driven on the branch and each produced what it produced before: the voting record with its indicator reading and its two times, the gate record with its armed flags and its blocker list, and one daily profit and loss file.

### The sentences this section leaves behind

Each sentence below stands in an earlier section and no longer describes the dialog. The earlier text stays where it is.

| Earlier sentence | What the dialog does now | | | |
|---|---|---|---|---|
| "Two checkboxes and four periodicity boxes. Every key below sits inside one stored `data_logging` group." | The dialog has no Logging page, and the store declares no such group. | | | |
| "Log TA signal samples with all values and timestamps - Writes each indicator reading out with its value and its time." | The voting record writes each indicator reading with its value and its time, for every fired trade, with no setting in front of it. | | | |
| "The first of the two flags on the page, and the one that decides whether the indicator record is written at all." | No flag decides it. The record is written on every fired trade. | | | |
| "Highlight entries near Scrumming Bot trades - Marks the log lines that sit close to a fire." | Both decision logs hold one row per fired trade, so every row already sits at a fire. | | | |
| "The second flag, built the same way and checked at build like the first." | Neither flag exists. | | | |
| "P/L Log Periodicity - Chooses the windows a profit and loss figure is summarised over." | No window is summarised. The daily file is the only one anything writes. | | | |
| "Four separate boxes rather than one control. Save walks them in page order and writes the checked ones into a list." | The four boxes are gone and the list is no longer stored. | | | |
| "Save collapses all six into one dictionary holding the two flags and the list of active periodicities." | Save writes three groups and none of them is this one. | | | |
| "No reader outside the dialog reads the group either, which leaves the six controls writing to a key nothing consults." | The six controls are off the page, so nothing writes to the key. | | | |
| "In development. What the load path should restore for this page has not been settled against the rest of the dialog, so nothing is proposed here." | The page is gone, so nothing restores it. | | | |
| "The load path restores the whole group: both flags and the list of active periodicities." | No group, and no rows to restore. | | | |
| "\ | Logging \ | Log TA signal samples with all values and timestamps, Highlight entries near Scrumming Bot trades, P/L Log Periodicity \ | " | No rows on this page, because there is no page. |

| Earlier sentence | What the dialog does now | | | |
|---|---|---|---|---|
| "Eleven tabs -- User, Exchanges, Trading, Profit Folding, TA Indicators, Phantom Bots, Theme, Logging, Sound, SMS and AI Monitor" | Ten tabs, and Logging is not one of them. | | | |

### What is still owed

The roll-up itself is absent, and taking the control off the page does not build it. A weekly, monthly or yearly profit and loss file needs something that calls the builder, and no such caller exists. That is a behaviour to build, not a setting to wire, and it belongs to whatever owns the profit and loss evidence layer.

In development.

## 2026-09-12 - The AI Monitor page reaches the monitor it names

Seven settings, one store group, one carry. The page was driven end to end on the React build, which is what the application draws, and again on the Qt build. Six of the seven arrive at the monitor. The seventh still arrives nowhere.

`src/core/settings.py` - the one declaration behind all seven

```
@dataclass
class AIMonitorSettings:
    """Field defaults for the ``AppSettings.ai_monitor`` group."""

    api_key: str = ""
    interval_hours: float = 4.0
    connect_phrase: str = ""
    confirm_phrase: str = ""
    enabled: bool = False
    auto_handshake: bool = True
    log_feedback: bool = True
```

### What the page was asked and what it answered

A store holding an obvious fake key and both phrases was opened, the dialog was built, and every control was read off the running object. All seven opened on the stored figures on both builds. Typing into them and pressing Save wrote all seven back. Reopening a fresh manager read all seven.

```
api_key         wrote the fake key          read the fake key
interval_hours  wrote 1.5                   read 1.5
connect_phrase  wrote u75-connect-phrase    read u75-connect-phrase
confirm_phrase  wrote u75-confirm-phrase    read u75-confirm-phrase
enabled         wrote True                  read True
auto_handshake  wrote False                 read False
log_feedback    wrote False                 read False
```

### The key is masked on both builds

One entry on the control list carries the masking, so neither build can lose it without the other losing it too. On the drawn page the key row is a password field, read off the browser twice in one process and `password` both times. On the Qt build the same row reads `EchoMode.Password`.

`src/gui/main_tabs/settings_dialog_surface.py` - the masked row

```
{
    "tab": AI_TAB,
    "group": AI_API_GROUP_TITLE,
    "label": "Anthropic API Key:",
    "name": "ai_api_key",
    "kind": LINE,
    "min_height": SMS_CONTROL_MIN_HEIGHT,
    "echo": "password",
    "placeholder": "sk-ant-api03-...",
}
```

Neither phrase row is masked, which is what the page already says about them.

### No provider was contacted to prove any of this

The two phrases and the journal hash were read out of the composed prompt, which is built without a request. The request arm itself was reached with the transport made unimportable, so the arm ran and no socket opened.

```
connect phrase in the prompt  True
confirm phrase in the prompt  True
journal hash in the prompt    True
the key in the prompt         False
handshake arm reached         no transport, no call
with no key                   {'authenticated': False, 'message': 'No API key'}
analysis with no key          {'status': 'disabled'}
```

The key never enters the prompt. It goes in one header field and nowhere else.

### A check interval the monitor cannot count is refused

The page holds the figure between half an hour and twenty four hours, so nothing the operator types can leave that range. The store is a file, and a figure that reaches it another way used to go straight through. Zero, a negative and a true flag each made the review due on every dashboard pass; a word and an empty value each raised inside the due test, once per pass, with nothing on screen.

`src/trading/live_monitor.py` - the refusal

```
    @classmethod
    def wait_hours(cls, raw) -> float:
        """Return ``raw`` as the wait ``should_check`` counts, else the default.

        A stored ``interval_hours`` of zero, a negative, a string or None would
        make ``should_check`` answer True on every tick or raise inside it, so
        ``configure_live_monitor`` cannot hand either through.
        """
        hours = float(raw) if isinstance(raw, (int, float)) else 0.0
        if not math.isfinite(hours) or hours <= 0:
            logger.warning(
                "LiveMonitor: check interval %r is not a wait; using %.1fh",
                raw,
                cls.DEFAULT_INTERVAL_HOURS,
            )
            return cls.DEFAULT_INTERVAL_HOURS
        return hours
```

Driven through the real manager, each refused figure now holds four hours and the review waits for it. The log names the figure it refused.

```
    interval 'four'  held 4.0  due right after a check  False
    interval 0.0     held 4.0  due right after a check  False
    interval -3.0    held 4.0  due right after a check  False
    interval None    held 4.0  due right after a check  False
    interval 1.5     held 1.5  due right after a check  False
    interval 1.5     held 1.5  one interval later       True
```

### The journal switch now writes a line

The switch is read when an answer arrives. The note it built was the record type the monitor's own journal keeps, and the journal the window holds is a different one that keeps a different entry. Driven with the note the window built, the write produced no file and no line, and the window logged that the note was not written.

```
    record handed over     TradeRecord, from live_monitor
    journal files          []
    lines on disk          0
    what it raised         'TradeRecord' object has no attribute 'symbol'
```

`src/gui/main_window.py` - the note as the held journal takes it

```
if ai_cfg.get("log_feedback"):
    try:
        # _journal is reconciliation.TradeJournal, so the note
        # goes in through record_from_trade as a JournalEntry.
        self._journal.record_from_trade(
            bot_id="AI_MONITOR",
            symbol="SYSTEM",
            action="AI_FEEDBACK",
            side="neutral",
            price=0.0,
            quantity=0.0,
            reason=feedback[:500],
        )
    except Exception:
        logger.exception(
            "AI feedback note was not written to " "the journal"
        )
```

The same call now lands one row, and the journal counts it.

```
journal files        ['journal_2026-09-12.jsonl']
on disk              AI_MONITOR SYSTEM AI_FEEDBACK | the answer text
lines on disk        1
entries in memory    1
statistics           {'total_entries': 1, 'actions': {'AI_FEEDBACK': 1}}
```

### What the drawn page now does with a figure it refuses

The number box reports whatever the browser read out of it, and an empty box reads as no figure at all. That reached the control, the control refused it, and the refusal left the dialog through the channel the page's console lines arrive on. The figure was kept and nothing said so.

`src/gui/react_settings_dialog.py` - the refused edit

```
try:
    self._holders[name].admit(asked.get("value"))
except Exception as exc:  # noqa: BLE001
    # A cleared number box reports null, which admit refuses; redraw
    # puts the held figure back instead of leaving the box empty.
    logger.warning(
        "Settings page sent %s a value its control refuses: %s",
        name,
        exc,
    )
    self.redraw()
    return
```

Driven on the drawn page, a cleared box is now named in the log and the held figure is kept. A typed phrase and a typed figure both still arrive, which is what proves the channel itself is alive.

```
typed 2.5 into the box        the dialog holds 2.5
typed a phrase into the row   the dialog holds the phrase
cleared the box               the dialog still holds 2.5, the refusal is logged
```

The box itself still draws empty until the next figure is typed, because the drawn controls take their figure once, when the page first builds them. Every control on every page of this dialog takes its figure that way, so a figure the dialog pushes after the page is up reaches the store and not the box. That is one change to how the page builds a control, and it is owed.

In development.

### The drawdown share says what it is

The monitor's own journal keeps the deepest fall from peak as a share of peak, and the name said nothing about that. The figure the review line prints and the figure the report file publishes are both unchanged.

`src/trading/live_monitor.py` - the running deepest fall

```
if s["peak"] > 0:
    dd_pct = (s["peak"] - trade.portfolio_value) / s["peak"] * 100
    s["max_dd_pct"] = max(s["max_dd_pct"], dd_pct)
```

Driven across a peak of two hundred and a fall to one hundred, the share reads twenty five at one hundred and fifty, holds at one hundred and eighty, and reads fifty at one hundred. The report's own field keeps the name it published.

```
value 200.0  peak 200.0  share  0.0
value 150.0  peak 200.0  share 25.0
value 180.0  peak 200.0  share 25.0
value 100.0  peak 200.0  share 50.0
report       {'advantage_pct': -50.0, 'max_dd': 50.0}
```

### Auto-handshake still reaches nothing, and why it is not wired

The box stores and restores. Nothing reads it. The phrase exchange it describes is what the review already does on its own, every time the connection is not yet proved, and it asks no switch.

`src/trading/live_monitor.py` - the review's own exchange

```
async def analyze(self, portfolio=0.0, passive=0.0, bots=0) -> dict:
    """Send performance snapshot. Auto-handshakes if needed."""
    if not self._enabled:
        return {"status": "disabled"}
    if not self._authenticated:
        hs = await self.handshake()
        if not hs.get("authenticated"):
            return {"status": "auth_failed", **hs}
```

The exchange has one caller in the whole application, and that is the line above. The box's ticked position is therefore the only behaviour there is, and its cleared position asks for an exchange the operator requests,

which nothing can request: Test Handshake requests none. Wiring the box today would make its cleared position a second way of turning the monitor off, which the box above it already does. The box waits on a Test Handshake that performs one.

In development.

## 2026-09-13 - The credential pair is masked and Username is not

The API Key row and the API Secret row are both hidden while they are typed. Two sections above record the pair as readable and say the change was left to a unit. This is that change. One declaration carries it, and the page reads that declaration for both rows.

`src/gui/main_tabs/settings_dialog_surface.py` - the two specs

```
{
    "label": "API Key:",
    "name": "new_api_key",
    "kind": LINE,
    "echo": "password",
},
{
    "label": "API Secret:",
    "name": "new_api_secret",
    "kind": TEXT_AREA,
    "echo": "password",
},
```

### How a multi-line secret is hidden

A single-line box hides its characters by taking a password type. A text area has no type attribute at all, so it cannot. The running build draws the page inside a web view, and that engine offers a style property that hides the characters of any element. The style sheet applies it to a text area carrying the masking mark.

`src/gui/web/settings_dialog.css` - the one rule a text area needs

```
textarea.acervator-settings-dialog[data-part="control"][data-echo="password"] {
  -webkit-text-security: disc;
}
```

### The three options and the reading that refused two

Three things were driven on the loaded page. The engine supports the property. A text area reports no type. A single-line box handed a two-line value gives one line back, so drawing the secret as a single line would flatten a pasted PEM and the venue would refuse the key.

```
CSS.supports("-webkit-text-security", "disc")   True
the text area's type attribute                  null
two lines into a single-line box, read back      "...secretsecond-line"
two lines into the text area, read back          "...secret\nsecond-line"
```

### Both halves now end in the same state

Read off the rendered page, with the pair beside the passphrase row that was already hidden. All three report the same hiding style.

```
API Key      INPUT     type password   data-echo password   text-security disc
API Secret   TEXTAREA  type null       data-echo password   text-security disc
Passphrase   INPUT     type password   data-echo password   text-security disc
```

### The hidden characters measured in the page's own layout

A hidden row draws one repeated mark for every character, so two values of the same length draw to the same size while two of different lengths still differ. The key row was measured across its rendered width and the secret row down its rendered height.

```
                        before     after
key, 200 wide chars     2254 px    986 px
key, 200 thin chars      593 px    986 px
key, 600 thin chars                2937 px
secret, 400 wide         206 px     98 px
secret, 400 thin          62 px     98 px
secret, 8 thin                       58 px
```

### What a hidden row still saves

Both rows were typed through the page and both getters the add path reads returned what was typed. The newline inside the secret survived, so the PEM conversion still receives the bytes it converts.

```
the add path reads the key      "u159-not-a-real-key-0000"
the add path reads the secret   "u159-not-a-real-secret\nsecond-line"
a newline survived the row      True
```

### The hints above the rows stay readable

Hiding the characters of a value does not hide the placeholder, so the Coinbase key shape and the PEM note still show on an empty row.

```
API Key      placeholder text-security none, colour rgb(117, 117, 117)
API Secret   placeholder text-security none, colour rgb(117, 117, 117)
Passphrase   placeholder text-security none, colour rgb(117, 117, 117)
```

### Username is not a credential row and stays readable

The Username row keeps a plain text field. It was typed on the loaded page, read back off the object the save path reads, saved, then read again from a second dialog built over the same store.

```
username row as drawn    INPUT  type text  text-security none
typed on the page        u159-fake-operator
read off the object      u159-fake-operator
store after Save         u159-fake-operator
a second dialog reads    u159-fake-operator
the reloaded page shows  u159-fake-operator
```

### What the Qt build still shows

The Qt dialog builds its own widgets and reads neither spec, so both rows there are unchanged and both stay readable. A Qt text edit has no echo mode. The only precedent for hiding one is the init wizard, which paints the text transparent. That also hides the placeholder and leaves the characters selectable, so applying it would trade one visible fault for two. The Qt pair is left as it is.

`src/gui/init_wizard.py` - the Qt precedent, and what it costs

```
self._api_secret.setStyleSheet(
    "color: transparent; background-selection-color: transparent;"
)
```

## 2026-09-13 - The first-run path keys no vault to a placeholder

A launch on a machine that has never run Acervator no longer writes a name into the store. The startup block reads the stored name, reports a first run in the log, and leaves the entry alone, so nothing claims the operator is called User.

`main.py` - the first-run block, after the change

```
stored_version = settings.get("app_version", "")
current_version = _acervator_version

if not settings.get("username", ""):
    log_manager.info("First run — no stored name, no wizard")
```

The version write that sat beside the name write is not lost. The next block compares the stored version against the running one, and on a fresh store those differ, so the version is written there as it always was.

### One declaration builds the phrase

The phrase the credential vault encrypts under is now written once, in the module that holds the encryption. Every site that seals or opens a stored credential calls it, so the four sites cannot disagree about the text.

`src/core/encryption.py` - the declaration and the fallback

```
MASTER_FORMAT = "qat_{username}_vault"
UNNAMED_OPERATOR = "user"


def vault_phrase(username: str) -> str:
    return MASTER_FORMAT.format(username=str(username) or UNNAMED_OPERATOR)
```

The four callers are the encrypt site on the Exchanges page, the two decrypt sites that open a stored secret, and the drawn page's own phrase builder.

```
src/gui/settings_dialog.py        _add_exchange                seals
src/gui/main_window.py            _connect_exchange_for_bot    opens
src/gui/widgets/api_tester_tab.py _do_connect                  opens
src/gui/main_tabs/settings_dialog_surface.py  _master_phrase   the drawn page
```

### The fallback fires on a store with no name

Driven on an empty directory, with the home redirected before the first import. The store declares the field, so a read returns the declared empty string and the fallback in each literal never had a chance to fire. The one declaration reads that empty string and substitutes the unnamed operator.

```
stored username            ''
phrase, before the change  'qat__vault'     fallback fired  False
phrase, after the change   'qat_user_vault' fallback fired  True
the drawn page and the declaration agree    True
sealed under the new phrase, opens again    True
```

### An existing install opens the same secrets

A store that already carries a name is untouched. The phrase built from a name is the same text before and after, and a secret sealed under the earlier phrase opens under the phrase the changed tree builds.

```
phrase on a named store, before   'qat_FAKEOPERATOR_vault'
phrase on a named store, after    'qat_FAKEOPERATOR_vault'
same phrase                        True
a secret sealed before opens now    True
```

Only a store with no name at all reads a different phrase than it did, and no such store can hold a credential, because the startup write filled the entry before any venue could be added.

### The wizard's secret row starts hidden

The install wizard builds its own widgets and reads neither the page spec nor the style sheet the drawn page uses. Its key row masked its value at build while its secret row did not, because the hiding lived only inside the Show credentials handler, which nothing called until the box was ticked. The handler now runs once as the wizard is built, and it carries the sheet as one declaration.

`src/gui/init_wizard.py` - the hidden state and the build seed

```
SECRET_HIDDEN_STYLE = (
    "color: transparent; selection-color: transparent; "
    f"placeholder-text-color: {ds.TEXT_PLACEHOLDER};"
)
...
self._toggle_visibility(self._show_key.isChecked())
```

Read off the rendered rows, with a monospace font loaded so two values of one length differ only in ink. A selection is read by comparing two such values rather than by counting the highlight, which moves with the text width.

| Row and state | which characters paint | the hint paints | a selection shows which |
|---|---|---|---|
| Secret, at build, before | 896 | 1122 | 896 |
| Secret, at build, after | 0 | 1122 | 0 |
| Secret, Show ticked, after | 896 | 1122 | 896 |
| Secret, hidden again, after | 0 | 1122 | 0 |

The Show ticked row is the control: it reports on every reading, so each zero is a fact about the state and not about the instrument. The hint now survives the hide, which the earlier sheet removed along with the characters. The key and passphrase rows report a masked display and a refused copy at build, both before and after, so the pair ends in the same state.

### What the install path still leaves open

A Qt text area hands its value to the clipboard whatever colour it paints in, while a single-line row in password mode refuses the copy. Measured on every state of the secret row, the copy returned the typed value; on the key row it returned nothing. No style sheet closes that, and the wizard draws on no path today.

```
API Secret     copy hands over the value   True   in all four states
API Key        copy hands over the value   False  at build and hidden
```

The wizard's name box still fills itself with User and still refuses an empty name. That collection reaches no caller, and the unlock it would feed is the Quintessence Wallet, so it waits on the wallet rather than being removed.

## 2026-09-13 - One venue list, one passphrase set

Four copies of two facts were measured, and the copies are gone. The connector decides which venues exist and which of them ask for a passphrase. Every screen now reads those two answers instead of keeping its own.

```
def exchange_ids() -> tuple:
    from ...exchange.ccxt_connector import SUPPORTED_EXCHANGES

    return tuple(sorted(SUPPORTED_EXCHANGES.keys()))
```

The screen offers the same fifteen venues in the same order as before, and the same three still ask for a passphrase. Nothing about what is offered changed.

### Where the two lists were declared

Six declarations were found, and three of them were copies of the other three.

| Fact | Declaration | Where |
|---|---|---|
| the venues | `SUPPORTED_EXCHANGES` | the connector, the authority |
| the venues | a frozen tuple | the install wizard page |

| Fact | Declaration | Where |
|---|---|---|
| the venues | a frozen tuple | the one-window bot runner |
| the passphrase venues | `PASSPHRASE_EXCHANGES` | the connector, the authority |
| the passphrase venues | a frozen set | the Settings page |
| the passphrase venues | a frozen tuple | the install wizard page |

Each copy is now a small function that asks the connector when it is called. The import sits inside the function, so loading a screen module still loads no exchange library and reads no settings file.

### The falsifier, one entry at a time

One venue was taken out of the connector inside a driving process, and every consumer was then asked what it offers. The tree itself still holds fifteen.

| Reading | Consumers asked | Still holding the removed venue |
|---|---|---|
| Before the change | 9 | 4 |
| After the change | 9 | 0 |

The same was done with one passphrase venue, against every place that decides whether the field is drawn.

| Reading | Consumers asked | That did not follow the connector |
|---|---|---|
| Before the change | 14 | 9 |
| After the change | 14 | 0 |

The four that carried the venue were the install wizard's ids, its labels, its bridge list and the bot runner's model. The nine that ignored the passphrase change included the ticked box and the drawn field on the Settings page, which is what the operator sees.

Why that matters, and it is not tidiness. A venue offered on a screen and absent from the connector is stored and then refused, before any object exists.

```
CCXTConnector('gemini') refused: Unsupported exchange 'gemini'. Supported:
['binance', 'coinbase', 'kraken', 'kucoin', 'bybit', 'okx', 'gateio', 'bitget',
'huobi', 'mexc', 'bitfinex', 'poloniex', 'bitstamp', 'cryptocom']
```

### Every reader of the chosen venue, driven

Five readers were named earlier as out of reach without a live window. All five were driven in one run, with real tab widgets and a real wizard, against a store holding two venues: one with an obvious fake credential and one with none.

| Reader | What it answered |
|---|---|
| the startup tab loop | two crypto tabs, Gemini and Kraken, no stock tab |
| the startup credential report | Gemini stored and unverified, Kraken with none |
| the bot connect lookup | both refusals, before anything was contacted |
| the bot wizard's venue picker | Gemini and Kraken, Gemini selected |
| the API panel's stored lookup | the no-credentials refusal |

One step is still out of reach. The Qt wizard is opened behind a modal call that waits for a person, so the handler that opens it cannot be run to the end without one. The wizard itself was built with the stored entries and its picker read.

### The diagnostic tool converts for every venue

The operator tool asked for a key at the keyboard and then converted a pasted private key's escaped newlines for Coinbase alone. Every other venue received an unusable key. The conversion now runs on the way in, for every venue, gated on the key's own armour lines.

```
secret = input("  API Secret: ").strip()
if looks_like_pem(secret):
    secret = unescape_pem_newlines(secret)
```

The tool also opened its fifteen public endpoints through the plain library call, which allows a file scheme, under a suppression marker that sat on the line above the real one. It now opens them through the project's own scheme-restricted request and opener.

### What the docstrings claimed

Two screen modules said in their own opening text that a value changed on one side alone is reported. Nothing reads prose, and the tree holds no test that compares them.

```
git ls-files tests/     conftest.py, 6 Solidity files, 1 data file, 150 reports
pytest --collect-only   no tests collected
```

The claim is removed rather than reworded. Each module now says plainly that nothing compares its written-out values, and names the values that are not written out and therefore cannot drift: the twelve indicator weights, the venue list and the passphrase set.

Two things this section does not change. Which venues are offered is still decided in one place only, the connector, and no venue was added or removed. The install wizard still writes its own shorter passphrase note beside a venue name while the Settings page writes the connector's longer one, because changing either would change words on a screen.

## 2026-09-13 - The exchange screen follows the store

Three things this page said were not things the program had done. A removed venue kept its tab until the next launch. An add reported a venue verified when nothing had been contacted. An add named the rows that were typed rather than the entry that was stored. Each one now reads off what happened.

### The tab bar takes a venue off

The sync added and never dropped. It now asks each layer for the ids the store no longer lists, takes those tabs off the bar, and returns how many went.

`src/gui/main_window.py` — `_drop_unlisted_exchange_tabs`, called by the sync

```
_gone = self._drop_unlisted_exchange_tabs(_wanted)
```

Driven on the running window, crypto layer, reading the tab captions and the rendered bar after each press.

| The press | Captions before | Captions after |
|---|---|---|
| Remove Selected on Kraken | `Kraken, Kucoin` | `Kucoin` |
| Test and Add Exchange for Gemini | `Kucoin` | `Kucoin, Gemini` |
| Remove Selected on the last two | `Kucoin, Gemini` | `Get Started` |

Before the change all three presses left the same three captions and the same rendered bar, so the picture agreed with the list and both were wrong. The sync still adds, which is the second row above.

The Get Started page comes back because each layer now keeps its placeholder object instead of dropping the reference on its first add. A tab taken off is asked to stop its news ticker and its data-pull countdown first; both keep running otherwise, because a tab removed from a bar is still a child of the layer's stack.

Nothing here changes what a removal permits. The store drops the entry exactly as it did, no bot is consulted and none is stood down.

### The add message reads the stored entry

The phrase came from the API Key row. It now comes from the entry the store holds after the write, through one function both builds call.

`src/gui/main_tabs/settings_dialog_surface.py` — `stored_credential_phrase`

```
def stored_credential_phrase(entry: Any) -> str:
    holder = entry if isinstance(entry, dict) else {}
    if any(holder.get(name) for name in CREDENTIAL_FIELDS):
        return ADDED_WITH_CREDENTIALS
    return ADDED_WITHOUT_CREDENTIALS
```

| The press | The line before | The line after |
|---|---|---|
| Key typed, secret empty, nothing stored | `Kucoin added with credentials (verified).` | `Kucoin added without credentials.` |
| Both rows empty over stored credentials | `Kucoin added without credentials.` | `Kucoin added with stored credentials.` |
| Both rows empty, nothing stored | `Kucoin added without credentials.` | `Kucoin added without credentials.` |

The Activity Log line and the confirmation box take the same phrase, so the three surfaces cannot disagree. The third row is the control and it did not move.

The first row is the press row 8 measured. A key with no secret never reaches the connection check and never reaches the store, so nothing was contacted and nothing was stored, and the word verified had no grounds behind it. The word is gone rather than reworded: no add can earn it without a connection, and the venue's own answer is already reported on that same line by the check that got it.

A blank re-add still keeps the stored tokens, which is row 8's repair. The second row above is that press, and the entry it names carries the same two tokens it carried before.

### What could not be driven

`QMessageBox.information` waits for a person, so the add cannot be driven past it on either build. The two lines written before it were read off the running dialog, and the box text was read off the Qt-free model, which records the box instead of showing it. All three take their phrase from the one function above.

## 2026-09-13 - The bot's increment style comes off BotConfig

The Increment Style row left the Trading page on 11 September and the bot field of the same name stayed. Nothing wrote that field and nothing read it. Both of its declarations are now gone.

```
# src/trading/container/config.py, both lines removed
increment_style: str = "linear"          # the BotConfig field
"increment_style",                       # the Scrumming-only name
```

The sweep was run twice on the same name, once matching case and once ignoring it, because a case-sensitive search has already missed a site twice in this audit.

| probe | before | after |
|---|---|---|
| occurrences in code | 2 | 0 |
| occurrences repo-wide, matching case | 22 | 20 |
| occurrences repo-wide, ignoring case | 22 | 20 |
| writers anywhere | 0 | 0 |
| readers anywhere | 0 | 0 |

The twenty that remain are this page, the Trading tab page and one captured report. None of them is code.

### A record carrying the retired name still restores

The restore path builds its arguments through one helper, and that helper keeps only the names the dataclass declares. A record holding a name the product has retired loses it at that filter instead of raising.

```
record            43 stored keys, four of them retired names
restored          1 bot, ScrummingBot, on the read-only restore placeholder
fields compared   65
fields different  1, the removed name
raised            nothing
```

The creation path reads that same declaration, so one removal reaches both paths at once. A wizard-shaped dictionary driven through the same two calls gave the same 65 compared against 1 different, and a record holding only a venue, a pair and a mode still restored with nothing raised.

### The bot's own behaviour does not move

One restore and one tick were driven before the change and after it, against the read-only placeholder the restore path already uses. That placeholder answers False to every order method, so no order can be placed from the run.

| reading | before | after |
|---|---|---|
| tick readings compared | 10 | 10 |

| reading | before | after |
|---|---|---|
| tick readings different | - | 0 |
| what the tick raised | the placeholder has no ticker reader | the same |

The comparison can report a difference. A control run moved the stored target balance and the stored read rate, and the same comparison then named three fields instead of one and two tick readings instead of none.

### The bot's spacing style was already gone

The bot-side spacing style was removed with the Settings-side one. Nothing was left to take out.

```
spacing_style   0 occurrences in src, tests, tools, desktop and dev_harness
                4 occurrences repo-wide, both files dated history
```

### The wizard stops advertising the retired Grid Bot

The wizard's opening description named a mode the product does not build, listed three rows already off the Settings dialog, and carried a version string. A second description inside the parameter page named the same mode again, with two more version strings.

```
# src/gui/bot_wizard.py, the opening description now reads
#   Accumulation Bot: Target Balance, Scrumming Interval, TA Engine, Phantoms
#   Extractor Bot: base-currency chunk, artillery sizing, compounding tier
#   Both: Visibility, Aggressive Trading
```

The page subtitle that named the retired mode is now one sentence with no version in it. No caller can draw it: the mode page answers False to the grid question on every call, so that branch has never been reached.

### The retired names this unit left in place

Each of these carries a reason, and none of them is in the wizard's own text.

| where | what it says | why it stays |
|---|---|---|
| the launcher | two screen strings naming the retired mode | that file does not pass its own coding verdict, so nothing ships from it |
| the mode page | a comment beside a constant return | it states why the method returns a constant |
| the restore path | one comment naming the retired mode | it explains why a stored mode is skipped |
| a captured report | one occurrence of the removed name | captured output, already owned by another row |

### The second wizard module keeps its stored-default read

The second wizard module reads a stored target balance that never arrives, because the shell's request carries the step values alone. The read itself is live: the model calls it on every build, and three sibling reads of the same bag sit beside it, two of them built deliberately by the phantom rows above.

```
_apply_stored_target_balance   called on every model build
sibling reads of the same bag  3
```

Removing it would delete one of four reads of one bag and settle three rows that are still open. It stays, and the empty bag is what those rows address.

## 2026-09-13 - The shell's wizard request carries the store

The desktop shell asks for the Create Auto Trader pages and sends the step values alone. The running program now puts its own stored settings into that request before the pages are laid out, so the wizard the shell draws opens on the same figures as the wizard the Qt window opens.

```
# src/gui/main_tabs/bot_wizard_surface.py, the handler the bridge registers
def live_view_model(params: dict, live: Any) -> dict:
    asked = dict(params or {})
    manager = getattr(live, "settings_manager", None)
    if asked.get("defaults") is None and hasattr(manager, "get_all"):
        asked["defaults"] = manager.get_all()
    return view_model(asked)
```

The bag the request carries went from no keys to seventeen. That was read on the request the shell sends when the New Bot space mounts, and again on the request every control press sends, so a press does not put the wizard back on its built-in figures.

### What the request now carries

Five Settings rows reach a control on the shell build. Two of them are lookups this section adds to the wizard surface; the other three were already written and had nothing to read.

| Settings row | store key | wizard control | opened at | opens at |
|---|---|---|---|---|
| Default Target Balance | `default_target_balance` | Target Balance | 200.00 | the stored figure |
| Bot Visibility | `bot_visibility` | Order Visibility | Order Book | the stored name |
| Enable aggressive trading mode | `aggressive_trading` | Aggressive Trading | clear | the stored flag |
| Enable Phantom Balance Bots for Scrumming | `default_enable_phantoms` | Enable Phantom Bots | clear | the stored flag |
| Lock duration (candles) | `default_lock_candle_count` | Candles to lock | 2 | the stored figure |

The two new lookups sit beside the three already there, and all five run inside the model build, before the page is drawn.

```
# src/gui/main_tabs/bot_wizard_surface.py, inside _build
self._apply_stored_target_balance()
self._apply_stored_visibility()
self._apply_stored_aggressive()
self._apply_stored_phantom_enable()
self._apply_stored_lock_candles()
```

### The five figures on both builds

One store, written with five values that differ from every built-in figure, was read by both wizards. Both opened on the same five.

| control | the store | the Qt wizard | the shell wizard |
|---|---|---|---|
| Target Balance | 777.00 | 777.00 | 777.00 |
| Order Visibility | internal | internal | internal |
| Aggressive Trading | on | on | on |
| Enable Phantom Bots | on | on | on |
| Candles to lock | 7 | 7 | 7 |

Before the change the shell wizard opened at 200.00, Order Book, clear, clear and 2, on the same store. A bot created from those pages carried those five figures, so the shell built a different bot from the Qt window on identical settings. It no longer does.

### An older store still opens the wizard

A store written before these rows existed carries none of the five keys. The request then hands over the twelve keys it does hold, every lookup falls to its own built-in figure, and the wizard lays out its six pages as usual.

```
bag keys 12    target balance 200.00    visibility Order Book
               aggressive clear         phantoms clear
               lock candles 2           pages 6
```

The shell reaches the store only while the trading program serves the bridge. Started as the surfaces alone, with no running system behind it, the bridge has no settings manager to read and the wizard opens on its built-in figures.

## 2026-09-13 - The store refuses a value of the wrong type

Two rows turned out to be one fault. `AppSettings` names a type for each of its seventeen fields, and nothing anywhere held a stored value to that type. Driven across all seventeen, the setter took a word where a number was declared and a word where a switch was declared, on seventeen of seventeen.

The store now answers for its own declaration. One function reads the type each field's own default carries, and both write paths ask it.

```
# src/core/settings.py
DECLARED_TYPES: dict[str, type] = {
    name: type(value) for name, value in asdict(AppSettings()).items()
}


def declared_type_holds(key: str, value: Any) -> bool:
    ...
```

A switch is refused where a number is declared, and a whole number is admitted where a dollar figure is declared, because a settings file writes 350 and reads 350 back.

### What the two write paths do now

`SettingsManager.set` raises and names the field, the type it takes and the type it was handed. The load path never raises, because a settings file that stops the program starting is worse than one field on its default.

| path | a value of the wrong type | what the operator sees |
|---|---|---|
| `set` | `TypeError: Setting 'aggressive_trading' takes bool, not str` | the Settings page's own save loop names the field and keeps going |
| the load path | the field's declared default, and one warning line | `Settings kept the default for default_target_balance: the file holds str, the field takes float.` |

Neither path changes a value into something else. A word is never turned into a number, because a changed value is a second way to be wrong.

### A settings file that stops the bot wizard opening

A hand-edited file holding a word where the target balance belongs stopped the wizard being built at all. The Settings page caught the same value and drew one dollar; the wizard had no guard beside its two siblings, which already refuse an unknown visibility name and admit only a switch.

| surface | before | after |
|---|---|---|
| the Qt wizard | `TypeError` at construction, no window | opens, Target Balance $200.00 |
| the shell wizard | `TypeError` at model build, no pages | opens, Target Balance 200.00 |
| the Settings page | opened, drew `$1.00` | opens, draws $200.00 |

Both wizards now keep their own figure when the value handed to them is not a number, and both still open on a figure the operator did set.

```
stored 777.00     Qt wizard 777.00      shell wizard 777.00
stored a word     Qt wizard 200.00      shell wizard 200.00
stored 350        Qt wizard 350.00      shell wizard 350.00
```

### A switch that read on whatever the file held

The aggressive trading flag is declared as a switch, and a word in the file read as on across every surface at once. Both surfaces agreed with each other, which is why nothing reported it.

| surface | before, file holds the word no | after |
|---|---|---|
| the Qt wizard box | on | clear |
| the shell wizard box | on | clear |
| the Settings page box | on | clear |
| the config a new bot is built from | on | clear |

A switch the operator really did set still reaches the wizard on: stored on opens the box on, which is the control that proves the reading is the store's and not the guard's.

### A store at plausible values loads unchanged

Every one of the seventeen fields was written at a value the operator could hold — a venue entry with both tokens, a target balance of 350.00, a chosen theme, an accent colour, twelve indicator weights, the sound group and the message channels group. It was written through both writers the store uses.

```
fields compared 17   different 0    written as settings.toml
fields compared 17   different 0    written as settings.json
control: the same 17 with two words planted -> different 2
         default_target_balance and aggressive_trading, both on their defaults
```

The control is the second half of that block. The comparison reports a difference when there is one, so the two zeroes above are a reading and not an empty instrument.

### The creation record names figures the wizard carries

Every bot ever created logged `Positions=0`. The record read a name off the wizard's dictionary that no branch of the wizard writes — it is a retired name, listed among the settings the bot config drops. An Extractor recorded a balance of zero for the same reason, because that mode's dictionary carries no target balance either.

```
# src/gui/main_window.py
if config.get("mode") == "extractor":
    _created = (
        f"Chunk size=${config.get('extractor_chunk_size_usd', 0):.2f}, "
        f"Scan top={config.get('extractor_scan_top_n', 0)}"
    )
else:
    _created = (
        f"Balance=${config.get('target_balance', 0):.2f}, "
        f"Tranche count={config.get('stack_tranche_count_target', 0)}"
    )
```

Both names are the wizard's own labels: Tranche Count on the parameters page and Chunk size (USD) on the Extractor page.

| mode | before | after |
|---|---|---|
| Scrumming | `Balance=$200.00, Positions=0` | Balance=$200.00, Tranche count=3 |
| Extractor | `Balance=$0.00, Positions=0` | Chunk size=$100.00, Scan top=8 |

Moving the two controls moves the line: a tranche count of 9 and a target balance of 512.00 recorded Balance=$512.00, Tranche count=9.

### The increment style sentence

The Trading page description above says a bot takes its own increment style from the wizard rather than from this page. That was true of neither half by the time it was written, and it is true of nothing now.

| what the sentence says | what is there |
|---|---|
| a wizard control sets an increment style | no wizard control does, on either build |
| a bot carries an increment style of its own | the bot config field was removed on 13 September |

Nothing sets an increment style and nothing reads one. The concept the sentence reaches for is Tranche Spacing, which the wizard does carry, and the derived placement the engine works out from Opposing Trade Distance is what decides where a tranche lands.

## 2026-09-13 - The wizard's Profit Folding page is removed

The bot creation wizard registered six pages and routed to five. The sixth was Profit Folding & Upward Distribution, and it is gone from both builds. Nothing a person can press changed, because nothing could reach it.

Driven with the home redirected into a scratch directory before the first product import, so the settings directory and the log root bound under that directory. No stored setting of the running install was read, no credentials file was opened and no venue was contacted.

### Which switch is which

Three controls wear almost the same words. Only one of them does anything, and this section removed neither that one nor the Settings row.

| The words on screen | Where it lives | What happened to it |
|---|---|---|
| Profit Folding Active | a running bot's own settings | untouched. It gates the fold and it works |
| Profit Folding / Upward Distribution Active | the Settings dialog's Profit Folding page | untouched. It is the last row on that page |
| Enable Profit Folding | the wizard's folding page | removed with the page it sat on |

The removed one is the wizard's. A bot already running reads its own flag from its saved record, and that reading is unchanged.

```
if not self.config.profit_folding_active:
```

### The page that no route reached

`nextId` was read from each page in turn with that page set as the start page, on both modes. No page answered the folding page in either.

```
before, accumulation      before, extractor
  asset  -> params          asset  -> params
  mode   -> asset           mode   -> pool
  params -> phantom         params -> finish
  pool   -> params          pool   -> params
  folding -> finish         folding -> finish     no page answers folding
  phantom -> finish         phantom -> finish
```

The Qt-free surface said the same in a named constant rather than by omission, and published it as a payload key. Both builds now register five pages, every one of the five is on a route, and the constant and its key are gone with the page that was the only entry either ever held.

### The bot the wizard builds, before and after

The collection was driven on both builds and both modes, and every key and every value was compared.

| Arm | keys before | keys after | key set identical | values different |
|---|---|---|---|---|
| Qt, accumulation | 49 | 49 | yes | 0 |
| shell, accumulation | 49 | 49 | yes | 0 |
| Qt, extractor | 49 | 49 | yes | 0 |
| shell, extractor | 49 | 49 | yes | 0 |
| Qt, on stored figures | 49 | 49 | yes | 0 |
| shell, on stored figures | 49 | 49 | yes | 0 |

The last two arms opened the wizard on a stored bag rather than on the build figures, and that bag moved five values on both builds either side of the change: target balance 200.00 to 512.00, the visibility from order book to internal, aggressive trading on, phantoms on, and the lock from 2 candles to 5. A bag that moved nothing would have made those two arms prove nothing.

The same comparison reported ten differences in the routing and payload readings taken in the same run, so a zero on the six config arms is a fact about the bot and not a blind comparator.

An Extractor still carries `profit_folding_active`, because both builds write it from a literal on the extractor branch and never from the folding page.

```
config["enable_phantoms"] = False
config["profit_folding_active"] = False
```

### The stored record that still loads

A record shaped like `bot_state.json` was written carrying all six of the page's names, and restored through the two calls the restore path makes.

| Stored name | On the built config, before | after |
|---|---|---|
| `fold_mode` | absent | absent |
| `fold_target` | absent | absent |
| `fold_target_count` | absent | absent |
| `distribute_target` | absent | absent |
| `distribute_target_count` | absent | absent |
| `profit_folding_active` | False | False |

Sixteen fields were compared and none differed. Nothing raised on either side. The five names stay on the retirement record in the container config, which is what drops them, so a record written before the page came off still builds.

### The fold ladder half nobody called

`stack_math` declared a matched pair of ladder-price helpers. The scrum half is called when a stack is split into tranches; the fold half had one definition and no call, no import and no aliased binding anywhere in the tree.

```
prices = scrum_ladder_prices(
```

The fold half is removed. The shared helper keeps its direction argument, because the Extractor passes the fold sign to the placement floor on a live path of its own.

### The sentences the wizard page leaves behind

Each sentence below stands in an earlier section and no longer describes the tree. The earlier text stays where it is.

| Earlier sentence | What is there now |
|---|---|
| "the wizard's folding page is unreachable - a route to the page and a call to its `get_config`" | The page is removed, so there is nothing to route to |
| "the box does not turn the page on - an enable rule over the three groups below it" | The wizard's box and its three groups are gone together |
| "No page answers the folding page. The only branch that would is guarded behind `is_grid()`" | That branch is removed and `is_grid` keeps its one live caller on the parameter page |
| "The field arrives the moment the folding page is reached and collected." | No page collects it. A new accumulation bot takes the declared default on its config |
| "Until the first of the three lands, a stored choice on this row has nowhere to arrive" | The first will not land. The Settings row's own answer is now free of the wizard |
| "The wizard registers six pages and routes to five." | It registers five and routes to all five |
| "the wizard lays out its six pages as usual" | It lays out five |
| "its view model reports the folding page unreachable" | The view model no longer carries an unreachable list, because no page is |

The Settings dialog's own Profit Folding row is untouched by this section and still holds the page alone.

## 2026-09-13 - The Settings dialog's Profit Folding page is removed

The page held one checkbox and nothing else. That checkbox reached no bot, no engine and no fold, so the page is gone from both builds and the dialog opens on nine tabs. Folding is still switched on and off per bot, on the running bot's own settings row, and that row is untouched.

The orphaned target-growth module goes with the page. `apply_profit_fold` had no caller anywhere in the tree, and the glossary named it as the fold's entry point while the live path is the fold tranches.

Driven with the home redirected into a scratch directory before the first product import, so the settings directory and the log root bound under that directory. No stored setting of the running install was read, no credentials file was opened and no venue was contacted. One connection to a venue host was attempted on purpose and refused.

### The three switches, and the one this section removed

Three controls wear almost the same words. Only one of them changes what a bot does, and this section left that one alone.

| The words on screen | Where it lives | What happened to it |
|---|---|---|
| Profit Folding Active | a running bot's own settings | untouched. It gates the fold and it works |
| Profit Folding / Upward Distribution Active | the Settings dialog's Profit Folding page | removed with the page |

| The words on screen | Where it lives | What happened to it |
|---|---|---|
| Enable Profit Folding | the wizard's folding page | removed earlier, with that page |

The stored names behind them are three different words, which is what keeps them apart.

```
per-bot switch    profit_folding_active     a BotConfig field, 10 read sites
Settings switch   profit_folding.active     an AppSettings group, 0 readers
wizard switch     -                         no longer in the tree
```

### The nine tabs the dialog draws now

Both builds take the tab list from one declaration, so removing the page removed the tab from the Qt window and from the shell together.

| | before | after |
|---|---|---|
| Tabs | 10 | 9 |
| Controls on the dialog | 67 | 66 |
| Controls on the Profit Folding tab | 1 | 0 |
| Layout entries | 10 | 9 |
| Groups the dialog persists | 5 | 4 |
| Keys a Save writes | 14 | 13 |

The tab list after the removal, read off the payload the shell draws:

```
User, Exchanges, Trading, TA Indicators, Phantom Bots,
Theme, Sound, SMS, AI Monitor
```

A tab holding no control is worse than no tab. What a reader of this page expects to find there is one master checkbox and three groups of radio buttons, eleven controls in all, and none of the eleven remains. Where folding is switched now is the running bot's own settings row, which the dated sections above already describe.

### The stored group a settings file still carries

A settings file written before the removal still carries its `profit_folding` entry. The loader copies only the fields the settings record declares, so an entry it no longer declares is passed over and every other value arrives.

Driven on a file carrying ten keys, one of them the folding group:

| | before | after |
|---|---|---|
| Fields on the settings record | 17 | 16 |
| The folding group is one of them | yes | no |
| Every other stored key arrived | yes | yes |
| Anything raised | no | no |

The next Save writes the file without the group.

### The per-bot switch, driven either side

A record shaped like `bot_state.json` was written, restored through the two calls the restore path makes, and the bot it built was driven with the switch on and then off. The readings are the same on both sides of the removal.

| The switch | applier returned | target after | preview returned | the gate read |
|---|---|---|---|---|
| on | $6.00 | $200.00 to $206.00 | $6.00 | on |
| off | $0.00 | held at $200.00 | $0.00 | off |

Two controls prove the comparison can report a difference rather than agreeing by being blind. Moving the units the preview is asked about took its answer from $6.00 to $4.00, and lowering the growth cap from three percent to one took the applier from $6.00 to $2.00 and the target to $202.00.

```
[switch] ON   apply 6.0  target 206.0  preview 6.0  gate true
[switch] OFF  apply 0.0  target 200.0  preview 0.0  gate false
[control] units 0.2        preview 4.0     moved
[control] growth cap 1.0%  apply   2.0     moved
```

The record and the wizard collection go through one declaration, so creation and restore cannot drift. Both carried twelve values into the config, sixty-four config fields were compared between the two, and none differed, on both sides of the removal.

### The orphaned target-growth module

`src/trading/profit_fold.py` held one function and forty-six lines. Its own module docstring recorded that no module imported it, and that was still true: four occurrences in the product, all four inside that file, three of them in the docstring and one on the definition line.

The live fold path is the tranche book, and the glossary now says so.

```
_fold_eligible_tranches      picks the tranches a fold may reach
_apply_fold_target_growth    grows the target from the fold surplus
```

### The sentences the Settings folding page leaves behind

Each sentence below stands in an earlier section and no longer describes the tree. The earlier text stays where it is.

| Earlier sentence | What is there now |
|---|---|
| "One master checkbox and three groups of radio buttons, eleven controls in all." | The page is removed and the dialog draws nine tabs |
| "Every key below sits inside one stored `profit_folding` group." | No page writes that group and the settings record no longer declares it |
| "The stored group holds one field after those removals." | The group is gone. A stored file still carrying it loads and the field is passed over |
| "Profit Folding / Upward Distribution Active - Turns the whole page on." | No page remains for it to turn on |

| Earlier sentence | What is there now |
|---|---|
| "This is the one control on the page the load path restores." | The load path restores no control on that page |
| "What the Profit Folding page holds now - One checkbox and nothing else." | It holds no checkbox, because the page is removed |
| "The master switch is the last row on the page, and its own verdict was that it waits on the wizard's folding page." | Both pages are removed. The verdict is settled by removal, not by waiting |
| "`SettingsManager._apply_dict` copies the whole `profit_folding` dict off the file, so a store written before the removal still carries its `mode` entry and still opens." | It copies the fields the record declares, and the folding group is no longer one. The store still opens |

Three earlier citations name a ladder helper removed with the wizard's folding page, and nobody had answered them. They are answered here.

| Earlier citation | What is there now |
|---|---|
| "`src/trading/stack_math.py:304` `fold_ladder_prices` - places fold prices from `stack_spacing_mode` and decides no size" | The fold half is removed. Its twin still places scrum prices |
| "`src/trading/stack_math.py:304` `fold_ladder_prices` - places fold prices and decides no size; its `levels` counts prices on a ladder" | The same helper, named a second time in the count table. It is gone |
| "`fold_ladder_prices(100.0, 4)` returned four prices and decided no size and no position." | That reading was taken before the helper was removed, and no call can be made now |

Five earlier citations name the target-growth module this section removes.

| Earlier citation | What is there now |
|---|---|
| "`src/trading/profit_fold.py:14` `apply_profit_fold` - grows one target balance" | The module is removed. The target grows through the tranche applier |
| "`src/trading/profit_fold.py:14` `apply_profit_fold` - grows one dollar target balance; its `target` parameter is annotated `float`" | The same module, named a second time in the count table |
| "`apply_profit_fold(..., fold_target_count=2)` - `TypeError`, unexpected keyword argument" | No call can be made. The refusal it recorded stands for the applier that replaced it |
| "`apply_profit_fold(..., target='x_buy')` - `TypeError`, a word compared against a number" | The same, and the point it made is unchanged: a fold is bounded in dollars |
| "`src/trading/profit_fold.py` records in its own module docstring that no module imports it." | The docstring went with the file. Nothing imported it, which is why it went |

The glossary entry that named the removed module as the fold's entry point is answered in place, in the ADR index and glossary.

## 2026-09-13 - A value pushed after the page is drawn

The dialog keeps its figures in Python and draws them in a web page. Until this section, a figure written after the page was up moved the store and left the screen alone. The page now takes every push.

```
# src/gui/react_settings_dialog.py, the write and the push it starts
def set_value(self, name: str, value: Any) -> None:
    self._model.admit(name, value)
    self.redraw()
```

### Where a push used to stop

React draws each control with a seeded value, which the browser reads once when the node is made. Every ordinary property is redrawn on a later push — whether a row is hidden, whether a button is greyed, the choices in a drop-down — but the figure inside a box is not one of those. The node kept whatever it was born with.

Diagram: flowchart, left to right

- the operator's own edit -> **the store**
- a write from the program -> **the store**
- the store -> **the payload**
- the payload -> **the page redraws**
- the page redraws -> **hidden, greyed, choices**
- the page redraws -> **the figure in the box**

### What each drawn control takes now

Each draw ends by writing the store's figure into every drawn control whose figure differs. A tick box takes a tick, a drop-down takes a row number, and every other control takes its text. Nothing is written where the two already agree, so a redraw that changes nothing touches nothing.

```
// src/gui/web/settings_dialog.js
function draw(target, node) {
  var root = rootFor(target);
  global.ReactDOM.flushSync(function () {
    root.render(node);
  });
  syncDrawn(target);
  return target;
}
```

The dialog draws 66 controls across nine tabs. Sixty-five of them take a seeded figure and are reached by this. The sixty-sixth is the configured exchange list, which is drawn as a list of lines and was always rebuilt.

| Tab | Controls | Tab | Controls |
|---|---|---|---|
| User | 1 | Theme | 2 |
| Exchanges | 6 | Sound | 10 |
| Trading | 3 | SMS | 22 |
| TA Indicators | 12 | AI Monitor | 7 |
| Phantom Bots | 3 | | |

### The credential rows, cleared and read off the page

Adding a venue empties the API Key, API Secret and Passphrase rows. Both figures below were read off the drawn nodes, never off the store.

| Driven on the rendered page | Before | Now |
|---|---|---|
| the key row after the clear | still showed the typed key | empty |
| the secret row after the clear | still showed the typed secret | empty |
| the key row after a tab switch | still showed the typed key | empty |
| Lock duration after a write of 3 | showed 7 | shows 3 |
| the phantom box after a tick | showed clear | shows ticked |
| Default Phantom Timeframes after a write of 1d | showed 4h | shows the row for 1d |

### A push cannot land on a half-typed value

The one control the page leaves alone is the one under the operator's cursor. A figure being typed has not been reported yet, so the store is behind the screen for that box and only for that box. Skipping it is what keeps a push from wiping out half a typed key.

```
// src/gui/web/settings_dialog.js, inside syncDrawn
var typing = owner ? owner.activeElement : null;
if (node === typing || kind === LIST_KIND) {
  return;
}
```

Driven both ways on the same push: with the key row focused and holding a half-typed value, the page kept it; with the same row left, the same push wrote the store's empty figure in. The skip is what saves it.

### An emptied number box is no longer reported

A number box with nothing in it is not a figure. The page used to hand the empty box back as nothing at all, the store refused it, and a warning line was written for a value nobody had entered. The page now puts the held number back in the box and reports nothing.

```
// src/gui/react_settings_dialog.py, the page's own reader
var value = readValue(node, kind);
if (typeof value === "number" && !isFinite(value)) {
  node.value = api.valueOf(name);
  return false;
}
```

### The TA row width the page checked against

The page checks the shape of everything it is handed. Its check on the indicator weight rows wanted three fields and every row carries four, the fourth being the slider's own name. The check reported twelve faults on every load, one for each indicator, and reports none now.

```
// src/gui/web/settings_dialog.js
var TA_ROW_WIDTH = THIRD + STEP;
```

### The six earlier rows this section drove

Every one of these was built on the older behaviour and every one was driven again afterwards.

| Earlier work | Driven after the change |
|---|---|
| the API Key and API Secret masked together | all three credential rows still masked, read off the drawn page |
| the wizard's stored defaults, filled before the draw | the five stored figures still reach the wizard, against the built-in figures with no store |
| Enable Phantom Balance Bots for Scrumming | opens on the stored setting |
| Default Phantom Timeframes | opens on the stored name |
| Lock duration (candles) | opens on the stored figure |
| the wizard's Enable Phantom Bots and Active Timeframes | both open on the stored setting, driven either way |

## 2026-09-13 - The radio drawing code comes off the Settings dialog

The dialog draws no radio button on any tab. The last three went off the page with the sell-side target group, and the machinery that drew them stayed behind: a kind constant, two lookup tables, a holder class, the grouping that cleared a button's siblings, a browser branch and a style rule. None of it had anything left to draw, and all of it is gone.

Driven with the home redirected into a scratch directory before the first product import, so the settings directory and the log root bound under that directory. No stored setting of the running install was read, no credentials file was opened, no bot was built and no venue was contacted. One connection to a venue host was attempted on purpose and refused.

### What drew a radio, and what asked for one

Four files carried the machinery. Zero control specs asked for it.

| What it was | Where it lived |
|---|---|
| the kind constant, and its rows in the signal and painted tables | `src/gui/main_tabs/settings_dialog_surface.py` |
| the holder class and the sibling grouping | `src/gui/react_settings_dialog.py` |
| the input-type branch and the shared group name | `src/gui/web/settings_dialog.js` |
| the style rule the tick box shared with it | `src/gui/web/settings_dialog.css` |

The count either side, over the five files that make the dialog:

| | before | after |
|---|---|---|
| Lines naming a radio | 26 | 0 |
| Occurrences, case-insensitive | 31 | 0 |
| Control specs of that kind | 0 | 0 |

### The dialog either side of the removal

Three surfaces were read in one run, before on a clean copy of the default branch and after on the branch:

| | before | after |
|---|---|---|
| Tabs, in the Qt window and in the shell payload | 9 and 9 | 9 and 9 |
| Control specs the payload carries | 66 | 66 |
| Tick boxes the Qt window draws | 23 | 23 |
| Drop-downs the Qt window draws | 6 | 6 |
| Radio buttons the Qt window draws | 0 | 0 |
| Radio inputs the browser page draws | 0 | 0 |
| Checkbox inputs the browser page draws | 207 | 207 |

The whole payload, the whole Qt reading and the whole rendered-page reading compare identical either side. The 207 is the positive control: the counter that reported zero radio inputs saw 23 tick boxes on each of the nine tab readings, so the zero is a fact about the page rather than about the counter.

### A settings file written while the radio buttons were there

The sell-side group stored one of three words, and the buy-side group stored one of another three. A settings file written before the groups came off still carries all of them.

```
buy side     all_buy    x_buy    most_recent_buy
sell side    all_sell   x_sell   most_recent_sell
```

Driven on a file carrying four top-level keys, one of them a folding group of six:

| | after |
|---|---|
| Anything raised | no |
| Username arrived | yes |
| Default Target Balance arrived | yes, 310.0 |
| The folding group | not declared, passed over |

### The sentence this removal leaves standing

None. The Profit Folding description above names no control and no radio button. Nothing on the dialog draws a radio button, and after this change nothing in the code can.

## 2026-09-13 - The distribution statistic comes off the bot record

One field on a bot's statistics record counted dollars toward a distribution the platform no longer performs. It was declared once and written by nothing, and it is gone.

Driven the same way as the section above, with the home redirected before the first product import. No stored record of the running install was read, no credentials file was opened, no bot was built and no venue was contacted.

### One declaration, no writer, no reader

`src/trading/container/config.py` — the line that went

```
accumulated_distribute: float = 0.0  # Tracks toward extended position
```

The name appeared once in the whole tree, the same count case-sensitive and case-insensitive, and that one appearance was its own declaration. No line set it, no line read it, and no page of this manual named it.

### A saved bot record carrying the retired figure

A save copies the statistics out with `asdict`, so the retired figure was written into every saved record. A restore copies back only the names the record still declares, and passes over the rest.

`src/trading/container/restore.py` — the gate a stored figure meets

```
saved_stats = bot_data.get("stats", {})
for key, val in saved_stats.items():
    if hasattr(bot.stats, key):
        setattr(bot.stats, key, val)
```

Driven on a record shaped like a saved bot, carrying every declared field, the retired figure at 12.5, and one planted name that has never been declared:

| | before | after |
|---|---|---|
| Fields the record declares | 36 | 35 |
| Keys in the stored record | 37 | 37 |
| Stored figures the gate admitted | 36 | 35 |
| Stored figures the gate passed over | 1 | 2 |
| Anything raised | no | no |

The planted name is the positive control. It was passed over on both sides, so the gate can report a skip, and the retired figure moving from admitted to passed over is the whole of the change.

### The directive this removal breaks

Nothing. The name appears in zero lines under `docs/`, so no sentence anywhere describes it. Its own comment named bookkeeping for the distribution mechanics the Profit Folding sections above already record as removed.

Back to the subsystem index.

# The Promotion Pipeline

Reference. A strategy earns its way to real money in four steps. Each step is a gate: it feeds the next one, and the next one refuses what the previous one could not produce.

Diagram: flowchart, left to right

- Market Inspector — scans markets, proposes topologies -> **Simulator — Stone Tablet history**
- Simulator — Stone Tablet history -> **Paper Trader — live feed, fake budget**
- Paper Trader — live feed, fake budget -> **Live — real orders on the venue**
- Market Inspector — scans markets, proposes topologies ..> **Live — real orders on the venue** *(adopt)*

## What each step contributes

| Step | Data source | What it must produce |
|---|---|---|
| Market Inspector | Higher-timeframe OHLCV from the connected venues | A signal table, an opposing-pair table and topology proposals |
| Simulator | Stone Tablets, read-only | The same gates latching on the same candles as live |
| Paper Trader | The live feed, in real time, against a budget that is not real | A trade record the History surface can grade |
| Live | The venue | Filled orders, and the exchange values every ledger reconciles to |

## One logic, three data sources

Live, Paper and the Simulator differ in one thing only: where the candles and the balances come from. The trading logic stays one body of pure code that all three call. Only the stateful shells fork, which keeps a simulated run from touching a live object.

The Simulator's fleet was the worked example. It built real bots against a fake exchange rather than a second bot class, and that fake exchange was a subclass of the same interface the live connector implements.

`FleetSimExchange`, in the Simulator fleet package the rebuild deleted

```
class FleetSimExchange(ExchangeInterface):
    """Real-symbol candle-driven fake exchange for Fleet Replay.
```

The rebuild replaced it. The Sim tab now reads Stone Tablet files through `TabletSource`, which holds no venue and defines no write.

## The dashed edge

One edge skips the two middle steps. The Market Inspector's adopt path creates real bots and draws real wires on the live fleet, which is why the handler opens a confirmation naming the new bot count, the new budget and every wire the adopt would change.

`src/gui/main_window.py` — `_topology_wire_collisions`

```
def _topology_wire_collisions(
    self, wires: list, asset_to_bot: dict
) -> list[dict]:
    """Return one dict per proposal wire whose bot pair is already wired."""
```

The read-only edge into the Simulator was a different hook, and it said so in its own words. It let the sim replay a proposal's shape across sim bots and created nothing. The rebuild removed that hook, so no proposal reaches the Sim tab today.

`set_topology_getter`, in the Simulator tab the rebuild deleted

```
    READ ONLY. This is not the adopt path — adopting a proposal
    creates real bots and wires on the live fleet, while this only
    lets the simulator replay a proposal's shape across sim bots.
    """
```

## Where the chain breaks today

The Paper Trader step has a module now, and issue #19 built it. No step hands its result to the next one: the Simulator hands nothing forward, Paper loads its own fleet from the stored bot record, and Live receives from the Market Inspector's adopt path instead. See paper-trader.md.

Back to the subsystem index.

# Live and Supporting Evidence

- YTD Exchange Data Covering the Entire Development Arc and Exchange Test Coverage

## The Year-to-Date Record and Exchange Test Coverage

Reference. What reads the venue's own trade record, and how far the tests around the exchange integration reach, measured rather than asserted.

Part 9 holds the fill record itself: its row counts, its fills per asset, the VWAP charts, the trade grading and the gate logs. This part covers the readers of that record and the exchange code beneath them.

The record is the operator's own venue export, and this repository holds no copy of it. The tree tracks no spreadsheet file at all, and of every path any commit has ever added, deleted or renamed, two carry a spreadsheet suffix and both are function-complexity inventories.

```
git ls-files "*.csv" "*.xlsx"                   nothing tracked

git log --all --diff-filter=ADR --name-only     every path the history touched
    docs/audits/2026-08-04_function_complexity_inventory.csv
    docs/engineering-notes/2026-08-04_function_complexity_inventory.csv
```

### The venue holds the authority

The reconciliation walk refills a bot's counters from the venue rather than from the bot's own ledger. It walks the venue's own trade call for the bot's symbol in 30-day windows from a fixed anchor, pages a fixed number of rows at a time, and de-duplicates by trade id across the windows. Each unique sell adds quantity times price to a year-to-date scrummed sum; each unique buy adds to a folded sum.

```
YTD_TRADE_ANCHOR_UTC = 1_775_001_600.0      # src/trading/scrumming/reconciliation.py
YTD_TRADE_PAGE_LIMIT = 500
YTD_TRADE_MAX_PAGES = 40

async def sync_ytd_trade_count(self) -> Optional[int]:
    """Walk ``get_my_trades`` in 30-day windows from the YTD anchor."""
```

| Field the walk writes | Value written |
|---|---|
| `stats.total_trades` | `max` of the persisted count, the previous exchange count, and the count just walked |
| `stats.exchange_trade_count` | the same value |
| `stats.ytd_scrummed_usd` | `max` of the persisted sum and the walked sell sum |
| `stats.ytd_folded_usd` | `max` of the persisted sum and the walked buy sum |
| `stats.exchange_data_fresh_ts` | the clock reading at the end of the walk |

Every counter write takes a `max`, so a walk raises a counter toward the venue and never lowers one. Three paths leave the persisted counters alone and answer nothing: a raise inside the walk, an absent exchange, and an exchange that does not serve the trade call. Eight checks drive it against a stubbed exchange — one pins the anchor, three drive those three paths, two pin that the count never falls, and two walk a paged, duplicated window set.

```
def test_anchor_is_2026_04_01_utc(self): ...        # tests/test_ytd_trade_sync.py
def test_single_page_under_limit(self): ...
def test_chunked_window_walk_dedupes_by_id(self): ...
def test_never_lowers_persisted_count(self): ...
def test_does_not_toggle_downward_when_prev_exchange_higher(self): ...
def test_returns_none_when_no_exchange(self): ...
def test_returns_none_when_method_missing(self): ...
def test_returns_none_on_api_exception(self): ...
```

The loop stops after twelve windows. Twelve 30-day windows reach 360 days past the anchor, the furthest forward one sync can carry.

The fleet aggregator sums those per-bot fields across every bot. Its two headline fields carry the year-to-date sum whenever that sum exceeds zero, and fall back to the platform-run accumulator otherwise. Two further fields carry the two figures apart, so a reader who wants one of them can have it unmixed.

```
def get_aggregate_stats(self) -> dict:      # src/trading/container/aggregation.py
    return {
        "total_scrummed_usd": ...,          # year-to-date, else the run accumulator
        "total_folded_usd": ...,            # the same rule
        "total_scrummed_usd_ytd": round(total_scrummed_ytd, 4),
        "total_scrummed_usd_lifetime": round(total_scrummed, 4),
    }
```

## Readers of the record

| Reader | Input | Output |
|---|---|---|
| `sync_ytd_trade_count` (`src/trading/scrumming/reconciliation.py`) | `get_my_trades` pages from the venue | the five `stats` fields above |
| `fetch_all_history_chunked` (`src/exchange/history_helpers.py`) | venue trades for every exchange and symbol a bot manager exposes | one row dict per fill, for the History tab |
| `compare_trades` (`src/trading/stone_tablets/parity_harness.py`) | live fills and Simulator fills | a `ParityReport` of matched, live-only and sim-only trades, paired inside `DEFAULT_TOLERANCE_S` of 300 seconds |
| `ytd_compounding_replay` (`dev_harness/harness/`) | the operator's export, read offline | an upper bound on what the surplus-drain formula would have added to each bot's target |

The Fleet Replay panel imports the parity comparison and records every call under a feature name. The telemetry behind that name counts calls, skips and exceptions per feature, and flags by name any feature that was never called.

```
TELEMETRY_PARITY = "sim.parity.compare_trades"      # src/gui/main_tabs/fleet_replay_panel_surface.py
The Simulator rebuild removed this file; it is not in the tree.

class FeatureCounter: ...       # src/core/feature_telemetry.py
class FeatureTelemetry: ...
def get_telemetry() -> FeatureTelemetry: ...
```

## The connectors

Three classes subclass `ExchangeInterface`. The interface declares seventeen members, sixteen of them abstract. The seventeenth is the venue trade call, and it carries a body that raises until a subclass overrides it.

```
class ExchangeInterface(ABC):       # src/exchange/base.py
    async def get_my_trades(...):   # the one non-abstract member, raises NotImplementedError
```

| Implementation | Module | Reaches |
|---|---|---|
| `CCXTConnector` | `src/exchange/ccxt_connector.py` | any venue id in `SUPPORTED_EXCHANGES`, through ccxt; or a backend passed to `attach_backend` |
| `FleetSimExchange` | `src/simulator/fleet/sim_exchange.py` | stored `CandleSeries` rows over real symbols; no venue |
| `NuclearSimExchange` | `src/simulator/nuclear_sim_exchange.py` | synthetic `TAPEA`/`TAPEB` tapes; no venue |

The Simulator rebuild removed the files above; they are not in the tree.

One connector method names the fifteen ccxt members a backend must serve, marks the connector connected and drops the rate-limit interval to zero. The Stone Tablet backend is such a backend: it answers those members from tablet rows instead of the network, and the replay controller attaches it to a real connector, so a replay runs the connector's own normalisation and fee code.

```
def attach_backend(self, backend: Any) -> None:     # src/exchange/ccxt_connector.py
    """Serve every ccxt call from *backend* and mark the connector connected.

    *backend* must provide the fifteen ccxt members reached through ``_ex``:
    fetch_ohlcv, fetch_ticker, fetch_tickers, fetch_balance, create_order,
    cancel_order, fetch_order, fetch_open_orders, fetch_my_trades,
    fetch_order_book, markets, market, precisionMode, amount_to_precision
    """

class TabletBackend: ...        # src/exchange/tablet_backend.py
```

The venue registry carries its own status sets, all in `ccxt_connector.py`.

| Set | Count |
|---|---|
| `SUPPORTED_EXCHANGES` | 15 |
| `PREFLIGHT_URLS` | 15 |
| `PASSPHRASE_EXCHANGES` | 3 |
| `US_IP_BLOCKED_EXCHANGES` | 2 |

| Set | Count |
|---|---|
| US_ACCOUNT_RESTRICTED_EXCHANGES | 2 |
| VERIFIED_EXCHANGES | 1 |

One function turns those sets into the picker text a user sees. A blocked venue is labelled as blocked, any other unverified venue is labelled untested, and a venue needing a passphrase says so. Driven over the whole registry, fourteen of the fifteen labels carry a caveat and one does not.

```
def exchange_label(exchange_id: str) -> str:        # src/exchange/ccxt_connector.py
    """Return the picker label for *exchange_id*, carrying its status notes."""
    label = exchange_id.capitalize()
    notes: list[str] = []
    if exchange_id in US_IP_BLOCKED_EXCHANGES:
        notes.append("blocked from US")
    elif exchange_id not in VERIFIED_EXCHANGES:
        notes.append("untested")
    if exchange_id in PASSPHRASE_EXCHANGES:
        notes.append("passphrase required")
    if notes:
        label += f" ({', '.join(notes)})"
    return label
```

Fifteen checks cover those sets. One resolves every registry id to an importable ccxt class through the connector's own resolver. One requires an https pre-flight URL for every supported id. One pins the verified set to a single member. One reads the label of every registry id against the rule above.

```
def resolve_ccxt_class(...): ...        # src/exchange/ccxt_connector.py
                                        # tests/test_exchange_registry.py, fifteen checks
```

The exchange package holds 23 modules beside its package initialiser. Twenty sit inside the static import closure of the entry point, which reaches 213 first-party modules and leaves 130 source modules outside it.

```
src/exchange/       23 modules, 20 inside the entry point's import closure
    api_docs            outside
    history_surface     outside
    market_data         outside
```

## What the exchange tests exercise

| Measurement | Count |
|---|---|
| test files under `tests/` | 535 |
| of those, naming `src.exchange` | 70 |
| `test_*` callables in those 70 files | 2,080 |

Sixteen of the seventeen interface members appear in at least one of those 70 files. The seventeenth, the asset logo lookup, appears in none.

```
member                  files naming it
    exchange_id             28
    connect                 22
    place_order              9
    get_ticker               7
    get_asset_logo_url       0
```

The same 70 files reach some modules far more than others, and leave five of the 23 with no test importing them at all.

```
module                  files importing it
    ccxt_connector          20
    base                    14
    currency_rate_monitor    7
    history_helpers          7
    history_read_contract    7
    tablet_backend           7

    api_docs                 0
    chart_data               0
    crypto_assets            0
    idempotency              0
    position_health          0
```

## How a test stands in for a venue

No test opens a socket to an exchange and none carries a credential. Each check measures the shape of a call and the code around it, never a venue's answer. Five stand-ins do that work.

| Stand-in | Where | Replaces |
|---|---|---|
| a fake `ccxt` module put into `sys.modules` | 2 of the 70 files | the venue library, sync and async support both |
| an emptied `PREFLIGHT_URLS` | `tests/test_connect_does_not_block_calling_thread.py` | the pre-flight HTTP call inside `sync_connect` |
| a `socket.socket.connect` that raises | 19 test files | the wire itself, for the whole file |
| `tests/fixtures/venue_precision_metadata.json` | `tests/test_venue_precision_is_decimal_places.py` | published market metadata for 13 venues, captured from a public `load_markets` with no credential |
| `TabletBackend` | `src/exchange/tablet_backend.py` | the ccxt surface beneath a real connector |

The fixture covers 13 of the 15 registry ids. The two ids missing from it are the pair in `US_IP_BLOCKED_EXCHANGES`, which refused the address the capture ran from.

The credential file name appears in the test tree three times, across two modules. Each of the three is a check that a tool raises rather than opening such a file, and each builds its target under a temporary path. One of them, the parametrised refusal over the runtime directories, sits beside a positive control that lets a path outside them through.

```
coinbase_credentials    3 occurrences under tests/
    tests/test_capture_live_baseline.py
    tests/test_migration_verifier.py
```

## The four refusals around an exchange call

| Refusal | Where | What it turns away |
|---|---|---|
| live-tree guard | `tests/conftest.py` | a suite run that created a path under `~/.acervator` or `~/.acervator_logs`, or changed anything under the Stone Tablet archive |
| `safe_urlopen` and `SafeRequest` | `src/core/safe_url.py` | a URL whose scheme is outside http and https, by policy at construction and at open, and by an opener carrying no file, ftp or data handler |
| `_redact` | `src/exchange/api_logger.py` | a param whose key holds any of nine credential substrings, swapping a mask in before `record` stores the entry |
| `guarded_place_order` | `src/trading/bot_container.py` | an order whose amount is not a finite positive number, at the single point every engine order passes through |

Each of the four carries checks that drive it to the refusal itself, so a green run says the refusal still fires.

- `_live_roots` is injectable, so `tests/test_live_tree_guard.py` drives the guard against temporary roots across 22 checks. Two of them replay the two isolation breaches this project has shipped: a telemetry file created in the live tree, and a reservation-state autosave.
- `tests/test_safe_url_scheme_policy.py` holds 40 checks, splitting the policy half from the transport half because either can fail alone.
- The redaction checks name 13 credential field spellings that must come back masked, five benign fields that must come back unchanged as the positive control, and read the emitted log line for any param value — with a control proving the same handler sees a value placed in a benign field. One substring covers two credential spellings at once.
- Two modules call the venue's order method directly: the bot container, inside the guard itself, and the volume guard the container dispatches to. Ten sites across three trading modules call the guard, and no other route out exists. Five checks cover it: one drives unusable amount shapes into a recorder standing where the exchange stands, one drives a real amount through as the positive control, and three more pin the text of the refusal and which check turns an undersized order back.

The modules on each side of the last two, and the one substring that covers two spellings:

```
tests/test_api_logger_redaction.py      "sign" masks signature and CB-ACCESS-SIGN
                                        the control places a value in reason

exchange.place_order called by      src/trading/bot_container.py
                                    src/trading/volume_guard.py
guarded_place_order called by       src/trading/scrumming/execution.py
                                    src/trading/scrumming_bot.py
                                    src/trading/extractor_bot.py
                                    ten sites, no other route out
tests/test_u6_venue_amount_gate.py      five checks
```

## When the order guard could not read the market's limits

A venue decides the smallest order it will take. The bot reads that figure off the market before it sends anything, and turns an order away that falls under it. The reading can fail — the venue can be down, or the market can be missing from the list it returns — and what the guard did with a failed reading was the defect.

A failed reading used to answer with a minimum of zero. Nothing is ever under zero, so the check that turns small orders away could not turn anything away, and the order went out unchecked. Nothing was said on any screen. The failed reading was also remembered, so one moment of trouble at the venue left that market unchecked until the bot was restarted.

`src/exchange/base.py` — what a market reports now

```
@dataclass(frozen=True)
class MarketRules:
    min_amount: Optional[float] = None       # base units
    min_cost: Optional[float] = None         # quote units
    amount_increment: Optional[float] = None # base units a size steps by
    read: bool = True
```

Nothing in that record stands in for a figure the venue did not give. A minimum that was never read is held as nothing at all, and nothing is not a number the check can be satisfied by. The `read` field separates the two cases that used to look the same: a venue that published no minimum, and a reading that never happened.

Three things now follow a failed reading. The check makes no comparison, because it has nothing to compare against and says so rather than pretending. A line goes to the Console under the bot's own name. The failure is not remembered, so the next order reads the market again and the check comes back as soon as the venue does.

```
LIMITS NOT READ: TONE/USD market record could not be obtained, so no minimum
size or cost is known for this SELL of 20.0000000000. The venue enforces its
own; Acervator checks nothing here.
```

### What a bot does differently

Nothing changes while the reading succeeds, which is every ordinary moment. The same sizes go to the venue and the same undersized orders are turned away. Read over four order sizes on two real Coinbase markets, every size and every refusal is identical before and after.

What changes is the bad moment. Where a reading failed, an order under the market's minimum used to go out silently and the bot stayed blind for the rest of its run. Now it still goes out — the venue applies its own minimum and refuses it there — but the Console says the check did not run, and the very next order after the venue recovers is checked again.

### The sentence this overtakes

The refusal table above carries this row, and it is kept as written:

> | `guarded_place_order` | `src/trading/bot_container.py` | *an order whose amount is not a finite positive number, at the single point every engine order passes through* |

That row is narrower than what the method does. The true sentence is: an order whose amount is not a finite positive number, an order under the market's published minimum size, or an order under its published minimum cost — at the single point every engine order passes through, with each refusal put on the Console, and with an order the guard could not check named there too.

### The broker side has no such step

The crypto side reads a market's limits and checks an order against them. The equities side does not. Measured across both broker files, no minimum size, no minimum value, no step size and no fractional flag is read anywhere.

```
src/stocks/broker_base.py         0 occurrences
src/stocks/alpaca_connector.py    0 occurrences
  searched for: min_amount, min_qty, min_notional, increment,
                precision, fractionable
```

Nothing on any screen builds a bot on that path today, so this is recorded and not repaired here.

## The simulation battery

The venue record above is live money. Beside it sits a second body of evidence that is not live at all: a battery of simulated runs, measured on an earlier release, which is where the platform's headline win rate comes from. It is set down here as the record of that run, with what this repository can and cannot show about it stated beside it.

The battery covered 738 simulations, spanning 27 portfolio configurations, capital from $400 up to $100K, six two-year historical periods — including the 2020 COVID crash, the 2021 ATH bubble and the 2022 brutal bear — and the venue's fee tiers from VIP-0 through VIP-3. Parameters were identical throughout, with no optimization per asset.

| Measurement | Result |
|---|---|
| Win rate | 715 of 738, 96.9% |
| Total advantage against buying and holding | +$78.0B |
| Canonical 39-sim battery | 39 of 39, 100% |
| Historical 2020 to 2022, 39 sims | 39 of 39, 100%, +$3.16M total advantage |
| Capital scaling, 156 sims | 156 of 156, 100%, superlinear at $100K |
| Bear regime stability, 9 sims | $4,262 to $4,427 total, 4% CV |

| Measurement | Result |
|---|---|
| Profitable fee ceiling | up to 0.50% roundtrip |

The 23 runs that are not wins are concentrated in extreme single-asset bear conditions, with 2022-class drawdowns of 64% to 94%, at the smallest capital levels. Losses in those cases are marginal: $10 to $157 on $400 deployments. The record makes no claim to win in a catastrophic single-asset collapse.

Six competing strategies were run over the same canonical 39 cases.

| Strategy compared against | Battery wins |
|---|---|
| Passive hold | 39 of 39, 100% |
| DCA weekly | 39 of 39, 100% |
| DCA daily | 39 of 39, 100% |
| Grid trading | 39 of 39, 100% |
| SMA 50/200 cross | 37 of 39, 94.9% |
| RSI mean-reversion | 35 of 39, 89.7% |

**This repository holds nothing behind those numbers.** No file in the tree carries the per-run rows, and no commit on any branch ever added one. The figures above are a record, not a measurement anything here reproduces.

**The Simulator tab did not produce them and could not.** It was not built when the table was recorded, and issue #117 has since rebuilt it. Its Portfolio Battery walks the RA-StoneTablets rather than the runs above, so nothing a reader can run in this product re-derives the table.

The queries, and the control that proves they can find a file:

```
git ls-files                                            no battery file in the tree
git log --all --diff-filter=ADR --name-only             no commit ever added one

git log --all --diff-filter=ADR --name-only -- generate_essay_ja.py
                                                        one commit, so the walk works
```

The third query is the control. The same walk over every branch does find a file that once existed and is gone, so the empty answer above is a fact about the battery and not about the query.

## Where a figure and a run carry less than they read

Two current-state findings bear on how a reader should take the numbers above.

**The header counters and their tooltips name different figures.** The header cells read the two headline fields out of the fleet snapshot, and that snapshot fills both from the year-to-date sum whenever the sum exceeds zero. The tooltips beside them describe a cumulative total “since the platform run started” that “resets to $0.00 only on a fresh process start” — the lifetime accumulator, which the same snapshot carries separately and neither cell reads. The card and the tooltip describe two different figures.

```
src/gui/main_tabs/header_strip_surface.py   counter_cells reads
                                                total_scrummed_usd
                                                total_folded_usd
src/gui/main_tabs/header_strip.py           the tooltips describe
                                                total_scrummed_usd_lifetime
```

**A Simulator run exercises an injected capital registry, not the live one.** The bot constructor takes a capital registry parameter. The two Simulator construction sites pass one; the three live sites pass none, and the reservation mixin then falls back to the process-wide registry. The sell path refuses an amount above the effective available quantity, but that pre-check sits inside a `try` whose handler logs at debug level and continues, so a raise there lets the sell proceed. A Simulator run says nothing about which registry a live bot resolves, and nothing about what follows when that resolution raises.

```
ScrummingBot.__init__(..., capital_registry=None)   src/trading/scrumming_bot.py

passes one      src/simulator/fleet/fleet_replay_controller.py
                src/simulator/nuclear_controller.py
The Simulator rebuild removed the files above; they are not in the tree.

passes none     src/gui/main_window.py
                src/gui/live_bot_window.py
                src/trading/container/restore.py

falls back      _crr in src/trading/scrumming/capital_reservation_mixin.py
the pre-check   effective_available, inside a try in
                src/trading/scrumming/execution.py
```

# Venue Compatibility

Reference. Every venue this platform can reach through an API, set beside the asset classes it serves, the shape its orders take, and whether an accumulation bot can trade on it unchanged.

A scrum sells the excess above a dollar target. That excess is an arbitrary fraction of the held asset, so one question decides each venue: does the fraction survive the venue's own size rule.

The index is README.md. The connectors behind the crypto path are described in 13-live-evidence.md, the screen that adds a venue is in 08-tabs/settings.md, and the per-class fractional reading the simulator works from is in 08-tabs/simulator.md.

No venue was contacted to write this page. Every venue fact below was read from the publisher's own page outside this repository and handed to the page on 2026-09-24. Every tree fact carries its file and line.

## What a scrum asks of a venue

A scrum computes a size and submits it. Two steps stand between that size and the venue: one truncates it to the venue's decimal places and refuses a size under the venue's minimum, and one rounds it again inside the crypto connector.

OVERTAKEN, and the sentence above is kept as written. Three steps stand between that size and the venue. The order gate floors the size onto the venue's own published `amount_increment`, it refuses a size that steps under the venue's `min_amount` or that floors to nothing, and the crypto connector rounds the result again. The flooring runs through `sized_order`, the one function the Simulator and the Paper Trader size with.

```
# src/trading/bot_container.py:260
        _sized = sized_order(
            _amt, unit_rule(CLASS_CRYPTO, self.config.exchange_id), _rules
        )

# src/trading/bot_container.py:283
        if 0.0 < _sized.units < _amt:

# src/exchange/ccxt_connector.py:1128
        amount = self._ex.amount_to_precision(symbol, amount)
```

Two citations in the block below quote lines `src/trading/bot_container.py` no longer holds at any line: `if _min_amount > 0 and _below_min:` and the docstring naming `(min_amount, min_cost, amount_precision)`. The block above carries the lines the file holds today.

```
# src/trading/scrumming/execution.py:1581
                amount=amount,

# src/trading/bot_container.py:226
        if _min_amount > 0 and _below_min:

# src/exchange/ccxt_connector.py:1061
        amount = self._ex.amount_to_precision(symbol, amount)
```

The minimum and the decimal places come off the market record the connector reads, held per symbol. A lookup that fails gives no minimum and eight decimal places, so the refusal stands down.

```
# src/trading/bot_container.py:116
        """Return ``(min_amount, min_cost, amount_precision)`` for
        ``symbol``, cached; ``(0.0, 0.0, 8)`` on any lookup failure."""
```

The broker path carries neither step. It names the size `quantity` and sends it unchanged.

```
# src/stocks/broker_base.py:145
    async def place_order(
        self,
        symbol: str,
        side: OrderSide,
        quantity: float,

# src/stocks/alpaca_connector.py:176
            "qty": str(quantity),
```

Nothing in the running program builds that path. Outside its own module and the stock window's own files, the broker connector is named once, in a docstring.

```
# src/simulator/portfolios.py:26
"""The ``SUPPORTED_BROKERS`` key of the one broker connector, ``AlpacaConnector``."""
```

## The venues and the classes they serve

Fifteen crypto venues are offered. One has ever traded. The equity venue list holds nine ids for eight firms, and two of those firms have no API to reach.

| Venue | Classes served | Reachable from the United States | Credential shape | Source |
|---|---|---|---|---|
| coinbase | crypto spot; US futures products over the same API | no refusal recorded, 2026-08-28 | key and secret | `src/exchange/ccxt_connector.py:67`; Coinbase developer documentation, Advanced Trade US derivatives |
| kraken | crypto spot | no refusal recorded, 2026-08-28 | key and secret | `src/exchange/ccxt_connector.py:67` |
| gateio | crypto spot | no refusal recorded, 2026-08-28 | key and secret | `src/exchange/ccxt_connector.py:67` |
| mexc | crypto spot | no refusal recorded, 2026-08-28 | key and secret | `src/exchange/ccxt_connector.py:67` |
| bitfinex | crypto spot | no refusal recorded, 2026-08-28 | key and secret | `src/exchange/ccxt_connector.py:67` |
| gemini | crypto spot | no refusal recorded, 2026-08-28 | key and secret | `src/exchange/ccxt_connector.py:67` |
| bitstamp | crypto spot | no refusal recorded, 2026-08-28 | key and secret | `src/exchange/ccxt_connector.py:67` |
| cryptocom | crypto spot | no refusal recorded, 2026-08-28 | key and secret | `src/exchange/ccxt_connector.py:67` |
| kucoin | crypto spot | no refusal recorded, 2026-08-28 | key, secret and passphrase | `src/exchange/ccxt_connector.py:102` |
| okx | crypto spot | no refusal recorded, 2026-08-28 | key, secret and passphrase | `src/exchange/ccxt_connector.py:102` |
| bitget | crypto spot | no refusal recorded, 2026-08-28 | key, secret and passphrase | `src/exchange/ccxt_connector.py:102` |
| binance | crypto spot | no, public endpoints refused a US address, 2026-08-28 | key and secret | `src/exchange/ccxt_connector.py:113` |

| Venue | Classes served | Reachable from the United States | Credential shape | Source |
|---|---|---|---|---|
| bybit | crypto spot | no, public endpoints refused a US address, 2026-08-28 | key and secret | `src/exchange/ccxt_connector.py:113` |
| poloniex | crypto spot | no, the terms refuse a US account, 2026-08-28 | key and secret | `src/exchange/ccxt_connector.py:119` |
| huobi | crypto spot | no, the terms refuse a US account, 2026-08-28 | key and secret | `src/exchange/ccxt_connector.py:119` |
| Alpaca | equities and exchange-traded funds, which is every commodity row this tree carries | yes, US broker | key and secret | `src/gui/main_tabs/asset_class_surface.py:33`; Alpaca fractional trading documentation, read 2026-09-24 |
| Interactive Brokers | equities, funds, futures and 100 or more currency pairs | yes, US broker | a TWS session, shape not stated by the pages read | `src/gui/main_tabs/asset_class_surface.py:33`; Interactive Brokers TWS API documentation, read 2026-09-24 |
| Schwab | equities and options, individual accounts | yes, US broker | issued on the Schwab developer portal, shape not stated by the pages read | `src/gui/main_tabs/asset_class_surface.py:33`; Schwab Trader API developer portal, read 2026-09-24 |
| tastytrade | equities, options, futures and crypto, through one API | yes, US broker | not stated by the pages read | `src/gui/main_tabs/asset_class_surface.py:33`; tastytrade API documentation, read 2026-09-24 |
| E*TRADE | equities | yes, US broker | not stated by the pages read | `src/gui/main_tabs/asset_class_surface.py:33`; E*TRADE developer documentation, read 2026-09-24 |
| Webull | equities | yes, US broker | not stated by the pages read | `src/gui/main_tabs/asset_class_surface.py:33`; the fractional-support table TradersPost publishes for the brokers it connects, read 2026-09-24 |
| TD Ameritrade | none reachable | no, the API was discontinued on 10 May 2024 and registrations did not carry over | none | `src/gui/main_tabs/asset_class_surface.py:33`; Charles Schwab notice of the discontinuation, read 2026-09-24 |
| Fidelity | none reachable | no retail trading API is published | none | `src/gui/main_tabs/asset_class_surface.py:33`; Fidelity, read 2026-09-24 |

| Venue | Classes served | Reachable from the United States | Credential shape | Source |
|---|---|---|---|---|
| OANDA | forex | yes, CFTC-registered and NFA-regulated | a v20 REST token, shape not stated by the pages read | OANDA v20 REST API documentation, read 2026-09-24 |
| FOREX.com | forex | yes, CFTC-registered and NFA-regulated | not stated by the pages read | FOREX.com, a StoneX brand, read 2026-09-24 |
| tastyfx | forex | yes, CFTC-registered and NFA-regulated | not stated by the pages read | tastyfx, the US brand of IG, read 2026-09-24 |
| Tradovate | commodity and index futures | yes, US broker | not stated by the pages read | Tradovate, read 2026-09-24 |
| NinjaTrader | commodity and index futures, with direct access to CME Group, Eurex, ICE and Coinbase Derivatives | yes, US broker | not stated by the pages read | NinjaTrader Clearing, read 2026-09-24 |

Two things in the table come from the tree rather than from a venue. The nine equity ids name eight firms, because one firm carries two.

```
# src/gui/main_tabs/asset_class_surface.py:33
EQUITY_VENUES = frozenset(
    {
        "alpaca",
        "ibkr",
        "schwab",
        "tdameritrade",
        "webull",
        "tastytrade",
        "fidelity",
        "etrade",
        "interactivebrokers",
    }
)
```

Every commodity row this tree carries is a US-listed fund share, so an equity broker serves that class. The four spot metal rows name no venue at all, and the fund rows name a price source rather than a place to send an order.

```
# src/trading/ata_asset_maps.py:437
METALS_SPOT: tuple[AssetListing, ...] = tuple(
    AssetListing(symbol=one, quote=USD, form=FORM_SPOT)
    for one in ("XAU/USD", "XAG/USD", "XPT/USD", "XPD/USD")
)

# src/trading/ata_asset_maps.py:444
METALS_PHYSICAL: tuple[AssetListing, ...] = tuple(
    AssetListing(symbol=one, quote=USD, venue=VENUE_YAHOO, ticker=one, form=FORM_ETF)
    for one in ("GLD", "SLV", "PPLT", "PALL")
)
```

## How an order names its size

A venue names an order's size in one of three ways: units of the base asset, a share count, or a whole contract. The smallest step and the minimum are the two figures that decide whether a scrum can run.

| Venue | How the size is named | Smallest step | Minimum | Source |
|---|---|---|---|---|
| coinbase, spot | units of the base currency | the product's own increment field | the product's own minimum size field | Coinbase Create a new order and Get Product, quoted with their read date in 08-tabs/simulator.md |
| coinbase, US futures | not stated by the pages read | not stated by the pages read | not stated by the pages read | Coinbase developer documentation, Advanced Trade US derivatives, read 2026-09-24 |
| the other fourteen crypto venues | units of the base currency | published per product by each venue, not stated by the pages read | published per product by each venue, not stated by the pages read | each venue's own product record |
| Alpaca | either a share quantity or a dollar amount, never both in one order | nine decimal places of a share | 1.00 USD on a buy | Alpaca fractional trading documentation, read 2026-09-24 |
| Interactive Brokers | a share or contract quantity | not stated by the pages read; a minimum price increment is published per contract | not stated by the pages read; an oversized order is refused with error 201 | Interactive Brokers TWS API order limitations and minimum increment pages, read 2026-09-24 |
| Schwab | not stated by the pages read | not stated by the pages read | not stated by the pages read | Schwab Trader API developer portal, read 2026-09-24 |
| tastytrade | not stated by the pages read | not stated by the pages read | not stated by the pages read | tastytrade API documentation, read 2026-09-24 |
| E*TRADE | not stated by the pages read | not stated by the pages read | not stated by the pages read | E*TRADE developer documentation, read 2026-09-24 |
| Webull | a share quantity | two decimal places of a share | not stated by the pages read | the fractional-support table TradersPost publishes for the brokers it connects, read 2026-09-24 |

| Venue | How the size is named | Smallest step | Minimum | Source |
|---|---|---|---|---|
| OANDA | units of the base currency | one unit, in place of a thousand-unit micro lot | one unit | OANDA v20 REST API documentation, read 2026-09-24 |
| FOREX.com, tastyfx | not stated by the pages read | not stated by the pages read | not stated by the pages read | each firm's own documentation, read 2026-09-24 |
| Tradovate, NinjaTrader | whole contracts | the contract specification of the product, not stated by the pages read | the contract specification of the product, not stated by the pages read | each product's contract specification |

A cell reading "not stated by the pages read" is a gap, not a zero. What closes each one is named beside it: the venue's product record for a crypto step, the firm's order specification page for a broker, and the exchange's contract specification for a futures product.

## Whether a scrum's excess survives the size rule

A scrum's excess is an arbitrary fraction. A venue whose smallest step is coarser than that fraction cannot carry the strategy unchanged.

Two questions decide a venue, and every row answers both with a yes or a no. The first is whether an order can be placed over an API at all. The second is whether the excess survives the venue's size rule. Readings taken 2026-09-24 from each venue's own published documentation, and from the fractional-support table TradersPost publishes for the brokers it connects.

| Venue | Orders over an API | Does the excess survive | Why, and where the reading came from |
|---|---|---|---|
| coinbase, spot | yes | yes | nine decimal places, and the size is truncated to the product's decimal places and refused under the product's minimum before the API is called, at `src/trading/bot_container.py:264`. Coinbase developer documentation, read 2026-09-24 |
| coinbase, US futures | yes | no | a futures contract is whole, so an excess under one contract cannot be sold. Coinbase developer documentation, Advanced Trade US derivatives, read 2026-09-24 |
| kraken, kucoin, okx, gateio, bitget, mexc, bitfinex, gemini, bitstamp, cryptocom | yes | yes | nine decimal places. Each venue's own documentation, read 2026-09-24 |
| binance, bybit | yes, and the address is refused from the United States | yes | nine decimal places. Each venue's own documentation, read 2026-09-24. The refusal is a tree reading dated 2026-08-28, at `src/exchange/ccxt_connector.py:113` |
| poloniex, huobi | yes, and a United States account is restricted | yes | nine decimal places. Each venue's own documentation, read 2026-09-24. The restriction is a tree reading dated 2026-08-28, at `src/exchange/ccxt_connector.py:119` |

| Venue | Orders over an API | Does the excess survive | Why, and where the reading came from |
|---|---|---|---|
| Alpaca | yes | yes | nine decimal places on a share quantity. The 1.00 USD floor is published for a buy, and the tree sends a share quantity and never a dollar amount, at `src/stocks/alpaca_connector.py:176`. Alpaca fractional trading documentation, read 2026-09-24 |
| Webull | yes | yes | two decimal places on a share quantity, which is finer than one dollar on any share this fleet holds. The fractional-support table TradersPost publishes, read 2026-09-24 |
| Interactive Brokers | yes | no | a quantity order rounds down to whole shares, and a fraction is reached only by naming a cash amount. Interactive Brokers TWS API documentation, read 2026-09-24 |
| Schwab | yes | no | the Trader API places no fractional order at all. Schwab Trader API developer portal, read 2026-09-24 |
| tastytrade | yes | no | a fractional quantity is not carried for automated trading. tastytrade API documentation, read 2026-09-24 |
| E*TRADE | yes | no | it sells no fractional share of an individual stock. E*TRADE developer documentation, read 2026-09-24 |
| TD Ameritrade | no | no | the API closed on 10 May 2024. Charles Schwab notice of the discontinuation, read 2026-09-24 |
| Fidelity | no | no | it publishes no trading API. Fidelity, read 2026-09-24 |
| OANDA, FOREX.com, tastyfx | yes | no | they size in whole currency units, so an excess under one unit cannot be sold and an excess above it is cut to whole units. Each firm's own documentation, read 2026-09-24 |
| Tradovate, NinjaTrader | yes | no | a futures contract is whole. Each product's contract specification, read 2026-09-24 |

Two limits apply to every row of that column. The truncation and the minimum refusal run only on the crypto path, so a broker order reaches the venue unsized. And a market lookup that fails leaves the bot with no minimum and eight assumed decimal places, which stands the refusal down for that symbol.

```
# src/trading/bot_container.py:130
            fallback = (0.0, 0.0, 8)
            self._market_limits_cache[symbol] = fallback
```

### 2026-09-24 - the failed-lookup half of that paragraph is repaired

The sentence above is kept as written:

> *And a market lookup that fails leaves the bot with no minimum and*
>
> *eight assumed decimal places, which stands the refusal down for that symbol.*

That is what the code did. The true sentence is: a market lookup that fails leaves the bot with no minimum, which it now holds as unknown rather than as zero, so the refusal makes no comparison instead of silently passing every size; the bot says so on the Console, and the failure is no longer remembered, so the next order reads the market again.

The block quoted above is replaced by the block below. The coinbase, spot row in the table still holds: while the lookup succeeds, the truncation and the refusal are unchanged.

`src/trading/bot_container.py` — what a failed lookup answers now

```
        unread = MarketRules(read=False)
        ...
            # A failure is not cached: caching it left the guard blind for the
            # container's life after one transient error.
            return unread
```

The record that replaced the tuple, and the comparison the guard now calls, are described under 13-live-evidence.md.

## Two variants cover every venue

The verdict column above falls into two groups and no more. A venue that reads yes needs nothing new. Every venue that reads no is served by one second shape, not by a shape of its own.

The first variant is the bot that runs today. It names a unit count, which is the share quantity the broker connector already sends, quoted higher up this page. It trades crypto and Alpaca unchanged.

The second variant names a cash amount instead of a unit count. Interactive Brokers reaches a fraction that way. Alpaca carries the same field beside its quantity. Schwab, tastytrade and E*TRADE take the whole-share order that shape produces. One variant covers all five.

```
# PROPOSED, not present. The broker contract names a quantity alone.
    async def place_order(
        self,
        symbol: str,
        side: OrderSide,
        quantity: float | None = None,
        notional_usd: float | None = None,
        order_type: OrderType = OrderType.MARKET,
    ) -> StockOrder:
```

Futures sit under the second variant and carry one further condition. A position small enough that its excess is under one contract cannot be scrummed at all, which is a reading each market answers for itself.

## Cells on this page that the 2026-09-24 readings overtake

The size table above was written from what each venue's own size pages state. The fractional readings taken on 2026-09-24 close several of its gaps. Each earlier cell is quoted whole, and the reading that overtakes it sits beneath.

The four broker rows of the size table:

```
| Interactive Brokers | a share or contract quantity | not stated by the pages read; a minimum price increment is published per
contract | not stated by the pages read; an oversized order is refused with error 201 | Interactive Brokers TWS API order
limitations and minimum increment pages, read 2026-09-24 |
| Schwab | not stated by the pages read | not stated by the pages read | not stated by the pages read | Schwab Trader API developer
portal, read 2026-09-24 |
| tastytrade | not stated by the pages read | not stated by the pages read | not stated by the pages read | tastytrade API
documentation, read 2026-09-24 |
| E\*TRADE, Tradier, TradeStation | not stated by the pages read | not stated by the pages read | not stated by the pages read |
each firm's developer documentation, read 2026-09-24 |
```

Overtaken on 2026-09-24. Each of those brokers steps in whole shares. Interactive Brokers rounds a quantity order down and reaches a fraction only through a cash amount. Schwab places no fractional order over the Trader API. tastytrade does not carry a fractional quantity for automated trading. E*TRADE sells no fractional share of an individual stock.

The futures and currency rows of the size table:

```
| coinbase, US futures | not stated by the pages read | not stated by the pages read | not stated by the pages read | Coinbase
developer documentation, Advanced Trade US derivatives, read 2026-09-24 |
| FOREX.com, tastyfx | not stated by the pages read | not stated by the pages read | not stated by the pages read | each firm's own
documentation, read 2026-09-24 |
| Tradovate, NinjaTrader | whole contracts | the contract specification of the product, not stated by the pages read | the contract
specification of the product, not stated by the pages read | each product's contract specification |
```

Overtaken on 2026-09-24. FOREX.com and tastyfx size in whole currency units, as OANDA does. A futures contract is whole, at Tradovate, at NinjaTrader and on Coinbase's US derivatives products, so one contract is the step.

The paragraph that follows the size table:

```
A cell reading "not stated by the pages read" is a gap, not a zero. What closes
each one is named beside it: the venue's product record for a crypto step, the
firm's order specification page for a broker, and the exchange's contract
specification for a futures product.
```

Overtaken on 2026-09-24 for the broker, the currency and the futures rows. The crypto steps are the gaps that remain, and each venue's own product record still closes one.

The row that recorded the last venue this page could not describe:

```
One id in the equity venue list has no reading behind it.

| webull | the id sits in `EQUITY_VENUES` and no page about it was read | Webull's own developer documentation: whether it publishes
a trading API, its order fields, its smallest share step and its minimum |
```

Overtaken on 2026-09-24. Webull publishes a trading API, and its share quantity carries two decimal places, so it sits in the tables above. Every id in the equity venue list is now described.

Tradier and TradeStation are not on this page. No reading of 2026-09-24 states whether either carries a fractional share.

The refusal quoted at the top of this page, cited there to line 226 of `src/trading/bot_container.py`:

```
        if _min_amount > 0 and _below_min:
```

Overtaken on 2026-09-24. The guard now reads a market record rather than three loose numbers, and the refusal it makes sits at line 236 of the same file. The verdict table above cites the new line.

## A market answers the same question for itself

The verdict column above is per venue and was read from published pages. A market answers the same question from the figures its own record carries, and the Market Inspector's ticker rows draw that answer.

```
def tradeable_answer(
    rules: Any,
    price: Optional[float] = None,
    excess_usd: float = REFERENCE_SCRUM_EXCESS_USD,
) -> str:
```

### One rule, three answers

A market can be traded by the built variant when the smallest order the venue accepts costs no more than the excess a scrum submits. The smallest order is the venue's published minimum size, ceiled onto its own size increment, valued at the venue's own last price, and never under its minimum order cost.

| answer | drawn on the row | when |
|---|---|---|
| yes | `can size a scrum` | the smallest order costs no more than the excess |
| no | `cannot size a scrum` | it costs more |
| unknown | `size rules not read` | no market record was obtained |

Unknown is a third answer and never folds into either other. `MarketRules` carries an unpublished rule as `None` and sets `read` to False when no record was obtained at all, so a venue that publishes nothing and a venue nobody reached are told apart.

```
# src/exchange/base.py:122
    ``None`` is a rule the venue did not publish, never a rule of zero, and
    ``read`` is False when no market record was obtained at all.
```

### The excess the rule measures against

The excess figure is the largest one the saved fleet produces. Every bot in it carries the same 5 per cent scrumming interval and the highest Target Balance is 350 dollars.

```
# src/trading/scrumming/sizing.py
LARGEST_FLEET_TARGET_USD = 350.0
FLEET_SCRUMMING_INTERVAL_PCT = 5.0

REFERENCE_SCRUM_EXCESS_USD = scrumming_interval_usd(
    LARGEST_FLEET_TARGET_USD, FLEET_SCRUMMING_INTERVAL_PCT
)
```

The largest is the right end of the fleet, not the smallest. `cannot size a scrum` has to mean no bot at any target this fleet carries could size one there. Read off a copy of the saved fleet: 38 bots, one interval value,

targets from 25 to 350 dollars, and an excess from 1 dollar 25 to 17 dollars 50. A caller that knows one bot's own interval passes it instead.

### Which venues this reaches today

Only the crypto connector loads a market table, so only its markets answer yes or no. Every other row answers unknown until a broker connector is wired. That matches the verdict column: no Coinbase spot product reads `cannot size a scrum`, because its minimum order cost is one dollar on most products and ten on ether, and both sit under the excess.

The whole-share brokers, the whole-unit currency dealers and the futures venues that read no in the verdict column are the markets the `no` answer is for. None of them is wired, so none is drawn today.

### What this entry overtakes on this page

Nothing. Every sentence above it stands as written, and this entry only adds the per-market answer beside the per-venue verdict.

The answer is for the one variant that exists, the bot that names a unit count. The second variant names a cash amount and nothing in the tree constructs it, so the ticker rows answer for the built variant alone and the offered list is narrowed by that answer. It widens when the second variant lands.

## 2026-09-25 - the price step and the fraction declaration

A venue publishes three order rules, not two. Beside the smallest size it will accept and the step that size moves by, it publishes the step a price moves by. Coinbase names that third one `quote_increment` on its own product record, and the record a bot reads now carries it.

```
# src/exchange/base.py:131
    min_amount: Optional[float] = None  # base units
    min_cost: Optional[float] = None  # quote units
    amount_increment: Optional[float] = None  # base units a size steps by
    price_increment: Optional[float] = None  # quote units a price steps by
    read: bool = True
```

Read off four recorded Coinbase products: bitcoin and ether step by one cent, TE-FOOD steps by a hundredth of a cent, and a product that publishes no step at all carries the step as absent rather than as zero.

### The price a venue books

A venue does not book the price a bot names. It moves that price to its own nearest step and books the result, so the money an order is worth is the size times the stepped price. The minimum order cost is now compared against that number.

```
# src/trading/bot_container.py:294
                # The venue books a price on its own tick, so min_cost is
                # compared against the notional at that price.
                _booked_px = _rules.price_on_tick(_px)
```

The refusal the operator reads on the Console names both figures, so a price that moved and a price that did not are told apart.

```
PRE-FLIGHT REJECTED: SELL A14/USD notional $0.3631 (0.0001000000 x
$3630.98000000) is below min_cost $10.0000. Priced at $3630.98000000 on a
price tick of 0.01. API not called.
```

A sub-cent market keeps its digits. A product priced at 0.0000037 dollars with a step of a ten-millionth of a dollar reads `$0.00000370` on that line, because every step is taken in exact decimal arithmetic and never by cutting a price to a fixed number of places.

### Whether a market can be held in fractions

The size step already answers it, and no separate field says it again. A market whose step is under one unit accepts a fraction of a unit; a market whose step is a whole unit or larger takes whole units only; a market whose step the venue never published answers neither.

```
# src/exchange/base.py:133
    amount_increment: Optional[float] = None  # base units a size steps by
```

Read off the recorded products: bitcoin steps by a hundred-millionth and ether by a ten-thousandth, so both take fractions. TE-FOOD and the expiring contract both step by 1, so both take whole units only.

A field carrying the same answer a second time was written and then taken out, because nothing in the running program read it. The reader arrives with the layer that sizes an amount onto the step, and with the layer that picks the bot variant a market needs. Until then the step is the answer.

### A floor of zero the venue never published

The unknown row of the three-answer table above is kept as written. It says the unknown answer is drawn when no market record was obtained.

The true row is: the unknown answer is drawn when no market record was obtained, and also when the venue published no size rule and no minimum order cost, because the smallest order then costs an unknown amount rather than nothing.

A market that published nothing used to be offered as tradeable with a smallest order of zero dollars. Zero is not a floor the venue gave; it is the figure left over when no floor was read.

```
# src/trading/scrumming/sizing.py:149
    if amount is None:
        return floor_usd if cost_published else None
```

A minimum cost the venue published **as** zero is a different thing, and it stays a known floor of zero. Both were driven, and they answer differently.

| What the venue published | Smallest order | Answer |
|---|---|---|
| nothing at all | not known | size rules not read |
| a minimum size alone | that size at the last price | yes or no |
| a size step alone | one step at the last price | yes or no |
| a minimum cost of zero, no size rule | zero dollars | yes |
| every rule | the larger of the two floors | yes or no |

Only the first row moves. Every market that published enough to answer still answers, including a market whose venue published a minimum size and no step. That matters for one venue Acervator offers: of 104 exchange classes in the trading library, 102 publish a step-shaped size rule and 2 do not, and one of those 2 is on the offered list.

Nothing else on this page is overtaken. The per-venue verdict column, the excess figure and the two-variant reading all stand as written.

### The tuple this record replaced

The block under *What a scrum asks of a venue* is kept as written. It quotes a docstring saying the lookup returns a minimum amount, a minimum cost and an amount precision, cached, and zero, zero and eight on any lookup failure.

That tuple is gone. The bot reads a record instead, and the record holds an unpublished rule as absent.

```
# src/trading/bot_container.py
    async def _get_market_rules(self, symbol: str) -> "MarketRules":
        """Return the venue's published ``MarketRules`` for ``symbol``, cached,
        and an all-``None`` record when the lookup fails or the venue lists no
        such market."""
```

### Which step actually changes the size

The first sentence of *What a scrum asks of a venue* is kept as written. It says two steps stand between the computed size and the venue, one of which truncates the size.

Driven on the seven order readings above, the size the bot computes reaches the connector unchanged in every one of them: 1234.56789 in and 1234.56789 out, 0.012345678912345 in and 0.012345678912345 out. The guard reads the market's rules to refuse a size under the minimum and a notional under the minimum cost, and it changes no size. The one place a size is stepped is the connector, at the line quoted in that section. The docstring that said otherwise now carries the true sentence beneath it.

### What the broker path still carries

Nothing. `src/stocks/broker_base.py` names no size rule, no price step and no market record, and no code in the tree constructs a broker connector, so there is nothing on that path for a rule to reach. The crypto record carries the rules; the two order contracts stay separate.

## Where each venue fact was read

Each publisher's page was opened as a document outside this repository and handed to this page on 2026-09-24. No endpoint was called, no credential was sent, and no account page was opened. A rule a page does not state is written as not stated rather than filled in.

```
Coinbase          Create a new order; Get Product; Advanced Trade US derivatives
Alpaca            fractional trading documentation
Interactive Brokers  TWS API order limitations; TWS API minimum increment
Schwab            Trader API developer portal; the TD Ameritrade discontinuation notice
tastytrade        API documentation
E*TRADE           developer documentation
Webull            the fractional-support table TradersPost publishes
Fidelity          the retail product pages, which publish no trading API
OANDA             v20 REST API documentation
FOREX.com         StoneX brand pages
tastyfx           IG US brand pages
Tradovate         product and API pages
NinjaTrader       NinjaTrader Clearing exchange access pages
```

The crypto venue set, the passphrase set and the two refusal sets are readings this repository holds, dated in the file that carries them.

```
# src/exchange/ccxt_connector.py:109
# Date the venue facts below were last measured.
VENUE_MEASUREMENT_DATE: str = "2026-08-28"
```

## 2026-09-25 - a back-tested order obeys the venue that would execute it

The Simulator and the Paper Trader reach no venue, so they could not read a venue's order rules. They sized every order from a two-row table of cited unit rules instead, and that table carries one fact: whether fractions are allowed. An order sized that way says nothing about whether the venue would take it.

The live connector already reads each market's record. It now writes what it read, once per read, into a recording under the runtime home, and the Simulator and the Paper Trader read the recording.

```
# src/exchange/market_rules_store.py:28
def store_path() -> Path:
    return Path.home() / ".acervator" / STORE_NAME
```

### What the recording changes about a back test

An order's amount is floored onto the venue's own size step, and an amount under the venue's own minimum fills nothing. Where no recording exists for the market, the cited table still answers, and the run says which of the two sized each order.

```
# src/trading/scrumming/sizing.py:272
def sized_order(units: float, rule: Optional[str], rules: Any = None) -> SizedOrder:
```

### The two figures a run reports

Every fill row on a Back Test report carries a **sized by** cell reading either `recorded venue rules` or `cited unit rule`. Every bot row carries an **orders refused** count, and the Activity Log names the reason behind each count.

Measured on `A34_5m_2026_coinbase`, 5,102 bars rolled to one hour, one bot at a $350 target, sockets refused and the home redirected:

| Recorded size step | Orders filled | Orders refused | Fills on the step | Whole-unit fills |
|---|---|---|---|---|
| a hundredth of a unit | 21 | 710 | 21 of 21 | 0 of 21 |
| one whole unit | 20 | 709 | 20 of 20 | 20 of 20 |
| a hundredth, minimum 200 units | 0 | 25 | none to read | none to read |
| nothing recorded | 21 | 0 | not measurable | 0 of 21 |

The last row is the reading the Simulator gave before this change: amounts such as `92.62189809510927` units, which sit on no step Coinbase publishes. The same tape with a recorded step reads `92.61`, and with a whole-unit step reads `92`.

### Why the refusal count runs high

A fold spends what its queued tranches hold. Many of those spends buy under a tenth of a unit, and Coinbase refuses an order that small. Those 710 orders were previously booked as fills. A refused fold consumes no tranche, so the tranches accumulate and a later fold fills at a legal size.

### What the recording does not yet reach

The minimum order cost is recorded and is not yet measured against a back-tested order's notional. The live order guard already compares it, at `src/trading/bot_container.py:305`. The live path also still sizes on the cited fractional rule and does not floor onto the step.

OVERTAKEN, and the sentence above is kept as written. The live path floors onto the venue's published step. `src/trading/bot_container.py:260` calls `sized_order` at the one site every live order passes, so a recorded `amount_increment` floors the amount, a recorded `min_amount` refuses it at `:264`, and an amount that floors to nothing refuses at `:273`. The sized amount reaches the VolumeGuard branch and `exchange.place_order` alike, and `min_cost` is measured on it.

Driven on a bare host with the home redirected, every socket but loopback refused, and an exchange stand-in that raises on every order call, an amount of `92.62189809510927` units reaches the venue as `92.62` on a recorded hundredth step and as `92.0` on a whole-unit step. The same amount reaches the venue unchanged where the venue published no step. Of 10 amount-and-rule pairs driven, **0** reach the venue larger than they were handed and **0** submit where they previously refused.

A market whose venue published no step is not sized and is not refused on that ground. `0.0 < _sized.units < _amt` guards the assignment, which refuses a larger amount whatever answers it.

## 2026-09-25 - every venue set beside every other, and the verdict per venue

This page already answers one question per venue: does a scrum's excess survive the venue's own size rule. That is one of five things the bot needs. This entry adds the other four, sets all five side by side, and closes with a single yes or no per venue.

Nothing above this heading is deleted or reworded. Two cells are overtaken and both are quoted whole under *What this entry overtakes and what it corrects*.

### What the bot as written requires of a venue

Five requirements, each read off the tree. A venue meeting all five is traded by the bot that exists today, with no variant.

```
1  a connector the running program constructs
   src/exchange/ccxt_connector.py:284   CCXTConnector, 30 importing sites
   src/stocks/broker_base.py:108        BrokerBase, no caller

2  an order whose size is a count of base units
   src/exchange/base.py:299   place_order(symbol, side, order_type, amount, price, ...)

3  a market order, a limit order, and a limit order carried immediate-or-cancel
   src/exchange/base.py:30    OrderType MARKET, LIMIT, IOC_LIMIT

4  a size rule the record can hold, and a smallest order under the excess
   src/trading/scrumming/sizing.py:272   sized_order

5  a market record the connector can read
   src/exchange/ccxt_connector.py:245    market_rules
```

### Whether Acervator itself can reach a venue today

The reachability column higher up this page answers whether the venue accepts a United States account. This one answers a different question: does the running program hold code that submits an order there.

| Venue | Class | A connector the program constructs | What was measured |
|---|---|---|---|
| the fifteen crypto ids | crypto | yes, one connector serves all fifteen | `git grep` finds 30 sites importing the crypto connector |
| Alpaca | stocks | no | the one broker connector is imported by nothing |
| Interactive Brokers, Schwab, tastytrade, E*TRADE, Webull, TD Ameritrade, Fidelity | stocks | no | no module names any of the seven |
| OANDA, FOREX.com, tastyfx | forex | no | no venue list holds a forex id |
| Tradovate, NinjaTrader | futures | no | no venue list holds a futures id |

Two searches over the git index give that column, and the second is the control for the first.

```
git grep -n alpaca_connector -- src/ main.py tools/
  -> src/stocks/alpaca_connector.py:2, its own docstring, and nothing else

git grep -n broker_base -- src/ main.py tools/
  -> src/stocks/alpaca_connector.py:14, and the module's own docstring

control, the same command on the crypto connector
git grep -n ccxt_connector -- src/ main.py tools/
  -> 30 sites, one of them importing it under an alias
```

The equities order path is a contract with no caller. A venue served only by that path cannot be traded today whatever its own rules allow, and that is the reason every stocks row above reads no.

### Every venue's order shape, set side by side

One order, two contracts, and the differences are visible in one table. The crypto contract names an amount and hands it to the trading library, which translates to each venue's own endpoint. The broker contract names a quantity and posts it as JSON.

| | Crypto, the fifteen ids | Alpaca, the one broker | Interactive Brokers, Schwab, tastytrade, E*TRADE | OANDA, FOREX.com, tastyfx | Tradovate, NinjaTrader |
|---|---|---|---|---|---|
| What an order must carry | symbol, side, type, amount, optional price, optional client order id | symbol, side, quantity, type, time in force, optional limit and stop prices | not stated by the pages read | not stated by the pages read | not stated by the pages read |
| Size named as | a count of base units | a share count, sent as a string | a share count | a count of currency units | a whole contract count |
| Price named as | a float moved to the venue's own tick | a string, only on a limit or stop order | a per-contract minimum increment | not stated by the pages read | the product's own tick |
| Order types the tree sends | market, limit, limit with immediate-or-cancel | market, limit, stop, stop limit, trailing stop are declared; nothing sends one | none, no connector exists | none, no connector exists | none, no connector exists |
| How it is submitted | the library's own create-order call | a POST to the broker's orders endpoint | not reached | not reached | not reached |
| Where it is read | `src/exchange/base.py:299` | `src/stocks/alpaca_connector.py:180` | this page's venue table | this page's venue table | this page's venue table |

Three differences the crypto path already absorbs per venue, and each is a line in one connector rather than a variant.

```
src/exchange/ccxt_connector.py:1169   the client-order-id field name per venue
                                      coinbase   client_order_id
                                      binance    newClientOrderId
                                      kraken     userref
                                      every other  clientOrderId

src/exchange/ccxt_connector.py:1183   no venue carries an immediate-or-cancel
                                      type, so it is sent as a limit order
                                      carrying timeInForce IOC

src/exchange/ccxt_connector.py:1132   a spot market buy on Coinbase needs a
                                      price, so a ticker is fetched first
```

### The order types each crypto venue declares

The trading library publishes a capability map per exchange class. Read as installed, with every socket refused and no venue contacted, it answers which order shapes each of the fifteen offered venues declares.

| Venue | market order | limit order | market buy by cash amount | market sell by cash amount |
|---|---|---|---|---|
| binance | yes | yes | yes | yes |
| bitfinex | yes | yes | not declared | not declared |
| bitget | yes | yes | yes | no |
| bitstamp | yes | yes | not declared | not declared |
| bybit | yes | yes | yes | yes |
| coinbase | yes | yes | yes | no |
| cryptocom | yes | yes | no | no |
| gateio | yes | yes | yes | no |
| gemini | **no** | yes | not declared | not declared |
| huobi | yes | yes | yes | no |
| kraken | yes | yes | yes | no |
| kucoin | yes | yes | yes | yes |
| mexc | yes | yes | yes | yes |
| okx | yes | yes | yes | yes |
| poloniex | yes | yes | yes | no |

Three counts fall out of that table and each one bears on a variant. Every one of the fifteen declares a limit order. Fourteen declare a market order, and the exception is gemini. Eleven declare a market buy by cash amount, and only five declare the sell side of it.

```
  createOrder                      15 of 15
  createLimitOrder                 15 of 15
  createMarketOrder                14 of 15   gemini declares it False
  createMarketBuyOrderWithCost     11 of 15
  createMarketSellOrderWithCost     5 of 15
  createStopLimitOrder             11 of 15
  createStopMarketOrder             8 of 15

  control, the same reader on a capability the library does not carry
    has["createOrder"]           -> True
    has["zzNoSuchCapability"]    -> None
    keys in coinbase.has         -> 246
```

The fifth line matters more than its size suggests. A scrum sells, and a cash amount reaches the sell side at a third of the offered venues.

### The size rule shape, and the two venues that publish none

The record a bot reads holds a size step. Of the 104 exchange classes the installed trading library carries, 102 publish a step-shaped size rule and 2 publish significant-digit precision instead. One of those two is on the offered list.

| Reading | Figure |
|---|---|
| exchange classes in the installed library | 104 |

| Reading | Figure |
|---|---|
| classes publishing a step-shaped size rule | 102 |
| classes publishing significant digits instead | 2, bitfinex and bithumb |
| of those two, on Acervator's offered list | 1, bitfinex |

Two controls sit beside that count. The reader answers a step under the step-shaped mode and nothing under the significant-digit one, which makes the figure 2 a fact about the venues rather than about the reader. Every class was read with the socket constructor replaced by one that raises.

```
  precision_to_increment("0.01", TICK_SIZE)           -> 0.01
  precision_to_increment("2",    DECIMAL_PLACES)      -> 0.01
  precision_to_increment("5",    SIGNIFICANT_DIGITS)  -> None

  104 classes instantiated, 0 failed, sockets refused for the whole census
```

Driven on the five rule shapes a record can carry, the order gate floors an amount only where a step was published. A venue publishing a minimum and no step keeps its minimum refusal and takes no flooring.

| What the record carries | An amount of 92.62189809510927 becomes | Sized by |
|---|---|---|
| step 0.01, minimum 0.1 | 92.62 | recorded venue rules |
| minimum 0.1, no step | 92.62189809510927 | recorded venue rules |
| no minimum, no step | 92.62189809510927 | cited unit rule |
| step 1.0, minimum 1.0 | 92.0 | recorded venue rules |
| no record read | 92.62189809510927 | cited unit rule |

The control on that table is an amount under the minimum, which refuses rather than sizing, so an empty refusal above is a reading and not a silence.

```
  sized_order(0.05, fractional, MarketRules(min_amount=0.1))
    -> units 0.0, refusal "below the venue's minimum size"
```

The trading library still rounds the amount inside the connector for every venue, so a significant-digit venue is not handed raw float digits. bitfinex carries its own override of that rounding. Read with a planted market record and no network, an amount of 92.62189809510927 came back as 92.62189 for a published 5 and as 92.62189809 for a published 8.

```
  coinbase, published step 0.01   -> "92.62"
  coinbase, published step 1.0    -> "92"
  bitfinex, published 5           -> "92.62189"
  bitfinex, published 8           -> "92.62189809"
  bithumb,  published 4           -> "92.6218"
```

### Can the bot as written trade here

The deliverable. One row per venue, two yes-or-no columns, and every no naming what would have to change. The first column asks whether the bot's own order shape fits the venue's rules. The second asks whether an order can reach the venue at all from this program and a United States account.

| Venue | The bot's shape fits | An order can reach it today | What the no is |
|---|---|---|---|
| coinbase, spot | yes | yes | — |
| kraken | yes | yes | — |
| kucoin | yes | yes | — |
| okx | yes | yes | — |
| gateio | yes | yes | — |
| bitget | yes | yes | — |
| mexc | yes | yes | — |
| bitstamp | yes | yes | — |
| cryptocom | yes | yes | — |
| bitfinex | yes | yes | — |
| gemini | **no** | yes | the venue declares no market order, and the bot names a market order at nine of the thirteen places it names an order type |
| binance | yes | **no** | the venue refuses a United States address; the bot's shape is not the obstacle |
| bybit | yes | **no** | the venue refuses a United States address; the bot's shape is not the obstacle |
| poloniex | yes | **no** | the venue restricts a United States account; the bot's shape is not the obstacle |
| huobi | yes | **no** | the venue restricts a United States account; the bot's shape is not the obstacle |
| coinbase, US futures | **no** | **no** | a contract is whole, and an excess under one contract cannot be sold |
| Alpaca | yes | **no** | nothing constructs the broker connector, so the order path has no caller |
| Webull | yes | **no** | nothing constructs a connector for it, and none exists |
| Interactive Brokers | **no** | **no** | a quantity order rounds down to whole shares, and no connector exists |
| Schwab | **no** | **no** | the Trader API places no fractional order, and no connector exists |
| tastytrade | **no** | **no** | no fractional quantity for automated trading, and no connector exists |
| E*TRADE | **no** | **no** | no fractional share of an individual stock, and no connector exists |
| TD Ameritrade | **no** | **no** | the API closed on 10 May 2024 |
| Fidelity | **no** | **no** | no retail trading API is published |
| OANDA | **no** | **no** | whole currency units, and no connector exists |
| FOREX.com | **no** | **no** | whole currency units, and no connector exists |
| tastyfx | **no** | **no** | whole currency units, and no connector exists |

| Venue | The bot's shape fits | An order can reach it today | What the no is |
|---|---|---|---|
| Tradovate | **no** | **no** | a contract is whole, and no connector exists |
| NinjaTrader | **no** | **no** | a contract is whole, and no connector exists |

Ten venues answer yes to both. Every one of the ten is crypto spot, and every one is reached by the single connector the program already constructs.

Four rows say no to the second column for a reason no variant can absorb: the venue itself refuses the account. Nine rows say no because no connector exists. Those nine are a construction that is missing, not a rule the bot cannot satisfy.

Two verdicts on this page are not answered and are written as such rather than guessed. Whether each whole-share broker accepts a sell named as a cash amount is not stated by any page read for this manual, and each firm's own order-submission page answers it. The order types Schwab, tastytrade, E*TRADE, Interactive Brokers and the three currency dealers accept are likewise not stated, and each firm's own order specification answers that.

### The variants this comparison implies

Three, and no more. Each is named by what it absorbs. None is built here, and one of the three cannot be designed until a decision on the issue is answered.

The first names a cash amount where the bot names a unit count. It is already proposed higher up this page and it absorbs a whole-share size rule. It reaches Interactive Brokers, Schwab, tastytrade and E*TRADE, and Alpaca carries the same field beside its quantity.

```
# PROPOSED, not present. Quoted from this page's own proposal above.
    async def place_order(
        self,
        symbol: str,
        side: OrderSide,
        quantity: float | None = None,
        notional_usd: float | None = None,
        order_type: OrderType = OrderType.MARKET,
    ) -> StockOrder:
```

The second sends a limit order where the bot sends a market order. It absorbs one venue, gemini, which is the only one of the fifteen that declares no market order. It is the smallest of the three and it changes one field.

```
# PROPOSED, not present. The order type a venue declines is replaced.
#   gemini declares createMarketOrder False; createLimitOrder True.
#   A market order becomes a limit order priced to cross.
```

The third holds an excess until it reaches one whole unit, then sells one. It absorbs every market whose smallest order costs more than a scrum's excess: each futures venue, the three currency dealers, and any market whose own row already reads that it cannot size a scrum. **It cannot be designed yet.** The issue's first open decision is whether such a market is refused or whether the scrum trigger changes for it, and the two answers build different variants.

```
In development. Decision 1 on the issue owns it.
```

Nineteen of the twenty-nine rows above need no variant at all. Ten trade today, four are refused by their own venue, and nine wait on a connector rather than on a shape. Saying so is the useful answer, and inventing a fourth variant to round the list out would not be.

### What this entry overtakes and what it corrects

One sentence of this page is overtaken. It is kept as written:

> *The verdict column above falls into two groups and no more. A venue that reads yes needs nothing new. Every venue that reads no is served by one second shape, not by a shape of its own.*

The true sentence is: the verdict column falls into two groups on the size rule alone, and the whole requirement set splits the no group into three. A whole-share venue is served by the cash-amount shape. A venue declining a market order is served by a limit-only shape, which the size rule cannot see. A market whose smallest order costs more than the excess is served by neither, and waits on the issue's first open decision.

One citation on this page has moved and the cells that carry it are kept as written. The venue table names the equity venue list at line 33 of its module, in nine of its source cells. That constant now sits at line 46, and line 33 holds a comment about button spacing.

```
src/gui/main_tabs/asset_class_surface.py:46   EQUITY_VENUES, nine ids
src/gui/main_tabs/asset_class_surface.py:65   LAYERED_CLASSES, crypto and stocks
```

Two further citations elsewhere in this arc have moved the same way, and both name a constant that exists. The cited unit rule table sits at line 57 of its module rather than line 46, and the per-venue candle lengths sit at line 47 of theirs rather than line 50.

Every other sentence above stands as written. The per-venue size table, the excess figure, the three-answer rule and the per-market answer are unchanged by this entry.

## 2026-09-25 - a venue's own rules select the bot variant

A venue's own published rules pick the bot's shape. No setting offers the choice and no screen exposes it. One function reads one market's record and answers with a variant name, and the order path acts on that name at the single site every live order passes. A record nothing has read selects the bot as written.

```
# src/trading/scrumming/sizing.py:280
def venue_variant(
    rules: Any,
    price: Optional[float] = None,
    excess_usd: float = REFERENCE_SCRUM_EXCESS_USD,
) -> str:
    if rules is None or not getattr(rules, "read", False):
        return VARIANT_NONE
    if tradeable_answer(rules, price, excess_usd) == TRADEABLE_NO:
        return VARIANT_WHOLE_UNIT
    if getattr(rules, "order_types", None) == ORDER_TYPES_LIMIT_ONLY:
        return VARIANT_LIMIT_ONLY
    return VARIANT_NONE
```

### Which markets each variant states it can trade

Four names exist, counting the bot as written, and each one is named by the market shape it absorbs. The program holds two of the four, and one function answers per market whether the variant that market selects is one of the two.

```
# src/trading/scrumming/sizing.py:264
VARIANT_MARKETS: dict[str, str] = {
    VARIANT_NONE: "a market naming a unit count on a venue taking a market order",
    VARIANT_LIMIT_ONLY: "a market on a venue declaring no market order",
    VARIANT_CASH_AMOUNT: "a market whose size is a whole share",
    VARIANT_WHOLE_UNIT: "a market whose smallest order costs more than the excess",
}

#: The variants the running program holds. ``VARIANT_CASH_AMOUNT`` has no caller
#: to reach it and ``VARIANT_WHOLE_UNIT`` waits on the scrum trigger's ruling.
VARIANTS_BUILT = frozenset({VARIANT_NONE, VARIANT_LIMIT_ONLY})
```

### The limit-only variant, and the one venue it reaches

The limit-only variant is built. It sends a limit order where the bot sends a market order, and it changes one field. Gemini is its one venue. The order-type table cites two pairs, Gemini is the only one of the two declaring no market order, and a pair absent from that table declares nothing and is never read as declining a type.

```
# src/trading/scrumming/sizing.py:147
CITED_VENUE_ORDER_TYPES: dict[tuple[str, str], str] = {
    (CLASS_CRYPTO, "coinbase"): ORDER_TYPES_WITH_MARKET,
    (CLASS_CRYPTO, "gemini"): ORDER_TYPES_LIMIT_ONLY,
}
```

### The two variants named and not built

The cash-amount variant is named and nothing reaches it, because the equities order path has no caller. The broker base declares the order method and no module calls it, and no file outside its own imports the one broker connector that exists.

```
src/stocks/broker_base.py:145      place_order, declared, called by nothing
src/stocks/alpaca_connector.py     imported by no file in src/, main.py or tools/
```

The whole-unit variant is named and waits on a decision that is the operator's. It would hold an excess until that excess reaches one whole unit, then sell one unit. Refusing a market too small to scrum and changing that market's scrum trigger build two different variants, and the issue's first open decision picks between them.

```
In development. Decision 1 on the issue owns it.
```

### A market no variant trades is scanned, charted and reported

Exclusion is from trading, not from sight. A scan keeps such a symbol in its own asset list and names it in a field of its own, so the market still draws its chart and still reports. Every order path refuses that market carrying the reason, and the reason names the variant the market needs beside the shape that variant absorbs.

```
# src/trading/ata_spm.py:965
    #: The symbols this scan read that no built bot variant trades, kept in
    #: ``assets`` so each one still charts and still reports.
    untradeable: tuple = ()
```

### The variant sentences this entry overtakes

Two passages above are overtaken. Both are kept as written. The first is the count under the variants this comparison implies:

> *Three, and no more. Each is named by what it absorbs. None is built here, and*
>
> *one of the three cannot be designed until a decision on the issue is answered.*

The true sentence is: four variant names exist, counting the bot as written, and the program holds two of them. The limit-only variant is built and Gemini is its one venue. The cash-amount variant is named and has no caller. The whole-unit variant is named and waits on the issue's first open decision.

The second is the heading over the earlier proposal:

> *Two variants cover every venue*

The true count is four names, two of them built. That heading counts the two shapes the size rule can see, a unit count and a cash amount. It counts neither the order-type shape, which the size rule cannot see, nor the whole-unit shape, which waits on a ruling.

## 2026-09-25 - what each count over the verdict table measures

The verdict table above holds twenty-nine rows, and the prose around it carries four counts of them. Three of the four are right about three different questions, and the fourth matches no set the table holds. The figures below name the question each count answers.

### The recount, and the control it was taken with

Read from the table's own two verdict columns, and from its reason column counted by phrase:

```
rows in the table                                    29
the bot's shape fits reads yes                       16
both columns read yes                                10
the reason names the venue refusing the account       4
the reason ends "and no connector exists"             9
the reason names a connector at all                  11
rows a named variant absorbs                         11
rows no named variant absorbs                        18
```

The counter was calibrated on those same twenty-nine rows before any figure above was taken. Three reason phrases the table carries returned 5, 3 and 1, and a phrase planted for the purpose returned nothing and exited non-zero.

Three different questions therefore produce three different counts over one set of rows, and none of the three is wrong about its own question. Sixteen rows need no variant, because the bot's shape already fits the venue. Eighteen rows are absorbed by no named variant, because no shape helps a venue that refuses the account or a connector that is missing. Eleven rows are absorbed by a named variant: four by the cash-amount shape, one by the limit-only shape, and six by the whole-unit shape.

### The counting sentences this entry overtakes

Two passages are overtaken. Both are kept as written. The first is the closing count:

> *Nineteen of the twenty-nine rows above need no variant at all. Ten trade today, four are refused by their own venue, and nine wait on a connector rather than on a shape.*

The true sentence is: sixteen of the twenty-nine rows read yes in the bot's-shape-fits column, which is the column that says a variant is not needed. The three figures beside that count answer three different questions and sum to twenty-three, leaving six rows outside all three — gemini, coinbase US futures, Alpaca, Webull, TD Ameritrade and Fidelity.

The second is the connector count above the variant list:

> *Nine rows say no because no connector exists. Those nine are a construction that is missing, not a rule the bot cannot satisfy.*

The true sentence is: nine rows carry a reason cell ending in that phrase, and ten rows name a connector that does not exist, because the Webull cell words the same absence differently. The Alpaca cell names a connector that does exist and has no caller, which is why the figure for a missing connector is ten rather than eleven.

## 2026-09-26 - a venue's expiry stops a buy and not a sale, and cash the venue has not settled is not spent twice

A venue can close a market on a date of its own, and it can take days to hand over the cash from a sale. The record a bot reads carries both of those facts now, beside the size and price rules already on this page. A venue that publishes neither leaves both absent, and absent changes nothing.

```
# src/exchange/base.py:141
    expiry_ms: Optional[float] = None  # epoch ms the venue closes the contract on
    settlement_days: Optional[float] = None  # days the venue takes to settle a sale
```

### The two rules the record carries beside the other four

Two members read the expiry. The first answers whether the venue published one at all. The second answers how many days remain, and it turns negative once the date has passed. A record holding no expiry answers False and nothing.

```
    @property
    def expires(self) -> bool:
        """True while ``expiry_ms`` is a finite positive epoch, and False for the
        None a venue publishing no expiry carries."""

    def days_to_expiry(self, moment_s: float) -> Optional[float]:
        """Days from ``moment_s`` to ``expiry_ms``, negative once it has passed.

        None while ``expires`` is False or ``moment_s`` is not a finite number.
        """
```

### A buy refuses on every order path, and a sale passes

A market that publishes an expiry selects a fifth variant name, and the program does not hold it. The function asks the expiry question first, before the size question and before the order-type question, so a market with a date on it never reaches either of them.

```
# src/trading/scrumming/sizing.py:331
VARIANT_ROLLING_POSITION = "rolling position"

VARIANT_MARKETS[VARIANT_ROLLING_POSITION] = "a market the venue expires on a date"

# src/trading/scrumming/sizing.py:375
    if getattr(rules, "expires", False):
        return VARIANT_ROLLING_POSITION
```

Three order paths read that answer, one per data source, and each of the three refuses a buy into such a market by name. The scan keeps the symbol in its own asset list and names it in a field of its own, exactly as it does for any other market no built variant trades.

```
src/trading/bot_container.py:370     the live order path
src/paper/paper_run.py:507           the Paper Trader's fold
src/simulator/back_test.py:1297      the back test's fold
src/trading/ata_spm.py:1323          the scan naming the market
```

### Why the two sides differ

The asymmetry looks inconsistent until a reader has the reason. Buying into a market the venue removes on its own date takes on a thing that ends; the bot would accumulate into a market that stops existing, and no rule in the tree yet says which contract a position rolls into. Selling out of a position already held takes on nothing. A bot holding such a position keeps scrumming it down by its own cycles and closes it before the date, and nothing rebuys it.

```
# src/trading/bot_container.py:375
        _closing = variant_permits_close(_variant) and side == OrderSide.SELL

        if not variant_built(_variant) and not _closing:
            self._refuse_order(...)
```

The sell that passes says so where the operator watches. The Console line names the days left and states that nothing will rebuy the position.

```
CLOSING AN EXPIRING MARKET: SELL <symbol> <units> is submitted where a BUY is
refused, because the venue expires this contract in <n> days (rolling
position). Nothing rebuys it.
```

### A fold waits while the venue holds the cash

A sale whose cash the venue has not handed over has not returned its money, and spending it would spend the same dollars twice. One figure answers how much of a fold's own cash sits inside the delay, and a second holds the rebuy to the wallet less that figure. With nothing to spend, the fold fills nothing, names the reason, and leaves its tranches queued for the next round.

```
# src/trading/scrumming/sizing.py:169
CITED_VENUE_SETTLEMENT: dict[tuple[str, str], float] = {
    (CLASS_CRYPTO, "coinbase"): 0.0,
}

HELD_UNSETTLED_CASH = "the venue has not settled the sale"

def unsettled_usd(tranches, moment_s, settlement_days) -> float:
def spend_less_unsettled_usd(spend, cash_usd, held_usd) -> float:
```

A cited delay of zero is a venue returning the cash at once, and it holds nothing. A pair absent from that table publishes no delay, which also holds nothing. Only a positive figure can hold a fold back.

### Where the settlement hold reaches

Two of the three folds read the delay at the point they size a rebuy. The third, the live autonomous fold, does not read it. Live's manual rebalance does read it and names the held amount on the Console.

```
src/paper/paper_run.py:542           the Paper Trader's fold reads the delay
src/simulator/back_test.py:1333      the back test's fold reads the delay
src/trading/scrumming/execution.py:953   live's manual rebalance reads the delay
the live autonomous fold             does not read the delay
```

The one venue this program records cites zero days, so no fold on any path holds cash today. A venue citing a positive figure would reach the two folds above and leave the live autonomous one spending cash it does not yet have.

### Which markets carry an expiry today

None of them. The recording every back test and every paper run reads holds one venue and 1,146 markets, and not one row carries an expiry. The recording holds only four field names, and the expiry is not among them, because the recording predates the field. Each such row answers absent, which is exactly what a venue publishing no expiry answers.

```
recorded coinbase markets                                1146
rows with a non-null expiry                                 0
CONTROL rows with a non-null minimum size                1146
CONTROL rows with a non-null minimum cost                1146
CONTROL rows with a planted absent key                      0
rows carrying a dated contract suffix                     100
dated rows with a non-null expiry                           0
CONTROL dated rows with a non-null minimum cost           100
distinct symbols the saved fleet names                     38
of those, symbols carrying a dated contract suffix          0
CONTROL of those, symbols carrying a quote separator       38
```

The controls sit in the same run as the figures they support, so a zero above is a reading of the file and not of the reader. Two of the rows matter together: one hundred recorded symbols carry a dated contract suffix and none of them carries an expiry, because the recording predates the field. A fresh recording of that venue would fill those hundred rows, and those hundred markets would then select the fifth variant.

This reaches no live market. No symbol the saved fleet trades carries a dated contract suffix, no recorded row carries an expiry, and the settlement table cites this venue at zero days.

### The sentences on this page that this entry overtakes

Nine passages are overtaken. Each one stays exactly as written, with the sentence that is true today beneath it.

The first is the count of a venue's order rules:

> *A venue publishes three order rules, not two.*

The true sentence is: a venue's own product record publishes five rules, not three. The four already named on this page sit beside the date the venue closes the contract on. Three further fields on the record come from cited tables rather than from the product record, and they are the trading session, the order types and the settlement delay.

The second is the field block under that sentence. It lists four rules and the read flag, and it stays as written. The true block carries eight fields before that flag: the four it already lists, then the trading session, the order types, the expiry and the settlement delay.

```
# src/exchange/base.py:135
    min_amount: Optional[float] = None  # base units
    min_cost: Optional[float] = None  # quote units
    amount_increment: Optional[float] = None  # base units a size steps by
    price_increment: Optional[float] = None  # quote units a price steps by
    session: Optional[str] = None  # the session name the venue publishes
    order_types: Optional[str] = None  # the order types the venue declares
    expiry_ms: Optional[float] = None  # epoch ms the venue closes the contract on
    settlement_days: Optional[float] = None  # days the venue takes to settle a sale
    read: bool = True
```

The third is the variant function quoted under *a venue's own rules select the bot variant*. The block stands as written, and the function answers the rolling-position name first, before the size question and the order-type question it already showed.

The fourth is the count of variant names:

> *Four names exist, counting the bot as written, and each one is named by the*
>
> *market shape it absorbs. The program holds two of the four, and one function*
>
> *answers per market whether the variant that market selects is one of the two.*

The true sentence is: five names exist, counting the bot as written, and the program still holds two of them. The fifth name is the rolling position, which the program names and does not build.

The fifth is the market table quoted under that count. It stands as written, and it carries a fifth row now, naming a market the venue expires on a date.

The sixth is the heading over the unbuilt variants:

> *The two variants named and not built*

The true count is three named and not built: the cash-amount variant with no caller, the whole-unit variant waiting on a decision, and the rolling position waiting on the rule that names which contract a position rolls into.

The seventh is the sentence about what an order path does with such a market:

> *Every order path refuses that market carrying the reason, and the reason names*
>
> *the variant the market needs beside the shape that variant absorbs.*

The true sentence is: every order path refuses a buy into that market carrying the reason, and one unbuilt variant lets a sale out of it through. The rolling position is that one, and only a sell passes.

The eighth and ninth are the two corrected counts written when the variants were first set out:

> *The true sentence is: four variant names exist, counting the bot as written,*
>
> *and the program holds two of them.*
>
> *The true count is four names, two of them built.*

The same figure overtakes both. Five variant names exist and the program holds two of them.

## 2026-09-26 - a market buy with no price is refused by the connector and names why

A market buy carries no price of its own. The venue prices it by multiplying the amount by a price, so it needs one. The connector fetches a ticker to supply it, for every venue, with no venue test in front of the fetch. That fetch fails in two ways: the call raises, or the ticker carries neither a last price nor an ask price.

Until now both failures dropped the price and sent the order anyway. The trading library then refused it, from inside its own order builder, with a sentence about its own argument list. The order reached no venue either way. What a reader could not tell was which of the two failures had happened.

The connector refuses first now. It names the venue, the market and which of the two failures occurred, writes one record at error level, counts one failure against that market's breaker, and sends nothing.

```
# src/exchange/ccxt_connector.py:1180
            if exec_price is None:
                from ccxt.base.errors import InvalidOrder

                refusal = (
                    f"{self._exchange_id} refuses a MARKET BUY on {symbol} with "
                    f"no price: {no_price_cause}"
                )
```

### What a reader sees when the ticker serves no price

Two sentences, one per failure, read off a run with every socket refused and the home redirected.

```
coinbase refuses a MARKET BUY on A15/USD with no price:
  fetch_ticker raised RuntimeError: ticker endpoint down

coinbase refuses a MARKET BUY on A15/USD with no price:
  fetch_ticker served no last and no ask price
```

A market buy that does get a price is unchanged, and so are a limit order and a market sell. Driven on the same run, the market buy reached the order call carrying the figure the ticker served, the limit order reached it carrying its own price with no ticker fetched, and the market sell reached it with no price and no ticker fetched.

### The sentence this entry adds beneath

The page already carries the fetch, and that sentence stands as written.

> *src/exchange/ccxt_connector.py:1132 a spot market buy on Coinbase needs a*
>
> *price, so a ticker is fetched first*

The sentence beneath it: when that ticker serves no price the connector refuses the order itself, names which step failed, and sends nothing to the venue.

The fetch stays unconditional, and the reason is a reading of the trading library. The requirement is not declared in the library's capability map for the venue the fleet trades. It sits in that class's own option map instead, where it defaults to on. A reader that asked the capability map whether a price is needed would answer no, and removing the price would break every live market buy on that venue.

Part 5

# Conceptual Hopscotch: The HOP Protocol, Rules Registry, and SADP v1 and v2

## Conceptual Hopscotch

Reference. Three subjects a reader meets around this repository and cannot find in the running code: the handoff file that carries a session across a context boundary, the rule registry no running module reads, and the development protocol, which exists in two versions — one with no source in this repository, one running today.

### The handoff file

`ACERVATOR_HOP8.md` at the repository root is the current handoff and the only one this manual describes. The earlier `ACERVATOR_HOP*.md` files at the root hold archives of sessions that closed, and nothing in them binds.

A root-file inventory test records why the handoff sits at the root rather than in a directory: the drift check looks for it there, and a fresh clone has to find its orientation without being told where to look. Each root file in that inventory carries a reason naming a mechanism, and a new root file without one fails the check.

```
def test_the_inventory_names_every_tracked_root_file() -> None: ...  # tests/test_repo_root_inventory.py
def test_the_inventory_names_no_file_that_has_left() -> None: ...
def test_every_inventory_entry_carries_a_reason() -> None: ...
```

The handoff holds current state and an index, never a history. Its own three standing instructions are to rewrite a section that has drifted rather than append to it, to move anything measurable into a tool rather than prose, and to keep every claim checkable in one command.

### What the drift check measures

`python -m tools.hop_check` reads the newest handoff and reports three quantities.

- Commits landed since the handoff was last written, from `git rev-list --count` against the commit that last touched it. `STALE_AFTER` is 15; at or past that count the report calls the file drifted.
- Cited paths that no longer exist, matched as a backticked path under one of eight tracked directories.
- Cited commits that no longer resolve, matched as a backticked hex string of seven characters or more and checked with `git cat-file -t`.

```
TRACKED = r"`((?:src|tests|tools|docs|desktop|dev_harness|harness_fixtures|\.github)/[\w./-]+)`"
```

Any one of the three returns exit 1. Run against the tree, the report reads 107 commits since the handoff was written, three paths declared absent on purpose, no missing path and no unresolved commit, and it exits 1 on the commit count alone.

`declared_absent` reads the paths under the `## CITED AS ABSENT` heading and drops them from the missing-path count. A path under that heading is skipped even when it sits on disk, which makes the heading a

record of a deliberate citation rather than a check on one.

### The sort key picks the wrong file at ten

The picker sorts the file names as strings and takes the last one. Every handoff suffix holds one digit today, so the answer is correct. Driven with a two-digit name added, `ACERVATOR_HOP10` sorts ahead of the single digits, the last entry stays `ACERVATOR_HOP8.md`, and the drift report then describes a superseded file. Driven again with only the two newest names present, the answer is the same. The check needs a numeric key before a tenth handoff exists.

```
def newest_hop() -> pathlib.Path | None:        # tools/hop_check.py
    """Returns the highest-numbered ACERVATOR_HOP*.md at the repository root."""
    found = sorted(ROOT.glob("ACERVATOR_HOP*.md"), key=lambda p: p.stem)
    return found[-1] if found else None
```

## The rules registry

The registry module declares 35 rule ids, R1 through R35, across ten lettered groups. It holds one entry per id in one of four states — LOCKED, UNLOCKED, SUSPENDED and DEPRECATED — and a parser turns a `RULE` line into a call on the registry. Five ids are marked CORE: R1, R5, R10, R11 and R12.

```
RULE_META = {...}       # src/core/rule_registry.py, 35 ids, R1 through R35
CORE_RULES = {r for r, m in RULE_META.items() if m["protection"] == "CORE"}
VALID_STATES = {"LOCKED", "UNLOCKED", "SUSPENDED", "DEPRECATED"}

class RuleEntry: ...
class RuleRegistry: ...
def parse_rule_command(...): ...
```

The suspend call refuses a CORE id. Driven both ways against a registry file in a temporary directory, R1 draws a refusal and stays LOCKED while R2 moves to SUSPENDED. The refusal is a real one rather than a declared one.

```
def suspend(self, rule: str, reason: str, expires: str = "") -> str: ...     # src/core/rule_registry.py
```

Nothing in the product reads any of it. The registry path resolves into a top-level directory that no commit in this repository has ever added. An import scan over 957 Python files — parsing each file and reading its import nodes, rather than matching text — finds one importer, `tests/test_rule_registry_validation.py`, which is also the only caller of any symbol in the module. The same scan against `src/core/log_paths.py` returns three importers, so the instrument does find an importer where one exists.

```
REGISTRY_PATH = ROOT / "sadp" / "RULE_REGISTRY.json"        # src/core/rule_registry.py
# sadp/ is not in the tree, so REGISTRY_PATH resolves to nothing
```

The module's own docstring says as much. The ids name nothing the running program consults, and they carry no authority.

## Version one of the protocol

Version one was never built here. It named a directory, and the history holds no commit that ever added that directory or anything inside it, the battery file and the registry file included. The version sweep still reads three of those paths: the battery file, the registry file, and a simulator module that is not there either. Run the same query against a file that is real and it answers with the commit that added it, which is the control saying the query works.

`git log --all --diff-filter=ADR --name-only`, one path per run

```
the query finds no commit that added the first two rows
sadp/, and RAIntSimBat.py and RULE_REGISTRY.json inside it   nothing
src/gui/simulator.py                                         nothing
src/core/log_paths.py                       the commit that added it
```

The live file behind the second row is `src/gui/simulator_tab/simulator_tab.py`. Version one's R-numbers were never live and must not be cited as though they were. The Simulator rebuild removed this file; it is not in the tree.

What did ship was a set of references to it, and one test module holds the line against their return. Two front-door documents and the build config may not name the protocol or its battery, no shipped module may import it, and no build may carry a path naming it. Fifty-one annotation comments naming those ids survive in the source across 21 files, and they bind nothing.

`tests/test_no_dead_sadp_references.py` — the seven checks

```
def test_sadp_directory_is_absent() -> None:
def test_build_config_names_no_dead_subsystem(name: str) -> None:
def test_front_door_docs_name_no_dead_subsystem(name: str) -> None:
def test_readme_says_the_harness_was_retired() -> None:
def test_no_shipped_python_imports_the_dead_package() -> None:
def test_no_build_datas_pair_ships_a_dead_subsystem() -> None:
def test_every_shipped_tool_imports() -> None:
```

The module records both directions of its own control: reverting the five files to their earlier state failed four of the seven, and restoring them passed all seven.

## Version two of the protocol

Version two is the system running now. It has three layers, and only the first is in this repository.

### The harness, in the repository

The harness directory holds twelve modules and the shared report contract. Each runs from the command line against one path and reports a `passed` boolean, exiting non-zero when it is false. That boolean needs five conditions, and the first four concern the run rather than the findings: an unhandled file type, an unscanned target, an analyzer that did not report ok, and a recorded error each hold it false. A guard around every subprocess raises on a tool that wrote to standard error and exited non-zero with empty output, which is how an installed-but-unconfigured linter used to report a clean run.

```
python -m dev_harness.harness.<name> <path>

passed                  dev_harness/harness/report.py
refuse_silent_failure   dev_harness/harness/report.py
```

Five archetypes, one per domain:

- `coding_archetype.py` — Python source through ruff, mypy, pyright, bandit, vulture and semgrep, each normalised into one `Finding` schema.
- `docs_archetype.py` — Markdown through proselint and vale, plus a structural check for one H1, no repeated heading text, and a mode signal in the opening 500 characters.
- `gui_archetype.py` — PySide6 widget source through a static AST pass, ruff and bandit. It rejects a colour read back from `styleSheet()`.
- `ta_archetype.py` — indicator source against its published formula, under rule ids TA000 through TA011.
- `watchdog_archetype.py` — the out-of-process crash watchdog, checking that every test pin reaches the one handler.

**Overtaken.** *"`coding_archetype.py` — Python source through ruff, mypy, pyright, bandit, vulture and semgrep, each normalised into one `Finding` schema."*

The bullet above keeps its wording. The coding archetype reads four languages, not one. Python takes the six analyzers the bullet names. JavaScript takes eslint. A shell script takes shellcheck. A YAML settings file takes yamllint. For a file of any other type the archetype answers `unhandled`, which is neither a pass nor a failure.

```
.py .pyi .pyw .spec    ruff, mypy, pyright, bandit, vulture, semgrep
.js .mjs .cjs .jsx     eslint
.sh .bash .zsh         shellcheck
.yml .yaml             yamllint
anything else          unhandled - no analyzer ran
```

The archetype marks an absent analyzer missing, and a report carrying a missing analyzer never counts as a pass.

The gate and the ledger:

- `check_release_readiness.py` — the release gate. It runs the suite, runs each archetype against its good fixture, and reads the claim ledger. It declines to declare a release ready when a step was skipped or when a green pytest run collected nothing. The older copy under `tools/`, and the whole harness directory beside it, are gone from the working tree.
- `claim_ledger.py` — one JSONL row per claim, each row carrying a status. A claim with no measurement behind it stays visible.

A claim holds one of three statuses.

| Status | What it records |
|---|---|
| open | Asserted, no measurement yet. A new claim starts here, and the pre-cascade check returns exit code 1 while one remains |

| Status | What it records |
|---|---|
| `verified` | Evidence attached, as a note and a file list. Only an open claim moves here |
| `refuted` | Evidence disproved the claim. Only an open claim moves here |

Four analysis tools, each answering one question about live behaviour:

- `decision_diff.py` — the live decision diff.
- `manual_fire_frame_check.py` — the Manual Fire target-frame split against live state.
- `reconcile_position_values.py` — displayed position values against their own inputs.
- `ytd_compounding_replay.py` — a venue year-to-date export walked per asset for fold-back opportunities.

Four rule modules run inside the archetypes rather than alone. The first catches a dead path, a dead subsystem name, an import of a module that is not there, a citation past the end of the file it names, and a dotted name split across string literals. The other three carry the numeric, scaffolding and slop families.

```
dev_harness/harness/rules/
    hallucination.py        dead path, dead subsystem, absent import, bad citation, split name
    numeric_guard.py
    scaffolding.py
    slop.py
```

The fixtures directory holds the calibration bodies — a good body and a bad body per archetype. The pair is the control on the gate, and a run of the archetypes that skips the pair proves nothing about the archetypes.

```
harness_fixtures/<archetype>/known_good.<ext>       exits 0
harness_fixtures/<archetype>/known_bad.<ext>        exits 1
```

### The skills, outside the repository

Twenty skills sit under `~/.claude/skills/`, one directory each with a `SKILL.md`. They are machine-local and none is committed, so a reader cannot locate them in this repository. They carry the standing rules that version one put in a numbered registry.

Five carry authority and process.

| Skill | What it governs |
|---|---|
| `harness-law` | Who may write code, and which archetype clears which domain |
| `found-it-own-it` | A defect found is a defect fixed in the same unit |
| `branch-discipline` | Work on a branch, never the tree the operator trades from |
| `unit-decomposition` | Sizing a task into units one pass can finish |
| `close-package` | Building the session close archive |

Four carry measurement.

| Skill | What it governs |
|---|---|
| `ocir` | Observe, calibrate, iterate, repeat, before trusting any zero or pass |
| `two-sided-control` | Driving a check to failure before trusting a pass |
| `job-watch` | Telling a slow background job from a stuck one |
| `log-pruning` | Bounding a log that grows without limit |

Seven carry writing.

| Skill | What it governs |
|---|---|
| `descriptive-comments-only` | A comment states a technical fact or it does not exist |
| `simple-technical-english` | Plain words and short sentences in everything the operator reads |
| `variable-naming-precision` | Short, functional and exact names |
| `docs-narrative` | Narrative shape, citation density, and where the code sits |
| `hyper-refocus` | Filtering a report against the item and the directives |
| `prompt-distillation` | Decomposing a dense request before the work starts |
| `anti-claudism` | The eight recurring failure behaviours |

Four carry the domain.

| Skill | What it governs |
|---|---|
| `acervator` | Project posture and operator identity |
| `ta-canon` | An indicator against its published formula |
| `archetype-peer-review` | Building or extending an archetype |
| `hop-protocol` | The handoff file described above |

### The skills are in the tree now

Two sentences above are overtaken. Each is quoted whole below, and the true sentence follows it.

> *Twenty skills sit under `~/.claude/skills/`, one directory each with a `SKILL.md`.*

Overtaken. Thirty-three skills sit there, one directory each with a `SKILL.md`, and the four tables above name twenty of them.

> *They are machine-local and none is committed, so a reader cannot locate them in this repository.*

Overtaken. All thirty-three are tracked under `dev_harness/skills/`, one directory per skill, so a reader of this repository finds every one of them. The copy outside the repository is the one the Skill tool loads, and the two hold the same text.

The tracked copies end each line with one byte and the loaded copies end it with two, so a comparison of the pair strips carriage returns before it reads the text. Comparing the raw bytes instead names six of the thirty-three as different when no rule in them differs.

### The hooks, outside the repository

Eight hooks sit under `~/.claude/hooks/`, also machine-local and also not committed. They are the blocking layer, and six of them deny a call outright rather than warn about it. - `archetype_gate.py` — in its pre mode, denies a write whose pending content would introduce a high or critical finding the file does not already carry; in its post mode, runs the archetype on what was written and reports the verdict. It blocks a rise, never a level. It routes a shell script and a workflow file to the coding archetype, and for a type with no analyzer it reports that the file was not examined rather than saying nothing. - `block_heavy_run.py` — denies a test run above the memory ceiling, a run while another is resident, and `-n auto` outright. - `block_heredoc.py` — denies a shell command carrying a heredoc. - `block_unanchored_docstring.py` — denies a Python docstring that argues rather than describes: a justification clause, or a sentence naming no identifier from its own file. - `check_directive_drift.py` — denies a dispatch brief that contradicts a pinned directive. - `verify_release_gate.py` — denies a write to either file that carries the version when the release-ready record is absent or older than an hour, and allows every other path through.

The two files that hook protects, and the record it reads:

```
src/__init__.py         __version__ = resolve_version()
main.py                 current_version = _acervator_version
.release_ready.json     written by the release gate, good for one hour
```

Two add context rather than deny, and both are machine-local, not in the tree: `prompt_router.py` names the archetype a task will need, and `session_stop_backstop.py` writes a forensic record of whatever is uncommitted at a session boundary.

The count reads the directory, not a list. Eight names end in `.py` and each of those runs. Six saved copies and one cache directory sit beside them, and the listing marks every entry the count leaves out.

```
~/.claude/hooks/, not committed and not in the tree
    archetype_gate.py                           counted, denies
    block_heavy_run.py                          counted, denies
    block_heredoc.py                            counted, denies
    block_unanchored_docstring.py               counted, denies
    check_directive_drift.py                    counted, denies
    verify_release_gate.py                      counted, denies
    prompt_router.py                            counted, adds context
    session_stop_backstop.py                    counted, adds context
    archetype_gate.py.before_pointer_repair     not counted, a saved copy
    prompt_router.py.before_catalogue_routing   not counted, a saved copy
    prompt_router.py.before_env_conflict        not counted, a saved copy
    prompt_router.py.before_pointer_repair      not counted, a saved copy
    prompt_router.py.before_skill_pointer_fix   not counted, a saved copy
    verify_release_gate.py.before_pointer_repair not counted, a saved copy
    __pycache__/                                not counted, a directory
```

### The hooks are in the tree now

Three sentences above are overtaken. Each is quoted whole below, and the true sentence follows it.

> *Eight hooks sit under `~/.claude/hooks/`, also machine-local and also not*

> *committed.*

Overtaken. Twenty-eight hook files are tracked in `dev_harness/hooks/`, and the same twenty-eight run from the machine profile.

> *The count reads the directory, not a list. Eight names end in `.py` and each of those runs.*

Overtaken. Twenty-eight names end in the Python extension. Twenty-two of them run at an event, and six sit in the directory with no event calling them.

> *The skills and the hooks run, and they are not in the tree — a reader who searches this repository for them finds nothing, and that is the correct result rather than a missing file.*

Overtaken. A reader who searches this repository for a hook finds its source, and `dev_harness/hooks/REGISTRATIONS.md` records the event each one runs at. The skills are tracked the same way, under `dev_harness/skills/`, one directory per skill.

### Where each layer resolves

The harness is in the tree, so every claim above about it resolves to a path a reader can open. The skills and the hooks run, and they are not in the tree — a reader who searches this repository for them finds nothing, and that is the correct result rather than a missing file. The contrast with version one is the point: version one's subject has no source anywhere, in this repository or on disk.

## Related parts

- 09-updates-and-versioning.md — the release gate in full, and where the version number comes from.
- 12-adr-index-and-glossary.md — where decisions are recorded, and the vocabulary this manual uses.

# ADR Index & Glossary

## Decision Records and Glossary

Reference. Where this repository records a decision, and the words the manual uses, each anchored to the module that implements it.

### Where a decision is recorded

Four places hold the decisions this repository has made. Each answers a different question.

- `docs/engineering-notes/` — 21 notes, ten with an evidence directory beside them holding the rows and the script that produced them. One note per question, each naming the standard it measures against and the control beside every measurement.
- `.claude/rules/` — three binding standards, on code comments, on documentation and on tests. `CLAUDE.md` at the root holds the repository rules above them: one role per folder, and no machine-specific path anywhere committed.
- Executable guards in the test suite. One holds a reason for every tracked file at the repository root, and a reason has to name a mechanism. A second holds the retirement of the older protocol, and it records driving itself both ways: reverting the five files it guards failed four of its checks, and restoring them passed all seven. A decision written as a guard fails when someone reverses it, which a decision written as prose does not.
- `CHANGELOG.md` and the commit history. See 09-updates-and-versioning.md for what the changelog holds today.

The two guards:

```
tests/test_repo_root_inventory.py         a reason per tracked root file
tests/test_no_dead_sadp_references.py     the older protocol's retirement
```

`docs/engineering-notes/ci_failure_ledger.md` is the narrowest of these: one row per continuous-integration failure, its class, the change that closed it, and the run that shows it closed. A row leaves the file after three consecutive green runs hold its tests, and the file states its own ceiling — past 20 rows, the work is producing failures faster than it closes them.

### Glossary

Every term below names something in the code, and carries the module or symbol that implements it. The technical-analysis vocabulary is in 07-indicators.md with the twelve published formulae, and is not repeated here.

#### The accumulation cycle

- **Target** — the dollar value a bot aims to hold in an asset. The scrum sells above it and the fold buys back beneath it.

- **Scrum** — a sell of the excess above the target. `ScrummingBot` in `src/trading/scrumming_bot.py` decides and places it.
- **Fold** — the buy back on the dip that follows a scrum, carried by `apply_profit_fold` in `src/trading/profit_fold.py`.
- **Fold, corrected 2026-09-13** — the sentence above names a module that no code ever called, and it is removed. A fold runs on the tranche book: `_fold_eligible_tranches` picks the tranches the fold may reach, and `_apply_fold_target_growth` in `src/trading/scrumming_bot.py` grows the target from the surplus, gated by the bot's own Profit Folding Active row.
- **Interval** — the smallest price move between two fires, in percent. `scrumming_interval_pct` in `src/trading/container/config.py`, default 1.0.
- **Scrum/Fold cycle** — the pair, run against volatility rather than against a price forecast. A completed cycle ends holding more of the asset than it started with.
- **Price floor** — every unit bought back after a fold costs at or under the lot's original price. `_fold_eligible_tranches` in `src/trading/scrumming/tick_phases.py` enforces it.
- **Fold tranche** — one queued slice of scrummed dollars waiting for its buy back. `src/trading/scrumming/fold_tranches.py` holds the book.
- **Merge** — `_top_up_remnant_fold_tranches` folds a new sell's dollars into part-spent tranches, and `_bound_new_fold_tranches` collapses one sell's fresh slice into a single record when the fill arrived as many lots. Both reduce the record count without losing a dollar.
- **Despawn** — `_despawn_aged_tranches` removes a tranche older than the bot's threshold. It removes the record; it does not delist it, and the count lands in the discarded lifetime counter rather than the closed one.
- **Clear** — `clear_fold_tranches` discards every queued tranche and trades nothing.

Merge, despawn and clear are the only three functions that collapse or remove a tranche. 08-tabs/bot-swarm.md describes each in full.

### Bots and what they hold

- **Scrumming Bot** — the accumulation bot, `ScrummingBot` in `src/trading/scrumming_bot.py`. One per traded symbol.
- **Extractor Bot** — `ExtractorBot` in `src/trading/extractor_bot.py`, a chunked state machine rather than a scrum/fold engine.
- **Smart Wire** — a capital reinforcement link between two bots, in `src/trading/smart_wire.py`.
- **Phantom balance** — a bot's view of holdings it may not sell yet, in `src/trading/phantom_balance.py`.
- **Capital claim** — one bot's hold on a quantity of an asset. `effective_available` in `src/trading/capital_reservation.py` answers holdings minus every other bot's claim on the same asset, never the caller's own.
- **Fleet** — the bots a run holds, with their wires, ledgers and tranches. All of it loads from `bot_state.json` through `StateManager`, which is the only source.

The classes behind the wire, the phantom balance and the fleet load:

```
class SmartWireManager: ...     # src/trading/smart_wire.py
class BotLedger: ...
class WireTransaction: ...

class PhantomBalanceManager: ...    # src/trading/phantom_balance.py
class PhantomBot: ...

class StateManager: ...     # src/core/state_manager.py
```

### Deciding a trade

- **Voter** — one indicator's contribution to a direction and a confidence. `VotingEngine` in `src/trading/ta_engine.py` runs the twelve.
- **Gate chain** — the ordered checks a candidate trade passes before it fires, in `src/trading/gate_chain.py`. The sell chain and the buy chain differ, and 07-indicators.md lists both.
- **Heikin-Ashi candle** — a smoothed price bar. `detect_landing_strip_v2` in `src/trading/indicators/landing_strip.py` reads them.
- **Landing Strip** — a tightening detector, `detect_landing_strip_v2` in `src/trading/indicators/landing_strip.py`.
- **Band travel** — the mean-reversion read, `MRInspector` in `src/trading/mr_inspector.py`.
- **Trade grade** — a letter per completed trade, from four scored axes in `src/trading/trade_grader.py`. The market regime is recorded beside them and never scored. See 10-live-trade-history.md.

The gate chain's two classes and the grader's five entry points:

```
class GateChain: ...    # src/trading/gate_chain.py
class GateContext: ...

def grade_trade(record, ctx) -> TradeGrade: ...     # src/trading/trade_grader.py
def _score_execution(...): ...
def _score_timing(...): ...
def _score_strategic(...): ...
def _score_outcome(...): ...
```

### Getting a strategy to real money

- **Promotion pipeline** — Market Inspector, then Simulator, then Paper Trader, then Live. Each step is a gate rather than a mode, and 08-tabs/promotion-pipeline.md holds the entry and exit criteria for each.
- **Gate-latch criterion** — the Simulator's own criterion. It passes when the gates latch identically on the same candles as live, not when profit or trade count matches. See 08-tabs/simulator.md.
- **Stone Tablet** — a stored candle history a Simulator run reads. `src/simulator/tablet_source.py` serves the candles, and `src/simulator/back_test.py` walks them through the live gate chain.
- **Paper Trader** — the step between Simulator and Live, defined by running the same logic against a real-time feed and a fake budget. Built under issue #19, in `src/paper/`. The shelved stock window still defaults to `paper_trader_available: bool = False`. See 08-tabs/paper-trader.md.

- **Proof of Accumulation** — the competition package, `src/competition/`. It holds the bot identity, the Merkle log, the challenge protocol, the token ledger and the local chain. Its two screens are shelved by the operator's own direction. See 08-tabs/proof-of-accumulation.md. The Simulator rebuild removed the fleet-replay files this glossary once named.

### The development harness

- **Archetype** — a domain quality gate under `dev_harness/harness/`, reporting a `passed` boolean.
- **Handoff** — `ACERVATOR_HOP8.md`, measured for drift by `tools/hop_check.py`.
- **Rule id** — an RN name in `RULE_META`, in `src/core/rule_registry.py`. The registry has no production importer and the ids bind nothing.

11-hop-protocol-and-rules-registry.md describes all three.

### The title page

- **Solve et coagula** — dissolve and reform, the sense the epigraph *Turbator aequilibrii dissolvendus reformandusque* carries on 01-title.md. One function letters SOLVE · ET · COAGULA around the outer arc of the harvest trophy, and that is the one place the phrase appears in the code.

```
def harvest_svg(d: TrophyData) -> str: ...      # src/competition/trophy_generator.py
```

## The operator's own words

These name a posture rather than a component. They appear in the manual because the operator uses them, and no module implements any of them.

- **Accumulation trading** — the product category he places Acervator in: harvesting volatility to end holding more of the asset, rather than predicting a direction.
- **Conceptual hopscotch** — his name for a session hopping across a context boundary, which the handoff file exists to carry.
- **Ekthelius the Accumulator** — his handle.

# Recent Updates and Version Tracking

## Version Tracking

Reference. Where the running version number comes from, what each part of it says, and the gate a release passes before any surface carries a new one.

### Recent updates

`CHANGELOG.md` at the repository root is the update record. It follows Keep a Changelog and groups entries under six headings. It currently carries an unreleased heading with no entry beneath it, so outside that file the commit history is the only record of what changed.

The instruction that file gives its own writers, and the one heading it carries:

```
Group entries under `Added`, `Changed`, `Deprecated`, `Removed`, `Fixed` or
`Security`.

## [Unreleased]
```

### Where the number comes from

One resolver answers the version, and no file in the tree writes it down. It runs `git describe` against a tag pattern that admits only a tag beginning with a v and a digit, then turns that output into the reported string. The package initialiser calls the resolver once at import and binds the answer.

```
TAG_GLOB = "v[0-9]*"                        # src/_version.py
def describe(root: str | Path) -> str: ...
def format_describe(text: str) -> str: ...
def resolve_version(root: str | Path | None = None) -> str: ...

__version__ = resolve_version()             # src/__init__.py
```

Three checks hold that shape in place. The first fails the moment the package initialiser gains a written-in number. The second drives a tree whose nearest tag is a local backup name and asserts the resolver skips it. The third drives the same tree without the tag pattern and asserts the backup tag does come back. That last one is the control on the second: without it, a green could mean nothing more than a tree carrying no backup tag.

```
def test_src_init_holds_no_version_literal() -> None: ...       # tests/test_version_resolution.py
def test_a_backup_tag_never_becomes_the_version(tmp_path) -> None: ...
def test_the_unguarded_form_would_adopt_a_backup_tag(tmp_path) -> None: ...
```

**Overtaken.** These two sentences stay whole here:

> *One resolver answers the version, and no file in the tree writes it down. It runs `git describe` against a tag pattern that admits only a tag beginning with a v and a digit, then turns that output into the reported string.*

The resolver declares the release number. The git call no longer decides it. That call matches the one tag the declared release names, and it adds the build count and the commit after a +.

```
RELEASE                                     # src/_version.py, the declared release
RELEASE_TAG = f"v{RELEASE}"
UNRESOLVED_LOCAL = "unknown"
UNKNOWN_VERSION = f"{RELEASE}+{UNRESOLVED_LOCAL}"
```

One commit reports one release number in every state. When the git call cannot derive a build count, the version says so and never takes one from an older tag.

| what the machine holds | what the version carries |
|---|---|
| the release tag | the release, the build count and the commit |
| an older version tag only | the release, the absent word and the commit |
| no tags | the release, the absent word and the commit |
| no git history | the release and the absent word |

## What the string says

The formatter returns a bare release number for one state only: a clean tree sitting exactly on a version tag. Distance past the tag, a modified working tree, and an unreachable version tag each append a PEP 440 local segment after a +. One check asserts every other shape carries that +.

```
def format_describe(text: str) -> str: ...      # src/_version.py

def test_only_an_exact_clean_tag_reports_a_release_number(described: str) -> None: ...  # tests/test_version_resolution.py
```

Two things follow. The number on a working checkout is rarely a release number, and the distance term inside it counts commits, not features. And the version tag the operator's own tree describes from is local to that machine: `git ls-remote --tags` lists an older version tag alone, so a fresh clone resolves a lower release number from the same commit.

**Overtaken.** That last sentence stays whole here:

> *And the version tag the operator's own tree describes from is local to that machine: `git ls-remote --tags` lists an older version tag alone, so a fresh clone resolves a lower release number from the same commit.*

The server carries both version tags. The newer one is an annotated tag, so the listing prints it twice, once for the tag object and once for the commit it points at. A fresh clone reports the release number the tree declares, and the release number no longer moves when a tag is absent.

```
git ls-remote --tags origin

refs/tags/build-<version>
refs/tags/<older version tag>
refs/tags/<newer version tag>
refs/tags/<newer version tag>^{}
```

## The six readers

Six modules read the resolved version, and none of them restates it.

- `src/__init__.py` — calls the resolver once at import and binds the answer.
- `main.py` reads the cached `src.__version__` for its startup log line, the application version and the splash paint, and falls back to the resolver and the baked reader when it checks a bundle against the source tree beside it.
- `tools/spec_common.py` bakes it into a build.
- `tools/build_release_zip.py` reads it for the release archive.
- `tools/capture_live_baseline.py` stamps a captured baseline with it.
- `src/core/version_sweep.py` takes it as the canonical value and reports any literal that shadows it.

## What a frozen bundle carries

A bundle ships no repository, so the git call finds nothing, returns an empty string, and the resolver falls through to the baked file. Two spec helpers resolve the version at build time and write it into a build subdirectory under a fixed file name. The second returns the pair that places that file inside the bundle, where the baked reader reads it back.

```
BAKED_FILENAME = "_baked_version.txt"                   # src/_version.py
def read_baked_version(root: str | Path) -> str: ...

BAKE_SUBDIR = os.path.join("build", "version")          # tools/spec_common.py
def read_acervator_version(project_root: str) -> str: ...
def bake_version_datas(project_root: str) -> list[tuple[str, str]]: ...
```

The frozen check makes a bundle prefer the baked value even when a repository sits around it, so a bundle unpacked inside a checkout reports its own build number rather than the enclosing tree's. Two tests drive it: one with a repository around the bundle, one with git taken away.

```
def is_frozen() -> bool: ...        # src/_version.py

def test_a_frozen_bundle_prefers_the_baked_value_over_a_repository() -> None: ...   # tests/test_version_resolution.py
def test_a_baked_tree_keeps_its_version_after_git_is_removed() -> None: ...
```

### What each variant ships

`datas_candidates` names every source folder a build copies into the bundle. Four folders go into every variant. The Electron shell files go into a React build alone.

| source folder | place in the bundle | qt | react |
|---|---|---|---|
| `src` | `src` | yes | yes |
| `resources` | `resources` | yes | yes |
| `data/historical` | `data/historical` | yes | yes |
| `desktop/renderer` | `desktop/renderer` | yes | yes |
| `desktop/main.js`, `desktop/preload.js`, `desktop/package.json` | `desktop` | no | yes |
| `desktop/node_modules/electron/dist` | `desktop/node_modules/electron/dist` | no | yes |

The renderer folder holds the page host that every React panel reads. The Status tab is a React panel under both builds, so both bundles carry that folder.

```
SHELL_RENDERER = "renderer"                     # tools/spec_common.py
def renderer_candidate(project_root: str) -> tuple[str, str]: ...
def shell_candidates(project_root: str) -> list[tuple[str, str]]: ...
def datas_candidates(project_root: str, variant: str) -> list[tuple[str, str]]: ...
```

`build_graceful_datas` prints and skips a source folder that is not on disk. A clone that never installed the shell therefore builds with no Electron runtime.

## A version that moves while the suite runs

Three test modules compare a freshly resolved version against the cached `src.__version__`. That cached value is computed once, when the package is first imported. The value it is measured against is computed when the assertion runs.

```
tests/test_specs_parity.py
tests/test_version_resolution.py
tests/test_build_variants_and_version_reach.py
```

A commit landing between those two moments changes the distance term, and the two strings disagree. That failure is a property of a git-derived version rather than a defect in the resolver: the derived number moves with every commit and the cached copy does not. Committing to the repository while the suite is running reproduces it.

## The release gate

One command is the gate a release passes before any surface carries a new number. It runs the suite, runs each archetype against its good fixture and requires `passed` true from every one, and reads the claim ledger for open claims. On success it prints a ready line and writes a sidecar record at the repository root. On failure it names the step that failed and exits non-zero.

```
python -m dev_harness.harness.check_release_readiness

[OK] Release-ready (vX.Y.Z, N tests)    printed on success
.release_ready.json                     the sidecar, holding version, tests, timestamp
```

The gate declines to declare a release ready when a check was skipped or when a green pytest run collected nothing. Two tests pin both: one drives the gate with each skip flag and asserts it declines to print the ready line, the other drives a pytest run that collected nothing and asserts the gate calls it a failure.

```
def test_any_skip_flag_refuses_to_declare_ready(...) -> None: ...   # tests/test_check_release_readiness.py
def test_green_pytest_with_zero_tests_is_a_failure(...) -> None: ...
```

The gate module is `dev_harness/harness/check_release_readiness.py`, which is the path that test imports. No copy of it lives under `tools/`.

## What the workflow runs on a pull request

Four lanes run on every pull request, and each one judges something the tree actually holds. One workflow file declares all of them.

```
.github/workflows/ci.yml

changes      diffs the branch and publishes the list of changed files
lint         black and flake8 over src, tests, tools and the root scripts
contracts    the six Solidity tests under tests/contracts, run by forge
archetypes   the owning archetype for each changed file
ci-gate      one status saying whether any required lane failed
```

### Which archetype owns which file

The archetype lane reads each changed file's type and runs the archetype that owns it. A file of any other type is named in the log and counted as unowned, because no archetype reads that type.

```
.py                          coding archetype
.js .mjs .cjs .css .html     GUI archetype
.md                          docs archetype
anything else                unowned, and the log names the file
```

### Shell scripts and workflow files reach the coding archetype

The sentence above is overtaken. Quoted whole:

> *A file of any other type is named in the log and counted as unowned,*
>
> *because no archetype reads that type.*

The coding archetype now reads two further types, and both routers send them to it. Shell scripts go through shellcheck and workflow files go through yamllint.

```
.sh .bash .zsh               coding archetype
.yml .yaml                   coding archetype
```

A type that still has no analyzer keeps the third outcome. The lane names the file and counts it as unowned, and the hook that runs on every save now says the same thing rather than staying silent: this file type was not examined, which is not the same as clean.

### What a green result means

Green means every changed file passed the archetype that owns it, the Solidity tests passed, and the formatting and lint lanes passed. It does not mean a Python test suite ran. The tree holds no Python test.

```
git ls-files tests/                    7 files
git ls-files tests/contracts/*.sol     6 files
git ls-files tests/test_*.py           0 files
```

### The two pytest lanes, overtaken

The release gate section above says this about the two tests that hold it down:

> *Two tests pin both: one drives the gate with*
>
> *each skip flag and asserts it declines to print the ready line, the other drives*
>
> *a pytest run that collected nothing and asserts the gate calls it a failure.*

**Overtaken.** Neither of those two files is in the tree. The workflow used to carry two pytest lanes that read the same empty suite, so both reported a failure on every run, for a reason that was never the change underneath them. Both lanes are gone.

```
pytest -m "not slow and not archetype"    exit 5, nothing collected
pytest -m "slow or archetype"             exit 5, nothing collected
```

### What the branch rules refuse

Two rulesets guard the branches. The first refuses a rewrite or a deletion of either branch. The second refuses a merge into the working branch until a pull request exists and the one status reports green.

```
current   a pull request is required before a merge
current   ci-gate must report success, and the branch must be up to date
current   a red ci-gate leaves the merge blocked, and no one may bypass it
current   a force-push is refused, and a deletion is refused
main      a force-push is refused, and a deletion is refused
main      a direct push is allowed, which is how the branch is synced
```

## How a build is produced

Two files at the repository root are the ones to open, one per variant. A third builds both. None of them holds any build logic: each hands its variant names to one shared launcher.

```
VARIANT = QT                                # Qt_BUILD.py
VARIANT = REACT                             # React_BUILD.py
def parse_variants(argv) -> tuple[str, ...]  # BUILD.py, every variant
def launch(variants: tuple[str, ...]) -> bool    # tools/build_launcher.py
```

The launcher reads the platform it is running on and picks that platform's build script. Windows gets the PowerShell script, macOS gets the shell script, and any other platform is refused by name rather than silently doing nothing.

```
WINDOWS_BUILDER = "build_windows.ps1"       # tools/build_launcher.py
MACOS_BUILDER = "build_mac.sh"
def builder_name() -> str: ...
def build_argv(script: str, variants: tuple[str, ...]) -> list[str]: ...
```

Each build claims a folder name that is not already taken, so nothing in `dist` is replaced and every earlier build stays runnable beside the new one.

```
dist/Acervator-<version>-<variant>/Acervator-<version>-<variant>.exe   Windows
dist/Acervator-<version>-<variant>.app                                  macOS
dist/Acervator-<version>-<variant>.dmg                                  macOS
```

A build that finishes sets the modification date on both single-variant entry points, and a build that fails sets nothing. That date is how a file listing shows whether a build happened.

```
def stamp_builder_dates(finished_at: float) -> list[str]: ...
```

**Overtaken.** These two sentences stay whole here:

> *Two files at the repository root are the ones to open, one per variant. A third builds both.*
>
> *A build that finishes sets the modification date on both single-variant entry points, and a build that fails sets nothing.*

Four files at the root are the ones to open, two per platform, and a fifth builds every variant for the platform it runs on. A finished build sets the date on its own platform's pair and never on the other, so a Windows date and a macOS date report different builds.

```
BUILDER_NAMES = ("Qt_BUILD.py", "React_BUILD.py")   # tools/build_launcher.py
MAC_BUILDER_NAMES = ("Qt_MAC_BUILD.py", "React_MAC_BUILD.py")
def stamp_builder_dates(finished_at, names=BUILDER_NAMES) -> list[str]: ...
```

### The macOS build runs on a runner, not on the operator's machine

The operator's machine runs Windows, so no macOS build can be produced there. A workflow builds both macOS variants on a GitHub macOS runner instead.

```
# .github/workflows/macos-build.yml
name: macOS build
runs-on: macos-latest
run: ./build_mac.sh --dmg
```

Two things start it: a push that changes one of the files the macOS build reads, and the Run workflow button on the repository's Actions tab.

```
on:
  workflow_dispatch:
  push:
    paths:
      - build_mac.sh
      - Acervator_mac.spec
      - tools/build_launcher.py
      - tools/build_variants.py
      - tools/spec_common.py
      - .github/workflows/macos-build.yml
```

**Overtaken.** That sentence stays whole here:

> *Two things start it: a push that changes one of the files the macOS build reads,*
>
> *and the Run workflow button on the repository's Actions tab.*

A third thing starts it, and it is the one the operator uses: a Mac builder at the repository root, double-clicked. Each one asks for a run on the branch his copy is on, or joins a run already going on that branch rather than starting a second one, because the workflow cancels an earlier run of the same group.

```
VARIANT = QT                                     # Qt_MAC_BUILD.py
VARIANT = REACT                                  # React_MAC_BUILD.py
def launch_macos(variants: tuple[str, ...]) -> bool   # tools/build_launcher.py
```

The run does not stop at a finished build. It reads the size of every bundle and every disk image, mounts each disk image and compares the application inside against the one that was built, then launches each application and reads the line the program writes to its own log once the main window is on screen.

```
log_manager.info("Application ready — main window displayed")   # main.py
```

### Where the macOS build is downloaded

The finished run page carries an Artifacts section. One entry holds both applications and both disk images as a single zip file; a second entry holds what the two launches produced, which is the evidence that each application started.

```
acervator-macos                  both .app bundles and both .dmg files
acervator-macos-launch-evidence  each launch's own log and screen capture
```

The download is a zip file. Unzipping it gives the two applications and the two disk images; dragging an application to the Applications folder installs it.

**Overtaken.** That last sentence stays whole here:

> *The download is a zip file. Unzipping it gives the two applications and the two*
>
> *disk images; dragging an application to the Applications folder installs it.*

The run page artefact remains a zip holding both bundles, and it no longer tells a person where to go. A release carries each disk image as a file of its own, so a visitor downloads one disk image and never a zip.

```
a run page artefact   one zip holding both bundles and both disk images
a release asset       one disk image, downloaded on its own
```

The operator downloads neither by hand. A Mac builder waits for its run, prints the elapsed time while it waits, then puts that variant's application and disk image straight into `dist` at the same level as the Windows folders, with no folder around them.

```
dist/Acervator-<version>-<variant>.app    what a Mac builder brings back
dist/Acervator-<version>-<variant>.dmg    its disk image, in the same folder
```

On a Mac the disk image is the one to open. A bundle's internal links become copies inside the zip the run page serves, so the folder arrives without the permissions a Mac needs while the disk image keeps them. A builder that cannot reach GitHub prints which of the two reasons stopped it, and starts nothing.

```
NOT_SIGNED_IN_NOTICE   # tools/build_launcher.py, gh answered exit 4
NO_ANSWER_NOTICE       # every other non-zero answer from gh api
```

### What the first launch looks like

The bundle carries no Apple Developer signature, so macOS refuses the first double-click. The way past it is to right-click the application and choose Open, which is needed once per build and not again.

```
------------------------------------------------------------
  IF macOS REFUSES TO OPEN THE APP:
  1. Right-click (or Control-click) the .app
  2. Choose Open
  3. Choose Open again in the dialog
------------------------------------------------------------
```

Signing removes that step and is not set up. It needs a paid Apple Developer account, a Developer ID Application certificate, and that certificate held as a repository secret. The build script already accepts the identity.

```
./build_mac.sh --sign "Developer ID Application: <name> (<team id>)"
```

## Where a person downloads the application

A release carries the built applications outside the repository. A visitor installs no Python, no dependencies and no builder, and the Releases panel on the repository front page links it. Git cannot hold the builds: `dist` measured 12,856 MB.

```
name: Release                       # .github/workflows/release.yml
on:
  workflow_dispatch:
jobs:
  windows: {runs-on: windows-latest}
  macos: {uses: ./.github/workflows/macos-build.yml}
  publish: {needs: [windows, macos]}
```

The Run workflow button on the Actions tab starts it. One run builds both platforms, launches every application it built, reads the line each one writes once its main window is on screen, and attaches four files to one release.

### The release a visitor downloads

Four files, one per platform and per variant, each named after the version and the commit the build resolved.

```
Acervator-<version>-qt-windows.zip      Windows, the Qt interface
Acervator-<version>-react-windows.zip   Windows, the React interface
Acervator-<version>-qt.dmg              macOS, the Qt interface
Acervator-<version>-react.dmg           macOS, the React interface
```

The tag carries that same version behind a prefix the version reader's own tag glob rejects, so publishing a release cannot move the version the next build resolves.

```
TAG_GLOB = "v[0-9]*"                # src/_version.py
build-<version>                     # a release tag, which that glob rejects
```

A runner resolves that version from the tags the remote carries, and the newer version tag is not one of them, so a runner build and a local build of one commit report different numbers. The commit is the same in both, and every file name carries it.

```
git ls-remote --tags origin    the older version tag, and the build- tags
a local clone                  the older and the newer version tag
```

Pushing the newer tag would change the number every runner build reports, which the release cascade above governs.

**Overtaken.** These two passages stay whole here:

> *A runner resolves that version from the tags the remote carries, and the newer version tag is not one of them, so a runner build and a local build of one commit report different numbers.*
>
> *Pushing the newer tag would change the number every runner build reports, which the release cascade above governs.*

The remote carries the newer version tag. A runner build and a local build of one commit report the same release number, because the resolver reads the release the tree declares and not a tag name. A tag that a runner cannot see now changes the build count alone, and the version names that count absent rather than counting from an older tag.

The publish step refuses a set whose file names do not all carry one version, and the Windows step refuses a build folder holding any logo file. Neither guard has a way to pass a mixed or a logo-carrying release.

```
files to attach: 4          two per platform, or the step fails
qt  logos    0              any other number fails the step
```

### The first launch on a Windows computer

Windows refuses the first run of an unsigned program. Two clicks pass it, once per download and not again.

```
------------------------------------------------------------
  IF WINDOWS BLOCKS THE EXE:
  1. Click "More info" then "Run anyway"
------------------------------------------------------------
```

That notice is the one the builder prints after a local build, so a downloaded build and a locally built one show the same step.

```
SMARTSCREEN_NOTICE                  # tools/build_launcher.py
```

A macOS download carries the same unsigned bundle a run page artefact carries, so its own first launch is the Gatekeeper step recorded above.

Part 8

# The Development Chronicle As Told By Claude

## The Development Chronicle

This is how Acervator got built. It runs from 10 April 2026 to 8 September 2026.

None of it is written from memory. Memory is the first thing this project stopped trusting. Every date below sits on an artefact — a commit, an issue, a file, a comment somebody dated. Where no artefact exists, the entry says so and stops there.

```
docs/audits/2026-09-08_development_map.md
    125 entries    120 artefact-recorded, 3 inferred, 2 gaps
    152 days       61 of them carry no usable record
    largest blank  27 June to 23 July
```

Sixty-one blank days out of a hundred and fifty-two is a poor record, and I am not going to dress it up. Those days are blank in the paperwork, not in the work. Read the gaps as gaps.

### How this chronicle was built

Seven sources carry the story, and each one has a reach and a limit. Nothing before 18 August is in the commit history. Four months arrived as a single import, so the first third of this chronicle is reconstructed rather than replayed.

```
version numbers written into code comments      20 April to today
the pre-git build archive                       10 to 24 April
the archived session chronicle                  11 April to 9 June
six handoff files at the repository root        18 April to 3 September
dated audit units inside the first commit       24 July to 15 August
the narrative changelog                         4 to 25 August
git log and the runtime directory               18 August to today
```

One source was tried and thrown out. File dates in my own working copy put five package roots on 23 April. A fresh checkout of the same commit gives those files today's date, because checkout writes them. That is a fact about one disk. It is not a fact about this project, so nothing here rests on it.

### 2026-04-11 — five days inside a chat window

We started in a chat window. Not an editor, not a terminal — a chat window with no access to the file system. Every session began by uploading a zip of the whole codebase, because the session could not read a file on its own.

Seventeen conversations, five days, about 27 MB of transcript. Out the far end came 74 Python files and roughly 34,000 lines. Day one produced a grid bot, then the accumulation engine, then a seven-indicator voting panel, the Phantom Balance, a window and a build script. That is not a normal first day and I know it.

```
docs-archive/llm-session-history/DEVELOPMENT_CHRONICLE.md
    lines 13 to 26     session one
    lines 304 to 308   ~27 MB, 74 files, ~34,000 lines
```

## 2026-04-12 — scrum and fold were the wrong way round

Our whole method is one cycle. Sell the excess above a dollar target, buy back more of the asset on the dip. Do that enough times and you end up holding more than you started with.

On day two we found the two halves inverted. Our engine was selling where it should have bought. Its own record calls this the most consequential defect in the project, and I agree with that. Get the direction wrong and every other thing you build is a faster way to lose.

```
archive chronicle lines 90 to 101   scrum and fold inverted, corrected
src/trading/otd_math.py             where the fold arithmetic lives today
```

## 2026-04-15 — the name

For the first four days this was not called Acervator. Sixteen builds stored on 11 April carry an earlier product name. Last build under the old name is stamped 14 April at 22:03. First build stamped Acervator reads 15 April at 01:35.

Acervator is Latin for one who heaps up. That is the whole product in one word. Our code folder kept the old name for another two days. That only bothers the person who renamed it.

```
build archive folder listing   old name through 14 April 22:03
build archive folder listing   Acervator from 15 April 01:35
src/core/event_bus.py          the name on the copyright line of every source file
```

## 2026-04-16 — the handoff, invented on the fly

Here is the problem a chat window gives you. Every session starts with nothing. No memory of the last one, no vocabulary, no idea which decisions are settled. You re-explain the entire product, and then you watch it re-litigate a thing you closed a week ago.

We wrote a file. One markdown document at the root of the project, carrying the vocabulary, the invariants and the standing rules forward into the next session. Read it first, then work. That is the whole protocol, and it was invented in an afternoon because the alternative was losing a day a week.

```
build archive package stored 16 April 14:59
    a 38,903-byte protocol document, a template, two readme files
ACERVATOR_HOP2.md    the earliest handoff still in the tree
tools/hop_check.py   what measures handoff drift today
```

That same release introduced numbered rules. Whenever a class of silent defect got fixed, we wrote a rule, and no rule was ever deleted. By the next day there were twenty-seven of them in six groups, because an ungrouped list of rules is a load nobody can hold in their head.

That habit did not survive contact. On 18 June I ruled that no part of the protocol built around those rules was valid or working. Two modules totalling 2,586 lines had been built to serve it. Nothing imports either one today.

```
src/core/rule_registry.py     every rule locked by default
src/core/version_sweep.py     built for the protocol, imported by nothing
```

## 2026-04-19 — the version went backwards and nobody wrote it down

In twelve hours the engine version reads 3.14.0, then 3.19.0, 3.21.0, 3.24.0 and 3.25.0. At 20:15 the same day it reads 3.9.0.

Why it went back is **not recorded**. No artefact states it. Both handoffs written that day and the next carry 3.9.0, so the lower number is the one the project kept, and that inference is the whole of the evidence. This entry is inferred and I am marking it inferred.

```
six archived builds, version read from each build's own entry point
ACERVATOR_HOP3.md and ACERVATOR_HOP4.md    both at 3.9.0
```

## 2026-04-20 — ten reviewers, four of whom would not give it money

Ten named reviewers went over the trading logic. Six called it work in progress. Four said plainly they would not trust capital to it.

They were right. Live decisions at that point used Bollinger Bands alone, at a fixed two per cent delta, across fifteen bots. Twelve indicators vote on a trade today, and none of that existed yet. That review proposed seven rules of its own and not one of the numbers survived; the first was claimed by an unrelated rule the following day.

```
docs-archive/llm-session-history/ACERVATOR_DEPT_LEAD_REVIEW_v3_12_0.md
    lines 13 to 26     the ten verdicts
    lines 364 to 368   Bollinger Bands alone, fifteen bots
```

## 2026-04-26 — three weeks I cannot account for

From 29 April to 18 May there are two dated points and nothing else. Our session chronicle jumps the span. Handoff jumps it too. Build archive stopped on 24 April. Sixteen of those twenty days are **unrecorded**.

What I can say is that three whole minor version series passed through that window. That makes it the fastest-moving stretch in the record and the worst documented, which is not a coincidence.

Two things are dated inside it, both from directives quoted in the running code. Venue is the authority on position health, and nothing recomputes locally what the exchange already reports. Our grid bot was deleted somewhere in here, and no commit ever added it, so a comment about a removed table is the only proof it existed.

```
src/exchange/position_health.py    the venue is the authority
src/exchange/timeframes.py         timeframes follow what each venue offers
```

## 2026-05-20 — four hundred gigabytes

Our log directory grew to about 400 GB and crashed the machine.

Cause was ordinary and the shape of it is worth keeping. Per-run file rotation had been added weeks earlier, capping each run at 400 MB. Nobody capped the number of runs. Every crash and restart wrote a fresh post-mortem bundle of a few hundred megabytes, and they accumulated for weeks until the disk went.

Three retention limits were written the same day. Bundle count, bundle age, and a size warning printed at startup so the next one is visible before it is fatal.

```
acervator_watchdog.py
    POSTMORTEM_KEEP_LATEST     most recent 20 bundles kept
    POSTMORTEM_MAX_AGE_DAYS    anything older than 30 days dropped
    POSTMORTEM_SIZE_WARN_BYTES 5 GB, reported on startup
src/core/log_paths.py          the one labelled root every log writes under today
```

That same day carried twenty entries. A declarative gate chain replaced the checks that had been scattered through the engine, and twenty-four standing test failures closed. Our Extractor skeleton and its four-state machine landed in the same stretch.

```
src/trading/extractor_bot.py   the four-state machine, still where it started
```

## 2026-06-05 — an address I never had

A session needed the project's source repository address. Instead of asking, it built one out of my email handle and shipped it.

That invented address landed in 25 files and in a package published to PyPI — the public index Python installers download from. Publishing is one way, so the wrong address could not be withdrawn; the only remedy was a version bump and a fresh upload. One good lesson came out of it. Do not infer an identifier. Ask, leave it blank, or fail loudly.

```
docs-archive/llm-session-history/DEVELOPMENT_CHRONICLE.md
    lines 18585 to 18587    fabricated address, 25 files, one published wheel
```

The same day carried a second one, and it is the better story. A screen defect was reported fixed, then reported fixed again, and it was not fixed either time. Its tests searched the source file for words and found them. They could not see that the words were wired to the wrong signal. The real repair came with six tests that drove the widget, and those tests were deliberately broken first — five of six failed with the exact symptom I had reported.

That is the difference between a test that proves something and a test that agrees with the file it is reading.

## 2026-06-27 — twenty-seven days of nothing

No commit. No handoff. No audit. No runtime file. No dated comment. No artefact of any kind falls inside it.

Twenty-seven consecutive days, the largest single blank in this record. Our own record does explain where it went: the changelog for that period was moved into an archive file during the 3.23 series and never replaced at the root, so the releases after it shipped with that step skipped. That archived file was never committed here. A history query for it across every branch returns nothing, against the same query for a known file, which returns its commit.

Two engine version series, 3.21 and 3.22, appear nowhere in the tree. Not one comment names either. They sit inside this blank.

## 2026-07-24 — the summer that lives in a commit

Everything from late July to mid-August survives in exactly one place: the first commit. Inside it sits a folder of dated audit units, one per piece of work, and it is the densest record of the whole pre-git period.

That folder has had a bad life. It was emptied on 26 August, restored on 27 August into a different directory, and lost again on 28 August inside two merges that reconciled two branches. A plain delete query cannot see the second loss, because the deletion happened in a merge.

Nineteen of those notes are in the tree today. The rest are still readable, because the commit that holds them is an ancestor of the current branch.

```
git show fc0d7788:docs/engineering-notes/<name>
    115 notes at that commit, 19 in the tree, 102 absent
    25 of the absent files carry a July date — 24 notes and one screenshot
```

I have not restored them. They are recoverable, they are dated, and this chronicle cites the commit rather than pretending the period is lost.

What that summer contains is the part of the project I am proudest of. Nine audits in one day walking the wizard panels. Design proposals for settlement across base currencies, the Market Inspector and the phantom bots. Fourteen audits on 6 August, almost all of them defects in the tranche lifecycle. Twenty-four releases on 7 August, each closing one numbered defect.

## 2026-08-01 — the archetypes

Three rules went in on one day, and they changed how everything after them got built. A cited path that does not exist on disk. A placeholder shipped as finished work. A duplicated block, an oversized function, a file nobody can read.

Those three grew into four review archetypes — one for code, one for arithmetic, one for the interface, one for documentation. An archetype is the authority in its domain. It reports a verdict, and the verdict is not a delta and not an opinion. This page you are reading was refused by the documentation archetype until it passed.

```
dev_harness/harness/rules/hallucination.py   a citation that resolves to nothing
dev_harness/harness/rules/scaffolding.py     a stub shipped as if finished
dev_harness/harness/rules/slop.py            duplication, oversized functions
dev_harness/harness/docs_archetype.py        what judged this page
```

Two rules of mine landed on 9 August and reshaped the rest. One entity writes arithmetic. One entity writes code, and it must check all of it adversarially. The day after, a rule against wiring anything to dead code; the release that followed deleted 457 lines and changed no behaviour at all.

## 2026-08-18 — four months arrive as one commit

650 files. 315,743 lines. 239 test files. Four archetypes already built. One commit.

Nothing dated before 24 July shipped in that import, apart from three interface backup files from June. The history that arrived is an upload log, not a development history — there is nothing to bisect, nothing to blame, nothing to revert to.

```
6e4f46b8   the import, and its twin be6aa048
git show 6e4f46b8:ver.txt   reads 3.25.5, three patches behind the module beside it
```

## 2026-08-22 — the first outside eyes

An outside audit filed issues 66 to 96. First review by anyone but me and the machine I was building with.

Its first finding was the one above: the history offers no bisect, blame or revert trail. It also found a fifteen-thousand-line engine file, an eight-thousand-line window file, five hand-copied dependency lists with no lockfile, a build that did not work, and committed backup copies sitting in the source tree. Every one of those was true.

Three days later a second engineer reformatted 464 files and moved all testing into the build service. That sweep also moved code anchors and emptied five test modules, both repaired the same day. Ten days after that the build service hit a billing wall and stopped running altogether, so a local replacement was written.

```
issue #66    git history is an upload log, not development history
issue #128   convert the interface from the old toolkit to React
issue #331   local build-service equivalent
```

## 2026-08-28 — the version restarts below one

First version tag in this project's life reads 0.1.0.

Four months of a private version ladder had already gone past three. Under git, the number is derived from the tag rather than typed into a file, and the tag started where a new project starts. I find that funny and I am leaving it in.

```
src/_version.py    derived from the git tag, or from a frozen build's baked value
ef105f55           the version stops being written and starts being derived
```

## 2026-09-06 — sixteen thousand seven hundred and eighty-eight tests

The suite reached 16,788 tests. Nobody could reason about it.

That is not a boast about coverage and it is not a complaint about slowness. It is a structural failure. Measured across 529 test files, 485 defined their own helpers and only 148 imported the shared ones. One helper name was rewritten in 72 separate files. Every rebuild rediscovered the same traps, and several of the rebuilds were themselves broken.

Then we read what they actually asserted. 267 of them opened the source file of the code under test and pattern-matched the text. 72 more carried hand-maintained registries of constants copied out of the source. Those tests cannot fail when the behaviour breaks, and they cannot survive a refactor that moves a line. They pass while the product is wrong, which is worse than having no test at all.

```
issue #411   222 of 525 test files assert on source text instead of behaviour
issue #330   three tests read source text and make comments load-bearing
05449360     the removal branch, not merged into origin/current
```

Verification moved to the debugger and the archetypes. A program runs, the debugger reports what it did, and an archetype owns the verdict.

## 2026-09-07 — a known defect, written down and left there

This is the worst one, and it is the reason this chronicle is worth writing.

The exchange connector fetches candles. Its docstring carried the words KNOWN DEFECT, followed by an accurate description of what was wrong. A comment two lines below said live callers land here, with the positional quirk documented above. Somebody found it, understood it, wrote it down correctly, and left it in place.

Underneath that comment, the count of candles was being passed into the library's start-time slot. That library filled the empty count slot with its own ceiling and kept rows from the start of the venue page, dropping the fifty newest. Every live indicator therefore read a candle window fifty five-minute bars behind the price. Four hours and ten minutes stale, on all 38 running bots, while the price beside it was five seconds old.

```
src/exchange/ccxt_connector.py   the docstring that named it and left it
docs/audits/2026-09-07_ta_window_lag.md
    141 of 141 recorded rows reproduce at an offset of -50 bars
    no other offset and no other timeframe fits better
b106df73   -50 bars on 1,215 pages before, 0 on 1,215 after
           the band latch differs on 88 of 162 recorded gate rows
```

Eighty-eight of a hundred and sixty-two decisions were made on the wrong window. A comment is not a fix. Writing a defect down and shipping it is the same as not knowing, except that it looks diligent.

## 2026-09-07 — the ninetieth percentile that was the maximum

An archive of simulation results arrived as source material for a new Simulator mode, and it looked wrong to me. It was.

Underneath, a real trading loop. One pass per candle, buy and sell branches, fees, a spread, a running ledger. Nobody typed the results in by hand. But the prices that loop ran on came from nowhere real: a start price, a middle price and an end price, joined by a smooth curve, with a random wiggle on every candle. Three numbers producing a year of hourly data, 8,760 candles at a time.

Then the headline figure. Ten runs per configuration, sorted, and the ninetieth percentile taken at index nine — the last item. At ten samples that index is always the maximum. Every reported ninetieth percentile equals the largest run, in 39 of 39 rows, across three separate reports.

```
docs/audits/2026-09-07_raintsimbat.md
    p90 = run_advs[min(n_runs-1, int(n_runs * 0.90))]
    39 of 39 rows: the ninetieth percentile equals the largest value
    all ten runs of a configuration share the same three anchor prices
```

All ten runs also shared those three anchors, so the spread between them measured the random wiggle and not market risk. Nothing from that archive entered this repository.

## 2026-09-08 — a producer that never read half its own manual

The manual you are holding is built by a tool that renders markdown into a PDF and then checks its own output.

Its manifest reader walked one table and one directory level. Thirteen of the manual's twenty-seven sections sat one level down, listed correctly in the contents and never rendered. The producer validated every page it built and was silent about the pages it never opened.

```
a8e7439c   269 pages to 346, 14 sections to 27
tools/build_product_manual.py   read_manifest now reads every table row
```

A check that only inspects what it already loaded is not a check. That is the same shape as the tests above and the same shape as the docstring, three times in three days, in three unrelated places.

## What the record does not hold

Six spans, sixty-one days, and here is what was looked at for each.

| Span | Days | What was looked at |
|---|---|---|
| 29 April to 5 May, 7 to 9 May, 11 to 18 May | 16 | Session chronicle and handoff both jump the span. Build archive ended 24 April. Two dated directives inside it, nothing else |
| 26 to 29 May | 4 | Absent from the archive histogram and the handoff entry series, between two dense weeks |
| 2, 3 and 7 June | 3 | Single days missing from an otherwise continuous run |
| 17 June, 19 to 25 June | 8 | Between two dated code comments, with nothing in between |
| 27 June to 23 July | 27 | Changelog moved to an archive file never committed here; the audit folders begin on 24 July |

| Span | Days | What was looked at |
|---|---|---|
| 29 July | 1 | One day between two dated audits |

The record also disagrees with itself in seven places, and I have not resolved any of them. Picking a side would put an invented fact in a document whose only claim is that it contains none.

Two are worth naming. One handoff states that an earlier handoff never existed; that file is in the tree at 11,445 lines and was in the very first commit. And the same handoff blames a size past 100 KB for killing the format, while the file it names is 712 KB — six times the threshold it says was fatal.

Session numbers are not a clock either. One handoff runs through sessions 18 to 22, then 24, then 26, then jumps to 72, then returns to 26. Dates order this record. Session numbers do not.

## What the mistakes bought

Read the entries above in order and the pattern is one thing, five times. A check that agrees with the thing it is checking.

```
tests that read the source          agreed with the file, not the behaviour
a docstring naming a defect         described it correctly, changed nothing
a ninetieth percentile of ten       reported the maximum and called it a tail
ten runs on three anchor prices     measured the noise, not the market
a producer validating its own build never opened half the pages
```

Everything durable in this project came out of that pattern, not out of the features. The archetypes exist because a green result was believed once too often. The handoff exists because a session with no memory re-litigates settled work. The rule that everything found is owned by whoever found it exists because eleven failures were once triaged into one mine and ten pre-existing, and the ten stayed broken.

I would rather ship a document that says sixty-one days are missing than one that quietly fills them in. Filling them in is exactly the failure this whole apparatus was built to catch.

## The thread underneath

The manual opens on a line of Latin, and it is not decoration.

> *Turbator aequilibrii dissolvendus reformandusque.*
>
> *The disturber of the balance is to be dissolved and reformed.*

That is *solve et coagula* — dissolve and recombine, the operation at the centre of hermetic practice. It is also, precisely, what the engine does to a position. Excess above the dollar target is dissolved into cash. Cash reforms into more of the asset on the dip. Every completed turn of the cycle ends holding more than it began with, and nothing about it requires knowing which way the price will go next.

The competition tiers carry the same thread, and I chose them deliberately. Five ranks, named for the stages of the Great Work.

| Tier | Stage | Supply |
|---|---|---|
| Harvest | Nigredo — the darkening | unlimited |
| Gold Fold | Albedo — the whitening | unlimited |

| Tier | Stage | Supply |
|---|---|---|
| Bear Slayer | Citrinitas — the solar dawn | 10,000 |
| Grand Accumulator | Rubedo — the great work | 1,000 |
| Ekthelius | Unio Mystica — return to the All | 21 ever |

```
src/competition/season_schedule.py    the five tiers and what earns each one
src/competition/trophy_generator.py   the stage, the motto and the colour
```

Twenty-one ever, and it takes a perfect score across every metric in a season. Nobody has one. I would be surprised if anybody ever does, which is the point of putting a ceiling on it.

The name at the top of that ladder is mine, and so is the one on the copyright line of every source file. Acervator — one who heaps up. What the software does is heap up, one dissolved and reformed position at a time, and this chronicle is the record of how badly and how well we learned to do it.

Part 9

# Live Trade History and Supporting Evidence

- VWAP Charts - YTD Data Analysis and Trade Grading - Deep Indicator Function Investigations Via Trade Logic Gate Logs

## VWAP Charts and Trade Grading

Reference. What a year of live fills measures, how the volume-weighted average price is drawn from it, and what the platform's own grading and gate logs add to a fill.

The fill record is the operator's own venue export. It is not committed to this repository and no row of it is reproduced anywhere in this manual. Every figure below is an aggregate over that record.

### The record behind the charts

The export holds 5,709 physical rows and 5,705 body rows. 5,661 of the body rows are fills.

Body rows by transaction type: Advanced Trade Buy 3,189; Advanced Trade Sell 2,472; Reward Income 16; Deposit 15; Buy 8; Sell 4; Withdrawal 1.

44 body rows carry no fill, and each one is accounted for by name: 16 Reward Income, 15 Deposit, 12 USDC cash legs of a trade priced elsewhere, and 1 Withdrawal. Nothing is dropped without a reason that says what it was.

40 distinct assets appear in the record. USD and USDC are quote currencies, which leaves 38 charted bases, one for each bot in the fleet, and each of the 38 has fills of its own.

Fills per asset, most to fewest: A01 900, A02 495, A03 459, A04 322, A05 261, A06 230, A07 226, A08 220, A09 196, A10 191, A11 178, A12 159, A13 147, A14 141, A15 124, A16 124, A17 119, A18 119, A19 113, A20 87, A21 83, A22 80, A23 77, A24 75, A25 74, A26 73, A27 73, A28 66, A29 63, A30 58, A31 25, A32 24, A33 23, A34 23, A35 16, A36 6, A37 6, A38 5. Those 38 counts sum to 5,661, the fill total above.

### Subtotal is the price

A row carries both a Subtotal and a Total, and only one of them belongs in a VWAP. On every row the absolute quantity times the price equals the absolute Subtotal to within 5e-6. Total is that same figure plus the fee.

A fee is a transaction cost, not a price. Fee drag measures between 0.92 and 1.20 percent per asset, so a VWAP built on Total would sit roughly that far above the price the venue actually filled at, on every buy in the record.

## How a VWAP is computed here

The running figure is a streaming sum of price times quantity over a streaming sum of quantity, both carried as floats. The price is the export's Price at Transaction column. The generator never reads Subtotal, and it does not need to. The section above measures quantity times price against Subtotal and finds the two equal to within 5e-6.

Run against the same rows with Python's exact `Fraction`, the two paths agree to 3.63e-15 across 26 assets, so the float path loses nothing a chart can show.

One control separates a weighted mean from an unweighted one. One unit bought at $10 and three at $20 give 17.50 by hand, and both the float path and the `Fraction` path return 17.50. An unweighted mean of the two prices returns 15.00. An implementation that forgot to weight by quantity would open a gap of 2.50 on that case rather than pass unnoticed.

## What the combined chart plots

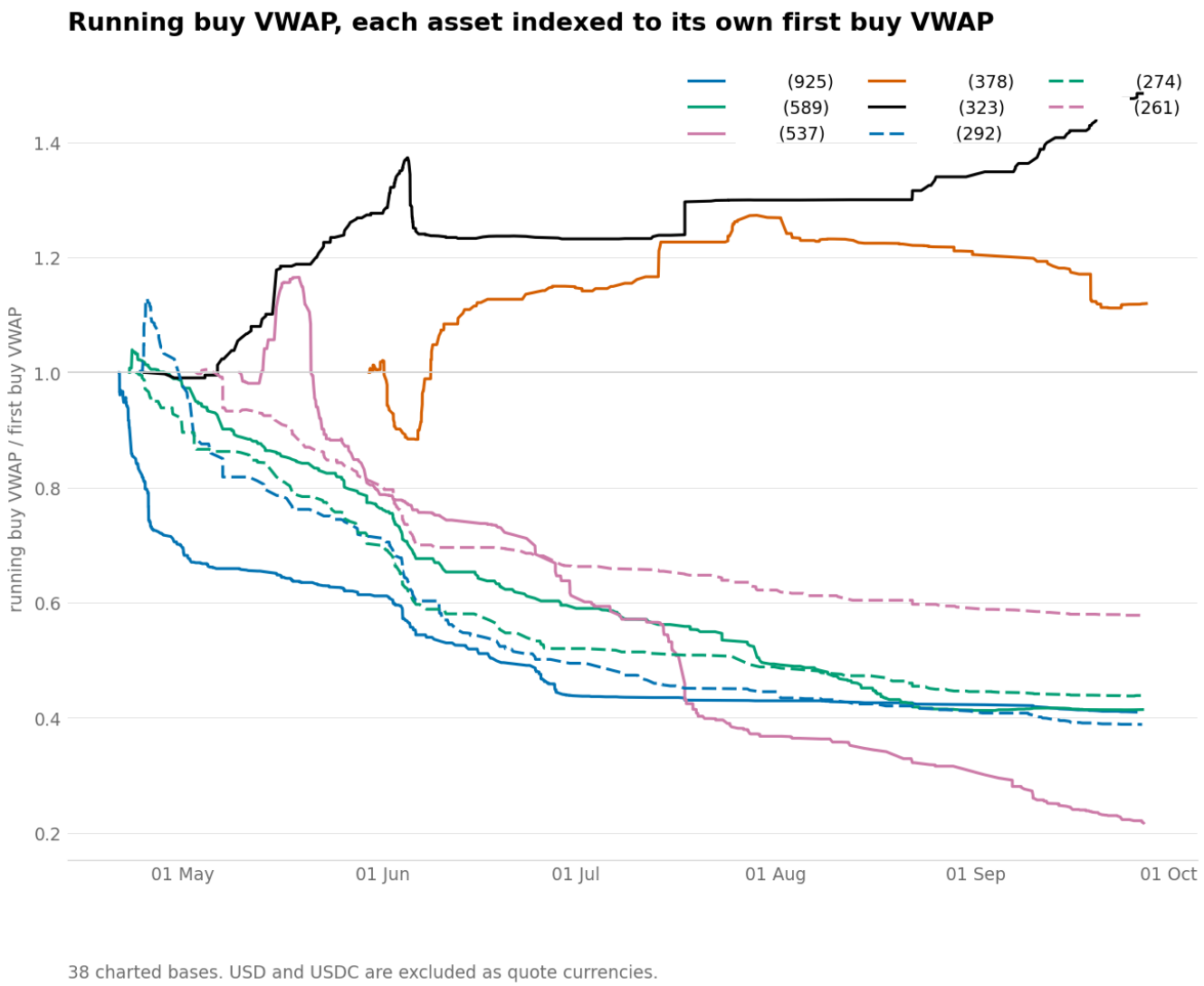
Each asset has a chart of its running buy VWAP against time. The line is the trajectory of the average as accumulation proceeds, and the shape of that trajectory is the reading.

Two metrics run through this part and they answer different questions. The trajectory answers what the units now held cost on average, and it keeps moving while the position is open. The sell-over-buy ratio answers whether the trades already closed sold above what they cost, and it is one settled figure per asset across the whole record. One reads the position, the other reads the realised trades. This part carries both, and neither stands in for the other.

The combined chart indexes every asset to its own first buy VWAP and plots the running buy VWAP divided by that first value, so an asset priced in cents and an asset priced in thousands share one axis and one scale.

Of the eight assets with the most fills, six trend down to between 0.4 and 0.6. A05 and A04 trend up.

Read the line as what it is. A falling running buy VWAP says the average cost of the base units now held fell as more were bought. It is not profit. It does not become profit until units are sold, and the index carries no sale in it at all.




The running buy VWAP of the eight assets with the most fills, each divided by its own first buy VWAP.

The eight lines are the eight assets with the most fills. Each one is that asset's running buy VWAP divided by its own first buy VWAP, which puts A10 at 5.71e-06 and A05 at 359.67 on one axis and one scale. The rule at 1.0 marks the first value. The legend names the asset and its fill count in three columns. The note under the axis reads:

```
38 charted bases. USD and USDC are excluded as quote currencies.
```

Five pens carry the eight lines. The chart takes the series palette from `src/design_system.py` and keeps only the colours reaching a contrast ratio of 3.0 against the page background. Five of the eight qualify. The five run solid for the first five assets and dashed for the last three.

| Series colour | Contrast on the page background | In the chart |
|---|---|---|
| #000000 | 21.00 | yes |
| #0072B2 | 5.19 | yes |
| #D55E00 | 3.87 | yes |

| Series colour | Contrast on the page background | In the chart |
|---|---|---|
| #009E73 | 3.42 | yes |
| #CC79A7 | 3.06 | yes |
| #56B4E9 | 2.31 | no |
| #E69F00 | 2.25 | no |
| #F0E442 | 1.32 | no |

Measured over the same 5,661 fills, the eight lines end here:

| Asset | Fills | First buy VWAP | Final buy VWAP | Final index |
|---|---|---|---|---|
| A01 | 900 | 1.5915 | 0.672605 | 0.423 |
| A02 | 495 | 0.10842 | 0.044733 | 0.413 |
| A03 | 459 | 0.11868 | 0.035741 | 0.301 |
| A04 | 322 | 0.2478 | 0.298528 | 1.205 |
| A05 | 261 | 359.67 | 485.048 | 1.349 |
| A06 | 230 | 0.01581 | 0.006453 | 0.408 |
| A07 | 226 | 0.05537 | 0.032580 | 0.588 |
| A08 | 220 | 0.0508 | 0.022611 | 0.445 |

Six of the eight end below 1.0 and two end above it. Of the six, five land between 0.408 and 0.588, and A03 lands at 0.301.

## The buy and sell ratio

The second metric is one figure per asset: the sell VWAP divided by the buy VWAP, across every fill in the record. Above 1.000 the average sale price beat the average buy price. Below it the average sale came in under the average cost. The figure reads how the closed trades performed, and nothing else.

No chart needs redrawing for it. Every per-asset chart already carries both lines, and the ratio is where the two of them finish: the sell line's last value over the buy line's last value. Both averages come from the same two columns as the charts, the export's Price at Transaction and its Quantity Transacted. Each side keeps its own pair of sums, so neither average pulls on the other.

Of the 38 charted bases, 35 have enough fills on both sides for the figure to mean anything. 23 of those finish above 1.000 and 12 below, and none lands exactly on it. A29 is the highest at 1.351 and A20 the lowest at 0.879.

| Asset | Buys | Sells | Buy VWAP | Sell VWAP | Ratio |
|---|---|---|---|---|---|
| A01 | 538 | 362 | 0.672605 | 0.610801 | 0.908 |
| A02 | 265 | 230 | 0.044733 | 0.0453964 | 1.015 |
| A03 | 275 | 184 | 0.0357413 | 0.040867 | 1.143 |
| A04 | 143 | 179 | 0.298528 | 0.328486 | 1.100 |
| A05 | 140 | 121 | 485.048 | 536.675 | 1.106 |
| A06 | 138 | 92 | 0.00645325 | 0.00627872 | 0.973 |

| Asset | Buys | Sells | Buy VWAP | Sell VWAP | Ratio |
|---|---|---|---|---|---|
| A07 | 155 | 71 | 0.0325805 | 0.0317443 | 0.974 |
| A08 | 140 | 80 | 0.0226113 | 0.0212018 | 0.938 |
| A09 | 113 | 83 | 1.38181 | 1.34298 | 0.972 |
| A10 | 104 | 87 | 5.98261e-06 | 6.10717e-06 | 1.021 |
| A11 | 75 | 103 | 0.0297901 | 0.036577 | 1.228 |
| A12 | 98 | 61 | 0.00762374 | 0.00758363 | 0.995 |
| A13 | 69 | 78 | 14.6904 | 15.4413 | 1.051 |
| A14 | 95 | 46 | 1996.6 | 2047.62 | 1.026 |
| A15 | 95 | 29 | 68918 | 68393.6 | 0.992 |
| A16 | 69 | 55 | 242.103 | 238.855 | 0.987 |
| A17 | 66 | 53 | 0.36133 | 0.38167 | 1.056 |
| A18 | 82 | 37 | 1.39501 | 1.40129 | 1.005 |
| A19 | 57 | 56 | 0.00276124 | 0.00283892 | 1.028 |
| A20 | 32 | 55 | 0.0336835 | 0.0296231 | 0.879 |
| A21 | 43 | 40 | 9.06192 | 9.27909 | 1.024 |
| A22 | 43 | 37 | 0.0881513 | 0.0882367 | 1.001 |
| A23 | 43 | 34 | 82.9528 | 83.4024 | 1.005 |
| A24 | 47 | 28 | 0.0116845 | 0.0116532 | 0.997 |
| A25 | 41 | 33 | 0.857863 | 0.824958 | 0.962 |
| A26 | 41 | 32 | 0.475448 | 0.497449 | 1.046 |
| A27 | 45 | 28 | 0.198142 | 0.21519 | 1.086 |
| A28 | 29 | 37 | 61.99 | 66.8959 | 1.079 |
| A29 | 23 | 40 | 0.00205112 | 0.00277049 | 1.351 |
| A30 | 27 | 31 | 2.03969 | 2.09155 | 1.025 |
| A31 | 16 | 9 | 0.0828047 | 0.0787786 | 0.951 |
| A32 | 8 | 16 | 0.431039 | 0.480832 | 1.116 |
| A33 | 8 | 15 | 0.145768 | 0.164984 | 1.132 |
| A34 | 10 | 13 | 0.174843 | 0.202243 | 1.157 |
| A35 | 9 | 7 | 0.160515 | 0.164726 | 1.026 |
| A36 | 3 | 3 | 46.4508 | 51.1833 | not read |
| A37 | 3 | 3 | 0.0589505 | 0.0591535 | not read |
| A38 | 1 | 4 | 2148.63 | 2576.19 | not read |

Three rows carry no ratio and the reason is the fill counts beside them. A38 has one buy, so its buy figure is a single price and not an average of anything. A36 and A37 have three buys and three sells each. A mean over three fills moves with any one of them, so a ratio built on it says more about which trade landed last than about how the asset performed. The table keeps both averages, since the export supports each one, and leaves the division out, since a number there would read as a result.

Every other row rests on at least seven fills a side, and 30 of the 38 rest on at least 23 a side.

The two metrics leave out different things and both gaps matter. The ratio counts only what completed on both sides, so it says nothing about units still held — and on most of these assets the held position is the larger part of the story. The trajectory counts every buy and no sale at all, so it says nothing about whether a sale ever cleared its cost. A cycle needs both readings. Either one alone overstates what it knows.

## Traps in the export

Three shapes in the record break a reader that takes every row the same way.

A plain `Sell` row carries a negative quantity against a positive Subtotal. An `Advanced Trade Sell` row pairs those signs the other way round. Taking the sign from one column and the magnitude from the other turns four trades backwards.

One row carries a negative fee.

Three A27 rows carry a Subtotal that rounds to $0.00 against a quantity that is real. A filter that drops zero-value rows drops those base units with them.

## Where the figures live

The charts are generated output, so they are not tracked. They belong beside the manual's other images in `artifacts/manual-figures/` under the repository root, a path `.gitignore` excludes. FIGURES.md inventories the images the manual itself embeds. No chart image is committed to this repository.

The 39 charts of this part live one level down, in a directory of their own beside the manual figures and under the same ignore rule. Keeping the two sets apart keeps the manual's own 38 images and this part's 39 charts from mixing. The combined view above sits in the first file below, and each charted base has one chart of its own.

```
docs/manual/figures/
    p<page>-i<index>.png        the manual's own 38 images
    vwap_combined.png           the eight-line combined chart
    vwap_a<NN>.png              one per charted base, 38 of them
```

## The charts are tracked, and they name no asset

Three sentences above are overtaken. They are quoted whole, and the sentence that replaces each one follows it.

> *The charts are generated output, so they are not tracked.*

The 39 charts are tracked, under `docs/manual/figures/`, so a clone can rebuild the manual without the venue export.

> *They belong beside the manual's other images in* `artifacts/manual-figures/`
>
> *under the repository root, a path* `.gitignore` *excludes.*

Both sets sit together in `docs/manual/figures/`, which `.gitignore` does not exclude.

*No chart image is committed to this repository.*

Every chart is committed, and every one of them has the asset name painted out of its title, out of its price-axis label, and out of the combined view's legend. The file name carries the label this part uses, `vwap_a01.png` through `vwap_a38.png`, and never the asset.

The repository carries one manual, and that manual names no traded asset. The build that names them runs only from a checkout holding the unobscured charts, which this repository does not carry.

## How to read a per-asset chart

Every chart carries two panels over one time axis.

The upper panel draws every fill as a dot at its price, buys in `COLORS["series"][0]` and sells in `COLORS["series"][5]`. Two lines cross it: the running buy VWAP and the running sell VWAP, in those same two colours. Each line steps at a fill of its own side and holds flat across a fill of the other side. The price axis turns logarithmic when the highest fill price exceeds the lowest more than eightfold, which covers A01 at 10.30 and A03 at 13.25 and no other asset. A chart of 30 fills or fewer marks each step with a dot: A31, A32, A33, A34, A35, A36, A37 and A38.

The lower panel draws net units accumulated — every bought quantity minus every sold quantity, running — in `COLORS["accent"]`, filled down to zero. It answers what the price panel cannot: whether the base position grew.

Every chart in this part draws its colours, its type sizes and its page size from `src/design_system.py`, which holds them under five names.

```
COLORS = {"bg": ..., "ink": ..., "accent": ..., "series": [...]}
TYPE = {"h1": ..., "body": ..., "cap": ...}
GRID = {"unit": 8, "page_w_in": 11.0, "page_h_in": 8.5}


def apply_rcparams(): ...


def contrast_ratio(fg: str, bg: str) -> float: ...
```

Read the buy line as a trajectory of average cost. A falling line says the average cost of the units bought so far fell. That fall is not profit, for the same reason the combined chart above carries none: no sale enters that line.

## The Bollinger band behind the price

A grey channel sits behind the fills on the upper panel. It is a Bollinger band, and it is there so a trade at a volatility extreme can be seen rather than asserted. A scrum sells into the top of the channel and a fold buys into the bottom, and the channel puts both on the picture.

The band is the published one. A middle line is a simple moving average over a fixed number of bars, and the outer lines sit a fixed number of population standard deviations above and below it, taken over the same bars.

```
Middle = SMA(period)
Upper  = Middle + std_dev * sigma(period)
Lower  = Middle - std_dev * sigma(period)
```

The setting is the conventional one: period 20, width 2 standard deviations.

### Where the band maths comes from

The platform already computes this band for its own trading decisions, and the charts read the same code rather than a second copy of the formula. The method returns one upper, middle and lower value per bar, and nothing before the twentieth bar, because no window has closed yet.

```
# src/trading/indicators/bollinger.py
BollingerBands(period=20, std_dev=2.0).bands(candles)
```

The deviation is the population one, divided by the window length, which is what Bollinger publishes. It is computed in `src/trading/indicators/helpers.py` and shared by every indicator that needs a rolling deviation.

### What a fill at a band edge means

The upper line is two standard deviations above the recent average price, so a sell dot sitting on it or above it is a sale made while the price was at the top of its own recent range. The lower line is the mirror of that, so a buy dot on it or below it is a purchase made into the bottom of that range. Those two readings are the whole reason the band is drawn.

A dot inside the channel is an ordinary trade. Nothing about a fill at an edge says it was profitable. It says where in the recent price distribution the trade landed.

### Which bar length the band uses

The export carries a timestamp and a price per fill and no candles, so the bars are built from the fills: the last fill in a bar closes it, and a bar with no fill closes where the one before it closed. That construction decides whether a short bar can carry a band at all. Twenty consecutive bars with no trade have no spread between them, so the deviation is zero and the band collapses onto the price line.

Measured over all 38 charted bases, counting every band the method returned and how many of them had zero width:

| Bar length | Bands drawn | Zero width | Share | At or above upper | At or below lower | Fills in a banded bar |
|---|---|---|---|---|---|---|
| 5 minutes | 1,288,981 | 1,064,990 | 82.6% | 2,921 | 3,134 | 6,868 |
| 15 minutes | 429,202 | 315,054 | 73.4% | 2,578 | 2,609 | 6,842 |
| 1 hour | 106,790 | 52,205 | 48.9% | 1,972 | 1,854 | 6,702 |
| 4 hours | 26,182 | 4,994 | 19.1% | 1,167 | 939 | 6,272 |
| 1 day | 3,792 | 35 | 0.9% | 605 | 460 | 4,955 |

At five minutes, 82.6 percent of the channel has no width. The band traces the price line as a hairline and shows no extreme at all. At one day, 0.9 percent has no width and the channel is wide enough to read across the whole window. The charts use one-day bars.

The cost of the coarse bar is the warm-up. Twenty one-day bars must close before the first band appears, so the first twenty days of each asset carry no channel, and 4,955 of the 6,997 fills land in a bar that has one.

### The band edges, counted over every base

One row per charted base, in fill-count order. The two edge columns count fills against the band of the bar holding them, on one-day bars at period 20 and width 2.

| Asset | Fills | Buy | Sell | At or above upper | At or below lower | In a banded bar | First buy VWAP | Final buy VWAP | Index | Window |
|---|---|---|---|---|---|---|---|---|---|---|
| A01 | 925 | 551 | 374 | 20 | 56 | 400 | 1.5915 | 0.652338 | 0.410 | 21 Apr to 25 Sep |
| A02 | 589 | 314 | 275 | 57 | 10 | 454 | 0.10842 | 0.0449093 | 0.414 | 22 Apr to 26 Sep |
| A03 | 537 | 315 | 222 | 13 | 28 | 409 | 0.11868 | 0.0257786 | 0.217 | 9 May to 27 Sep |
| A04 | 378 | 170 | 208 | 21 | 13 | 225 | 0.2478 | 0.277471 | 1.120 | 29 May to 27 Sep |
| A05 | 323 | 172 | 151 | 35 | 28 | 283 | 359.67 | 534.024 | 1.485 | 24 Apr to 26 Sep |
| A06 | 292 | 165 | 127 | 21 | 11 | 218 | 0.01581 | 0.00614089 | 0.388 | 24 Apr to 26 Sep |
| A08 | 274 | 165 | 109 | 20 | 19 | 209 | 0.0508 | 0.0223169 | 0.439 | 23 Apr to 27 Sep |
| A07 | 261 | 171 | 90 | 10 | 14 | 198 | 0.05537 | 0.0320055 | 0.578 | 3 May to 27 Sep |
| A10 | 237 | 122 | 115 | 25 | 24 | 167 | 5.71e-06 | 5.84811e-06 | 1.024 | 12 Apr to 27 Sep |
| A11 | 220 | 95 | 125 | 49 | 7 | 166 | 0.01722 | 0.033729 | 1.959 | 13 Jul to 25 Sep |
| A09 | 215 | 118 | 97 | 18 | 9 | 138 | 1.4622 | 1.38509 | 0.947 | 27 Apr to 27 Sep |
| A12 | 210 | 118 | 92 | 26 | 20 | 181 | 0.009814 | 0.00778479 | 0.793 | 27 Apr to 27 Sep |
| A13 | 205 | 93 | 112 | 21 | 7 | 170 | 15.7253 | 16.3105 | 1.037 | 9 May to 27 Sep |
| A14 | 169 | 108 | 61 | 18 | 31 | 149 | 2308.68 | 2037.61 | 0.883 | 23 Apr to 24 Sep |
| A16 | 157 | 80 | 77 | 13 | 20 | 141 | 273.4 | 243.486 | 0.891 | 2 May to 27 Sep |
| A17 | 155 | 82 | 73 | 10 | 4 | 132 | 0.35136 | 0.364156 | 1.036 | 7 May to 27 Sep |
| A15 | 152 | 107 | 45 | 11 | 39 | 143 | 77782.1 | 70016.9 | 0.900 | 23 Apr to 24 Sep |
| A18 | 148 | 97 | 51 | 18 | 22 | 123 | 1.4278 | 1.39586 | 0.978 | 23 Apr to 26 Sep |

| Asset | Fills | Buy | Sell | At or above upper | At or below lower | In a banded bar | First buy VWAP | Final buy VWAP | Index | Window |
|---|---|---|---|---|---|---|---|---|---|---|
| A19 | 147 | 79 | 68 | 2 | 3 | 40 | 0.00395 | 0.0024861 | 0.629 | 9 Aug to 27 Sep |
| A25 | 121 | 55 | 66 | 19 | 5 | 101 | 1.0673 | 0.850641 | 0.797 | 9 May to 27 Sep |
| A20 | 116 | 44 | 72 | 0 | 1 | 42 | 0.0697757 | 0.0309713 | 0.444 | 9 Aug to 27 Sep |
| A22 | 112 | 56 | 56 | 15 | 22 | 103 | 0.09814 | 0.0884351 | 0.901 | 25 Apr to 26 Sep |
| A24 | 108 | 62 | 46 | 4 | 3 | 73 | 0.02809 | 0.0104166 | 0.371 | 8 Jul to 27 Sep |
| A21 | 107 | 53 | 54 | 10 | 18 | 102 | 9.296 | 9.38981 | 1.010 | 24 Apr to 25 Sep |
| A23 | 101 | 52 | 49 | 11 | 24 | 88 | 85.32 | 84.0803 | 0.985 | 23 Apr to 25 Sep |
| A29 | 98 | 40 | 58 | 15 | 5 | 85 | 0.001603 | 0.00239143 | 1.492 | 7 Jul to 27 Sep |
| A30 | 98 | 40 | 58 | 21 | 2 | 69 | 2.411 | 2.2074 | 0.916 | 29 May to 27 Sep |
| A27 | 87 | 51 | 36 | 9 | 4 | 44 | 0.205737 | 0.197296 | 0.959 | 29 May to 25 Sep |
| A33 | 86 | 36 | 50 | 20 | 0 | 60 | 0.14222 | 0.155404 | 1.093 | 22 Aug to 26 Sep |
| A26 | 85 | 46 | 39 | 6 | 1 | 42 | 0.5153 | 0.471505 | 0.915 | 13 Jul to 27 Sep |
| A28 | 81 | 35 | 46 | 9 | 0 | 46 | 64.42 | 63.7886 | 0.990 | 29 May to 27 Sep |
| A32 | 49 | 15 | 34 | 15 | 0 | 37 | 0.42307 | 0.465448 | 1.100 | 23 Jul to 26 Sep |
| A31 | 42 | 21 | 21 | 9 | 4 | 28 | 0.09457 | 0.0825252 | 0.873 | 29 May to 24 Sep |
| A34 | 36 | 15 | 21 | 7 | 0 | 26 | 0.1696 | 0.179876 | 1.061 | 23 Jul to 26 Sep |
| A35 | 35 | 15 | 20 | 13 | 0 | 27 | 0.1695 | 0.166708 | 0.984 | 13 Jul to 25 Sep |
| A36 | 18 | 6 | 12 | 8 | 0 | 17 | 46.33 | 47.883 | 1.034 | 23 Jul to 25 Sep |
| A37 | 15 | 7 | 8 | 1 | 5 | 12 | 0.05997 | 0.057538 | 0.959 | 23 Jul to 25 Sep |
| A38 | 8 | 2 | 6 | 5 | 1 | 7 | 2148.63 | 2168.26 | 1.009 | 23 Jul to 21 Sep |

Across the 38, 605 fills sit at or above the upper band and 460 at or below the lower one. A02 and A11 sell into the top most often, at 57 and 49. A01 buys into the bottom most often, at 56.

## The newest export, beside the one the figures above measure

The figures and tables above were measured on an earlier export. A later one covers the same account over a longer window. Both sets of numbers are below, so a reader holding either can tell which is which.

The following sentence is overtaken:

> *The export holds 5,709 physical rows and 5,705 body rows. 5,661 of the body rows are fills.*

The later export holds 7,049 physical rows and 7,045 body rows, and 6,997 of the body rows are fills on a charted base.

The following sentence is overtaken:

> *Of the eight assets with the most fills, six trend down to between 0.4 and 0.6. A05 and A04 trend up.*

On the later export, five of the eight trend down to between 0.388 and 0.578, A03 falls further to 0.217, and A05 and A04 still trend up, to 1.485 and 1.120.

Side by side:

| Reading | Earlier export | Later export |
|---|---|---|
| Physical rows | 5,709 | 7,049 |
| Body rows | 5,705 | 7,045 |
| Fills on a charted base | 5,661 | 6,997 |
| Charted bases | 38 | 38 |
| First fill | 12 Apr | 12 Apr |
| Last fill | 3 Sep | 27 Sep |
| Rows carrying no fill | 44 | 48 |

The charted bases are the same 38 assets in both. The later export adds 1,336 fills and 24 days, and its extra rows carrying no fill are three more reward payments and one more cash leg.

One check separates a longer record from a different calculation. Of the 38 first buy VWAP values the table above publishes, 37 come out of the later export identical to the digit, and the one that differs is A20, published rounded to 0.069776 against a computed 0.0697757. A different price column or a different weighting would have moved all 38.

## What this repository holds for these charts

The rows behind the charts take the shape `fetch_all_history_chunked` in `src/exchange/history_helpers.py` returns for the History tab, one dict per fill.

The closest committed figure to the buy line is the average entry price `compute_position_health` derives in `src/exchange/position_health.py` from the records a venue returns for its own trade history. The two differ, and the difference matters to a reader holding a chart beside the platform. The average entry follows the open position only: a buy re-weights it, a sell leaves it alone, and a full close resets it to zero. The buy line here counts every buy in the record and never resets.

```
if t.side == OrderSide.BUY:     # t is one get_my_trades record
    new_qty = qty + t.amount
    if new_qty > 0:
        avg_entry = (qty * avg_entry + t.amount * t.price) / new_qty
    qty = new_qty
elif t.side == OrderSide.SELL:
    qty -= t.amount
    if qty < 1e-12:
        qty = 0.0
        avg_entry = 0.0
```

No committed module computes an average sell price, and none computes a ratio of one average over the other. Five names were searched across the source, the harness and the tools, ignoring case, and every one of them returns no file. The control returns files for both of its terms and none for a coined one.

```
searched in src/, dev_harness/ and tools/, ignoring case
    avg_sell             0 files
    average_sell         0 files
    sell_vwap            0 files
    avg_sell_price       0 files
    sb_ratio             0 files
control
    avg_entry           13 files
    vwap                 6 files
    a coined term        0 files
```

The chart module computes no ratio either. It draws the two lines and stops there. The ratio table above comes from the export directly, on the same two columns the charts read.

No committed file produces these charts. A walk of every path any commit ever added, renamed or deleted reaches 1,626 distinct paths, and exactly one of them carries the letters vwap: a Simulator test for the price band. A pickaxe over every Python file in every commit returns nothing for the three names a chart builder would carry. The charts and the record behind them belong to the operator. This repository cannot regenerate either.

```
git log --all --diff-filter=ADR --name-only
    1,626 distinct paths
    tests/test_vwap_band_scales_to_price.py     the one vwap name
                                                deleted; not in the tree

git log -S<name> -- '*.py'
    draw_combined         0 commits
    vwap_combined         0 commits
    buy_vwap              0 commits
control
    avg_entry            16 commits
    sync_ytd_trade_count  7 commits
    a coined term         0 commits
```

## The 38 per-asset charts

One chart per charted base, in fill-count order. The table gives what each chart's two lines start and end at, and the window its axis spans.

| Asset | Fills | Buy | Sell | First buy VWAP | Final buy VWAP | Index | Window |
|---|---|---|---|---|---|---|---|
| A01 | 900 | 538 | 362 | 1.5915 | 0.672605 | 0.423 | 21 Apr to 2 Sep |
| A02 | 495 | 265 | 230 | 0.10842 | 0.044733 | 0.413 | 22 Apr to 3 Sep |
| A03 | 459 | 275 | 184 | 0.11868 | 0.035741 | 0.301 | 9 May to 2 Sep |
| A04 | 322 | 143 | 179 | 0.2478 | 0.298528 | 1.205 | 29 May to 31 Aug |
| A05 | 261 | 140 | 121 | 359.67 | 485.048 | 1.349 | 24 Apr to 3 Sep |
| A06 | 230 | 138 | 92 | 0.01581 | 0.006453 | 0.408 | 24 Apr to 3 Sep |
| A07 | 226 | 155 | 71 | 0.05537 | 0.032580 | 0.588 | 3 May to 3 Sep |
| A08 | 220 | 140 | 80 | 0.0508 | 0.022611 | 0.445 | 23 Apr to 3 Sep |
| A09 | 196 | 113 | 83 | 1.4622 | 1.381809 | 0.945 | 27 Apr to 3 Sep |
| A10 | 191 | 104 | 87 | 5.71e-06 | 5.98261e-06 | 1.048 | 12 Apr to 3 Sep |
| A11 | 178 | 75 | 103 | 0.01722 | 0.029790 | 1.730 | 13 Jul to 3 Sep |
| A12 | 159 | 98 | 61 | 0.009814 | 0.007624 | 0.777 | 27 Apr to 3 Sep |
| A13 | 147 | 69 | 78 | 15.7253 | 14.690407 | 0.934 | 9 May to 1 Sep |
| A14 | 141 | 95 | 46 | 2308.68 | 1996.595 | 0.865 | 23 Apr to 21 Aug |
| A15 | 124 | 95 | 29 | 77782.1 | 68917.999 | 0.886 | 23 Apr to 3 Sep |
| A16 | 124 | 69 | 55 | 273.4 | 242.103 | 0.886 | 2 May to 1 Sep |
| A17 | 119 | 66 | 53 | 0.35136 | 0.361330 | 1.028 | 7 May to 3 Sep |
| A18 | 119 | 82 | 37 | 1.4278 | 1.395015 | 0.977 | 23 Apr to 3 Sep |
| A19 | 113 | 57 | 56 | 0.00395 | 0.002761 | 0.699 | 9 Aug to 31 Aug |
| A20 | 87 | 32 | 55 | 0.069776 | 0.033684 | 0.483 | 9 Aug to 2 Sep |
| A21 | 83 | 43 | 40 | 9.296 | 9.061918 | 0.975 | 24 Apr to 3 Sep |
| A22 | 80 | 43 | 37 | 0.09814 | 0.088151 | 0.898 | 25 Apr to 3 Sep |
| A23 | 77 | 43 | 34 | 85.32 | 82.952844 | 0.972 | 23 Apr to 1 Sep |
| A24 | 75 | 47 | 28 | 0.02809 | 0.011685 | 0.416 | 8 Jul to 31 Aug |
| A25 | 74 | 41 | 33 | 1.0673 | 0.857863 | 0.804 | 9 May to 3 Sep |
| A26 | 73 | 41 | 32 | 0.5153 | 0.475448 | 0.923 | 13 Jul to 30 Aug |
| A27 | 73 | 45 | 28 | 0.205737 | 0.198142 | 0.963 | 29 May to 28 Aug |
| A28 | 66 | 29 | 37 | 64.42 | 61.990016 | 0.962 | 29 May to 30 Aug |
| A29 | 63 | 23 | 40 | 0.001603 | 0.002051 | 1.280 | 7 Jul to 30 Aug |
| A30 | 58 | 27 | 31 | 2.411 | 2.039694 | 0.846 | 29 May to 3 Sep |
| A31 | 25 | 16 | 9 | 0.09457 | 0.082805 | 0.876 | 29 May to 22 Aug |
| A32 | 24 | 8 | 16 | 0.42307 | 0.431039 | 1.019 | 23 Jul to 3 Sep |
| A33 | 23 | 8 | 15 | 0.14222 | 0.145768 | 1.025 | 22 Aug to 3 Sep |

| Asset | Fills | Buy | Sell | First buy VWAP | Final buy VWAP | Index | Window |
|---|---|---|---|---|---|---|---|
| A34 | 23 | 10 | 13 | 0.1696 | 0.174843 | 1.031 | 23 Jul to 3 Sep |
| A35 | 16 | 9 | 7 | 0.1695 | 0.160515 | 0.947 | 13 Jul to 2 Sep |
| A36 | 6 | 3 | 3 | 46.33 | 46.450791 | 1.003 | 23 Jul to 3 Sep |
| A37 | 6 | 3 | 3 | 0.05997 | 0.058950 | 0.983 | 23 Jul to 16 Aug |
| A38 | 5 | 1 | 4 | 2148.63 | 2148.63 | 1.000 | 23 Jul to 25 Aug |

Those 38 fill counts sum to 5,661, the fill total this part opens with. Eleven of the 38 end at or above 1.000 and 27 end below it.

The 39 charts of this part, the combined view above included, come from one module, `vwap_charts`. It reads the export once, groups the fills by asset, and writes one PNG per charted base.

The line each chart draws is a running mean of price weighted by quantity. The price is the export's Price at Transaction column and the quantity is its Quantity Transacted column. Buys and sells accumulate in two separate pairs of sums, so a fill on one side steps that side's line and leaves the other flat. The lower panel is the same walk of quantities, added on a buy and taken away on a sell.

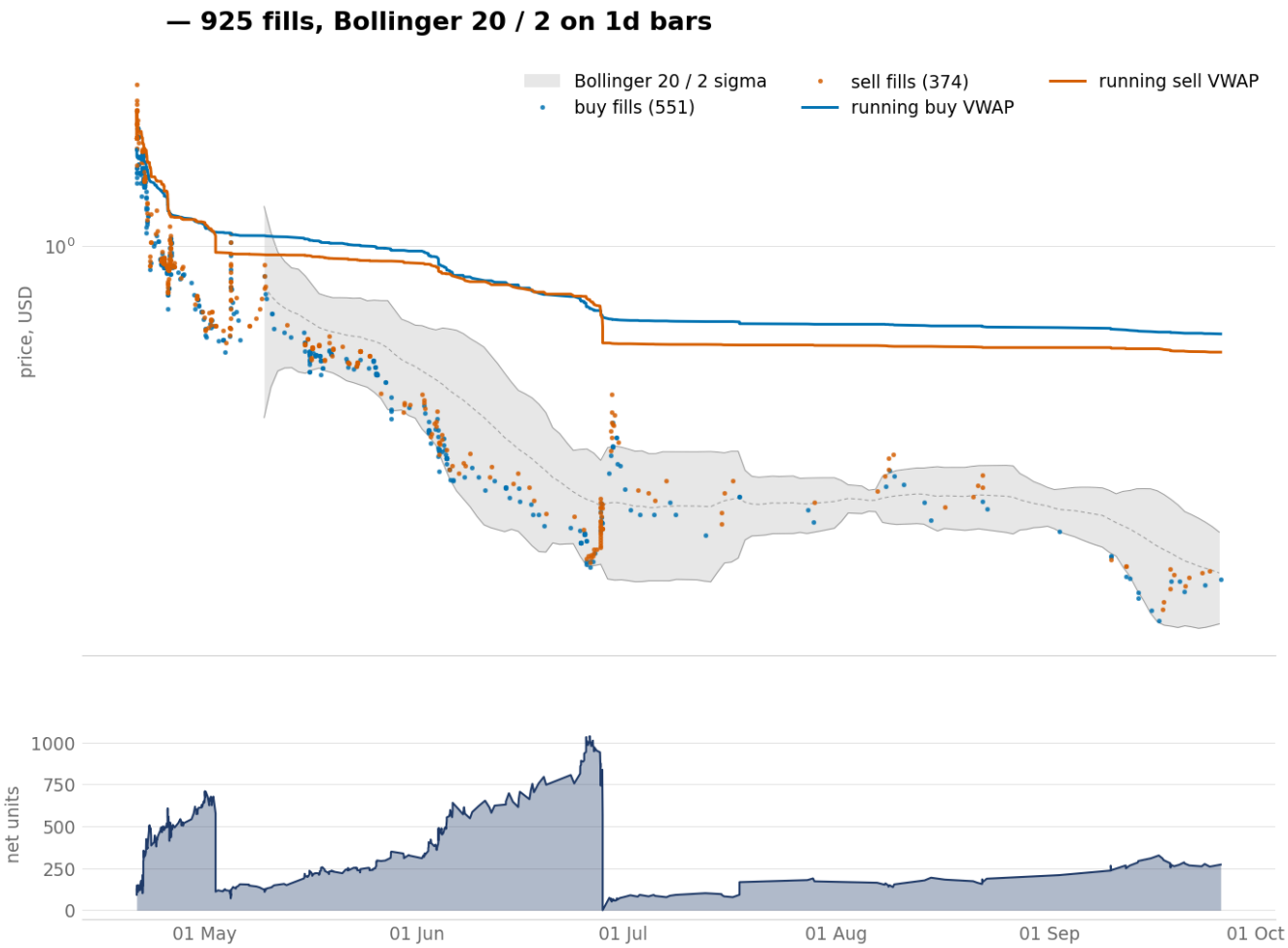
The module is not committed to this repository. Neither is the export it reads.

```
def load_fills(src: Path):
    """Return the Fill list and a Counter keyed by drop cause."""
    ...
    price = parse_money(r[idx["Price at Transaction"]])
    qty = abs(parse_money(r[idx["Quantity Transacted"]]))
    ...
    fills.append(Fill(asset, when, side, price, qty))


def trajectory(fills):
    """Return times and the running buy VWAP, sell VWAP and net units."""
    ordered = sorted(fills, key=lambda f: f.when)
    t, buy_v, sell_v, held = [], [], [], []
    bn = bd = sn = sd = pos = 0.0
    for f in ordered:
        if f.side == "buy":
            bn += f.price * f.qty
            bd += f.qty
            pos += f.qty
        else:
            sn += f.price * f.qty
            sd += f.qty
            pos -= f.qty
        t.append(f.when)
        buy_v.append(bn / bd if bd > 0 else float("nan"))
        sell_v.append(sn / sd if sd > 0 else float("nan"))
        held.append(pos)
    return t, buy_v, sell_v, held
```

Four of the 38 charts below carry a block of their own. Each one is a chart where the module takes a turn it does not take for the rest: a logarithmic price axis, the minor ticks that axis needs, a tick label small enough to

go scientific, and a fill count low enough to mark every step. The other 34 are drawn by the two functions above and nothing else.

### A01



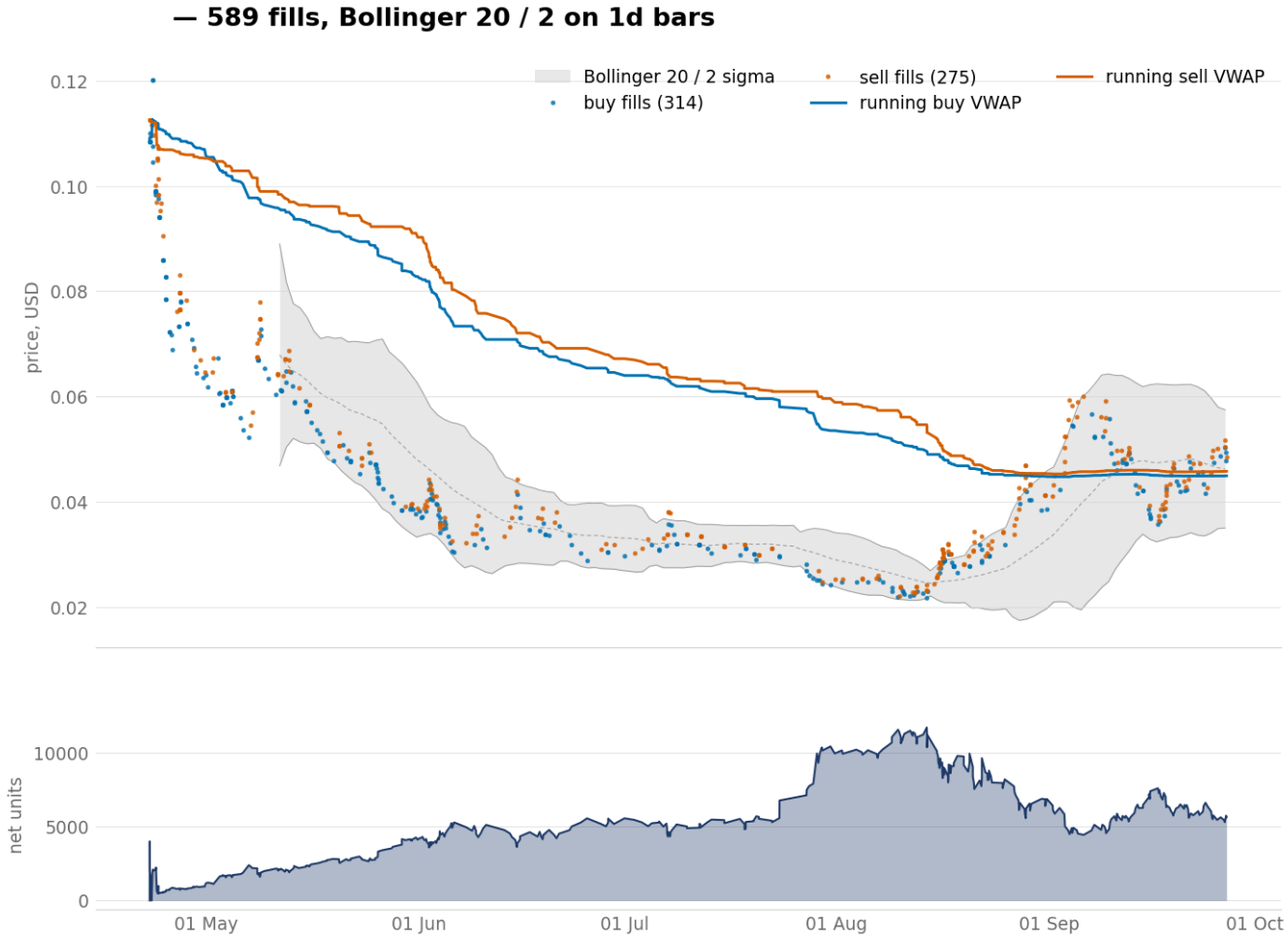

A01, 900 fills, a logarithmic price axis, and a net-units panel that empties twice.

The most-traded base of the 38, and one of the two charts whose price axis runs logarithmic. Both lines step down together to about 1.00 by early May, then to about 0.67 at the end of June, where the sell line settles below the buy line and stays there. The net-units panel peaks at 1,037 units in late June and ends at 209, with two near-vertical drops.

The axis is a measurement, not a choice. A01's fills run from 0.2118 to 2.1805, a span of 10.30, and the module turns the axis logarithmic above eight. A03 is the only other chart over that line.

```
lo = min(f.price for f in fills)
hi = max(f.price for f in fills)
if hi / lo > 8:
    ax.set_yscale("log")
```

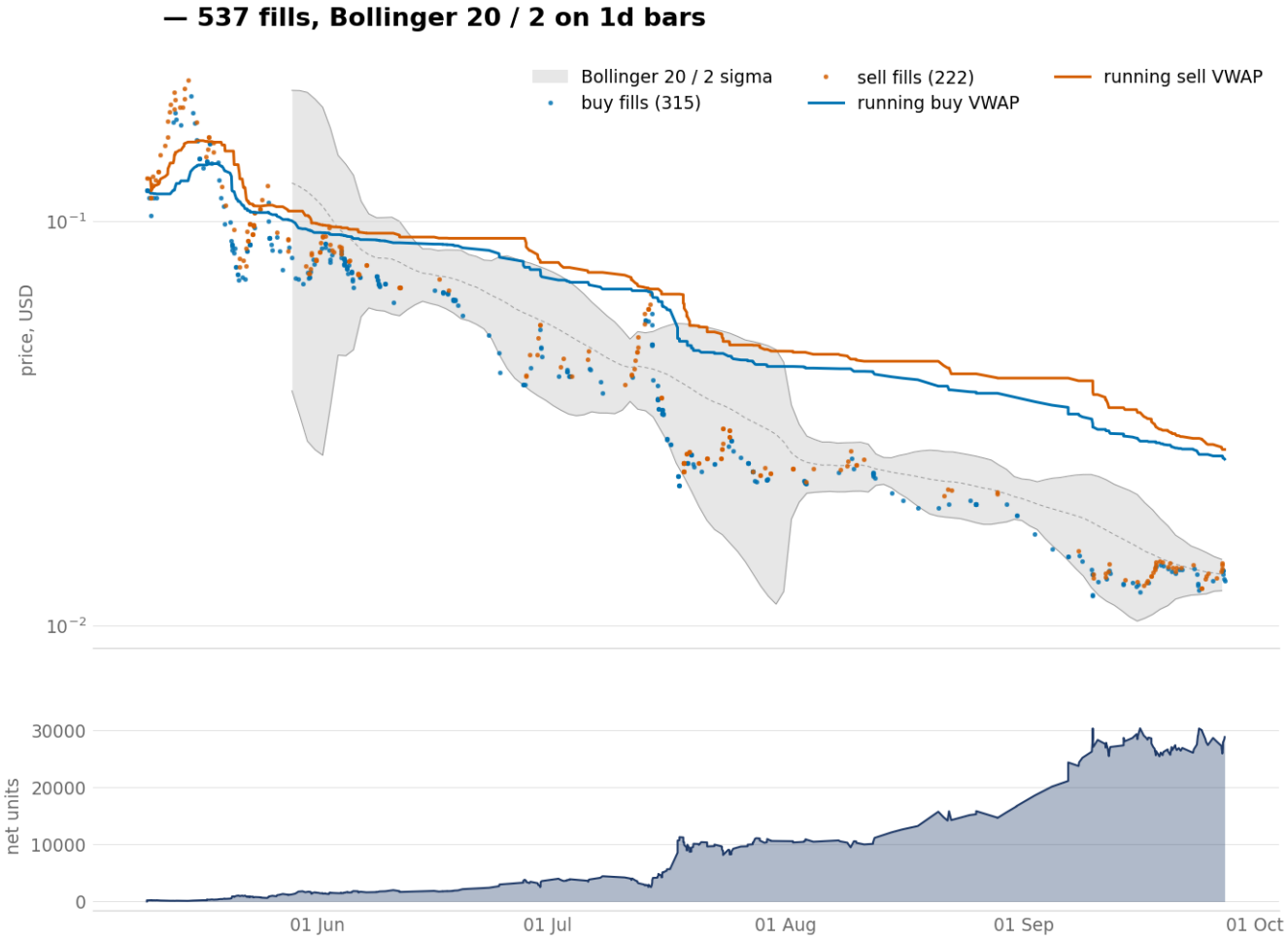
## A02




A02, 495 fills, with the sell line above the buy line for the whole window.

The sell line runs above the buy line across the whole window and the two meet in September. Net units climb to 11,699 in mid-August and end at 4,765.

### A03



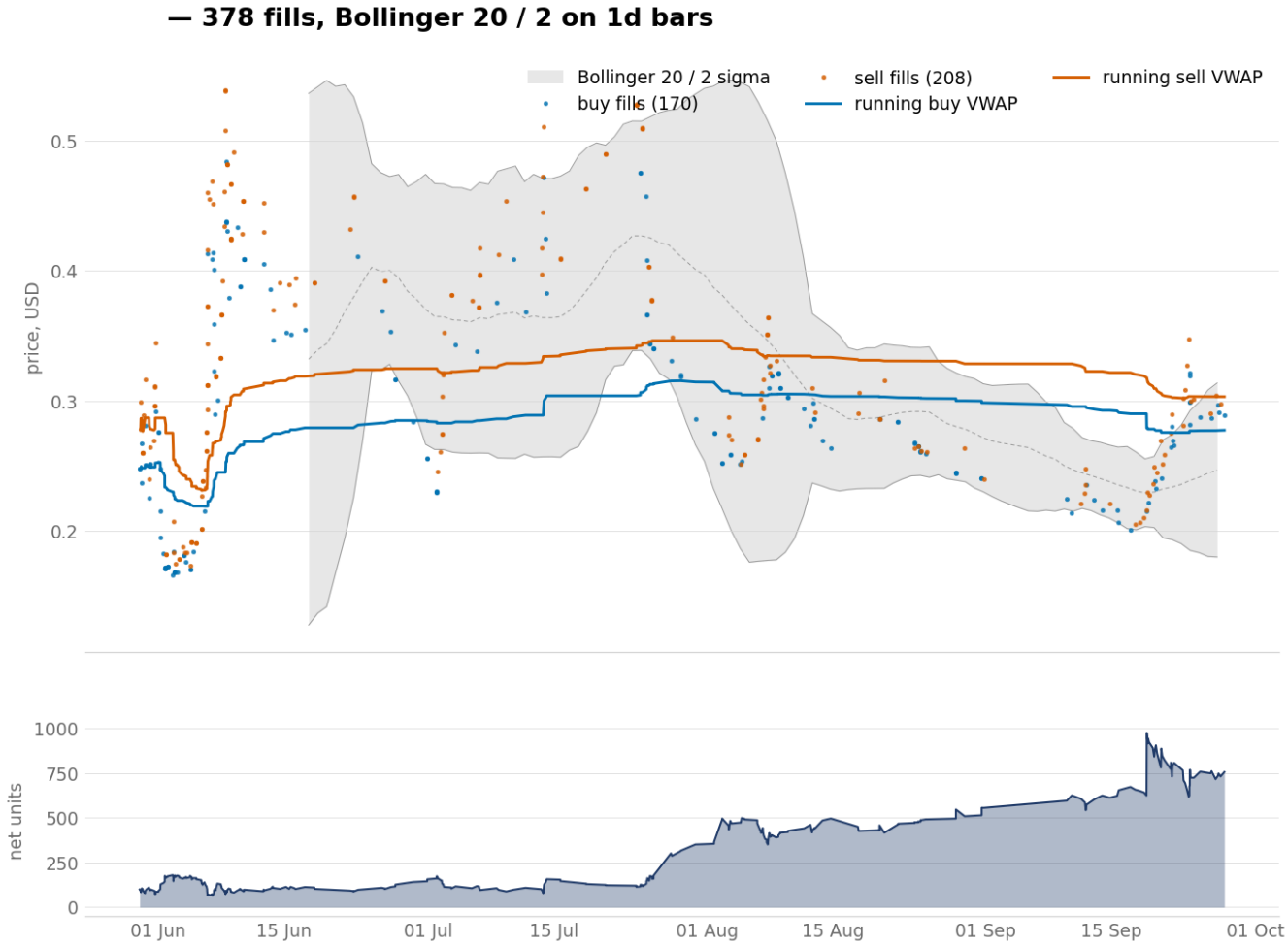

A03, 459 fills, ending at the lowest index of the eight most-traded assets.

The second logarithmic axis, at a span of 13.25. Both lines hold near 0.09 through June, then break down in mid-July. Net units end at the highest point of the window, 18,510.

A logarithmic axis labels the powers of ten. A03's fills run from 0.01682 to 0.22281, a window holding one of them, and A01's window holds one as well. The module adds five minor ticks per decade and gives them the same price formatter as the major ones, which is what puts 0.2000 down to 0.0200 on this axis.

```
SUBS = (1.0, 2.0, 3.0, 5.0, 7.0)

ax.yaxis.set_minor_locator(LogLocator(base=10.0, subs=SUBS, numticks=99))
ax.yaxis.set_minor_formatter(FuncFormatter(fmt_price))
ax.tick_params(axis="y", which="minor", labelsize=TYPE["cap"]["size"])
```

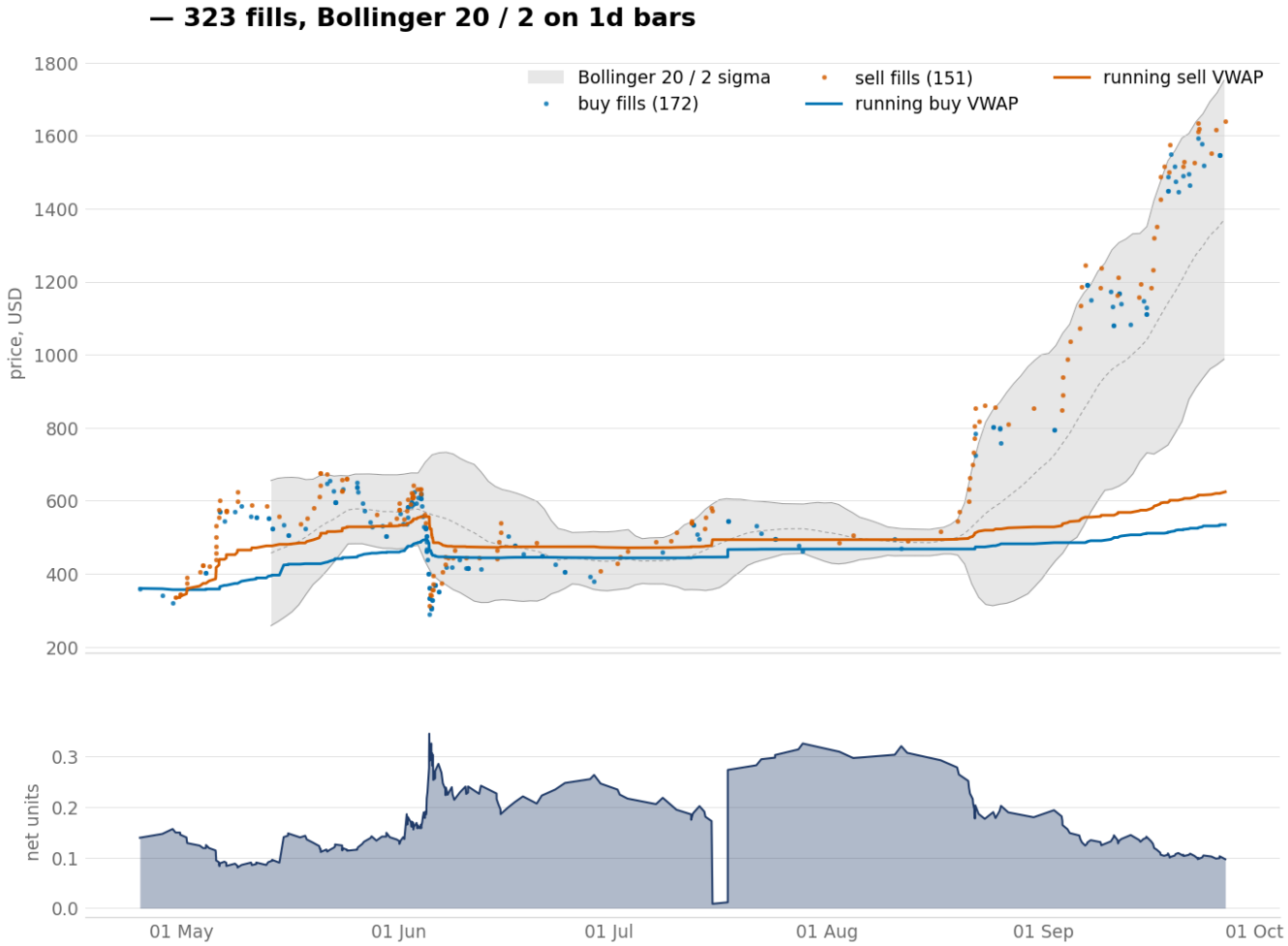
## A04




A04, 322 fills, with more sells than buys and both lines rising.

One of eleven charts with more sells than buys. Both lines dip to about 0.22 in early June and climb after it. Net units step from about 120 to about 500 through late July.

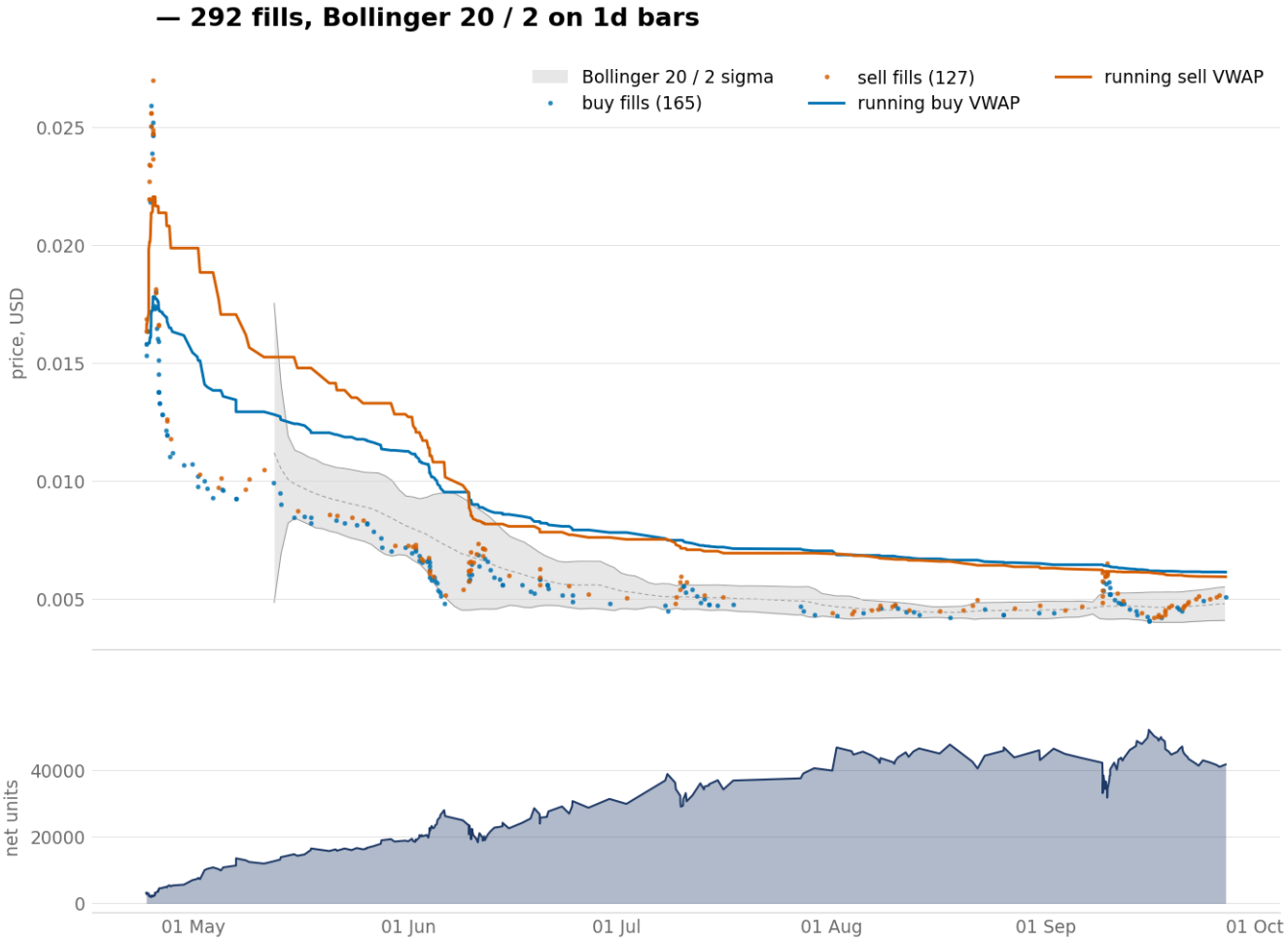
## A05




A05, 261 fills, the highest final index of the eight most-traded assets.

The buy line rises from 359.67 to 485.05, the largest climb among the eight in the combined chart. Net units fall to near zero in mid-July, then rebuild to about 0.20.

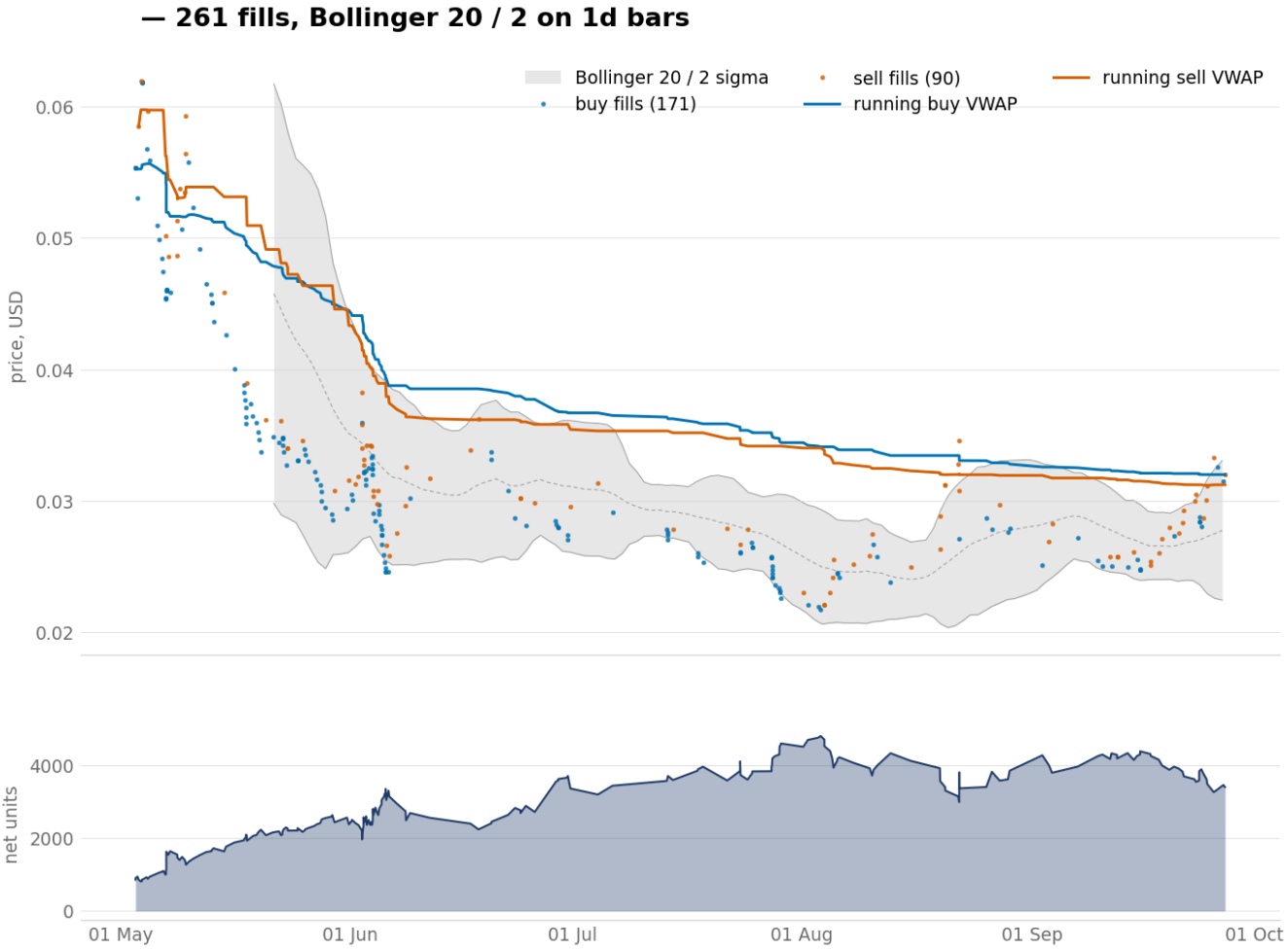
## A06




A06, 230 fills, with the two lines converging from mid-June.

The sell line runs above the buy line until mid-June, then the two converge and the sell line finishes just below. Net units grow to 47,574 and end at 44,795.

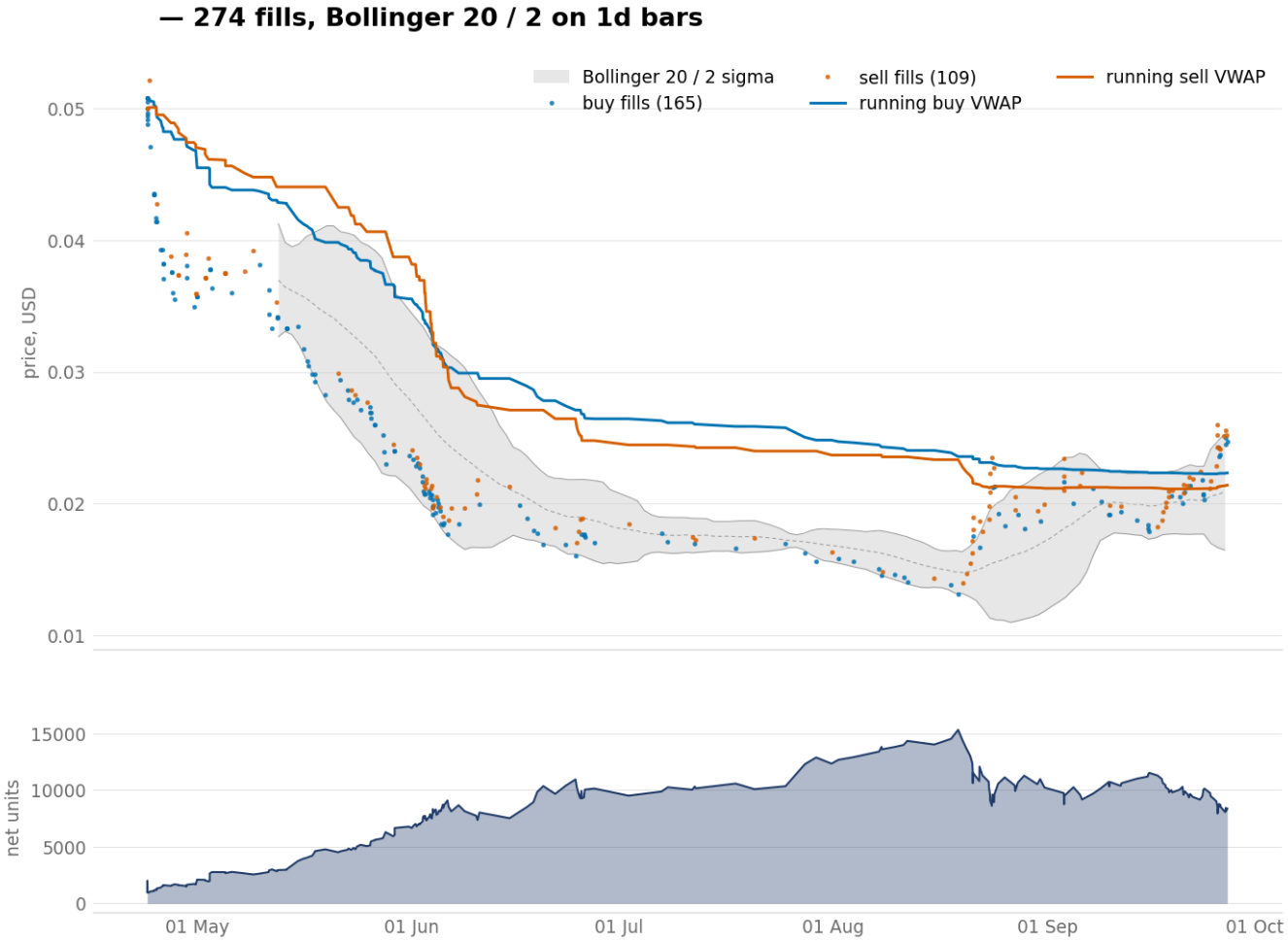
## A07




A07, 226 fills, with a step down at the start of June.

One step down at the start of June takes the buy line from about 0.044 to about 0.039, and a slow decline follows. The sell line crosses below the buy line at that same step.

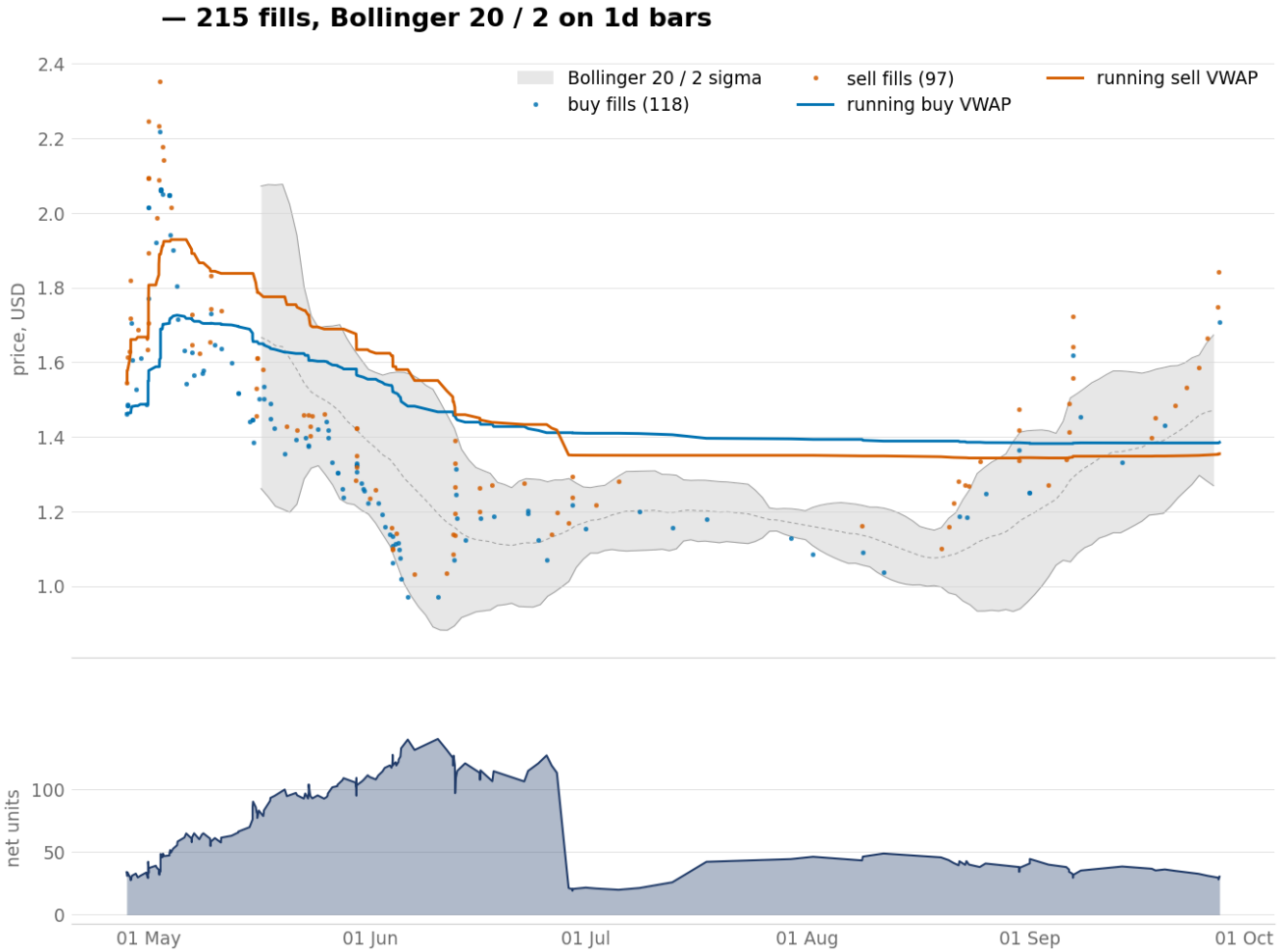
## A08




A08, 220 fills, with the sell line crossing below the buy line in mid-June.

The crossing holds for the rest of the window. Net units peak at 15,301 in mid-August, then fall to 9,483.

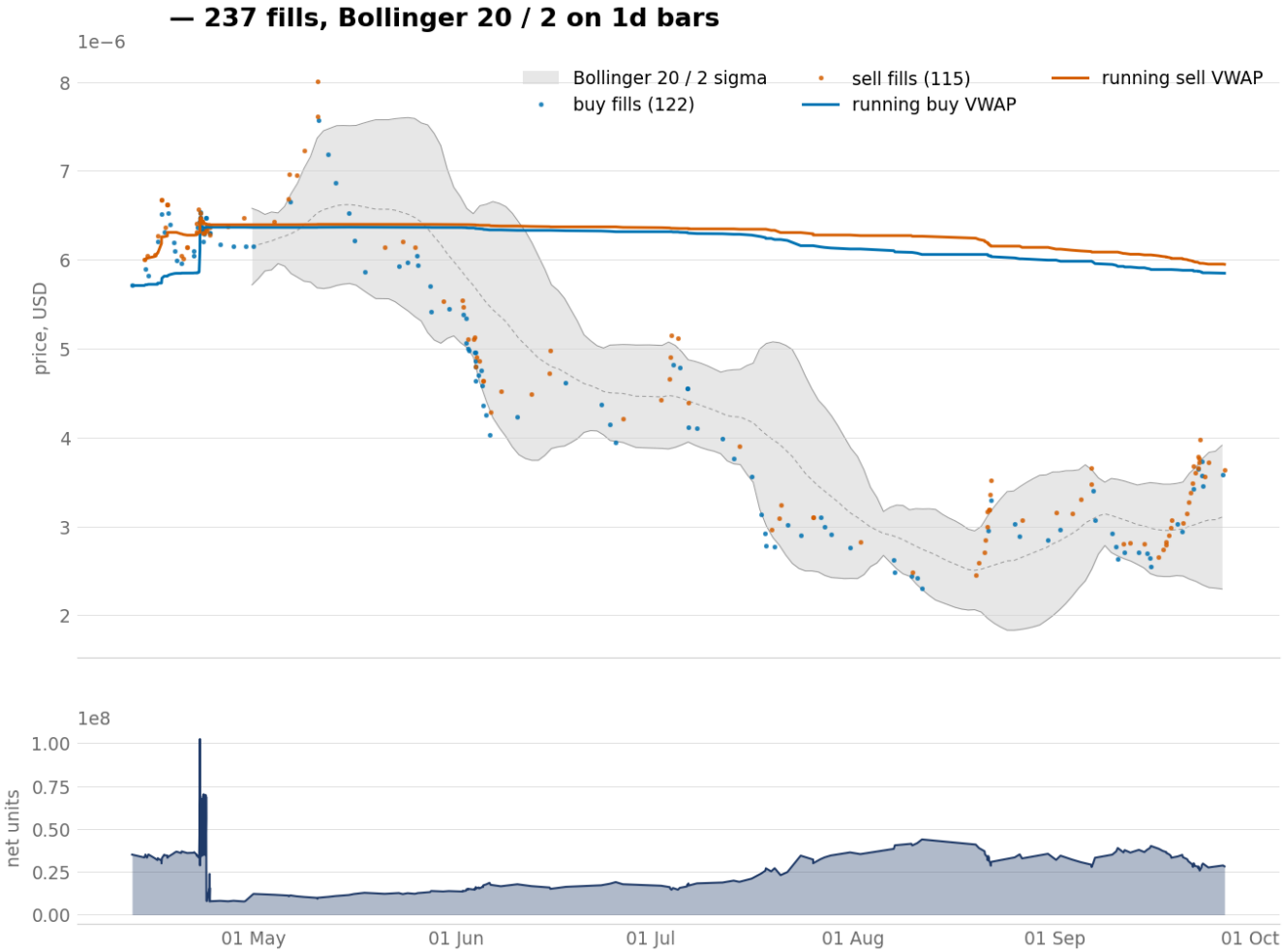
## A09




A09, 196 fills, with both lines flat from July onward.

Both lines rise to a peak near 1 May, then fall to a flat run from July. Net units drop from 140 to about 20 at the end of June and hold near 40 after that.

### A10



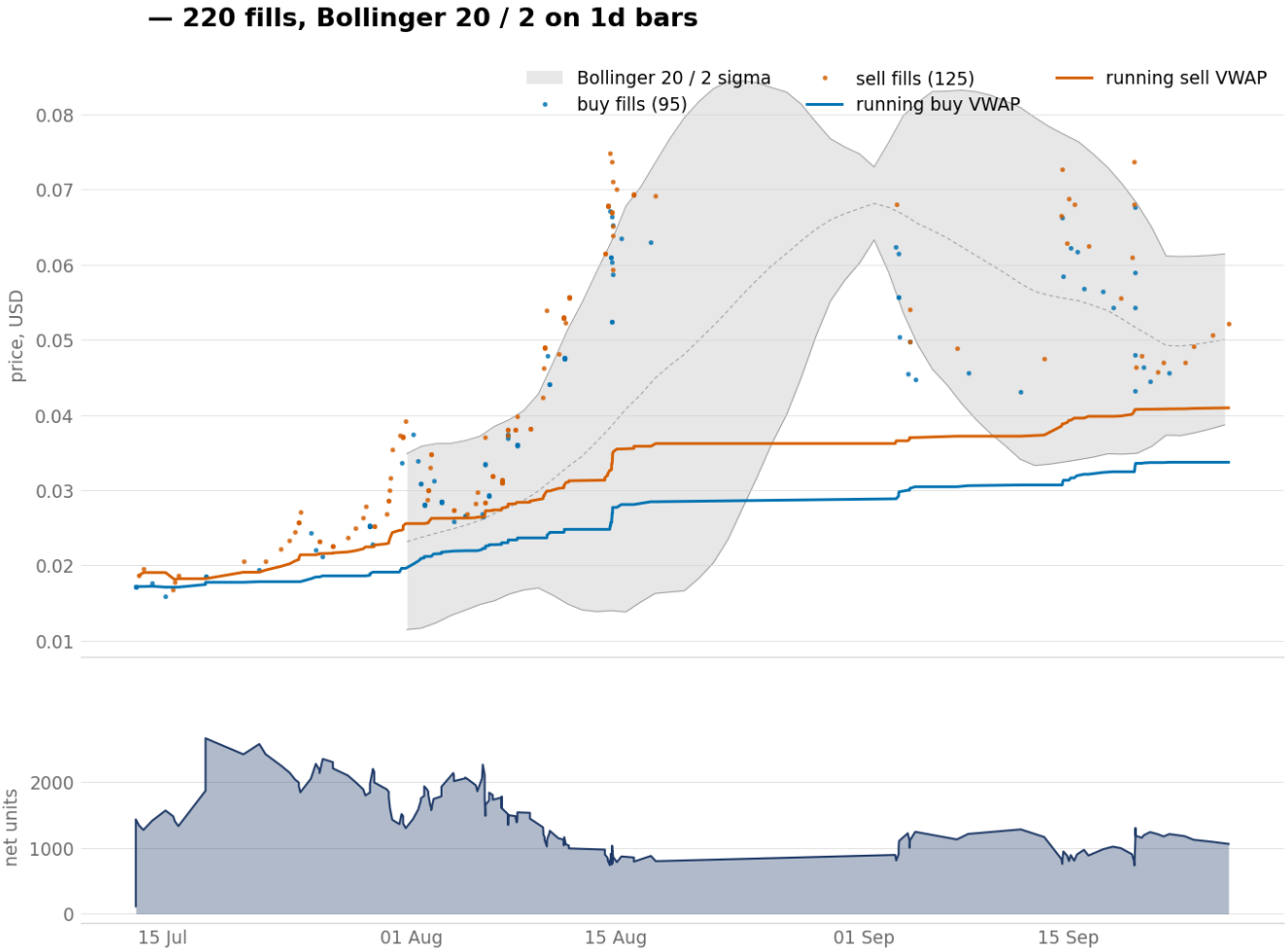

A10, 191 fills, priced near six millionths of a dollar.

The lowest-priced base in the record. The buy line holds near 6.3e-06 from late April while the fill dots fall from 8.0e-06 to 2.3e-06. Net units spike to 1.02e+08 in late April and end at 3.22e+07.

A10 is the only base whose fills fall under a tenth of a cent. The next lowest is A29 at 0.001479, so A10 alone reaches the last branch of the tick formatter, and this is the only chart of the 38 labelled in scientific notation.

```
def fmt_price(v, _pos=None) -> str:
    """Format a y-axis price tick, keeping a micro-cap significand visible."""
    if v == 0:
        return "0"
    if abs(v) >= 1000:
        return f"{v:,.0f}"
    if abs(v) >= 1:
        return f"{v:,.2f}"
    if abs(v) >= 0.001:
        return f"{v:.4f}"
    return f"{v:.2e}"
```

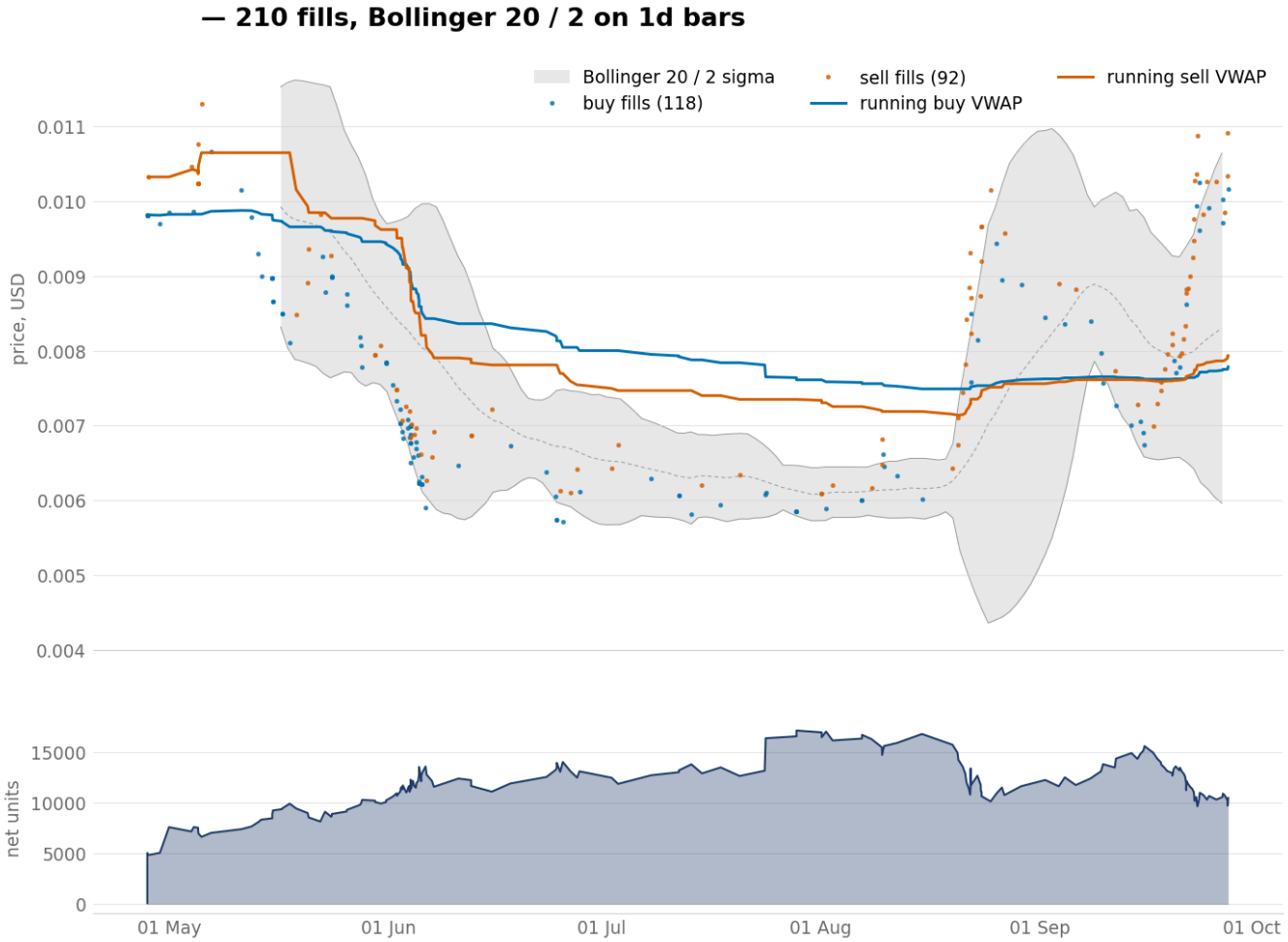
## A11




A11, 178 fills, and the largest rise of the 38.

The buy line rises from 0.01722 to 0.02979, an index of 1.730 and the largest of the 38. Both lines climb through August. Net units drop across that same climb, 2,669 down to 1,107.

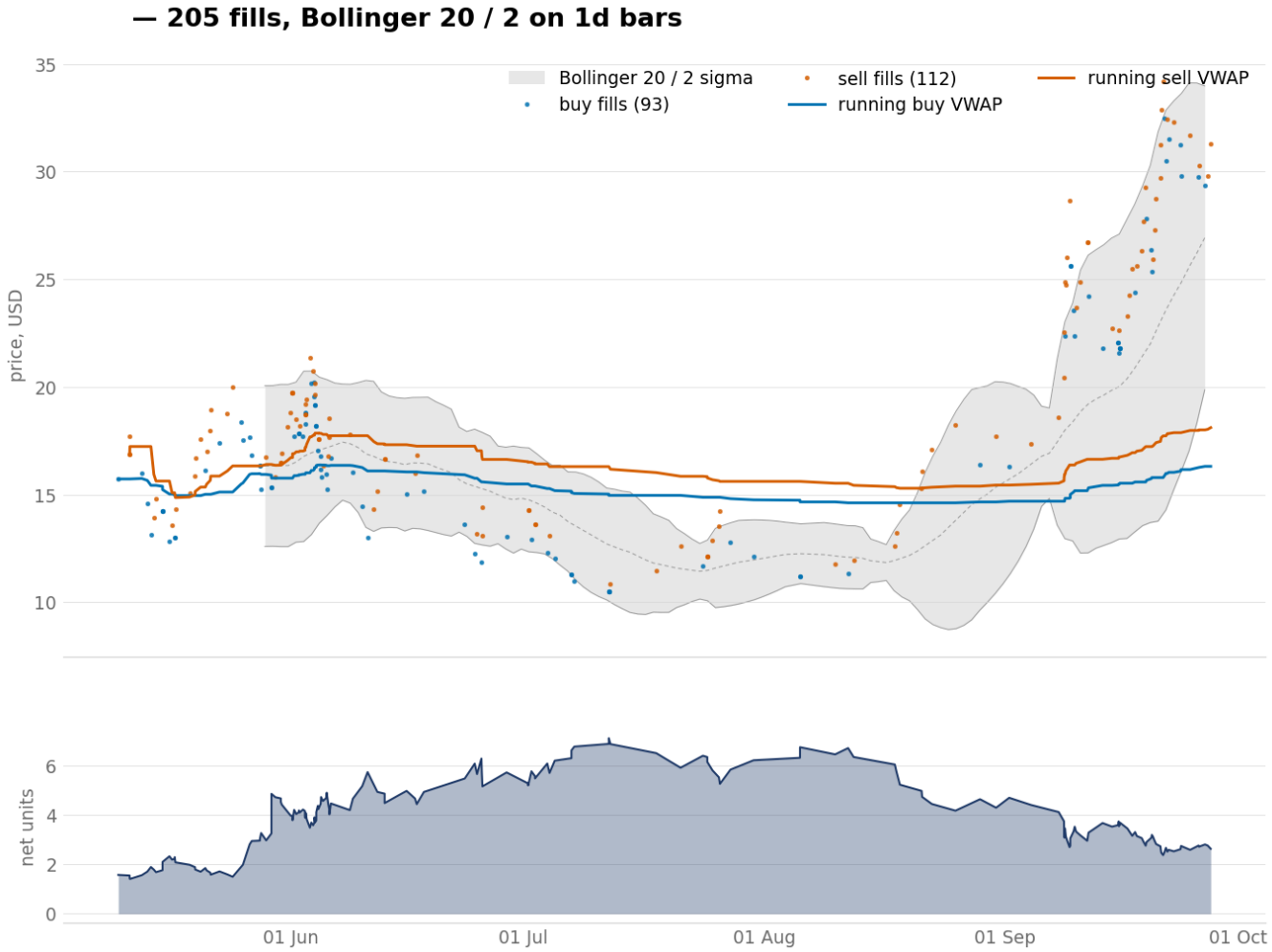
## A12




A12, 159 fills, with the sell line dropping below the buy line at the start of June.

The buy line holds near 0.0098 through May and the sell line near 0.0105, then both step down at the start of June. Net units reach 17,089 in early August and end at 11,612.

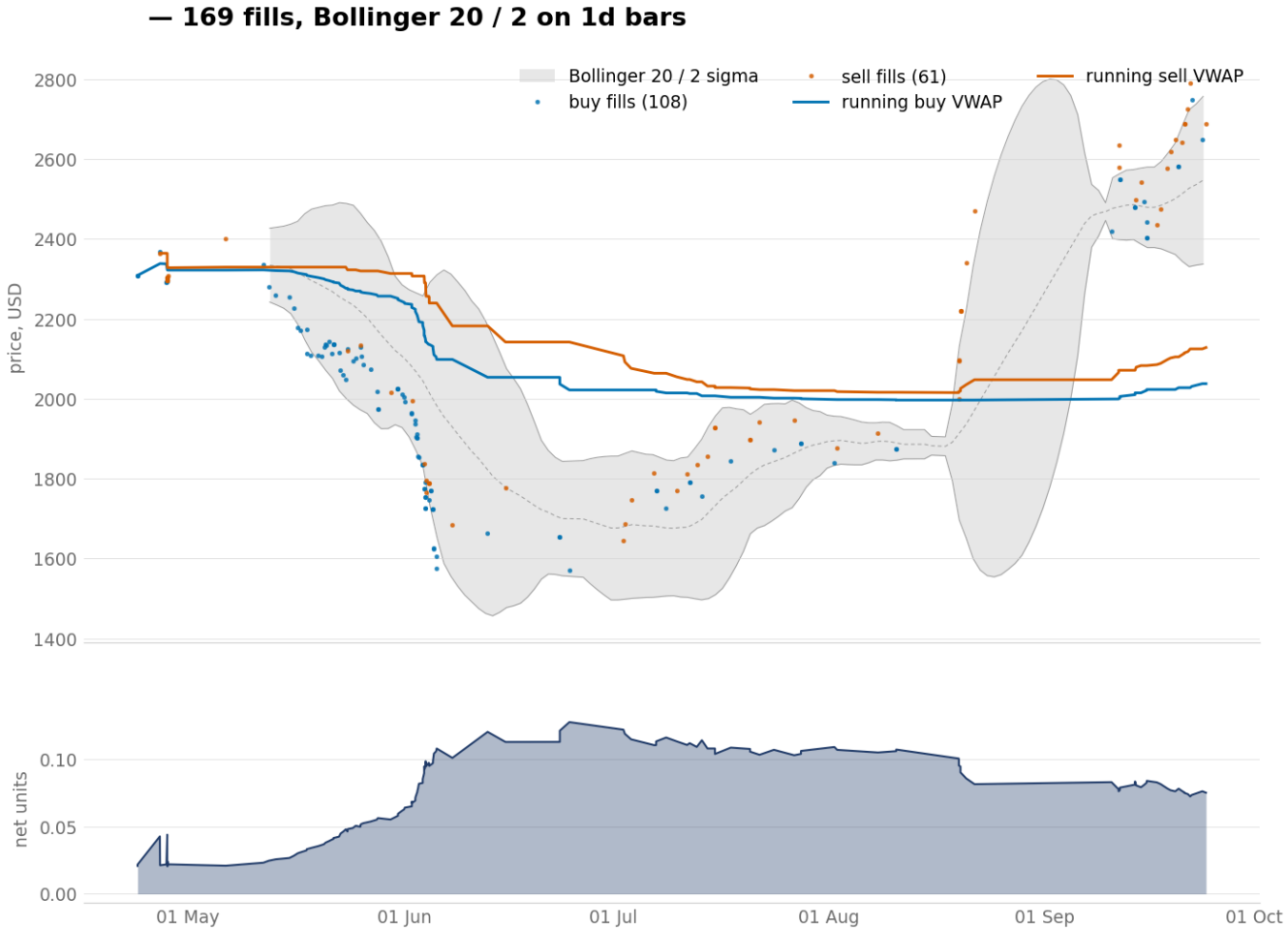
## A13




A13, 147 fills, with the sell line above the buy line throughout.

The sell line stays above the buy line for the whole window, ending near 15.4 against 14.7. Net units peak at 7.13 and end at 4.71.

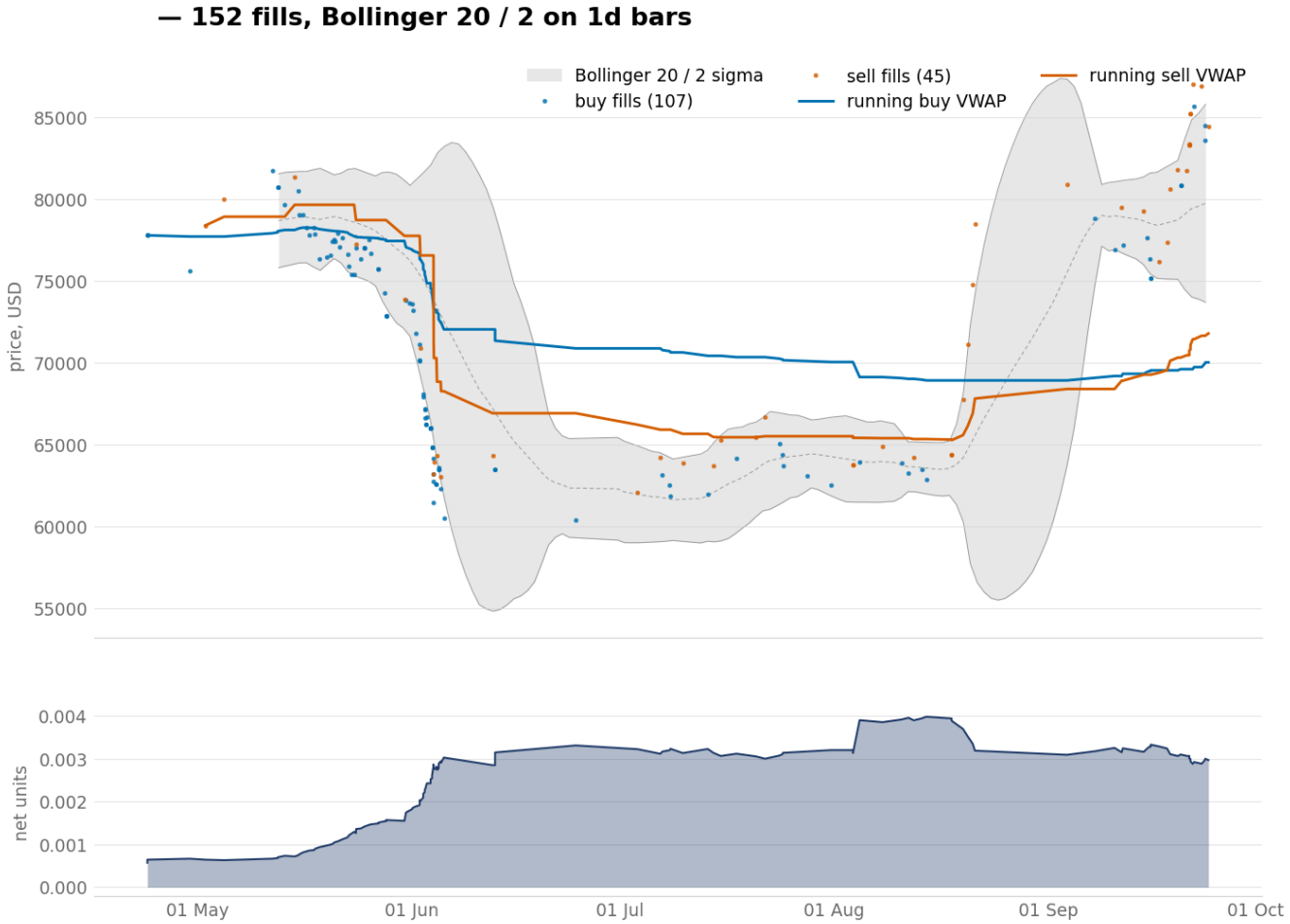
## A14




A14, 141 fills, ending 21 August.

The window closes on 21 August, the earliest close of the 38 apart from A37. The two lines run together until the start of June, then split, with the sell line about 100 dollars above through June and July. They meet again in August.

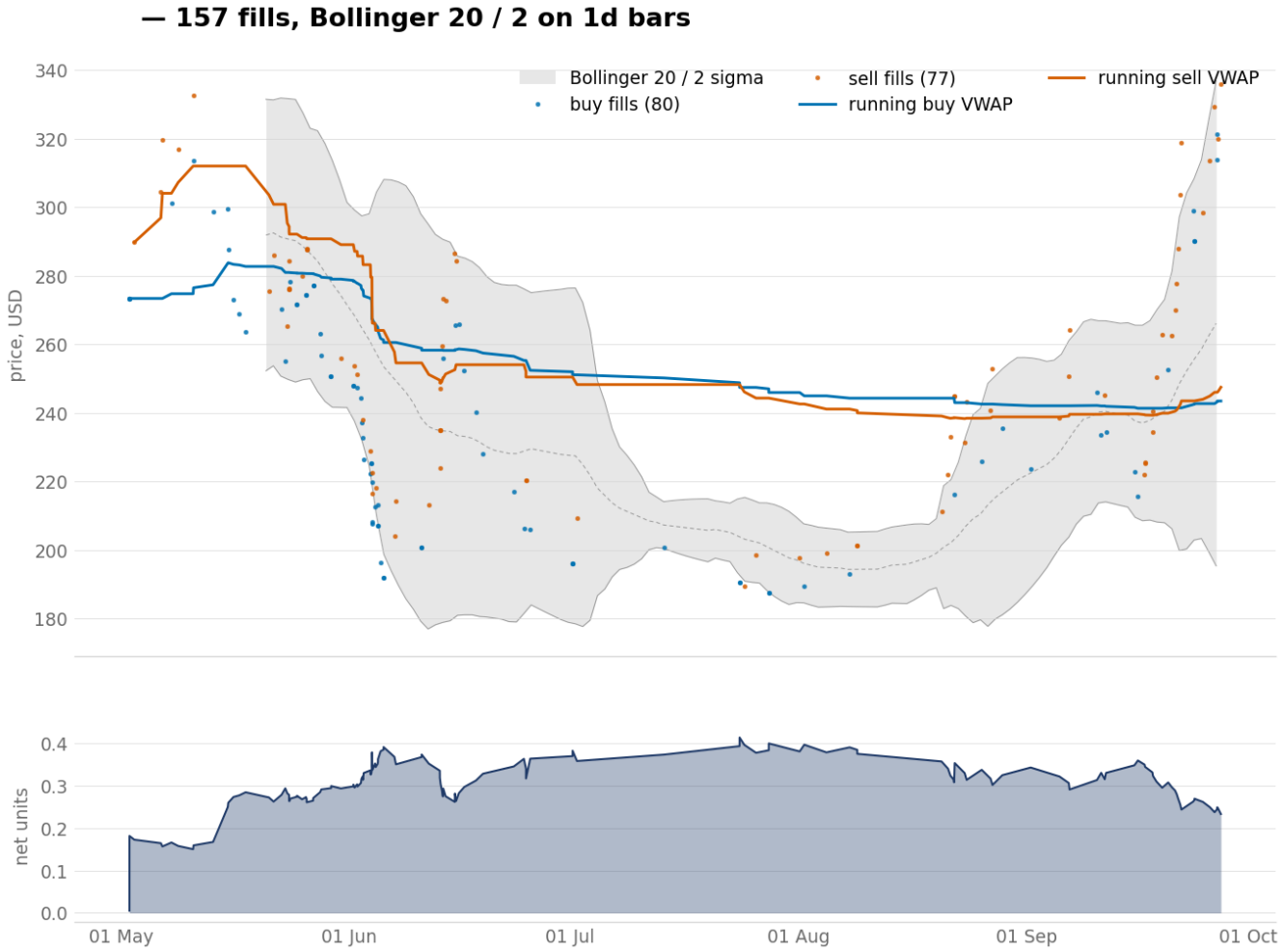
## A15




A15, 124 fills, with the two lines splitting at the start of June.

The highest-priced base in the record. Both lines hold near 78,000 through May. At the start of June the sell line drops to about 68,000 and the buy line to about 72,000, and the gap holds until late August. Net units end at 0.0031.

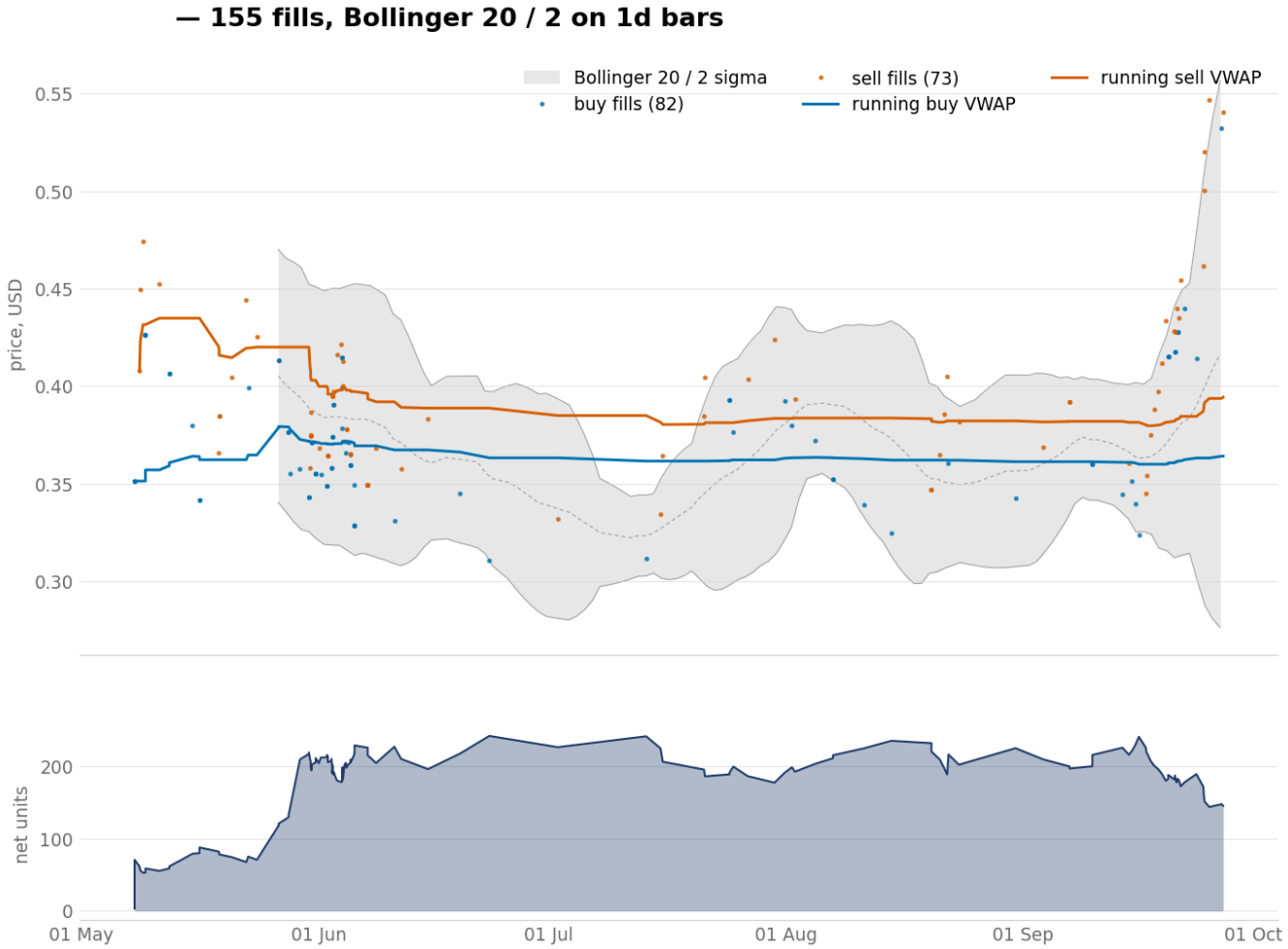
## A16




A16, 124 fills, with the sell line above the buy line until mid-June.

The sell line peaks near 312 in mid-May, about 30 dollars above the buy line, and crosses below it in mid-June. Net units end at 0.34.

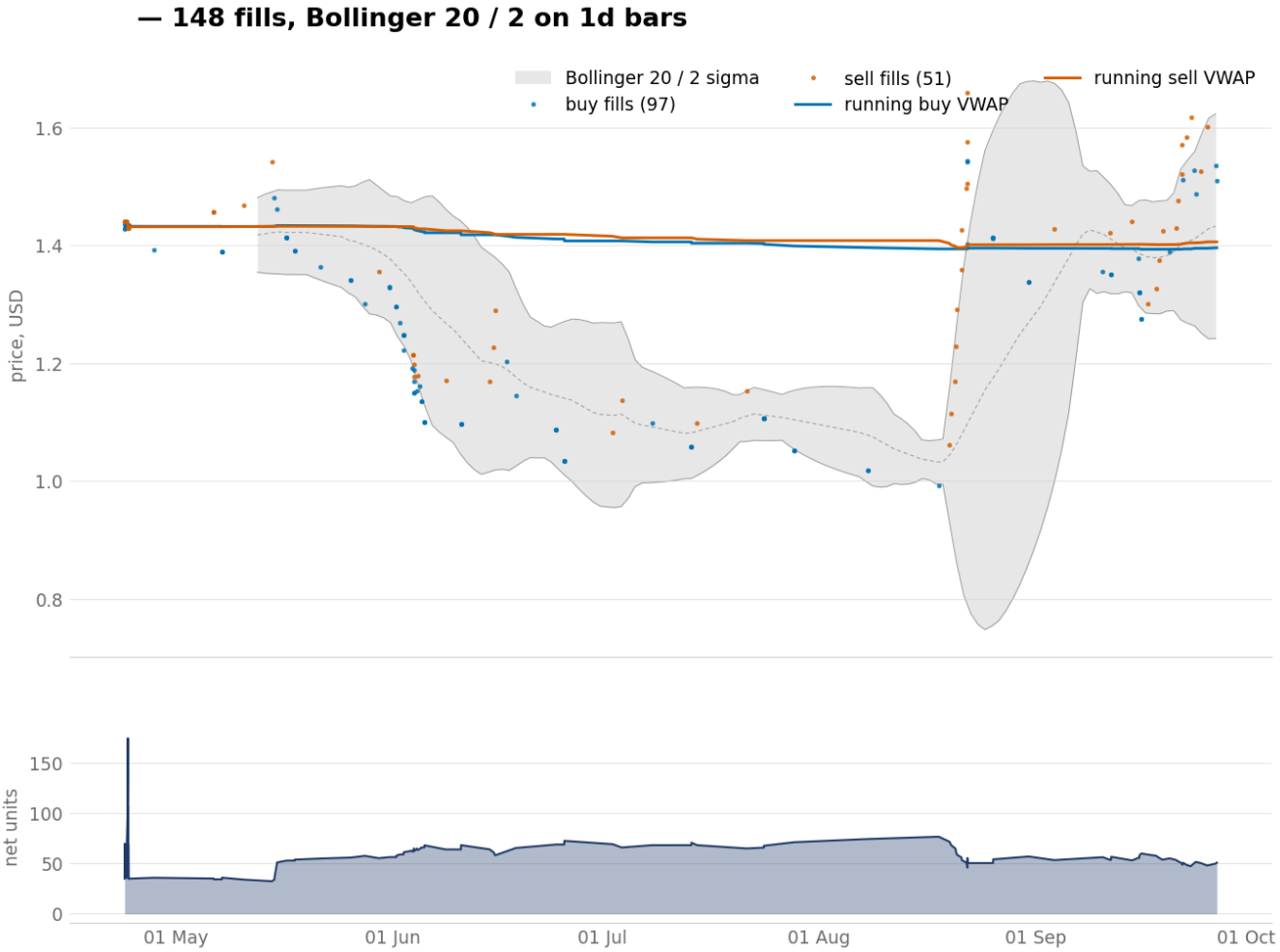
## A17




A17, 119 fills, with both lines nearly flat after June.

The buy line sits near 0.362 and the sell line near 0.382 from mid-June to the close, a gap of about 0.02 held across eleven weeks.

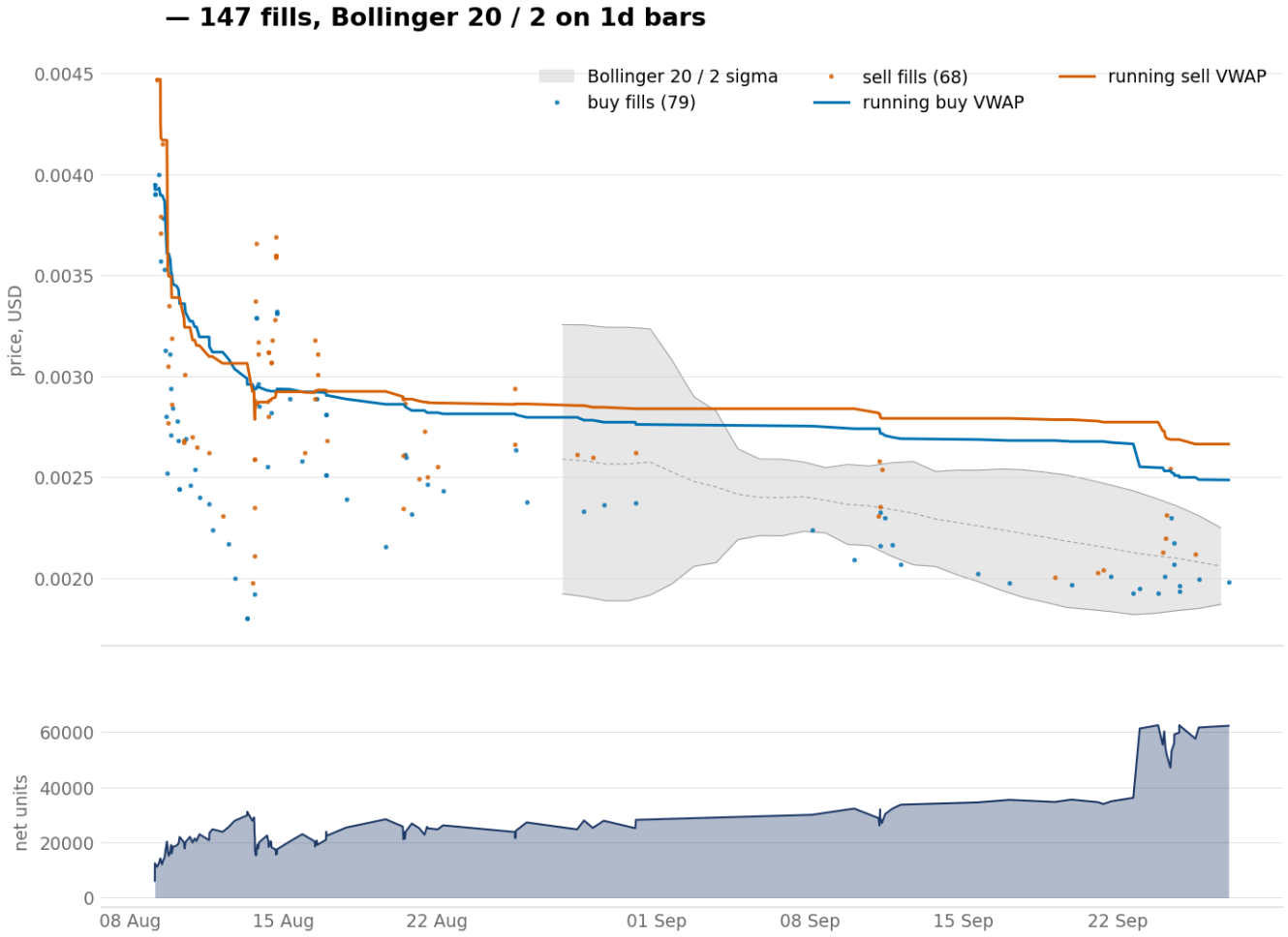
## A18




A18, 119 fills, with the two lines almost touching across the window.

The two lines run within about 0.01 of each other for the whole window. Net units spike to 175 at the start, then hold near 55.

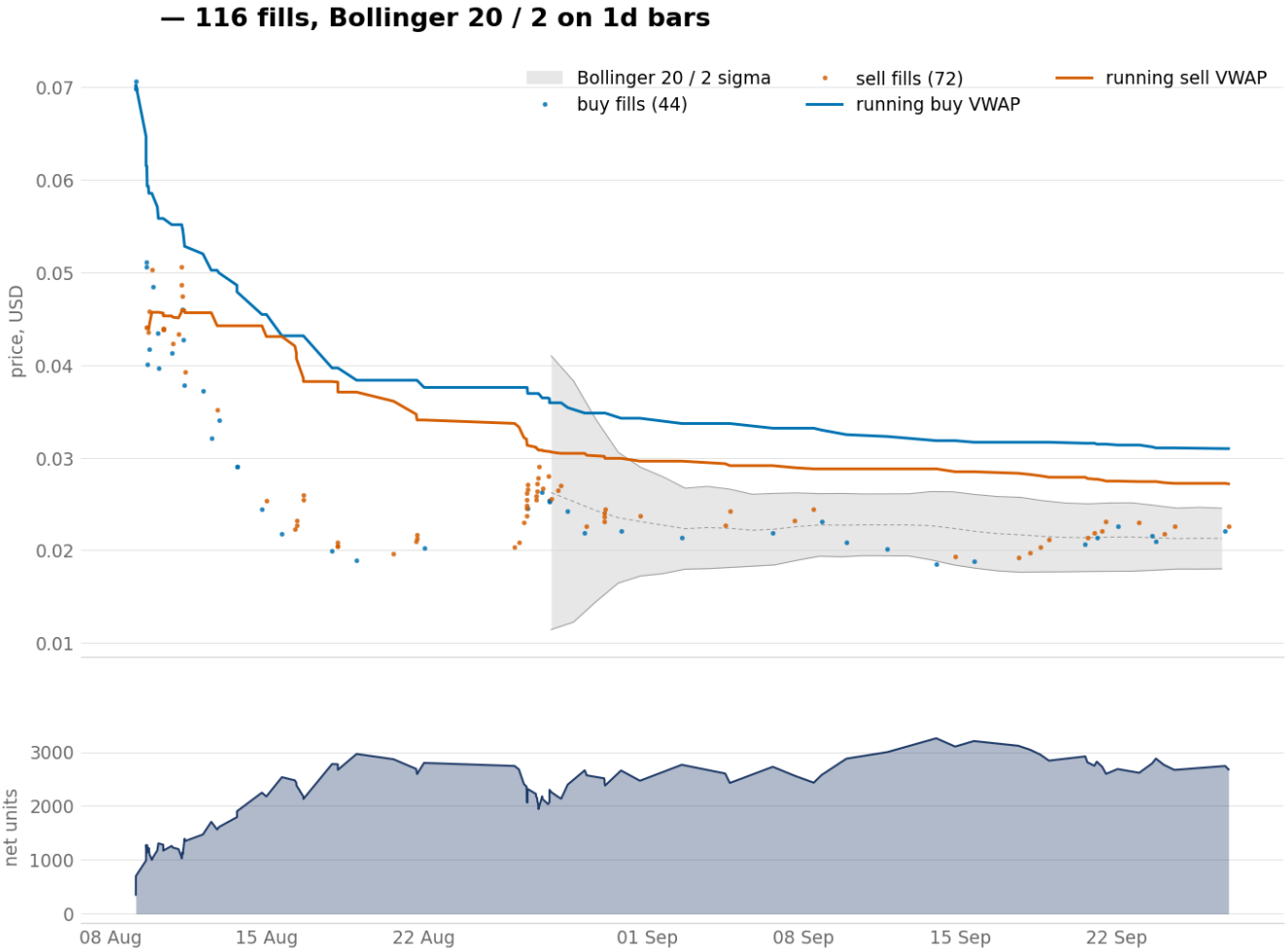
### A19




A19, 113 fills across 22 days in August.

The second-shortest window of the 38. The buy line falls from 0.00395 to about 0.0030 in the first four days, then flattens. The two lines nearly overlap after 17 August.

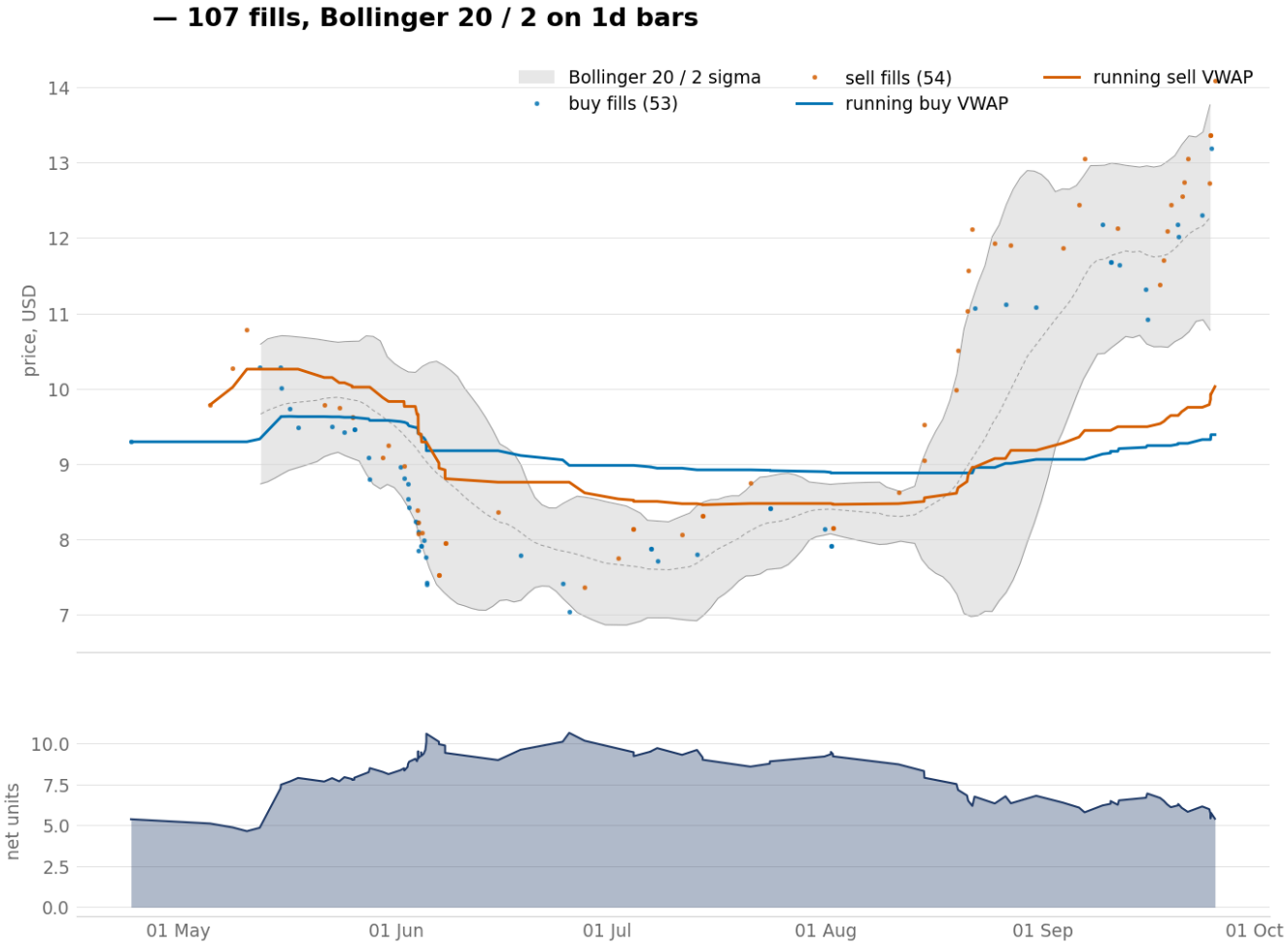
## A20




A20, 87 fills, with the sell line crossing below the buy line in mid-August.

Buys number 32 against 55 sells. The buy line starts at 0.0698 and falls to 0.0337 across 24 days, the steepest fall of the three assets that start in August.

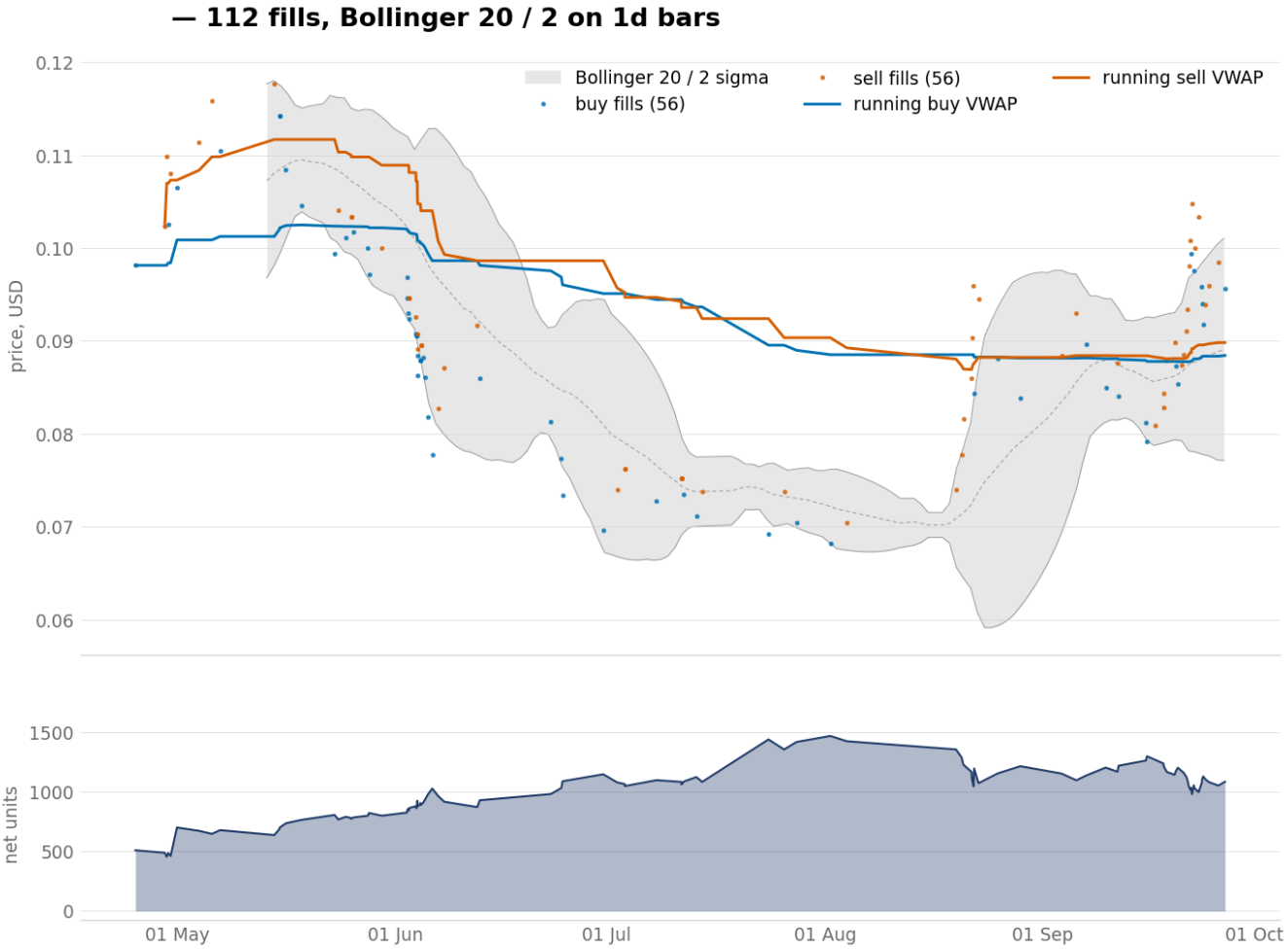
## A21




A21, 83 fills, with both lines turning up in the last two weeks.

The sell line falls below the buy line at the start of June and stays there until late August, when both turn up and cross again.

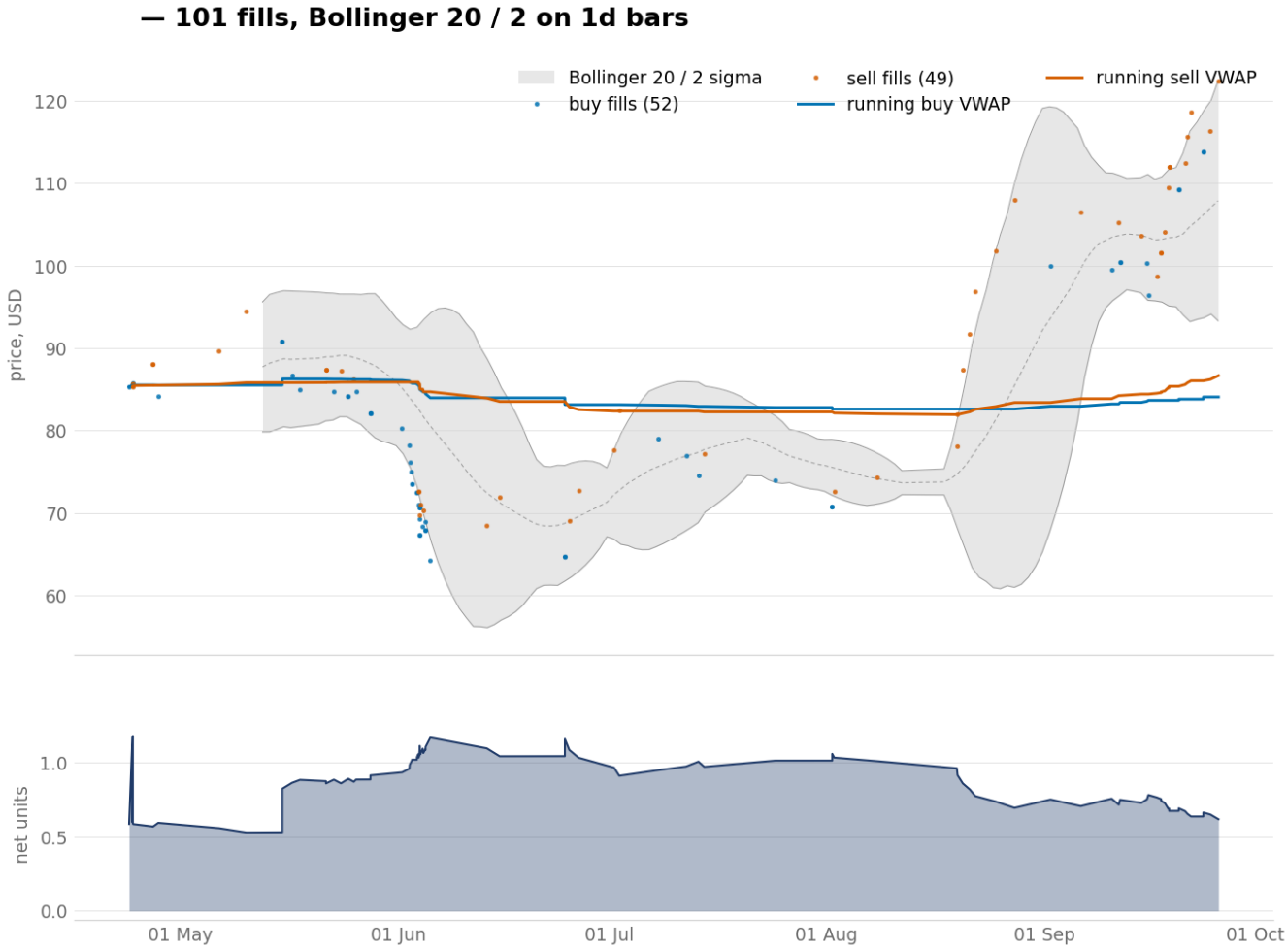
## A22




A22, 80 fills, with the two lines meeting in mid-August.

The sell line runs above the buy line from May to mid-August, then the two converge near 0.088 and finish together. Net units end at 1,153.

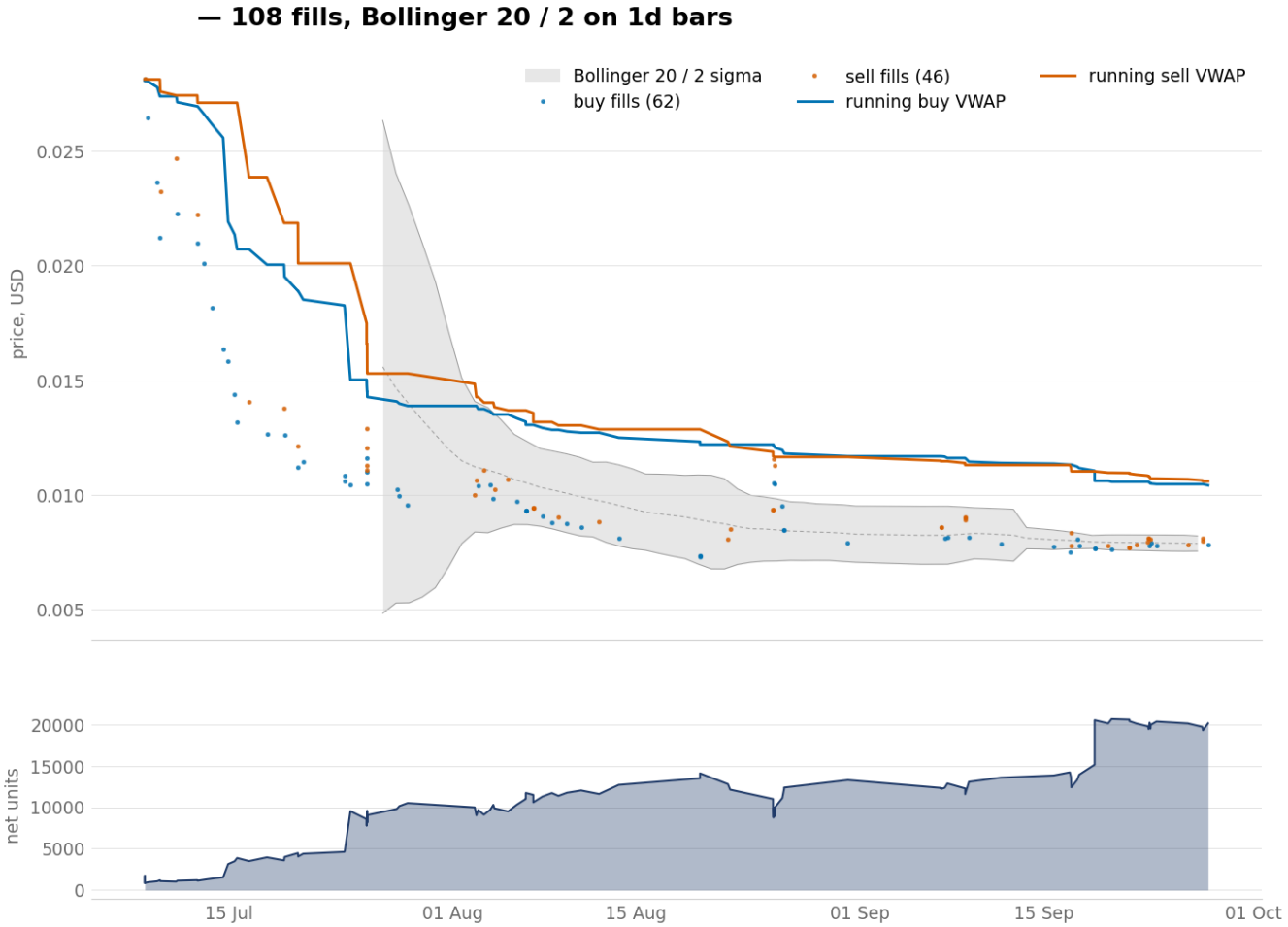
## A23




A23, 77 fills, with the two lines within about a dollar of each other.

Both lines hold near 85 through May, step to about 83 at the start of June, and end within a dollar of each other. Net units end below one whole unit, at 0.75.

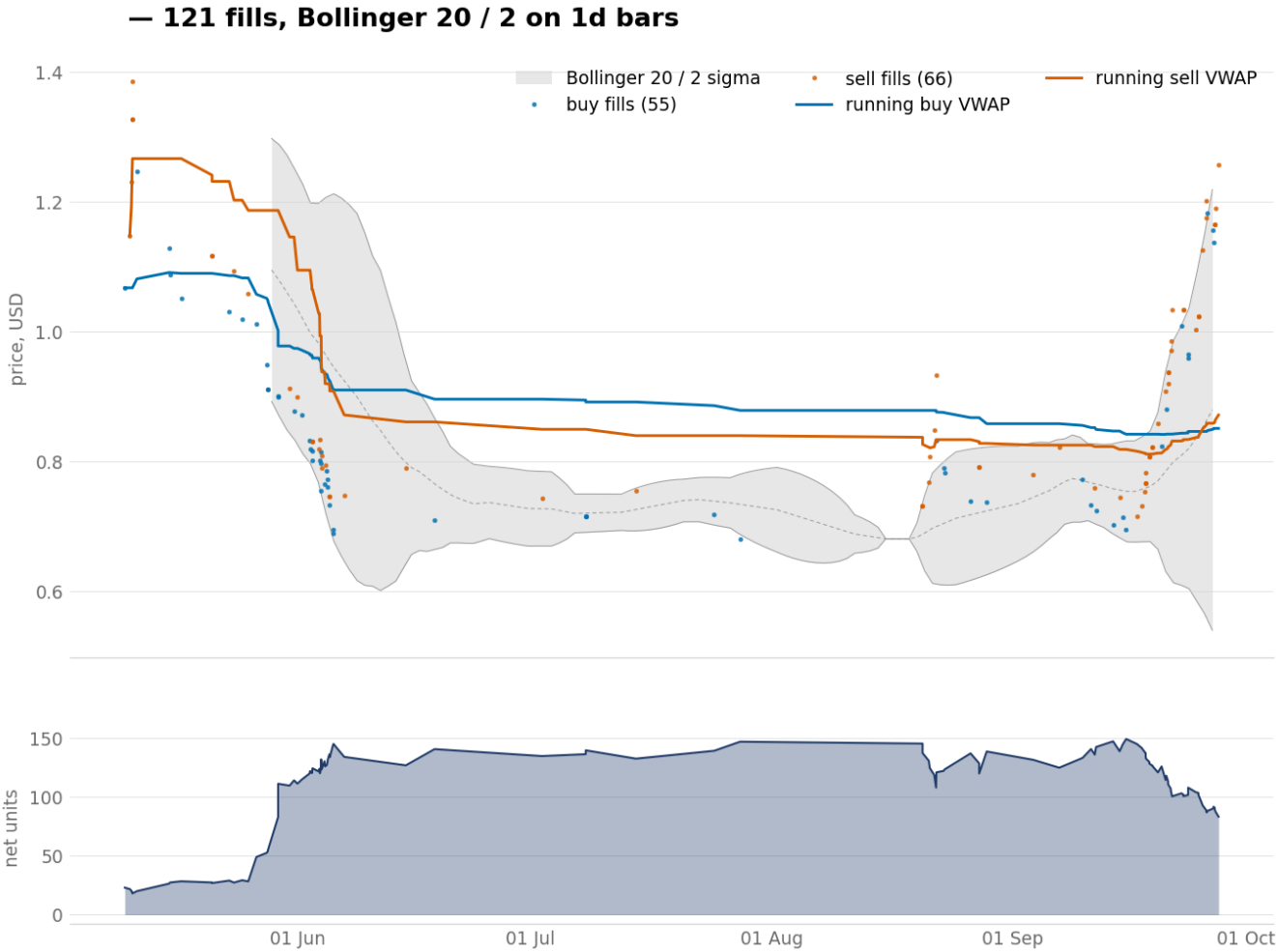
## A24




A24, 75 fills, with both lines falling by more than half.

A July starter. The buy line falls from 0.02809 to 0.01168 in eight weeks, and the sell line converges onto it by the close. Net units end at 13,268.

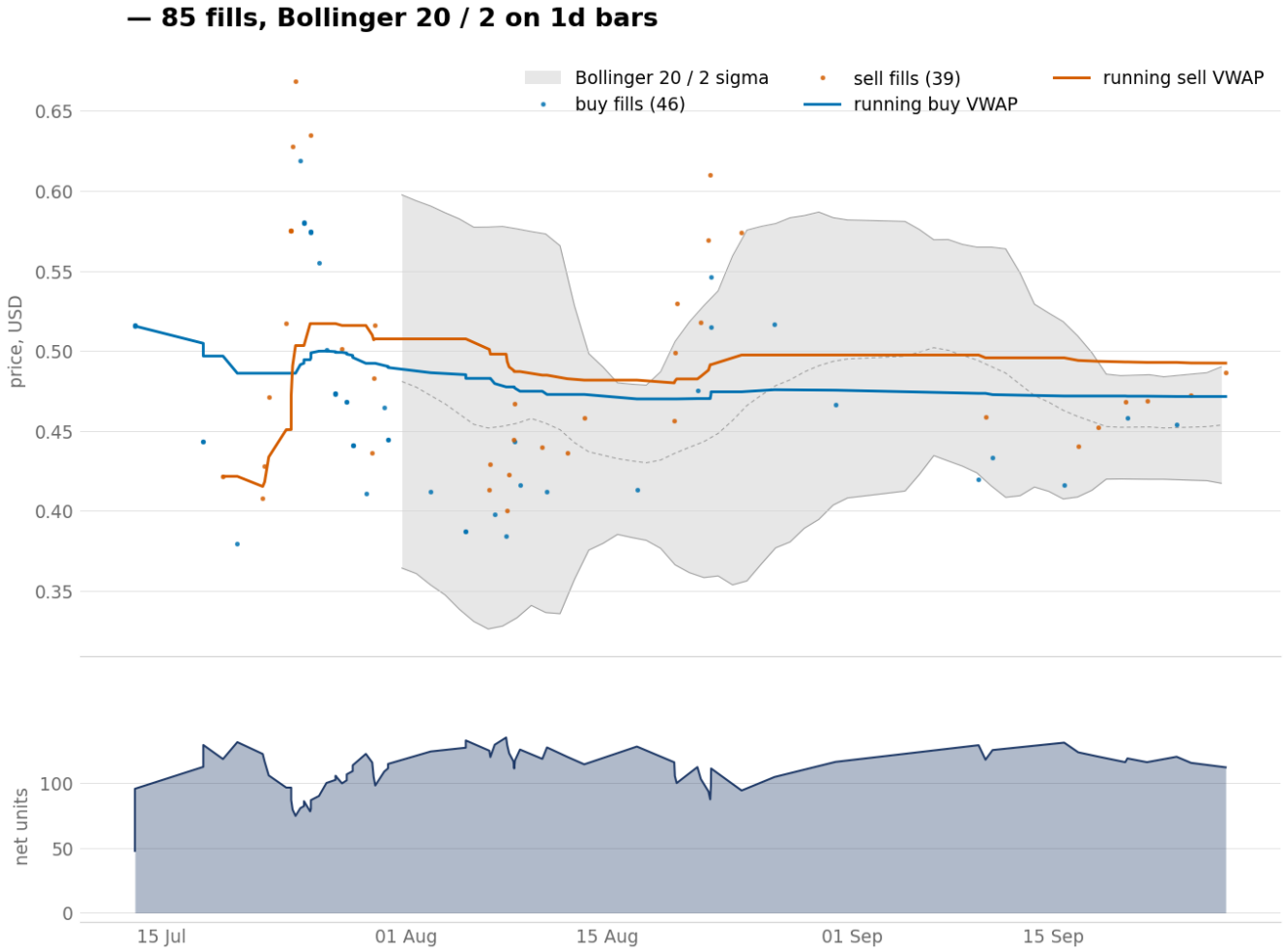
### A25




A25, 74 fills, with a step down at the start of June.

The sell line falls from 1.27 to 0.87 across late May and early June, crossing below the buy line, and the two hold about 0.04 apart afterwards.

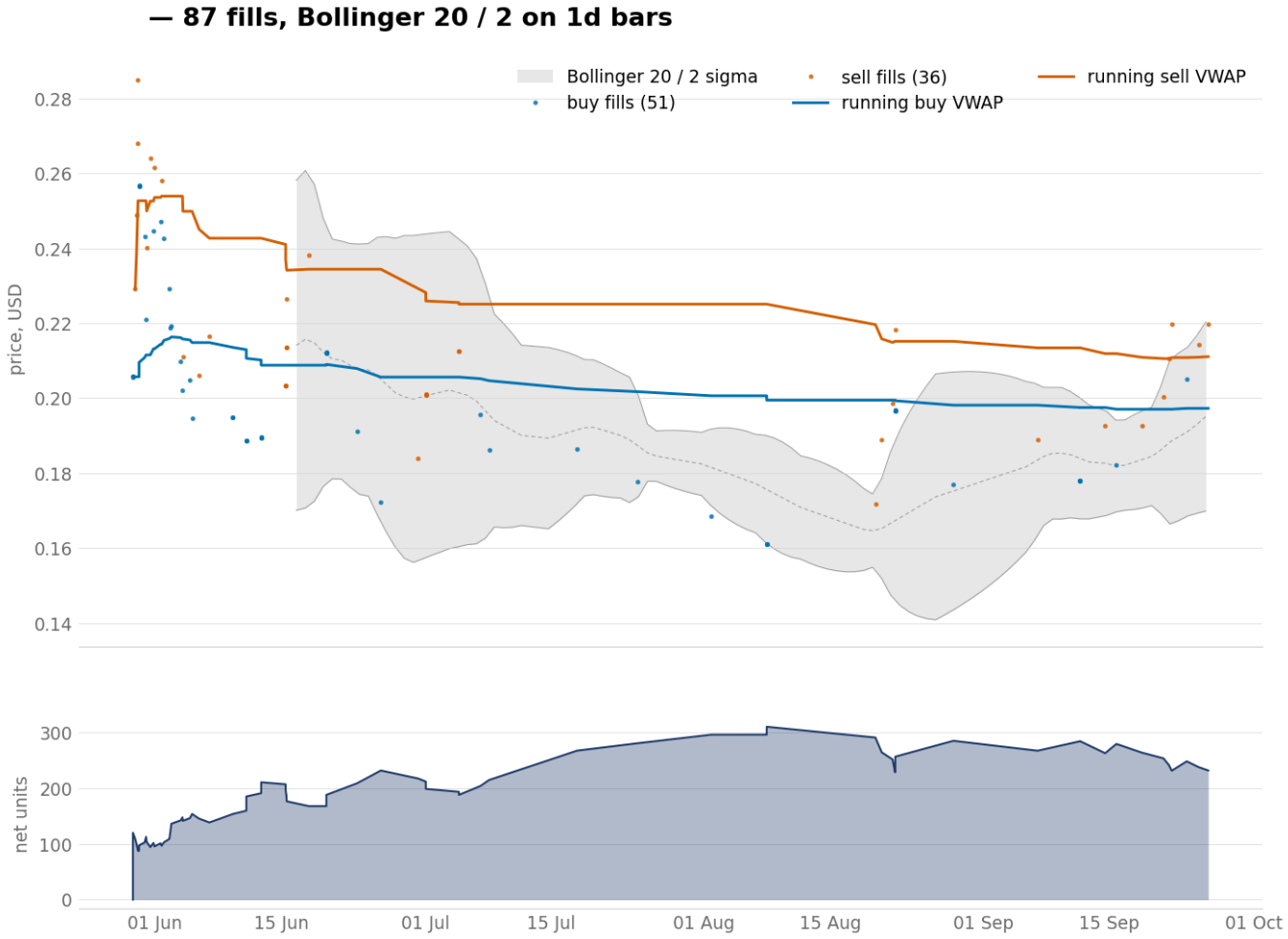
## A26




A26, 73 fills, with the sell line above the buy line from late July.

The sell line jumps from 0.42 to 0.52 in the last week of July, then settles near 0.50 against a buy line near 0.476.

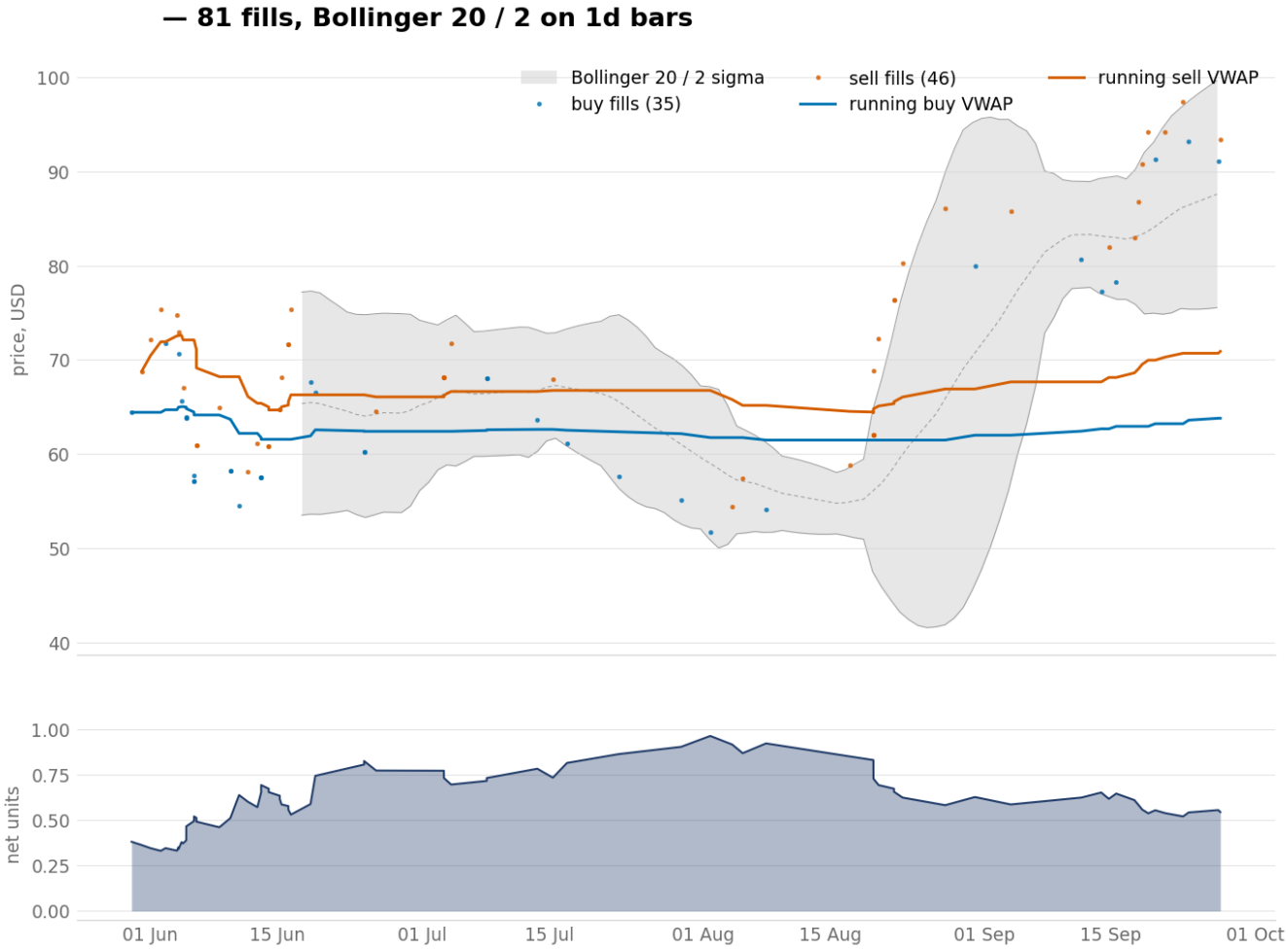
## A27




A27, 73 fills, with the sell line about 0.02 above the buy line throughout.

The gap between the two lines holds across the whole window, and both drift down. Net units grow to 311 and end at 285.

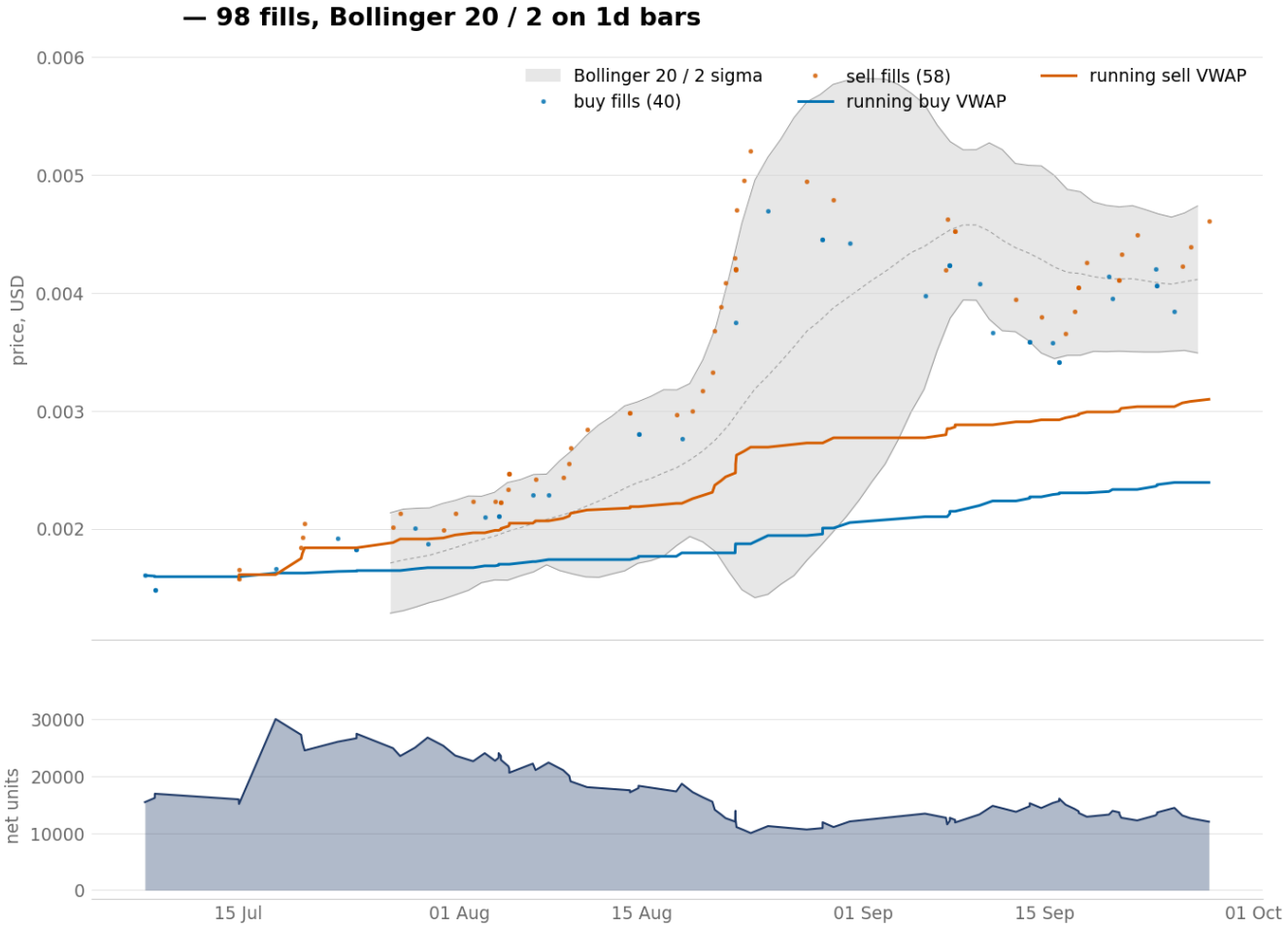
## A28




A28, 66 fills, with more sells than buys.

29 buys against 37 sells. The sell line peaks near 72.6 in early June and ends near 67, and the buy line holds between 61 and 65 across the window.

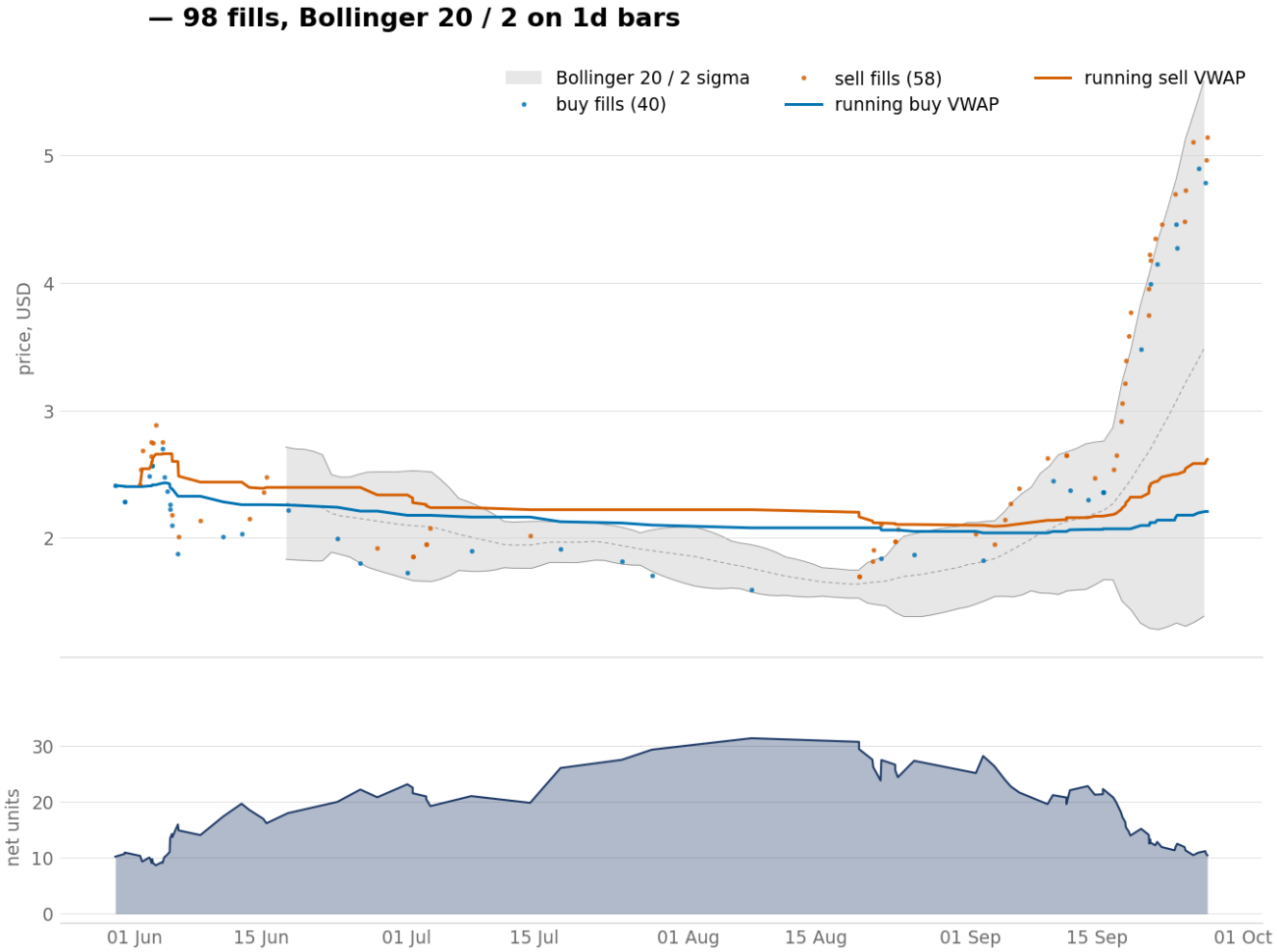
## A29




A29, 63 fills, with both lines rising through August.

23 buys against 40 sells. The buy line rises from 0.001603 to 0.002051, and the sell line rises faster, ending about 0.0007 above it. Net units drop across the window, 29,987 down to 12,032.

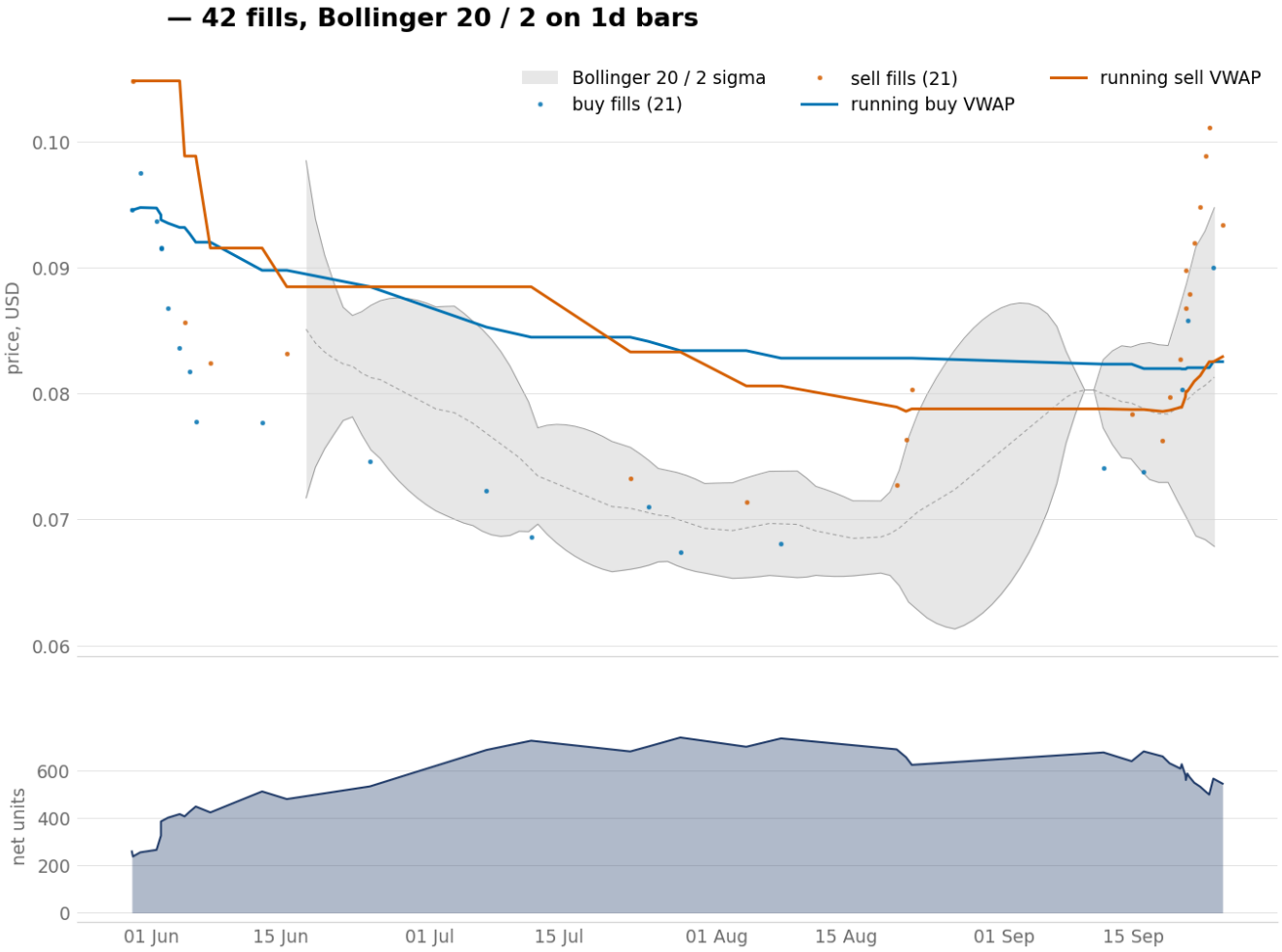
## A30




A30, 58 fills, with both lines falling steadily.

The sell line runs about 0.1 above the buy line from June to late August, then the two converge near 2.05. Net units end at 26.

### A31



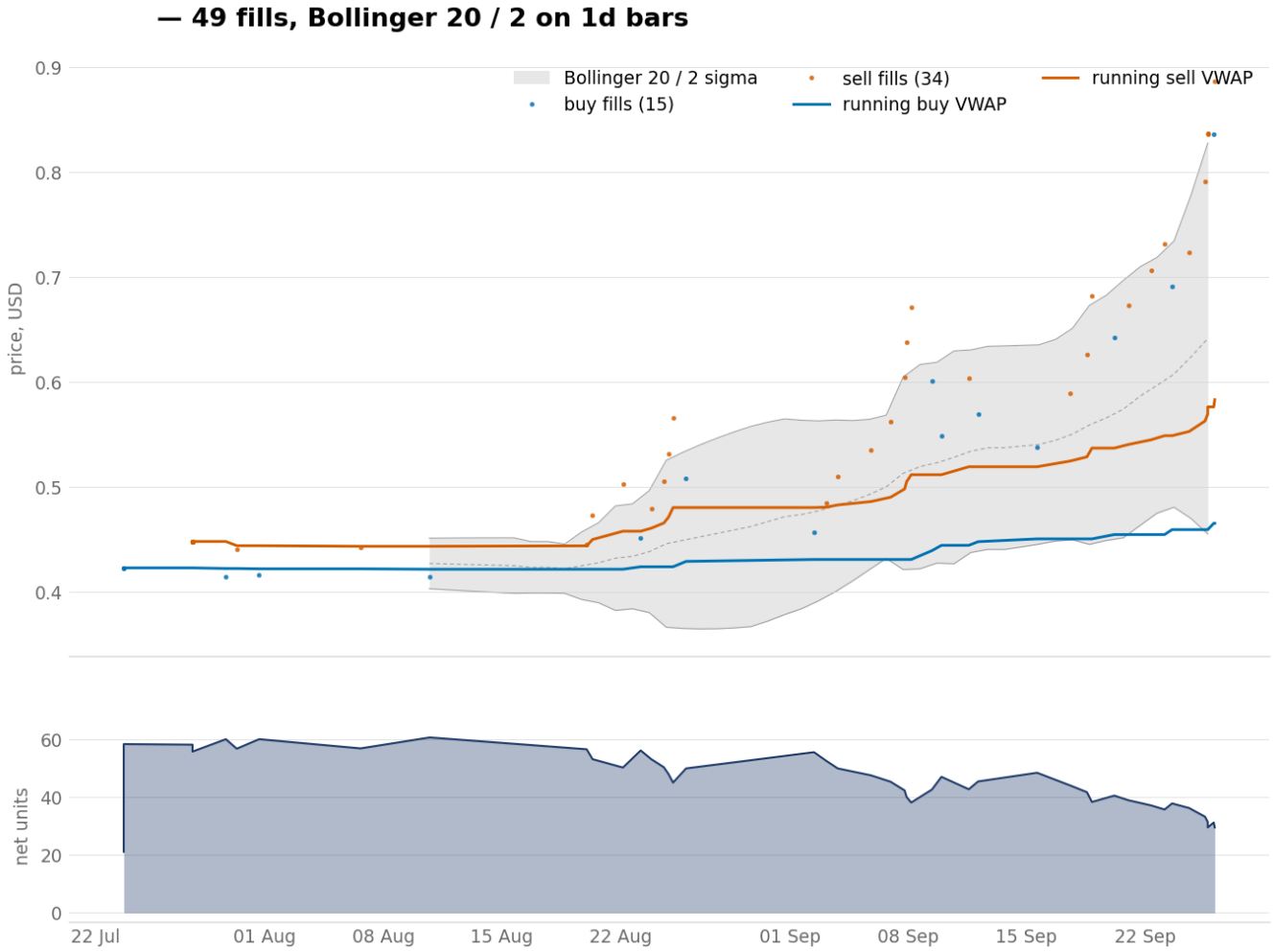

A31, 25 fills, thin enough for the two lines to carry a dot per step.

The first of the eight thin charts. The sell line starts at 0.1048, steps down through June and July, and crosses below the buy line in late July. Net units end at 626.

25 fills is under the module's threshold of 30, so both lines carry a marker at every step. Sixteen dots sit on the buy line and nine on the sell line. The seven charts after this one are drawn the same way.

```
THIN_FILLS = 30

dot = "o" if len(fills) <= THIN_FILLS else None
ax.plot(t, buy_v, color=COLORS["series"][0], lw=2.0, marker=dot,
        markersize=4, label="running buy VWAP")
```

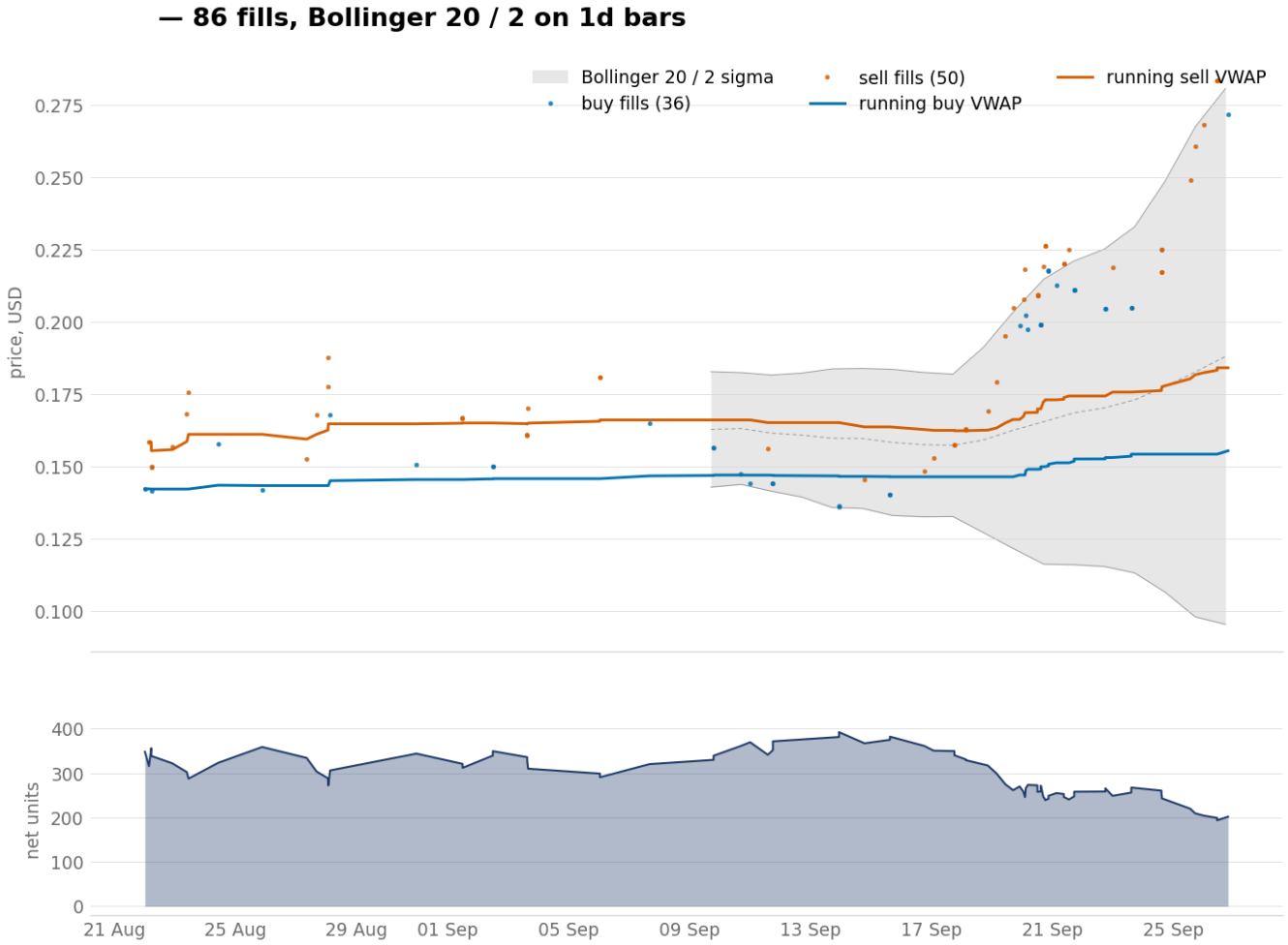
## A32




A32, 24 fills, with eight buys and sixteen sells.

A thin chart. The buy line holds near 0.4225 until late August, then rises to 0.4310. The sell line jumps from 0.444 to 0.481 in the same week.

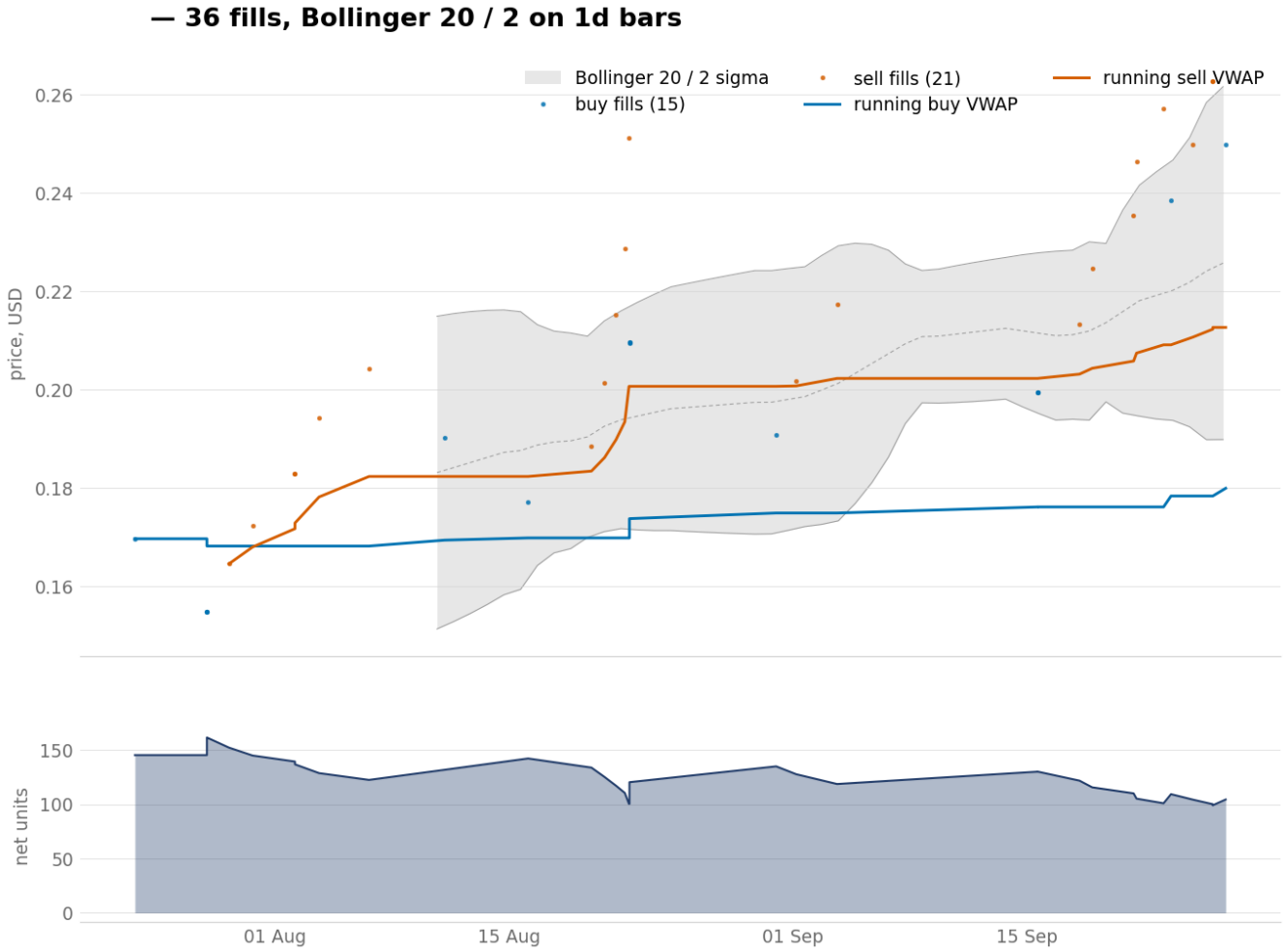
## A33




A33, 23 fills across 12 days, the shortest window of the 38.

A thin chart, and the shortest window here. Eight buys against fifteen sells. Both lines step up once, on 28 August.

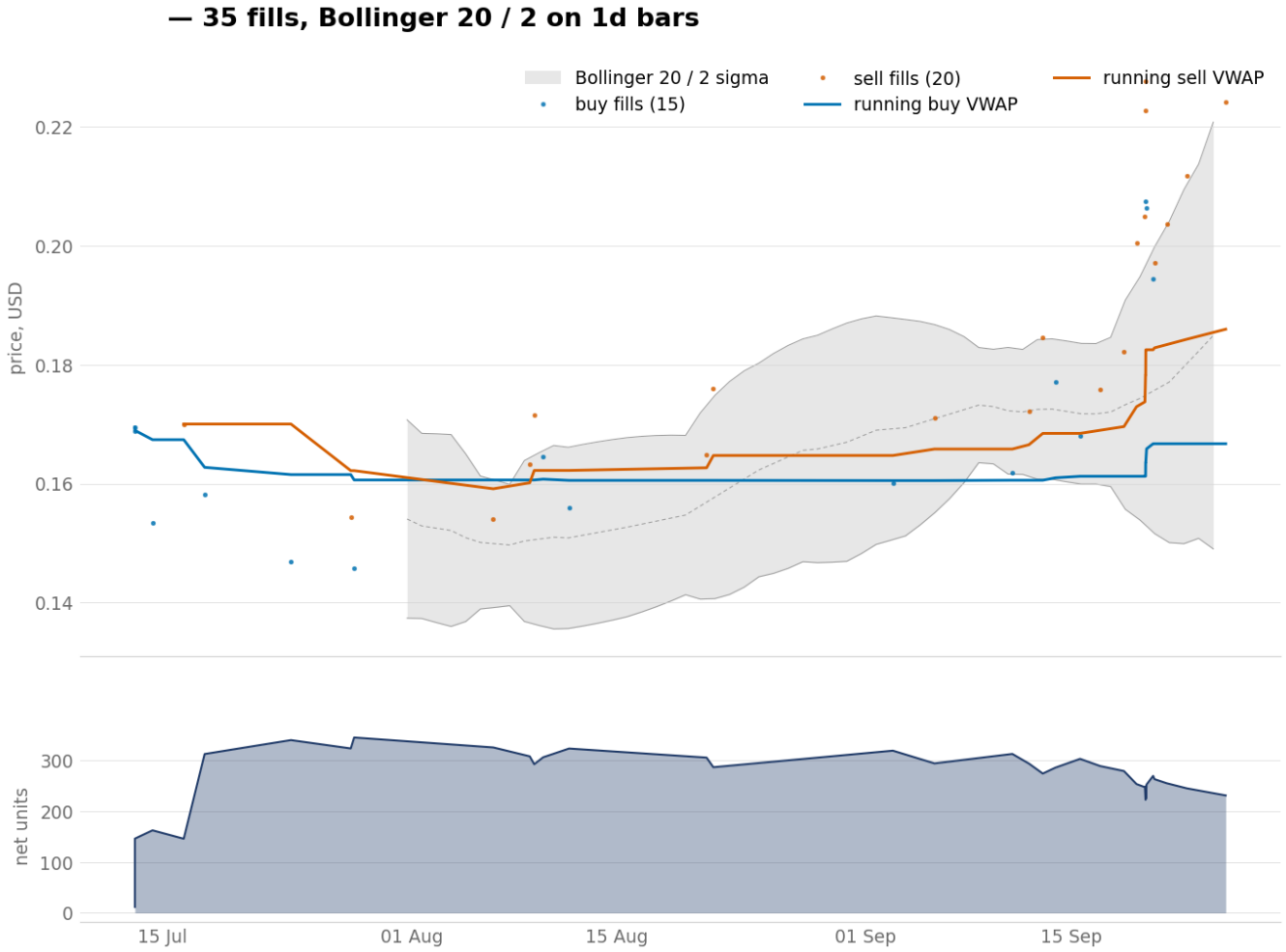
## A34




A34, 23 fills, with the sell line stepping up sharply on 22 August.

A thin chart. The sell line climbs from 0.1645 to about 0.202, while the buy line moves from 0.1696 to 0.1748 across the same six weeks.

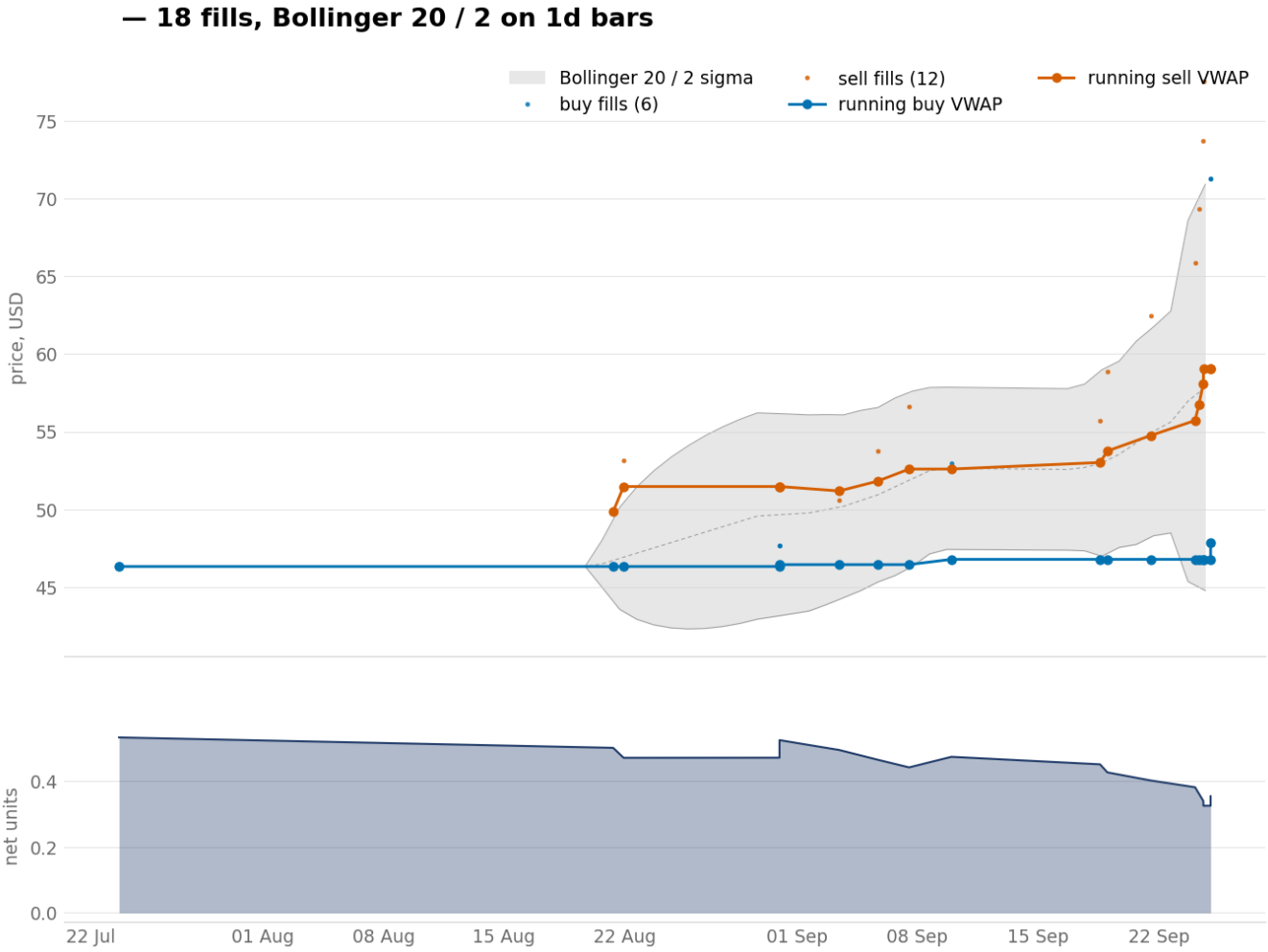
## A35




A35, 16 fills, with the buy line flat from late July.

A thin chart. The buy line settles at 0.1605 in late July and holds. The sell line dips to 0.159 in early August, then steps back to 0.1648.

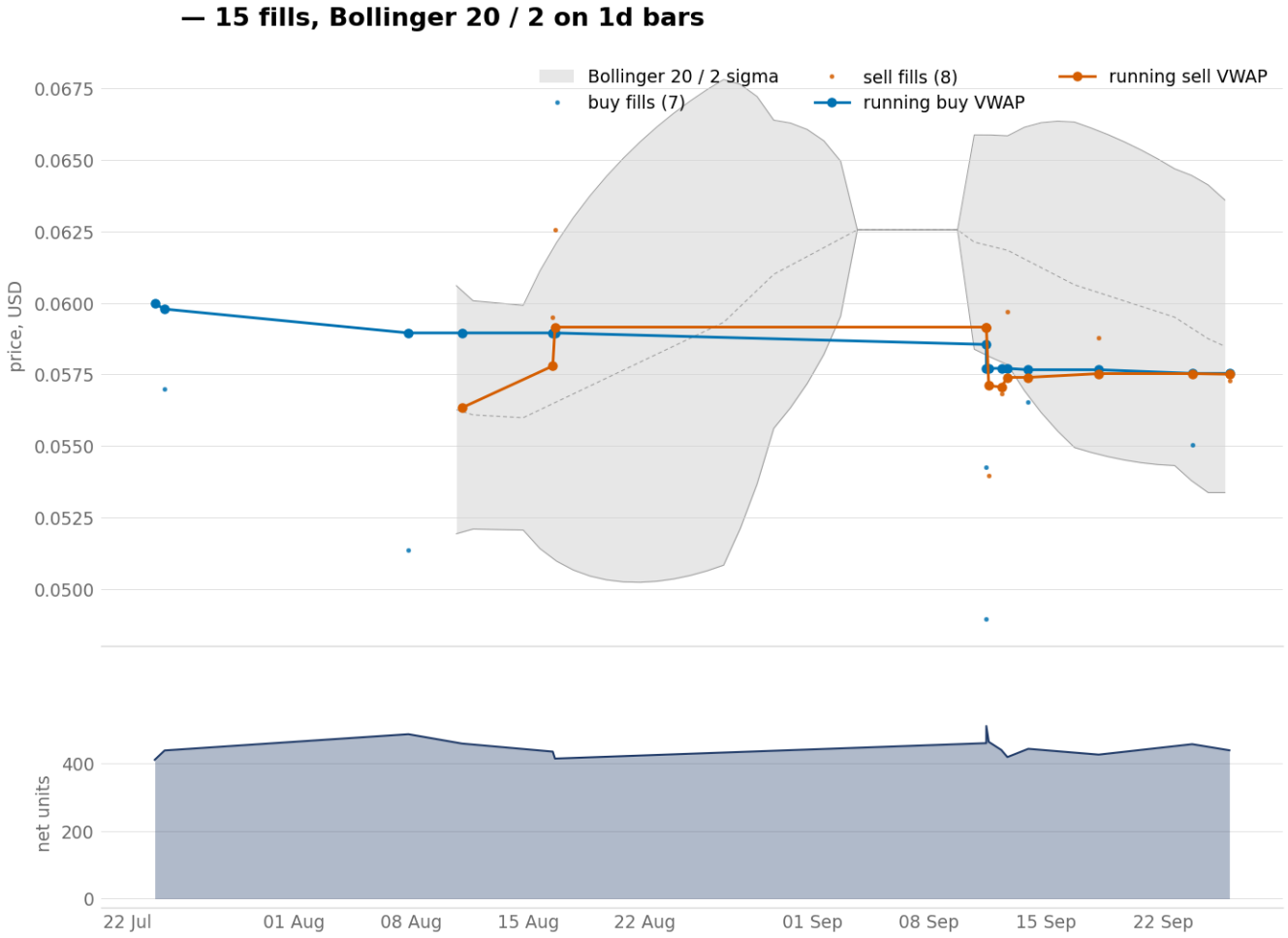
## A36




A36, 6 fills, three buys and three sells.

The buy line is nearly flat at 46.33, since all three buys land near one price. The sell line runs about 5 dollars above it. Net units end at 0.49.

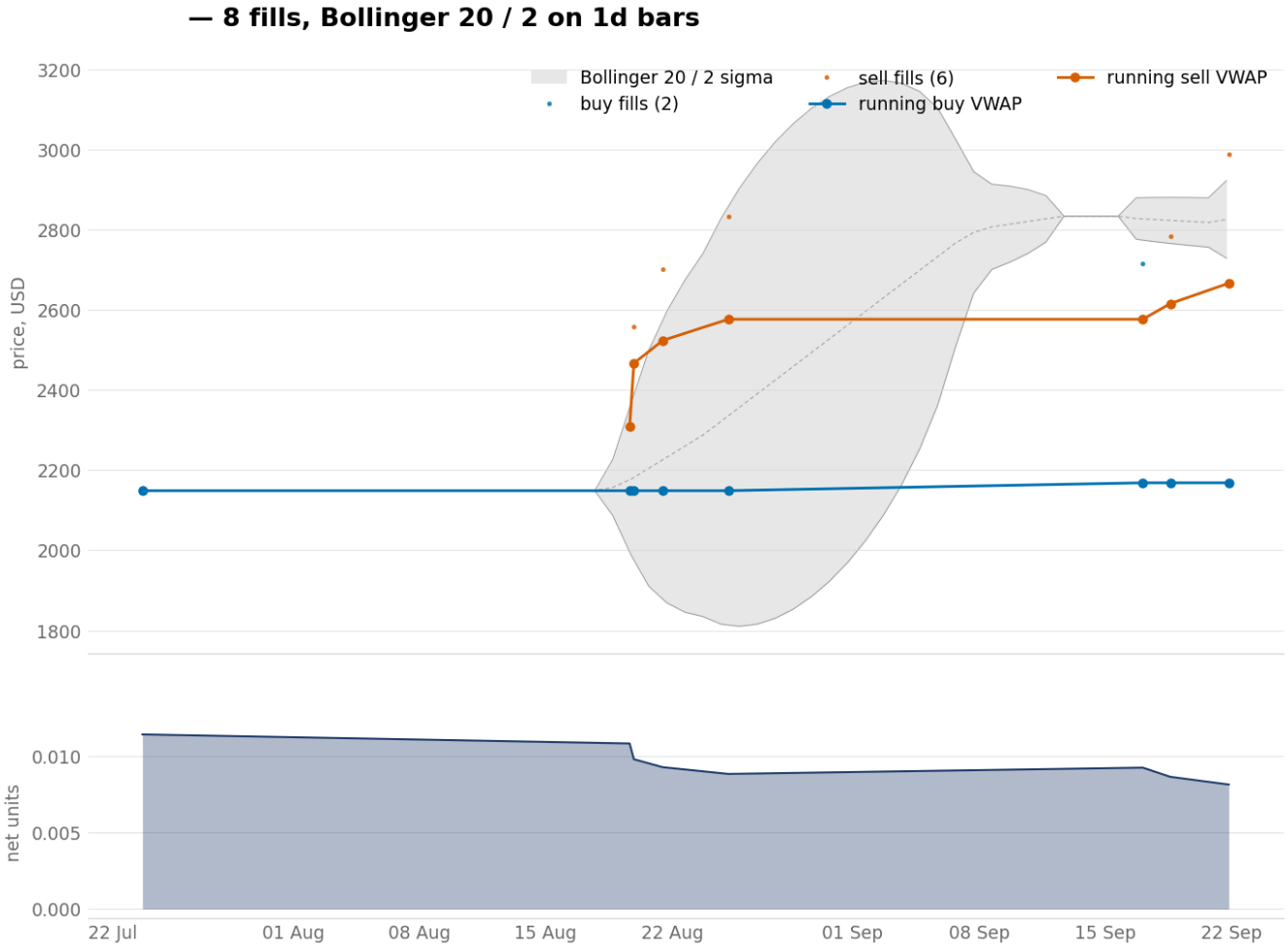
## A37




A37, 6 fills, closing 16 August.

The earliest close of the 38. Three buys and three sells, all inside four weeks. The buy line drifts from 0.0600 to 0.0590.

**A38**




A38, 5 fills, one buy and four sells.

The only base in the record with a single buy. One buy makes the buy line a single horizontal line at 2,148.63 and its index exactly 1.000. The sell line rises across the four sells, 2,307 up to 2,577. Net units drop across the window, 0.0114 down to 0.0088.

## Trade grading

One function in `src/trading/trade_grader.py` grades a completed fill, on four axes and no more. Execution scores against a reference price at the decision, timing across the prices after the trade, strategy across the shift in the rolling S/B figure, and outcome against realised profit per unit. An axis whose price inputs are absent is left out, and the overall figure is the unweighted mean of the axes that scored — 0.5 when none of them could.

```
def grade_trade(record: TradeRecord, ctx: PriceContext) -> TradeGrade: ...


def _score_execution(...): ...
def _score_timing(...): ...
def _score_strategic(...): ...
def _score_outcome(...): ...
```

A second function maps that mean to a letter.

```
def _letter_from_numeric(num: float) -> str:
    if num >= 0.93:
        return "A+"
    elif num >= 0.85:
        return "A"
    elif num >= 0.70:
        return "B"
    elif num >= 0.55:
        return "C"
    elif num >= 0.40:
        return "D"
    else:
        return "F"
```

The market regime rides beside the grade rather than inside it. `PriceContext.regime_tag` reaches `TradeGrade.regime` and the rationale text, and it carries no sub-score of its own, which is why a rationale can read five terms while the grade rests on four.

`grade_trades` runs the batch.

The screen these grades reach is in the History tab reference.

## Gate logs behind each trade

A fill says what happened. The gate log says which decision produced it.

`gate_light_row` in `src/trading/gate_vocabulary.py` returns nineteen lights for one decision, the scrum bank first and the fold bank second, each entry carrying its bank, label, state and colour. The History table and the Simulator draw their lights from that one function, so both surfaces name the same gates in the same order. The labels themselves, and the indicator readings behind them, are in the indicator reference.

`lookup_gate_entry` in `src/exchange/history_helpers.py` pairs each fill with the nearest gate entry for the same bot. The tolerance is 60 seconds either side of the fill, and a fill that finds an entry inside that window carries its gate. A fill that finds none carries no gate.

```
JOIN_TOLERANCE_SECONDS = 60.0


def gate_cell_text(entry: Optional[dict]) -> str:
    if not entry:
        return "no record"
```

A fill with no entry reads "no record" in the Gates column, and its tooltip says no gate record joined to the trade. Nothing measures coverage over a set of fills, and nothing names a reason a fill has no gate.